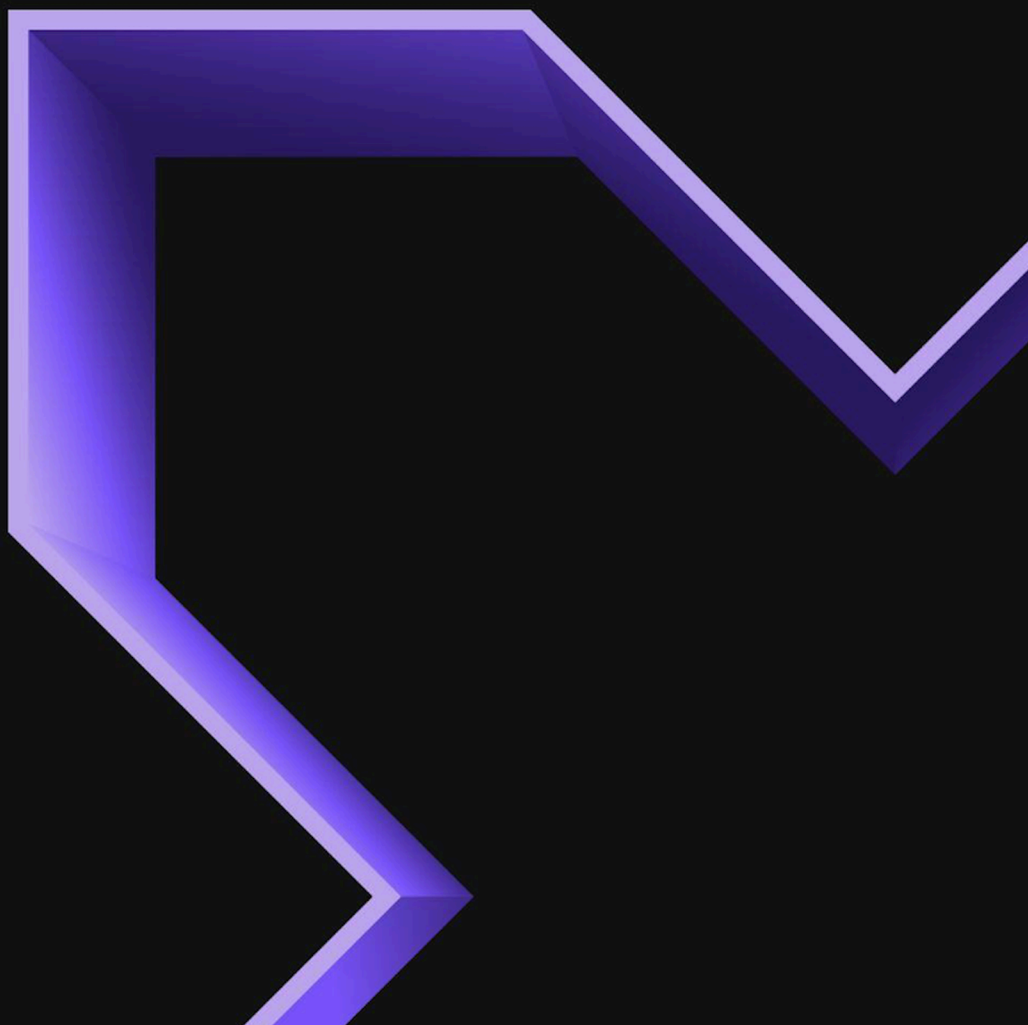
Nutanix Kubernetes® Platform Guide

Nutanix Kubernetes Platform 2.12

October 6, 2025



NUTANIX

Contents

1. Nutanix Kubernetes Platform Overview....11
   - Architecture....11
   - Supported Infrastructure Operating Systems....12

2. Downloading NKP....16

3. Getting Started with NKP....17
   - NKP Concepts and Terms....18
     - Cluster Types....19
     - CAPI Concepts and Terms....20
     - Air-Gapped or Non-Air-Gapped Environment....22
     - Pre-provisioned Infrastructure....23
   - Licenses....24
     - NKP Starter License....26
     - NKP Pro License....27
     - NKP Ultimate License....29
     - Adding an NKP License....31
     - Removing an NKP License....32
   - Commands within a kubeconfig File....32
   - Storage....33
     - Default Storage Providers....34
     - Change or Manage Multiple StorageClasses....35
     - Provisioning a Static Local Volume....38
   - Resource Requirements....39
     - General Resource Requirements....40
     - Infrastructure Provider-Specific Requirements....41
     - Kommander Component Requirements....42
     - Managed Cluster Requirements....42
     - Management Cluster Application Requirements....43
     - Workspace Platform Application Defaults and Resource Requirements....44
   - Prerequisites for Installation....47
   - Installing NKP....50

4. Basic Installations by Infrastructure....52
   - Nutanix Installation Options....52
     - Nutanix Basic Prerequisites....53
     - Nutanix Non-Air-gapped Installation....58
     - Nutanix Air-gapped Installation....63
   - Pre-provisioned Installation Options....69
     - Pre-provisioned Installation....70
     - Pre-provisioned Air-gapped Installation....83
     - Pre-provisioned FIPS Install....100
     - Pre-provisioned FIPS Air-gapped Install....113
     - Pre-provisioned with GPU Install....130
     - Pre-provisioned Air-gapped with GPU Install....145
   - AWS Installation Options....163

AWS Installation....163
AWS Air-gapped Installation....175
AWS with FIPS Installation....189
AWS Air-gapped with FIPS Installation....201
AWS with GPU Installation....214
AWS Air-gapped with GPU Installation....226
EKS Installation Options....239
EKS Installation....240
EKS: Minimal User Permission for Cluster Creation....241
EKS: Cluster IAM Policies and Roles....243
EKS: Create an EKS Cluster....247
EKS: Grant Cluster Access....251
EKS: Retrieve kubeconfig for EKS Cluster....252
EKS: Attach a Cluster....254
vSphere Installation Options....259
vSphere Prerequisites: All Installation Types....259
vSphere Installation....266
vSphere Air-gapped Installation....279
vSphere with FIPS Installation....294
vSphere Air-gapped FIPS Installation....307
VMware Cloud Director Installation Options....322
Azure Installation Options....322
Azure Installation....323
Azure: Creating an Image....324
Azure: Creating the Management Cluster....325
Azure: Install Kommander....326
Azure: Verifying your Installation and UI Log in....328
Azure: Creating Managed Clusters Using the NKP CLI....330
AKS Installation Options....332
AKS Installation....333
AKS: Create an AKS Cluster....335
AKS: Retrieve kubeconfig for AKS Cluster....337
AKS: Attach a Cluster....338
GCP Installation Options....341
GCP Installation....342

5. Cluster Operations Management....353
Operations....353
Access Control....354
Identity Providers....364
Infrastructure Providers....374
Header, Footer, and Logo Implementation....389
Applications....390
Customizing Your Application....390
Printing and Reviewing the Current State of an AppDeployment Resource....391
Deployment Scope....392
Logging Stack Application Sizing Recommendations....392
Rook Ceph Cluster Sizing Recommendations....394
Application Management Using the UI....396
Platform Applications....400
Setting Priority Classes in NKP Applications....408
AppDeployment Resources....410
Workspaces....411
Creating a Workspace....411
Adding or Editing Workspace Annotations and Labels....411

Deleting a Workspace....412
Workspace Applications....412
Workplace Catalog Applications....420
Configuring Workspace Role Bindings....435
Multi-Tenancy in NKP....436
Generating a Dedicated Login URL for Each Tenant....438
Projects....439
Creating a Project Using the UI....439
Creating a Project Using the CLI....440
Project Applications....441
Project Deployments....456
Project Role Bindings....463
Project Roles....466
Project ConfigMaps....469
Project Secrets....470
Project Quotas and Limit Ranges....471
Project Network Policies....473
Cluster Management....478
Creating a Managed Nutanix Cluster Through the NKP UI....479
Creating a Managed Azure Cluster Through the NKP UI....483
Creating a Managed vSphere Cluster Through the NKP UI....484
Creating a Managed Cluster on VCD Through the NKP UI....489
Attaching an Existing Kubernetes Cluster....493
Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster....535
Creating Advanced CLI Clusters....552
Custom Domains and Certificates Configuration for All Cluster Types....553
Disconnecting or Deleting Clusters....558
Management Cluster....559
Cluster Limitations....560
Cluster Statuses....560
Cluster Resources....561
NKP Platform Applications....562
Cluster Applications and Statuses....562
Custom Cluster Application Dashboard Cards....563
Kubernetes Cluster Federation (KubeFed)....564
Stopping an NKP Cluster....564
Backup and Restore....567
Velero Configuration....567
Velero Backup....585
Logging....589
Logging Operator....590
Logging Stack....591
Admin-level Logs....593
Workspace-level Logging....594
Multi-Tenant Logging....601
Fluent Bit....607
Configuring Loki to Use AWS S3 Storage in NKP....611
Customizing Logging Stack Applications....612
Security....613
OpenID Connect (OIDC)....614
Identity Providers....614
Login Connectors....615
Access Token Lifetime....616
Authentication....616
Connecting Kommander to an IdP Using SAML....617

Enforcing Policies Using Gatekeeper....618
Traefik-Forward-Authentication in NKP (TFA).... 621
Local Users....623
Networking....627
Networking Service....627
Required Domains.... 631
Load Balancing.... 632
Ingress....633
Configuring Ingress for Load Balancing....633
Istio as a Microservice.... 635
GPUs....637
Creating a GPU Image....637
Adding GPU to a Nutanix Cluster....638
Adding GPU Node Pool to a Nutanix Cluster....639
Configuring GPU for Kommander Clusters.... 640
Enabling the NVIDIA Platform Application on a Management Cluster....640
Enabling the NVIDIA Platform Application on Attached or Managed Clusters....642
Validating the Application....643
NVIDIA GPU Monitoring.... 644
Configuring MIG for NVIDIA.... 644
Troubleshooting NVIDIA GPU Operator on Kommander....645
Disabling NVIDIA GPU Operator Platform Application on Kommander....647
GPU Toolkit Versions....647
Enabling GPU After Installing NKP....648
Monitoring and Alerts....648
Recommendations.... 649
Grafana Dashboards....651
Cluster Metrics.... 653
Alerts Using AlertManager....653
Centralized Monitoring.... 658
Centralized Metrics.... 659
Centralized Alerts....660
Federating Prometheus Alerting Rules.... 660
Centralized Cost Monitoring....660
Application Monitoring using Prometheus.... 662
Setting Storage Capacity for Prometheus.... 664
Storage for Applications....665
Rook Ceph in NKP....665
Bring Your Own Storage (BYOS) to NKP Clusters....670

6. Custom Installation and Infrastructure Tools....677
Universal Configurations for all Infrastructure Providers....677
Container Runtime Engine (CRE).... 677
Configuring an HTTP or HTTPS Proxy....677
Output Directory Flag....682
Customization of Cluster CAPI Components.... 682
Registry and Registry Mirrors....682
Managing Subnets and Pods.... 684
Creating a Bastion Host....685
Provision Flatcar Linux OS....687
Load Balancers.... 687
Inspect Cluster for Issues....689
Nutanix Infrastructure.... 690
Nutanix Infrastructure Prerequisites....690
Nutanix Installation in a Non-air-gapped Environment....709

Nutanix Installation in an Air-Gapped Environment..... 722
Nutanix Management Tools..... 737
Configuring NTP on Nutanix NKP Clusters..... 738
Pre-provisioned Infrastructure..... 743
Pre-provisioned Prerequisites and Environment Variables..... 744
Pre-provisioned Cluster Creation Customization Choices..... 751
Pre-provisioned Installation in a Non-air-gapped Environment..... 762
Pre-provisioned Installation in an Air-gapped Environment..... 772
Pre-Provisioned Management Tools..... 784
AWS Infrastructure..... 791
AWS Prerequisites and Permissions..... 792
AWS Installation in a Non-air-gapped Environment..... 808
AWS Installation in an Air-gapped Environment..... 822
AWS Management Tools..... 838
EKS Infrastructure..... 855
EKS Introduction..... 855
EKS Prerequisites and Permissions..... 856
Creating an EKS Cluster from the CLI..... 864
Create an EKS Cluster from the UI..... 869
Granting Cluster Access..... 872
Exploring your EKS Cluster..... 872
Attach an Existing Cluster to the Management Cluster..... 875
Deleting the EKS Cluster from CLI..... 879
Deleting EKS Cluster from the NKP UI..... 880
Manage EKS Node Pools..... 882
Azure Infrastructure..... 884
Azure Prerequisites..... 884
Azure Non-air-gapped Install..... 889
Azure Management Tools..... 900
AKS Infrastructure..... 904
Use Nutanix Kubernetes Platform to Create a New AKS Cluster..... 905
Create a New AKS Cluster from the NKP UI..... 907
Explore New AKS Cluster..... 909
Delete an AKS Cluster..... 912
vSphere Infrastructure..... 912
vSphere Prerequisites..... 914
vSphere Installation in a Non-air-gapped Environment..... 922
vSphere Installation in an Air-Gapped Environment..... 937
vSphere Management Tools..... 954
VMware Cloud Director Infrastructure..... 963
VMware Cloud Director Prerequisites..... 964
Cloud Director Configure the Organization..... 967
Cloud Director Install NKP..... 976
Cloud Director Management Tools..... 986
Google Cloud Platform (GCP) Infrastructure..... 994
GCP Prerequisites..... 994
GCP Installation in a Non-air-gapped Environment..... 998
GCP Management Tools..... 1009

7. Additional Kommander Configuration..... 1016
Kommander Installation Based on Your Environment..... 1016
Installing Kommander in an Air-gapped Environment..... 1017
Installing Kommander in a Non-Air-gapped Environment..... 1021
Installing Kommander in a Pre-provisioned Air-gapped Environment..... 1024
Installing Kommander in a Pre-provisioned, Non-Air-gapped Environment..... 1029

Installing Kommander in a Small Environment....1032
Verifying Kommander Installation....1034
Configuring Applications After Installing Kommander....1035
Installing Kommander with a Configuration File....1036
Installling Kommander....1036
Dashboard UI Functions....1037
Logging into the UI with Kommander....1037
Default StorageClass....1038
Identifying and Modifying Your StorageClass....1038
Kommander Configuration Reference....1038
Configuring the Kommander Installation with a Custom Domain and Certificate....1042
Reasons for Setting Up a Custom Domain or Certificate....1042
Certificate Issuer and KommanderCluster Concepts....1043
Certificate Authority....1044
Certificate Configuration Options....1044
Advanced Configuration: ClusterIssuer....1047
Configuring a Custom Domain Without a Custom Certificate....1048
Verifying and Troubleshooting the Domain and Certificate Customization....1049
DNS Record Creation with External DNS....1050
Configuring External DNS with the CLI: Management or Pro Cluster....1050
Configuring the External DNS Using the UI....1052
Customizing the Traefik Deployment Using the UI....1053
Verifying Your External DNS Configuration....1054
Verifying Whether the DNS Deployment Is Successful....1054
Examining the Cluster's Ingress....1054
Verifying the DNS Record....1055
External Load Balancer....1056
Configuring Kommander to Use an External Load Balancer....1056
Configuring the External Load Balancer to Target the Specified Ports....1057
HTTP Proxy Configuration Considerations....1058
Configuring HTTP proxy for the Kommander Clusters....1058
Enabling Gatekeeper....1059
Creating Gatekeeper ConfigMap in the Kommander Namespace....1060
Installing Kommander Using the Configuration Files and ConfigMap....1061
Configuring the Workspace or Project....1061
Configuring HTTP Proxy in Attached Clusters....1061
Creating Gatekeeper ConfigMap in the Workspace Namespace....1061
Configuring Your Applications....1062
Configuring Your Application Manually....1062
NKP Catalog Applications Enablement after Installing NKP....1063
Configuring a Default Ultimate Catalog after Installing NKP....1063
NKP Catalog Application Labels....1064

8. Additional Konvoy Configurations....1065
FIPS 140-2 Compliance....1065
FIPS Support in NKP....1065
Infrastructure Requirements for FIPS 140-2 Mode....1066
Deploying Clusters in FIPS Mode....1066
FIPS 140 Images: Non-Air-Gapped Environments....1067
FIPS 140 Images: Air-gapped Environment....1067
Validate FIPS 140 in Cluster....1068
FIPS 140 Mode Performance Impact....1069
Registry Mirror Tools....1069
Air-gapped vs. Non-air-gapped Environments....1069
Local Registry Tools Compatible with NKP....1070

Using a Registry Mirror....1071
Seeding the Registry for an Air-gapped Cluster....1073
Configure the Control Plane....1074
Modifying Audit Logs....1074
Viewing the Audit Logs....1079
Updating Cluster Node Pools....1080
Cluster and NKP Installation Verification....1081
Checking the Cluster Infrastructure and Nodes....1081
Monitor the CAPI Resources....1082
Verify all Pods....1082
Troubleshooting....1082
GPU for Konvoy....1082
Delete a NKP Cluster with One Command....1083

9. Konvoy Image Builder....1084
Creating an Air-gapped Package Bundle....1086
Use KIB with AWS....1087
Creating Minimal IAM Permissions for KIB....1087
Integrating your AWS Image with NKP CLI....1090
Create a Custom AMI....1091
KIB for EKS....1097
Using KIB with Azure....1097
KIB for AKS....1100
Using KIB with GCP....1100
Building the GCP Image....1101
Creating a Network (Optional)....1102
Using KIB with GPU....1102
Verification....1104
Using KIB with vSphere....1104
Create a vSphere Base OS Image....1105
Create a vSphere Virtual Machine Template....1106
Using KIB with Pre-provisioned Environments....1111
Customize your Image....1112
Customize your Image YAML or Manifest File....1112
Customize your Packer Configuration....1114
Adding Custom Tags to your Image....1116
Ansible Variables....1117
Use Override Files with Konvoy Image Builder....1118
Konvoy Image Builder CLI....1127
konvoy-image build....1127
konvoy-image completion....1131
konvoy-image generate-docs....1134
konvoy-image generate....1134
konvoy-image provision....1136
konvoy-image upload....1136
konvoy-image validate....1137
konvoy-image version....1138

10. Nutanix Image Builder....1139
Using an Image provided by Nutanix....1139
Building a Custom Image....1140
Building a Custom Image Use Cases....1141

11. Upgrade NKP 1144
Upgrade Compatibility Tables 1145
Supported Operating Systems 1145
Konvoy Image Builder 1145
Supported Kubernetes Cluster Versions 1145
Upgrading Cluster Node Pools 1146
Upgrade Prerequisites 1147
Prerequisites for the Kommander Component 1147
Prerequisites for the Konvoy Component 1148
Upgrade: For Air-gapped Environments Only 1149
Downloading all Images for Air-gapped Deployments 1149
Extracting Air-gapped Images and Set Variables 1150
Loading Images for Deployments - Konvoy Pre-provisioned 1150
Load Images to your Private Registry - Konvoy 1151
Load Images to your Private Registry - Kommander 1151
Load Images to your Private Registry - NKP Catalog Applications 1152
Next Step 1152
Upgrade NKP Ultimate 1152
Ultimate: For Air-gapped Environments Only 1153
Ultimate: Upgrade the Management Cluster and Platform Applications 1154
Ultimate: Upgrade Platform Applications on Managed and Attached Clusters 1156
Ultimate: Upgrade Workspace NKP Catalog Applications 1157
Ultimate: Upgrade Project Catalog Applications 1162
Ultimate: Upgrade Custom Applications 1163
Ultimate: Upgrade the Management Cluster CAPI Components 1164
Ultimate: Upgrade the Management Cluster Core Addons 1164
Ultimate: Upgrading the Management Cluster Kubernetes Version 1166
Ultimate: Upgrading Managed Clusters 1169
Ultimate: Upgrade images used by Catalog Applications 1171
Upgrade NKP Pro 1173
NKP Pro: For Air-gapped Environments Only 1174
NKP Pro: Upgrade the Cluster and Platform Applications 1175
NKP Pro: Upgrade the Cluster CAPI Components 1176
NKP Pro: Upgrade the Cluster Core Addons 1177
NKP Pro: Upgrading the Kubernetes Version 1178

12. Troubleshooting Guide 1183
Gather Environment Data for Nutanix NKP Support 1183
Generate a Support Bundle 1183
Using SSH Fallback and Ansible 1185
Create Diagnostic Bundles using Custom Collectors 1186
Common Information Discovery Commands 1189
Application Discovery Commands 1190
Workspace Discovery Commands 1191
Application Troubleshooting 1192
Applications Context 1192
Troubleshooting Approach 1192
Related Topics 1193
Applications Deployment CLI vs UI 1193
Application YAML Customizations 1197
Overriding ALL Clusters within a Workspace 1197
Available Customizations 1198
How do I Customize? 1199

Enable or Disable an App per Cluster....1200
Object Locations and Explanations....1201
Kommander....1202
Kommander Resource Locations....1203

13. AI Navigator....1205
AI Navigator User Agreement and Guidelines....1205
Accept the User Agreement....1205
AI Navigator Guidelines....1206
AI Navigator Installation and Upgrades....1207
Installing AI Navigator....1207
Disabling AI Navigator....1207
Upgrades to 2.7.0 or Later....1207
Related Information....1208
Accessing the AI Navigator....1208
AI Navigator Queries....1210
What Goes in a Prompt....1210
Inline Commands or Code Snippets....1210
Code Blocks....1212
Selected Prompt Examples....1212
AI Navigator Cluster Info Agent: Obtain Live Cluster Information....1214
Enable the NKP AI Navigator Cluster Info Agent....1215
Customizing the AI Navigator Cluster Info Agent....1215
Data Privacy FAQs....1215

14. Nutanix Kubernetes Platform Insights Guide....1217
Nutanix Kubernetes Platform Insights Overview....1217
NKP Insights Alert Table....1217
NKP Insights Engine....1218
NKP Insights Architecture....1218
Nutanix Kubernetes Platform Insights Setup....1218
NKP Insights Resource Requirements....1218
NKP Insights Setup and Configuration....1219
Grant View Rights to Users Using the UI....1221
Uninstall NKP Insights....1224
NKP Insights Bring Your Own Storage (BYOS) to Insights....1227
Requirements....1228
Create a Secret to support BYOS for Nutanix Kubernetes Platform Insights....1228
Helm Values for Insights Storage....1228
Installing NKP Insights Special Storage in the UI....1229
Installing Insights Storage using CLI....1230
Installing Nutanix Kubernetes Platform Insights in an Air-gapped Environment....1230
Manually Creating the Object Bucket Claim....1231
Upgrade NKP Insights to 1.0.0....1232
Nutanix Kubernetes Platform Insights Alerts....1232
Resolving or Muting Alerts....1233
Viewing Resolved or Muted Alerts....1234
Insight Alert Usage Tips....1235
NKP Insight Alert Details....1235
NKP Insights Alert Notifications With Alertmanager....1235
Enable NKP-Related Insights Alerts....1243
Configuration Anomalies....1244

1

NUTANIX KUBERNETES PLATFORM OVERVIEW

Architecture

Kubernetes® creates the foundation for the Nutanix Kubernetes® Platform (NKP) cluster. This topic briefly overviews the native Kubernetes architecture, a simplified version of the NKP architecture for both Pro and Ultimate versions. The architecture also depicts the operational workflow for an NKP cluster.

NKP for Kubernetes v1.29.6. Kubernetes® is a registered trademark of The Linux Foundation in the United States and other countries and is used according to a license from The Linux Foundation.

Components of the Kubernetes Control Plane

The native Kubernetes cluster consists of components in the cluster's control plane and worker nodes that run containers and maintain the runtime environment.

NKP supplements the native Kubernetes cluster by including a pre-defined and pre-configured set of applications. As the pre-defined set of applications critical features for managing a Kubernetes cluster in a production environment, the default set is identified as the NKP platform applications.

To view the full set of NKP platform services, see Platform Applications on page 400.

The following illustration depicts the NKP architecture and the workflow of the key components:

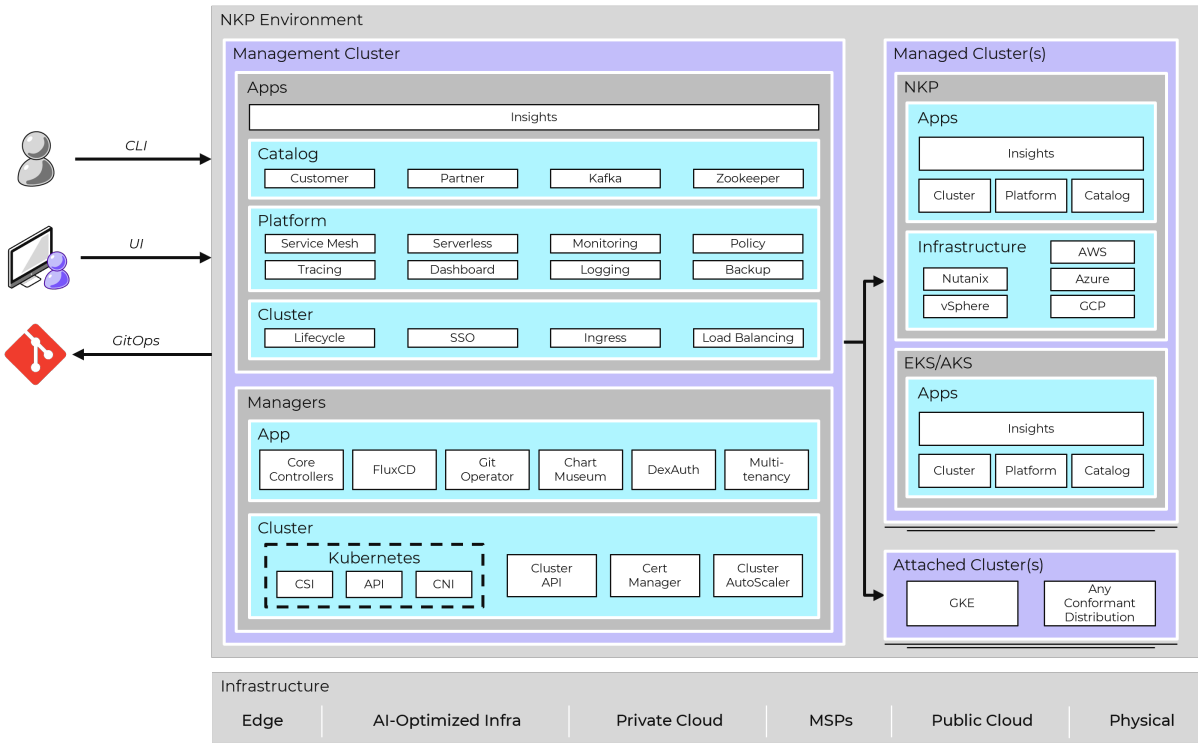


Figure 1: NKP Architecture

Related Information

For information on related topics or procedures, see the Kubernetes documentation.

Supported Infrastructure Operating Systems

This topic contains all the supported and tested operating systems (OS) that are currently supported for use with Nutanix Kubernetes Platform (NKP).

> **Note:** Arm-based systems are not supported. Only x86/amd64 architectures are supported.

Table 1: Nutanix

Operating System	Kernel	Default Config	FIPS	Air-gapped	FIPS Air-gapped	GPU Support	GPU Air-gapped	Nutanix Image Builder
Rocky Linux 9.4	5.14.0-162.6.1.el9_1.x86_64	Yes	-	Yes	-	-	-	Yes
Ubuntu 22.04	5.15.0-91-generic	Yes	-	Yes	-	Yes	Yes	Yes

Table 2: Amazon Web Services (AWS)

Operating System	Kernel	Default Config	FIPS	Air-gapped	FIPS Air-gapped	GPU Support	GPU Air-gapped	Konvoy Image Builder
RHEL 8.6	5.14.0-162.6.1.el9_1.x86_64	Yes	Yes	Yes	Yes	Yes	Yes	Yes
RHEL 8.8	5.14.0-162.6.1.el9_1.x86_64	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Ubuntu 18.04 (Bionic Beaver)	5.4.0-1103-aws	Yes	-		-	-	-	Yes
Ubuntu 20.04 (Focal Fossa)	5.15.0-1068-aws	Yes	-	-	-	Yes	-	Yes
Oracle Linux RH 8.9	4.18.0-513.5.1.el8_9.x86_64	Yes	Yes	-	-	-	-	Yes
Rocky Linux 9.1	5.14.0-162.12.1.el9_1.0.2.x86_64	Yes		Yes				Yes
Flatcar 3033.3.x	5.10.198-flatcar	Yes	-	-	-	-	-	Yes

Table 3: Microsoft Azure

Operating System	Kernel	Default Config	FIPS	Air-gapped	FIPS Air-gapped	GPU Support	GPU Air-gapped	Konvoy Image Builder
Ubuntu 20.04 (Focal Fossa)	5.15.0-1070-azure	Yes	-	-	-	-	-	Yes
Rocky Linux 9.1	5.14.0-162.12.1.el9_1.0.2.x86_64	Yes	Yes		-	-	-	Yes

Table 4: Google Cloud Platform (GCP)

Operating System	Kernel	Default Config	FIPS	Air-gapped	FIPS Air-gapped	GPU Support	GPU Air-gapped	Konvoy Image Builder
Ubuntu 18.04	5.4.0-1106-gcp	Yes	-	-	-	-	-	Yes
Ubuntu 20.04	5.15.0-1047-gcp	Yes	-	-	-	-	-	Yes

Table 5: Pre-provisioned

Operating System	Kernel	Default Config	FIPS	Air-gapped	FIPS Air-gapped	GPU Support	GPU Air-gapped	Konvoy Image Builder
RHEL 8.6	4.18.0-372.70.1.el8_6.x86_64	Yes	Yes	Yes	Yes	Yes	Yes	Yes
RHEL 8.8	4.18.0-477.36.1.el8_8.x86_64	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Flatcar 3033.3.x	5.10.198-flatcar	Yes	-	-	-	-	-	Yes
Ubuntu 20.04	5.15.0-1048-aws	Yes	-	-	-	Yes	-	Yes
Oracle Linux RH 8.9	4.18.0-513.5.1.el8_9.x86_64	Yes	-	-	-	-	-	Yes
Rocky Linux 9.1	5.14.0-162.12.1.el9_1.0.2.x86_64	Yes	Yes	Yes				Yes

Table 6: Pre-provisioned on Azure

Operating System	Kernel	Default Config	FIPS	Air-gapped	FIPS Air-gapped	GPU Support	GPU Air-gapped	Konvoy Image Builder
RHEL 8.6	4.18.0-372.91.1.el8_6.x86_64	Yes	Yes	Yes	Yes	-	-	Yes

Table 7: vSphere

Operating System	Kernel	Default Config	FIPS	Air-gapped	FIPS Air-gapped	GPU Support	GPU Air-gapped	Konvoy Image Builder
RHEL 8.6	4.18.0-372.9.1.el8.x86_64	Yes	Yes	Yes	Yes	-	-	Yes
RHEL 8.8	4.18.0-477.10.1.el8_8.x86_64	Yes	Yes	Yes	Yes	-	-	Yes
Ubuntu 20.04	5.4.0-125-generic	Yes	-	Yes	-	-	-	Yes
Ubuntu 22.04	5.15.0-122-generic	Yes	-	-	-	-	-	Yes
Rocky Linux 9.1	5.14.0-162.6.1.el9_1.x86_64	Yes	Yes	Yes	-	-	-	Yes
Flatcar 3033.3.x	5.10.188-flatcar	Yes	-	-	-	-	-	Yes

Table 8: VMware Cloud Director (VCD)

Operating System	Kernel	Default Config	FIPS	Air-gapped	FIPS Air-gapped	GPU Support	GPU Air-gapped	Konvoy Image Builder
Ubuntu 20.04	5.4.0-125-generic	Yes	-	-	-	-	-	Yes

Table 9: Amazon Elastic Kubernetes (EKS)

Operating System	Kernel	Default Config	FIPS	Air-gapped	FIPS Air-gapped	GPU Support	GPU Air-gapped	Konvoy Image Builder
Amazon Linux 2 v7.9	5.10.223-211.872.amzn2.x86_64	-	-	-	-	-	-	-

Table 10: Azure Kubernetes Service (AKS)

Operating System	Kernel	Default Config	FIPS	Air-gapped	FIPS Air-gapped	GPU Support	GPU Air-gapped	Konvoy Image Builder
Ubuntu 22.04.2 LTS	5.15.0-1068-azure	Yes	-	-	-	-	-	-

2

DOWNLOADING NKP

You can download NKP from the Nutanix Support portal.

Procedure

1. From the Nutanix Download Site, select the NKP binary for either Darwin (MacOS) or Linux OS.
2. Extract the `.tar` file that is compatible with your OS as follows.
   - For *MacOS or Darwin*:
     1. Right-click on the `.tar` file using a file manager program such as 7-Zip.
     2. Select **Open With** and then **7-Zip File Manager**.
     3. Select **Extract** and choose a location to save the extracted files.
   - For *Linux*:
     1. To extract the `.tar` file to the current directory using the CLI, type `tar -xvf filename.tar`.
     2. If the file is compressed with gzip, add the `tar -xvzf` option.

3

GETTING STARTED WITH NKP

At Nutanix, we partner with you throughout the entire cloud-native journey as follows:

About this task

- Help you in getting started with Nutanix Kubernetes Platform (NKP) is the planning phase that introduces definitions and concepts.
- Guide you with the Basic Installations by Infrastructure on page 52 through the NKP software installation and start-up.
- Guide you with the Cluster Operations Management on page 353, which involves customizing applications and managing operations.

You can install in multiple ways:

- On Nutanix infrastructure.
- On a public cloud infrastructure, such as Amazon Web Services (AWS), Google Cloud Platform (GCP), or Azure.
- On an internal network, on-premises environment, or with a physical or virtual infrastructure.
- On an air-gapped environment.
- With or without Federal Information Processing Standards (FIPS) and graphics processing unit (GPU).

Before you install NKP:

Procedure

1. Complete the prerequisites (see Prerequisites for Installation on page 47) required to install NKP.
2. Determine the infrastructure (see Resource Requirements on page 39) on which you want to deploy NKP.
3. After you choose your environment, download NKP, and select the Basic Installations by Infrastructure on page 52 for your infrastructure provider and environment.

   The basic installations set up the cluster with the Konvoy component and then install the Kommander component to access the dashboards through the NKP UI. The topics in the Basic Installations by Infrastructure on page 52 chapter help you explore ##NKP# and prepare clusters for production to deploy and enable the applications that support Cluster Operations Management on page 353.
4. (Optional) After you complete the basic installation and are ready to customize, perform Custom Installation and Additional Infrastructure Tools, if required.
5. To prepare the software, perform the steps described in the Cluster Operations Management chapter.
6. Deploy and test your workloads.

NKP Concepts and Terms

This topic describes the terminology used in Nutanix Kubernetes Platform (NKP) to help you understand important NKP concepts and terms.

NKP is composed of three main components: Konvoy, Kommander, and Konvoy Image Builder (KIB). These three components work together to provide a single and centralized control point for an organization's application infrastructure. NKP empowers organizations to deploy, manage, and scale Kubernetes workloads in production environments more efficiently.

Each of the three main components specifically manages the following:

- *Konvoy* is the cluster life cycle manager component of NKP. Konvoy relies on Cluster Application Programming Interface (API), Calico, and other open-source and proprietary software to provide simple cluster life cycle management for conformant Kubernetes clusters with networking and storage capabilities.

  Konvoy uses industry-standard tools to provision certified Kubernetes clusters on multiple cloud providers, vSphere, and on-premises hardware in connected and air-gapped environments. Konvoy contains the following components:

  Cluster Manager consists of Cluster API, Container Storage Interface (CSI), Container Network Interface (CNI), Cluster AutoScaler, Cert Manager, and MetalLB.

  For *Networking*, Kubernetes uses CNI (Container Network Interface) as an interface between network infrastructure and Kubernetes pod networking. In NKP, the Nutanix provider uses the Cilium CNI. All other providers use Calico CNI.

  The Konvoy component is installed according to the cluster's infrastructure. Remember:

  1. To install NKP quickly and without much customization, see Basic Installations by Infrastructure on page 52.
  2. To choose more environments and cluster customizations, see Custom Installation and Additional Infrastructure Tools.

- *Kommander* is the fleet management component of NKP. Kommander delivers centralized observability, control, governance, unified policy, and better operational insights. With NKP Pro, Kommander manages a single Kubernetes cluster.

  In NKP Ultimate, Kommander supports attaching workload clusters and life cycle management of clusters using Cluster API. NKP Ultimate also offers life cycle management of applications through FluxCD. Kommander contains the following components:

  - User interface, Security, Observability, Networking, and Application Management.
  - Platform Applications: Applications such as observability, cost management, monitoring, and logging are available with NKP and making NKP clusters production-ready right through. Platform applications are a choice of selected applications from the open-source community consumed by the platform.
  - Pro Platform Applications: Monitoring, Logging, Backup or Restore, Policy Agent, External DNS, Load Balance, Ingress, SSO, Service Mesh.
  - Ultimate Platform Applications: Includes all of the Pro Platform applications, plus additional Access Control and Centralized Cost Management.
  - Catalog Applications: Applications in NKP Ultimate that are deployed to be used for customer workloads, such as Kafka, Spark, and ZooKeeper.

  The Kommander component is installed according to the cluster's environment type. For more information, see Installing Kommander by Environment .

- *Konvoy Image Builder (KIB)* creates Cluster API-compliant machine images. It configures only those images to contain all the necessary software to deploy Kubernetes cluster nodes. For more information, see Konvoy Image Builder.

Cluster Types

Cluster types such as Management clusters, Managed clusters, and Attached clusters are key concepts in understanding and getting the most out of Nutanix Kubernetes Platform (NKP) Pro versus Ultimate environments.

Multi-cluster Environment

- **Management Cluster**: Is the cluster where you install NKP, and it is self-managed. In a multi-cluster environment, the Management cluster also manages other clusters. Customers with an Ultimate license need to run workloads on **Managed** and **Attached** clusters, not on the **Management** cluster. For more information, see License Packaging.
- **Managed Cluster**: Also called an “NKP cluster,” this is a type of workload cluster that you can create with NKP. The NKP Management cluster manages its infrastructure, its life cycle, and its applications.
- **Attached Cluster**: This is a type of workload cluster that is created outside of NKP but is then connected to the NKP Management Cluster so that NKP can manage it. In these cases, the NKP Management cluster only manages the attached cluster’s applications.

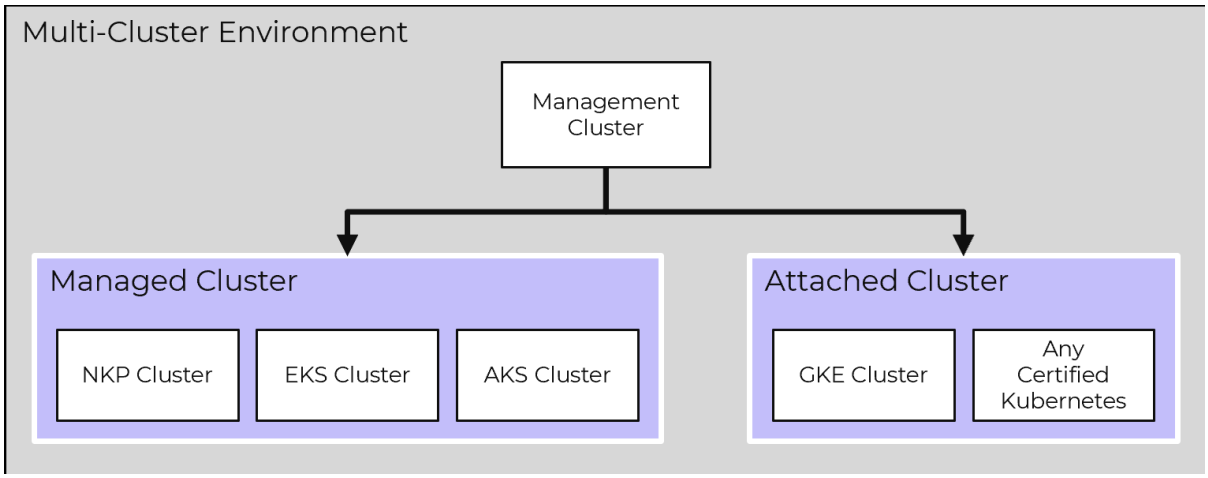


Figure 2: Multi-cluster Environment

Single-cluster Environment

NKP Pro Cluster: Is the cluster where you install NKP. A NKP Pro cluster is a stand-alone cluster. It is self-managed and, therefore, capable of provisioning itself. In this single-cluster environment, you cannot attach other clusters; all workloads are run on your NKP Pro cluster. You can, however, have several separate NKP Pro instances, each with its own license.

Customers with a Pro license can run workloads on their NKP Pro cluster.

> **Note:** If you have not decided which license to get but plan on adding one or several clusters to your environment and managing them centrally, Nutanix recommends obtaining a license.

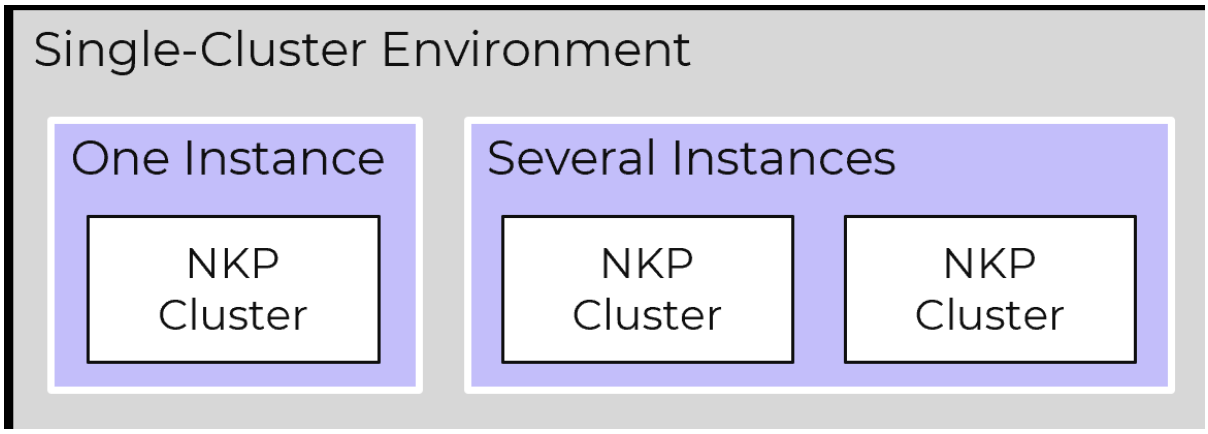


Figure 3: Single-cluster Environment

Self-Managed Cluster

In the Nutanix Kubernetes Platform (NKP) landscape, the NKP Starter, Pro, and Ultimate Management clusters are self-managed. You can run multiple self-managed management clusters using NKP Starter, Pro, and Ultimate. Self-managed clusters manage the provisioning and deployment of their own nodes through CAPI controllers. The CAPI controllers are a managing entity that automatically manages the life cycle of a cluster's nodes based on a customizable definition of the resources.

A self-managed cluster is one in which the CAPI resources and controllers that describe and manage it run on the same cluster they are managing. As part of the underlying processing using the `--self-managed` flag, the NKP CLI does the following:

- Creates a bootstrap cluster.
- Creates a workload cluster.
- Moves CAPI controllers from the bootstrap cluster to the workload cluster, making it self-managed.
- Deletes the bootstrap cluster.

Network-Restricted Cluster

A network-restricted cluster is not the same as an air-gapped cluster.

A firewall secures a network-restricted or firewalled cluster, Perimeter Network, Network Address Translation (NAT) gateway, or proxy server requires additional access information. Network-restricted clusters are usually located in remote locations or at the edge and, therefore, not in the same network as the Management cluster.

The main difference between network-restricted and air-gapped clusters is that network-restricted clusters can reach external networks (like the Internet), but their services or ingresses cannot be accessed from outside. Air-gapped clusters, however, do not allow ingress or egress traffic.

In a multi-cluster environment, NKP supports attaching a network-restricted cluster to an NKP Management cluster. You can also enable a proxied access pipeline through the Management cluster, which allows you to access the network-restricted cluster's dashboards without being in the same network.

CAPI Concepts and Terms

Nutanix Kubernetes Platform (NKP) uses ClusterAPI (CAPI) technology to create and manage the life cycle of Kubernetes Clusters. A basic understanding of CAPI concepts and terms helps understand how to install and maintain NKP. You can find a deeper discussion of the architecture in the ClusterAPI Book.

CAPI makes use of a bootstrap cluster for provisioning and starting clusters. A bootstrap cluster handles the following actions:

- Generating the cluster certificates if they are not otherwise specified.
- Initialize the control plane and manage the creation of other nodes until it is complete.
- Joining control plane and worker nodes to the cluster.
- Installing and configuring the networking plugin (Calico CNI), Container Storage Interface (CSI) volume provisioners, and cluster autoscalercore Kubernetes components.

BootstrapData

BootstrapData is machine or node role-specific data, such as cloud initialization data, used to bootstrap a “machine” onto a node.

For customers using Kommander for multi-cluster management, a **management cluster** manages the life cycle of workload clusters. As the management cluster, NKP Kommander works with bootstrap and infrastructure providers and maintains cluster resources such as bootstrap configurations and templates. If you are working with only one cluster, Kommander will provide you with add-on (platform application) management for that cluster but not others.

Workload Cluster

A workload cluster is a Kubernetes cluster whose life cycle is managed by a management cluster. It provides the platform to deploy, execute, and run workloads.

These additional concepts are essential for understanding the upgrade. They are part of a collection of Custom Resource Definitions (CRDs) that extend the Kubernetes API.

ClusterResourceSet

A ClusterResourceSet Kubernetes cluster created by CAPI is functionally minimal. Crucial components like CSI and CNI are not in the default cluster spec. A ClusterResourceSet is a custom resource definition (CRD) that can be used to group and deploy core cluster components after the installation of the Kubernetes cluster.

When you create a bootstrap cluster, you can find all the components in the default namespace, and we move them to the workload cluster while making the cluster self-managed.

A machine is a declarative specification for a platform or infrastructure component that hosts a Kubernetes node as a bare metal server or a VM. CAPI uses provider-specific controllers to provision and install new hosts that register as nodes. When you update a machine spec other than for specific values, such as annotations, status, and labels, the controller deletes the host and creates a new one that conforms to the latest spec. This is called machine immutability. If you delete a machine, the controller deletes the infrastructure and the node. Provider-specific information is not portable between providers.

Within CAPI, you use declarative **MachineDeployments** to handle changes to machines by replacing them like a core Kubernetes Deployment replaces Pods. MachineDeployments reconcile changes to machine specs by rolling out changes to two **MachineSets** (similar to a ReplicaSet), both the old and the newly updated.

MachineHealthCheck

A MachineHealthCheck (MHC) identifies unhealthy node conditions and initiates remediation for nodes owned by a MachineSet.

In a MachineHealthCheck, the default value of `maxunhealthy` is set to 40%. You can customize this value as per your requirements for self-healing.

Related Information

For information on related topics or procedures, see:

- ClusterAPI Book: https://cluster-api.sigs.k8s.io/user/concepts.html
- Customizing CAPI Components for a Cluster: Customization of Cluster CAPI Components on page 682

Configuring Self-healing
In a MachineHealthCheck (MHC) resource, you can customize the value of `spec.maxUnhealthy` as per your requirements for self-healing.

About this task

To configure self-healing, follow these steps:

Procedure

1. Open a command line interface (CLI).
2. View the MachineHealthCheck resources:

   ```
   $ kubectl get MachineHealthCheck
   ```

   The list of MachineHealthCheck resources are displayed. For example:

   ```
   $ kubectl get MachineHealthCheck
   NAME              CLUSTER   EXPECTEDMACHINES   MAXUNHEALTHY   CURRENTHEALTHY   AGE
   nkp-j5h4n         nkp       3                  40%            3                8h
   nkp-md-0-qgxr8    nkp       4                  40%            4                8h
   ```

   The MachineHealthChecks for a control node pool named `nkp-j5h4n` and a worker node pool named `nkp-md-0-qgxr8` are displayed.
3. Edit a MachineHealthCheck resource:

   ```
   $ kubectl edit machinehealthcheck <mhc-name>
   ```

   Replace `<mhc-name>` with the name of the MachineHealthCheck resource of the control pane node pool or the worker pane node pool.
4. Search for `spec.maxUnhealthy`.
   The default value is `40%`.
5. Update the value of `spec.maxUnhealthy` as per your requirements for self-healing.
   For example, you can update the value from `40%` to `50%`:

   ```
   [...]
   spec:
     clusterName: nkp
     maxUnhealthy: 50%
     nodeStartupTimeout: 10m0s
   [...]
   ```
6. Save the changes.
7. Verify this update:

   ```
   $ kubectl describe machinehealthcheck <mhc-name>maxUnhealthmaxUnhealth
   ```

Air-Gapped or Non-Air-Gapped Environment

How to know if you are in an air-gapped or a non-air-gapped environment?

Air-Gapped Environments

In an air-gapped environment, your environment is isolated from unsecured networks like the Internet. Running your workloads in an air-gapped environment is expected in scenarios where security is a determining factor. Nutanix Kubernetes Platform (NKP) in air-gapped environments allows you to manage your clusters while shielding them from undesired external influences.

You can create an air-gapped cluster in on-premises environments or any other environment. In this configuration, you are responsible for providing an image registry. You must also retrieve required artifacts and configure NKP to use those from a local directory when creating and managing NKP clusters. See Supported Infrastructure Operating Systems on page 12.

Air-gapped environments provide secure interactions with other networks. You can perform actions in several ways that require incoming data from other networks regardless of your environment's isolation. In some configurations, air-gapped clusters allow inbound connections but cannot initiate outbound connections. In other configurations, you can set up a bastion host, which serves as a gateway between the Internet (or other untrusted networks) and your environment and facilitates the download of install, upgrade, etc., files and images that are required for other machines to run in air-gapped environments.

Common Industry Synonyms: Fettered, disconnected, restricted, Session Initiation Protocol (SIPREC), etc.

Non-Air-Gapped Environments

In a non-air-gapped environment, two-way access to and from the Internet exists. You can create a non-air-gapped cluster on pre-provisioned (on-premises) environments or any cloud infrastructure.

NKP in a non-air-gapped environment allows you to manage your clusters while facilitating connections and offering integration with other tools and systems.

Common Industry Synonyms: Open, accessible (to the Internet), not restricted, Non-classified Internet Protocol (IP) Router Network (NIPRNet), etc.

Pre-provisioned Infrastructure

The pre-provisioned infrastructure allows the deployment of Kubernetes using Nutanix Kubernetes Platform (NKP) to pre-existing machines. Other providers, such as vSphere, AWS, or Azure, create or provision the machines before Kubernetes is deployed. On most infrastructures (including vSphere and cloud providers), NKP provisions the actual nodes automatically as part of deploying a cluster. It creates the virtual machine (VM) using the appropriate image and then handles the networking and installation of Kubernetes.

However, NKP can also work with pre-provisioned infrastructure in which you provision the VMs for the nodes. You can pre-provision nodes for NKP on bare metal, vSphere, or cloud. Pre-provisioned and vSphere combine the physical (on-premises bare metal) and virtual servers (VMware vSphere).

Usage of Pre-provisioned Environments

Pre-provisioned environments is often used in bare metal deployments, where you deploy your OS (see Cluster Types on page 19 (such as Red Hat Enterprise Linux (RHEL) or Ubuntu, and so on) on physical machines. Creating a pre-provisioned cluster as an Infrastructure Operations Manager, you are responsible for allocating compute resources, setting up networking, and collecting IP and Secure Shell (SSH) information to NKP. You can then provide all the required details to the pre-provisioned provider to deploy Kubernetes. These operations are done manually or with the help of other tools.

In pre-provisioned environments, NKP handles your cluster's life cycle (installation, upgrade, node management, and so on). NKP installs Kubernetes, performs monitoring and logging applications, and has its own UI.

The main use cases for the pre-provisioned provider are:

- On-premises clusters.

- Cloud or Infrastructure as a Service (IaaS) environments that do not currently have a Nutanix-supported infrastructure provider.
- Cloud environments, you must use pre-defined infrastructure instead of having one of the supported cloud providers create it for you.

In an environment with access to the Internet, you can retrieve artifacts from specialized repositories dedicated to them, such as Docker images from the DockerHub and Helm Charts from a dedicated Helm Chart repository. However, in an air-gapped environment, you need local repositories to store Helm charts, Docker images, and other artifacts. Tools such as JFrog, Harbor, and Nexus handle multiple types of artifacts in a single local repository.

Related Information

For information on related topics or procedures see Pre-provisioned Installation Options on page 69.

Licenses

This chapter describes the Nutanix Kubernetes Platform (NKP) licenses. The license type you subscribe to determines what NKP features are available to you. Features compatible with all versions of NKP can be activated by purchasing additional Add-on licenses.

> **Note:** You cannot downgrade a license once you register the license for a cluster. For example, you cannot downgrade to the Pro license once you register for the Ultimate license.

The NKP licenses available are:

- NKP Starter License on page 26
- NKP Pro License on page 27
- NKP Ultimate License on page 29

Table 11: Feature Support Matrix

Feature	NKP Starter	NKP Pro	NKP Ultimate
APPLICATIONS			
Workspace Platform Applications		X	X
Prometheus		X	X
Kubernetes Dashboard		X	X
Reloader	X	X	X
Traefik	X	X	X
Project Level Platform Applications			X
Catalog Applications (Kafka and Zookeeper)			X
Custom Applications			X
Partner Applications			X
COST MONITORING (Kubecost)			
AI OPS			X
Insights			X

Feature	NKP Starter	NKP Pro	NKP Ultimate
AI Navigator		X	X
CLUSTER MANAGEMENT			
LCM Management Cluster	X	X	X
LCM Workload Clusters	X	X	X
Workload Cluster Creation using UI, CLI, or YAML	X	X	X
Attaching Workload Cluster			X
Upgrade Management Cluster	X	X	X
Upgrade Workload Clusters	X	X	X
Third-Party Kubernetes Management			
LCM of EKS Cluster			X
LCM of AKS Cluster			X
GITOPS			
Continuous Deployment (FluxCD)			X
FluxCD (as an application)		X	X
UX			
NKP CLI	X	X	X
NKP UI	X	X	X
Workspaces Management			X
Projects			X
Add new Infrastructure Provider			X
LOGGING			
Workspace Level Logging		X	X
Fluentbit		X	X
Multi-tenant Logging			X
ADDITIONAL TOOLING			
Backup & Restore based on Velero		X	X
NVIDIA GPU		X	X
SECURITY			
Single Sign On	X	X	X
Policy control using Gatekeeper		X	X
CLUSTER PROVISIONING			
NKP on Nutanix Infrastructure	X	X	X
NKP on AWS		X	X
NKP on Azure		X	X
NKP on GCP		X	X

Feature	NKP Starter	NKP Pro	NKP Ultimate
NKP on vSphere		X	X
Pre-provisioned		X	X
VMware Cloud Director		X	X
EKS Provisioning			X
Multi-Cloud, Hybrid Cloud (Management and Workload clusters on different infrastructures)			X
SECURITY			
FIPS Compliant Build		X	X
Konvoy Image Builder or Bring your own OS		X	X
Nutanix provided OS Image (Rocky Linux)	X	X	X
Air-Gapped Deployments	X	X	X
RBAC			
RBAC - Admin role only	X	X	X
RBAC - Kubernetes	X	X	X
NKP RBAC		X	X
Customize UI Banners		X	X
Upload custom Logo			X

Purchase of a License

NKP Licenses are sold in units of cores. To learn more about licenses and to obtain a valid license:

- See the Nutanix Cloud Platform Software Options page.
- Contact a Nutanix sales representative.
- Download the binary files from the Nutanix Support portal.

NKP Starter License

Nutanix Kubernetes Platform (NKP) Starter license is a Kubernetes solution that offers a feature-rich, easy-to-deploy, and easy-to-manage entry-level cloud container platform. The NKP Starter license provides access to the entire Konvoy cluster environment and basic access to the NKP UI dashboard. NKP Starter can only be deployed on the Nutanix Infrastructure.

NKP Starter is bundled with *NKP FullStack Starter*, *NCI Pro*, and *NCI Ultimate*.

Compatible Infrastructure

NKP Starter operates across Nutanix's entire range of on-premises, edge, and air-gapped infrastructures and has support for Rocky Linux-managed operating system. To view the supported Rocky Linux operating system version, see Supported Infrastructure Operating Systems on page 12.

To understand the NKP Starter cluster in one of the listed environments of your choice, see Basic Installations by Infrastructure on page 52 or Custom Installation and Infrastructure Tools on page 677.

Cluster Manager

Konvoy is the Kubernetes installer component of NKP Starter that uses industry-standard tools to create a certified Kubernetes cluster. These industry standard tools create a cluster management system that includes:

- *Control Plane*: Manages the worker nodes and pods in the cluster.
- *Worker Nodes*: Used to run containerized applications and handle networking to ensure that the traffic between applications across and outside the cluster is facilitated correctly.
- *Container Networking Interface (CNI)*: Cilium's open-source networking and network security solution for containers, virtual machines, and native host-based workloads.
- *Container Storage Interface (CSI)*: A common abstraction to container orchestrations for interacting with storage subsystems of various types.
- *Kubernetes Cluster API (CAPI)*: Cluster API uses Kubernetes-style APIs and patterns to automate cluster life cycle management for platform operators. For more information on how CAPI is integrated into NKP Pro, see CAPI Concepts and Terms on page 20.
- *Cert Manager*: A Kubernetes addon to automate the management and issuance of TLS certificates from various issuing sources.
- *Cluster Autoscaler*: A component that automatically adjusts the size of a Kubernetes cluster so that all pods have a location to run and there are no unwanted nodes.

Platform Applications

NKP Starter deploys only a basic set of platform applications.

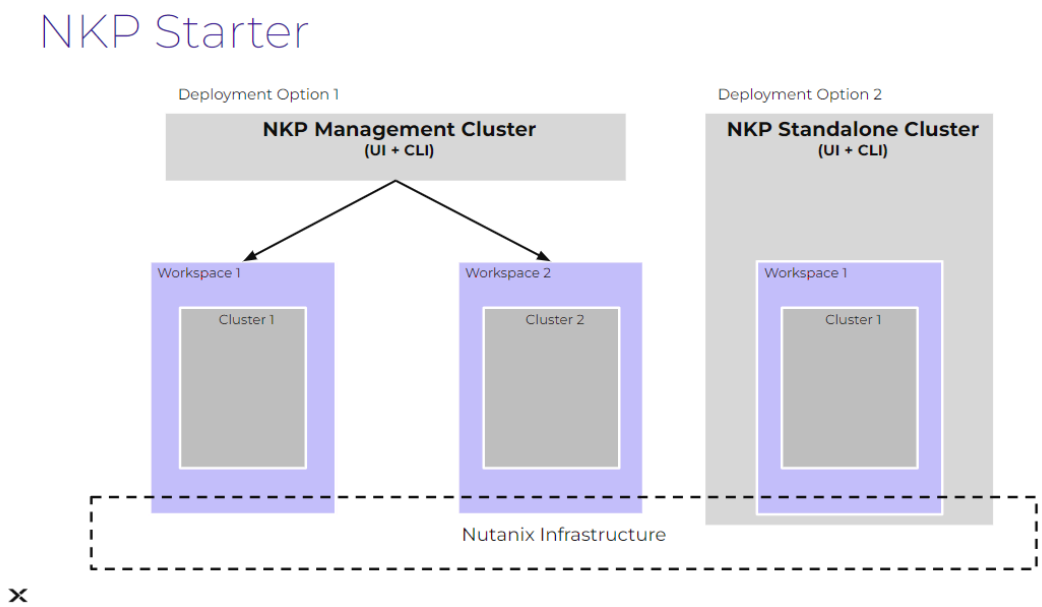


Figure 4: NKP Starter Diagram

NKP Pro License

Nutanix Kubernetes Platform (NKP) Pro is an enterprise-ready license version that is stack-ready to move applications in a cluster to production.

NKP Pro is a multi-cluster life cycle management Kubernetes solution centered around a management cluster that manages many managed Kubernetes clusters through a centralized management dashboard. The management dashboard provides a single observability point and control throughout your managed clusters. The NKP Pro license gives you access to the entire Konvoy cluster environment, and the NKP UI dashboard that deploys platform and catalog applications provides multi-cluster management, as well as comprehensive compatibility with a complete range of infrastructure deployment options.

Compatible Infrastructure

NKP Pro operates across Nutanix entire range of cloud, on-premises, edge, and air-gapped infrastructures and has support for various operating systems, including immutable operating systems. For a complete list of compatible infrastructure, see Supported Infrastructure Operating Systems on page 12. NKP Pro also operates across many popular Cloud infrastructure providers and vSphere.

For instructions on standing up an NKP Pro cluster in one of the listed environments, see Basic Installations by Infrastructure on page 52 or Custom Installation and Infrastructure Tools on page 677.

Cluster Manager

Konvoy is the Kubernetes installer component of NKP Pro that uses industry-standard tools to create a certified Kubernetes cluster. These industry standard tools create a cluster management system that includes:

- *Control Plane*: Manages the worker nodes and pods in the cluster.
- *Worker Nodes*: Used to run containerized applications and handle networking to ensure that the traffic between applications across and outside the cluster is facilitated correctly.
- *Container Networking Interface (CNI)*: Calico's open-source networking and network security solution for containers, virtual machines, and native host-based workloads.
- *Container Storage Interface (CSI)*: A common abstraction to container orchestrations for interacting with storage subsystems of various types.
- *Kubernetes Cluster API (CAPI)*: Cluster API uses Kubernetes-style APIs and patterns to automate cluster life cycle management for platform operators. For more information on how CAPI is integrated into NKP Pro, see CAPI Concepts and Terms on page 20.
- *Cert Manager*: A Kubernetes addon to automate the management and issuance of TLS certificates from various issuing sources.
- *Cluster Autoscaler*: A component that automatically adjusts the size of a Kubernetes cluster so that all pods have a location to run and there are no unwanted nodes.

Builtin GitOps

NKP Pro is bundled with GitOps, an operating model for Kubernetes and other cloud native technologies, providing a set of best practices that unify Git deployment, management, and monitoring for containerized clusters and applications. GitOps uses Git as a single source of truth for declarative infrastructure and applications. With GitOps, software agents can alert any divergence between Git and what it is running in a cluster and if there's a difference. Kubernetes reconcilers automatically update or roll back the cluster depending on the case.

Platform Applications

When creating a cluster, the application manager deploys specific platform applications on the newly created cluster. You can deploy applications in any NKP managed cluster with the complete flexibility to operate across cloud, on-premises, edge, and air-gapped scenarios. Customers can also use the NKP UI to customize the required platform applications to deploy to the cluster in a given workspace.

With NKP Pro, you can use the NKP UI to customize which platform applications to deploy in the cluster in a workspace. For a list of available platform applications included with NKP, see Workspace Platform Application Resource Requirements on page 408.

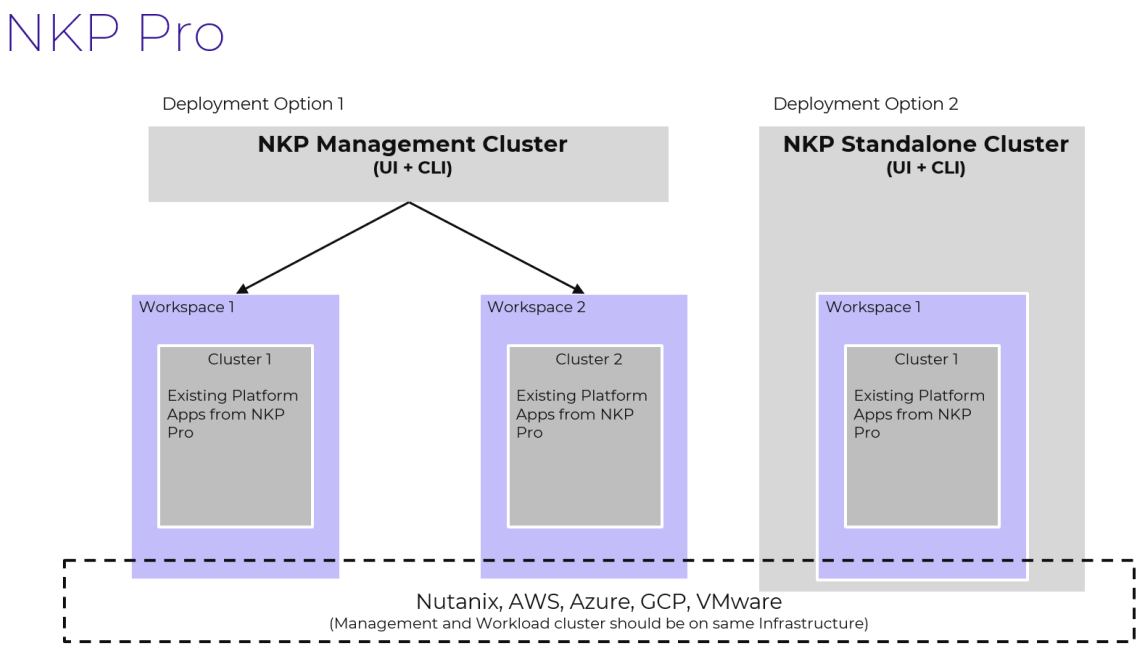


Figure 5: NKP Pro Diagram

NKP Ultimate License

Nutanix Kubernetes Platform (NKP) Ultimate provides true fleet management for clusters running on-premises, in the cloud, or anywhere. It is a multi-cluster life cycle management Kubernetes solution centered around a *management cluster* that manages many `attached` or `managed` Kubernetes clusters through a centralized management dashboard. The management dashboard provides a single observability point and control throughout your attached or managed clusters. The NKP Ultimate license gives you access to the Konvoy cluster environment, the NKP UI dashboard that deploys platform and catalog applications, multi-cluster management, and comprehensive compatibility with the complete infrastructure deployment options.

Compatible Infrastructure

NKP Ultimate operates across Nutanix entire range of cloud, on-premises, edge, and air-gapped infrastructures and has support for various operating systems, including immutable operating systems. See Supported Infrastructure Operating Systems on page 12 for a complete list of compatible infrastructure. NKP Ultimate also operates across many popular Cloud infrastructure providers and vSphere.

For the basics on standing up a NKP Ultimate cluster in one of the listed environments of your choice, see Basic Installs by Infrastructure or Custom Installation and Infrastructure Tools on page 677.

Cluster Manager

Konvoy is the Kubernetes installer component of NKP Pro that uses industry-standard tools to create a certified Kubernetes cluster. These industry standard tools create a cluster management system that includes:

- *Control Plane*: Manages the worker nodes and pods in the cluster.

- *Worker Nodes*: Used to run containerized applications and handle networking to ensure that the traffic between applications across and outside the cluster is facilitated correctly.
- *Container Networking Interface (CNI)*: Calico's open-source networking and network security solution for containers, virtual machines, and native host-based workloads.
- *Container Storage Interface (CSI)*: A common abstraction to container orchestrations for interacting with storage subsystems of various types.
- *Kubernetes Cluster API (CAPI)*: Cluster API uses Kubernetes-style APIs and patterns to automate cluster life cycle management for platform operators. For more information on how CAPI is integrated into NKP Pro, see CAPI Concepts and Terms on page 20.
- *Cert Manager*: A Kubernetes addon to automate the management and issuance of TLS certificates from various issuing sources.
- *Cluster Autoscaler*: A component that automatically adjusts the size of a Kubernetes cluster so that all pods have a location to run and there are no unwanted nodes.

Builtin GitOps

NKP Ultimate is bundled with GitOps, an operating model for Kubernetes and other cloud native technologies, providing a set of best practices that unify Git deployment, management, and monitoring for containerized clusters and applications. GitOps uses Git as a single source of truth for declarative infrastructure and applications. With GitOps, software agents can alert any divergence between Git and what it is running in a cluster and if there's a difference. Kubernetes reconcilers automatically update or roll back the cluster depending on the case.

Platform Applications

When creating a cluster, the application manager deploys specific platform applications on the newly created cluster. Applications can be deployed in any NKP managed cluster, giving you complete flexibility to operate across cloud, on-premises, edge, and air-gapped scenarios. Customers can also use the UI with Kommander to customize which platform applications to deploy to the cluster in a given workspace.

With NKP Ultimate, you can use the Kommander UI to customize which platform applications to deploy to the cluster in a workspace. For a list of available platform applications included with NKP, see Workspace Platform Application Resource Requirements on page 408.

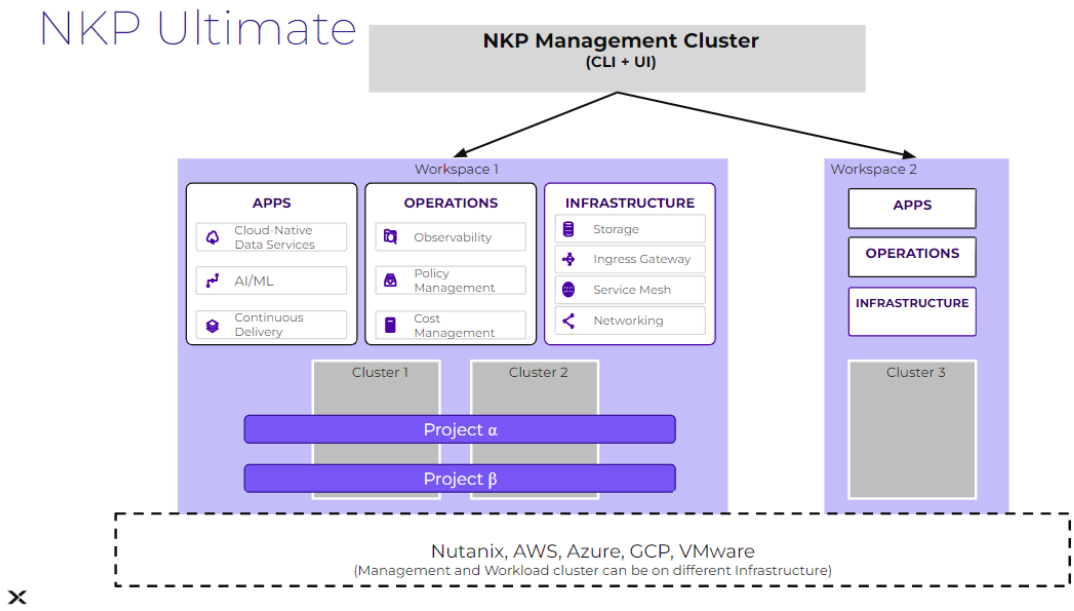


Figure 6: NKP Ultimate Diagram

Adding an NKP License

Add your license to the cluster using the NKP UI.

About this task

For licenses purchased directly from D2iQ or Nutanix, you can obtain the license token from the Nutanix Support Portal. Insert this token in the last step of this procedure. Newly created clusters have a *placeholder* license; you must remove this license before adding your new license obtained by the portal.

Note:

- You must be an administrator to add licenses to NKP. Also, when adding a NKP Pro or Ultimate license, ensure that your cluster has enough resources to accommodate the additional applications that will be deployed.
- If you upgrade the license from NKP Starter to NKP Pro or NKP Ultimate, ensure that you size the worker nodes appropriately to support the installation of default platform services.

Procedure

1. Log into the NKP UI dashboard.
2. In the workspace header drop-down, select **Global**.
3. In the sidebar menu, select **Administration** > **Licensing**.
4. If there is a license currently listed, remove it.
5. To enter the Activate License form, select **+** and **Activate License**.
6. On the Activate License form page, select either **Nutanix** or D2iQ, depending where you purchased your license from.

7. Inside the **License Key** field, paste your license token in the **Enter License** section.
8. Click **Save**.

Removing an NKP License

Remove an NKP license using the NKP UI.

About this task

If your license information has changed, to add a new one, you might need to remove an existing license from NKP.

> **Note:** Only NKP administrators can remove licenses.

Procedure

1. Log into the NKP UI dashboard.
2. In the workspace header drop-down, select **Global**.
3. In the sidebar menu, select **Administration** > **Licensing**.
   Your existing licenses will be listed.
4. Select **Remove License** on the license and follow the prompts.

Commands within a kubeconfig File

This topic specifies some basic recommendations regarding the `kubeconfig` file related to target clusters and the `--kubeconfig=<CLUSTER_NAME>.conf` flag. For more information, see Kubernetes Documentation.

For `kubectl` and NKP commands to run, it is often necessary to specify the environment or cluster in which you want to run them. This also applies to commands that create, delete, or update a cluster's resources.

There are two options:

Table 12: Table

Export an Environment Variable	Specify the Target Cluster in the Command
Export an environment variable from a cluster's `kubeconfig` file, which sets the environment for the commands you run after exporting it.	Specify an environment variable for one command at a time by running it with the `--kubeconfig=<CLUSTER_NAME>.conf` flag.
Better suited for single-cluster environments.	Better suited for multi-cluster environments.

Single-cluster Environment

In a single-cluster environment, you do not need to switch between clusters to run commands and perform operations. However, specifying an environment for each terminal session is still necessary. Hence, the NKP CLI runs the operations on the NKP cluster and does not accidentally run operations on, for example, the bootstrap cluster.

To set the environment variable for all your operations using the `kubeconfig` file, you must first set the variable:

- When you create a cluster, a `kubeconfig` file is generated automatically. Get the `kubeconfig` file and write it to the `${CLUSTER_NAME}.conf` variable :

  ```
  nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
  ```

- Set the context by exporting the `kubeconfig` file from the source file and executing the command for each terminal session using the `--kubeconfig` file except the current session:

  ```
  export KUBECONFIG=${CLUSTER_NAME}.conf
  ```

Multi-cluster Environment

Having multiple clusters means switching between two clusters to run operations. Nutanix recommends two approaches:

- You can start several terminal sessions, one per cluster, and set the environment variable as shown in the single-cluster environment example above, one time per cluster.
- You can use a single terminal session and run the commands with a flag every time. The flag specifies the target cluster for the operation every time so that you can run the same command several times but with a different flag.
- You can use a flag to reference the target cluster. The `--kubeconfig=<CLUSTER_NAME>.conf` flag defines the configuration file for the cluster that you configure and try to access.

  This is the easiest way to ensure you are working on the correct cluster when operating and using multiple clusters. If you create additional clusters and do not store the name as an environment variable, you can enter the cluster name followed by `.conf` to access your cluster.

  > **Note:** Ensure that you run `nkp get kubeconfig` for each cluster you want to create to generate a `kubeconfig` file.

Storage

This document describes the model used in Kubernetes for managing persistent, cluster-scoped storage for workloads requiring access to persistent data.

A workload on Kubernetes typically requires two types of storage:

- Ephemeral Storage
- Persistent Volume

Ephemeral Storage

Ephemeral storage, by its name, is ephemeral because it is cleaned up when the workload is deleted or the container crashes. For example, the following are examples of ephemeral storage provided by Kubernetes:

Table 13: Types of Ephemeral Storage

Ephemeral Storage Type	Location
EmptyDir volume	Managed by kubelet under `/var/lib/kubelet`
Container logs	Typically under `/var/logs/containers`

Ephemeral Storage Type	Location
Container image layers	Managed by container runtime (for example, under `/var/lib/containerd`)
Container writable layers	Managed by container runtime (e.g., under `/var/lib/containerd`)

Kubernetes automatically manages ephemeral storage and typically does not require explicit settings. However, you might need to express capacity requests for temporary storage so that `kubelet` can use that information to ensure that each node has enough.

Persistent Volume

A persistent volume claim (PVC) is a storage request. A workload that requires persistent volumes uses a persistent volume claim (PVC) to express its request for persistent storage. A PVC can request a specific size and Access Modes (for example, they can be mounted after read/write or many times read-only).

Any workload can specify a PersistentVolumeClaim. For example, a Pod may need a volume that is at least 4Gi large or a volume mounted under `/data` in the container's filesystem. If a PersistentVolume (PV) satisfies the specified requirements in the PersistentVolumeClaim (PVC), it will be bound to the PVC before the Pod starts.

Default Storage Providers

When deploying Nutanix Kubernetes Platform (NKP) using a supported cloud provider (AWS, Azure, or GCP), NKP automatically configures native storage drivers for the target platform. In addition, NKP deploys a default storage class (see Storage Classes) for provisioning dynamic volumes (see Dynamic Volume Provisioning) creation.

The following table lists the driver and default StorageClass for each supported cloud provisioner:

Table 14: Default StorageClass for Supported Cloud Provisioners

Cloud Provisioner	Version	Driver	Default Storage Class
AWS	`1.23` The AWS CSI driver is upgraded to a new minor version in NKP 2.7, which has an upgraded version (see https://github.com/kubernetes-sigs/aws-ebs-csi-driver/blob/master/CHANGELOG.md#urgent-upgrade-notes of the package.	aws-ebs-csi-driver	`ebs-sc`
Nutanix	`3.0.0`	nutanix-csi-driver	
Azure	`1.29.6`	azuredisk-csi-driver	`azuredisk-sc`
Pre-provisioned	`2.5.0`	local-static-provisioner	`localvolumeprovisioner`
vSphere	`2.12.0`	vsphere-csi-driver	`vsphere-raw-block-sc`

Cloud Provisioner	Version	Driver	Default Storage Class
Cloud Director (VCD)	`1.4.0-d2iq.0`	cloud-director-named-csi-driver	`vcd-disk-sc`
GCP	`1.10.3`	gcp-compute-persistent-disk-csi-driver	`csi-gce-pd`

Note: NKP uses the local static provisioner as the default storage provider for pre-provisioned clusters. However, `localvolumeprovisioner` is not suitable for production use. Use a Kubernetes CSI (see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types that is compatible with storage that is suitable for production.

You can choose from any storage option https://kubernetes.io/docs/concepts/storage/volumes/#volume-types available for Kubernetes. To disable the default that Konvoy deploys, set the default StorageClass `localvolumeprovisioner` as non-default. Then, set the newly created StorageClass to default by following the commands in the Changing the default StorageClass topic in the Kubernetes documentation (see https://kubernetes.io/docs/tasks/administer-cluster/change-default-storage-class/).

When a default StorageClass is specified, you can create PVCs without specifying the StorageClass. For instance, to request a volume using the default provisioner, create a PVC with the following configurations:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pv-claim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 4Gi
```

To start the provisioning of a volume, launch a pod that references the PVC:

```
...
    volumeMounts:
    - mountPath: /data
      name: persistent-storage
...
  volumes:
  - name: persistent-storage
    persistentVolumeClaim:
      claimName: my-pv-claim
```

Note: To specify a StorageClass that references a storage policy when making a PVC (see https://kubernetes.io/docs/concepts/storage/persistent-volumes/#class-1) and specify a name in `storageClassName`. If left blank, the default StorageClass is used.

Change or Manage Multiple StorageClasses

The default `StorageClass` provisioned with Nutanix Kubernetes Platform (NKP) is acceptable for installation but unsuitable for production. If your workload has different requirements, you can create additional. You can use a single terminal session and run the commands with a flag every time. The flag specifies the target cluster for the operation every time so that you can run the same command several times but with a different flag.

- `StorageClass` types with specific configurations. You can change the default `StorageClass` using these steps from the Kubernetes site: Changing the default storage class

Ceph can also be used as Container Storage Interface (CSI) storage. For information on how to use Rook Ceph, see Rook Ceph in NKP on page 665.

Driver Information

Below is infrastructure provider CSI driver specifics.

Amazon Elastic Block Store (EBS) CSI Driver

NKP EBS default `StorageClass`:

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  annotations:
    storageclass.kubernetes.io/is-default-class: "true" # This tells kubernetes to make
 this the default storage class
  name: ebs-sc
provisioner: ebs.csi.aws.com
reclaimPolicy: Delete  # volumes are automatically reclaimed when no longer in use and
 PVCs are deleted
volumeBindingMode: WaitForFirstConsumer #  Physical volumes will not be created until a
 pod is created that uses the PVC, required to use CSI's Topology feature
parameters:
  csi.storage.k8s.io/fstype: ext4
  type: gp3 # General Purpose SSD
```

NKP deploys with gp3 (general purpose SSDs) EBS volumes.

- Driver documentation: aws-ebs-csi-driver
- Volume types and pricing: volume types

Nutanix CSI Driver

NKP default storage class for Nutanix supports dynamic provisioning and static provisioning of block volumes.

- Driver documentation: Nutanix CSI Driver Configuration
- Nutanix Volumes documentation: Nutanix Creating a Storage Class - Nutanix Volumes
- Hypervisor Attached Volumes documentation: Nutanix Creating a Storage Class - Hypervisor Attached Volumes

The CLI and UI allow you to enable or disable Hypervisor Attached volumes. The selection passes to the CSI driver's storage class. See Manage Hypervisor.

```
allowVolumeExpansion: true
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
    name: default-hypervisorattached-storageclass
parameters:
   csi.storage.k8s.io/fstype: file-system type
   hypervisorAttached: ENABLED | DISABLED <==========
   flashMode: ENABLED | DISABLED
   storageContainer: storage-container-name
   storageType: NutanixVolumes
provisioner: csi.nutanix.com
```

```
reclaimPolicy: Delete | Retain
mountOptions:
   -option1
   -option2
```

Azure CSI Driver

NKP deploys with StandardSSD_LRS for Azure Virtual Disks.

- Driver documentation: azuredisk-csi-driver
- Volume types and pricing: volume types
- Specifics for Azure using Pre-provisioning can be found here: Pre-provisioned Azure-only Configurations

vSphere CSI Driver

NKP default storage class for vSphere supports dynamic provisioning and static provisioning of block volumes.

- Driver documentation: VMware vSphere Container Storage Plug-in Documentation
- Specifics for using vSphere storage driver: Using vSphere Container Storage Plug-in

VMware Cloud Director

In VMware Cloud Director:

- The purpose is to manage the life cycle of the load balancers as well as associate Kubernetes nodes with virtual machines in the infrastructure. See Cloud provider component (CPI)
- Cluster API for VMware Cloud Director (CAPVCD) is a component that runs in a Kubernetes cluster that connects to the VCD API. It uses the Cloud Provider Interface (CPI) to create and manage the infrastructure.
- For storage, NKP has a CSI plugin for interfacing with CPI and can create disks dynamically and become the StorageClass

Pre-provisioned CSI Driver

In a Pre-provisioned environment, NKP will also deploy a CSI-compatible driver and configure a default StorageClass - `localvolumeprovisioner`. See pre-provisioned.

- Driver documentation: local-static-provisioner

NKP uses (`localvolumeprovisioner`) as the default storage provider for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. Use an alternate compatible storage that is suitable for production. See local-static-provisioner and Kubernetes CSI.

To disable the default that Konvoy deploys, set the default StorageClass `localvolumeprovisioner` as non-default. Then, set your newly created StorageClass by following the steps in the Kubernetes documentation: See Change the default StorageClass. You can choose from any of the storage options available for Kubernetes and make your storage choice the default storage. See Storage choice

Ceph can also be used as CSI storage. For information on how to use Rook Ceph, see Rook Ceph in NKP on page 665.

GCP CSI Driver

This driver allows volumes backed by Google Cloud Filestore instances to be dynamically created and mounted by workloads.

- Driver documentation: gcp-filestore-csi-driver
- Persistent volumes and dynamic provisioning: volume types

Provisioning a Static Local Volume

You can provision a static local volume for a Nutanix Kubernetes Platform (NKP) cluster.

About this task

You can choose from any of the storage options available for Kubernetes. To disable the default that Konvoy deploys, set the default StorageClass `localvolumeprovisioner` to non-default. Then, set the newly created StorageClass by following the steps in the Kubernetes documentation (see Change the Default Storage Class).

For the Pre-provisioned infrastructure, the `localvolumeprovisioner` component uses the local volume static provisioner (see https://github.com/kubernetes-sigs/sig-storage-local-static-provisioner) to manage persistent volumes for pre-allocated disks. The volume static provisioners do this by watching the `/mnt/disks` folder on each host and creating persistent volumes in the `localvolumeprovisioner` storage class for each disk it discovers in this folder.

For Nutanix, see documentation topics in the Nutanix Portal: Creating a Persistent Volume Claim.

- Persistent volumes with a *Filesystem* volume mode are discovered if you mount them under `/mnt/disks`.
- Persistent volumes with a *Block* volume-mode are discovered if you create a symbolic link to the block device in `/mnt/disks`.

For additional NKP documentation regarding StorageClass, see Default Storage Providers on page 34.

> **Note:** When creating a pre-provisioned infrastructure cluster, NKP uses `localvolumeprovisioner` as the default storage provider (see https://d2iq.atlassian.net/wiki/spaces/DENT/pages/29919120). However, `localvolumeprovisioner` is not suitable for production use. Use Kubernetes CSI (see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types) to check for compatible storage suitable for production.

Before starting, verify the following:

- You can access a Linux, macOS, or Windows computer with a supported OS version.
- You have a provisioned NKP cluster that uses the `localvolumeprovisioner` platform application but has not added any other Kommander applications to the cluster yet.

This distinction between provisioning and deployment is important because some applications depend on the storage class provided by the `localvolumeprovisioner` component and can fail to start if not configured.

Procedure

1. Create a pre-provisioned cluster by following the steps outlined in the pre-provisioned infrastructure topic.

   As volumes are created or mounted on the nodes, the local volume provisioner detects each volume in the `/mnt/disks` directory. It adds it as a persistent volume with the `localvolumeprovisioner` StorageClass. For more information, see the documentation regarding Kubernetes Local Storage.

2. Create at least one volume in `/mnt/disks` on each host.

   For example, mount a `tmpfs` volume.

   ```
   mkdir -p /mnt/disks/example-volume && mount -t tmpfs example-volume /mnt/disks/
   example-volume
   ```

3. Verify the persistent volume by running the following command.

   ```
   kubectl get pv
   ```

4. Claim the persistent volume using a PVC by running the following command.

   ```
   cat <<EOF | kubectl create -f -
   kind: PersistentVolumeClaim
   apiVersion: v1
   metadata:
     name: example-claim
   spec:
     accessModes:
     - ReadWriteOnce
     resources:
       requests:
         storage: 100Mi
     storageClassName: localvolumeprovisioner
   EOF
   ```

5. Reference the persistent volume claim in a pod by running the following command.

   ```
   cat <<EOF | kubectl create -f -
   apiVersion: v1
   kind: Pod
   metadata:
     name: pod-with-persistent-volume
   spec:
     containers:
       - name: frontend
         image: nginx
         volumeMounts:
           - name: data
             mountPath: "/var/www/html"
     volumes:
       - name: data
         persistentVolumeClaim:
           claimName: example-claim
   EOF
   ```

6. Verify the persistent volume claim using the command `kubectl get pvc`.

   Example output:

   ```
   NAME            STATUS   VOLUME              CAPACITY   ACCESS MODES   STORAGECLASS
             AGE
   example-claim   Bound    local-pv-4c7fc8ba   3986Mi     RWO
    localvolumeprovisioner   78s
   ```

Resource Requirements

To ensure a successful Nutanix Kubernetes Platform (NKP) installation, you must meet certain resource requirements for the control plane nodes and worker nodes. These resource requirements can be slightly different for different infrastructure providers and license type.

General Resource Requirements

To install NKP with the minimum amount of resources. Review the requirements list for your install before beginning installation.

For more information, see Installing NKP on page 50.

Pro and Ultimate Cluster Minimum Requirements

To install Pro and Ultimate Clusters NKP with minimum resources, review the following before beginning installation. The specific number of worker nodes required for your environment can vary depending on the cluster workload and size of the nodes.

Note: The default number of Control Plane Nodes is three (3). The default is set at three to provide High Availability (HA).

Caution: It is possible to create a working cluster with only one (1) Control Plane Node. However, Nutanix discourages this; you will lose the cluster if the single Control Plane node has any corruption or problems.

Table 15: General Resource Requirements for Pro and Ultimate Clusters

Resources	Control Plane Nodes	Worker Nodes
Minimum Node Requirements	3	4
vCPU Count	4	8
Memory	16 GB	32 GB
Disk Volume	Approximately 80 GiB: Used for `/var/lib/kubelet` and `/var/lib/containerd`	Approximately 95 GiB: used for `/var/lib/kubelet` and `/var/lib/containerd`
Root Volume	Disk usage must be below 85%	Disk usage must be below 85%

Note: If you use the instructions to create a cluster using the NKP default settings your cluster is deployed with three control plane nodes, and four worker nodes which match the requirements above.

Starter Cluster Minimum Requirements

To install the ##NKP# Starter Cluster with the minimum amount of resources, review the resource table before starting the installation. The specific number of worker nodes required for your environment can vary depending on the cluster workload and size of the nodes.

Note:

- The Starter License is supported exclusively with the Nutanix Infrastructure.
- The default number of Control Plane Nodes is three (3). The default is set at three to provide High Availability (HA).
- If you have 2 worker nodes in your Starter cluster, and you apply a Pro or Ultimate license, the additional Pro or Ultimate features will fail because there aren't enough cluster resources to deploy them.

Caution: It is possible to create a working cluster with only one (1) Control Plane Node. However, Nutanix discourages this; you will lose the cluster if the single Control Plane node has any corruption or problems.

Table 16: Management Cluster

Resources	Control Plane Nodes	Worker Nodes
Minimum Node Requirements	3	2
vCPU Count	2	4
Memory	6 GB	4 GB
Disk Volume	Approximately 80 GiB: Used for `/var/lib/kubelet` and `/var/lib/containerd`	Approximately 80 GiB: used for `/var/lib/kubelet` and `/var/lib/containerd`
Root Volume	Disk usage must be below 85%	Disk usage must be below 85%

Non-Default Flags in the CLI	`--control-plane-vcpus 2 \`
	`--control-plane-memory 6 \`
	`--worker-replicas 2 \`
	`--worker-vcpus 4 \`
	`--worker-memory 4`	

Table 17: Managed Cluster

Resources	Control Plane Nodes	Worker Nodes
Minimum Node Requirements	3	2
vCPU Count	2	3
Memory	4 GB	3 GB
Disk Volume	Approximately 80 GiB: Used for `/var/lib/kubelet` and `/var/lib/containerd`	Approximately 80 GiB: used for `/var/lib/kubelet` and `/var/lib/containerd`
Root Volume	Disk usage must be below 85%	Disk usage must be below 85%

Non-Default Flags in the CLI	`--control-plane-vcpus 2 \`
	`--control-plane-memory 4 \`
	`--worker-replicas 2 \`
	`--worker-vcpus 3 \`
	`--worker-memory 3`	

Infrastructure Provider-Specific Requirements

For specific infrastructure providers, additional requirements might apply. For example, NKP on Azure defaults to deploying a Standard_D4s_v3 VM with a 128 GiB volume for the OS and an 80 GiB volume for etcd storage, which meets the above requirements. For specific additional resource information:

- Nutanix Installation Options on page 52
- Pre-provisioned Installation Options on page 69
- AWS Installation Options on page 163
- Azure Installation Options on page 322
- vSphere Installation Options on page 259
- VMware Cloud Director Installation Options on page 322
- EKS Installation Options on page 239
- AKS Installation Options on page 332

Kommander Component Requirements

- Management Cluster Application Requirements
- Workspace Platform Application Defaults and Resource Requirements

Managed Cluster Requirements

To create additional clusters in that environment, ensure that you have at least the minimal recommended resources below.

Minimum Recommendation for Managed Clusters

Worker Nodes: For a default installation, at least four worker nodes with:

- 8 CPU cores each
- 12 GiB of memory
- A storage class and disk mounts that can accommodate at least four persistent volumes

> **Note:** Four worker nodes are required to support upgrades to the `rook-ceph` platform application. `rook-ceph` supports the logging stack, the `velero` backup tool, and NKP Insights. If you have disabled the `rook-ceph` platform application, only three worker nodes are required.

Control Plane Nodes:

- 8 CPU cores each
- 12 GiB of memory

Cluster Needs:

- Default Storage Class and four volumes of 32GiB, 32GiB, 2GiB, and 100GiB or the ability to create those volumes depending on the Storage Class:

```
$ kubectl get pv -A
NAME                                       CAPACITY   ACCESS MODES   RECLAIM POLICY
 STATUS   CLAIM
    STORAGECLASS   REASON   AGE
pvc-08de8c06-bd66-40a3-9dd4-b0aece8ccbe8   32Gi       RWO            Delete
 Bound    kommander-default-workspace/kubecost-cost-analyzer
    ebs-sc                 124m
pvc-64552486-7f4c-476a-a35d-19432b3931af   32Gi       RWO            Delete
 Bound    kommander-default-workspace/kubecost-prometheus-server
    ebs-sc                 124m
```

```
pvc-972c3ee3-20bd-449b-84d9-25b7a06a6630   2Gi        RWO            Delete
 Bound    kommander-default-workspace/kubecost-prometheus-alertmanager
    ebs-sc                  124m
pvc-98ab93f1-2c2f-46b6-b7d3-505c55437fbb   100Gi      RWO            Delete
       Bound    kommander-default-workspace/db-prometheus-kube-prometheus-stack-
prometheus-0   ebs-sc                  123m
```

Note: Actual workloads might demand more resources depending on the usage.

Management Cluster Application Requirements

This topic only details requirements for management cluster-specific applications in the Kommander component. For the list of all platform applications, see Platform Application Configuration Requirements.

The following table describes the workspace platform applications specific to the management cluster, minimum resource requirements, minimum persistent storage requirements, and default priority class values:

Common Name	**App ID** (for App versions, see the Release Notes)	**Deployed by default**	**Minimum Resources Suggested**	**Minimum Persistent Storage Required**	**Default Priority Class**
Centralized Grafana*	centralized-grafana	Yes	cpu: 200m memory: 100Mi		NKP Critical (100002000)
Centralized Kubecost*	centralized-kubecost	Yes	cpu: 1200m memory: 4151Mi	# of PVs: 1 PV sizes: 32Gi	NKP High (100001000)
Chartmuseum	chartmuseum	Yes		# of PVs: 1 PV sizes: 2Gi	NKP Critical (100002000)
Dex	dex	Yes	cpu: 100m memory: 50Mi		NKP Critical (100002000)
Dex Authenticator	dex-k8s-authenticator	Yes	cpu: 100m memory: 128Mi		NKP High (100001000)
NKP Insights Management	NKP-insights-management	No	cpu: 100m memory: 128Mi		NKP Critical (100002000)
Karma*	karma	Yes			NKP Critical (100002000)
Kommander	kommander	No	cpu: 1100m memory: 896Mi		NKP Critical (100002000)
Kommander AppManagement	kommander-appmanagement	Yes	cpu: 300m memory: 256Mi		NKP Critical (100002000)

Kommander Flux	kommander-flux	Yes	cpu: 5000m memory: 5Gi	NKP Critical (100002000)
Kommander UI	kommander-ui	No	cpu: 100m memory: 256Mi	NKP Critical (100002000)
Kubefed	kube fed	Yes	cpu: 300m memory: 192Mi	NKP Critical (100002000)
Kubetunnel	kubetunnel	Yes	cpu: 200m memory: 148Mi	NKP Critical (100002000)
Thanos*	thanos	Yes		NKP Critical (100002000)
Traefik ForwardAuth	traefik-forward-auth-mgmt	Yes	cpu: 100m memory: 128Mi	NKP Critical (100002000)

Note: Applications with an asterisk (“*”) are NKPUltimate-only apps deployed by default for NKP Ultimate customers only.

Workspace Platform Application Defaults and Resource Requirements

This topic lists the platform applications available in Nutanix Kubernetes Platform (NKP) with the Kommander component. Some are deployed by default on attachment, while others require manual installation by deploying platform applications (see Platform Applications) through the CLI under Cluster Operations.

Workspace platform applications require more resources than solely deploying or attaching clusters to a workspace. Your cluster must have sufficient resources when deploying or attaching to ensure the platform services are installed successfully.

The following table describes all the workspace platform applications that are available to the clusters in a workspace, minimum resource requirements, whether they are enabled by default, and their default priority classes:

Table 18: Available Workspace Platform Applications

Common Name	App ID (for App versions, see the Release Notes)	Deployed by default	Minimum Resources Suggested	Minimum Persistent Storage Required	Default Priority Class
Cert Manager	cert-manager	Yes	cpu: 10m memory: 32Mi		system-cluster-critical (2000000000)
External DNS	external-dns	No			NKP High (100001000)

Common Name	App ID (for App versions, see the Release Notes)	Deployed by default	Minimum Resources Suggested	Minimum Persistent Storage Required	Default Priority Class
Fluent Bit	fluent-bit	No	cpu: 350m memory: 350Mi		NKP Critical (100002000)
Gatekeeper	gatekeeper	Yes	cpu: 300m memory: 768Mi		system-cluster-critical (2000000000)
Grafana	grafana-logging	No	cpu: 200m memory: 100Mi		NKP Critical (100002000)
Loki	grafana-loki	No		# of PVs: 8 PV sizes: 10Gi x 8 (total: 80Gi)	NKP Critical (100002000)
Istio	istio	No	cpu: 1270m memory: 4500Mi		NKP Critical (100002000)
Jaeger	jaeger	No			NKP High (100001000)
Kiali	kiali	No	cpu: 20m memory: 128Mi		NKP High (100001000)
Knative	knative	No	cpu: 610m memory: 400Mi		NKP High (100001000)
Kube OIDC Proxy	kube-oidc-proxy	Yes			NKP Critical (100002000)
Kube Prometheus Stack	kube-prometheus-stack	Yes	cpu: 1300m memory: 4300Mi	# of PVs: 1 PV sizes: 100Gi	NKP Critical (100002000)
Kubecost	kubecost	Yes	cpu: 700m memory: 1700Mi	# of PVs: 3 PV sizes: 2Gi, 32Gi, 32Gi (total: 66Gi)	NKP High (100001000)
Kubernetes Dashboard	kubernetes-dashboard	Yes	cpu: 250m memory: 300Mi		NKP High (100001000)
Logging Operator	logging-operator	No	cpu: 350m * # of nodes + 600m memory: 228Mi + 350Mi * # of nodes	# of PVs: 1 PV sizes: 10Gi	NKP Critical (100002000)

Common Name	App ID (for App versions, see the Release Notes)	Deployed by default	Minimum Resources Suggested	Minimum Persistent Storage Required	Default Priority Class
NFS Server Provisioner	nfs-server-provisioner	No	# of PVs: 1 PV size: 100Gi		NKP High (100001000)
NVIDIA GPU Operator	nvidia-gpu-operator	No	cpu: 100m memory: 128Mi		system-cluster-critical (2000000000)
Prometheus Adapter	prometheus-adapter	Yes	cpu: 1000m memory: 1000Mi		NKP Critical (100002000)
Reloader	reloader	Yes	cpu: 100m memory: 128Mi		NKP High (100001000)
Rook Ceph	rook-ceph	Yes	cpu: 100m memory: 128Mi		system-cluster-critical (2000000000)
Rook Ceph Cluster	rook-ceph-cluster	Yes	cpu 2500m mem 8Gi	# of PVs: 4 PV sizes: 40Gi	NKP Critical (100002000) system-cluster-critical (2000000000) system-node-critical
Traefik	traefik	Yes	cpu: 500m		NKP Critical (100002000)
Traefik ForwardAuth	traefik-forward-auth	Yes	cpu: 100m memory: 128Mi		NKP Critical (100002000)
Velero	velero	No	cpu: 1000m memory: 1024Mi		NKP Critical (100002000)

- Currently, NKP only supports a single deployment of `cert-manager` per cluster. Because of this, `cert-manager` cannot be installed on any Konvoy managed clusters or clusters with cert-manager pre-installed.
- Only a single deployment of traefik per cluster is supported.
- NKP automatically manages the deployment of `traefik-forward-auth` and `kube-oidc-proxy` when clusters are attached to the workspace. These applications are not shown in the NKP UI.
- Applications are enabled in NKP and then deployed to attached clusters. To confirm that the application you enabled is deployed successfully and verified through the CLI, see Deployment of Catalog Applications in Workspaces on page 424.

Prerequisites for Installation

Before you create a Nutanix Kubernetes Platform (NKP) image and deploy the initial NKP cluster, the operator's machine must be either a Linux-based or MacOS machine of a supported version. Ensure you have met all the prerequisites for the Konvoy and Kommander components.

- Prerequisites for the Konvoy component:
  - Non-air-gapped (all environments)
  - Air-gapped (additional prerequisites)
- Prerequisites for the Kommander component:
  - Non-air-gapped (all environments)
  - Air-gapped only (additional prerequisites)

  Note: Additional prerequisites are necessary for air-gapped; verify that all the non-air-gapped conditions are met and then add any additional air-gapped prerequisites listed.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50

Prerequisites for the Konvoy Component

For NKP and Konvoy Image Builder to run, the operator machine requires:

Non-air-gapped (all environments)

In a non-air-gapped environment, your environment has two-way access to and from the Internet. The prerequisites required if installing in a non-air-gapped environment are as follows:

- An x86_64-based Linux or MacOS machine.
- The NKP binary on the bastion by downloading NKP (see Downloading NKP on page 16).

  To check which version of NKP you installed for compatibility reasons, run the `nkp version` command.
- You must run all the NKP commands from a host or VM that has network connectivity with the infrastructure provider and has the NKP binary installed.
- A container engine or runtime installed is required to install NKP and bootstrap:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
- `kubectl` (see https://kubernetes.io/docs/tasks/tools/#kubectl) for interacting with the cluster running on the host where the NKP Konvoy CLI runs.
- Konvoy Image Builder in KIB (see Konvoy Image Builder).

- Valid provider account with the credentials configured.
  - AWS credentials (see https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-profiles.html) that can manage CloudFormation Stacks, IAM Policies, IAM Roles, and IAM Instance Profiles.
  - Azure credentials (see https://github.com/kubernetes-sigs/cluster-api-provider-azure/blob/master/docs/book/src/topics/getting-started.md#prerequisites).
- CLI tooling of the cloud provider to deploy NKP commands:
  - `aws-cli` (see https://docs.aws.amazon.com/cli/latest/userguide/cli-chap-install.html)
  - `googlecloud-cli` (see https://cloud.google.com/sdk/docs/install)
  - `azure-cli` (see https://learn.microsoft.com/en-us/cli/azure/install-azure-cli)
- **Elastic Kubernetes Service (EKS) and Azure Kubernetes Service (AKS)**: A management cluster (self-managed) cluster is required:
  - If you follow the instructions in the Basic Installations by Infrastructure on page 52 topic for installing NKP, use the `--self-managed` flag for a self-managed cluster. If you use the instructions in Custom Installation and Additional Infrastructure Tools, ensure that you perform the self-managed process on your new cluster:
  - A self-managed AWS cluster.
  - A self-managed Azure cluster.
- **Pre-provisioned only**:
  - Pre-provisioned hosts with SSH access enabled.
  - An unencrypted SSH private key, whose public key is configured on the hosts.
  - Pre-provisioned Override Files if needed.
- **vSphere only**:
  - A valid VMware vSphere account with credentials configured.

Air-gapped Only (additional prerequisites) In an air-gapped environment, your environment is isolated from unsecured networks, like the Internet, and therefore requires additional considerations for installation. Configure the following additional prerequisites if installing in an air-gapped environment:

- Linux machine (bastion) that has access to the existing Virtual Private Cloud (VPC) instead of an x86_64-based Linux or macOS machine.
- Ability to download artifacts from the internet and then copy those onto your Bastion machine.
- Download the complete NKP air-gapped bundle `NKP-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` for this release (see Downloading NKP on page 16).
  - To use a local registry, whether air-gapped or non-air-gapped environment, download and extract the complete NKP air-gapped bundle for this release (that is, `NKP-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`) to load your registry.
- An existing local registry to seed the air-gapped environment.

Prerequisites for the Kommander Component

Non-air-gapped (all environments)

In a non-air-gapped environment, your environment has two-way access to and from the Internet. Below are the prerequisites required if installing in a non-air-gapped environment:

- The version of the CLI that matches the NKP version you want to install.
- Review the Management Cluster Application Requirements and Workspace Platform Application Defaults and Resource Requirements to ensure your cluster has sufficient resources.
- Ensure you have a default StorageClass (see Default Storage Providers on page 34) configured (the Konvoy component is responsible for configuring one).
- A load balancer to route external traffic.

  In cloud environments, this information is provided by your cloud provider. You can configure MetalLB for on-premises and vSphere deployments. It is also possible to use Virtual IP. For more details, see Load Balancing on page 632.
- Ensure your firewall allows connections to github.com.
- For pre-provisioned on-premises environments:
  - Ensure you meet the storage requirements (see Storage on page 33), default storage class, and Workspace Platform Application Defaults and Resource Requirements on page 44.
  - Ensure you have added at least 40 GB of raw storage to your clusters' worker nodes.

Air-gapped Only (additional prerequisites)

In an air-gapped environment, your environment is isolated from unsecured networks, like the Internet, and therefore requires additional considerations for installation. Below are the additional prerequisites required if installing in an air-gapped environment:

- A local registry (see Registry Mirror Tools on page 1069) containing all the necessary installation images, including the Kommander images, which were downloaded in the air-gapped bundle above, `NKP-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`. To view how to push images required to this registry and load the registry under each provider in the Basic Installations by Infrastructure on page 52 section.
- Connectivity with the clusters attached to the management cluster:
  - Both management and attached clusters must connect to the registry.
  - The management cluster must connect to all the attached cluster’s API servers.
  - The management cluster must connect to any load balancers created for platform services on the management cluster.

  Note: If you want to customize your cluster’s domain or certificate, ensure you review the respective documentation sections:

  - Configure custom domains and certificates during the Kommander installation.
  - Configure custom domains and certificates after Kommander has been installed.
- For pre-provisioned environments:
  - Ensure you meet the storage requirements.
  - Ensure you have added at least 40 GB of raw storage to each of your cluster worker nodes.

Installing NKP

The topic lists the basic package requirements for your environment to perform a successful installation of Nutanix Kubernetes Platform (NKP). Next, install NKP, and then you can begin any custom configurations based on your environment.

About this task

Perform the following steps to install NKP:

Procedure

1. Install the required packages.

   In most cases, you can install the required software using your preferred package manager. For example, on a macOS computer, use Homebrew (see Homebrew Documentation) to install `kubectl` and the `aws` command-line utility by running the following command. Replace `aws` with your provider.

   ```
   brew install kubernetes-cli awscli
   ```

   Replace `aws` with the name of the provider.

2. Check the Kubernetes client version.

   Many important Kubernetes functions do not work if your client is outdated. You can verify the version of `kubectl` you have installed to check whether it is supported by running the following command.

   ```
   kubectl version --short=true
   ```

3. Check the supported Kubernetes versions after finding your version with the preceding command.

4. For air-gapped environments, create a bastion host for the cluster nodes to use within the air-gapped network.

   The bastion host needs access to a local registry instead of an Internet connection to export images. The recommended template naming pattern is `../folder-name/NKP-e2e-bastion-template` or similar. Each infrastructure provider has its own set of bastion host instructions. For specific details of your provider, see the respective provider's sit for more information: Azure (see https://learn.microsoft.com/en-us/azure/bastion/quickstart-host-portal, AWS (see https://aws.amazon.com/solutions/implementations/linux-bastion/, GCP https://blogs.vmware.com/cloud/2021/06/02/intro-google-cloud-vmware-engine-bastion-host-access-iap/, or vSphere (see https://docs.vmware.com/en/VMware-vSphere/7.0/com.vmware.vsphere.security.doc/GUID-6975426F-56D0-4FE2-8A58-580B40D2F667.html).

5. Create NKP machine images by downloading the Konvoy Image Builder and extracting it.

6. Download NKP. For more information, see Downloading NKP on page 16.

7. Verify that you have valid cloud provider security credentials to deploy the cluster.

   > **Note:** This step regarding the provider security credentials is not required if you install NKP on an on-premises environment. For information about installing NKP in an on-premises environment, see Pre-provisioned Infrastructure on page 743.

8. Install the Konvoy component depending on which infrastructure you have. For more information, see Basic Installations by Infrastructure on page 52. To use customized YAML and other advanced features, see Custom Installation and Infrastructure Tools on page 677.

9. Configure the Kommander component by initializing the configuration file under the Kommander Installer Configuration File component of NKP. For more information, see Kommander Installation Based on Your Environment on page 1016.

10. (Optional) Test operations by deploying a sample application, customizing the cluster configuration, and checking the status of cluster components.

11. Initialize the configuration file under the Kommander Installer Configuration File component of NKP. For more information, see Installling Kommander on page 1036.

What to do next

Here are some links to the NKP installation-specific information:

- To view supported Kubernetes versions, see Upgrade Compatibility Tables.
- To view the list of NKP versions and compatibility software, see Konvoy Image Builder .
- For details about default storage providers and drivers, see Default Storage Providers.
- For supported FIPS builds, see Deploying a Cluster in FIPS mode.

4

BASIC INSTALLATIONS BY INFRASTRUCTURE

This topic provides basic installation instructions for your infrastructure using combinations of providers and other variables.

A *basic* cluster contains nodes and a running instance of Nutanix Kubernetes Platform (NKP) but is not yet a `production` cluster. While you might not be ready to deploy a workload after completing the basic installation procedures, you will familiarize yourself with NKP and view the cluster structure.

> **Note:** For custom installation procedures, see Custom Installation and Additional Infrastructure Tools.

Production cluster configuration allows you to deploy and enable the cluster management applications and your workload applications that you need for production operations. For more information, see Cluster Operations Management on page 353.

For virtualized environments, NKP can provision the virtual machines necessary to run Kubernetes clusters. If you want to allow NKP to manage your infrastructure, select your supported infrastructure provider installation choices below.

> **Note:** If you want to provision your nodes in a bare metal environment or manually, see Pre-provisioned Infrastructure on page 743.

If not already done, perform the procedures in the following topics:

- Resource Requirements on page 39
- Prerequisites for Installation on page 47
- Installing NKP on page 50

Nutanix Installation Options

This chapter describes the installation options for environments on the Nutanix infrastructure.

For additional options to customize YAML Ain't Markup Language (YAML), see Custom Installation and Infrastructure Tools on page 677.

To determine whether your OS is supported, see Supported Infrastructure Operating Systems on page 12.

The process of configuring Nutanix and NKP comprises the following steps:

1. Configure Nutanix to provide the required elements described in Nutanix Infrastructure Prerequisites on page 690.
2. **For air-gapped environments**, create a bastion VM host (see Creating a Bastion Host on page 685).
3. Create a base OS image (see Nutanix Base OS Image Requirements on page 706).
4. Create a new self-managed cluster.
5. Verify and log on to the UI.

Nutanix Basic Prerequisites

This topic contains the prerequisites specific to the Nutanix infrastructure.

These prerequisites are above and beyond the ones mentioned in Prerequisites for Installation on page 47. Fulfilling the prerequisites for Nutnanix involves completing the following two tasks:

1. NKP Prerequisites on page 53
2. Nutanix Prerequisites on page 53

NKP Prerequisites

Before using NKP to create a Nutanix cluster, verify that you have the following:

- An x86_64-based Linux or macOS machine.
- Download the NKP binaries and Konvoy Image Builder (KIB) bundle. For more information, see Downloading NKP on page 16 and Konvoy Image Builder on page 1084.
- Install a container engine or runtime to install NKP and bootstrap:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
- A registry is required in your environment:
  - Install kubectl 1.28.x to interact with the running cluster on the host where the NKP Konvoy CLI runs. For more information, see https://kubernetes.io/docs/tasks/tools/#kubectl.
  - A valid Nutanix account with credentials configured.

Note:

- NKP uses the Nutanix CSI Volume Driver (CSI) 3.0 as the default storage provider. For more information on the default storage providers, see Default Storage Providers on page 34.
- For compatible storage suitable for production, choose from any of the storage options available for Kubernetes. For more information, see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types.
- To turn off the default StorageClass that Konvoy deploys:
  1. Set the default StorageClass as *non-default*.
  2. Set your newly created StorageClass as *default*.

  For information on changing the default storage class, see https://kubernetes.io/docs/tasks/administer-cluster/change-default-storage-class/.

Nutanix Prerequisites

Before installing, verify that your environment meets the following basic requirements:

- NKP 2.12 is supported with Prism Central versions 2024.1 and 2024.2.

  Caution: NKP 2.12.x that ships with Nutanix CSI 3.0 is incompatible with Prism Central 2024.3. You must upgrade NKP 2.12.x to NKP 2.13 before upgrading to Prism Central 2024.3.

- Nutanix Prism Central version 2024.1 has role credentials configured with administrator privileges.

- AOS 6.5, 6.8, 6.10 and later.
- Configure valid values in Prism Central (see Prism Central Settings (Infrastructure)).
  - Pre-designated subnets.
  - A subnet with unused IP addresses. The number of IP addresses required is computed as follows:
    - One IP address for each node in the Kubernetes cluster. The default cluster size has three control plane nodes and four worker nodes. So, this requires seven IP addresses.
    - One IP address in the same Classless Inter-Domain Routing (CIDR) as the subnet but not part of the address pool for the Kubernetes API server (kubevip).
    - One IP address in the same CIDR as the subnet but not part of an address pool for the Loadbalancer service used by Traefik (metallb).
    - Additional IP addresses may be assigned to accommodate other services such as NDK, that also need the Loadbalancer service used by Metallb. For more information, see the Prerequisites and Limitations section in the *Nutanix Data Services for Kubernetes* guide at https://portal.nutanix.com/page/documents/details?targetId=Nutanix-Data-Services-for-Kubernetes-v1_1:top-prerequisites-k8s-c.html.
  - For air-gapped environments, a bastion VM host template with access to a configured local registry. The recommended template naming pattern is `../folder-name/NKP-e2e-bastion-template` or similar. Each infrastructure provider has its own bastion host instructions (see Creating a Bastion Host on page 685.
  - Access to a bastion VM or other network-connected host running Konvoy Image Builder.

    Note: Nutanix provides a full image built on Nutanix with base images if you do not want to create your own from a BaseOS image.

    - You must be able to reach the Nutanix endpoint where the Konvoy CLI runs.
  - **Note:**

    - For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

      ```
      tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
      ```

    - Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

      For example:

      ```
      export TMPDIR=/path/to/your/directory
      ```

  Note: For troubleshooting or additional information, see Nutanix Knowledge Base.

Prism Central Credential Management
Create the required credentials for Nutanix Prism Central (PC).

Nutanix Kubernetes Platform(NKP) infrastructure cluster uses Prism Central credentials for the following:

1. To manage the cluster, such as listing subnets and other infrastructure and creating VMs in Prism Central used by the Cluster API Provider Nutanix Cloud Infrastructure (CAPX) infrastructure provider.
2. To manage persistent storage used by Nutanix CSI providers.
3. To discover node metadata used by the Nutanix Cloud Cost Management (CCM) provider.

Prism Central (PC) credentials are required to authenticate the PC APIs. CAPX currently supports two mechanisms for assigning the required credentials.

- Credentials injected into the CAPX manager deployment.
- Workload cluster-specific credentials.

For examples, see Credential Management at https://opendocs.nutanix.com/capx/v1.5.x/credential_management/.

Injected Credentials

By default, credentials are injected into the CAPX manager deployment when CAPX is initialized. For information about getting started with CAPX, see *Getting Started* in https://opendocs.nutanix.com/capx/v1.5.x/getting_started/.

Upon initialization, a `nutanix-creds` secret is automatically created in the `capx-system` namespace. This secret contains the values specified in the `NUTANIX_USER` and `NUTANIX_PASSWORD` parameters.

The `nutanix-creds` secret is used for workload cluster deployments if no other credentials are supplied.

Workload Cluster Credentials

Users can override the credentials injected in CAPX manager deployment by supplying a credential specific to a workload cluster. See *Credentials injected into the CAPX manager deployment* in https://opendocs.nutanix.com/capx/v1.5.x/credential_management/#credentials-injected-into-the-capx-manager-deployment. The credentials are provided by creating a secret in the same namespace as the `NutanixCluster` namespace.

The secret is referenced by adding a `credentialRef` inside the `prismCentral` attribute contained in the `NutanixCluster`. See Prism Central Admin Center Guide. The secret is also deleted when the `NutanixCluster` is deleted.

> **Note:** There is a 1:1 relation between the secret and the `NutanixCluster` object.

Prism Central Role

For NKP 2.12, you can only use an Admin user.

Configuring the Role with an Authorization Policy

When provisioning Kubernetes clusters with NKP on Nutanix infrastructure, a pre-defined role that contains the minimum permissions to deploy clusters is also provisioned.

About this task

On the **Kubernetes Infrastructure Provision Role Details** screen, you are assigned to the system-defined roles by creating an authorization policy. For more information, see Configuring an Authorization Policy in the *Security Guide*.

Procedure

1. Log in to **Prism Central** as an administrator.
2. In the Application Switcher, select **Admin Center**.

3. Select **IAM** and go to **Authorization Policies**.
4. To create an authorization policy, select **Create New Authorization Policy**.
   The Create New Authorization Policy window appears.
5. In the **Choose Role** step, enter a role name in the **Select the role to add to this policy** field and select **Next**.
   You can enter any built-in or custom roles.
6. In the **Define Scope** step, select one of the following.
   - » **Full Access**: provides all existing users access to all entity types in the associated role.
   - » **Configure Access**: provides you with the option to configure the entity types and instances for the added users in the associated role.
7. Click **Next**.
8. In the **Assign Users** step, do the following.
   - » From the dropdown list, select **Local User** to add a local user or group to the policy. Search a user or group by typing the first few letters of the name in the text field.
   - » From the dropdown list, select the available directory to add a directory user or group. Search a user or group by typing the first few letters of the name in the text field.
9. Click **Save**.

   Note: To display role permissions for any built-in role, see Displaying Role Permissions. in the *Security Guide*.

   The authorization policy configurations are saved and the authorization policy is listed in the **Authorization Policies** window.

BaseOS Image Requirements
The base OS image is used by Nutanix Kubernetes Platform Image Builder (NIB) to create a custom image.

For the NKP Starter license tier, use the pre-built Rocky Linux 9.4 image downloaded along with the binary file. The downloaded image must then be uploaded to the Prism Central images folder.

The base OS image is used by Nutanix Kubernetes Platform (NKP) Image Builder (NIB) to create a custom image. For a base OS image, you have two options:

- Use the pre-built Rocky Linux 9.4 image downloaded from the portal.

  Note: Starter license level workload clusters are only licensed to use Nutanix pre-built images.

- Create your custom image for the supported operating systems listed in Table1.Nutanix in Supported Infrastructure Operating Systems on page 12. If not using the out-of-box image, see the topics in the Custom Installation section Create the OS Image for Prism Central or Create the Air-gapped OS Image for Prism Central.

The BaseOS requirements are as follows:

Caution: Beginning with NKP Release 2.12.x, the base image name must be unique. Uploading multiple images with the same name to a PC results in cluster creation or update failure.

- Configure Prism Central. For more information, see Prism Central Admin Center Guide.

- Upload the image downloaded with the NKP binary to the Prism Central images folder.
- Configure the network by downloading NKP Image Builder (NIB) and installing packages to activate the network.
- Install a container engine or runtime to install NKP and bootstrap:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
- For Nutanix, the *Kubernetes Infrastructure Provision* role in Prism Central is required as a minimum permission set for managing clusters. You can also use the *Administrator* role with administrator privileges.

Advanced Users can use image UUID instead of the image name, by using the --dry-run to generate their cluster configuration using the --dry-run option and edit the image name section as shown in the example.

```
- type: name
  name: <node image name >
```

```
 - type: uuid
    uuid: <uuid of the image>
```

If you need to create or update a cluster with an image that does not have a unique name on the PC. First, log in to the PC VM (CVM) and run the command `acli image. list` to find the image UUID. For more information, see https://portal.nutanix.com/page/documents/details?targetId=Command-Ref-AOS-v6_8:acl-acli-image-auto-r.html.

Migrating VMs from VLAN to OVN

Describes how to create and migrate a subnet.

About this task

Migrating Virtual Machines (VMs) from VLAN basic to OVN VLAN is not done through `atlas_cli` , which is recommended by other projects in Nutanix.

Some subnets reserved by Kubernetes can prevent proper cluster deployment if you unknowingly configure Nutanix Kubernetes Platform (NKP) so that the Node subnet collides with either the Pod or Service subnet. Ensure your subnets do not overlap with your host subnet because the subnets cannot be changed after cluster creation.

Note: The default subnets used in NKP are:

```
spec:
  clusterNetwork:
    pods:
      cidrBlocks:
      - 192.168.0.0/16
    services:
      cidrBlocks:
      - 10.96.0.0/12
```

The existing VLAN implementation is *basic* VLAN. However, *advanced* VLAN uses OVN as the control plane instead of the Acropolis. The subnet creation workflow is from Prism Central (PC) rather than Prism Element (PE). Subnet creation can be done using API or through the UI.

Procedure

1. Naviagate to PC settings > Network Controller.
2. Select the option next to **Use the VLAN migrate workflow to convert VLAN Basic subnets to Network Controller managed VLAN Subnets**.
3. In the NKP UI, **Create Subnet**.
4. Under **Advanced Configuration**, remove the check from the checkbox next to **VLAN Basic Networking** to change from Basic to Advanced OVN.
5. Modify the subnet specification in the control plane and worker nodes to use the new subnet. `kubectl edit cluster <clustername>`.

   CAPX will roll out the new control plane and worker nodes in the new Subnet and destroy the old ones.

   > **Note:** You can choose Basic or Advanced OVN when creating the subnet(s) you used during cluster creation. If you created the cluster with basic, you can migrate to OVN.

   To modify the service subnet, add or edit the `configmap`. See the topic Managing Subnets and Pods for more details.

Nutanix Non-Air-gapped Installation

This topic provides instructions on how to install NKP in a Nutanix non-air-gapped environment.

For additional options to customize YAML Ain't Markup Language (YAML), see Custom Installation and Infrastructure Tools on page 677.

If not already done, perform the procedures in the following topics:

- Resource Requirements on page 39
- Prerequisites for Installation on page 47
- Installing NKP on page 50

Nutanix Non-Air-gapped: Installing NKP
Create a Nutanix cluster and install the UI in a non-air-gapped environment.

About this task

Use this procedure to create a self-managed Management cluster with NKP. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster that they are managing.

> **Note:** For Virtual Private Cloud (VPC) installation, see the topic Nutanix with VPC Creating a New Cluster.

> **Note:** NKP uses the Nutanix CSI driver as the default storage provider. For more information see, Default Storage Providers on page 34.

> **Note:** NKP uses a CSI storage container on your Prism Element (PE). The CSI Storage Container image names must be the same for every PE environment in which you deploy an NKP cluster.

Before you begin

Decide on your Base OS image selection. See BaseOS Image Requirements in the Prerequisites section.

Specify a name for your cluster. The cluster name must only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. For more instructions on naming, see *Object Names and IDs* at https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

The installation prompt will pull information from the information you have configured in Prism Central (PC) and Prism Element (PE), such as Cluster name, Storage Containers, and Images.

Procedure

1. Set the environment variable for the cluster name using the following command.

   ```
   export CLUSTER_NAME=<my-nutanix-cluster>
   ```

2. Export Nutanix PC credentials.

   ```
   export NUTANIX_USER=<user>
   export NUTANIX_PASSWORD=<password>
   ```

3. Ensure your subnets do not overlap with your host subnet because they cannot be changed after cluster creation. You must do this at the cluster creation stage to change the Kubernetes subnets. The default subnets used in NKP are /16 for pods, and /20 for service. Nutanix recommends that you use the default values.

   ```
   spec:
     clusterNetwork:
       pods:
         cidrBlocks:
         - 192.168.0.0/16
       services:
         cidrBlocks:
         - 10.96.0.0/12
   ```

   For more information, see Managing Subnets and Pods on page 684.

4. Create a Kubernetes cluster. The following example shows a standard configuration.

   ```
   nkp create cluster nutanix
   ```

   Throughout the prompt-based CLI installation of NKP, there will be options for navigating the screens. Navigation inside the pages is shown in the legend at the bottom of each screen.

   » `Press 'ctrl+p' to go to the previous page.`

   » `Press 'ctrl+n' to go to the next page.`

   » `Press 'ctrl+c' to Quit.`

   » `Press 'shift+tab' to move up.`

   » `Press 'tab' to move down.`

5. Enter your Nutanix Prism Central details. Required fields are denoted with a red asterisk (*). All other fields are optional.

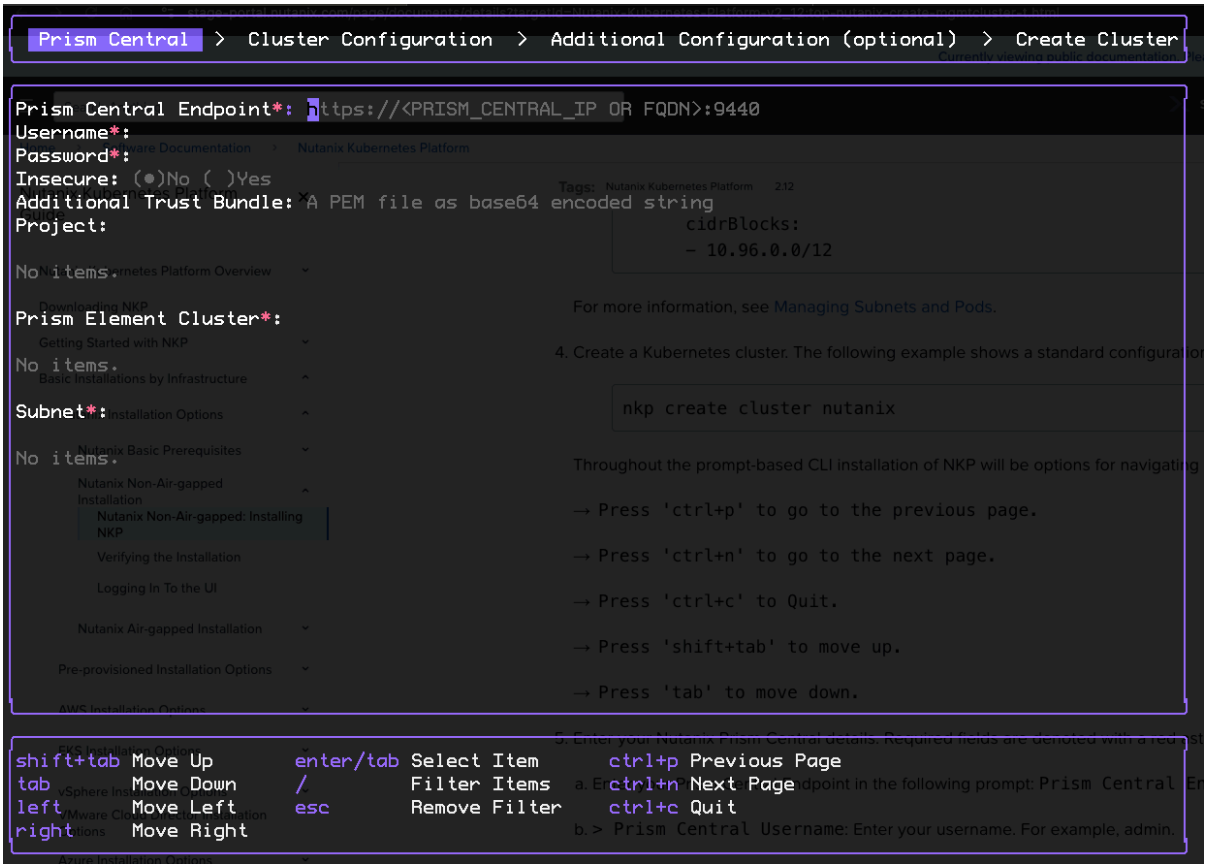


Figure 7: Prism Central User Interface

a. Enter your Prism Central Endpoint in the following prompt: `https://<PRISM_CENTRAL_IP or FQDN>:9440`.

b. `> Prism Central Username`: Enter your username. For example, admin.

c. `> Prism Central Password`: Enter your password.

d. Enter **yes** or **no** for the prompt `Insecure Mode`

e. (Optional) Enter trust information in the prompt for `Additional Trust Bundle: A PEM file as base64 encoded string`

f. `Project:` Select the project name from the generated list.

g. `Prism Element Cluster*:` Select the PE cluster name from the generated list.

h. `Subnet*:` Select subnet information from PE/PC.

6. On the next screen, enter additional information on the `Cluster Configuration` screen. Required fields are denoted with a red asterisk (*). Other fields are optional.

   » `Cluster Name*`

   » `Control Plane Endpoint*`

   » `VM Image*`: A generated list appears from PC images where you select the desired image.

   » `Kubernetes Service Load Balancer IP Range*`

   » `Pod Network`

   » `Service Network`

   » `Reclaim Policy`

   » `File System`

   » `Hypervisor Attached Volumes`

   » `Storage Container*`

7. Enter any additional optional information on the `Additonal Configuration: (optional)` screen.

   » `ACME Configuration: ACME Server address issuing the certificates`.

   » `Email address for ACME Server:`

   » `Ingress certificate file`

   » `Ingress Private key file`

   » `CA chain file`

   » `Registry URL`

   » `Registry CA Certificate`

   » `Registry Username`

   » `Registry Password`

   » `SSH User`

   » If you entered an SSH Username, you must enter the SSH path. `Path to file containing SSH Key for user`

   » `HTTP Proxy:`

   » `HTTPS Proxy:`

   » `No Proxy List:`

8. Review and confirm your changes to create the cluster.

9. Select one of the choices for creating your cluster.

   Create NKP Cluster?

   » `( ) Create`

   » `( ) Dry Run`

   After the installation, the required components are installed, and the Kommander component deploys the minimum applications needed by default. For more information, see NKP Concepts and Terms or NKP Catalog Applications Enablement after Installing NKP .

   > **Caution:** You cannot use the NKP CLI to re-install the Kommander component on a cluster created using the interactive prompt-based CLI. If you need to re-install or reconfigure Kommander to enable more applications, contact Nutanix Support.

Verifying the Installation

Verify your Kommander installation.

About this task

After you build the Konvoy cluster and install the Kommander component for the UI, you can verify your installation. By default, it waits for all applications to be ready.

Procedure

Run the following command to check the installation status.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer using the CLI and not waiting for all applications to be available, you can set the flag to `--wait=false`.

The first wait is for each of the helm charts to reach the `Ready` condition, eventually resulting in an output as follows:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

What to do next

If an application fails to deploy, check the status of the `HelmRelease` using the following command.

```
kubectl -n kommander get helmrelease <HELMRELEASE_NAME>
```

If you find any `HelmReleases` in a *broken* release state, such as *exhausted* or *another rollback/release in progress*, trigger a reconciliation of the `HelmRelease` using the following commands. `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Logging In To the UI

Log in to the UI dashboard.

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using the following command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret nkp-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following command.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use static credentials to access the UI for configuring an external identity provider (see Identity Providers on page 364). Treat them as back up credentials rather than using them for normal access.

   a. Rotate the password using the following command.

      ```
      nkp experimental rotate dashboard-password
      ```

      The output displays the new password.

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Nutanix Air-gapped Installation

Instructions for installing NKP in a Nutanix air-gapped environment.

For additional options to customize YAML Ain't Markup Language (YAML), see Custom Installation and Infrastructure Tools on page 677.

If not already done, perform the procedures in the following topics:

- Resource Requirements on page 39
- Prerequisites for Installation on page 47
- Installing NKP on page 50

Note:

- For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

  ```
  tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
  ```

- Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

  For example:

  ```
  export TMPDIR=/path/to/your/directory
  ```

Nutanix Air-gapped: Loading the Registry
Load the images needed to create an air-gapped Kubernetes cluster in a local registry.

About this task

Before creating an air-gapped Kubernetes cluster, you must load the required images in a local registry for the Konvoy component. The complete NKP air-gapped bundle contains all the NKP components required for an air-gapped environment installation and a local registry in a non-air-gapped environment.

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images, is required. This registry must be accessible from both the bastion machine and other machines that will be created for the Kubernetes cluster.

Before you begin

- Set up a local registry. For more information, see Local Registry Tools.
- Create a Bastion or Jump Host. For more information, see Creating a Bastion Host on page 685.

Procedure

1. Download the air-gapped bundle from the Nutanix support site.
2. Upload the bundle to the Bastion host using Secure Copy Protocol (SCP) or WinSCP.
3. Unpack the air-gapped bundle and move the tar file to the extracted directory using the command `nkp-<version>`.
   Example:

   ```
   tar -xvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz

   cd nkp-v2.12.0
   ```

4. Load the `nkp-image-builder-<version>` and the `nkp bootstrap-<version>` server image into Docker or Podman on the Bastion host.
   Docker Example:

   ```
   docker load -i nkp-image-builder-image-v0.13.1.tar
   docker load -i konvoy-bootstrap-image-v2.12.0.tar
   ```

5. Move the NKP binary to the directory in the $PATH variable, using the command `sudo mv cli/nkp /usr/local/bin`.

6. On the Bastion, load the Konvoy bootstrap image, using either the Docker or Podman command.

   Docker example:

   ```
   docker load -i konvoy-bootstrap-image-v2.12.0.tar
   ```

   Podman example:

   ```
   podman load -i konvoy-bootstrap-image-v2.12.0.tar
   podman image tag konvoy-bootstrap:2.12.0 docker.io/konvoy-bootstrap:v2.12.0
   ```

Nutanix Air-gapped: Creating the OS Bundle
Package the OS dependencies and create a Tar GZ file.

Before you begin

Part of this process is to download OS packages; this step needs to be completed by a machine with connectivity.

About this task

Create the Tar GZ file by completing the following task.

Procedure

Package all the OS dependencies for the given OS and create a .tar.gz using the command `nkp create package-bundle`.

Example:

```
export OS_BUNDLE_DIR=kib/artifacts
export OS=ubuntu-22.04

nkp create package-bundle --artifacts-directory ${OS_BUNDLE_DIR} ${OS}
```

Nutanix Air-gapped: Creating the Nutanix Image
Steps for creating the Nutanix image.

Procedure

1. Create the Nutanix image using the command `nkp create image nutanix`.

   The `--artifacts-directory` is the KIB directory in the air-gapped bundle extract, which was updated when the OS bundle was created. The `--artifacts-directory` flag tells the image builder that the binaries, images, etc., should be picked from the specified directory instead of the internet.

   Example:

   ```
   export NUTANIX_USER=admin
   export NUTANIX_PASSWORD=<password>
   export BASE_IMAGE=ubuntu-22.04-server-cloudimg-amd64.img
   export SUBNET=<VLAN Subnet>
   export PC_ENDPOINT=<PC Hostname>
   export CLUSTER=<PE Cluster name>

   nkp create image nutanix ${OS} --endpoint ${PC_ENDPOINT} --cluster ${CLUSTER} --
   subnet ${SUBNET}
   ```

```
 --source-image ${BASE_IMAGE} --artifacts-directory ${OS_BUNDLE_DIR}
```

> **Note:** If no base image is specified, the cloud image for the given OS is downloaded from the internet, and then Image Builder will run it. To use an existing base image, populate the `--source-image` flag with an appropriate image name.

In a restricted environment where the target network is not directly reachable, a Bastion host with a reachable IP is created to connect to the network. This Bastion host is then used to connect to the target VM.

Example:

```
export NUTANIX_USER=admin
export NUTANIX_PASSWORD=<password>
export BASE_IMAGE=ubuntu-22.04-server-cloudimg-amd64.img
export SUBNET=<VLAN Subnet>
export PC_ENDPOINT=<PC Hostname>
export CLUSTER=<PE Cluster name>
export BASTION_HOST=<Reachable Bastion host IP>
export BASTION_SSH_USER=<SSH User to connect to the Bastion>
export BASTION_KEY=<SSH Private Key to connect to the Bastion>
export BASTION_PORT=22
nkp create image nutanix ${OS} --endpoint ${PC_ENDPOINT} --cluster ${CLUSTER} --
subnet ab-overlay
 --source-image ${BASE_IMAGE} --artifacts-directory ${OS_BUNDLE_DIR} --bastion-host
 ${BASTION_HOST}
 --bastion-private-key-file ${BASTION_KEY} --bastion-username ${BASTION_SSH_USER} --
bastion-port
${BASTION_PORT}
```

2. Create a temporary VM using the base image.
3. Run the Image Builder process on the VM and create an Image as an output. This image is used as an input in the cluster-building process.

   Example Output:

```
==> Builds finished. The artifacts of successful builds are:
--> nutanix.kib_image: nkp-ubuntu-22.04-1.29.6-20240905184139
--> nutanix.kib_image: nkp-ubuntu-22.04-1.29.6-20240905184139
--> nutanix.kib_image: nkp-ubuntu-22.04-1.29.6-20240905184139
```

Nutanix Air-gapped: Pushing Container Images to the Registry Used by NKP

Push container images to the registry used by NKP.

Procedure

1. Push the container images to the registry used by NKP using the command `nkp push bundle`.

   > **Note:** The `--to-registry-ca-cert-file` flag is optional and is used if an internal signing authority signs the cert used by the registry.

   Example

```
export REGISTRY_URL=<REGISTRY_URL>
export REGISTRY_USERNAME=<user>
export REGISTRY_PASSWORD=<password>
export REGISTRY_CACERT=<cacert file for the registry> #Note only required for certs
 signed
 by internal authority
```

```
nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-
registry=${REGISTRY_URL}
 --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=
${REGISTRY_PASSWORD}
 [--to-registry-ca-cert-file=${REGISTRY_CACERT}]

nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar
 --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME}
 --to-registry-password=${REGISTRY_PASSWORD} [--to-registry-ca-cert-file=
${REGISTRY_CACERT}]

nkp push bundle --bundle ./container-images/nkp-catalog-applications-image-bundle-
v2.12.0.tar
 --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME}
 --to-registry-password=${REGISTRY_PASSWORD} [--to-registry-ca-cert-file=
${REGISTRY_CACERT}]
```

2. Download the file to be used in the `REGISTRY_CERT` variable using the command `openssl s_client -showcerts -servername`.

   Example:

```
openssl s_client -showcerts -servername ${REGISTRY_HOST} -connect ${REGISTRY_HOST}:
${REGISTRY_PORT}
 </dev/null | openssl x509 -outform PEM > registry-ca.crt
```

Nutanix Air-gapped: Creating the NKP Cluster

Create your NKP Cluster.

Procedure

Create the cluster using the OS image and Image Registry configured in the previous steps.

This creates a bootstrap cluster, deploys 67 CAPI resources, and creates an NKP base cluster (konvoy) using the CAPI components in the bootstrap cluster. Once the NKP base cluster is provisioned, the following automatically happens:

- Deploy CAPI components on the new cluster.
- Moves the CAPI components from the bootstrap cluster to the new cluster.
- Deletes the Bootstrap cluster
- Deploy the Kommander components on top of the new base NKP cluster.

Example:

```
export IMAGE=<Image Builder Output Image Name>
export SUBNET=<PC subnet>
export CLUSTER_NAME=<NKP Cluster name>
export CONTROLPLANE_VIP=<IP To be Assigned as Control Plane VIP>
export METALLB_IP_RANGE=<Ingress IP Range Start Address-Ingress IP Range End Address>
export NUTANIX_USER=<PC Username>
export NUTANIX_PASSWORD=<PC Password>
export NUTANIX_ENDPOINT=https://<PC Hostname>:9440
export CLUSTER=<PE Cluster Name>
export STORAGE_CONTAINER=<Prism Storage Container to use for Volumes>
export SSH_PUBLIC_KEY=<Optional ssh public key for ssh acces to nodes for konvoy user>


nkp create cluster nutanix \
    --cluster-name=${CLUSTER_NAME} \
    --control-plane-prism-element-cluster=${CLUSTER} \
    --worker-prism-element-cluster=${CLUSTER} \
```

```
    --control-plane-subnets=${SUBNET} \
    --worker-subnets=${SUBNET} \
    --control-plane-endpoint-ip=${CONTROLPLANE_VIP} \
    --csi-storage-container=${STORAGE_CONTAINER} \
    --endpoint=${NUTANIX_ENDPOINT} \
    --control-plane-vm-image=${IMAGE} \
    --worker-vm-image=${IMAGE} \
    --registry-mirror-url=${REGISTRY_URL} \
    --registry-mirror-username=${REGISTRY_USERNAME} \
    --registry-mirror-password=${REGISTRY_PASSWORD} \
    --registry-mirror-cacert=${REGISTRY_CACERT} \
    --kubernetes-service-load-balancer-ip-range=${METALLB_IP_RANGE} \
    --ssh-public-key-file=${SSH_PUBLIC_KEY} \
    --airgapped \
    --self-managed
```

Verifying the Installation
Verify your Kommander installation.

About this task

After you build the Konvoy cluster and install the Kommander component for the UI, you can verify your installation. It waits for all applications to be ready by default.

Procedure

Run the following command to check the status of the installation.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer using the CLI to not wait for all applications to be available, you can set the flag to `--wait=false`.

The first wait is for each of the helm charts to reach the `Ready` condition, eventually resulting in an output as follows:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Logging In To the UI
Log in to the Dashboard UI.

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using the following command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret nkp-credentials -o go-template='Username:
    {{.data.username
   |base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}{{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following command.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with index
   .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use static credentials to access the UI for configuring an external identity provider (see Identity Providers on page 364). Treat them as back up credentials rather than using them for normal access.

   a. Rotate the password using the following command.

      ```
      nkp experimental rotate dashboard-password
      ```

      The output displays the new password.

      ```
      Password: <new nkp dashboard password>
      ```

Pre-provisioned Installation Options

Pre-provisioned infrastructure is provided for non-air-gapped and air-gapped environments.

For more information, see Pre-provisioned Infrastructure on page 743.

The required specific machine resources are as follows:

- *Control Plane machines*:
  - 15% of free space is available on the root file system.
  - Multiple ports are open as described in NKP ports.
  - `firewalld` systemd service is disabled. If it exists and is enabled, use the commands `systemctl stop firewalld` and `systemctl disable firewalld` to disable `firewalld`after the machine restarts.

- *Worker machines*:
  - 15% of free space is available on the root file system.
  - Multiple ports are open as described in the NKP ports.
  - If you plan to use local volume provisioning to provide persistent volumes for your workloads, you must mount at least four volumes to the /mnt/disks/ mount point on each machine. Each volume must have at least 100 GiB of capacity.
  - Ensure your disk meets the resource requirements for Rook Ceph in Block mode for ObjectStorageDaemons as specified in the requirements table.
  - `firewalld` systemd service disabled. If it exists and is enabled, use the commands `systemctl stop firewalld` then `systemctl disable firewalld`, so that `firewalld` remains disabled after the machine restarts.

    Note: Swap is disabled. `kubelet` does not support swapping. Due to variable commands, see the respective Operating System documentation.

Installation Scenarios

Select your installation scenario:

Pre-provisioned Installation

This section provides installation instructions for NKP in a pre-provisioned, non-air-gapped environment.

Pre-provisioned: Defining the Infrastructure
Define the cluster hosts and infrastructure in a pre-provisioned environment.

About this task

The Konvoy component of NKP must know how to access your cluster hosts. Hence, you must define the cluster hosts and infrastructure using the inventory resources. For the initial cluster creation, define a control plane and at least one worker pool.

Set the necessary environment variables as follows:

Procedure

1. Export the following environment variables, ensuring that all the control plane and worker nodes are included.

```
export CONTROL_PLANE_1_ADDRESS="<control-plane-address-1>"
export CONTROL_PLANE_2_ADDRESS="<control-plane-address-2>"
export CONTROL_PLANE_3_ADDRESS="<control-plane-address-3>"
export WORKER_1_ADDRESS="<worker-address-1>"
export WORKER_2_ADDRESS="<worker-address-2>"
export WORKER_3_ADDRESS="<worker-address-3>"
export WORKER_4_ADDRESS="<worker-address-4>"
export SSH_USER="<ssh-user>"
export SSH_PRIVATE_KEY_SECRET_NAME="$CLUSTER_NAME-ssh-key"
```

2. Use the following template to define your infrastructure. The environment variables that you set in the previous step automatically replaces the variable names when the inventory YAML file is created.

```
cat <<EOF > preprovisioned_inventory.yaml
---
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
```

```
metadata:
  name: $CLUSTER_NAME-control-plane
  namespace: default
  labels:
    cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
    clusterctl.cluster.x-k8s.io/move: ""
spec:
  hosts:
    # Create as many of these as needed to match your infrastructure
    # Note that the command line parameter --control-plane-replicas determines how
 many control plane nodes will actually be used.
    #
    - address: $CONTROL_PLANE_1_ADDRESS
    - address: $CONTROL_PLANE_2_ADDRESS
    - address: $CONTROL_PLANE_3_ADDRESS
  sshConfig:
    port: 22
    # This is the username used to connect to your infrastructure. This user must be
 root or
    # have the ability to use sudo without a password
    user: $SSH_USER
    privateKeyRef:
      # This is the name of the secret you created in the previous step. It must
 exist in the same
      # namespace as this inventory object.
      name: $SSH_PRIVATE_KEY_SECRET_NAME
      namespace: default
---
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: $CLUSTER_NAME-md-0
  namespace: default
  labels:
    cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
    clusterctl.cluster.x-k8s.io/move: ""
spec:
  hosts:
    - address: $WORKER_1_ADDRESS
    - address: $WORKER_2_ADDRESS
    - address: $WORKER_3_ADDRESS
    - address: $WORKER_4_ADDRESS
  sshConfig:
    port: 22
    user: $SSH_USER
    privateKeyRef:
      name: $SSH_PRIVATE_KEY_SECRET_NAME
      namespace: default
EOF
```

Pre-provisioned: Control Plane Endpoints

Define the control plane endpoints for your cluster as well as the connection mechanism. A control plane must have three, five, or seven nodes so it can remain available if one or more nodes fail. A control plane with one node cannot be used in production.

In addition, the control plane needs an endpoint that remains available if some nodes fail.

```
                          -------- cp1.example.com:6443
    |
    lb.example.com:6443 ---------- cp2.example.com:6443
    |
```

```
      -------- cp3.example.com:6443
```

In this example, the control plane endpoint host is `lb.example.com` , and the control plane endpoint port is `6443`. The control plane nodes are `cp1.example.com`, `cp2.example.com`, and `cp3.example.com`. The port of each API server is `6443`.

Connection Mechanism Selection

A virtual IP is the address that the client uses to connect to the service. A load balancer is a device that distributes the client connections to the backend servers. Before you create a new NKP cluster, choose an external load balancer (LB) or virtual IP.

- *External load balancer*: Nutanix recommends that an external load balancer be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines that are responding to API requests.
- *Built-in virtual IP*: If an external load balancer is not available, use the built-in virtual IP. The virtual IP is not a load balancer; it does not distribute request load among the control plane machines. However, if the machine receiving requests does not respond to them, the virtual IP automatically moves to another machine.

Single-Node Control Plane

> **Caution:** Do not use a single-node control plane in a production cluster.

A control plane with one node can use its single node as the endpoint, so you will not require an external load balancer or a built-in virtual IP. At least one control plane node must always be running. Therefore, to upgrade a cluster with one control plane node, a spare machine must be available in the control plane inventory. This machine is used to provision the new node before the old node is deleted. When the Application Programming Interface (API) server endpoints are defined, you can create the cluster using the link in the next step below. .

> **Note:** Modify control plane audit log settings using the information in the **Configure the Control Plane** page. See Configuring the Control Plane.

Known Limitations

The control plane endpoint port is also used as the API server port on each control plane machine. The default port is 6443. Before you create the cluster, ensure the port is available for use on each control plane machine.

Pre-provisioned: Creating the Management Cluster

Create a new pre-provisioned Kubernetes cluster in a non-air-gapped environment.

About this task

After you define the infrastructure and control plane endpoints, proceed with creating the cluster by following the steps below to create a new pre-provisioned cluster. This process creates a self-managed cluster for use as the Management cluster.

Before you begin

Specify a name for your cluster and run the command to deploy it. When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects (see Pre-provisioned Air-gapped: Configure Environment on page 83).

Note: The cluster can contain only the following characters: `a-z`, `0-9`,`.`, and `-`. The cluster creation will fail if the name has capital letters. For more instructions on naming, see *Object Names and IDs* at https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Procedure

1. Enter a unique name for your cluster that is suitable for your environment.
2. Set the environment variable for the cluster name using the following command.

   ```
   export CLUSTER_NAME=preprovisioned-example
   ```

3. Create a Kubernetes Cluster.

   After you define the infrastructure and control plane endpoints, you can proceed to create the cluster by following these steps to create a new Pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

What to do next

Before you create a new NKP cluster below, choose an external load balancer (LB) or virtual IP and use the corresponding `nkp create cluster` command.

In a pre-provisioned environment, use the Kubernetes CSI and third party drivers for local volumes and other storage devices in your data center.

NKP uses local static provisioner as the default storage provider for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. You can use a Kubernetes CSI compatible storage that is suitable for production.

After disabling `localvolumeprovisioner`, you can choose from any of the storage options available for Kubernetes. To make that storage the default storage, use the commands shown in this section of the Kubernetes documentation: https://kubernetes.io/docs/tasks/administer-cluster/change-default-storage-class/

For Pre-provisioned environments, you define a set of nodes that already exist. During the cluster creation process, Konvoy Image Builder (KIB) is built into NKP and automatically runs the machine configuration process (which KIB uses to build images for other providers) against the set of nodes that you defined. This results in your pre-existing or pre-provisioned nodes being configured properly.

The following command relies on the pre-provisioned cluster API infrastructure provider to initialize the Kubernetes control plane and worker nodes on the hosts defined in the inventory YAML previously created.

The create cluster command below includes the --self-managed flag. A self-managed cluster refers to one in which the Cluster API (CAPI) resources and controllers that describe and manage it are running on the same cluster they are managing.

This command uses the default external load balancer (LB) option (see alternative Step 1 for virtual IP):

```
nkp create cluster preprovisioned \
  --cluster-name ${CLUSTER_NAME} \
  --control-plane-endpoint-host <control plane endpoint host> \
  --control-plane-endpoint-port <control plane endpoint port, if different than 6443> \
  --pre-provisioned-inventory-file preprovisioned_inventory.yaml \
  --ssh-private-key-file <path-to-ssh-private-key> \
  --self-managed
```

> **Note:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

1. **ALTERNATIVE Virtual IP** - if you don't have an external LB, and want to use a VIRTUAL IP provided by `kube-vip`, specify these flags example below:

```
nkp create cluster preprovisioned \
    --cluster-name ${CLUSTER_NAME} \
    --control-plane-endpoint-host 196.168.1.10 \
    --virtual-ip-interface eth1
```

2. Use the wait command to monitor the cluster control-plane readiness:

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=30m
```

   Output:

```
cluster.cluster.x-k8s.io/preprovisioned-example condition met
```

> **Note:** Depending on the cluster size, it will take a few minutes to create.

When the command completes, you will have a running Kubernetes cluster! For bootstrap and custom YAML cluster creation, see the Additional Infrastructure Customization section of the documentation for Pre-provisioned: Pre-provisioned Infrastructure

Use this command to get the Kubernetes `kubeconfig` for the new cluster and proceed to installing the NKP Kommander UI:

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

> **Note:** If changing the Calico encapsulation, Nutanix recommends changing it after cluster creation, but before production. See Calico encapsulation.

> **Important:** If you need to increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster, by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command. See Docker Hub's rate limit.

Audit Logs

To modify Control Plane Audit logs settings using the information contained in the page Configure the Control Plane.

Further Steps

For more customized cluster creation, access the Pre-Provisioned Additional Configurations section. That section is for Pre-Provisioned Override Files, custom flags, and more that specify the secret as part of the create cluster command. If these are not specified, the overrides for your nodes will not be applied.

MetalLB Configuration

Create a MetalLB configmap for your pre-provisioned infrastructure.

Nutanix recommends that an external load balancer be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines that are responding to API requests. If you do not have one, you can use Metal LB to create a MetalLB configmap for your pre-provisioned infrastructure.

Choose one of the following two protocols you want to use to define service IPs. If your environment is not currently equipped with a load balancer, use MetalLB. Otherwise, your load balancer will work, and

you can continue with the installation process. To use MetalLB, create a MetalLB configMap for your pre-provisioned infrastructure. MetalLB uses one of two protocols for exposing Kubernetes services:

- Layer 2, with Address Resolution Protocol (ARP)
- Border Gateway Protocol (BGP)

Layer 2 Configuration

Layer 2 mode is the simplest to configure. In many cases, you do not require any protocol-specific configuration, only IP addresses.

Layer 2 mode does not require the IPs to be bound to the network interfaces of your worker nodes. It works by responding to ARP requests on your local network directly to give the machine's MAC address to clients.

- MetalLB IP address ranges or Classless Inter-Domain Routing (CIDR) needs to be within the node's primary network subnets. For more information, see Managing Subnets and Pods on page 684.
- MetalLB IP address ranges or CIDRs and node subnets must not conflict with the Kubernetes cluster pod and service subnets.

For example, the following configuration gives MetalLB control over IPs from 192.168.1.240 to 192.168.1.250 and configures Layer 2 mode:

The following values are generic; enter your specific values into the fields where applicable.

```
cat << EOF > metallb-conf.yaml
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: default
  namespace: metallb-system
spec:
  addresses:
  - 192.168.1.240-192.168.1.250
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: default
  namespace: metallb-system
spec:
  ipAddressPools:
  - default
EOF
```

After this is complete, run the `kubectl apply -f metallb-conf.yaml` command.

Border Gateway Protocol (BGP) Configuration

For a basic configuration featuring one BGP router and one IP address range, you need the following four pieces of information:

- The router IP address that MetalLB must connect to.
- The router's autonomous systems (AS) number.
- The AS number MetalLB to use.
- An IP address range is expressed as a CIDR prefix.

As an example, if you want to specify the MetalLB range as 192.168.10.0/24 and AS number as 64500 and connect it to a router at 10.0.0.1 with AS number 64501, your configuration will be as follows:

```
cat << EOF > metallb-conf.yaml
apiVersion: metallb.io/v1beta2
kind: BGPPeer
metadata:
  name: default
  namespace: metallb-system
spec:
  myASN: 64500
  peerASN: 64501
  peerAddress: 10.0.0.1
---
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: default
  namespace: metallb-system
spec:
  addresses:
  - 192.168.10.0/24
---
apiVersion: metallb.io/v1beta1
kind: BGPAdvertisement
metadata:
  name: example
  namespace: metallb-system
spec:
  ipAddressPools:
  - default
EOF
```

After this is complete, run the `kubectl apply -f metallb-conf.yaml` command.

Pre-provisioned: Installing Kommander

This section describes the installation instructions for the Kommander component of NKP in a non-air-gapped pre-provisioned environment.

About this task

After installing the Konvoy component of NKP, continue with the installation of the Kommander component that enables you to bring up the UI dashboard.

Note:

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands using the `kubeconfig` file.
- Applications take longer to deploy and sometimes time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period (for example, 1 h) to allocate more time to the deployment of applications.
- If the Kommander installation fails and you want PVC-based storage that requires your CSI provider to support, rerun the `install` command to retry.

Caution: Nutanix support is unable to recover forgotten credentials. Prevent Kommander UI lockouts with the following best practices.

- Backup the kubeconfig file.

- Save your credentials.
- Configure an IDP (identity provider) instead of using a username and password credentials.

Before you begin

- Ensure you review all the prerequisites required for the installation.
- Ensure you have a default StorageClass (see Identifying and Modifying Your StorageClass on page 1038).
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to search and find it.

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. Edit the installer file to include configuration overrides for the `rook-ceph-cluster`. NKP's default configuration ships Ceph with PVC based storage that requires your CSI povider to support for PVC with type `volumeMode: Block`. As this is not possible with the default local static provisioner, you can install Ceph in host storage mode and choose whether Ceph's object storage daemon (osd) pods can consume all or just some of the devices on your nodes. Include one of the following overrides.

   a. To automatically assign all raw storage devices on all nodes to the Ceph cluster.

   ```
   rook-ceph-cluster:
       enabled: true
       values: |
         cephClusterSpec:
           storage:
             storageClassDeviceSets: []
             useAllDevices: true
             useAllNodes: true
             deviceFilter: "<<value>>"
   ```

   b. To assign specific storage devices on all nodes to the Ceph cluster.

   ```
   rook-ceph-cluster:
       enabled: true
       values: |
         cephClusterSpec:
           storage:
             storageClassDeviceSets: []
             useAllNodes: true
             useAllDevices: false
   ```

```
          deviceFilter: "^sdb."
```

> **Note:** If you want to assign specific devices to specific nodes using the `deviceFilter` option, refer to Specific Nodes and Devices. For general information on the `deviceFilter` value, refer to Storage Selection Settings.

5. (Optional) Customize your `kommander.yaml`.

   Some options include custom domains and certificates, HTTP proxy, and external load balancer.

6. Enable NKP catalog applications and install Kommander in the same `kommander.yaml`, add these values (if you are enabling NKP catalog applications) in `dkp-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP catalog applications after installing NKP, see Configuring Applications After Installing Kommander on page 1035.

Verifying your Installation

Verify Kommander installation. After you build the Konvoy cluster and you install Kommander, verify your installation. The cluster waits for all the applications to be ready by default.

About this task

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer using the CLI to not wait for all applications to be available, you can set the flag to `--wait=false`.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output.

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

What to do next

If an application fails to deploy, check the status of the `HelmRelease` using the following command.

```
kubectl -n kommander get helmrelease <HELMRELEASE_NAME>
```

If you find any `HelmReleases` in a *broken* release state, such as *exhausted* or *another rollback/release in progress*, trigger a reconciliation of the `HelmRelease` using the following commands. `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Logging In To the UI

Log in to the UI Dashboard. After you build the Konvoy cluster and install Kommander, verify your installation. The cluster waits for all the applications to be ready by default.

Procedure

1. By default, you can log in to the UI in Kommander with the credentials provided in this command.

   ```
   NKP open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

```
kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
 index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
dashboard{{ "\n"}}'
```

Only use static credentials to access the UI for configuring an external identity provider (see Identity Providers on page 364). Treat them as back up credentials rather than using them for normal access.

a. Rotate the password using the following command.

```
NKP experimental rotate dashboard-password
```

The output displays the new password.

```
Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
```

What to do next

After installing the Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations. For more information, Cluster Operations Management. The majority of the customization such as attaching clusters and deploying applications takes place in the dashboard or the NKP UI.

Create Managed Clusters Using the NKP CLI

This topic explains how to continue using the CLI to create managed clusters rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed, which allows it to be a Management cluster or a stand-alone cluster. Subsequent new clusters are not self-managed as they are likely to be managed or attached clusters to this Management Cluster.

> **Note:** When creating managed clusters, do not create and move CAPI objects or install the Kommander component. Those tasks are only done on Management clusters.
>
> Your new managed cluster must be part of a workspace under a management cluster. To make the new managed cluster a part of a workspace, set that workspace's environment variable.

Procedure

1. If you have an existing workspace name, run this command to find the name.

```
kubectl get workspace -A
```

2. After you find the workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace_namespace>
```

If you need to create a new workspace, see Creating a Workspace on page 411.

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to createthat cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster that can be used as the management cluster.

First, you must name your cluster. Then, you run the command to deploy it.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

```
export CLUSTER_NAME=<preprovisioned-additional>
```

Create a Kubernetes Cluster

About this task

After you have defined the infrastructure and control plane endpoints, you can proceed to createthat cluster by following these steps to create a new pre-provisioned cluster.

This process creates a self-managed cluster that can be used as the management cluster.

> **Tip:** Before you create a new Nutanix Kubernetes Platform (NKP) cluster below, choose an external load balancer (LB) or virtual IP and use the corresponding `NKP create cluster` command.

In a Pre-provisioned environment, use the Kubernetes CSI and third-party drivers for local volumes and other storage devices in your datacenter.

> **Caution:** NKP uses a local static provisioner as the default storage provider for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. Use Kubernetes CSI compatible storage that is suitable for production.

After turning off `localvolumeprovisioner`, you can choose from any of the storage options available for Kubernetes. To make that storage the default storage, use the commands shown in this section of the Kubernetes documentation: Change the default StorageClass.

For Pre-provisioned environments, you define a set of nodes that already exist. During the cluster creation process, Konvoy Image Builder(KIB) is built into NKP and automatically runs the machine configuration process (which KIB uses to build images for other providers) against the set of nodes that you defined. This results in your pre-existing or pre-provisioned nodes being configured properly.

The following command relies on the pre-provisioned cluster Application Programming Interface (API) infrastructure provider to initialize the Kubernetes control plane and worker nodes on the hosts defined in the inventory YAML previously created.

Procedure

1. This command uses the default external load balancer (LB) option.

```
nkp create cluster pre-provisioned
  --cluster-name ${CLUSTER_NAME}
```

```
 --control-plane-endpoint-host <control plane endpoint host>
 --control-plane-endpoint-port <control plane endpoint port, if different than
6443>
 --pre-provisioned-inventory-file preprovisioned_inventory.yaml
 --ssh-private-key-file <path-to-ssh-private-key>
```

2. Use the wait command to monitor the cluster control-plane readiness.

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=30m
```

> **Note:** NOTE: Depending on the cluster size, it will take a few minutes to create.

```
cluster.cluster.x-k8s.io/preprovisioned-additional condition met
```

> **Tip:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

```
echo ${MANAGED_CLUSTER_NAME}
```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

```
nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
 ${MANAGED_CLUSTER_NAME}.conf
```

3.

> **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

You can now either attach it in the UI, link to attaching it to the workspace through the UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

```
kubectl get workspaces -A
```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace-namespace>
```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

```
kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
template='{{.data.value}}{{ "\n"}}'
```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

```
apiVersion: v1
kind: Secret
metadata:
  name: <your-managed-cluster-name>-kubeconfig
  labels:
    cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
type: cluster.x-k8s.io/secret
data:
  value: <value-you-copied-from-secret-above>
```

8. Create this secret in the desired workspace.

```
kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
 ${WORKSPACE_NAMESPACE}
```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI, and you can confirm its status by running the command below. It might take a few minutes to reach "Joined" status.

```
kubectl get kommanderclusters -A
```

If you have several Pro Clusters and want to turn one of them into a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

Pre-provisioned Air-gapped Installation

Installation instructions for installing NKP in a pre-provisioned air-gapped environment.

Note: For air-gapped, ensure you have downloaded `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , so you can extract the tarball to a local registry.

```
tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
```

Pre-provisioned Air-gapped: Configure Environment

In order to create a cluster in a Pre-provisioned Air-gapped environment, you must first prepare the environment.

The instructions below outline how to fulfill the requirements for using pre-provisioned infrastructure in an air-gapped environment. In order to create a cluster, you must first set uppre-provisioned air-gapped need to be placed on the environment with necessary artifacts. All artifacts for Pre-provisioned Air-gapped need

to get onto the bastion host. Artifacts needed by nodes must be unpacked and distributed on the bastion before other provisioning will work in the absence of an internet connection.

There is an air-gapped bundle available to download NKP. In the previous NKP releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts, with the exception of the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Containerd tar file

1. Downloading NKP on page 16 `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tar file to a local directory:

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.
3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the new NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`. Example command:

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   NOTE: For FIPS, pass the flag: `--fips`

   NOTE: For RHEL OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`

   Example command:

   ```
   export RHSM_ACTIVATION_KEY="-ci"
   export RHSM_ORG_ID="1232131"
   ```

Setup Process

1. The bootstrap image must be extracted and loaded onto the bastion host.
2. Artifacts must be copied onto cluster hosts for nodes to access.
3. If using a graphics processing unit (GPU), those artifacts must be positioned locally.
4. Registry seeded with images locally.

Load the Bootstrap Image

1. Assuming you have downloaded `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` from the Downloading NKP on page 16 mentioned above and extracted the tar file, you will load the bootstrap.
2. Load the bootstrap image on your bastion machine:

   ```
   docker load -i konvoy-bootstrap-image-v2.12.0.tar
   ```

Copy air-gapped artifacts onto cluster hosts

.

Using the Konvoy Image Builder, you can copy the required artifacts onto your cluster hosts.

1. Assuming you have downloaded `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`, extract the tar file to a local directory:

2. The Kubernetes image bundle will be located in `kib/artifacts/images` , and you will want to verify the image and artifacts.

   1. Verify the image bundles exist in `artifacts/images`:

      ```
      $ ls artifacts/images/
      Kubernetes-images-1.29.6-d2iq.1.tar Kubernetes-images-1.29.6-d2iq.1-fips.tar
      ```

   2. Verify the artifacts for your OS exist in the `artifacts/` directory and export the appropriate variables:

      ```
      $ ls artifacts/
      1.29.6_centos_7_x86_64.tar.gz     1.29.6_redhat_8_x86_64_fips.tar.gz
       containerd-1.6.28-d2iq.1-rhel-8.6-x86_64_fips.tar.gz pip-packages.tar.gz
      1.29.6_centos_7_x86_64_fips.tar.gz  1.29.6_rocky_9_x86_64.tar.gz
       containerd-1.6.28-d2iq.1-rocky-9.0-x86_64.tar.gz
      1.29.6_redhat_7_x86_64.tar.gz     1.29.6_ubuntu_20_x86_64.tar.gz
       containerd-1.6.28-d2iq.1-rocky-9.1-x86_64.tar.gz
      1.29.6_redhat_7_x86_64_fips.tar.gz  containerd-1.6.28-d2iq.1-ubuntu-20.04-
      x86_64.tar.gz
      1.29.6_redhat_8_x86_64.tar.gz
      ```

   3. For example, for RHEL 8.6, you set:

      ```
      export OS_PACKAGES_BUNDLE=1.29.6_redhat_8_x86_64.tar.gz
      export CONTAINERD_BUNDLE=containerd-1.6.28-d2iq.1-rhel-8.6-x86_64.tar.gz
      ```

3. Export the following environment variables, ensuring that all control plane and worker nodes are included:

   ```
   export CONTROL_PLANE_1_ADDRESS="<control-plane-address-1>"
   export CONTROL_PLANE_2_ADDRESS="<control-plane-address-2>"
   export CONTROL_PLANE_3_ADDRESS="<control-plane-address-3>"
   export WORKER_1_ADDRESS="<worker-address-1>"
   export WORKER_2_ADDRESS="<worker-address-2>"
   export WORKER_3_ADDRESS="<worker-address-3>"
   export WORKER_4_ADDRESS="<worker-address-4>"
   export SSH_USER="<ssh-user>"
   export SSH_PRIVATE_KEY_FILE="<private key file>"
   ```

   `SSH_PRIVATE_KEY_FILE` must be either the name of the SSH private key file in your working directory or an absolute path to the file in your user's home directory.

4. Generate an `inventory.yaml` , which is automatically picked up by the `konvoy-image upload` in the next step.

   ```
   cat <<EOF > inventory.yaml
   all:
     vars:
       ansible_user: $SSH_USER
       ansible_port: 22
       ansible_ssh_private_key_file: $SSH_PRIVATE_KEY_FILE
     hosts:
       $CONTROL_PLANE_1_ADDRESS:
         ansible_host: $CONTROL_PLANE_1_ADDRESS
       $CONTROL_PLANE_2_ADDRESS:
         ansible_host: $CONTROL_PLANE_2_ADDRESS
       $CONTROL_PLANE_3_ADDRESS:
         ansible_host: $CONTROL_PLANE_3_ADDRESS
       $WORKER_1_ADDRESS:
         ansible_host: $WORKER_1_ADDRESS
       $WORKER_2_ADDRESS:
         ansible_host: $WORKER_2_ADDRESS
       $WORKER_3_ADDRESS:
         ansible_host: $WORKER_3_ADDRESS
       $WORKER_4_ADDRESS:
   ```

```
      ansible_host: $WORKER_4_ADDRESS
EOF
```

5. Upload the artifacts onto cluster hosts with the following command:

```
konvoy-image upload artifacts \
              --container-images-dir=./artifacts/images/ \
              --os-packages-bundle=./artifacts/$OS_PACKAGES_BUNDLE \
              --contained-bundle=artifacts/$CONTAINERD_BUNDLE \
              --pip-packages-bundle=./artifacts/pip-packages.tar.gz
```

   KIB uses variable overrides to specify the base image and container images to use in your new machine image. The variable overrides files for NVIDIA and Federal Information Processing Standards (FIPS) can be ignored unless an overlay feature is added.

Pre-provisioned Air-gapped: Configuring the Environment
In order to create a cluster in a pre-provisioned air-gapped environment, you must first prepare the environment.

The instructions in this topic describes using pre-provisioned infrastructure in an air-gapped environment. To create a cluster, you must first set up the environment with the necessary artifacts. All the artifacts for pre-provisioned air-gapped environments must be hosted onto the bastion host. Unpack the artifacts required by the nodes and distribute them on the bastion before other provisioning starts working in the absence of an internet connection.

There is an air-gapped bundle available for download in the Nutanix Support portal (see Downloading NKP on page 16). In previous NKP releases, the distro package bundles were included in the downloaded air-gapped bundle. Now, that air-gapped bundle contains the following artifacts, with the exception of the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Contains the tarball `.tar` file.

Pre-provisioned Air-gapped: Load the Registry
Before creating an air-gapped Kubernetes cluster, you need to load the required images in a local registry for the Konvoy component.

About this task

The complete Nutanix Kubernetes Platform (NKP) air-gapped bundle is needed for an air-gapped environment but can also be used in a non-air-gapped environment. The bundle contains all the NKP components needed for an air-gapped environment installation and also for using a local registry in a non-air-gapped environment.

> **Note:** If you do not already have a local registry set up, see the Local Registry Tools page for more information.

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images, is required. This registry must be accessible from both the bastion machine and either the Amazon Web Services (AWS) EC2 instances (if deploying to AWS) or other machines that will be created for the Kubernetes cluster.

Procedure

1. If not already done in prerequisites, download the air-gapped bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tar file to a local directory.

```
tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
```

2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. For example, for the bootstrap cluster, change your directory to the `nkp-<version>` directory, similar to the example below, depending on your current location

```
cd nkp-v2.12.0
```

3. Set an environment variable with your registry address and any other needed variables using this command.

```
export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
export REGISTRY_USERNAME=<username>
export REGISTRY_PASSWORD=<password>
export REGISTRY_CA=<path to the cacert file on the bastion>
```

4. Execute the following command to load the air-gapped image bundle into your private registry using any of the relevant flags to apply the variables above.

```
nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-
registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-
password=${REGISTRY_PASSWORD}
```

> **Note:** It might take some time to push all the images to your image registry, depending on the performance of the network between the machine you are running the script on and the registry.

> **Important:** To increase Docker Hub's rate limit use your Docker Hub credentials when creating the cluster, by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

5. Load the Kommander component images to your private registry using the command.

```
nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-
registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-
password=${REGISTRY_PASSWORD}
```

Optional: This step is required only if you have an Ultimate license.

For NKP Catalog Applications available with the Ultimate license, perform this image load by running the following command to load the `dkp-catalog-applications` image bundle into your private registry:

```
nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-
v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME}
 --to-registry-password=${REGISTRY_PASSWORD}
```

Pre-provisioned Air-gapped: Define Infrastructure

Define the cluster hosts and infrastructure in a pre-provisioned environment.

About this task

The Konvoy component of Nutanix Kubernetes Platform (NKP) needs to know how to access your cluster hosts so you must define the cluster hosts and infrastructure. This is done using inventory resources. For initial cluster creation, you must define a control-plane and at least one worker pool.

This procedure sets the necessary environment variables.

Procedure

1. Export the following environment variables, ensuring that all control plane and worker nodes are included:

```
export CONTROL_PLANE_1_ADDRESS="<control-plane-address-1>"
export CONTROL_PLANE_2_ADDRESS="<control-plane-address-2>"
```

```
export CONTROL_PLANE_3_ADDRESS="<control-plane-address-3>"
export WORKER_1_ADDRESS="<worker-address-1>"
export WORKER_2_ADDRESS="<worker-address-2>"
export WORKER_3_ADDRESS="<worker-address-3>"
export WORKER_4_ADDRESS="<worker-address-4>"
export SSH_USER="<ssh-user>"
export SSH_PRIVATE_KEY_SECRET_NAME="$CLUSTER_NAME-ssh-key"
```

2. Use the following template to help you define your infrastructure. The environment variables that you set in the previous step automatically replace the variable names when the inventory YAML Ain't Markup Language (YAML) file is created.

```
cat <<EOF > preprovisioned_inventory.yaml
---
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: $CLUSTER_NAME-control-plane
  namespace: default
  labels:
    cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
    clusterctl.cluster.x-k8s.io/move: ""
spec:
  hosts:
    # Create as many of these as needed to match your infrastructure
    # Note that the command line parameter --control-plane-replicas determines how
 many control plane nodes will actually be used.
    #
    - address: $CONTROL_PLANE_1_ADDRESS
    - address: $CONTROL_PLANE_2_ADDRESS
    - address: $CONTROL_PLANE_3_ADDRESS
  sshConfig:
    port: 22
    # This is the username used to connect to your infrastructure. This user must be
 root or
    # have the ability to use sudo without a password
    user: $SSH_USER
    privateKeyRef:
      # This is the name of the secret you created in the previous step. It must
 exist in the same
      # namespace as this inventory object.
      name: $SSH_PRIVATE_KEY_SECRET_NAME
      namespace: default
---
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: $CLUSTER_NAME-md-0
  namespace: default
  labels:
    cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
    clusterctl.cluster.x-k8s.io/move: ""
spec:
  hosts:
    - address: $WORKER_1_ADDRESS
    - address: $WORKER_2_ADDRESS
    - address: $WORKER_3_ADDRESS
    - address: $WORKER_4_ADDRESS
  sshConfig:
    port: 22
    user: $SSH_USER
    privateKeyRef:
```

```
      name: $SSH_PRIVATE_KEY_SECRET_NAME
      namespace: default
EOF
```

Pre-provisioned Air-gapped: Define Control Plane Endpoint

Define the control plane endpoint for your cluster and the connection mechanism. A control plane needs to have three, five, or seven nodes so it can remain available if one or more nodes fail. A control plane with one node is not for production use.

In addition, the control plane needs an endpoint that remains available if some nodes fail.

```
                          -------- cp1.example.com:6443
     |
     lb.example.com:6443 ---------- cp2.example.com:6443
     |
     -------- cp3.example.com:6443
```

In this example, the control plane endpoint host is `lb.example.com`, and the control plane endpoint port is `6443`. The control plane nodes are `cp1.example.com`, `cp2.example.com`, and `cp3.example.com`. The port of each API server is `6443`.

Select your Connection Mechanism

A virtual IP is the address that the client uses to connect to the service. A load balancer is a device that distributes the client connections to the backend servers. Before you create a new Nutanix Kubernetes Platform (NKP) cluster, choose an external load balancer (LB) or virtual IP.

- **External load balancer**

  It is recommended that an external load balancer be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines that are responding to Application Programming Interface (API) requests.

- **Built-in virtual IP**

  If an external load balancer is not available, use the built-in virtual IP. The virtual IP is not a load balancer; it does not distribute request load among the control plane machines. However, if the machine receiving requests does not respond to them, the virtual IP automatically moves to another machine.

Single-Node Control Plane

> **Caution:** Do not use a single-node control plane in a production cluster.

A control plane with one node can use its single node as the endpoint, so you will not require an external load balancer or a built-in virtual IP. At least one control plane node must always be running. Therefore, to upgrade a cluster with one control plane node, a spare machine must be available in the control plane inventory. This machine is used to provision the new node before the old node is deleted. When the API server endpoints are defined, you can create the cluster using the link in the Next Step below.

> **Note:** Modify Control Plane Audit logs settings using the information contained in the page Configure the Control Plane.

Known Limitations

The control plane endpoint port is also used as the API server port on each control plane machine. The default port is 6443. Before you create the cluster, ensure the port is available for use on each control plane machine.

Pre-provisioned Air-gapped: Creating a Management Cluster

Create a new Pre-provisioned Kubernetes cluster in an air-gapped environment.

About this task

After you have defined the infrastructure and control plane endpoints, you can proceed to create the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster that can be used as the management cluster.

Before you begin

First, you must name your cluster. Then, you run the command to deploy it. When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

> **Note:** The cluster name might only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. For more naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable: `export CLUSTER_NAME=preprovisioned-example`

What to do next

Create a Kubernetes Cluster

If your cluster is air-gapped or you have a local registry, you must provide additional arguments when creating the cluster. These tell the cluster where to locate the local registry to use by defining the URL.

```
export REGISTRY_URL=<https/http>://<registry-address>:<registry-port>
export REGISTRY_CA=<path to the CA on the bastion>
export REGISTRY_USERNAME=<username>
export REGISTRY_PASSWORD=<password>
```

- `REGISTRY_URL`: the address of an existing registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.
- `REGISTRY_CA`: (optional) the path on the bastion machine to the registry CA. Konvoy will configure the cluster nodes to trust this CA. This value is only needed if the registry is using a self-signed certificate and the AMIs are not already configured to trust this CA.
- `REGISTRY_USERNAME`: optional, set to a user that has pull access to this registry.
- `REGISTRY_PASSWORD`: optional if the username is not set.

Before you create a new NKP cluster below, you might choose an external load balancer or virtual IP and use the corresponding `nkp create cluster` command example from that page in the docs from the links below. Other customizations are available but require different flags during `nkp create cluster` command also. Refer to Pre-provisioned Cluster Creation Customization Choices for more cluster customizations.

When you create a new NKP cluster below, choose an external load balancer (LB) or virtual IP and use the corresponding `nkp create cluster` command.

In a pre-provisioned environment, use the Kubernetes CSI and third-party drivers for local volumes and other storage devices in your datacenter.

> **Note:** NKP uses a local static provisioner as the default storage provider for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. Use Kubernetes CSI compatible storage that is suitable for production.

After turning off `localvolumeprovisioner`, you can choose from any of the storage options available for Kubernetes. To make that storage the default storage, use the commands shown in this section of the Kubernetes documentation: Changing the Default Storage Class.

> **Note:** (Optional) Use a registry mirror. Configure your cluster to use an existing local registry as a mirror when attempting to pull images previously pushed to your registry when defining your infrastructure. Instructions in the expandable Custom Installation section. For registry mirror information, see topics Using a Registry Mirror and Registry Mirror Tools.

The create cluster command below includes the `--self-managed` flag. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing.

This command uses the **default external load balancer (LB)** option (see alternative Step 1 for virtual IP):

```
nkp create cluster pre-provisioned --cluster-name ${CLUSTER_NAME} \
  --control-plane-endpoint-host <control plane endpoint host> \
  --control-plane-endpoint-port <control plane endpoint port, if different than 6443> \
  --pre-provisioned-inventory-file preprovisioned_inventory.yaml \
  --ssh-private-key-file <path-to-ssh-private-key> \
  --registry-mirror-url=${REGISTRY_URL} \
  --registry-mirror-cacert=${REGISTRY_CA} \
  --registry-mirror-username=${REGISTRY_USERNAME} \
  --registry-mirror-password=${REGISTRY_PASSWORD} \
  --self-managed
```

> **Note:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

1. **ALTERNATIVE Virtual IP** - if you don't have an external LB and want to use a VIRTUAL IP provided by `kube-vip`, specify these flags example below:

   ```
   nkp create cluster pre-provisioned \
       --cluster-name ${CLUSTER_NAME} \
       --control-plane-endpoint-host 196.168.1.10 \
       --virtual-ip-interface eth1
   ```

2. Use the wait command to monitor the cluster control-plane readiness:

   ```
   kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
   timeout=30m
   ```

   Output:

   ```
   cluster.cluster.x-k8s.io/preprovisioned-example condition met
   ```

   > **Note:** Depending on the cluster size, it will take a few minutes to create.

When the command is complete, you will have a running Kubernetes cluster! For bootstrap and custom YAML cluster creation, refer to the Additional Infrastructure Customization section of the documentation for Pre-provisioned: Pre-provisioned Infrastructure.

Use this command to get the Kubernetes `kubeconfig` for the new cluster and proceed to install the NKP Kommander UI:

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

Note: If changing the Calico encapsulation, Nutanix recommends doing so after cluster creation but before production.

Audit Logs

To modify Control Plane Audit logs settings using the information contained on the page Configure the Control Plane.

Configure Air-gapped MetalLB

Create a MetalLB configmap for your Pre-provisioned Infrastructure.

It is recommended that an external load balancer (LB) be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines that are responding to API requests. If you do not have one, you can use Metal LB to create a MetalLB configmap for your Pre-provisioned infrastructure.

Choose one of the following two protocols you want to use to announce service IPs. If your environment is not currently equipped with a load balancer, you can use MetalLB. Otherwise, your own load balancer will work and you can continue the installation process with Pre-provisioned: Install Kommander . To use MetalLB, create a MetalLB configMap for your Pre-provisioned infrastructure. MetalLB uses one of two protocols for exposing Kubernetes services:

- Layer 2, with Address Resolution Protocol (ARP)
- Border Gateway Protocol (BGP)

  Select one of the following procedures to create your MetalLB manifest for further editing.

Layer 2 Configuration

Layer 2 mode is the simplest to configure: in many cases, you don’t need any protocol-specific configuration, only IP addresses.

Layer 2 mode does not require the IPs to be bound to the network interfaces of your worker nodes. It works by responding to ARP requests on your local network directly, to give the machine’s MAC address to clients.

- MetalLB IP address ranges or Classless Inter-Domain Routing (CIDR) need to be within the node’s primary network subnet.
- MetalLB IP address ranges or CIDRs and node subnet must not conflict with the Kubernetes cluster pod and service subnets.

For example, the following configuration gives MetalLB control over IPs from 192.168.1.240 to 192.168.1.250, and configures Layer 2 mode:

The following values are generic, enter your specific values into the fields where applicable.

```
cat << EOF > metallb-conf.yaml
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: default
  namespace: metallb-system
spec:
  addresses:
  - 192.168.1.240-192.168.1.250
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
```

```
  name: default
  namespace: metallb-system
spec:
  ipAddressPools:
  - default
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

BGP Configuration

For a basic configuration featuring one BGP router and one IP address range, you need 4 pieces of information:

- The router IP address that MetalLB needs to connect to.
- The router's autonomous systems (AS) number.
- The AS number MetalLB to be used.
- An IP address range expressed as a CIDR prefix.

As an example, if you want to give MetalLB the range 192.168.10.0/24 and AS number 64500, and connect it to a router at 10.0.0.1 with AS number 64501, your configuration will look like:

```
cat << EOF > metallb-conf.yaml
apiVersion: metallb.io/v1beta2
kind: BGPPeer
metadata:
  name: default
  namespace: metallb-system
spec:
  myASN: 64500
  peerASN: 64501
  peerAddress: 10.0.0.1
---
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: default
  namespace: metallb-system
spec:
  addresses:
  - 192.168.10.0/24
---
apiVersion: metallb.io/v1beta1
kind: BGPAdvertisement
metadata:
  name: example
  namespace: metallb-system
spec:
  ipAddressPools:
  - default
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

Pre-provisioned Air-gapped: Kommander Installation

Installation instructions for installing the Kommander component of Nutanix Kubernetes Platform (NKP) in a air-gapped pre-provisioned environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you want to reconfigure applications, rerun the `install` command to retry.

Caution: Nutanix support is unable to recover forgotten credentials. Prevent Kommander UI lockouts with the following best practices.

- Backup the kubeconfig file.
- Save your credentials.
- Configure an IDP (identity provider) instead of using a username and password credentials.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Installation.
- Ensure you have a default StorageClass.
- Ensure you have loaded all the necessary images for your configuration. See: Load the Images into Your Registry: Air-gapped Environments.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

```
export CLUSTER_NAME=<your-management-cluster-name>
```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

```
nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
```

3. Create a configuration file for the deployment.

```
nkp install kommander --init --airgapped > kommander.yaml
```

4. Edit the installer file to include configuration overrides for the `rook-ceph-cluster`. NKP's default configuration ships Ceph with PersistentVolumeClaim (PVC) based storage, which requires your CSI provider to support PVC with type `volumeMode: Block`. As this is not possible with the default local static provisioner, you can install Ceph in host storage mode. You can choose whether Ceph's object

storage daemon (osd) pods can consume all or just some of the devices on your nodes. Include one of the following Overrides.

a. To automatically assign all raw storage devices on all nodes to the Ceph cluster.

```
rook-ceph-cluster:
    enabled: true
    values: |
      cephClusterSpec:
        storage:
          storageClassDeviceSets: []
          useAllDevices: true
          useAllNodes: true
          deviceFilter: "<<value>>"
```

b. To assign specific storage devices on all nodes to the Ceph cluster.

```
rook-ceph-cluster:
    enabled: true
    values: |
      cephClusterSpec:
        storage:
          storageClassDeviceSets: []
          useAllNodes: true
          useAllDevices: false
          deviceFilter: "^sdb."
```

> **Note:** If you want to assign specific devices to specific nodes using the `deviceFilter` option, refer to Specific Nodes and Devices. For general information on the `deviceFilter` value, refer to Storage Selection Settings.

5. If required: Customize your `kommander.yaml`.

   a. See the Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `NKP-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see the topic Configuring NKP Catalog Applications after Installing NKP.

Verifying your Installation and UI Log in

Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the command-line interface (CLI) to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   NKP open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      NKP experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Create Managed Clusters Using the NKP CLI

This topic explains how to continue using the command-line interface (CLI) to create managed clusters in an air-gapped Pre-provisioned environment rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed which allows it to be a Management cluster or a stand alone cluster. Subsequent new clusters are not self-managed as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects, or install the Kommander component. Those tasks are only done on Management clusters.
>
> To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, find the name using the command `kubectl get workspace -A`.
2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable using the command `export WORKSPACE_NAMESPACE=<workspace_namespace>`.

> **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First you must name your cluster. Then you run the command to deploy it.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. For more naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable using the command `export CLUSTER_NAME=<preprovisioned-additional>`.

Create a Kubernetes Cluster

About this task

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster.

This process creates a self-managed cluster that can be used as the Management cluster.

> **Tip:** Before you create a new NKP cluster below, choose an external load balancer (LB) or Pre-provisioned Built-in Virtual IP on page 755 and use the corresponding `NKP create cluster` command.

In a Pre-provisioned environment, use the Kubernetes CSI and third-party drivers for local volumes and other storage devices in your data center.

> **Caution:** NKP uses a local static provisioner as the default storage provider for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. Use Kubernetes CSI-compatible storage that is suitable for production. For more information, see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types.

After turning off `localvolumeprovisioner`, you can choose from any of the storage options available for Kubernetes. To make that storage the default storage, use the commands shown in the Change the default StorageClass section of the Kubernetes documentation. For more information, see https://kubernetes.io/docs/tasks/administer-cluster/change-default-storage-class/

For Pre-provisioned environments, you define a set of nodes that already exist. During the cluster creation process, Konvoy Image Builder (KIB) is built into NKP and automatically runs the machine configuration process (which KIB uses to build images for other providers) against the set of nodes that you defined. This results in your pre-existing or pre-provisioned nodes being configured properly.

The following command relies on the pre-provisioned cluster API infrastructure provider to initialize the Kubernetes control plane and worker nodes on the hosts defined in the inventory YAML previously created.

Procedure

1. This command uses the default external load balancer (LB) option.

```
nkp create cluster earlier earlierisioned --cluster-name ${CLUSTER_NAME}
  --control-plane-endpoint-host <control plane endpoint host>
  --control-plane-endpoint-port <control plane endpoint port, if different than 6443>
  --pre-provisioned-inventory-file preprovisioned_inventory.yaml
  --ssh-private-key-file <path-to-ssh-private-key>
  --registry-mirror-url=${REGISTRY_URL} \
  --registry-mirror-cacert=${REGISTRY_CA} \
  --registry-mirror-username=${REGISTRY_USERNAME} \
  --registry-mirror-password=${REGISTRY_PASSWORD}
```

2. Use the wait command to monitor the cluster control-plane readiness.

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=30m
```

> **Note:** NOTE: Depending on the cluster size, it will take a few minutes to create.

```
cluster.cluster.x-k8s.io/preprovisioned-additional condition met
```

> **Tip:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Clusters with HTTP or HTTPS Proxy on page 680.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set using the command `echo ${MANAGED_CLUSTER_NAME}` .
2. Retrieve your kubeconfig from the cluster you have created without setting a workspace using the command `nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} > ${MANAGED_CLUSTER_NAME}.conf`.

   You can now either attach it in the UI, link to attaching it to the workspace through earlier UI, or attach your cluster to the workspace you want in the CLI.

> **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

3. Retrieve the workspace where you want to attach the cluster using the command `kubectl get workspaces -A`.
4. Set the `WORKSPACE_NAMESPACE` environment variable using the command `export WORKSPACE_NAMESPACE=<workspace-namespace>`.
5. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster using the command `kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-template='{{.data.value}}{{ "\n"}}'`.
6. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

```
apiVersion: v1
kind: Secret
metadata:
  name: <your-managed-cluster-name>-kubeconfig
  labels:
    cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
type: cluster.x-k8s.io/secret
data:
  value: <value-you-copied-from-secret-above>
```

7. Create this secret in the desired workspace using the command `kubectl apply -f attached-cluster-kubeconfig.yaml --namespace ${WORKSPACE_NAMESPACE}`
8. Create this `kommandercluster` object to attach the cluster to the workspace.

   Example:

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

9. You can now view this cluster in your Workspace in the UI, and you can confirm its status by using the command `kubectl get kommanderclusters -A`.

   It might take a few minutes to reach "Joined" status.

   If you have several Pro Clusters and want to turn one of them into a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion:

Pre-provisioned FIPS Install

This section provides instructions to install NKP in a Pre-provisioned non-air-gapped environment with FIPS requirements.

Ensure Configuration

If not already done, see the documentation for:

- Resource Requirements on page 39

- Installing NKP on page 50
- Prerequisites for Installation on page 47

Pre-provisioned FIPS: Define Infrastructure
Define the cluster hosts and infrastructure in a pre-provisioned environment.

About this task

Konvoy needs to know how to access your cluster hosts. This is done using inventory resources. For initial cluster creation, you must define a controlcontrol-plane controland at least one worker pool.

This procedure sets the necessary environment variables.

Procedure

1. Export the following environment variables, ensuring that all control plane and worker nodes are included:

```
export CONTROL_PLANE_1_ADDRESS="<control-plane-address-1>"
export CONTROL_PLANE_2_ADDRESS="<control-plane-address-2>"
export CONTROL_PLANE_3_ADDRESS="<control-plane-address-3>"
export WORKER_1_ADDRESS="<worker-address-1>"
export WORKER_2_ADDRESS="<worker-address-2>"
export WORKER_3_ADDRESS="<worker-address-3>"
export WORKER_4_ADDRESS="<worker-address-4>"
export SSH_USER="<ssh-user>"
export SSH_PRIVATE_KEY_SECRET_NAME="$CLUSTER_NAME-ssh-key"
```

2. Use the following template to help you define your infrastructure. The environment variables that you set in the previous step automatically replace the variable names when the inventory YAML file is created.

```
cat <<EOF > preprovisioned_inventory.yaml
---
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: $CLUSTER_NAME-control-plane
  namespace: default
  labels:
    cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
    clusterctl.cluster.x-k8s.io/move: ""
spec:
  hosts:
    # Create as many of these as needed to match your infrastructure
    # Note that the command-line parameter—-control-plane-replicas determines how
 many control plane nodes will actually be used.
    #
    - address: $CONTROL_PLANE_1_ADDRESS
    - address: $CONTROL_PLANE_2_ADDRESS
    - address: $CONTROL_PLANE_3_ADDRESS
  sshConfig:
    port: 22
    # This is the username used to connect to your infrastructure. This user must be
 root or
    # have the ability to use sudo without a password
    user: $SSH_USER
    privateKeyRef:
      # This is the name of the secret you created in the previous step. It must
 exist in the same
```

```
      # namespace as this inventory object.
      name: $SSH_PRIVATE_KEY_SECRET_NAME
      namespace: default
---
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: $CLUSTER_NAME-md-0
  namespace: default
  labels:
    cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
    clusterctl.cluster.x-k8s.io/move: ""
spec:
  hosts:
    - address: $WORKER_1_ADDRESS
    - address: $WORKER_2_ADDRESS
    - address: $WORKER_3_ADDRESS
    - address: $WORKER_4_ADDRESS
  sshConfig:
    port: 22
    user: $SSH_USER
    privateKeyRef:
      name: $SSH_PRIVATE_KEY_SECRET_NAME
      namespace: default
EOF
```

Pre-provisioned FIPS: Define Control Plane Endpoint

Define the control plane endpoint for your cluster and the connection mechanism. A control plane needs to have three, five, or seven nodes so it can remain available if one or more nodes fail. A control plane with one node, is not for production use.

In addition, the control plane needs an endpoint that remains available if some nodes fail.

```
                            -------- cp1.example.com:6443
     |
     lb.example.com:6443 ---------- cp2.example.com:6443
     |
     -------- cp3.example.com:6443
```

In this example, the control plane endpoint host is `lb.example.com`, and the control plane endpoint port is `6443`. The control plane nodes are `cp1.example.com`, `cp2.example.com`, and `cp3.example.com`. The port of each API server is `6443`.

Select your Connection Mechanism

A virtual IP is the address that the client uses to connect to the service. A load balancer is a device that distributes the client connections to the backend servers. Before you create a new NKP cluster, choose an external load balancer (LB) or virtual IP.

- **External load balancer**

  It is recommended that an external load balancer be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines that are responding to API requests.

- **Built-in virtual IP**

  If an external load balancer is not available, use the built-in virtual IP. The virtual IP is not a load balancer; it does not distribute request load among the control plane machines. However, if the machine receiving requests does not respond to them, the virtual IP automatically moves to another machine.

Single-Node Control Plane

> **Caution:** Do not use a single-node control plane in a production cluster.

A control plane with one node can use its single node as the endpoint, so you will not require an external load balancer, or a built-in virtual IP. At least one control plane node must always be running. Therefore, to upgrade a cluster with one control plane node, a spare machine must be available in the control plane inventory. This machine is used to provision the new node before the old node is deleted. When the API server endpoints are defined, you can create the cluster using the link in the Next Step below.

> **Note:** Modify Control Plane Audit logs settings using the information contained in the page Configure the Control Plane.

Known Limitations

The control plane endpoint port is also used as the API server port on each control plane machine. The default port is 6443. Before you create the cluster, ensure the port is available for use on each control plane machine.

Pre-provisioned FIPS: Creating the Management Cluster

Create a new Pre-provisioned Kubernetes cluster in a non-air-gapped environment using the steps below.

About this task

After you have defined the infrastructure and control plane endpoints, you can proceed to create the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster that can be used as the management cluster.

Before you begin

First, you must name your cluster. Then, you run the command to deploy it. When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

> **Note:** The cluster name might only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. For more naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable: `export CLUSTER_NAME=preprovisioned-example`

What to do next

Create a Kubernetes Cluster

After you have defined the infrastructure and control plane endpoints, you can proceed to create the cluster by following these steps to create a new Pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

Before you create a new NKP cluster below, choose an external load balancer (LB) or virtual IP and use the corresponding `nkp create cluster` command.

In a pre-provisioned environment, use the Kubernetes CSI and third party drivers for local volumes and other storage devices in your data center.

NKP uses a local static provisioner as the default storage provider for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. You can use a Kubernetes CSI that is suitable for production. For more information, see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types

After turning off `localvolumeprovisioner`, you can choose from any of the storage options available for Kubernetes. To make that storage the default storage, use the commands shown in this section of the Kubernetes documentation: Changing the Default Storage Class.

For Pre-provisioned environments, you define a set of nodes that already exist. During the cluster creation process, Konvoy Image Builder(KIB) is built into NKP and automatically runs the machine configuration process (which KIB uses to build images for other providers) against the set of nodes that you defined. This results in your pre-existing or pre-provisioned nodes being configured properly.

The following command relies on the pre-provisioned cluster API infrastructure provider to initialize the Kubernetes control plane and worker nodes on the hosts defined in the inventory YAML previously created.

The create cluster command below includes the --self-managed flag. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing.

This command uses the default external load balancer (LB) option (see alternative Step 1 for virtual IP):

```
nkp create cluster preprovisioned \
  --cluster-name ${CLUSTER_NAME} \
  --control-plane-endpoint-host <control plane endpoint host> \
  --control-plane-endpoint-port <control plane endpoint port, if different than 6443> \
  --pre-provisioned-inventory-file preprovisioned_inventory.yaml \
  --ssh-private-key-file <path-to-ssh-private-key> \
  --self-managed
```

> **Note:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

1. **ALTERNATIVE Virtual IP** - if you don't have an external LB and want to use a VIRTUAL IP provided by `kube-vip`, specify these flags example below:

   ```
   nkp create cluster preprovisioned \
       --cluster-name ${CLUSTER_NAME} \
       --control-plane-endpoint-host 196.168.1.10 \
       --virtual-ip-interface eth1
   ```

2. Use the wait command to monitor the cluster control-plane readiness:

   ```
   kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
   timeout=30m
   ```

   Output:

   ```
   cluster.cluster.x-k8s.io/preprovisioned-example condition met
   ```

   > **Note:** Depending on the cluster size, it will take a few minutes to create.

3. When the command completes, you will have a running Kubernetes cluster! For bootstrap and custom YAML cluster creation, refer to the Additional Infrastructure Customization section of the documentation for Pre-provisioned: Pre-Provisioned Infrastructure.

   Use this command to get the Kubernetes `kubeconfig` for the new cluster and proceed to install the NKP Kommander UI:

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
   ```

   > **Important:** If you need to increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://`

`registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command. See Docker Hub's rate limit.

Note: If changing the Calico encapsulation, Nutanix recommends doing so after cluster creation, but before production. See Calico encapsulation.

Configure MetalLB

Create a MetalLB configmap for your Pre-provisioned Infrastructure.

Choose one of the following two protocols you want to use to announce service IPs. If your environment is not currently equipped with a load balancer, you can use MetalLB. Otherwise, your own load balancer will work, and you can continue the installation process with Pre-provisioned: Install Kommander. To use MetalLB, create a MetalLB configMap for your Pre-provisioned infrastructure. MetalLB uses one of two protocols for exposing Kubernetes services:

- Layer 2, with Address Resolution Protocol (ARP)
- Border Gateway Protocol (BGP)

  Select one of the following procedures to create your MetalLB manifest for further editing.

Layer 2 Configuration

Layer 2 mode is the simplest to configure: in many cases, you don't need any protocol-specific configuration, only IP addresses.

Layer 2 mode does not require the IPs to be bound to the network interfaces of your worker nodes. It works by responding to ARP requests on your local network directly and giving the machine's MAC address to clients.

- MetalLB IP address ranges or CIDRs needs to be within the node's primary network subnet.
- MetalLB IP address ranges or CIDRs and node subnets must not conflict with the Kubernetes cluster pod and service subnets.

For example, the following configuration gives MetalLB control over IPs from 192.168.1.240 to 192.168.1.250, and configures Layer 2 mode:

The following values are generic, enter your specific values into the fields where applicable.

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    address-pools:
    - name: default
      protocol: layer2
      addresses:
      - 192.168.1.240-192.168.1.250
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

BGP Configuration

For a basic configuration featuring one BGP router and one IP address range, you need 4 pieces of information:

- The router IP address that MetalLB needs to connect to.
- The router's autonomous systems (AS) number.
- The AS number MetalLB to be used.
- An IP address range expressed as a Classless Inter-Domain Routing (CIDR) prefix.

As an example, if you want to give MetalLB the range 192.168.10.0/24 and AS number 64500, and connect it to a router at 10.0.0.1 with AS number 64501, your configuration will look like:

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    peers:
    - peer-address: 10.0.0.1
      peer-asn: 64501
      my-asn: 64500
    address-pools:
    - name: default
      protocol: bgp
      addresses:
      - 192.168.10.0/24
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

Pre-provisioned FIPS: Install Kommander

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped pre-provisioned environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Caution: Nutanix support is unable to recover forgotten credentials. Prevent Kommander UI lockouts with the following best practices.

- Backup the kubeconfig file.
- Save your credentials.
- Configure an IDP (identity provider) instead of using a username and password credentials.

Before you begin:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. Edit the installer file to include configuration overrides for the `rook-ceph-cluster`. NKP's default configuration ships Ceph with PersistentVolumeClaim (PVC) based storage which requires your CSI provider to support PVC with type `volumeMode: Block`. As this is not possible with the default local static provisioner, you can install Ceph in host storage mode. You can choose whether Ceph's object storage daemon (osd) pods can consume all or just some of the devices on your nodes. Include one of the following Overrides.

   a. To automatically assign all raw storage devices on all nodes to the Ceph cluster.

   ```
   rook-ceph-cluster:
       enabled: true
       values: |
         cephClusterSpec:
           storage:
             storageClassDeviceSets: []
             useAllDevices: true
             useAllNodes: true
             deviceFilter: "<<value>>"
   ```

   b. To assign specific storage devices on all nodes to the Ceph cluster.

   ```
   rook-ceph-cluster:
       enabled: true
       values: |
         cephClusterSpec:
           storage:
             storageClassDeviceSets: []
             useAllNodes: true
             useAllDevices: false
             deviceFilter: "^sdb."
   ```

   Note: If you want to assign specific devices to specific nodes using the `deviceFilter` option, refer to Specific Nodes and Devices. For general information on the `deviceFilter` value, refer to Storage Selection Settings.

5. If required: Customize your `kommander.yaml`.

   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy and External Load Balancer.

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `NKP-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Enable NKP Catalog Applications after Installing NKP.

Verifying your Installation and UI Log in
Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   NKP open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

```
kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
 index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
dashboard{{ "\n"}}'
```

Only use the static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

a. Rotate the password using the following command.

```
NKP experimental rotate dashboard-password
```

The output displays the new password:

```
Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
```

Create Managed Clusters Using the NKP CLI

This topic explains how to continue using the CLI to create managed clusters in a Pre-provisioned environment with FIPS rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed which allows it to be a Management cluster or a stand alone cluster. Subsequent new clusters are not self-managed as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects, or install the Kommander component. Those tasks are only done on Management clusters!
>
> To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

```
kubectl get workspace -A
```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace_namespace>
```

> **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First you must name your cluster. Then you run the command to deploy it.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

```
export CLUSTER_NAME=<preprovisioned-additional>
```

Create a Kubernetes Cluster

About this task

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster.

This process creates a self-managed cluster to be used as the Management cluster.

Tip: Before you create a new NKP cluster below, choose an external load balancer (LB) or virtual IP and use the corresponding `NKP create cluster` command.

In a Pre-provisioned environment, use the Kubernetes CSI and third party drivers for local volumes and other storage devices in your data center.

Caution: NKP uses local static provisioner as the default storage provider for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. Use Kubernetes CSI compatible storage that is suitable for production.

After disabling `localvolumeprovisioner`, you can choose from any of the storage options available for Kubernetes. To make that storage the default storage, use the commands shown in this section of the Kubernetes documentation: Change the default StorageClass

For Pre-provisioned environments, you define a set of nodes that already exist. During the cluster creation process, Konvoy Image Builder(KIB) is built into NKP and automatically runs the machine configuration process (which KIB uses to build images for other providers) against the set of nodes that you defined. This results in your pre-existing or pre-provisioned nodes being configured properly.

The following command relies on the pre-provisioned cluster API infrastructure provider to initialize the Kubernetes control plane and worker nodes on the hosts defined in the inventory YAML previously created.

Procedure

1. This command uses the default external load balancer (LB) option.

```
nkp create cluster preprovisioned \
  --cluster-name ${CLUSTER_NAME} \
  --control-plane-endpoint-host <control plane endpoint host> \
  --control-plane-endpoint-port <control plane endpoint port, if different than 6443>
 \
  --pre-provisioned-inventory-file preprovisioned_inventory.yaml \
  --ssh-private-key-file <path-to-ssh-private-key> \
  --kubernetes-version=v1.29.6+fips.0 \
  --etcd-version=3.5.10+fips.0 \
  --kubernetes-image-repository=docker.io/mesosphere
```

2. Use the wait command to monitor the cluster control-plane readiness.

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=30m
```

> **Note:** NOTE: Depending on the cluster size, it will take a few minutes to create.

```
cluster.cluster.x-k8s.io/preprovisioned-additional condition met
```

> **Tip:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

```
echo ${MANAGED_CLUSTER_NAME}
```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

```
nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
 ${MANAGED_CLUSTER_NAME}.conf
```

3. > **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

```
kubectl get workspaces -A
```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace-namespace>
```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

```
kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
template='{{.data.value}}{{ "\n"}}'
```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

```
apiVersion: v1
kind: Secret
metadata:
  name: <your-managed-cluster-name>-kubeconfig
  labels:
    cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
```

```
type: cluster.x-k8s.io/secret
data:
  value: <value-you-copied-from-secret-above>
```

8. Create this secret in the desired workspace.

```
kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
 ${WORKSPACE_NAMESPACE}
```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

```
kubectl get kommanderclusters -A
```

If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

Pre-provisioned FIPS Air-gapped Install

This section provides instructions to install NKP in a Pre-provisioned air-gapped environment with FIPS requirements.

Ensure Configuration

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Note:

- For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

  ```
  tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
  ```

- Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

  For example:

  ```
  export TMPDIR=/path/to/your/directory
  ```

Pre-provisioned Air-gapped FIPS: Configure Environment

In order to create a cluster in a Pre-provisioned Air-gapped environment, you must first prepare the environment.

The instructions below outline how to fulfill the requirements for using pre-provisioned infrastructure in an air-gapped environment. In order to create a cluster, you must first setup the environment with necessary artifacts. All artifacts for Pre-provisioned Air-gapped need to get onto the bastion host. Artifacts needed by nodes must be unpacked and distributed on the bastion before other provisioning will work in the absence of an internet connection.

There is an air-gapped bundle available to download. In previous NKP releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts with the exception of the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Containerd tarball

1. Download `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarball to a local directory:

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.
3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the new NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`. Example command:

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   NOTE: For FIPS, pass the flag: `--fips`

   NOTE: For RHEL OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`

   Example command:

   ```
   export RHSM_ACTIVATION_KEY="-ci"
   export RHSM_ORG_ID="1232131"
   ```

Setup Process

1. The bootstrap image must be extracted and loaded onto the bastion host.
2. Artifacts must be copied onto cluster hosts for nodes to access.
3. If using GPU, those artifacts must be positioned locally.
4. Registry seeded with images locally.

Load the Bootstrap Image

1. Assuming you have downloaded `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` from the download site mentioned above and extracted the tarball, you will load the bootstrap.
2. Load the bootstrap image on your bastion machine:

```
docker load -i konvoy-bootstrap-image-v2.12.0.tar
```

Copy air-gapped artifacts onto cluster hosts

Using the Konvoy Image Builder, you can copy the required artifacts onto your cluster hosts.

1. Assuming you have downloaded `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , extract the tarball to a local directory:
2. The kubernetes image bundle will be located in `kib/artifacts/images` and you will want to verify image and artifacts.
   1. Verify the image bundles exist in `artifacts/images`:

      ```
      $ ls artifacts/images/
      kubernetes-images-1.29.6-d2iq.1.tar kubernetes-images-1.29.6-d2iq.1-fips.tar
      ```

   2. Verify the artifacts for your OS exist in the `artifacts/` directory and export the appropriate variables:

      ```
      $ ls artifacts/
      1.29.6_centos_7_x86_64.tar.gz     1.29.6_redhat_8_x86_64_fips.tar.gz
       containerd-1.6.28-d2iq.1-rhel-8.6-x86_64_fips.tar.gz pip-packages.tar.gz
      1.29.6_centos_7_x86_64_fips.tar.gz  1.29.6_rocky_9_x86_64.tar.gz
       containerd-1.6.28-d2iq.1-rocky-9.0-x86_64.tar.gz
      1.29.6_redhat_7_x86_64.tar.gz     1.29.6_ubuntu_20_x86_64.tar.gz
       containerd-1.6.28-d2iq.1-rocky-9.1-x86_64.tar.gz
      1.29.6_redhat_7_x86_64_fips.tar.gz  containerd-1.6.28-d2iq.1-ubuntu-20.04-
      x86_64.tar.gz
      1.29.6_redhat_8_x86_64.tar.gz
      ```

   3. For example, for RHEL 8.6 you set:

      ```
      export OS_PACKAGES_BUNDLE=1.29.6_redhat_8_x86_64.tar.gz
      export CONTAINERD_BUNDLE=containerd-1.6.28-d2iq.1-rhel-8.6-x86_64.tar.gz
      ```

3. Export the following environment variables, ensuring that all control plane and worker nodes are included:

   ```
   export CONTROL_PLANE_1_ADDRESS="<control-plane-address-1>"
   export CONTROL_PLANE_2_ADDRESS="<control-plane-address-2>"
   export CONTROL_PLANE_3_ADDRESS="<control-plane-address-3>"
   export WORKER_1_ADDRESS="<worker-address-1>"
   export WORKER_2_ADDRESS="<worker-address-2>"
   export WORKER_3_ADDRESS="<worker-address-3>"
   export WORKER_4_ADDRESS="<worker-address-4>"
   export SSH_USER="<ssh-user>"
   export SSH_PRIVATE_KEY_FILE="<private key file>"
   ```

   `SSH_PRIVATE_KEY_FILE` must be either the name of the SSH private key file in your working directory or an absolute path to the file in your user's home directory.
4. Generate an `inventory.yaml` which is automatically picked up by the `konvoy-image upload` in the next step.

   ```
   cat <<EOF > inventory.yaml
   all:
     vars:
       ansible_user: $SSH_USER
       ansible_port: 22
       ansible_ssh_private_key_file: $SSH_PRIVATE_KEY_FILE
   ```

```
  hosts:
    $CONTROL_PLANE_1_ADDRESS:
      ansible_host: $CONTROL_PLANE_1_ADDRESS
    $CONTROL_PLANE_2_ADDRESS:
      ansible_host: $CONTROL_PLANE_2_ADDRESS
    $CONTROL_PLANE_3_ADDRESS:
      ansible_host: $CONTROL_PLANE_3_ADDRESS
    $WORKER_1_ADDRESS:
      ansible_host: $WORKER_1_ADDRESS
    $WORKER_2_ADDRESS:
      ansible_host: $WORKER_2_ADDRESS
    $WORKER_3_ADDRESS:
      ansible_host: $WORKER_3_ADDRESS
    $WORKER_4_ADDRESS:
      ansible_host: $WORKER_4_ADDRESS
EOF
```

5. Upload the artifacts onto cluster hosts with the following command:

```
konvoy-image upload artifacts \
              --container-images-dir=./artifacts/images/ \
              --os-packages-bundle=./artifacts/$OS_PACKAGES_BUNDLE \
              --containerd-bundle=artifacts/$CONTAINERD_BUNDLE \
              --pip-packages-bundle=./artifacts/pip-packages.tar.gz
```

KIB uses variable overrides to specify base image and container images to use in your new machine image. The variable overrides files for NVIDIA and FIPS can be ignored unless adding an overlay feature.

- Use the --overrides flag and reference either `fips.yaml` or `offline-fips.yaml` manifests located in the overrides directory or see these pages in the documentation:
  - FIPS Overrides
  - Create FIPS 140 Images

Pre-provisioned Air-gapped FIPS: Load the Registry

Before creating an air-gapped Kubernetes cluster, you need to load the required images in a local registry for the Konvoy component.

About this task

The complete NKP air-gapped bundle is needed for an air-gapped environment but can also be used in a non-air-gapped environment. The bundle contains all the NKP components needed for an air-gapped environment installation and also to use a local registry in a non-air-gapped environment.

> **Note:** If you do not already have a local registry set up, see the Local Registry Tools page for more information.

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images is required. This registry must be accessible from both the bastion machine and either the AWS EC2 instances (if deploying to AWS) or other machines that will be created for the Kubernetes cluster.

Procedure

1. If not already done in prerequisites, download the air-gapped bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarball to a local directory.

```
tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
```

2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. EX: For the bootstrap cluster, change your directory to the `nkp-<version>` directory similar to example below depending on your current location

```
cd nkp-v2.12.0
```

3. Set an environment variable with your registry address and any other needed variables using this command.

```
export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
export REGISTRY_USERNAME=<username>
export REGISTRY_PASSWORD=<password>
export REGISTRY_CA=<path to the cacert file on the bastion>
```

4. Execute the following command to load the air-gapped image bundle into your private registry using any of the relevant flags to apply variables above.

```
nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-
registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-
password=${REGISTRY_PASSWORD}
```

> **Note:** It may take some time to push all the images to your image registry, depending on the performance of the network between the machine you are running the script on and the registry.

> **Important:** To increase Docker Hub's rate limit use your Docker Hub credentials when creating the cluster, by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

5. Load the Kommander component images to your private registry using the command.

```
nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-
registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-
password=${REGISTRY_PASSWORD}
```

Optional: This step is required only if you have an Ultimate license.

For NKP Catalog Applications available with the Ultimate license, perform this image load by running the following command to load the `dkp-catalog-applications` image bundle into your private registry:

```
nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-
v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME}
 --to-registry-password=${REGISTRY_PASSWORD}
```

Pre-provisioned Air-gapped FIPS: Define Infrastructure

Define the cluster hosts and infrastructure in a pre-provisioned environment.

About this task

Konvoy needs to know how to access your cluster hosts. This is done using inventory resources. For initial cluster creation, you must define a control-plane and at least one worker pool.

This procedure sets the necessary environment variables.

Procedure

1. Export the following environment variables, ensuring that all control plane and worker nodes are included:

```
export CONTROL_PLANE_1_ADDRESS="<control-plane-address-1>"
export CONTROL_PLANE_2_ADDRESS="<control-plane-address-2>"
export CONTROL_PLANE_3_ADDRESS="<control-plane-address-3>"
```

```
export WORKER_1_ADDRESS="<worker-address-1>"
export WORKER_2_ADDRESS="<worker-address-2>"
export WORKER_3_ADDRESS="<worker-address-3>"
export WORKER_4_ADDRESS="<worker-address-4>"
export SSH_USER="<ssh-user>"
export SSH_PRIVATE_KEY_SECRET_NAME="$CLUSTER_NAME-ssh-key"
```

2. Use the following template to help you define your infrastructure. The environment variables that you set in the previous step automatically replace the variable names when the inventory YAML file is created.

```
cat <<EOF > preprovisioned_inventory.yaml
---
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: $CLUSTER_NAME-control-plane
  namespace: default
  labels:
    cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
    clusterctl.cluster.x-k8s.io/move: ""
spec:
  hosts:
    # Create as many of these as needed to match your infrastructure
    # Note that the command line parameter --control-plane-replicas determines how
 many control plane nodes will actually be used.
    #
    - address: $CONTROL_PLANE_1_ADDRESS
    - address: $CONTROL_PLANE_2_ADDRESS
    - address: $CONTROL_PLANE_3_ADDRESS
  sshConfig:
    port: 22
    # This is the username used to connect to your infrastructure. This user must be
 root or
    # have the ability to use sudo without a password
    user: $SSH_USER
    privateKeyRef:
      # This is the name of the secret you created in the previous step. It must
 exist in the same
      # namespace as this inventory object.
      name: $SSH_PRIVATE_KEY_SECRET_NAME
      namespace: default
---
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: $CLUSTER_NAME-md-0
  namespace: default
  labels:
    cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
    clusterctl.cluster.x-k8s.io/move: ""
spec:
  hosts:
    - address: $WORKER_1_ADDRESS
    - address: $WORKER_2_ADDRESS
    - address: $WORKER_3_ADDRESS
    - address: $WORKER_4_ADDRESS
  sshConfig:
    port: 22
    user: $SSH_USER
    privateKeyRef:
      name: $SSH_PRIVATE_KEY_SECRET_NAME
```

```
      namespace: default
EOF
```

Pre-provisioned Air-gapped FIPS: Define Control Plane Endpoint

Define the control plane endpoint for your cluster as well as the connection mechanism. A control plane needs to have three, five, or seven nodes, so it can remain available if one or more nodes fail. A control plane with one node, is not for production use.

In addition, the control plane needs an endpoint that remains available if nodes fail.

```
                          -------- cp1.example.com:6443
     |
     lb.example.com:6443 ---------- cp2.example.com:6443
     |
     -------- cp3.example.com:6443
```

In this example, the control plane endpoint host is `lb.example.com`, and the control plane endpoint port is `6443`. The control plane nodes are `cp1.example.com`, `cp2.example.com`, and `cp3.example.com`. The port of each API server is `6443`.

Select your Connection Mechanism

A virtual IP is the address that the client uses to connect to the service. A load balancer is the device that distributes the client connections to the backend servers. Before you create a new NKP cluster, choose an external load balancer (LB) or virtual IP.

- **External load balancer**

  It is recommended that an external load balancer be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines that are responding to API requests.

- **Built-in virtual IP**

  If an external load balancer is not available, use the built-in virtual IP. The virtual IP is not a load balancer; it does not distribute request load among the control plane machines. However, if the machine receiving requests does not respond to them, the virtual IP automatically moves to another machine.

Single-Node Control Plane

> **Caution:** Do not use a single-node control plane in a production cluster.

A control plane with one node can use its single node as the endpoint, so you will not require an external load balancer, or a built-in virtual IP. At least one control plane node must always be running. Therefore, to upgrade a cluster with one control plane node, a spare machine must be available in the control plane inventory. This machine is used to provision the new node before the old node is deleted. When the API server endpoints are defined, you can create the cluster using the link in Next Step below.

> **Note:** Modify Control Plane Audit logs settings using the information contained in the page Configure the Control Plane.

Known Limitations

The control plane endpoint port is also used as the API server port on each control plane machine. The default port is 6443. Before you create the cluster, ensure the port is available for use on each control plane machine.

Pre-provisioned Air-gapped FIPS: Creating a Management Cluster

Create a new Pre-provisioned Kubernetes cluster in an air-gapped environment.

About this task

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster.

Before you begin

First you must name your cluster. Then you run the command to deploy it. When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

> **Note:** When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Procedure

1. Give your cluster a unique name suitable for your environment.

   > **Note:** The cluster name may only contain the following characters: a-z, 0-9,. , and -. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

2. Set the environment variable: `export CLUSTER_NAME=<preprovisioned-example>`

What to do next

Create a Kubernetes Cluster

If your cluster is air-gapped or you have a local registry, you must provide additional arguments when creating the cluster. These tell the cluster where to locate the local registry to use by defining the URL.

```
export REGISTRY_URL=<https/http>://<registry-address>:<registry-port>
export REGISTRY_CA=<path to the CA on the bastion>
export REGISTRY_USERNAME=<username>
export REGISTRY_PASSWORD=<password>
```

- `REGISTRY_URL`: the address of an existing registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.
- `REGISTRY_CA`: (optional) the path on the bastion machine to the registry CA. Konvoy will configure the cluster nodes to trust this CA. This value is only needed if the registry is using a self-signed certificate and the AMIs are not already configured to trust this CA.
- `REGISTRY_USERNAME`: optional, set to a user that has pull access to this registry.
- `REGISTRY_PASSWORD`: optional if username is not set.

Before you create a new NKP cluster below, you may choose an external load balancer or virtual IP and use the corresponding `nkp create cluster` command example from that page in the docs from the links below. Other customizations are available, but require different flags during `nkp create cluster` command also. Refer to Pre-provisioned Cluster Creation Customization Choices for more cluster customizations.

When you create a new NKP cluster below, choose an external load balancer (LB) or virtual IP and use the corresponding `nkp create cluster` command.

In a pre-provisioned environment, use the Kubernetes CSI and third party drivers for local volumes and other storage devices in your data center.

> **Note:** NKP uses local static provisioner as the default storage provider for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. Use Kubernetes CSI compatible storage that is suitable for production.

After disabling `localvolumeprovisioner`, you can choose from any of the storage options available for Kubernetes. To make that storage the default storage, use the commands shown in this section of the Kubernetes documentation: Changing the Default Storage Class

> **Important:** If you need to increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster, by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command. See Docker Hub's rate limit.

> **Note:** (Optional) Use a registry mirror. Configure your cluster to use an existing local registry as a mirror when attempting to pull images previously pushed to your registry when defining your infrastructure. Instructions in the expandable Custom Installation section. For registry mirror information, see topics Using a Registry Mirror and Registry Mirror Tools.

The create cluster command below includes the `--self-managed` flag. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing.

This command uses the **default external load balancer (LB)** option (see alternative Step 1 for virtual IP):

```
nkp create cluster preprovisioned --cluster-name ${CLUSTER_NAME} \
  --control-plane-endpoint-host <control plane endpoint host> \
  --control-plane-endpoint-port <control plane endpoint port, if different than 6443> \
  --pre-provisioned-inventory-file preprovisioned_inventory.yaml \
  --ssh-private-key-file <path-to-ssh-private-key> \
  --registry-mirror-url=${REGISTRY_URL} \
  --registry-mirror-cacert=${REGISTRY_CA} \
  --registry-mirror-username=${REGISTRY_USERNAME} \
  --registry-mirror-password=${REGISTRY_PASSWORD} \
  --self-managed
```

> **Note:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

1. **ALTERNATIVE Virtual IP** - if you don't have an external LB, and want to use a VIRTUAL IP provided by `kube-vip`, specify these flags example below:

   ```
   nkp create cluster preprovisioned \
       --cluster-name ${CLUSTER_NAME} \
       --control-plane-endpoint-host 196.168.1.10 \
       --virtual-ip-interface eth1
   ```

2. Use the wait command to monitor the cluster control-plane readiness:

   ```
   kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
   timeout=30m
   ```

   Output:

   ```
   cluster.cluster.x-k8s.io/preprovisioned-example condition met
   ```

   > **Note:** Depending on the cluster size, it will take a few minutes to create.

When the command completes, you will have a running Kubernetes cluster! For bootstrap and custom YAML cluster creation, refer to the Additional Infrastructure Customization section of the documentation for Pre-provisioned: Pre-provisioned Infrastructure

Use this command to get the Kubernetes `kubeconfig` for the new cluster and proceed to installing the NKP Kommander UI:

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

Note: If changing the Calico encapsulation, Nutanix recommends changing it after cluster creation, but before production.

Audit Logs

To modify Control Plane Audit logs settings using the information contained in the page Configure the Control Plane.

Configure Air-gapped MetalLB

Create a MetalLB configmap for your Pre-provisioned Infrastructure.

Choose one of the following two protocols you want to use to announce service IPs. If your environment is not currently equipped with a load balancer, you can use MetalLB. Otherwise, your own load balancer will work and you can continue the installation process with Pre-provisioned: Install Kommander . To use MetalLB, create a MetalLB configMap for your Pre-provisioned infrastructure. MetalLB uses one of two protocols for exposing Kubernetes services:

- Layer 2, with Address Resolution Protocol (ARP)
- Border Gateway Protocol (BGP)

  Select one of the following procedures to create your MetalLB manifest for further editing.

Layer 2 Configuration

Layer 2 mode is the simplest to configure: in many cases, you don't need any protocol-specific configuration, only IP addresses.

Layer 2 mode does not require the IPs to be bound to the network interfaces of your worker nodes. It works by responding to ARP requests on your local network directly, to give the machine's MAC address to clients.

- MetalLB IP address ranges or CIDRs needs to be within the node's primary network subnet.
- MetalLB IP address ranges or CIDRs and node subnet must not conflict with the Kubernetes cluster pod and service subnets.

For example, the following configuration gives MetalLB control over IPs from 192.168.1.240 to 192.168.1.250, and configures Layer 2 mode:

The following values are generic, enter your specific values into the fields where applicable.

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    address-pools:
    - name: default
      protocol: layer2
      addresses:
      - 192.168.1.240-192.168.1.250
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

BGP Configuration

For a basic configuration featuring one BGP router and one IP address range, you need 4 pieces of information:

- The router IP address that MetalLB needs to connect to.
- The router's autonomous systems (AS) number.
- The AS number MetalLB to be used.
- An IP address range expressed as a Classless Inter-Domain Routing (CIDR) prefix.

As an example, if you want to give MetalLB the range 192.168.10.0/24 and AS number 64500, and connect it to a router at 10.0.0.1 with AS number 64501, your configuration will look like:

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    peers:
    - peer-address: 10.0.0.1
      peer-asn: 64501
      my-asn: 64500
    address-pools:
    - name: default
      protocol: bgp
      addresses:
      - 192.168.10.0/24
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

Pre-provisioned Air-gapped FIPS: Install Kommander

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped pre-provisioned environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Ensure you have loaded all the necessary images for your configuration. See: Load the Images into Your Registry: Air-gapped Environments.

- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. Edit the installer file to include configuration overrides for the `rook-ceph-cluster`. NKP's default configuration ships Ceph with PVC based storage which requires your CSI provider to support PVC with type `volumeMode: Block`. As this is not possible with the default local static provisioner, you can install Ceph in host storage mode. You can choose whether Ceph's object storage daemon (osd) pods can consume all or just some of the devices on your nodes. Include one of the following Overrides.

   a. To automatically assign all raw storage devices on all nodes to the Ceph cluster.

   ```
   rook-ceph-cluster:
       enabled: true
       values: |
         cephClusterSpec:
           storage:
             storageClassDeviceSets: []
             useAllDevices: true
             useAllNodes: true
             deviceFilter: "<<value>>"
   ```

   b. To assign specific storage devices on all nodes to the Ceph cluster.

   ```
   rook-ceph-cluster:
       enabled: true
       values: |
         cephClusterSpec:
           storage:
             storageClassDeviceSets: []
             useAllNodes: true
             useAllDevices: false
             deviceFilter: "^sdb."
   ```

   Note: If you want to assign specific devices to specific nodes using the `deviceFilter` option, refer to Specific Nodes and Devices. For general information on the `deviceFilter` value, refer to Storage Selection Settings.

5. If required: Customize your `kommander.yaml`.

   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy and External Load Balancer.

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `dkp-catalog-applications`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Enable NKP Catalog Applications after Installing NKP.

Verifying your Installation and UI Log in
Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   NKP open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use the static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      NKP experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Create Managed Clusters Using the NKP CLI

This topic explains how to continue using the CLI to create managed clusters in an air-gapped Pre-provisioned environment with FIPS rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed which allows it to be a Management cluster or a stand alone cluster. Subsequent new clusters are not self-managed as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects, or install the Kommander component. Those tasks are only done on Management clusters!
>
> To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

   ```
   kubectl get workspace -A
   ```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

   > **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First you must name your cluster. Then you run the command to deploy it.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

   ```
   export CLUSTER_NAME=<preprovisioned-additional>
   ```

Create a Kubernetes Cluster

About this task

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster.

This process creates a self-managed cluster to be used as the Management cluster.

> **Tip:** Before you create a new NKP cluster below, choose an external load balancer (LB) or virtual IP and use the corresponding `NKP create cluster` command.

In a Pre-provisioned environment, use the Kubernetes CSI and third party drivers for local volumes and other storage devices in your data center.

> **Caution:** NKP uses local static provisioner as the default storage provider for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. Use Kubernetes CSI compatible storage that is suitable for production.

After disabling `localvolumeprovisioner`, you can choose from any of the storage options available for Kubernetes. To make that storage the default storage, use the commands shown in this section of the Kubernetes documentation: Change the default StorageClass

For Pre-provisioned environments, you define a set of nodes that already exist. During the cluster creation process, Konvoy Image Builder(KIB) is built into NKP and automatically runs the machine configuration process (which KIB uses to build images for other providers) against the set of nodes that you defined. This results in your pre-existing or pre-provisioned nodes being configured properly.

The following command relies on the pre-provisioned cluster API infrastructure provider to initialize the Kubernetes control plane and worker nodes on the hosts defined in the inventory YAML previously created.

Procedure

1. This command uses the default external load balancer (LB) option.

```
nkp create cluster preprovisioned --cluster-name ${CLUSTER_NAME} \
  --control-plane-endpoint-host <control plane endpoint host> \
  --control-plane-endpoint-port <control plane endpoint port, if different than 6443>
 \
  --pre-provisioned-inventory-file preprovisioned_inventory.yaml \
  --ssh-private-key-file <path-to-ssh-private-key> \
  --registry-mirror-url=${REGISTRY_URL} \
  --registry-mirror-cacert=${REGISTRY_CA} \
  --registry-mirror-username=${REGISTRY_USERNAME} \
  --registry-mirror-password=${REGISTRY_PASSWORD}
```

2. Use the wait command to monitor the cluster control-plane readiness.

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=30m
```

> **Note:** NOTE: Depending on the cluster size, it will take a few minutes to create.

```
cluster.cluster.x-k8s.io/preprovisioned-additional condition met
```

> **Tip:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

   ```
   echo ${MANAGED_CLUSTER_NAME}
   ```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

   ```
   nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
    ${MANAGED_CLUSTER_NAME}.conf
   ```

3. **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

   ```
   kubectl get workspaces -A
   ```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace-namespace>
   ```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

   ```
   kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
   template='{{.data.value}}{{ "\n"}}'
   ```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <your-managed-cluster-name>-kubeconfig
     labels:
       cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
   type: cluster.x-k8s.io/secret
   data:
     value: <value-you-copied-from-secret-above>
   ```

8. Create this secret in the desired workspace.

   ```
   kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
    ${WORKSPACE_NAMESPACE}
   ```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

   ```
   cat << EOF | kubectl apply -f -
   apiVersion: kommander.mesosphere.io/v1beta1
   kind: KommanderCluster
   metadata:
   ```

```
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

    ```
    kubectl get kommanderclusters -A
    ```

    If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

Pre-provisioned with GPU Install

This section provides instructions to install NKP in a Pre-provisioned non-air-gapped environment with GPU.

Ensure Configuration

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Pre-provisioned GPU: Nodepool Secrets and Overrides
Install NVIDIA runfile and place it in the artifacts directory.

About this task

For pre-provisioned environments, NKP has introduced the `nvidia-runfile` flag for Air-gapped Pre-provisioned environments. If the NVIDIA runfile installer has not been downloaded, then retrieve and install the download first by running the following command. The first line in the command below downloads and installs the runfile and the second line places it in the artifacts directory (you must create an `artifacts` directory if it doesn't already exist).

```
curl -O https://download.nvidia.com/XFree86/Linux-x86_64/470.82.01/NVIDIA-Linux-x86_64-470.82.01.run mv NVIDIA-Linux-x86_64-470.82.01.run artifacts
```

> **Note:** NKP supports NVIDIA version is 470.x. For more information, see NVIDIA driver.

Procedure

1. Create the secret that GPU nodepool uses. This secret is populated from the KIB overrides.

   Example output of a file named `overrides/nvidia.yaml`.

   ```
   gpu:
     types:
       - nvidia
   build_name_extra: "-nvidia"
   ```

2. Create a secret on the bootstrap cluster that is populated from the above file. We will name it `${CLUSTER_NAME}-user-overrides`

```
kubectl create secret generic ${CLUSTER_NAME}-user-overrides --from-
file=overrides.yaml=overrides/nvidia.yaml
```

3. Create an inventory and nodepool with the instructions below and use the `${CLUSTER_NAME}-user-overrides` secret.

   a. Create an inventory object that has the same name as the node pool you're creating, and the details of the pre-provisioned machines that you want to add to it. For example, to create a node pool named `gpu-nodepool` an inventory named `gpu-nodepool` must be present in the same namespace.

```
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: ${MY_NODEPOOL_NAME}
spec:
  hosts:
    - address: ${IP_OF_NODE}
  sshConfig:
    port: 22
    user: ${SSH_USERNAME}
    privateKeyRef:
      name: ${NAME_OF_SSH_SECRET}
      namespace: ${NAMESPACE_OF_SSH_SECRET}
```

   b. (Optional) If your pre-provisioned machines have overrides, you must create a secret that includes all of the overrides you want to provide in one file. Create an override secret using the instructions detailed on this page.

   c. Once the `PreprovisionedInventory` object and overrides are created, create a node pool.

```
nkp create nodepool preprovisioned -c ${MY_CLUSTER_NAME} ${MY_NODEPOOL_NAME} --
override-secret-name ${MY_OVERRIDE_SECRET}
```

> **Note:** Advanced users can use a combination of the `--dry-run` and `--output=yaml` or `--output-directory=<existing-directory>` flags to get a complete set of node pool objects to modify locally or store in version control.

> **Note:** For more information regarding this flag or others, see the `nkp create nodepool` section of the documentation for either cluster or nodepool and select your provider.

Pre-provisioned GPU: Define Infrastructure
Define the cluster hosts and infrastructure in a pre-provisioned environment.

About this task

Konvoy needs to know how to access your cluster hosts. This is done using inventory resources. For initial cluster creation, you must define a control-plane and at least one worker pool.

This procedure sets the necessary environment variables.

Procedure

1. Export the following environment variables, ensuring that all control plane and worker nodes are included:

```
export CONTROL_PLANE_1_ADDRESS="<control-plane-address-1>"
export CONTROL_PLANE_2_ADDRESS="<control-plane-address-2>"
export CONTROL_PLANE_3_ADDRESS="<control-plane-address-3>"
```

```
export WORKER_1_ADDRESS="<worker-address-1>"
export WORKER_2_ADDRESS="<worker-address-2>"
export WORKER_3_ADDRESS="<worker-address-3>"
export WORKER_4_ADDRESS="<worker-address-4>"
export SSH_USER="<ssh-user>"
export SSH_PRIVATE_KEY_SECRET_NAME="$CLUSTER_NAME-ssh-key"
```

2. Use the following template to help you define your infrastructure. The environment variables that you set in the previous step automatically replace the variable names when the inventory YAML file is created.

```
cat <<EOF > preprovisioned_inventory.yaml
---
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: $CLUSTER_NAME-control-plane
  namespace: default
  labels:
    cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
    clusterctl.cluster.x-k8s.io/move: ""
spec:
  hosts:
    # Create as many of these as needed to match your infrastructure
    # Note that the command line parameter --control-plane-replicas determines how
 many control plane nodes will actually be used.
    #
    - address: $CONTROL_PLANE_1_ADDRESS
    - address: $CONTROL_PLANE_2_ADDRESS
    - address: $CONTROL_PLANE_3_ADDRESS
  sshConfig:
    port: 22
    # This is the username used to connect to your infrastructure. This user must be
 root or
    # have the ability to use sudo without a password
    user: $SSH_USER
    privateKeyRef:
      # This is the name of the secret you created in the previous step. It must
 exist in the same
      # namespace as this inventory object.
      name: $SSH_PRIVATE_KEY_SECRET_NAME
      namespace: default
---
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: $CLUSTER_NAME-md-0
  namespace: default
  labels:
    cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
    clusterctl.cluster.x-k8s.io/move: ""
spec:
  hosts:
    - address: $WORKER_1_ADDRESS
    - address: $WORKER_2_ADDRESS
    - address: $WORKER_3_ADDRESS
    - address: $WORKER_4_ADDRESS
  sshConfig:
    port: 22
    user: $SSH_USER
    privateKeyRef:
      name: $SSH_PRIVATE_KEY_SECRET_NAME
```

```
      namespace: default
EOF
```

Pre-provisioned GPU: Define Control Plane Endpoint

Define the control plane endpoint for your cluster as well as the connection mechanism. A control plane needs to have three, five, or seven nodes, so it can remain available if one or more nodes fail. A control plane with one node, is not for production use.

In addition, the control plane needs an endpoint that remains available if some nodes fail.

```
                          -------- cp1.example.com:6443
     |
     lb.example.com:6443 ---------- cp2.example.com:6443
     |
     -------- cp3.example.com:6443
```

In this example, the control plane endpoint host is `lb.example.com`, and the control plane endpoint port is `6443`. The control plane nodes are `cp1.example.com`, `cp2.example.com`, and `cp3.example.com`. The port of each API server is `6443`.

Select your Connection Mechanism

A virtual IP is the address that the client uses to connect to the service. A load balancer is the device that distributes the client connections to the backend servers. Before you create a new NKP cluster, choose an external load balancer (LB) or virtual IP.

- **External load balancer**

  It is recommended that an external load balancer be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines that are responding to API requests.

- **Built-in virtual IP**

  If an external load balancer is not available, use the built-in virtual IP. The virtual IP is not a load balancer; it does not distribute request load among the control plane machines. However, if the machine receiving requests does not respond to them, the virtual IP automatically moves to another machine.

Single-Node Control Plane

> **Caution:** Do not use a single-node control plane in a production cluster.

A control plane with one node can use its single node as the endpoint, so you will not require an external load balancer, or a built-in virtual IP. At least one control plane node must always be running. Therefore, to upgrade a cluster with one control plane node, a spare machine must be available in the control plane inventory. This machine is used to provision the new node before the old node is deleted. When the API server endpoints are defined, you can create the cluster using the link in Next Step below.

> **Note:** Modify Control Plane Audit logs settings using the information contained in the page Configure the Control Plane.

Known Limitations

The control plane endpoint port is also used as the API server port on each control plane machine. The default port is 6443. Before you create the cluster, ensure the port is available for use on each control plane machine.

Pre-provisioned GPU: Creating the Management Cluster

Create a new Pre-provisioned Kubernetes cluster in a non-air-gapped environment with the steps below.

About this task

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

If a custom AMI was created using Konvoy Image Builder, the custom `ami` id is printed and written to `packer.pkr.hcl`.

To use the built `ami` with Konvoy, specify it with the `--ami` flag when calling cluster create.

For GPU Steps in Pre-provisioned section of the documentation to use the overrides/nvidia.yaml.

Additional helpful information can be found in the NVIDIA Device Plug-in for Kubernetes instructions and the Installation Guide of Supported Platforms.

Before you begin

First you must name your cluster. Then you run the command to deploy it. When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable: `export CLUSTER_NAME=preprovisioned-example`

What to do next

Create a Kubernetes Cluster

Create a Kubernetes Cluster

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new Pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster. By default, the control-plane Nodes will be created in 3 different zones. However, the default worker Nodes will reside in a single Availability Zone. You may create additional node pools in other Availability Zones with the `nkp create nodepool` command.

Before you create a new NKP cluster below, choose an external load balancer (LB) or virtual IP and use the corresponding `nkp create cluster` command.

In a pre-provisioned environment, use the Kubernetes CSI and third party drivers for local volumes and other storage devices in your data center.

NKP uses local static provisioner as the default storage provider for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. You can use a Kubernetes CSI compatible storage that is suitable for production.

After disabling `localvolumeprovisioner`, you can choose from any of the storage options available for Kubernetes. To make that storage the default storage, use the commands shown in this section of the Kubernetes documentation: Changing the Default Storage Class

For Pre-provisioned environments, you define a set of nodes that already exist. During the cluster creation process, Konvoy Image Builder(KIB) is built into NKP and automatically runs the machine configuration process (which KIB uses to build images for other providers) against the set of nodes that you defined. This results in your pre-existing or pre-provisioned nodes being configured properly.

The following command relies on the pre-provisioned cluster API infrastructure provider to initialize the Kubernetes control plane and worker nodes on the hosts defined in the inventory YAML previously created.

The create cluster command below includes the `--self-managed` flag. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing.

This command uses the default external load balancer (LB) option (see alternative Step 1 for virtual IP):

```
nkp create cluster preprovisioned \
  --cluster-name=${CLUSTER_NAME} \
  --control-plane-endpoint-host <control plane endpoint host> \
  --control-plane-endpoint-port <control plane endpoint port, if different than 6443> \
  --pre-provisioned-inventory-file preprovisioned_inventory.yaml \
  --ssh-private-key-file <path-to-ssh-private-key> \
  --ami <ami> \
  --self-managed
```

> **Note:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

1. **ALTERNATIVE Virtual IP** - if you don't have an external LB, and want to use a VIRTUAL IP provided by `kube-vip`, specify these flags example below:

   ```
   nkp create cluster preprovisioned \
       --cluster-name ${CLUSTER_NAME} \
       --control-plane-endpoint-host 196.168.1.10 \
       --virtual-ip-interface eth1
   ```

2. Create the node pool after cluster creation:

   ```
   nkp create nodepool aws -c ${CLUSTER_NAME} \
   --instance-type p2.xlarge \
   --ami-id=${AMI_ID_FROM_KIB} \
   --replicas=1 ${NODEPOOL_NAME} \
   --kubeconfig=${CLUSTER_NAME}.conf
   ```

3. Use the wait command to monitor the cluster control-plane readiness:

   ```
   kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
   timeout=30m
   ```

   Output:

   ```
   cluster.cluster.x-k8s.io/preprovisioned-example condition met
   ```

   > **Note:** Depending on the cluster size, it will take a few minutes to create.

When the command completes, you will have a running Kubernetes cluster! For bootstrap and custom YAML cluster creation, refer to the Additional Infrastructure Customization section of the documentation for Pre-provisioned: Pre-Provisioned Infrastructure

Use this command to get the Kubernetes `kubeconfig` for the new cluster and proceed to installing the NKP Kommander UI:

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

> **Note:** If changing the Calico encapsulation, Nutanix recommends changing it after cluster creation, but before production.

> **Important:** If you need to increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster, by setting the following flag `--registry-mirror-url=https://`

```
registry-1.docker.io --registry-mirror-username= --registry-mirror-password=
```
on the `nkp create cluster` command.

Audit Logs

To modify Control Plane Audit logs settings using the information contained in the page Configure the Control Plane.

Further Steps

For more customized cluster creation, access the Pre-Provisioned Infrastructure section. That section is for Pre-Provisioned Override Files, custom flags, and more that specify the secret as part of the create cluster command. If these are not specified, the overrides for your nodes will not be applied.

Cluster Verification: If you want to monitor or verify the installation of your clusters, refer to: Verify your Cluster and NKP Installation.

Configure MetalLB with GPU

Create a MetalLB configmap for your Pre-provisioned Infrastructure.

Choose one of the following two protocols you want to use to announce service IPs. If your environment is not currently equipped with a load balancer, you can use MetalLB. Otherwise, your own load balancer will work and you can continue the installation process with Pre-provisioned: Install Kommander . To use MetalLB, create a MetalLB configMap for your Pre-provisioned infrastructure. MetalLB uses one of two protocols for exposing Kubernetes services:

- Layer 2, with Address Resolution Protocol (ARP)
- Border Gateway Protocol (BGP)

  Select one of the following procedures to create your MetalLB manifest for further editing.

Layer 2 Configuration

Layer 2 mode is the simplest to configure: in many cases, you don't need any protocol-specific configuration, only IP addresses.

Layer 2 mode does not require the IPs to be bound to the network interfaces of your worker nodes. It works by responding to ARP requests on your local network directly, to give the machine's MAC address to clients.

- MetalLB IP address ranges or CIDRs needs to be within the node's primary network subnet.
- MetalLB IP address ranges or CIDRs and node subnet must not conflict with the Kubernetes cluster pod and service subnets.

For example, the following configuration gives MetalLB control over IPs from 192.168.1.240 to 192.168.1.250, and configures Layer 2 mode:

The following values are generic, enter your specific values into the fields where applicable.

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    address-pools:
    - name: default
      protocol: layer2
      addresses:
```

```
      - 192.168.1.240-192.168.1.250
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

BGP Configuration

For a basic configuration featuring one BGP router and one IP address range, you need 4 pieces of information:

- The router IP address that MetalLB needs to connect to.
- The router's autonomous systems (AS) number.
- The AS number MetalLB to be used.
- An IP address range expressed as a Classless Inter-Domain Routing (CIDR) prefix.

As an example, if you want to give MetalLB the range 192.168.10.0/24 and AS number 64500, and connect it to a router at 10.0.0.1 with AS number 64501, your configuration will look like:

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    peers:
    - peer-address: 10.0.0.1
      peer-asn: 64501
      my-asn: 64500
    address-pools:
    - name: default
      protocol: bgp
      addresses:
      - 192.168.10.0/24
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

Pre-provisioned GPU: Install Kommander

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped pre-provisioned environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. Edit the installer file to include configuration overrides for the `rook-ceph-cluster`. NKP's default configuration ships Ceph with PersistentVolumeClaim (PVC) based storage which requires your CSI provider to support PVC with type `volumeMode: Block`. As this is not possible with the default local static provisioner, you can install Ceph in host storage mode. You can choose whether Ceph's object storage daemon (osd) pods can consume all or just some of the devices on your nodes. Include one of the following Overrides.

   a. To automatically assign all raw storage devices on all nodes to the Ceph cluster.

   ```
   rook-ceph-cluster:
       enabled: true
       values: |
         cephClusterSpec:
           storage:
             storageClassDeviceSets: []
             useAllDevices: true
             useAllNodes: true
             deviceFilter: "<<value>>"
   ```

   b. To assign specific storage devices on all nodes to the Ceph cluster.

   ```
   rook-ceph-cluster:
       enabled: true
       values: |
         cephClusterSpec:
           storage:
             storageClassDeviceSets: []
             useAllNodes: true
             useAllDevices: false
             deviceFilter: "^sdb."
   ```

   Note: If you want to assign specific devices to specific nodes using the `deviceFilter` option, refer to Specific Nodes and Devices. For general information on the `deviceFilter` value, refer to Storage Selection Settings.

5. If required: Customize your `kommander.yaml`.

   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy and External Load Balancer.

6. Enable NVIDIA platform services in the same `kommander.yaml`file. for GPU resources.

```
apps:
  nvidia-gpu-operator:
   enabled: true
```

7. Append the correct Toolkit version based on your OS.

   a. RHEL 8.6

   If you're using RHEL 8.6 as the base operating system for your GPU enabled nodes, set the `toolkit.version` parameter in your Kommander Installer Configuration file or `<kommander.yaml>` to the following

```
kind: Installation
apps:
  nvidia-gpu-operator:
   enabled: true
   values: |
     toolkit:
       version: v1.14.6-ubi8
```

   b. Ubuntu 18.04 and 20.04

   If you're using Ubuntu 18.04 or 20.04 as the base operating system for your GPU enabled nodes, set the `toolkit.version` parameter in your Kommander Installer Configuration file or `<kommander.yaml>` to the following

```
kind: Installation
apps:
  nvidia-gpu-operator:
   enabled: true
   values: |
     toolkit:
       version: v1.14.6-ubuntu20.04
```

8. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `NKP-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

9. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see the topic Configuring NKP Catalog Applications after Installing NKP.

Verifying your Installation and UI Log in

Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   NKP open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use the static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      NKP experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Create Managed Clusters Using the NKP CLI

This topic explains how to continue using the CLI to create managed clusters in a Pre-provisioned environment with GPU rather than switching to the UI dashboard.

About this task

After initial cluster creation, you can create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed, which allows it to be a Management cluster or a stand-alone cluster. Subsequent new clusters are not self-managed, as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects, or install the Kommander component. Those tasks are only done on Management clusters!

To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

```
kubectl get workspace -A
```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace_namespace>
```

Note: If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to create the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster that can be used as the management cluster.

First, you must name your cluster. Then, you run the command to deploy it.

Note: The cluster name might only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

```
export CLUSTER_NAME=<preprovisioned-additional>
```

Create a Cluster with GPU AMI

Procedure

- If a custom AMI was created using Konvoy Image Builder, use the `--ami` flag. The custom `ami` id is printed and written to `./manifest.json`. To use the built `ami` with Konvoy, specify it with the `--ami` flag when calling cluster create in Step 1 in the next section where you create your Kubernetes cluster.

Create a Kubernetes Cluster

About this task

After you have defined the infrastructure and control plane endpoints, you can proceed to create the cluster by following these steps to create a new pre-provisioned cluster.

This process creates a self-managed cluster that can be used as the management cluster.

> **Tip:** Before you create a new NKP cluster below, choose an external load balancer (LB) or virtual IP and use the corresponding `NKP create cluster` command.

In a Pre-provisioned environment, use the Kubernetes CSI and third-partythird-party drivers for local volumes and other storage devices in your data center.

> **Caution:** NKP uses a local static provisioner as the default storage provider for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. Use Kubernetes CSI compatible storage that is suitable for production.

After turning off `localvolumeprovisioner`, you can choose from any of the storage options available for Kubernetes. To make that storage the default storage, use the commands shown in this section of the Kubernetes documentation: Change the default StorageClass.

For Pre-provisioned environments, you define a set of nodes that already exist. During the cluster creation process, Konvoy Image Builder(KIB) is built into NKP and automatically runs the machine configuration process (which KIB uses to build images for other providers) against the set of nodes that you defined. This results in your pre-existing or pre-provisioned nodes being configured properly.

The following command relies on the pre-provisioned cluster API infrastructure provider to initialize the Kubernetes control plane and worker nodes on the hosts defined in the inventory YAML previously created.

Procedure

1. This command uses the default external load balancer (LB) option.

```
nkp create cluster pre-provisioned --cluster-name ${CLUSTER_NAME} \
  --control-plane-endpoint-host <control plane endpoint host> \
  --control-plane-endpoint-port <control plane endpoint port, if different than 6443>
 \
  --pre-provisioned-inventory-file preprovisioned_inventory.yaml \
  --ssh-private-key-file <path-to-ssh-private-key> \
  --registry-mirror-url=${REGISTRY_URL} \
  --registry-mirror-cacert=${REGISTRY_CA} \
  --registry-mirror-username=${REGISTRY_USERNAME} \
  --registry-mirror-password=${REGISTRY_PASSWORD}
```

2. Use the wait command to monitor the cluster control-plane readiness.

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=30m
```

> **Note:** NOTE: Depending on the cluster size, it will take a few minutes to create.

```
cluster.cluster.x-k8s.io/preprovisioned-additional condition met
```

> **Tip:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created

in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

   ```
   echo ${MANAGED_CLUSTER_NAME}
   ```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

   ```
   nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
    ${MANAGED_CLUSTER_NAME}.conf
   ```

3. **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

   ```
   kubectl get workspaces -A
   ```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace-namespace>
   ```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

   ```
   kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
   template='{{.data.value}}{{ "\n"}}'
   ```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <your-managed-cluster-name>-kubeconfig
     labels:
       cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
   type: cluster.x-k8s.io/secret
   data:
     value: <value-you-copied-from-secret-above>
   ```

8. Create this secret in the desired workspace.

   ```
   kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
    ${WORKSPACE_NAMESPACE}
   ```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

   ```
   cat << EOF | kubectl apply -f -
   apiVersion: kommander.mesosphere.io/v1beta1
   kind: KommanderCluster
   metadata:
     name: ${MANAGED_CLUSTER_NAME}
     namespace: ${WORKSPACE_NAMESPACE}
   spec:
     kubeconfigRef:
       name: ${MANAGED_CLUSTER_NAME}-kubeconfig
     clusterRef:
   ```

```
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

    ```
    kubectl get kommanderclusters -A
    ```

    If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

Pre-provisioned Air-gapped with GPU Install

This section provides instructions to install NKP in a Pre-provisioned air-gapped environment with GPU.

Ensure Configuration

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Note:

- For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

  ```
  tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
  ```

- Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

  For example:

  ```
  export TMPDIR=/path/to/your/directory
  ```

Pre-provisioned Air-gapped with GPU: Configure Environment

In order to create a cluster in a Pre-provisioned Air-gapped environment with GPU, you must first prepare the environment.

Note: If the NVIDIA runfile installer has not been downloaded, then retrieve and install the download first by running the following command. The first line in the command below downloads and installs the runfile, and the second line places it in the artifacts directory.

```
curl -O https://download.nvidia.com/XFree86/Linux-x86_64/470.82.01/NVIDIA-Linux-x86_64-470.82.01.run
mv NVIDIA-Linux-x86_64-470.82.01.run artifacts
```

The instructions below outline how to fulfill the requirements for using pre-provisioned infrastructure in an air-gapped environment. In order to create a cluster, you must first set up the environment with the necessary artifacts. All artifacts for Pre-provisioned Air-gapped need to get onto the bastion host. Artifacts

needed by nodes must be unpacked and distributed on the bastion before other provisioning will work in the absence of an internet connection.

There is an air-gapped bundle available to download. In previous NKP releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts, with the exception of the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- ContainerContainersd tar file

1. Download `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tar file to a local directory:

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.
3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the new NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`. Example command:

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   Note:

   - For FIPS, pass the flag: `--fips`.
   - For Red Hat Enterprise Linux (RHEL) OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`

     Example command:

     ```
     export RHSM_ACTIVATION_KEY="-ci"
     export RHSM_ORG_ID="1232131"
     ```

Setup Process

1. The bootstrap image must be extracted and loaded onto the bastion host.
2. Artifacts must be copied onto cluster hosts for nodes to access.
3. If using GPU, those artifacts must be positioned locally.
4. Registry seeded with images locally.

Load the Bootstrap Image

1. Assuming you have downloaded `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` from the download site mentioned above and extracted the tar file, you will load the bootstrap.
2. Load the bootstrap image on your bastion machine:

   ```
   docker load -i konvoy-bootstrap-image-v2.12.0.tar
   ```

Copy air-gapped artifacts onto cluster hosts

.

Using the Konvoy Image Builder, you can copy the required artifacts onto your cluster hosts.

1. Assuming you have downloaded `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , extract the tar file to a local directory:
2. The Kubernetes image bundle will be located in `kib/artifacts/images` and you will want to verify image and artifacts.

   1. Verify the image bundles exist in `artifacts/images`:

      ```
      $ ls artifacts/images/
      kubernetes-images-1.29.6-d2iq.1.tar kubernetes-images-1.29.6-d2iq.1-fips.tar
      ```

   2. Verify the artifacts for your OS exist in the `artifacts/` directory and export the appropriate variables:

      ```
      $ ls artifacts/
      1.29.6_centos_7_x86_64.tar.gz     1.29.6_redhat_8_x86_64_fips.tar.gz
       containerd-1.6.28-d2iq.1-rhel-8.6-x86_64_fips.tar.gz pip-packages.tar.gz
      1.29.6_centos_7_x86_64_fips.tar.gz  1.29.6_rocky_9_x86_64.tar.gz
       containerd-1.6.28-d2iq.1-rocky-9.0-x86_64.tar.gz
      1.29.6_redhat_7_x86_64.tar.gz     1.29.6_ubuntu_20_x86_64.tar.gz
       containerd-1.6.28-d2iq.1-rocky-9.1-x86_64.tar.gz
      1.29.6_redhat_7_x86_64_fips.tar.gz  containerd-1.6.28-d2iq.1-ubuntu-20.04-
      x86_64.tar.gz
      1.29.6_redhat_8_x86_64.tar.gz
      ```

   3. For example, for RHEL 8.6 you set:

      ```
      export OS_PACKAGES_BUNDLE=1.29.6_redhat_8_x86_64.tar.gz
      export CONTAINERD_BUNDLE=containerd-1.6.28-d2iq.1-rhel-8.6-x86_64.tar.gz
      ```

3. Export the following environment variables, ensuring that all control plane and worker nodes are included:

   ```
   export CONTROL_PLANE_1_ADDRESS="<control-plane-address-1>"
   export CONTROL_PLANE_2_ADDRESS="<control-plane-address-2>"
   export CONTROL_PLANE_3_ADDRESS="<control-plane-address-3>"
   export WORKER_1_ADDRESS="<worker-address-1>"
   export WORKER_2_ADDRESS="<worker-address-2>"
   export WORKER_3_ADDRESS="<worker-address-3>"
   export WORKER_4_ADDRESS="<worker-address-4>"
   export SSH_USER="<ssh-user>"
   export SSH_PRIVATE_KEY_FILE="<private key file>"
   ```

   `SSH_PRIVATE_KEY_FILE` must be either the name of the SSH private key file in your working directory or an absolute path to the file in your user's home directory.
4. Generate an `inventory.yaml` which is automatically picked up by the `konvoy-image upload` in the next step.

   ```
   cat <<EOF > inventory.yaml
   all:
     vars:
       ansible_user: $SSH_USER
       ansible_port: 22
       ansible_ssh_private_key_file: $SSH_PRIVATE_KEY_FILE
     hosts:
       $CONTROL_PLANE_1_ADDRESS:
         ansible_host: $CONTROL_PLANE_1_ADDRESS
       $CONTROL_PLANE_2_ADDRESS:
         ansible_host: $CONTROL_PLANE_2_ADDRESS
       $CONTROL_PLANE_3_ADDRESS:
         ansible_host: $CONTROL_PLANE_3_ADDRESS
       $WORKER_1_ADDRESS:
         ansible_host: $WORKER_1_ADDRESS
       $WORKER_2_ADDRESS:
         ansible_host: $WORKER_2_ADDRESS
       $WORKER_3_ADDRESS:
   ```

```
      ansible_host: $WORKER_3_ADDRESS
    $WORKER_4_ADDRESS:
      ansible_host: $WORKER_4_ADDRESS
EOF
```

5. Upload the artifacts onto cluster hosts with the following command:

```
konvoy-image upload artifacts --inventory-file=gpu_inventory.yaml \
              --container-images-dir=./artifacts/images/ \
              --os-packages-bundle=./artifacts/$OS_PACKAGES_BUNDLE \
              --containerd-bundle=artifacts/$CONTAINERD_BUNDLE \
              --pip-packages-bundle=./artifacts/pip-packages.tar.gz \
              --nvidia-runfile=./artifacts/NVIDIA-Linux-x86_64-470.82.01.run
```

   The `konvoy-image upload artifacts` command copies all OS packages and other artifacts onto each of the machines in your inventory. When you create the cluster, the provisioning process connects to each node and runs commands to install those artifacts and consequently Kubernetes running.. KIB uses variable overrides to specify base image and container images to use in your new machine image. The variable overrides files for NVIDIA and FIPS can be ignored unless adding an overlay feature. Use the `--overrides overrides/fips.yaml,overrides/offline-fips.yaml` flag with manifests located in the overrides directory

Pre-provisioned Air-gapped GPU: Load the Registry

Before creating an air-gapped Kubernetes cluster, you need to load the required images in a local registry for the Konvoy component.

About this task

The complete NKP air-gapped bundle is needed for an air-gapped environment but can also be used in a non-air-gapped environment. The bundle contains all the NKP components needed for an air-gapped environment installation and also for using a local registry in a non-air-gapped environment.

> **Note:** If you do not already have a local registry set up, see the Local Registry Tools page for more information.

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images, is required. This registry must be accessible from both the bastion machine and either the AWS EC2 instances (if deploying to AWS) or other machines that will be created for the Kubernetes cluster.

Procedure

1. If not already done in prerequisites, download the air-gapped bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tar file to a local directory.

```
tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
```

2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. EX: For the bootstrap cluster, change your directory to the `nkp-<version>` directory, similar to the example below, depending on your current location

```
cd nkp-v2.12.0
```

3. Set an environment variable with your registry address and any other needed variables using this command.

```
export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
export REGISTRY_USERNAME=<username>
export REGISTRY_PASSWORD=<password>
export REGISTRY_CA=<path to the cacert file on the bastion>
```

4. Execute the following command to load the air-gapped image bundle into your private registry using any of the relevant flags to apply the variables above.

```
nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

> **Note:** It might take some time to push all the images to your image registry, depending on the network performance between the machine you are running the script on and the registry.

> **Important:** To increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

5. Load the Kommander component images to your private registry using the command.

```
nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

Optional: This step is required only if you have an Ultimate license.

For NKP Catalog Applications available with the Ultimate license, perform this image load by running the following command to load the `dkp-catalog-applications` image bundle into your private registry:

```
nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

Pre-provisioned GPU: Nodepool Secrets and Overrides
Install NVIDIA runfile and place it in the artifacts directory.

About this task

For pre-provisioned environments, NKP has introduced the `nvidia-runfile` flag for Air-gapped Pre-provisioned environments. If the NVIDIA runfile installer has not been downloaded, then retrieve and install the download first by running the following command. The first line in the command below downloads and installs the runfile and the second line places it in the artifacts directory (you must create an `artifacts` directory if it doesn't already exist).

```
curl -O https://download.nvidia.com/XFree86/Linux-x86_64/470.82.01/NVIDIA-Linux-x86_64-470.82.01.run mv NVIDIA-Linux-x86_64-470.82.01.run artifacts
```

> **Note:** NKP supports NVIDIA driver version is 470.x. For more information, see NVIDIA driver.

Procedure

1. Create the secret that the GPU node pool uses. This secret is populated from the KIB overrides.

   Example output of a file named `overrides/nvidia.yaml`.

```
gpu:
  types:
    - nvidia
build_name_extra: "-nvidia"
```

2. Create a secret on the bootstrap cluster that is populated from the above file. We will name it `${CLUSTER_NAME}-user-overrides`

```
kubectl create secret generic ${CLUSTER_NAME}-user-overrides --from-
file=overrides.yaml=overrides/nvidia.yaml
```

3. Create an inventory and nodepool with the instructions below and use the `${CLUSTER_NAME}-user-overrides` secret.

   a. Create an inventory object that has the same name as the node pool you're creating and the details of the pre-provisioned machines that you want to add to it. For example, to create a node pool named `gpu-nodepool` an inventory named `gpu-nodepool` must be present in the same namespace.

```
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: ${MY_NODEPOOL_NAME}
spec:
  hosts:
    - address: ${IP_OF_NODE}
  sshConfig:
    port: 22
    user: ${SSH_USERNAME}
    privateKeyRef:
      name: ${NAME_OF_SSH_SECRET}
      namespace: ${NAMESPACE_OF_SSH_SECRET}
```

   b. (Optional) If your pre-provisioned machines have overrides, you must create a secret that includes all of the overrides you want to provide in one file. Create an override secret using the instructions detailed on this page.

   c. Once the `PreprovisionedInventory` object and overrides are created, create a node pool.

```
nkp create nodepool preprovisioned -c ${MY_CLUSTER_NAME} ${MY_NODEPOOL_NAME} --
override-secret-name ${MY_OVERRIDE_SECRET}
```

> **Note:** Advanced users can use a combination of the `--dry-run` and `--output=yaml` or `--output-directory=<existing-directory>` flags to get a complete set of node pool objects to modify locally or store in version control.

> **Note:** For more information regarding this flag or others, see the `nkp create node pool` section of the documentation for either cluster or nodepool and select your provider.

Pre-provisioned Air-gapped GPU: Define Infrastructure
Define the cluster hosts and infrastructure in a pre-provisioned environment.

About this task

Konvoy needs to know how to access your cluster hosts. This is done using inventory resources. For initial cluster creation, you must define a control plane and at least one worker pool.

This procedure sets the necessary environment variables.

Procedure

1. Export the following environment variables, ensuring that all control plane and worker nodes are included:

```
export CONTROL_PLANE_1_ADDRESS="<control-plane-address-1>"
export CONTROL_PLANE_2_ADDRESS="<control-plane-address-2>"
export CONTROL_PLANE_3_ADDRESS="<control-plane-address-3>"
```

```
export WORKER_1_ADDRESS="<worker-address-1>"
export WORKER_2_ADDRESS="<worker-address-2>"
export WORKER_3_ADDRESS="<worker-address-3>"
export WORKER_4_ADDRESS="<worker-address-4>"
export SSH_USER="<ssh-user>"
export SSH_PRIVATE_KEY_SECRET_NAME="$CLUSTER_NAME-ssh-key"
```

2. Use the following template to help you define your infrastructure. The environment variables that you set in the previous step automatically replace the variable names when the inventory YAML file is created.

```
cat <<EOF > preprovisioned_inventory.yaml
---
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: $CLUSTER_NAME-control-plane
  namespace: default
  labels:
    cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
    clusterctl.cluster.x-k8s.io/move: ""
spec:
  hosts:
    # Create as many of these as needed to match your infrastructure
    # Note that the command-line parameter—-control-plane-replicas determines how
 many control plane nodes will actually be used.
    #
    - address: $CONTROL_PLANE_1_ADDRESS
    - address: $CONTROL_PLANE_2_ADDRESS
    - address: $CONTROL_PLANE_3_ADDRESS
  sshConfig:
    port: 22
    # This is the username used to connect to your infrastructure. This user must be
 root or
    # have the ability to use sudo without a password
    user: $SSH_USER
    privateKeyRef:
      # This is the name of the secret you created in the previous step. It must
 exist in the same
      # namespace as this inventory object.
      name: $SSH_PRIVATE_KEY_SECRET_NAME
      namespace: default
---
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: $CLUSTER_NAME-md-0
  namespace: default
  labels:
    cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
    clusterctl.cluster.x-k8s.io/move: ""
spec:
  hosts:
    - address: $WORKER_1_ADDRESS
    - address: $WORKER_2_ADDRESS
    - address: $WORKER_3_ADDRESS
    - address: $WORKER_4_ADDRESS
  sshConfig:
    port: 22
    user: $SSH_USER
    privateKeyRef:
      name: $SSH_PRIVATE_KEY_SECRET_NAME
```

```
      namespace: default
EOF
```

Pre-provisioned Air-gapped GPU: Define Control Plane Endpoint

Define the control plane endpoint for your cluster and the connection mechanism. A control plane needs to have three, five, or seven nodes so it can remain available if one or more nodes fail. A control plane with one node is not for production use.

In addition, the control plane needs an endpoint that remains available if some nodes fail.

```
                          -------- cp1.example.com:6443
     |
     lb.example.com:6443 ---------- cp2.example.com:6443
     |
     -------- cp3.example.com:6443
```

In this example, the control plane endpoint host is `lb.example.com`, and the control plane endpoint port is `6443`. The control plane nodes are `cp1.example.com`, `cp2.example.com`, and `cp3.example.com`. The port of each API server is `6443`.

Select your Connection Mechanism

A virtual IP is the address that the client uses to connect to the service. A load balancer is a device that distributes the client connections to the backend servers. Before you create a new NKP cluster, choose an external load balancer (LB) or virtual IP.

- **External load balancer**

  It is recommended that an external load balancer be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines that are responding to API requests.

- **Built-in virtual IP**

  If an external load balancer is not available, use the built-in virtual IP. The virtual IP is not a load balancer; it does not distribute request load among the control plane machines. However, if the machine receiving requests does not respond to them, the virtual IP automatically moves to another machine.

Single-Node Control Plane

> **Caution:** Do not use a single-node control plane in a production cluster.

A control plane with one node can use its single node as the endpoint, so you will not require an external load balancer, or a built-in virtual IP. At least one control plane node must always be running. Therefore, to upgrade a cluster with one control plane node, a spare machine must be available in the control plane inventory. This machine is used to provision the new node before the old node is deleted. When the API server endpoints are defined, you can create the cluster using the link in the Next Step below.

> **Note:** Modify Control Plane Audit logs settings using the information contained in the page Configure the Control Plane.

Known Limitations

The control plane endpoint port is also used as the API server port on each control plane machine. The default port is 6443. Before you create the cluster, ensure the port is available for use on each control plane machine.

Pre-provisioned Air-gapped GPU: Creating a Management Cluster

Create a new Pre-provisioned Kubernetes cluster in an air-gapped environment.

About this task

After you have defined the infrastructure and control plane endpoints, you can proceed to create the cluster by following these steps to create a new pre-provisioned cluster.

Before you begin

First, you must name your cluster. Then yo,u run the command to deploy it. When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Procedure

1. Give your cluster a unique name suitable for your environment.

   > **Note:** The cluster name might only contain the following characters: a-z, 0-9, , and -. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

2. Set the environment variable: `export CLUSTER_NAME=<preprovisioned-example>`

What to do next

Create a Kubernetes Cluster

- the

  > **Note:** a

Turning offthird-partymight.

```
nkp create cluster pre-provisioned --cluster-name ${CLUSTER_NAME} \
  --control-plane-endpoint-host <control plane endpoint host> \
  --control-plane-endpoint-port <control plane endpoint port, if different than 6443> \
  --pre-provisioned-inventory-file preprovisioned_inventory.yaml \
  --ssh-private-key-file <path-to-ssh-private-key> \
  --registry-mirror-url=${REGISTRY_URL} \
  --registry-mirror-cacert=${REGISTRY_CA} \
  --registry-mirror-username=${REGISTRY_USERNAME} \
  --registry-mirror-password=${REGISTRY_PASSWORD} \
  --self-managed
```

> **Note:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

1. **ALTERNATIVE Virtual IP** - if you don't have an external LB, and want to use a VIRTUAL IP provided by `kube-vip`, specify these flags example below:

   ```
   nkp create cluster preprovisioned \
       --cluster-name ${CLUSTER_NAME} \
       --control-plane-endpoint-host 196.168.1.10 \
       --virtual-ip-interface eth1
   ```

2. Use the wait command to monitor the cluster control-plane readiness:

   ```
   kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
   timeout=30m
   ```

   Output:

   ```
   cluster.cluster.x-k8s.io/preprovisioned-example condition met
   ```

   > **Note:** Depending on the cluster size, it will take a few minutes to create.

When the command completes, you will have a running Kubernetes cluster! For bootstrap and custom YAML cluster creation, refer to the Additional Infrastructure Customization section of the documentation for Pre-provisioned: Pre-provisioned Infrastructure

Use this command to get the Kubernetes `kubeconfig` for the new cluster and proceed to installing the NKP Kommander UI:

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

> **Note:** If changing the Calico encapsulation, Nutanix recommends changing it after cluster creation, but before production.

Audit Logs

To modify Control Plane Audit logs settings using the information contained in the page Configure the Control Plane.

Configure Air-gapped MetalLB with GPU
Create a MetalLB configmap for your Pre-provisioned Infrastructure.

Choose one of the following two protocols you want to use to announce service IPs. If your environment is not currently equipped with a load balancer, you can use MetalLB. Otherwise, your own load balancer will work, and you can continue the installation process with Pre-provisioned: Install Kommander. To use MetalLB, create a MetalLB configMap for your Pre-provisioned infrastructure. MetalLB uses one of two protocols for exposing Kubernetes services:

- Layer 2, with Address Resolution Protocol (ARP)
- Border Gateway Protocol (BGP)

  Select one of the following procedures to create your MetalLB manifest for further editing.

Layer 2 Configuration

Layer 2 mode is the simplest to configure: in many cases, you don't need any protocol-specific configuration, only IP addresses.

Layer 2 mode does not require the IPs to be bound to the network interfaces of your worker nodes. It works by responding to ARP requests on your local network directly and give the machine's MAC address to clients.

- MetalLB IP address ranges or CIDRs needs to be within the node's primary network subnet.
- MetalLB IP address ranges or CIDRs and node subnets must not conflict with the Kubernetes cluster pod and service subnets.

For example, the following configuration gives MetalLB control over IPs from 192.168.1.240 to 192.168.1.250, and configures Layer 2 mode:

The following values are generic; enter your specific values into the fields where applicable.

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    address-pools:
    - name: default
      protocol: layer2
      addresses:
```

```
      - 192.168.1.240-192.168.1.250
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

BGP Configuration

For a basic configuration featuring one BGP router and one IP address range, you need four pieces of information:

- The router IP address that MetalLB needs to connect to.
- The router's autonomous systems (AS) number.
- The AS number MetalLB to be used.
- An IP address range is expressed as a Classless Inter-Domain Routing (CIDR) prefix.

As an example, if you want to give MetalLB the range 192.168.10.0/24 and AS number 64500, and connect it to a router at 10.0.0.1 with AS number 64501, your configuration will look like this:

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    peers:
    - peer-address: 10.0.0.1
      peer-asn: 64501
      my-asn: 64500
    address-pools:
    - name: default
      protocol: bgp
      addresses:
      - 192.168.10.0/24
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

Pre-provisioned Air-gapped GPU: Install Kommander

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped pre-provisioned environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Ensure you have loaded all the necessary images for your configuration. See: Load the Images into Your Registry: Air-gapped Environments.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. Edit the installer file to include configuration overrides for the `rook-ceph-cluster`. NKP's default configuration ships Ceph with PersistentVolumeClaim (PVC) based storage which requires your CSI provider to support PVC with type `volumeMode: Block`. As this is not possible with the default local static provisioner, you can install Ceph in host storage mode. You can choose whether Ceph's object storage daemon (osd) pods can consume all or just some of the devices on your nodes. Include one of the following Overrides.

   a. To automatically assign all raw storage devices on all nodes to the Ceph cluster.

   ```
   rook-ceph-cluster:
       enabled: true
       values: |
         cephClusterSpec:
           storage:
             storageClassDeviceSets: []
             useAllDevices: true
             useAllNodes: true
             deviceFilter: "<<value>>"
   ```

   b. To assign specific storage devices on all nodes to the Ceph cluster.

   ```
   rook-ceph-cluster:
       enabled: true
       values: |
         cephClusterSpec:
           storage:
             storageClassDeviceSets: []
             useAllNodes: true
             useAllDevices: false
   ```

```
          deviceFilter: "^sdb."
```

Note: If you want to assign specific devices to specific nodes using the `deviceFilter` option, refer to Specific Nodes and Devices. For general information on the `deviceFilter` value, refer to Storage Selection Settings.

5. If required: Customize your `kommander.yaml`.

   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy and External Load Balancer.

6. Enable NVIDIA platform services in the same `kommander.yaml`file. for GPU resources.

```
apps:
  nvidia-gpu-operator:
   enabled: true
```

7. Append the correct Toolkit version based on your OS.

   a. RHEL 8.6

   If you're using RHEL 8.6 as the base operating system for your GPU enabled nodes, set the `toolkit.version` parameter in your Kommander Installer Configuration file or `<kommander.yaml>` to the following

```
kind: Installation
apps:
  nvidia-gpu-operator:
   enabled: true
   values: |
     toolkit:
       version: v1.14.6-ubi8
```

   b. Ubuntu 18.04 and 20.04

   If you're using Ubuntu 18.04 or 20.04 as the base operating system for your GPU enabled nodes, set the `toolkit.version` parameter in your Kommander Installer Configuration file or `<kommander.yaml>` to the following

```
kind: Installation
apps:
  nvidia-gpu-operator:
   enabled: true
   values: |
     toolkit:
       version: v1.14.6-ubuntu20.04
```

8. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `NKP-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
```

```
      ref:
        tag: v2.12.0
```

9. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see the topic Configuring NKP Catalog Applications after Installing NKP.

Verifying your Installation and UI Log in

Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the command `kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m`.

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using the command `NKP open dashboard --kubeconfig=${CLUSTER_NAME}.conf`.
2. Retrieve your credentials at any time using the command `kubectl -n kommander get secret NKP-credentials -o go-template='Username: {{.data.username|base64decode}} {{ "\n"}}Password: {{.data.password|base64decode}}{{ "\n"}}'`.
3. Retrieve the URL used for accessing the UI using the command `kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/dashboard{{ "\n"}}'`
   Only use the static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the command `NKP experimental rotate dashboard-password`.
      The example output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Create Managed Clusters Using the NKP CLI

This topic explains how to continue using the CLI to create managed clusters in an air-gapped Pre-provisioned environment with GPU rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed which allows it to be a Management cluster or a stand alone cluster. Subsequent new clusters are not self-managed as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects, or install the Kommander component. Those tasks are only done on Management clusters!
>
> To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

   ```
   kubectl get workspace -A
   ```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

   Note: If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First you must name your cluster. Then you run the command to deploy it.

Note: The cluster name might only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable using the command `export CLUSTER_NAME=<preprovisioned-additional>`

Create a Cluster with GPU AMI

Procedure

- If a custom AMI was created using Konvoy Image Builder, use the `--ami` flag. The custom `ami` id is printed and written to `./manifest.json`. To use the built `ami` with Konvoy, specify it with the `--ami` flag when calling cluster create in Step 1 in the next section where you create your Kubernetes cluster.

Create a Kubernetes Cluster

About this task

After you have defined the infrastructure and control plane endpoints, you can proceed to create the cluster by following these steps to create a new pre-provisioned cluster.

This process creates a self-managed cluster that can be used as the managementthird-party cluster.

Tip: Before you create a new NKP cluster below, choose an external load balancer (LB) or virtual IP and use the corresponding `NKP create cluster` command.

In a Pre-provisioned environment, use the Kubernetes CSI and third party drivers for local volumes and other storage devices in your data center.

> **Caution:** NKP uses local static provisioners as the Default Storage Providers on page 34 for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. Use Kubernetes CSI compatible storage that is suitable for production.

After disabling `localvolumeprovisioner`, you can choose from any of the storage options available for Kubernetes. To make that storage the default storage, use the commands shown in this section of the Kubernetes documentation: Change or Manage Multiple StorageClasses on page 35

For Pre-provisioned environments, you define a set of nodes that already exist. During the cluster creation process, Konvoy Image Builder(KIB) is built into NKP and automatically runs the machine configuration process (which KIB uses to build images for other providers) against the set of nodes that you defined. This results in your pre-existing or pre-provisioned nodes being configured properly.

The following command relies on the pre-provisioned cluster API infrastructure provider to initialize the Kubernetes control plane and worker nodes on the hosts defined in the inventory YAML previously created.

Procedure

1. This command uses the default external load balancer (LB) option.

```
nkp create cluster pre-provisioned \
  --cluster-name ${CLUSTER_NAME} \
  --control-plane-endpoint-host <control plane endpoint host> \
  --control-plane-endpoint-port <control plane endpoint port, if different than 6443>
 \
  --pre-provisioned-inventory-file preprovisioned_inventory.yaml \
  --ssh-private-key-file <path-to-ssh-private-key> \
  --registry-mirror-url=${_REGISTRY_URL} \
  --registry-mirror-cacert=${_REGISTRY_CA} \
  --registry-mirror-username=${_REGISTRY_USERNAME} \
  --registry-mirror-password=${_REGISTRY_PASSWORD}
```

2. Use the wait command to monitor the cluster control-plane readiness.

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=30m
```

> **Note:** NOTE: Depending on the cluster size, it will take a few minutes to create.

```
cluster.cluster.x-k8s.io/preprovisioned-additional condition met
```

> **Tip:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

```
echo ${MANAGED_CLUSTER_NAME}
```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

```
nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
 ${MANAGED_CLUSTER_NAME}.conf
```

3. **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to the workspace through earlier UI, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

```
kubectl get workspaces -A
```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace-namespace>
```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

```
kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
template='{{.data.value}}{{ "\n"}}'
```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

```
apiVersion: v1
kind: Secret
metadata:
  name: <your-managed-cluster-name>-kubeconfig
  labels:
    cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
type: cluster.x-k8s.io/secret
data:
  value: <value-you-copied-from-secret-above>
```

8. Create this secret in the desired workspace.

```
kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
 ${WORKSPACE_NAMESPACE}
```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
```

```
EOF
```

10. You can now view this cluster in your Workspace in the UI, and you can confirm its status by running the command below. It might take a few minutes to reach "Joined" status.

    ```
    kubectl get kommanderclusters -A
    ```

    If you have several Pro Clusters and want to turn one of them into a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

AWS Installation Options

For an environment that is on the AWS Infrastructure, installation options based on those environment variables are provided for you in this location.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operative in the most common scenarios.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

> **Note:** Additional Resource Information specific to AWS is below.

- **Control Plane Nodes** - NKP on AWS defaults to deploying an `m5.xlarge` instance with an 80GiB root volume for control plane nodes, which meets the above resource requirements.
- **Worker Nodes** - NKP on AWS defaults to deploying a`m5.2xlarge` instance with an 80GiB root volume for worker nodes, which meets the above resource requirements.

AWS Installation

Installation instructions for installing NKP in an AWS non-air-gapped environment.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

AWS Prerequisites

Before you begin using Konvoy with AWS, you must:

1. Follow the steps to create permissions and roles on the Minimal Permissions and Role to Create Clusters page.
2. Create Cluster IAM Policies and Roles
3. Export the AWS region where you want to deploy the cluster:

   ```
   export AWS_REGION=us-west-2
   ```

4. Export the AWS profile with the credentials you want to use to create the Kubernetes cluster:

```
export AWS_PROFILE=<profile>
```

If using AWS ECR as your local private registry, more information can be found on the Registry Mirror Tools page.

To deploy a cluster with a custom image in a region where CAPI images are not provided, you need to use Konvoy Image Builder to create your own image for the region.

> **Note:** For multi-tenancy, every tenant needs to be in a different AWS account to ensure they are truly independent of other tenants in order to enforce security.

AWS: Creating an Image
Learn how to build a custom AMI for use with NKP.

About this task

This procedure describes how to use the Konvoy Image Builder (KIB) to create a Cluster API compliant Amazon Machine Image (AMI). KIB uses variable overrides to specify base image and container images to use in your new AMI.

AMI images contain configuration information and software to create a specific, pre-configured, operating environment. For example, you can create an AMI image of your current computer system settings and software. The AMI image can then be replicated and distributed, creating your computer system for other users. You can use overrides files to customize some of the components installed on your machine image. For example, having the FIPS versions of the Kubernetes components installed by KIB.

In previous NKP releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder. Explore the Customize your Image topic for more options about overrides.

The **prerequisites** to use Konvoy Image Builder are:

Procedure

1. Download the KIB bundle for your version of NKP prefixed with `konvoy-image-bundle` for your OS.
2. Check the Supported Infrastructure Operating Systems.
3. Check the Supported Kubernetes Version for your Provider.
4. Create a working registry.
   - » Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
   - » Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
5. Ensure you have met the minimal set of permissions from the AWS Image Builder Book.
6. Minimal IAM permissions to create an image for an AWS account using Konvoy Image Builder. For more information, see Creating Minimal IAM Permissions for KIB on page 1087.

Extract the KIB Bundle

About this task

If not done previously during Konvoy Image Builder download in Prerequisites, extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION` folder. Otherwise, proceed to Build the Image below.

In previous NKP releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts with the exception of the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Containerd tarball

Procedure

1. Downloading NKP on page 16`nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.

3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the new NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`. Example command.

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   Note:

   - For FIPS, pass the flag: `--fips`
   - **Note:** For RHEL OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`. Example command:

     ```
     export RHSM_ACTIVATION_KEY="-ci"
     export RHSM_ORG_ID="1232131"
     ```

4. Follow the instructions below to build an AMI.

   Note: The `konvoy-image` binary and all supporting folders are also extracted. When run, `konvoy-image` bind mounts the current working directory (`${PWD}`) into the container to be used.

   Set the environment variables for AWS access. The following variables must be set using your credentials including required IAM:

   ```
   export AWS_ACCESS_KEY_ID
   export AWS_SECRET_ACCESS_KEY
   export AWS_DEFAULT_REGION
   ```

   If you have an override file to configure specific attributes of your AMI file, add it. Instructions for customizing an override file are found on this page: Image Overrides

Build the Image

About this task

Depending on which version of NKP you are running, steps and flags will be different. To deploy in a region where CAPI images are not provided, you need to use KIB to create your own image for the region. For a list of supported AWS regions, refer to the Published AMI information from AWS.

Procedure

Run the `konvoy-image` command to build and validate the image.

```
konvoy-image build aws images/ami/rhel-86.yaml
```

a. By default, it builds in the `us-west-2` region. to specify another region set the `--region` flag as shown in the command below.

```
konvoy-image build aws --region us-east-1 images/ami/rhel-86.yaml
```

> **Note:** Ensure you have named the correct AMI image YAML file for your OS in the `konvoy-image build` command.

What to do next

After KIB provisions the image successfully, the `ami`

id is printed and written to the `packer.pkr.hcl` (Packer config) file. This file has an `artifact_id` field whose value provides the name of the AMI ID as shown in the example below. That is the `ami` you use in the `NKP create cluster` command.


```
{
  "builds": [
    {
      "name": "kib_image",
      "builder_type": "amazon-ebs",
      "build_time": 1698086886,
      "files": null,
      "artifact_id": "us-west-2:ami-04b8dfef8bd33a016",
      "packer_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05",
      "custom_data": {
        "containerd_version": "",
        "distribution": "RHEL",
        "distribution_version": "8.6",
        "kubernetes_cni_version": "",
        "kubernetes_version": "1.26.6"
      }
    }
  ],
  "last_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05"
}
```


What to do next

1. To use a custom AMI when creating your cluster, you must create that AMI using KIB first. Then perform the export and name the custom AMI for use in the command `nkp create cluster`:

   ```
   export AWS_AMI_ID=ami-<ami-id-here>
   ```

   > **Note:** Inside the sections for either Non-air-gapped or Air-gapped cluster creation, you will find the instructions for how to apply custom images.

Related Information

Procedure

- To use a local registry even in a non-air-gapped environment, download and extract the bundle. Downloading NKP on page 16 the Complete NKP Air-gapped Bundle for this release (that is. `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`) to load registry
- To view the complete set of instructions, see Load the Registry.

AWS: Creating the Management Cluster
Create an AWS Management Cluster in a non-air-gapped environment.

About this task

Use this procedure to create a self-managed **Management cluster** with NKP. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing.

If you use these instructions to create a cluster on AWS using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes.

NKP uses AWS CSI as the default storage provider. You can use a Kubernetes CSI compatible storage solution that is suitable for production. See the Kubernetes documentation called Changing the Default Storage Class for more information.

In previous NKP releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder.

Before you begin

First you must name your cluster.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable:

```
export CLUSTER_NAME=<aws-example>
```

3. There are two approaches to supplying the ID of your AMI. Either provide the ID of the AMI or provide a way for NKP to discover the AMI using location, format and OS information.

   a. Option One - Provide the ID of your AMI.
   Use the example command below leaving the existing flag that provides the AMI ID: `--ami AMI_ID`

   b. Option Two - Provide a path for your AMI with the information required for image discover.

   - Where the AMI is published using your AWS Account ID: `--ami-owner AWS_ACCOUNT_ID`
   - The format or string used to search for matching AMIs and ensure it references the Kubernetes version plus the base OS name: `--ami-base-os ubuntu-20.04`
   - The base OS information: `--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*'`

   Note:

   - The AMI must be created with Konvoy Image Builder in order to use the registry mirror feature.

     ```
     export AWS_AMI_ID=<ami-...>
     ```

   - (Optional) Registry Mirror - Configure your cluster to use an existing local registry as a mirror when attempting to pull images. Below is an AWS ECR example where `REGISTRY_URL`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.:

     ```
     export REGISTRY_URL=<ecr-registry-URI>
     ```

4. Run this command to create your Kubernetes cluster by providing the image ID and using any relevant flags.

   ```
   nkp create cluster aws \
   --cluster-name=${CLUSTER_NAME} \
   --ami=${AWS_AMI_ID} \
   --additional-tags=owner=$(whoami) \
   --with-aws-bootstrap-credentials=true \
   --self-managed
   ```

   If providing the AMI path, use these flags in place of `AWS_AMI_ID`:

   ```
   --ami-owner AWS_ACCOUNT_ID \
   --ami-base-os ubuntu-20.04 \
   --ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*' \
   ```

   » Additional cluster creation flags based on your environment:

   » Optional Registry flag: `--registry-mirror-url=${REGISTRY_URL}`

   » Flatcar OS flag to instruct the bootstrap cluster to make changes related to the installation paths: `--os-hint flatcar`

   » HTTP or HTTPS flags if you use proxies: `--http-proxy`, `--https-proxy`, and `--no-proxy`

AWS: Install Kommander
This section provides installation instructions for the Kommander component of NKP in a non-air-gapped AWS environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

```
export CLUSTER_NAME=<your-management-cluster-name>
```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

```
nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
```

3. Create a configuration file for the deployment.

```
nkp install kommander --init > kommander.yaml
```

4. If required: Customize your `kommander.yaml`.

   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy and External Load Balancer.

5. Only required if your cluster uses a custom AWS VPC and requires an internal load-balancer, set the traefik annotation to create an internal-facing ELB.

```
apps:
  traefik:
    enabled: true
    values: |
      service:
        annotations:
          service.beta.kubernetes.io/aws-load-balancer-internal: "true"
```

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `NKP-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Configuring NKP Catalog Applications after Installing NKP.

AWS: Verifying your Installation and UI Log in
Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the installation command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      nkp experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Day 2 Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of NKP. The Day 2 section allows you to manage cluster operations and their application workloads to optimize your organization's productivity.

- Continue to the NKP Dashboard.

AWS: Creating Managed Clusters Using the NKP CLI

This topic explains how to continue using the CLI to create managed clusters rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed which allows it to be a Management cluster or a stand alone cluster. Subsequent new clusters are not self-managed as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects, or install the Kommander component. Those tasks are only done on Management clusters!
>
> Your new managed cluster needs to be part of a workspace under a management cluster. To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

   ```
   kubectl get workspace -A
   ```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

   > **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First you must name your cluster. Then you run the command to deploy it.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

```
export MANAGED_CLUSTER_NAME=<aws-additional>
```

Create a Kubernetes Cluster

About this task

The below instructions tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, it will be created in the `default` workspace, and you need to take additional steps to attach to a workspace later. For instructions on how to do this, see Attach a Kubernetes Cluster.

Procedure

Execute this command to create your additional Kubernetes cluster using any relevant flags. This will create a new non-self-managed cluster that can be managed by management cluster you created in the previous section.

```
nkp create cluster aws \
--cluster-name=${MANAGED_CLUSTER_NAME} \
--additional-tags=owner=$(whoami) \
--kubeconfig=<management-cluster-kubeconfig-path> \
--namespace ${WORKSPACE_NAMESPACE}
```

> **Tip:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Clusters with HTTP or HTTPS Proxy on page 680.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

```
echo ${MANAGED_CLUSTER_NAME}
```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

```
nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
 ${MANAGED_CLUSTER_NAME}.conf
```

> **Caution:** Nutanix support is unable to recover forgotten credentials. The following best practices will prevent Kommander UI lockouts.
>
> - Backup the kubeconfig file.
> - Save your credentials.
> - Configure an IDP (identity provider) instead of using a username and password credentials.

3. > **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

```
kubectl get workspaces -A
```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace-namespace>
```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

```
kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
template='{{.data.value}}{{ "\n"}}'
```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

```
apiVersion: v1
kind: Secret
metadata:
  name: <your-managed-cluster-name>-kubeconfig
  labels:
    cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
type: cluster.x-k8s.io/secret
data:
  value: <value-you-copied-from-secret-above>
```

8. Create this secret in the desired workspace.

```
kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
 ${WORKSPACE_NAMESPACE}
```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
```

```
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

    ```
    kubectl get kommanderclusters -A
    ```

    If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

AWS Air-gapped Installation

This installation provides instructions to install NKP in an Amazon Web Services (AWS) air-gapped environment.

Remember, there are always more options for custom YAML Ain't Markup Language (YAML) in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Note:

- For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

  ```
  tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
  ```

- Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

  For example:

  ```
  export TMPDIR=/path/to/your/directory
  ```

AWS Prerequisites

Before you begin using Konvoy with AWS, you must:

1. Follow the steps to create permissions and roles on the Minimal Permissions and Role to Create Clusters page.
2. Create Cluster IAM Policies and Roles
3. Export the AWS region where you want to deploy the cluster:

   ```
   export AWS_REGION=us-west-2
   ```

4. Export the AWS profile with the credentials you want to use to create the Kubernetes cluster:

   ```
   export AWS_PROFILE=<profile>
   ```

If using AWS ECR as your local private registry, more information can be found on the Registry Mirror Tools page.

To deploy a cluster with a custom image in a region where CAPI images are not provided, you need to use Konvoy Image Builder to create your own image for the region.

> **Note:** For multi-tenancy, every tenant needs to be in a different AWS account to ensure they are truly independent of other tenants in order to enforce security.

AWS Air-gapped: Creating an Image
Learn how to build a custom AMI for use with NKP.

About this task

This procedure describes how to use the Konvoy Image Builder (KIB) to create a Cluster API compliant Amazon Machine Image (AMI). KIB uses variable overrides variable overrides to specify base image and container images to use in your new AMI.

AMI images contain configuration information and software to create a specific, pre-configured, operating environment. For example, you can create an AMI image of your current computer system settings and software. The AMI image can then be replicated and distributed, creating your computer system for other users. You can use overrides files to customize some of the components installed on your machine image. For example, you can tell KIB to install the FIPS versions of the Kubernetes components.

In previous NKP releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder. Explore the Create a Custom AMI on page 1091 topic for more options about overrides.

The **prerequisites** to use Konvoy Image Builder are:

Procedure

1. Download the KIB bundle for your version of NKP prefixed with `konvoy-image-bundle` for your OS.
2. Check the Supported Infrastructure Operating Systems
3. Check the Supported Kubernetes Version for your Provider.
4. Create a working registry.
   - » Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
   - » Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
5. Ensure you have met the minimal set of permissions from the AWS Image Builder Book.
6. A Minimal IAM Permissions for KIB to create an Image for an AWS account using Konvoy Image Builder. For more information, see Creating Minimal IAM Permissions for KIB on page 1087.

Extract the KIB Bundle

About this task

If not done previously during Konvoy Image Builder download in Prerequisites, extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION` folder. Otherwise, proceed to Build the Image below.

In previous NKP releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts with the exception of the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Containerd tarball

Procedure

1. Download `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`, and extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.

3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the new NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`.

   Example

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   Note:

   - For FIPS, pass the flag: `--fips`
   - For Red Hat Enterprise Linux (RHEL) OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`. Example command:

     ```
     export RHSM_ACTIVATION_KEY="-ci"
     export RHSM_ORG_ID="1232131"
     ```

     OR

     ```
     export RHSM_USER=""
     export RHSM_PASS=""
     ```

4. Follow the instructions below to build an AMI.

   Note: The `konvoy-image` binary and all supporting folders are also extracted. When run, `konvoy-image` bind mounts the current working directory (`${PWD}`) into the container to be used.

   Set the environment variables for AWS access. The following variables must be set using your credentials including required IAM:

   ```
   export AWS_ACCESS_KEY_ID
   export AWS_SECRET_ACCESS_KEY
   export AWS_DEFAULT_REGION
   ```

   If you have an override file to configure specific attributes of your AMI file, add it. For more information on customizing an override file, see Image Overrides on page 1123.

Build the Image

About this task

Depending on which version of NKP you are running, steps and flags will be different. To deploy in a region where CAPI images are not provided, you need to use KIB to create your own image for the region. For a list of supported AWS regions, refer to the Published AMI information from AWS.

Procedure

Run the `konvoy-image` command to build and validate the image.

```
konvoy-image build aws images/ami/rhel-86.yaml
```

a. By default, it builds in the `us-west-2` region. to specify another region set the `--region` flag as shown in the command below.

```
konvoy-image build aws --region us-east-1 images/ami/rhel-86.yaml
```

> **Note:** Ensure you have named the correct AMI image YAML file for your OS in the `konvoy-image build` command.

What to do next

After KIB provisions the image successfully, the `ami`

id is printed and written to the `packer.pkr.hcl` (Packer config) file. This file has an `artifact_id` field whose value provides the name of the AMI ID as shown in the example below. That is the `ami` you use in the `nkp create cluster` command.

```
...
    amazon-ebs.kib_image: Adding tag: "distribution_version": "8.6"
    amazon-ebs.kib_image: Adding tag: "gpu_nvidia_version": ""
    amazon-ebs.kib_image: Adding tag: "kubernetes_cni_version": ""
    amazon-ebs.kib_image: Adding tag: "build_timestamp": "20231023182049"
    amazon-ebs.kib_image: Adding tag: "gpu_types": ""
    amazon-ebs.kib_image: Adding tag: "kubernetes_version": "1.28.7"
==> amazon-ebs.kib_image: Creating snapshot tags
    amazon-ebs.kib_image: Adding tag: "ami_name": "konvoy-ami-
rhel-8.6-1.26.6-20231023182049"
==> amazon-ebs.kib_image: Terminating the source AWS instance...
==> amazon-ebs.kib_image: Cleaning up any extra volumes...
==> amazon-ebs.kib_image: No volumes to clean up, skipping
==> amazon-ebs.kib_image: Deleting temporary security group...
==> amazon-ebs.kib_image: Deleting temporary keypair...
==> amazon-ebs.kib_image: Running post-processor:  (type manifest)
Build 'amazon-ebs.kib_image' finished after 26 minutes 52 seconds.

==> Wait completed after 26 minutes 52 seconds

==> Builds finished. The artifacts of successful builds are:
--> amazon-ebs.kib_image: AMIs were created:
us-west-2: ami-04b8dfef8bd33a016

--> amazon-ebs.kib_image: AMIs were created:
us-west-2: ami-04b8dfef8bd33a016
```

What to do next

1. To use a custom AMI when creating your cluster, you must create that AMI using KIB first. Then perform the export and name the custom AMI for use in the command `nkp create cluster`:

   ```
   export AWS_AMI_ID=ami-<ami-id-here>
   ```

   > **Note:** Inside the sections for either Non-air-gapped or Air-gapped cluster creation, you will find the instructions for how to apply custom images.

Related Information

Procedure

- To use a local registry even in a non-air-gapped environment, download and extract the bundle. Download the Complete NKP Air-gapped Bundle for this release (that is. `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`) to load registry
- To view the complete set of instructions, see Load the Registry.

AWS Air-gapped: Load the Registry

Before creating an air-gapped Kubernetes cluster, you need to load the required images in a local registry for the Konvoy component.

About this task

The complete NKP air-gapped bundle is needed for an air-gapped environment but can also be used in a non-air-gapped environment. The bundle contains all the NKP components needed for an air-gapped environment installation and also to use a local registry in a non-air-gapped environment.

> **Note:** If you do not already have a local registry set up, see the Local Registry Tools page for more information.

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images is required. This registry must be accessible from both the bastion machine and either the AWS EC2 instances (if deploying to AWS) or other machines that will be created for the Kubernetes cluster.

Procedure

1. If not already done in prerequisites, download the air-gapped bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
   ```

2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. EX: For the bootstrap cluster, change your directory to the `nkp-<version>` directory similar to example below depending on your current location

   ```
   cd nkp-v2.12.0
   ```

3. Set an environment variable with your registry address and any other needed variables using this command.

   ```
   export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
   export REGISTRY_USERNAME=<username>
   export REGISTRY_PASSWORD=<password>
   export REGISTRY_CA=<path to the cacert file on the bastion>
   ```

4. Execute the following command to load the air-gapped image bundle into your private registry using any of the relevant flags to apply variables above.

```
nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

> **Note:** It may take some time to push all the images to your image registry, depending on the performance of the network between the machine you are running the script on and the registry.

> **Important:** To increase Docker Hub's rate limit use your Docker Hub credentials when creating the cluster, by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

5. Load the Kommander component images to your private registry using the command.

```
nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

Optional: This step is required only if you have an Ultimate license.

For NKP Catalog Applications available with the Ultimate license, perform this image load by running the following command to load the `dkp-catalog-applications` image bundle into your private registry:

```
nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

6. On the Bastion, load the Konvoy bootstrap image, using either the Docker or Podman command.

Docker example:

```
docker load -i konvoy-bootstrap-image-v2.12.0.tar
```

Podman example:

```
podman load -i konvoy-bootstrap-image-v2.12.0.tar
podman image tag konvoy-bootstrap:2.12.0 docker.io/konvoy-bootstrap:v2.12.0
```

AWS Air-gapped: Creating the Management Cluster

Create an AWS Management Cluster in an air-gapped environment.

About this task

Use this procedure to create a self-managed Management cluster with NKP. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing.

If you use these instructions to create a cluster on AWS using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes.

NKP uses AWS CSI as the default storage provider. You can use a Kubernetes CSI compatible storage solution that is suitable for production. See the Kubernetes documentation called Changing the Default Storage Class for more information.

In previous NKP releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder.

Before you begin

First you must name your cluster.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

   ```
   export CLUSTER_NAME=<aws-example>
   ```

3. Export variables for the existing infrastructure details.

   ```
   export AWS_VPC_ID=<vpc-...>
   export AWS_SUBNET_IDS=<subnet-...,subnet-...,subnet-...>
   export AWS_ADDITIONAL_SECURITY_GROUPS=<sg-...>
   export AWS_AMI_ID=<ami-...>
   ```

   - `AWS_VPC_ID`: the VPC ID where the cluster will be created. The VPC requires the following AWS VPC Endpoints to be already present:
     - `ec2` - `com.amazonaws.{region}.ec2`
     - `elasticloadbalancing` - `com.amazonaws.{region}.elasticloadbalancing`
     - `secretsmanager` - `com.amazonaws.{region}.secretsmanager`
     - `autoscaling` - `com.amazonaws.{region}.autoscaling`
     - `ecr` - `com.amazonaws.{region}.ecr.api` - `(authentication)`
     - `ecr` - `com.amazonaws.{region}.ecr.dkr` -

     More details about AWS service using an interface VPC endpoint and AWS VPC endpoints list at https://docs.aws.amazon.com/vpc/latest/privatelink/create-interface-endpoint.html and https://docs.aws.amazon.com/vpc/latest/privatelink/aws-services-privatelink-support.html respectively.
   - `AWS_SUBNET_IDS`: a comma-separated list of one or more private Subnet IDs with each one in a different Availability Zone. The cluster control-plane and worker nodes will automatically be spread across these Subnets.
   - `AWS_ADDITIONAL_SECURITY_GROUPS`: a comma-seperated list of one or more Security Groups IDs to use in addition to the ones automatically created by CAPA. For more information, see https://github.com/kubernetes-sigs/cluster-api-provider-aws.
   - `AWS_AMI_ID`: the AMI ID to use for control-plane and worker nodes. The AMI must be created by the `konvoy-image-builder`.

     > **Note:** In previous NKP releases, AMI images provided by the upstream CAPA project would be used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder on page 1084.

   There are two approaches to supplying the ID of your AMI. Either provide the ID of the AMI or provide a way for NKP to discover the AMI using location, format and OS information:

4. Use one of the following options:

   - Option One - Provide the ID of your AMI: Use the example command below leaving the existing flag that provides the AMI ID: `--ami AMI_ID`.
   - Option Two - Provide a path for your AMI with the information required for image discover.
     - Where the AMI is published using your AWS Account ID: `--ami-owner AWS_ACCOUNT_ID`
     - The format or string used to search for matching AMIs and ensure it references the Kubernetes version plus the base OS name: `--ami-base-os ubuntu-20.04`
     - The base OS information: `--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*'`

5. (Optional) Configure your cluster to use an existing container registry as a mirror when attempting to pull images. The example below is for AWS ECR:

   Warning: Ensure that the local registry is set up if you do not have this set up already.

   Warning: The AMI must be created with Konvoy Image Builder in order to use the registry mirror feature.

   ```
   export AWS_AMI_ID=<ami-...>
   ```

   - (Optional) Registry Mirror - Configure your cluster to use an existing local registry as a mirror when attempting to pull images. Below is an AWS ECR example where `REGISTRY_URL`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.:

     ```
     export REGISTRY_URL=<ecr-registry-URI>
     ```

   - `REGISTRY_URL`: the address of an existing registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.
   - Other local registries may use the options below:
     - JFrog - `REGISTRY_CA`: (optional) the path on the bastion machine to the registry CA. This value is only needed if the registry is using a self-signed certificate and the AMIs are not already configured to trust this CA.
     - `REGISTRY_USERNAME`: optional, set to a user that has pull access to this registry.
     - `REGISTRY_PASSWORD`: optional if username is not set.

6. Create a Kubernetes cluster. The following example shows a common configuration. For the complete list of cluster creation options, see the `nkp create cluster aws` CLI Command reference.

   Note: NKP uses AWS CSI as the default storage provider. You can use a Kubernetes CSI (see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types) compatible storage solution that is suitable for production. For more information, see the https://kubernetes.io/docs/tasks/administer-cluster/change-default-storage-class/ topic in the Kubernetes documentation.

7. Do one of the following:

   - Option1 - Run this command to create your Kubernetes cluster using any relevant flags for Option One explained above providing the AMI ID:

     ```
     nkp create cluster aws --cluster-name=${CLUSTER_NAME} \
     --additional-tags=owner=$(whoami) \
     --with-aws-bootstrap-credentials=true \
     ```

```
--vpc-id=${AWS_VPC_ID} \
--ami=${AWS_AMI_ID} \
--subnet-ids=${AWS_SUBNET_IDS} \
--internal-load-balancer=true \
--additional-security-group-ids=${AWS_ADDITIONAL_SECURITY_GROUPS} \
--registry-mirror-url=<YOUR_ECR_URL> \
--self-managed
```

- Option 2 - Run the command as shown from the explanation above to allow discovery of your AMI:

```
nkp create cluster aws \
--cluster-name=${CLUSTER_NAME} \
--additional-tags=owner=$(whoami) \
--with-aws-bootstrap-credentials=true \
--vpc-id=${AWS_VPC_ID} \
--ami-owner AWS_ACCOUNT_ID \
--ami-base-os ubuntu-20.04 \
--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*' \
--subnet-ids=${AWS_SUBNET_IDS} \
--internal-load-balancer=true \
--additional-security-group-ids=${AWS_ADDITIONAL_SECURITY_GROUPS} \
--registry-mirror-url=<YOUR_ECR_URL> \
--self-managed
```

8. Additional configurations that you can perform:

   » Additional cluster creation flags based on your environment:

   » Optional Registry flag: `--registry-mirror-url=${REGISTRY_URL}`

   » Flatcar OS flag to instruct the bootstrap cluster to make changes related to the installation paths: `--os-hint flatcar`

   » HTTP or HTTPS flags if you use proxies: `--http-proxy`, `--https-proxy`, and `--no-proxy`

AWS Air-gapped: Install Kommander

This section provides installation instructions for the Kommander component of NKP in an air-gapped AWS environment.

About this task

Once you have installed the Konvoy component and created a cluster, continue with the installation of the Kommander component which will allow you to access the UI and attach new or existing clusters to monitor.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all the prerequisites required for the installation. For more information, see Prerequisites for Installation on page 47.
- Ensure you have a default StorageClass. For more information, see Creating a Default StorageClass on page 493.
- Ensure you have loaded all necessary images for your configuration. For more information, see AWS Air-gapped: Load the Registry on page 179.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. (Optional) Customize your `kommander.yaml`.

   a. For customization options, see Additional Kommander Configuration on page 1016. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.

5. (Optional) If your cluster uses a custom AWS VPC and requires an internal load-balancer, set the traefik annotation to create an internal-facing ELB.

   ```
   apps:
     traefik:
       enabled: true
       values: |
         service:
           annotations:
             service.beta.kubernetes.io/aws-load-balancer-internal: "true"
   ```

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `nkp-catalog-appliations`.

   ```
   apiVersion: config.kommander.mesosphere.io/v1alpha1
   kind: Installation
   catalog:
     repositories:
       - name: dkp-catalog-applications
         labels:
           kommander.d2iq.io/project-default-catalog-repository: "true"
           kommander.d2iq.io/workspace-default-catalog-repository: "true"
           kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
         gitRepositorySpec:
           url: https://github.com/mesosphere/dkp-catalog-applications
           ref:
             tag: v2.12.0
   ```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Configuring Applications After Installing Kommander on page 1035.

AWS Air-gapped: Verifying your Installation and UI Log in

Verify the Kommander installation and log in to the Dashboard UI. After you build the Konvoy cluster and you install Kommander, verify your installation. It waits for all applications to be ready by default.

About this task

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command

```
kubectl -n kommander get helmrelease <HELMRELEASE_NAME>
```

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the following commands.

```
kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op":
 "replace", "path": "/spec/suspend", "value": true}]'
kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op":
 "replace", "path": "/spec/suspend", "value": false}]'
```

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      nkp experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Day 2 Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of NKP. The Day 2 section allows you to manage cluster operations and their application workloads to optimize your organization’s productivity.

- Continue to the NKP Dashboard.

AWS Air-gapped: Creating Managed Clusters Using the NKP CLI
This topic explains how to continue using the CLI to create managed clusters rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed which allows it to be a Management cluster or a stand alone cluster. Subsequent new clusters are not self-managed as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects, or install the Kommander component. Those tasks are only done on Management clusters!
>
> Your new managed cluster needs to be part of a workspace under a management cluster. To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

   ```
   kubectl get workspace -A
   ```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

   > **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First you must name your cluster. Then you run the command to deploy it.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

   ```
   export MANAGED_CLUSTER_NAME=<aws-additional>
   ```

Create a Kubernetes Cluster

About this task

The below instructions tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, it will be created in the `default` workspace, and you need

to take additional steps to attach to a workspace later. For instructions on how to do this, see Attach a Kubernetes Cluster.

Procedure

Execute this command to create your additional Kubernetes cluster using any relevant flags. This will create a new non-self-managed cluster that can be managed by management cluster you created in the previous section.

```
nkp create cluster aws --cluster-name=${MANAGED_CLUSTER_NAME} \
--additional-tags=owner=$(whoami) \
--namespace ${WORKSPACE_NAMESPACE}
--kubeconfig=<management-cluster-kubeconfig-path> \
--with-aws-bootstrap-credentials=true \
--vpc-id=${AWS_VPC_ID} \
--ami=${AWS_AMI_ID} \
--subnet-ids=${AWS_SUBNET_IDS} \
--internal-load-balancer=true \
--additional-security-group-ids=${AWS_ADDITIONAL_SECURITY_GROUPS} \
--registry-mirror-url=${REGISTRY_URL} \
--registry-mirror-cacert=${REGISTRY_CA} \
--registry-mirror-username=${REGISTRY_USERNAME} \
--registry-mirror-password=${REGISTRY_PASSWORD} \
--kubeconfig=<management-cluster-kubeconfig-path> \
```

Tip: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Clusters with HTTP or HTTPS Proxy on page 680.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

   ```
   echo ${MANAGED_CLUSTER_NAME}
   ```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

   ```
   nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
    ${MANAGED_CLUSTER_NAME}.conf
   ```

3. **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

   ```
   kubectl get workspaces -A
   ```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace-namespace>
   ```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

```
kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
template='{{.data.value}}{{ "\n"}}'
```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

```
apiVersion: v1
kind: Secret
metadata:
  name: <your-managed-cluster-name>-kubeconfig
  labels:
    cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
type: cluster.x-k8s.io/secret
data:
  value: <value-you-copied-from-secret-above>
```

8. Create this secret in the desired workspace.

```
kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
 ${WORKSPACE_NAMESPACE}
```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

```
kubectl get kommanderclusters -A
```

If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

AWS with FIPS Installation

This installation provides instructions to install NKP in an AWS non-air-gapped environment using FIPS.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50

- Prerequisites for Installation on page 47

AWS Prerequisites

Before you begin using Konvoy with AWS, you must:

1. Follow the steps to create permissions and roles on the Minimal Permissions and Role to Create Clusters page.
2. Create Cluster IAM Policies and Roles
3. Export the AWS region where you want to deploy the cluster:

   ```
   export AWS_REGION=us-west-2
   ```

4. Export the AWS profile with the credentials you want to use to create the Kubernetes cluster:

   ```
   export AWS_PROFILE=<profile>
   ```

If using AWS ECR as your local private registry, more information can be found on the Registry Mirror Tools page.

To deploy a cluster with a custom image in a region where CAPI images are not provided, you need to use Konvoy Image Builder to create your own image for the region.

> **Note:** For multi-tenancy, every tenant needs to be in a different AWS account to ensure they are truly independent of other tenants in order to enforce security.

AWS FIPS: Creating an Image

Learn how to build a custom AMI for use with NKP.

About this task

This procedure describes how to use the Konvoy Image Builder (KIB) to create a Cluster API compliant Amazon Machine Image (AMI). KIB uses variable overrides to specify base image and container images to use in your new AMI.

AMI images contain configuration information and software to create a specific, pre-configured, operating environment. For example, you can create an AMI image of your current computer system settings and software. The AMI image can then be replicated and distributed, creating your computer system for other users. You can use overrides files to customize some of the components installed on your machine image. For example, having the FIPS versions of the Kubernetes components installed by KIB.

In previous NKP releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder. Explore the Customize your Image topic for more options about overrides.

The **prerequisites** to use Konvoy Image Builder are:

Procedure

1. Download the KIB bundle for your version of NKP prefixed with `konvoy-image-bundle` for your OS.
2. Check the Supported Infrastructure Operating Systems
3. Check the Supported Kubernetes Version for your Provider.

4. Create a working registry.

   » Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.

   » Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.

5. Ensure you have met the minimal set of permissions from the AWS Image Builder Book.

6. A Minimal IAM Permissions for KIB to create an Image for an AWS account using Konvoy Image Builder.

Extract the KIB Bundle

About this task

If not done previously during Konvoy Image Builder download in Prerequisites, extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION` folder. Otherwise, proceed to Build the Image below.

In previous NKP releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts with the exception of the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Containerd tarball

Procedure

1. Downloading NKP on page 16`nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarball to a local directory

   For example:

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.

3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the new NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`. Example command.

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   Note:

   - For FIPS, pass the flag: `--fips`
   - For Red Hat Enterprise Linux (RHEL) OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`. Example command:

     ```
     export RHSM_ACTIVATION_KEY="-ci"
     export RHSM_ORG_ID="1232131"
     ```

4. Follow the instructions below to build an AMI.

   Note: The `konvoy-image` binary and all supporting folders are also extracted. When run, `konvoy-image` bind mounts the current working directory (`${PWD}`) into the container to be used.

   Set the environment variables for AWS access. The following variables must be set using your credentials including required IAM:

   ```
   export AWS_ACCESS_KEY_ID
   export AWS_SECRET_ACCESS_KEY
   export AWS_DEFAULT_REGION
   ```

   If you have an override file to configure specific attributes of your AMI file, add it. Instructions for customizing an override file are found on this page: Image Overrides

Build the Image

About this task

Depending on which version of NKP you are running, steps and flags will be different. To deploy in a region where CAPI images are not provided, you need to use KIB to create your own image for the region. For a list of supported AWS regions, refer to the Published AMI information from AWS.

Procedure

Run the `konvoy-image` command to build and validate the image.

```
konvoy-image build aws images/ami/rhel-86.yaml
```

a. By default, it builds in the `us-west-2` region. to specify another region set the `--region` flag as shown in the command below.

   ```
   konvoy-image build aws --region us-east-1 images/ami/rhel-86.yaml
   ```

   Note: Ensure you have named the correct AMI image YAML file for your OS in the `konvoy-image build` command.

What to do next

After KIB provisions the image successfully, the `ami`

id is printed and written to the `packer.pkr.hcl` (Packer config) file. This file has an `artifact_id` field whose value provides the name of the AMI ID as shown in the example below. That is the `ami` you use in the `NKP create cluster` command.

```
{
  "builds": [
    {
      "name": "kib_image",
      "builder_type": "amazon-ebs",
      "build_time": 1698086886,
      "files": null,
      "artifact_id": "us-west-2:ami-04b8dfef8bd33a016",
      "packer_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05",
      "custom_data": {
        "containerd_version": "",
        "distribution": "RHEL",
        "distribution_version": "8.6",
        "kubernetes_cni_version": "",
        "kubernetes_version": "1.26.6"
      }
    }
```

```
  ],
  "last_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05"
}
```

What to do next

1. To use a custom AMI when creating your cluster, you must create that AMI using KIB first. Then perform the export and name the custom AMI for use in the command `nkp create cluster`:

   ```
   export AWS_AMI_ID=ami-<ami-id-here>
   ```

   > **Note:** Inside the sections for either Non-air-gapped or Air-gapped cluster creation, you will find the instructions for how to apply custom images.

Related Information

Procedure

- To use a local registry even in a non-air-gapped environment, download and extract the bundle. Downloading NKP on page 16 the Complete NKP Air-gapped Bundle for this release (that is. `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`) to load registry
- To view the complete set of instructions, see Load the Registry.

AWS FIPS: Creating the Management Cluster

Create an AWS Management Cluster in a non-air-gapped environment using FIPS.

About this task

Use this procedure to create a self-managed **Management cluster** with NKP. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing. First you must name your cluster.

Name Your Cluster

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. For more naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable using the command `export CLUSTER_NAME=<aws-example>`.

Create a New AWS Kubernetes Cluster

About this task

If you use these instructions to create a cluster on AWS using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes.

NKP uses AWS CSI as the default storage provider. You can use a Kubernetes CSI compatible storage solution that is suitable for production. See the Kubernetes documentation called Changing the Default Storage Class for more information.

In previous NKP releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always

be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder.

There are two approaches to supplying the ID of your AMI. Either provide the ID of the AMI or provide a way for NKP to discover the AMI using location, format and OS information:

Procedure

1. Option One - Provide the ID of your AMI.
   a. Use the example command below leaving the existing flag that provides the AMI ID: `--ami AMI_ID`
2. Option Two - Provide a path for your AMI with the information required for image discover.
   a. Where the AMI is published using your AWS Account ID: `--ami-owner AWS_ACCOUNT_ID`
   b. The format or string used to search for matching AMIs and ensure it references the Kubernetes version plus the base OS name: `--ami-base-os ubuntu-20.04`
   c. The base OS information: `--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*'`

   Note:

   - The AMI must be created with Konvoy Image Builder in order to use the registry mirror feature.

     ```
     export AWS_AMI_ID=<ami-...>
     ```

   - (Optional) Registry Mirror - Configure your cluster to use an existing local registry as a mirror when attempting to pull images. Below is an AWS ECR example where `REGISTRY_URL`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.:

     ```
     export REGISTRY_URL=<ecr-registry-URI>
     ```

3. Run this command to create your Kubernetes cluster using any relevant flags.

   ```
   nkp create cluster aws \
   --cluster-name=${CLUSTER_NAME} \
   --additional-tags=owner=$(whoami) \
   --with-aws-bootstrap-credentials=true \
   --ami=${AWS_AMI_ID} \
   --kubernetes-version=v1.29.6+fips.0 \
   --etcd-version=3.5.10+fips.0 \
   --kubernetes-image-repository=docker.io/mesosphere \
   --self-managed
   ```

   If providing the AMI path, use these flags in place of `AWS_AMI_ID`:

   ```
   --ami-owner AWS_ACCOUNT_ID \
   --ami-base-os ubuntu-20.04 \
   --ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*' \
   ```

   » Additional cluster creation flags based on your environment:

   » Optional Registry flag: `--registry-mirror-url=${REGISTRY_URL}`

   » Flatcar OS flag to instruct the bootstrap cluster to make changes related to the installation paths: `--os-hint flatcar`

» HTTP or HTTPS flags if you use proxies: `--http-proxy`, `--https-proxy`, and `--no-proxy`

If you want to monitor or verify the installation of your clusters, refer to the topic: Verify your Cluster and NKP Installation

AWS FIPS: Install Kommander

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped AWS environment using FIPS.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

```
export CLUSTER_NAME=<your-management-cluster-name>
```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

```
nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
```

3. Create a configuration file for the deployment.

```
nkp install kommander --init > kommander.yaml
```

4. If required: Customize your `kommander.yaml`.

   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy and External Load Balancer.

5. Only required if your cluster uses a custom AWS VPC and requires an internal load-balancer, set the traefik annotation to create an internal-facing ELB.

```
apps:
```

```
  traefik:
    enabled: true
    values: |
      service:
        annotations:
          service.beta.kubernetes.io/aws-load-balancer-internal: "true"
```

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `NKP-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see the topic Configuring NKP Catalog Applications after Installing NKP.

AWS FIPS: Verifying your Installation and UI Log in
Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the command `kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m`.

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using the command `nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf`.
2. Retrieve your credentials at any time using the command `kubectl -n kommander get secret NKP-credentials -o go-template='Username: {{.data.username|base64decode}} {{ "\n"}}Password: {{.data.password|base64decode}}{{ "\n"}}'`.

3. Retrieve the URL used for accessing the UI using the command `kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/dashboard{{ "\n"}}'`.
   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      nkp experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Day 2 Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of NKP. The Day 2 section allows you to manage cluster operations and their application workloads to optimize your organization's productivity.

- Continue to the NKP Dashboard.

AWS FIPS: Creating Managed Clusters Using the NKP CLI
This topic explains how to continue using the CLI to create managed clusters rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed which allows it to be a Management cluster or a stand alone cluster. Subsequent new clusters are not self-managed as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects, or install the Kommander component. Those tasks are only done on Management clusters!
>
> Your new managed cluster needs to be part of a workspace under a management cluster. To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

   ```
   kubectl get workspace -A
   ```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

   > **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First you must name your cluster. Then you run the command to deploy it.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

```
export MANAGED_CLUSTER_NAME=<aws-additional>
```

Create a Kubernetes Cluster

About this task

The below instructions tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, it will be created in the `default` workspace, and you need to take additional steps to attach to a workspace later. For instructions on how to do this, see Attach a Kubernetes Cluster.

Procedure

Execute this command to create your additional Kubernetes cluster using any relevant flags. This will create a new non-self-managed cluster that can be managed by management cluster you created in the previous section.

```
nkp create cluster aws \
--cluster-name=${MANAGED_CLUSTER_NAME} \
--additional-tags=owner=$(whoami) \
--namespace ${WORKSPACE_NAMESPACE}
--with-aws-bootstrap-credentials=true \
--kubernetes-version=v1.29.6+fips.0 \
--kubernetes-image-repository=docker.io/mesosphere
--kubeconfig=<management-cluster-kubeconfig-path>
```

> **Tip:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Clusters with HTTP or HTTPS Proxy on page 680.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

   ```
   echo ${MANAGED_CLUSTER_NAME}
   ```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

   ```
   nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
    ${MANAGED_CLUSTER_NAME}.conf
   ```

3. **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

   ```
   kubectl get workspaces -A
   ```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace-namespace>
   ```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

   ```
   kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
   template='{{.data.value}}{{ "\n"}}'
   ```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <your-managed-cluster-name>-kubeconfig
     labels:
       cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
   type: cluster.x-k8s.io/secret
   data:
     value: <value-you-copied-from-secret-above>
   ```

8. Create this secret in the desired workspace.

   ```
   kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
    ${WORKSPACE_NAMESPACE}
   ```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

   ```
   cat << EOF | kubectl apply -f -
   apiVersion: kommander.mesosphere.io/v1beta1
   kind: KommanderCluster
   metadata:
   ```

```
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

    ```
    kubectl get kommanderclusters -A
    ```

    If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

AWS Air-gapped with FIPS Installation

Installation instructions for installing NKP in an Amazon Web Services (AWS) air-gapped environment.

Remember, there are always more options for custom YAML Ain't Markup Language (YAML) in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Note:

- For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

  ```
  tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
  ```

- Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

  For example:

  ```
  export TMPDIR=/path/to/your/directory
  ```

AWS Prerequisites

Before you begin using Konvoy with AWS, you must:

1. Follow the steps to create permissions and roles on the Minimal Permissions and Role to Create Clusters page.
2. Create Cluster IAM Policies and Roles
3. Export the AWS region where you want to deploy the cluster:

   ```
   export AWS_REGION=us-west-2
   ```

4. Export the AWS profile with the credentials you want to use to create the Kubernetes cluster:

```
export AWS_PROFILE=<profile>
```

If using AWS ECR as your local private registry, more information can be found on the Registry Mirror Tools page.

To deploy a cluster with a custom image in a region where CAPI images are not provided, you need to use Konvoy Image Builder to create your own image for the region.

> **Note:** For multi-tenancy, every tenant needs to be in a different AWS account to ensure they are truly independent of other tenants in order to enforce security.

AWS Air-gapped FIPS: Creating an Image
Learn how to build a custom AMI for use with NKP.

About this task

This procedure describes how to use the Konvoy Image Builder (KIB) to create a Cluster API compliant Amazon Machine Image (AMI). KIB uses variable overrides to specify base image and container images to use in your new AMI.

AMI images contain configuration information and software to create a specific, pre-configured, operating environment. For example, you can create an AMI image of your current computer system settings and software. The AMI image can then be replicated and distributed, creating your computer system for other users. You can use overrides files to customize some of the components installed on your machine image. For example, having the FIPS versions of the Kubernetes components installed by KIB. components.

In previous Nutanix Kubernetes Platform (NKP) releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder. Explore the Customize your Image topic for more options about overrides.

The **prerequisites** to use Konvoy Image Builder are:

Procedure

1. Download the KIB bundle for your version of NKP prefixed with `konvoy-image-bundle` for your OS.
2. Check the Supported Infrastructure Operating Systems.
3. Check the Supported Kubernetes Version for your Provider.
4. Create a working registry.
   - » Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
   - » Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
5. Ensure you have met the minimal set of permissions from the AWS Image Builder Book.
6. A Minimal IAM Permissions for KIB to create an Image for an AWS account using Konvoy Image Builder.

Extract the KIB Bundle

About this task

If not done previously during Konvoy Image Builder download in Prerequisites, extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION` folder. Otherwise, proceed to Build the Image below.

In previous NKP releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts with the exception of the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Containerd tarball

Procedure

1. Downloading NKP on page 16`nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.

3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`. Example command.

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   Note:

   - For FIPS, pass the flag: `--fips`
   - For Red Hat Enterprise Linux (RHEL) OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`. Example command:

     ```
     export RHSM_ACTIVATION_KEY="-ci"
     export RHSM_ORG_ID="1232131"
     ```

4. Follow the instructions below to build an AMI.

   Note: The `konvoy-image` binary and all supporting folders are also extracted. When run, `konvoy-image` bind mounts the current working directory (`${PWD}`) into the container to be used.

   Set the environment variables for AWS access. The following variables must be set using your credentials including required IAM:

   ```
   export AWS_ACCESS_KEY_ID
   export AWS_SECRET_ACCESS_KEY
   export AWS_DEFAULT_REGION
   ```

   If you have an override file to configure specific attributes of your AMI file, add it. Instructions for customizing an override file are found on this page: Image Overrides

Build the Image

About this task

Depending on which version of NKP you are running, steps and flags will be different. To deploy in a region where CAPI images are not provided, you need to use KIB to create your own image for the region. For a list of supported AWS regions, refer to the Published AMI information from AWS.

Procedure

Run the `konvoy-image` command to build and validate the image.

```
konvoy-image build aws images/ami/rhel-86.yaml
```

a. By default, it builds in the `us-west-2` region. to specify another region set the `--region` flag as shown in the command below.

```
konvoy-image build aws --region us-east-1 images/ami/rhel-86.yaml
```

> **Note:** Ensure you have named the correct AMI image YAML file for your OS in the `konvoy-image build` command.

What to do next

After KIB provisions the image successfully, the `ami`

id is printed and written to the `packer.pkr.hcl` (Packer config) file. This file has an `artifact_id` field whose value provides the name of the AMI ID as shown in the example below. That is the `ami` you use in the `NKP create cluster` command.


```
{
  "builds": [
    {
      "name": "kib_image",
      "builder_type": "amazon-ebs",
      "build_time": 1698086886,
      "files": null,
      "artifact_id": "us-west-2:ami-04b8dfef8bd33a016",
      "packer_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05",
      "custom_data": {
        "containerd_version": "",
        "distribution": "RHEL",
        "distribution_version": "8.6",
        "kubernetes_cni_version": "",
        "kubernetes_version": "1.26.6"
      }
    }
  ],
  "last_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05"
}
```


What to do next

1. To use a custom AMI when creating your cluster, you must create that AMI using KIB first. Then perform the export and name the custom AMI for use in the command `nkp create cluster`:

```
export AWS_AMI_ID=ami-<ami-id-here>
```

> **Note:** Inside the sections for either Non-air-gapped or Air-gapped cluster creation, you will find the instructions for how to apply custom images.

Related Information

Procedure

- To use a local registry even in a non-air-gapped environment, download and extract the bundle. Downloading NKP on page 16 the Complete NKP Air-gapped Bundle for this release (that is. `nkp-air-gapped-bundle_v2..0_linux_amd64.tar.gz`) to load registry
- To view the complete set of instructions, see Load the Registry.

AWS Air-gapped FIPS: Loading the Registry
Before creating an air-gapped Kubernetes cluster, you need to load the required images in a local registry for the Konvoy component.

About this task

The complete NKP air-gapped bundle is needed for an air-gapped environment but can also be used in a non-air-gapped environment. The bundle contains all the NKP components needed for an air-gapped environment installation and also to use a local registry in a non-air-gapped environment.

> **Note:** If you do not already have a local registry set up, see the Local Registry Tools page for more information.

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images is required. This registry must be accessible from both the bastion machine and either the AWS EC2 instances (if deploying to AWS) or other machines that will be created for the Kubernetes cluster.

Procedure

1. If not already done in prerequisites, download the air-gapped bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
   ```

2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. EX: For the bootstrap cluster, change your directory to the `nkp-<version>` directory similar to example below depending on your current location

   ```
   cd nkp-v2.12.0
   ```

3. Set an environment variable with your registry address and any other needed variables using this command.

   ```
   export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
   export REGISTRY_USERNAME=<username>
   export REGISTRY_PASSWORD=<password>
   export REGISTRY_CA=<path to the cacert file on the bastion>
   ```

4. Execute the following command to load the air-gapped image bundle into your private registry using any of the relevant flags to apply variables above.

```
nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

> **Note:** It may take some time to push all the images to your image registry, depending on the performance of the network between the machine you are running the script on and the registry.

> **Important:** To increase Docker Hub's rate limit use your Docker Hub credentials when creating the cluster, by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

5. Load the Kommander component images to your private registry using the command.

```
nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

Optional: This step is required only if you have an Ultimate license.

For NKP Catalog Applications available with the Ultimate license, perform this image load by running the following command to load the `dkp-catalog-applications` image bundle into your private registry:

```
nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

6. On the Bastion, load the Konvoy bootstrap image, using either the Docker or Podman command.

Docker example:

```
docker load -i konvoy-bootstrap-image-v2.12.0.tar
```

Podman example:

```
podman load -i konvoy-bootstrap-image-v2.12.0.tar
podman image tag konvoy-bootstrap:2.12.0 docker.io/konvoy-bootstrap:v2.12.0
```

AWS Air-gapped FIPS: Creating the Management Cluster

Create an AWS Management Cluster in an air-gapped environment using FIPS.

About this task

Use this procedure to create a self-managed **Management cluster** with NKP. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing.

If you use these instructions to create a cluster on AWS using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes.

NKP uses AWS CSI as the default storage provider. You can use a Kubernetes CSI compatible storage solution that is suitable for production. See the Kubernetes documentation called Changing the Default Storage Class for more information.

In previous NKP releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder.

Before you begin

First you must name your cluster.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable:

   ```
   export CLUSTER_NAME=<aws-example>
   ```

3. Export variables for the existing infrastructure details.

   ```
   export AWS_VPC_ID=<vpc-...>
   export AWS_SUBNET_IDS=<subnet-...,subnet-...,subnet-...>
   export AWS_ADDITIONAL_SECURITY_GROUPS=<sg-...>
   export AWS_AMI_ID=<ami-...>
   ```

   `AWS_VPC_ID`: the VPC ID where the cluster will be created. The VPC requires the following AWS VPC Endpoints to be already present

4. There are two approaches to supplying the ID of your AMI. Either provide the ID of the AMI or provide a way for NKP to discover the AMI using location, format and OS information:

   a. Option One - Provide the ID of your AMI.

   Use the example command below leaving the existing flag that provides the AMI ID: `--ami AMI_ID`

   b. Option Two - Provide a path for your AMI with the information required for image discover.

   - Where the AMI is published using your AWS Account ID: `--ami-owner AWS_ACCOUNT_ID`
   - The format or string used to search for matching AMIs and ensure it references the Kubernetes version plus the base OS name: `--ami-base-os ubuntu-20.04`
   - The base OS information: `--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*'`

   > **Note:**
   >
   > - The AMI must be created with Konvoy Image Builder in order to use the registry mirror feature.
   >
   >   ```
   >   export AWS_AMI_ID=<ami-...>
   >   ```
   >
   > - (Optional) Registry Mirror - Configure your cluster to use an existing local registry as a mirror when attempting to pull images. Below is an AWS ECR example where `REGISTRY_URL`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.:
   >
   >   ```
   >   export REGISTRY_URL=<ecr-registry-URI>
   >   ```

5. Run this command to create your Kubernetes cluster by providing the image ID and using any relevant flags.

   ```
   nkp create cluster aws --cluster-name=${CLUSTER_NAME} \
   --additional-tags=owner=$(whoami) \
   --with-aws-bootstrap-credentials=true \
   --vpc-id=${AWS_VPC_ID} \
   ```

```
--ami=${AWS_AMI_ID} \
--subnet-ids=${AWS_SUBNET_IDS} \
--internal-load-balancer=true \
--additional-security-group-ids=${AWS_ADDITIONAL_SECURITY_GROUPS} \
--registry-mirror-url=${REGISTRY_URL} \
--registry-mirror-cacert=${REGISTRY_CA} \
--registry-mirror-username=${REGISTRY_USERNAME} \
--registry-mirror-password=${REGISTRY_PASSWORD} \
--kubernetes-version=v1.29.6+fips.0 \
--etcd-version=3.5.10+fips.0 \
--kubernetes-image-repository=docker.io/mesosphere \
--self-managed
```

If providing the AMI path, use these flags in place of `AWS_AMI_ID`:

```
--ami-owner AWS_ACCOUNT_ID \
--ami-base-os ubuntu-20.04 \
--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*' \
```

» Additional cluster creation flags based on your environment:

» Optional Registry flag: `--registry-mirror-url=${REGISTRY_URL}`

» Flatcar OS flag to instruct the bootstrap cluster to make changes related to the installation paths: `--os-hint flatcar`

» HTTP or HTTPS flags if you use proxies: `--http-proxy`, `--https-proxy`, and `--no-proxy`

AWS Air-gapped FIPS: Install Kommander

This section provides installation instructions for the Kommander component of NKP in an air-gapped AWS environment with FIPS.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass. For more information, see Creating a Default StorageClass on page 493.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

```
export CLUSTER_NAME=<your-management-cluster-name>
```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

```
nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
```

3. Create a configuration file for the deployment.

```
nkp install kommander --init > kommander.yaml
```

4. If required: Customize your `kommander.yaml`.

   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy and External Load Balancer.

5. Only required if your cluster uses a custom AWS VPC and requires an internal load-balancer, set the traefik annotation to create an internal-facing ELB.

```
apps:
  traefik:
    enabled: true
    values: |
      service:
        annotations:
          service.beta.kubernetes.io/aws-load-balancer-internal: "true"
```

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `nkp-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP,see the topic Configuring NKP Catalog Applications after Installing NKP.

AWS Air-gapped FIPS: Verifying your Installation and UI Log in
Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      nkp experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Day 2 Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of NKP. The Day 2 section allows you to manage cluster operations and their application workloads to optimize your organization's productivity.

- Continue to the NKP Dashboard.

AWS Air-gapped FIPS: Creating Managed Clusters Using the NKP CLI
This topic explains how to continue using the CLI to create managed clusters rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed which allows it to be a Management cluster or a stand alone cluster. Subsequent new clusters are not self-managed as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects, or install the Kommander component. Those tasks are only done on Management clusters!
>
> Your new managed cluster needs to be part of a workspace under a management cluster. To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

   ```
   kubectl get workspace -A
   ```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

   > **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First you must name your cluster. Then you run the command to deploy it.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable using the command `export MANAGED_CLUSTER_NAME=<aws-additional>`

Create a Kubernetes Cluster

About this task

The below instructions tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, it will be created in the `default` workspace, and you need to take additional steps to attach to a workspace later. For instructions on how to do this, see Attach a Kubernetes Cluster.

Procedure

Execute this command to create your additional Kubernetes cluster using any relevant flags. This will create a new non-self-managed cluster that can be managed by management cluster you created in the previous section.

```
nkp create cluster aws
--cluster-name=${MANAGED_CLUSTER_NAME} \
--additional-tags=owner=$(whoami) \
--namespace ${WORKSPACE_NAMESPACE}
--with-aws-bootstrap-credentials=true \
--vpc-id=${AWS_VPC_ID} \
```

```
--ami=${AWS_AMI_ID} \
--subnet-ids=${AWS_SUBNET_IDS} \
--internal-load-balancer=true \
--additional-security-group-ids=${AWS_ADDITIONAL_SECURITY_GROUPS} \
--registry-mirror-url=${REGISTRY_URL} \
--registry-mirror-cacert=${REGISTRY_CA} \
--registry-mirror-username=${REGISTRY_USERNAME} \
--registry-mirror-password=${REGISTRY_PASSWORD} \
--kubeconfig=<management-cluster-kubeconfig-path> \
```

Tip: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Clusters with HTTP or HTTPS Proxy on page 680.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set using the command `echo ${MANAGED_CLUSTER_NAME}`
2. Retrieve your kubeconfig from the cluster you have created without setting a workspace using the command `nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} > ${MANAGED_CLUSTER_NAME}.conf`
3. You can now either attach it in the UI, link to attaching it to workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

   Note: This is only necessary if you never set the workspace of your cluster upon creation.
4. Retrieve the workspace where you want to attach the cluster using the command `kubectl get workspaces -A`.
5. Set the `WORKSPACE_NAMESPACE` environment variable using the command `export WORKSPACE_NAMESPACE=<workspace-namespace>`
6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster using the command `kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-template='{{.data.value}}{{ "\n"}}'`.
7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

   Example:

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <your-managed-cluster-name>-kubeconfig
     labels:
       cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
   type: cluster.x-k8s.io/secret
   data:
     value: <value-you-copied-from-secret-above>
   ```

8. Create this secret in the desired workspace using the command `kubectl apply -f attached-cluster-kubeconfig.yaml --namespace ${WORKSPACE_NAMESPACE}`

9. Create this `kommandercluster` object to attach the cluster to the workspace.

   Example:

   ```
   cat << EOF | kubectl apply -f -
   apiVersion: kommander.mesosphere.io/v1beta1
   kind: KommanderCluster
   metadata:
     name: ${MANAGED_CLUSTER_NAME}
     namespace: ${WORKSPACE_NAMESPACE}
   spec:
     kubeconfigRef:
       name: ${MANAGED_CLUSTER_NAME}-kubeconfig
     clusterRef:
       capiCluster:
         name: ${MANAGED_CLUSTER_NAME}
   EOF
   ```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by using the command `kubectl get kommanderclusters -A`.

    It may take a few minutes to reach "Joined" status. If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, review Platform Expansion.

AWS with GPU Installation

This installation provides instructions to install NKP in an AWS non-air-gapped environment.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

AWS Prerequisites

Before you begin using Konvoy with AWS, you must:

1. Follow the steps to create permissions and roles on the Minimal Permissions and Role to Create Clusters page.
2. Create Cluster IAM Policies and Roles
3. Export the AWS region where you want to deploy the cluster:

   ```
   export AWS_REGION=us-west-2
   ```

4. Export the AWS profile with the credentials you want to use to create the Kubernetes cluster:

   ```
   export AWS_PROFILE=<profile>
   ```

If using AWS ECR as your local private registry, more information can be found on the Registry Mirror Tools page.

To deploy a cluster with a custom image in a region where CAPI images are not provided, you need to use Konvoy Image Builder to create your own image for the region.

Note: For multi-tenancy, every tenant needs to be in a different AWS account to ensure they are truly independent of other tenants in order to enforce security.

GPU Prerequisites

Before you begin, you must:

- Ensure nodes provide an NVIDIA GPU
- If you are using a public cloud services, such as AWS, create an AMI with KIB using the Instructions on the KIB for GPU topic.
- If you are deploying in a pre-provisioned environment, ensure that you have created the appropriate secret for your GPU nodepool and have uploaded the appropriate artifacts to each node.

AWS with GPU: Using Node Label Auto Configuration

When using GPU nodes, it is important they have the proper label identifying them as Nvidia GPU nodes. Node feature discovery (NFD), by default labels PCI hardware as:

```
"feature.node.kubernetes.io/pci-<device label>.present": "true"
```

where `<device label>` is by default as defined in this topic:

```
< class > _ < vendor >
```

However, because there is a wide variety in devices and their assigned PCI classes, you may find that the labels assigned to your GPU nodes do not always properly identify them as containing an Nvidia GPU.

If the default detection does not work, you can manually change the daemonset that the GPU operator creates by running the following command:

```
          nodeSelector:
            feature.node.kubernetes.io/pci-< class > _ < vendor>.present: "true"
```

where `class` is any 4 digit number starting with `03xy` and the vendor for Nvidia is `10de`. If this is already deployed, you can always change the `daemonset` and change the `nodeSelector` field so that it deploys to the right nodes.

AWS with GPU: Creating an Image

Learn how to build a custom AMI for use with NKP.

About this task

This procedure describes how to use the Konvoy Image Builder (KIB) to create a Cluster API compliant Amazon Machine Image (AMI). KIB uses variable overrides to specify base image and container images to use in your new AMI.

AMI images contain configuration information and software to create a specific, pre-configured, operating environment. For example, you can create an AMI image of your current computer system settings and software. The AMI image can then be replicated and distributed, creating your computer system for other users. You can use overrides files to customize some of the components installed on your machine image. For example, having the FIPS versions of the Kubernetes components installed by KIB.

In previous NKP releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder. Explore the Customize your Image topic for more options about overrides.

The **prerequisites** to use Konvoy Image Builder are:

Procedure

1. Download the KIB bundle for your version of NKP prefixed with `konvoy-image-bundle` for your OS.
2. Check the Supported Infrastructure Operating Systems
3. Check the Supported Kubernetes Version for your Provider.
4. Create a working registry.
   - » Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
   - » Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
5. Ensure you have met the minimal set of permissions from the AWS Image Builder Book.
6. A Minimal IAM Permissions for KIB to create an Image for an AWS account using Konvoy Image Builder.

Extract the KIB Bundle

About this task

If not done previously during Konvoy Image Builder download in Prerequisites, extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION` folder. Otherwise, proceed to Build the Image below.

In previous NKP releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts with the exception of the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Containerd tarball

Procedure

1. Downloading NKP on page 16`nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.
3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the new NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`. Example command.

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   Note: For FIPS, pass the flag: `--fips`

   Note: For Red Hat Enterprise Linux (RHEL) OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`. Example command:

   ```
   export RHSM_ACTIVATION_KEY="-ci"
   ```

```
export RHSM_ORG_ID="1232131"
```

4. Follow the instructions below to build an AMI.

   Note: The `konvoy-image` binary and all supporting folders are also extracted. When run, `konvoy-image` bind mounts the current working directory (`${PWD}`) into the container to be used.

   Set the environment variables for AWS access. The following variables must be set using your credentials including required IAM:

   ```
   export AWS_ACCESS_KEY_ID
   export AWS_SECRET_ACCESS_KEY
   export AWS_DEFAULT_REGION
   ```

   If you have an override file to configure specific attributes of your AMI file, add it. Instructions for customizing an override file are found on this page: Image Overrides

Build the Image

About this task

Depending on which version of NKP you are running, steps and flags will be different. To deploy in a region where CAPI images are not provided, you need to use KIB to create your own image for the region. For a list of supported AWS regions, refer to the Published AMI information from AWS.

Procedure

Run the `konvoy-image` command to build and validate the image.

```
konvoy-image build aws images/ami/rhel-86.yaml
```

a. By default, it builds in the `us-west-2` region. to specify another region set the `--region` flag as shown in the command below.

   ```
   konvoy-image build aws --region us-east-1 images/ami/rhel-86.yaml
   ```

   Note: Ensure you have named the correct AMI image YAML file for your OS in the `konvoy-image build` command.

What to do next

After KIB provisions the image successfully, the `ami`

id is printed and written to the `packer.pkr.hcl` (Packer config) file. This file has an `artifact_id` field whose value provides the name of the AMI ID as shown in the example below. That is the `ami` you use in the `NKP create cluster` command.

```
{
  "builds": [
    {
      "name": "kib_image",
      "builder_type": "amazon-ebs",
      "build_time": 1698086886,
      "files": null,
      "artifact_id": "us-west-2:ami-04b8dfef8bd33a016",
      "packer_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05",
      "custom_data": {
        "containerd_version": "",
        "distribution": "RHEL",
        "distribution_version": "8.6",
        "kubernetes_cni_version": "",
        "kubernetes_version": "1.26.6"
```

```
      }
    }
  ],
  "last_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05"
}
```

What to do next

1. To use a custom AMI when creating your cluster, you must create that AMI using KIB first. Then perform the export and name the custom AMI for use in the command `nkp create cluster`:

   ```
   export AWS_AMI_ID=ami-<ami-id-here>
   ```

   > **Note:** Inside the sections for either Non-air-gapped or Air-gapped cluster creation, you will find the instructions for how to apply custom images.

Related Information

Procedure

- To use a local registry even in a non-air-gapped environment, download and extract the bundle. Downloading NKP on page 16 the Complete NKP Air-gapped Bundle for this release (that is. `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`) to load registry
- To view the complete set of instructions, see Load the Registry.

AWS with GPU: Creating the Management Cluster

Create an AWS Management Cluster in a non-air-gapped environment using GPU.

About this task

Use this procedure to create a self-managed **Management cluster** with NKP. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing.

If you use these instructions to create a cluster on AWS using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes.

NKP uses AWS CSI as the default storage provider. You can use a Kubernetes CSIcompatible storage solution that is suitable for production. See the Kubernetes documentation called Changing the Default Storage Class for more information.

In previous NKP releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder.

Before you begin

First you must name your cluster.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

Procedure

1. Give your cluster a unique name suitable for your environment.

2. Set the environment variable:

```
export CLUSTER_NAME=<aws-example>
```

3. Option One - Provide the ID of your AMI.

   a. Option One

   Use the example command below leaving the existing flag that provides the AMI ID: `--ami AMI_ID`

   b. Option Two - Provide a path for your AMI with the information required for image discover.

   - Where the AMI is published using your AWS Account ID: `--ami-owner AWS_ACCOUNT_ID`
   - The format or string used to search for matching AMIs and ensure it references the Kubernetes version plus the base OS name: `--ami-base-os ubuntu-20.04`
   - The base OS information: `--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*'`

   Note:

   - The AMI must be created with Konvoy Image Builder in order to use the registry mirror feature.

     ```
     export AWS_AMI_ID=<ami-...>
     ```

   - (Optional) Registry Mirror - Configure your cluster to use an existing local registry as a mirror when attempting to pull images. Below is an AWS ECR example where `REGISTRY_URL`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.:

     ```
     export REGISTRY_URL=<ecr-registry-URI>
     ```

4. Run this command to create your Kubernetes cluster by providing the image ID and using any relevant flags.

```
nkp create cluster aws \
--cluster-name=${CLUSTER_NAME} \
--ami=${AWS_AMI_ID} \
--additional-tags=owner=$(whoami) \
--with-aws-bootstrap-credentials=true \
--self-managed
```

   If providing the AMI path, use these flags in place of `AWS_AMI_ID`:

```
--ami-owner AWS_ACCOUNT_ID \
--ami-base-os ubuntu-20.04 \
--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*' \
```

   » Additional cluster creation flags based on your environment:

   » Optional Registry flag: `--registry-mirror-url=${REGISTRY_URL}`

   » Flatcar OS flag to instruct the bootstrap cluster to make changes related to the installation paths: `--os-hint flatcar`

   » HTTP or HTTPS flags if you use proxies: `--http-proxy`, `--https-proxy`, and `--no-proxy`

5. After cluster creation, create the node pool after cluster creation.

```
nkp create nodepool aws -c ${CLUSTER_NAME} \
--instance-type p2.xlarge \
--ami-id=${AMI_ID_FROM_KIB} \
```

```
--replicas=1 ${NODEPOOL_NAME} \
--kubeconfig=${CLUSTER_NAME}.conf
```

AWS with GPU: Install Kommander

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped AWS environment using GPU.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. If required: Customize your `kommander.yaml`.

   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy and External Load Balancer.

5. Only required if your cluster uses a custom AWS VPC and requires an internal load-balancer, set the traefik annotation to create an internal-facing ELB.

   ```
   apps:
     traefik:
       enabled: true
   ```

```
    values: |
      service:
        annotations:
          service.beta.kubernetes.io/aws-load-balancer-internal: "true"
```

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `NKP-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP,see the topic Configuring NKP Catalog Applications after Installing NKP.

AWS with GPU: Verifying your Installation and UI Log in
Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

```
kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
 index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
dashboard{{ "\n"}}'
```

Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

a. Rotate the password using the following command.

```
nkp experimental rotate dashboard-password
```

The output displays the new password:

```
Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Day 2 Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of NKP. The Day 2 section allows you to manage cluster operations and their application workloads to optimize your organization's productivity.

- Continue to the NKP Dashboard.

AWS with GPU: Creating Managed Clusters Using the NKP CLI

This topic explains how to continue using the CLI to create managed clusters rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed which allows it to be a Management cluster or a stand alone cluster. Subsequent new clusters are not self-managed as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects, or install the Kommander component. Those tasks are only done on Management clusters!
>
> Your new managed cluster needs to be part of a workspace under a management cluster. To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

```
kubectl get workspace -A
```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace_namespace>
```

> **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First you must name your cluster. Then you run the command to deploy it.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

```
export MANAGED_CLUSTER_NAME=<aws-additional>
```

Create a Kubernetes Cluster

About this task

The below instructions tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, it will be created in the `default` workspace, and you need to take additional steps to attach to a workspace later. For instructions on how to do this, see Attach a Kubernetes Cluster.

Procedure

1. Execute this command to create your additional Kubernetes cluster using any relevant flags. This will create a new non-self-managed cluster that can be managed by management cluster you created in the previous section.

```
nkp create cluster aws \
--cluster-name=${MANAGED_CLUSTER_NAME} \
--additional-tags=owner=$(whoami) \
--kubeconfig=<management-cluster-kubeconfig-path> \
--namespace ${WORKSPACE_NAMESPACE}
--with-aws-bootstrap-credentials=true \
--kubeconfig=<management-cluster-kubeconfig-path> \
```

> **Tip:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Clusters with HTTP or HTTPS Proxy on page 680.

2. Create the node pool after cluster creation.

```
nkp create nodepool aws -c ${CLUSTER_NAME} \
--instance-type p2.xlarge \
--ami-id=${AMI_ID_FROM_KIB} \
```

```
--replicas=1 ${NODEPOOL_NAME} \
--kubeconfig=${CLUSTER_NAME}.conf \
```

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

   ```
   echo ${MANAGED_CLUSTER_NAME}
   ```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

   ```
   nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
    ${MANAGED_CLUSTER_NAME}.conf
   ```

3. **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

   ```
   kubectl get workspaces -A
   ```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace-namespace>
   ```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

   ```
   kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
   template='{{.data.value}}{{ "\n"}}'
   ```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <your-managed-cluster-name>-kubeconfig
     labels:
       cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
   type: cluster.x-k8s.io/secret
   data:
     value: <value-you-copied-from-secret-above>
   ```

8. Create this secret in the desired workspace.

   ```
   kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
    ${WORKSPACE_NAMESPACE}
   ```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

```
kubectl get kommanderclusters -A
```

If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

AWS Air-gapped with GPU Installation

Installation instructions for installing NKP in an Amazon Web Services (AWS) air-gapped environment.

Remember, there are always more options for custom YAML Ain't Markup Language (YAML) in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Note:

- For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

  ```
  tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
  ```

- Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

  For example:

  ```
  export TMPDIR=/path/to/your/directory
  ```

AWS Prerequisites

Before you begin using Konvoy with AWS, you must:

1. Follow the steps to create permissions and roles on the Minimal Permissions and Role to Create Clusters page.

2. Create Cluster IAM Policies and Roles
3. Export the AWS region where you want to deploy the cluster:

```
export AWS_REGION=us-west-2
```

4. Export the AWS profile with the credentials you want to use to create the Kubernetes cluster:

```
export AWS_PROFILE=<profile>
```

If using AWS ECR as your local private registry, more information can be found on the Registry Mirror Tools page.

To deploy a cluster with a custom image in a region where CAPI images are not provided, you need to use Konvoy Image Builder to create your own image for the region.

> **Note:** For multi-tenancy, every tenant needs to be in a different AWS account to ensure they are truly independent of other tenants in order to enforce security.

AWS Air-gapped with GPU: Using Node Label Auto Configuration

When using GPU nodes, it is important they have the proper label identifying them as Nvidia GPU nodes. Node feature discovery (NFD), by default labels PCI hardware as:

```
"feature.node.kubernetes.io/pci-<device label>.present": "true"
```

where `<device label>` is by default as defined in this topic:

```
< class > _ < vendor >
```

However, because there is a wide variety in devices and their assigned PCI classes, you may find that the labels assigned to your GPU nodes do not always properly identify them as containing an Nvidia GPU.

If the default detection does not work, you can manually change the daemonset that the GPU operator creates by running the following command:

```
          nodeSelector:
            feature.node.kubernetes.io/pci-< class > _ < vendor>.present: "true"
```

where`class` is any 4 digit number starting with `03xy` and the vendor for Nvidia is `10de`. If this is already deployed, you can always change the `daemonset` and change the `nodeSelector` field so that it deploys to the right nodes.

AWS Air-gapped with GPU: Creating an Image

Learn how to build a custom AMI for use with NKP.

About this task

This procedure describes how to use the Konvoy Image Builder (KIB) to create a Cluster API compliant Amazon Machine Image (AMI). KIB uses variable overrides to specify base image and container images to use in your new AMI.

AMI images contain configuration information and software to create a specific, pre-configured, operating environment. For example, you can create an AMI image of your current computer system settings and software. The AMI image can then be replicated and distributed, creating your computer system for other users. You can use overrides files to customize some of the components installed on your machine image. For example, having the FIPS versions of the Kubernetes components installed by KIB. components.

In previous NKP releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder. Explore the Customize your Image topic for more options about overrides.

The **prerequisites** to use Konvoy Image Builder are:

Procedure

1. Download the KIB bundle for your version of NKP prefixed with `konvoy-image-bundle` for your OS.
2. Check the Supported Infrastructure Operating Systems
3. Check the Supported Kubernetes Version for your Provider.
4. Create a working registry.
   - » Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
   - » Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
5. Ensure you have met the minimal set of permissions from the AWS Image Builder Book.
6. A Minimal IAM Permissions for KIB to create an Image for an AWS account using Konvoy Image Builder.

Extract the KIB Bundle

About this task

If not done previously during Konvoy Image Builder download in Prerequisites, extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION` folder. Otherwise, proceed to Build the Image below.

In previous NKP releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts with the exception of the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Containerd tarball

Procedure

1. Downloading NKP on page 16`nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.
3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the new NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`. Example command.

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   Note: For FIPS, pass the flag: `--fips`

   Note: For Red Hat Enterprise Linux (RHEL) OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`. Example command:

   ```
   export RHSM_ACTIVATION_KEY="-ci"
   ```

```
export RHSM_ORG_ID="1232131"
```

4. Follow the instructions below to build an AMI.

   Note: The `konvoy-image` binary and all supporting folders are also extracted. When run, `konvoy-image` bind mounts the current working directory (`${PWD}`) into the container to be used.

   Set the environment variables for AWS access. The following variables must be set using your credentials including required IAM:

   ```
   export AWS_ACCESS_KEY_ID
   export AWS_SECRET_ACCESS_KEY
   export AWS_DEFAULT_REGION
   ```

   If you have an override file to configure specific attributes of your AMI file, add it. Instructions for customizing an override file are found on this page: Image Overrides

Build the Image

About this task

Depending on which version of NKP you are running, steps and flags will be different. To deploy in a region where CAPI images are not provided, you need to use KIB to create your own image for the region. For a list of supported AWS regions, refer to the Published AMI information from AWS.

Procedure

Run the `konvoy-image` command to build and validate the image.

```
konvoy-image build aws images/ami/rhel-86.yaml
```

a. By default, it builds in the `us-west-2` region. to specify another region set the `--region` flag as shown in the command below.

   ```
   konvoy-image build aws --region us-east-1 images/ami/rhel-86.yaml
   ```

   Note: Ensure you have named the correct AMI image YAML file for your OS in the `konvoy-image build` command.

What to do next

After KIB provisions the image successfully, the `ami`

id is printed and written to the `packer.pkr.hcl` (Packer config) file. This file has an `artifact_id` field whose value provides the name of the AMI ID as shown in the example below. That is the `ami` you use in the `NKP create cluster` command.

```
{
  "builds": [
    {
      "name": "kib_image",
      "builder_type": "amazon-ebs",
      "build_time": 1698086886,
      "files": null,
      "artifact_id": "us-west-2:ami-04b8dfef8bd33a016",
      "packer_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05",
      "custom_data": {
        "containerd_version": "",
        "distribution": "RHEL",
        "distribution_version": "8.6",
        "kubernetes_cni_version": "",
        "kubernetes_version": "1.26.6"
```

```
      }
    }
  ],
  "last_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05"
}
```

What to do next

1. To use a custom AMI when creating your cluster, you must create that AMI using KIB first. Then perform the export and name the custom AMI for use in the command `nkp create cluster`:

   ```
   export AWS_AMI_ID=ami-<ami-id-here>
   ```

   > **Note:** Inside the sections for either Non-air-gapped or Air-gapped cluster creation, you will find the instructions for how to apply custom images.

Related Information

Procedure

- To use a local registry even in a non-air-gapped environment, download and extract the bundle. Downloading NKP on page 16 the Complete NKP Air-gapped Bundle for this release (that is. `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`) to load registry
- To view the complete set of instructions, see Load the Registry.

AWS Air-gapped with GPU: Loading the Registry
Before creating an air-gapped Kubernetes cluster, you need to load the required images in a local registry for the Konvoy component.

About this task

The complete NKP air-gapped bundle is needed for an air-gapped environment but can also be used in a non-air-gapped environment. The bundle contains all the NKP components needed for an air-gapped environment installation and also to use a local registry in a non-air-gapped environment.

> **Note:** If you do not already have a local registry set up, see the Local Registry Tools page for more information.

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images is required. This registry must be accessible from both the bastion machine and either the AWS EC2 instances (if deploying to AWS) or other machines that will be created for the Kubernetes cluster.

Procedure

1. If not already done in prerequisites, download the air-gapped bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
   ```

2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. EX: For the bootstrap cluster, change your directory to the `nkp-<version>` directory similar to example below depending on your current location

   ```
   cd nkp-v2.12.0
   ```

3. Set an environment variable with your registry address and any other needed variables using this command.

```
export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
export REGISTRY_USERNAME=<username>
export REGISTRY_PASSWORD=<password>
export REGISTRY_CA=<path to the cacert file on the bastion>
```

4. Execute the following command to load the air-gapped image bundle into your private registry using any of the relevant flags to apply variables above.

```
nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-
registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-
password=${REGISTRY_PASSWORD}
```

> **Note:** It may take some time to push all the images to your image registry, depending on the performance of the network between the machine you are running the script on and the registry.

> **Important:** To increase Docker Hub's rate limit use your Docker Hub credentials when creating the cluster, by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

5. Load the Kommander component images to your private registry using the command.

```
nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-
registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-
password=${REGISTRY_PASSWORD}
```

Optional: This step is required only if you have an Ultimate license.

For NKP Catalog Applications available with the Ultimate license, perform this image load by running the following command to load the `dkp-catalog-applications` image bundle into your private registry:

```
nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-
v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME}
 --to-registry-password=${REGISTRY_PASSWORD}
```

6. On the Bastion, load the Konvoy bootstrap image, using either the Docker or Podman command.

Docker example:

```
docker load -i konvoy-bootstrap-image-v2.12.0.tar
```

Podman example:

```
podman load -i konvoy-bootstrap-image-v2.12.0.tar
podman image tag konvoy-bootstrap:2.12.0 docker.io/konvoy-bootstrap:v2.12.0
```

AWS Air-gapped with GPU: Creating the Management Cluster

Create an AWS Management Cluster in an air-gapped environment using GPU.

About this task

Use this procedure to create a self-managed **Management cluster** with NKP. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing.

If you use these instructions to create a cluster on AWS using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes.

NKP uses AWS CSI as the default storage provider. You can use a Kubernetes CSI compatible storage solution that is suitable for production. See the Kubernetes documentation called Changing the Default Storage Class for more information.

In previous NKP releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder.

Before you begin

First you must name your cluster.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable:

   ```
   export CLUSTER_NAME=<aws-example>
   ```

3. Export variables for the existing infrastructure details.

   ```
   export AWS_VPC_ID=<vpc-...>
   export AWS_SUBNET_IDS=<subnet-...,subnet-...,subnet-...>
   export AWS_ADDITIONAL_SECURITY_GROUPS=<sg-...>
   export AWS_AMI_ID=<ami-...>
   ```

   `AWS_VPC_ID`: the VPC ID where the cluster will be created. The VPC requires the following AWS VPC Endpoints to be already present

4. There are two approaches to supplying the ID of your AMI. Either provide the ID of the AMI or provide a way for NKP to discover the AMI using location, format and OS information.

   a. Option One - Provide the ID of your AMI.
   Use the example command below leaving the existing flag that provides the AMI ID: `--ami AMI_ID`

   b. Option Two - Provide a path for your AMI with the information required for image discover.

   - Where the AMI is published using your AWS Account ID: `--ami-owner AWS_ACCOUNT_ID`
   - The format or string used to search for matching AMIs and ensure it references the Kubernetes version plus the base OS name: `--ami-base-os ubuntu-20.04`
   - The base OS information: `--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*'`

   > **Note:**
   >
   > - The AMI must be created with Konvoy Image Builder in order to use the registry mirror feature.
   >
   >   ```
   >   export AWS_AMI_ID=<ami-...>
   >   ```
   >
   > - (Optional) Registry Mirror - Configure your cluster to use an existing local registry as a mirror when attempting to pull images. Below is an AWS ECR example where

`REGISTRY_URL`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.:

```
export REGISTRY_URL=<ecr-registry-URI>
```

5. Run this command to create your Kubernetes cluster by providing the image ID and using any relevant flags.

```
nkp create cluster aws --cluster-name=${CLUSTER_NAME} \
--additional-tags=owner=$(whoami) \
--with-aws-bootstrap-credentials=true \
--vpc-id=${AWS_VPC_ID} \
--ami=${AWS_AMI_ID} \
--subnet-ids=${AWS_SUBNET_IDS} \
--internal-load-balancer=true \
--additional-security-group-ids=${AWS_ADDITIONAL_SECURITY_GROUPS} \
--registry-mirror-url=${REGISTRY_URL} \
--registry-mirror-cacert=${REGISTRY_CA} \
--registry-mirror-username=${REGISTRY_USERNAME} \
--registry-mirror-password=${REGISTRY_PASSWORD} \
--kubernetes-version=v1.29.6+fips.0 \
--etcd-version=3.5.10+fips.0 \
--kubernetes-image-repository=docker.io/mesosphere \
--self-managed
```

If providing the AMI path, use these flags in place of `AWS_AMI_ID`:

```
--ami-owner AWS_ACCOUNT_ID \
--ami-base-os ubuntu-20.04 \
--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*' \
```

» Additional cluster creation flags based on your environment:

» Optional Registry flag: `--registry-mirror-url=${REGISTRY_URL}`

» Flatcar OS flag to instruct the bootstrap cluster to make changes related to the installation paths: `--os-hint flatcar`

» HTTP or HTTPS flags if you use proxies: `--http-proxy`, `--https-proxy`, and `--no-proxy`

AWS Air-gapped with GPU: Install Kommander

This section provides installation instructions for the Kommander component of NKP in an air-gapped AWS environment with GPU.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. If required: Customize your `kommander.yaml`.

   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy and External Load Balancer.

5. Only required if your cluster uses a custom AWS VPC and requires an internal load-balancer, set the traefik annotation to create an internal-facing ELB.

   ```
   apps:
     traefik:
       enabled: true
       values: |
         service:
           annotations:
             service.beta.kubernetes.io/aws-load-balancer-internal: "true"
   ```

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `nkp-catalog-appliations`.

   ```
   apiVersion: config.kommander.mesosphere.io/v1alpha1
   kind: Installation
   catalog:
     repositories:
       - name: dkp-catalog-applications
         labels:
           kommander.d2iq.io/project-default-catalog-repository: "true"
           kommander.d2iq.io/workspace-default-catalog-repository: "true"
           kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
         gitRepositorySpec:
           url: https://github.com/mesosphere/dkp-catalog-applications
           ref:
             tag: v2.12.0
   ```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see the topic Configuring NKP Catalog Applications after Installing NKP.

AWS Air-gapped with GPU: Verifying your Installation and UI Log in
Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      nkp experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Day 2 Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of NKP. The Day 2 section allows you to manage cluster operations and their application workloads to optimize your organization’s productivity.

- Continue to the NKP Dashboard.

AWS Air-gapped GPU: Creating Managed Clusters Using the NKP CLI
This topic explains how to continue using the CLI to create managed clusters rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed which allows it to be a Management cluster or a stand alone cluster. Subsequent new clusters are not self-managed as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects, or install the Kommander component. Those tasks are only done on Management clusters!
>
> Your new managed cluster needs to be part of a workspace under a management cluster. To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

```
kubectl get workspace -A
```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace_namespace>
```

> **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First you must name your cluster. Then you run the command to deploy it.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

```
export MANAGED_CLUSTER_NAME=<aws-additional>
```

Create a Kubernetes Cluster

About this task

The below instructions tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, it will be created in the `default` workspace, and you need

to take additional steps to attach to a workspace later. For instructions on how to do this, see Attach a Kubernetes Cluster.

Procedure

1. Execute this command to create your additional Kubernetes cluster using any relevant flags. This will create a new non-self-managed cluster that can be managed by management cluster you created in the previous section.

```
nkp create cluster aws --cluster-name=${MANAGED_CLUSTER_NAME} \
--additional-tags=owner=$(whoami) \
--namespace ${WORKSPACE_NAMESPACE}
--with-aws-bootstrap-credentials=true \
--vpc-id=${AWS_VPC_ID} \
--ami=${AWS_AMI_ID} \
--subnet-ids=${AWS_SUBNET_IDS} \
--internal-load-balancer=true \
--additional-security-group-ids=${AWS_ADDITIONAL_SECURITY_GROUPS} \
--registry-mirror-url=${REGISTRY_URL} \
--registry-mirror-cacert=${REGISTRY_CA} \
--registry-mirror-username=${REGISTRY_USERNAME} \
--registry-mirror-password=${REGISTRY_PASSWORD} \
--kubeconfig=<management-cluster-kubeconfig-path> \
```

> **Tip:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Clusters with HTTP or HTTPS Proxy on page 680.

2. Create the node pool after cluster creation.

```
nkp create nodepool aws -c ${CLUSTER_NAME} \
--instance-type p2.xlarge \
--ami-id=${AMI_ID_FROM_KIB} \
--replicas=1 ${NODEPOOL_NAME} \
--kubeconfig=${CLUSTER_NAME}.conf
```

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

```
echo ${MANAGED_CLUSTER_NAME}
```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

```
nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
 ${MANAGED_CLUSTER_NAME}.conf
```

3. > **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

```
kubectl get workspaces -A
```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace-namespace>
```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

```
kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
template='{{.data.value}}{{ "\n"}}'
```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

```
apiVersion: v1
kind: Secret
metadata:
  name: <your-managed-cluster-name>-kubeconfig
  labels:
    cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
type: cluster.x-k8s.io/secret
data:
  value: <value-you-copied-from-secret-above>
```

8. Create this secret in the desired workspace.

```
kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
 ${WORKSPACE_NAMESPACE}
```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

```
kubectl get kommanderclusters -A
```

If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

EKS Installation Options

For an environment that is on the EKS Infrastructure, install options based on those environment variables are provided for you in this location.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operative in the most common scenarios.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

> **Note:** An EKS cluster cannot be a Management or Pro cluster. To install NKP on your EKS cluster, first, ensure you have a Management cluster with NKP and the Kommander component installed that handles the life cycle of your EKS cluster.

In order to install Kommander, you need to have CAPI components, cert-manager, etc on a self-managed cluster. The CAPI components mean you can control the life cycle of the cluster, and other clusters. However, because EKS is semi-managed by AWS, the EKS clusters are under AWS control and don't have those components. Therefore, Kommander will not be installed.

EKS Installation

This installation provides instructions to install NKP in an AWS non-air-gapped environment.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

> **Note:** Ensure that the `KUBECONFIG` environment variable is set to the Management cluster by running `export KUBECONFIG=<Management_cluster_kubeconfig>.conf`.

AWS Prerequisites

Before you begin using Konvoy with AWS, you must:

1. A Management cluster with the Kommander component installed.
2. You have a valid AWS account with credentials configured that can manage CloudFormation Stacks, IAM Policies, and IAM Roles.
3. You will need to have the AWS CLI utility installed.
4. Install aws-iam-authenticator. This binary is used to access your cluster using kubectl.

> **Note:** In order to install Kommander, you need to have CAPI components, cert-manager, etc on a self-managed cluster. The CAPI components mean you can control the life cycle of the cluster, and other clusters. However, because EKS is semi-managed by AWS, the EKS clusters are under AWS control and don't have those components. Therefore, Kommander will not be installed and these clusters will be attached to the management cluster.

If you are found using AWS ECR as your local private registry; more information is available on the Registry Mirror Tools page.

To deploy a cluster with a custom image in a region where CAPI images are not provided, you need to use Konvoy Image Builder to create your image for the region.

EKS: Minimal User Permission for Cluster Creation

The following is a CloudFormation stack which adds a policy named eks-bootstrapper to manage EKS cluster to the NKP-bootstrapper-role created by the CloudFormation stack for AWS in the Minimal Permissions and Role to Create Cluster section. Consult the Leveraging the Role section for an example of how to use this role and how a system administrator wants to expose using the permissions.

EKS CloudFormation Stack:

> **Note:** If your role is not named `NKP-bootstrapper-role` **change the parameter on line 6 of the file**.

```
AWSTemplateFormatVersion: 2010-09-09
Parameters:
  existingBootstrapperRole:
    Type: CommaDelimitedList
    Description: 'Name of existing minimal role you want to add to add EKS cluster
 management permissions to'
    Default: NKP-bootstrapper-role
Resources:
  EKSMinimumPermissions:
    Properties:
      Description: Minimal user policy to manage eks clusters
      ManagedPolicyName: eks-bootstrapper
      PolicyDocument:
        Statement:
          - Action:
              - 'ssm:GetParameter'
            Effect: Allow
            Resource:
              - 'arn:*:ssm:*:*:parameter/aws/service/eks/optimized-ami/*'
          - Action:
              - 'iam:CreateServiceLinkedRole'
            Condition:
              StringLike:
                'iam:AWSServiceName': eks.amazonaws.com
            Effect: Allow
            Resource:
              - >-
                arn:*:iam::*:role/aws-service-role/eks.amazonaws.com/
AWSServiceRoleForAmazonEKS
          - Action:
              - 'iam:CreateServiceLinkedRole'
            Condition:
              StringLike:
                'iam:AWSServiceName': eks-nodegroup.amazonaws.com
            Effect: Allow
            Resource:
              - >-
                arn:*:iam::*:role/aws-service-role/eks-nodegroup.amazonaws.com/
AWSServiceRoleForAmazonEKSNodegroup
          - Action:
              - 'iam:CreateServiceLinkedRole'
            Condition:
              StringLike:
                'iam:AWSServiceName': eks-fargate.amazonaws.com
            Effect: Allow
            Resource:
              - >-
                arn:aws:iam::*:role/aws-service-role/eks-fargate-pods.amazonaws.com/
AWSServiceRoleForAmazonEKSForFargate
          - Action:
              - 'iam:GetRole'
```

```
      - 'iam:ListAttachedRolePolicies'
    Effect: Allow
    Resource:
      - 'arn:*:iam::*:role/*'
  - Action:
      - 'iam:GetPolicy'
    Effect: Allow
    Resource:
      - 'arn:aws:iam::aws:policy/AmazonEKSClusterPolicy'
  - Action:
      - 'eks:DescribeCluster'
      - 'eks:ListClusters'
      - 'eks:CreateCluster'
      - 'eks:TagResource'
      - 'eks:UpdateClusterVersion'
      - 'eks:DeleteCluster'
      - 'eks:UpdateClusterConfig'
      - 'eks:UntagResource'
      - 'eks:UpdateNodegroupVersion'
      - 'eks:DescribeNodegroup'
      - 'eks:DeleteNodegroup'
      - 'eks:UpdateNodegroupConfig'
      - 'eks:CreateNodegroup'
      - 'eks:AssociateEncryptionConfig'
      - 'eks:ListIdentityProviderConfigs'
      - 'eks:AssociateIdentityProviderConfig'
      - 'eks:DescribeIdentityProviderConfig'
      - 'eks:DisassociateIdentityProviderConfig'
    Effect: Allow
    Resource:
      - 'arn:*:eks:*:*:cluster/*'
      - 'arn:*:eks:*:*:nodegroup/*/*/*'
  - Action:
      - 'ec2:AssociateVpcCidrBlock'
      - 'ec2:DisassociateVpcCidrBlock'
      - 'eks:ListAddons'
      - 'eks:CreateAddon'
      - 'eks:DescribeAddonVersions'
      - 'eks:DescribeAddon'
      - 'eks:DeleteAddon'
      - 'eks:UpdateAddon'
      - 'eks:TagResource'
      - 'eks:DescribeFargateProfile'
      - 'eks:CreateFargateProfile'
      - 'eks:DeleteFargateProfile'
    Effect: Allow
    Resource:
      - '*'
  - Action:
      - 'iam:PassRole'
    Condition:
      StringEquals:
        'iam:PassedToService': eks.amazonaws.com
    Effect: Allow
    Resource:
      - '*'
  - Action:
      - 'kms:CreateGrant'
      - 'kms:DescribeKey'
    Condition:
      'ForAnyValue:StringLike':
        'kms:ResourceAliases': alias/cluster-api-provider-aws-*
```

```
          Effect: Allow
          Resource:
            - '*'
      Version: 2012-10-17
    Roles: !Ref existingBootstrapperRole
  Type: 'AWS::IAM::ManagedPolicy'
```

To create the resources in the cloudformation stack, copy the contents above into a file. Before executing the following command, replace `MYFILENAME.yaml` and `MYSTACKNAME` with the intended values for your system when running the command to create the AWS cloudformation stack:

```
aws cloudformation create-stack --template-body=file://MYFILENAME.yaml --stack-
name=MYSTACKNAME --capabilities CAPABILITY_NAMED_IAM
```

EKS: Cluster IAM Policies and Roles

This section guides a NKP user in creating IAM Policies and Instance Profiles that governs who has access to the cluster. The IAM Role is used by the cluster's control plane and worker nodes using the provided AWS CloudFormation Stack specific to EKS. This CloudFormation Stack has additional permissions that are used to delegate access roles for other users.

Prerequisites from AWS:

Before you begin, ensure you have met the AWS prerequisites:

- The user you delegate from your role must have a minimum set of permissions, see User Roles and Instance Profiles page for AWS.
- Create the Cluster IAM Policies in your AWS account.

EKS IAM Artifacts

Policies

- `controllers-eks.cluster-api-provider-aws.sigs.k8s.io` - enumerates the Actions required by the workload cluster to create and modify EKS clusters in the user's AWS Account. It is attached to the existing `control-plane.cluster-api-provider-aws.sigs.k8s.io` role
- `eks-nodes.cluster-api-provider-aws.sigs.k8s.io` - enumerates the Actions required by the EKS workload cluster's worker machines. It is attached to the existing `nodes.cluster-api-provider-aws.sigs.k8s.io`

Roles

- `eks-controlplane.cluster-api-provider-aws.sigs.k8s.io` - is the Role associated with EKS cluster control planes

NOTE: `control-plane.cluster-api-provider-aws.sigs.k8s.io` and `nodes.cluster-api-provider-aws.sigs.k8s.io` roles were created by Cluster IAM Policies and Roles in AWS.

Below is a CloudFormation stack that includes IAM policies and roles required to setup EKS Clusters.

> **Note:** To create the resources in the CloudFormation stack, copy the contents above into a file and execute the following command after replacing `MYFILENAME.yaml` and `MYSTACKNAME` with the intended values:
>
> ```
> aws cloudformation create-stack
>             --template-body=file://MYFILENAME.yaml --stack-name=MYSTACKNAME --
> capabilities
>             CAPABILITY_NAMED_IAM
> ```

```
AWSTemplateFormatVersion: 2010-09-09
Parameters:
```

```
  existingControlPlaneRole:
    Type: CommaDelimitedList
    Description: 'Names of existing Control Plane Role you want to add to the newly
 created EKS Managed Policy for AWS cluster API controllers'
    Default: control-plane.cluster-api-provider-aws.sigs.k8s.io
  existingNodeRole:
    Type: CommaDelimitedList
    Description: 'ARN of the Nodes Managed Policy to add to the role for nodes'
    Default: nodes.cluster-api-provider-aws.sigs.k8s.io
Resources:
  AWSIAMManagedPolicyControllersEKS:
    Properties:
      Description: For the Kubernetes Cluster API Provider AWS Controllers
      ManagedPolicyName: controllers-eks.cluster-api-provider-aws.sigs.k8s.io
      PolicyDocument:
        Statement:
          - Action:
              - 'ssm:GetParameter'
            Effect: Allow
            Resource:
              - 'arn:*:ssm:*:*:parameter/aws/service/eks/optimized-ami/*'
          - Action:
              - 'iam:CreateServiceLinkedRole'
            Condition:
              StringLike:
                'iam:AWSServiceName': eks.amazonaws.com
            Effect: Allow
            Resource:
              - >-
                arn:*:iam::*:role/aws-service-role/eks.amazonaws.com/
AWSServiceRoleForAmazonEKS
          - Action:
              - 'iam:CreateServiceLinkedRole'
            Condition:
              StringLike:
                'iam:AWSServiceName': eks-nodegroup.amazonaws.com
            Effect: Allow
            Resource:
              - >-
                arn:*:iam::*:role/aws-service-role/eks-nodegroup.amazonaws.com/
AWSServiceRoleForAmazonEKSNodegroup
          - Action:
              - 'iam:CreateServiceLinkedRole'
            Condition:
              StringLike:
                'iam:AWSServiceName': eks-fargate.amazonaws.com
            Effect: Allow
            Resource:
              - >-
                arn:aws:iam::*:role/aws-service-role/eks-fargate-pods.amazonaws.com/
AWSServiceRoleForAmazonEKSForFargate
          - Action:
              - 'iam:GetRole'
              - 'iam:ListAttachedRolePolicies'
            Effect: Allow
            Resource:
              - 'arn:*:iam::*:role/*'
          - Action:
              - 'iam:GetPolicy'
            Effect: Allow
            Resource:
              - 'arn:aws:iam::aws:policy/AmazonEKSClusterPolicy'
```

```
        - Action:
            - 'eks:DescribeCluster'
            - 'eks:ListClusters'
            - 'eks:CreateCluster'
            - 'eks:TagResource'
            - 'eks:UpdateClusterVersion'
            - 'eks:DeleteCluster'
            - 'eks:UpdateClusterConfig'
            - 'eks:UntagResource'
            - 'eks:UpdateNodegroupVersion'
            - 'eks:DescribeNodegroup'
            - 'eks:DeleteNodegroup'
            - 'eks:UpdateNodegroupConfig'
            - 'eks:CreateNodegroup'
            - 'eks:AssociateEncryptionConfig'
            - 'eks:ListIdentityProviderConfigs'
            - 'eks:AssociateIdentityProviderConfig'
            - 'eks:DescribeIdentityProviderConfig'
            - 'eks:DisassociateIdentityProviderConfig'
          Effect: Allow
          Resource:
            - 'arn:*:eks:*:*:cluster/*'
            - 'arn:*:eks:*:*:nodegroup/*/*/*'
        - Action:
            - 'ec2:AssociateVpcCidrBlock'
            - 'ec2:DisassociateVpcCidrBlock'
            - 'eks:ListAddons'
            - 'eks:CreateAddon'
            - 'eks:DescribeAddonVersions'
            - 'eks:DescribeAddon'
            - 'eks:DeleteAddon'
            - 'eks:UpdateAddon'
            - 'eks:TagResource'
            - 'eks:DescribeFargateProfile'
            - 'eks:CreateFargateProfile'
            - 'eks:DeleteFargateProfile'
          Effect: Allow
          Resource:
            - '*'
        - Action:
            - 'iam:PassRole'
          Condition:
            StringEquals:
              'iam:PassedToService': eks.amazonaws.com
          Effect: Allow
          Resource:
            - '*'
        - Action:
            - 'kms:CreateGrant'
            - 'kms:DescribeKey'
          Condition:
            'ForAnyValue:StringLike':
              'kms:ResourceAliases': alias/cluster-api-provider-aws-*
          Effect: Allow
          Resource:
            - '*'
      Version: 2012-10-17
    Roles: !Ref existingControlPlaneRole
  Type: 'AWS::IAM::ManagedPolicy'
AWSIAMManagedEKSNodesPolicy:
  Properties:
    Description: Additional Policies to nodes role to work for EKS
```

```
    ManagedPolicyName: eks-nodes.cluster-api-provider-aws.sigs.k8s.io
    PolicyDocument:
      Statement:
        - Action:
            - "ec2:AssignPrivateIpAddresses"
            - "ec2:AttachNetworkInterface"
            - "ec2:CreateNetworkInterface"
            - "ec2:DeleteNetworkInterface"
            - "ec2:DescribeInstances"
            - "ec2:DescribeTags"
            - "ec2:DescribeNetworkInterfaces"
            - "ec2:DescribeInstanceTypes"
            - "ec2:DetachNetworkInterface"
            - "ec2:ModifyNetworkInterfaceAttribute"
            - "ec2:UnassignPrivateIpAddresses"
          Effect: Allow
          Resource:
            - '*'
        - Action:
            - ec2:CreateTags
          Effect: Allow
          Resource:
            - arn:aws:ec2:*:*:network-interface/*
        - Action:
            - "ec2:DescribeInstances"
            - "ec2:DescribeInstanceTypes"
            - "ec2:DescribeRouteTables"
            - "ec2:DescribeSecurityGroups"
            - "ec2:DescribeSubnets"
            - "ec2:DescribeVolumes"
            - "ec2:DescribeVolumesModifications"
            - "ec2:DescribeVpcs"
            - "eks:DescribeCluster"
          Effect: Allow
          Resource:
            - '*'
      Version: 2012-10-17
    Roles: !Ref existingNodeRole
  Type: 'AWS::IAM::ManagedPolicy'
AWSIAMRoleEKSControlPlane:
  Properties:
    AssumeRolePolicyDocument:
      Statement:
        - Action:
            - 'sts:AssumeRole'
          Effect: Allow
          Principal:
            Service:
              - eks.amazonaws.com
      Version: 2012-10-17
    ManagedPolicyArns:
      - 'arn:aws:iam::aws:policy/AmazonEKSClusterPolicy'
    RoleName: eks-controlplane.cluster-api-provider-aws.sigs.k8s.io
  Type: 'AWS::IAM::Role'
```

Add EKS CSI Policy

AWS CloudFormation does not support attaching an existing IAM Policy to an existing IAM Role. Add the necessary IAM policy to your worker instance profile using the `aws` CLI:

```
aws iam attach-role-policy --role-name
             nodes.cluster-api-provider-aws.sigs.k8s.io --policy-arn
             arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy
```

In other infrastructures, you create an image next. However, AWS EKS best practices discourage building custom images. The Amazon EKS Optimized AMI is the preferred way to deploy containers for EKS. If the image is customized, it breaks some of the autoscaling and security capabilities of EKS. Therefore, you will proceed to creating your EKS cluster.

EKS: Create an EKS Cluster

EKS clusters can be created from the UI or CLI, but require permissions first.

About this task

When creating a Managed cluster on your EKS infrastructure, you can choose from multiple configuration types. The steps for creating and accessing your cluster are listed below after setting minimal permissions and creating IAM Policies and Roles. For more information about custom EKS configurations, refer to the EKS Infrastructure under Custom Installation and Additional Infrastructure Tools.

Access your Cluster

Use the previously installed aws-iam-authenticator to access your cluster using kubectl. Amazon EKS uses IAM to provide authentication to your Kubernetes cluster.

Procedure

1. Export the AWS region where you want to deploy the cluster.

   ```
   export AWS_REGION=us-west-2
   ```

2. Export the AWS profile with the credentials you want to use to create the Kubernetes cluster.

   ```
   export AWS_PROFILE=<profile>
   ```

3. Name your cluster

   Give your cluster a unique name suitable for your environment. In AWS it is critical that the name is unique, as no two clusters in the same AWS account can have the same name.

4. Set the environment variable.

   ```
   export CLUSTER_NAME=<aws-example>
   ```

   Note: The cluster name may only contain the following characters: `a-z`, `0-9`, `.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

Known Limitations

About this task

Be aware of these limitations in the current release of Konvoy:

Procedure

- The Konvoy version used to create a workload cluster must match the Konvoy version used to delete a workload cluster.
- EKS clusters cannot be Self-managed.
- Konvoy supports deploying one workload cluster. Konvoy generates a set of objects for one Node Pool.
- Konvoy does not validate edits to cluster objects.

Create an EKS Cluster from the CLI

Create an EKS cluster using the cli rather than in the UI.

About this task

If you prefer to work in the shell, you can continue by creating a new cluster following these steps. If you prefer to log in to the NKP UI, you can create a new cluster from there using the steps on this page: Create an EKS Cluster from the NKP UI

Procedure

1. Set the environment variable to the name you assigned this cluster.

```
export CLUSTER_NAME=eks-example
```

2. Make sure your AWS credentials are up-to-date. Refresh the credentials command is only necessary if you are using Access Keys. For more information, see Leverage the NKP Create Cluster Role on page 799 otherwise, if you are using role-based authentication on a bastion host, proceed to step 3.

```
nkp update bootstrap credentials aws
```

3. Create the cluster.

```
nkp create cluster eks \
  --cluster-name=${CLUSTER_NAME} \
  --additional-tags=owner=$(whoami)
```

4. Inspect or edit the cluster objects. Familiarize yourself with Cluster API before editing the cluster objects as edits can prevent the cluster from deploying successfully. See Customizing CAPI Clusters.

5. Wait for the cluster control-plane to be ready.

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=20m
```

   The `READY` status will become `True` after the cluster control-plane becomes ready in one of the following steps.

6. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. Konvoy provides a command to describe the current status of the cluster.

```
nkp describe cluster -c ${CLUSTER_NAME}
```

```
NAME                                                                  READY  SEVERITY
 REASON  SINCE  MESSAGE
Cluster/eks-example                                                   True
       10m
##ControlPlane - AWSManagedControlPlane/eks-example-control-plane  True
       10m
##Workers
  ##MachineDeployment/eks-example-md-0                                 True
       26s
 ##Machine/eks-example-md-0-78fcd7c7b7-66ntt                    True
    84s
 ##Machine/eks-example-md-0-78fcd7c7b7-b9qmc                    True
    84s
 ##Machine/eks-example-md-0-78fcd7c7b7-v5vfq                    True
    84s
 ##Machine/eks-example-md-0-78fcd7c7b7-zl6m2                    True
    84s
```

7. As they progress, the controllers also create Events. List the Events using this command.

```
kubectl get events | grep ${CLUSTER_NAME}
```

For brevity, the example uses `grep`. It is also possible to use separate commands to get Events for specific objects. For example, `kubectl get events --field-selector involvedObject.kind="AWSCluster"` and `kubectl get events --field-selector involvedObject.kind="AWSMachine"`.

```
46m         Normal   SuccessfulCreateVPC
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed VPC
 "vpc-05e775702092abf09"
46m         Normal   SuccessfulSetVPCAttributes
 awsmanagedcontrolplane/eks-example-control-plane   Set managed VPC attributes for
 "vpc-05e775702092abf09"
46m         Normal   SuccessfulCreateSubnet
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Subnet
 "subnet-0419dd3f2dfd95ff8"
46m         Normal   SuccessfulModifySubnetAttributes
 awsmanagedcontrolplane/eks-example-control-plane   Modified managed Subnet
 "subnet-0419dd3f2dfd95ff8" attributes
46m         Normal   SuccessfulCreateSubnet
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Subnet
 "subnet-0e724b128e3113e47"
46m         Normal   SuccessfulCreateSubnet
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Subnet
 "subnet-06b2b31ea6a8d3962"
46m         Normal   SuccessfulModifySubnetAttributes
 awsmanagedcontrolplane/eks-example-control-plane   Modified managed Subnet
 "subnet-06b2b31ea6a8d3962" attributes
46m         Normal   SuccessfulCreateSubnet
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Subnet
 "subnet-0626ce238be32bf98"
46m         Normal   SuccessfulCreateSubnet
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Subnet
 "subnet-0f53cf59f83177800"
46m         Normal   SuccessfulModifySubnetAttributes
 awsmanagedcontrolplane/eks-example-control-plane   Modified managed Subnet
 "subnet-0f53cf59f83177800" attributes
46m         Normal   SuccessfulCreateSubnet
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Subnet
 "subnet-0878478f6bbf153b2"
46m         Normal   SuccessfulCreateInternetGateway
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Internet
 Gateway "igw-09fb52653949d4579"
46m         Normal   SuccessfulAttachInternetGateway
 awsmanagedcontrolplane/eks-example-control-plane   Internet Gateway
 "igw-09fb52653949d4579" attached to VPC "vpc-05e775702092abf09"
46m         Normal   SuccessfulCreateNATGateway
 awsmanagedcontrolplane/eks-example-control-plane   Created new NAT Gateway
 "nat-06356aac28079952d"
46m         Normal   SuccessfulCreateNATGateway
 awsmanagedcontrolplane/eks-example-control-plane   Created new NAT Gateway
 "nat-0429d1cd9d956bf35"
46m         Normal   SuccessfulCreateNATGateway
 awsmanagedcontrolplane/eks-example-control-plane   Created new NAT Gateway
 "nat-059246bcc9d4e88e7"
46m         Normal   SuccessfulCreateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Created managed RouteTable
 "rtb-01689c719c484fd3c"
46m         Normal   SuccessfulCreateRoute
 awsmanagedcontrolplane/eks-example-control-plane   Created route {...
```

```
46m         Normal   SuccessfulAssociateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Associated managed RouteTable
 "rtb-01689c719c484fd3c" with subnet "subnet-0419dd3f2dfd95ff8"
46m         Normal   SuccessfulCreateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Created managed RouteTable
 "rtb-065af81b9752eeb69"
46m         Normal   SuccessfulCreateRoute
 awsmanagedcontrolplane/eks-example-control-plane   Created route {...
46m         Normal   SuccessfulAssociateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Associated managed RouteTable
 "rtb-065af81b9752eeb69" with subnet "subnet-0e724b128e3113e47"
46m         Normal   SuccessfulCreateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Created managed RouteTable
 "rtb-03eeff810a89afc98"
46m         Normal   SuccessfulCreateRoute
 awsmanagedcontrolplane/eks-example-control-plane   Created route {...
46m         Normal   SuccessfulAssociateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Associated managed RouteTable
 "rtb-03eeff810a89afc98" with subnet "subnet-06b2b31ea6a8d3962"
46m         Normal   SuccessfulCreateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Created managed RouteTable
 "rtb-0fab36f8751fdee73"
46m         Normal   SuccessfulCreateRoute
 awsmanagedcontrolplane/eks-example-control-plane   Created route {...
46m         Normal   SuccessfulAssociateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Associated managed RouteTable
 "rtb-0fab36f8751fdee73" with subnet "subnet-0626ce238be32bf98"
46m         Normal   SuccessfulCreateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Created managed RouteTable
 "rtb-0e5c9c7bbc3740a0f"
46m         Normal   SuccessfulCreateRoute
 awsmanagedcontrolplane/eks-example-control-plane   Created route {...
46m         Normal   SuccessfulAssociateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Associated managed RouteTable
 "rtb-0e5c9c7bbc3740a0f" with subnet "subnet-0f53cf59f83177800"
46m         Normal   SuccessfulCreateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Created managed RouteTable
 "rtb-0bf58eb5f73c387af"
46m         Normal   SuccessfulCreateRoute
 awsmanagedcontrolplane/eks-example-control-plane   Created route {...
46m         Normal   SuccessfulAssociateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Associated managed RouteTable
 "rtb-0bf58eb5f73c387af" with subnet "subnet-0878478f6bbf153b2"
46m         Normal   SuccessfulCreateSecurityGroup
 awsmanagedcontrolplane/eks-example-control-plane   Created managed SecurityGroup
 "sg-0b045c998a120a1b2" for Role "node-eks-additional"
46m         Normal   InitiatedCreateEKSControlPlane
 awsmanagedcontrolplane/eks-example-control-plane   Initiated creation of a new EKS
 control plane default_eks-example-control-plane
37m         Normal   SuccessfulCreateEKSControlPlane
 awsmanagedcontrolplane/eks-example-control-plane   Created new EKS control plane
 default_eks-example-control-plane
37m         Normal   SucessfulCreateKubeconfig
 awsmanagedcontrolplane/eks-example-control-plane   Created kubeconfig for cluster
 "eks-example"
37m         Normal   SucessfulCreateUserKubeconfig
 awsmanagedcontrolplane/eks-example-control-plane   Created user kubeconfig for
 cluster "eks-example"
27m         Normal   SuccessfulCreate                                awsmachine/
eks-example-md-0-4t9nc                 Created new node instance with id
 "i-0aecc1897c93df740"
```

```
26m         Normal   SuccessfulDeleteEncryptedBootstrapDataSecrets   awsmachine/eks-
example-md-0-4t9nc                 AWS Secret entries containing userdata deleted
26m         Normal   SuccessfulSetNodeRef                           machine/eks-
example-md-0-78fcd7c7b7-fn7x9          ip-10-0-88-24.us-west-2.compute.internal
26m         Normal   SuccessfulSetNodeRef                           machine/eks-
example-md-0-78fcd7c7b7-g64nv          ip-10-0-110-219.us-west-2.compute.internal
26m         Normal   SuccessfulSetNodeRef                           machine/eks-
example-md-0-78fcd7c7b7-gwc5j          ip-10-0-101-161.us-west-2.compute.internal
26m         Normal   SuccessfulSetNodeRef                           machine/eks-
example-md-0-78fcd7c7b7-j58s4          ip-10-0-127-49.us-west-2.compute.internal
46m         Normal   SuccessfulCreate                               machineset/eks-
example-md-0-78fcd7c7b7            Created machine "eks-example-md-0-78fcd7c7b7-
fn7x9"
46m         Normal   SuccessfulCreate                               machineset/eks-
example-md-0-78fcd7c7b7            Created machine "eks-example-md-0-78fcd7c7b7-
g64nv"
46m         Normal   SuccessfulCreate                               machineset/eks-
example-md-0-78fcd7c7b7            Created machine "eks-example-md-0-78fcd7c7b7-
j58s4"
46m         Normal   SuccessfulCreate                               machineset/eks-
example-md-0-78fcd7c7b7            Created machine "eks-example-md-0-78fcd7c7b7-
gwc5j"
27m         Normal   SuccessfulCreate                               awsmachine/
eks-example-md-0-7whkv                 Created new node instance with id
 "i-06dfc0466b8f26695"
26m         Normal   SuccessfulDeleteEncryptedBootstrapDataSecrets   awsmachine/eks-
example-md-0-7whkv                 AWS Secret entries containing userdata deleted
27m         Normal   SuccessfulCreate                               awsmachine/
eks-example-md-0-ttgzv                 Created new node instance with id
 "i-0544fce0350fd41fb"
26m         Normal   SuccessfulDeleteEncryptedBootstrapDataSecrets   awsmachine/eks-
example-md-0-ttgzv                 AWS Secret entries containing userdata deleted
27m         Normal   SuccessfulCreate                               awsmachine/
eks-example-md-0-v2hrf                 Created new node instance with id
 "i-0498906edde162e59"
26m         Normal   SuccessfulDeleteEncryptedBootstrapDataSecrets   awsmachine/eks-
example-md-0-v2hrf                 AWS Secret entries containing userdata deleted
46m         Normal   SuccessfulCreate
 machinedeployment/eks-example-md-0               Created MachineSet "eks-example-
md-0-78fcd7c7b7"
```

EKS: Grant Cluster Access

This topic explains how to Grant Cluster Access.

About this task

You can access your cluster using AWS IAM roles in the dashboard. When you create an EKS cluster, the IAM entity is granted `system:masters` permissions in Kubernetes Role Based Access Control (RBAC) configuration.

> **Note:** More information about the configuration of the EKS control plane can be found on the EKS Cluster IAM Policies and Roles page.

If the EKS cluster was created as a cluster using a self-managed AWS cluster that uses IAM Instance Profiles, you will need to modify the `IAMAuthenticatorConfig` field in the `AWSManagedControlPlane` API object to allow other IAM entities to access the EKS workload cluster. Follow the steps below:

Procedure

1. Run the following command with your KUBECONFIG configured to select the self-managed cluster previously used to create the workload EKS cluster. Ensure you substitute ${CLUSTER_NAME} and ${CLUSTER_NAMESPACE} with their corresponding values for your cluster.

```
kubectl edit awsmanagedcontrolplane ${CLUSTER_NAME}-control-plane -n
 ${CLUSTER_NAMESPACE}
```

2. Edit the `IamAuthenticatorConfig` field with the IAM Role to the corresponding Kubernetes Role. In this example, the IAM role `arn:aws:iam::111122223333:role/PowerUser` is granted the cluster role `system:masters`. Note that this example uses example AWS resource ARNs, remember to substitute real values in the corresponding AWS account.

```
iamAuthenticatorConfig:
    mapRoles:
      - groups:
        - system:bootstrappers
        - system:nodes
        rolearn: arn:aws:iam::111122223333:role/my-node-role
        username: system:node:{{EC2PrivateDNSName}}
      - groups:
        - system:masters
        rolearn: arn:aws:iam::111122223333:role/PowerUser
        username: admin
```

   For further instructions on changing or assigning `roles` or `clusterroles` to which you can map IAM users or roles, see Amazon Enabling IAM access to your cluster.

EKS: Retrieve kubeconfig for EKS Cluster

This guide explains how to use the command line to interact with your newly deployed Kubernetes cluster.

About this task

Before you start, make sure you have created a workload cluster, as described in EKS: Create an EKS Cluster. When the workload cluster is created, the cluster life cycle services generate a `kubeconfig` file for the workload cluster, and write it to a Secret. The kubeconfig file is scoped to the cluster administrator.

Procedure

1. Get a kubeconfig file for the workload cluster from the Secret, and write it to a file using this command.

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

2. List the Nodes using this command.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get nodes
```

   Output will be similar to:

```
NAME                                          STATUS   ROLES    AGE   VERSION
ip-10-0-122-211.us-west-2.compute.internal    Ready    <none>   35m   v1.27.12-eks-
ae9a62a
ip-10-0-127-74.us-west-2.compute.internal     Ready    <none>   35m   v1.27.12-eks-
ae9a62a
ip-10-0-71-155.us-west-2.compute.internal     Ready    <none>   35m   v1.27.12-eks-
ae9a62a
```

```
ip-10-0-93-47.us-west-2.compute.internal     Ready    <none>   35m   v1.27.12-eks-
ae9a62a
```

> **Note:** It may take a few minutes for the Status to move to `Ready` while the Pod network is deployed. The node status will change to `Ready` soon after the `calico-node` DaemonSet Pods are Ready.

3. List the Pods using this command.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get --all-namespaces pods
```

Output will be similar to:

```
NAMESPACE                NAME                                       READY
 STATUS     RESTARTS      AGE
calico-system            calico-kube-controllers-7d6749878f-ccsx9   1/1
 Running    0             34m
calico-system            calico-node-2r6l8                          1/1
 Running    0             34m
calico-system            calico-node-5pdlb                          1/1
 Running    0             34m
calico-system            calico-node-n24hh                          1/1
 Running    0             34m
calico-system            calico-node-qrh7p                          1/1
 Running    0             34m
calico-system            calico-typha-7bbcb87696-7pk45              1/1
 Running    0             34m
calico-system            calico-typha-7bbcb87696-t4c8r              1/1
 Running    0             34m
calico-system            csi-node-driver-bz48k                      2/2
 Running    0             34m
calico-system            csi-node-driver-k5mmk                      2/2
 Running    0             34m
calico-system            csi-node-driver-nvcck                      2/2
 Running    0             34m
calico-system            csi-node-driver-x4xnh                      2/2
 Running    0             34m
kube-system              aws-node-2xp86                             1/1
 Running    0             35m
kube-system              aws-node-5f2kx                             1/1
 Running    0             35m
kube-system              aws-node-6lzm7                             1/1
 Running    0             35m
kube-system              aws-node-pz8c6                             1/1
 Running    0             35m
kube-system              cluster-autoscaler-789d86b489-sz9x2        0/1
 Init:0/1   0             36m
kube-system              coredns-57ff979f67-pk5cg                   1/1
 Running    0             75m
kube-system              coredns-57ff979f67-sf2j9                   1/1
 Running    0             75m
kube-system              ebs-csi-controller-5f6bd5d6dc-bplwm        6/6
 Running    0             36m
kube-system              ebs-csi-controller-5f6bd5d6dc-dpjt7        6/6
 Running    0             36m
kube-system              ebs-csi-node-7hmm5                         3/3
 Running    0             35m
kube-system              ebs-csi-node-l4vfh                         3/3
 Running    0             35m
kube-system              ebs-csi-node-mfr7c                         3/3
 Running    0             35m
kube-system              ebs-csi-node-v8krq                         3/3
 Running    0             35m
```

```
kube-system             kube-proxy-7fc5x                                  1/1
 Running    0            35m
kube-system             kube-proxy-vvkmk                                  1/1
 Running    0            35m
kube-system             kube-proxy-x6hcc                                  1/1
 Running    0            35m
kube-system             kube-proxy-x8frb                                  1/1
 Running    0            35m
kube-system             snapshot-controller-8ff89f489-4cfxv               1/1
 Running    0            36m
kube-system             snapshot-controller-8ff89f489-78gg8               1/1
 Running    0            36m
node-feature-discovery  node-feature-discovery-master-7d5985467-52fcn     1/1
 Running    0            36m
node-feature-discovery  node-feature-discovery-worker-88hr7               1/1
 Running    0            34m
node-feature-discovery  node-feature-discovery-worker-h95nq               1/1
 Running    0            35m
node-feature-discovery  node-feature-discovery-worker-lfghg               1/1
 Running    0            34m
node-feature-discovery  node-feature-discovery-worker-prc8p               1/1
 Running    0            35m
tigera-operator         tigera-operator-6dcd98c8ff-k97hq                  1/1
 Running    0            36m
```

EKS: Attach a Cluster

You can attach existing Kubernetes clusters to the Management Cluster with the instructions below.

About this task

After attaching the cluster, you can use the UI to examine and manage this cluster. The following procedure shows how to attach an existing Amazon Elastic Kubernetes Service (EKS) cluster.

This procedure assumes you have an existing and spun up Amazon EKS cluster(s) with administrative privileges. Refer to the Amazon EKS for setup and configuration information.

- Install aws-iam-authenticator. This binary is used to access your cluster using kubectl.

Attach a Pre-existing EKS Cluster

Ensure that the `KUBECONFIG` environment variable is set to the Management cluster before attaching by running:

Note:

```
export KUBECONFIG=<Management_cluster_kubeconfig>.conf
```

Access Your EKS Clusters

Procedure

1. Ensure you are connected to your EKS clusters. Enter the following commands for each of your clusters.

```
kubectl config get-contexts
kubectl config use-context <context for first eks cluster>
```

2. Confirm `kubectl` can access the EKS cluster.

```
kubectl get nodes
```

Create a kubeconfig File

About this task

To get started, ensure you have kubectl set up and configured with ClusterAdmin for the cluster you want to connect to Kommander.

Procedure

1. Create the necessary service account.

```
kubectl -n kube-system create serviceaccount kommander-cluster-admin
```

2. Create a token secret for the `serviceaccount`.

```
kubectl -n kube-system create  -f - <<EOF
apiVersion: v1
kind: Secret
metadata:
  name: kommander-cluster-admin-sa-token
  annotations:
    kubernetes.io/service-account.name: kommander-cluster-admin
type: kubernetes.io/service-account-token
EOF
```

   For more information on Service Account Tokens, refer to this article in our blog.

3. Verify that the `serviceaccount` token is ready by running this command.

```
kubectl -n kube-system get secret kommander-cluster-admin-sa-token -oyaml
```

   Verify that the `data.token` field is populated.

   Example output:

```
apiVersion: v1
data:
  ca.crt: LS0tLS1CRUdJTiBDR...
  namespace: ZGVmYXVsdA==
  token: ZXlKaGJHY2lPaUpTVX...
kind: Secret
metadata:
  annotations:
    kubernetes.io/service-account.name: kommander-cluster-admin
    kubernetes.io/service-account.uid: b62bc32e-b502-4654-921d-94a742e273a8
  creationTimestamp: "2022-08-19T13:36:42Z"
  name: kommander-cluster-admin-sa-token
  namespace: default
  resourceVersion: "8554"
  uid: 72c2a4f0-636d-4a70-9f1c-55a75f15e520
type: kubernetes.io/service-account-token
```

4. Configure the new service account for `cluster-admin` permissions.

```
cat << EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: kommander-cluster-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
```

```
- kind: ServiceAccount
  name: kommander-cluster-admin
  namespace: kube-system
EOF
```

5. Set up the following environment variables with the access data that is needed for producing a new `kubeconfig` file.

```
export USER_TOKEN_VALUE=$(kubectl -n kube-system get secret/kommander-cluster-admin-
sa-token -o=go-template='{{.data.token}}' | base64 --decode)
export CURRENT_CONTEXT=$(kubectl config current-context)
export CURRENT_CLUSTER=$(kubectl config view --raw -o=go-
template='{{range .contexts}}{{if eq .name "'''${CURRENT_CONTEXT}'''"}}
{{ index .context "cluster" }}{{end}}{{end}}')
export CLUSTER_CA=$(kubectl config view --raw -o=go-template='{{range .clusters}}{{if
 eq .name "'''${CURRENT_CLUSTER}'''"}}"{{with index .cluster "certificate-authority-
data" }}{{.}}{{end}}"{{ end }}{{ end }}')
export CLUSTER_SERVER=$(kubectl config view --raw -o=go-template='{{range .clusters}}
{{if eq .name "'''${CURRENT_CLUSTER}'''"}}{{ .cluster.server }}{{end}}{{ end }}')
```

6. Confirm these variables have been set correctly.

```
export -p | grep -E 'USER_TOKEN_VALUE|CURRENT_CONTEXT|CURRENT_CLUSTER|CLUSTER_CA|
CLUSTER_SERVER'
```

7. Generate a kubeconfig file that uses the environment variable values from the previous step.

> **Caution:** Nutanix support is unable to recover forgotten credentials. Prevent Kommander UI lockouts with the following best practices.
>
> - Backup the kubeconfig file.
> - Save your credentials.
> - Configure an IDP (identity provider) instead of using a username and password credentials.

```
cat << EOF > kommander-cluster-admin-config
apiVersion: v1
kind: Config
current-context: ${CURRENT_CONTEXT}
contexts:
- name: ${CURRENT_CONTEXT}
  context:
    cluster: ${CURRENT_CONTEXT}
    user: kommander-cluster-admin
    namespace: kube-system
clusters:
- name: ${CURRENT_CONTEXT}
  cluster:
    certificate-authority-data: ${CLUSTER_CA}
    server: ${CLUSTER_SERVER}
users:
- name: kommander-cluster-admin
  user:
    token: ${USER_TOKEN_VALUE}
EOF
```

8. This process produces a file in your current working directory called `kommander-cluster-admin-config`. The contents of this file are used in Kommander to attach the cluster. Before importing this configuration, verify the `kubeconfig` file can access the cluster.

```
kubectl --kubeconfig $(pwd)/kommander-cluster-admin-config get all --all-namespaces
```

Attach EKS Cluster Manually Using the CLI

These steps are only applicable if you do not set a `WORKSPACE_NAMESPACE` when creating a cluster. If you already set a `WORKSPACE_NAMESPACE`, then you do not need to perform these steps since the cluster is already attached to the workspace.

About this task

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments.

However, if you do not set a **workspace**, the attached cluster will be created in the `default` workspace. To ensure that the attached cluster is created in your desired `workspace namespace`, follow these instructions:

Procedure

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

```
echo ${MANAGED_CLUSTER_NAME}
```

2. Retrieve your `kubeconfig` from the cluster you have created without setting a workspace.

```
nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
 ${MANAGED_CLUSTER_NAME}.conf
```

3. You can now either attach it in the UI, or attach your cluster to the workspace you want in the CLI. This is only necessary if you never set the workspace of your cluster upon creation.

4. Retrieve the workspace where you want to attach the cluster:

```
kubectl get workspaces -A
```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace-namespace>
```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

```
kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
template='{{.data.value}}{{ "\n"}}'
```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml` file.

```
apiVersion: v1
kind: Secret
metadata:
  name: <your-managed-cluster-name>-kubeconfig
  labels:
    cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
type: cluster.x-k8s.io/secret
data:
  value: <value-you-copied-from-secret-above>
```

8. Create this secret in the desired workspace.

```
kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
 ${WORKSPACE_NAMESPACE}
```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

```
kubectl get kommanderclusters -A
```

Attach EKS Cluster from the UI Dashboard
Attach your new EKS cluster using the UI.

About this task

Now that you have a `kubeconfig` from the previous page, go to the NKP UI and follow these steps below:

Procedure

1. From the top menu bar, select your target workspace.
2. On the Dashboard page, select the **Add Cluster** option in the Actions dropdown list at the top right.
3. Select **Attach Cluster**.
4. Select the **No additional networking restrictions** card. Alternatively, if you must use network restrictions, stop following the steps below, and see the instructions on the page Attach a cluster WITH network restrictions.
5. Upload the kubeconfig file you created in the previous section (or copy its contents) into the **Cluster Configuration** section.
6. The **Cluster Name** field automatically populates with the name of the cluster in the kubeconfig. You can edit this field with the name you want for your cluster.
7. Add labels to classify your cluster as needed.
8. Select **Create** to attach your cluster.

> **Note:** If a cluster has limited resources to deploy all the federated platform services, it will fail to stay attached in the NKP UI. If this happens, ensure your system has sufficient resources for all pods.

vSphere Installation Options

For an environment that is on the vSphere Infrastructure, install options based on those environment variables are provided for you in this location.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operative in the most common scenarios.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

vSphere Overview

vSphere is a more complex setup than some of the other providers and infrastructures, so an overview of steps has been provided to help. To confirm that your OS is supported, see Supported Operating System.

The overall process for configuring vSphere and NKP together includes the following steps:

1. Configure vSphere to provide the needed elements described in the vSphere Prerequisites: All Installation Types.
2. For more information on air-gapped environments, see Creating a Bastion Host on page 685.
3. Create a base OS image (for use in the OVA package containing the disk images packaged with the OVF).
4. Create a CAPI VM image template that uses the base OS image and adds the needed Kubernetes cluster components.
5. Create a new self-managing cluster on vSphere.
6. Install Kommander.
7. Verify and log on to the UI.

vSphere Prerequisites: All Installation Types

This section contains all the prerequisite information specific to VMware vSphere infrastructure. These are above and beyond all of the NKP prerequisites for Install. Fulfilling the prerequisites involves completing these two areas:

1. NKP prerequisites
2. vSphere prerequisites - vCenter Server + ESXi

1. NKP Prerequisites

Before using NKP to create a vSphere cluster, verify that you have:

- An x86_64-based Linux or macOS machine.
- Download NKP binaries and Konvoy Image Builder (KIB) image bundle for Linux or macOS.
- A Container engine/runtime installed is required to install NKP and bootstrap:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.

- A registry needs to be installed on the host where the NKP Konvoy CLI runs. For example, if you are installing Konvoy on your laptop, ensure the laptop has a supported version of Docker or other registry. On macOS, Docker runs in a virtual machine. Configure this virtual machine with at least 8GB of memory.
- CLI tool Kubectl 1.21.6 for interacting with the running cluster, installed on the host where the NKP Konvoy command line interface (CLI) runs. For more information, see https://kubernetes.io/docs/tools/#kubectl.
- A valid VMware vSphere account with credentials configured.

  Note: NKP uses the vsphere CSI driver as the default storage provider. Use a Kubernetes CSI-compatible storage that is suitable for production. For more information, see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types.

  Note: You can choose from any of the storage options available for Kubernetes. To turn off the default that Konvoy deploys, set the default StorageClass as non-default. Then, set your newly created StorageClass to be the default by following the commands in the Kubernetes documentation called Changing the Default Storage Class.

VMware vSphere Prerequisites

Before installing, verify that your VMware vSphere Client environment meets the following basic requirements:

- Access to a bastion VM or other network-connected host, running vSphere version 7.0.3 or later.
  - You must be able to reach the vSphere API endpoint from where the Konvoy command line interface (CLI) runs.
- vSphere account with credentials configured - this account must have Administrator privileges.
- A Red Hat subscription with a username and password for downloading DVD ISOs.
- For air-gapped environments, a bastion VM host template with access to a configured local registry. The recommended template naming pattern is `../folder-name/NKP-e2e-bastion-template` or similar. Each infrastructure provider has its own set of bastion host instructions. For more information on Creating a Bastion Host on page 685, see your provider's documentation:
  - AWS: https://aws.amazon.com/solutions/implementations/linux-bastion/
  - Azure: https://learn.microsoft.com/en-us/azure/bastion/quickstart-host-portal
  - GCP: https://blogs.vmware.com/cloud/2021/06/02/intro-google-cloud-vmware-engine-bastion-host-access-iap/
  - vSphere: https://docs.vmware.com/en/VMware-vSphere/7.0/com.vmware.vsphere.security.doc/GUID-6975426F-56D0-4FE2-8A58-580B40D2F667.html
  - VMware: https://docs.vmware.com/en/VMware-vSphere/7.0/com.vmware.vsphere.security.doc/GUID-6975426F-56D0-4FE2-8A58-580B40D2F667.html.

- Valid vSphere values for the following:
  - vCenter API server URL
  - Datacenter name
  - Zone name that contains ESXi hosts for your cluster's nodes. For more information, see https://docs.vmware.com/en/VMware-vSphere/7.0/com.vmware.esxi.install.doc/GUID-B2F01BF5-078A-4C7E-B505-5DFFED0B8C38.html
  - Datastore name for the shared storage resource to be used for the VMs in the cluster.
    - Use of PersistentVolumes in your cluster depends on Cloud Native Storage (CNS), available in vSphere v7.0.3 and later versions. CNS depends on this shared Datastore's configuration.
  - Datastore URL from the datastore record for the shared datastore you want your cluster to use.
    - You need this URL value to ensure that the correct Datastore is used when NKP creates VMs for your cluster in vSphere.
  - Folder name.
  - Base template name, such as base-rhel-8 or base-rhel-7.
  - Name of a Virtual Network that has DHCP enabled for both air-gapped and non-air-gapped environments.
  - Resource Pools - at least one resource pool is needed, with every host in the pool having access to shared storage, such as VSAN.
    - Each host in the resource pool needs access to shared storage, such as NFS or VSAN, to make use of machine deployments and high-availability control planes.

vSphere Roles
When provisioning Kubernetes clusters with the Nutanix Kubernetes Platform (NKP) vSphere provider, four roles are needed for NKP to provide proper permissions.

About this task

Roles in vSphere are more like policy statements for the objects in a vSphere inventory. The Role is assigned to a user and the Object assignment can be inherited by any siblings if desired through propagation.

Add the permission at the highest level and set to propagate the permissions. In small vSphere environments, with just a few hosts, assigning the role or user at the top level and propagating to the child resources is appropriate. However, in the majority of cases, this is not possible since security teams will enforce strict restrictions on who has access to specific resources.

The table below describes the level at which these permissions are assigned, followed by the steps to **Add Roles** in vCenter. These roles provide user permissions that are less than those of the admin.

Procedure

Table 19: vSphere Permissions Propagation

Level	Required	Propagate to Child
vCenter Server (Top Level)	No	No
Data Center	Yes	No

Level	Required	Propagate to Child
Resource Pool	Yes	No
Folder	Yes	Yes
Template	Yes	No

1. Open a vSphere Client connection to the vCenter Server, described in the Prerequisites.
2. Select Home > Administration > Roles > Add Role.
3. Give the new Role a name from the four choices detailed in the next section.
4. Select the Privileges from the permissions directory tree dropdown list below each of the four roles.
   - The list of permissions can be set so that the provider is able to create, modify, or delete resources or clone templates, VMs, disks, attach network, etc.

vSphere: Create the Four Roles

The following four Roles need to be created for proper Nutanix Kubernetes Platform (NKP) access to the required Resource(s) on the correct level of vCenter and resource pools.

About this task

Set your roles to contain the permissions shown in the permissions directory tree under each role. Set the four roles - `NKP-vcenter`, `NKP-datacenter`, `NKP-k8srole`, `NKP-readonly` in the steps below.

Open vCenter:

Procedure

1. `nkp-vcenter` - This root-level permissions role applies to the Resource.
   - 1. `vcenter root`
        1. Resource: View
        2. Cns: Searchable
        3. Profile-driven storage: Profile-driven storage view
        4. Network - Session: ValidateSession
2. `nkp-datacenter` This role allows CAPV to create resources and assign networks. It is the most extensive permissions, but is only assigned on folder, resource pool, data store and network level so it can easily be separated from other environments. It applies to the Resources.
   - Do not propagate them because it gives the user view privileges on all folders and resource pools:
     1. datacenter
     2. cluster
     3. esx host 1
     4. esx host 2

Resource		
	X	View
Data Center Data Center		
	X	View

Cluster		
	X	View
ESX Host 1		
	X	View
ESX Host 2		
	X	View

3. `nkp-k8srole` - This role allows CAPV to create resources and assign networks. It is the most extensive permissions, but is only assigned on folder, resource pool, data store and network level so it can easily be separated from other environments. It applies to the Resources.

   - This role can be propagated to other Resources if desired.

     1. `resource pool`
     2. `nkp folder`
     3. `nkp data store`
     4. `network`

Resource		
	X	View
Datastore		
	X	Allocate space
	X	Browse
	X	Delete File
	X	File Management
	X	Update Virtual Machine File
	X	Update Virtual Machine Data
Global		
	X	Set Custom Field
Network		
	X	Assign network
Resource		
	X	Assign vApp to Pool
	X	Assign VM to Pool
Scheduled Task		
	X	Create
	X	Delete
	X	Edit
	X	Run

	Session	
	X	ValidateSession
Storage Profile		
	X	View
Storage Views		
	X	View

4. `nkp-readonly` - This optional role allows the role to clone from templates in other folders and data stores, but not have write access. It applies to the Resources.

   a. `templates folder`

   b. `templates data store`

Datastore		
	X	View
Folder		
	X	View
vApp		
	X	Clone
	X	Export
Provisioning		
	X	Clone
	X	Clone template
	X	Deploy template

vSphere: Minimum User Permissions

When a user needs permissions less than Admin, a role must be created with those permissions.

In small vSphere environments, with just a few hosts, assigning the role or user at the top level and propagating to the child resources is appropriate, as shown on this page in the permissions tree below.

However, in the majority of cases, this is not possible, as security teams will enforce strict restrictions on who needs access to specific resources.

The process for configuring a vSphere role with the permissions for provisioning nodes and installing includes the following steps:

1. Open a vSphere Client connection to the vCenter Server, as described in the Prerequisites.
2. Select Home > Administration > Roles > Add Role.
3. Give the new role a name, then select these Privileges:

Cns		
	X	Searchable
Datastore		
	X	Allocate space

	X	Browse datastore
	X	Low-level file operations
Host		
	Configuration	
	X	Storage partition configuration
Profile-driven storage		
	X	Profile-driven storage view
Network		
	X	Assign network
Resource		
		Assign virtual machine to resource pool.
Virtual machine		
	Change Configuration - from the list in that section, select these permissions below:	
	X	Add new disk
	X	Add existing disk
	X	Add or remove device
	X	Advanced configuration
	X	Change CPU count
	X	Change Memory
	X	Change resource
	X	Change Settings
	X	Reload from path
	Edit inventory	
	X	Create from existing
	X	Remove
	Interaction	
	X	Power off
	X	Power on
	Provisioning	
	X	Clone template
	X	Deploy template
	Session	
	X	ValidateSession

In the table below we describe the level at which these permissions get assigned.

Level	Required	Propagate to Child
vCenter Server (Top Level)	No	No
Data Center	Yes	No
Resource Pool	Yes	No
Folder	Yes	Yes
Template	Yes	No

vSphere Storage Options

Explore storage options and considerations for using NKP with VMware vSphere.

The vSphere Container Storage plugin supports shared NFS, vNFS, and vSAN. You need to provision your storage options in vCenter prior to creating a CAPI image in NKP for use with vSphere.

NKP has integrated the CSI 2.x driver used in vSphere. When creating your NKP cluster, NKP uses whatever configuration you provide for the Datastore name. vSAN is not required. Using NFS can reduce the amount of tagging and permission granting required to configure your cluster.

vSphere Installation

This topic provides instructions on how to install NKP in a vSphere non-air-gapped environment.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Further vSphere Prerequisites

Before you begin using Nutanix Kubernetes Platform (NKP), you must ensure you already meet the other prerequisites in the vSphere Prerequisites: All Installation Types section.

vSphere: Image Creation Overview

This diagram illustrates the image creation process:

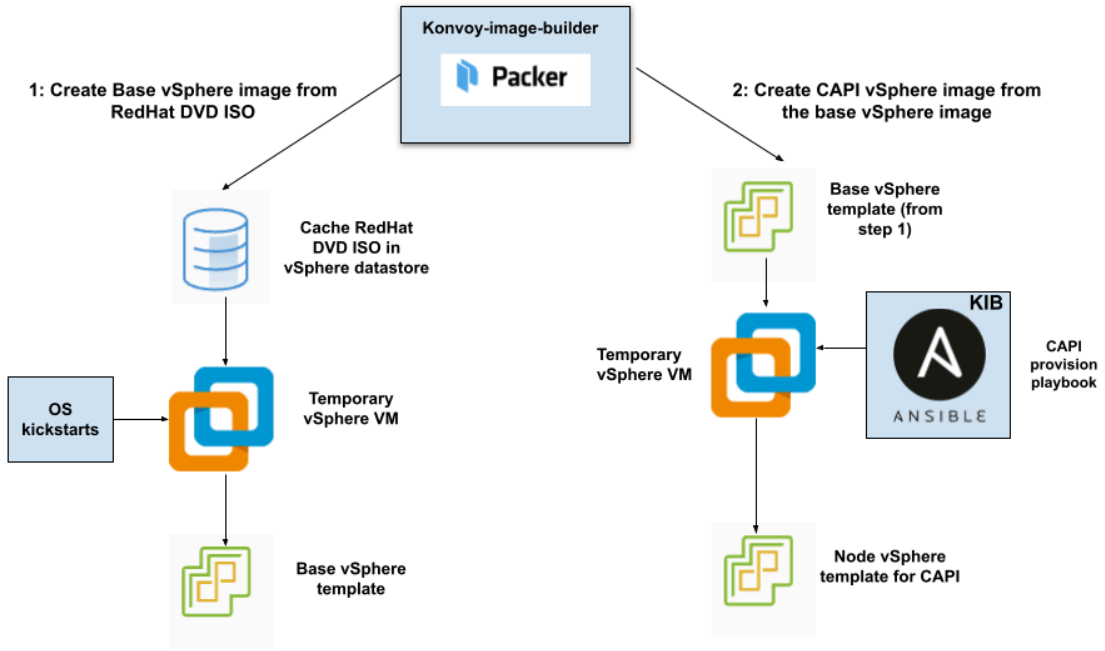


Figure 8: vSphere Image Creation Process

The workflow on the left shows the creation of a base OS image in the vCenter vSphere client using inputs from Packer. The workflow on the right shows how NKP uses that same base OS image to create CAPI-enabled VM images for your cluster.

After creating the base image, the NKP image builder uses it to create a CAPI-enabled vSphere template that includes the Kubernetes objects for the cluster. You can use that resulting template with the NKP `create cluster` command to create the VM nodes in your cluster directly on a vCenter server. From that point, you can use NKP to provision and manage your cluster.

NKP communicates with the code in vCenter Server as the management layer for creating and managing virtual machines after ESXi v7.0.3 or later is installed and configured. For more information, see https://docs.vmware.com/en/VMware-vSphere/7.0/com.vmware.esxi.install.doc/GUID-B2F01BF5-078A-4C7E-B505-5DFFED0B8C38.html.

Next Step

vSphere Air-gapped: Create an Image

vSphere: BaseOS Image in vCenter

Creating a base OS image from DVD ISO files is a one-time process. The base OS image file is created in the vSphere Client for use in the vSphere VM template. Therefore, the base OS image is used by Konvoy Image Builder (KIB) to create a VM template to configure Kubernetes nodes by the NKP vSphere provider.

The Base OS Image

For vSphere, a username is populated by `SSH_USERNAME` , and the user can use authorization through `SSH_PASSWORD` or `SSH_PRIVATE_KEY_FILE` environment variables and required by default by the packer. This user needs administrator privileges. It is possible to configure a custom user and password when building the OS image; however, that requires the Konvoy Image Builder (KIB) configuration to be overridden.

While creating the base OS image, it is important to take into consideration the following elements:

- Storage configuration: Nutanix recommends customizing disk partitions and not configuring a SWAP partition.
- Network configuration: as KIB must download and install packages, activating the network is required.
- Connect to Red Hat: if using Red Hat Enterprise Linux (RHEL), registering with Red Hat is required to configure software repositories and install software packages.
- Software selection: Nutanix recommends choosing **Minimal Install**.
- NKP recommends installing with the packages provided by the operating system package managers. Use the version that corresponds to the major version of your operating system.

vSphere: Creating a CAPI VM Template

The Konvoy Image Builder (KIB) uses the values in `image.yaml` and the input base OS image to create a vSphere template directly on the vCenter server.

About this task

You must have at least one image before creating a new cluster. If you have an image, this step in your configuration is not required each time since that image can be used to spin up a new cluster. However, if you need different images for different environments or providers, you will need to create a new custom image.

Procedure

1. Users need to perform the steps in the topic vSphere: Creating an Image before starting this procedure.
2. Build an image template with Konvoy Image Builder (KIB).

Create a vSphere Template for Your Cluster from a Base OS Image

Procedure

1. Set the following vSphere environment variables on the bastion VM host.

```
export VSPHERE_SERVER=your_vCenter_APIserver_URL
export VSPHERE_USERNAME=your_vCenter_user_name
export VSPHERE_PASSWORD=your_vCenter_password
```

2. Copy the base OS image file created in the vSphere Client to your desired location on the bastion VM host and make a note of the path and file name.

3. Create an `image.yaml` file and add the following variables for vSphere. NKP uses this file and these variables as inputs in the next step. To customize your `image.yaml` file, refer to this section: Customize your Image.

   Note: This example is Ubuntu 20.04. You will need to replace the OS name below based on your OS. Also, refer to the example YAML files located here: OVA YAML.

```
---
download_images: true
build_name: "ubuntu-2004"
packer_builder_type: "vsphere"
guestinfo_datasource_slug: "https://raw.githubusercontent.com/vmware/cloud-init-
vmware-guestinfo"
guestinfo_datasource_ref: "v1.4.0"
guestinfo_datasource_script: "{{guestinfo_datasource_slug}}/
{{guestinfo_datasource_ref}}/install.sh"
packer:
  cluster: "<VSPHERE_CLUSTER_NAME>"
  datacenter: "<VSPHERE_DATACENTER_NAME>"
  datastore: "<VSPHERE_DATASTORE_NAME>"
  folder: "<VSPHERE_FOLDER>"
  insecure_connection: "false"
  network: "<VSPHERE_NETWORK>"
  resource_pool: "<VSPHERE_RESOURCE_POOL>"
  template: "os-qualification-templates/d2iq-base-Ubuntu-20.04" # change default
 value with your base template name
  vsphere_guest_os_type: "other4xLinux64Guest"
  guest_os_type: "ubuntu2004-64"
  # goss params
  distribution: "ubuntu"
  distribution_version: "20.04"
# Use following overrides to select the authentication method that can be used with
 base template
# ssh_username: ""  # can be exported as environment variable 'SSH_USERNAME'
# ssh_password: "" # can be exported as environment variable 'SSH_PASSWORD'
# ssh_private_key_file = "" # can be exported as environment variable
 'SSH_PRIVATE_KEY_FILE'
# ssh_agent_auth: false  # is set to true, ssh_password and ssh_private_key will be
 ignored
```

4. Create a vSphere VM template with your variation of the following command.

```
konvoy-image build images/ova/<image.yaml>
```

   - Any additional configurations can be added to this command using `--overrides` flags as shown below:
     1. Any credential overrides: `--overrides overrides.yaml`
     2. for FIPS, add this flag: `--overrides overrides/fips.yaml`
     3. for air-gapped, add this flag: `--overrides overrides/offline-fips.yaml`

5. The Konvoy Image Builder (KIB) uses the values in `image.yaml` and the input base OS image to create a vSphere template directly on the vCenter server. This template contains the required artifacts needed to create a Kubernetes cluster. When KIB successfully provisions the OS image, it creates a manifest file. The `artifact_id` field of this file contains the name of the AMI ID (AWS), template name (vSphere), or image name (GCP/Azure), for example.

```
{
      "name": "vsphere-clone",
      "builder_type": "vsphere-clone",
      "build_time": 1644985039,
```

```
    "files": null,
    "artifact_id": "konvoy-ova-vsphere-rhel-86-1.21.6-1644983717",
    "packer_run_uuid": "260e8110-77f8-ca94-e29e-ac7a2ae779c8",
    "custom_data": {
      "build_date": "2022-02-16T03:55:17Z",
      "build_name": "vsphere-rhel-86",
      "build_timestamp": "1644983717",
      [...]
    }
  }
```

Tip: Recommendation: Now that you can now see the template created in your vCenter, it is best to rename it to `nkp-<NKP_VERSION>-k8s-<K8S_VERSION>-<DISTRO>`, like `nkp-2.4.0-k8s-1.24.6-ubuntu` to keep templates organized.

6. : The next steps are to deploy a NKP cluster using your vSphere template.

Next Step

Procedure

- vSphere: Creating the Management Cluster

vSphere: Creating the Management Cluster

Create a vSphere Management Cluster in a non-air-gapped environment.

About this task

Use this procedure to create a self-managed **Management cluster** with NKP. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing.

Note: To increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username=<username> --registry-mirror-password=<password>` on the `nkp create cluster` command.

Before you begin

First, you must name your cluster.

Note: The cluster name might only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable:

```
export CLUSTER_NAME=<my-vsphere-cluster>
```

3. Use the following command to set the environment variables for vSphere.

```
export VSPHERE_SERVER=example.vsphere.url
export VSPHERE_USERNAME=user@example.vsphere.url
```

```
export VSPHERE_PASSWORD=example_password
```

Note: NKP uses the vSphere CSI driver as the default storage provider. Use a Kubernetes CSI compatibleCSI-compatibleat is suitable for production. See the Kubernetes documentation called Changing the Default Storage Class for more information. If you're not using the default, you cannot deploy an alternate provider until after the `nkp create cluster` is finished. However, this must be determined before the installation

4. Generate the Kubernetes cluster objects by copying and editing this command to include the correct values, including the VM template name you assigned in the previous procedure.

```
nkp create cluster vsphere \
  --cluster-name ${CLUSTER_NAME} \
  --network <NETWORK_NAME> \
  --control-plane-endpoint-host <xxx.yyy.zzz.000> \
  --data-center <DATACENTER_NAME> \
  --data-store <DATASTORE_NAME> \
  --folder <FOLDER_NAME> \
  --server <VCENTER_API_SERVER_URL> \
  --ssh-public-key-file <SSH_PUBLIC_KEY_FILE> \
  --resource-pool <RESOURCE_POOL_NAME> \
  --vm-template <TEMPLATE_NAME> \
  --virtual-ip-interface <ip_interface_name> \
  --self-managed
```

Important: If you need to increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster, by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

» Flatcar OS flag: Flatcar OS use `--os-hint flatcar` to instruct the bootstrap cluster to make some changes related to the installation paths:

» HTTP: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

Next Step

Procedure

- vSphere: Configure MetalLB

vSphere: Configure MetalLB

Create a MetalLB configmap for your Pre-provisioned Infrastructure.

It is recommended that an external load balancer (LB) be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines that are responding to API requests. If you do not have one, you can use Metal LB to create a MetalLB configmap for your Pre-provisioned infrastructure.

Choose one of the following two protocols you want to use to announce service IPs. If your environment is not currently equipped with a load balancer, you can use MetalLB. Otherwise, your load balancer will work, and you can continue the installation process with Pre-provisioned: Install Kommander. To use MetalLB, create a MetalLB configMap for your Pre-provisioned infrastructure. MetalLB uses one of two protocols for exposing Kubernetes services:

- Layer 2, with Address Resolution Protocol (ARP)

- Border Gateway Protocol (BGP)

  Select one of the following procedures to create your MetalLB manifest for further editing.

Layer 2 Configuration

Layer 2 mode is the simplest to configure: in many cases, you don't need any protocol-specific configuration, only IP addresses.

Layer 2 mode does not require the IPs to be bound to the network interfaces of your worker nodes. It works by responding to ARP requests on your local network directly and give the machine's MAC address to clients.

- and giving IP address ranges or CIDR needs to be within the node's primary network subnet.
- MetalLB IP address ranges or CIDRs and node subnets must not conflict with the Kubernetes cluster pod and service subnets.

For example, the following configuration gives MetalLB control over IPs from 192.168.1.240 to 192.168.1.250 and configures Layer 2 mode:

The following values are generic; enter your specific values into the fields where applicable.

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    address-pools:
    - name: default
      protocol: layer2
      addresses:
      - 192.168.1.240-192.168.1.250
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

BGP Configuration

For a basic configuration featuring one BGP router and one IP address range, you need four pieces of information:

- The router IP address that MetalLB needs to connect to.
- The router's autonomous systems (AS) number.
- The AS number MetalLB to be used.
- An IP address range is expressed as a CIDR prefix.

As an example, if you want to give MetalLB the range 192.168.10.0/24 and AS number 64500, and connect it to a router at 10.0.0.1 with AS number 64501, your configuration will look like this:

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
```

```
    peers:
    - peer-address: 10.0.0.1
      peer-asn: 64501
      my-asn: 64500
    address-pools:
    - name: default
      protocol: bgp
      addresses:
      - 192.168.10.0/24
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

vSphere: Kommander Installation
This section provides installation instructions for the Kommander component of NKP in a non-air-gapped vSphere environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

> **Tip:** Tips and Recommendations
>
> - The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
> - Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period (for example, 1h) to allocate more time to the deployment of applications.
> - If the Kommander installation fails, or you want to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for installation.
- Ensure you have a Default StorageClass on page 1038.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. If required: Customize your `kommander.yaml`.

   a. See the Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.

5. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `dkp-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: nkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

6. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see the topic Configuring NKP Catalog Applications after Installing NKP.

Next Step

Procedure

- vSphere: Verify Install and Log in to the UI

vSphere: Verifying your Installation and UI Log in

Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

```
kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
 index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
dashboard{{ "\n"}}'
```

Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

a. Rotate the password using the following command.

```
nkp experimental rotate dashboard-password
```

The output displays the new password:

```
Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of NKP. The Cluster Operations section allows you to manage cluster operations and their application workloads to optimize your organization's productivity.

- Continue to the NKP Dashboard.

vSphere: Creating Managed Clusters Using the NKP CLI

This topic explains how to continue using the CLI to create managed clusters rather than switching to the UI dashboard.

About this task

After ,stand-alonethe initial cluster creation; you can create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed, which allows it to be a Management cluster or a stand-alone cluster. Subsequent new clusters are not self-managed, as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects or install the Kommander component. Those tasks are only done on Management clusters!
>
> Your new managed cluster needs to be part of a workspace under a management cluster. To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

```
kubectl get workspace -A
```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace_namespace>
```

> **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to create the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster that can be used as the management cluster.

First, you must name your cluster. Then, you run the command to deploy it.

> **Note:** The cluster name might only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

```
export CLUSTER_NAME=<my-managed-vsphere-cluster>
```

Create a Kubernetes Cluster

About this task

The below instructions tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, it will be created in the `default` workspace, and you need to take additional steps to attach to a workspace later. For instructions on how to do this, see Attach a Kubernetes Cluster.

Procedure

1. Use the following command to set the environment variables for vSphere.

```
export VSPHERE_SERVER=example.vsphere.url
export VSPHERE_USERNAME=user@example.vsphere.url
export VSPHERE_PASSWORD=example_password
```

2. Generate the Kubernetes cluster objects by copying and editing this command to include the correct values, including the VM template name you assigned in the previous procedure:.

```
nkp create cluster vsphere \
  --cluster-name ${MANAGED_CLUSTER_NAME} \
  --additional-tags=owner=$(whoami) \
  --namespace ${WORKSPACE_NAMESPACE} \
  --network <NETWORK_NAME> \
  --control-plane-endpoint-host <xxx.yyy.zzz.000> \
  --data-center <DATACENTER_NAME> \
  --data-store <DATASTORE_NAME> \
  --folder <FOLDER_NAME> \
  --server <VCENTER_API_SERVER_URL> \
  --ssh-public-key-file <SSH_PUBLIC_KEY_FILE> \
  --resource-pool <RESOURCE_POOL_NAME> \
  --virtual-ip-interface <ip_interface_name> \
```

```
--vm-template <TEMPLATE_NAME> \
--kubeconfig=<management-cluster-kubeconfig-path> \
```

Tip: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

   ```
   echo ${MANAGED_CLUSTER_NAME}
   ```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

   ```
   nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
    ${MANAGED_CLUSTER_NAME}.conf
   ```

3. **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

   ```
   kubectl get workspaces -A
   ```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace-namespace>
   ```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

   ```
   kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
   template='{{.data.value}}{{ "\n"}}'
   ```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <your-managed-cluster-name>-kubeconfig
     labels:
       cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
   type: cluster.x-k8s.io/secret
   data:
     value: <value-you-copied-from-secret-above>
   ```

8. Create this secret in the desired workspace.

```
kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
 ${WORKSPACE_NAMESPACE}
```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

```
kubectl get kommanderclusters -A
```

If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

Next Step

Procedure

- Cluster Operations Management

vSphere Air-gapped Installation

This installation provides instructions on how to install Nutanix Kubernetes Platform (NKP) in a vSphere air-gapped environment.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Note:

- For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

  ```
  tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
  ```

- Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

  For example:

  ```
  export TMPDIR=/path/to/your/directory
  ```

Further vSphere Prerequisites

Before you begin using NKP, you must ensure you already meet the other prerequisites in the vSphere Prerequisites: All Installation Types section.

vSphere Air-gapped: Image Creation Overview

This diagram illustrates the image creation process:

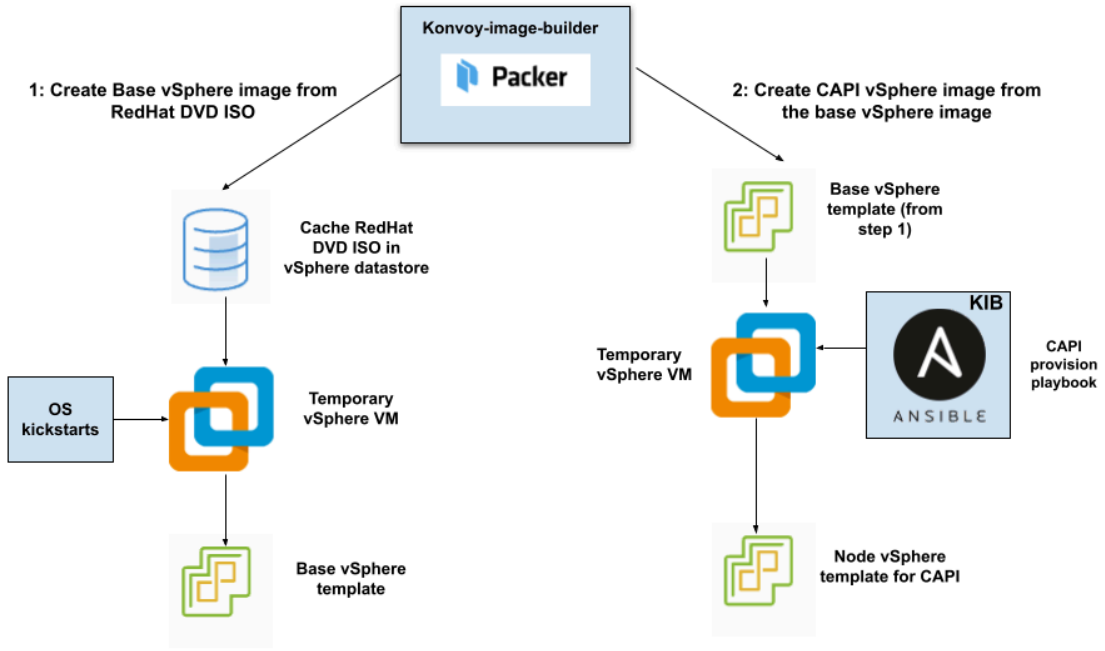


Figure 9: vSphere Image Creation Process

The workflow on the left shows the creation of a base OS image in the vCenter vSphere client using inputs from Packer. The workflow on the right shows how NKP uses that same base OS image to create CAPI-enabled VM images for your cluster.

After creating the base image, the NKP image builder uses it to create a CAPI-enabled vSphere template that includes the Kubernetes objects for the cluster. You can use that resulting template with the NKP `create cluster` command to create the VM nodes in your cluster directly on a vCenter server. From that point, you can use NKP to provision and manage your cluster.

vSphere Air-gapped: BaseOS Image in vCenter

Creating a base OS image from DVD ISO files is a one-time process. The base OS image file is created in the vSphere Client for use in the vSphere VM template. Therefore, the base OS image is used by Konvoy Image Builder (KIB) to create a VM template to configure Kubernetes nodes by the NKP vSphere provider.

The Base OS Image

For vSphere, a username is populated by `SSH_USERNAME` , and the user can use authorization through `SSH_PASSWORD` or `SSH_PRIVATE_KEY_FILE` environment variables and required by default bythe packer. This user needs administrator privileges. It is possible to configure a custom user and password when building the OS image; however, that requires the Konvoy Image Builder (KIB) configuration to be overridden.

While creating the base OS image, it is important to take into consideration the following elements:

- Storage configuration: Nutanix recommends customizing disk partitions and not configuring a SWAP partition.
- Network configuration: as KIB must download and install packages, activating the network is required.
- Connect to Red Hat: if using Red Hat Enterprise Linux (RHEL), registering with Red Hat is required to configure software repositories and install software packages.
- Software selection: Nutanix recommends choosing **Minimal Install**.
- NKP recommends installing with the packages provided by the operating system package managers. Use the version that corresponds to the major version of your operating system.

vSphere Air-gapped: Loading the Registry

Before creating an air-gapped Kubernetes cluster, you need to load the required images in a local registry for the Konvoy component.

About this task

If you do not already have a local registry set up, see the Local Registry Tools page for more information.

Note:

- For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

  ```
  tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
  ```

- Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

  For example:

  ```
  export TMPDIR=/path/to/your/directory
  ```

Procedure

1. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. EX: For the bootstrap cluster, change your directory to the `nkp-<version>` directory similar to example below depending on your current location.

   ```
   cd nkp-v2.12.0
   ```

2. Set an environment variable with your registry address and any other needed variables using this command.

```
export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
export REGISTRY_USERNAME=<username>
export REGISTRY_PASSWORD=<password>
export REGISTRY_CA=<path to the cacert file on the bastion>
```

3. Execute the following command to load the air-gapped image bundle into your private registry using any of the relevant flags to apply variables above.

```
nkp push bundle --bundle ./container-images/konvoy-image-bundle-
<nkp_version_number>.tar --to-registry=${REGISTRY_URL} --to-registry-username=
${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

> **Note:** It may take some time to push all the images to your image registry, depending on the performance of the network between the machine you are running the script on and the registry.

Kommander Load Images

About this task

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander component images, is required. See below for how to push the necessary images to this registry.

Procedure

1. Load the Kommander images into your private registry using the command below to load the image bundle.

```
nkp push bundle --bundle ./container-images/kommander-image-bundle-
<nkp_version_number>.tar --to-registry=${REGISTRY_URL} --to-registry-username=
${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

2. Optional Step for **Ultimate License** to load **NKP Catalog Applications** images.

```
nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-
<nkp_version_number>.tar --to-registry=${REGISTRY_URL} --to-registry-username=
${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

vSphere Air-gapped: Creating a CAPI VM Template

The Konvoy Image Builder (KIB) uses the values in `image.yaml` and the input base OS image to create a vSphere template directly on the vCenter server.

About this task

You must have at least one image before creating a new cluster. As long as you have an image, this step in your configuration is not required each time since that image can be used to spin up a new cluster. However, if you need different images for different environments or providers, you will need to create a new custom image.

> **Note:** Users need to perform the steps in the topic vSphere: Creating an Image before starting this procedure.

Procedure

1. Assuming you have downloaded `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`, extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.

3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the new NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`. Example command.

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   » For FIPS, pass the flag: `--fips`

   » For Red Hat Enterprise Linux (RHEL) OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`. Example command:

   ```
   export RHSM_ACTIVATION_KEY="-ci"
   export RHSM_ORG_ID="1232131"
   ```

4. Build image template with Konvoy Image Builder (KIB).

5. Follow the instructions to build a vSphere template below and if applicable, set the override `--overrides overrides/offline.yaml` flag described in Step 4 below.

Create a vSphere Template for Your Cluster from a Base OS Image

Procedure

1. Set the following vSphere environment variables on the bastion VM host.

   ```
   export VSPHERE_SERVER=your_vCenter_APIserver_URL
   export VSPHERE_USERNAME=your_vCenter_user_name
   export VSPHERE_PASSWORD=your_vCenter_password
   ```

2. Copy the base OS image file created in the vSphere Client to your desired location on the bastion VM host and make a note of the path and file name.

3. Create an `image.yaml` file and add the following variables for vSphere. NKP uses this file and these variables as inputs in the next step. To customize your `image.yaml` file, refer to this section: Customize your Image.

   Note: This example is Ubuntu 20.04. You will need to replace OS name below based on your OS. Also refer to example YAML files located here: OVA YAML

   ```
   ---
   download_images: true
   build_name: "ubuntu-2004"
   packer_builder_type: "vsphere"
   guestinfo_datasource_slug: "https://raw.githubusercontent.com/vmware/cloud-init-
   vmware-guestinfo"
   guestinfo_datasource_ref: "v1.4.0"
   guestinfo_datasource_script: "{{guestinfo_datasource_slug}}/
   {{guestinfo_datasource_ref}}/install.sh"
   packer:
   ```

```
  cluster: "<VSPHERE_CLUSTER_NAME>"
  datacenter: "<VSPHERE_DATACENTER_NAME>"
  datastore: "<VSPHERE_DATASTORE_NAME>"
  folder: "<VSPHERE_FOLDER>"
  insecure_connection: "false"
  network: "<VSPHERE_NETWORK>"
  resource_pool: "<VSPHERE_RESOURCE_POOL>"
  template: "os-qualification-templates/d2iq-base-Ubuntu-20.04" # change default
 value with your base template name
  vsphere_guest_os_type: "other4xLinux64Guest"
  guest_os_type: "ubuntu2004-64"
  # goss params
  distribution: "ubuntu"
  distribution_version: "20.04"
# Use following overrides to select the authentication method that can be used with
 base template
# ssh_username: ""  # can be exported as environment variable 'SSH_USERNAME'
# ssh_password: "" # can be exported as environment variable 'SSH_PASSWORD'
# ssh_private_key_file = "" # can be exported as environment variable
 'SSH_PRIVATE_KEY_FILE'
# ssh_agent_auth: false  # is set to true, ssh_password and ssh_private_key will be
 ignored
```

4. Create a vSphere VM template with your variation of the following command.

   ```
   konvoy-image build images/ova/<image.yaml>
   ```

   - Any additional configurations can be added to this command using `--overrides` flags as shown below:

     1. Any credential overrides: `--overrides overrides.yaml`
     2. for FIPS, add this flag: `--overrides overrides/fips.yaml`
     3. for air-gapped, add this flag: `--overrides overrides/offline-fips.yaml`

5. The Konvoy Image Builder (KIB) uses the values in `image.yaml` and the input base OS image to create a vSphere template directly on the vCenter server. This template contains the required artifacts needed to create a Kubernetes cluster. When KIB provisions the OS image successfully, it creates a manifest file. The `artifact_id` field of this file contains the name of the AMI ID (AWS), template name (vSphere), or image name (GCP/Azure), for example.

   ```
   {
        "name": "vsphere-clone",
        "builder_type": "vsphere-clone",
        "build_time": 1644985039,
        "files": null,
        "artifact_id": "konvoy-ova-vsphere-rhel-8-1.21.6-1644983717",
        "packer_run_uuid": "260e8110-77f8-ca94-e29e-ac7a2ae779c8",
        "custom_data": {
          "build_date": "2022-02-16T03:55:17Z",
          "build_name": "vsphere-rhel-86",
          "build_timestamp": "1644983717",
          [...]
        }
      }
   ```

   Tip: Recommendation: Now that you can now see the template created in your vCenter, it is best to rename it to `nkp-<NKP_VERSION>-k8s-<K8S_VERSION>-<DISTRO>`, like `nkp-2.4.0-k8s-1.24.6-ubuntu` to keep templates organized.

6. Next steps are to deploy a NKP cluster using your vSphere template.

vSphere Air-gapped: Creating the Management Cluster

Create a vSphere Management Cluster in an air-gapped environment.

About this task

If you use these instructions to create a cluster on vSphere using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes. First you must name your cluster.

Before you begin

Name Your Cluster

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. For more naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable using the command `export CLUSTER_NAME=<my-vsphere-cluster>`.
3. Use the following command to set the environment variables for vSphere.

```
export VSPHERE_SERVER=example.vsphere.url
export VSPHERE_USERNAME=user@example.vsphere.url
export VSPHERE_PASSWORD=example_password
```

> **Note:** NKP uses the vSphere CSI driver as the default storage provider. Use a Kubernetes CSI compatible storage that is suitable for production. See the Kubernetes documentation called Changing the Default Storage Class for more information. If you're not using the default, you cannot deploy an alternate provider until after the `nkp create cluster` is finished. However, it must be determined before Kommander installation.

4. Load the image, using either the `docker` or `podman` command

   » `docker load -i konvoy-bootstrap-image-v2.12.0.tar`

   » `podman load -i konvoy-bootstrap-image-v2.12.0.tar`

   ```
   podman image tag localhost/mesosphere/konvoy-bootstrap:2.12.0 docker.io/
   mesosphere/konvoy-bootstrap:v2.12.0
   ```

   The bootstrap image is loaded.

5. Generate the Kubernetes cluster objects by copying and editing this command to include the correct values, including the VM template name you assigned in the previous procedure.

```
nkp create cluster vsphere \
  --cluster-name ${CLUSTER_NAME} \
  --network <NETWORK_NAME> \
  --control-plane-endpoint-host <xxx.yyy.zzz.000> \
  --data-center <DATACENTER_NAME> \
  --data-store <DATASTORE_NAME> \
  --folder <FOLDER_NAME> \
  --server <VCENTER_API_SERVER_URL> \
  --ssh-public-key-file <SSH_PUBLIC_KEY_FILE> \
  --resource-pool <RESOURCE_POOL_NAME> \
  --vm-template <TEMPLATE_NAME> \
  --virtual-ip-interface <ip_interface_name> \
```

```
  --self-managed
```

> **Important:** If you need to increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster, by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

» Flatcar OS flag: Flatcar OS use `--os-hint flatcar` to instruct the bootstrap cluster to make some changes related to the installation paths:

» HTTP: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

vSphere Air-gapped: Configure MetalLB

Create a MetalLB configmap for your Pre-provisioned Insfrastructure.

It is recommended that an external load balancer (LB) be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines that are responding to API requests. If you do not have one, you can use Metal LB to create a MetalLB configmap for your Pre-provisioned infrastructure.

Choose one of the following two protocols you want to use to announce service IPs. If your environment is not currently equipped with a load balancer, you can use MetalLB. Otherwise, your own load balancer will work and you can continue the installation process with Pre-provisioned: Install Kommander . To use MetalLB, create a MetalLB configMap for your Pre-provisioned infrastructure. MetalLB uses one of two protocols for exposing Kubernetes services:

- Layer 2, with Address Resolution Protocol (ARP)
- Border Gateway Protocol (BGP)

  Select one of the following procedures to create your MetalLB manifest for further editing.

Layer 2 Configuration

Layer 2 mode is the simplest to configure: in many cases, you don’t need any protocol-specific configuration, only IP addresses.

Layer 2 mode does not require the IPs to be bound to the network interfaces of your worker nodes. It works by responding to ARP requests on your local network directly, to give the machine’s MAC address to clients.

- MetalLB IP address ranges or CIDRs need to be within the node’s primary network subnet.
- MetalLB IP address ranges or CIDRs and node subnet must not conflict with the Kubernetes cluster pod and service subnets.

For example, the following configuration gives MetalLB control over IPs from 192.168.1.240 to 192.168.1.250, and configures Layer 2 mode:

The following values are generic, enter your specific values into the fields where applicable.

```
cat << EOF > metallb-conf.yaml
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: default
  namespace: metallb-system
spec:
  addresses:
```

```
  - 192.168.1.240-192.168.1.250
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: default
  namespace: metallb-system
spec:
  ipAddressPools:
  - default
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

BGP Configuration

For a basic configuration featuring one BGP router and one IP address range, you need 4 pieces of information:

- The router IP address that MetalLB needs to connect to.
- The router's autonomous systems (AS) number.
- The AS number MetalLB to be used.
- An IP address range expressed as a CIDR prefix.

As an example, if you want to give MetalLB the range 192.168.10.0/24 and AS number 64500, and connect it to a router at 10.0.0.1 with AS number 64501, your configuration will look like:

```
cat << EOF > metallb-conf.yaml
apiVersion: metallb.io/v1beta2
kind: BGPPeer
metadata:
  name: default
  namespace: metallb-system
spec:
  myASN: 64500
  peerASN: 64501
  peerAddress: 10.0.0.1
---
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: default
  namespace: metallb-system
spec:
  addresses:
  - 192.168.10.0/24
---
apiVersion: metallb.io/v1beta1
kind: BGPAdvertisement
metadata:
  name: example
  namespace: metallb-system
spec:
  ipAddressPools:
  - default
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

vSphere Air-gapped: Installing Kommander

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped vSphere environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

> **Tip:** Tips and Recommendations
>
> - The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
> - Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
> - If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. If required: Customize your `kommander.yaml`.

   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy and External Load Balancer.

5. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `dkp-catalog-appliations`.

   ```
   apiVersion: config.kommander.mesosphere.io/v1alpha1
   kind: Installation
   catalog:
     repositories:
       - name: dkp-catalog-applications
         labels:
   ```

```
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
    gitRepositorySpec:
      url: https://github.com/mesosphere/dkp-catalog-applications
      ref:
        tag: v2.12.0
```

6. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see the topic Configuring NKP Catalog Applications after Installing NKP.

vSphere Air-gapped: Verifying your Installation and UI Log in
Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      nkp experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of NKP. The Cluster Operations section allows you to manage cluster operations and their application workloads to optimize your organization’s productivity.

- Continue to the NKP Dashboard.

vSphere Air-gapped: Creating Managed Clusters Using the NKP CLI
This topic explains how to continue using the CLI to create managed clusters rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed which allows it to be a Management cluster or a stand alone cluster. Subsequent new clusters are not self-managed as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects, or install the Kommander component. Those tasks are only done on Management clusters!
>
> Your new managed cluster needs to be part of a workspace under a management cluster. To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

   ```
   kubectl get workspace -A
   ```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

   > **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First you must name your cluster. Then you run the command to deploy it.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

   ```
   export CLUSTER_NAME=<my-managed-vsphere-cluster>
   ```

Create a Kubernetes Cluster

About this task

The below instructions tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, it will be created in the `default` workspace, and you need to take additional steps to attach to a workspace later. For instructions on how to do this, see Attach a Kubernetes Cluster.

Procedure

1. Configure your cluster to use an existing local registry as a mirror when attempting to pull images: IMPORTANT: The image must be created by Konvoy Image Builder in order to use the registry mirror feature..

```
export REGISTRY_URL=<https/http>://<registry-address>:<registry-port>
export REGISTRY_CA=<path to the CA on the bastion>
export REGISTRY_USERNAME=<username>
export REGISTRY_PASSWORD=<password>
```

2. Load the images, using either the `docker` or `podman` command.

   » `docker load -i konvoy-bootstrap-image-v2.12.0.tar`

   » `podman load -i konvoy-bootstrap-image-v2.12.0.tar`

3. Generate the Kubernetes cluster objects by copying and editing this command to include the correct values, including the VM template name you assigned in the previous procedure:.

```
nkp create cluster vsphere \
  --cluster-name=${MANAGED_CLUSTER_NAME} \
  --additional-tags=owner=$(whoami) \
  --kubeconfig=<management-cluster-kubeconfig-path> \
  --namespace ${WORKSPACE_NAMESPACE}
  --network <NETWORK_NAME> \
  --control-plane-endpoint-host <CONTROL_PLANE_IP> \
  --data-center <DATACENTER_NAME> \
  --data-store <DATASTORE_NAME> \
  --folder <FOLDER_NAME> \
  --server <VCENTER_API_SERVER_URL> \
  --ssh-public-key-file </path/to/key.pub> \
  --resource-pool <RESOURCE_POOL_NAME> \
  --vm-template konvoy-ova-vsphere-os-release-k8s_release-vsphere-timestamp \
  --virtual-ip-interface <ip_interface_name> \
  --extra-sans "127.0.0.1" \
  --registry-mirror-url=${REGISTRY_URL} \
  --registry-mirror-cacert=${REGISTRY_CA} \
  --registry-mirror-username=${REGISTRY_USERNAME} \
  --registry-mirror-password=${REGISTRY_PASSWORD} \
```

Tip: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

   ```
   echo ${MANAGED_CLUSTER_NAME}
   ```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

   ```
   nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
    ${MANAGED_CLUSTER_NAME}.conf
   ```

3. **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

   ```
   kubectl get workspaces -A
   ```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace-namespace>
   ```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

   ```
   kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
   template='{{.data.value}}{{ "\n"}}'
   ```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <your-managed-cluster-name>-kubeconfig
     labels:
       cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
   type: cluster.x-k8s.io/secret
   data:
     value: <value-you-copied-from-secret-above>
   ```

8. Create this secret in the desired workspace.

   ```
   kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
    ${WORKSPACE_NAMESPACE}
   ```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

   ```
   cat << EOF | kubectl apply -f -
   apiVersion: kommander.mesosphere.io/v1beta1
   kind: KommanderCluster
   metadata:
   ```

```
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

    ```
    kubectl get kommanderclusters -A
    ```

    If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

Next Step

Procedure

- Cluster Operations Management

vSphere with FIPS Installation

This installation provides instructions to install NKP in a vSphere non-air-gapped environment.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Further vSphere Prerequisites

Before you begin using NKP, you must ensure you already meet the other prerequisites in the vSphere Prerequisites: All Installation Types section.

vSphere FIPS: Image Creation Overview

This diagram illustrates the image creation process:

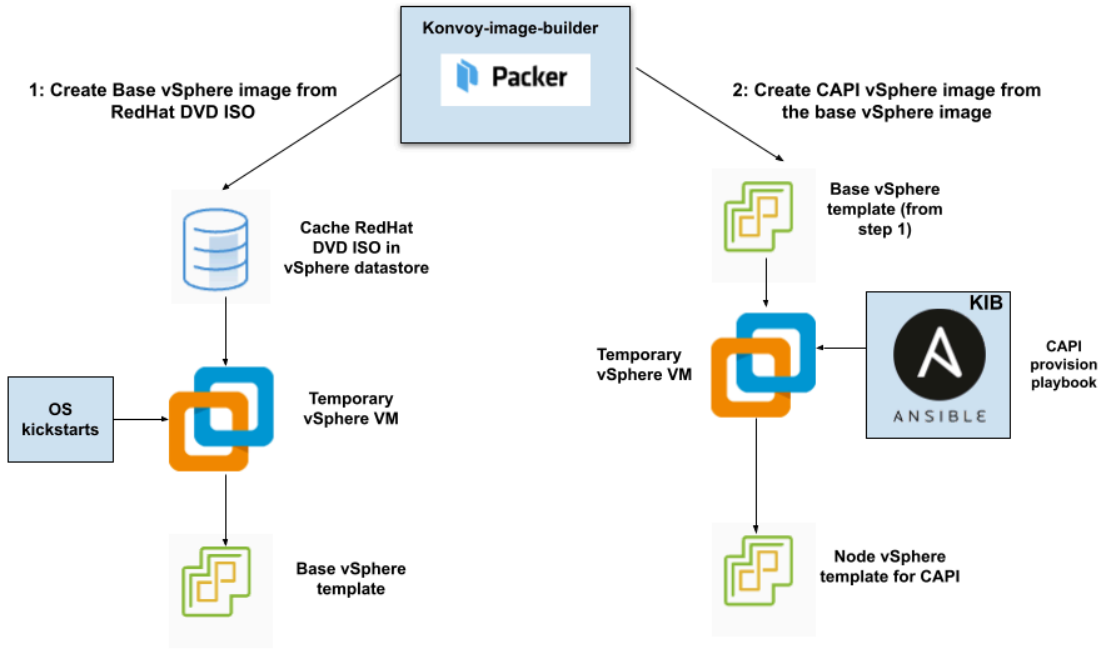


Figure 10: vSphere Image Creation Process

The workflow on the left shows the creation of a base OS image in the vCenter vSphere client using inputs from Packer. The workflow on the right shows how NKP uses that same base OS image to create CAPI-enabled VM images for your cluster.

After creating the base image, the NKP image builder uses it to create a CAPI-enabled vSphere template that includes the Kubernetes objects for the cluster. You can use that resulting template with the NKP `create cluster` command to create the VM nodes in your cluster directly on a vCenter server. From that point, you can use NKP to provision and manage your cluster.

vSphere FIPS: BaseOS Image in vCenter

Creating a base OS image from DVD ISO files is a one-time process. The base OS image file is created in the vSphere Client for use in the vSphere VM template. Therefore, the base OS image is used by Konvoy Image Builder (KIB) to create a VM template to configure Kubernetes nodes by the NKP vSphere provider.

The Base OS Image

For vSphere, a username is populated by `SSH_USERNAME` and the user can use authorization through `SSH_PASSWORD` or `SSH_PRIVATE_KEY_FILE` environment variables and required by default by packer. This user needs administrator privileges. It is possible to configure a custom user and password when building the OS image, however, that requires the Konvoy Image Builder (KIB) configuration to be overridden.

While creating the base OS image, it is important to take into consideration the following elements:

- Storage configuration: Nutanix recommends customizing disk partitions and not configuring a SWAP partition.

- Network configuration: as KIB must download and install packages, activating the network is required.
- Connect to Red Hat: if using Red Hat Enterprise Linux (RHEL), registering with Red Hat is required to configure software repositories and install software packages.
- Software selection: Nutanix recommends choosing **Minimal Install**.
- NKP recommends to install with the packages provided by the operating system package managers. Use the version that corresponds to the major version of your operating system.

vSphere FIPS: Creating a CAPI VM Template
The Konvoy Image Builder (KIB) uses the values in `image.yaml` and the input base OS image to create a vSphere template directly on the vCenter server.

About this task

You must have at least one image before creating a new cluster. As long as you have an image, this step in your configuration is not required each time since that image can be used to spin up a new cluster. However, if you need different images for different environments or providers, you will need to create a new custom image.

Procedure

1. Users need to perform the steps in the topic vSphere FIPS: Creating an Image before starting this procedure.
2. Build image template with Konvoy Image Builder (KIB).

Create a vSphere Template for Your Cluster from a Base OS Image

Procedure

1. Set the following vSphere environment variables on the bastion VM host.

```
export VSPHERE_SERVER=your_vCenter_APIserver_URL
export VSPHERE_USERNAME=your_vCenter_user_name
export VSPHERE_PASSWORD=your_vCenter_password
```

2. Copy the base OS image file created in the vSphere Client to your desired location on the bastion VM host and make a note of the path and file name.
3. Create an `image.yaml` file and add the following variables for vSphere. NKP uses this file and these variables as inputs in the next step. To customize your `image.yaml` file, refer to this section: Customize your Image.

   Note: This example is Ubuntu 20.04. You will need to replace OS name below based on your OS. Also refer to example YAML files located here: OVA YAML

```
---
download_images: true
build_name: "ubuntu-2004"
packer_builder_type: "vsphere"
guestinfo_datasource_slug: "https://raw.githubusercontent.com/vmware/cloud-init-
vmware-guestinfo"
guestinfo_datasource_ref: "v1.4.0"
guestinfo_datasource_script: "{{guestinfo_datasource_slug}}/
{{guestinfo_datasource_ref}}/install.sh"
packer:
  cluster: "<VSPHERE_CLUSTER_NAME>"
```

```
  datacenter: "<VSPHERE_DATACENTER_NAME>"
  datastore: "<VSPHERE_DATASTORE_NAME>"
  folder: "<VSPHERE_FOLDER>"
  insecure_connection: "false"
  network: "<VSPHERE_NETWORK>"
  resource_pool: "<VSPHERE_RESOURCE_POOL>"
  template: "os-qualification-templates/d2iq-base-Ubuntu-20.04" # change default
 value with your base template name
  vsphere_guest_os_type: "other4xLinux64Guest"
  guest_os_type: "ubuntu2004-64"
  # goss params
  distribution: "ubuntu"
  distribution_version: "20.04"
# Use following overrides to select the authentication method that can be used with
 base template
# ssh_username: ""  # can be exported as environment variable 'SSH_USERNAME'
# ssh_password: "" # can be exported as environment variable 'SSH_PASSWORD'
# ssh_private_key_file = "" # can be exported as environment variable
 'SSH_PRIVATE_KEY_FILE'
# ssh_agent_auth: false  # is set to true, ssh_password and ssh_private_key will be
 ignored
```

4. Create a vSphere VM template with your variation of the following command.

   ```
   konvoy-image build images/ova/<image.yaml>
   ```

   - Any additional configurations can be added to this command using `--overrides` flags as shown below:

     1. Any credential overrides: `--overrides overrides.yaml`
     2. for FIPS, add this flag: `--overrides overrides/fips.yaml`
     3. for air-gapped, add this flag: `--overrides overrides/offline-fips.yaml`

5. The Konvoy Image Builder (KIB) uses the values in `image.yaml` and the input base OS image to create a vSphere template directly on the vCenter server. This template contains the required artifacts needed to create a Kubernetes cluster. When KIB provisions the OS image successfully, it creates a manifest file. The `artifact_id` field of this file contains the name of the AMI ID (AWS), template name (vSphere), or image name (GCP or Azure), for example.

   ```
   {
        "name": "vsphere-clone",
        "builder_type": "vsphere-clone",
        "build_time": 1644985039,
        "files": null,
        "artifact_id": "konvoy-ova-vsphere-rhel-86-1.21.6-1644983717",
        "packer_run_uuid": "260e8110-77f8-ca94-e29e-ac7a2ae779c8",
        "custom_data": {
          "build_date": "2022-02-16T03:55:17Z",
          "build_name": "vsphere-rhel-86",
          "build_timestamp": "1644983717",
          [...]
        }
      }
   ```

   Tip: Recommendation: Now that you can now see the template created in your vCenter, it is best to rename it to `nkp-<NKP_VERSION>-k8s-<K8S_VERSION>-<DISTRO>`, like `nkp-2.4.0-k8s-1.24.6-ubuntu` to keep templates organized.

6. Next steps are to deploy a NKP cluster using your vSphere template.

vSphere FIPS: Creating the Management Cluster

Create a vSphere Management Cluster in a non-air-gapped environment using FIPS.

About this task

Use this procedure to create a self-managed **Management cluster** with NKP. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing. First you must name your cluster.

Before you begin

Procedure

- The table below identifies the current FIPS and `etcd` versions for this release.

Table 20: Supported FIPS Builds

Component	Repository	Version
Kubernetes	docker.io/mesosphere	v1.29.6+fips.0
etcd	docker.io/mesosphere	3.5.10+fips.0

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. For more Kubernetes naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

- Name your cluster and give it a unique name suitable for your environment.
- Set the environment variable:

```
export CLUSTER_NAME=<my-vsphere-cluster>
```

Create a New vSphere Kubernetes Cluster

About this task

The below instructions tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, it will be created in the `default` workspace, and you need to take additional steps to attach to a workspace later. For instructions on how to do this, see Attaching an Existing Kubernetes Cluster on page 493.

Procedure

1. Use the following command to set the environment variables for vSphere.

```
export VSPHERE_SERVER=example.vsphere.url
export VSPHERE_USERNAME=user@example.vsphere.url
export VSPHERE_PASSWORD=example_password
```

2. Generate the Kubernetes cluster objects by copying and editing this command to include the correct values, including the VM template name you assigned in the previous procedure.

```
nkp create cluster vsphere \
  --cluster-name ${CLUSTER_NAME} \
  --network <NETWORK_NAME> \
  --control-plane-endpoint-host <xxx.yyy.zzz.000> \
```

```
--data-center <DATACENTER_NAME> \
--data-store <DATASTORE_NAME> \
--folder <FOLDER_NAME> \
--server <VCENTER_API_SERVER_URL> \
--ssh-public-key-file <SSH_PUBLIC_KEY_FILE> \
--resource-pool <RESOURCE_POOL_NAME> \
--vm-template <TEMPLATE_NAME> \
--virtual-ip-interface <ip_interface_name> \
--kubernetes-version=v1.29.6+fips.0 \
--kubernetes-image-repository=docker.io/mesosphere \
--etcd-image-repository=docker.io/mesosphere \
--etcd-version=3.5.10+fips.0 \
--self-managed
```

Note: To increase Dockerhub's rate limit use your Dockerhub credentials when creating the cluster, by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

- Flatcar OS flag: Flatcar OS use `--os-hint flatcar` to instruct the bootstrap cluster to make some changes related to the installation paths:
- HTTP: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

Next Step

Procedure

- vSphere FIPS: Configure MetalLB

vSphere FIPS: Configure MetalLB

Create a MetalLB configmap for your Pre-provisioned Insfrastructure.

It is recommended that an external load balancer (LB) be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines that are responding to API requests. If you do not have one, you can use Metal LB to create a MetalLB configmap for your Pre-provisioned infrastructure.

Choose one of the following two protocols you want to use to announce service IPs. If your environment is not currently equipped with a load balancer, you can use MetalLB. Otherwise, your own load balancer will work and you can continue the installation process with Pre-provisioned: Install Kommander . To use MetalLB, create a MetalLB configMap for your Pre-provisioned infrastructure. MetalLB uses one of two protocols for exposing Kubernetes services:

- Layer 2, with Address Resolution Protocol (ARP)
- Border Gateway Protocol (BGP)

  Select one of the following procedures to create your MetalLB manifest for further editing.

Layer 2 Configuration

Layer 2 mode is the simplest to configure: in many cases, you don’t need any protocol-specific configuration, only IP addresses.

Layer 2 mode does not require the IPs to be bound to the network interfaces of your worker nodes. It works by responding to ARP requests on your local network directly, to give the machine's MAC address to clients.

- MetalLB IP address ranges or CIDRs need to be within the node's primary network subnet.
- MetalLB IP address ranges or CIDRs and node subnet must not conflict with the Kubernetes cluster pod and service subnets.

For example, the following configuration gives MetalLB control over IPs from 192.168.1.240 to 192.168.1.250, and configures Layer 2 mode:

The following values are generic, enter your specific values into the fields where applicable.

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    address-pools:
    - name: default
      protocol: layer2
      addresses:
      - 192.168.1.240-192.168.1.250
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

BGP Configuration

For a basic configuration featuring one BGP router and one IP address range, you need 4 pieces of information:

- The router IP address that MetalLB needs to connect to.
- The router's autonomous systems (AS) number.
- The AS number MetalLB to be used.
- An IP address range expressed as a CIDR prefix.

As an example, if you want to give MetalLB the range 192.168.10.0/24 and AS number 64500, and connect it to a router at 10.0.0.1 with AS number 64501, your configuration will look like:

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    peers:
    - peer-address: 10.0.0.1
      peer-asn: 64501
      my-asn: 64500
    address-pools:
    - name: default
      protocol: bgp
      addresses:
      - 192.168.10.0/24
```

```
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

vSphere FIPS: Installing Kommander

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped vSphere environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

> **Tip:** Tips and Recommendations
>
> - The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
> - Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
> - If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. If required: Customize your `kommander.yaml`.

   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy and External Load Balancer.

5. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `dkp-catalog-appliations`.

   ```
   apiVersion: config.kommander.mesosphere.io/v1alpha1
   kind: Installation
   ```

```
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

6. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see the topic Configuring NKP Catalog Applications after Installing NKP.

vSphere FIPS: Verifying your Installation and UI Log in
Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      nkp experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Cluster Operations

Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of NKP. The Cluster Operations section allows you to manage cluster operations and their application workloads to optimize your organization's productivity.

- Continue to the NKP Dashboard.

vSphere FIPS: Creating Managed Clusters Using the NKP CLI
This topic explains how to continue using the CLI to create managed clusters rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed which allows it to be a Management cluster or a stand alone cluster. Subsequent new clusters are not self-managed as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects, or install the Kommander component. Those tasks are only done on Management clusters!
>
> Your new managed cluster needs to be part of a workspace under a management cluster. To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

   ```
   kubectl get workspace -A
   ```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

   > **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First you must name your cluster. Then you run the command to deploy it.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.

2. Set the environment variable.

```
export CLUSTER_NAME=<my-managed-vsphere-cluster>
```

Create a Kubernetes Cluster

About this task

The below instructions tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, it will be created in the `default` workspace, and you need to take additional steps to attach to a workspace later. For instructions on how to do this, see Attach a Kubernetes Cluster.

Procedure

1. Use the following command to set the environment variables for vSphere.

```
export VSPHERE_SERVER=example.vsphere.url
export VSPHERE_USERNAME=user@example.vsphere.url
export VSPHERE_PASSWORD=example_password
```

2. Generate the Kubernetes cluster objects by copying and editing this command to include the correct values, including the VM template name you assigned in the previous procedure:.

```
nkp create cluster vsphere \
  --cluster-name ${MANAGED_CLUSTER_NAME} \
  --additional-tags=owner=$(whoami) \
  --namespace ${WORKSPACE_NAMESPACE} \
  --network <NETWORK_NAME> \
  --control-plane-endpoint-host <xxx.yyy.zzz.000> \
  --data-center <DATACENTER_NAME> \
  --data-store <DATASTORE_NAME> \
  --folder <FOLDER_NAME> \
  --server <VCENTER_API_SERVER_URL> \
  --ssh-public-key-file <SSH_PUBLIC_KEY_FILE> \
  --resource-pool <RESOURCE_POOL_NAME> \
  --virtual-ip-interface <ip_interface_name> \
  --vm-template <TEMPLATE_NAME> \
  --kubeconfig=<management-cluster-kubeconfig-path> \
```

Tip: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Use HTTP or HTTPS Proxy with KIB Images on page 1126.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

```
echo ${MANAGED_CLUSTER_NAME}
```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

```
nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
 ${MANAGED_CLUSTER_NAME}.conf
```

3. **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

```
kubectl get workspaces -A
```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace-namespace>
```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

```
kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
template='{{.data.value}}{{ "\n"}}'
```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

```
apiVersion: v1
kind: Secret
metadata:
  name: <your-managed-cluster-name>-kubeconfig
  labels:
    cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
type: cluster.x-k8s.io/secret
data:
  value: <value-you-copied-from-secret-above>
```

8. Create this secret in the desired workspace.

```
kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
 ${WORKSPACE_NAMESPACE}
```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

    ```
    kubectl get kommanderclusters -A
    ```

    If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

Next Step

Procedure

- Cluster Operations Management

vSphere Air-gapped FIPS Installation

This installation provides instructions on how to install NKP in a vSphere air-gapped environment using FIPS.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Note:

- For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

  ```
  tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
  ```

- Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

  For example:

  ```
  export TMPDIR=/path/to/your/directory
  ```

Further vSphere Prerequisites

Before you begin using NKP, you must ensure you already meet the other prerequisites in the vSphere Prerequisites: All Installation Types section.

vSphere Air-gapped FIPS: Image Creation Overview

This diagram illustrates the image creation process:

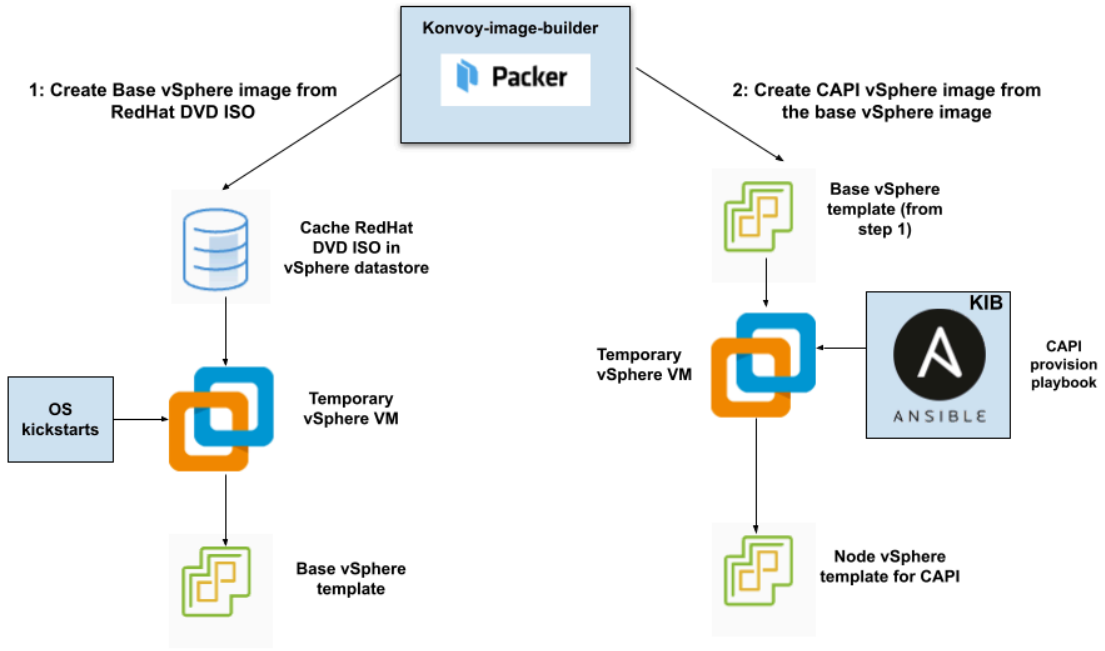


Figure 11: vSphere Image Creation Process

The workflow on the left shows the creation of a base OS image in the vCenter vSphere client using inputs from Packer. The workflow on the right shows how NKP uses that same base OS image to create CAPI-enabled VM images for your cluster.

After creating the base image, the NKP image builder uses it to create a CAPI-enabled vSphere template that includes the Kubernetes objects for the cluster. You can use that resulting template with the NKP `create cluster` command to create the VM nodes in your cluster directly on a vCenter server. From that point, you can use NKP to provision and manage your cluster.

vSphere Air-gapped FIPS: BaseOS Image in vCenter

Creating a base OS image from DVD ISO files is a one-time process. The base OS image file is created in the vSphere Client for use in the vSphere VM template. Therefore, the base OS image is used by Konvoy Image Builder (KIB) to create a VM template to configure Kubernetes nodes by the Nutanix Kubernetes Platform (NKP) vSphere provider.

The Base OS Image

For vSphere, a username is populated by `SSH_USERNAME` , and the user can use authorization through `SSH_PASSWORD` or `SSH_PRIVATE_KEY_FILE` environment variables and required by default by the packer. This user needs administrator privileges. It is possible to configure a custom user and password when building the OS image; however, that requires the Konvoy Image Builder (KIB) configuration to be overridden.

While creating the base OS image, it is important to take into consideration the following elements:

- Storage configuration: Nutanix recommends customizing disk partitions and not configuring a SWAP partition.
- Network configuration: as KIB must download and install packages, activating the network is required.
- Connect to Red Hat: if using Red Hat Enterprise Linux (RHEL), registering with Red Hat is required to configure software repositories and install software packages.
- Software selection: Nutanix recommends choosing **Minimal Install**.
- NKP recommends installing with the packages provided by the operating system package managers. Use the version that corresponds to the major version of your operating system.

vSphere Air-gapped FIPS: Loading the Registry
Before creating an air-gapped Kubernetes cluster, you need to load the required images in a local registry for the Konvoy component.

About this task

If you do not already have a local registry set up, see the Local Registry Tools page for more information.

Note:

- For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

  ```
  tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
  ```

- Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

  For example:

  ```
  export TMPDIR=/path/to/your/directory
  ```

Procedure

1. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. Example: For the bootstrap cluster, change your directory to the `nkp-<version>` directory, similar to the example below, depending on your current location.

   ```
   cd nkp-v2.12.0
   ```

2. Set an environment variable with your registry address and any other needed variables using this command.

   ```
   export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
   export REGISTRY_USERNAME=<username>
   export REGISTRY_PASSWORD=<password>
   export REGISTRY_CA=<path to the cacert file on the bastion>
   ```

3. Execute the following command to load the air-gapped image bundle into your private registry using any of the relevant flags to apply variables above.

```
nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

> **Note:** It might take some time to push all the images to your image registry, depending on the network performance of the machine you are running the script on and the registry.

Kommander Load Images

About this task

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander component images, is required. See below for instructions on how to push the necessary images to this registry.

Procedure

1. Load the Kommander images into your private registry using the command below to load the image bundle.

```
nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

2. Optional Step for **Ultimate License** to load **NKP Catalog Applications** images.

```
nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

vSphere Air-gapped FIPS: Creating a CAPI VM Template

The Konvoy Image Builder (KIB) uses the values in `image.yaml` and the input base OS image to create a vSphere template directly on the vCenter server.

About this task

You must have at least one image before creating a new cluster. If you have an image, this step in your configuration is not required each time since that image can be used to spin up a new cluster. However, if you need different images for different environments or providers, you will need to create a new custom image.

> **Note:** Users need to perform the steps in the topic vSphere: Creating an Image before starting this procedure.

Procedure

1. Assuming you have downloaded `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`, extract the tar file to a local directory.

```
tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
```

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.

3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the new NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`. Example command.

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   » For FIPS, pass the flag: `--fips`

   » For Red Hat Enterprise Linux (RHEL) OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`. Example command:

   ```
   export RHSM_ACTIVATION_KEY="-ci"
   export RHSM_ORG_ID="1232131"
   ```

4. Build an image template with Konvoy Image Builder (KIB).

5. Follow the instructions to build a vSphere template below, and if applicable, set the override `--overrides overrides/offline.yaml` flag described in Step 4 below.

Create a vSphere Template for Your Cluster from a Base OS Image

Procedure

1. Set the following vSphere environment variables on the bastion VM host.

   ```
   export VSPHERE_SERVER=your_vCenter_APIserver_URL
   export VSPHERE_USERNAME=your_vCenter_user_name
   export VSPHERE_PASSWORD=your_vCenter_password
   ```

2. Copy the base OS image file created in the vSphere Client to your desired location on the bastion VM host and make a note of the path and file name.

3. Create an `image.yaml` file and add the following variables for vSphere. NKP uses this file and these variables as inputs in the next step. To customize your `image.yaml` file, refer to this section: Customize your Image.

   Note: This example is Ubuntu 20.04. You will need to replace the OS name below based on your OS. Also, refer to the example YAML files located here: OVA YAML.

   ```
   ---
   download_images: true
   build_name: "ubuntu-2004"
   packer_builder_type: "vsphere"
   guestinfo_datasource_slug: "https://raw.githubusercontent.com/vmware/cloud-init-
   vmware-guestinfo"
   guestinfo_datasource_ref: "v1.4.0"
   guestinfo_datasource_script: "{{guestinfo_datasource_slug}}/
   {{guestinfo_datasource_ref}}/install.sh"
   packer:
     cluster: "<VSPHERE_CLUSTER_NAME>"
     datacenter: "<VSPHERE_DATACENTER_NAME>"
     datastore: "<VSPHERE_DATASTORE_NAME>"
     folder: "<VSPHERE_FOLDER>"
     insecure_connection: "false"
     network: "<VSPHERE_NETWORK>"
     resource_pool: "<VSPHERE_RESOURCE_POOL>"
     template: "os-qualification-templates/d2iq-base-Ubuntu-20.04" # change default
    value with your base template name
     vsphere_guest_os_type: "other4xLinux64Guest"
   ```

```
  guest_os_type: "ubuntu2004-64"
  # goss params
  distribution: "ubuntu"
  distribution_version: "20.04"
# Use following overrides to select the authentication method that can be used with
 base template
# ssh_username: ""  # can be exported as environment variable 'SSH_USERNAME'
# ssh_password: "" # can be exported as environment variable 'SSH_PASSWORD'
# ssh_private_key_file = "" # can be exported as environment variable
 'SSH_PRIVATE_KEY_FILE'
# ssh_agent_auth: false  # is set to true, ssh_password and ssh_private_key will be
 ignored
```

4. Create a vSphere VM template with your variation of the following command.

```
konvoy-image build images/ova/<image.yaml>
```

   - Any additional configurations can be added to this command using `--overrides` flags as shown below:

     1. Any credential overrides: `--overrides overrides.yaml`
     2. for FIPS, add this flag: `--overrides overrides/fips.yaml`
     3. for air-gapped, add this flag: `--overrides overrides/offline-fips.yaml`

5. The Konvoy Image Builder (KIB) uses the values in `image.yaml` and the input base OS image to create a vSphere template directly on the vCenter server. This template contains the required artifacts needed to create a Kubernetes cluster. When KIB successfully provisions the OS image, it creates a manifest file. The `artifact_id` field of this file contains the name of the AMI ID (AWS), template name (vSphere), or image name (GCP/Azure), for example.

```
{
      "name": "vsphere-clone",
      "builder_type": "vsphere-clone",
      "build_time": 1644985039,
      "files": null,
      "artifact_id": "konvoy-ova-vsphere-rhel-8-1.21.6-1644983717",
      "packer_run_uuid": "260e8110-77f8-ca94-e29e-ac7a2ae779c8",
      "custom_data": {
        "build_date": "2022-02-16T03:55:17Z",
        "build_name": "vsphere-rhel-8",
        "build_timestamp": "1644983717",
        [...]
      }
    }
```

   Tip: Recommendation: Now that you can now see the template created in your vCenter, it is best to rename it to `nkp-<NKP_VERSION>-k8s-<K8S_VERSION>-<DISTRO>`, like `nkp-2.4.0-k8s-1.24.6-ubuntu` to keep templates organized.

6. The next steps are to deploy a NKP cluster using your vSphere template.

vSphere Air-gapped FIPS: Creating the Management Cluster

Create a vSphere Management Cluster in an air-gapped environment using FIPS.

About this task

Use this procedure to create a self-managed **Management cluster** with NKP. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing. First, you must name your cluster.

Before you begin

Procedure

- The table below identifies the current FIPS and `etcd` versions for this release.

Table 21: Supported FIPS Builds

Component	Repository	Version
Kubernetes	docker.io/mesosphere	v1.29.6+fips.0
etcd	docker.io/mesosphere	3.5.10+fips.0

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

- Name your cluster and give it a unique name suitable for your environment.
- Set the environment variable:

```
export CLUSTER_NAME=<my-vsphere-cluster>
```

Create a New vSphere Kubernetes Cluster

About this task

The below instructions tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, it will be created in the `default` workspace, and you need to take additional steps to attach to a workspace later. For instructions on how to do this, see Attach a Kubernetes Cluster.

Procedure

1. Configure your cluster to use an existing local registry as a mirror when attempting to pull images: IMPORTANT: The image must be created by Konvoy Image Builder in order to use the registry mirror feature. Use the following command to set the environment variables for vSphere.

```
export VSPHERE_SERVER=example.vsphere.url
export VSPHERE_USERNAME=user@example.vsphere.url
export VSPHERE_PASSWORD=example_password
```

2. Load the image, using either the `docker` or `podman` command

   » `docker load -i konvoy-bootstrap-image-v2.12.0.tar`

   » `podman load -i konvoy-bootstrap-image-v2.12.0.tar`

3. Generate the Kubernetes cluster objects by copying and editing this command to include the correct values, including the VM template name you assigned in the previous procedure.

```
nkp create cluster vsphere
  --cluster-name ${CLUSTER_NAME} \
  --network <NETWORK_NAME> \
  --control-plane-endpoint-host <CONTROL_PLANE_IP> \
  --data-center <DATACENTER_NAME> \
```

```
--data-store <DATASTORE_NAME> \
--folder <FOLDER_NAME> \
--server <VCENTER_API_SERVER_URL> \
--ssh-public-key-file </path/to/key.pub> \
--resource-pool <RESOURCE_POOL_NAME> \
--vm-template konvoy-ova-vsphere-os-release-k8s_release-vsphere-timestamp \
--virtual-ip-interface <ip_interface_name> \
--extra-sans "127.0.0.1" \
--registry-mirror-url=${REGISTRY_URL} \
--registry-mirror-cacert=${REGISTRY_CA} \
--registry-mirror-username=${REGISTRY_USERNAME} \
--registry-mirror-password=${REGISTRY_PASSWORD} \
--kubernetes-version=v1.29.6+fips.0 \
--kubernetes-image-repository=docker.io/mesosphere \
--etcd-image-repository=docker.io/mesosphere --etcd-version=3.5.10+fips.0 \
--self-managed
```

Note: To increase Dockerhub's rate limit, use your Dockerhub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

» Flatcar OS flag: Flatcar OS use `--os-hint flatcar` to instruct the bootstrap cluster to make some changes related to the installation paths:

» HTTP: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

vSphere Air-gapped FIPS: Configure MetalLB

Create a MetalLB configmap for your Pre-provisioned Insfrastructure.

It is recommended that an external load balancer (LB) be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines that are responding to API requests. If you do not have one, you can use Metal LB to create a MetalLB configmap for your Pre-provisioned infrastructure.

Choose one of the following two protocols you want to use to announce service IPs. If your environment is not currently equipped with a load balancer, you can use MetalLB. Otherwise, your load balancer will work, and you can continue the installation process with Pre-provisioned: Install Kommander. To use MetalLB, create a MetalLB configMap for your Pre-provisioned infrastructure. MetalLB uses one of two protocols for exposing Kubernetes services:

- Layer 2, with Address Resolution Protocol (ARP)
- Border Gateway Protocol (BGP)

  Select one of the following procedures to create your MetalLB manifest for further editing.

Layer 2 Configuration

Layer 2 mode is the simplest to configure: in many cases, you don’t need any protocol-specific configuration, only IP addresses.

Layer 2 mode does not require the IPs to be bound to the network interfaces of your worker nodes. It works by responding to ARP requests on your local network directly and giving the machine’s MAC address to clients.

- MetalLB IP address ranges or CIDR need to be within the node’s primary network subnet.

- MetalLB IP address ranges or CIDRs and node subnets must not conflict with the Kubernetes cluster pod and service subnets.

For example, the following configuration gives MetalLB control over IPs from 192.168.1.240 to 192.168.1.250 and configures Layer 2 mode:

The following values are generi; enterr your specific values into the fields where applicable.

; enter

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    address-pools:
    - name: default
      protocol: layer2
      addresses:
      - 192.168.1.240-192.168.1.250
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

BGP Configuration

For a basic configuration featuring one BGP router and one IP address range, you need four pieces of information:

- The router IP address that MetalLB needs to connect to.
- The router's autonomous systems (AS) number.
- The AS number MetalLB is to be used.
- An IP address range is expressed as a CIDR prefix.

As an example, if you want to give MetalLB the range 192.168.10.0/24 and AS number 64500 and connect it to a router at 10.0.0.1 with AS number 64501, your configuration will look like this:

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    peers:
    - peer-address: 10.0.0.1
      peer-asn: 64501
      my-asn: 64500
    address-pools:
    - name: default
      protocol: bgp
      addresses:
      - 192.168.10.0/24
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

vSphere Air-gapped FIPS: Installing Kommander

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped vSphere environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

> **Tip:** Tips and Recommendations
>
> - The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
> - Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
> - If the Kommander installation fails, or you want to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. If required: Customize your `kommander.yaml`.

   a. See the Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.

5. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `dkp-catalog-appliations`.

   ```
   apiVersion: config.kommander.mesosphere.io/v1alpha1
   kind: Installation
   catalog:
     repositories:
       - name: dkp-catalog-applications
         labels:
   ```

```
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
    gitRepositorySpec:
      url: https://github.com/mesosphere/dkp-catalog-applications
      ref:
        tag: v2.12.0
```

6. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see the topic Configuring NKP Catalog Applications after Installing NKP.

vSphere Air-gapped FIPS: Verifying your Installation and UI Log in
Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      nkp experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of NKP. The Cluster Operations section allows you to manage cluster operations and their application workloads to optimize your organization’s productivity.

- Continue to the NKP Dashboard.

vSphere Air-gapped FIPS: Creating Managed Clusters Using the NKP CLI
This topic explains how to continue using the CLI to create managed clusters rather than switching to the UI dashboard.

About this task

After initial cluster creation, you have the ability to create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed which allows it to be a Management cluster or a stand alone cluster. Subsequent new clusters are not self-managed as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects or install the Kommander component. Those tasks are only done on Management clusters!
>
> Your new managed cluster needs to be part of a workspace under a management cluster. To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

   ```
   kubectl get workspace -A
   ```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

   > **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to create the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First, you must name your cluster. Then, you run the command to deploy it.

> **Note:** The cluster name intomight only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

   ```
   export CLUSTER_NAME=<my-managed-vsphere-cluster>
   ```

Create a Kubernetes Cluster

About this task

The below instructions tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, it will be created in the `default` workspace, and you need to take additional steps to attach to a workspace later. For instructions on how to do this, see Attach a Kubernetes Cluster.

Procedure

1. Configure your cluster to use an existing local registry as a mirror when attempting to pull images: IMPORTANT: The image must be created by Konvoy Image Builder in order to use the registry mirror feature..

```
export REGISTRY_URL=<https/http>://<registry-address>:<registry-port>
export REGISTRY_CA=<path to the CA on the bastion>
export REGISTRY_USERNAME=<username>
export REGISTRY_PASSWORD=<password>
```

2. Load the images, using either the `docker` or `podman` command.

   » `docker load -i konvoy-bootstrap-image-v2.12.0.tar`

   » `podman load -i konvoy-bootstrap-image-v2.12.0.tar`

3. Generate the Kubernetes cluster objects by copying and editing this command to include the correct values, including the VM template name you assigned in the previous procedure.

```
nkp create cluster vsphere \
  --cluster-name ${MANAGED_CLUSTER_NAME} \
  --additional-tags=owner=$(whoami) \
  --namespace ${WORKSPACE_NAMESPACE}
  --network <NETWORK_NAME> \
  --control-plane-endpoint-host <CONTROL_PLANE_IP> \
  --data-center <DATACENTER_NAME> \
  --data-store <DATASTORE_NAME> \
  --folder <FOLDER_NAME> \
  --server <VCENTER_API_SERVER_URL> \
  --ssh-public-key-file </path/to/key.pub> \e
  --resource-pool <RESOURCE_POOL_NAME> \
  --vm-template konvoy-ova-vsphere-os-release-k8s_release-vsphere-timestamp \
  --virtual-ip-interface <ip_interface_name> \
  --extra-sans "127.0.0.1" \
  --registry-mirror-url=${REGISTRY_URL} \
  --registry-mirror-cacert=${REGISTRY_CA} \
  --registry-mirror-username=${REGISTRY_USERNAME} \
  --registry-mirror-password=${REGISTRY_PASSWORD} \
  --kubernetes-version=v1.29.6+fips.0 \
  --kubernetes-image-repository=docker.io/mesosphere \
  --etcd-image-repository=docker.io/mesosphere \
  --etcd-version=3.5.10+fips.0 \
  --kubeconfig=<management-cluster-kubeconfig-path> \
```

> **Tip:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the Nutanix Kubernetes Platform (NKP) CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

   ```
   echo ${MANAGED_CLUSTER_NAME}
   ```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

   ```
   nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
    ${MANAGED_CLUSTER_NAME}.conf
   ```

3. **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to the workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

   ```
   kubectl get workspaces -A
   ```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace-namespace>
   ```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

   ```
   kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
   template='{{.data.value}}{{ "\n"}}'
   ```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <your-managed-cluster-name>-kubeconfig
     labels:
       cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
   type: cluster.x-k8s.io/secret
   data:
     value: <value-you-copied-from-secret-above>
   ```

8. Create this secret in the desired workspace.

   ```
   kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
    ${WORKSPACE_NAMESPACE}
   ```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

   ```
   cat << EOF | kubectl apply -f -
   apiVersion: kommander.mesosphere.io/v1beta1
   kind: KommanderCluster
   metadata:
   ```

```
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

    ```
    kubectl get kommanderclusters -A
    ```

    If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

Next Step

Procedure

- Cluster Operations Management

VMware Cloud Director Installation Options

There is not a “Basic Install” for VMWare Cloud Director since each tenant will have different needs. For Managed Service Providers (MSPs), refer to the Cloud Director for Service Providers section of the documentation regarding installation plus image and OVA export and import for your Tenant Organizations.

Before continuing to install Nutanix Kubernetes Platform (NKP) on Cloud Director, verify that your VMware vSphere Client environment is running vSphere Client version v7.0.3 or later version with ESXi. You must be able to reach the vSphere API endpoint from where the Konvoy command line interface (CLI) runs and have a vSphere account containing Administrator privileges. A RedHat subscription is required with username and password for downloading DVD ISOs and valid vSphere values for the following: vCenter API server URL Datacenter name Zone name that contains ESXi hosts for your cluster’s nodes.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Next Step

Continue to the VMware Cloud Director Infrastructure ice Providers section of the Custom Install and Infrastructure Tools chapter.

Azure Installation Options

For an environment that is on the Azure Infrastructure, install options based on those environment variables are provided for you in this location.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operative in the most common scenarios.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Additional Resource Information Specific to Azure

- Control plane nodes - NKP on Azure defaults to deploying a `Standard_D4s_v3` virtual machine with a 128 GiB volume for the OS and an 80GiB volume for etcd storage, which meets the above resource requirements.
- Worker nodes - NKP on Azure defaults to deploying a `Standard_D8s_v3` virtual machine with an 80 GiB volume for the OS, which meets the above resource requirements.

Azure Installation

This installation provides instructions on how to install NKP in an Azure non-air-gapped environment.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Azure Prerequisites

Before you begin using Konvoy with Azure, you must:

1. Sign in to Azure:

```
az login
```

```
[
  {
    "cloudName": "AzureCloud",
    "homeTenantId": "a1234567-b132-1234-1a11-1234a5678b90",
    "id": "b1234567-abcd-11a1-a0a0-1234a5678b90",
    "isDefault": true,
    "managedByTenants": [],
    "name": "Nutanix Developer Subscription",
    "state": "Enabled",
    "tenantId": "a1234567-b132-1234-1a11-1234a5678b90",
    "user": {
      "name": "user@azuremesosphere.onmicrosoft.com",
      "type": "user"
    }
  }
]
```

2. Create an Azure Service Principal (SP) by running the following commands:
   1. If you have more than one Azure account, run this command to identify your account:

      ```
      echo $(az account show --query id -o tsv)
      ```

   2. Run this command to ensure you are pointing to the correct Azure subscription ID:

      ```
      az account set --subscription "Nutanix Developer Subscription"
      ```

   3. If an SP with the name exists, this command rotates the password:

      ```
      az ad sp create-for-rbac --role contributor --name "$(whoami)-konvoy" --scopes=/
      subscriptions/$(az account show --query id -o tsv) --query "{ client_id: appId,
       client_secret: password, tenant_id: tenant }"
      ```

      Output:

      ```
      {
        "client_id": "7654321a-1a23-567b-b789-0987b6543a21",
        "client_secret": "Z79yVstq_E.R0R7RUUck718vEHSuyhAB0C",
        "tenant_id": "a1234567-b132-1234-1a11-1234a5678b90"
      }
      ```

3. Set the `AZURE_CLIENT_SECRET` environment variable:

   ```
   export AZURE_CLIENT_SECRET="<azure_client_secret>" #
    Z79yVstq_E.R0R7RUUck718vEHSuyhAB0C
   export AZURE_CLIENT_ID="<client_id>"               # 7654321a-1a23-567b-
   b789-0987b6543a21
   export AZURE_TENANT_ID="<tenant_id>"               # a1234567-
   b132-1234-1a11-1234a5678b90
   export AZURE_SUBSCRIPTION_ID="<subscription_id>"   # b1234567-abcd-11a1-
   a0a0-1234a5678b90
   ```

4. Ensure you have an override file to configure specific attributes of your Azure image.

Azure: Creating an Image

Learn how to build a custom image for use with NKP.

About this task

Extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION_$OS` folder. The bundled version of `konvoy-image` contains an embedded `docker` image that contains all the requirements for the building.

> **Note:** The `konvoy-image` binary and all supporting folders are also extracted. When extracted, `konvoy-image` bind mounts the current working directory (`${PWD}`) into the container to be used.

This procedure describes how to use the Konvoy Image Builder (KIB) to create a Cluster API compliant Amazon Machine Image (AMI). KIB uses variable overrides to specify base images and container images to use in your new AMI.

> **Note:** The default Azure image is not recommended for use in production. We suggest using KIB for Azure to build the image to take advantage of enhanced cluster operations. For more options, see the Customize your Image topic.
>
> For more information regarding using the image in creating clusters, see Creating a New Azure Cluster on page 892.

Before you begin

- Download the Konvoy Image Builder bundle for your version of NKP.

- Check the Supported Kubernetes Version for your Provider.
- Create a working `Docker` setup.

Extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION_$OS` folder. The bundled version of `konvoy-image` contains an embedded `docker` image that contains all the requirements for the building.

The `konvoy-image` binary and all supporting folders are also extracted. When extracted, `konvoy-image` bind mounts the current working directory (`${PWD}`) into the container to be used.

Procedure

Run the `konvoy-image` command to build and validate the image.

```
konvoy-image build azure --client-id ${AZURE_CLIENT_ID} --tenant-id ${AZURE_TENANT_ID}
 --overrides override-source-image.yaml images/azure/ubuntu-2004.yaml
```

By default, the image builder builds in the `westus2` location. To specify another location, set the `--location` flag (shown in the example below is how to change the location to `eastus`):

```
konvoy-image build azure --client-id ${AZURE_CLIENT_ID} --tenant-id ${AZURE_TENANT_ID}
 --location eastus --overrides override-source-image.yaml images/azure/centos-7.yaml
```

When the command is complete, the image id is printed and written to the `./packer.pkr.hcl` file. This file has an `artifact_id` field whose value provides the name of the image. Then, specify this image ID when creating the cluster.

Image Gallery

About this task

By default Konvoy Image Builder will create a Resource Group, Gallery, and Image Name to store the resulting image in.

Procedure

- To specify a specific Resource Group, Gallery, or Image Name flags might be specified:

```
--gallery-image-locations string     a list of locations to publish the image
 (default same as location)
--gallery-image-name string          the gallery image name to publish the image to
--gallery-image-offer string         the gallery image offer to set (default "nkp")
--gallery-image-publisher string     the gallery image publisher to set (default
 "nkp")
--gallery-image-sku string           the gallery image sku to set
--gallery-name string                the gallery name to publish the image in
 (default "nkp")
--resource-group string              the resource group to create the image in
 (default "nkp")
```

Azure: Creating the Management Cluster

Create an Azure Management Cluster in a non-air-gapped environment.

About this task

Use this procedure to create a self-managed **Management cluster** with NKP. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing. First, you must name your cluster.

Name Your Cluster

Note: The cluster name might only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. For more naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable using the command `export CLUSTER_NAME=<azure-example>`.

Encode your Azure Credential Variables:

Procedure

Base64 encodes the Azure environment variables set in the Azure install prerequisites step.

```
export AZURE_SUBSCRIPTION_ID_B64="$(echo -n "${AZURE_SUBSCRIPTION_ID}" | base64 | tr -d
 '\n')"
export AZURE_TENANT_ID_B64="$(echo -n "${AZURE_TENANT_ID}" | base64 | tr -d '\n')"
export AZURE_CLIENT_ID_B64="$(echo -n "${AZURE_CLIENT_ID}" | base64 | tr -d '\n')"
export AZURE_CLIENT_SECRET_B64="$(echo -n "${AZURE_CLIENT_SECRET}" | base64 | tr -d
 '\n')"
```

Create an Azure Kubernetes Cluster

About this task

If you use these instructions to create a cluster on Azure using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes.

NKP uses Azure CSI as the default storage provider. You can use a Kubernetes CSIcompatible storage solution that is suitable for production. See the Kubernetes documentation called Changing the Default Storage Class for more information.

Availability zones (AZs) are isolated locations within datacenter regions from which public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single AZ; you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple AZs.

Procedure

Run this command to create your Kubernetes cluster using any relevant flags.

```
nkp create cluster azure \
--cluster-name=${CLUSTER_NAME} \
--self-managed
```

Note: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

If you want to monitor or verify the installation of your clusters, refer to the topic: Verify your Cluster and NKP Installation

Azure: Install Kommander

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped Azure environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you want to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

```
export CLUSTER_NAME=<your-management-cluster-name>
```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

```
nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
```

3. Create a configuration file for the deployment.

```
nkp install kommander --init > kommander.yaml
```

4. If required: Customize your `kommander.yaml`.

   a. See the Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.

5. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `NKP-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
```

```
      url: https://github.com/mesosphere/dkp-catalog-applications
      ref:
        tag: v2.12.0
```

6. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see the topic Configuring NKP Catalog Applications after Installing NKP.

Azure: Verifying your Installation and UI Log in

Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      nkp experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Day 2 Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of NKP. The Day 2 section allows you to manage cluster operations and their application workloads to optimize your organization’s productivity.

- Continue to the NKP Dashboard.

Azure: Creating Managed Clusters Using the NKP CLI

This topic explains how to continue using the CLI to create managed clusters rather than switching to the UI dashboard.

About this task

After ,initial cluster creation; you can create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed, which allows it to be a Management cluster or a stand-alone cluster. Subsequent new clusters are not self-managed, as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects or install the Kommander component. Those tasks are only done on Management clusters!
>
> Your new managed cluster needs to be part of a workspace under a management cluster. To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

   ```
   kubectl get workspace -A
   ```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

   > **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to creating the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First, you must name your cluster. Then yo,u run the command to deploy it.

> **Note:** The cluster name might only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

   ```
   export MANAGED_CLUSTER_NAME=<MANAGED_CLUSTER_NAME>
   ```

Create a Kubernetes Cluster

About this task

The below instructions tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, it will be created in the `default` workspace, and you need to take additional steps to attach to a workspace later. For instructions on how to do this, see Attach a Kubernetes Cluster.

Procedure

Execute this command to create your additional Kubernetes cluster using any relevant flags. This will create a new non-self-managed cluster that can be managed by management cluster you created in the previous section.

```
nkp create cluster azure \
--cluster-name=${MANAGED_CLUSTER_NAME} \
--namespace=${WORKSPACE_NAMESPACE} \
--additional-tags=owner=$(whoami) \
--kubeconfig=<management-cluster-kubeconfig-path>
```

> **Tip:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

   ```
   echo ${MANAGED_CLUSTER_NAME}
   ```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

   ```
   nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
    ${MANAGED_CLUSTER_NAME}.conf
   ```

3. > **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

   ```
   kubectl get workspaces -A
   ```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace-namespace>
   ```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

```
kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
template='{{.data.value}}{{ "\n"}}'
```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

```
apiVersion: v1
kind: Secret
metadata:
  name: <your-managed-cluster-name>-kubeconfig
  labels:
    cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
type: cluster.x-k8s.io/secret
data:
  value: <value-you-copied-from-secret-above>
```

8. Create this secret in the desired workspace.

```
kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
 ${WORKSPACE_NAMESPACE}
```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the below command. It may take a few minutes to reach "Joined" status.

```
kubectl get kommanderclusters -A
```

If you have several Pro Clusters and want to turn one of them to a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

Next Step

Procedure

- Cluster Operations Management

AKS Installation Options

For an environment that is on the Azure Kubernetes Service (AKS) Infrastructure, installation options based on those environment variables are provided for you in this location.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operative in the most common scenarios.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

> **Note:** An AKS cluster cannot be a Management or Pro cluster. Before installing NKP on your EKS cluster, first ensure you have a Management cluster with NKP and the Kommander component installed that handles the life cycle of your AKS cluster.

Installing Kommander requires you to have CAPI components, cert-manager, etc, on a self-managed cluster. The CAPI components mean you can control the life cycle of the cluster and other clusters. However, because AKS is semi-managed by Azure, the AKS clusters are under Azure's control and don't have those components. Therefore, Kommander will not be installed.

AKS Installation

Nutanix Kubernetes Platform (NKP) installation on Azure Kubernetes Service (AKS).

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

For additional custom YAML Ain't Markup Language (YAML) options, see Custom Installation and Additional Infrastructure Tools.

AKS Prerequisites

Before you begin using Konvoy with AKS, you must:

1. Sign in to Azure using the command `az login`. For example:

```
[
  {
    "cloudName": "AzureCloud",
    "homeTenantId": "a1234567-b132-1234-1a11-1234a5678b90",
    "id": "b1234567-abcd-11a1-a0a0-1234a5678b90",
    "isDefault": true,
    "managedByTenants": [],
    "name": "Nutanix Developer Subscription",
    "state": "Enabled",
    "tenantId": "a1234567-b132-1234-1a11-1234a5678b90",
    "user": {
      "name": "user@azuremesosphere.onmicrosoft.com",
      "type": "user"
    }
  }
]
```

2. Create an Azure Service Principal (SP) by using the command `az ad sp create-for-rbac --role contributor --name "$(whoami)-konvoy" --scopes=/subscriptions/$(az account show --query id -o tsv)`.

3. Set the Azure client secret environment variable using the command `AZURE_CLIENT_SECRET`. Example output:

```
export AZURE_CLIENT_SECRET="<azure_client_secret>" #
 Z79yVstq_E.R0R7RUUck718vEHSuyhAB0C
export AZURE_CLIENT_ID="<client_id>"               # 7654321a-1a23-567b-
b789-0987b6543a21
export AZURE_TENANT_ID="<tenant_id>"               # a1234567-
b132-1234-1a11-1234a5678b90
export AZURE_SUBSCRIPTION_ID="<subscription_id>"   # b1234567-abcd-11a1-
a0a0-1234a5678b90
```

4. Base64 encodes the same environment variables:

```
export AZURE_SUBSCRIPTION_ID_B64="$(echo -n "${AZURE_SUBSCRIPTION_ID}" | base64 | tr
 -d '\n')"
export AZURE_TENANT_ID_B64="$(echo -n "${AZURE_TENANT_ID}" | base64 | tr -d '\n')"
export AZURE_CLIENT_ID_B64="$(echo -n "${AZURE_CLIENT_ID}" | base64 | tr -d '\n')"
export AZURE_CLIENT_SECRET_B64="$(echo -n "${AZURE_CLIENT_SECRET}" | base64 | tr -d
 '\n')"
```

5. Check to see what version of Kubernetes is available in your region. When deploying with AKS, you must pick a version of Kubernetes that is available in AKS and use that version for subsequent steps. To find out the list of available Kubernetes versions in the Azure Region you are using, run the following command, substituting `<your-location>` for the Azure region you're deploying to:

   1. `az aks get-versions -o table --location <your-location>`
   2. The output resembles the following:

```
az aks get-versions -o table --location westus
KubernetesVersion    Upgrades
-------------------  ----------------------------------------
1.27.6(preview)      None available
1.27.3(preview)      1.27.6(preview)
1.27.1(preview)      1.27.3(preview)
1.26.6               1.27.1(preview), 1.27.3(preview)
1.26.3               1.26.6, 1.27.1(preview), 1.27.3(preview)
1.25.11              1.26.3, 1.26.6
1.25.6               1.25.11, 1.26.3, 1.26.6
1.24.15              1.25.6, 1.25.11
1.24.10              1.24.15, 1.25.6, 1.25.11
```

6. Choose a version of Kubernetes for installation from the list using the command `KubernetesVersion` The example shows the selected version is 1.29.0.

```
export KUBERNETES_VERSION=1.29.0
```

   For the list of compatible supported Kubernetes versions, see Supported Kubernetes Versions.

NKP Prerequisites

Before starting the NKP installation, verify that you have:

- A Management cluster with NKP and the Kommander component installed.

  Note: An AKS cluster cannot be a Management or Pro cluster. Before installing NKP on your AKS cluster, ensure you have a Management cluster with NKP and the Kommander component installed, that handles the life cycle of your AKS cluster.

  - An x86_64-based Linux or macOS machine with a supported version of the operating system.
  - A Self-managed Azure cluster, if you used the Day 1-Basic Installation for Azure instructions, your cluster was created using `--self-managed` flag and therefore is already a self-managed cluster.
  - Download the `NKP`binary for Linux, or macOS. To check which version of NKP you installed for compatibility reasons, run the `NKP version -h` command.
  - Docker https://docs.docker.com/get-docker/ version 27.4.0.
  - kubectl https://kubernetes.io/docs/tasks/tools/#kubectl for interacting with the running cluster.
  - The Azure CLI https://docs.microsoft.com/en-us/cli/azure/install-azure-cli.
  - A valid Azure account used to sign in to the Azure CLI https://docs.microsoft.com/en-us/cli/azure/authenticate-azure-cli?view=azure-cli-latest.
  - All Resource requirements.

  Note: Kommander installation requires you to have Cluster API (CAPI) components, cert-manager, etc on a self-managed cluster. The CAPI components mean you can control the life cycle of the cluster, and other clusters. However, because AKS is semi-managed by Azure, the AKS clusters are under Azure control and don't have those components. Therefore, Kommander will not be installed and these clusters will be attached to the management cluster.

To deploy a cluster with a custom image in a region where CAPI images https://cluster-api-aws.sigs.k8s.io/topics/images/built-amis.html are not provided, you need to use Konvoy Image Builder to create your own image for the region.

AKS best practices discourage building custom images. If the image is customized, it breaks some of the autoscaling and security capabilities of AKS. Since custom virtual machine images are discouraged in AKS, Konvoy Image Builder (KIB) does not include any support for building custom machine images for AKS.

AKS: Create an AKS Cluster

About this task

When creating a Managed cluster on your AKS infrastructure, you can choose from multiple configuration types.

Procedure

Use NKP to create a new AKS cluster

Ensure that the `KUBECONFIG` environment variable is set to the Management cluster by running :

```
export KUBECONFIG=<Management_cluster_kubeconfig>.conf
```

Name Your Cluster

Procedure

1. Give your cluster a unique name suitable for your environment.

2. Set the environment variable.

```
export CLUSTER_NAME=<aks-example>
```

> **Note:** The cluster name might only contain the following characters: `a-z`, `0-9`, `.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

Create a New AKS Kubernetes Cluster from the CLI

About this task

Procedure

1. Set the environment variable to the name you assigned this cluster.

```
export CLUSTER_NAME=<aks-example>
```

2. Check to see what version of Kubernetes is available in your region. When deploying with AKS, you need to declare the version of Kubernetes you want to use by running the following command, substituting <your-location> for the Azure region you're deploying to.

```
az aks get-versions -o table --location <your-location>
```

3. Set the Kubernetes version you have chosen.

```
export KUBERNETES_VERSION=1.27.6
```

4.

> **Note:** Refer to the current release Kubernetes compatibility table for the correct version to use and choose an available 1.27.x version. The version listed in the command is an example.

Create the cluster.

```
nkp create cluster aks --cluster-name=${CLUSTER_NAME} --additional-tags=owner=
$(whoami) --kubernetes-version=${KUBERNETES_VERSION}
```

5. Inspect or edit the cluster objects. Familiarize yourself with Cluster API before editing the cluster objects, as edits can prevent the cluster from deploying successfully. See Customizing CAPI Clusters.

6. Wait for the cluster control-plane to be ready.

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=20m
```

The `READY` status will become `True` after the cluster control-plane becomes ready in one of the following steps.

7. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. Konvoy provides a command to describe the current status of the cluster.

```
nkp describe cluster -c ${CLUSTER_NAME}
```

```
NAME                                                    READY  SEVERITY  REASON
 SINCE  MESSAGE
Cluster/aks-example                                     True
 48m
##ClusterInfrastructure - AzureManagedCluster/aks-example
##ControlPlane - AzureManagedControlPlane/aks-example
```

8. As they progress, the controllers also create Events. List the Events using this command.

```
kubectl get events | grep ${CLUSTER_NAME}
```

For brevity, the example uses `grep`. It is also possible to use separate commands to get Events for specific objects. For example, `kubectl get events --field-selector involvedObject.kind="AKSCluster"` and `kubectl get events --field-selector involvedObject.kind="AKSMachine"`.

```
48m         Normal    SuccessfulSetNodeRefs               machinepool/aks-
example-md-0                  [{Kind: Namespace: Name:aks-mp6gglj-41174201-
vmss000000 UID:e3c30389-660d-46f5-b9d7-219f80b5674d APIVersion: ResourceVersion:
 FieldPath:} {Kind: Namespace: Name:aks-mp6gglj-41174201-vmss000001 UID:300d71a0-
f3a7-4c29-9ff1-1995ffb9cfd3 APIVersion: ResourceVersion: FieldPath:} {Kind:
 Namespace: Name:aks-mp6gglj-41174201-vmss000002 UID:8eae2b39-a415-425d-8417-
d915a0b2fa52 APIVersion: ResourceVersion: FieldPath:} {Kind: Namespace: Name:aks-
mp6gglj-41174201-vmss000003 UID:3e860b88-f1a4-44d1-b674-a54fad599a9d APIVersion:
 ResourceVersion: FieldPath:}]
6m4s        Normal    AzureManagedControlPlane available   azuremanagedcontrolplane/
aks-example          successfully reconciled
48m         Normal    SuccessfulSetNodeRefs               machinepool/aks-
example                       [{Kind: Namespace: Name:aks-mp6gglj-41174201-
vmss000000 UID:e3c30389-660d-46f5-b9d7-219f80b5674d APIVersion: ResourceVersion:
 FieldPath:} {Kind: Namespace: Name:aks-mp6gglj-41174201-vmss000001 UID:300d71a0-
f3a7-4c29-9ff1-1995ffb9cfd3 APIVersion: ResourceVersion: FieldPath:} {Kind:
 Namespace: Name:aks-mp6gglj-41174201-vmss000002 UID:8eae2b39-a415-425d-8417-
d915a0b2fa52 APIVersion: ResourceVersion: FieldPath:}]
```

AKS: Retrieve kubeconfig for AKS Cluster

Learn to interact with your AKS Kubernetes cluster.

About this task

This guide explains how to use the command line to interact with your newly deployed Kubernetes cluster. Before you start, make sure you have created a workload cluster, as described in AKS: Create an AKS Cluster.

Explore the new AKS cluster with the steps below.

Procedure

1. Get a `kubeconfig` file for the workload cluster. When the workload cluster is created, the cluster life cycle services generate a `kubeconfig` file for the workload cluster and write it to a Secret. The `kubeconfig` file is scoped to the cluster administrator. Get the `kubeconfig` from the Secret, and write it to a file using this command.

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

2. List the Nodes using this command.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get nodes
```

```
NAME                                STATUS   ROLES   AGE   VERSION
aks-cp6dsz8-41174201-vmss000000   Ready    agent   56m   v1.29.6
aks-cp6dsz8-41174201-vmss000001   Ready    agent   55m   v1.29.6
aks-cp6dsz8-41174201-vmss000002   Ready    agent   56m   v1.29.6
aks-mp6gglj-41174201-vmss000000   Ready    agent   55m   v1.29.6
aks-mp6gglj-41174201-vmss000001   Ready    agent   55m   v1.29.6
aks-mp6gglj-41174201-vmss000002   Ready    agent   55m   v1.29.6
```

```
aks-mp6gglj-41174201-vmss000003   Ready    agent   56m   v1.29.6
```

> **Note:**
>
> It might take a few minutes for the Status to move to `Ready` while the Pod network is deployed. The Node Status will change to Ready soon after the `calico-node` DaemonSet Pods are Ready.

3. List the Pods using the command `kubectl --kubeconfig=${CLUSTER_NAME}.conf get --all-namespaces pods`.

   Example output:

```
NAMESPACE                   NAME                                       READY
 STATUS     RESTARTS   AGE
calico-system               calico-kube-controllers-5dcd4b47b5-tgslm   1/1
 Running    0          3m58s
calico-system               calico-node-46dj9                          1/1
 Running    0          3m58s
calico-system               calico-node-crdgc                          1/1
 Running    0          3m58s
calico-system               calico-node-m7s7x                          1/1
 Running    0          3m58s
calico-system               calico-node-qfkqc                          1/1
 Running    0          3m57s
calico-system               calico-node-sfqfm                          1/1
 Running    0          3m57s
calico-system               calico-node-sn67x                          1/1
 Running    0          3m53s
calico-system               calico-node-w2pvt                          1/1
 Running    0          3m58s
calico-system               calico-typha-6f7f59969c-5z4t5              1/1
 Running    0          3m51s
calico-system               calico-typha-6f7f59969c-ddzqb              1/1
 Running    0          3m58s
calico-system               calico-typha-6f7f59969c-rr4lj              1/1
 Running    0          3m51s
kube-system                 azure-ip-masq-agent-4f4v6                  1/1
 Running    0          4m11s
kube-system                 azure-ip-masq-agent-5xfh2                  1/1
 Running    0          4m11s
kube-system                 azure-ip-masq-agent-9hlk8                  1/1
 Running    0          4m8s
kube-system                 azure-ip-masq-agent-9vsgg                  1/1
 Running    0          4m16s
```

AKS: Attach a Cluster

You can attach existing Kubernetes clusters to the Management Cluster using the instructions below.

About this task

After attaching the cluster, you can use the UI to examine and manage this cluster. The following procedure shows how to attach an existing Azure Kubernetes Service (AKS) cluster.

This procedure assumes you have an existing and spun up Azure AKS cluster(s) with administrative privileges. Refer to the Azure site regarding AKS for setup and configuration information.

spun-up Azure AKS cluster(s) with administrative privileges. For setup and configuration information, refer to the Azure site regarding ##AKS# that the `KUBECONFIG` environment variable is set to the Management cluster before attaching by running:

Note:

```
export KUBECONFIG=<Management_cluster_kubeconfig>.conf
```

Access Your AKS Clusters

Procedure

1. Ensure you are connected to your AKS clusters. Enter the following commands for each of your clusters.

   ```
   kubectl config get-contexts
   kubectl config use-context <context for first aks cluster>
   ```

2. Confirm `kubectl` can access the EKS cluster.

   ```
   kubectl get nodes
   ```

Create a kubeconfig File

About this task

To get started, ensure you have kubectl set up and configured with ClusterAdmin for the cluster you want to connect to Kommander.

Procedure

1. Create the necessary service account.

   ```
   kubectl -n kube-system create serviceaccount kommander-cluster-admin
   ```

2. Create a token secret for the `serviceaccount`.

   ```
   kubectl -n kube-system create  -f - <<EOF
   apiVersion: v1
   kind: Secret
   metadata:
     name: kommander-cluster-admin-sa-token
     annotations:
       kubernetes.io/service-account.name: kommander-cluster-admin
   type: kubernetes.io/service-account-token
   EOF
   ```

   For more information on Service Account Tokens, refer to this article in our blog.

3. Verify that the `serviceaccount` token is ready by running this command.

   ```
   kubectl -n kube-system get secret kommander-cluster-admin-sa-token -oyaml
   ```

   Verify that the `data.token` field is populated, as seen in the example output.

   ```
   apiVersion: v1
   data:
     ca.crt: LS0tLS1CRUdJTiBDR...
     namespace: ZGVmYXVsdA==
     token: ZXlKaGJHY2lPaUpTVX...
   kind: Secret
   metadata:
     annotations:
       kubernetes.io/service-account.name: kommander-cluster-admin
       kubernetes.io/service-account.uid: b62bc32e-b502-4654-921d-94a742e273a8
     creationTimestamp: "2022-08-19T13:36:42Z"
   ```

```
  name: kommander-cluster-admin-sa-token
  namespace: default
  resourceVersion: "8554"
  uid: 72c2a4f0-636d-4a70-9f1c-55a75f15e520
type: kubernetes.io/service-account-token
```

4. Configure the new service account for `cluster-admin` permissions.

```
cat << EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: kommander-cluster-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
  name: kommander-cluster-admin
  namespace: kube-system
EOF
```

5. Set up the following environment variables with the access data that is needed for producing a new `kubeconfig` file.

```
export USER_TOKEN_VALUE=$(kubectl -n kube-system get secret/kommander-cluster-admin-
sa-token -o=go-template='{{.data.token}}' | base64 --decode)
export CURRENT_CONTEXT=$(kubectl config current-context)
export CURRENT_CLUSTER=$(kubectl config view --raw -o=go-
template='{{range .contexts}}{{if eq .name "'''${CURRENT_CONTEXT}'''"}}
{{ index .context "cluster" }}{{end}}{{end}}')
export CLUSTER_CA=$(kubectl config view --raw -o=go-template='{{range .clusters}}{{if
 eq .name "'''${CURRENT_CLUSTER}'''"}}"{{with index .cluster "certificate-authority-
data" }}{{.}}{{end}}"{{ end }}{{ end }}')
export CLUSTER_SERVER=$(kubectl config view --raw -o=go-template='{{range .clusters}}
{{if eq .name "'''${CURRENT_CLUSTER}'''"}}{{ .cluster.server }}{{end}}{{ end }}')
```

6. Confirm these variables have been set correctly.

```
export -p | grep -E 'USER_TOKEN_VALUE|CURRENT_CONTEXT|CURRENT_CLUSTER|CLUSTER_CA|
CLUSTER_SERVER'
```

7. Generate a kubeconfig file that uses the environment variable values from the previous step.

> **Caution:** Nutanix support is unable to recover forgotten credentials. Prevent Kommander UI lockouts with the following best practices.
>
> - Backup the kubeconfig file.
> - Save your credentials.
> - Configure an IDP (identity provider) instead of using a username and password credentials.

```
cat << EOF > kommander-cluster-admin-config
apiVersion: v1
kind: Config
current-context: ${CURRENT_CONTEXT}
contexts:
- name: ${CURRENT_CONTEXT}
  context:
```

```
    cluster: ${CURRENT_CONTEXT}
    user: kommander-cluster-admin
    namespace: kube-system
clusters:
- name: ${CURRENT_CONTEXT}
  cluster:
    certificate-authority-data: ${CLUSTER_CA}
    server: ${CLUSTER_SERVER}
users:
- name: kommander-cluster-admin
  user:
    token: ${USER_TOKEN_VALUE}
EOF
```

8. This process produces a file in your current working directory called `kommander-cluster-admin-config`. The contents of this file are used in Kommander to attach the cluster. Before importing this configuration, verify the `kubeconfig` file can access the cluster.

   ```
   kubectl --kubeconfig $(pwd)/kommander-cluster-admin-config get all --all-namespaces
   ```

Finalize attaching your cluster from the UI

About this task

Now that you have the `kubeconfig`, go to the NKP UI and follow these steps below:

Procedure

From the top menu bar, select your target workspace.

a. On the Dashboard page, select the **Add Cluster** option in the Actions dropdown menu at the top right.

b. Select **Attach Cluster**.

c. Select the **No additional networking restrictions** card. Alternatively, if you must use network restrictions, stop following the steps below and see the Attach a cluster WITH network restrictions.

d. Upload the `kubeconfig` file you created in the previous section (or copy its contents) into the **Cluster Configuration** section.

e. The **Cluster Name** field automatically populates with the name of the cluster in the kubeconfig. You can edit this field using the name you want for your cluster.

f. Add labels to classify your cluster as needed.

g. Select **Create** to attach your cluster. Next Step

GCP Installation Options

For an environment that is on the GCP Infrastructure, install options based on those environment variables are provided for you in this location.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operative in the most common scenarios.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Additional Resource Information Specific to GCP

- Control plane nodes - NKP on GCP defaults to deploying an `n2-standard-4` instance with an 80GiB root volume for control plane nodes, which meets the above requirements.
- Worker nodes - NKP on GCP defaults to deploying a `n2-standard-8` instance with an 80GiB root volume for worker nodes, which meets the above requirements.

GCP Installation

This installation provides instructions to install NKP in an GCP non-air-gapped environment.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

GCP Prerequisites

Verify that your Google Cloud project does not have the Enable OS Login feature enabled.

> **Note:**
>
> The Enable OS Login feature is sometimes enabled by default in GCP projects. If the OS login feature is enabled, KIB will not be able to `ssh` to the VM instances it creates and will not be able to create an image successfully.
>
> To check if it is enabled, use the commands on this page Set and remove custom metadata | Compute Engine Documentation | Google Cloud to inspect the metadata configured in your project. If you find the `enable-oslogin` flag set to TRUE, you must remove it (or set it to FALSE) to use KIB.

The user creating the Service Accounts needs additional privileges in addition to the Editor role.

> **Note:** See GCP Roles for more information.

GCP: Creating an Image

Learn how to build a custom image for use with NKP.

About this task

This procedure describes how to use the Konvoy Image Builder (KIB) to create a Cluster API compliant GCP image. GCP images contain configuration information and software to create a specific, pre-configured, operating environment. For example, you can create a GCP image of your current computer system settings and software. The GCP image can then be replicated and distributed, creating your computer system for other users. KIB uses variable overrides to specify the base image and container images to use in your new AMI.

The **prerequisites** to use Konvoy Image Builder are:

Procedure

1. Download the KIB bundle for your version of NKP prefixed with `konvoy-image-bundle` for your OS.
2. Check the Supported Infrastructure Operating Systems.

3. Check the Supported Kubernetes Version for your Provider.
4. Create a working registry.
   - » Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
   - » Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
5. On Debian-based Linux distributions, install a version of the cri-tools package known to be compatible with both the Kubernetes and container runtime versions.
6. **Note:** The Enable OS Login feature is sometimes enabled by default in GCP projects. If the OS login feature is enabled, KIB will not be able to `ssh` to the VM instances it creates and will not be able to create an image successfully.

   To check if it is enabled, use the commands on this page Set and remove custom metadata | Compute Engine Documentation | Google Cloud to inspect the metadata configured in your project. If you find the `enable-oslogin` flag set to TRUE, you must remove (or set it to FALSE) to use KIB successfully.

   Verify that your Google Cloud project does not have the Enable OS Login feature enabled. See below for more information.

GCP Prerequisite Roles

About this task

If not done previously during Konvoy Image Builder download in Prerequisites, extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION` folder. Otherwise, proceed to the steps below.

Procedure

1. If you are creating your image on either a non-GCP instance or one that does not have the required roles, you must either:
   - » Create a GCP service account.
   - » If you have already created a service account, retrieve the credentials for an existing service account.
2. Export the static credentials that will be used to create the cluster.

```
export GCP_B64ENCODED_CREDENTIALS=$(base64 < "${GOOGLE_APPLICATION_CREDENTIALS}" | tr
 -d '\n')
```

Build the GCP Image

About this task

Depending on which version of NKP you are running, steps and flags will be different.

Procedure

1. Run the `konvoy-image` command to build and validate the image.

```
./konvoy-image build gcp --project-id ${GCP_PROJECT} --network ${NETWORK_NAME}
 images/gcp/ubuntu-2004.yaml
```

2. KIB will run and print out the name of the created image; you will use this name when creating a Kubernetes cluster. See the sample output below.

> **Note:** Ensure you have named the correct YAML file for your OS in the `konvoy-image build` command.

```
...
==> ubuntu-2004-focal-v20220419: Deleting instance...
    ubuntu-2004-focal-v20220419: Instance has been deleted!
==> ubuntu-2004-focal-v20220419: Creating image...
==> ubuntu-2004-focal-v20220419: Deleting disk...
    ubuntu-2004-focal-v20220419: Disk has been deleted!
==> ubuntu-2004-focal-v20220419: Running post-processor: manifest
Build 'ubuntu-2004-focal-v20220419' finished after 7 minutes 46 seconds.

==> Wait completed after 7 minutes 46 seconds

==> Builds finished. The artifacts of successful builds are:
--> ubuntu-2004-focal-v20220419: A disk image was created: konvoy-
ubuntu-2004-1-23-7-1658523168
--> ubuntu-2004-focal-v20220419: A disk image was created: konvoy-
ubuntu-2004-1-23-7-1658523168
```

3. To find a list of images you have created in your account, run the following command.

```
gcloud compute images list --no-standard-images
```

Related Information

Procedure

- To use a local registry, even in a non-air-gapped environment, download and extract the bundle. Downloading NKP on page 16 the Complete NKP Air-gapped Bundle for this release (that is. `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`) to load registry
- To view the complete set of instructions, see Load the Registry.

GCP: Creating the Management Cluster

Create a GCP Management Cluster in a non-air-gapped environment.

About this task

Use this procedure to create a self-managed **Management cluster** with NKP. A self-managed cluster refers to one in which the CAPI resources and controllers that describe and manage it are running on the same cluster they are managing. First, you must name your cluster.

Name Your Cluster

> **Note:** The cluster name might only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. For more naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

Note: NKP uses the GCP CSI driver as the default storage provider. Use a Kubernetes CSI compatible storage that is suitable for production.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable using the command `export CLUSTER_NAME=<gcp-example>`.

Create a GCP Kubernetes Cluster

About this task

If you use these instructions to create a cluster on GCP using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes.

By default, the control-plane Nodes will be created in 3 different zones. However, the default worker Nodes will reside in a single zone. You might create additional node pools in other zones with the `nkp create nodepool` command.

Availability zones (AZs) are isolated locations within data center regions from which public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single AZ, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple AZs.

Procedure

1. Create an image using Konvoy Image Builder (KIB) and then export the image name.

   ```
   export IMAGE_NAME=projects/${GCP_PROJECT}/global/images/<image_name_from_kib>
   ```

2. Ensure your subnets do not overlap with your host subnet because they cannot be changed after cluster creation. If you need to change the Kubernetes subnets, you must do this at cluster creation. The default subnets used in NKP are.

   ```
   spec:
     clusterNetwork:
       pods:
         cidrBlocks:
         - 192.168.0.0/16
       services:
         cidrBlocks:
         - 10.96.0.0/12
   ```

   - (Optional) Modify Control Plane Audit logs - Users can make modifications to the `KubeadmControlplane` cluster-api object to configure different `kubelet` options. See the following guide if you wish to configure your control plane beyond the existing options that are available from flags.
   - (Optional) Determine what VPC Network to use. All GCP accounts come with a preconfigured VPC Network named default, which will be used if you do not specify a different network. To use a different VPC network for your cluster, create one by following these instructions for Create and Manage VPC Networks. Then specify the `--network <new_vpc_network_name>` option on the create cluster command below. More information is available on GCP Cloud Nat and network flag.

3. Create a Kubernetes cluster. The following example shows a common configuration.

   ```
   nkp create cluster gcp \
   --cluster-name=${CLUSTER_NAME} \
   --additional-tags=owner=$(whoami) \
   ```

```
--project=${GCP_PROJECT} \
--image=${IMAGE_NAME} \
--self-managed
```

> **Note:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

If you want to monitor or verify the installation of your clusters, refer to the topic: Verify your Cluster and NKP Installation

GCP: Installing Kommander

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped GCP environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue with the installation of the Kommander component that will bring up the UI dashboard.

> **Tip:** Tips and Recommendations
>
> - The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
> - Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
> - If the Kommander installation fails, or you want to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. If required: Customize your `kommander.yaml`.

   a. See the Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.

5. Only required if your cluster uses a custom AWS VPC and requires an internal load-balancer; set the traefik annotation to create an internal-facing ELB.

```
apps:
  traefik:
    enabled: true
    values: |
      service:
        annotations:
          service.beta.kubernetes.io/aws-load-balancer-internal: "true"
```

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `nkp-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see the topic Configuring NKP Catalog Applications after Installing NKP.

GCP: Verifying your Installation and UI Log in

Verify Kommander installation and log in to the Dashboard UI.

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

Note: If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

```
kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
 index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
dashboard{{ "\n"}}'
```

Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

a. Rotate the password using the following command.

```
nkp experimental rotate dashboard-password
```

The output displays the new password:

```
Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Day 2 Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of NKP. The Day 2 section allows you to manage cluster operations and their application workloads to optimize your organization's productivity.

- Continue to the NKP dashboard.

GCP: Creating Managed Clusters Using the NKP CLI

This topic explains how to continue using the CLI to create managed clusters rather than switching to the UI dashboard.

About this task

After the initial cluster creation, you can create additional clusters from the CLI. In a previous step, the new cluster was created as Self-managed, which allows it to be a Management cluster or a stand-alone cluster. Subsequent new clusters are not self-managed, as they will likely be Managed or Attached clusters to this Management Cluster.

> **Note:** When creating Managed clusters, you do not need to create and move CAPI objects or install the Kommander component. Those tasks are only done on Management clusters!
>
> Your new managed cluster needs to be part of a workspace under a management cluster. To make the new managed cluster a part of a Workspace, set that workspace environment variable.

Procedure

1. If you have an existing Workspace name, run this command to find the name.

```
kubectl get workspace -A
```

2. When you have the Workspace name, set the `WORKSPACE_NAMESPACE` environment variable.

```
export WORKSPACE_NAMESPACE=<workspace_namespace>
```

> **Note:** If you need to create a new Workspace, follow the instructions to Create a New Workspace

Name Your Cluster

About this task

Each cluster must have an original name.

After you have defined the infrastructure and control plane endpoints, you can proceed to create the cluster by following these steps to create a new pre-provisioned cluster. This process creates a self-managed cluster to be used as the Management cluster.

First, you must name your cluster. Then, you run the command to deploy it.

> **Note:** The cluster name might only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
>
> When specifying the `cluster-name`, you must use the same `cluster-name` as used when defining your inventory objects.

Perform both steps to name the cluster:

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable.

```
export MANAGED_CLUSTER_NAME=<gcp-additional>
```

Create a Kubernetes Cluster

About this task

The instructions below tell you how to create a cluster and have it automatically attach to the workspace you set above. If you do not set a workspace, the cluster will be created in the #default# workspace, and you will need to take additional steps to attach to a workspace later. For instructions on how to do this, see Attach a Kubernetes Cluster.

> **Note:** Google Cloud Platform does not publish images. You must first build the image using Konvoy Image Builder.

Procedure

1. Create an image using Konvoy Image Builder (KIB) and then export the image name.

```
export IMAGE_NAME=projects/${GCP_PROJECT}/global/images/<image_name_from_kib>
```

2. Execute this command to create your additional Kubernetes cluster using any relevant flags. This will create a new non-self-managed cluster that can be managed by the management cluster you created in the previous section.

```
nkp create cluster gcp \
--cluster-name=${MANAGED_CLUSTER_NAME} \
--additional-tags=owner=$(whoami) \
--namespace ${WORKSPACE_NAMESPACE} \
--project=${GCP_PROJECT} \
--image=${IMAGE_NAME} \
```

```
--kubeconfig=<management-cluster-kubeconfig-path>
```

> **Tip:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

Manually Attach an NKP CLI Cluster to the Management Cluster

Procedure

When you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments. However, if you do not set a workspace, the attached cluster will be created in the default workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

1. Confirm you have your `MANAGED_CLUSTER_NAME` variable set with the following command.

   ```
   echo ${MANAGED_CLUSTER_NAME}
   ```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

   ```
   nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
    ${MANAGED_CLUSTER_NAME}.conf
   ```

3. > **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

   You can now either attach it in the UI, link to attaching it to the workspace through UI that was earlier, or attach your cluster to the workspace you want in the CLI.

4. Retrieve the workspace where you want to attach the cluster.

   ```
   kubectl get workspaces -A
   ```

5. Set the `WORKSPACE_NAMESPACE` environment variable.

   ```
   export WORKSPACE_NAMESPACE=<workspace-namespace>
   ```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the `kubeconfig` secret value of your cluster.

   ```
   kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
   template='{{.data.value}}{{ "\n"}}'
   ```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml file`.

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <your-managed-cluster-name>-kubeconfig
     labels:
       cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
   type: cluster.x-k8s.io/secret
   data:
     value: <value-you-copied-from-secret-above>
   ```

8. Create this secret in the desired workspace.

```
kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
 ${WORKSPACE_NAMESPACE}
```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI, and you can confirm its status by running the command below. It may take a few minutes to reach "Joined" status.

```
kubectl get kommanderclusters -A
```

If you have several Pro Clusters and want to turn one of them into a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

Next Step

Procedure

- Cluster Operations Management

5

CLUSTER OPERATIONS MANAGEMENT

Manage your NKP environment using the Cluster Operations Management features.

The Cluster Operations Management section allows you to manage cluster operations and their application workloads to optimize your organization's productivity.

- Operations on page 353
- Applications on page 390
- Workspaces on page 411
- Projects on page 439
- Cluster Management on page 478
- Backup and Restore on page 567
- Logging on page 589
- Security on page 613
- Networking on page 627
- GPUs on page 637
- Monitoring and Alerts on page 648
- Storage for Applications on page 665

Operations

You can manage your cluster and deployed applications using platform applications.

After you deploy an NKP cluster and the platform applications you want to use, you are ready to begin managing cluster operations and their application workloads to optimize your organization's productivity.

In most cases, a production cluster requires additional advanced configuration tailored for your environment, ongoing maintenance, authentication and authorization, and other common activities. For example, it is important to monitor cluster activity and collect metrics to ensure application performance and response time, evaluate network traffic patterns, manage user access to services, and verify workload distribution and efficiency.

In addition to the configurations, you can also control the appearance of your NKP UI by adding banners and footers. There are different options available depending on the NKP level that you license and install.

- Access Control on page 354
- Identity Providers on page 364
- Kubectl API Access Using an Identity Provider on page 371
- Infrastructure Providers on page 374

- Header, Footer, and Logo Implementation on page 389

Access Control

You can centrally manage access across clusters and define role-based authorization within the NKP UI to control resource access on the management cluster for a set or all of the target clusters. These resources are similar to Kubernetes RBAC but with crucial differences, and they make it possible to define the roles and role bindings once and federate them to clusters within a given scope.

NKP UI has two conceptual groups of resources that are used to manage access control:

- *Kommander Roles*: control access to resources on the management clusters.
- *Cluster Roles*: control access to resources on all target clusters.

Use these two groups of resources to manage access control within three levels of scope:

Table 22: Managing Access Across Scopes

Environment Context	Kommander Roles	Cluster Roles
Global : Manages access to the entire environment.	Create ClusterRoles on the management cluster.	Federates ClusterRoles on all target clusters across all workspaces.
Workspace: Manages access to clusters in a specific workspace, for example, in the scope of multi-tenancy. See Multi-Tenancy in NKP on page 436.	Create namespaced Roles on the management cluster in the workspace namespace.	Federates ClusterRoles on all target clusters in the workspace.
Project: Manages access for clusters in a specific project, for example, in the scope of multi-tenancy. See Multi-Tenancy in NKP on page 436.	Create namespaced Roles on the management cluster in the project namespace.	Federates namespaced Roles on all target clusters in the project in the project namespace.

Create the role bindings for each level and type create `RoleBindings` or `ClusterRoleBindings` on the clusters that apply to each category.

This approach gives you maximum flexibility over who has access to what resources, conveniently mapped to your existing identity providers' claims.

Limitation for Kommander Roles

In addition to granting a Kommander Role, you must also grant the appropriate NKP role to allow external users and groups into the UI. For details about the built-in NKP roles, see Types of Access Control Objects on page 355. Here are examples of `ClusterRoleBindings` that grant an Identity provider (IdP) group (in this example, the user group *engineering*) is provided with admin access to the Kommander routes:

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: eng-kommander-dashboard
  labels:
    "workspaces.kommander.mesosphere.io/rbac": ""
roleRef:
```

```
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: dkp-kommander-admin
subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: Group
    name: oidc:engineering
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: eng-nkp-routes
  labels:
    "workspaces.kommander.mesosphere.io/rbac": ""
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: dkp-admin
subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: Group
    name: oidc:engineering
EOF
```

Types of Access Control Objects

Manage Kubernetes role-based access control with three different object categories: Groups, Roles, and Policies.

Groups

Access control groups are configured in the Groups tab of the Identity Providers page.

You can map group and user claims made by your configured identity providers to Kommander groups by selecting administration or identity providers in the left sidebar in the global workspace level, and then select the **Groups** tab.

Roles

`ClusterRoles` are named collections of rules defining which verbs can be applied to what resources.

- Kommander Roles apply specifically to resources in the management cluster.
- Cluster Roles apply to target clusters within their scope at these levels:
  - *Global level* - this is all target clusters in all workspaces,
  - *Workspace level* - all target clusters in the workspaces.
  - *Project level* - this i all target clusters that are added to the project.

Propagating Workspace Roles to Projects
By default, users are granted the Kommander Workspace Admin, Edit, or View roles. You are also granted the equivalent Kommander Project Admin, Edit, or View role for any project created in the workspace. Other workspace roles are not automatically propagated to the equivalent role for a project in the workspace.

About this task

Each workspace has roles defined using `KommanderWorkspaceRole` resources. Automatic propagation is controlled using the annotation `"workspace.kommander.mesosphere.io/sync-to-project": "true"` on a `KommanderWorkspaceRole` resource. You can manage this only by using the CLI.

Procedure

1. Run the command `kubectl get kommanderworkspaceroles -n <WORKSPACE_NAMESPACE>`.

```
NAME                          DISPLAY NAME                       AGE
kommander-workspace-admin     Kommander Workspace Admin Role     2m18s
kommander-workspace-edit      Kommander Workspace Edit Role      2m18s
kommander-workspace-view      Kommander Workspace View Role      2m18s
```

2. To prevent propagation of the `kommander-workspace-view` role, remove this annotation from the `KommanderWorkspaceRole` resource.

```
kubectl annotate kommanderworkspacerole -n <WORKSPACE_NAMESPACE> kommander-workspace-
view workspace.kommander.mesosphere.io/sync-to-project-
```

3. To enable propagation of the role, add this annotation to the relevant `KommanderWorkspaceRole` resource.

```
kubectl annotate kommanderworkspacerole -n <WORKSPACE_NAMESPACE> kommander-workspace-
view workspace.kommander.mesosphere.io/sync-to-project=true
```

Limitation for Workspace

During the inheritance of the Project Role, when granting users access to a workspace, you must manually grant access to the projects within that workspace. Each project is created with a set of admin, edit, or view roles, and you can choose to add `RoleBinding` to each group or user of the workspace for one of these project roles. Usually, these are prefixed with one of the roles `kommander-project-(admin/edit/view)`.

This is an example of RoleBinding that grants the *Kommander Project Admin* role access for the project namespace to the engineering group:

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: workspace-admin-project1-admin
  namespace: <my-project-namespace-xxxxx>
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: <kommander-project-admin-xxxxx>
subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: Group
    name: oidc:engineering
EOF
```

Role Bindings

Kommander role bindings, cluster role bindings, and project role bindings bind a Kommander group to any number of roles. All groups defined in the **Groups** tab are present at the global, workspace, or project levels and are ready for you to assign roles to them.

Access to Kubernetes and Kommander Resources

You can grant access to Kommander and Kubernetes resources using RBAC.

Initially, users and groups from an external identity provider have no access to Kubernetes resources. Privileges must be granted explicitly by interacting with the RBAC API. This section provides some basic examples for general usage. For more information on the RBAC API, see the *Using RBAC Authorization* section in the Kubernetes documentation at https://kubernetes.io/docs/reference/access-authn-authz/rbac/.

Kubernetes does not provide an identity database for standard users. A trusted identity provider must provide users and group membership. In Kubernetes, RBAC policies are additive, which means that a subject (user, group, or service account) is denied access to a resource unless explicitly granted access by a cluster administrator. You can grant access by binding a subject to a role, which grants some level of access to one or more resources. Kubernetes is shipped with some default role, which aids in creating broad access control policies. For more information, see *Default roles and role bindings* in the Kubernetes documentation https://kubernetes.io/docs/reference/access-authn-authz/rbac/#default-roles-and-role-bindings.

For example, if you want to make `mary@example.com` a cluster administrator, bind their username to the *cluster-admin* default role as follows:

```
cat << EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: mary-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: mary@example.com
EOF
```

User to Namespace Restriction

A common example is granting users access to specific namespaces by creating a RoleBinding (RoleBindings are namespaced scoped). For example, to make the user `bob@example.com` a reader of the `baz` namespace, bind the user to the *view* role:

```
cat << EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: bob-view
  namespace: baz
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: view
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: bob@example.com
EOF
```

The user can now only perform non-destructive operations targeting resources in the #baz# namespace.

Groups

If your external identity provider supports group claims, you can also bind groups to roles. To make the engineering LDAP group administrators of the production namespace bind the group to the admin role:

```
cat << EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: engineering-admin
  namespace: production
roleRef:
```

```
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: admin
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: Group
  name: oidc:engineering
EOF
```

One important distinction from adding users is that all external groups are prefixed with `oidc:`, so a group name is `oidc:devops`. This prevents collision with locally defined groups.

NKP UI Authorization

The NKP UI and other HTTP applications protected by Kommander forward authentication, are also authorized by the Kubernetes RBAC API. In addition to the Kubernetes API resources, it is possible to define rules which map to HTTP URIs and HTTP verbs. Kubernetes RBAC refer to these as nonResourceURLs, Kommander forward authentication uses these rules to grant or deny access to HTTP endpoints.

Default Roles

Roles are created to grant access to the dashboard and select applications that expose an HTTP server through the ingress controller. The `cluster-admin` role is a system role that grants permission to all actions (verbs) on any resource, including non-resource URLs. The default dashboard user is bound to this role.

> **Note:** Granting user administrator privileges on `/dkp/*` grants `admin` privileges to all sub-resources, even if the bindings exist for sub-resources with fewer privileges.

Table 23: Table

Dashboard	Role	Path	access
*	cluster-admin	*	read, write, delete
kommander	dkp-view	/dkp/*	read
kommander	dkp-edit	/dkp/*	read, write
kommander	dkp-admin	/dkp/*	read, write, delete
kommander-dashboard	dkp-kommander-view	/dkp/kommander/dashboard/*	read
kommander-dashboard	dkp-kommander-edit	/dkp/kommander/dashboard/*	read, write
kommander-dashboard	dkp-kommander-admin	/dkp/kommander/dashboard/*	read, write, delete
alertmanager	dkp-kube-prometheus-stack-alertmanager-view	/dkp/alertmanager/*	read
alertmanager	dkp-kube-prometheus-stack-alertmanager-edit	/dkp/alertmanager/*	read, write
alertmanager	dkp-kube-prometheus-stack-alertmanager-admin	/dkp/alertmanager/*	read, write, delete

Dashboard	Role	Path	access
centralized-grafana	dkp-centralized-grafana-grafana-view	/dkp/kommander/monitoring/grafana/*	read
centralized-grafana	dkp-centralized-grafana-grafana-edit	/dkp/kommander/monitoring/grafana/*	read, write
centralized-grafana	dkp-centralized-grafana-grafana-admin	/dkp/kommander/monitoring/grafana/*	read, write, delete
centralized-kubecost	dkp-centralized-kubecost-view	/dkp/kommander/kubecost/*	read
centralized-kubecost	dkp-centralized-kubecost-edit	/dkp/kommander/kubecost/*	read, write
centralized-kubecost	dkp-centralized-kubecost-admin	/dkp/kommander/kubecost/*	read, write, delete
grafana	dkp-kube-prometheus-stack-grafana-view	/dkp/grafana/*	read
grafana	dkp-kube-prometheus-stack-grafana-edit	/dkp/grafana/*	read, write
grafana	dkp-kube-prometheus-stack-grafana-admin	/dkp/grafana/*	read, write, delete
grafana-logging	dkp-grafana-logging-view	/dkp/logging/grafana/*	read
grafana-logging	dkp-grafana-logging-edit	/dkp/logging/grafana/*	read, write
grafana-logging	dkp-grafana-logging-admin	/dkp/logging/grafana/*	read, write, delete
karma	dkp-karma-view	/dkp/kommander/monitoring/karma/*	read
karma	dkp-karma-edit	/dkp/kommander/monitoring/karma/*	read, write
karma	dkp-karma-admin	/dkp/kommander/monitoring/karma/*	read, write, delete
kubernetes-dashboard	dkp-kubernetes-dashboard-view	/dkp/kubernetes/*	read
kubernetes-dashboard	dkp-kubernetes-dashboard-edit	/dkp/kubernetes/*	read, write
kubernetes-dashboard	dkp-kubernetes-dashboard-admin	/dkp/kubernetes/*	read, write, delete
prometheus	dkp-kube-prometheus-stack-prometheus-view	/dkp/prometheus/*	read
prometheus	dkp-kube-prometheus-stack-prometheus-edit	/dkp/prometheus/*	read, write
prometheus	dkp-kube-prometheus-stack-prometheus-admin	/dkp/prometheus/*	read, write, edit
traefik	dkp-traefik-view	/dkp/traefik/*	read
traefik	dkp-traefik-edit	/dkp/traefik/*	read, edit

Dashboard	Role	Path	access
traefik	dkp-traefik-admin	/dkp/traefik/*	read, edit, delete
thanos	dkp-thanos-query-view	/dkp/kommander/monitoring/query/*	read
thanos	dkp-thanos-query-edit	/dkp/kommander/monitoring/query/*	read, write
thanos	dkp-thanos-query-admin	/dkp/kommander/monitoring/query/*	read, write, delete

Examples of Default Roles
This topic provides a few examples of binding subjects to the default roles defined for the NKP UI endpoints.

User

To grant the user `mary@example.com` administrative access to all Kommander resources, bind the user to the `dkp-admin` role:

```
cat << EOF | kubectl apply -f -
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: dkp-admin-mary
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: dkp-admin
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: mary@example.com
EOF
```

If you inspect the role, you can see what access is now granted:

```
kubectl describe clusterroles dkp-admin
```

```
Name:         dkp-admin
Labels:       app.kubernetes.io/instance=kommander
              app.kubernetes.io/managed-by=Helm
              app.kubernetes.io/version=v2.13.1
              helm.toolkit.fluxcd.io/name=kommander
              helm.toolkit.fluxcd.io/namespace=kommander
              rbac.authorization.k8s.io/aggregate-to-admin=true
Annotations:  meta.helm.sh/release-name: kommander
              meta.helm.sh/release-namespace: kommander
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  ---------  -----------------  --------------  -----
             [/dkp/*]           []              [delete]
             [/dkp]             []              [delete]
             [/dkp/*]           []              [get]
             [/dkp]             []              [get]
             [/dkp/*]           []              [head]
             [/dkp]             []              [head]
             [/dkp/*]           []              [post]
             [/dkp]             []              [post]
             [/dkp/*]           []              [put]
```

```
            [/dkp]              []              [put]
```

The user can now use the HTTP verbs HEAD, GET, DELETE, POST, and PUT when accessing any URL at or under `/dkp`. The downstream application follows REST conventions. This effectively allows privileges to be read, edited, and deleted.

> **Note:** To enable users to access the NKP UI, ensure they have the appropriate `dkp-kommander` role and the Kommander roles granted in the NKP UI.

Group

To grant view access to the `/dkp/*` endpoints and edit access to the grafana logging endpoint to group `logging-ops`, create the following ClusterRoleBindings:

```
cat << EOF | kubectl apply -f -
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: nkp-view-logging-ops
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: dkp-view
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: Group
  name: oidc:logging-ops
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: dkp-logging-edit-logging-ops
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: dkp-logging-edit
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: Group
  name: oidc:logging-ops
EOF
```

> **Note:** External groups must be prefixed by `oidc`:

Members of `logging-ops` to view all the resources under `/dkp` and edit all the resources under `/dkp/logging/grafana`.

Creating Custom Roles

If one of the predefined roles from NKP does not include all the permissions you need, you can create a custom role.

About this task

Perform the following tasks to assign actions and permissions to roles:

Procedure

1. In the **Administration** section of the sidebar menu, select **Access Control**.
2. Select the **Cluster Roles** tab, and then select **+ Create Role** .

3. Enter a descriptive name for the role and ensure that **Cluster Role** is selected as the type.
4. For example, to configure a read-only role, select **Add Rule**.
   a. In the **Resources** input, select **All Resource Types**.
   b. Select the **get**, **list**, and **watch** options.
   c. Click **Save**.

   You can assign your newly created role to the developer's group.

Kubernetes Dashboard

The Kubernetes dashboard displays information that offloads authorization directly to the Kubernetes API server. Once authenticated, all users may access the dashboard at `/dkp/kubernetes/` without needing an `dkp` role. However, the cluster RBAC policy protects access to the underlying Kubernetes resources exposed by the dashboard.

Access to the Kubernetes dashboard at the `/dkp/kubernetes/` URL requires an `dkp` role that grants access to that URL path. NKP provides several pre-defined `ClusterRoles` to provide access such as: dkp-kubernetes-dashboard-view, dkp-kubernetes-dashboard-edit, and dkp-kubernetes-dashboard-admin

For example, to provide all authenticated users in the `oidc:developers` group access to Kubernetes dashboard, add the following `ClusterRoleBinding`:

```
cat << EOF | kubectl apply -f -
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: developer-dashboard-access
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: dkp-kubernetes-dashboard-admin
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: Group
  name: oidc:developers
EOF
```

However, access to the underlying Kubernetes resources exposed by the dashboard are protected by the cluster RBAC policy.

This topic describes some basic examples of operations that provide the building blocks for creating an access control policy. For more information about creating your roles and advanced policies, see *Using RBAC Authorization* in the Kubernetes documentation at https://kubernetes.io/docs/reference/access-authn-authz/rbac/. For information on adding a user to a cluster as an administrator, see Onboarding a User to an NKP Cluster on page 362.

Onboarding a User to an NKP Cluster

After you install NKP and create a cluster, you can add new users to your environment.

Before you begin

You must have administrator rights. Also, ensure that:

- You have an LDAP Connector.
- You are a cluster administrator.
- You have a valid NKP license (Starter or Pro)

- You have a running cluster.

For information about adding users using other types of connectors, see:

- https://dexidp.io/docs/connectors/oidc/
- https://dexidp.io/docs/connectors/saml/
- https://dexidp.io/docs/connectors/github/

To onboard a user:

Procedure

1. Create an LDAP Connector definition and name the file `ldap.yaml`.

```
apiVersion: v1
kind: Secret
metadata:
  name: ldap-password
  namespace: kommander
type: Opaque
stringData:
  password: superSecret
---
apiVersion: dex.mesosphere.io/v1alpha1
kind: Connector
metadata:
  name: ldap
  namespace: kommander
spec:
  enabled: true
  type: ldap
  displayName: LDAP Test Connector
  ldap:
    host: ldapdce.testdomain
    insecureNoSSL: true
    bindDN: cn=ldapconnector,cn=testgroup,ou=testorg,dc=testdomain
    bindSecretRef:
      name: ldap-password
    userSearch:
      baseDN: dc=testdomain
      filter: "(objectClass=inetOrgPerson)"
      username: uid
      idAttr: uid
      emailAttr: uid
    groupSearch:
      baseDN: ou=testorg,dc=testdomain
      filter: "(objectClass=posixGroup)"
      userMatchers:
      - userAttr: uid
        groupAttr: memberUid
      nameAttr: cn
```

2. Add the connector and use the command `kubectl apply -f ldap.yaml`.
   The following output is displayed.

```
secret/ldap-password created
connector.dex.mesosphere.io/ldap created
```

3. Add the appropriate role bindings and name the file `new_user.yaml`.
   See the following examples for both Single User and Group Bindings.

   » For Single Users:

   ```
   apiVersion: rbac.authorization.k8s.io/v1
   kind: ClusterRoleBinding
   metadata:
     name: cluster-admin
   roleRef:
     apiGroup: rbac.authorization.k8s.io
     kind: ClusterRole
     name: cluster-admin
   subjects:
   - apiGroup: rbac.authorization.k8s.io
     kind: User
     name: newUser
   ```

   » For Group Binding:

   ```
   apiVersion: rbac.authorization.k8s.io/v1
   kind: RoleBinding
   metadata:
     name: cluster-admin
     namespace: ml
   roleRef:
     apiGroup: rbac.authorization.k8s.io
     kind: ClusterRole
     name: cluster-admin
   subjects:
   - apiGroup: rbac.authorization.k8s.io
     kind: Group
     name: oidc:kommanderAdmins
   ```

4. Add the role binding(s) use the command `kubectl apply -f new_user.yaml`.

   Note:

   - *ClusterRoleBindings* permissions are applicable at the global level.
   - *RoleBindings* permissions are applicable at the namespace level.

Identity Providers

You can grant access to users in your organization.

NKP supports GitHub Identity Provider Configuration on page 366, Adding an LDAP Connector on page 367, SAML,and standard OIDC identity providers such as Google. These identity management providers support the login and authentication process for NKP and your Kubernetes clusters.

You can configure as many identity providers as you want, and users can select from any method when logging in. If you have multiple workspaces in your environment, you can use a single identity provider to manage access for all of them or choose to configure an identity provider per workspace.

Configuring a dedicated identity provider per workspace can be useful if you want to retain access to your workspaces separately. In this case, users of a specific workspace have a dedicated login 2-factor authentication page with the identity provider options configured for their workspace. This setup is particularly helpful if you have multiple tenants. For more information, see Multi-Tenancy in NKP on page 436.

Advantages of Using an External Identity Provider

Using an external identity provider is beneficial for:

- Centralized management of multiple users and multiple clusters.
- Centralized management of password rotation, expiration, and so on.
- Support of 2-factor-authentication methods for increased security.
- Separate storage of user credentials.

Access Limitations

- The GitHub provider allows you to specify any organizations and teams that are eligible for access.
- The LDAP provider allows you to configure search filters for either users or groups.
- The OIDC provider cannot limit users based on identity.
- The SAML provider allows users to log in using a single sign-on (SSO).

Configuring an Identity Provider Through the UI

You can configure an identity provider through the UI.

Before you begin

To configure an identity provider:

Procedure

1. Log into the Kommander UI. See Logging In To the UI on page 79.
2. From the dropdown list, select the **Global** workspace.
3. Select **Administration** > **Identity Providers.**
4. Select the **Identity Providers** tab.
5. Select **Add Identity Provider**.
6. Select an identity provider.
7. Select the **target workspace** for the identity provider and complete the fields with the relevant details.

   Note: You can configure an identity provider globally for your entire organization using the**All Workspaces** option or per workspace, enabling multi-tenancy.

8. Click **Save**.

Disabling an Identity Provider

You can temporarily disable an identity provider.

Procedure

1. Select the three-dot button on the Identity Providers table.
2. Select **Disable** from the dropdown menu.
   The identity provider is disabled. You can view the disabled identity providers in the **Identity Providers** tab.

GitHub Identity Provider Configuration

You can configure GitHub as an identity provider and grant access to NKP.

NKP allows authorizing access to your clusters and the UI with GitHub credentials but it must be configured in the dashboard. To ensure every developer in your GitHub organization has access to your Kubernetes clusters using their GitHub credentials, add that option for login by adding an identity provider with the information from your GitHub profile in the OAuth application settings

The first login requires you to authorize the GitHub account. As an administrator of the cluster, select the **Authorize github-username** button on the page that follows the login. After setting up the GitHub authorization, the future login screens will have the **Log in with github-auth** button as an option.

Adding an Identity Provider Using GitHub

To authorize all developers to access your clusters using their GitHub credentials, set up GitHub as an identity provider login option.

Procedure

1. Start by creating a new OAuth Application in your GitHub organization by completing the registration form. To view the form, see https://github.com/settings/applications/new.
2. In the **Application name** field, enter a name for your application.
3. In the **Homepage URL** field, enter your cluster URL.
4. In the **Authorization callback URL** field, use your cluster URL followed by `/dex/callback` by adding this to the end of your URL.
5. Click **Register application**.
   After you complete the application, the **Settings** page. appears
6. You need the **Client ID** and **Client Secret** from this page for the NKP UI.
   If you do not have a Client Secret for the application, to generate a new client secret, select **Generate a new client secret**.
7. Log in to your NKP UI from the top menu bar, and select the **Global** workspace.
8. Select **Identity Providers** in the **Administration** section of the sidebar menu.
9. Select the **Identity Providers** tab and then click **Add Identity Provider** .
10. Select **GitHub** as the identity provider type, and select the target workspace.
11. Copy the **Client ID** and **Client Secret** values from GitHub into this form.
12. To configure `dex` to load all the groups configured in the user's GitHub identity, select the **Load All Groups** checkbox.
    This allows you to configure group-specific access to NKP and Kubernetes resources.

    Note: Do not select the **Enable Device Flow** checkbox before selecting **<Register the Application>** .
13. Click **Save**.

Mapping the Identity Provider Groups to the Kubernetes Groups

You can map the identity provider groups to the Kubernetes groups.

Procedure

1. In the NKP UI, select the **Groups** tab from the **Identity Provider** screen, and then click **Create Group**.
2. In the **Enter Name** field, enter a descriptive name.
3. Add the groups or teams from your GitHub provider under **Identity Provider Groups**.

   For more information on finding the teams to which you are assigned in GitHub, see the *Changing team visibility* section at https://docs.github.com/en/organizations/organizing-members-into-teams/changing-team-visibility.
4. Click **Save**.

Assigning a Role to the Developers Group

After defining a group, bind one or more roles to this group. This topic describes how to bind the group to the View Only role.

Procedure

1. In the NKP UI, from the top menu bar, select **Global** or the target workspace.
2. Select the **Cluster Role Bindings** tab and then select **Add roles**.
3. Select **View Only** role from the **Roles** dropdown list and select **Save**.

   For more information on granting users access to Kommander paths on your cluster, see Access to Kubernetes and Kommander Resources on page 356.
4. At a minimum, add a read only path for access to all the Kommander Dashboard views:

Table 24: Kommander Dashboard Views

Dashboard	Role	Path	Access
kommander-dashboard	nkp-kommander-view	/nkp/kommander/dashboard/*	read

When you check your attached clusters and login as a user from your matched groups, every resource, is listed. Do delete or edit them.

External LDAP Directory Configuration

You can connect your cluster to an external LDAP directory. Configure your NKP cluster for logging in with the credentials stored in an external LDAP directory service.

Adding an LDAP Connector

Each LDAP directory is set up in unique ways. So, these steps are important. Add the LDAP authentication mechanism using the CLI or UI.

About this task

This topic describes the configuration of an NKP cluster to connect to the *Online LDAP Test Server* in Forum Systems Web site at https://www.forumsys.com/tutorials/integration-how-to/ldap/online-ldap-test-server/. For demonstration purpose, the configuration shown uses `insecureNoSSL: true`. In production, you should protect LDAP communication with a properly configured transport layer security (TLS). When

using TLS, as an admin, you can add `insecureSkipVerify: true` to `spec.ldap` to skip server certificate verification, if needed.

> **Note:** This topic does not cover all possible configurations. For more information, see *Dex LDAP connector reference* documentation on GitHub at https://github.com/dexidp/dex/blob/v2.22.0/Documentation/connectors/ldap.md.

Procedure

Choose whether to establish an external LDAP globally or for a specific workspace.

» Global LDAP - identity provider serves all workspaces: Create and apply the following objects:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Secret
metadata:
  name: ldap-password
  namespace: kommander
type: Opaque
stringData:
  password: password
---
apiVersion: dex.mesosphere.io/v1alpha1
kind: Connector
metadata:
  name: ldap
  namespace: kommander
spec:
  enabled: true
  type: ldap
  displayName: LDAP Test
  ldap:
    host: ldap.forumsys.com:389
    insecureNoSSL: true
    bindDN: cn=read-only-admin,dc=example,dc=com
    bindSecretRef:
      name: ldap-password
    userSearch:
      baseDN: dc=example,dc=com
      filter: "(objectClass=inetOrgPerson)"
      username: uid
      idAttr: uid
      emailAttr: mail
    groupSearch:
      baseDN: dc=example,dc=com
      filter: "(objectClass=groupOfUniqueNames)"
      userMatchers:
      - userAttr: DN
        groupAttr: uniqueMember
      nameAttr: ou
EOF
```

> **Note:** The value for the LDAP connector `spec:displayName` (here **LDAP Test**) appears on the **Login** button for this identity provider in the NKP UI. Enter a name for the users.

» Workspace LDAP - identity provider serves a specific workspace: Create and apply the following objects:

Note: Establish LDAP for a specific workspace in the scope of multiple tenants..

- **1.** Obtain the workspace name for which you are establishing an LDAP authentication server.

  ```
  kubectl get workspaces
  ```

  Note down the value under the `WORKSPACE NAMESPACE` column.

  2. Set the `WORKSPACE_NAMESPACE` environment variable to that namespace.

  ```
  export WORKSPACE_NAMESPACE=<your-namespace>
  ```

  3. Create and apply the following objects on that workspace.

  ```
  cat <<EOF | kubectl apply -f -
  apiVersion: v1
  kind: Secret
  metadata:
    name: ldap-password
    namespace: ${WORKSPACE_NAMESPACE}
  type: Opaque
  stringData:
    password: password
  ---
  apiVersion: dex.mesosphere.io/v1alpha1
  kind: Connector
  metadata:
    name: ldap
    namespace: ${WORKSPACE_NAMESPACE}
  spec:
    enabled: true
    type: ldap
    displayName: LDAP Test
    ldap:
      host: ldap.forumsys.com:389
      insecureNoSSL: true
      bindDN: cn=read-only-admin,dc=example,dc=com
      bindSecretRef:
        name: ldap-password
      userSearch:
        baseDN: dc=example,dc=com
        filter: "(objectClass=inetOrgPerson)"
        username: uid
        idAttr: uid
        emailAttr: mail
      groupSearch:
        baseDN: dc=example,dc=com
        filter: "(objectClass=groupOfUniqueNames)"
        userMatchers:
        - userAttr: DN
          groupAttr: uniqueMember
        nameAttr: ou
  EOF
  ```

  Note: The value for the LDAP connector `spec:displayName` (here **LDAP Test**) appears on the **Login** button for this identity provider in the NKP UI. Choose a name for the users.

Testing the LDAP Connector

You can test the LDAP connector.

Procedure

1. To retrieve a list of connectors using the `kubectl get connector.dex.mesosphere.io -A` command.
2. Run the `kubectl get Connector.dex.mesosphere.io -n kommander <LDAP-CONNECTOR-NAME> -o yaml` command to verify that the LDAP connector is created successfully.

Logging In for Global LDAP

Global LDAP identity provider serves all workspaces.

Procedure

1. Visit **https://<YOUR-CLUSTER-HOST>/token** and initiate a login flow.
2. On the login page, click **Log in with <ldap-name>**.
3. Enter the LDAP credentials and log in.

> **Note:** In the UI, after the LDAP authentication is enabled, additional access rights must be configured using the **Add Identity Provider** page in the UI.

Logging In for Workspace LDAP

Workspace LDAP identity provider serves a specific workspace.

Procedure

1. Complete the steps in Generating a Dedicated Login URL for Each Tenant on page 438.
2. On the login page, click **Log in with <ldap-name>**.
3. Enter the LDAP credentials and log in.

> **Note:** In the UI, after the LDAP authentication is enabled, additional access rights must be configured using the **Add Identity Provider** page in the UI.

LDAP Troubleshooting

If the Dex LDAP connector configuration is incorrect, debug the problem, and iterate on it. The Dex log output contains helpful error messages, as indicated in the following examples:

Reading Errors During Dex Startup

About this task

If the Dex configuration fragment provided results in an invalid Dex config log file, Dex does not properly start up. The, read the error details by reviewing the Dex logs.

Procedure

1. Use the `kubectl logs -f dex-66675fcb7c-snxb8 -n kommander` command to retrieve the Dex logs. You may see an error similar to the following example:

   ```
   error parse config file /etc/dex/cfg/config.yaml: error unmarshaling JSON: parse
    connector config: illegal base64 data at input byte 0
   ```

2. Another reason for Dex not starting up correctly is that `https://<YOUR-CLUSTER-HOST>/token` displays a 5xx HTTP error response after timing out.

Errors Upon Login

Most problems with the Dex LDAP connector configuration become apparent only after a login attempt. A login that fails from misconfiguration results in an error displaying only `Internal Server Error` and `Login error`. You can find the root cause by reading the Dex log, as shown in the following example.

```
kubectl logs -f dex-5d55b6b94b-9pm2d -n kommander
```

You can look for output similar to this example.

```
[...]
time="2019-07-29T13:03:57Z" level=error msg="Failed to login user: failed to connect:
 LDAP Result Code 200 \"Network Error\": dial tcp: lookup freeipa.example.com on
 10.255.0.10:53: no such host"
```

Here, the directory's DNS name was misconfigured, which should be easy to address.

A more difficult problem occurs when a login through Dex through LDAP fails because Dex cannot find the specified user unambiguously in the directory. That is the result of an invalid LDAP user search configuration. Here's an example error message from the Dex log.

```
time="2019-07-29T14:21:27Z" level=info msg="performing ldap search
 cn=users,cn=compat,dc=demo1,dc=freeipa,dc=org sub (&(objectClass=posixAccount)
(uid=employee))"
time="2019-07-29T14:21:27Z" level=error msg="Failed to login user: ldap: filter
 returned multiple (2) results: \"(&(objectClass=posixAccount)(uid=employee))\""
```

Solving problems like this requires you to review the directory structures carefully. Directory structures can be very different between different LDAP setups. You must carefully assemble a user search configuration matching the directory structure.

Notably, with some directories, it can be hard to distinguish between the cases such as `properly configured and user not found` where login fails in an expected way and displays `not properly configured, and therefore user not found` where login fails in an unexpected way.

Successful Login Example

For comparison, here are some sample log lines issued by Dex for a successful login:

```
time="2019-07-29T15:35:51Z" level=info msg="performing ldap search
 cn=accounts,dc=demo1,dc=freeipa,dc=org sub (&(objectClass=posixAccount)
(uid=employee))"
time="2019-07-29T15:35:52Z" level=info msg="username \"employee\" mapped to entry
 uid=employee,cn=users,cn=accounts,dc=demo1,dc=freeipa,dc=org"
time="2019-07-29T15:35:52Z" level=info msg="login successful: connector \"ldap\",
 username=\"\", email=\"employee@demo1.freeipa.org\", groups=[]"
```

Kubectl API Access Using an Identity Provider
After installing NKP, a single user with admin rights and static credentials is available. However, static credentials are hard to manage and replace.

To allow other users and user groups to access your environment, Nutanix recommends setting up an external identity provider. Users added through an identity provider do not have static credentials, but have to generate a token to gain access to your environment's kubectl API. This token ensures that certificates are rotated continuously for security reasons.

There are two options for the generation of this token:

Table 25: Token Generation

Method	How Often Does the User Have to Generate a Token?
Generating a token	User must log in with credentials and manually generate a `kubeconfig` file with a fresh token every 24 hours.
Enabling the Konvoy Async Plugin	User configures the `Konvoy Async Plugin` so the authentication is routed through Dex's `oidc` and the token is generated automatically. By enabling the plugin, the user is routed to an additional login procedure for authentication, but they no longer have to generate a token manually in the UI.

The instructions for either generating a token manually or enabling the Konvoy Async Plugin differ slightly depending on whether you configured the identity provide globally for all the workspaces, or individually for a single workspace.

Configuring Token Authentication for Global Identity Providers

In this scenario, the Identity Provider serves all workspaces.

About this task

> **Note:** You must manually generate a new token every 24 hours:

About this task

Procedure

1. Log in to the NKP UI with your credentials.
2. Select your username.
3. Select **Generate Token**.
4. Login again.
5. If there are several clusters, select the target cluster.
6. Follow the instructions on the displayed page.

Enabling the Konvoy Async Plugin for Global Identity Providers

Enable the Konvoy Async Plugin to automatically update the token.

Before you begin

You or a global admin must configure an identity provider to see this option.

Procedure

1. Open the login URL.
2. To authenticate, select **Konvoy credentials plugin instructions**.
3. Follow the instructions on the displayed **(Konvoy) Credentials plugin instructions**page.
   If you use **Method 1** in the instructions documented in the **(Konvoy) Credentials plugin instructions**, then download a `kubeconfig` file that includes the contexts for all clusters.
   Alternatively, to switch between clusters, you can use **Method 2** to create a `kubeconfig` file per cluster and use the `--kubeconfig=` flag or `export KUBECONFIG=` commands.

   Warning: If you choose **Method 2**, the **Set profile name** field is not optional if you have multiple clusters in your environment. Ensure you change the name of the profile for each cluster for which you want to generate a `kubeconfig` file. Otherwise, all clusters will use the same token, which makes cluster authentication vulnerable and can let users access clusters for which they do not have authorization.

Configuring Token Authentication for Workspace Identity Providers

In this scenario, the identity provider serves a specific workspace or tenant.

About this task

Note: You must manually generate a new token every 24 hours:

Procedure

1. Open the login link you obtained from the global administrator, which they generated for your workspace or tenant.
2. Select **Generate Kubectl Configuration**.
3. If there are several clusters in the workspace, select the cluster for which you want to generate a token.
4. Log in with your credentials.
5. Follow the instructions on the page displayed.

Enabling the Konvoy Async Plugin for Workspace Identity Providers

Enable the Konvoy Async Plugin to automatically update the token.

Before you begin

You or a global admin must configure a **workspace-scoped** identity provider to see this option.

Procedure

1. Open the login link you obtained from the global administrator, which they generated for your workspace or tenant.
2. Select **Credentials plugin instructions**.

3. Follow the instructions on the **(Konvoy) Credentials plugin instructions** page.

   If you use **Method 1** in the instructions documented in the **(Konvoy) Credentials plugin instructions**, then download a `kubeconfig` file that includes the contexts for all clusters.

   Alternatively, to switch between clusters, you can use **Method 2** to create a `kubeconfig` file per cluster and use the `--kubeconfig=` flag or `export KUBECONFIG=` commands.

   > **Warning:** If you choose **Method 2**, the **Set profile name** field is not optional if you have multiple clusters in your environment. Ensure you change the name of the profile for each cluster for which you want to generate a `kubeconfig` file. Otherwise, all clusters will use the same token, which makes cluster authentication vulnerable and can let users access clusters for which they do not have authorization.

Infrastructure Providers

Infrastructure providers, such as AWS, Azure, and vSphere, provide the infrastructure for your Management clusters. You may have many accounts for a single infrastructure provider. To automate cluster provisioning, NKP needs authentication keys for your preferred infrastructure provider.

To provision new clusters and manage them in the NKP UI, NKP also needs infrastructure provider credentials. Currently, you can create infrastructure providers records for:

- AWS: Creating an AWS Infrastructure Provider with a User Role on page 375
- Azure: Creating an Azure Infrastructure Provider in the UI on page 387
- vSphere: Creating a vSphere Infrastructure Provider in the UI on page 387

Infrastructure provider credentials are configured in each workspace. The name you assign must be unique across all the other namespaces in your cluster.

Viewing and Modifying Infrastructure Providers

You can use the NKP UI to view, create, and delete infrastructure provider records.

Procedure

1. From the top menu bar, select your target workspace.
2. In the **Administration** section of the sidebar menu, select **Infrastructure Providers**.

   - **AWS**:
     - Configure AWS Provider with Role Credentials (Recommended if using AWS).
     - Configure AWS Provider with static credentials.
   - **Azure**:
     -
       - Create an Azure Infrastructure Provider in the NKP UI.
       - Create a managed Azure cluster in the NKP UI.
   - **vSphere**:
     -
       - Create a vSphere Infrastructure Provider in the NKP UI.
       - Create a managed cluster on vSphere in the NKP UI.
   - **VMware Cloud Director (VCD)**:
     -
       - Create a VMware Cloud Director Infrastructure Provider in the NKP UI.

- Create a managed Cluster on VCD in the NKP UI

Deleting an infrastructure provider
Before deleting an infrastructure provider, NKP verifies whether any existing managed clusters were created using this provider.

Procedure

To delete an infrastructure provider, delete all the other clusters created with that infrastructure provider first.

This ensures that NKP has access to your infrastructure provider to remove all the resources created for a managed cluster.

Creating an AWS Infrastructure Provider with a User Role
You can create an AWS Infrastructure Provider in the NKP UI. Create your provider to add resources to your AWS account.

About this task

> **Important:** Nutanix recommends using the role-based method as this is more secure.

> **Note:** The role authentication method can only be used if your management cluster is running in AWS.

For more flexible credential configuration, we offer a role-based authentication method with an optional External ID for third party access. For more information, see the *IAM roles for Amazon EC2* in the AWS documentation at https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/iam-roles-for-amazon-ec2.html.

Procedure

1. Complete the steps in Create a Role Manually on page 375.
2. From the top menu bar, select your target workspace.
3. In the **Administration** section of the sidebar menu, select **Infrastructure Providers**.
4. Select **Add Infrastructure Provider**.
5. Select the **Amazon Web Services (AWS)** option.
6. Ensure **Role** is selected as the **Authentication Method**.
7. Enter a name for your infrastructure provider.
   Select a name that matches the AWS user.
8. Enter the **Role ARN**.
9. If you want to share the role with a 3rd party, add an **External ID**. External IDs secure your environment from accidentally used roles. For more information see *How to use an external ID when granting access to your AWS resources to a third party* in the AWS documentation at https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_create_for-user_externalid.html.
10. Click **Save**.

Create a Role Manually

Create a role manually before configuring an AWS Infrastructure Provider with a User Role.

About this task

The role should grant permissions to create the following resources in the AWS account:

- EC2 Instances
- VPC
- Subnets
- Elastic Load Balancer (ELB)
- Internet Gateway
- NAT Gateway
- Elastic Block Storage (EBS) Volumes
- Security Groups
- Route Tables
- IAM Roles

Procedure

1. The user you delegate from your role must have a minimum set of permissions. The following snippet is the minimal IAM policy required.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:AllocateAddress",
        "ec2:AssociateRouteTable",
        "ec2:AttachInternetGateway",
        "ec2:AuthorizeSecurityGroupIngress",
        "ec2:CreateInternetGateway",
        "ec2:CreateNatGateway",
        "ec2:CreateRoute",
        "ec2:CreateRouteTable",
        "ec2:CreateSecurityGroup",
        "ec2:CreateSubnet",
        "ec2:CreateTags",
        "ec2:CreateVpc",
        "ec2:ModifyVpcAttribute",
        "ec2:DeleteInternetGateway",
        "ec2:DeleteNatGateway",
        "ec2:DeleteRouteTable",
        "ec2:DeleteSecurityGroup",
        "ec2:DeleteSubnet",
        "ec2:DeleteTags",
        "ec2:DeleteVpc",
        "ec2:DescribeAccountAttributes",
        "ec2:DescribeAddresses",
        "ec2:DescribeAvailabilityZones",
        "ec2:DescribeInstances",
        "ec2:DescribeInternetGateways",
        "ec2:DescribeImages",
        "ec2:DescribeNatGateways",
        "ec2:DescribeNetworkInterfaces",
        "ec2:DescribeNetworkInterfaceAttribute",
```

```
        "ec2:DescribeRouteTables",
        "ec2:DescribeSecurityGroups",
        "ec2:DescribeSubnets",
        "ec2:DescribeVpcs",
        "ec2:DescribeVpcAttribute",
        "ec2:DescribeVolumes",
        "ec2:DetachInternetGateway",
        "ec2:DisassociateRouteTable",
        "ec2:DisassociateAddress",
        "ec2:ModifyInstanceAttribute",
        "ec2:ModifyNetworkInterfaceAttribute",
        "ec2:ModifySubnetAttribute",
        "ec2:ReleaseAddress",
        "ec2:RevokeSecurityGroupIngress",
        "ec2:RunInstances",
        "ec2:TerminateInstances",
        "tag:GetResources",
        "elasticloadbalancing:AddTags",
        "elasticloadbalancing:CreateLoadBalancer",
        "elasticloadbalancing:ConfigureHealthCheck",
        "elasticloadbalancing:DeleteLoadBalancer",
        "elasticloadbalancing:DescribeLoadBalancers",
        "elasticloadbalancing:DescribeLoadBalancerAttributes",
        "elasticloadbalancing:ApplySecurityGroupsToLoadBalancer",
        "elasticloadbalancing:DescribeTags",
        "elasticloadbalancing:ModifyLoadBalancerAttributes",
        "elasticloadbalancing:RegisterInstancesWithLoadBalancer",
        "elasticloadbalancing:DeregisterInstancesFromLoadBalancer",
        "elasticloadbalancing:RemoveTags",
        "autoscaling:DescribeAutoScalingGroups",
        "autoscaling:DescribeInstanceRefreshes",
        "ec2:CreateLaunchTemplate",
        "ec2:CreateLaunchTemplateVersion",
        "ec2:DescribeLaunchTemplates",
        "ec2:DescribeLaunchTemplateVersions",
        "ec2:DeleteLaunchTemplate",
        "ec2:DeleteLaunchTemplateVersions",
        "ec2:DescribeKeyPairs"
      ],
      "Resource": ["*"]
    },
    {
      "Effect": "Allow",
      "Action": [
        "autoscaling:CreateAutoScalingGroup",
        "autoscaling:UpdateAutoScalingGroup",
        "autoscaling:CreateOrUpdateTags",
        "autoscaling:StartInstanceRefresh",
        "autoscaling:DeleteAutoScalingGroup",
        "autoscaling:DeleteTags"
      ],
      "Resource": [
        "arn:*:autoscaling:*:*:autoScalingGroup:*:autoScalingGroupName/*"
      ]
    },
    {
      "Effect": "Allow",
      "Action": ["iam:CreateServiceLinkedRole"],
      "Resource": [
        "arn:*:iam::*:role/aws-service-role/autoscaling.amazonaws.com/
AWSServiceRoleForAutoScaling"
      ],
```

```
      "Condition": {
        "StringLike": { "iam:AWSServiceName": "autoscaling.amazonaws.com" }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["iam:CreateServiceLinkedRole"],
      "Resource": [
        "arn:*:iam::*:role/aws-service-role/elasticloadbalancing.amazonaws.com/
AWSServiceRoleForElasticLoadBalancing"
      ],
      "Condition": {
        "StringLike": {
          "iam:AWSServiceName": "elasticloadbalancing.amazonaws.com"
        }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["iam:CreateServiceLinkedRole"],
      "Resource": [
        "arn:*:iam::*:role/aws-service-role/spot.amazonaws.com/
AWSServiceRoleForEC2Spot"
      ],
      "Condition": {
        "StringLike": { "iam:AWSServiceName": "spot.amazonaws.com" }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["iam:PassRole"],
      "Resource": ["arn:*:iam::*:role/*.cluster-api-provider-aws.sigs.k8s.io"]
    },
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:CreateSecret",
        "secretsmanager:DeleteSecret",
        "secretsmanager:TagResource"
      ],
      "Resource": ["arn:*:secretsmanager:*:*:secret:aws.cluster.x-k8s.io/*"]
    },
    {
      "Effect": "Allow",
      "Action": ["ssm:GetParameter"],
      "Resource": ["arn:*:ssm:*:*:parameter/aws/service/eks/optimized-ami/*"]
    },
    {
      "Effect": "Allow",
      "Action": ["iam:CreateServiceLinkedRole"],
      "Resource": [
        "arn:*:iam::*:role/aws-service-role/eks.amazonaws.com/
AWSServiceRoleForAmazonEKS"
      ],
      "Condition": {
        "StringLike": { "iam:AWSServiceName": "eks.amazonaws.com" }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["iam:CreateServiceLinkedRole"],
      "Resource": [
```

```
        "arn:*:iam::*:role/aws-service-role/eks-nodegroup.amazonaws.com/
AWSServiceRoleForAmazonEKSNodegroup"
      ],
      "Condition": {
        "StringLike": { "iam:AWSServiceName": "eks-nodegroup.amazonaws.com" }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["iam:CreateServiceLinkedRole"],
      "Resource": [
        "arn:aws:iam::*:role/aws-service-role/eks-fargate-pods.amazonaws.com/
AWSServiceRoleForAmazonEKSForFargate"
      ],
      "Condition": {
        "StringLike": { "iam:AWSServiceName": "eks-fargate.amazonaws.com" }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["iam:GetRole", "iam:ListAttachedRolePolicies"],
      "Resource": ["arn:*:iam::*:role/*"]
    },
    {
      "Effect": "Allow",
      "Action": ["iam:GetPolicy"],
      "Resource": ["arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"]
    },
    {
      "Effect": "Allow",
      "Action": [
        "eks:DescribeCluster",
        "eks:ListClusters",
        "eks:CreateCluster",
        "eks:TagResource",
        "eks:UpdateClusterVersion",
        "eks:DeleteCluster",
        "eks:UpdateClusterConfig",
        "eks:UntagResource",
        "eks:UpdateNodegroupVersion",
        "eks:DescribeNodegroup",
        "eks:DeleteNodegroup",
        "eks:UpdateNodegroupConfig",
        "eks:CreateNodegroup",
        "eks:AssociateEncryptionConfig"
      ],
      "Resource": ["arn:*:eks:*:*:cluster/*", "arn:*:eks:*:*:nodegroup/*/*/*"]
    },
    {
      "Effect": "Allow",
      "Action": [
        "eks:ListAddons",
        "eks:CreateAddon",
        "eks:DescribeAddonVersions",
        "eks:DescribeAddon",
        "eks:DeleteAddon",
        "eks:UpdateAddon",
        "eks:TagResource",
        "eks:DescribeFargateProfile",
        "eks:CreateFargateProfile",
        "eks:DeleteFargateProfile"
      ],
```

```
      "Resource": ["*"]
    },
    {
      "Effect": "Allow",
      "Action": ["iam:PassRole"],
      "Resource": ["*"],
      "Condition": {
        "StringEquals": { "iam:PassedToService": "eks.amazonaws.com" }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["kms:CreateGrant", "kms:DescribeKey"],
      "Resource": ["*"],
      "Condition": {
        "ForAnyValue:StringLike": {
          "kms:ResourceAliases": "alias/cluster-api-provider-aws-*"
        }
      }
    }
  ]
}
```

Make sure to also add a correct trust relationship to the created role.

This preceding example allows everyone within the same account to assign `AssumeRole` with the created role.

2. Replace `YOURACCOUNTRESTRICTION` with the AWS Account ID that you want `AssumeRole` from.

   Note: Never add a `*/` wildcard. This opens your account to the public.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "ec2.amazonaws.com",
        "AWS": "arn:aws:iam::YOURACCOUNTRESTRICTION:root"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

3. To use the role created, attach the following policy to the role which is already attached to your managed or attached cluster. Replace `YOURACCOUNTRESTRICTION` with the AWS Account ID where the role `AssumeRole` is saved. Also, replace `THEROLEYOUCREATED` with the AWS Role name.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AssumeRoleKommander",
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::YOURACCOUNTRESTRICTION:role/THEROLEYOUCREATED"
    }
  ]
}
```

Configuring an AWS Infrastructure Provider with Static Credentials

When configuring an infrastructure provider with static credentials, you need an access ID and secret key for a user with a set of minimum capabilities.

About this task

To create an AWS infrastructure provider with static credentials:

Procedure

1. In NKP, select the workspace associated with the credentials that you are adding.
2. Navigate to **Administration** > **Infrastructure Providers**, and click **Add Infrastructure Provider** .
3. Select the **Amazon Web Services (AWS)** option.
4. Ensure **Static** is selected as the authentication method.
5. Enter a name for your infrastructure provider for later reference.
   Consider choosing a name that matches the AWS user.
6. Enter a access ID and secret keys using the keys generated above.
7. click **Save** to save your provider.

Creating a New User Using CLI

You can create a new user using CLI.

Before you begin

You must install the AWS CLI utility. For more information, see *Install or update to the latest version of the AWS CLI* in the AWS documentation at https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html.

Procedure

Create a new user with the following AWS CLI commands:

- ```
  aws iam create-user --user-name Kommander
  ```
- ```
  aws iam create-policy --policy-name kommander-policy --policy-document
   '{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Action":
  ["ec2:AllocateAddress","ec2:AssociateRouteTable","ec2:AttachInternetGateway","ec2:AuthorizeSecur
  ["*"]},{"Effect":"Allow","Action":
  ["autoscaling:CreateAutoScalingGroup","autoscaling:UpdateAutoScalingGroup","autoscaling:CreateOr
  ["arn:*:autoscaling:*:*:autoScalingGroup:*:autoScalingGroupName/*"]},
  {"Effect":"Allow","Action":["iam:CreateServiceLinkedRole"],"Resource":
  ["arn:*:iam::*:role/aws-service-role/autoscaling.amazonaws.com/
  AWSServiceRoleForAutoScaling"],"Condition":{"StringLike":
  {"iam:AWSServiceName":"autoscaling.amazonaws.com"}}},
  {"Effect":"Allow","Action":["iam:CreateServiceLinkedRole"],"Resource":
  ["arn:*:iam::*:role/aws-service-role/elasticloadbalancing.amazonaws.com/
  AWSServiceRoleForElasticLoadBalancing"],"Condition":{"StringLike":
  {"iam:AWSServiceName":"elasticloadbalancing.amazonaws.com"}}},
  {"Effect":"Allow","Action":["iam:CreateServiceLinkedRole"],"Resource":
  ["arn:*:iam::*:role/aws-service-role/spot.amazonaws.com/
  AWSServiceRoleForEC2Spot"],"Condition":{"StringLike":
  {"iam:AWSServiceName":"spot.amazonaws.com"}}},{"Effect":"Allow","Action":
  ["iam:PassRole"],"Resource":["arn:*:iam::*:role/*.cluster-
  ```

```
api-provider-aws.sigs.k8s.io"]},{"Effect":"Allow","Action":
["secretsmanager:CreateSecret","secretsmanager:DeleteSecret","secretsmanager:TagResource"],"Reso
["arn:*:secretsmanager:*:*:secret:aws.cluster.x-k8s.io/*"]},
{"Effect":"Allow","Action":["ssm:GetParameter"],"Resource":["arn:*:ssm:*:*:parameter/
aws/service/eks/optimized-ami/*"]},{"Effect":"Allow","Action":
["iam:CreateServiceLinkedRole"],"Resource":["arn:*:iam::*:role/aws-service-
role/eks.amazonaws.com/AWSServiceRoleForAmazonEKS"],"Condition":{"StringLike":
{"iam:AWSServiceName":"eks.amazonaws.com"}}},{"Effect":"Allow","Action":
["iam:CreateServiceLinkedRole"],"Resource":["arn:*:iam::*:role/aws-service-role/
eks-nodegroup.amazonaws.com/AWSServiceRoleForAmazonEKSNodegroup"],"Condition":
{"StringLike":{"iam:AWSServiceName":"eks-nodegroup.amazonaws.com"}}},
{"Effect":"Allow","Action":["iam:CreateServiceLinkedRole"],"Resource":
["arn:aws:iam::*:role/aws-service-role/eks-fargate-pods.amazonaws.com/
AWSServiceRoleForAmazonEKSForFargate"],"Condition":{"StringLike":
{"iam:AWSServiceName":"eks-fargate.amazonaws.com"}}},{"Effect":"Allow","Action":
["iam:GetRole","iam:ListAttachedRolePolicies"],"Resource":["arn:*:iam::*:role/
*"]},{"Effect":"Allow","Action":["iam:GetPolicy"],"Resource":
["arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"]},{"Effect":"Allow","Action":
["eks:DescribeCluster","eks:ListClusters","eks:CreateCluster","eks:TagResource","eks:UpdateClust
["arn:*:eks:*:*:cluster/*","arn:*:eks:*:*:nodegroup/*/*/*"]},
{"Effect":"Allow","Action":
["eks:ListAddons","eks:CreateAddon","eks:DescribeAddonVersions","eks:DescribeAddon","eks:DeleteA
["*"]},{"Effect":"Allow","Action":["iam:PassRole"],"Resource":
["*"],"Condition":{"StringEquals":{"iam:PassedToService":"eks.amazonaws.com"}}},
{"Effect":"Allow","Action":["kms:CreateGrant","kms:DescribeKey"],"Resource":
["*"],"Condition":{"ForAnyValue:StringLike":{"kms:ResourceAliases":"alias/cluster-
api-provider-aws-*"}}}]}'
```

- ```
  aws iam attach-user-policy --user-name Kommander --policy-arn $(aws iam list-policies
   --query 'Policies[?PolicyName==`kommander-policy`].Arn' | grep -o '".*"' | tr -d
   '"')
  ```

- ```
  aws iam create-access-key --user-name Kommander
  ```

Using an Existing User to Configure an AWS Infrastructure

You can use an existing AWS user with the credentials configured.

Before you begin

For more information, see *Configuration and credential file settings* in the AWS documentation at https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-files.html. The user must be authorized to create the following resources in the AWS account:

- EC2 Instances
- VPC
- Subnets
- Elastic Load Balancer (ELB)
- Internet Gateway
- NAT Gateway
- Elastic Block Storage (EBS) Volumes
- Security Groups
- Route Tables

- IAM Roles

Procedure

The following is the minimal IAM policy required.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:AllocateAddress",
        "ec2:AssociateRouteTable",
        "ec2:AttachInternetGateway",
        "ec2:AuthorizeSecurityGroupIngress",
        "ec2:CreateInternetGateway",
        "ec2:CreateNatGateway",
        "ec2:CreateRoute",
        "ec2:CreateRouteTable",
        "ec2:CreateSecurityGroup",
        "ec2:CreateSubnet",
        "ec2:CreateTags",
        "ec2:CreateVpc",
        "ec2:ModifyVpcAttribute",
        "ec2:DeleteInternetGateway",
        "ec2:DeleteNatGateway",
        "ec2:DeleteRouteTable",
        "ec2:DeleteSecurityGroup",
        "ec2:DeleteSubnet",
        "ec2:DeleteTags",
        "ec2:DeleteVpc",
        "ec2:DescribeAccountAttributes",
        "ec2:DescribeAddresses",
        "ec2:DescribeAvailabilityZones",
        "ec2:DescribeInstances",
        "ec2:DescribeInternetGateways",
        "ec2:DescribeImages",
        "ec2:DescribeNatGateways",
        "ec2:DescribeNetworkInterfaces",
        "ec2:DescribeNetworkInterfaceAttribute",
        "ec2:DescribeRouteTables",
        "ec2:DescribeSecurityGroups",
        "ec2:DescribeSubnets",
        "ec2:DescribeVpcs",
        "ec2:DescribeVpcAttribute",
        "ec2:DescribeVolumes",
        "ec2:DetachInternetGateway",
        "ec2:DisassociateRouteTable",
        "ec2:DisassociateAddress",
        "ec2:ModifyInstanceAttribute",
        "ec2:ModifyNetworkInterfaceAttribute",
        "ec2:ModifySubnetAttribute",
        "ec2:ReleaseAddress",
        "ec2:RevokeSecurityGroupIngress",
        "ec2:RunInstances",
        "ec2:TerminateInstances",
        "tag:GetResources",
        "elasticloadbalancing:AddTags",
        "elasticloadbalancing:CreateLoadBalancer",
        "elasticloadbalancing:ConfigureHealthCheck",
        "elasticloadbalancing:DeleteLoadBalancer",
```

```
        "elasticloadbalancing:DescribeLoadBalancers",
        "elasticloadbalancing:DescribeLoadBalancerAttributes",
        "elasticloadbalancing:ApplySecurityGroupsToLoadBalancer",
        "elasticloadbalancing:DescribeTags",
        "elasticloadbalancing:ModifyLoadBalancerAttributes",
        "elasticloadbalancing:RegisterInstancesWithLoadBalancer",
        "elasticloadbalancing:DeregisterInstancesFromLoadBalancer",
        "elasticloadbalancing:RemoveTags",
        "autoscaling:DescribeAutoScalingGroups",
        "autoscaling:DescribeInstanceRefreshes",
        "ec2:CreateLaunchTemplate",
        "ec2:CreateLaunchTemplateVersion",
        "ec2:DescribeLaunchTemplates",
        "ec2:DescribeLaunchTemplateVersions",
        "ec2:DeleteLaunchTemplate",
        "ec2:DeleteLaunchTemplateVersions",
        "ec2:DescribeKeyPairs"
      ],
      "Resource": ["*"]
    },
    {
      "Effect": "Allow",
      "Action": [
        "autoscaling:CreateAutoScalingGroup",
        "autoscaling:UpdateAutoScalingGroup",
        "autoscaling:CreateOrUpdateTags",
        "autoscaling:StartInstanceRefresh",
        "autoscaling:DeleteAutoScalingGroup",
        "autoscaling:DeleteTags"
      ],
      "Resource": [
        "arn:*:autoscaling:*:*:autoScalingGroup:*:autoScalingGroupName/*"
      ]
    },
    {
      "Effect": "Allow",
      "Action": ["iam:CreateServiceLinkedRole"],
      "Resource": [
        "arn:*:iam::*:role/aws-service-role/autoscaling.amazonaws.com/
AWSServiceRoleForAutoScaling"
      ],
      "Condition": {
        "StringLike": { "iam:AWSServiceName": "autoscaling.amazonaws.com" }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["iam:CreateServiceLinkedRole"],
      "Resource": [
        "arn:*:iam::*:role/aws-service-role/elasticloadbalancing.amazonaws.com/
AWSServiceRoleForElasticLoadBalancing"
      ],
      "Condition": {
        "StringLike": {
          "iam:AWSServiceName": "elasticloadbalancing.amazonaws.com"
        }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["iam:CreateServiceLinkedRole"],
      "Resource": [
```

```
        "arn:*:iam::*:role/aws-service-role/spot.amazonaws.com/
AWSServiceRoleForEC2Spot"
      ],
      "Condition": {
        "StringLike": { "iam:AWSServiceName": "spot.amazonaws.com" }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["iam:PassRole"],
      "Resource": ["arn:*:iam::*:role/*.cluster-api-provider-aws.sigs.k8s.io"]
    },
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:CreateSecret",
        "secretsmanager:DeleteSecret",
        "secretsmanager:TagResource"
      ],
      "Resource": ["arn:*:secretsmanager:*:*:secret:aws.cluster.x-k8s.io/*"]
    },
    {
      "Effect": "Allow",
      "Action": ["ssm:GetParameter"],
      "Resource": ["arn:*:ssm:*:*:parameter/aws/service/eks/optimized-ami/*"]
    },
    {
      "Effect": "Allow",
      "Action": ["iam:CreateServiceLinkedRole"],
      "Resource": [
        "arn:*:iam::*:role/aws-service-role/eks.amazonaws.com/
AWSServiceRoleForAmazonEKS"
      ],
      "Condition": {
        "StringLike": { "iam:AWSServiceName": "eks.amazonaws.com" }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["iam:CreateServiceLinkedRole"],
      "Resource": [
        "arn:*:iam::*:role/aws-service-role/eks-nodegroup.amazonaws.com/
AWSServiceRoleForAmazonEKSNodegroup"
      ],
      "Condition": {
        "StringLike": { "iam:AWSServiceName": "eks-nodegroup.amazonaws.com" }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["iam:CreateServiceLinkedRole"],
      "Resource": [
        "arn:aws:iam::*:role/aws-service-role/eks-fargate-pods.amazonaws.com/
AWSServiceRoleForAmazonEKSForFargate"
      ],
      "Condition": {
        "StringLike": { "iam:AWSServiceName": "eks-fargate.amazonaws.com" }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["iam:GetRole", "iam:ListAttachedRolePolicies"],
```

```
      "Resource": ["arn:*:iam::*:role/*"]
    },
    {
      "Effect": "Allow",
      "Action": ["iam:GetPolicy"],
      "Resource": ["arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"]
    },
    {
      "Effect": "Allow",
      "Action": [
        "eks:DescribeCluster",
        "eks:ListClusters",
        "eks:CreateCluster",
        "eks:TagResource",
        "eks:UpdateClusterVersion",
        "eks:DeleteCluster",
        "eks:UpdateClusterConfig",
        "eks:UntagResource",
        "eks:UpdateNodegroupVersion",
        "eks:DescribeNodegroup",
        "eks:DeleteNodegroup",
        "eks:UpdateNodegroupConfig",
        "eks:CreateNodegroup",
        "eks:AssociateEncryptionConfig"
      ],
      "Resource": ["arn:*:eks:*:*:cluster/*", "arn:*:eks:*:*:nodegroup/*/*/*"]
    },
    {
      "Effect": "Allow",
      "Action": [
        "eks:ListAddons",
        "eks:CreateAddon",
        "eks:DescribeAddonVersions",
        "eks:DescribeAddon",
        "eks:DeleteAddon",
        "eks:UpdateAddon",
        "eks:TagResource",
        "eks:DescribeFargateProfile",
        "eks:CreateFargateProfile",
        "eks:DeleteFargateProfile"
      ],
      "Resource": ["*"]
    },
    {
      "Effect": "Allow",
      "Action": ["iam:PassRole"],
      "Resource": ["*"],
      "Condition": {
        "StringEquals": { "iam:PassedToService": "eks.amazonaws.com" }
      }
    },
    {
      "Effect": "Allow",
      "Action": ["kms:CreateGrant", "kms:DescribeKey"],
      "Resource": ["*"],
      "Condition": {
        "ForAnyValue:StringLike": {
          "kms:ResourceAliases": "alias/cluster-api-provider-aws-*"
        }
      }
    }
  ]
```

```
}
```

Creating an Azure Infrastructure Provider in the UI

You can create an Azure Infrastructure Provider in the NKP UI.

Before you begin

Before you provision Azure clusters using the NKP UI, you must first create an Azure infrastructure provider to contain your Azure credentials:

Procedure

1. Log in to the Azure command line.

   ```
   az login
   ```

2. Create an Azure Service Principal (SP) by running the following command.

   ```
   az ad sp create-for-rbac --role contributor --name "$(whoami)-konvoy" --scopes=/
   subscriptions/$(az account show --query id -o tsv)
   ```

3. From the Dashboard menu, select **Infrastructure Providers**.
4. Select **Add Infrastructure Provider**.
5. If you are already in a workspace, the provider is automatically created in that workspace.
6. Select **Microsoft Azure**.
7. Add a **Name** for your Infrastructure Provider.
8. Copy and paste the following values into the indicated fields.
   - Copy the `id` output from the login command above and paste it into the **Subscription ID** field.
   - Copy the `tenant` used in step 2 and paste it into the **Tenant ID** field.
   - Copy the `appId` used in step 2 and paste it into the **Client ID** field.
   - Copy the `password` used in step 2 and paste it into the **Client Secret** field.
9. Click **Save**.

Creating a vSphere Infrastructure Provider in the UI

You can create a vSphere Infrastructure Provider in the NKP UI.

Procedure

1. Log in to your NKP Ultimate UI and access the NKP home page.
2. From the left navigation menu, select **Infrastructure Providers**.
3. Select **Add Infrastructure Provider**.
4. If you are already in a workspace, the new infrastructure provider is automatically created in that workspace.

5. Select **vSphere** and add the following information.

   - Add a **Name** for your infrastructure provider.
   - In the **Username** field, enter a valid vSphere vCenter username.
   - In the **Password** field, enter a valid vSphere vCenter user password.
   - In the **Host URL** field, enter the vCenter Server URL.

     This field must contain only the domain for the URL, such as `vcenter.ca1.your-org-platform.domain.cloud`. Do not specify the protocols `http://` or `https://` to avoid errors during cluster creation.
   - (Optional) Enter a valid **TLS Certificate Thumbprint** value.

     The TLS Certificate Thumbprint helps in creating a secure connection to VMware vCenter. If you do not have a thumprint, your connection might be marked as insecure. This field is optional because you might not have a self-signed vCenter instance, and you only need the thumbprint if you do. The command to obtain this SHA-1 thumbprint for the vSphere's server's TLS certificate is listed under the field in the interface.

6. click **Save**.

Creating a VMware Cloud Director Infrastructure Provider in the UI
You can create a VMware Cloud Director Infrastructure Provider in the NKP UI.

Before you begin

Before you provision VMware Cloud Director (VCD) clusters using the NKP UI, you must

- Complete the VMware Cloud Director Prerequisites on page 964 for the VMware Cloud Director.
- Create a VCD infrastructure provider to contain your credentials.

Procedure

1. Log in to your NKP Ultimate UI.
2. From the left-navigation menu, select **Infrastructure Providers**.
3. Select **Add Infrastructure Provider**.
4. For referencing this infrastructure provider, add a **Provider Name**.
5. Specify a **Refresh Token** name that you created in VCD.
   You can generate API Tokens to grant programmatic access to VCD for both providers and tenant users. For more information, see *Cloud Director API Token* in the VMWare website at https://blogs.vmware.com/cloudprovider/2022/03/cloud-director-api-token.html. Automation scripts or third-party solutions use the API token to make requests of VCD on the user's behalf. You must generate and name the token within VCD, and also grant tenants the right to use and manage them.

6. Specify a **Site URL** value, which must begin with `https://`. For example, `"https://vcd.example.com"`. Do not use a trailing forward slash character.

> **Warning:** Ensure to make a note of the Refresh Token, as it displays only one time, and cannot be retrieved afterwards.

> **Note:** Editing a VCD infrastructure provider means that you are changing the credentials under which NKP connects to VMware Cloud Director. This can have negative effects on any existing cluster that use that infrastructure provider record.
>
> To prevent errors, NKP first checks if there are any existing clusters for the selected infrastructure provider. If a VCD infrastructure provider has existing clusters, NKP displays an error message and prevents you from editing the infrastructure provider.

Header, Footer, and Logo Implementation

Use the customizable **Banners** page to add header banners, footer banners, and select the colors for them. You can define header and/or footer banners for your NKP pages and turn them on and off, as needed.

NKP displays your header and footer banner in a default typeface and size, which cannot be changed.

Creating a Header Banner

The text you type in the **Text** field appears centered at the top of the screen. The text length is limited to 100 characters, including spaces. The text color is determined by the background color and automatically calculates an appropriate light or dark color for you.

About this task

The **Color** selection control uses the style of your browser for its color picker tool. This control allows you to select a color for your header banner:

Procedure

1. Enter the color's Hex code.
2. Select a general color range, and then select a specific shade or tint. The color input uses the style of your browser for its color selection tool.
3. Select the eyedropper, move it to a sample of the color you want and select once to select that color's location.

Creating a Footer Banner

The text you type in the **Text** field appears centered at the bottom of the screen.

About this task

The **Color** selection control uses the style of your browser for its color picker tool. This control allows you to select a color for your footer banner:

Procedure

1. Enter the color's Hex code.
2. Select a general color range from the slider bar, and then select a specific shade or tint with your mouse cursor.
3. Select the eyedropper, move it to a sample of the color you want and select once to select that color's location.

Adding Your Organization's Logo Using the Drag and Drop Option

When you license and install NKP Ultimate or Gov Advanced, you also have the option to add your organization's logo to the header. The width of the header banner automatically adjusts to contain your logo. NKP automatically places your logo on the left side of the header and centers it vertically.

Before you begin

Your logo graphic must meet the following criteria:

- Use a suggested file format: PNG, SVG, or JPEG.
- The file size cannot exceed 200 KB.

Error messages affecting the file to upload appear below the image in red, inside the shaded logo area.

> **Note:** To provide security against certain kinds of malicious activity, your browser has a *same-origin policy* for accessing resources. When you upload a file, the browser creates a unique identifier for the file. This prevents you from selecting a file more than once.

Procedure

1. Locate the required file in the MacOS Finder or Windows File Explorer.
2. Drag and drop an image of the appropriate file type into the shaded area to see a preview of the image and display the file name.

   You can select **X** on the upper-right or **Remove** on the lower-right to clear the image, if needed.
3. Click **Save**.

> **Warning:** You cannot select a file for drag-and-drop if it does not have a valid image format.

Adding Your Organization's Logo Using the Upload Option

Upload a logo image to the header by browsing your file.

Procedure

1. Select **Browse Files**.
2. To clear the image, select **X** or click the **Remove** link, if needed.
3. Click **Save**.

Applications

This section includes information on the applications you can deploy in NKP.

Customizing Your Application

If you want to customize an application or change how a specific application is deployed, you can create a ConfigMap to change or add values to the information that is stored in the `HelmRelease`. Override the default configuration of an application by setting the configOverrides field on the AppDeployment to that ConfigMap. This overrides the configuration of the app for all clusters within the workspace.

About this task

For workspace applications, you can also enable and customize them on a per-cluster basis. For instructions on how to enable and customize an application per cluster in a given workspace, see Cluster-scoped Application for Existing AppDeployments on page 414.

Before you begin

Set the `WORKSPACE_NAMESPACE` environment variable to the name of the workspace's namespace where the cluster is attached:

```
export WORKSPACE_NAMESPACE=<your_workspace_namespace>
```

You can now copy the following commands without replacing the placeholder with your workspace namespace every time you run a command.

Here's an example of how to customize the AppDeployment of Kube Prometheus Stack:

Procedure

1. Provide the name of a ConfigMap with the custom configuration in the AppDeployment.

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: kube-prometheus-stack
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    name: kube-prometheus-stack-46.8.0
    kind: ClusterApp
  configOverrides:
    name: kube-prometheus-stack-overrides-attached
EOF
```

2. Create the ConfigMap with the name provided in the previous step, which provides the custom configuration on top of the default configuration.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: ${WORKSPACE_NAMESPACE}
  name: kube-prometheus-stack-overrides-attached
data:
  values.yaml: |
    prometheus:
      prometheusSpec:
        storageSpec:
          volumeClaimTemplate:
            spec:
              resources:
                requests:
                  storage: 150Gi
EOF
```

Printing and Reviewing the Current State of an AppDeployment Resource

If you want to know how the AppDeployment resource is currently configured, use the commands below to print a table of the declared information. If the AppDeployment is configured for several clusters in a workspace, a column will display a list of the clusters.

About this task

You can review all the AppDeployments in a workspace or a specific AppDeployments of an application in a workspace.

Procedure

You can run the following commands to review AppDeployments.

» **All AppDeployments in a workspace**: To review the state of the AppDeployment resource for a specific workspace, run the `get` command with the name of your workspace. Here's as example:

```
nkp get appdeployments -w kommander-workspace
```

The output displays a list of all your applications:

```
NAME APP CLUSTERS
[...]
kube-oidc-proxy                 kube-oidc-proxy-0.3.2           host-cluster
kube-prometheus-stack           kube-prometheus-stack-46.8.0    host-cluster
kubecost                        kubecost-0.35.1                 host-cluster
[...]
```

» **Specific AppDeployment of an application in a workspace**: To review the state of a specific AppDeployment of an application, run the `get` command with the name of the application and your workspace. Here's an example:

```
nkp get appdeployment kube-prometheus-stack -w kommander-workspace
```

The output is as follows:

```
NAME APP CLUSTERS
kube-prometheus-stack kube-prometheus-stack-46.8.0 host-cluster
```

Note: For more information on how to `create`, or `get` an `AppDeployment`, see the CLI documentation.

Deployment Scope

In a single-cluster environment with an Starter license, AppDeployments enable customizing any platform application. In a multi-cluster environment with a Starter license, AppDeployments enable workspace-level, project-level, and per-cluster deployment and customization of workspace applications.

Logging Stack Application Sizing Recommendations

Sizing recommendations for Logging Stack applications.

For information on how you customize your AppDeployments, see AppDeployment Resources on page 410.

Note: When configuring storage for `logging-operator-logging-overrides`, ensure that you create a ConfigMap in your workspace namespace for every cluster in that workspace.

Keep in mind that you can configure `logging-operator-logging-overrides` only through the CLI.

Table 26: Table

No. of Worker Nodes	Log Generating Load	Application	Suggested Configuration

50	1.4 MB/s	Logging Operator Logging Override Config	`values.yaml: |-`
			`  clusterOutputs:`
			`  - name: loki`
			`  spec:`
			`  loki:`
			`  # change ${WORKSPACE_NAMESPACE} to the actual value of your workspace namespace`
			`  url: http://grafana-loki-loki-distributed-gateway.${WORKSPACE_NAMESPACE}.svc.cluster.local:8`
			`  extract_kubernetes_labels: true`
			`  configure_kubernetes_labels: true`
			`  buffer:`
			`  disabled: true`
			`  retry_forever: false`
			`  retry_max_times: 5`
			`  flush_mode: interval`
			`  flush_interval: 10s`
			`  flush_thread_count: 8`
			`  extra_labels:`
			`  log_source: kubernetes_container`
			`  fluentbit:`
			`  inputTail:`
			`  Mem_Buf_Limit: 512MB`
			`  fluentd:`
			`  bufferStorageVolume:`
			`  emptyDir:`
			`  medium: Memory`
			`  disablePvc: true`
			`  scaling:`
			`  replicas: 10`
			`  resources:`
			`  requests:`
			`  memory: 1000Mi`
			`  cpu: 1000m`
			`  limits:`
			`  memory: 2000Mi`
			`  cpu: 1000m`

		Loki	`ingester:`
			`  replicas: 10`
			`distributor:`
			`  replicas: 2`

100	8.5 MB/s	Logging Operator Logging Override Config	`values.yaml: |-`
			`  clusterOutputs:`
			`  name: loki`
			`  spec:`
			`  loki:`
			`  # change ${WORKSPACE_NAMESPACE} to`

Rook Ceph Cluster Sizing Recommendations

Sizing recommendations for Logging Stack applications.

For information on how you customize your AppDeployments, see AppDeployment Resources on page 410.

> **Note:** To add more storage to `rook-ceph-cluster`, copy and paste `storageClassDeviceSets` list from the `rook-ceph-cluster-`*`your-ceph-version`*`-d2iq-defaults` ConfigMap into your workspace where `rook-ceph-cluster` is present and then modify `count` and `volumeClaimTemplates.spec.resource.requests.storage` .

Table 27: Table

No. of Worker Nodes	Application	Suggested Configuration
50	Rook Ceph Cluster	cephClusterSpec: labels: monitoring: prometheus.kommander.d2iq.io/select: "true" storage: storageClassDeviceSets: - name: rook-ceph-osd-set1 count: 4 portable: true encrypted: false placement: topologySpreadConstraints: - maxSkew: 1 topologyKey: topology.kubernetes.io/zone # The nodes in the same rack have the same topology.kubernetes.io/zone label. whenUnsatisfiable: ScheduleAnyway labelSelector: matchExpressions: - key: app operator: In values: - rook-ceph-osd - rook-ceph-osd- prepare - maxSkew: 1 topologyKey: kubernetes.io/ hostname whenUnsatisfiable: ScheduleAnyway labelSelector: matchExpressions: - key: app operator: In values: - rook-ceph-osd - rook-ceph-osd- prepare volumeClaimTemplates: # If there are some faster devices and some slower devices, it is more efficient to use # separate metadata, wal, and data devices. # Refer https://rook.io/docs/ rook/v1.10/CRDs/Cluster/pvc-cluster/ #dedicated-metadata-and-wal-device-for- osd-on-pvc - metadata: name: data spec: resources: requests: storage: 120Gi volumeMode: Block accessModes: - ReadWriteOnce dkp: grafana-loki: additionalConfig:

Application Management Using the UI

Choose your license type for instructions on how to enable and customize an application and then verify it has been deployed correctly.

Ultimate: Application Management Using the UI

You can deploy and uninstall application using the UI.

> **Note:** To use the CLI to deploy or uninstall applications, see Deploying Platform Applications Using CLI on page 403.

Ultimate: Enabling an Application Using the UI

This topic describes how to enable your platform applications from the UI.

Procedure

1. From the top menu bar, select your target workspace.
2. From the sidebar, browse through the available applications from your configured repositories, and select **Applications**.
3. Select the three-dot button of the desired application card > **Enable**.
4. If available, select a version from the dropdown list.
   The dropdown list is only visible if there are more than one versions to choose from.
5. Select the clusters where you want to deploy the application.
6. **For customizations only**: to override the default configuration values, in the sidebar, select **Configuration**.

   > **Note:** If there are customization Overrides at the workspace and cluster level, they are combined for implementation. Cluster-level Overrides take precedence over Workspace Overrides.

   a. To customize an application for all clusters in a workspace, copy your customized values into the text editor under **Workspace Application Configuration** or upload your YAML file that contains the values.

   ```
   someField: someValue
   ```

   b. To add a customization per cluster, copy the customized values into the text editor of each cluster under **Cluster Application Configuration Override** or upload your YAML file that contains the values.

   ```
   someField: someValue
   ```

7. Verify that the details are correct and select **Enable**.
   There may be dependencies between the applications, which are listed in Platform Applications Dependencies on page 404. Review them carefully before customizing to ensure that the applications are deployed successfully.

Ultimate: Customizing an Application Using the UI

You can enable an application and customize it using the UI.

About this task

To customize the applications that are deployed to a workspace's cluster using the UI:

Procedure

1. From the top menu bar, select your target workspace.
2. From the sidebar, browse through the available applications from your configured repositories, and select **Applications**.
3. In the **Application** card you want to customize, select the three dot menu and **Edit**.
4. To override the default configuration values, select **Configuration** in the sidebar.

   > **Note:** If there are customization Overrides at the workspace and cluster levels, they are combined for implementation. Cluster-level Overrides take precedence over Workspace Overrides.

   a. To customize an application for all clusters in a workspace, copy your customized values into the text editor under **Workspace Application Configuration** or upload your YAML file that contains the values.

   ```
   someField: someValue
   ```

   b. To add a customization per cluster, copy the customized values into the text editor of each cluster under **Cluster Application Configuration Override** or upload your YAML file that contains the values.

   ```
   someField: someValue
   ```

5. Verify that the details are correct and select **Save**.

Ultimate: Customizing an Application For a Specific Cluster

You can also customize an application for a specific cluster from the **Clusters** view:

Procedure

1. From the sidebar menu, select **Clusters**.
2. Select the target cluster.
3. Select the **Applications** tab.
4. Navigate to the target **Applications** card.
5. Select the three-dot menu > **Edit**.

Ultimate: Verifying an Application using the UI

The application has now been enabled.

About this task

To verify that the application is deployed correctly:

Procedure

1. From the top menu bar, select your target workspace.

2. Select the cluster you want to verify.
   a. Select **Management Cluster** if your target cluster is the Management Cluster Workspace.
   b. Otherwise, select **Clusters**, and choose your target cluster.
3. Select the **Applications** tab and navigate to the application you want to verify.
4. If the application was deployed successfully, the status **Deployed** appears in the application card. Otherwise, hover over the failed status to obtain more information on why the application failed to deploy.

> **Note:** It can take several minutes for the application to deploy completely. If the **Deployed** or **Failed** status is not displayed, the deployment process is not finished.

Ultimate: Disabling an Application Using the UI

You can disable an application using the UI.

About this task

Follow these steps to disable an application with the UI:

Procedure

1. From the top menu bar, select your target workspace.
2. From the sidebar, browse through the available applications from your configured repositories, and select **Applications**.
3. Select the three-dot button of the desired application card > **Uninstall**.
4. Follow the instruction on the confirmation pop-up message and select **Uninstall Application**.

Pro: Application Management Using the UI

You can enable an application and customize it using the UI.

You can deploy and uninstall application using the UI.

> **Note:** To use the CLI to deploy or uninstall applications, see Deploying Platform Applications Using CLI on page 403.

Pro: Enabling an Application Using the UI

About this task

To enable your platform applications from the UI in Kommander:

Procedure

1. From the sidebar to browse through the available applications from your configured repositories, select **Applications**.
2. Select the three-dot button of the desired application card > **Enable**.
3. If available, select a version from the dropdown list. This dropdown list is only visible if there is more than one version to choose from.

4. Select the clusters where you want to deploy the application.
5. **For customizations only**: to override the default configuration values, select **Configuration** in the sidebar.
   a. To customize an application for all clusters in a workspace, copy your customized values into the text editor under **Workspace Application Configuration** or upload your YAML file that contains the values.
   ```
   someField: someValue
   ```
6. Confirm the details are correct and then select **Enable**.
   There may be dependencies between the applications, which are listed in Platform Applications Dependencies on page 404. Review them carefully before customizing to ensure that the applications are deployed successfully.

Pro: Customizing an Application Using the UI

About this task

To customize the applications that are deployed to yout Management Cluster cluster using the UI:

Procedure

1. From the sidebar, browse through the available applications from your configured repositories and select **Applications**
2. In the **Application** card you want to customize, select the three dot menu and **Edit**.
3. To override the default configuration values, select **Configuration** in the sidebar.

   Note: If there are customization Overrides at the workspace and cluster level, they are combined for implementation. Cluster-level Overrides take precedence over Workspace Overrides.

   a. To customize an application for all clusters in a workspace, copy your customized values into the text editor under **Workspace Application Configuration** or upload your YAML file that contains the values.
   ```
   someField: someValue
   ```
4. Verify that the details are correct and select **Save**.

Pro: Verifying an Application using the UI

The application has now been enabled.

About this task

To ensure that the application is deployed correctly:

Procedure

1. From the sidebar, select **Management Cluster**.
2. Select the **Applications** tab and navigate to the application you want to verify.

3. If the application was deployed successfully, the status **Deployed** appears in the application card. Otherwise, hover over the failed status to obtain more information on why the application failed to deploy.

> **Note:** It can take several minutes for the application to deploy completely. If the **Deployed** or **Failed** status is not displayed, the deployment process is not finished.

Pro: Disabling an Application Using the UI

About this task

To disable an application with the UI:

Procedure

1. From the sidebar, browse through the available applications from your configured repositories and select **Applications**
2. Select the three-dot button of the desired application card > **Uninstall**.
3. Follow the instruction on the confirmation pop-up message, and select **Uninstall Application**.

Platform Applications

When attaching a cluster, NKP deploys certain platform applications on the newly attached cluster. Operators can use the NKP UI to customize which platform applications to deploy to the attached clusters in a given workspace. For more information and to check the default and their current versions. see *Nutanix Kubernetes Platform Release Notes*

Default Foundational Applications

These applications provide the foundation for all platform application capabilities and deployments on Managed Clusters. These applications must be enabled for any Platform Applications to work properly. For current NKP release Helm Values and NKP versions, see the **Components and Application** values of the NKP Release Notes:

The foundational applications are comprised of the following Platform Applications:

- Cert Manager: Automates TLS certificate management and issuance. For more information, see https://cert-manager.io/docs/.
- Reloader: A controller that watches changes on ConfigMaps and Secrets, and automatically triggers updates on the dependent applications. For more information, see https://github.com/stakater/Reloader.
- traefik: Provides an HTTP reverse proxy and load balancer. Requires cert-manager and reloader. For more information, see https://traefik.io/.
- Chart Museum: An Open source Helm Chart (collection of files that describe a set of Kubernetes resources) repository. For more information, see https://chartmuseum.com/.
    - **Air-gapped environments only**: ChartMuseum is used on air-gapped installations to store the Helm Charts for air-gapped installations. In non-air-gapped installations, the charts are fetched from upstream repositories and Chartmuseum is not installed.

Common Platform Application Name	APP ID
Cert-Manager	cert-manager
Logging Operator	logging-operator

Common Platform Application Name	APP ID
Reloader	reloader
Traefik	traefik
Traefik ForwardAuth	traefik-forward-auth
ChartMuseum	chartmuseum

1. To see which applications are enabled or disabled in each category, verify the status.

   ```
   kubectl get apps,clusterapps,appdeployments -A
   ```

2. After deployment, the applications will be enabled.

   ```
   kubectl get helmreleases istio -n ${WORKSPACE_NAMESPACE} -w
   ```

   To check whether enabled or not, connect to the attached cluster and watch the `HelmReleases` to verify the deployment. In this example, we are checking if `istio` got deployed correctly:
3. You should eventually see the `HelmRelease` marked as `Ready`:

   ```
   NAMESPACE               NAME        READY   STATUS                              AGE
   workspace-test-vjsfq    istio       True    Release reconciliation succeeded    7m3s
   ```

Logging

Collects logs over time from Kubernetes and applications deployed on managed clusters. Also provides the ability to visualize and query the aggregated logs.

- Fluent-Bit: Open source and multi-platform log processor tool which aims to be a generic. For example, Swiss knife for logs processing and distribution. For more information, see https://docs.fluentbit.io/manual.
- Grafana: Log into the dashboard to view logs aggregated to Grafana Loki. For more information, see https://grafana.com/oss/grafana/.
- Logging operation: A horizontally-scalable, highly-available, multi-tenant log aggregation system inspired by Prometheus. For more information, see Logging operator.
- Rook Ceph: Automates the deployment and configuration of a Kubernetes logging pipeline. For more information, see https://rook.io/docs/rook/v1.10/Helm-Charts/operator-chart/.
- Rook Ceph Cluster: A Kubernetes-native high performance object store with an S3-compatible API that supports deploying into private and public cloud infrastructures. For more information, see https://rook.io/docs/rook/v1.10/Helm-Charts/ceph-cluster-chart/.

Note: Currently, the monitoring stack is deployed by default. The logging stack is not.

Common Platform Application Name	APP ID
Fluent Bit	fluent-bit
Grafana Logging	grafana-logging
Logging Operator	logging-operator
Grafana Loki (project)	project-grafana-loki
Rook Ceph	rook-ceph
Rook Ceph Cluster	rook-ceph-cluster

Monitoring

Provides monitoring capabilities by collecting metrics, including cost metrics for Kubernetes and applications deployed on managed clusters. Also provides visualization of metrics and evaluates rule expressions to trigger alerts when specific conditions are observed.

- Kubecost: provides real-time cost visibility and insights for teams using Kubernetes, helping you continuously reduce your cloud costs. For more information, see https://kubecost.com/
- kubernetes-dashboard: A general purpose, web-based UI for Kubernetes clusters. It allows users to manage applications running in the cluster, troubleshoot them and manage the cluster itself. For more information, see https://kubernetes.io/docs/tasks/access-application-cluster/web-ui-dashboard/
- kube-prometheus-stack: A stack of applications that collect metrics and provide visualization and alerting capabilities. For more information, see https://github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack

  > **Note:** Prometheus, Prometheus Alertmanager, and Grafana are included in the bundled installation. For more information, see https://prometheus.io/, https://prometheus.io/docs/alerting/latest/alertmanager and https://grafana.com/.

- nvidia-gpu-operator: The NVIDIA GPU Operator manages NVIDIA GPU resources in a Kubernetes cluster and automates tasks related to bootstrapping GPU nodes. For more information, see https://catalog.ngc.nvidia.com/orgs/nvidia/containers/gpu-operator.
- prometheus-adapter: Provides cluster metrics from Prometheus. For more information, see https://github.com/DirectXMan12/k8s-prometheus-adapter.

Common Platform Application Name	APP ID
Kubecost	kubecost
Kubernetes Dashboard	kubernetes-dashboard
Full Prometheus Stack	kube-prometheus-stack
Prometheus Adapter	prometheus-adapter
NVIDIA GPU Operator	nvidia-gpu-operator

Security

Allows management of security constraints and capabilities for the clusters and users.

- gatekeeper: A policy Controller for Kubernetes. For more information, see https://github.com/open-policy-agent/gatekeeper

Platform Application	APP ID
Gatekeeper	gatekeeper

Single Sign On (SSO)

Group of platform applications that allow enabling SSO on attached clusters. SSO is a centralized system for connecting attached clusters to the centralized authority on the management cluster.

- kube-oidc-proxy: A reverse proxy server that authenticates users using OIDC to Kubernetes API servers where OIDC authentication is not available. For more information, see https://github.com/jetstack/kube-oidc-proxy

- traefik-forward-auth: Installs a forward authentication application providing Google OAuth based authentication for Traefik. For more information, see https://github.com/thomseddon/traefik-forward-auth.

Platform Application	APP ID
Kube OIDC Proxy	kube-oidc-proxy
Traefik ForwardAuth	traefik-forward-auth

Backup

This platform application assists you with backing up and restoring your environment:

- velero: An open source tool for safely backing up and restoring resources in a Kubernetes cluster, performing disaster recovery, and migrating resources and persistent volumes to another Kubernetes cluster.For more information, see https://velero.io/.

Platform Application	APP ID
Velero	velero

Review the Workspace Platform Application Defaults and Resource Requirements on page 44 to ensure that the attached clusters have sufficient resources.

When deploying and upgrading applications, platform applications come as a bundle; they are tested as a single unit, and you must deploy or upgrade them in a single process, for each workspace. This means all clusters in a workspace have the same set and versions of platform applications deployed.

Deploying Platform Applications Using CLI

This topic describes how to use the CLI to enable an application to deploy to managed and attached clusters in a workspace.

Before you begin

Before you begin, you must have:

- A running cluster with Kommander installed.
- An existing Kubernetes cluster attached to Kommander (see Attaching an Existing Kubernetes Cluster on page 493).
- Determine the name of the workspace where you wish to perform the deployments. You can use the `nkp get workspaces` command to view the list of workspace names and their corresponding namespaces.
- Set the `WORKSPACE_NAMESPACE` environment variable to the name of the workspace's namespace where the cluster is attached:

  ```
  export WORKSPACE_NAMESPACE=<workspace_namespace>
  ```

- Set the `WORKSPACE_NAME` environment variable to the name of the workspace where the cluster is attached:

  ```
  export WORKSPACE_NAME=<workspace_name>
  ```

  Note: From the CLI, you can enable applications to deploy in the workspace. Verify that the application has successfully deployed through the CLI.

To create the AppDeployment, enable a supported application to deploy to your existing attached or managed cluster with an AppDeployment resource (see AppDeployment Resources on page 410).

Procedure

1. Obtain the **APP ID** and **Version** of the application from the "Components and Applications" section in the *Nutanix Kubenetes Platform Release Notes*.You must add them in the `<APP-ID>-<Version>` format, for example, `istio-1.17.2`.
2. Run the following command and define the `--app` flag to specify which platform application and version will be enabled.

   ```
   nkp create appdeployment istio --app istio-1.17.2 --workspace ${WORKSPACE_NAME}
   ```

   Note:

   - The `--app` flag must match the `APP NAME` from the list of available platform applications.
   - Observe that the `nkp create` command must be run with the `WORKSPACE_NAME` instead of the `WORKSPACE_NAMESPACE` flag.

   This instructs Kommander to create and deploy the `AppDeployment` to the `KommanderClusters` in the specified `WORKSPACE_NAME`.

Verifying the Deployed Platform Applications
You must verify the status of your platform applications after the platform applications are deployed.

Procedure

Connect to the attached cluster and watch the `HelmReleases` to verify the deployment. In this example, we are checking whether `istio` is deployed correctly.

```
kubectl get helmreleases istio -n ${WORKSPACE_NAMESPACE} -w
```

`HelmRelease` must be marked as `Ready`.

```
NAMESPACE               NAME        READY   STATUS                             AGE
workspace-test-vjsfq    istio       True    Release reconciliation succeeded   7m3s
```

Some supported applications have dependencies on other applications. For more information, see Platform Applications Dependencies on page 404.

Platform Applications Dependencies
Platform applications that are deployed to a workspace's attached clusters can depend on each other. It is important to note these dependencies when customizing the workspace platform applications to ensure that your applications are properly deployed to the clusters. .

For more information on how to customize workspace platform applications, see Platform Applications on page 400.

When deploying or troubleshooting platform applications, it helps to understand how platform applications interact and may require other platform applications as dependencies.

If a platform application's dependency does not successfully deploy, the platform application requiring that dependency does not successfully deploy.

The following table details information about the workspace platform application:

fluent-bit	-	-
grafana-logging	grafana-loki	-
grafana-loki	rook-ceph-cluster	-
logging-operator	-	-
rook-ceph	-	-

rook-ceph-cluster	rook-ceph	kube-prometheus-stack

Users can override the configuration to remove the dependency, as needed.

Foundational Applications

Provides the foundation for all platform application capabilities and deployments on managed clusters. These applications must be enabled for any platform applications to work properly.

The foundational applications are comprised of the following platform applications:

- cert-manager (see https://cert-manager.io/docs): Automates TLS certificate management and issuance.
- reloader (see https://github.com/stakater/Reloader): A controller that watches changes on ConfigMaps and Secrets, and automatically triggers updates on the dependent applications.
- traefik (see https://traefik.io/): Provides an HTTP reverse proxy and load balancer. Requires cert-manager and reloader.

Table 28: Foundational Applications

Platform Application	Required Dependencies
cert-manager	-
reloader	-
traefik	cert-manager, reloader

Logging

Logs are collected over a period of time from Kubernetes and applications are deployed on managed clusters. Also provides the ability to visualize and query the aggregated logs.

- fluent-bit: Open source and multi-platform log processor tool which aims to be a generic Swiss knife for logs processing and distribution. For more information, see https://docs.fluentbit.io/manual/.
- grafana-logging: Logging dashboard used to view logs aggregated to Grafana Loki. For more information, see https://grafana.com/oss/grafana/.
- grafana-loki: A horizontally-scalable, highly-available, multi-tenant log aggregation system inspired by Prometheus. For more information, see https://grafana.com/oss/loki/.
- logging-operator (see https://banzaicloud.com/docs/one-eye/logging-operator/): Automates the deployment and configuration of a Kubernetes logging pipeline. For more information, see
- rook-ceph: A Kubernetes-native high performance object store with an S3-compatible API that supports deploying into private and public cloud infrastructures. For more information, see https://rook.io/docs/rook/v1.10/Helm-Charts/operator-chart/) and rook-ceph-cluster (see https://rook.io/docs/rook/v1.10/Helm-Charts/ceph-cluster-chart/

Table 29: Logging

Platform Application	Required Dependencies	Optional Dependencies
fluent-bit	-	-
grafana-logging	grafana-loki	-

Platform Application	Required Dependencies	Optional Dependencies
grafana-loki	rook-ceph-cluster	-
logging-operator	-	-
rook-ceph	-	-
rook-ceph-cluster	rook-ceph	kube-prometheus-stack **Note:** Users can override the configuration to remove the dependency, as needed.

Monitoring

Provides monitoring capabilities by collecting metrics, including cost metrics, for Kubernetes and applications deployed on managed clusters. Also provides visualization of metrics and evaluates rule expressions to trigger alerts when specific conditions are observed.

- Kubecost: provides real-time cost visibility and insights for teams using Kubernetes, helping you continuously reduce your cloud costs. For more information, see .https://kubecost.com/
- kubernetes-dashboard: A general purpose, web-based UI for Kubernetes clusters. It allows users to manage applications running in the cluster, troubleshoot them and manage the cluster itself. For more information, see https://kubernetes.io/docs/tasks/access-application-cluster/web-ui-dashboard/.
- kube-prometheus-stack: A stack of applications that collect metrics and provide visualization and alerting capabilities. For more information, see https://github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack.

  Note: Prometheus, Prometheus Alertmanager, and Grafana are included in the bundled installation. For more information, see https://prometheus.io/, https://prometheus.io/docs/alerting/latest/alertmanager, and https://grafana.com/.
- nvidia-gpu-operator: The NVIDIA GPU Operator manages NVIDIA GPU resources in a Kubernetes cluster and automates tasks related to bootstrapping GPU nodes. For more information, see https://catalog.ngc.nvidia.com/orgs/nvidia/containers/gpu-operator.
- prometheus-adapter: Provides cluster metrics from Prometheus. For more information, see https://github.com/DirectXMan12/k8s-prometheus-adapter.

Table 30: Monitoring

Platform Application	Required Dependencies
kubecost	-
kubernetes-dashboard	-
kube-prometheus-stack	-
prometheus-adapter	kube-prometheus-stack
nvidia-gpu-operator	-

Security

Allows management of security constraints and capabilities for the clusters and users.

- gatekeeper: A policy Controller for Kubernetes.
- For more information, see https://github.com/open-policy-agent/gatekeeper.

Table 31: Security

Platform Application	Required Dependencies
gatekeeper	gatekeeper

Single Sign On (SSO)

Group of platform applications that allow enabling SSO on attached clusters. SSO is a centralized system for connecting attached clusters to the centralized authority on the management cluster.

- kube-oidc-proxy (see): A reverse proxy server that authenticates users using OIDC to Kubernetes API servers where OIDC authentication is not available. For more information, see https://github.com/jetstack/kube-oidc-proxy.
- traefik-forward-auth: Installs a forward authentication application providing Google OAuth based authentication for Traefik. For more information, see https://github.com/thomseddon/traefik-forward-auth.

Table 32: SSO

Platform Application	Required Dependencies
kube-oidc-proxy	cert-manager, traefik
traefik-forward-auth	traefik

Backup

This platform application assists you with backing up and restoring your environment:

- velero: An open source tool for safely backing up and restoring resources in a Kubernetes cluster, performing disaster recovery, and migrating resources and persistent volumes to another Kubernetes cluster.For more information, see https://velero.io/.

Table 33: Backup

Platform Application	APP ID
Velero	rook-ceph-cluster

Service Mesh

Allows deploying service mesh on clusters, enabling the management of microservices in cloud-native applications. Service mesh can provide a number of benefits, such as providing observability into communications, providing secure connections, or automating retries and backoff for failed requests.

- istio: Addresses the challenges developers and operators face with a distributed or microservices architecture. For more information, see https://istio.io/latest/about/service-mesh/.

- jaeger: A distributed tracing system used for monitoring and troubleshooting microservices-based distributed systems.For more information, see https://www.jaegertracing.io/.
- kiali: A management console for an Istio-based service mesh. It provides dashboards, observability, and lets you operate your mesh with robust configuration and validation capabilities. For more information, see https://kiali.io/.

Table 34: Service Mesh

Platform Application	Required Dependencies	Optional Dependencies
istio	kube-prometheus-stack	-
jaeger	istio	-
kiali	istio	jaeger (optional for monitoring purposes)
knative	istio	-

NKP AI Navigator Cluster Info Agent

Coupled with the AI Navigator, it analyses your cluster's data to include live information on queries made through the AI Navigator chatbot.

- `ai-navigator-info-api`: This is the collector of the application's API service, which performs all data abstraction data structuring services. This component is enabled by default and included in the AI Navigator.
- `ai-navigator-info-agent`: After manually enabling this platform application, the agent starts collecting pro or management cluster data and injecting it into the Cluster Info Agent database.

Table 35: NKP AI Navigator

Platform Application	Required Dependencies	Optional Dependencies
ai-navigator-info-agent	On the Management/Pro Cluster • ai-navigator-info-api (included in ai-navigator-app)	-

Workspace Platform Application Resource Requirements

See Workspace Platform Application Defaults and Resource Requirements on page 44 for a list of all platform applications, their default deployment configuration, required resources, and storage minimums.

Setting Priority Classes in NKP Applications

In NKP, your workloads can be prioritized to ensure that your critical components stay running in any situation. Priority Classes can be set for any application in NKP, including Platform Applications, Catalog applications, and even your own Custom Applications.

About this task

By default, the priority classes of Platform Applications are set by NKP.

For more information about the default priority classes for NKP applications, see the following pages:

- Workspace Platform Application Defaults and Resource Requirements on page 44
- Management Cluster Application Requirements on page 43
- Project Platform Application Configuration Requirements on page 445

This topic provides instructions on how to override the default priority class of any application in NKP to a different one.

NKP Priority Classes: The priority classes that are available in NKP are as follows:

Before you begin

Table 36: Priority Classes

Class	Value Name	Value	Description
NKP High	`nkp-high-priority`	100001000	This is the priority class that is used for high priority NKP workloads.
NKP Critical	`nkp-critical-priority`	100002000	This is the highest priority class that is used for critical priority NKP workloads.

1. Set the `WORKSPACE_NAMESPACE` environment variable to the name of the workspace's namespace where the cluster is attached: `xport WORKSPACE_NAMESPACE=<your_workspace_namespace>`.

   ```
   export WORKSPACE_NAMESPACE=<your_workspace_namespace>
   ```

2. You are now able to copy the following commands without having to replace the placeholder with your workspace namespace every time you run a command.

Follow these steps.

> **Note:** Keep in mind that the overrides for each application appears differently and is dependent on how the application's helm chart values are configured.
>
> For more information about the helm chart values used in the NKP, see "Component and Applications" section in the *Nutanix Kubernetes Platform Release Notes*.
>
> Generally speaking, performing a search for the `priorityClassName` field allows you to find out how you can set the priority class for a component.
>
> In the example below which uses the helm chart values in Grafana Loki, the referenced `priorityClassName` field is nested under the `ingester` component. The priority class can be set for several other components, including `distributor`, `ruler`, and on a `global` level.

Procedure

1. Create a `ConfigMap` with custom priority class configuration values for Grafana Loki.

   The following example sets the priority class of ingester component to the NKP critical priority class.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: v1
   kind: ConfigMap
   metadata:
     namespace: ${WORKSPACE_NAMESPACE}
     name: grafana-loki-overrides
   data:
   ```

```
  values.yaml: |
    ingester:
      priorityClassName: nkp-critical-priority
EOF
```

2. Edit the `grafana-loki` AppDeployment to set the value of `spec.configOverrides.name` to `grafana-loki-overrides`.

   After your editing is complete, the AppDeployment resembles this example.

```
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: grafana-loki
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    name: grafana-loki-0.69.16
    kind: ClusterApp
  configOverrides:
    name: grafana-loki-overrides
```

3. It will take a few minutes to reconcile but you can check the ingester pod's priority class after reconciling.

```
kubectl get pods -n ${WORKSPACE_NAMESPACE} -o custom-
columns=NAME:.metadata.name,PRIORITY:.spec.priorityClassName,PRIORITY:.spec.priority
 |grep ingester
```

   The results appears as follows::

```
NAME                                                              PRIORITY
       PRIORITY
grafana-loki-loki-distributed-ingester-0                          nkp-critical-
priority     100002000
```

AppDeployment Resources

Use AppDeployments to deploy and customize platform, NKP catalog, and custom applications.

An AppDeployment is a custom resource (see Custom Resource in https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/ created by NKP with the purpose of deploying applications (platform, NKP catalog and custom applications) in the management cluster, managed clusters, or both. Customers of both Pro and Ultimate products use `AppDeployments`, regardless of their setup (air-gapped, non-air-gapped, etc.), and their infrastructure provider.

When installing NKP, an `AppDeployment` resource is created for each enabled Platform Application. This `AppDeployment` resource references a `ClusterApp`, which then references the repository that contains a concrete declarative and preconfigured setup of an application, usually in the form of a `HelmRelease`. `ClusterApps` are cluster-scoped so that these platform applications are deployable to all workspaces or projects.

In the case of NKP catalog and custom applications, the `AppDeployment` references an `App` instead of a `ClusterApp`, which also references the repository containing the installation and deployment information. `Apps` are namespace-scoped and are meant to only be deployable to the workspace or project in which they have been created.

For example, this is the default `AppDeployment` for the Kube Prometheus Stack platform application:

```
apiVersion: apps.kommander.nutanix.io/v1alpha3
kind: AppDeployment
metadata:
  name: kube-prometheus-stack
  namespace: ${WORKSPACE_NAMESPACE}
```

```
spec:
  appRef:
    name: kube-prometheus-stack-46.8.0
    kind: ClusterApp
```

Workspaces

Allow teams or tenants to manage their own clusters using workspaces. Workspaces are a logical grouping of clusters that maintain a similar configuration, with certain configurations automatically federated to those clusters. Workspaces give you the flexibility to represent your organization in a way that makes sense for your teams or tenants. For example, you can create workspaces to separate clusters according to departments, products, or business functions.

The following procedures are supported for workspaces:

- Deploying Platform Applications Using CLI on page 403
- Platform Applications on page 400
- Platform Applications Dependencies on page 404
- Workspace Platform Application Defaults and Resource Requirements on page 44

Global or Workspace UI

The UI is designed to be accessible for different roles at different levels:

- *Global*: At the top level, IT administrators manage all clusters across all workspaces.
- *Workspace*: DevOps administrators manage multiple clusters within a workspace.
- *Projects*: DevOps administrators or developers manage configuration and services across multiple clusters.

Default Workspace

To get started immediately, you can use the default workspace deployed in NKP. However, take into account that you cannot move clusters from one workspace to another after creating/attaching them.

Creating a Workspace

In NKP, you can create your own workspaces.

About this task

To create a workspace:

Procedure

1. From the workspace selection dropdown list in the top menu bar, select **Create Workspace**.
2. Type a name and description.
3. Click **Save**.
   The workspace is now accessible from the workspace selection dropdown list.

Adding or Editing Workspace Annotations and Labels

When creating or editing a workspace, you can use the **Advanced Options** to add, edit, or delete annotations and labels to your workspace. Both the annotations and labels are applied to the workspace namespace.

About this task

To perform an action in workspace:

Procedure

1. From the top menu bar, select your target workspace.
2. Select the **Actions** from the dropdown list and click **Edit**.
3. Enter in new **Key** and **Value** labels for your workspace, or edit existing **Key** and **Value** labels.

> **Note:** Labels that are added to a workspace are also applied to the `kommanderclusters` object and as well as to all the clusters in the workspace.

Deleting a Workspace

In NKP, you can delete existing workspaces.

About this task

To delete a workspace:

> **Note:** Workspaces can only be deleted if all the clusters in the workspace have been deleted or detached.

Procedure

1. From the top menu bar, select **Global**.
2. From the sidebar menu, select **Workspaces**.
3. Select the three-dot button to the right of the workspace you want to delete, and then click **Delete**.
4. Confirm the workspace deletion in the **Delete Workspace** dialog box.

Workspace Applications

This topic describes the applications and application types that you can use with NKP.

Application types are either pre-packaged applications from the Nutanix Application Catalog or custom applications that you maintain for your teams or organization.

- Platform Applications on page 400 are applications integrated into NKP.
- Cluster-scoped Application Configuration from the NKP UI on page 412
- Cluster-scoped Application for Existing AppDeployments on page 414

Cluster-scoped Application Configuration from the NKP UI

When you enable an application for a workspace, you deploy this application to all clusters within that workspace. You can also choose to enable or customize an application on certain clusters within a workspace.

This functionality allows you to use NKP in a multi-cluster scenario without restricting the management of multiple clusters from a single workspace.

> **Note:** NKP Pro users are only be able to configure and deploy applications to a single cluster within a workspace. Selecting an application to deploy to a cluster skips cluster selection and takes you directly to the workspace configuration overrides page.

Enabling a Cluster-scoped Application

Before you begin

Ensure that you've provisioned or attached clusters in one of the following environments:

- Amazon Web Services (AWS): Creating a New AWS Air-gapped Cluster on page 829
- Amazon Elastic Kubernetes Service (EKS):Create an EKS Cluster from the UI on page 869
- Microsoft Azure:Creating a Managed Azure Cluster Through the NKP UI on page 483

For more information, see the current list of Catalog and Platform Applications:

- Workplace Catalog Applications on page 420
- Platform Applications on page 400

Navigate to the workspace containing the clusters you want to deploy to by selecting the appropriate workspace name from the dropdown list at the top of the NKP dashboard.

Procedure

1. From the left navigation pane, find the application you want to deploy to the cluster, and select **Applications**.
2. Select the three-dot menu in the desired application's tile and select **Enable**.

   > **Note:** You can also access the **Application Enablement** by selecting the three-dot menu > **View** > **Details**. Then, select **Enable** from the application's details page.

   The **Application Enablement** page appears.
3. Select the cluster(s) that you want to deploy the application to.
   The available clusters are sorted by **Name**, **Type**, **Provider** and any **Labels** that you added.
4. In the top-right corner of the **Application Enablement** page, deploy the application to the clusters by selecting **Enable**.
   You are automatically redirected to either the **Applications** or **View Details** page.

   To view the application enabled in your chosen cluster, navigate to the **Clusters** page on the left navigation bar. The application appears in the **Applications** pane of the appropriate cluster.

   > **Note:** Once you enable an application at the workspace level, NKP automatically enables that app on any other cluster you create or attach.

Configuring a Cluster-scoped Application

About this task

For scenarios where applications require different configurations on a per-cluster basis, navigate to the **Applications** page and select **Edit** from the three-dot menu of the appropriate application to return to the application enablement page.

Procedure

1. Select the cluster(s) that you want to deploy the application to.
   The available clusters can be sorted by **Name**, **Type**, **Provider** and any **Labels** you've added.
2. Select the **Configuration** tab.

3. The **Configuration** tab contains two separate types of code editors, where you can enter your specified overrides and configurations.

   » **Workspace Application Configuration**: A workspace-level code editor that applies all configurations and overrides to the entirety of the workspace and its clusters for this application.

   » **Cluster Application Configuration Override**: A cluster-scoped code editor that applies configurations and overrides to the cluster specified. These customizations will merge with the workspace application configuration. If there is no cluster-scoped configuration, the workspace configuration applies.

4. If you already have a configuration to apply in a text or `.yaml` file, you can upload the file by selecting **Upload File.** If you want to download the displayed set of configurations, select **Download File**.

5. Finish configuring the cluster-scoped applications by selecting **Save** in the top right corner of the **Application Enablement** page.
   You are automatically redirected to either the **Applications** or **View Details** page. To view the custom configurations of the application in the cluster, select the **Configurations** tab on the details page of the application.

   Note:

   Editing is disabled in the code boxes displayed in the application's details page. To edit the configuration, click **Edit** in the top right of the page and repeat the steps in this section.

Removing a Cluster-scoped Application

About this task

Navigate to the cluster you've deployed your applications to by selecting **Clusters** from the left navigation bar.

Procedure

1. Click on the **Applications** tab.

2. Select the three-dot menu in the application tile that you want and select **Uninstall**.
   A prompt appears to confirm your decision to uninstall the application.

3. Follow the instructions in the prompt and select **Uninstall**

4. Refresh the page to confirm that the application has been removed from the cluster.
   This process only removes the application from the specific cluster you've navigated to. To remove this application from other clusters, navigate to the **Clusters** page and repeat the process.

Cluster-scoped Application for Existing AppDeployments

This topic describes how to enable cluster-scoped configuration of applications for existing AppDeployments.

When you enable an application for a workspace, you deploy this application to all clusters within that workspace. You can also choose to enable or customize an application on certain clusters within a workspace. This functionality allows you to use NKP in a multi-cluster scenario without restricting the management of multiple clusters from a single workspace.

Your NKP cluster comes bundled with a set of default application configurations. If you want to override the default configuration of your applications, you can define workspace `configOverrides` on top of the default workspace configuration. And if you want to further customize your workplace by enabling

applications on a per-cluster basis or by defining per-cluster customizations, you can create and apply `clusterConfigOverrides`.

The cluster-scoped enablement and customization of applications is an *Ultimate-only* feature, which allows the configuration of all workspace Platform Applications on page 400, Workplace Catalog Applications on page 420, and Custom Applications on page 428 through the CLI in your managed and attached clusters regardless of your environment setup (air-gapped or non-air-gapped). This capability is not provided for project applications.

Enabling an Application Per Cluster

Before you begin

- Any application you wish to enable or customize at a cluster level, first needs to be enabled at the workspace-level through an `AppDeployment`. See Deploying Platform Applications Using CLI on page 403 and Workplace Catalog Applications on page 420.
- For custom configurations, you must created a ConfigMap. For all the required `spec` fields for each customization you want to add to an application in a cluster, see AppDeployment Resources on page 410.

  You can apply a `ConfigMap` to several clusters, or create a `ConfigMap` for each cluster, but the `ConfigMap` object must exist in the Management cluster.
- Determine the name of the workspace where you wish to perform the deployments. You can use the `nkp get workspaces` command to see the list of workspace names and their corresponding namespaces.
- Set the `WORKSPACE_NAMESPACE` environment variable to the name of the workspace's namespace where the cluster is attached.

  ```
  export WORKSPACE_NAMESPACE=<workspace_namespace>
  ```

- Set the `WORKSPACE_NAME` environment variable to the name of the workspace where the cluster is attached.

  ```
  export WORKSPACE_NAME=<workspace_name>
  ```

When you enable an application on a workspace, it is deployed to all clusters in the workspace by default. If you want to deploy it only to a subset of clusters when enabling it on a workspace for the first time, you can follow the steps:

To enable an application per cluster for the first time:

Procedure

1. Create an `AppDeployment` for your application, selecting a subset of clusters within the workspace to enable it on. You can use the `nkp get clusters --workspace ${WORKSPACE_NAME}` command to see the list of clusters in the workspace.

   The following snippet is an example. Replace the application name, version, workspace name and cluster names according to your requirements. For compatible versions, see the "Components and Applications" section in the *Nutanix Kubernetes Platforms Release Notes*.

   ```
   nkp create appdeployment kube-prometheus-stack --app kube-prometheus-stack-46.8.0 --
   workspace ${WORKSPACE_NAME} --clusters attached-cluster1,attached-cluster2
   ```

2. (Optional) Check the current status of the `AppDeployment` to see the names of the clusters where the application is currently enabled.

Enabling or Disabling an Application Per Cluster

You can enable or disable an application per cluster after it has been enabled at the workspace level.

About this task

You can enable or disable applications at any time. After you have enabled the application at the workspace level, the `spec.clusterSelector` field populates.

> **Note:** For clusters that are newly attached into the workspace, all applications enabled for the workspace are automatically enabled on and deployed to the new clusters.

If you want to see on what clusters your application is currently deployed, see the print and review the current state of your AppDeployment. For more information, see AppDeployment Resources on page 410.

Procedure

Edit the `AppDeployment` YAML by adding or removing the names of the clusters where you want to enable your application in the `clusterSelector` section:

The following snippet is an example. Replace the application name, version, workspace name and cluster names according to your requirements.

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: kube-prometheus-stack
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    name: kube-prometheus-stack-46.8.0
    kind: ClusterApp
  clusterSelector:
    matchExpressions:
    - key: kommander.d2iq.io/cluster-name
      operator: In
      values:
      - attached-cluster1
      - attached-cluster3-new
EOF
```

Customizing an Application Per Cluster

You can customize the application for each cluster occurrence of said application. If you want to customize the application for a cluster that is **not yet attached**, refer to the instructions below, so the application is deployed with the custom configuration during attachment.

About this task

To enable per-cluster customizations:

Procedure

1. Reference the name of the `ConfigMap` to be applied per cluster in the `spec.clusterConfigOverrides` fields. In this example, you have three different customizations specified in three different `ConfigMaps` for three different clusters in one workspace.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: apps.kommander.d2iq.io/v1alpha3
   kind: AppDeployment
   ```

```
metadata:
  name: kube-prometheus-stack
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    name: kube-prometheus-stack-46.8.1
    kind: ClusterApp
  clusterSelector:
    matchExpressions:
    - key: kommander.d2iq.io/cluster-name
      operator: In
      values:
      - attached-cluster1
      - attached-cluster2
      - attached-cluster3-new
  clusterConfigOverrides:
    - configMapName: kps-cluster1-overrides
      clusterSelector:
        matchExpressions:
        - key: kommander.d2iq.io/cluster-name
          operator: In
          values:
          - attached-cluster1
    - configMapName: kps-cluster2-overrides
      clusterSelector:
        matchExpressions:
        - key: kommander.d2iq.io/cluster-name
          operator: In
          values:
          - attached-cluster2
    - configMapName: kps-cluster3-overrides
      clusterSelector:
        matchExpressions:
        - key: kommander.d2iq.io/cluster-name
          operator: In
          values:
          - attached-cluster3-new
EOF
```

2. If you have not done so yet, create the ConfigMaps referenced in each `clusterConfigOverrides` entry.

Note:

- The changes are applied only if the YAML file has a valid syntax.
- Set up only one cluster override `ConfigMap` per cluster. If there are several `ConfigMaps` configured for a cluster, only one will be applied.
- Cluster override `ConfigMaps` must be created on the Management cluster.

Customizing an Application Per Cluster at Attachment

You can customize the application configuration for a cluster prior to its attachment, so that the application is deployed with this custom configuration on attachment. This is preferable, if you do not want to redeploy the application with an updated configuration after it has been initially installed, which may cause downtime.

About this task

To enable per-cluster customizations, follow these steps before attaching the cluster

Procedure

1. Set the `CLUSTER_NAME` environment variable to the cluster name that you will give your to-be-attached cluster.

   ```
   export CLUSTER_NAME=<your_attached_cluster_name>
   ```

   Reference the name of the `ConfigMap` you want to apply to this cluster in the

   `spec.clusterConfigOverrides` fields. **You do not need to update the** `spec.clusterSelector` **field**. In this example, you have the `kps-cluster1-overrides` customization specified for `attached-cluster-1` and a different customization (in `kps-your-attached-cluster-overrides` `ConfigMap`) for your to-be-attached cluster.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: apps.kommander.d2iq.io/v1alpha3
   kind: AppDeployment
   metadata:
     name: kube-prometheus-stack
     namespace: ${WORKSPACE_NAMESPACE}
   spec:
     appRef:
       name: kube-prometheus-stack-46.8.1
       kind: ClusterApp
     clusterSelector:
       matchExpressions:
       - key: kommander.d2iq.io/cluster-name
         operator: In
         values:
         - attached-cluster1
     clusterConfigOverrides:
       - configMapName: kps-cluster1-overrides
         clusterSelector:
           matchExpressions:
           - key: kommander.d2iq.io/cluster-name
             operator: In
             values:
             - attached-cluster1
       - configMapName: kps-your-attached-cluster-overrides
         clusterSelector:
           matchExpressions:
           - key: kommander.d2iq.io/cluster-name
             operator: In
             values:
             - ${CLUSTER_NAME}
   EOF
   ```

2. If you have not done so yet, create the ConfigMap referenced for your to-be-attached cluster.

   Note:

   - The changes are applied only if the YAML file has a valid syntax.
   - Cluster override `ConfigMaps` must be created on the Management cluster.

Verify the Configuration of your Application

Procedure

1. To verify whether the applications connect to the managed or attached cluster and check the status of the deployments, see Workplace Catalog Applications on page 420.
2. If you want to know how the `AppDeployment` resource is currently configured, refer to the print and review the state of your AppDeployments.

Disabling the Custom Configuration of an Application Per Cluster

Enabled customizations are defined in a `ConfigMap` which, in turn, is referenced in the `spec.clusterConfigOverrides` object of your `AppDeployment`.

Procedure

1. Review your current configuration to establish what you want to remove.

   ```
   kubectl get appdeployment -n ${WORKSPACE_NAMESPACE} kube-prometheus-stack -o yaml
   ```

   The result appears as follows.

   ```
   apiVersion: apps.kommander.d2iq.io/v1alpha3
   kind: AppDeployment
   metadata:
     name: kube-prometheus-stack
     namespace: ${WORKSPACE_NAMESPACE}
   spec:
     appRef:
       name: kube-prometheus-stack-46.8.0
       kind: ClusterApp
     configOverrides:
       name: kube-prometheus-stack-overrides-attached
     clusterSelector:
       matchExpressions:
       - key: kommander.d2iq.io/cluster-name
         operator: In
         values:
         - attached-cluster1
         - attached-cluster2
     clusterConfigOverrides:
       - configMapName: kps-cluster1-overrides
         clusterSelector:
           matchExpressions:
           - key: kommander.d2iq.io/cluster-name
             operator: In
             values:
             - attached-cluster1
       - configMapName: kps-cluster2-overrides
         clusterSelector:
           matchExpressions:
           - key: kommander.d2iq.io/cluster-name
             operator: In
             values:
             - attached-cluster2
   ```

   Here you can see that `kube-prometheus-stack` has been enabled for the `attached-cluster1` and `attached-cluster2`. There is also a custom configuration for each of the clusters: `kps-cluster1-overrides` and `kps-cluster2-overrides`.

2. To delete the customization, delete the `configMapName` entry of the cluster. This is located under `clusterConfigOverrides`.

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: kube-prometheus-stack
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    kind: ClusterApp
    name: kube-prometheus-stack-46.8.0
  configOverrides:
    name: kube-prometheus-stack-ws-overrides
  clusterSelector:
    matchExpressions:
      - key: kommander.d2iq.io/cluster-name
        operator: In
        values:
        - attached-cluster1
  clusterConfigOverrides:
    - configMapName: kps-cluster1-overrides
      clusterSelector:
        matchExpressions:
        - key: kommander.d2iq.io/cluster-name
          operator: In
          values:
          - attached-cluster1
EOF
```

> **Note:** Compare steps one and two for a reference of how an entry should be deleted.

3. Before deleting a `ConfigMap` that contains your customization, ensure you will NOT require it at a later time. It is not possible to restore a deleted `ConfigMap`. If you choose to delete it, run.

```
kubectl delete configmap <name_configmap> -n ${WORKSPACE_NAMESPACE}
```

> **Note:** It is not possible to delete a `ConfigMap` that is being actively used and referenced in the `configOverride` of any `AppDeployment`.

Workplace Catalog Applications

Catalog applications are any third-party or open source applications that appear in the Catalog. These applications are deployed to be used for customer workloads. Nutanix provides Workplace Catalog Applications for use in your environment.

Installing the NKP Catalog Application Using the CLI

Catalog applications are applications provided by Nutanix for use in your environment.

Before you begin

- Ensure your clusters run on a supported Kubernetes version and that this Kubernetes version is also compatible with your catalog application version.
- For customers with an NKP Ultimate License on page 29 and a multi-cluster environment, Nutanix recommends keeping all clusters on the same Kubernetes version. This ensures your NKP catalog application can run on all clusters in a given workspace.

- Ensure that your NKP Catalog application is compatible with:
  - The Kubernetes version in all the Managed and Attached clusters of the workspace where you want to install the catalog application.
  - The range of Kubernetes versions supported in this release of NKP.
- If your current Catalog application version is not compatible, upgrade the application to a compatible version.

> **Note:** With the latest NKP version, only the following versions of Catalog applications are supported. All the previous versions and any other applications previously included in the Catalog are now deprecated.

Table 37: Supported Catalog Applications

Name	App ID	Compatible Kubernetes Versions	Application Version
kafka-operator-0.25.1	kafka-operator	1.21-1.27	0.25.1
zookeeper-operator-0.2.16-nkp.1	zookeeper-operator	1.26-1.27	0.2.15

About this task

Follow these steps to install the NKP catalog from the CLI.

Procedure

1. If you are running in air-gapped environment, install Kommander in an Air-gapped environment. For more information, see Installing Kommander in an Air-gapped Environment on page 1017.
2. Set the `WORKSPACE_NAMESPACE` environment variable to the name of your workspace's namespace.

```
export WORKSPACE_NAMESPACE=<workspace namespace>
```

3. Create the `GitRepository`.

```
kubectl apply -f - <<EOF
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: nkp-catalog-applications
  namespace: ${WORKSPACE_NAMESPACE}
  labels:
    kommander.d2iq.io/gitapps-gitrepository-type: nkp
    kommander.d2iq.io/gitrepository-type: catalog
spec:
  interval: 1m0s
  ref:
    tag: v2.12.1
  timeout: 20s
  url:https://github.com/mesosphere/dkp-catalog-applications
EOF
```

4. Verify that you can see the NKP workspace catalog `Apps` available in the UI (in the Applications section in said workspace), and in the CLI, using `kubectl`.

   ```
   kubectl get apps -n ${WORKSPACE_NAMESPACE}
   ```

Kafka Operator in a Workspace

Apache Kafka is an open-source distributed event streaming platform used for high-performance data pipelines, streaming analytics, data integration, and mission-critical applications. The Kafka Operator is a Kubernetes operator to automate provisioning, management, autoscaling, and operations of Apache Kafka clusters deployed to Kubernetes. It works by watching custom resources, such as `KafkaClusters`, `KafkaUsers`, and `KafkaTopics`, to provision underlying Kubernetes resources (that is `StatefulSets`) required for a production-ready Kafka Cluster.

Usage of Custom Image for a Zookeeper Cluster

> **Warning:** If you use a custom version of KafkaCluster with `cruise.control`, ensure you use the custom resource image version `2.5.123` in the `.cruiseControlConfig.image` field for both air-gapped and non-air-gapped environments.

To avoid the critical CVEs associated with the official kafka image in version `v0.25.1`, a custom image must be specified when creating a zookeeper cluster.

Specify the following custom values in `KafkaCluster CRD`:

- `.spec.clusterImage` to `ghcr.io/banzaicloud/kafka:2.13-3.4.1`
- `.spec.cruiseControlConfig.initContainers[*].image` to `ghcr.io/banzaicloud/cruise-control:2.5.123`

Installing Kafka Operator in a Workspace

This topic describes the Kafka operator running in a workspace namespace, and how to create and manage Kafka clusters in any project namespaces.

About this task

Follow these steps to install the Kafka operator in a workspace.

> **Note:** Only install the Kafka operator once per workspace.

For more information, see Deploying Kafka in a Project on page 448.

Procedure

1. Follow the generic installation instructions for workspace catalog applications on the **Application Deployment** page.
2. Within the `AppDeployment`, update the `appRef` to specify the correct `kafka-operator` App. You can find the `appRef.name` by listing the available `Apps` in the workspace namespace.

   ```
   kubectl get apps -n ${WORKSPACE_NAMESPACE}
   ```

   For details on custom configuration for the operator, see Kafka operator Helm Chart documentation at https://github.com/banzaicloud/koperator/tree/master/charts/kafka-operator#configuration.

Uninstalling Kafka Operator Using the CLI

Uninstalling the Kafka operator does not affect existing `KafkaCluster` deployments. After uninstalling the operator, you must manually remove any remaining Custom Resource Definitions (CRDs) from the operator.

Procedure

1. Delete all of the deployed Kafka custom resources.
   For more information, see Deleting Kafka in a Project on page 449.
2. Uninstall a Kafka operator `AppDeployment`.

```
kubectl -n <workspace namespace> delete AppDeployment <name of AppDeployment>
```

3. Remove Kafka CRDs.

> **Note:** The CRDs are not finalized for deletion until you delete the associated custom resources.

```
kubectl delete crds kafkaclusters.kafka.banzaicloud.io
 kafkausers.kafka.banzaicloud.io kafkatopics.kafka.banzaicloud.io
```

Zookeeper Operator in Workspace

ZooKeeper is a centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services. The ZooKeeper operator is a Kubernetes operator that handles the provisioning and management of ZooKeeper clusters. It works by watching custom resources, such as `ZookeeperClusters`, to provision the underlying Kubernetes resources (`StatefulSets`) required for a production-ready ZooKeeper Cluster

Usage of Custom Image for a Zookeeper Cluster

To avoid the critical CVEs associated with the official zookeeper image in version `v0.2.15`, a custom image must be specified when creating a zookeeper cluster.

```
apiVersion: "zookeeper.pravega.io/v1beta1"
kind: "ZookeeperCluster"
# ...
spec:
  image:
    repository: ghcr.io/mesosphere/zookeeper
    tag: v0.2.15-d2iq
```

For more information about custom images go to Ultimate: Upgrade Project Catalog Applications on page 1162.

Installing Zookeeper Operator in a Workspace

This topic describes the ZooKeeper operator running in a workspace namespace, and how to create and manage ZooKeeper clusters in any project namespaces.

About this task

Follow these steps to install the Zookeeper operator in a workspace.

> **Note:** Only install the Zookeeper operator once per workspace.

For more information, see Deploying ZooKeeper in a Project on page 447.

Procedure

1. Follow the generic installation instructions for workspace catalog applications in **Application Deployment** page.
2. Within the `AppDeployment`, update the `appRef` to specify the correct `zookeeper-operator` App. You can find the `appRef.name` by listing the available `Apps` in the workspace namespace.

   ```
   kubectl get apps -n ${WORKSPACE_NAMESPACE}
   ```

   For details on custom configuration for the operator, see ZooKeeper operator Helm Chart documentation at https://github.com/pravega/zookeeper-operator/tree/master/charts/zookeeper-operator#configuration.

Uninstalling Zookeeper Operator Using the CLI

Uninstalling the ZooKeeper operator will not directly affect any running `ZookeeperClusters`. By default, the operator waits for any `ZookeeperClusters` to be deleted before it will fully uninstall (you can set `hooks.delete: true` in the application configuration to disable this behavior). After uninstalling the operator, you need to manually clean up any leftover Custom Resource Definitions (CRDs).

Procedure

1. Delete all `ZookeeperClusters`.

   For more information, see Deleting Zookeeper in a Project on page 448.
2. Uninstall a ZooKeeper operator `AppDeployment`.

   ```
   kubectl -n <workspace namespace> delete AppDeployment <name of AppDeployment>
   ```
3. Remove Zookeeper CRDs.

   > **Warning:** After you remove the CRDs, all deployed ZookeeperClusters will be deleted!

   ```
   kubectl delete crds zookeeperclusters.zookeeper.pravega.io
   ```

Deployment of Catalog Applications in Workspaces

Deploy applications to attached clusters using the CLI. This topic describes how to use the CLI to deploy a workspace catalog application to attached clusters within a workspace.

To deploy an application to selected clusters within a workspace, see Cluster-scoped Application for Existing AppDeployments on page 414.

Enabling the Catalog Application Using the UI

Before you begin

Before you begin, you must have:

- A running cluster with Kommander installed. The cluster must be on a supported Kubernetes version for this release of NKP and also compatible with the catalog application version you want to install.
- Attach an Existing Kubernetes Cluster section of the documentation completed. For more information, see Attaching an Existing Kubernetes Cluster on page 493.

- Set the `WORKSPACE_NAMESPACE` environment variable to the name of the workspace's namespace the attached cluster exists in.

  ```
  export WORKSPACE_NAMESPACE=<workspace_namespace>
  ```

- A Git Repository. After creating a GitRepository, use either the NKP UI or the CLI to enable your catalog applications. For more information, see Creating a Public Git Repository on page 430.

  > **Note:** From within a workspace, you can enable applications to deploy. Verify that an application has successfully deployed through the CLI.

About this task

Follow these steps to enable your catalog applications from the NKP UI:

Procedure

1. **Ultimate only**: From the top menu bar, select your target workspace.
2. From the sidebar menu to browse the available applications from your configured repositories and select **Applications**.
3. Select the three dot button on the required application tile and select **Enable**.
4. If available, select a version from the dropdown list.

   This dropdown list will only be visible if there is more than one version.
5. (Optional) If you want to override the default configuration values, copy your customized values into the text editor under **Configure Service** or upload your YAML file that contains the values.

   ```
   someField: someValue
   ```

6. Confirm the details are correct, and then click **Enable**.

   For all applications, you must provide a display name and an ID which is automatically generated based on what you enter for the display name, unless or until you edit the ID directly. The ID must be compliant with Kubernetes DNS subdomain name validation rules in the Kubernetes documentation.

   Alternately, you can use the CLI to enable your catalog applications.

Enabling the Catalog Application Using the CLI

See Workspace Catalog Applications for the list of available applications that you can deploy on the attached cluster.

Before you begin

Procedure

1. Enable a supported application to deploy to your attached Kubernetes cluster with an `AppDeployment` resource.

   For more information, see Attaching an Existing Kubernetes Cluster on page 493.
2. Within the `AppDeployment`, define the `appRef` to specify which `App` to enable.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: apps.kommander.d2iq.io/v1alpha3
   kind: AppDeployment
   metadata:
   ```

```
  name: kafka-operator
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    name: kafka-operator-0.25.1
    kind: App
EOF
```

Note:

- The `appRef.name` must match the app `name` from the list of available catalog applications.
- Create the resource in the workspace you just created, which instructs Kommander to deploy the `AppDeployment` to the `KommanderClusters` in the same workspace.

Enabling the Catalog Application With a Custom Configuration Using the CLI

About this task

To enable the catalog application:

Procedure

1. Provide the name of a `ConfigMap` in the `AppDeployment`, which provides custom configuration on top of the default configuration.

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: kafka-operator
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    name: kafka-operator-0.25.1
    kind: App
  configOverrides:
    name: kafka-operator-overrides
EOF
```

2. Create the `ConfigMap` with the name provided in the step above, with the custom configuration.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: ${WORKSPACE_NAMESPACE}
  name: kafka-operator-overrides
data:
  values.yaml: |
    operator:
      verboseLogging: true
EOF
```

   Kommander waits for the `ConfigMap` to be present before deploying the `AppDeployment` to the managed or attached clusters.

Verify the Catalog Applications

The applications are now enabled.

Procedure

Connect to the attached cluster and check the `HelmReleases` to verify the deployment.

```
kubectl get helmreleases -n ${WORKSPACE_NAMESPACE}
```

The result appears as follows.

```
NAMESPACE               NAME               READY   STATUS
 AGE
workspace-test-vjsfq    kafka-operator     True    Release reconciliation succeeded
 7m3s
```

Workspace Catalog Application Upgrade

Upgrade catalog applications using the CLI or UI.

Before upgrading, keep in mind the distinction between Platform applications and Catalog applications. Platform applications are deployed and upgraded as a set for each cluster or workspace. Catalog applications are deployed separately, so that you can deploy and upgrade them individually for each workspace or project.

Upgrading the Catalog Applications Using the UI

Before you begin

Complete the upgrade prerequisites tasks. For more information, see Upgrade Prerequisites on page 1147.

About this task

To upgrade an application from the NKP UI:

Procedure

1. From the top menu bar, select your target workspace.
2. From the sidebar menu, select **Applications**.
3. Select the three dot button on the required application tile, and then select **Edit**.
4. Select the **Version** from the dropdown list and select a new version.
   This dropdown list is only available if there is a newer version to upgrade to.
5. Click **Save**.

Upgrading the Catalog Applications Using the CLI

Before you begin

> **Note:** The commands use the workspace *name* and not *namespace*.
>
> You can retrieve the workspace name by running the following command.
>
> ```
> nkp get workspaces
> ```
>
> To view a list of the deployed `apps` to your workspace, run the following command.
>
> ```
> nkp get appdeployments --workspace=<workspace-name>
> ```

Complete the upgrade prerequisites tasks. For more information, see Upgrade Prerequisites on page 1147.

About this task

To upgrade an application from the CLI:

Procedure

1. To see what app(s) and app versions are available to upgrade, run the following command.

   > **Note:** You can reference the app version by going into the app name (e.g. <APP ID>-<APP VERSION>.

   ```
   kubectl get apps -n ${WORKSPACE_NAMESPACE}
   ```

   You can also use this command to display the apps and app versions, for example.

   ```
   kubectl get apps -n ${WORKSPACE_NAMESPACE} -o jsonpath='{range .items[*]}
   {@.spec.appId}{"----"}{@.spec.version}{"\n"}{end}'
   ```

   This is an example of an output that displays the different application and application versions.

   ```
   kafka-operator----0.20.0
   kafka-operator----0.20.2
   kafka-operator----0.23.0-dev.0
   kafka-operator----0.25.1
   zookeeper-operator----0.2.13
   zookeeper-operator----0.2.14
   zookeeper-operator----0.2.15
   ```

2. Run the following command to upgrade an application from the NKP CLI.

   ```
   nkp upgrade catalogapp <appdeployment-name> --workspace=<my-workspace-name> --to-
   version=<version.number>
   ```

   The following command upgrades the Kafka Operator application, named `kafka-operator-abc` in a workspace to version `0.25.1`.

   ```
   nkp upgrade catalogapp kafka-operator-abc --workspace=my-workspace --to-
   version=0.25.1
   ```

   > **Note:** Platform applications cannot be upgraded on a one-off basis, and must be upgraded in a single process for each workspace. If you attempt to upgrade a platform application with these commands, you receive an error and the application is not upgraded.

Custom Applications
Custom applications are third-party applications you have added to the NKP Catalog.

Custom applications are any third-party applications that are not provided in the NKP Application Catalog. Custom applications can leverage applications from the NKP Catalog or be fully-customized. There is no expectation of support by Nutanix for a Custom application. Custom applications can be deployed on Konvoy clusters or on any Nutanix supported 3rd party Kubernetes distribution.

Git Repository Structure

Git repositories must be structured in a specific manner for defined applications to be processed by Kommander.

You must structure your git repository based on the following guidelines, for your applications to be processed properly by Kommander so that they can be deployed.

Git Repository Directory Structure

Use the following basic directory structure for your git repository.

```
### helm-repositories
#   ### <helm repository 1>
#   #   ### kustomization.yaml
#   #   ### <helm repository name>.yaml
#   ### <helm repository 2>
#       ### kustomization.yaml
#       ### <helm repository name>.yaml
### services
    ### <app name>
    #   ### <app version1> # semantic version of the app helm chart. e.g., 1.2.3
    #   #   ### defaults
    #   #   #   ### cm.yaml
    #   #   #   ### kustomization.yaml
    #   #   ### <app name>.yaml
    #   #   ### kustomization.yaml
    #   ### <app version2> # another semantic version of the app helm chart. e.g.,
 2.3.4
    #   #   ### defaults
    #   #   #   ### cm.yaml
    #   #   #   ### kustomization.yaml
    #   #   ### <app name>.yaml
    #   #   ### kustomization.yaml
    #   ### metadata.yaml
    ### <another app name>
    ...
```

Remember the following guidelines:

- Define applications in the `services/` directory.
- You can define multiple versions of an application, under different directories nested under the `services/<app name>/` directory.
- Define application manifests.

  For more information, see HelmRelease in the Flux documentation. Under each versioned directory `services/<app name>/<version>/`in the `<app name>.yaml` file that which is listed in the `kustomization.yaml` file.

  Kubernetes Kustomization file. For more information, see the Kubernetes Kustomization docs. For more information, see the The Kustomization File in the SIG CLI documentation.
- Define the default values ConfigMap for `HelmReleases` in the `services/<app name>/<version>/defaults` directory, accompanied by a `kustomization.yaml` Kubernetes Kustomization file pointing to the `ConfigMap` file.
- Define the `metadata.yaml` of each application under the `services/<app name>/` directory. For more information, see Workspace Application Metadata on page 432

For an example of how to structure custom catalog Git repositories, see https://github.com/mesosphere/dkp-catalog-applications.

Helm Repositories

You must include the `HelmRepository` that is referenced in each `HelmRelease`'s `Chart` spec.

Each `services/<app name>/<version>/kustomization.yaml` must include the path of the YAML file that defines the `HelmRepository`. For example.

```
# services/<app name>/<version>/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
```

```
kind: Kustomization
resources:
  - <app name>.yaml
  - ../../../helm-repositories/<helm repository 1>
```

For more information, see the flux documentation about Helm Repositories in the Flux documentation.

Substitution Variables

Some substitution variables are provided. For more information, see Kustomization in the Flux documentation.

- `${releaseName}`: For each App deployment, this variable is set to the `AppDeployment` name. Use this variable to prefix the names of any resources that are defined in the application directory in the Git repository so that multiple instances of the same application can be deployed. If you create resources without using the `releaseName` prefix (or suffix) in the name field, there can be conflicts if the same named resource is created in that same namespace.
- `${releaseNamespace}`: The namespace of the Workspace.
- `${workspaceNamespace}`: The namespace of the Workspace that the Workspace belongs to.

Creating a Public Git Repository

Use the CLI to create the GitRepository resource and add a new repository to your Workspace.

About this task

Create a public Git Repository in the Workspace namespace. For information on creating a catalog in a private repository, see Create a Private Git Repository on page 431.

Procedure

1. If you are running in an air-gapped environment, complete the steps in Installing Kommander in an Air-gapped Environment on page 1017.
2. Set the `WORKSPACE_NAMESPACE` environment variable to the name of your workspace's namespace.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

3. Adapt the URL of your Git repository.

   ```
   kubectl apply -f - <<EOF
   apiVersion: source.toolkit.fluxcd.io/v1
   kind: GitRepository
   metadata:
     name: example-repo
     namespace: ${WORKSPACE_NAMESPACE}
     labels:
       kommander.d2iq.io/gitapps-gitrepository-type: catalog
       kommander.d2iq.io/gitrepository-type: catalog
   spec:
     interval: 1m0s
     ref:
       branch: <your-target-branch-name> # e.g., main
     timeout: 20s
     url: https://github.com/<example-org>/<example-repo>
   EOF
   ```

4. Ensure the status of the `GitRepository` signals a ready state.

```
kubectl get gitrepository example-repo -n ${WORKSPACE_NAMESPACE}
```

The repository commit also displays the ready state.

```
NAME         URL                                                          READY
 STATUS                                                                 AGE
example-repo https://github.com/example-org/example-repo                 True
 Fetched revision: master/6c54bd1722604bd03d25dcac7a31c44ff4e03c6a   11m
```

Create a Private Git Repository

For information on the GitRepository resource fields and how to make Flux aware of credentials required to access a private Git repository, see the Secret Reference section in the Flux documentation at https://fluxcd.io/flux/components/source/gitrepositories/#secret-reference.

Creating an NKP Catalog using the CLI

You can create an NKP catalog using the CLI.

About this task

To create an NKP catalog using the CLI, follow these steps:

Procedure

1. In the terminal of a machine that has network access to the management cluster and has the NKP CLI binary installed, set the `WORKSPACE_NAMESPACE` environment variable to the name of your workspace's namespace:

```
export WORKSPACE_NAME=<workspace_name>
```

2. View the workspace list in the management cluster:

```
nkp get workspaces
```

3. Create a catalog on the management cluster:

```
nkp create catalog nutanix-product-apps-catalog
```

Example:

```
nkp create catalog nutanix-apps-catalog \
-w <workspace_name> \
--branch main \
--url https://github.com/nutanix-cloud-native/nkp-nutanix-product-catalog \
```

4. List the status of gitRepository/catalog instance on the management cluster:

```
kubectl get gitRepository -A
```

Troubleshooting a Git Repository

Logs used for troubleshooting issues with adding the GitRepository.

Procedure

Review the following logs:

```
kubectl -n kommander-flux logs -l app=source-controller
[...]
kubectl -n kommander-flux logs -l app=kustomize-controller
[...]
kubectl -n kommander-flux logs -l app=helm-controller
[...]
```

Workspace Application Metadata

You can define how custom applications display in the NKP UI by defining a `metadata.yaml` file for each application in the git repository. You must define this file at `services/<application>/metadata.yaml` for it to process correctly.

> **Note:** To display more information about custom applications in the UI, define a `metadata.yaml` file for each application in the Git repository.

You can define the following fields:

Table 38: Workplace Application Metadata

Field	Default	Description
displayName	falls back to App ID	Display name of the application for the UI.
description	“”	Short description, should be a sentence or two, displayed in the UI on the application card.
category	general	1 or more categories for this application. Categories are used to group applications in the UI.
overview		Markdown overview used on the application detail page in the UI.
icon		Base64 encoded icon SVG file used for application logos in the UI.
scope	project	List of scopes, can be set only to project or workspace currently.

None of these fields are required for the application to display in the UI.

Here is an example `metadata.yaml` file.

```
displayName: Prometheus Monitoring Stack
description: Stack of applications that collect metrics and provides visualization and
 alerting capabilities. Includes Prometheus, Prometheus Alertmanager and Grafana.
category:
  - monitoring
overview: >
  # Overview
  A stack of applications that collects metrics and provides visualization and alerting
 capabilities. Includes Prometheus, Prometheus Alertmanager and Grafana.

  ## Dashboards
  By deploying the Prometheus Monitoring Stack, the following platform applications and
 their respective dashboards are deployed. After deployment to clusters in a workspace,
 the dashboards are available to access from a respective cluster's detail page.
```

```
  ### Prometheus

  A software application for event monitoring and alerting. It records real-time
 metrics in a time series database built using a HTTP pull model, with flexible and
 real-time alerting.

  - [Prometheus Documentation - Overview](https://prometheus.io/docs/introduction/
overview/)

  ### Prometheus Alertmanager
  A Prometheus component that enables you to configure and manage alerts sent by the
 Prometheus server and to route them to notification, paging, and automation systems.

  - [Prometheus Alertmanager Documentation - Overview](https://prometheus.io/docs/
alerting/latest/alertmanager/)

  ### Grafana
  A monitoring dashboard from Grafana that can be used to visualize metrics collected
 by Prometheus.

  - [Grafana Documentation](https://grafana.com/docs/)
icon:
 PHN2ZyB4bWxucz0iaHR0cDovL3d3dy53My5vcmcvMjAwMC9zdmciIHZpZXdCb3g9IjAgMCAzMDAgMzAwIiBzdHlsZT0iZW5hY
```

Custom Application from the Workspace Catalog

Enable a Custom Application from the Workspace Catalog

After creating a GitRepository, you can either use the NKP UI or the CLI to enable your custom applications. To deploy an application to selected clusters within a workspace, see Cluster-scoped Application Configuration from the NKP UI on page 412.

From within a workspace, you can enable applications to deploy. Verify that an application has successfully deployed through the CLI.

Enabling the Custom Application Using the UI

About this task

To enabling the custom application using the UI:

Procedure

1. From the top menu bar, select your target workspace.
2. From the sidebar menu to browse the available applications from your configured repositories, select **Applications**.
3. Select the three dot button on the required application tile and click **Enable**.
4. If available, select a version from the dropdown list.
   This dropdown list will only be visible if there is more than one version.
5. (Optional) If you want to override the default configuration values, copy your customized values into the text editor under **Configure Service** or upload your YAML file that contains the values.

   ```
   someField: someValue
   ```

6. Confirm the details are correct, and then click **Enable**.

   For all applications, you must provide a display name and an ID which is automatically generated based on what you enter for the display name, unless or until you edit the ID directly. The ID must be compliant with Kubernetes DNS subdomain name validation rules. For more information, see DNS Subdomain Names section in the Kubernetes documentation.

   Alternately, you can use the CLI to enable your catalog applications.

Enabling the Custom Application Using the CLI

Before you begin

- Determine the name of the workspace where you wish to perform the deployments. You can use the `nkp get workspaces` command to see the list of workspace names and their corresponding namespaces.
- Set the `WORKSPACE_NAMESPACE` environment variable to the name of the workspace's namespace where the cluster is attached:

  ```
  export WORKSPACE_NAMESPACE=<workspace_namespace>
  ```

To enabling the custom application using the CLI:

Procedure

1. Get the list of available applications to enable using the following command.

   ```
   kubectl get apps -n ${WORKSPACE_NAMESPACE}
   ```

2. Deploy one of the supported applications from the list with an `AppDeployment` resource.
3. Within the `AppDeployment`, define the `appRef` to specify which `App` to enable.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: apps.kommander.d2iq.io/v1alpha3
   kind: AppDeployment
   metadata:
     name: my-custom-app
     namespace: ${WORKSPACE_NAMESPACE}
   spec:
     appRef:
       name: custom-app-0.0.1
       kind: App
   EOF
   ```

   Note:

   - The `appRef.name` must match the app `name` from the list of available catalog applications.
   - Create the resource in the workspace you just created, which instructs Kommander to deploy the `AppDeployment` to the `KommanderClusters` in the same workspace.

Enabling the Custom Application With Custom Configuration Using the CLI

About this task

To enabling the custom application with custom configuration using the CLI:

Procedure

1. Provide the name of a `ConfigMap` in the `AppDeployment`, which provides custom configuration on top of the default configuration.

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: my-custom-app
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    name: custom-app-0.0.1
    kind: App
  configOverrides:
    name: my-custom-app-overrides
EOF
```

2. Create the `ConfigMap` with the name provided in the step above, with the custom configuration.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: ${WORKSPACE_NAMESPACE}
  name: my-custom-app-overrides
data:
  values.yaml: |
    someField: someValue
EOF
```

   Kommander waits for the `ConfigMap` to be present before deploying the `AppDeployment` to the managed or attached clusters.

Verify the Custom Applications

After completing the previous steps, your applications are enabled.

Procedure

Connect to the attached cluster and check the `HelmReleases` to verify the deployment.

```
kubectl get helmreleases -n ${WORKSPACE_NAMESPACE}
```

The output is as follows.

```
NAMESPACE               NAME            READY   STATUS                              AGE
workspace-test-vjsfq    my-custom-app   True    Release reconciliation succeeded    7m3s
```

Configuring Workspace Role Bindings

Workspace Role Bindings grant access to specified Workspace Roles for a specified group of people.

Before you begin

Before you can create a Workspace Role Binding, ensure you have created a workspace **Group**. A Group can contain one or several Identity Provider users, groups or both.

> **Note:** The syntax for the Identity Provider groups you add to a NKP Group varies depending on the context for which you have established an Identity Provider.

- If you have set up an identity provider globally, for **All Workspaces**:
  - For groups: Add an **Identity Provider Group** in the `oidc:<IdP_user_group>` format. For example, `oidc:engineering`.
  - For users: Add an **Identity Provider User** in the `<user_email>`. For example, `jane.doe@example.com`.
- If you have set up an identity provider for a **Specific Workspace**:
  - For groups: Add an **Identity Provider Group** in the `oidc:<workspace_name>:<IdP_user_group>` format. For example, `oidc:tenant-z:engineering`.
  - For users: Add an **Identity Provider User** in the `<workspace_ID>:<user_email>` format. For example, `tenant-z:jane.doe@example.com`.

Run `kubectl get workspaces` to obtain a list of all existing workspaces. The `workspace_ID` is listed under the `NAME` column.

About this task

You can assign a role to this Kommander Group:

Procedure

1. From the top menu bar, select your target workspace.
2. Select **Access Control** in the **Administration** section of the sidebar menu.
3. Select the **Cluster Role Bindings** tab, and then select **Add Roles** next to the group you want.
4. Select the Role, or Roles, you want from the dropdown list and click **Save**.
   It will take a few minutes for the resource to be created.

Multi-Tenancy in NKP

You can use workspaces to manage your tenants' environments separately, while still maintaining control over clusters and environments centrally. For example, if you operate as a Managed Service Provider (MSP), you can manage your clients clusters' life cycles, resources, and applications. If you operate as an environment administrator, you can manage these resources per department, division, employee group, etc.

Here are some important concepts:

- **Multi-tenancy** in NKP is an architecture model where a single NKP Ultimate instance serves multiple organization’s divisions, customers or tenants. In NKP, each tenant system is represented by a workspace. Each workspace and its resources can be isolated from other workspaces (by using separate Identity Providers), even though they all fall under a single Ultimate license.

  Multi-tenant environments have at least two participating parties: the Ultimate license administrator (for example, an MSP), and one or several tenants.
- **Managed Service Providers** or MSPs are partner organizations that use NKP to facilitate cloud infrastructure services to their customers or tenants.

- **Tenants** can be customers of Managed Service Provider partners. They outsource their cloud management requirements to MSPs, so they can focus on the development of their products.

  Tenants can also be divisions within an organization that require a strict isolation from other divisions, for example, through differentiated access control.

  In NKP, a workspace is assigned to a tenant.

Access Control in Multi-Tenant Environments

To isolate each tenant's information and environment, multi-tenancy allows you to configure an identity provider per workspace or tenant. In this setup, NKP keeps all workspaces and tenants separate and isolated from each other.

You, as a global administrator, manage tenant access at the Workspace level. A tenant can further adapt user access at the Project level.

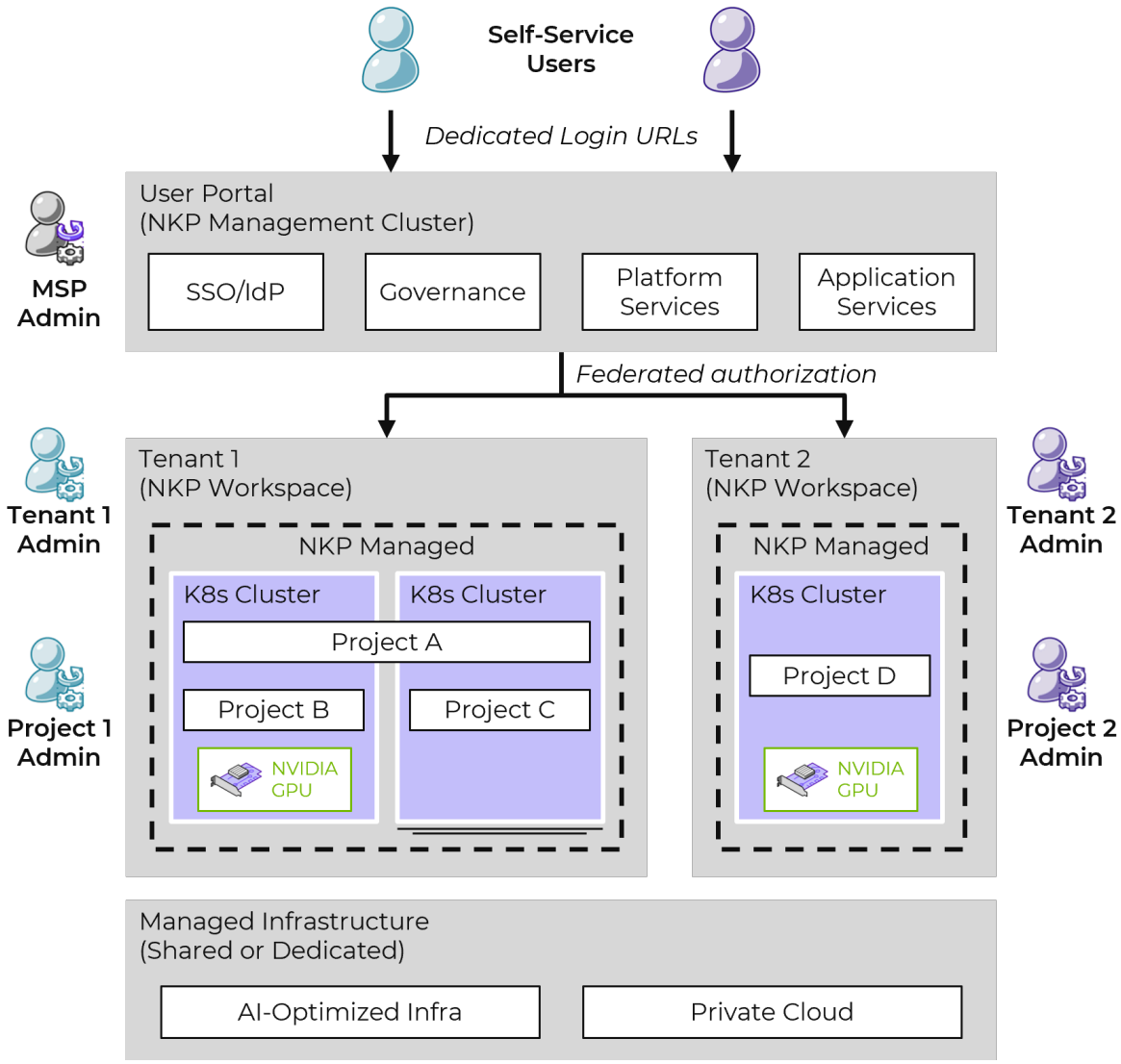


Figure 12: Multi-tenant Cluster

Here are some important concepts:

- **Workspaces**: In a multi-tenant system, workspaces and tenants are synonymous. You can set up an identity provider to control all workspaces, including the Management cluster's `kommander` workspace. You can then set up additional identity providers for each workspace/tenant, and generate a dedicated Login URL so each tenant has its own user access.

  For more information see, Generating a Dedicated Login URL for Each Tenant on page 438.

- **Projects**: After you set up an identity provider per workspace or tenant, the tenant can choose to further narrow down access with an additional layer. A tenant can choose to organize clusters into projects and assign differentiated access to user groups with Project Role Bindings.

  For more information, see Project Role Bindings on page 463.

By assigning clusters to one or several projects, you can enable more complex user access.

Multi-Tenancy Enablement

To enable multi-tenancy, you must:

- If you want to use a single IdP to access all of your tenant's environments, configure an Identity Provider globally.
- Configure an Identity Provider per workspace. This way, each tenant has a dedicated IdP to access their workspace.
- Create NKP Identity Provider groups with the correct prefixes to map your existing IdP groups.
- Create a dedicated login URL for each tenant. You can provide a workspace login link to each tenant for access to the NKP UI and for the generation of kubectl API access tokens.

To enforce security, every tenant should be in a different AWS account, so they are truly independent of each other.

Generating a Dedicated Login URL for Each Tenant

This page contains instructions on how to generate a workspace-specific URL to access the NKP UI.

About this task

By making this URL available to your tenant, you provide them with a dedicated login page, where users can enter their SSO credentials to access their workspace in the NKP UI and to where users can create a token to access a cluster's kubectl API. Other tenants and their SSO configurations are not visible.

Before you begin

- Complete the steps in Multi-Tenancy in NKP on page 436.
- Ensure you have administrator permissions and access to all workspaces.

Procedure

1. Set an environment variable to point at the workspace for which you want to generate a URL:

   Replace `<name_target_workspace>` with the workspace name. If you do not know the exact name of the workspace, run `kubectl get workspace` to get a list of all workspace names.

   ```
   export WORKSPACE_NAME=<name_target_workspace>
   ```

2. Generate an NKP UI login URL for that workspace.

```
echo https://$(kubectl get kommandercluster -n kommander host-cluster -o
 jsonpath='{ .status.ingress.address }')/token/landing/${WORKSPACE_NAME}
```

The output is as follows.

```
https://example.com/token/landing/<WORKSPACE_NAME>
```

3. Share the output login URL with your tenant, so users can start accessing their workspace from the NKP UI.

Note: The login page displays:

- Identity providers set globally.
- Identity providers set for that specific workspace.

The login page does not display any resources or workspaces for which the tenant has no permissions.

Projects

Multi-cluster Configuration Management

Projects support the management of configMaps, continuous deployments, secrets, services, quotas, and role-based access control and multi-tenant logging by leveraging federated resources. When a Project is created, NKP creates a federated namespace that is propagated to the Kubernetes clusters associated with this Project.

Federation in this context means that a common configuration is pushed out from a central location (NKP) to all Kubernetes clusters, or a pre-defined subset group, under NKP management. This pre-defined subset group of Kubernetes clusters is called a Project.

Projects enable teams to deploy their configurations and services to clusters in a consistent way. Projects enable central IT or a business unit to share their Kubernetes clusters among several teams. Using Projects, NKP leverages Kubernetes Cluster Federation (KubeFed) to coordinate the configuration of multiple Kubernetes clusters.

Kommander allows a user to use labels to select, manually or dynamically, the Kubernetes clusters associated with a Project.

Project Namespaces

A Project Namespace is a Kommander-specific concept. Project Namespaces isolate configurations across clusters and are created on all clusters matching the project labels. When creating a new project, you can customize the name of the Kubernetes namespace that is created.

Creating a Project Using the UI

About this task

When you create a Project, you must specify a Project Name, a Namespace Name (optional) and a way to allow Kommander to determine which Kubernetes clusters will be part of this project.

As mentioned previously, a Project Namespace corresponds to a Kubernetes Federated Namespace. By default, the name of the namespace is auto-generated based on the project name (first 57 characters) plus 5 unique alphanumeric characters. You can specify a namespace name, but you must ensure it does not conflict with any existing namespace on the target Kubernetes clusters, that will be a part of the Project.

To determine which Kubernetes clusters will be part of this project, you can either select manually existing clusters or define labels that Kommander will use to dynamically add clusters. The latter is recommended because it will allow you to deploy additional Kubernetes clusters later and to have them automatically associated with Projects based on their labels.

To create a Project, you can either use the NKP UI or create a Project object on the Kubernetes cluster where Kommander is running (using kubectl or the Kubernetes API). The latter allows you to configure Kommander resources in a declarative way. It is available for all kinds of Kommander resources.

To create a project using the NKP UI:

Procedure

1. Enter the following fields:
   - **Project Name**
   - **ID / Namespace**
   - **Description**
2. In the **Clusters** field, add one or more active clusters.
3. Click **Create**.

Creating a Project Using the CLI

About this task

The following sample is a YAML Kubernetes object for creating a Kommander Project. This example does not work verbatim because it depends on a workspace name that has been previously created and does not exist by default in your cluster.

Procedure

Use this as an example format and fill in the workspace name and namespace name appropriately along with the proper labels.

```
apiVersion: workspaces.kommander.mesosphere.io/v1alpha1
kind: Project
metadata:
  name: My-Project-Name
  namespace: my-project-k8s-namespace-name
spec:
  workspaceRef:
    name: myworkspacename
  namespaceName: myworkspacename-di3tx
  placement:
    clusterSelector:
      matchLabels:
        cluster: prod
```

The following procedures are supported for projects:

- Project Applications on page 441
- Project Deployments on page 456
- Project Role Bindings on page 463
- Project Roles on page 466
- Project ConfigMaps on page 469

- Project Secrets on page 470
- Project Quotas and Limit Ranges on page 471
- Project Network Policies on page 473

Project Applications

This section documents the applications and application types that you can utilize with NKP.

Application types are:

- Workplace Catalog Applications on page 420 that are either pre-packaged applications from the Nutanix Application Catalog or custom applications that you maintain for your teams or organization.
  - NKP Applications on page 390 are applications that are provided by Nutanix and added to the Catalog.
  - Custom Applications on page 428 are applications integrated into Kommander.
- Platform Applications on page 400

When deploying and upgrading applications, platform applications come as a bundle; they are tested as a single unit and you must deploy or upgrade them in a single process, for each workspace. This means all clusters in a workspace have the same set and versions of platform applications deployed. Whereas catalog applications are individual, so you can deploy and upgrade them individually, for each project.

Project Platform Applications

How project Platform applications work

The following table describes the list of applications that can be deployed to attached clusters within a project.

Review the Project Platform Application Configuration Requirements on page 445 to ensure that the attached clusters in the project have sufficient resources.

From within a project, you can enable applications to deploy. Verify that an application has successfully deployed through the CLI.

Platform Applications

Table 39: Platform Applications

Name	APP ID	Deployed by default
project-grafana-logging-6.57.4	project-grafana-logging	False
project-grafana-loki-0.69.16	project-grafana-loki	False
project-logging-1.0.3	project-logging	False

Enabling the Platform Application Using the UI

About this task

Follow these steps to:

Procedure

1. From the top menu bar, select your target workspace.
2. Select **Projects** from the sidebar menu.
3. Select your project from the list.
4. Select **Applications** tab to browse the available applications.
5. Select the three dot button from the bottom-right corner of the desired application tile, and then select **Enable**.
6. If you want to override the default configuration values, copy your customized values into the text editor under **Configure Service** or upload your YAML file that contains the values.

   ```
   someField: someValue
   ```

7. Confirm the details are correct, and then click **Enable**.

   To use the CLI to enable or disable applications, see Deploying Platform Applications Using CLI on page 403

   > **Warning:** There may be dependencies between the applications, which are listed in Project Platform Application Dependencies on page 444. Review them carefully prior to customizing to ensure that the applications are deployed successfully.

Platform Applications Upgrade Using the CLI

Platform Applications within a Project are automatically upgraded when the Workspace that a Project belongs to is upgraded.

For more information on how to upgrade these applications, see Ultimate: Upgrade Platform Applications on Managed and Attached Clusters on page 1156.

Deploying Project Platform Applications Using the CLI

Deploy applications to attached clusters in a project using the CLI.

About this task

This topic describes how to use the CLI to deploy an application to attached clusters within a project.

For a list of all applications and those that are enabled by default, see Project Platform Applications.

Before you begin

Ensure that you have:

- A running cluster with Kommander installed.
- An existing Kubernetes cluster attached to Kommander.
- Set the `WORKSPACE_NAME` environment variable to the name of the workspace where the cluster is attached.

  ```
  export WORKSPACE_NAME=<workspace_name>
  ```

- Set the `WORKSPACE_NAMESPACE` environment variable to the namespace of the above workspace.

```
export WORKSPACE_NAMESPACE=$(kubectl get namespace --
selector='workspaces.kommander.mesosphere.io/workspace-name=${WORKSPACE_NAME}' -o
 jsonpath='{.items[0].metadata.name}')
```

- Set the `PROJECT_NAME` environment variable to the name of the project in which the cluster is included:

```
export PROJECT_NAME=<project_name>
```

- Set the `PROJECT_NAMESPACE` environment variable to the name of the above project's namespace:

```
export PROJECT_NAMESPACE=$(kubectl get project ${PROJECT_NAMESPACE} -n
 ${WORKSPACE_NAMESPACE} -o jsonpath='{.status.namespaceRef.name}')
```

Procedure

1. Deploy one of the supported applications to your existing attached cluster with an `AppDeployment` resource. Provide the `appRef` and application version to specify which `App` is deployed.

```
nkp create appdeployment project-grafana-logging --app project-grafana-logging-6.38.1
 --workspace ${WORKSPACE_NAME} --project ${PROJECT_NAME}
```

2. Create the resource in the project you just created, which instructs Kommander to deploy the `AppDeployment` to the `KommanderClusters` in the same project.

> **Note:**
>
> - The `appRef.name` must match the app `name` from the list of available catalog applications.
> - Observe that the `nkp create` command must be run with both the `--workspace` and `--project` flags for project platform applications.

Deploying the Project Platform Application With Custom Configuration Using the CLI

About this task

To perform custom configuration using the CLI:

Procedure

1. Create the `AppDeployment` and provide the name of a `ConfigMap`, which provides custom configuration on top of the default configuration.

```
nkp create appdeployment project-grafana-logging --app project-grafana-logging-6.38.1
 --config-overrides project-grafana-logging-overrides --workspace ${WORKSPACE_NAME}
 --project ${PROJECT_NAMESPACE}
```

2. Create the `ConfigMap` with the name provided in the step above, with the custom configuration.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: ${PROJECT_NAMESPACE}
  name: project-grafana-logging-overrides
data:
  values.yaml: |
    datasources:
```

```
      datasources.yaml:
        apiVersion: 1
        datasources:
        - name: Loki
          type: loki
          url: "http://project-grafana-loki-loki-distributed-gateway"
          access: proxy
          isDefault: false
EOF
```

Kommander waits for the `ConfigMap` to be present before deploying the `AppDeployment` to the managed or attached clusters.

Verify the Project Platform Applications

After completing the previous steps, your applications are enabled.

Procedure

1. Export the `project_namespace` with this command.

   ```
   export PROJECT_NAMESPACE=<project_namespace>
   ```

2. Connect to the attached cluster and check the `HelmReleases` to verify the deployment.

   ```
   kubectl get helmreleases -n ${PROJECT_NAMESPACE}
   NAMESPACE               NAME                        READY   STATUS
            AGE
   project-test-vjsfq      project-grafana-logging     True    Release reconciliation
    succeeded   7m3s
   ```

> **Note:** Some of the supported applications have dependencies on other applications. See Project Platform Application Dependencies on page 444 for that table.

Project Platform Application Dependencies

Dependencies between project platform applications.

There are many dependencies between the applications that are deployed to a project's attached clusters. It is important to note these dependencies when customizing the platform applications to ensure that your services are properly deployed to the clusters. For more information on how to customize platform applications, see Project Platform Applications on page 441.

Application Dependencies

When deploying or troubleshooting applications, it helps to understand how applications interact and may require other applications as dependencies.

If an application's dependency does not successfully deploy, the application requiring that dependency does not successfully deploy.

The following sections detail information about the platform applications.

Logging

Collects logs over time from Kubernetes pods deployed in the project namespace. Also provides the ability to visualize and query the aggregated logs.

- *project-logging*: Defines resources for the Logging Operator which uses them to direct the project's logs to its respective Grafana Loki application. For more information, see https://grafana.com/oss/grafana/.

- **project-grafana-loki**: A horizontally-scalable, highly-available, multi-tenant log aggregation system inspired by Prometheus. For more information, see https://grafana.com/oss/loki/.
- **project-grafana-logging**: Logging dashboard used to view logs aggregated to Grafana Loki. For more information, see https://grafana.com/oss/grafana/.

> **Warning:** The project logging applications depend on the Enabling Logging Applications Using the UI on page 595 being deployed.

Table 40: Project Platform Application Dependencies

Application	Required Dependencies
project-logging	logging-operator (workspace)
project-grafana-loki	project-logging, grafana-loki (workspace), logging-operator (workspace)
project-grafana-logging	project-grafana-loki

Project Platform Application Configuration Requirements

Project Platform Application Descriptions and Resource Requirements

Platform applications require more resources than solely deploying or attaching clusters into a project. Your cluster must have sufficient resources when deploying or attaching to ensure that the applications are installed successfully.

The following table describes all the platform applications that are available to the clusters in a project, minimum resource and persistent storage requirements, and whether they are enabled by default.

Table 41: Project Platform Application Configuration Requirements

Name	Minimum Resources Suggested	Minimum Persistent Storage Required	Deployed by Default	Default Priority Class
project-grafana-logging	cpu: 200m memory: 100Mi		No	NKP Critical (100002000)
project-grafana-loki		# of PVs: 3 PV sizes: 10Gi x 3 (total: 30Gi)	No	NKP Critical (100002000)
project-logging			No	NKP Critical (100002000)

Project Catalog Applications

Catalog applications are any third-party or open source applications that appear in the Catalog. These can be NKP applications provided by Nutanix for use in your environment or that can be used but are not supported by Nutanix.

For more information, see:

- Project Catalog Applications on page 445
- Project-level NKP Applications on page 447
- Usage of Custom Resources with Workspace Catalog Applications on page 450
- Custom Project Applications on page 450

Upgrading Project Catalog Applications Using the UI

Before upgrading your catalog applications, verify the current and supported versions of the application. Also, keep in mind the distinction between Platform applications and Catalog applications. Platform applications are deployed and upgraded as a set for each cluster or workspace. Catalog applications are deployed separately, so that you can deploy and upgrade them individually for each project.

About this task

Catalog applications must be upgraded to the latest version BEFORE upgrading the Konvoy component for Managed clusters or Kubernetes version for attached clusters.

To upgrade an application from the NKP UI:

Procedure

1. From the top menu bar, select your target workspace.
2. From the side menu bar, select **Projects**.
3. Select your target project.
4. Select **Applications** from the project menu bar.
5. Select the three dot button from the bottom-right corner of the desired application tile, and then click **Edit**.
6. Select the **Version** dropdown list, and select a new version. This dropdown list will only be available if there is a newer version to upgrade to.
7. Click **Save**.

Upgrading Project Catalog Applications Using the CLI

About this task

To upgrade project catalog applications:

Procedure

1. To see what app(s) and app versions are available to upgrade, run the following command:

   Note: The `APP ID` column displays the available apps and the versions available to upgrade.

   ```
   kubectl get apps -n ${PROJECT_NAMESPACE}
   ```

2. Run the following command to upgrade an application from the NKP CLI.

```
nkp upgrade catalogapp <appdeployment-name> --workspace=my-workspace --project=my-project --to-version=<version.number>
```

As an example, the following command upgrades the Kafka Operator application, named `kafka-operator-abc`, in a workspace to version `0.25.1`.

```
nkp upgrade catalogapp kafka-operator-abc --workspace=my-workspace --to-version=0.25.1
```

> **Note:** Platform applications cannot be upgraded on a one-off basis, and must be upgraded in a single process for each workspace. If you attempt to upgrade a platform application with these commands, you receive an error and the application is not upgraded.

Project-level NKP Applications

NKP applications are catalog applications provided by Nutanix for use in your environment.

Some NKP workspace catalog applications will provision `CustomResourceDefinitions`, which allow you to deploy Custom Resources to a Project. See your NKP workspace catalog application's documentation for instructions.

Deploying ZooKeeper in a Project

To get started with creating ZooKeeper clusters in your project namespace, you first need to deploy the Zookeeper operator in the workspace where the project exists.

About this task

After you deploy the ZooKeeper operator, you can create ZooKeeper Clusters by applying a `ZookeeperCluster` custom resource on each attached cluster in a project's namespace.

A Helm chart exists in the ZooKeeper operator repository that can assist with deploying ZooKeeper clusters. For more information, see https://github.com/pravega/zookeeper-operator/tree/master/charts/zookeeper.

> **Note:** If you need to manage these custom resources across all clusters in a project, it is recommended you use Project Deployments on page 456 which enables you to leverage GitOps to deploy the resources. Otherwise, you will need to create the resources manually in each cluster.

Follow these steps to deploy a ZooKeeper cluster in a project namespace. This procedure results in a running ZooKeeper cluster, ready for use in your project's namespace.

Before you begin

You must first deploy the Zookeeper operator. For more information, see Zookeeper Operator in Workspace on page 423.

Procedure

1. Set the `PROJECT_NAMESPACE` environment variable to the name of your project's namespace.

```
export PROJECT_NAMESPACE=<project namespace>
```

2. Create a ZooKeeper Cluster custom resource in your project namespace.

```
kubectl apply -f - <<EOF
apiVersion: zookeeper.pravega.io/v1beta1
kind: ZookeeperCluster
```

```
metadata:
  name: zookeeper
  namespace: ${PROJECT_NAMESPACE}
spec:
  replicas: 1
EOF
```

3. Check the status of your ZooKeeper cluster using `kubectl`.

```
kubectl get zookeeperclusters -n ${PROJECT_NAMESPACE}
```

```
NAME        REPLICAS   READY REPLICAS    VERSION   DESIRED VERSION   INTERNAL
 ENDPOINT    EXTERNAL ENDPOINT   AGE
zookeeper   1          1                0.2.15    0.2.15
 10.100.200.18:2181   N/A                94s
```

Deleting Zookeeper in a Project

About this task

To delete the Zookeeper clusters:

Procedure

1. View `ZookeeperClusters`in all namespaces.

```
kubectl get zookeeperclusters -A
```

2. Delete a specific `ZookeeperCluster`.

```
kubectl -n ${PROJECT_NAMESPACE} delete zookeepercluster <name of zookeepercluster>
```

Deploying Kafka in a Project

After you deploy the Kafka operator, you can create Kafka clusters by applying a `KafkaCluster` custom resource on each attached cluster in a project's namespace.

About this task

Refer to the Kafka operator repository for examples of the custom resources and their configurations. For more information, see https://github.com/banzaicloud/koperator/tree/master/config/samples.

Before you begin

To get started with creating and managing a Kafka Cluster in a project, you must:

- Deploy the Kafka operator in the workspace where the project exists. See Kafka Operator in a Workspace on page 422.
- Deploy the ZooKeeper operator the workspace where the project exists. See Zookeeper Operator in Workspace on page 423.
- Deploy the ZooKeeper operator in the workspace where the project exists. Deploy Zookeeper in a Project in the same project where you want to enable Kafka. See Deploying ZooKeeper in a Project on page 447.

> **Note:** If you need to manage these custom resources across all clusters in a project, it is recommended you use project deployments which enables you to leverage GitOps to deploy the resources. Otherwise, you must create the custom resources manually in each cluster.

Procedure

1. Ensure you deployed Zookeeper in a project.
   See Deploying ZooKeeper in a Project on page 447
2. Set the `PROJECT_NAMESPACE` environment variable to the name of your project's namespace.

   ```
   export PROJECT_NAMESPACE=<project namespace>
   ```

3. Obtain the Kafka Operator version you deployed in the workspace.
   Replace `<target_namespace>` with the namespace where you deployed Kafka.

   ```
   kubectl get appdeployments.apps.kommander.d2iq.io -n <target_namespace> -o
    template="{{ .spec.appRef.name }}" kafka-operator
   ```

   The output prints the Kafka Operator version.
4. Use the Kafka Operator version to download the `simplekafkacluster.yaml` file you require.

   In the following URL, replace `/v0.25.1/` with the Kafka version you obtained in the previous step and download the file.

   ```
   https://raw.githubusercontent.com/banzaicloud/koperator/v0.25.1/config/samples/
   simplekafkacluster.yaml
   ```

   In order to use a CVE-free kafka image, set `clusterImage` value to `ghcr.io/banzaicloud/kafka:2.13-3.4.1` (similarly to workspace installation in).
5. Open and edit the downloaded file to use the correct Zookeeper Cluster address.
   Replace `<project_namespace>` with the target project namespace.

   ```
   zkAddresses:
       - "zookeeper-client.<project_namespace>:2181"
   ```

6. Apply the `KafkaCluster` configuration to your project's namespace:.

   ```
   kubectl apply -n ${PROJECT_NAMESPACE} -f simplekafkacluster.yaml
   ```

7. Check the status of your Kafka cluster using `kubectl`.

   ```
   kubectl -n ${PROJECT_NAMESPACE} get kafkaclusters
   ```

   The output should look similar to this.

   ```
   NAME    CLUSTER STATE    CLUSTER ALERT COUNT   LAST SUCCESSFUL UPGRADE   UPGRADE
    ERROR COUNT   AGE
   kafka   ClusterRunning   0                                               0
             79m
   ```

   With both the ZooKeeper cluster and Kafka cluster running in your project's namespace, refer to the Kafka Operator documentation. When performing those steps, ensure you substitute: `zookeeper-client.<project namespace>:2181` anywhere that the zookeeper client address is mentioned. For more information on how to test and verify they are working as expected, see Kafka Operator documentation.

Deleting Kafka in a Project

About this task

To delete the Kafka custom resources:

Procedure

1. View all Kafka resources in the cluster.

```
kubectl get kafkaclusters -A
kubectl get kafkausers -A
Kubectl get kafkatopics -A
```

2. Delete a `KafkaCluster` example.

```
kubectl -n ${PROJECT_NAMESPACE} delete kafkacluster <name of KafkaCluster>
```

Usage of Custom Resources with Workspace Catalog Applications

Some workspace catalog applications provision some `CustomResourceDefinition`, which allow you to deploy Custom Resources. Refer to your workspace catalog application's documentation for instructions.

Custom Project Applications

Custom applications are third-party applications you have added to the Kommander Catalog.

Custom applications are any third-party applications that are not provided in the NKP Application Catalog. Custom applications can leverage applications from the NKP Catalog or be fully-customized. There is no expectation of support by Nutanix for a Custom application. Custom applications can be deployed on Konvoy clusters or on any Nutanix supported 3rd party Kubernetes distribution.

- Projects: Git Repository Structure on page 450
- Project: Workspace Application Metadata on page 453
- Enabling a Custom Application From the Project Catalog Using the UI on page 454 and Enabling a Custom Application From the Project Catalog Using the CLI on page 455

Projects: Git Repository Structure

Git repositories must be structured in a specific manner for defined applications to be processed by Kommander.

You must structure your git repository based on the following guidelines, for your applications to be processed properly by Kommander so that they can be deployed.

Git Repository Directory Structure

Use the following basic directory structure for your git repository.

```
### helm-repositories
#   ### <helm repository 1>
#   #   ### kustomization.yaml
#   #   ### <helm repository name>.yaml
#   ### <helm repository 2>
#       ### kustomization.yaml
#       ### <helm repository name>.yaml
### services
    ### <app name>
    #   ### <app version1> # semantic version of the app helm chart. e.g., 1.2.3
    #   #   ### defaults
    #   #   #   ### cm.yaml
    #   #   #   ### kustomization.yaml
    #   #   ### <app name>.yaml
    #   #   ### kustomization.yaml
```

```
   #   ### <app version2> # another semantic version of the app helm chart. e.g.,
2.3.4
   #   #   ### defaults
   #   #   #   ### cm.yaml
   #   #   #   ### kustomization.yaml
   #   #   ### <app name>.yaml
   #   #   ### kustomization.yaml
   #   ### metadata.yaml
   ### <another app name>
   ...
```

Remember the following guidelines:

- Define applications in the `services/` directory.
- You can define multiple versions of an application, under different directories nested under the `services/<app name>/` directory.
- Define application manifests, such as a HelmRelease, under each versioned directory `services/<app name>/<version>/`.

  in `<app name>.yaml` in which is listed in the `kustomization.yaml` Kubernetes Kustomization file. For more information, see https://fluxcd.io/docs/components/helm/helmreleases/ and https://kubectl.docs.kubernetes.io/references/kustomize/kustomization/.
- Define the default values ConfigMap for `HelmReleases` in the `services/<app name>/<version>/defaults` directory, accompanied by a `kustomization.yaml` Kubernetes Kustomization file pointing to the `ConfigMap` file.
- Define the `metadata.yaml` of each application under the `services/<app name>/` directory. For more information, see Workspace Application Metadata on page 432

For an example of how to structure custom catalog Git repositories, see the NKP Catalog repository at https://github.com/mesosphere/dkp-catalog-applications.

Helm Repositories

You must include the `HelmRepository` that is referenced in each `HelmRelease`'s `Chart` spec.

Each `services/<app name>/<version>/kustomization.yaml` must include the path of the YAML file that defines the `HelmRepository`. For example.

```
# services/<app name>/<version>/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - <app name>.yaml
  - ../../../helm-repositories/<helm repository 1>
```

For more information, see the flux documentation at:

- HelmRepositories: https://fluxcd.io/docs/components/source/helmrepositories/
- Manage Helm Releases: https://fluxcd.io/flux/guides/helmreleases/

Substitution Variables

Some substitution variables are provided. For more information, see https://fluxcd.io/docs/components/kustomize/kustomization/#variable-substitution.

- `${releaseName}`: For each application deployment, this variable is set to the `AppDeployment` name. Use this variable to prefix the names of any resources that are defined in the application directory in the Git repository so that multiple instances of the same application can be deployed. If you create

resources without using the `releaseName` prefix (or suffix) in the name field, there can be conflicts if the same named resource is created in that same namespace.

- `${releaseNamespace}`: The namespace of the project.
- `${workspaceNamespace}`: The namespace of the workspace that the project belongs to.

Project: Creating a Git Repository

Use the CLI to create the GitRepository resource and add a new repository to your Project.

About this task

To ceate a Git repository in the Project namespace.

Procedure

1. Refer to Installing Kommander in an Air-gapped Environment on page 1017 setup instructions section, if you are running in air-gapped environment.
2. Set the `WORKSPACE_NAMESPACE` environment variable to the name of your project's namespace.

   ```
   export PROJECT_NAMESPACE=<project_namespace>
   ```

3. Adapt the URL of your Git repository.

   ```
   kubectl apply -f - <<EOF
   apiVersion: source.toolkit.fluxcd.io/v1
   kind: GitRepository
   metadata:
     name: example-repo
     namespace: ${PROJECT_NAMESPACE}
   spec:
     interval: 1m0s
     ref:
       branch: <your-target-branch-name> # e.g., main
     timeout: 20s
     url: https://github.com/<example-org>/<example-repo>
   EOF
   ```

4. Ensure the status of the `GitRepository` signals a ready state.

   ```
   kubectl get gitrepository example-repo -n ${PROJECT_NAMESPACE}
   ```

   The repository commit also displays the ready state.

   ```
   NAME         URL                                                        READY
    STATUS                                                             AGE
   example-repo https://github.com/example-org/example-repo                True
    Fetched revision: master/6c54bd1722604bd03d25dcac7a31c44ff4e03c6a   11m
   ```

   For more information on the GitRepository resource fields and how to make Flux aware of credentials required to access a private Git repository, see the Flux documentation at https://fluxcd.io/flux/components/source/gitrepositories/#secret-reference.

   Note: To troubleshoot issues with adding the GitRepository, review the following logs:

   ```
   kubectl -n kommander-flux logs -l app=source-controller
   [...]
   kubectl -n kommander-flux logs -l app=kustomize-controller
   [...]
   kubectl -n kommander-flux logs -l app=helm-controller
   ```

```
[...]
```

Project: Workspace Application Metadata

You can define how custom applications display in the NKP UI by defining a `metadata.yaml` file for each application in the git repository. You must define this file at `services/<application>/metadata.yaml` for it to process correctly.

> **Note:** To display more information about custom applications in the UI, define a `metadata.yaml` file for each application in the Git repository.

You can define the following fields:

Table 42: Workplace Application Metadata

Field	Default	Description
displayName	falls back to App ID	Display name of the application for the UI.
description	“”	Short description, should be a sentence or two, displayed in the UI on the application card.
category	general	1 or more categories for this application. Categories are used to group applications in the UI.
overview	-	Markdown overview used on the application detail page in the UI.
icon	-	Base64 encoded icon SVG file used for application logos in the UI.
scope	project	List of scopes, can be set only to project or workspace currently.

None of these fields are required for the application to display in the UI.

Here is an example `metadata.yaml` file:

```
displayName: Prometheus Monitoring Stack
description: Stack of applications that collect metrics and provides visualization and
 alerting capabilities. Includes Prometheus, Prometheus Alertmanager and Grafana.
category:
  - monitoring
overview: >
  # Overview
  A stack of applications that collects metrics and provides visualization and alerting
 capabilities. Includes Prometheus, Prometheus Alertmanager and Grafana.

  ## Dashboards
  By deploying the Prometheus Monitoring Stack, the following platform applications and
 their respective dashboards are deployed. After deployment to clusters in a workspace,
 the dashboards are available to access from a respective cluster's detail page.

  ### Prometheus

  A software application for event monitoring and alerting. It records real-time
 metrics in a time series database built using a HTTP pull model, with flexible and
 real-time alerting.
```

```
  - [Prometheus Documentation - Overview](https://prometheus.io/docs/introduction/
overview/)

  ### Prometheus Alertmanager
  A Prometheus component that enables you to configure and manage alerts sent by the
 Prometheus server and to route them to notification, paging, and automation systems.

  - [Prometheus Alertmanager Documentation - Overview](https://prometheus.io/docs/
alerting/latest/alertmanager/)

  ### Grafana
  A monitoring dashboard from Grafana that can be used to visualize metrics collected
 by Prometheus.

  - [Grafana Documentation](https://grafana.com/docs/)
icon:
 PHN2ZyB4bWxucz0iaHR0cDovL3d3dy53My5vcmcvMjAwMC9zdmciIHZpZXdCb3g9IjAgMCAzMDAgMzAwIiBzdHlsZT0iZW5hY
```

Enabling a Custom Application From the Project Catalog Using the UI

Enable a Custom Application from the Project Catalog. After creating a GitRepository, you can either use the NKP UI or the CLI to enable your custom applications.

About this task

> **Note:** From within a project, you can enable applications to deploy. Verify that an application has successfully deployed through the CLI.

Procedure

1. From the top menu bar, select your target workspace.
2. From the side menu bar, select **Projects**.
3. Select your target project from the list.
4. Select **Applications** from the sidebar menu to browse the available applications from your configured repositories.
5. Select the three dot button from the bottom-right corner of the desired application tile, and then select **Enable**.
6. If available, select a version from the dropdown list. This dropdown list will only be visible if there is more than one version.
7. (Optional) If you want to override the default configuration values, copy your customized values into the text editor under **Configure Service**or upload your YAML file that contains the values.

   ```
   someField: someValue
   ```

8. Confirm the details are correct, and then click **Enable**.

   For all applications, you must provide a display name and an ID which is automatically generated based on what you enter for the display name, unless or until you edit the ID directly. The ID must be compliant with Kubernetes DNS subdomain name validation rules. For more information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/#dns-subdomain-names.

   Alternately, you can use the CLI to enable your catalog applications. For more information, see Deployment of Catalog Applications in Workspaces on page 424.

Enabling a Custom Application From the Project Catalog Using the CLI

Enable a Custom Application from the Project Catalog. After creating a GitRepository, you can either use the NKP UI or the CLI to enable your custom applications.

About this task

> **Note:** From within a project, you can enable applications to deploy. Verify that an application has successfully deployed through the CLI.

Procedure

1. Set the `PROJECT_NAMESPACE` environment variable to the name of the above project's namespace.

   ```
   export PROJECT_NAMESPACE=<project_namespace>
   ```

2. Get the list of available applications to enable using the following command.

   ```
   kubectl get apps -n ${PROJECT_NAMESPACE}
   ```

3. Enable one of the supported applications from the list with an `AppDeployment` resource.

4. Within the `AppDeployment` resource. Provide the `appRef` and application version to specify which `App` is deployed.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: apps.kommander.d2iq.io/v1alpha3
   kind: AppDeployment
   metadata:
     name: my-custom-app
     namespace: ${PROJECT_NAMESPACE}
   spec:
     appRef:
       name: custom-app-0.0.1
       kind: App
   EOF
   ```

   > **Note:** The `appRef.name` must match the app `name` from the list of available catalog applications.

Enabling a Custom Application Configuration With Custom Configuration Using the CLI

About this task

Follow these steps:

Procedure

1. Provide the name of a `ConfigMap` in the `AppDeployment`, which provides custom configuration on top of the default configuration.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: apps.kommander.d2iq.io/v1alpha3
   kind: AppDeployment
   metadata:
     name: my-custom-app
     namespace: ${PROJECT_NAMESPACE}
   spec:
     appRef:
       name: custom-app-0.0.1
       kind: App
   ```

```
  configOverrides:
    name: my-custom-app-overrides
EOF
```

2. Create the `ConfigMap` with the name provided in the step above, with the custom configuration.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: ${PROJECT_NAMESPACE}
  name: my-custom-app-overrides
data:
  values.yaml: |
    someField: someValue
EOF
```

   Kommander waits for the `ConfigMap` to be present before deploying the `AppDeployment` to the attached clusters in the Project.

Project: Verify the Custom Applications

After completing the previous steps, your applications are enabled.

Procedure

Connect to the attached cluster and check the `HelmReleases` to verify the deployment.

```
kubectl get helmreleases -n ${PROJECT_NAMESPACE}
```

The output looks similar to this:

```
NAMESPACE               NAME            READY   STATUS                             AGE
project-test-vjsfq      my-custom-app   True    Release reconciliation succeeded   7m3s
```

Custom Applications Upgrade

You must maintain your custom applications manually.

When upgrading NKP, ensure you validate for compatibility issues any custom applications you run against the current version of Kubernetes. We recommend upgrading to the latest compatible application versions as soon as possible.

Project AppDeployments
An `AppDeployment` is a Custom Resource created by NKP with the purpose of deploying applications (platform, NKP catalog and custom applications).

For more information about these Custom Resources and how to customize them, see Printing and Reviewing the Current State of an AppDeployment Resource on page 391 section of this guide.

Project Deployments

Use Project Deployments to manage GitOps based Continuous Deployments.

You can configure Kommander Projects with GitOps-based Continuous Deployments for federation of your Applications to associated clusters of the project. This is backed by Flux, which enables software and applications to be continuously deployed (CD) using GitOps processes. GitOps enables the application to be deployed as per a manifest that is stored in a Git repository. This ensures that the application deployment can be automated, audited, and declaratively deployed to the infrastructure.

GitOps
GitOps is a modern software deployment strategy. The configuration that describes how your application is deployed to a cluster are stored in a Git repository. The configuration is continuously synchronized from the Git repository to the cluster, ensuring that the specified state of the cluster always matches what is defined in the “GitOps” Git repository.

The benefits of using a GitOps deployment strategy are:

- Familiar, collaborative change and review process. Engineers are intimately familiar with Git-based workflows: branches, pull requests, code reviews, etc. GitOps leverages this experience to control the deployment of software and updates to catch issues early.
- Clear change log and audit trail. The Git commit log serves as an audit trail to answer the question: “who changed what, and when?” Having such information available, you can contact the right people when fixing or prioritizing a production incident to determine the why and correctly resolve the issue as quickly as possible. Additionally, Kommander’s CD component (Flux CD) maintains a separate audit trail in the form of Kubernetes Events, as changes to a Git repository don’t include exactly when those changes were deployed.
- Avoid configuration drift. The scope of manual changes made by operators expands over time. It soon becomes difficult to know which cluster configuration is critical and which is left over from temporary workarounds or live debugging. Over time, changing a project configuration or replicating a deployment to a new environment becomes a daunting task. GitOps supports simple, reproducible deployment to multiple different clusters by having a single source of truth for cluster and application configuration.

That said, there are some cases when live debugging is necessary in order to resolve an incident in the minimum amount of time. In such cases, pull-request-based workflow adds precious time to resolution for critical production outages. Kommander’s CD strategy supports this scenario by letting you disable the auto sync feature. After auto sync is disabled, Flux will stop synchronizing the cluster state from the GitOps git repository. This lets you use `kubectl`, `helm`, or whichever tool you need to resolve the issue.

Continuous Delivery with GitOps
NKP enables software and applications to be continuously delivered (CD) using GitOps processes. GitOps enables you to deploy an application according to a manifest that is stored in a Git repository. This ensures that the application deployment can be automated, audited, and declaratively deployed to the infrastructure.

This section contains step-by-step tutorials for performing some common deployment-related tasks using NKP. All tutorials begin with a Prerequisites section that contains links to any steps that need to be taken first. This means you can visit any tutorial to get started.

- Secrets Stored in GitOps Repository Using SealedSecrets on page 457
- Deploying a Sample Application from NKP GitOps on page 458

Secrets Stored in GitOps Repository Using SealedSecrets
Securely managing secrets in a GitOps workflow using SealedSecrets

For security reasons, Kubernetes secrets are usually the only resource that cannot be managed with a GitOps workflow. Instead of managing secrets outside of GitOps and having to use a third-party tool like Vault, SealedSecrets provides a way to keep all the advantages of using a GitOps workflow while avoiding exposing secrets. SealedSecrets is composed of two main components:

- A CLI (Kubeseal) to encrypt secrets.
- A cluster-side controller to decrypt the sealed secrets into regular Kubernetes secrets. Only this controller can decrypt sealed secrets, not even the original author.
- This tutorial describes how to install these two components, configure the controller, and add or remove sealed secrets.

Deploying a Sample Application from NKP GitOps

Use this procedure to deploy a sample `podinfo` application from NKP Ultimate GitOps.

Before you begin

- Install NKP: For more information, see Installing NKP on page 50
- Github account and personal access token. For more information, see https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/creating-a-personal-access-token
- Add cluster to Kommander: For more information, see Attaching an Existing Kubernetes Cluster on page 493
- Setup Workspace and Projects: For more information, see Workspaces on page 411.

> **Note:** This procedure was run on an AWS cluster with NKP installed.

Follow these steps:

Procedure

1. Ensure you are on the **Default Workspace** (or other workspace you have access to) so that you can create a project.
2. Create a project, as described in Projects on page 439.In the working example we name the project **pod-info**. When you create a namespace, Kommander appends five alphanumeric characters. You can opt to select a target cluster for this project from one of the available attached clusters, and then this **(pod-info-xxxxx)** is the namespace used for deployments under the project.
3. [Optional] Create a secret in order to pull from the repository, for private repositories.
   a. Select the Secrets tab and set up your secret according to the Continuous Deployment on page 459 documentation.
   b. Add a **key** and **value** pair for the GitHub personal access token and then select **Create**.
4. Verify that the secret `podinfo-secret` is created on the project namespace in the managed or attached cluster.

```
kubectl get secrets -n pod-info-xt2sz --kubeconfig=${CLUSTER_NAME}.conf
```

```
NAME                          TYPE                                  DATA   AGE
default-token-k685t           kubernetes.io/service-account-token   3      94m
pod-info-xt2sz-token-p9k5z    kubernetes.io/service-account-token   3      94m
podinfo-secret                Opaque                                1      1s
tls-root-ca                   Opaque                                1      93m
```

5. Select your project and then select the **CD** tab.
6. Add a GitOps Source, complete the required fields, and then **Save**.
   There are several configurable options such as selecting the **Git Ref Type** but in this example we use the master branch. The **Path** value should contain where the manifests are located. Additionally, the **Primary Git Secret** is the secret (**podinfo-secret**) that you created in the previous step, if you need to access private repositories. This can be disregarded for public repositories.

7. Do the following.

   a. Verify the status of `gitrepository` creation with this command (on the attached or managed cluster), and if **READY** is marked as **True**.

```
kubectl get gitrepository -A --kubeconfig=${CLUSTER_NAME}.conf
```

```
NAMESPACE        NAME           URL
          AGE    READY   STATUS
kommander-flux   management     https://git-operator-git.git-operator-system.svc/
repositories/kommander/kommander.svc/kommander/kommander.git   134m   True
 stored artifact for revision 'main/4fbee486076778c85e14f3196e49b8766e50e6ce'
pod-info-xt2sz   podinfo-source https://github.com/stefanprodan/podinfo
                    116m   True    stored artifact for revision 'master/
b3b00fe35424a45d373bf4c7214178bc36fd7872'
```

8. Verify the `Kustomization` with this command below (on the attached or managed cluster), and if **READY** is marked as **True**.

```
kubectl get kustomizations -n pod-info-xt2sz --kubeconfig=${CLUSTER_NAME}.conf
```

```
NAME                 AGE    READY    STATUS
originalpodinfo       10m    True     Applied revision: master/
b3b00fe35424a45d373bf4c7214178bc36fd7872
podinfo-source        113m   True     Applied revision: master/
b3b00fe35424a45d373bf4c7214178bc36fd7872
project               116m   True     Applied revision:
 main/4fbee486076778c85e14f3196e49b8766e50e6ce
project-tls-root-ca   117m   True     Applied revision:
 main/4fbee486076778c85e14f3196e49b8766e50e6ce
```

   Note the

```
port
```

   so that you can use to verify if the app is deployed correctly (on the attached or managed cluster).

```
kubectl get deployments,services -n pod-info-xt2sz --kubeconfig=
${CLUSTER_NAME}.conf
```

```
NAME                       READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/podinfo    2/2      2             2            118m

NAME              TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)             AGE
service/podinfo   ClusterIP   10.99.239.120   <none>        9898/TCP,9999/TCP
 118m
```

9. Port forward the **podinfo** service (port **9898**) to verify (on the attached or managed cluster):

```
kubectl port-forward svc/podinfo -n pod-info-xt2sz 9898:9898 --kubeconfig=
${CLUSTER_NAME}.conf
```

```
Forwarding from 127.0.0.1:9898 -> 9898
Forwarding from [::1]:9898 -> 9898
Handling connection for 9898
Handling connection for 9898
Handling connection for 9898
```

10. Open a browser and type in **localhost:9898**. A successful deployment of the podinfo app gives you this page.

Continuous Deployment
After installing Kommander and configuring your project and its clusters, navigate to the Continuous Deployment (CD) tab under your Project.

Here you create a GitOps source which is a source code management (SCM) repository hosting the application definition. Nutanix recommends that you create a secret first then create a GitOps source accessed by the secret.

Setting Up a Secret for Accessing GitOps

You can create a secret that Kommander uses to deploy the contents of your GitOps repository.

About this task

> **Note:** This dialog box creates a `types.kubefed.io/v1beta1, Kind=FederatedSecret` and this is not yet supported by NKP CLI. Use the GUI, as described above, to create a federated secret or create a `FederatedSecret` manifest and apply it to the project namespace. For more information about secrets, see Project Secrets on page 470

Kommander secrets (for CD) can be configured to support any of the following three authentication methods:

- HTTPS Authentication (described above)
- HTTPS self-signed certificates
- SSH Authentication

The following table describes the fields required for each authentication method.

Procedure

Table 43: Secret in GitOps

HTTP Auth	HTTPS Auth (Self-signed)	SSH Auth
username	username	identity
password	password	identity.pub
	caFile	known_hosts

If you are using a GitHub personal access token, you do not need to have a key:value pair of `username`.

1. If you are using GitOps by using a GitHub repo as your source, you can create your secret with a personal access token. Then, in the NKP UI, in your project, create a Secret, with a key:value pair of `password`: `<your-token-created-on-github>`. If you are using a GitHub personal access token, you do not need to have a key:value pair of `username`: `<your-github-username>`.
2. If you are using a secret with your GitHub username and your password, you will need one secret created in the NKP UI, with key:value pairs of `username`: `<your-github-username>` and `password`: `<your-github-password>`.

   > **Note:** If you have multi-factor authentication turned on in your GitHub account, this will not work.

   > **Note:** Using a token without a username is valid for GitHub, but other providers (such as GitLab) require both username and tokens.

   > **Warning:** If you are using a public GitHub repository, you do not need to use a secret.

Creating the GitOps Source

After the secret is created, you can view it in the `Secrets` tab. Configure the GitOps source accessed by the secret.

About this task

> **Note:** If using an SSH secret, the SCM repo URL needs to be an SSH address. It does not support SCP syntax. The URL format is `ssh://user@host:port/org/repository`.

It takes a few moments for the GitOps Source to be reconciled and the manifests from the SCM repository at the given path to be federated to attached clusters. After the sync is complete, manifests from GitOps source are created in attached clusters.

After a GitOps Source is created, there are various commands that can be executed from the CLI to check the various stages of syncing the manifests.

Procedure

On the management cluster, check your `GitopsRepository` to ensure that the `CD` manifests have been created successfully.

```
kubectl describe gitopsrepositories.dispatch.d2iq.io -n<PROJECT_NAMESPACE> gitopsdemo
```

```
Name:         gitopsdemo
Namespace:    <PROJECT_NAMESPACE>
...
Events:
  Type    Reason               Age    From                         Message
  ----    ------               ----   ----                         -------
  Normal  ManifestSyncSuccess  1m7s  GitopsRepositoryController  manifests synced to
 bootstrap repo
...
```

On the attached cluster, check for your `Kustomization` and `GitRepository` resources. The `status` field reflects the syncing of manifests.

```
kubectl get kustomizations.kustomize.toolkit.fluxcd.io -n<PROJECT_NAMESPACE>
 <GITOPS_SOURCE_NAME> -oyaml
```

```
...
status:
  conditions:
  - reason: ReconciliationSucceeded
    status: "True"
    type: Ready
    ...
...
```

Similarly, with `GitRepository` resource.

```
kubectl get gitrepository.source.toolkit.fluxcd.io -n<PROJECT_NAMESPACE>
 <GITOPS_SOURCE_NAME> -oyaml
```

```
...
status:
  conditions:
  - reason: GitOperationSucceed
    status: "True"
    type: Ready
    ...
...
```

If there are errors creating the manifests, those events are populated in the status field of the `GitopsRepository` resource on the management cluster, the `GitRepository` and `Kustomization` resources on the attached cluster(s), or both.

Editing the GitOps Source

About this task

To edit the GitOPs Source from within the UI::

Procedure

1. From the top menu bar, select your target workspace.
2. Select **Projects** from the sidebar menu.
3. Select your project from the list.
4. Select the **Continuous Deployment (CD)** tab.
5. Select the **GitOps Sources** button.

   From here, you can edit the ID (name), Repository URL, Git Ref Type , **Branch Name**, **Type**, Path, and **Primary Git Secret**.

Suspending the GitOps Source

There may be times when you need to suspend the auto-sync between the GitOps repository and the associated clusters. This live debugging may be necessary to resolve an incident in the minimum amount of time without the overhead of pull request based workflows.

About this task

To Suspend the GitOps Source from the NKP UI:

Procedure

1. From the top menu bar, select your target workspace.
2. Select **Projects** from the sidebar menu.
3. Select your project from the list.
4. Select the **Continuous Deployment (CD)** tab.
5. Select the three dot button to the right of the desired GitOps Source.
6. **Suspend** to manually suspend the GitOps reconciliation.

   This lets you use `kubectl`, `helm`, or another tool to resolve the issue. After the issue is resolved select **Resume** to sync the updated contents of the GitOps source to the associated clusters.

   Similar to **Suspend/Resume**, you can use the **Delete** action to remove the GitOps source. Removing the GitOps source results in removal of all the manifests applied from the GitOps source.

   You can have more than one GitOps Source in your Project to deploy manifests from various sources.

   Kommander deployments are backed by FluxCD. For Flux docs for advanced configuration and more examples, see Source Controller at https://fluxcd.io/docs/components/source/ and Kustomize controller at https://fluxcd.io/docs/components/kustomize/.

Project Deployments Troubleshooting

- View the events and logs for `deployments/kommander-cm` in Kommander namespace, if there are any unexpected errors.
- Enabling the Kommander repository controller for your project namespace causes a number of related Flux controller components to deploy into the namespace. These are necessary for the proper operation of the repository controller and should not be removed. For more information, see https://toolkit.fluxcd.io/components/.
- Ensure your GitOps repository does not contain any manifests that are cluster-scoped - for example, `Namespace`, `ClusterRole`, `ClusterRoleBinding`, etc. All of the manifests must be namespace-scoped.
- Ensure your GitOps repository does not contain any `HelmRelease` and `Kustomization` resources that are targeting a different namespace than the project namespace.

Viewing Helm Releases

In addition to viewing the current GitOps Sources, you can also view the currently deployed Helm Releases.

Procedure

1. From the top menu bar, select your target workspace.
2. Select **Projects** from the sidebar menu.
3. Select your project from the list.
4. Select the **Continuous Deployment (CD)** tab.
5. Click **Helm Releases** .

   All of the current Helm Release charts are displayed with their Chart Version and the names of the clusters.

   Note: If an error occurs with the Helm Releases charts deployment, an “Install Failed” error status appears in the **Kommander Host** field .

   Select the error status to open a screen that details specific issues related to the error.

Project Role Bindings

Project Role Bindings grant access to a specified Project Role for a specified group of people.

Configuring Project Role Bindings Using the UI

Before you can create a Project Role Binding, ensure you have created a Group. A Kommander Group can contain one or several Identity Provider users or groups.

About this task

You can assign a role to this Kommander Group:

Procedure

1. From the **Projects** page, select your project.
2. Select the **Role Bindings** tab, then select **Add Roles** next to the group you want.
3. Select the **Role** you want from the dropdown list, and then click **Save**.

Configuring Project Role Bindings Using the CLI

Procedure

A Project role binding can also be created using kubectl.

```
cat << EOF | kubectl create -f -
apiVersion: workspaces.kommander.mesosphere.io/v1alpha1
kind: VirtualGroupProjectRoleBinding
metadata:
  generateName: projectpolicy-
  namespace: ${projectns}
spec:
  projectRoleRef:
    name: ${projectrole}
  virtualGroupRef:
    name: ${virtualgroup}
EOF
```

Configure Project Role Bindings to Bind to WorkspaceRoles Using the CLI

You can also create a Project role binding to bind to a WorkspaceRole in certain instances.

Procedure

1. To list the WorkspaceRoles that you can bind to a Project, run the following command.

   ```
   kubectl get workspaceroles -n ${workspacens} -o=jsonpath="{.items[?
   (@.metadata.annotations.workspace\.kommander\.d2iq\.io\/project-default-workspace-
   role-for==\"${projectns}\")].metadata.name}"
   ```

   You can bind to any of the above WorkspaceRoles by setting `spec.workspaceRoleRef` in the project role binding.

   ```
   cat << EOF | kubectl create -f -
   apiVersion: workspaces.kommander.mesosphere.io/v1alpha1
   kind: VirtualGroupProjectRoleBinding
   metadata:
     generateName: projectpolicy-
     namespace: ${projectns}
   spec:
     workspaceRoleRef:
       name: ${workspacerole}
     virtualGroupRef:
       name: ${virtualgroup}
   EOF
   ```

   Note that you must specify either workspaceRoleRef or projectRoleRef to be validated by the admission webhook. Specifying both values is not valid and will cause an error.

   Ensure the `projectns`, `workspacens`, `projectrole` (or `workspacerole`) and the `virtualgroup` variables are set before executing the command.

   When a Project Role Binding is created, Kommander creates a Kubernetes `FederatedRoleBinding` on the Kubernetes cluster where Kommander is running. You can view this by first finding the name of the project role binding that you created: `kubectl -n ${projectns} get federatedrolebindings.types.kubefed.io`.

   Then, view the details like in this example:

   ```
   kubectl -n ${projectns} get federatedrolebindings.types.kubefed.io projectpolicy-
   gtct4-rdkwq -o yaml
   ```

   Output.

   ```
   apiVersion: types.kubefed.io/v1beta1
   ```

```
kind: FederatedRoleBinding
metadata:
  creationTimestamp: "2020-06-04T16:19:27Z"
  finalizers:
  - kubefed.io/sync-controller
  generation: 1
  name: projectpolicy-gtct4-rdkwq
  namespace: project1-5ljs9-lhvjl
  ownerReferences:
  - apiVersion: workspaces.kommander.mesosphere.io/v1alpha1
    blockOwnerDeletion: true
    controller: true
    kind: VirtualGroupProjectRoleBinding
    name: projectpolicy-gtct4
    uid: 19614de2-4593-433e-82fa-96dc9470e07a
  resourceVersion: "196270"
  selfLink: /apis/types.kubefed.io/v1beta1/namespaces/project1-5ljs9-lhvjl/
federatedrolebindings/projectpolicy-gtct4-rdkwq
  uid: beaffc29-edec-4258-9813-3a17ba27a2a6
spec:
  placement:
    clusterSelector: {}
  template:
    roleRef:
      apiGroup: rbac.authorization.k8s.io
      kind: Role
      name: admin-dbfpj-l6s9g
    subjects:
    - apiGroup: rbac.authorization.k8s.io
      kind: User
      name: user1@d2iq.lab
status:
  clusters:
  - name: konvoy-5nr5h
  conditions:
  - lastTransitionTime: "2020-06-04T16:19:27Z"
    lastUpdateTime: "2020-06-04T16:19:27Z"
    status: "True"
    type: Propagation
  observedGeneration: 1
```

2. Then, if you run the following command on a Kubernetes cluster associated with the Project, you'll see a Kubernetes RoleBinding Object, in the corresponding namespace.

```
kubectl -n ${projectns} get rolebinding projectpolicy-gtct4-rdkwq -o yaml
```

Output:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  creationTimestamp: "2020-06-04T16:19:27Z"
  labels:
    kubefed.io/managed: "true"
  name: projectpolicy-gtct4-rdkwq
  namespace: project1-5ljs9-lhvjl
  resourceVersion: "125392"
  selfLink: /apis/rbac.authorization.k8s.io/v1/namespaces/project1-5ljs9-lhvjl/
rolebindings/projectpolicy-gtct4-rdkwq
  uid: 2938398d-437b-4f3a-9cb9-c92e50139196
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
```

```
  name: admin-dbfpj-l6s9g
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: user1@d2iq.lab
```

Role Binding with VirtualGroup
In NKP, `VirtualGroup` is a list of subjects that can be assigned to several different kinds of roles, including:

- `ClusterRole` for cluster-scoped objects
- `WorkspaceRole` for workspace-scoped objects
- `ProjectRole` for project-scoped objects

In order to define which `VirtualGroup`(s) is assigned to one of these roles, administrators can create corresponding role bindings such as `VirtualGroupClusterRoleBinding`, `VirtualGroupWorkspaceRoleBinding`, and `VirtualGroupProjectRoleBinding`.

Note that for `WorkspaceRole` and `ProjectRole`, the referenced `VirtualGroup` and corresponding role and role binding objects need to be in the same namespace. If they are not in the same namespace, the role will not bind to the `VirtualGroup` since it is assumed that the rules set in the role apply to objects that live in that namespace. Whereas for `ClusterRole` which is cluster-scoped, the `VirtualGroupClusterRoleBinding` is also cluster-scoped, even though it references a namespace-scoped `VirtualGroup`.

Project Roles

Project Roles are used to define permissions at the namespace level.

Configuring the Project Role Using the UI
You can create a Project Role with several Rules.

About this task

Create a project role with one or more rules.

Procedure

Create a Project Role with a single rule.

In this example, the Project Role corresponds to an admin role:

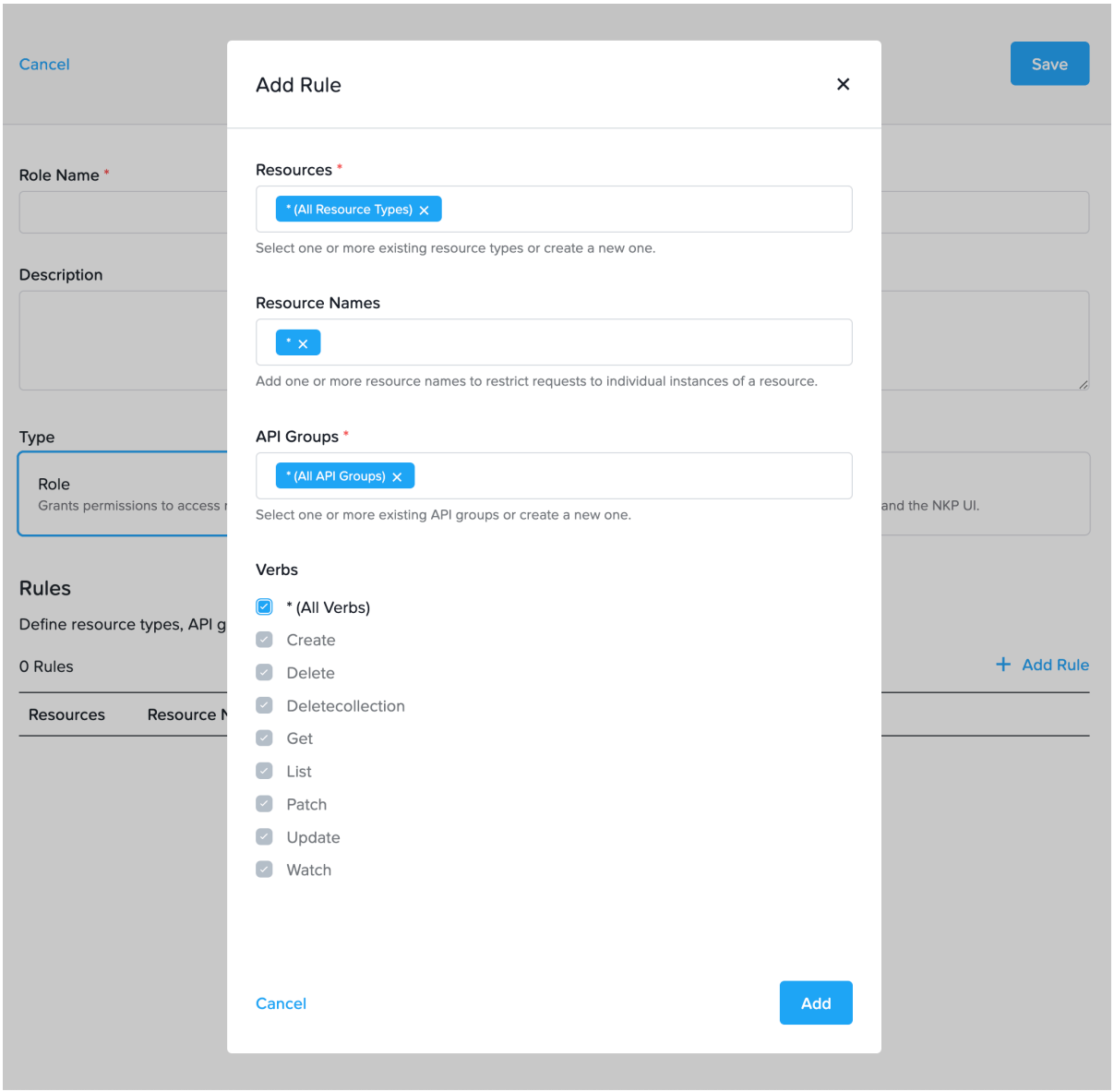


Figure 13: Adding a Project Role using the UI

Configuring the Project Role Using the CLI

About this task

In the example below, a Project Role is created with a single Rule. This Project Role corresponds to an admin role.

Procedure

1. The same Project Role can also be created using kubectl:

```
cat << EOF | kubectl create -f -
apiVersion: workspaces.kommander.mesosphere.io/v1alpha1
kind: ProjectRole
metadata:
  annotations:
    kommander.mesosphere.io/display-name: Admin
  generateName: admin-
```

```
  namespace: ${projectns}
spec:
  rules:
  - apiGroups:
    - '*'
    resources:
    - '*'
    verbs:
    - '*'
EOF
```

Note: Ensure the `projectns` variable is set before executing the command.

2. You can set it using the following command (for a Kommander Project called `project1`, and after setting the `workspacens` as explained in the previous section).

```
projectns=$(kubectl -n ${workspacens} get projects.workspaces.kommander.mesosphere.io
 -o jsonpath='{.items[?
(@.metadata.generateName=="project1-")].status.namespaceRef.name}')
```

When a Project Role is created, Kommander creates a Kubernetes `FederatedRole` on the Kubernetes cluster where Kommander is running.

```
kubectl -n ${projectns} get federatedroles.types.kubefed.io admin-dbfpj-l6s9g -o yaml
apiVersion: types.kubefed.io/v1beta1
kind: FederatedRole
metadata:
  creationTimestamp: "2020-06-04T11:54:26Z"
  finalizers:
  - kubefed.io/sync-controller
  generation: 1
  name: admin-dbfpj-l6s9g
  namespace: project1-5ljs9-lhvjl
  ownerReferences:
  - apiVersion: workspaces.kommander.mesosphere.io/v1alpha1
    blockOwnerDeletion: true
    controller: true
    kind: ProjectRole
    name: admin-dbfpj
    uid: e5f3b2ca-16bf-474d-8305-7be04c034793
  resourceVersion: "75680"
  selfLink: /apis/types.kubefed.io/v1beta1/namespaces/project1-5ljs9-lhvjl/
federatedroles/admin-dbfpj-l6s9g
  uid: 1e5a3d98-b223-4605-bba1-16276a3eb47c
spec:
  placement:
    clusterSelector: {}
  template:
    rules:
    - apiGroups:
      - '*'
      resourceNames:
      - '*'
      resources:
      - '*'
      verbs:
      - '*'
status:
  clusters:
  - name: konvoy-5nr5h
  conditions:
  - lastTransitionTime: "2020-06-04T11:54:26Z"
    lastUpdateTime: "2020-06-04T11:54:26Z"
```

```
    status: "True"
    type: Propagation
  observedGeneration: 1
```

3. Then, if you run the following command on a Kubernetes cluster associated with the Project, you see a Kubernetes Role object in the corresponding namespace.

```
kubectl -n ${projectns} get role admin-dbfpj-l6s9g -o yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  creationTimestamp: "2020-06-04T11:54:26Z"
  labels:
    kubefed.io/managed: "true"
  name: admin-dbfpj-l6s9g
  namespace: project1-5ljs9-lhvjl
  resourceVersion: "29218"
  selfLink: /apis/rbac.authorization.k8s.io/v1/namespaces/project1-5ljs9-lhvjl/roles/
admin-dbfpj-l6s9g
  uid: f05b998c-4649-4e73-bbfe-c12bc4c86a3c
rules:
- apiGroups:
  - '*'
  resourceNames:
  - '*'
  resources:
  - '*'
  verbs:
  - '*'
```

Project ConfigMaps

Use ConfigMaps to automate ConfigMaps creation on your clusters

Project ConfigMaps can be created to make sure Kubernetes ConfigMaps are automatically created on all Kubernetes clusters associated with the Project, in the corresponding namespace.

As reference, a ConfigMap is a key-value pair to store some type of non-confidential data like “name=bob” or “state=CA”. For a full reference to the concept, consult the Kubernetes documentation on the topic of ConfigMaps. For more information, see https://kubernetes.io/docs/concepts/configuration/configmap/.

Configuring Project ConfigMaps Using the UI

About this task

The below Project ConfigMap form can be navigated to by:

Procedure

1. From the top menu bar, select your target workspace.
2. Select **Projects** from the sidebar menu.
3. Select your project from the list.
4. Select the **ConfigMaps** tab to browse the deployed ConfigMaps.
5. Click **+ Create ConfigMap** .
6. Enter an ID, Description and Data for the ConfigMap, and click **Create**.

Configuring Project ConfigMaps Using the CLI

Procedure

1. A Project ConfigMap is simply a Kubernetes FederatedConfigMap and can be created using kubectl with YAML.

```
cat << EOF | kubectl create -f -
apiVersion: types.kubefed.io/v1beta1
kind: FederatedConfigMap
metadata:
  generateName: cm1-
  namespace: ${projectns}
spec:
  placement:
    clusterSelector: {}
  template:
    data:
      key: value
EOF
```

   Note: Ensure the `projectns` variable is set before executing the command. This variable is the project namespace (the Kubernetes Namespace associated with the project) that was defined/created when the project itself was initially created.

```
projectns=$(kubectl -n ${workspacens} get
 projects.workspaces.kommander.mesosphere.io -o jsonpath='{.items[?
(@.metadata.generateName=="project1-")].status.namespaceRef.name}')
```

2. Then, if you run the following command on a Kubernetes cluster associated with the Project, you'll see a Kubernetes ConfigMap Object, in the corresponding namespace.

```
kubectl -n ${projectns} get configmap cm1-8469c -o yaml
apiVersion: v1
data:
  key: value
kind: ConfigMap
metadata:
  creationTimestamp: "2020-06-04T16:37:10Z"
  labels:
    kubefed.io/managed: "true"
  name: cm1-8469c
  namespace: project1-5ljs9-lhvjl
  resourceVersion: "131844"
  selfLink: /api/v1/namespaces/project1-5ljs9-lhvjl/configmaps/cm1-8469c
  uid: d32acb98-3d57-421f-a677-016da5dab980
```

Project Secrets

Project Secrets can be created to make sure a Kubernetes Secrets are automatically created on all the Kubernetes clusters associated with the Project, in the corresponding namespace.

Configuring the Project Secrets Using the UI

About this task

Context for the current task

Procedure

1. Select the workspace your project was created in from the workspace selection dropdown in the header.

2. In the sidebar menu, select **Projects**.
3. Select the project you want to configure from the table.
4. Select the **Secrets** tab, and then click **Create Secret** .
5. Complete the form and click **Create**.

Configuring the Project Secrets Using the CLI

Procedure

1. A Project Secret is simply a Kubernetes FederatedConfigSecret and can also be created using kubectl.

```
cat << EOF | kubectl create -f -
apiVersion: types.kubefed.io/v1beta1
kind: FederatedSecret
metadata:
  generateName: secret1-
  namespace: ${projectns}
spec:
  placement:
    clusterSelector: {}
  template:
    data:
      key: dmFsdWU=
EOF
```

   Ensure the `projectns` variable is set before executing the command.

```
projectns=$(kubectl -n ${workspacens} get projects.workspaces.kommander.mesosphere.io
 -o jsonpath='{.items[?
(@.metadata.generateName=="project1-")].status.namespaceRef.name}')
```

   Note: The value of the key is base64 encoded.

2. If you run the following command on a Kubernetes cluster associated with the Project, you see a Kubernetes Secret Object, in the corresponding namespace.

```
kubectl -n ${projectns} get secret secret1-r9vk2 -o yaml
apiVersion: v1
data:
  key: dmFsdWU=
kind: Secret
metadata:
  creationTimestamp: "2020-06-04T16:51:59Z"
  labels:
    kubefed.io/managed: "true"
  name: secret1-r9vk2
  namespace: project1-5ljs9-lhvjl
  resourceVersion: "137215"
  selfLink: /api/v1/namespaces/project1-5ljs9-lhvjl/secrets/secret1-r9vk2
  uid: e5c6fc1d-93e7-47fe-ae1e-f418f8e35d72
type: Opaque
```

Project Quotas and Limit Ranges

Project Quotas and Limit Ranges can be set up to limit the number of resources the Project team uses.

Creating Project Quotas and Limit Ranges Using the UI

About this task

Project Quotas and Limit Ranges can be set up to limit the number of resources the Project team uses. Quotas and Limit Ranges are applied to all project clusters.

Procedure

1. Select the workspace your project was created in from the workspace selection dropdown in the header.
2. In the sidebar menu, select **Projects**.
3. Select the project you want to configure from the table.
4. Select the **Quotas & Limit Ranges** tab, and then select **Edit**.

   Kommander provides a set of default resources for which you can set Quotas. You can also define Quotas for custom resources. We recommend that you set Quotas for CPU and Memory. By using Limit Ranges, you can restrict the resource consumption of individual Pods, Containers, and Persistent Volume Claims in the project namespace. You can also constrain memory and CPU resources consumed by Pods and Containers, and storage resources consumed by Persistent Volume Claims.
5. To add a custom quota, scroll to the bottom of the form and select **Add Quota**.
6. When you are finished, click **Save**.

Creating Project Quotas and Limit Ranges Using the CLI

Procedure

1. All the Project Quotas are defined using a Kubernetes FederatedResourceQuota called `kommander` which you can also create/update using kubectl.

   ```
   cat << EOF | kubectl apply -f -
   apiVersion: types.kubefed.io/v1beta1
   kind: FederatedResourceQuota
   metadata:
     name: kommander
     namespace: ${projectns}
   spec:
     placement:
       clusterSelector: {}
     template:
       spec:
         hard:
           limits.cpu: "10"
           limits.memory: 1024.000Mi
   EOF
   ```

   Ensure the `projectns` variable is set before executing the command.

   ```
   projectns=$(kubectl -n ${workspacens} get projects.workspaces.kommander.mesosphere.io
    -o jsonpath='{.items[?
   (@.metadata.generateName=="project1-")].status.namespaceRef.name}')
   ```
2. Then, if you run the following command on a Kubernetes cluster associated with the Project, you'll see a Kubernetes Resource Quota in the corresponding namespace.

   ```
   kubectl -n ${projectns} get resourcequota kommander -o yaml
   apiVersion: v1
   kind: ResourceQuota
   metadata:
   ```

```
  creationTimestamp: "2020-06-05T08:04:37Z"
  labels:
    kubefed.io/managed: "true"
  name: kommander
  namespace: project1-5ljs9-lhvjl
  resourceVersion: "470822"
  selfLink: /api/v1/namespaces/project1-5ljs9-lhvjl/resourcequotas/kommander
  uid: 925b61b4-134b-4c45-915c-96a05b63d3c3
spec:
  hard:
    limits.cpu: "10"
    limits.memory: 1Gi
status:
  hard:
    limits.cpu: "10"
    limits.memory: 1Gi
  used:
    limits.cpu: "0"
    limits.memory: "0"
```

Similarly, Project Limit Ranges are defined using a FederatedLimitRange object with name `kommander` in the project namespace.

```
cat << EOF | kubectl apply -f -
apiVersion: types.kubefed.io/v1beta1
kind: FederatedLimitRange
metadata:
  name: kommander
  namespace: ${projectns}
spec:
  placement:
    clusterSelector: {}
  template:
    spec:
      limits:
      - type: "Pod"
        max:
          cpu: 500m
          memory: 50Gi
        min:
          cpu: 100m
          memory: 10Gi
      - type: "Container"
        max:
          cpu: 2
          memory: 100Mi
        min:
          cpu: 1
          memory: 10Mi
      - type: "PersistentVolumeClaim"
        max:
          storage: 3Gi
        min:
          storage: 1Gi
EOF
```

Project Network Policies

Projects are created with a secure-by-default network policy and users needing more flexibility can edit or add more policies to tailor to their unique security needs.

Cluster networking is a critical and central part of Kubernetes that can also be quite challenging. All network communication within and between clusters depends on the presence of a Container Network Interface (CNI) plugin.

By default, and to enable fluid communications within and between clusters, all traffic is authorized between nodes and pods. Most production environments require some kind of traffic flow control at the IP address or port level. An application-centric approach to this uses Network Policies. Pods are isolated by having a Network Policy that selects them, and the configuration of the policy limits the kind of traffic they can receive or send. Network policies do not conflict because they are additive. Pods selected by more than one policy are subject to the union of the policies' ingress and egress rules.

A Network Policy's rules define ingress and egress for network communications between pods and across namespaces. Successful traffic control using network policies is bi-directional. You have to configure both the egress policy on the source pod and the ingress policy on the destination pod to enable the traffic. If either end denies the traffic, it will not flow between the pods.

Since the Kubernetes default is to allow all traffic, it is a common practice to create a default "deny all traffic" rule, and then specifically open up some combination of the pods, ports, and applications as needed.

Network Plugins

Network plugins ensure that Kubernetes networking requirements are met and surface features needed by network administrators, such as enforcing network policies. Common Network Plugins include Flannel, Calico, and Weave among many others. As an example, Nutanix Konvoy uses the Calico CNI plugin by default, but can support others. Nutanix AHV uses Cilium instead of Calico.

Since pods are short-lived, the cluster needs a way to configure the network dynamically as pods are created and destroyed. Plugins provision and manage IP addresses to interfaces and let administrators manage IPs and their assignments to containers, in addition to connections to more than one host, when needed.

Network Policies

You can create network policies in three main parts:

- General information
- Ingress rules
- Egress rules

General Information section

The fields in this part of the form allow you to create a name and description for this policy. Creating a detailed **Description** helps to keep policy functions understandable for additional use and maintenance.

This section also contains the **Pod Selector** fields for selecting pods using either Labels or Expressions. **Labels** added to pod declarations are a common means of identifying individual pods, or creating groups of pods, in a Kubernetes cluster. **Expressions** are similar to Labels, but allow you to define parameters that identify a range of pods.

The **Policy Types** selections help to define the type of Network Policy you are creating:

- **Default** - automatically includes ingress, and egress is set only if the network policy defines egress rules.
- **Ingress** - this policy applies to ingress traffic for the selected pods, to namespaces using the options you define below, or both.
- **Egress** - this policy applies to egress traffic for the selected pods, to namespaces using the options you define below, or both.

If the Default policy type is too rigid or does not offer what you need, you can select the Ingress or Egress type, or both, and explicitly define the policy with the options that follow. For example, if you do not want this policy to apply to ingress traffic, you only select Egress, and then define the policy.

To deny all ingress traffic, select the **Ingress** option here and then leave the ingress rules empty.

To deny all egress traffic, select the **Egress** option here and then leave the egress rules empty.

Ingress rules section

Ingress rules use a combination of **Port**/ **Protocol** and **Source** to define the incoming traffic allowed to some or all of the pods in this namespace.

The options under **Sources: From** enable you to define a source either by using the pod selector or by defining an IP block. When using the pod selector method, you can define the namespace, the pods within that namespace, or both.

Namespaces - Selecting a namespace in an ingress rule source permits the pods selected by the pod selector, in your selected namespaces, to receive incoming traffic that meets the other defined criteria. If you have not selected any pods, the rule permits traffic from all pods in the selected namespaces.

Pods - This option selects specific Pods which should be allowed as ingress sources or egress destinations. If you have not selected any namespaces in the namespace selector, this option selects all matching pods in the project namespace. Otherwise, this option selects all matching pods in the selected namespaces.

There also are options to select all namespaces, all pods, or both.

When defining ingress rules using the IP Block method, you define a CIDR and exception conditions. CIDR stands for Classless Inter-Domain Routing and is an IP standard for creating unique network and device identifiers. When grouped together so that they share an initial sequence of bits in their binary representation, the range of addresses creates a CIDR block. The block identity is in an IPv4-like notation including a dotted-decimal address, followed by a slash, then a colon and a number from 0 through 32, for example, 127.0.26.33:31.

Egress rules section

Egress rules use a similar combination of options to define the outgoing traffic from pods, ranges of pods, or namespaces in a Kommander Project. Port, Protocol, and Destination options for egress rules define the outgoing traffic. You can define your egress rules under **Destination: To**. Ensure the egress policy on the source pods, and the ingress policy on the destination pods, permit traffic in order for the pods to be able to communicate over the network.

Network Policy Examples

Before you begin each example, ensure you're on the Network Policy page for your project

- Navigating to the Network Policy Page on page 475
- Ingress: Permit Access to API Service Pods from All Namespaces on page 476
- Ingress: Limit Pods That Access a Database to a Namespace on page 477
- Ingress: Disable But Not Delete Ingress Rules on page 478
- Egress: Deny all Egress Traffic from Restricted Pods on page 478

Navigating to the Network Policy Page

About this task

To navigate to your project's Network Policy page:

Procedure

1. From the top menu bar, select your target workspace.
2. Select **Projects** from the sidebar menu.
3. Select your project from the list.
4. Select the **Network Policy** tab.

Ingress: Permit Access to API Service Pods from All Namespaces

Suppose you need to create a network policy to permit incoming traffic to API service pods in a specific Kommander Project's namespace from any other pod in any namespace that has the label, `service.corp/users-api-role: client`. For this example, API service pods are those pods created with the Label, `service.corp/users-api-role: api`.

You can limit the policy to just incoming traffic from select namespaces by adding an ingress rule with these characteristics:

- Use Port 8080 to receive incoming TCP traffic
- Refuse traffic from pods unless they are client pods that have a specific Label, such as `service.corp/users-api-role: client`. This example follows a common microservice architecture pattern, microservice-tier-role: access_mode

Configuring General Information to Access the API Service Pods

Procedure

1. Select **+ Create Network Policy** .
2. Type "microsvc-users-api-allow" in the **ID Name** field.
3. Type "Allow Users microservice clients to reach the APIs provided in this namespace" in the **Description** field.
4. Select **Add** under **Pod Selector** and then select **Match Label**.
5. Set the **Key** to "service.corp/users-api-role" and the **Value** to "API".

Creating an Ingress Rule to Access API Service Pods

Procedure

1. Leave **Policy Types** set to Default.
2. Scroll down to **Ingress Rules** and select **+ Add an Ingress Rule**.
3. Select **+ Add Port**, and set the **Port** to "8080" and the **Protocol** to TCP.

Adding Sources to Access API Service Pods

Procedure

1. Select **+ Add Source** and mark the **Select All Namespaces**check box.
2. Select **+ Add Pod Selector**.

3. Select **Match Label**.
4. Set the **Key** value to “service.corp/users-api-role” and set the **Value** to “client”.
5. Scroll up and click **Save**.

Ingress: Limit Pods That Access a Database to a Namespace

Suppose that while deploying an application in a project, you want to protect its database pods by permitting ingress only from API service pods in the current namespace, and prevent ingress from pods in any other namespace.

You can limit the database pods to just the incoming traffic from the current namespaces by adding an ingress rule with these characteristics:

- Use Port 3306 to receive incoming TCP traffic for pods that have the label, `tier: database`
- Refuse traffic from pods unless they have the label, `tier: api`

Configuring General Information to Access a Database

About this task

To configure general information to access a database:

Procedure

1. Select **+ Create Network Policy** .
2. Type “database-access-api-only” in the **ID Name** field.
3. Type “Allow MySQL access only from API pods in this namespace” in the **Description** field.
4. Select **Add** under **Pod Selector** and then select **Match Label**.
5. Set the **Key** to “tier” and the **Value** to “database”.

Creating an Ingress Rule to Access a Database

Procedure

1. Leave **Policy Types** set to Default.
2. Scroll down to **Ingress Rules** and select **+ Add an Ingress Rule**.
3. Select **+ Add Port**, and set the **Port** to "3306" and the **Protocol** to TCP.

Adding Sources to Access a Database

Procedure

1. Select **+ Add Source**.
2. Select **+ Add Pod Selector**.
3. Select **Match Label**.
4. Set the **Key** value to “tier” and set the **Value** to “database”.

5. Scroll up and click **Save**.

Ingress: Disable But Not Delete Ingress Rules
Suppose that you want to disable ingress rules temporarily for testing or triaging purposes.

First, you need to create a network policy with one or more ingress rules. Follow one of the preceding procedures. Then, edit the policy to match the following example:

Editing Your Network Policy

Procedure

In the table row belonging to your network policy, click the context menu at the right of the row and select **Edit**.

Disabling Ingress Rules

Procedure

1. Update the **Policy Types** so that only **Egress** is selected. If you don't want to deny all egress traffic, ensure that you add an egress rule that suits your preferred level of access. You can add an empty rule to allow all egress traffic.
2. Scroll up and click **Save**.

Egress: Deny all Egress Traffic from Restricted Pods
Suppose that you need to deny all egress traffic from a group of restricted pods. This is a simple egress rule and you can create it following this example and steps:

Configuring General Information to Deny Egress

Procedure

1. Select **+ Add Network Policy** .
2. Type "deny-restricted-egress" in the **ID Name** field.
3. Type "Deny egress traffic from restricted pods" in the **Description** field.
4. Select **Add** under **Pod Selector** and then select **Match Label**.
5. Set the **Key** to "access" and the **Value** to "restricted".

Denying Egress Traffic

Procedure

1. Update the **Policy Types** so that only **Egress** is selected. Do not add any egress rules.
2. Scroll up and click **Save**.

Cluster Management

View clusters created with Kommander or any connected Kubernetes cluster

Kommander allows you to monitor and manage very large numbers of clusters. Use the features described in this area to connect existing clusters, or to create new clusters whose life cycle is managed by Konvoy. You can view clusters from the Clusters tab in the navigation pane on the left. You can see the details for a cluster by selecting the **View Details** link at the bottom of the cluster card or the cluster name in either the card or the table view.

Creating a Managed Nutanix Cluster Through the NKP UI

You can use the NKP UI to provision a Nutanix cluster quickly and easily.

About this task

Provisioning a production-ready cluster requires you to specify a number of parameters. Breaking up the form sections, as done in this documentation section, makes it a little easier to complete.

Before you begin

Ensure you have fulfilled the Prism Central requirements. To Provision a Nutanix Cluster, you must also create a Nutanix infrastructure before you can create additional clusters.

> **Caution:** Before provisioning a managed cluster, ensure your network allows seamless access from the management cluster to the Prism Central (PC) environment to avoid provisioning issues.

Select the View Details link (on the cluster card's bottom left corner) to see additional information about this cluster.

Procedure

Open the NKP UI.

What to do next

See the topic Specifying Nutanix Cluster Information.

Specifying Nutanix Cluster Information

About this task

In the section of the provisioning form, you give the cluster a name and provide some basic information:

Procedure

1. In the selected workspace Dashboard, select the **Add Cluster** button at the top right to display the Add Cluster page.
2. Select the **Create Cluster** card.

3. Provide these cluster details in the form:

   - **Cluster Name**: A valid Kubernetes name for the cluster.
   - **Add Labels**: Add any required Labels the cluster needs for your environment by selecting the `+ Add Label` link.

     Adding a cluster label might add the cluster to NKP projects.
   - **Infrastructure Provider**: This field's value corresponds to the Nutanix infrastructure provider you created.
   - **Kubernetes Version**: Select a supported version of Kubernetes for this version of NKP.
   - **SSH Username**: This field corresponds to the name of the SSH user to create or use. Leaving this field blank creates the default user name 'konvoy' for the specified SSH public key.
   - **SSH Public Key**: Paste the SSH public key that specifies the user's authorized key.

Configuring Nutanix Node Pool Information

About this task

You must configure node pool information for your control plane and worker nodes. The form splits these information sets into two groups.

Procedure

1. Provide the control plane node pool name and resource sizing information.

   - **Nutanix Prism Project**: Select Nutanix Prism Project. Selecting this will associate all the NKP control plane node virtual machines to this project.
   - **Nutanix AOS Cluster**: Nutanix AOS cluster is used to host the control plane virtual machines. If a Nutanix Prism Project is chosen, only the AOS clusters linked to that project will be shown.
   - **Subnet**: Subnet used for the control plane nodes. Subnet must be precreated in Nutanix Prism Central. If a Nutanix Prism project is chosen, only the subnets associated with the Nutanix Prism Project will be shown. At least one subnet must be selected.
   - **OS Image**: Select an NKP compliant OS image uploaded in Prism Central for the Kubernetes control plane nodes.
   - **Nutanix Prism Categories**: Add additional Nutanix Prism Categories to the NKP control plane VMs. The categories must exist in Prism Central.
   - **Control Plane Endpoint IP** : Endpoint IP is used for the NKP Kubernetes API VIP.

     Note: This IP address should not belong to a DHCP range or Nutanix IPAM address pool.

   - **Control plane Endpoint port** : Port used for NKP Kubernetes API server. By default the value is 6443.
   - **Control plane Node Count** : This field corresponds to the amount of control plane nodes for the NKP cluster. The default value is 3, other options are 1 or 5.

     Caution: Do not use a single-node control plane in a production cluster. Recommended is 3 or more.

   - **CPU per Node (vCPU)** : The amount of vCPUs per control plane node.
   - **Memory per node (GiB)** : The amount of Memory per control plane node (in GiB).
   - **Disk Size per node (GiB)** : The amount of disk size per control plane node (in GiB).

     Note: When you select a project, AOS cluster, subnets, and images in the control plane section, these selections will automatically populate the worker node pool section. This eliminates the need to input the same information twice manually. However, if desired, you can modify these selections for the worker node pool.

2. Provide the worker node pool name and resource sizing information.

   - **Nutanix Prism Project**: Select Nutanix Prism Project. Selecting this will associate all the NKP worker node virtual machines to this project.
   - **Nutanix AOS Cluster**: Nutanix AOS cluster is used to host the control plane virtual machines. If a Nutanix Prism Project is chosen, only the AOS clusters linked to that project will be shown.
   - **Subnet**: Subnet used for the control plane nodes. Subnet must be precreated in Nutanix Prism Central. If a Nutanix Prism project is chosen, only the subnets associated with the Nutanix Prism Project will be shown. At least one subnet must be selected.
   - **OS Image**: Select an NKP compliant OS image uploaded in Prism Central for the Kubernetes worker nodes.
   - **Nutanix Prism Categories**: Add additional Nutanix Prism Categories to the NKP worker node VMs. The categories must exist in Prism Central.
   - **Worker node Autoscaling**: Enable or disable worker node autoscaling. If enabled, NKP will automatically add or remove nodes based on workload demands. This is disabled by default.
     - **If worker node autoscaling is enabled** :
       - **Minimum number of nodes**: Minimum amount of worker nodes in the worker node pool.
       - **Maximum number of nodes**: Maximum amount of worker nodes in the worker node pool.
     - **If worker node autoscaling is disabled:**:
       - **Worker node count:**: The amount of worker nodes in the node pool.
   - **CPU per Node (vCPU)**: The amount of vCPUs per worker node.
   - **Memory per node (GiB)**: The amount of Memory per worker node (in GiB).
   - **Disk Size per node (GiB)**: The amount of disk size per worker node (in GiB).

3. Provide the Storage information.

   - **Hypervisor attached Volumes**: The hypervisor attached Nutanix Volume uses the hypervisor internal network for data traffic instead of external iSCSI connections. Enabled by default.
   - **Nutanix Storage container**: The Storage container is used for the Nutanix Volumes. Nutanix Storage container must be pre-created in Prism Central.
   - **Reclaim Policy**: This field corresponds to the Reclaim policy for the persistent volumes. The allowed values are Retain and Delete. Default is Delete.

4. Provide the Networking information.

   - **Pod Network**: The Kubernetes Pod network CIDR to use in the cluster (Default is 192.168.0.0/16).

     Note: Ensure the CIDRs do not overlap with your host subnets because they cannot be changed after cluster creation.

   - **Service Network**: The Kubernetes Service CIDR to use in the cluster (Default is 10.96.0.0/12).

     Note: Ensure the CIDRs do not overlap with your host subnets because they cannot be changed after cluster creation.

   - **Service load balancer start IP**: Enter the first IP address in the private range you're allocating for load balancing.

     Note: These IP addresses should not belong to a DHCP range or Nutanix IPAM address pool.

   - **Service load balancer end IP**: Enter the last IP address in the private range you're allocating for load balancing.

     Note: These IP addresses should not belong to a DHCP range or Nutanix IPAM address pool.

5. Provide the Image registries information.

   - **Image Registry Mirror**: Use an image registry mirror as a local copy of public registries. Defining a mirror registry is recommended if you have an air-gapped environment or want to avoid restrictions with firewalls.

     - **URL**: Enter the valid URL for the image registry mirror.
     - **Username**: Enter the Username for the registry mirror.
     - **Password**: Enter the password for the registry mirror.
     - **CA certificate**: Enter the CA certificate for the registry mirror. This is required in case of self-signed certificates.

   - **Private Registry**: Use a private image registry for your application images.

     - **URL**: Enter the valid URL for the image registry mirror.
     - **Username**: Enter the Username for the registry mirror.
     - **Password**: Enter the password for the registry mirror.
     - **CA certificate**: Enter the CA certificate for the registry mirror. This is required in case of self-signed certificates.

Creating a Managed Azure Cluster Through the NKP UI

NKP UI allows you to provision an Azure cluster from your browser.

Before you begin

Before you provision an Azure cluster using the NKP UI, you must first create an Azure infrastructure provider. For more information on how to hold your Azure credentials, see Creating an Azure Infrastructure Provider in the UI on page 387.

Follow these steps to provision an Azure cluster:

Procedure

1. From the top menu bar, select your target workspace.
2. Select **Clusters** > **Add Cluster**.
3. Choose **Create Cluster**.
4. Enter the **Cluster Name**.
5. From Select Infrastructure Provider, choose the provider created in the prerequisites section.
6. If available, choose a **Kubernetes Version**. Otherwise, the review of the Supported Kubernetes Versions. installs.
7. Select a datacenter **location** or specify a custom **location**.
8. Edit your worker **Node Pools** as necessary. You can choose the **Number of Nodes**, the **Machine Type**, and for the worker nodes, you can choose a **Worker Availability Zone**.
9. Add any additional **Labels** or **Infrastructure Provider Tags** as necessary.
10. Review your inputs to ensure they meet the predefined criteria, and select **Create**.

    Note: It can take up to 15 minutes for your cluster to appear in the **Provisioned** status.

    You are then redirected to the **Clusters** page, where you'll see your new cluster in the **Provisioning** status. Hover over the status to view the details.

Creating a Managed vSphere Cluster Through the NKP UI

You can use the NKP UI to provision a vSphere cluster quickly and easily.

About this task

Provisioning a production-ready cluster in vSphere requires you to specify a fairly large number of parameters. Breaking up the sections of the form, as done below, makes it a little easier to complete.

Before you begin

Before you begin these procedures, ensure that you have fulfilled the vSphere vCenter configuration prerequisites described in vSphere Prerequisites.

Note:

You must also create a vSphere infrastructure provider before you can create additional vSphere clusters.

To Provision a vSphere Cluster.

Procedure

Complete these procedures to provision a vSphere cluster.

- Provide Basic Cluster Information
- Specifying the Cluster Resources and Network Information on page 485
- Configure Node Pool Information
- Set Virtual IP Parameters

- Supply MetalLB Information
- Configure the StorageClass Options
- Advanced Configuration Parameters

What to do next

Select the **View Details** link (on the cluster card's bottom left corner) to see additional information about this cluster.

Specifying Basic Cluster Information

About this task

In the section of the provisioning form, you give the cluster a name and provide some basic information:

Procedure

1. In the selected workspace Dashboard, select the **Add Cluster** button at the top right to display the Add Cluster page.
2. Select the **Create Cluster** card.
3. Provide these cluster details in the form:
   - **Cluster Name**: A valid Kubernetes name for the cluster.
   - **Add Labels**: Add any required Labels the cluster needs for your environment by selecting the `+ Add Label` link.

     By default, your cluster has labels that reflect the infrastructure provider provisioning. For example, your vSphere cluster might have a label for the datacenter and `provider: vsphere`. Cluster labels are matched to the selectors created for Projects. Changing a cluster label might add or remove the cluster from projects. For more information, see Projects on page 439.
   - **Infrastructure Provider**: The value in this field corresponds to the vSphere infrastructure provider you created while fulfilling the prerequisites.
   - **Kubernetes Version**: Select a supported version of Kubernetes for this version of NKP.
   - **SSH Public Key**: Paste into this field the public key value for a user who is authorized to create the public key value for a user who is authorized to create vSphere clusters into this field clusters.
   - **Workspace**: The workspace where this cluster belongs (if within the Global works).

Specifying the Cluster Resources and Network Information

About this task

This section of the form identifies resources already present in your VMware vCenter configuration. Refer to your vCenter configuration to find the necessary values.

Procedure

1. Provide the following values for the **Resources** that are specific to vSphere.

   - **Datacenter**: Select an existing data center name.

     The datacenter is the top level organizational unit in vSphere.

   - **Datastore**: Enter a valid vSphere datastore name.

     Datastores in vSphere are storage resources that provide storage infrastructure for virtual machines within a datacenter. They are a subset of datacenter resources, with each datastore being associated with a specific datacenter.

   - **Folder**: Enter a valid, existing folder name, or leave it blank to use the vSphere root folder.

     When provisioning a Kubernetes cluster on vSphere using Cluster API and `clusterctl`, vSphere uses the folder parameter to specify the vSphere folder where it creates and manages the virtual machines for the Kubernetes cluster. Specifying the folder helps maintain an organized inventory of your virtual machines and other resources in your vSphere environment.

2. Enter the values for the network information in the lower half of this section.

   - **Network**: Enter an existing network name you want the new cluster to use.

     You need to create required network resources, such as port groups or distributed port groups, in the vSphere Client or using the vSphere API before you use NKP to create a new cluster.

   - **Resource Pool**: Enter the name of a logical resource pool for the cluster's resources.

     In vSphere, resource pools are a logical abstraction that allows you to allocate and manage computing resources, such as CPU and memory, for a group of virtual machines. Use resource pools only when needed, as they can add complexity to your environment.

   - **Virtual Machine Template**: Enter the name of the virtual machine template to use for the managed cluster's virtual machines.

     In vSphere, a virtual machine (VM) template is a pre-configured virtual machine that you can use to create new virtual machines with identical configurations quickly. The template contains the basic configuration settings for the VM, such as the operating system, installed software, and hardware configurations.

   - **Storage Policy**: Enter the name of a valid vSphere storage policy. This field is optional.

     A storage policy in vSphere specifies the storage requirements for virtual machine disks and files. It consists of a rule set that defines the storage capabilities required, tags to identify them, profiles that collect settings and requirements, and storage requirements that include storage performance, capacity, redundancy, and other attributes necessary for the virtual machine to function properly. By creating and applying a storage policy to a specific datastore or group of datastores, you can ensure that virtual machines using that datastore meet the specified storage requirements.

Configuring Node Pool Information

About this task

You need to configure node pool information for both your control plane nodes and your worker nodes. The form splits these information sets into two groups.

Procedure

1. Provide the control plane node pool name and resource sizing information.

   - **Node Pool Name**: NKP sets this field's value, `control plane`, and you cannot change it.
   - **Disk**: Enter the amount of disk space allocated for each control plane node. The default value is 80 GB. The specified custom disk size must be equal to, or larger than, the size of the base OS image root file system. This is because a root file system cannot be reduced automatically when a machine first boots.
   - **Memory**: The amount of memory for each control plane node in GB. The default value is 16 GB.
   - **Number of CPUs**: Enter the number of virtual processors in each control plane node. The default value is 4 CPUs per control plane node.
   - **Replicas**: Enter the number of control plane nodes to create for your new cluster.

     Valid values for production clusters are 3 or 5. You can enter one if you are creating a test cluster, but a single control plane is not a valid production configuration. You must enter an odd number to allow for internal leader selection processes to provide proper failover for high availability. The default value is three control plane nodes.

2. Provide the worker node pool name and resource sizing information.

   - **Node Pool Name**: Enter a node pool name for the worker nodes. NKP sets this field's default value to `worker-0`.
   - **Disk**: Enter the amount of disk space allotted for each worker node. The default value is 80GB. The specified custom disk size must be equal to, or larger than, the size of the base OS image root file system. This is because a root file system cannot be reduced automatically when a machine first boots.
   - **Memory**: The amount of memory for each worker node in GB. The default value is 32 GB.
   - **Number of CPUs**: Enter the number of virtual processors in each worker node. The default value is 8 CPUs per node.
   - **Replicas**: Enter the number of worker nodes to create for your new cluster. The default value is four worker nodes.

Setting Virtual IP Parameters

About this task

In this section of the form, you configure the built-in virtual IP.

Procedure

Provide the Virtual IP information needed for managing this cluster with NKP.

- **Interface**: Enter the name of the network used for the virtual IP control plane endpoint.

  This value is specific to your environment and cannot be inferred by NKP. An example value is `eth0` or `ens5`.

- **Host**: Enter the control plane endpoint address.

  To use an external load balancer, set this value to the load balancer's IP address or hostname. To use the built-in virtual IP, set to a static IPv4 address in the Layer 2 network of the control plane machines.

- **Port**: Enter the control plane's endpoint port.

  The default port value is 6443. To use an external load balancer, see this value in the load balancer's listening port.

Specifying the MetalLB Information
Specify the MetalLB Information.

Procedure

The MetalLB load balancer is needed for cluster installation, and requires these values.

- Provide a **Starting IP** address range value for the load balancing allocation.
- Provide an **Ending IP** address range value for the load balancing allocation.

Configuring the StorageClass Options

About this task

In this section of the form, you configure the storage options for your vSphere cluster. The StorageClass defines the provisioning properties and requirements for the storage used to store the persistent data of the Kubernetes application.

You can provide either the Datastore URL or the Storage Policy Name in this section.

Procedure

1. Select **Datastore URL** if it is not already highlighted, and then in the **Datastore URL** field, enter a unique identifier in URL format used by vSphere to access specific storage locations. A typical example of the field's format is `ds:///vmfs/volumes/<datastore_uuid>/`.
2. Select **Storage Policy Name** if it is not already highlighted, and then in the **Storage Policy Name** field, enter the name of the storage policy to use with the cluster's StorageClass.

Advanced Configuration Parameters

You can open the Advanced configuration parameters sections by selecting the **Show Advanced** link.

Configuring CIDR Values for the Pod Network and Kubernetes Services

About this task

In this section of the form, you configure Classless Inter-Domain Routing (CIDR) Values that your vSphere cluster uses.

Procedure

Specify the following values.

- Enter a CIDR value for the Pod network in the **Pod Network CIDR** field. The default value is 192.168.0.0/16.
- Enter a CIDR value for Kubernetes Services in the **Service CIDR** field. The default value is 10.96.0.0/12.

Configuring the Docker Registry Mirror

About this task

In this section, you configure a registry mirror for container images. The first time you request an image from your local registry mirror, it pulls the image from a public registry and stores it locally before handing it back to you. On subsequent requests, the local registry mirror serves the image from its own storage.

Procedure

Configure the image registry mirror.

- **Registry Mirror URL**: Enter the URL of a container registry to use as a registry mirror.
- **Registry Mirror Username**: Enter the name of a user who can authenticate to the registry mirror.
- **Registry Mirror Password**: Enter the password for the username in the previous entry.
- **Registry Mirror CA Cert**: Upload a certificate file or copy the CA certificate chain value into the provided field to use while communicating with the registry mirror using Transport Layer Security (TLS).

  This value is a trusted root certificate (or chain of certificates) that validates the SSL/TLS connection between clients and the registry mirror, ensuring secure and trustworthy communications.

Creating the Managed Cluster on vSphere

About this task

This step may take a few minutes, as the cluster must be ready and fully deploy its components. The cluster automatically tries to join the management cluster for federation and fleet operations and should resolve after it is fully provisioned.

While NKP provisions the new cluster, you can access the **Clusters** page to view the new cluster. A new cluster card with the name of your cluster appears and shows a “Pending” cluster status when the cluster comes up and joins the management cluster.

Procedure

Select the **Create** button (at the page’s top right corner) to begin provisioning the cluster.

Creating a Managed Cluster on VCD Through the NKP UI

About this task

After configuring VMware Cloud Director (VCD), you can use the NKP UI to provision a VCD cluster quickly and easily.

> **Note:** You must also create a VCD infrastructure provider before you can create additional VCD clusters.

Before you begin

Ensure that you have fulfilled the VMware Cloud Director configuration prerequisites. For more information, see VMware Cloud Director Prerequisites on page 964.

Provisioning a production-ready cluster in VCD requires you to specify a fairly large number of parameters. Breaking up the sections of the form, as done below, makes it a little easier to complete.

To Provision a VCD Cluster:

Procedure

Complete these procedures to provision a VCD cluster:

- Provide basic cluster information.
- Specify an SSH user and public key.
- Define the cluster's VCD resources.
- Configure node pools for the cluster.
- Specify pod and service CIDR values, if any.
- Define a registry mirror, if needed.
- Specify Control Plane endpoints.

Specifying Basic Cluster Information

About this task

In this section of the provisioning form, you give the cluster a name and provide some basic information:

Procedure

Provide these cluster details in the form:

- **Cluster Name**: A valid Kubernetes name for the cluster.
- **Add Labels**: Add any required Labels the cluster needs for your environment by selecting the `+ Add Label` link. Changing a cluster Label might add or remove the cluster from Projects.
- **Infrastructure Provider**: This field's value corresponds to the VCD infrastructure provider you created while fulfilling the prerequisites.
- **Kubernetes Version**: Select a supported version of Kubernetes for this version of NKP.

Specifying an SSH User

About this task

In this section, you specify the user name and public key values for an SSH user who has access to the VCD cluster after its creation. This group of credentials enables password-less SSH access for administrative tasks and debugging purposes, which improves security. NKP adds the public key to the `authorized_keys` file on each node, allowing the corresponding private key holder to access the nodes using SSH.

Procedure

1. Type an **SSH Username** to which the SSH Public Key belongs.

   Leaving this field blank means that NKP assigns the user name `konvoy` to the public key value you copy into the next field.

2. Copy and paste an entire, valid **SSH Public Key** to go with the SSH Username in the previous field.

Defining the Cluster's VCD Resources

About this task

This section of the form identifies resources already in your VMware Cloud Director configuration. For necessary values, refer to your VCD configuration.

Procedure

Provide the following values for the **Resources** that are specific to VCD.

- **Datacenter**: Select an existing Organization's Virtual Datacenter(VDC) name where you want to deploy the cluster.
- **Network**: Select the Organization's virtual datacenter Network that the new cluster uses.
- **Organization**: The Organization name under which you want to deploy the cluster.
- **Catalog**: The name of the VCD Catalog that hosts the virtual machine templates used for cluster creation.
- **vApp Template**: The vApp template used to create the virtual machines that comprise the cluster. A NKP cluster becomes a vApp in VCD.

  Storage Profile: The name of a VCD storage profile you want to use for a virtual machine. The default value in NKP is " * " and selects the policy defined as the default storage policy in VCD.

Configuring Node Pools

About this task

You need to configure node pool information for both your control plane nodes and your worker nodes. The form splits these information sets into two groups.

Procedure

1. Provide the control plane node pool name and resource sizing information.

   - **Node Pool Name**: NKP sets this field's value, `control plane`, and you cannot change it.
   - **Number of Nodes**: Enter the number of control plane nodes to create for your new cluster.

     Valid values for production clusters are three or five. You can enter one if you are creating a test cluster, but a single control plane is not a valid production configuration. You must enter an odd number to allow for internal leader selection processes to provide proper failover for high availability. The default value is three control plane nodes.
   - **Placement Policy**: The placement policy for control planes to be used on this machine.

     A VM placement policy defines the placement of a virtual machine on a host or group of hosts. It is a mechanism for cloud provider administrators to create a named group of hosts within a provider VDC. The named group of hosts is a subset of hosts within the provider VDC clusters that might be selected based on any criteria such as performance tiers or licensing. You can expand the scope of a VM placement policy to more than one provider VDC.
   - **Sizing Policy**: The sizing policy for control planes to be used on this machine.

     A VM sizing policy defines the computing resource allocation for virtual machines within an organization's VDC. The compute resource allocation includes CPU and memory allocation, reservations, limits, and shares.

2. Provide the worker node pool name and resource sizing information.

   - **Node Pool Name**: Enter a node pool name for the worker nodes. NKP sets this field's default value to `worker-0`.
   - **Replicas**: Enter the number of worker nodes to create for your new cluster. The default value is four worker nodes.
   - **Placement Policy**: The placement policy for workers to be used on this machine.

     A VM placement policy defines the placement of a virtual machine on a host or group of hosts. It is a mechanism for cloud provider administrators to create a named group of hosts within a provider VDC. The named group of hosts is a subset of hosts within the provider VDC clusters that might be selected based on any criteria such as performance tiers or licensing. You can expand the scope of a VM placement policy to more than one provider VDC.
   - **Sizing Policy**: The sizing policy for workers to be used on this machine.

     A VM sizing policy defines the computing resource allocation for virtual machines within an organization's VDC. The compute resource allocation includes CPU and memory allocation, reservations, limits, and shares.

Advanced Configuration Parameters

Configure the following advanced configuration parameters:

Specifying CIDR Values (Group)

About this task

In this section, you specify the CIDR blocks for Pods and Services in your VCD cluster. You also need to reserve a separate CIDR block for Services. These ranges must not overlap each other or the existing network. Incorrect configuration can cause network conflicts and disrupt cluster operations.

Procedure

1. Specify the **Pod Network CIDR** to use in VCD clusters.
   The default value is 192.168.0.0/16.
2. Specify the **Service CIDR** to use in VCD clusters.
   The default value is 10.96.0.0/12.

Defining a Registry Mirror (Group)

About this task

A registry mirror is a local caching proxy for a container image repository. When clients request an image, the mirror first tries to serve the image from its cache. If the image is not available in the cache, the mirror fetches it from the primary repository, caches it, and then serves it to the client.

Procedure

1. Enter the **URL** of a container registry to use as a mirror in the cluster.
2. Type the **Username** for the account to use to authenticate to the registry mirror.
3. Type the **Password** for the account to authenticate to the registry mirror.
4. Copy and paste a **CA Certificate** chain to use while communicating with the registry mirror using TLS.

Specifying the Control Plane Endpoint

About this task

In this section, you specify the control plane endpoint host and port values that enable both IP addresses and DNS names to map to the IP address for this cluster.

Procedure

1. Type the **Host** name as either the control plane endpoint IP or a hostname.
2. Enter a **Port** value for the control plane endpoint port. The default value is 6443. To use an external load balancer, set this port value to the load balancer's listening port number.

Attaching an Existing Kubernetes Cluster

Attach an existing Kubernetes cluster using kubeconfig

You can attach an existing cluster (whether it is a cluster created with the NKP CLI or by means of another platform) to NKP. At the time of attachment, certain namespaces are created on the cluster, and workspace platform applications are deployed automatically into the newly-created namespaces.

Review the Workspace Platform Application Defaults and Resource Requirements on page 44 to ensure the attached cluster has sufficient resources. For more information on platform applications and customizing them, see Workspace Applications on page 412.

If the cluster you want to attach was created using Amazon EKS, Azure AKS, or Google GKE, create a service account as described in Cluster Attachment with No Networking Restrictions on page 497.

> **Note:** Starting with NKP 2.6.0, NKP supports the attachment of all Kubernetes Conformant clusters, but only x86-64 architecture is supported, not ARM.

Platform applications extend the functionality of Kubernetes and provide ready-to-use logging and monitoring stacks. Platform applications are deployed when a cluster is attached to Kommander.

Requirements for Attaching an Existing Cluster

These are the requirements for attaching an existing cluster.

Basic Requirements

To attach an existing cluster in the UI, the Application Management cluster must be able to reach the services and the `api-server` of the target cluster.

The cluster you want to attach can be a **NKP-CLI-created** cluster (which will become a Managed cluster upon attachment), or **another Kubernetes cluster** like AKS, EKS, or GKE (which will become an Attached cluster upon attachment).

Creating a Default StorageClass

About this task

To deploy many of the services on the attached cluster, a default #StorageClass# must be configured.

Procedure

1. Run the following command on the cluster you want to attach.

   ```
   kubectl get sc
   ```

   The output should look similar to this. Note the `(default)` after the name:

   ```
   NAME               PROVISIONER       RECLAIMPOLICY   VOLUMEBINDINGMODE
    ALLOWVOLUMEEXPANSION   AGE
   ebs-sc (default)   ebs.csi.aws.com   Delete          WaitForFirstConsumer   false
                41s
   ```

   If the `StorageClass` is not set as default, add the following annotation to the `StorageClass` manifest.

   ```
   annotations:
     storageclass.kubernetes.io/is-default-class: "true"
   ```

2.

Projects and Workspaces

Before you attach clusters, you need to create one or more Workspaces, and we recommend that you also create Projects within your Workspaces. Workspaces give you a logical way to represent your teams and specific configurations. Projects let you define one or more clusters as a group to which Kommander pushes a common configuration. Grouping your existing clusters in Kommander projects and workspaces makes managing their platform services and resources easier and supports monitoring and logging.

> **Note:** Do not attach a cluster in the "Management Cluster Workspace" workspace. This workspace is reserved for your Application Management cluster only.

Platform Application Requirements

In addition to the basic cluster requirements, the platform services you want NKP to manage on those clusters will impact the total cluster requirements. The specific combinations of platform applications will affect the requirements for the cluster nodes and their resources (CPU, memory, and storage).

To view a list of platform applications that NKP provides by default, see Platform Applications on page 400.

Requirements for Attaching Existing AKS, EKS, and GKE Clusters

Attaching an existing AWS cluster requires that the cluster be fully configured and running. You must create a separate service account when attaching existing AKS, EKS, or Google GKE Kubernetes clusters. This is necessary because the `kubeconfig` files generated from those clusters are not usable out-of-the-box by Kommander. The `kubeconfig` files call CLI commands, such as `azure, aws` , or `gcloud`, and use locally-obtained authentication tokens. Having a separate service account also allows you to keep access to the cluster-specific to and isolated from Kommander.

The suggested default cluster configuration includes a control plane pool containing three m5.xlarge nodes and a worker pool containing four m5.2xlarge nodes.

Consider the additional resource requirements for running the platform services you want NKP to manage and ensure that your existing clusters comply.

To attach an existing EKS cluster, see EKS Cluster Attachment on page 497.

To attach an existing GKE cluster, see GKE Cluster Attachment on page 504.

Requirements for Attaching Clusters with an Existing cert-manager Installation

If you are attaching clusters that already have cert-manager installed, the cert-manager `HelmRelease` provided by NKP will fail to deploy, due to the existing cert-manager installation. As long as the pre-existing cert-manager functions as expected, you can ignore this failure. It will have no impact on the operation of the cluster.

Creating a kubeconfig File for Attachment

Create a separate service account when attaching existing clusters (for example, Amazon EKS or Azure AKS clusters).

About this task

If you already have a `kubeconfig` file to attach your cluster, go directly to Attaching a Cluster with No Networking Restrictions on page 497 or Cluster Attachment with Networking Restrictions on page 508

The `kubeconfig` files generated from existing clusters are not usable out-of-the-box because they call provisioner-specific CLI commands (like `aws` commands) and use locally-obtained authentication tokens that are not compatible with NKP. Having a separate service account also allows you to have a dedicated identity for all NKP operations.

To get started, ensure you have kubectl set up and configured with ClusterAdmin for the cluster you want to connect to Kommander.

Procedure

1. Create the necessary service account.

   ```
   kubectl -n kube-system create serviceaccount kommander-cluster-admin
   ```

2. Create a token secret for the `serviceaccount`:

   ```
   kubectl -n kube-system create  -f - <<EOF
   apiVersion: v1
   kind: Secret
   metadata:
     name: kommander-cluster-admin-sa-token
     annotations:
       kubernetes.io/service-account.name: kommander-cluster-admin
   type: kubernetes.io/service-account-token
   EOF
   ```

3. Verify that the `serviceaccount` token is ready by running the `kubectl -n kube-system get secret kommander-cluster-admin-sa-token -oyaml` command.

   Verify that the `data.token` field is populated. The output should be similar to this example:

   ```
   apiVersion: v1
   data:
     ca.crt: LS0tLS1CRUdJTiBDR...
     namespace: ZGVmYXVsdA==
     token: ZXlKaGJHY2lPaUpTVX...
   kind: Secret
   metadata:
     annotations:
       kubernetes.io/service-account.name: kommander-cluster-admin
       kubernetes.io/service-account.uid: b62bc32e-b502-4654-921d-94a742e273a8
     creationTimestamp: "2022-08-19T13:36:42Z"
     name: kommander-cluster-admin-sa-token
     namespace: default
     resourceVersion: "8554"
     uid: 72c2a4f0-636d-4a70-9f1c-55a75f15e520
   ```

```
type: kubernetes.io/service-account-token
```

4. Configure the new service account for `cluster-admin` permissions.

```
cat << EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: kommander-cluster-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
  name: kommander-cluster-admin
  namespace: kube-system
EOF
```

5. Set up the following environment variables with the access data that is needed for producing a new `kubeconfig` file.

```
export USER_TOKEN_VALUE=$(kubectl -n kube-system get secret/kommander-cluster-admin-
sa-token -o=go-template='{{.data.token}}' | base64 --decode)
export CURRENT_CONTEXT=$(kubectl config current-context)
export CURRENT_CLUSTER=$(kubectl config view --raw -o=go-
template='{{range .contexts}}{{if eq .name "'''${CURRENT_CONTEXT}'''"}}
{{ index .context "cluster" }}{{end}}{{end}}')
export CLUSTER_CA=$(kubectl config view --raw -o=go-template='{{range .clusters}}{{if
 eq .name "'''${CURRENT_CLUSTER}'''"}}"{{with index .cluster "certificate-authority-
data" }}{{.}}{{end}}"{{ end }}{{ end }}')
export CLUSTER_SERVER=$(kubectl config view --raw -o=go-template='{{range .clusters}}
{{if eq .name "'''${CURRENT_CLUSTER}'''"}}{{ .cluster.server }}{{end}}{{ end }}')
```

6. Confirm these variables have been set correctly.

```
export -p USER_TOKEN_VALUE CURRENT_CONTEXT CURRENT_CLUSTER CLUSTER_CA CLUSTER_SERVER
```

7. Generate a `kubeconfig` file that uses the environment variable values from the previous step.

```
cat << EOF > kommander-cluster-admin-config
apiVersion: v1
kind: Config
current-context: ${CURRENT_CONTEXT}
contexts:
- name: ${CURRENT_CONTEXT}
  context:
    cluster: ${CURRENT_CONTEXT}
    user: kommander-cluster-admin
    namespace: kube-system
clusters:
- name: ${CURRENT_CONTEXT}
  cluster:
    certificate-authority-data: ${CLUSTER_CA}
    server: ${CLUSTER_SERVER}
users:
- name: kommander-cluster-admin
  user:
    token: ${USER_TOKEN_VALUE}
EOF
```

8. This process produces a file in your current working directory called `kommander-cluster-admin-config`. The contents of this file are used in Kommander to attach the cluster.

   Before importing this configuration, verify the `kubeconfig` file can access the cluster.

   ```
   kubectl --kubeconfig $(pwd)/kommander-cluster-admin-config get all --all-namespaces
   ```

What to do next

Use this `kubeconfig` to:

- Attaching a Cluster with No Networking Restrictions on page 497
- Cluster Attachment with Networking Restrictions on page 508

> **Note:** If a cluster has limited resources to deploy all the federated platform services, it will fail to stay attached to the NKP UI. If this happens, check if any pods are not getting the resources required.

Cluster Attachment with No Networking Restrictions

Configure the non-network restricted cluster settings.

Attaching a Cluster with No Networking Restrictions

About this task

Using the **Add Cluster** option, you can attach an existing Kubernetes or NKP cluster directly to NKP. This enables you to access the multi-cluster management and monitoring benefits that NKP provides, while keeping your existing cluster on its current provider and infrastructure.

Use this option when you want to attach a cluster that does not require additional access information.

Procedure

1. From the top menu bar, select your target workspace.
2. On the Dashboard page, select the **Add Cluster** option in the **Actions** dropdown menu at the top right.
3. Select **Attach Cluster**.
4. Select the **No additional networking restrictions** card. Alternatively, if you must use network restrictions, perform the steps in Cluster Attachment with Networking Restrictions on page 508.
5. In the **Cluster Configuration** section, paste your `kubeconfig` file into the field, or select **Upload kubeconfig File** to specify the file.
6. The **Cluster Name** field will automatically populate with the name of the cluster is in the `kubeconfig`. You can edit this field with the name you want for your cluster.
7. The **Context** select list is populated from the `kubeconfig`. Select the desired context with admin privileges from the **Context** select list.
8. Add labels to classify your cluster as needed.
9. Select **Create** to attach your cluster.

EKS Cluster Attachment

Attach an existing EKS cluster

You can attach existing Kubernetes clusters to the Management Cluster. After attaching the cluster, you can use the UI to examine and manage this cluster. The following procedure shows how to attach an existing Amazon Elastic Kubernetes Service (EKS) cluster.

EKS: Preparing the Cluster

About this task

This procedure requires the following items and configurations:

This procedure assumes you have an existing and spun-up Amazon EKS cluster(s) with administrative privileges. For more information, see the Amazon for setup and configuration information, see https://aws.amazon.com/eks/.

- A fully configured and running Amazon EKS cluster with administrative privileges.
- The current version NKP Ultimate is on your cluster.
- Ensure you have installed `kubectl` in your Management cluster.
- Attach Amazon EKS Clusters. Ensure that the `KUBECONFIG` environment variable is set to the Management cluster before attaching by running:

  ```
  export KUBECONFIG=<Management_cluster_kubeconfig>.conf
  ```

Ensure you have access to your EKS clusters.

Procedure

Ensure you are connected to your EKS clusters. Enter the following commands for each of your clusters.

```
kubectl config get-contexts
kubectl config use-context <context for first eks cluster>
```

Confirm `kubectl` can access the EKS cluster:

```
kubectl get nodes
```

EKS: Creating a kubeconfig File

Create a `kubeconfig` file for your EKS cluster

About this task

To get started, ensure you have kubectl set up and configured with ClusterAdmin for the cluster you want to connect to Kommander. Create the necessary service account:

Procedure

1. Create the necessary service account.

   ```
   kubectl -n kube-system create serviceaccount kommander-cluster-admin
   ```

2. Create a token secret for the `serviceaccount`.

   ```
   kubectl -n kube-system create  -f - <<EOF
   apiVersion: v1
   kind: Secret
   metadata:
     name: kommander-cluster-admin-sa-token
     annotations:
       kubernetes.io/service-account.name: kommander-cluster-admin
   ```

```
type: kubernetes.io/service-account-token
EOF
```

For more information on Service Account Tokens, See https://eng.d2iq.com/blog/service-account-tokens-in-kubernetes-v1.24/#whats-changed-in-kubernetes-v124

3. Verify that the `serviceaccount` token is ready by running this command.

```
kubectl -n kube-system get secret kommander-cluster-admin-sa-token -oyaml
```

Verify that the `data.token` field is populated.

Example output:

```
apiVersion: v1
data:
  ca.crt: LS0tLS1CRUdJTiBDR...
  namespace: ZGVmYXVsdA==
  token: ZXlKaGJHY2lPaUpTVX...
kind: Secret
metadata:
  annotations:
    kubernetes.io/service-account.name: kommander-cluster-admin
    kubernetes.io/service-account.uid: b62bc32e-b502-4654-921d-94a742e273a8
  creationTimestamp: "2022-08-19T13:36:42Z"
  name: kommander-cluster-admin-sa-token
  namespace: default
  resourceVersion: "8554"
  uid: 72c2a4f0-636d-4a70-9f1c-55a75f15e520
type: kubernetes.io/service-account-token
```

4. Configure the new service account for `cluster-admin` permissions.

```
cat << EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: kommander-cluster-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
  name: kommander-cluster-admin
  namespace: kube-system
EOF
```

5. Set up the following environment variables with the access data that is needed for producing a new `kubeconfig` file.

```
export USER_TOKEN_VALUE=$(kubectl -n kube-system get secret/kommander-cluster-admin-
sa-token -o=go-template='{{.data.token}}' | base64 --decode)
export CURRENT_CONTEXT=$(kubectl config current-context)
export CURRENT_CLUSTER=$(kubectl config view --raw -o=go-
template='{{range .contexts}}{{if eq .name "'''${CURRENT_CONTEXT}'''"}}
{{ index .context "cluster" }}{{end}}{{end}}')
export CLUSTER_CA=$(kubectl config view --raw -o=go-template='{{range .clusters}}{{if
 eq .name "'''${CURRENT_CLUSTER}'''"}}"{{with index .cluster "certificate-authority-
data" }}{{.}}{{end}}"{{ end }}{{ end }}')
export CLUSTER_SERVER=$(kubectl config view --raw -o=go-template='{{range .clusters}}
{{if eq .name "'''${CURRENT_CLUSTER}'''"}}{{ .cluster.server }}{{end}}{{ end }}')
```

6. Confirm these variables have been set correctly.

```
export -p | grep -E 'USER_TOKEN_VALUE|CURRENT_CONTEXT|CURRENT_CLUSTER|CLUSTER_CA|
CLUSTER_SERVER'
```

7. Generate a kubeconfig file that uses the environment variable values from the previous step.

> **Caution:** Nutanix support is unable to recover forgotten credentials. Prevent Kommander UI lockouts with the following best practices.
>
> - Backup the kubeconfig file.
> - Save your credentials.
> - Configure an IDP (identity provider) instead of using a username and password credentials.

```
cat << EOF > kommander-cluster-admin-config
apiVersion: v1
kind: Config
current-context: ${CURRENT_CONTEXT}
contexts:
- name: ${CURRENT_CONTEXT}
  context:
    cluster: ${CURRENT_CONTEXT}
    user: kommander-cluster-admin
    namespace: kube-system
clusters:
- name: ${CURRENT_CONTEXT}
  cluster:
    certificate-authority-data: ${CLUSTER_CA}
    server: ${CLUSTER_SERVER}
users:
- name: kommander-cluster-admin
  user:
    token: ${USER_TOKEN_VALUE}
EOF
```

8. This process produces a file in your current working directory called `kommander-cluster-admin-config`. The contents of this file are used in Kommander to attach the cluster. Before importing this configuration, verify the `kubeconfig` file can access the cluster.

```
kubectl --kubeconfig $(pwd)/kommander-cluster-admin-config get all --all-namespaces
```

EKS: Finalizing Attaching the Cluster Through the UI

About this task

Now that you have `kubeconfig` file, go to the NKP UI and follow these steps below:

Procedure

1. From the top menu bar, select your target workspace.
2. On the Dashboard page, select the **Add Cluster** option in the **Actions** dropdown menu at the top right.
3. Select **Attach Cluster**.
4. Select the **No additional networking restrictions** card. Alternatively, if you must use network restrictions, follow the steps in Cluster Attachment with Networking Restrictions on page 508.

5. Upload the `kubeconfig` file you created in the previous section (or copy its contents) into the **Cluster Configuration** section.
6. The **Cluster Name** field automatically populates with the name of the cluster in the `kubeconfig` file. You can edit this field using the name you want for your cluster.
7. Add labels to classify your cluster as needed.
8. Select **Create** to attach your cluster.

> **Note:** If a cluster has limited resources to deploy all the federated platform services, it will fail to stay attached to the NKP UI. If this happens, ensure your system has sufficient resources for all pods.

AKS Cluster Attachment

Attach an existing AKS cluster

You can attach existing Kubernetes clusters to the Management Cluster. After attaching the cluster, you can use the UI to examine and manage this cluster. The following procedure shows how to attach an existing Azure Kubernetes Service (AKS) cluster.

AKS: Preparing the Cluster

About this task

This procedure requires the following items and configurations:

- A fully configured and running Azure AKS cluster with administrative privileges.
- The current version NKP Ultimate is installed on your cluster.
- Ensure you have installed `kubectl` in your Management cluster.
- Attach AKS Clusters. Ensure that the `KUBECONFIG` environment variable is set to the Management cluster before attaching by running:

```
export KUBECONFIG=<Management_cluster_kubeconfig>.conf
```

> **Note:**
>
> This procedure assumes you have an existing and spun-up Azure AKS cluster(s) with administrative privileges. For information on the Azure AKS for setup and configuration, see https://azure.microsoft.com/en-us/products/kubernetes-service/.
>
> Ensure you have access to your AKS clusters.

Procedure

1. Ensure you are connected to your AKS clusters. Enter the following commands for each of your clusters.

```
kubectl config get-contexts
kubectl config use-context <context for first AKS cluster>
```

2. Confirm `kubectl` can access the AKS cluster.

```
kubectl get nodes
```

AKS: Creating a kubeconfig File

Create a `kubeconfig` file for your AKS cluster

About this task

To get started, ensure you have `kubectl` set up and configured with ClusterAdmin for the cluster you want to connect to Kommander. Create the necessary service account:

Procedure

1. Create the necessary service account.

```
kubectl -n kube-system create serviceaccount kommander-cluster-admin
```

2. Create a token secret for the `serviceaccount`.

```
kubectl -n kube-system create  -f - <<EOF
apiVersion: v1
kind: Secret
metadata:
  name: kommander-cluster-admin-sa-token
  annotations:
    kubernetes.io/service-account.name: kommander-cluster-admin
type: kubernetes.io/service-account-token
EOF
```

3. Verify that the `serviceaccount` token is ready by running this command.

```
kubectl -n kube-system get secret kommander-cluster-admin-sa-token -oyaml
```

   Verify that the `data.token` field is populated. The output must be similar to this:

```
apiVersion: v1
data:
  ca.crt: LS0tLS1CRUdJTiBDR...
  namespace: ZGVmYXVsdA==
  token: ZXlKaGJHY2lPaUpTVX...
kind: Secret
metadata:
  annotations:
    kubernetes.io/service-account.name: kommander-cluster-admin
    kubernetes.io/service-account.uid: b62bc32e-b502-4654-921d-94a742e273a8
  creationTimestamp: "2022-08-19T13:36:42Z"
  name: kommander-cluster-admin-sa-token
  namespace: default
  resourceVersion: "8554"
  uid: 72c2a4f0-636d-4a70-9f1c-55a75f15e520
type: kubernetes.io/service-account-token
```

4. Configure the new service account for `cluster-admin` permissions.

```
cat << EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: kommander-cluster-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
```

```
  name: kommander-cluster-admin
  namespace: kube-system
EOF
```

5. Set up the following environment variables with the access data that is needed for producing a new `kubeconfig` file.

```
export USER_TOKEN_VALUE=$(kubectl -n kube-system get secret/kommander-cluster-admin-
sa-token -o=go-template='{{.data.token}}' | base64 --decode)
export CURRENT_CONTEXT=$(kubectl config current-context)
export CURRENT_CLUSTER=$(kubectl config view --raw -o=go-
template='{{range .contexts}}{{if eq .name "'''${CURRENT_CONTEXT}'''"}}
{{ index .context "cluster" }}{{end}}{{end}}')
export CLUSTER_CA=$(kubectl config view --raw -o=go-template='{{range .clusters}}{{if
 eq .name "'''${CURRENT_CLUSTER}'''"}}"{{with index .cluster "certificate-authority-
data" }}{{.}}{{end}}"{{ end }}{{ end }}')
export CLUSTER_SERVER=$(kubectl config view --raw -o=go-template='{{range .clusters}}
{{if eq .name "'''${CURRENT_CLUSTER}'''"}}{{ .cluster.server }}{{end}}{{ end }}')
```

6. Confirm these variables have been set correctly.

```
export -p | grep -E 'USER_TOKEN_VALUE|CURRENT_CONTEXT|CURRENT_CLUSTER|CLUSTER_CA|
CLUSTER_SERVER'
```

7. Generate a kubeconfig file that uses the environment variable values from the previous step.

> **Caution:** Nutanix support is unable to recover forgotten credentials. Prevent Kommander UI lockouts with the following best practices.
>
> - Backup the kubeconfig file.
> - Save your credentials.
> - Configure an IDP (identity provider) instead of using a username and password credentials.

```
cat << EOF > kommander-cluster-admin-config
apiVersion: v1
kind: Config
current-context: ${CURRENT_CONTEXT}
contexts:
- name: ${CURRENT_CONTEXT}
  context:
    cluster: ${CURRENT_CONTEXT}
    user: kommander-cluster-admin
    namespace: kube-system
clusters:
- name: ${CURRENT_CONTEXT}
  cluster:
    certificate-authority-data: ${CLUSTER_CA}
    server: ${CLUSTER_SERVER}
users:
- name: kommander-cluster-admin
  user:
    token: ${USER_TOKEN_VALUE}
EOF
```

8. This process produces a file in your current working directory called `kommander-cluster-admin-config`. The contents of this file are used in Kommander to attach the cluster. Before importing this configuration, verify the `kubeconfig` file can access the cluster.

   ```
   kubectl --kubeconfig $(pwd)/kommander-cluster-admin-config get all --all-namespaces
   ```

AKS: Finalizing Attaching the Cluster Through the UI

About this task

Now that you have `kubeconfig` file, go to the NKP UI and follow these steps below:

Procedure

1. From the top menu bar, select your target workspace.
2. On the Dashboard page, select the **Add Cluster** option in the **Actions** dropdown menu at the top right.
3. Select **Attach Cluster**.
4. Select the **No additional networking restrictions** card. Alternatively, if you must use network restrictions, follow the steps in Cluster Attachment with Networking Restrictions on page 508.
5. Upload the `kubeconfig` file you created in the previous section (or copy its contents) into the **Cluster Configuration** section.
6. The **Cluster Name** field automatically populates with the name of the cluster in the `kubeconfig` file. You can edit this field using the name you want for your cluster.
7. Add labels to classify your cluster as needed.
8. Select **Create** to attach your cluster.

> **Note:** If a cluster has limited resources to deploy all the federated platform services, it will fail to stay attached in the NKP UI. If this happens, ensure your system has sufficient resources for all pods.

GKE Cluster Attachment

Attach an existing GKE cluster in NKP.

You can attach existing Kubernetes clusters to the Management Cluster. After attaching the cluster, you can use the UI to examine and manage this cluster. The following procedure shows how to attach an existing Google Kubernetes Engine (GKE) cluster.

GKE: Preparing the Cluster

About this task

This procedure requires the following items and configurations:

- A fully configured and running with a GKE cluster supported Kubernetes version cluster with administrative privileges.
- The current version NKP Ultimate is installed on your cluster.

- Ensure you have installed `kubectl` in your Management cluster.

  Note: This procedure assumes you have an existing and spun-up GKE cluster with administrator privileges.

- Attach GKE Clusters.
- Ensure that you have access to your GKE clusters. Enter the following commands for each of your clusters:c

```
kubectl config get-contexts
kubectl config use-context <context for first gcloud cluster>
```

- Confirm `kubectl` can access the GKE cluster.

  Note:

  Ensure you have access to your GKE clusters.

Procedure

1. Ensure you are connected to your GKE clusters. Enter the following commands for each of your clusters.
2. Confirm `kubectl` can access the GKE cluster.

```
kubectl get nodes
```

GKE: Creating a kubeconfig File

Create a `kubeconfig` file for your GKE cluster

About this task

To get started, ensure you have kubectl set up and configured with ClusterAdmin for the cluster you want to connect to Kommander.

Caution: Nutanix support is unable to recover forgotten credentials. The following best practices will prevent Kommander UI lockouts.

- Backup the kubeconfig file.
- Save your credentials.
- Configure an IDP (identity provider) instead of using a username and password credentials.

Create the necessary service account:

Procedure

1. Create the necessary service account.

```
kubectl -n kube-system create serviceaccount kommander-cluster-admin
```

2. Create a token secret for the `serviceaccount`.

```
kubectl -n kube-system create  -f - <<EOF
apiVersion: v1
kind: Secret
```

```
metadata:
  name: kommander-cluster-admin-sa-token
  annotations:
    kubernetes.io/service-account.name: kommander-cluster-admin
type: kubernetes.io/service-account-token
EOF
```

For more information on Service Account Tokens, see https://eng.d2iq.com/blog/service-account-tokens-in-kubernetes-v1.24/#whats-changed-in-kubernetes-v124.

3. Verify that the `serviceaccount` token is ready by running this command.

```
kubectl -n kube-system get secret kommander-cluster-admin-sa-token -oyaml
```

Verify that the `data.token` field is populated.

Example output:

```
apiVersion: v1
data:
  ca.crt: LS0tLS1CRUdJTiBDR...
  namespace: ZGVmYXVsdA==
  token: ZXlKaGJHY2lPaUpTVX...
kind: Secret
metadata:
  annotations:
    kubernetes.io/service-account.name: kommander-cluster-admin
    kubernetes.io/service-account.uid: b62bc32e-b502-4654-921d-94a742e273a8
  creationTimestamp: "2022-08-19T13:36:42Z"
  name: kommander-cluster-admin-sa-token
  namespace: default
  resourceVersion: "8554"
  uid: 72c2a4f0-636d-4a70-9f1c-55a75f15e520
type: kubernetes.io/service-account-token
```

4. Configure the new service account for `cluster-admin` permissions.

```
cat << EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: kommander-cluster-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
  name: kommander-cluster-admin
  namespace: kube-system
EOF
```

5. Set up the following environment variables with the access data that is needed for producing a new `kubeconfig` file.

```
export USER_TOKEN_VALUE=$(kubectl -n kube-system get secret/kommander-cluster-admin-
sa-token -o=go-template='{{.data.token}}' | base64 --decode)
export CURRENT_CONTEXT=$(kubectl config current-context)
export CURRENT_CLUSTER=$(kubectl config view --raw -o=go-
template='{{range .contexts}}{{if eq .name "'''${CURRENT_CONTEXT}'''"}}
{{ index .context "cluster" }}{{end}}{{end}}')
export CLUSTER_CA=$(kubectl config view --raw -o=go-template='{{range .clusters}}{{if
 eq .name "'''${CURRENT_CLUSTER}'''"}}"{{with index .cluster "certificate-authority-
data" }}{{.}}{{end}}"{{ end }}{{ end }}')
```

```
export CLUSTER_SERVER=$(kubectl config view --raw -o=go-template='{{range .clusters}}
{{if eq .name "'''${CURRENT_CLUSTER}'''"}}{{ .cluster.server }}{{end}}{{ end }}')
```

6. Confirm these variables have been set correctly.

   c

```
export -p | grep -E 'USER_TOKEN_VALUE|CURRENT_CONTEXT|CURRENT_CLUSTER|CLUSTER_CA|
CLUSTER_SERVER'
```

7. Generate a kubeconfig file that uses the environment variable values from the previous step.

```
cat << EOF > kommander-cluster-admin-config
apiVersion: v1
kind: Config
current-context: ${CURRENT_CONTEXT}
contexts:
- name: ${CURRENT_CONTEXT}
  context:
    cluster: ${CURRENT_CONTEXT}
    user: kommander-cluster-admin
    namespace: kube-system
clusters:
- name: ${CURRENT_CONTEXT}
  cluster:
    certificate-authority-data: ${CLUSTER_CA}
    server: ${CLUSTER_SERVER}
users:
- name: kommander-cluster-admin
  user:
    token: ${USER_TOKEN_VALUE}
EOF
```

8. This process produces a file in your current working directory called `kommander-cluster-admin-config`. The contents of this file are used in Kommander to attach the cluster. Before importing this configuration, verify the `kubeconfig` file can access the cluster.

```
kubectl --kubeconfig $(pwd)/kommander-cluster-admin-config get all --all-namespaces
```

GKE: Finalizing Attaching the Cluster Through the UI

About this task

Now that you have the `kubeconfig` file, go to the NKP UI and follow these steps below:

Procedure

1. From the top menu bar, select your target workspace.
2. On the Dashboard page, select the **Add Cluster** option in the **Actions** dropdown menu at the top right.
3. Select **Attach Cluster**.
4. Select the **No additional networking restrictions** card. Alternatively, if you must use network restrictions, follow the steps in Cluster Attachment with Networking Restrictions on page 508.
5. Upload the `kubeconfig` file you created in the previous section (or copy its contents) into the **Cluster Configuration** section.
6. The **Cluster Name** field automatically populates with the name of the cluster in the `kubeconfig` file. You can edit this field using the name you want for your cluster.

7. Add labels to classify your cluster as needed.
8. Select **Create** to attach your cluster.

> **Note:** If a cluster has limited resources to deploy all the federated platform services, it will fail to stay attached to the NKP UI. If this happens, ensure your system has sufficient resources for all pods.

Cluster Attachment with Networking Restrictions

Configure the network-resrtricted cluster settings.

Need for a Secure Tunnel

When attaching a cluster to NKP, the Management cluster initiates an outbound connection to the cluster you want to attach. This is not possible if the cluster you want to attach (Managed or Attached) has networking restrictions and is not exposed, for example, because it is in a private network or its API is not accessible from the same network as the Management cluster. This is what we call a network-restricted cluster.

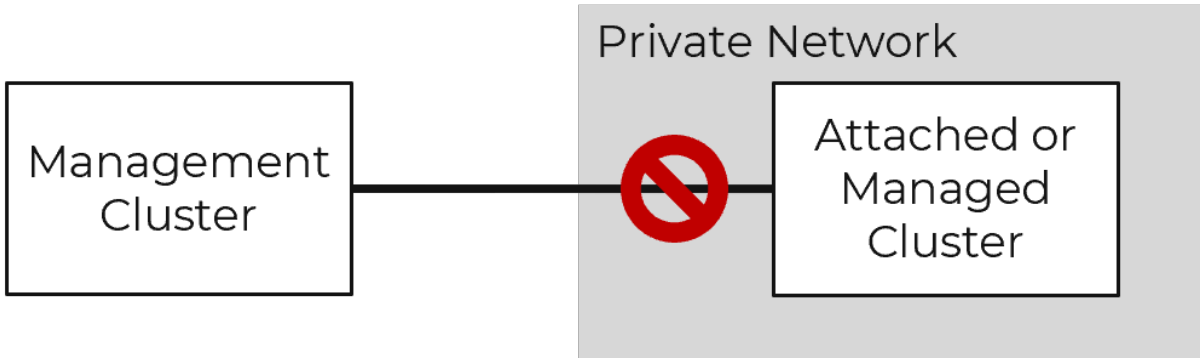


Figure 14: Network-restricted Cluster

Tunneled Attachment Workflow

NKP can create a secure tunnel to enable the attachment of clusters that are not directly reachable.

To create a secure tunnel, you must provide a configuration for the tunnel in the cluster you want to attach. After you apply that configuration, the cluster you want to attach will establish a secure tunnel with the Management cluster and make an attachment request.

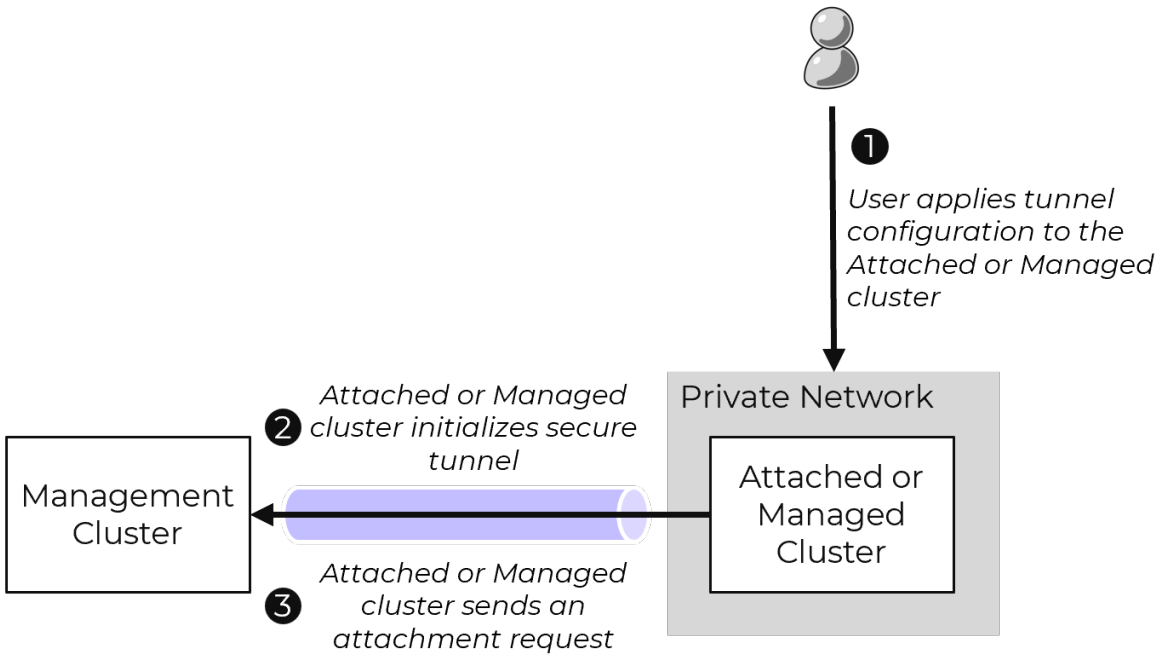


Figure 15: Secure Tunnel Attachment Workflow

After the attachment request is accepted and the connection between clusters is established, both clusters will allow bilateral communication.

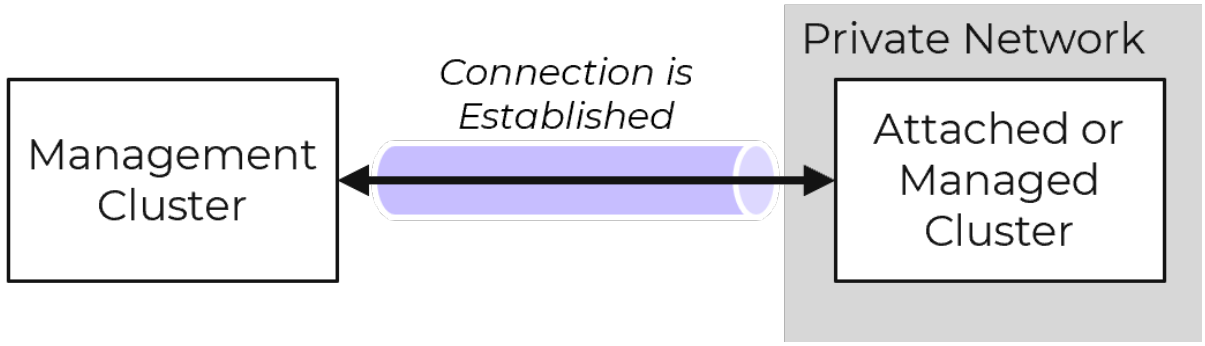


Figure 16: Connection Establishment of the Secure Tunnel Attachment

Prerequisites for a Tunneled Attachment

Before you begin

- Gain more understanding of this approach by reviewing UI: Attaching a Network-Restricted Cluster Using a Tunnel Through the UI on page 510
- Ensure you have reviewed the general https://d2iq.atlassian.net/wiki/spaces/DENT/pages/29920407 .
- **Firewall Rules**:

Table 44: Table

	The ingress rule on the Management cluster network must allow:	The egress rule on the Attached or Managed cluster private network must allow:
Protocol	HTTPS (TCP/443) and WebSocket	HTTPS (TCP/443) and WebSocket
Source	Any	Any node of the Attached or Managed cluster
Destination	NKP Traefik Service External IP/ URL	NKP Traefik Service on the Management cluster

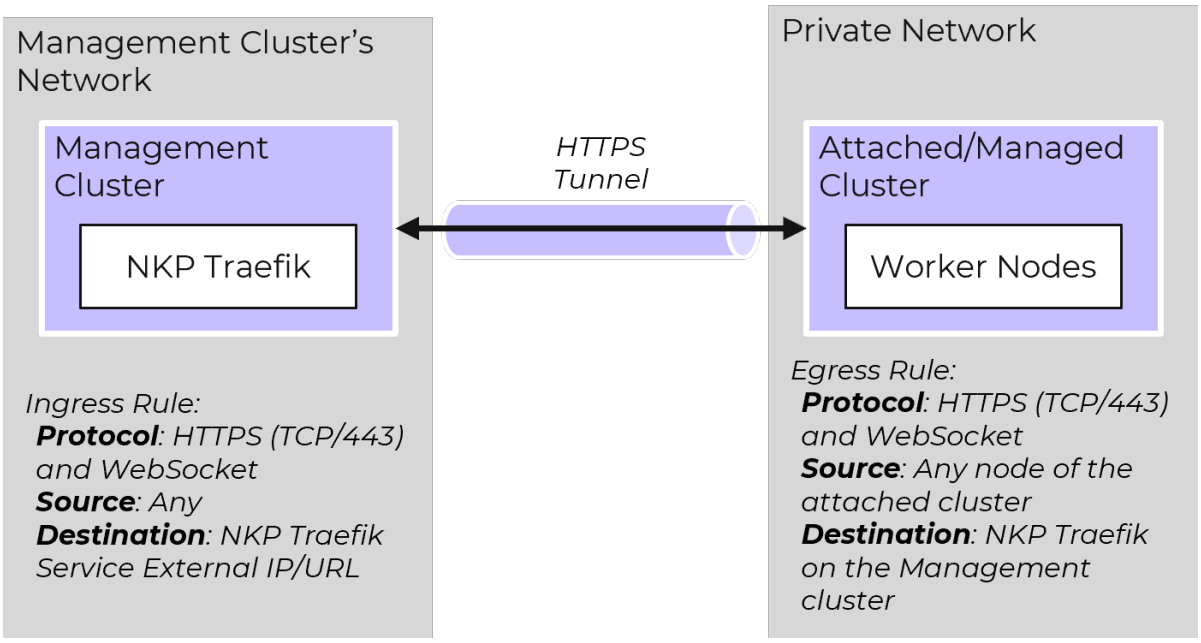


Figure 17: Tunnel Attachment

Procedure

UI: Attaching a Network-Restricted Cluster Using a Tunnel Through the UI

Use the UI to attach a network-restricted cluster.

Before you begin

Ensure you have reviewed and followed the steps in Prerequisites for a Tunneled Attachment on page 509.

To attach a cluster:

Procedure

1. From the top menu bar, select your target workspace.
2. On the Dashboard page, select the `Add Cluster` option in the **Actions** dropdown menu at the top right.

3. Select **Attach Cluster**.
4. Select the **Cluster has networking restrictions** card to display the configuration page.
5. **Establish the configuration parameters for the attachment:** Enter the Cluster Name of the cluster you're attaching.
6. Create additional new **Labels** as needed.
7. Select the hostname that is the Ingress for the cluster from the **Load Balancer Hostname** dropdown menu. The hostname must match the Kommander Host cluster to which you are attaching your existing cluster with network restrictions.
8. Specify the **URL Path Prefix** for your Load Balancer Hostname. This URL path will serve as the prefix for the specific tunnel services you want to expose on the Kommander management cluster. If no value is specified, the value defaults to `/nkp/tunnel`.

   Kommander uses Traefik 2 ingress, which requires the explicit definition of strip prefix middleware as a Kubernetes API object, as opposed to a simple annotation. Kommander provides default middleware that supports creating tunnels only on the `/nkp/tunnel URL` prefix. This is indicated by using the extra annotation, `traefik.ingress.kubernetes.io/router.middlewares: kommander-stripprefixes-kubetunnel@kubernetescrd` as shown in the code sample that follows. If you want to expose a tunnel on a different URL prefix, you must manage your own middleware configuration.
9. (Optional): Enter a value for the **Hostname** field.
10. Provide a secret for your certificate in the **Root CA Certificate** dropdown list.

    a. For environments where the Management cluster uses a publicly-signed CA (like ZeroSSL or Let's Encrypt), select **Use Publicly Trusted CA**.

    b. If you manually created a secret in advance, select it from the dropdown list.

    c. For all other cases, select **Create a new secret**. Then, run the following command on the Management cluster to obtain the `caBundle` key:

    ```
    kubectl get kommandercluster -n kommander host-cluster -o go-
    template='{{ .status.ingress.caBundle }}'
    ```

    Copy and paste the output into the **Root CA Certificate** field.
11. Add any **Extra Annotations** as needed.
12. **(Optional) Enable a Proxied Access** Activate a proxied access to enable kubectl access and dashboard observability for the network-restricted cluster from the Management cluster. For more information, see Proxied Access to Network-Restricted Clusters on page 525 .

    Select **Show Advanced**.

13. Add a **Cluster Proxy Domain**.

    Note:

    - If you previously configured a domain wildcard for your cluster, a Cluster Proxy Domain is suggested automatically based on your cluster name. Replace the suggestion if you want to assign a different domain for the proxied cluster.
    - If you want to use the `external-dns` service, specify a **Cluster Proxy Domain** that is within the zones specified in the `--domain-filter` argument of the external-dns deployment manifest stored on the **Management cluster**.

      For example, if the filter is set to `example.com`, a possible domain for the `TUNNEL_PROXY_EXTERNAL_DOMAIN` is `myclusterproxy.example.com`.

14. Establish a DNS record and certificate configuration for the **Cluster Proxy Domain**. You can choose between the default and a custom option.

    Table 45: Table

	DNS record creation	Certificate Management
Default settings: box checked icon	Automatic, handled by `external-dns`	Automatic, handled by `kommander-ca`
Custom settings: box unchecked icon	Manually create a DNS record. The record's A/CNAME value must point to the Management cluster's Traefik IP address, URL or domain. OR Enable `external-dns` with an annotation that points to the **Cluster Proxy Domain**.	Select an existing **TLS** certificate. OR Select an existing **Issuer** or **ClusterIssuer**.

15. Select **Save & Generate kubeconfig** to generate a file required to finish attaching the cluster. A new window appears with instructions on how to finalize attaching the cluster.

What to do next

UI: Finishing Attaching the Existing Cluster on page 512.

UI: Finishing Attaching the Existing Cluster

How to apply the kubeconfig file to create the network tunnel to attach a network-restricted cluster.

About this task

After you have configured your cluster's attachment in UI: Attaching a Network-Restricted Cluster Using a Tunnel Through the UI on page 510, finalize attaching the cluster. Now you must apply the generated manifest to create the network tunnel and complete the attachment process:

Procedure

1. Select the **Download Manifest** link to download the file you generated previously.
2. Copy the `kubectl apply` command from the UI and paste it into your terminal session. Do not run it yet.
3. Ensure you substitute the actual name of the file for the variable. Also ensure you use the `--kubeconfig=<managed_cluster_kubeconfig.conf>` flag to run the command on the Attached or Managed cluster. Run the command.

   Running this command starts the attachment process, which might take several minutes to complete. The Cluster details page is appears automatically when the cluster attachment process completes.
4. (Optional) Select **Verify Connection to Cluster** to send a request to Kommander to refresh the connection information. You can use this option to check to see if the connection is complete, though the Cluster Details page displays automatically when the connection is complete.

   Note: After the initial connection is made and your cluster becomes viewable as attached in the NKP UI, the attachment, federated add-ons, and platform services will still need to be completed. This might take several additional minutes. If a cluster has limited resources to deploy all the federated platform services, the installation of the federated resources will fail, and the cluster may become unreachable in the NKP UI. If this happens, check whether any pods are not getting the resources required.

What to do next

CLI: Using the Network-restricted Cluster on page 519

CLI: Attaching a Network-Restricted Cluster Using a Tunnel Through the CLI

Before you begin

Ensure you have reviewed and followed the steps in Prerequisites for a Tunneled Attachment on page 509 before proceeding.

Procedure

1. Identify the Management Cluster Endpoint.

   Run the following command on the Management cluster to obtain the hostname and CA certificate.

   ```
   hostname=$(kubectl get service -n kommander kommander-traefik -o go-template='{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}')
   b64ca_cert=$(kubectl get secret -n cert-manager kommander-ca -o=go-
   template='{{index .data "tls.crt"}}')
   ```
2. Specify a Workspace Namespace

   » Obtain the desired workspace namespace on the Management cluster for the tunnel gateway:

   ```
   namespace=$(kubectl get workspace default-workspace -o
    jsonpath="{.status.namespaceRef.name}")
   ```

   » Alternatively, you can create a new workspace instead of using an existing workspace: Run the following command, and replace the `<workspace_name>` with the new workspace name:

   ```
   workspace=<workspace_name>
   ```

   Finish creating the workspace:

   ```
   namespace=${workspace}
   ```

```
cat > workspace.yaml <<EOF
apiVersion: workspaces.kommander.mesosphere.io/v1alpha1
kind: Workspace
metadata:
  annotations:
    kommander.mesosphere.io/display-name: ${workspace}
  name: ${workspace}
spec:
  namespaceName: ${namespace}
EOF

kubectl apply -f workspace.yaml
```

You can verify the workspace exists using:

```
kubectl get workspace ${workspace}
```

3. Create a Tunnel Gateway. Create a tunnel gateway on the Management cluster to listen for tunnel agents on remote clusters.

   Note: Kommander uses Traefik 2 ingress, which requires explicit definition of strip prefix middleware as a Kubernetes API object, as opposed to a simple annotation. Kommander provides default middleware that supports creating tunnels only on the `/nkp/tunnel URL` prefix. This is indicated by using the extra annotation,

   `traefik.ingress.kubernetes.io/router.middlewares: kommander-stripprefixes-kubetunnel@kubernetescrd`

   as shown in the code sample that follows. If you want to expose a tunnel on a different URL prefix, you must manage your own middleware configuration.

   a. Establish variables for the certificate secret and gateway. Replace the `<gateway_name>` placeholder with the name of the gateway.

   ```
   cacert_secret=kubetunnel-ca
   gateway=<gateway_name>
   ```

   b. Create the `Secret` and `TunnelGateway` objects.

   ```
   cat > gateway.yaml <<EOF
   apiVersion: v1
   kind: Secret
   metadata:
     namespace: ${namespace}
     name: ${cacert_secret}
   data:
     ca.crt:
       ${b64ca_cert}
   ---
   apiVersion: kubetunnel.d2iq.io/v1alpha1
   kind: TunnelGateway
   metadata:
     namespace: ${namespace}
     name: ${gateway}
   spec:
     ingress:
       caSecretRef:
         namespace: ${namespace}
         name: ${cacert_secret}
       loadBalancer:
         hostname: ${hostname}
       urlPathPrefix: /nkp/tunnel
       extraAnnotations:
   ```

```
      kubernetes.io/ingress.class: kommander-traefik
      traefik.ingress.kubernetes.io/router.tls: "true"
      traefik.ingress.kubernetes.io/router.middlewares: kommander-stripprefixes-
kubetunnel@kubernetescrd
EOF

kubectl apply -f gateway.yaml
```

c. You can verify the gateway exists using the command.

```
kubectl get tunnelgateway -n ${namespace} ${gateway}
```

CLI: Creating a Tunnel Connector

Connect a remote, edge, or network-restricted cluster

About this task

Create a tunnel connector on the Management cluster for the remote cluster.

Procedure

1. Establish a variable for the connector. Provide the name of the connector, by replacing the `<connector_name>` placeholder:

   ```
   connector=<connector_name>
   ```

2. Create the `TunnelConnector` object.

   ```
   cat > connector.yaml <<EOF
   apiVersion: kubetunnel.d2iq.io/v1alpha1
   kind: TunnelConnector
   metadata:
     namespace: ${namespace}
     name: ${connector}
   spec:
     gatewayRef:
       name: ${gateway}
   EOF

   kubectl apply -f connector.yaml
   ```

   After you create the `TunnelConnector` object, NKP creates a `manifest.yaml`. This `manifest.yaml` contains the configuration information for the components required by the tunnel for a specific cluster.

3. Verify the connector exists.

   ```
   kubectl get tunnelconnector -n ${namespace} ${connector}
   ```

4. Wait for the tunnel connector to reach the `Listening` state and export the agent manifest.

   ```
   while [ "$(kubectl get tunnelconnector -n ${namespace} ${connector} -o
    jsonpath="{.status.state}")" != "Listening" ]
   do
     sleep 5
   done

   manifest=$(kubectl get tunnelconnector -n ${namespace} ${connector} -o
    jsonpath="{.status.tunnelAgent.manifestsRef.name}")
   while [ -z ${manifest} ]
   do
     sleep 5
   ```

```
 manifest=$(kubectl get tunnelconnector -n ${namespace} ${connector} -o
 jsonpath="{.status.tunnelAgent.manifestsRef.name}")
done
```

The `manifest.yaml` is applied successfully after the command completes.

5. Fetch the `manifest.yaml` to use it in the following section.

```
kubectl get secret -n ${namespace} ${manifest} -o jsonpath='{.data.manifests\.yaml}'
 | base64 -d > manifest.yaml
```

> **Note:** When attaching several clusters, ensure that you fetch the `manifest.yaml` of the cluster you are attempting to attach. Using the wrong combination of `manifest.yaml` and cluster will cause the attachment to fail.

CLI: Setting Up the Network-restricted Cluster

About this task

In the following commands, the `--kubeconfig` flag ensures that you set the context to the Attached or Managed cluster. For alternatives and recommendations around setting your context, see Commands within a kubeconfig File on page 32.

Procedure

1. Apply the `manifest.yaml` file to the Attached or Managed cluster and deploy the tunnel agent.

```
kubectl apply --kubeconfig=<managed_cluster_kubeconfig.conf> -f manifest.yaml
```

2. Check the status of the created pods using:

```
kubectl get pods --kubeconfig=<managed_cluster_kubeconfig.conf> -n kubetunnel
```

After a short time, expect to see a `post-kubeconfig` pod that reaches `Completed` state and a `tunnel-agent` pod that stays in `Running` state.

```
NAME                            READY   STATUS      RESTARTS   AGE
post-kubeconfig-j2ghk           0/1     Completed   0          14m
tunnel-agent-f8d9f4cb4-thx8h    1/1     Running     0          14m
```

CLI: Adding the Network-restricted Cluster Into Kommander

When you create a cluster using the NKP CLI, it does not attach automatically.

About this task

Procedure

1. On the Management cluster, wait for the tunnel to be connected by the tunnel agent.

```
while [ "$(kubectl get tunnelconnector -n ${namespace} ${connector} -o
 jsonpath="{.status.state}")" != "Connected" ]
do
  sleep 5
done
```

2. Establish variables for the managed cluster. Replace the `<private_cluster>` placeholder with the name of the managed cluster:

```
managed=<private-cluster>
```

```
display_name=${managed}
```

3. Update the `KommanderCluster` object:

```
cat > kommander.yaml <<EOF
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  namespace: ${namespace}
  name: ${managed}
  annotations:
    kommander.mesosphere.io/display-name: ${display_name}
spec:
  clusterTunnelConnectorRef:
    name: ${connector}
EOF

kubectl apply -f kommander.yaml
```

4. Wait for the Attached or Managed cluster to join.

```
while [ "$(kubectl get kommandercluster -n ${namespace} ${managed} -o
 jsonpath='{.status.phase}')" != "Joined" ]
do
  sleep 5
done

kubefed=$(kubectl get kommandercluster -n ${namespace} ${managed} -o
 jsonpath="{.status.kubefedclusterRef.name}")
while [ -z "${kubefed}" ]
do
  sleep 5
  kubefed=$(kubectl get kommandercluster -n ${namespace} ${managed} -o
 jsonpath="{.status.kubefedclusterRef.name}")
done

kubectl wait --for=condition=ready --timeout=60s kubefedcluster -n kube-federation-
system ${kubefed}
```

After the command is executed, your cluster becomes visible in the NKP UI, and you can start using it. Its metrics will be accessible through different dashboards, such as Grafana, Karma, etc.

CLI: Creating a Network Policy for the Tunnel Server

About this task

This step is optional but improves security by restricting which remote hosts can connect to the tunnel.

Procedure

1. Apply a network policy that restricts tunnel access to specific namespaces and IP blocks.
   The following example permits connections from:

   - Pods running in the `kommander` and `kube-federation-system` namespace.
   - Remote clusters with IP addresses in the ranges 192.0.2.0 to 192.0.2.255 and 203.0.113.0 to 203.0.113.255.
   - Pods running in namespaces with a label `kubetunnel.d2iq.io/networkpolicy` that match the tunnel name and namespace.

```
cat > net.yaml <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  namespace: ${namespace}
  name: ${connector}-deny
  labels:
    kubetunnel.d2iq.io/tunnel-connector: ${connector}
    kubetunnel.d2iq.io/networkpolicy-type: "tunnel-server"
spec:
  podSelector:
    matchLabels:
      kubetunnel.d2iq.io/tunnel-connector: ${connector}
  policyTypes:
  - Ingress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  namespace: ${namespace}
  name: ${connector}-allow
  labels:
    kubetunnel.d2iq.io/tunnel-connector: ${connector}
    kubetunnel.d2iq.io/networkpolicy-type: "tunnel-server"
spec:
  podSelector:
    matchLabels:
      kubetunnel.d2iq.io/tunnel-connector: ${connector}
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: "kube-federation-system"
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: "kommander"
    - namespaceSelector:
        matchLabels:
          kubetunnel.d2iq.io/networkpolicy: ${connector}-${namespace}
    - ipBlock:
        cidr: 192.0.2.0/24
    - ipBlock:
        cidr: 203.0.113.0/24
EOF

kubectl apply -f net.yaml
```

2. To enable applications running in another namespace to access the attached cluster, add the label `kubetunnel.d2iq.io/networkpolicy=${connector}-${namespace}` to the target namespace.

```
kubectl label ns ${namespace} kubetunnel.d2iq.io/networkpolicy=${connector}-
${namespace}
```

   All pods in the target namespace can now reach the attached cluster services.

What to do next

- (Optional): If you want to access the network-restricted attached cluster from the Management cluster, Enabling Proxied Access Using the CLI on page 527
- (Alternatively), start CLI: Using the Network-restricted Cluster on page 519

CLI: Using the Network-restricted Cluster

About this task

To access services running on the remote, edge, or network-restricted cluster from the Management cluster, connect to the tunnel proxy.

Procedure

1. You can use these three methods.

   » If the client program supports the use of a `kubeconfig` file, use the network-restricted cluster's kubeconfig.

   » If the client program supports SOCKS5 proxies, use the proxy directly.

   » Otherwise, deploy a proxy server on the Management cluster.

2. **Network-restricted Cluster Service** These sections require a service to run on the Attached or Managed network-restricted cluster.

   As an example, start the following service.

```
service_namespace=test
service_name=webserver
service_port=8888
service_endpoint=${service_name}.${service_namespace}.svc.cluster.local:
${service_port}

cat > nginx.yaml <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: ${service_namespace}
---
apiVersion: apps/v1
kind: Deployment
metadata:
  namespace: ${service_namespace}
  name: nginx-deployment
  labels:
    app: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-app
```

```
  template:
    metadata:
      labels:
        app: nginx-app
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  namespace: ${service_namespace}
  name: ${service_name}
spec:
  selector:
    app: nginx-app
  type: ClusterIP
  ports:
  - targetPort: 80
    port: ${service_port}
EOF

kubectl apply -f nginx.yaml

kubectl rollout status deploy -n ${service_namespace} nginx-deployment
```

On the Attached or Managed cluster, a client Job can access this service using.

```
cat > curl.yaml <<EOF
apiVersion: batch/v1
kind: Job
metadata:
  name: curl
spec:
  template:
    spec:
      containers:
      - name: curl
        image: curlimages/curl:7.76.0
        command: ["curl", "--silent", "--show-error", "http://${service_endpoint}"]
      restartPolicy: Never
  backoffLimit: 4
EOF

kubectl apply -f curl.yaml

kubectl wait --for=condition=complete job curl

podname=$(kubectl get pods --selector=job-name=curl --field-
selector=status.phase=Succeeded -o jsonpath='{.items[0].metadata.name}')

kubectl logs ${podname}
```

The final command returns the default Nginx web page.

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
```

```
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support, see
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

What to do next

(Optional): If you want to manage the attached cluster from the Management cluster, Enabling Proxied Access Using the UI on page 526.

Using kubeconfig File

About this task

This is primarily useful for running `kubectl` commands on the Management cluster to monitor the network-restricted, Managed or Attached cluster.

On the Management cluster, a `kubeconfig` file for the Attached or Managed cluster configured to use the tunnel proxy is available as a Secret.

Procedure

1. The Secret's name can be identified using.

```
kubeconfig_secret=$(kubectl get tunnelconnector -n ${namespace} ${connector} -o
 jsonpath='{.status.kubeconfigRef.name}')
```

2. After setting `service_namespace` and `service_name` to the service resource, run this command on the Management cluster.

```
cat > get-service.yaml <<EOF
apiVersion: batch/v1
kind: Job
metadata:
  name: get-service
spec:
  template:
    spec:
      containers:
      - name: kubectl
        image: bitnami/kubectl:1.19
        command: ["kubectl", "get", "service", "-n", "${service_namespace}",
 "${service_name}"]
        env:
        - name: KUBECONFIG
          value: /tmp/kubeconfig/kubeconfig
```

```
        volumeMounts:
        - name: kubeconfig
          mountPath: /tmp/kubeconfig
      volumes:
      - name: kubeconfig
        secret:
          secretName: "${kubeconfig_secret}"
      restartPolicy: Never
  backoffLimit: 4
EOF

kubectl apply -n ${namespace} -f get-service.yaml

kubectl wait --for=condition=complete --timeout=5m job -n ${namespace} get-service

podname=$(kubectl get pods -n ${namespace} --selector=job-name=get-service --field-
selector=status.phase=Succeeded -o jsonpath='{.items[0].metadata.name}')

kubectl logs -n ${namespace} ${podname}
```

Using SOCKS5 Proxy Directly

About this task

Procedure

1. To use the SOCKS5 proxy directly, obtain the SOCKS5 proxy endpoint using.

```
proxy_service=$(kubectl get tunnelconnector -n ${namespace} ${connector} -o
 jsonpath='{.status.tunnelServer.serviceRef.name}')

socks_proxy=$(kubectl get service -n ${namespace} "${proxy_service}" -o
 jsonpath='{.spec.clusterIP}{":"}{.spec.ports[?(@.name=="proxy")].port}')
```

2. Provide the value of `${socks_proxy}` as the SOCKS5 proxy to your client.

   For example, since `curl` supports SOCKS5 proxies, the Attached or Managed service started above can be accessed from the Management cluster by adding the SOCKS5 proxy to the `curl` command. After setting `service_endpoint` to the service endpoint, on the Management cluster run.

```
cat > curl.yaml <<EOF
apiVersion: batch/v1
kind: Job
metadata:
  name: curl
spec:
  template:
    spec:
      containers:
      - name: curl
        image: curlimages/curl:7.76.0
        command: ["curl", "--silent", "--show-error", "--socks5-hostname",
 "${socks_proxy}", "http://${service_endpoint}"]
      restartPolicy: Never
  backoffLimit: 4
EOF

kubectl apply -f curl.yaml

kubectl wait --for=condition=complete --timeout=5m job curl
```

```
podname=$(kubectl get pods --selector=job-name=curl --field-
selector=status.phase=Succeeded -o jsonpath='{.items[0].metadata.name}')

kubectl logs ${podname}
```

The final command returns the same output as for the job on the Attached or Managed cluster, demonstrating that the job on the Management cluster accessed the service running on the Attached or Managed cluster.

Using Deployed Proxy

About this task

Procedure

1. To deploy a proxy on the Management cluster, obtain the SOCKS5 proxy endpoint using.

```
proxy_service=$(kubectl get tunnelconnector -n ${namespace} ${connector} -o
 jsonpath='{.status.tunnelServer.serviceRef.name}')

socks_proxy=$(kubectl get service -n ${namespace} "${proxy_service}" -o
 jsonpath='{.spec.clusterIP}{":"}{.spec.ports[?(@.name=="proxy")].port}')
```

2. Provide the value of `${socks_proxy}` as the SOCKS5 proxy to a proxy deployed on the Management cluster. After setting `service_endpoint` to the service endpoint, on the Management cluster run.

```
cat > nginx-proxy.yaml <<EOF
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: nginx-proxy-crt
spec:
  secretName: nginx-proxy-crt-secret
  dnsNames:
  - nginx-proxy-service.${namespace}.svc.cluster.local
  issuerRef:
    group: cert-manager.io
    kind: ClusterIssuer
    name: kubernetes-ca
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-proxy
  labels:
    app: nginx-proxy-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx-proxy-app
  template:
    metadata:
      labels:
        app: nginx-proxy-app
    spec:
      containers:
      - name: nginx-proxy
        image: mesosphere/ghostunnel:v1.5.3-server-backend-proxy
        args:
        - "server"
```

```
        - "--listen=:443"
        - "--target=${service_endpoint}"
        - "--cert=/etc/certs/tls.crt"
        - "--key=/etc/certs/tls.key"
        - "--cacert=/etc/certs/ca.crt"
        - "--unsafe-target"
        - "--disable-authentication"
        env:
        - name: ALL_PROXY
          value: socks5://${socks_proxy}
        ports:
        - containerPort: 443
        volumeMounts:
        - name: certs
          mountPath: /etc/certs
      volumes:
      - name: certs
        secret:
          secretName: nginx-proxy-crt-secret
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-proxy-service
spec:
  selector:
    app: nginx-proxy-app
  type: ClusterIP
  ports:
  - targetPort: 443
    port: 8765
EOF

kubectl apply -n ${namespace} -f nginx-proxy.yaml

kubectl rollout status deploy -n ${namespace} nginx-proxy

proxy_port=$(kubectl get service -n ${namespace} nginx-proxy-service -o
 jsonpath='{.spec.ports[0].port}')
```

3. Any client running on the Management cluster can now access the service running on the Attached or Managed cluster using the proxy service endpoint. Note that the #curl# job runs in the same namespace as the proxy to provide access to the CA certificate secret.

```
cat > curl.yaml <<EOF
apiVersion: batch/v1
kind: Job
metadata:
  name: curl
spec:
  template:
    spec:
      containers:
      - name: curl
        image: curlimages/curl:7.76.0
        command:
        - curl
        - --silent
        - --show-error
        - --cacert
        - /etc/certs/ca.crt
        - https://nginx-proxy-service.${namespace}.svc.cluster.local:${proxy_port}
```

```
        volumeMounts:
        - name: certs
          mountPath: /etc/certs
      volumes:
      - name: certs
        secret:
          secretName: nginx-proxy-crt-secret
      restartPolicy: Never
  backoffLimit: 4
EOF

kubectl apply -n ${namespace} -f curl.yaml

kubectl wait --for=condition=complete --timeout=5m job -n ${namespace} curl

podname=$(kubectl get pods -n ${namespace} --selector=job-name=curl --field-
selector=status.phase=Succeeded -o jsonpath='{.items[0].metadata.name}')

kubectl logs -n ${namespace} ${podname}
```

The final command returns the same output as the job on the Attached or Managed cluster, demonstrating that the job on the Management cluster accessed the service running on the network-restricted cluster.

Proxied Access to Network-Restricted Clusters

Enabling a proxied access allows you to access Attached and Managed clusters that are network-restricted, in a private network, firewalled, or at the edge.

> **Note:** This section only applies to clusters with networking restrictions that were attached through a secure tunnel

You can attach clusters that are in a private network (clusters that have networking restrictions or are at the edge). Nutanix provides the option of using a secure tunnel or a tunneled attachment to attach a Kubernetes cluster to the Management cluster. To access these attached clusters through `kubectl` or monitor its resources through the Management cluster, you have to be in the same network, or enable a proxied access.

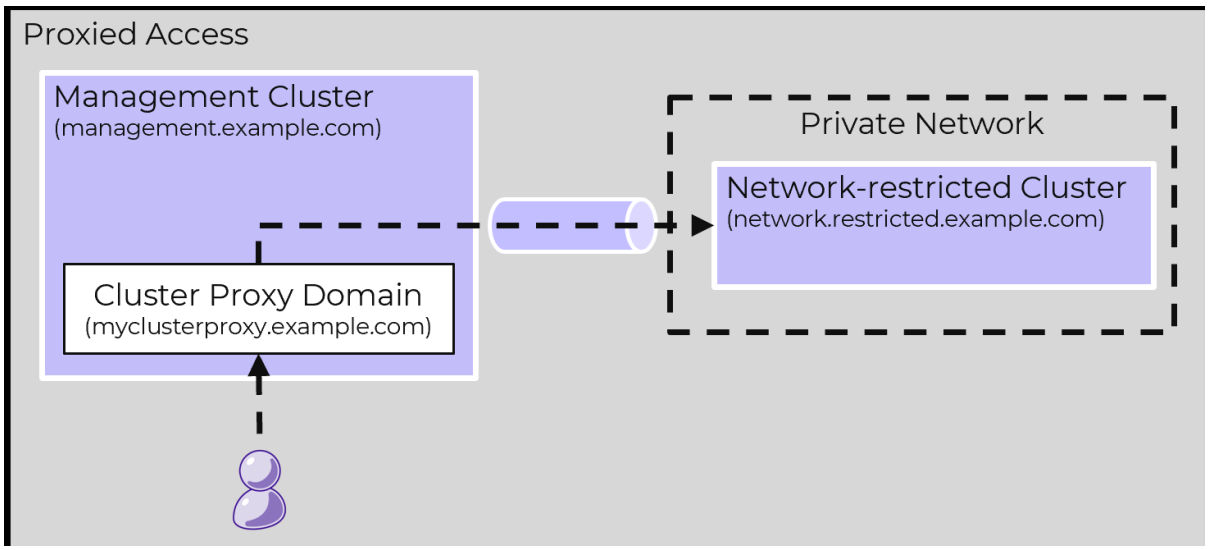


Figure 18: Proxied Access

Enabling the proxied access for a network-restricted cluster makes it possible for NKP to authenticate user requests (regardless of the identity provider) through the Management cluster's authentication proxy. This is helpful,when the cluster you are trying to reach is in a different network. The proxied access allows you to:

- Access and observe the cluster's monitoring and logging services from the Management cluster, for example:
  - Access the cluster's Grafana, Kubernetes, and Kubecost dashboards from the Management cluster.
  - Use the CLI to print a cluster's service UR so that you can access the cluster's dashboards.
- Access and perform operations on the network-restricted cluster from the Management cluster, for example:
  - Generate an API token (Generate Token option, from the upper right corner of the UI) that allows you to authenticate to the network-restricted cluster.
  - Upon authentication, use `kubectl` to manage the network-restricted cluster.

You can perform the previous actions without being in the same network as the network-restricted cluster.

Enabling Proxied Access Using the UI

Enable proxied access using the UI.

Procedure

For instructions on how to enable a proxied access while you attach a cluster with the UI, see Cluster Attachment with Networking Restrictions on page 508.

If you attached your Kubernetes cluster to your NKP environment already, it is not possible to enable the proxied access using the UI. Use the CLI as explained in Enabling Proxied Access Using the CLI on page 527.

Configuring a Wildcard Domain for Your Proxied Cluster

About this task

Wildcard domains are helpful in multi-cluster environments. When you set up a wildcard domain, every time you attach an additional network-restricted cluster through a proxy, the NKP UI pre-fills a domain for the cluster automatically.

After you set up a wildcard domain in the `kommander.yaml` as explained in the following section, NKP will suggest domains for attached clusters automatically based on the wildcard domain + the name of the cluster, for example:

Procedure

Table 46: Wildcard Domain

Wildcard Domain	Cluster Name	Cluster Domain
*.example.com	development.cluster	development.cluster.example.com
*.example.com	janedoe	janedoe.example.com

To Set up a Wildcard Domain:

1. Open the `kommander.yaml` file.

   a. If you have not installed the Kommander component yet, initialize the configuration file, so you can edit it in the following steps.

      Warning: Initialize this file only once. Otherwise, you will overwrite previous customizations.

   b. If you have installed the Kommander component already, open the existing `kommander.yaml` with the editor of your choice.

2. Adjust the `apps` section of your `kommander.yaml` file to include these values.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
apps:
[...]
  kubetunnel:
    enabled: true
    values: |
      proxiedAccess:
         clusterDomainWildcard: "*.example.com"
```

3. Use the configuration file to install or update the Kommander component.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

   Whenever you attach a network-restricted cluster, the UI will suggest a new domain based on the wildcard domain and cluster name.

4. Configure a DNS Record or DNS Service

   The clusters will not be available through the established domain until you manually create a DNS record or enable `external-dns` to automatically manage the creation of records.

   Nutanix recommends enabling the External DNS service to manage your records automatically. However, you can choose to create your records manually.

Enabling Proxied Access Using the CLI

Enabling a proxied access allows you to access Attached and Managed clusters that are network-restricted, in a private network, firewalled, or at the edge.

About this task

This section only applies to clusters with networking restrictions that were attached through a secure tunnel.

Before you begin

- You have attached a network-restricted cluster.
- The Management and network-restricted cluster are on the same NKP version.
- You have a domain that you can use to put on top of the network-restricted cluster’s domain to redirect user requests (Cluster Proxy Domain).

- A DNS record to map your domain to your cluster. There are two supported options for this:
  - Manual DNS record creation:

    Create a DNS record manually. The record's A/CNAME value must point to the Management cluster's Traefik IP address, URL, or domain. Use one record per proxied cluster.
  - Automatic DNS record creation:

    A service that creates and maintains your DNS record automatically. For this method, enable the external-dns service on the Management cluster before configuring the proxy. For more information, see external-dns service on the Management cluster

The following pages walk you through enabling the proxied access on the network-restricted cluster. Establish the following environment variables on the Management cluster. For more information on switching cluster contexts, see Commands within a kubeconfig File on page 32.

The following commands allow you to run most commands without replacing the information manually.

Procedure

1. Set the `WORKSPACE_NAMESPACE` environment variable to the name of your network-restricted cluster's workspace namespace.

   ```
   export WORKSPACE_NAMESPACE=<workspace namespace>
   ```

2. Set the variable to the proxy domain through which your cluster should be available.

   TUNNEL_PROXY_EXTERNAL_DOMAIN=<myclusterproxy.example.com>

   If you want to use the `external-dns` service, specify a `TUNNEL_PROXY_EXTERNAL_DOMAIN` that is within the zones specified in the `--domain-filter` argument of the external-dns deployment manifest stored on the Management cluster.

   For example, if the filter is set to `example.com`, a possible domain for the `TUNNEL_PROXY_EXTERNAL_DOMAIN` is `myclusterproxy.example.com`.

3. Establish a variable that points to the name of the network-restricted cluster.

   The name of the network-restricted cluster is established in the KommanderCluster object.

   ```
   NETWORK_RESTRICTED_CLUSTER=<name_of_restricted_cluster>
   ```

4. Given that each cluster can only have one proxy domain, reuse the name of the network-restricted cluster for the proxy object.

   ```
   TUNNEL_PROXY_NAME=${NETWORK_RESTRICTED_CLUSTER}
   ```

5. Obtain the name of the connector and set it to a variable.

   ```
   TUNNEL_CONNECTOR_NAME=$(kubectl get kommandercluster -n
    ${WORKSPACE_NAMESPACE} ${NETWORK_RESTRICTED_CLUSTER} -o
    template='{{ .spec.clusterTunnelConnectorRef.name }}')
   ```

Creating a TunnelProxy Object

Before you begin

Procedure

1. In the Management cluster, create a `TunnelProxy` object for your proxied cluster and assign it a unique domain. This domain forwards all user authentication requests through the Management cluster and is used to generate a URL that exposes the cluster's dashboards (`clusterProxyDomain`).
2. Do one of the following.
   - » To back the domain, you require both a certificate and a DNS record. If you choose the default configuration, NKP will handle the certificate creation (self-signed certificate), but you must create a DNS record manually.
   - » Alternatively, you can set up a different Certificate Authority to handle the certificate creation and rotation for your domain. You can also set up the `external-dns` service to automatically create a DNS record.
3. Here are some examples of possible configuration combinations.

Note:

- Ensure you set the required variables.
- Ensure you run the following commands on the Management cluster. For more information on switching cluster contexts, Commands within a kubeconfig File on page 32.
- Example 1: Example 1: Domain with Default Certificate and Automatic DNS Record Creation (Requires External DNS) on page 529
- Example 2: Example 2: Domain with Default Certificate and Default DNS Setup (Requires Manually-created DNS) on page 530
- Example 3: Example 3: Domain with Auto-generated ACME Certificate and Automatic DNS Record Creation (Requires External DNS) on page 530
- Example 4: Example 4: Domain with Custom Certificate (Requires Certificate Secret) and Automatic DNS Record Creation (Requires External DNS) on page 531

Example 1: Domain with Default Certificate and Automatic DNS Record Creation (Requires External DNS)

In this example, the following configuration applies:

- **Certificate** - The domain uses a self-signed certificate created by NKP.
- **DNS record** - The external-dns manages the creation of a DNS record automatically. For it to work, ensure you have enabled Configuring External DNS with the CLI: Management or Pro Cluster on page 1050.

```
cat > tunnelproxy.yaml <<EOF | kubectl apply -f -
apiVersion: kubetunnel.d2iq.io/v1alpha1
kind: TunnelProxy
metadata:
  name: ${TUNNEL_PROXY_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  clusterProxyDomain: ${TUNNEL_PROXY_EXTERNAL_DOMAIN}
  tunnelConnectorRef:
    name: ${TUNNEL_CONNECTOR_NAME}
  ingress:
```

```
    annotations:
      external-dns.alpha.kubernetes.io/hostname: ${TUNNEL_PROXY_EXTERNAL_DOMAIN}
EOF
```

The **spec.ingress.annotations** field contains the annotation required for DNS record management. For more information, see DNS Record Creation with External DNS on page 1050.

Example 2: Domain with Default Certificate and Default DNS Setup (Requires Manually-created DNS)

In this example, the following configuration applies:

- **Certificate** - The domain uses a self-signed certificate created by NKP.
- **DNS record** - For the domain to be recognized by the cluster, ensure you manually create a DNS record. The record's A/CNAME value must point to the Management cluster's Traefik IP address, URL, or domain. Create a record per proxied cluster.

```
In this example, the following configuration applies:Certificate - The domain uses a
 self-signed certificate created by NKP.DNS record - For the domain to be recognized by
 the cluster, ensure you manually create a DNS record. The record’s A/CNAME value must
 point to the Management cluster’s Traefik IP address, URL or domain. Create a record
 per proxied cluster.cat > tunnelproxy.yaml <<EOF | kubectl apply -f -
apiVersion: kubetunnel.d2iq.io/v1alpha1
kind: TunnelProxy
metadata:
  name: ${TUNNEL_PROXY_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  clusterProxyDomain: ${TUNNEL_PROXY_EXTERNAL_DOMAIN}
  tunnelConnectorRef:
    name: ${TUNNEL_CONNECTOR_NAME}
EOF
```

Example 3: Domain with Auto-generated ACME Certificate and Automatic DNS Record Creation (Requires External DNS)

In this example, the following configuration applies:

- **Certificate** - The domain uses a cert-manager to enable an ACME-based Certificate Authority. This CA automatically issues and rotates your certificates. By default, NKP uses Let's Encrypt.
- **DNS record** -

  The external-dns manages the creation of a DNS record automatically. For it to work, ensure you have enabled Configuring External DNS with the CLI: Management or Pro Cluster on page 1050.

1. Set the environment variable for your issuing object:

   This can be a ClusterIssuer or Issuer. For more information, see Advanced Configuration: ClusterIssuer on page 1047.

   ```
   ISSUER_KIND=ClusterIssuer
   ```

2. Set the environment variable for your CA:

   Replace letsEncrypt if you are using another ACME-based certificate authority.

   ```
   ISSUER_NAME=letsEncrypt
   ```

3. Create the TunnelProxy:

   ```
   cat > tunnelproxy.yaml <<EOF | kubectl apply -f -
   apiVersion: kubetunnel.d2iq.io/v1alpha1
   ```

```
kind: TunnelProxy
metadata:
  name: ${TUNNEL_PROXY_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  clusterProxyDomain: ${TUNNEL_PROXY_EXTERNAL_DOMAIN}
  tunnelConnectorRef:
    name: ${TUNNEL_CONNECTOR_NAME}
  ingress:
    annotations:
      external-dns.alpha.kubernetes.io/hostname: ${TUNNEL_PROXY_EXTERNAL_DOMAIN}
    certificate:
      issuerRef:
        kind: ${ISSUER_KIND}
        name: ${ISSUER_NAME}
EOF
```

For more information, see DNS Record Creation with External DNS on page 1050.

Example 4: Domain with Custom Certificate (Requires Certificate Secret) and Automatic DNS Record Creation (Requires External DNS)

In this example, the following configuration applies:

- **Certificate** - The domain uses a custom certificate created manually. Ensure you reference the `<certificate_secret_name>`.
- **DNS record** -

  The external-dns manages the creation of a DNS record automatically. For it to work, ensure you have enabled Configuring External DNS with the CLI: Management or Pro Cluster on page 1050.

1. Set an environment variable for the name of your custom certificate.

   For more information, see Configuring the Kommander Installation with a Custom Domain and Certificate on page 1042.

   ```
   CERTIFICATE_SECRET_NAME=<custom_certificate_secret_name>
   ```

2. (Optional): If you do not have a secret yet and want to create one pointing at the certificate, run the following command.

   ```
   kubectl create secret tls ${CERTIFICATE_SECRET_NAME} -n ${WORKSPACE_NAMESPACE} --
   key="tls.key" --cert="tls.crt"
   ```

3. Create the TunnelProxy:

   ```
   cat > tunnelproxy.yaml <<EOF | kubectl apply -f -
   apiVersion: kubetunnel.d2iq.io/v1alpha1
   kind: TunnelProxy
   metadata:
     name: ${TUNNEL_PROXY_NAME}
     namespace: ${WORKSPACE_NAMESPACE}
   spec:
     clusterProxyDomain: ${TUNNEL_PROXY_EXTERNAL_DOMAIN}
     tunnelConnectorRef:
       name: ${TUNNEL_CONNECTOR_NAME}
     ingress:
       annotations:
         external-dns.alpha.kubernetes.io/hostname: ${TUNNEL_PROXY_EXTERNAL_DOMAIN}
       certificate:
         certificateSecretRef:
            name: ${CERTIFICATE_SECRET_NAME}
   ```

```
EOF
```

For more information, see Configure Custom Domains or Custom Certificates post Kommander Installation on page 555.

Enabling the TunnelProxy Object in KommanderCluster

Enable the TunnelProxy Object in KommanderCluster.

Before you begin

- Ensure to set the required variables as described in Enabling Proxied Access Using the CLI on page 527 and create a TunnelProxy object as described in Creating a TunnelProxy Object on page 528 before you run the following commands.
- Ensure you run the following command on the Management cluster.

  For more information on switching cluster contexts, see Commands within a kubeconfig File on page 32.

To enable the TunnelProxy, reference the object in the KommanderCluster object:

Procedure

On the Management cluster, patch the `KommanderCluster` object with the name of the `TunnelProxy` you created on the previous page.

```
kubectl patch --type merge kommanderclusters -n ${WORKSPACE_NAMESPACE}
 ${NETWORK_RESTRICTED_CLUSTER} --patch "{\"spec\": {\"clusterTunnelProxyConnectorRef\":
 { \"name\": \"${TUNNEL_PROXY_NAME}\"}}}"
```

Verifying the Proxy

Verify the proxy.

Before you begin

Ensure you run the following command on the Management cluster. For more information around switching cluster contexts, see Commands within a kubeconfig File on page 32.

On the Management cluster:

Procedure

1. Verify that the following conditions for the `TunnelProxy` configuration are met.

   ```
   kubectl wait --for=condition=ClientAuthReady=true --timeout=300s -n
    ${WORKSPACE_NAMESPACE} tunnelproxy/${TUNNEL_PROXY_NAME}
   kubectl wait --for=condition=ReverseProxyReady=true --timeout=300s -n
    ${WORKSPACE_NAMESPACE} tunnelproxy/${TUNNEL_PROXY_NAME}
   kubectl wait --for=condition=available -n ${WORKSPACE_NAMESPACE} deploy  -l control-
   plane=${TUNNEL_PROXY_NAME}-kubetunnel-reverse-proxy-rp
   ```

   The output should look like this.

   ```
   tunnelproxy.kubetunnel.d2iq.io/test condition met
   tunnelproxy.kubetunnel.d2iq.io/test condition met
   deployment.apps/${TUNNEL_PROXY_NAME}-kubetunnel-reverse-proxy-rp condition met
   ```

2. Verify that the `TunnelProxy` is correctly assigned and connected to your cluster.

```
curl -Lk -s -o /dev/null -w "%{http_code}" https://${TUNNEL_PROXY_EXTERNAL_DOMAIN}/
nkp/grafana
```

The output should return a successful HTTP response status.

```
200
```

You can access the network-restricted cluster dashboards and use `kubectl` to manage its resources from the Management cluster.

NKP-created Kubernetes Cluster

Starting with NKP 2.6, when you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments.

If you have several Pro Clusters and want to turn one of them into a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535

Attaching an NKP-created Cluster Using the CLI

About this task

> **Note:** These steps are only applicable if you do not set a `WORKSPACE_NAMESPACE` when creating a cluster. If you already set a `WORKSPACE_NAMESPACE`, then you do not need to perform these steps since the cluster is already attached to the workspace.

Starting with NKP 2.6, when you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments.

However, if you do not set a workspace, the attached cluster will be created in the `default` workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

Procedure

1. Confirm you have your MANAGED_CLUSTER_NAME variable set with the following command:

```
echo ${MANAGED_CLUSTER_NAME}
```

2. Retrieve your kubeconfig from the cluster you have created without setting a workspace.

```
nkp get kubeconfig --cluster-name ${MANAGED_CLUSTER_NAME} >
 ${MANAGED_CLUSTER_NAME}.conf
```

> **Caution:** Nutanix support is unable to recover forgotten credentials. The following best practices will prevent Kommander UI lockouts.
>
> - Backup the kubeconfig file.
> - Save your credentials.
> - Configure an IDP (identity provider) instead of using a username and password credentials.

3. You can now either [attach it in the UI] (link to attaching it to the workspace through UI that was earlier) or attach your cluster to the workspace you want in the CLI.

> **Note:** This is only necessary if you never set the workspace of your cluster upon creation.

4. Retrieve the workspace where you want to attach the cluster:

```
kubectl get workspaces -A
```

5. Set the WORKSPACE_NAMESPACE environment variable.

```
export WORKSPACE_NAMESPACE=<workspace-namespace>
```

6. You need to create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the kubeconfig secret value of your cluster.

```
kubectl -n default get secret ${MANAGED_CLUSTER_NAME}-kubeconfig -o go-
template='{{.data.value}}{{ "\n"}}'
```

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new attached-cluster-kubeconfig.yaml file.

```
apiVersion: v1
kind: Secret
metadata:
  name: <your-managed-cluster-name>-kubeconfig
  labels:
    cluster.x-k8s.io/cluster-name: <your-managed-cluster-name>
type: cluster.x-k8s.io/secret
data:
  value: <value-you-copied-from-secret-above>
```

8. Create this secret in the desired workspace:

```
kubectl apply -f attached-cluster-kubeconfig.yaml --namespace
 ${WORKSPACE_NAMESPACE}
```

9. Create this `kommandercluster` object to attach the cluster to the workspace.

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  kubeconfigRef:
    name: ${MANAGED_CLUSTER_NAME}-kubeconfig
  clusterRef:
    capiCluster:
      name: ${MANAGED_CLUSTER_NAME}
EOF
```

10. You can now view this cluster in your Workspace in the UI and you can confirm its status by running the command below. It might take a few minutes to reach "Joined" status.

```
kubectl get kommanderclusters -A
```

If you have several Pro Clusters and want to turn one of them into a Managed Cluster to be centrally administrated by a Management Cluster, refer to Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

Accessing a Managed or Attached Cluster
How to access a Managed or Attached cluster with credentials

About this task

Access your Clusters Using your UI Administrator Credentials.

After the cluster is attached, retrieve a custom `kubeconfig` file from the UI.

> **Caution:** Nutanix support is unable to recover forgotten credentials. Prevent Kommander UI lockouts with the following best practices.
>
> - Backup the kubeconfig file.
> - Save your credentials.
> - Configure an IDP (identity provider) instead of using a username and password credentials.

Procedure

1. Select the username in the top right corner, and then select **Generate Token**.
2. Select the cluster name and follow the instructions to assemble a `kubeconfig` for accessing its Kubernetes API.

   > **Note:** If the UI prompts you to log inlog on, use the credentials you normally use to access the UI.

   You can also retrieve a custom `kubeconfig` file by visiting the `/token` endpoint on the Kommander cluster domain (example URL: `https://your-server-name.your-region-2.elb.service.com/token/`. Selecting the cluster's name displays the instructions to assemble a `kubeconfig` for accessing its Kubernetes API.

Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster

If you are an NKP Pro customer, you can easily convert your independent clusters into a multi-cluster environment if you upgrade to an Ultimate license.

This section provides information on how you can turn your Pro Clusters into Managed clusters.

Prerequisites: General Prerequisites for Your Cluster Conversion
General requirements for converting your DKP Essential or NKP Pro Clusters into NKP Ultimate Managed Clusters.

Starting with DKP 2.5, you can convert your DKP Essential Clusters into DKP Managed Enterprise or NKP Ultimate Clusters.

Starting with NKP 2.12, you can convert your NKP Pro Clusters into NKP Ultimate Clusters.

To convert your DKP or NKP clusters, ensure you meet the following requirements:

- An NKP Management cluster with a valid NKP Ultimate license installed.
- At least one installed and running standalone DKP Essential or NKP Pro cluster.
- All NKP Pro Clusters are upgraded to the same NKP version as the NKP Managed Ultimate cluster. For more information, see Upgrade NKP on page 1144.
- The cluster you want to convert is self-managed. For more information, see Cluster Types on page 19.
- The NKP Pro Cluster you want to convert only contains its own Cluster API resources and does not contain Cluster API resources from other clusters.

For more information on purchasing an NKP Ultimate license, see Licenses on page 24.

Note: Attaching DKP Enterprise or NKP Ultimate clusters is not supported.

Downtime Considerations

Your NKP Pro cluster will not be accessible externally for several minutes during the expansion process. Any configuration of the cluster's `Ingress` that requires `traefik-forward-auth` authentication will be affected.

Note: Access from within the cluster through Kubernetes service hostname (for example, http://SERVICE_NAME.NAMESPACE:PORT) is unaffected.

Affected NKP Services

- `nkp-ceph-cluster-dashboard`
- `grafana-logging`
- `kube-prometheus-stack-alertmanager`
- `kube-prometheus-stack-grafana`
- `kube-prometheus-stack-prometheus`
- `kubernetes-dashboard`
- `traefik-dashboard`

Other Services

To verify if your services are affected by `traefik-forward-auth` downtime, run the command `kubectl get ingress -n NS <your_customized_ingress_name>`.

Look for the `traefik.ingress.kubernetes.io/router.middlewares` field in the output. The downtime will affect your service if this field contains the value #kommander-forwardauth@kubernetescrd#.

Duration

The `traefik-forward-auth` service is affected starting with the `PreAttachmentCleanup` conversion stage and will run normally again after `ResumeFluxOperations` is completed. Observe the conversion progress to monitor your cluster's current status.

Prerequisites: Cluster Configurations

Before you convert a Pro cluster into a Managed cluster, review the following information.

SSO Configuration

After attachment, the SSO configuration of the Management cluster applies to the Managed (formerly Pro) cluster. Any SSO configuration of the former Pro cluster will be deleted.

- If the Pro cluster has SSO configured but the Management cluster does not, you can copy your Pro cluster's SSO configuration (`dex-controller` resources) to the Management cluster before conversion.
- If your Management cluster has SSO configured and the Pro cluster has another SSO configuration, you can choose to keep one or both. To keep the configuration of your Pro cluster, manually copy the `dex-controller` resources to the Management cluster before conversion. NKP maintains the SSO configuration of your Management cluster automatically unless you manually delete it.

Domains and Certificates

Any custom domain or certificate configuration you have set up for your Pro cluster remains functional after you turn it into a Managed cluster.

Warning:

After conversion, any domain or certificate customizations you want to apply to your Managed cluster must be done through the `KommanderCluster`. This object is now stored in the Management cluster.

Prerequisites: Cloning Git Repository from Git Operator

About this task

After your NKP Pro Cluster is converted into a NKP Ultimate Managed cluster, the old instance of Git Operator that is used to host all Git repositories in the NKP Pro Cluster will not be preserved.

Perform the steps on this page to ensure that you have a local copy of the Management Git Repository in the state it was in prior to undergoing the expansion process.

Note: All NKP Platform Applications will be migrated from the NKP Pro Cluster to the NKP Ultimate Managed Cluster.

Procedure

1. Prior to turning an NKP Pro cluster to an NKP Ultimate Managed Cluster, clone the Management Git Repository using the following command:

   ```
   nkp experimental gitops clone
   ```

2. Verify that the Git Repository has been successfully cloned to your local environment.

   ```
   cd kommander
   git remote -v
   # output from git remote -v look like
   # origin  https://<YOUR_CLUSTER_INGRESS_HOSTNAME>/nkp/kommander/git-operator/
   repositories/kommander/kommander.git (fetch)
   ```

Cluster Applications and Persistent Volumes Backup

Note: Back up and restore your cluster's applications before attempting to convert your NKP Pro cluster into a NKP Ultimate Managed cluster.

The instructions differ depending on the infrastructure provider of your NKP Pro cluster.

For AWS, see AWS Cluster Backup on page 537.

For Azure, vSphere, GCP, and pre-provisioned environments, see Azure, vSphere, GCP, or Pre-provisioned Cluster Backup on page 541.

AWS Cluster Backup

This section contains the instructions necessary to back up and restore the NKP Pro cluster on the AWS environment.

Preparing Your Cluster for Backup

About this task

This section describes how to prepare your cluster on an AWS environment so that it can be backed up.

Before you begin

- Ensure Velero is installed on your Pro cluster. Use at least Velero CLI version 1.10.1.

  For more information, see Velero Installation Using CLI on page 585.
- Ensure kubectl is installed.

  For more information, see https://kubernetes.io/docs/tasks/tools/.
- Ensure you have admin rights to the NKP Pro cluster.

Procedure

Prepare your cluster.

Run the following commands in the NKP Pro cluster. For general guidelines on how to set the context, see Commands within a kubeconfig File on page 32.

Preparing Velero

Enable the CSI snapshotting plug-in by providing a custom configuration of Velero.

Procedure

1. Create an Override with the custom configuration:

```
cat << EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: velero-overrides
  namespace: kommander
data:
  values.yaml: |
    ---
    configuration:
      features: EnableCSI
    initContainers:
      - name: velero-plugin-for-aws
        image: velero/velero-plugin-for-aws:v1.5.2
        imagePullPolicy: IfNotPresent
        volumeMounts:
          - mountPath: /target
            name: plugins
      - name: velero-plugin-for-csi
        image: velero/velero-plugin-for-csi:v0.4.2
        imagePullPolicy: IfNotPresent
        volumeMounts:
          - mountPath: /target
            name: plugins
EOF
```

2. Update the `AppDeployment` to apply the new configuration.

```
cat << EOF | kubectl -n kommander patch appdeployment velero --type='merge' --patch-
file=/dev/stdin
spec:
  configOverrides:
    name: velero-overrides
EOF
```

3. Verify the configuration has been updated before proceeding with the next section.

   ```
   kubectl -n kommander wait --for=condition=Ready kustomization velero
   ```

   The output should look similar to this.

   ```
   kustomization.kustomize.toolkit.fluxcd.io/velero condition met
   ```

Preparing the AWS IAM Permission

About this task

When creating a cluster on AWS, you must provide additional permission as specified in AWS Cluster Identity and Access Management Policies and Roles on page 800.

For the CSI plugin to function correctly, update the existing IAM role to include an additional policy.

Procedure

Add the https://docs.aws.amazon.com/aws-managed-policy/latest/reference/AmazonEBSCSIDriverPolicy.html policy to the control plane role `control-plane.cluster-api-provider-aws.sigs.k8s.io`.

```
aws iam attach-role-policy \
  --role-name control-plane.cluster-api-provider-aws.sigs.k8s.io \
  --policy-arn arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy
```

This will allow the EBS CSI driver, a volume manager, to have enough permissions to create volume snapshots.

> **Warning:** The default control plane role name is `control-plane.cluster-api-provider-aws.sigs.k8s.io`. If you customized this name when creating the AWS cluster, replace the default control-plane role with the name you assigned to it.

Preparing the CSI Configuration

About this task

Procedure

Configure a `VolumeSnapshotClass` object on the cluster so that Velero can create a volume snapshot:

```
cat << EOF | kubectl apply -f -
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: aws
  labels:
    velero.io/csi-volumesnapshot-class: "true"
driver: ebs.csi.aws.com
deletionPolicy: Delete
parameters:
EOF
```

Backing Up the AWS Cluster

About this task

With this workflow, you can back up and restore your cluster's applications. The backup contains all Kubernetes objects and the Persistent Volumes of your NKP applications.

When backing up a cluster that runs on a cloud provider, Velero captures the state of your cluster in a snapshot. Review Velero's list of cloud providers for CSI compatibility. For more information, see https://velero.io/docs/v1.10/supported-providers/.

> **Warning:** Run the following commands in the NKP Pro cluster. For general guidelines on how to set the context, see Commands within a kubeconfig File on page 32.

Procedure

1. Configure Velero to use CSI snapshotting.

```
velero client config set features=EnableCSI
```

2. Create a backup with Velero. Use the following flags to reduce the scope of the backup and only include the applications that are affected during the expansion.

```
velero backup create pre-expansion \
  --include-namespaces="kommander,kommander-default-workspace,kommander-
flux,kubecost" \
  --include-cluster-resources \
  --wait
```

After completion, the output should look similar to this.

```
Backup request "pre-expansion" submitted successfully.
Waiting for backup to complete. You may safely press ctrl-c to stop waiting - your
 backup will continue in the background.
.....................................................................................
Backup completed with status: Completed. You may check for more information using
 the commands `velero backup describe pre-expansion` and `velero backup logs pre-
expansion`.
```

3. Verify the Backup. Review the backup has been completed successfully.

```
velero backup describe pre-expansion
```

The following example output will vary depending on your cloud provider. Verify that it shows no errors and the `Phase` is `Completed`.

```
Name:         pre-expansion
Namespace:    kommander
Labels:       velero.io/storage-location=default
Annotations:  velero.io/source-cluster-k8s-gitversion=v1.25.5
              velero.io/source-cluster-k8s-major-version=1
              velero.io/source-cluster-k8s-minor-version=25

Phase:  Completed

Errors:    0
Warnings:  0

Namespaces:
  Included:  kommander, kommander-default-workspace, kommander-flux, kubecost
  Excluded:  <none>

Resources:
  Included:        *
  Excluded:        <none>
  Cluster-scoped:  included

Label selector:  <none>

Storage Location:  default
```

```
Velero-Native Snapshot PVs:  auto

TTL:  720h0m0s

CSISnapshotTimeout:  10m0s

Hooks:  <none>

Backup Format Version:  1.1.0

Started:    2023-03-15 10:40:25 -0400 EDT
Completed:  2023-03-15 10:44:39 -0400 EDT

Expiration:  2023-04-14 10:40:24 -0400 EDT

Total items to be backed up:  5188
Items backed up:              5188

Velero-Native Snapshots: <none included>
```

Azure, vSphere, GCP, or Pre-provisioned Cluster Backup

This section contains the instructions necessary to back up and restore an NKP Pro cluster on Azure, vSphere, GCP or Pre-provisioned environments.

Preparing Your Cluster for Backup

This section describes how to prepare your cluster on AWS, Azure, vSphere, Google Cloud, or pre-provisioned environment, so it can be backed up.

Before you begin

- Ensure Velero is installed on your Pro cluster. Use at least Velero CLI version 1.10.1.

  For more information, see Velero Installation Using CLI on page 585.
- Ensure kubectl is installed.

  For more information, see https://kubernetes.io/docs/tasks/tools/.
- Ensure you have admin rights to the NKP Pro cluster.

Procedure

Prepare you cluster.

Run the following commands in the NKP Pro cluster. For general guidelines on how to set the context, see Commands within a kubeconfig File on page 32.

Preparing Velero

Enable `restic` backup capabilities by providing a custom configuration of Velero.

Procedure

1. Create an Override with a custom configuration for Velero. This custom configuration deploys the `node-agent` service, which enables `restic`.

   ```
   cat << EOF | kubectl apply -f -
   ```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: velero-overrides
  namespace: kommander
data:
  values.yaml: |
    ---
    deployNodeAgent: true
EOF
```

2. Reference the created Override in Velero's `AppDeployment` to apply the new configuration.

```
cat << EOF | kubectl -n kommander patch appdeployment velero --type='merge' --patch-
file=/dev/stdin
spec:
  configOverrides:
    name: velero-overrides
EOF
```

3. Wait until the `node-agent` has been deployed:

```
until kubectl get daemonset -A | grep -m 1 "node-agent"; do 0.1 ; done
```

The `node-agent` is ready after a similar output appears:

```
kommander                node-agent
3         3        0       3           0           <none>
```

4. Verify the configuration has been updated before proceeding with the next section.

```
kubectl -n kommander wait --for=condition=Ready kustomization velero
```

The output should look similar to this.

```
kustomization.kustomize.toolkit.fluxcd.io/velero condition met
```

Preparing the Pods for Backup

Annotate the pod to ensure `restic` backs up the Persistent Volumes (PVs) of the pods that will be affected during the expansion process. These volumes contain the Git repository information of your NKP Pro cluster.

About this task

> **Warning:** Run the following commands in the NKP Pro cluster. For general guidelines on how to set the context, see Commands within a kubeconfig File on page 32.

Procedure

Run the following command.

```
kubectl -n git-operator-system annotate pod git-operator-git-0 backup.velero.io/backup-
volumes=data
```

Backing Up the Azure, vSphere, GCP, or Pre-provisioned Cluster

With this workflow, you can back up and restore your cluster's applications. This backup contains Kubernetes objects and the Persistent Volumes (PVs) of Git Operator pods. Given that Git Operator's PVs store information on your cluster's state, you will be able to restore your cluster if required.

About this task

> **Warning:** Run the following commands in the NKP Pro cluster. For general guidelines on how to set the context, see Commands within a kubeconfig File on page 32.

Procedure

1. Create a backup with Velero. Use the following flags to reduce the scope of the backup and only include the applications that are affected during the expansion.

```
velero backup create pre-expansion \
  --include-namespaces="git-operator-system,kommander,kommander-default-
workspace,kommander-flux,kubecost" \
  --include-cluster-resources \
  --snapshot-volumes=false --wait \
  --namespace kommander
```

   After completion, the output should look similar to this.

```
Backup request "pre-expansion" submitted successfully.
Waiting for backup to complete. You may safely press ctrl-c to stop waiting - your
 backup will continue in the background.
.....................................................................................
Backup completed with status: Completed. You may check for more information using
 the commands `velero backup describe pre-expansion` and `velero backup logs pre-
expansion`.
```

2. Verify the Backup. Ensure the backup has been completed successfully.

```
velero backup describe pre-expansion --namespace kommander
```

   The following example output will vary depending on your cloud provider. Verify that it shows no errors and the `Phase` is `Completed`.

```
Name:         pre-expansion
Namespace:    kommander
Labels:       velero.io/storage-location=default
Annotations:  velero.io/source-cluster-k8s-gitversion=v1.25.5
              velero.io/source-cluster-k8s-major-version=1
              velero.io/source-cluster-k8s-minor-version=25

Phase:  Completed

Errors:    0
Warnings:  0

Namespaces:
  Included:  git-operator-system, kommander, kommander-default-workspace, kommander-
flux, kubecost
  Excluded:  <none>

Resources:
  Included:        *
  Excluded:        <none>
  Cluster-scoped:  included

Label selector:  <none>

Storage Location:  default

Velero-Native Snapshot PVs:  auto

TTL:  720h0m0s
```

```
CSISnapshotTimeout:  10m0s

Hooks:  <none>

Backup Format Version:  1.1.0

Started:    2023-03-15 10:40:25 -0400 EDT
Completed:  2023-03-15 10:44:39 -0400 EDT

Expiration:  2023-04-14 10:40:24 -0400 EDT

Total items to be backed up:  5188
Items backed up:              5188

Velero-Native Snapshots: <none included>
```

3. Ensure that the `PodVolumeBackup` objects have been created,

```
kubectl get podvolumebackups -A
```

The output should look similar to this.

```
NAMESPACE   NAME        STATUS      CREATED   NAMESPACE   POD                  VOLUME
   REPOSITORY ID
                                   UPLOADER TYPE   STORAGE LOCATION   AGE
kommander   ash-5vsbf   Completed   42s       git-operator-system   git-operator-
git-0             data     s3:https://a54904d80411e4d64b572b96cb3ddb62-477717230.us-
west-2.elb.amazonaws.com:8085/nkp-velero/restic/kommander   restic          default
          42s
```

Converting a Pro Cluster Into a Managed Cluster Using the UI

Instructions on how you can convert a Pro Cluster to a Managed cluster with no Networking Restrictions with the NKP UI.

About this task

> **Warning:** `Ingress` that contains Traefik-Forward-Authentication in NKP (TFA) on page 621 configuration will not be available during the expansion process. Therefore, your NKP Pro cluster will not be accessible externally for several minutes. Access from within the cluster through Kubernetes service hostname (for example, http://SERVICE_NAME.NAMESPACE:PORT) is not affected.
>
> For more information., see Downtime Considerations on page 536.

To attach an existing cluster that has no additional networking restrictions:

Use this option when you want to attach a cluster that does not require additional access information.

Procedure

1. From the top menu bar, select your target workspace.
2. On the Dashboard page, select the **Add Cluster** option in the **Actions** dropdown list on the top right.
3. Select **Attach Cluster**.
4. Select the **No additional networking restrictions** card. Alternatively, if you must use network restrictions, skip the following steps and perform the steps in Cluster Attachment with Networking Restrictions on page 508.

5. In the **Cluster Configuration** section, paste your `kubeconfig` file into the field or select **Upload kubeconfig File** to specify the file.
6. The **Cluster Name** field will automatically populate with the name of the cluster in the `kubeconfig`. You can edit this field using the name you want for your cluster.
7. The **Context** select list is populated from the `kubeconfig`. Select the desired context with admin privileges from the **Context** select list.
8. Add labels to classify your cluster as needed.
9. Select **Create** to attach your cluster.
10. Verify the Conversion.

    Warning: Run the following commands in the Management cluster. For general guidelines on setting the context, see Commands within a kubeconfig File on page 32.

    a. Export the environment variable for the workspace namespace.

    ```
    export WORKSPACE_NAMESPACE=<workspace_namespace>
    ```

    b. To verify that the conversion is successful, check the `KommanderCluster` object.

    ```
    kubectl wait --for=condition=AttachmentCompleted kommandercluster <cluster name>
     -n ${WORKSPACE_NAMESPACE} --timeout 30m
    ```

    The following output appears if the conversion is successful.

    ```
    kommandercluster.kommander.mesosphere.io/<cluster name> condition met
    ```

    Note: After conversion, all Platform Applications will be in the Kommander Namespace in the Managed Cluster.

What to do next

Post Conversion: Cleaning Clusters Running on Different Cloud Platforms on page 548

Converting a Pro Cluster Into a Managed Cluster Using the CLI

Instructions on how you can convert an NKP Pro Cluster to an NKP Ultimate Managed Cluster with no Networking Restrictions through the CLI.

About this task

Warning:

`Ingress` that contains Traefik-Forward-Authentication in NKP (TFA) on page 621 configuration will not be available during the expansion process, therefore, your NKP Pro cluster will not be accessible externally for several minutes. Access from within the cluster through Kubernetes service hostname (for example, http://SERVICE_NAME.NAMESPACE:PORT) is not affected.

For more information, see Downtime Considerations on page 536.

Procedure

1. Run the following command in the NKP Ultimate cluster.

   The cluster name and kubeconfig are from the cluster you are attaching and want to convert to initiate the process of turning an NKP Pro cluster into an NKP Ultimate Managed cluster. The workspace is the workspace name you want the attached cluster to go into.

   ```
   nkp attach cluster --name <pro-cluster-name> --attached-kubeconfig <kubeconfig-file-of-pro-cluster> --workspace <workspace-name>
   ```

2. Verify the conversion.

   Warning: Run the following commands in the Management cluster. For general guidelines on setting the context, see Commands within a kubeconfig File on page 32.

   a. Export the environment variable for the workspace namespace.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

   b. To verify that the conversion is successful, check the `KommanderCluster` object.

   ```
   kubectl wait --for=condition=AttachmentCompleted kommandercluster <cluster name> -n ${WORKSPACE_NAMESPACE} --timeout 30m
   ```

   The following output appears if the conversion is successful.

   ```
   kommandercluster.kommander.mesosphere.io/<cluster name> condition met
   ```

   Note: After conversion, all Platform Applications will be in the Kommander Namespace in the Managed Cluster.

What to do next

Post Conversion: Cleaning Up Cluster Autoscaler Configuration on page 546

Post Conversion: Cleaning Up Cluster Autoscaler Configuration

Follow the steps on this page to ensure that the Cluster Autoscaler is able to function properly after turning your NKP Pro cluster into an NKP Ultimate Managed cluster.

About this task

After converting your cluster from Pro to Ultimate, the Cluster Life cycle Management responsibilities are moved to a single Management cluster.

The Cluster Autoscaler feature also depends on the same Cluster Life Cycle Management components. If you are using the Cluster Autoscaler feature in NKP, you must perform the following steps for this feature to continue to work correctly:

Note: Run the following commands in the Management cluster. For general guidelines on how to set the context, see Commands within a kubeconfig File on page 32.

Procedure

1. Set the following environment variables with your cluster's details.

   ```
   export CLUSTER_NAME=<>
   export WORKSPACE_NAMESPACE=<>
   ```

2. Apply the Cluster Autoscaler `Deployment` and supporting resources.

   ```
   cat <<EOF | kubectl apply -f -
   ```

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: cluster-autoscaler-${CLUSTER_NAME}
  name: cluster-autoscaler-${CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: cluster-autoscaler-${CLUSTER_NAME}
  template:
    metadata:
      labels:
        app: cluster-autoscaler-${CLUSTER_NAME}
    spec:
      containers:
      - args:
        - --cloud-provider=clusterapi
        - --node-group-auto-discovery=clusterapi:clusterName=${CLUSTER_NAME}
        - --kubeconfig=/workload-cluster/kubeconfig
        - --clusterapi-cloud-config-authoritative
        - -v5
        command:
        - /cluster-autoscaler
        image: us.gcr.io/k8s-artifacts-prod/autoscaling/cluster-autoscaler:v1.25.0
        name: cluster-autoscaler
        volumeMounts:
        - mountPath: /workload-cluster
          name: kubeconfig
          readOnly: true
      serviceAccountName: cluster-autoscaler-${CLUSTER_NAME}
      terminationGracePeriodSeconds: 10
      tolerations:
      - effect: NoSchedule
        key: node-role.kubernetes.io/master
      - effect: NoSchedule
        key: node-role.kubernetes.io/control-plane
      volumes:
      - name: kubeconfig
        secret:
          items:
          - key: value
            path: kubeconfig
          secretName: ${CLUSTER_NAME}-kubeconfig
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: cluster-autoscaler-management-${CLUSTER_NAME}
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-autoscaler-management-${CLUSTER_NAME}
subjects:
- kind: ServiceAccount
  name: cluster-autoscaler-${CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
---
apiVersion: v1
```

```
kind: ServiceAccount
metadata:
  name: cluster-autoscaler-${CLUSTER_NAME}
  namespace: ${WORKSPACE_NAMESPACE}
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-autoscaler-management-${CLUSTER_NAME}
rules:
- apiGroups:
  - cluster.x-k8s.io
  resources:
  - machinedeployments
  - machinedeployments/scale
  - machines
  - machinesets
  verbs:
  - get
  - list
  - update
  - watch
EOF
```

3. Verify the output is similar to the following.

```
deployment.apps/cluster-autoscaler-<cluster-name> created
clusterrolebinding.rbac.authorization.k8s.io/cluster-autoscaler-management-<cluster-
name> created
serviceaccount/cluster-autoscaler-<cluster-name> created
clusterrole.rbac.authorization.k8s.io/cluster-autoscaler-management-<cluster-name>
 created
```

4. To check that the status of the deployment has the expected `AVAILABLE` count of `1`, run the following command and verify that the output is similar.

```
$ kubectl get deployment -n $WORKSPACE_NAMESPACE cluster-autoscaler-$CLUSTER_NAME
NAME                                 READY   UP-TO-DATE   AVAILABLE   AGE
cluster-autoscaler-<cluster-name>   1/1     1            1           1m
```

What to do next

Post Conversion: Cleaning Clusters Running on Different Cloud Platforms on page 548.

Post Conversion: Cleaning Clusters Running on Different Cloud Platforms

About this task

Before you begin

Prior to running these commands, you must ensure that the NKP Management Ultimate cluster is configured with the necessary platform specific permissions to manage the incoming CAPI objects that backs the infrastructure resources in the target cloud platform.

For example, for the NKP Ultimate Managed cluster to manage CAPI clusters in AWS, see https://cluster-api-aws.sigs.k8s.io/topics/iam-permissions.html.

NKP supports expanding your platform in the following scenarios:

Prior to running these commands, you must ensure that the NKP Management Ultimate cluster is configured with the necessary platform specific permissions to manage the incoming CAPI objects that

backs the infrastructure resources in the target cloud platform.platform-specific permissions to manage the incoming CAPI objects that back

For example, for the NKP Ultimate Managed cluster to manage CAPI clusters in AWS, refer to.

NKP supports expanding your platform in the following scenarios:

Table 47: Table

NKP Ultimate Management cluster host provider	NKP Ultimate Management cluster IAM permissions	NKP Pro cluster host provider
AWS	https://cluster-api-aws.sigs.k8s.io/topics/iam-permissions.html	AWS, GCP, vSphere, Pre-provisioned
GCP	https://cloud.google.com/iam/docs/overview	AWS, GCP, vSphere, Pre-provisioned
vSphere	https://docs.vmware.com/en/vRealize-Operations/Cloud/com.vmware.vcom.config.doc/GUID-F85638E3-937E-4E31-90D0-9D4A5E479292.html	AWS, GCP, vSphere, Pre-provisioned
Azure	https://learn.microsoft.com/en-us/azure/active-directory/fundamentals/active-directory-ops-guide-iam	Azure
Pre-provisioned	NA	AWS, GCP, vSphere, Pre-provisioned

To move the CAPI resources:

Procedure

1. Following the conversion into an NKP Ultimate managed cluster, run the following command to move the CAPI Objects.

```
nkp move capi-resources --from-kubeconfig <essential_cluster_kubeconfig> --to-kubeconfig <ultimate_cluster_kubeconfig> --to-namespace ${WORKSPACE_NAMESPACE}
```

2. Verify that the output looks similar to the following.

```
# Moving cluster resources
You can now view resources in the moved cluster by using the --kubeconfig flag with
 kubectl. For example: kubectl --kubeconfig=<ultimate_cluster_kubeconfig> get nodes
```

3. After moving the resources, run the following command to remove the CAPI controller manager deployments.

```
nkp delete capi-components --kubeconfig <essential_cluster_kubeconfig>
```

Troubleshooting: Cluster Management

Verify the Conversion Status of your Cluster

When an error or failure occurs when converting a NKP Pro cluster to a NKP Ultimate Managed cluster, NKP automatically keeps retrying the cluster's conversion and attachment process. You do not need to trigger it manually.

If the state does not improve after a while, here are some ways in which you can check or troubleshoot the failed conversion:

> **Note:** Run the following commands in the Management cluster. For general guidelines on how to set the context, see Commands within a kubeconfig File on page 32.

1. Export the environment variable for the workspace namespace.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

2. To verify that the conversion is successful, check the `KommanderCluster` object:

   ```
   kubectl wait --for=condition=AttachmentCompleted kommandercluster <cluster name> -n
    ${WORKSPACE_NAMESPACE} --timeout 30m
   ```

   The following output appears if the conversion is successful.

   ```
   kommandercluster.kommander.mesosphere.io/<cluster name> condition met
   ```

3. If the condition is not met yet, you can observe the conversion process.
4. Export the environment variable for the workspace namespace:

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

5. Print the state of your cluster's `KommanderCluster` object through the CLI and observe the cluster conversion process,

   ```
   kubectl get kommandercluster -n ${WORKSPACE_NAMESPACE} <essential_cluster_name>
    -o go-template='{{range .status.conditions }}type: {{.type}} {{"\n"}}status:
    {{.status}} {{"\n"}}reason: {{.reason}} {{"\n"}}lastTxTime: {{.lastTransitionTime}}
    {{"\n"}}message: {{.message}} {{"\n\n"}}{{end}}'
   ```

6. The output looks similar to this.

   ```
   type: IngressAddressReady
   status: False
   reason: IngressServiceNotFound
   lastTxTime: 2023-02-24T14:58:18Z
   message: Ingress service object was not found in the cluster

   type: IngressCertificateReady
   status: True
   reason: Ready
   lastTxTime: 2023-02-24T14:49:09Z
   message: Certificate is up to date and has not expired

   type: CAPIResourceMoved
   status: True
   reason: Succeeded
   lastTxTime: 2023-02-24T14:50:56Z
   message: Moved CAPI resources from the attached cluster to management cluster

   type: PreAttachmentCleanup
   status: True
   reason: Succeeded
   lastTxTime: 2023-02-24T14:54:47Z
   message: pre-attach cleanup succeeded

   # [...]
   ```

Errors Related to CAPI Resources

Failed Condition Reason: `FailedToIdentifyCAPIResources`

kubeconfigRef points to the wrong secret. Verify the `KommanderCluster` object of both your Pro and Ultimate clusters. The `spec.kubeconfigRef.name` of each object should point to a valid `kubeconfig` secret.

1. Download the referenced `kubeconfig` to your local machine:

   ```
   kubectl get secret -n <WORKSPACE_NAMESPACE> <cluster_name>-kubeconfig -o
    jsonpath='{.data.value}' | base64 --decode > <cluster_name>-kubeconfig
   ```

2. Verify if the `kubeconfig` is valid:

   ```
   kubectl get namespaces -A --kubeconfig <cluster_name>-kubeconfig
   ```

3. Verify the output of the previous command:

   - **No errors in Ouput**: If your output shows no errors, the error message is not related to a `kubeconfig`.
   - **Errors in Output**: If the output shows an error, delete the `KommanderCluster` object through CLI:

     ```
     kubectl delete kommandercluster -n <WORKSPACE_NAMESPACE> <WRONG_KOMMANDER_CLUSTER>
     ```

4. At this particular stage and in the context of converting your cluster, deleting your `KommanderCluster` will not affect your environment. However, DO NOT delete your `KommanderCluster` in other scenarios, as it detaches the referenced cluster from the Management cluster.

   Finally, restart the cluster conversion process with the UI. For more information, see Converting a Pro Cluster Into a Managed Cluster Using the UI on page 544.

The pro cluster has more than one instance of `v1beta1/clusters.cluster.x-k8s.io`

> **Warning:** Nutanix does not support converting Pro clusters that contain the Cluster API resources of more than one cluster.

Ensure your Pro cluster only contains its own CAPI resources and does not contain the CAPI resources of other clusters.

Restoring Backup and Retrying Cluster Expansion

When an error or failure occurs when converting a NKP Pro cluster to a NKP Ultimate Managed cluster, NKP automatically keeps retrying the cluster's conversion and attachment process. You do not need to trigger it manually.

If you must interrupt the expansion process to restore your cluster and retry the expansion procedure, follow these instructions:

Restoring Your Cluster

Prerequisites: You have backed up your cluster. The cluster expansion you attempted was not successful.

> **Warning:** Switch between NKP Ultimate Management and NKP Pro clusters for the following commands. For general guidelines on how to set the context, see Commands within a kubeconfig File on page 32.

1. Delete the `KommanderCluster` object on the **NKP Ultimate Management** cluster:

   ```
   kubectl -n <WORKSPACE_NAMESPACE> delete kommandercluster <KOMMANDER_CLUSTER_NAME> --
   wait=false
   ```

2. Disable the Flux controllers on the **NKP Pro** cluster to interrupt the expansion process:

```
kubectl -n kommander-flux delete deployment -l app.kubernetes.io/instance=kommander-flux
```

3. Delete the `kube-federation-system` namespace on the **NKP Pro** cluster:

```
kubectl get ns kube-federation-system -o json | jq '.spec.finalizers = []' | kubectl replace --raw "/api/v1/namespaces/kube-federation-system/finalize" -f -
```

4. Restore your cluster's configuration on the **NKP Pro**cluster:

```
velero restore create pre-expansion --from-backup pre-expansion --existing-resource-policy update --wait --namespace kommander
```

Moving the Cluster's CAPI Resources

Because the backup created does not include CAPI resources, you will also have to move them back to the NKP Pro cluster.

> **Warning:** Ensure you replace <CAPI_CLUSTER_NAME> with the name of the NKP Pro cluster you were converting in the current workflow. If you accidentally provide the CAPI cluster name of another Managed cluster, the command will move the CAPI resources of the incorrect cluster to the NKP Pro cluster.

1. Retrieve your Managed cluster's kubeconfig and write it to the `<pro.conf>` file:

```
KUBECONFIG=management.conf ./nkp get kubeconfig -n <WORKSPACE_NAMESPACE> -c <CAPI_CLUSTER_NAME> > pro.conf
```

2. Move your CAPI resources back to the NKP Pro cluster:

```
nkp move capi-resources --from-kubeconfig management.conf --to-kubeconfig <pro.conf> -n <WORKSPACE_NAMESPACE> --to-namespace default
```

Verifying the Restore Process and Retrying the Expansion

1. Verify that you successfully restored your cluster:

```
velero restore describe pre-expansion --namespace kommander
```

The output looks similar to this:

```
Name:         pre-expansion
Namespace:    kommander
Labels:       <none>
Annotations:  <none>

Phase:                       Completed
Total items to be restored:  2411
Items restored:              2411

...
```

2. Retry the Expansion Process.

Run the cluster expansion again, as described in Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

Creating Advanced CLI Clusters

Create CLI clusters.

About this task

Warning: This feature is for advanced users and users in unique environments only. We highly recommend using other documented methods to create clusters whenever possible.

Procedure

1. Generate Cluster Objects.

   You must set the target `namespace` with the name of the workspace you are creating the cluster in using the `nkp create cluster ... --namespace <WORKSPACE_NAME> --dry-run --output=yaml` command. In other words, the `--namespace` flag should equal the workspace name.

   Depending on your infrastructure, NKP CLI can generate a set of cluster objects that can be customized for unusual use cases. For an example of how to use the `--output` flags to create a set of cluster objects, see Creating a New AWS Cluster on page 811.

2. In the selected workspace Dashboard, select the **Add Cluster** option in the **Actions** dropdown list on the top-right.

3. In the **Add Cluster** page, select **Upload YAML to Create a Cluster** and provide advanced cluster details.

   - **Workspace**: The workspace where this cluster belongs (if within the Global workspace).
   - **Cluster YAML**: Paste or upload your customized set of cluster objects into this field. Only valid YAML is accepted.
   - **Add Labels**: By default, your cluster has labels that reflect the infrastructure provider provisioning. For example, your AWS cluster might have a label for the datacenter region and `provider: aws`. Cluster labels are matched to the selectors created for Projects on page 439. Changing a cluster label might add or remove the cluster from projects.

4. To begin provisioning the NKP CLI cluster, click **Create**. This step might take a few minutes for the cluster to be ready and fully deploy its components. The cluster automatically tries to join and should resolve after it is fully provisioned.

Custom Domains and Certificates Configuration for All Cluster Types

Configure a custom domain and certificate during the installation of the Kommander component.

You can perform this configuration on either managed or attached clusters. For more information, see Cluster Types on page 19.

There are two configuration methods:

Table 48: Configuration Methods

Configuration Methods	Supported cluster types
While installing the Kommander component	Only Pro or Management clusters
After installing the Kommander component	Go to Configuring the Kommander Installation with a Custom Domain and Certificate on page 1042

NKP supports configuring a custom domain name for accessing the UI and other platform services, as well as setting up manual or automatic certificate renewal or rotation. This section provides instructions and examples on how to configure a customized domain and certificate on your Pro, Managed, or attached clusters.

Reasons For Setting Up a Custom Domain or Certificate

Reasons For Setting Up a Custom Domain or Certificate.

Reasons for Using a Custom DNS Domain

NKP supports the customization of domains to allow you to use your own domain or hostname for your services. For example, you can set up your NKP UI or any of your clusters to be accessible with your custom domain name instead of the domain provided by default.

To set up a custom domain (without a custom certificate), see Configuring a Custom Domain Without a Custom Certificate on page 1048.

Reasons for Using a Custom Certificate

NKP's default CA identity supports the encryption of data exchange and traffic (between your client and your environment's server). To configure an additional security layer that validates your environment's server authenticity, NKP supports configuring a custom certificate issued by a trusted Certificate Authority either directly in a Secret or managed automatically using the ACME protocol (for example, Let's Encrypt).

Changing the default certificate for any of your clusters can be helpful. For example, you can adapt it to classify your NKP UI or any other type of service as trusted (when accessing a service through a browser).

To set up a custom domain and certificate, refer to the following pages respectively:

- Configure a custom domain and certificate as part of the cluster's installation process. This is only possible for your Management/Pro cluster.
- Update your cluster's current domain and certificate configuration as part of your cluster management operations. For information, see Cluster Operations Management on page 353. You can do this for any cluster type in your environment.

KommanderCluster and Certificate Issuer Concepts

This topic provides information about KommanderCluster and Certificate Issuer.

`KommanderCluster` Object

The `KommanderCluster` resource is an object that contains key information for all types of clusters that are part of your environment, such as:

- Cluster access and endpoint information
- Cluster attachment information
- Cluster status and configuration information

Issuer Objects: `Issuer`, `ClusterIssuer` or `certificateSecret`

If you use a certificate issued and managed automatically by `cert-manager`, you need an `Issuer` or `ClusterIssuer`that you reference in your `KommanderCluster` resource. The referenced object must contain information about your certificate provider.

If you want to use a manually-created certificate, you need a `certificateSecret` that you reference in your `KommanderCluster` resource.

Location of the `KommanderCluster` and Issuer Objects: Management, Managed or Attached Cluster

In the Management or Pro cluster, both the `KommanderCluster` and issuer objects are stored on the same cluster. The issuer can be referenced as an `Issuer`, `ClusterIssuer` , or `certificateSecret`.

In the Managed and attached clusters, the `KommanderCluster` object is stored on the Management cluster. The `Issuer`, `ClusterIssuer` , or `certificateSecret` is stored on the Managed or Attached cluster.

HTTP or DNS Solver

When configuring a certificate for your NKP cluster, you can set up an HTTP solver or a DNS solver. The HTTP protocol exposes your cluster to the public Internet, whereas DNS keeps your traffic hidden. If you use HTTP, your cluster must be publicly accessible (through the ingress or load balancer). If you use DNS, this is not a requirement. For HTTP and DNS configuration options, see Advanced Configuration: ClusterIssuer on page 1047.

If you are enabling access for a network-restricted cluster, this configuration is restricted to DNS. For more information, see Proxied Access to Network-Restricted Clusters on page 525.

Configure Custom Domains or Custom Certificates post Kommander Installation

This page contains instructions on how to set up custom certificates for any cluster type after installing NKP.

There are two configuration methods:

Table 49: Configuration Methods

Configuration Methods	Supported cluster types
While installing the Kommander component	Only Pro or Management clusters
While configuring the Kommander Installation with a Custom Domain and Certificate.	Remain in this page

NKP supports configuring a custom domain name for accessing the UI and other platform services, as well as setting up manual or automatic certificate renewal or rotation. This section provides instructions and examples on how to configure a customized domain and certificate on Pro, Management, Managed, or Attached clusters.

NKP supports configuring a custom domain name for accessing the UI and other platform services, as well as setting up manual or automatic certificate renewal or rotation. This section provides instructions and examples on how to configure the NKP installation to add a customized domain and certificate on your Pro cluster or your Management cluster.

Configuration Options

After you have installed the Kommander component of NKP, you can configure a custom domain and certificate by modifying the `KommanderCluster` object of your cluster. You have several options to establish a custom domain and certificate.

> **Note:** If you want the `cert-manager` to automatically handle certificate renewal and rotation, choose an ACME-supported Certificate Authority.

Using an Automatically-generated Certificate with ACME

About this task

Use a certificate that is managed automatically and supported by `cert-manager`:

- Update the `KommanderCluster` by referencing the **name of the created** `Issuer` or `ClusterIssuer` in the `spec.ingress.issuerRef` field. Enter the custom domain name in the `spec.ingress.hostname` field:

Procedure

1. Create an `Issuer` or `ClusterIssuer` with your certificate provider information. Store this object in the cluster where you want to customize the certificate and domain.

   a. If you want to use NKP's default certificate authority, see Configuring a Custom Certificate With Let's Encrypt on page 557.

   b. For an advanced configuration example, select **I want to use an automatically-generated certificate with ACME and require advanced configuration**.

2. Update the `KommanderCluster` by referencing the **name of the created** `Issuer` or `ClusterIssuer` in the `spec.ingress.issuerRef` field.

   Enter the custom domain name in the `spec.ingress.hostname` field.

```
cat <<EOF | kubectl -n <workspace_namespace> --kubeconfig
 <management_cluster_kubeconfig> patch \
kommandercluster <cluster_name>  --type='merge' --patch-file=/dev/stdin
spec:
  ingress:
    hostname: <cluster_hostname>
    issuerRef:
      name: <issuer_name>
      kind: Issuer # or ClusterIssuer depending on the issuer config
EOF
```

   Warning: Certificates issued by another Issuer .--You can also configure a certificate issued by another Certificate Authority. In this case, the CA will determine which information to include in the configuration.

   - For configuration examples, see https://cert-manager.io/docs/configuration/.
   - The `ClusterIssuer`'s name MUST BE `kommander-acme-issuer`.

Using a manually Manually-generated Certificate

About this task

Use a manually created certificate that is **customized for your hostname**.

Procedure

1. Obtain or create a certificate that is customized for your hostname. Store this object in the workspace namespace of the target cluster.

2. Create a secret with the certificate in the cluster's namespace. Give it a name by replacing `<certificate_secret_name>`:

```
kubectl create secret generic -n "${WORKSPACE_NAMESPACE}" <certificate_secret_name> \
  --from-file=ca.crt=$CERT_CA_PATH \
  --from-file=tls.crt=$CERT_PATH \
  --from-file=tls.key=$CERT_KEY_PATH \
  --type=kubernetes.io/tls
```

3. Update the `KommanderCluster` by referencing this secret in the `spec.ingress.certificateSecretRef` field and provide the custom domain name in the `spec.ingress.hostname`.

```
cat <<EOF | kubectl -n <workspace_namespace> --kubeconfig
 <management_cluster_kubeconfig> patch \
kommandercluster <cluster_name>  --type='merge' --patch-file=/dev/stdin
```

```
spec:
  ingress:
    hostname: <cluster_hostname>
    certificateSecretRef:
      name: <certificate_secret_name>
EOF
```

> **Note:** For Kommander to access the secret containing the certificate, it must be located in the workspace namespace of the target cluster.

Configuring a Custom Certificate With Let's Encrypt

About this task

Let's Encrypt is one of the Certificate Authorities (CA) supported by `cert-manager`. To set up a Let's Encrypt certificate, create an `Issuer` or `ClusterIssuer` in the target cluster and then reference it in the `issuerRef` field of the `KommanderCluster` resource.

Procedure

1. Create the Let's Encrypt ACME cert-manager issuer.

```
cat <<EOF | kubectl apply -f -
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: kommander-acme-issuer
spec:
  acme:
    email: <your_email>
    server: https://acme-v02.api.letsencrypt.org/directory
    privateKeySecretRef:
      name: kommander-acme-issuer-account
    solvers:
      - dns01:
          route53:
            region: us-east-1
            role: arn:aws:iam::YYYYYYYYYYY:role/dns-manager
EOF
```

2. Configure the Management cluster to use your `custom-domain.example.com` with a certificate issued by Let's Encrypt by referencing the created `ClusterIssuer`.

```
cat <<EOF | kubectl -n kommander --kubeconfig <management_cluster_kubeconfig> patch \
 kommandercluster host-cluster  --type='merge' --patch-file=/dev/stdin
spec:
  ingress:
    hostname: custom-domain.example.com
    issuerRef:
      name: custom-acme-issuer
      kind: ClusterIssuer
EOF
```

Troubleshooting Domain and Certificate Customization

About this task

If you want to ensure the customization for a domain and certificate is completed, or if you want to obtain more information on the status of the customization, display the status information for the `KommanderCluster`. On the Management cluster:

Procedure

1. Inspect the modified `KommanderCluster` object.

   ```
   kubectl describe kommandercluster -n <workspace_name> <cluster_name>
   ```

2. If the ingress is still being provisioned, the output looks similar to this.

   ```
   [...]
   Conditions:
     Last Transition Time:  2022-06-24T07:48:31Z
     Message:               Ingress service object was not found in the cluster
     Reason:                IngressServiceNotFound
     Status:                False
     Type:                  IngressAddressReady
   [...]
   ```

   If the provisioning has been completed, the output looks similar to this.

   ```
   [...]
   Conditions:
     Last Transition Time:  2022-06-28T13:43:33Z
     Message:               Ingress service address has been provisioned
     Reason:                IngressServiceAddressFound
     Status:                True
     Type:                  IngressAddressReady
     Last Transition Time:  2022-06-28T13:42:24Z
     Message:               Certificate is up to date and has not expired
     Reason:                Ready
     Status:                True
     Type:                  IngressCertificateReady
   [...]
   ```

   The same command also prints the actual customized values for the `KommanderCluster.Status.Ingress`. Here is an example.

   ```
   [...]
      ingress:
       address: 172.20.255.180
       caBundle: LS0tLS1CRUdJTiBD...<output has been shortened>...DQVRFLS0tLS0K
   [...]
   ```

Disconnecting or Deleting Clusters

Disconnect or delete a cluster.

About this task

When you attach a cluster that was not created with Kommander, you can later detach it. This does not alter the cluster's running state but simply removes it from the NKP UI. User workloads, platform services, and other Kubernetes resources are not cleaned up at detach.

> **Warning:** After successfully detaching the cluster, manually disconnect the attached cluster's Flux installation from the management Git repository. Otherwise, changes to apps in the managed cluster's workspace will still be reflected on the cluster you just detached. Ensure your `nkp` configuration references

the target cluster. You can do this by setting the `KUBECONFIG` environment variable to the appropriate `kubeconfig` file location. For more information, see https://kubernetes.io/docs/tasks/access-application-cluster/configure-access-multiple-clusters/. An alternative to initializing the `KUBECONFIG` environment variable is to use the `–kubeconfig=cluster_name.conf` flag. Then, run `kubectl -n kommander-flux patch gitrepo management -p '{"spec":{"suspend":true}}' --type merge` to make the cluster's workloads not managed by Kommander, anymore.

If you created a managed cluster with Kommander, you cannot disconnect it, but you can delete it. This completely removes the cluster and all of its cloud assets.

We recommend deleting a managed cluster through the NKP UI.

Warning: If you delete the Management (Konvoy) cluster, you can not use Kommander to delete any Managed clusters created by Kommander. If you want to delete all clusters, ensure you delete any Managed clusters before finally deleting the Management cluster.

Statuses: For a list of possible states a cluster can have when it is getting disconnected or deleted, see Cluster Statuses on page 560.

Troubleshooting: I cannot detach an attached cluster that is “Pending,” OR the cluster I deleted through the CLI still appears in the UI with an “Error” state.

Sometimes, detaching or deleting a Kubernetes cluster causes that cluster to get stuck in a “Pending” or “Error” state. This can happen because the wrong `kubeconfig` file is used or the cluster is just not reachable. In order to detach the cluster so it does not show in the UI, follow these steps:

Procedure

1. Determine the `KommanderCluster` resource backing the cluster you tried to attach/detach.

   ```
   kubectl -n WORKSPACE_NAMESPACE get kommandercluster
   ```

   Replace `WORKSPACE_NAMESPACE` with the actual current workspace name. You can find this name by going to `https://YOUR_CLUSTER_DOMAIN_OR_IP_ADDRESS/nkp/kommander/dashboard/workspaces` in your browser.

2. Delete the cluster.

   ```
   kubectl -n WORKSPACE_NAMESPACE delete kommandercluster CLUSTER_NAME
   ```

3. If the resource does not go after a short time, remove its finalizers.

   ```
   kubectl -n WORKSPACE_NAMESPACE patch kommandercluster CLUSTER_NAME --type json -p
    '[{"op":"remove", "path":"/metadata/finalizers"}]'
   ```

   This removes the cluster from the NKP UI.

Management Cluster

A guide for the Management Cluster and the Management Cluster Workspace

When you install Kommander, the host cluster is attached to the Management Cluster Workspace, which is called Management Cluster in the Global workspace dashboard, and Kommander Host inside the Management Cluster Workspace. This allows the Management Cluster to be included in Projects on page 439 and enables the management of its Platform Applications on page 400 from the Management Cluster Workspace.

Note: Do not attach a cluster in the "Management Cluster Workspace" workspace. This workspace is reserved for your Kommander Management cluster only.

Editing

As an attached cluster, you can edit the Management Cluster to add or remove Labels and then use these labels to include the Management Cluster in Projects inside of the Management Cluster Workspace.

Disconnecting

The Management Cluster cannot be disconnected from the GUI like other attached clusters. Because of this, the Management Cluster Workspace cannot be deleted from the GUI as it will always have the Management Cluster inside itself.

Cluster Limitations

This page details the limitations of the clusters.

Clusters have the following limitations:

- Unregistering a cluster on Nutanix Kubernetes Platform (NKP) is not supported.
- Reconfiguring the IP address of the PC VM on Nutanix Kubernetes Platform (NKP) is not supported.
- Deploying Objects on the cluster after changing the PC VM IP address on Nutanix Kubernetes Platform (NKP) is not supported.

Cluster Statuses

A cluster card's status line displays both the current status and the version of Kubernetes running in the cluster.

These statuses only appear on Managed clusters.

Table 50: Table

Status	Description
Pending	This is the initial state when a cluster is created or connected.
Pending Setup	The cluster has networking restrictions that require additional setup and is not yet connected or attached.
Loading Data	The cluster has been added to Kommander, and we are fetching details about it. This is the status before `Active`.
Active	The cluster is connected to the API server.
Provisioning*	The cluster is being created on your cloud provider. This process might take some time.
Provisioned*	The cluster's infrastructure has been created and configured.
Joining	The cluster is being joined to the management cluster for the federation.
Joined	The join process is done and waiting for the first data from the cluster to arrive.

Status	Description
Deleting*	The cluster and its resources are being removed from your cloud provider. This process might take some time.
Error	There has been an error connecting to the cluster or retrieving data from the cluster.
Join Failed	This status can appear when kubefed does not have permission to create entities in the target cluster.
Unjoining	Kubefed is cleaning up after itself, removing all installed resources on the target cluster.
Unjoined	The cluster has been disconnected from the management cluster.
Unjoin Failed	The Unjoin from kubefed failed, or there is some other error with deleting or disconnecting.
Unattached*	The cluster was created manually, and the infrastructure was configured. However, the cluster is not attached. To resolve this status, see Attaching an NKP-created Cluster Using the CLI on page 533.

Cluster Resources

The Resource graphs on a cluster card show you a cluster's resource requests, limits, and usage. This allows a quick, visual scan of cluster health. Hover over each resource to get specific details for that specific cluster resource.

Table 51: Table

Resource	Description
CPU Requests	The requested portion of the total allocatable CPU resource for the cluster is measured in number of cores, such as 0.5 cores.
CPU Limits	The portion of the total allocatable CPU resource to which the cluster is limited is measured in number of cores, such as 0.5 cores.
CPU Usage	The amount of the allocatable CPU resource being consumed. It cannot be higher than the configured CPU limit. Measured in number of cores, such as 0.5 cores)
Memory Requests	The requested portion of the cluster's total allocatable memory resource is measured in bytes, such as 64 GiB.
Memory Limits	The portion of the allocatable memory resource to which the cluster is limited is measured in bytes, such as 64 GiB.

Resource	Description
Memory Usage	The amount of the allocatable memory resource being consumed. It cannot be higher than the configured memory limit. It is measured in bytes, such as 64 GiB.
Disk Requests	The requested portion of the allocatable ephemeral storage resource for the cluster is measured in bytes, such as 64 GiB.
Disk Limits	The portion of the allocatable ephemeral storage resource to which the cluster is limited is measured in bytes, such as 64 GiB.

NKP Platform Applications

Platform Applications are applications that NKP provides out-of-the-box with functionality such as observability, cost management, monitoring, logging, making NKP clusters production-ready right from installation.

Platform applications are the applications selected by Nutanix from the open-source community for use by the NKP platform. You can visit a cluster's detail page to see which platform applications are enabled under the "Platform Applications" section.

To ensure that the attached clusters have sufficient resources, see Workspace Platform Application Defaults and Resource Requirements on page 44. For more information on platform applications and how to customize them, see Platform Applications on page 400.

Cluster Applications and Statuses

The management cluster installs applications. You can visit a cluster's detail page to see the application dashboards enabled from the deployed applications under the **Application Dashboards** section.

Under the **Applications** section of the cluster's detail page, you can view the workspace applications enabled for the cluster, grouped by category.

In this section, you can also view the current status of the enabled applications on the cluster on each application card. Hovering on the status displays details about the application's status.

To ensure that the attached clusters have sufficient resources, see Workspace Platform Application Defaults and Resource Requirements on page 44. For more information on applications and how to customize them, see Workplace Catalog Applications on page 420

Cluster applications can have one of the following statuses

Table 52: Table

Status	Description
Enabled	The application is enabled, but the status on the cluster is not available.
Pending	The application is waiting to be deployed.
Deploying	The application is currently being deployed to the cluster.
Deployed	The application has successfully been deployed to the cluster.

Status	Description
Deploy Failed	The application failed to deploy to the cluster.

Custom Cluster Application Dashboard Cards

You can add custom application dashboard cards to the cluster detail page's Applications section by creating a `ConfigMap` on the cluster. The `ConfigMap` must have a `kommander.d2iq.io/application` label applied through the CLI and must contain both `name` and `dashboardLink` data keys to be displayed. Upon creation of the `ConfigMap`, the NKP UI displays a card corresponding to the data provided in the `ConfigMap`. Custom application cards have a Kubernetes icon and can link to a service running in the cluster or use an absolute URL to link to any accessible URL.

ConfigMap Example

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: "my-app"
  namespace: "app-namespace"
  labels:
    "kommander.d2iq.io/application": "my-app"
data:
  name: "My Application"
  dashboardLink: "/path/to/app"
```

Table 53: Table

Key	Description	Required
`metadata.labels."kommander.d2iq.io/application"`	The application name (ID).	X
`data.name`	The display name that describes the application and displays on the custom application card in the UI.	X
`data.dashboardLink`	The link to the application. This can be an absolute link, `https://www.d2iq.com` or a relative link, `/nkp/kommander/dashboard.` If you use a relative link, the link is built using the cluster's path as the base of the URL to the application.	X
`data.docsLink`	Link to documentation about the application. This is displayed on the application card but is omitted if it is not present.	
`data.category`	Category with which to group the custom application. If not provided, the application is grouped under the category, "None."	

Key	Description	Required
`data.version`	A version string for the application. If not provided, “N/A” is displayed on the application card in the UI.	

Use a command similar to this to create a new custom application `ConfigMap`:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: "my-app"
  namespace: "default"
  labels:
    "kommander.d2iq.io/application": "my-app"
data:
  name: "My Application"
  dashboardLink: "/path/to/app"
EOF
```

Kubernetes Cluster Federation (KubeFed)

Kubernetes Cluster Federation (KubeFed) allows you to coordinate the configuration of multiple Kubernetes clusters from a single set of APIs in a hosting cluster. KubeFed aims to provide mechanisms for expressing which clusters should have their configuration managed and what that configuration should be. The mechanisms that KubeFed provides are intentionally low-level and intended to be foundational for more complex multicluster use cases, such as deploying multi-geo applications and disaster recovery.

For more information, see https://github.com/kubernetes-retired/kubefed.

NKP uses KubeFed to manage multiple clusters from the management cluster and also to federate various resources. A `KubefedCluster` object is automatically created for each attached cluster and joined to the management cluster. After they are joined, namespaces can be federated to the clusters - this is how you get workspace and project namespaces created on the attached clusters. From here, other resources can be federated into those namespaces, such as ConfigMaps, RBAC, and so on.

See the following pages for more information:

- https://github.com/kubernetes-sigs/kubefed/blob/master/docs/concepts.md
- https://github.com/kubernetes-sigs/kubefed/blob/master/docs/userguide.md

Stopping an NKP Cluster

Gracefully stop an NKP cluster.

Before you begin

- Stop the applications and pods in the NKP cluster. This helps reduce the possibility of data corruption.
- Take a backup of the Kubernetes cluster. For more information, see Velero Backup.

> **Note:** This procedure is applicable for all the infrastructure providers. However, the example provided in this topic is based on Nutanix deployment.

About this task

To shut down an NKP cluster, follow these steps:

Procedure

1. Skip remediation of the workload cluster on the management cluster:

   ```
   kubectl patch cluster <clustername> -n <namespace> --type merge -p '{"spec": {"paused": true}}'
   ```

   This annotates the workload cluster to stop remediation.

2. In Prism Central UI, shut down the nodes.

   a. Log in to Prism Central.

   b. In the top-left corner, click the menu icon.

   c. Select **Compute** > **VMs** > **Filters**.

   d. Select the **Name** checkbox, change the option to **Starts with**, and type the following:

   `nkp_cluster_name-`

   Where, `nkp_cluster_name-` is the name of the NKP cluster.

   e. Ensure that the filter displays only the desired Kubernetes cluster nodes.

   f. Select the virtual machines with the text **md** in their name.

3. Soft shutdown the worker nodes.

   a. With the worker nodes selected, click **Actions** > **Guest Shutdown**.

   b. Click **OK** to confirm.

4. After the worker nodes are powered off, perform the following for control plane nodes.

   a. Clear the worker nodes.

   b. Select the virtual machines.

   c. With the control plane nodes selected, click **Actions** > **Guest Shutdown**.

   d. Click **OK** to confirm.

5. Soft shutdown the control plane.

   a. With the worker nodes selected, click **Actions** > **Guest Shutdown**.

   b. Click **OK** to confirm.

6. Restart the cluster. For more information, see Starting an NKP Cluster on page 565.

Starting an NKP Cluster

Starting an NKP cluster after shutdown.

About this task

To start an NKP cluster, follow these steps:

> **Note:** This procedure is applicable for all the infrastructure providers.

Procedure

1. Use Prism Central UI to select the control plane nodes.
   a. Log in to Prism Central.
   b. In the top-left corner, click the menu icon.
   c. Select **Compute** > **VMs** > **Filters**.
   d. Select the **Name** checkbox, change the option to **Starts with**, and type the following:

      `nkp_cluster_name-`

      Where, `nkp_cluster_name-` is the name of the NKP cluster.
   e. Ensure that the filter only displays the desired Kubernetes cluster nodes.
   f. Select the virtual machines.
2. Start all control plane nodes.
   a. Perform Step 1 for control plane nodes.
   b. Power on the control plane.
3. Use SSH to log on to one of the control plane nodes to check the control plane status.
   a. To verify that the control plane nodes are up, use the `kubectl` to check for the Ready status.

      $ watch kubectl get nodes
   b. If the control plane nodes are NotReady, then check whether there are connection issues between the control plane nodes and etcd nodes.
4. Start all the worker nodes.
   a. Perform Step 1 for worker nodes.
      - Clear the control plane nodes.
      - Select the virtual machines with the text **worker** in their name.
   b. Power on the workers nodes.
5. Use SSH to log on to one of the control plane nodes to check the cluster status and uncordon the worker nodes.
   a. To verify that all worker nodes are up, use the kubectl to check for the `Ready`, `SchedulingDisabled` status.

      ```
      $ watch kubectl get nodes
      ```
   b. Verify that all pods are up. Use the kubectl to check for the `Running` status.

      ```
      $ watch kubectl get pods --all-namespaces
      ```
6. Re-enable remediation for the cluster on the management cluster:

   ```
   kubectl patch cluster <clustername> -n <namespace> --type merge -p '{"spec":
   {"paused": false}}'
   ```

   This annotates the cluster to re-enable remediation.
7. In the NKP UI, verify that the cluster status is healthy.

Backup and Restore

For production clusters, regular maintenance should include routine backup operations to ensure data integrity and reduce the risk of data loss due to unexpected events. Backup operations should include the cluster state, application state, and the running configuration of both stateless and stateful applications in the cluster.

NKP stores all data as CRDs in the Kubernetes API, and you can back it up and restore it. Choose a procedure depending on your infrastructure provider:

- Velero Configuration on page 567
- Velero Backup on page 585

Velero Configuration

For default installations, NKP deploys Velero integrated with Rook Ceph, operating inside the same cluster.

For more information on Velero, see https://velero.io/. For more information on Rook Ceph, see https://rook.io/.

For production usecases, Nutanix advises providing an external storage class to use with Rook Ceph, see Rook Ceph in NKP on page 665.

Usage of Velero with Nutanix Objects Bucket

Configure Velero to use Nutanix Objects Bucket as storage for backup operations.

Follow these procedures to set up Velero with Nutanix Objects.

Velero with Nutanix Objects: Preparing your Environment

Before you begin

- Ensure you have installed Velero (included in the default NKP installation).
- Ensure you have installed the Velero CLI. For more information, see Velero Installation Using CLI on page 585.
- Ensure you have created an S3 bucket with Nutanix Objects and configured access. For more information, https://portal.nutanix.com/page/documents/details?targetId=Objects-v5_0:top-create-configure-buckets-t.html.

> **Note:** This is an S3-compliant bucket, and many configuration parameters use the S3 style.

Procedure

1. Set the environment variables.

   a. Set the `BUCKET` environment variable to the name of the Nutanix Objects bucket you want to use as backup storage.

   Example:

   ```
   export BUCKET=<nutanix-objects-bucket-name>
   ```

   b. Set the `WORKSPACE_NAMESPACE` environment variable to the name of the workspace's namespace. Replace `<workspace_namespace>` with the name of the target workspace.

   Example:

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

   This can be the kommander namespace for the Management cluster or any other additional workspace namespace for attached or managed clusters. To list all available workspace namespaces, use the `kubectl get kommandercluster -A` command.

   c. To point to the correct kubeconfig, set the `CLUSTER_NAME` environment variable. Replace `<target_cluster>` with the cluster name where you want to set up Velero.

   Example:

   ```
   export CLUSTER_NAME=<target_cluster>
   ```

2. Prepare your Nutanix Objects bucket credentials.

   > **Note:** If you use different credentials to access the BackupStorageLocation (BSL), you can create multiple secrets, enabling the creation of more than one BSL. The secret ties together the BSLs using the `AWS_PROFILE` field.

   For more information on generating Access Keys, see https://portal.nutanix.com/page/documents/details?targetId=Objects-v5_0:top-generate-access-key-t.html

   a. Create a secret on the cluster where you install and configure Velero by referencing the file created in the previous step. This can be the Management, Managed, or an Attached cluster.

   > **Important:** Create this secret on the cluster where you deploy Velero. If you are an NKP Pro license user with multiple workload clusters attached to a Workspace, create this secret in each cluster before updating the AppDeployment using the overrides. For more information, see Velero with Nutanix: Configuring Velero to use Nutanix Objects Bucket as Default BSL on page 569.

   In this example, the secret's name is `velero-nutanix-credentials`, and the profile's name is `ntnx-object-nkp`.

   > **Tip:** Use the same name for `AWS_PROFILE BUCKET` and `BSL_NAME` in the next step.

   Example:

   ```
   export NUTANIX_OBJECTS_SECRET=velero-nutanix-credentials
   export AWS_PROFILE=ntnx-object-nkp
   export NUTANIX_OBJECTS_ACCESS_KEY_ID=<nutanix-objets-access-key>
   export NUTANIX_OBJECTS_SECRET_ACCESS_KEY=<nutanix-objets-secret-Key>

   kubectl --kubeconfig=${CLUSTER_NAME}.conf apply -f - <<EOF
   apiVersion: v1
   kind: Secret
   metadata:
     name: ${NUTANIX_OBJECTS_SECRET}
     namespace: ${WORKSPACE_NAMESPACE}
   ```

```
type: Opaque
stringData:
  aws: |
    [${AWS_PROFILE}]
    aws_access_key_id = ${NUTANIX_OBJECTS_ACCESS_KEY_ID}
    aws_secret_access_key = ${NUTANIX_OBJECTS_SECRET_ACCESS_KEY}
EOF
```

Velero with Nutanix: Configuring Velero to use Nutanix Objects Bucket as Default BSL

Customize Velero to allow a non-default backup location configuration.

About this task

Note: This method will overwrite the default BackupStorageLocation (BSL) to use the specified Nutanix Objects bucket. If that is not the desired outcome, see Velero with Nutanix: Adding additional Backup Storage Location on page 571 or modify the `backupStorageLocation` to include the default existing BSL configured with the same Ceph Object store as the backend.

Procedure

1. Create a `ConfigMap` to enable Velero to use Nutanix Objects Bucket as a backup storage location. This will require additional environment variables with the connection details for the Nutanix Objects Bucket to be used by the Velero BSL. The secret with access keys created in the Velero with Nutanix Objects: Preparing your Environment on page 567 section is referenced for authentication to the bucket.

   In this example, the Bucket name is `ntnx-object-nkp` and the same name is used for the BSL resource.

   Note: The `BackupStorageLocation` field of the configuration is a list, so more than one BSL can be added if required. These can, however, be added later by directly connecting to the cluster, as shown in Velero with Nutanix: Adding additional Backup Storage Location on page 571.

   Backup to Nutanix Objects with Cert:

```
export BUCKET=ntnx-object-nkp
export BSL_NAME=ntnx-object-nkp
export NUTANIX_OBJECTS_HOST=<nutanix-objects-hostname-or-ip>
export NUTANIX_OBJECTS_PORT=443
export NUTANIX_OBJECTS_SECRET=velero-nutanix-credentials
export AWS_PROFILE=ntnx-object-nkp

kubectl --kubeconfig ${CLUSTER_NAME}.conf apply -f - <<EOF
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: ${WORKSPACE_NAMESPACE}
  name: velero-overrides
data:
  values.yaml: |
    credentials:
      extraSecretRef: ""
    configuration:
      backupStorageLocation:
        - name: ${BSL_NAME}
          bucket: ${BUCKET}
          provider: "aws"
          default: true
          config:
            region: us-east-1
```

```
            s3ForcePathStyle: "true"
            insecureSkipTLSVerify: "false"
            s3Url: "https://${NUTANIX_OBJECTS_HOST}:${NUTANIX_OBJECTS_PORT}"
            # profile should be set to the AWS profile name mentioned in the secret
            profile: ${AWS_PROFILE}
          credential:
            key: aws
            name: ${NUTANIX_OBJECTS_SECRET}
            caCert:
 LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSURmekNDQW1lZ0F3SUJBZ0lRRS9OMWcydktjaU5nZVBBd3R5bzNqekF
EOF
```

Backup to Nutanix Objects without Cert:

```
export BUCKET=ntnx-object-nkp
export BSL_NAME=ntnx-object-nkp
export NUTANIX_OBJECTS_HOST=<nutanix-objects-hostname-or-ip>
export NUTANIX_OBJECTS_PORT=443
export NUTANIX_OBJECTS_SECRET=velero-nutanix-credentials
export AWS_PROFILE=ntnx-object-nkp

kubectl --kubeconfig ${CLUSTER_NAME}.conf apply -f - <<EOF
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: ${WORKSPACE_NAMESPACE}
  name: velero-overrides
data:
  values.yaml: |
    credentials:
      extraSecretRef: ""
    configuration:
      backupStorageLocation:
        - name: ${BSL_NAME}
          bucket: ${BUCKET}
          provider: "aws"
          default: true
          config:
            region: us-east-1
            s3ForcePathStyle: "true"
            insecureSkipTLSVerify: "true"
            s3Url: "http://${NUTANIX_OBJECTS_HOST}:${NUTANIX_OBJECTS_PORT}"
            # profile should be set to the AWS profile name mentioned in the secret
            profile: ${AWS_PROFILE}
          credential:
            key: aws
            name: ${NUTANIX_OBJECTS_SECRET}
EOF
```

2. Patch the Velero `AppDeployment` to reference the created `ConfigMap` with the Velero overrides.

   a. To update Velero in all clusters in a workspace:

   ```
   kubectl --kubeconfig=${CLUSTER_NAME}.conf -n ${WORKSPACE_NAMESPACE} patch
    appdeployment velero --type="merge" --patch-file=/dev/stdin <<EOF
   spec:
     configOverrides:
       name: velero-overrides
   EOF
   ```

3. Check the `ConfigMap` on the `HelmRelease` object.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get hr -n kommander velero -o
 jsonpath='{.spec.valuesFrom[?(@.name=="velero-overrides")]}'
```

   The output looks like this if the deployment is successful:

```
{"kind":"ConfigMap","name":"velero-overrides"}
```

4. Verify that the Velero pod is running.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get pods -A --kubeconfig=
${CLUSTER_NAME}.conf |grep velero
```

5. Verify BSL. It should have the phase displayed as `Available`.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get pods -n ${WORKSPACE_NAMESPACE} default
```

Velero with Nutanix: Adding additional Backup Storage Location

Process for creating Multiple BackupStorageLocation (BSL) resources that can point to different Nutanix Objects Buckets.

Procedure

1. If different credentials or key pairs are used to access each bucket, create a secret.

   Note: Multiple secrets can be created to create more than one BackupStorageLocation (BSL) if different credentials are used to access them. The AWS_PROFILE field is the secret, and the BSL ties the two together.

```
export NUTANIX_OBJECTS_SECRET=velero-nutanix-two
export AWS_PROFILE=ntnx-objects-two
export NUTANIX_OBJECTS_ACCESS_KEY_ID=<nutanix-objets-access-key>
export NUTANIX_OBJECTS_SECRET_ACCESS_KEY=<nutanix-objets-secret-Key>
export BUCKET=ntnx-obj-two
export BSL_NAME=ntnx-obj-two
export NUTANIX_OBJECTS_HOST=<nutanix-objects-hostname-or-ip>
export NUTANIX_OBJECTS_PORT=443

kubectl --kubeconfig=${CLUSTER_NAME}.conf apply -f - <<EOF
apiVersion: v1
kind: Secret
metadata:
  name: ${NUTANIX_OBJECTS_SECRET}
  namespace: ${WORKSPACE_NAMESPACE}
type: Opaque
stringData:
  aws: |
    [${AWS_PROFILE}]
    aws_access_key_id = ${NUTANIX_OBJECTS_ACCESS_KEY_ID}
    aws_secret_access_key = ${NUTANIX_OBJECTS_SECRET_ACCESS_KEY}
EOF
```

2. Create a Backup Storage Location.

   a. Create a location for the backup by pointing to an existing Nutanix bucket.

```
velero --kubeconfig ${CLUSTER_NAME}.conf backup-location create
 ${BSL_NAME} -n ${WORKSPACE_NAMESPACE} --provider aws --bucket ${BUCKET}
```

```
 --credential ${NUTANIX_OBJECTS_SECRET}=aws --config region=us-
east-1,insecureSkipTLSVerify="true",s3ForcePathStyle="true",profile=
${AWS_PROFILE},s3Url=https://${NUTANIX_OBJECTS_HOST}:${NUTANIX_OBJECTS_PORT}
```

b. Check that the backup storage location is `Available` and that it references the correct Nutanix bucket.

```
kubectl get backupstoragelocations -n ${WORKSPACE_NAMESPACE} -oyaml
```

> **Note:** If the BackupStorageLocation is not Available, view any error events by using: `kubectl describe backupstoragelocations -n ${WORKSPACE_NAMESPACE}`

3. Create a test backup.

a. Create a test backup stored in the location you created in the previous section.

```
velero backup create nutanix-velero-testbackup -n ${WORKSPACE_NAMESPACE} --
kubeconfig=${CLUSTER_NAME}.conf --storage-location <aws-backup-location-name> --
snapshot-volumes=false
```

b. View your backup.

```
velero backup describe nutanix-velero-testbackup
```

Usage of Velero with AWS S3 Buckets

Configure Velero to use AWS S3 buckets as storage for its backup operations.

Follow these procedures to set up Velero with AWS S3:

Velero with AWS: Preparing your Environment

Before you begin

- Ensure you have installed Velero (included in the default NKP installation).
- Ensure you have installed the Velero CLI. For more information, see Velero Installation Using CLI on page 585.
- Ensure that you have created an S3 bucket with AWS. For more information, https://docs.aws.amazon.com/AmazonS3/latest/userguide/creating-bucket.html.

Procedure

1. Set the environment variables.

   a. Set the `BUCKET` environment variable to the name of the S3 bucket you want to use as backup storage.

   ```
   export BUCKET=<aws-bucket-name>
   ```

   b. Set the `WORKSPACE_NAMESPACE` environment variable to the name of the workspace's namespace. Replace `<workspace_namespace>` with the name of the target workspace.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

   This can be the `kommander` namespace for the Management cluster or any other additional workspace namespace for attached or managed clusters. To list all available workspace namespaces, use the `kubectl get kommandercluster -A` command.

   c. Set the `CLUSTER_NAME` environment variable. Replace `<target_cluster>` with the name of the cluster where you want to set up Velero.

   ```
   export CLUSTER_NAME=<target_cluster>
   ```

2. Prepare your AWS credentials.

   For details on how to use IAM roles instead of static credentials, see https://github.com/vmware-tanzu/velero-plugin-for-aws.

   a. Create a file containing your static AWS credentials.

   In this example, the file's name is `aws-credentials`.

   ```
   cat << EOF > aws-credentials
   [default]
   aws_access_key_id=<REDACTED>
   aws_secret_access_key=<REDACTED>
   EOF
   ```

   b. Create a secret on the cluster where you are installing and configuring Velero by referencing the file created in the previous step. This can be the Management, a Managed, or an Attached cluster.

   In this example, the secret's name is `velero-aws-credentials`.

   ```
   kubectl create secret generic -n ${WORKSPACE_NAMESPACE} velero-aws-credentials --
   from-file=aws=aws-credentials --kubeconfig=${CLUSTER_NAME}.conf
   ```

Velero with AWS: Configuring Velero

Customize Velero to allow the configuration of a non-default backup location.

Procedure

1. Create a `ConfigMap` to enable Velero to use AWS S3 buckets as backup storage location.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: v1
   kind: ConfigMap
   metadata:
     namespace: ${WORKSPACE_NAMESPACE}
     name: velero-overrides
   data:
     values.yaml: |
       credentials:
   ```

```
     extraSecretRef: ""
   configuration:
     backupStorageLocation:
     - bucket: ${BUCKET}
       name: <YOUR-NEW-BACKUP-STORAGE-LOCATION-NAME> # If you don't specify a unique
 name, it will use "default" as the name of the bakcupstoragelocation
       provider: "aws"
       config:
         region: <your-aws-region> # such as us-west-2
         s3ForcePathStyle: "false"
         insecureSkipTLSVerify: "false"
         s3Url: ""
         profile: <your-aws-profile> # profile should be set to the AWS profile name
 mentioned in the secret
       credential:
         name: velero-aws-credentials
         key: aws
EOF
```

2. Patch the Velero `AppDeployment` to reference the created `ConfigMap` with the Velero overrides.

   a. To update Velero in all clusters in a workspace:

   ```
   cat << EOF | kubectl -n ${WORKSPACE_NAMESPACE} patch appdeployment velero --
   type="merge" --patch-file=/dev/stdin
   spec:
     configOverrides:
       name: velero-overrides
   EOF
   ```

   To update Velero for a specific cluster in a workspace and customize an application per cluster, see Customizing an Application Per Cluster on page 416.

3. Check the `ConfigMap` on the `HelmRelease` object.

   ```
   kubectl get hr -n kommander velero -o jsonpath='{.spec.valuesFrom[?(@.name=="velero-
   overrides")]}'
   ```

   The output looks like this if the deployment is successful:

   ```
   {"kind":"ConfigMap","name":"velero-overrides"}
   ```

4. Verify that the Velero pod is running.

   ```
   kubectl get pods -A --kubeconfig=${CLUSTER_NAME}.conf |grep velero
   ```

Velero with AWS: Configuring Velero By Editing the kommander.yaml File

This is an alternative configuration path for management or Pro clusters. You can also configure Velero by editing the `kommander.yaml` and rerunning the installation. To follow this alternative configuration path, expand the following section:

About this task

Configure Velero on the Management Cluster:

Procedure

1. Refresh the `kommander.yaml` to add the customization of Velero.

   Warning: Before running this command, ensure the `kommander.yaml` is the configuration file you are currently using for your environment. Otherwise, your previous configuration will be lost.

   ```
   nkp install kommander -o yaml --init > kommander.yaml
   ```

2. Configure NKP to load the plugins and to include the secret in the `apps.velero` section.

   This process has been tested to work with plugins for AWS v1.1.0 and Azure v1.5.1. More recent versions of these plugins can be used, but have not been tested by Nutanix.

   ```
   ...
    velero:
      values: |
        configuration:
        backupStorageLocation:
          bucket: ${BUCKET}
          config:
            region: <AWS_REGION> # such as us-west-2
            s3ForcePathStyle: "false"
            insecureSkipTLSVerify: "false"
            s3Url: ""
            # profile should be set to the AWS profile name mentioned in the secret
            profile: default
        credentials:
          # With the proper IAM permissions with access to the S3 bucket,
          # you can attach the EC2 instances using the IAM Role, OR fill in
    "existingSecret" OR "secretContents" below.
          #
          # Name of a pre-existing secret (if any) in the Velero namespace
          # that should be used to get IAM account credentials.
          existingSecret: velero-aws-credentials
          # The key must be named "cloud", and the value corresponds to the entire
    content of your IAM credentials file.
          # For more information, consult the documentation for the velero plugin for
    AWS at:
          # [AWS] https://github.com/vmware-tanzu/velero-plugin-for-aws/blob/main/
   README.md
          secretContents:
            # cloud: |
            #   [default]
            #   aws_access_key_id=<REDACTED>
            #   aws_secret_access_key=<REDACTED>
   ...
   ```

3. Use the modified `kommander.yaml` configuration to install this Velero configuration.

   ```
   nkp install kommander --installer-config kommander.yaml --kubeconfig=
   ${CLUSTER_NAME}.conf
   ```

4. Check the `ConfigMap` on the `HelmRelease` object.

   ```
   kubectl get hr -n kommander velero -o jsonpath='{.spec.valuesFrom[?(@.name=="velero-
   overrides")]}'
   ```

   The output looks like this if the deployment is successful:

   ```
   {"kind":"ConfigMap","name":"velero-overrides"}
   ```

5. Verify that the Velero pod is running.

```
kubectl get pods -A --kubeconfig=${CLUSTER_NAME}.conf |grep velero
```

Velero with AWS: Establishing a Backup Location

Procedure

1. Create a Backup Storage Location.
   a. Create a new backup storage location by specifying a unique name during the creation of velero-overrides ConfigMap. For creating velero-overrides ConfigMap, see Velero with AWS: Configuring Velero.
   b. Check that the backup storage location is `Available` and that it references the correct S3 bucket.

```
kubectl get backupstoragelocations -n ${WORKSPACE_NAMESPACE} -oyaml
```

> **Note:** If the BackupStorageLocation is not Available, view any error events by using: `kubectl describe backupstoragelocations -n ${WORKSPACE_NAMESPACE}`

2. Create a test backup.
   a. Create a test backup that is stored in the location you created in the previous section.

```
velero backup create aws-velero-testbackup -n ${WORKSPACE_NAMESPACE} --kubeconfig=
${CLUSTER_NAME}.conf --storage-location <aws-backup-location-name> --snapshot-
volumes=false
```

   b. View your backup.

```
velero backup describe aws-velero-testbackup
```

Usage of Velero with Azure Blob Containers

Configure Velero to use Azure Blob Storage as storage for its backup operations.

Follow these procedures to set up Velero with Azure Blob Storage:

Velero with Azure: Preparing your Environment

Before you begin

- Ensure you have installed Velero (included in the default NKP installation).
- Ensure you have installed the Velero CLI. For more information, see Velero Installation Using CLI on page 585.
- Ensure you have installed the Azure CLI. For more information, see https://learn.microsoft.com/en-us/cli/azure/install-azure-cli?view=azure-cli-latest.
- Ensure you have sufficient access rights to the Azure storage environment and blob container you want to use for backup. For more information on data authorization and Azure blob storage, see https://learn.microsoft.com/en-us/azure/storage/common/authorize-data-access?toc=%2Fazure%2Fstorage%2Fblobs%2Ftoc.json&bc=%2Fazure%2Fstorage%2Fblobs%2Fbreadcrumb%2Ftoc.json&tabs=blobs.

Procedure

1. Prepare your Environment.

   a. Create a container in Azure blob storage.

      For more information, see https://learn.microsoft.com/en-us/azure/storage/blobs/storage-quickstart-blobs-portal#create-a-container.

   b. Set the `BLOB_CONTAINER` environment variable to the name of the blob container you created to use as backup storage.

      ```
      export BLOB_CONTAINER=<Azure-blob-container-name>
      ```

   c. Set up a storage account and resource group.

      For more information, see https://learn.microsoft.com/en-us/azure/storage/common/storage-account-create?tabs=azure-cli#create-a-storage-account-1.

   d. Set the `AZURE_BACKUP_RESOURCE_GROUP` variable to the name of the resource group you created.

      ```
      AZURE_BACKUP_RESOURCE_GROUP=<azure-resource-group-name>
      ```

   e. Set the `AZURE_STORAGE_ACCOUNT_ID` variable to the unique identifier of the storage account you want to use for the backup.

      To obtain the ID, get the resource ID for a storage account. For more information, see https://learn.microsoft.com/en-us/azure/storage/common/storage-account-get-info?toc=%2Fazure%2Fstorage%2Fblobs%2Ftoc.json&bc=%2Fazure%2Fstorage%2Fblobs%2Fbreadcrumb%2Ftoc.json&tabs=azure-cli#get-the-resource-id-for-a-storage-account. The output shows the entire location path of the storage account. You only need the last part, or storage account name, to set the variable.

      ```
      AZURE_STORAGE_ACCOUNT_ID=<storage-account-name>
      ```

   f. Set the `AZURE_BACKUP_SUBSCRIPTION_ID` variable to the unique identifier of the subscription you want to use for the backup.

      To obtain the ID and Azure account list , see https://learn.microsoft.com/en-us/cli/azure/account?view=azure-cli-latest#az-account-list.

      ```
      AZURE_BACKUP_SUBSCRIPTION_ID=<azure-subscription-id>
      ```

   g. Set the `WORKSPACE_NAMESPACE` environment variable to the name of the workspace's namespace.

      Replace `<workspace_namespace>` with the name of the target workspace:

      ```
      export WORKSPACE_NAMESPACE=<workspace_namespace>
      ```

      This can be the `kommander` namespace for the Management cluster or any other additional workspace namespace for Attached or Managed clusters. To list all available workspace namespaces, use the `kubectl get kommandercluster -A` command.

   h. Set the `CLUSTER_NAME` environment variable. Replace `<target_cluster>` with the name of the cluster where you want to set up Velero.

      ```
      export CLUSTER_NAME=<target_cluster>
      ```

2. Prepare your Azure credentials.

   For more details on authorization, choose how to authorize access to blob data in the Azure portalhttps://learn.microsoft.com/en-us/azure/storage/blobs/authorize-data-operations-portal.

   a. Create a `credentials-velero` file with the information required to create a secret. Use the same credentials that you employed when creating the cluster.

      These credentials should not be Base64 encoded because Velero will not read them properly.

      Replace the variables in `<...>` with your environment's information. See your Microsoft Azure account to look up the values.

      ```
      cat << EOF > ./credentials-velero
      AZURE_SUBSCRIPTION_ID=${AZURE_BACKUP_SUBSCRIPTION_ID}
      AZURE_TENANT_ID=<AZURE_TENANT_ID>
      AZURE_CLIENT_ID=<AZURE_CLIENT_ID>
      AZURE_CLIENT_SECRET=<AZURE_CLIENT_SECRET>
      AZURE_BACKUP_RESOURCE_GROUP=${AZURE_BACKUP_RESOURCE_GROUP}
      AZURE_CLOUD_NAME=AzurePublicCloud
      EOF
      ```

   b. Use the `credentials-velero` file to create the secret.

      ```
      kubectl create secret generic -n ${WORKSPACE_NAMESPACE} velero-azure-credentials
       --from-file=azure=credentials-velero --kubeconfig=${CLUSTER_NAME}.conf
      ```

Velero with Azure: Configuring Velero

Customize Velero to allow the configuration of a non-default backup location.

Procedure

1. Create a `ConfigMap` to enable Velero to use Azure blob containers as backup storage location.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: v1
   kind: ConfigMap
   metadata:
     namespace: ${WORKSPACE_NAMESPACE}
     name: velero-overrides
   data:
     values.yaml: |
       initContainers:
         - name: velero-plugin-for-microsoft-azure
           image: velero/velero-plugin-for-microsoft-azure:v1.5.1
           imagePullPolicy: IfNotPresent
           volumeMounts:
             - mountPath: /target
               name: plugins
       credentials:
         extraSecretRef: velero-azure-credentials
   EOF
   ```

2. Patch the Velero `AppDeployment` to reference the created `ConfigMap` with the Velero overrides.

   a. To update Velero in all clusters in a workspace:

      ```
      cat << EOF | kubectl -n ${WORKSPACE_NAMESPACE} patch appdeployment velero --
      type="merge" --patch-file=/dev/stdin
      spec:
        configOverrides:
      ```

```
    name: velero-overrides
EOF
```

To update Velero for a specific cluster in a workspace and customize an application per cluster, see Customizing an Application Per Cluster on page 416.

3. Check the `ConfigMap` on the `HelmRelease` object.

```
kubectl get hr -n kommander velero -o jsonpath='{.spec.valuesFrom[?(@.name=="velero-overrides")]}'
```

The output looks like this if the deployment is successful:

```
{"kind":"ConfigMap","name":"velero-overrides"}
```

4. Verify that the Velero pod is running.

```
kubectl get pods -A --kubeconfig=${CLUSTER_NAME}.conf |grep velero
```

Velero with Azure: Configuring Velero By Editing the kommander.yaml File

This is an alternative configuration path for management or Pro clusters. You can also configure Velero by editing the `kommander.yaml` and rerunning the installation. To follow this alternative configuration path, expand the following section:

About this task

Configure Velero on the Management Cluster:

Procedure

1. Refresh the `kommander.yaml` to add the customization of Velero.

> **Warning:** Before running this command, ensure the `kommander.yaml` is the configuration file you are currently using for your environment. Otherwise, your previous configuration will be lost.

```
nkp install kommander -o yaml --init > kommander.yaml
```

2. Configure NKP to load the plugins and to include the secret in the `apps.velero` section.

This process has been tested to work with plugin Azure v1.5.1. More recent versions of these plugins can be used, but Nutanix has not tested them.

```
...
  velero:
    values: |
      initContainers:
        - name: velero-plugin-for-microsoft-azure
          image: velero/velero-plugin-for-microsoft-azure:v1.5.1
          imagePullPolicy: IfNotPresent
          volumeMounts:
            - mountPath: /target
              name: plugins
      credentials:
        extraSecretRef: velero-azure-credentials
...
```

3. Use the modified `kommander.yaml` configuration to install this Velero configuration.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

4. Check the `ConfigMap` on the `HelmRelease` object.

```
kubectl get hr -n kommander velero -o jsonpath='{.spec.valuesFrom[?(@.name=="velero-overrides")]}'
```

The output looks like this if the deployment is successful:

```
{"kind":"ConfigMap","name":"velero-overrides"}
```

5. Verify that the Velero pod is running.

```
kubectl get pods -A --kubeconfig=${CLUSTER_NAME}.conf |grep velero
```

Velero with Azure: Establishing a Backup Location

Procedure

1. Create a Backup Storage Location.

   a. Create a location for the backup by pointing to an existing Azure bucket.
   Replace `<azure-backup-location-name>` with a name for the backup location.

```
velero backup-location create <azure-backup-location-name> -n
 ${WORKSPACE_NAMESPACE} \
--provider azure \
--bucket ${BLOB_CONTAINER} \
--config resourceGroup=${AZURE_BACKUP_RESOURCE_GROUP},storageAccount=
${AZURE_STORAGE_ACCOUNT_ID},subscriptionId=${AZURE_BACKUP_SUBSCRIPTION_ID} \
--credential=velero-azure-credentials=azure --kubeconfig=${CLUSTER_NAME}.conf
```

   b. Check that the backup storage location is `Available` and that it references the correct Azure bucket.

```
kubectl get backupstoragelocations -n ${WORKSPACE_NAMESPACE} -oyaml
```

2. Create a test backup.

   a. Create a test backup that is stored in the location you created in the previous section.

```
velero backup create azure-velero-testbackup -n ${WORKSPACE_NAMESPACE} \
--kubeconfig=${CLUSTER_NAME}.conf \
--storage-location <azure-backup-location-name> \
--snapshot-volumes=false
```

   b. View your backup.

```
velero backup describe azure-velero-testbackup
```

Note: If your backup wasn't created, Velero might have had an issue installing the plugin.

- 1. If the plugin was not installed, run this command:

```
velero plugin add velero/velero-plugin-for-microsoft-azure:v1.5.1 -n
 ${WORKSPACE_NAMESPACE}
```

  2. Confirm your `backupstoragelocation` was configured correctly.

```
kubectl get backupstoragelocations -n ${WORKSPACE_NAMESPACE}
```

  If your backup storage location is "Available", proceed to create a test backup.

```
NAME             PHASE       LAST VALIDATED   AGE   DEFAULT
```

```
      <azure-backup-location-name>       Available   38s             60m
```

Usage of Velero with Google Cloud Storage Platform
Configure Velero to use Google Cloud Storage Platform as storage for its backup operations.

Follow these procedures to set up Velero with Google Cloud Storage Platform:

Velero with GCP: Preparing your Environment

Before you begin

- Ensure you have installed Velero (included in the default NKP installation).
- Ensure you have installed the Velero CLI. For more information, see Velero Installation Using CLI on page 585.
- You have installed the gcloud CLI. For more information, see https://cloud.google.com/sdk/docs/install.
- (Optional) You can install the gsutil CLI or opt to create buckets through the GCS Console. For more information, see https://cloud.google.com/storage/docs/gsutil_install.
- Ensure you have created a GCS bucket. For more information, see https://cloud.google.com/storage/docs/creating-buckets.
- Ensure you have sufficient access rights to the bucket you want to use for backup. For more information on GCP-related access control, see https://cloud.google.com/storage/docs/access-control.

Procedure

1. Set the environment variables.
   a. Set the `BUCKET` environment variable to the name of the GCS container you want to use as backup storage.
      ```
      export BUCKET=<GCS-bucket-name>
      ```
   b. Set the `WORKSPACE_NAMESPACE` environment variable to the name of the workspace's namespace.
      Replace `<workspace_namespace>` with the name of the target workspace:
      ```
      export WORKSPACE_NAMESPACE=<workspace_namespace>
      ```
      This can be the `kommander` namespace for the Management cluster or any other additional workspace namespace for Attached or Managed clusters. To list all available workspace namespaces, use the`kubectl get kommandercluster -A` command.
   c. Set the `CLUSTER_NAME` environment variable.
      Replace `<target_cluster>` with the name of the cluster where you want to set up Velero:
      ```
      export CLUSTER_NAME=<target_cluster>
      ```

2. Prepare your Google Cloud Platform credentials.

   You can store your backups in Google Cloud Platform/GCS buckets. For more information on setting up access to your bucket, see https://cloud.google.com/storage/docs/creating-buckets#required-roles.

   a. Create a `credentials-velero` file with the information required to create a secret. Use the same credentials that you employed

      Replace `<service-account-email>` with the email address you used to grant permissions to your bucket. The address usually follows the format `<service-account-user>@<gcp-project>.iam.gserviceaccount.com`.

      ```
      gcloud iam service-accounts keys create credentials-velero \
          --iam-account <service-account-email>
      ```

   b. Use the `credentials-velero` file to create the secret.

      ```
      kubectl create secret generic -n ${WORKSPACE_NAMESPACE} velero-gcp-credentials --
      from-file=gcp=credentials-velero --kubeconfig=${CLUSTER_NAME}.conf
      ```

Velero with GCP: Configuring Velero

Customize Velero to allow the configuration of a non-default backup location.

Procedure

1. Create a `ConfigMap` to enable Velero to use GCS buckets as backup storage location.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: v1
   kind: ConfigMap
   metadata:
     namespace: ${WORKSPACE_NAMESPACE}
     name: velero-overrides
   data:
     values.yaml: |
       initContainers:
         - name: velero-plugin-for-gcp
           image: velero/velero-plugin-for-gcp:v1.5.0
           imagePullPolicy: IfNotPresent
           volumeMounts:
             - mountPath: /target
               name: plugins
       credentials:
         extraSecretRef: velero-gcp-credentials
   EOF
   ```

2. Patch the Velero `AppDeployment` to reference the created `ConfigMap` with the Velero overrides.

   a. To update Velero in all clusters in a workspace:

      ```
      cat << EOF | kubectl -n ${WORKSPACE_NAMESPACE} patch appdeployment velero --
      type="merge" --patch-file=/dev/stdin
      spec:
        configOverrides:
          name: velero-overrides
      EOF
      ```

      To update Velero for a specific cluster in a workspace and customize an application per cluster, see Customizing an Application Per Cluster on page 416.

3. Check the `ConfigMap` on the `HelmRelease` object.

```
kubectl get hr -n kommander velero -o jsonpath='{.spec.valuesFrom[?(@.name=="velero-overrides")]}'
```

The output looks like this if the deployment is successful:

```
{"kind":"ConfigMap","name":"velero-overrides"}
```

4. Verify that the Velero pod is running.

```
kubectl get pods -A --kubeconfig=${CLUSTER_NAME}.conf |grep velero
```

Velero with GCP: Configuring Velero By Editing the kommander.yaml File

This is an alternative configuration path for management or Pro clusters. You can also configure Velero by editing the `kommander.yaml` and rerunning the installation.

About this task

Configure Velero on the Management Cluster:

Procedure

1. Refresh the `kommander.yaml` to add the customization of Velero.

> **Warning:** Before running this command, ensure the `kommander.yaml` is the configuration file you are currently using for your environment. Otherwise, your previous configuration will be lost.

```
nkp install kommander -o yaml --init > kommander.yaml
```

2. Configure NKP to load the plugins and to include the secret in the `apps.velero` section.

This process has been tested to work with plugin GCP v1.5.0. More recent versions of these plugins can be used, but Nutanix has not tested them.

```
...
  velero:
    values: |
      initContainers:
        - name: velero-plugin-for-gcp
          image: velero/velero-plugin-for-gcp:v1.5.0
          imagePullPolicy: IfNotPresent
          volumeMounts:
            - mountPath: /target
              name: plugins
      credentials:
        extraSecretRef: velero-gcp-credentials
...
```

3. Use the modified `kommander.yaml` configuration to install this Velero configuration.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

4. Check the `ConfigMap` on the `HelmRelease` object.

```
kubectl get hr -n kommander velero -o jsonpath='{.spec.valuesFrom[?(@.name=="velero-overrides")]}'
```

The output looks like this if the deployment is successful:

```
{"kind":"ConfigMap","name":"velero-overrides"}
```

5. Verify that the Velero pod is running.

```
kubectl get pods -A --kubeconfig=${CLUSTER_NAME}.conf |grep velero
```

Velero with GCP: Establishing a Backup Location

Procedure

1. Create a backup storage location.

   a. Create a location for the backup by pointing to an existing GCS bucket.

   Ensure you set the required environment variables as specified in Velero with GCP: Preparing your Environment on page 581.

   Replace `<gcp-backup-location-name>` with a name for the backup location.

```
velero backup-location create <gcp-backup-location-name> -n ${WORKSPACE_NAMESPACE}
 \
--provider gcp \
--bucket $BUCKET \
--credential=velero-gcp-credentials=gcp
```

   b. Check that the backup storage location is `Available` and that it references the correct GCS bucket.

```
kubectl get backupstoragelocations -n ${WORKSPACE_NAMESPACE} -oyaml
```

2. Create a test backup.

   a. Create a test backup that is stored in the location you created in the previous section.

```
velero backup create gcp-velero-testbackup -n ${WORKSPACE_NAMESPACE} \
--kubeconfig=${CLUSTER_NAME}.conf \
--storage-location <gcp-backup-location-name> \
--snapshot-volumes=false
```

   b. View your backup.

```
velero backup describe gcp-velero-testbackup
```

Note: If your backup wasn't created, Velero might have had an issue installing the plugin.

- 1. If the plugin was not installed, run this command:

```
velero plugin add velero/velero-plugin-for-gcp:v1.5.0 -n
 ${WORKSPACE_NAMESPACE}
```

  2. Confirm your `backupstoragelocation` was configured correctly.

```
kubectl get backupstoragelocations -n ${WORKSPACE_NAMESPACE}
```

  If your backup storage location is "Available", proceed to create a test backup.

```
NAME              PHASE       LAST VALIDATED   AGE   DEFAULT
```

```
      <gcp-backup-location-name>       Available   38s             60m
```

Velero Backup

If you do not want to use Rook Ceph to store Velero backups, you can configure Velero to use the default cloud provider storage.

Velero Installation Using CLI

Although installing the Velero command-line interface is optional and independent of deploying the NKP cluster, having access to it provides several benefits. For example, you can use it up or, restore a cluster on demand or modify certain settings without changing the Velero configuration.

- By default, NKP sets up Velero to use Rook Ceph over TLS using a self-signed certificate.
- As a result, when using certain commands, you might be asked to use the `--insecure-skip-tls-verify` flag.

Again, the default setup is not suitable for production use cases.

Install the Velero command-line interface. For more information, see https://velero.io/docs/v1.5/basic-install/#install-the-cli.

In NKP, the Velero platform application is installed in the `kommander` namespace instead of `velero`. Thus, after installing the CLI, we recommend that you set the Velero CLI `namespace` config option so that subsequent Velero CLI invocations will use the correct namespace:

```
velero client config set namespace=kommander
```

Backup Operations

Velero provides the following basic administrative functions to back up production clusters:

Note:

- If you want to back up your cluster in the scope of Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535, that is, from NKP Pro cluster to an NKP Ultimate Managed cluster, see Cluster Applications and Persistent Volumes Backup on page 537.
- If you require a custom backup location, see how to create one for Velero with AWS: Establishing a Backup Location on page 576, Velero with Azure: Establishing a Backup Location on page 580, and Velero with GCP: Establishing a Backup Location on page 584.

Preparing Your Environment for Backup

About this task

Before you modify a schedule or create an on-demand backup, set the following environment variables:

Procedure

1. Specify the workspace namespace of the cluster for which you want to configure the backup.

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

2. Specify the cluster for which you want to create the backup.

```
export CLUSTER_NAME=<target_cluster_name>
```

Setting a Backup Schedule

About this task

By default, NKP configures a regular, automatic backup of the cluster's state in Velero. The default settings do the following:

Procedure

1. Create daily backups.
2. Save the data from all namespaces.

   Warning: NKP default backups do not support the creation of Volume Snapshots.

   These default settings take effect after the cluster is created. If you install NKP with the default platform services deployed, the initial backup starts after the cluster is successfully provisioned and ready for use.

Creating Backup Schedules

About this task

The Velero CLI provides an easy way to create alternate backup schedules. For example:

Procedure

Run the following command.

```
velero create schedule <backup-schedule-name> -n ${WORKSPACE_NAMESPACE} \
--kubeconfig=${CLUSTER_NAME}.conf \
--snapshot-volumes=false \
--schedule="@every 8h"
```

Changing the Default Backup Service Settings

Procedure

1. Check the backup schedules currently configured for the cluster.

   ```
   velero get schedules
   ```

2. Delete the `velero-default` schedule.

   ```
   velero delete schedule velero-default
   ```

3. Replace the default schedule with your custom settings.

   ```
   velero create schedule velero-default -n ${WORKSPACE_NAMESPACE} \
   --kubeconfig=${CLUSTER_NAME}.conf \
   --snapshot-volumes=false \
   --schedule="@every 24h"
   ```

Creating a Backup Schedule for a Specific Namespace

About this task

You can also create backup schedules for specific namespaces.

Procedure

Creating a backup for a specific namespace can be useful for clusters running multiple apps operated by multiple teams. For example.

```
velero create schedule <backup-schedule-name> \
--include-namespaces=kube-system,kube-public,kommander \
--snapshot-volumes=false \
--schedule="@every 24h"
```

The Velero command line interface provides many more options worth exploring. For more information on disaster recovery, see https://velero.io/docs/v0.11.0/disaster-case/. For more information on cluster migration, see https://velero.io/docs/v0.11.0/migration-case/.

Backing Up on Demand

About this task

In some cases, you might find it necessary to create a backup outside the regularly scheduled interval. For example, if you are preparing to upgrade any components or modify your cluster configuration, perform a backup before taking that action.

Procedure

Create a backup by running the following command.

```
velero backup create <backup-name> -n ${WORKSPACE_NAMESPACE} \
--kubeconfig=${CLUSTER_NAME}.conf \
--snapshot-volumes=false
```

Restoring a Cluster from Backup

About this task

When restoring a backup to the Management Cluster, you must adjust the configuration to avoid restore errors.

Before you begin

Before attempting to restore the cluster state using the Velero command-line interface, verify the following requirements:

- The backend storage, Rook Ceph Cluster, is still operational.
- The Velero platform service in the cluster is still operational.
- The Velero platform service is set to a `restore-only-mode` to avoid having backups run while restoring.

Procedure

1. Prior to restoring a backup, run the following commands.

   a. Ensure the following `ResourceQuota` setup is not configured on your cluster (this `ResourceQuota` will be automatically restored)

   ```
   kubectl -n kommander delete resourcequota one-kommandercluster-per-kommander-workspace
   ```

   b. Turn off the `Workspace` validation webhooks. Otherwise, you will not restore Workspaces with pre-configured namespaces. If the validation webhook named `kommander-validating` is present, it must be modified with this command.

   ```
   kubectl patch validatingwebhookconfigurations kommander-validating \
     --type json \
     --patch '[
       {
         "op": "remove",
         "path": "/webhooks/0/rules/3/operations/0"
       }
     ]'
   ```

2. Restore from backup.

   a. To list the available backup archives for your cluster, run the following command.

   ```
   velero backup get
   ```

   b. Check your deployment to verify that the configuration change was applied correctly.

   ```
   helm get values -n kommander velero
   ```

   c. If restoring your cluster from a backup, use Read Only Backup Storage. To restore cluster data on demand from a selected backup snapshot available in the cluster, run a command similar to the following.

   ```
   velero restore create --from-backup <BACKUP-NAME>
   ```

   If you are restoring using Velero from the default setup (and not using an external bucket or blob to store your backups), you might see an error when describing or viewing the logs of your backup restore. This is a known issue when restoring from an object store that is not accessible from outside your cluster. However, you can review the success of the backup restore by confirming the Phase is `Completed` and not in error, and viewing the logs by running these `kubectl` commands:

   ```
   kubectl logs -l name=velero -n kommander --tail -1
   ```

3. After restoring a backup, perform the following commands.

   a. Verify that the `ResourceQuota` named `one-kommandercluster-per-kommander-workspace` is restored.

   b. Add the removed `CREATE` webhook rule operation with.

   ```
   kubectl patch validatingwebhookconfigurations kommander-validating \
     --type json \
     --patch '[
       {
         "op": "add",
         "path": "/webhooks/0/rules/3/operations/0",
         "value": "CREATE"
       }
   ```

```
  ]'
```

Backup Service Diagnostics

You can check whether the Velero service is currently running on your cluster through the Kubernetes dashboard (accessible through the NKP UI on the Management Cluster), or by running the following `kubectl` command:

```
kubectl get all -A | grep velero
```

If the Velero platform service application is currently running, you can generate diagnostic information about Velero backup and restore operations. For example, you can run the following commands to retrieve, back up, and restore information that you can use to assess the overall health of Velero in your cluster:

```
velero get schedules
velero get backups
velero get restores
velero get backup-locations
velero get snapshot-locations
```

Logging

Nutanix Kubernetes Platform (NKP) comes with a pre-configured logging stack that allows you to collect and visualize pod and admin log data at the Workspace level. The logging stack is also multi-tenant capable, and multi-tenancy is enabled at the Project level through role-based access control (RBAC).

By default, logging is disabled on managed and attached clusters. You need to enable the logging stack applications explicitly on the workspace to make use of these capabilities.

The primary components of the logging stack include these platform services:

- BanzaiCloud Logging-operator
- Grafana and Grafana Loki
- Fluent Bit and Fluentd

In addition to these platform services, logging relies on other software and system facilities, including the container runtime, the journal facility, and system configuration, to collect logs and messages from all the machines in the cluster.

The following diagram illustrates how different components of the logging stack collect log data and provide information about clusters:

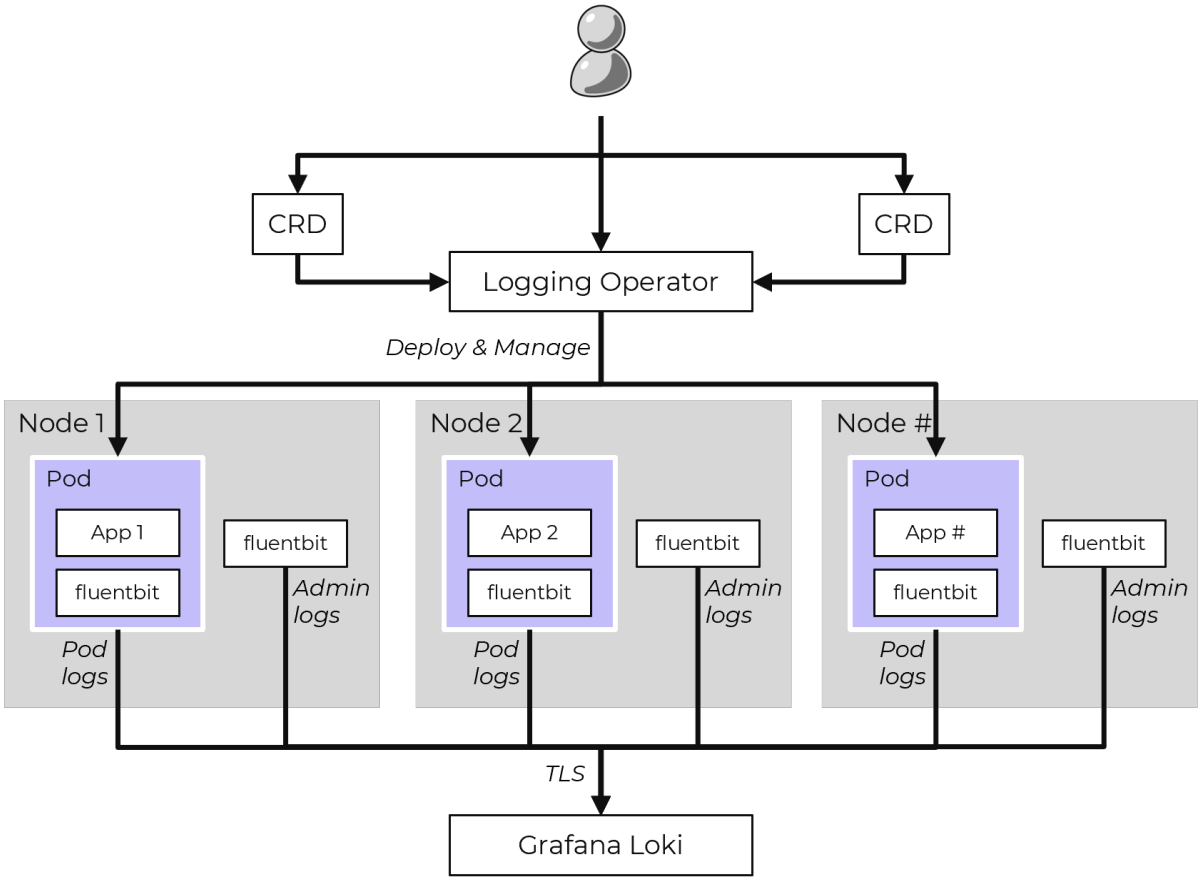


Figure 19: Logging Architecture

The NKP logging stack aggregates logs from applications and nodes running inside your cluster.

NKP uses the BanzaiCloud Logging operator to manage the Fluent Bit and Fluentd deployments that collect pod logs using Kubernetes API extensions called custom resources. The custom resources allow users to declare logging configurations using `kubectl` commands. The Fluent Bit instance deployed by the Logging-operator gathers pod logs data and sends it to Fluentd, which forwards it to the appropriate Grafana Loki servers based on the configuration defined in custom resources.

Loki then indexes the log data by label and stores it for querying. Loki maintains log order integrity but does not index the log messages themselves, which improves its efficiency and lowers its footprint.

Logging Operator

This section contains information about setting up the logging operator to manage the Fluent Bit and Fluentd resources.

Fluent Bit Buffer Information

Fluent Bit collects container logs from the host filesystem and performs the following:

- Maintains a small buffer of logs (5 MB in memory)
- Maintains a checkpoint on each host for each file it's consumed, so if it’s restarted, it can resumed where it left off.
- Flushes the buffer every one second.

- There is a five-second grace period to flush the buffer on exit (along with a 30-second termination grace period on the pod).

Every pod restart should result in the buffer being flushed to `fluentd`(allows up to five seconds for that flush to happen).

If the buffer is not fully flushed during that five seconds, a small amount of log data might be dropped, but if `fluentd`is functional, it is unlikely that fluent-bit will be unable to flush its logs on pod termination.

Fluent Bit can be configured to use a `hostPath` volume to store the buffer information, so it can be picked up again when Fluent Bit restarts.

For more information on Fluent Bit and Fluent Bit log collector, see https://kube-logging.dev/docs/logging-infrastructure/fluentbit/#hostpath-volumes-for-buffers-and-positions.

For more information on Logging in relation to how it is used in NKP, refer to these pages in our Help Center:

- Admin-level Logs on page 593
- Workspace-level Logging on page 594
- Multi-Tenant Logging on page 601
- Fluent Bit on page 607
- Logging Stack on page 591
- Configuring Loki to Use AWS S3 Storage in NKP on page 611
- Customizing Logging Stack Applications on page 612

Logging Stack

Depending on the application workloads you run on your clusters, you might find that the default settings for the NKP logging stack do not meet your needs. In particular, if your workloads produce lots of log traffic, you might find you need to adjust the logging stack components to capture all the log traffic properly. Follow the suggestions below to tune the logging stack components as needed.

Logging Operator

In a high log traffic environment, `fluentd` usually becomes the bottleneck of the logging stack.

According to scaling (see https://kube-logging.dev/docs/operation/scaling/), the typical sign of this is when `fluentd` cannot handle its buffer (see https://kube-logging.dev/docs/configuration/plugins/outputs/buffer/ directory size growth for more than the configured or calculated (timekey + timekey_wait) flush interval. For metrics to monitor by Prometheus, see https://docs.fluentd.org/monitoring-fluentd/monitoring-prometheus#metrics-to-monitor.

Grafana Dashboard

In NKP, if the `Prometheus Monitoring` (kube-prometheus-stack) platform application is enabled, you can view the Logging Operator dashboard in the Grafana UI.

You can also improve Fluentd throughput by turning off the buffering for `loki` clusterOutput.

Example Configuration

You can see an example configuration of the logging operator in Logging Stack Application Sizing Recommendations on page 392.

For more information on performance tuning, Fluentd 1.0, see https://docs.fluentd.org/deployment/performance-tuning-single-process.

Grafana Loki
NKP deploys Loki in Microservice mode. This provides you with the highest flexibility in terms of scaling.

For more information on Microservice mode, see https://grafana.com/docs/loki/latest/get-started/deployment-modes/#microservices-mode.

In a high log traffic environment, Nutanix recommends:

- Ingester should be the first component to be considered for scaling up.
- Distributor should be scaled up only when the existing Distributor is experiencing stress due to high computing resource usage.
    - Usually, the number of Distributor pods should be much lower than the number of Ingester pods.

Grafana Dashboard

In NKP, if `Prometheus Monitoring` (kube-prometheus-stack) platform app is enabled, you can view the Loki dashboards in Grafana UI.

Example Configuration

You can see an example configuration of the logging operator in Logging Stack Application Sizing Recommendations on page 392.

For more information:

- On Loki components, see https://grafana.com/docs/loki/latest/fundamentals/architecture/components/
- On Scale Loki, see https://grafana.com/docs/loki/latest/operations/scalability/
- On Label best practices, see https://grafana.com/docs/loki/latest/best-practices/.

Rook Ceph
Ceph is the default S3 storage provider. In NKP, a Rook Ceph Operator and a Rook Ceph Cluster are deployed together to have a Ceph Cluster.

Grafana Dashboard

In NKP, if the `Prometheus Monitoring` (kube-prometheus-stack) platform app is enabled, you can view the Ceph dashboards in the Grafana UI.

Example Configuration

You can see an example configuration in Rook Ceph Cluster Sizing Recommendations on page 394.

Storage

The default configuration of Rook Ceph Cluster in NKP has a 33% overhead in data storage for redundancy. Meaning, if the data disks allocated for your Rook Ceph Cluster is 1000Gb, 750Gb will be used to store your data. Thus, it is important to account for that in planning the capacity of your data disks to prevent issues.

ObjectBucketClaim Storage Limit

ObjectBucketClaim has a storage limit option to prevent your S3 bucket from growing over a limit. In NKP this is enabled by default.

Thus, after you size up your Rook Ceph Cluster for more storage, it is important to also increase the storage limit of your ObjectBucketClaims of your `grafana-loki` and/or `project-grafana-loki`.

To change it for `grafana-loki` , provide an override configmap in `rook-ceph-cluster` platform app to override `nkp.grafana-loki.maxSize`

To change it for `project-grafana-loki` , provide an override configmap in `project-grafana-loki` platform app to override `nkp.project-grafana-loki.maxSize`

Ceph OSD CPU considerations

ceph-osd is the object storage daemon for the Ceph distributed file system. It is responsible for storing objects on a local file system and providing access to them over the network.

If you determine that the Ceph OSD component is the bottleneck, then you may wish to consider increasing the CPU allocated to it.

For more information on Ceph OSD CPU Scaling, see https://ceph.io/en/news/blog/2022/ceph-osd-cpu-scaling/.

Audit Log

Overhead

Enabling audit logging requires additional computing and storage resources.

When you enable audit logging by enabling the `kommander-fluent-bit` AppDeployment, inbound traffic to the logging stack increases the log traffic by approximately 3-4 more times.

Thus, when enabling the audit log, consider scaling up all components in the logging stack mentioned above.

Fine-tuning audit log Fluent Bit

If you are certain that you only need to collect a subset of the logs that the default config makes the `kommander-fluent-bit` pods collect, you can add your own override configmap to `kommander-fluent-bit` with proper Fluent Bit `INPUT` , `FILTER`, `OUTPUT` settings. This helps reduce the audit log traffic.

To see the default configuration of Fluent Bit, see the *Nutanix Kubernetes Platform Release Notes*.

For more information:

- On admin-level logs, see Admin-level Logs on page 593.
- On configuration files, see https://docs.fluentbit.io/manual/administration/configuring-fluent-bit/classic-mode/configuration-file.

Admin-level Logs

NKP also includes a Fluentbit instance to collect admin-level log information, which is sent to the workspace Grafana Loki, which is running on the cluster. The admin log information includes:

- Logs for host processes managed by systemd
- Kernel logs
- Kubernetes audit logs

This approach helps to isolate the more sensitive logs from the Logging-operator, eliminating the possibility that users might gain inadvertent access to that data.

For more information on these logs, see Fluent Bit on page 607.

> **Warning:** On the Management cluster, the Fluentbit application is disabled by default. The amount of admin logs ingested to Loki requires additional disk space to be configured on the `rook-ceph-cluster`. Enabling admin logs might take around 2GB/day per node. See
>
> For more details on how to configure the Ceph Cluster, see Rook Ceph in NKP on page 665.

Workspace-level Logging

How to enable Workspace-level Logging for use with NKP.

Logging is disabled by default on managed and attached clusters. You will need to enable logging features explicitly at the Workspace level if you want to capture and view log data.

> **Warning:** You must perform these procedures to enable multi-tenant logging at the Project level as well.

Logging Architecture

The NKP logging stack architecture provides a comprehensive logging solution for the NKP platform. It combines Fluent Bit, Fluentd, Loki, and Grafana components to collect, process, store, and visualize log data. The architecture establishes a robust logging solution by assigning specific roles to each of those components.

Components:

- Fluent Bit - Fluent Bit is a lightweight log processor and forwarder that collects log data from various sources, such as application logs or Kubernetes components. It forwards the collected logs to Fluentd for further processing.
- Fluentd - Fluentd is a powerful and flexible log aggregator that receives log data from Fluent Bit, processes it, and forwards it to the Loki Distributor. Fluentd can handle various log formats and enrich the log data with additional metadata before forwarding it.
- Loki - Loki is a horizontally-scalable, highly-available, multi-tenant log aggregation system. Loki components include:- Compactor: Responsible for compacting index files and chunks to improve query performance and reduce storage usage.
    - Distributor: Receives log streams, partitions them into chunks, and forwards these chunks to the Loki Ingester component.
    - Gateway: Acts as a single access point to various Loki components, routing requests between the Distributor, Query Frontend, and other components as needed.
    - Ingester: Compresses, indexes, and persists received log chunks.
    - Querier: Fetches log chunks from the Ingester, decompresses and filters them based on the query, and returns the results to the Query Frontend.
    - Query Frontend: Splits incoming queries into smaller parts, forwards these to the Loki Querier component, and combines the results from all Queriers before returning the final result.
    - Grafana: Grafana is a visualization and analytics platform that supports Loki as one of its data sources. Grafana provides a user-friendly interface for querying and visualizing log data based on user-defined dashboards and panels.
- Grafana - Grafana is a visualization and analytics platform that supports Loki as one of its data sources. Grafana provides a user-friendly interface for querying and visualizing log data based on user-defined dashboards and panels.

Workflow

- Write Path:
  - Fluent Bit instances running on each node collect log data from various sources, like application logs or Kubernetes components.
  - Fluent Bit forwards the collected log data to the Fluentd instance.
  - Fluentd processes the received log data and forwards it to the Loki Distributor through the Loki Gateway.
  - The Loki Distributor receives the log streams, partitions them into chunks, and forwards these chunks to the Loki Ingester component.
  - Loki Ingesters are responsible for compressing, indexing, and persisting the received log chunks.
- Read Path:
  - When a user queries logs through Grafana, the request goes to the Loki Gateway, which routes it to the Loki Query Frontend.
  - The Query Frontend splits the query into smaller parts and forwards these to the Loki Querier component.
  - Loki Queriers fetch the log chunks from the Loki Ingester, decompress and filter them based on the query, and return the results to the Query Frontend.
  - The Query Frontend combines the results from all Queriers and returns the final result to Grafana through the Loki Gateway.
  - Grafana visualizes the log data based on the user's dashboard configuration.

Enabling Logging Applications Using the UI

How to enable the logging stack through the UI for Workspace-level logging

About this task

You can enable the Workspace logging stack to all attached clusters within the Workspace through the UI. If you prefer to enable the logging stack with `kubectl`, review how you Creating AppDeployments to Enable Workspace Logging on page 596.

To enable workspace-level logging in NKP using the UI, follow these steps:

Procedure

1. From the top menu bar, select your target workspace.
2. Select **Applications** from the sidebar menu.
3. Ensure `traefik` and `cert-manager` are enabled on your cluster. These are deployed by default unless you modify your configuration.
4. Scroll to the **Logging** applications section.
5. Select the three-dot button from the bottom-right corner of the cards for Rook Ceph and Rook Ceph Cluster, then click **Enable**. On the **Enable Workspace Platform Application** page, you can add a customized configuration for settings that best fit your organization. You can leave the configuration settings unchanged to enable with default settings.
6. Select **Enable** at the top right of the page.

7. Repeat the process for the Grafana Loki, Logging Operator, and Grafana Logging applications.
8. You can verify the cluster logging stack installation by waiting until the cards have a Deployed checkmark on the Cluster Application page, or you can verify the Cluster Logging Stack installation through the CLI
9. Then, you can view cluster log data.

> **Warning:** We do not recommend installing Fluent Bit, which is responsible for collecting admin logs, unless you have configured the Grafana Loki Ceph Cluster Bucket with sufficient storage space. The amount of admin logs ingested to Loki requires additional disk space to be configured on the `rook-ceph-cluster`. Enabling admin logs might use around 2GB/day per node. For details on how to configure the Ceph Cluster, see Rook Ceph in NKP on page 665.

Creating AppDeployments to Enable Workspace Logging

How to create AppDeployments to enable Workspace-level logging

About this task

Workspace logging AppDeployments enable and deploy the logging stack to all attached clusters within the workspace. Use the NKP UI to enable the logging applications, or, alternately, use the CLI to create the AppDeployments.

To enable logging in NKP using the CLI, follow these steps on the management cluster:

Procedure

1. Execute the following command to get the name and namespace of your workspace.

   ```
   nkp get workspaces
   ```

   And copy the values under the `NAME` and `NAMESPACE` columns for your workspace.

2. Export the `WORKSPACE_NAME` variable.

   ```
   export WORKSPACE_NAME=<WORKSPACE_NAME>
   ```

3. Export the `WORKSPACE_NAMESPACE` variable.

   ```
   export WORKSPACE_NAMESPACE=<WORKSPACE_NAMESPACE>
   ```

4. Ensure that Cert-Manager and Traefik are enabled in the workspace. If you want to find out if the applications are enabled on the management cluster workspace, you can run them.

   ```
   nkp get appdeployments --workspace ${WORKSPACE_NAME}
   ```

5. You can confirm that the applications are deployed on the managed or attached cluster by running this `kubectl` command in that cluster.

   Ensure you switch to the correct context or kubeconfig of the attached cluster for the following `kubectl` command. For more information, see https://kubernetes.io/docs/tasks/access-application-cluster/configure-access-multiple-clusters/).

   ```
   kubectl get helmreleases -n ${WORKSPACE_NAMESPACE}
   ```

6. Copy these commands and run them on the management cluster from a command line to create the Logging-operator, Grafana-loki, and Grafana-logging AppDeployments.

   ```
   nkp create appdeployment logging-operator --app logging-operator-3.17.9 --workspace
    ${WORKSPACE_NAME}
   nkp create appdeployment rook-ceph --app rook-ceph-1.10.3 --workspace
    ${WORKSPACE_NAME}
   ```

```
nkp create appdeployment rook-ceph-cluster --app rook-ceph-cluster-your-ceph-version
 --workspace ${WORKSPACE_NAME}
nkp create appdeployment grafana-loki --app grafana-loki-0.69.16 --workspace
 ${WORKSPACE_NAME}
nkp create appdeployment grafana-logging --app grafana-logging-6.57.4 --workspace
 ${WORKSPACE_NAME}
```

Then, you can verify the cluster logging stack installation. For more information, see Verifying the Cluster Logging Stack Installation on page 599.

To deploy the applications to selected clusters within the workspace, refer to the Cluster-scoped Application Configuration from the NKP UI on page 412.

> **Warning:** We do not recommend installing Fluent Bit, which is responsible for collecting admin logs unless you have configured the Rook Ceph Cluster with sufficient storage space. Enabling admin logs through Fluent Bit might use around 2GB/day per node. For more information on how to configure the Rook Ceph Cluster, see Rook Ceph in NKP on page 665.

7. To install Fluent Bit, create the AppDeployment.

```
nkp create appdeployment fluent-bit --app fluent-bit-0.20.9 --workspace
 ${WORKSPACE_NAME}
```

Overriding ConfigMap to Restrict Logging

About this task

How to override the logging configMap to restrict logging to specific namespaces.

As a cluster administrator, you may need to limit or restrict logging activities to certain namespaces. Kommander allows you to do this by creating an override configMap that modifies the logging configuration created in Creating AppDeployments to Enable Workspace Logging on page 596.

Before you begin

- Implement each of the steps listed in Workspace-level Logging on page 594.
- Ensure that log data is available before you run this procedure.

To create and use the override configMap entries, follow these steps:

Procedure

1. Execute the following command to get the namespace of your workspace.

```
nkp get workspaces
```

And copy the value under the `NAMESPACE` column for your workspace.

2. Set the `WORKSPACE_NAMESPACE` variable to the namespace copied in the previous step.

```
export WORKSPACE_NAMESPACE=<WORKSPACE_NAMESPACE>
```

3. Identify one or more namespaces to which you want to restrict logging.

4. Create a file named `logging-operator-logging-overrides.yaml` and paste the following YAML code into it to create the overrides configMap.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: logging-operator-logging-overrides
  namespace: ${WORKSPACE_NAMESPACE}
```

```
data:
  values.yaml: |
    ---
    clusterFlows:
    - name: cluster-containers
      spec:
        globalOutputRefs:
        - loki
        match:
        - exclude:
            namespaces:
            - <your-namespace>
            - <your-other-namespace>
```

5. Add the relevant namespace values for `metadata.namespace` and the `clusterFlows[0].spec.match[0].exclude.namespaces` values at the end of the file, and save the file.
6. Use the following command to apply the YAML file.

   ```
   kubectl apply -f logging-operator-logging-overrides.yaml
   ```

7. Edit the `logging-operator` AppDeployment to set the value of `spec.configOverrides.name` to `logging-operator-logging-overrides`.

   For more information, see AppDeployment Resources on page 410.

   ```
   nkp edit appdeployment -n ${WORKSPACE_NAMESPACE} logging-operator
   ```

   After your editing is complete, the AppDeployment resembles this example:

   ```
   apiVersion: apps.kommander.d2iq.io/v1alpha3
   kind: AppDeployment
   metadata:
     name: logging-operator
     namespace: ${WORKSPACE_NAMESPACE}
   spec:
     appRef:
       name: logging-operator-3.17.7
       kind: ClusterApp
     configOverrides:
       name: logging-operator-logging-overrides
   ```

8. Perform actions that generate log data, both in the specified namespaces and the namespaces you mean to exclude.
9. Verify that the log data contains only the data you expected to receive.

Overriding ConfigMap to Modify the Storage Retention

NKP configures Grafana Loki to retain log metadata and logs for one week using the Compactor. This retention policy can be customized.

About this task

For more information on Compactor, see https://grafana.com/docs/loki/latest/operations/storage/boltdb-shipper/#compactor.

The minimum retention period is 24 hours.

To customize the retention policy using `configOverrides`, run these commands on the management cluster:

Procedure

1. Execute the following command to get the namespace of your workspace.

   ```
   nkp get workspaces
   ```

   Copy the value under the `NAMESPACE` column for your workspace.

2. Set the `WORKSPACE_NAMESPACE` variable to the namespace copied in the previous step.

   ```
   export WORKSPACE_NAMESPACE=<WORKSPACE_NAMESPACE>
   ```

3. Create a `ConfigMap` with custom configuration values for Grafana Loki.

   Since the retention configuration is nested in a `config` string, you must copy the entire block. The following example sets the retention period to 360 hours (15 days). For more information on this field, see https://grafana.com/docs/loki/latest/operations/storage/retention/#retention-configuration.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: v1
   kind: ConfigMap
   metadata:
     name: grafana-loki-custom-overrides
     namespace: ${WORKSPACE_NAMESPACE}
   data:
     values.yaml: |
       loki:
         structuredConfig:
           limits_config:
             retention_period: 360h
   EOF
   ```

4. Edit the `grafana-loki` AppDeployment to set the value of `spec.configOverrides.name` to `grafana-loki-custom-overrides`

   For more information on deploying a service with a custom configuration, see AppDeployment Resources on page 410.

   ```
   nkp edit appdeployment -n ${WORKSPACE_NAMESPACE} grafana-loki
   ```

   After your editing is complete, the AppDeployment resembles this example.

   ```
   apiVersion: apps.kommander.d2iq.io/v1alpha3
   kind: AppDeployment
   metadata:
     name: grafana-loki
     namespace: ${WORKSPACE_NAMESPACE}
   spec:
     appRef:
       name: grafana-loki-0.69.10
       kind: ClusterApp
     configOverrides:
       name: grafana-loki-custom-overrides
   ```

Verifying the Cluster Logging Stack Installation

How to verify the cluster's logging stack has been installed successfully.

About this task

You must wait for the cluster’s logging stack `HelmReleases` to deploy before attempting to configure or use the logging features.

Run the following commands on the management cluster:

Procedure

1. Execute the following command to get the namespace of your workspace.

   ```
   nkp get workspaces
   ```

   Copy the value under the `NAMESPACE` column for your workspace.

2. Set the `WORKSPACE_NAMESPACE` variable to the namespace copied in the previous step.

   ```
   export WORKSPACE_NAMESPACE=<WORKSPACE_NAMESPACE>
   ```

   Run the following commands on the managed or attached cluster. Ensure you switch to the correct context or `kubeconfig` of the attached cluster for the following `kubectl` commands. For more information, see https://kubernetes.io/docs/tasks/access-application-cluster/configure-access-multiple-clusters/.

3. Check the deployment status using this command on the attached cluster.

   ```
   kubectl get helmreleases -n ${WORKSPACE_NAMESPACE}
   ```

   > **Note:** It may take some time for these changes to take effect, based on the duration configured for the Flux GitRepository reconciliation.

   When the logging stack is successfully deployed, you will see output that includes the following `HelmReleases`:

   ```
   NAME                          READY   STATUS                             AGE
   grafana-logging               True    Release reconciliation succeeded   15m
   logging-operator              True    Release reconciliation succeeded   15m
   logging-operator-logging      True    Release reconciliation succeeded   15m
   grafana-loki                  True    Release reconciliation succeeded   15m
   rook-ceph                     True    Release reconciliation succeeded   15m
   rook-ceph-cluster             True    Release reconciliation succeeded   15m
   object-bucket-claims          True    Release reconciliation succeeded   15m
   ```

What to do next

Viewing Cluster Log Data on page 600

Viewing Cluster Log Data

How to view the cluster's log data after enabling logging.

About this task

Though you enable logging at the Workspace level, you can view the log data at the cluster level using the cluster’s Grafana logging URL.

Run the following commands on the management cluster:

Procedure

1. Execute the following command to get the namespace of your workspace.

   ```
   nkp get workspaces
   ```

   And copy the value under the `NAMESPACE` column for your workspace.

2. Set the `WORKSPACE_NAMESPACE` variable to the namespace copied in the previous step.

```
export WORKSPACE_NAMESPACE=<WORKSPACE_NAMESPACE>
```

Run the following commands on the attached cluster to access the Grafana UI.

Ensure you switch to the correct context or `kubeconfig` of the attached cluster for the following `kubectl` commands. For more information, see https://kubernetes.io/docs/tasks/access-application-cluster/configure-access-multiple-clusters/.

3. Get the Grafana URL.

```
kubectl get ingress -n ${WORKSPACE_NAMESPACE} grafana-logging -o go-
template='https://{{with index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}
{{end}}{{with index .spec.rules 0}}{{with index .http.paths 0}}{{.path }}{{end}}
{{end}}{{"\n"}}'
```

To view logs in Grafana:

- 1. Go to the Explore tab:

```
kubectl get ingress -n ${WORKSPACE_NAMESPACE} grafana-logging -o
 go-template='https://{{with index .status.loadBalancer.ingress 0}}
{{or .hostname .ip}}{{end}}{{with index .spec.rules 0}}{{with index .http.paths
 0}}{{.path }}{{end}}{{end}}/explore{{"\n"}}'
```

2. You might be prompted to log on using the SSO flow. See

for more information.

3. At the top of the page, change the data source to `Loki`.

See the

for more on how to use the interface to view and query logs.

> **Warning:** Cert-Manager and Traefik must be deployed in the attached cluster to be able to access the Grafana UI. These are deployed by default on the workspace.

Multi-Tenant Logging

Multi-tenant logging in Kommander is built on the NKP Logging architecture. Multi-tenant logging uses the same components as the base logging architecture:

- BanzaiCloud logging-operator
- Grafana Loki
- Grafana

> **Note:**
>
> You must perform the Workspace-level Logging on page 594 procedures as a prerequisite to enable multi-tenant logging at the Project level as well.

Access to log data is done at the namespace level through the use of Projects within Kommander, as shown in the diagram:

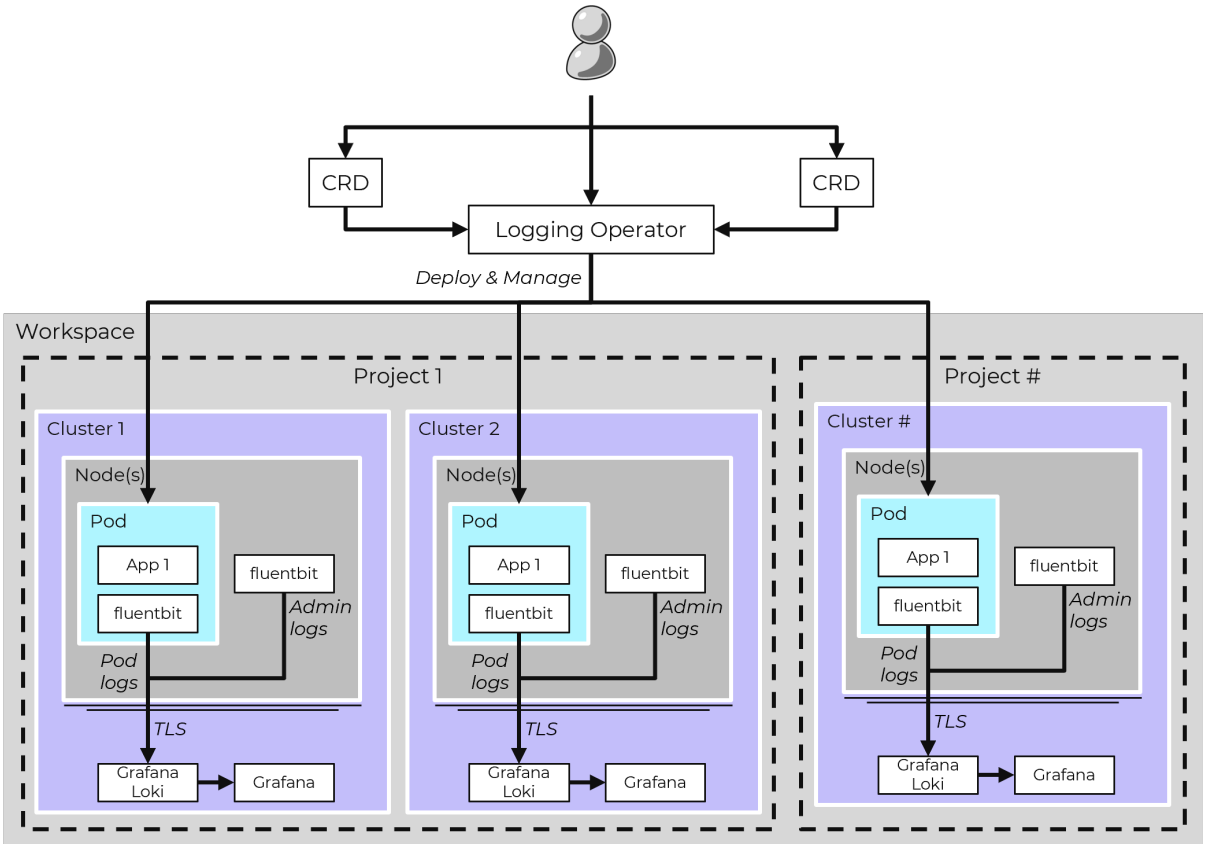


Figure 20: Multi-tenant Logging Architecture

Each Project namespace has a logging-operator, “Flow” that sends its pod logs to its own Loki server. A custom controller deploys corresponding Loki and Grafana servers in each namespace, and defines a logging-operator Flow in each namespace that forwards its pod logs to its respective Loki server. There is a corresponding Grafana server for visualizations for each namespace.

For the convenience of cluster Administrators, a cluster-scoped Loki/Grafana instance pair is deployed with a corresponding Logging-operator `ClusterFlow` that directs pod logs from all namespaces to the pair. A cluster Administrator can grant access either to none of the logs, or to all logs collected from all pods in a given namespace. Assigning teams to specific namespaces enables the team members to see only the logs for the namespaces they own.

As with any endpoint, if an Ingress controller is in use in the environment, take care that the ingress rules do not supersede the RBAC permissions and thus prevent access to the logs.

> **Note:** Cluster Administrators will need to monitor and adjust resource usage to prevent operational difficulties or excessive use on a per namespace basis.

Enabling Multi-tenant Logging

About this task

Context for the current task

Before you begin

- Enable workspace-level logging before you can configure multi-tenant logging. For more information, see Workspace-level Logging on page 594.

- Be a cluster administrator with permissions to configure cluster-level platform services.

Multi-tenant Logging Enablement Process

The steps required to enable multi-tenant logging include:

Procedure

1. Get started with multi-tenant logging by Creating a Project for Logging on page 603.
2. Create the required Project-level AppDeployments.
3. Verifying the Project Logging Stack Installation on page 605
4. Viewing Project Log Data on page 606

Creating a Project for Logging

About this task

To enable multi-tenant logging:

Procedure

1. You must first create a Project and its namespace.

   Users assigned to this namespace will be able to access log data for only that namespace and not others.
2. Then, you can create project-level AppDeployments for use in multi-tenant logging.

Overriding ConfigMap to Modify the Storage Retention

NKP configures Grafana Loki to retain log metadata and logs for one week using the Compactor. This retention policy can be customized.

About this task

For more information on Compactor, see https://grafana.com/docs/loki/latest/operations/storage/boltdb-shipper/#compactor.

The minimum retention period is 24 hours.

To customize the retention policy using `configOverrides`, run these commands on the management cluster:

Procedure

1. Determine the namespace of the workspace that your project is in. You can use the `nkp get workspaces` command to see the list of workspace names and their corresponding namespaces.

   ```
   nkp get workspaces
   ```

   Copy the value under the `NAMESPACE` column for your workspace. This might NOT be identical to the Display Name of the `Workspace`.
2. Set the `WORKSPACE_NAMESPACE` variable to the namespace copied in the previous step.

   ```
   export WORKSPACE_NAMESPACE=<WORKSPACE_NAMESPACE>
   ```

3. Get the namespace of your project.

```
kubectl get projects --namespace ${WORKSPACE_NAMESPACE}
```

Copy the value under the `PROJECT NAMESPACE` column for your workspace. This may NOT be identical to the Display Name of the `Project`.

4. Set the `PROJECT_NAMESPACE` variable to the namespace copied in the previous step.

```
export PROJECT_NAMESPACE=<PROJECT_NAMESPACE>
```

5. Create a `ConfigMap` with custom configuration values for Grafana Loki.

Since the retention configuration is nested in a `config` string, you must copy the entire block. The following example sets the retention period to 360 hours (15 days). For more information on Grafana Loki's Retention Configuration, see https://grafana.com/docs/loki/latest/operations/storage/retention/#retention-configuration.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: project-grafana-loki-custom-overrides
  namespace: ${PROJECT_NAMESPACE}
data:
  values.yaml: |
    loki:
      structuredConfig:
        limits_config:
          retention_period: 360h
EOF
```

6. Run the following command on the management cluster to reference the `configOverrides` in `project-grafana-loki` AppDeployment.

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: project-grafana-loki
  namespace: ${PROJECT_NAMESPACE}
spec:
  appRef:
    name: project-grafana-loki-0.48.6
    kind: ClusterApp
  configOverrides:
    name: project-grafana-loki-custom-overrides
EOF
```

After your editing is complete, the AppDeployment resembles this example.

```
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: grafana-loki
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    name: grafana-loki-0.69.10
    kind: ClusterApp
  configOverrides:
    name: grafana-loki-custom-overrides
```

Verifying the Project Logging Stack Installation

How to verify the project logging stack installation for multi-tenant logging.

About this task

You must wait for the project's logging stack `HelmReleases` , to deploy before attempting to configure or use the project-level logging features, including multi-tenancy.

Run the following commands on the management cluster:

Procedure

1. Determine the namespace of the workspace that your project is in. You can use the `nkp get workspaces` command to see the list of workspace names and their corresponding namespaces.

   ```
   nkp get workspaces
   ```

   Copy the value under the `NAMESPACE` column for your workspace.

2. Export the `WORKSPACE_NAMESPACE` variable.

   ```
   export WORKSPACE_NAMESPACE=<WORKSPACE_NAMESPACE>
   ```

3. Execute the following command to get the namespace of your project.

   ```
   kubectl get projects -n ${WORKSPACE_NAMESPACE}
   ```

   And copy the value under the `PROJECT NAMESPACE` column for your project. This might not be identical to the Display Name of the `Project`.

   .

4. Export the `PROJECT_NAMESPACE` variable.

   ```
   export PROJECT_NAMESPACE=<PROJECT_NAMESPACE>
   ```

   Run the following commands on the managed or attached cluster. Ensure you switch to the correct context or `kubeconfig` of the attached cluster for the following `kubectl` commands. For more information, see https://kubernetes.io/docs/tasks/access-application-cluster/configure-access-multiple-clusters/.

5. Check the deployment status using this command on the attached cluster.

   ```
   kubectl get helmreleases -n ${PROJECT_NAMESPACE}
   ```

   > **Note:** It may take some time for these changes to take effect, based on the duration configured for the Flux GitRepository reconciliation.

   When the logging stack is successfully deployed, you will see output that includes the following `HelmReleases`:

   ```
   NAMESPACE              NAME                                  READY   STATUS
                  AGE
   ${PROJECT_NAMESPACE}   project-grafana-logging               True    Release
    reconciliation succeeded   15m
   ${PROJECT_NAMESPACE}   project-grafana-loki                  True    Release
    reconciliation succeeded   11m
   ${PROJECT_NAMESPACE}   project-loki-object-bucket-claims     True    Release
    reconciliation succeeded   11m
   ```

What to do next

Viewing Project Log Data on page 606

Viewing Project Log Data

How to view the project log data within multi-tenant logging.

About this task

You can only view the log data for a Project to which you have been granted access.

To access Project Grafana's UI:

Run the following commands on the management cluster:

Procedure

1. Determine the namespace of the workspace that your project is in. You can use the `nkp get workspaces` command to see the list of workspace names and their corresponding namespaces.

   ```
   nkp get workspaces
   ```

   Copy the value under the `NAMESPACE` column for your workspace.

2. Export the `WORKSPACE_NAMESPACE` variable.

   ```
   export WORKSPACE_NAMESPACE=<WORKSPACE_NAMESPACE>
   ```

3. Execute the following command to get the namespace of your project.

   ```
   kubectl get projects -n ${WORKSPACE_NAMESPACE}
   ```

   And copy the value under the `PROJECT NAMESPACE` column for your project. This might NOT be identical to the Display Name of the `Project`.

4. Export the `PROJECT_NAMESPACE` variable.

   ```
   export PROJECT_NAMESPACE=<PROJECT_NAMESPACE>
   ```

   Run the following commands on the attached cluster to access the Grafana UI.

   Ensure you switch to the correct context or `kubeconfig` of the attached cluster for the following `kubectl` commands. For more information, see https://kubernetes.io/docs/tasks/access-application-cluster/configure-access-multiple-clusters/.

5. Get the Grafana URL.

   ```
   kubectl get ingress -n ${PROJECT_NAMESPACE} ${PROJECT_NAMESPACE}-project-grafana-
   logging -o go-template='https://{{with index .status.loadBalancer.ingress 0}}
   {{or .hostname .ip}}{{end}}{{with index .spec.rules 0}}{{with index .http.paths 0}}
   {{.path }}{{end}}{{end}}{{"\n"}}'
   ```

   To view logs in Grafana:

   - 1. Go to the Explore tab:

     ```
     kubectl get ingress -n ${PROJECT_NAMESPACE} ${PROJECT_NAMESPACE}-
     project-grafana-logging -o go-template='https://{{with
      index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}{{with
      index .spec.rules 0}}{{with index .http.paths 0}}{{.path }}{{end}}{{end}}/
     explore{{"\n"}}'
     ```

     2. You might be prompted to log on using the SSO flow.

        For more information, see Authentication on page 616 and Authorization.

3. At the top of the page, change the data source to `Loki`.

For more on how to use the interface to view and query logs, see https://grafana.com/docs/grafana/v7.5/datasources/loki/.

> **Warning:** Cert-Manager and Traefik must be deployed in the attached cluster to be able to access the Grafana UI. These are deployed by default on the workspace.

You can configure the workspace policy to restrict access to the Project logging Grafana UI. For more information, see Logging on page 589.

Each Grafana instance in a Project has a unique URL at the cluster level. Consider creating a `WorkspaceRoleBinding` that maps to a `ClusterRoleBinding`, on attached cluster(s), for each Project level Grafana instance. For example, If you have a group named `sample-group` and two projects named `first-project` and `second-project` in `sample-workspace` workspace, then the Role Bindings will look similar to the following image:

Select the correct role bindings for each group for a project at the workspace level.

Fluent Bit

Fluent Bit is the NKP choice of open-source log collection and forwarding tool.

For more information, see https://fluentbit.io/.

> **Warning:** The Fluentbit application is disabled by default on the management cluster. To ingest Loki the required amount of admin logs, additional disk space mustquires additional disk space to be configured on the `rook-ceph-cluster`. Enabling admin logs might use around 2 GB/day per node.

For more details on how to configure the Rook Ceph Cluster, see Rook Ceph on page 592.

Audit Log Collection

Auditing in Kubernetes provides a way to document the actions taken on a cluster chronologically. For more information, see https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/.

On Kommander, by default, audit logs are collected and stored for quick indexing. Viewing and accessing can be done through the Grafana logging UI.

To adjust the default Audit Policy log backend configuration (see https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/#log-backend), you must modify the log retention settings by configuring the control plane (see Configure the Control Plane on page 1074) before creating the cluster. This needs to be done prior to creating the cluster since it cannot be edited after creation (see https://d2iq.atlassian.net/wiki/spaces/DENT/pages/120980357/Collecting+systemd+Logs+from+a+Non-default+Path).

Collecting systemd Logs from a Non-default Path

About this task

By default, Fluent Bit pods are configured to collect `systemd` logs from the `/var/log/journal/` path on cluster nodes.

If `systemd-journald` running as a part of the OS on the nodes uses a different path for writing logs, you will need to override the configuration of the `fluent-bit` AppDeployment to make Fluent Bit collect `systemd` logs.

To configure the Fluent Bit AppDeployment to collect `systemd` logs from a non-default path, follow these steps (all `kubectl` and `nkp` invocations refer to the management cluster):

Procedure

1. To get the namespace of the workspace where you want to configure Fluent Bit, run the following command.

   ```
   nkp get workspaces
   ```

   Copy the value under the `NAMESPACE` column for your workspace.

2. Set the `WORKSPACE_NAMESPACE` variable to the namespace copied in the previous step.

   ```
   export WORKSPACE_NAMESPACE=<WORKSPACE_NAMESPACE>
   ```

3. Identify the `systemd-journald` log data storage path on the nodes of the clusters in the workspace by using the OS documentation and examining the `systemd` configuration.

   Usually, it will be either `/var/log/journal` (typically used when `systemd-journald` is configured to store logs permanently; in this case, the default Fluent Bit configuration should work) or `/run/log/journal` (typically used when `systemd-journald` is configured to use volatile storage).

4. Extract the default Helm values used by the Fluent Bit App.

   ```
   kubectl get -n ${WORKSPACE_NAMESPACE} configmaps fluent-bit-0.20.9-d2iq-defaults -
   o=jsonpath='{.data.values\.yaml}' > fluent-bit-values.yaml
   ```

5. Edit the resulting file `fluent-bit-values.yaml` by removing all sections except for `extraVolumes`, `extraVolumeMounts` and `config.inputs`. The result should look similar to this.

   ```
   extraVolumes:
   # we create this to have a persistent tail-db directory an all nodes
   # otherwise a restarted fluent-bit would rescrape all tails
   - name: tail-db
     hostPath:
       path: /var/log/tail-db
       type: DirectoryOrCreate
   # we create this to get rid of error messages that would appear on non control-
   plane nodes
   - name: kubernetes-audit
     hostPath:
       path: /var/log/kubernetes/audit
       type: DirectoryOrCreate
   # needed for kmsg input plugin
   - name: uptime
     hostPath:
       path: /proc/uptime
       type: File
   - name: kmsg
     hostPath:
       path: /dev/kmsg
       type: CharDevice

   extraVolumeMounts:
   - name: tail-db
     mountPath: /tail-db
   - name: kubernetes-audit
     mountPath: /var/log/kubernetes/audit
   - name: uptime
     mountPath: /proc/uptime
   - name: kmsg
     mountPath: /dev/kmsg

   config:
     inputs: |
   ```

```
    # Collect audit logs, systemd logs, and kernel logs.
    # Pod logs are collected by the fluent-bit deployment managed by logging-
operator.
    [INPUT]
        Name tail
        Alias kubernetes_audit
        Path /var/log/kubernetes/audit/*.log
        Parser kubernetes-audit
        DB /tail-db/audit.db
        Tag audit.*
        Refresh_Interval 10
        Rotate_Wait 5
        Mem_Buf_Limit 135MB
        Buffer_Chunk_Size 5MB
        Buffer_Max_Size 20MB
        Skip_Long_Lines Off
    [INPUT]
        Name systemd
        Alias kubernetes_host
        DB /tail-db/journal.db
        Tag host.*
        Max_Entries 1000
        Read_From_Tail On
        Strip_Underscores On
    [INPUT]
        Name kmsg
        Alias kubernetes_host_kernel
        Tag kernel
```

6. Add the following item to the list under the `extraVolumes` key.

```
- name: kubernetes-host
  hostPath:
    path: <path to systemd logs on the node>
    type: Directory
```

7. Add the following item to the list under the `extraVolumeMounts` key.

```
- name: kubernetes-host
  mountPath: <path to systemd logs on the node>
```

These items will make Kubernetes mount logs into Fluent Bit pods.

8. Add the following line into the `[INPUT]` entry identified by `Name systemd` and `Alias kubernetes_host`.

```
Path <path to systemd logs on the node>
```

This is needed to make Fluent Bit actually collect the mounted logs.

9. Assuming that the path to systemd logs on the node is `/run/log/journal`, the result will look similar to this.

```
extraVolumes:
# we create this to have a persistent tail-db directory an all nodes
# otherwise a restarted fluent-bit would rescrape all tails
- name: tail-db
  hostPath:
    path: /var/log/tail-db
    type: DirectoryOrCreate
# we create this to get rid of error messages that would appear on non control-
plane nodes
- name: kubernetes-audit
  hostPath:
```

```
    path: /var/log/kubernetes/audit
    type: DirectoryOrCreate
# needed for kmsg input plugin
- name: uptime
  hostPath:
    path: /proc/uptime
    type: File
- name: kmsg
  hostPath:
    path: /dev/kmsg
    type: CharDevice
- name: kubernetes-host
  hostPath:
    path: /run/log/journal
    type: Directory

extraVolumeMounts:
- name: tail-db
  mountPath: /tail-db
- name: kubernetes-audit
  mountPath: /var/log/kubernetes/audit
- name: uptime
  mountPath: /proc/uptime
- name: kmsg
  mountPath: /dev/kmsg
- name: kubernetes-host
  mountPath: /run/log/journal

config:
  inputs: |
    # Collect audit logs, systemd logs, and kernel logs.
    # Pod logs are collected by the fluent-bit deployment managed by logging-
operator.
    [INPUT]
        Name tail
        Alias kubernetes_audit
        Path /var/log/kubernetes/audit/*.log
        Parser kubernetes-audit
        DB /tail-db/audit.db
        Tag audit.*
        Refresh_Interval 10
        Rotate_Wait 5
        Mem_Buf_Limit 135MB
        Buffer_Chunk_Size 5MB
        Buffer_Max_Size 20MB
        Skip_Long_Lines Off
    [INPUT]
        Name systemd
        Alias kubernetes_host
        Path /run/log/journal
        DB /tail-db/journal.db
        Tag host.*
        Max_Entries 1000
        Read_From_Tail On
        Strip_Underscores On
    [INPUT]
        Name kmsg
        Alias kubernetes_host_kernel
        Tag kernel
```

10. Create a `ConfigMap` manifest with override values from `fluent-bit-values.yaml`.

```
cat <<EOF >fluent-bit-overrides.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: ${WORKSPACE_NAMESPACE}
  name: fluent-bit-overrides
data:
  values.yaml: |
$(cat fluent-bit-values.yaml | sed 's/^/    /g')
EOF
```

11. Create a `ConfigMap` from the manifest above.

```
kubectl apply -f fluent-bit-overrides.yaml
```

12. Edit the `fluent-bit` AppDeployment to set the value of `spec.configOverrides.name` to the name of the created `ConfigMap`. You can use the steps in the procedure, and deploy an application with a custom configuration.

```
nkp edit appdeployment -n ${WORKSPACE_NAMESPACE} fluent-bit
```

After your editing is complete, the AppDeployment resembles this example.

```
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: fluent-bit
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    name: fluent-bit-0.20.9
    kind: ClusterApp
  configOverrides:
    name: fluent-bit-overrides
```

13. Log in to the Grafana logging UI of your workspace and verify that logs with a label `log_source=kubernetes_host` are now present in Loki.

Configuring Loki to Use AWS S3 Storage in NKP

About this task

Follow the instructions on this page to configure Loki to use AWS S3 Storage in NKP .

Procedure

1. Execute the following command to get the namespace of your workspace.

```
nkp get workspaces
```

2. Set the `WORKSPACE_NAMESPACE` variable to the namespace copied in the previous step.

```
export WORKSPACE_NAMESPACE=<WORKSPACE_NAMESPACE>
```

3. Create a secret containing the static AWS S3 credentials. The secret is then mounted into each of the Grafana Loki pods as environment variables.

```
kubectl create secret generic nkp-aws-s3-creds -n${WORKSPACE_NAMESPACE} \
--from-literal=AWS_ACCESS_KEY_ID=<key id> \
```

```
--from-literal=AWS_SECRET_ACCESS_KEY=<secret key>
```

4. Create a config that overrides ConfigMap to update the storage configuration.

   Note: This can also be added to the installer configuration if you are configuring Grafana Loki on the Management Cluster.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: grafana-loki-overrides
  namespace: ${WORKSPACE_NAMESPACE}
data:
  values.yaml: |
    loki:
      annotations:
        secret.reloader.stakater.com/reload: nkp-aws-s3-creds
      structuredConfig:
        storage_config:
          aws:
            s3: s3://<region>/<bucket name>
    ingester:
      extraEnvFrom:
        - secretRef:
            name: nkp-aws-s3-creds
    querier:
      extraEnvFrom:
        - secretRef:
            name: nkp-aws-s3-creds
    queryFrontend:
      extraEnvFrom:
        - secretRef:
            name: nkp-aws-s3-creds
    compactor:
      extraEnvFrom:
        - secretRef:
            name: nkp-aws-s3-creds
    ruler:
      extraEnvFrom:
        - secretRef:
            name: nkp-aws-s3-creds
    distributor:
      extraEnvFrom:
        - secretRef:
            name: nkp-aws-s3-creds
EOF
```

5. Update the `grafana-loki` AppDeployment to apply the configuration override.

   Note: If you use the Kommander CLI installation configuration file, you don't need this step

```
cat << EOF | kubectl -n ${WORKSPACE_NAMESPACE} patch appdeployment grafana-loki --
type="merge" --patch-file=/dev/stdin
spec:
  configOverrides:
    name: grafana-loki-overrides
EOF
```

Customizing Logging Stack Applications

About this task

This page provides instructions on how you can customize the Logging Stack Applications in NKP.

Procedure

1. Retrieve the Workspace Namespace

   a. On the management cluster, run the following command to get the namespace of your workspace.

   ```
   nkp get workspaces
   ```

   b. Copy the value under the `NAMESPACE` column for your workspace.

   c. Set the `WORKSPACE_NAMESPACE` variable to the namespace copied in the previous step.

   ```
   export WORKSPACE_NAMESPACE=<WORKSPACE_NAMESPACE>
   ```

2. Customize Logging Stack applications.

   a. On the Attached or Managed Cluster, retrieve the kubeconfig for the cluster.

   b. Apply the ConfigMap directly to the managed/attached cluster using the name, `logging-operator-logging-overrides`.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: v1
   kind: ConfigMap
   metadata:
     name: logging-operator-logging-overrides
     namespace: ${WORKSPACE_NAMESPACE}
   data:
     values.yaml: |
       <insert config here>
   EOF
   ```

   This is an example of a ConfigMap that contains customized resource requests and limit values for `fluentd`:

   ```
   apiVersion: v1
   kind: ConfigMap
   metadata:
     name: logging-operator-logging-overrides
     namespace: kommander
   data:
     values.yaml: |
       fluentd:
         resources:
           limits:
             cpu: 1
             memory: 2000Mi
           requests:
             cpu: 1
             memory: 1500Mi
   ```

Security

Details on distributed authentication and authorization between clusters

Authentication

NKP UI comes with a pre-configured authentication Dex identity broker and provider.

Warning: Kubernetes, Kommander, and Dex do not store any user identities. The Kommander installation comes with default admin static credentials. These credentials should only be used to access the NKP UI for configuring an external identity provider. Currently, there is no way to update these credentials, so they should be treated as backup credentials and not used for normal access.

The NKP UI admin credentials are stored as a secret. They never leave the boundary of the UI cluster and are never shared with any other cluster.

The Dex service issues an OIDC ID token (see https://openid.net/specs/openid-connect-core-1_0.html#IDToken) on successful user authentication. Other platform services use ID tokens as proof of authentication. User identity to the Kubernetes API server is provided by the `kube-oidc-proxy` platform service that reads the identity from an ID token. Web requests to NKP UI access are authenticated by the traefik forward auth platform service (see https://github.com/mesosphere/traefik-forward-auth).

Note: The kube-oidc-proxy service (see https://github.com/jetstack/kube-oidc-proxy) authenticates kubectl CLI requests using the Kubernetes API Server Go library. This library requires that if an email_verified claim is present, it must be set to true, even if the `insecureSkipEmailVerified: true` flag is configured in the Dex connector. Thus, ensure that the OIDC provider is configured to set the email_verified field to 'true'.

A user identity is shared across a UI cluster and all other attached clusters.

Attached Clusters

A newly attached cluster has federated `kube-oidc-proxy`, `dex-k8s-authenticator`, and `traefik-forward-auth`platform applications. These platform applications are configured to accept the Management or Pro cluster (see Cluster Types on page 19), Dex issues ID tokens.

When the `traefik-forward-auth` is used as a Traefik Ingress authenticator (see https://doc.traefik.io/traefik/v2.4/providers/kubernetes-ingress/), it checks if the user identity was issued by the Kommander cluster Dex service issued the user identity to the Management/Pro cluster Dex service to authenticate and confirm their identity.

The Kommander cluster Dex service issued the user identity for the attached clusters. On the ManagementPro cluster, use the static admin credentials or an external identity provider (IDP).

Authorization

Kommander does not have a centralized authorization component, and the service makes its own authorization decisions based on user identity.

OpenID Connect (OIDC)

An introduction to OpenID Connect (OIDC) Authentication in Kubernetes.

All Kubernetes clusters have two categories of users: service accounts and normal users. Kubernetes manages authentication for service accounts, but the cluster administrator, or a separate service, manages authentication for normal users.

Kommander configures the cluster to use OpenID Connect (OIDC), a popular and extensible user authentication method and installs Dex. This popular, open-source software product integrates your existing Identity Providers with Kubernetes.

To begin, set up an Identity Provider with Dex, then use OIDC as the Authentication method.

Identity Providers

An Identity Provider (IdP) is a service that lets you manage identity information for users, including groups. A cluster created in Kommander uses Dex as its IdP. Dex, in turn, delegates to one or more external IdPs.

If you already use one or more of the following IdPs, you can configure Dex to use them:

Table 54: Table

Name	Supports Refresh Tokens	Supports Groups Claim	Supports `preferred_user` Claim	Status	Notes
LDAP	yes	yes	yes	stable	
GitHub	yes	yes	yes	stable	
SAML 2.0	no	yes	no	stable	
GitLab	yes	yes	yes	beta	
OpenID Connect	yes	yes	yes	beta	Includes Salesforce, Azure, etc.
Google	yes	yes	yes	alpha	
LinkedIn	yes	no	no	beta	
Microsoft	yes	yes	no	beta	
AuthProxy	no	no	no	alpha	Authentication proxies such as Apache2 mod_auth, etc.
Bitbucket Cloud	yes	yes	no	alpha	
OpenShift	no	yes	no	stable	

Note:

These are the Identity Providers supported by Dex 2.22.0, the version used by NKP.

Login Connectors

Kommander uses Dex to provide OpenID Connect single sign-on (SSO) to the cluster. Dex can be configured to use multiple connectors, including GitHub, LDAP, and SAML 2.0. The Dex Connector documentation describes how to configure different connectors. You can add the configuration as the values field in the Dex application. An example Dex configuration provided to the Kommander CLI's `install` command is similar to this:

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
apps:
  dex:
    values: |
      config:
        connectors:
        - type: oidc
          id: google
          name: Google
          config:
            issuer: https://accounts.google.com/o/oauth2/v2/auth
            clientID: YOUR_CLIENT_ID
            clientSecret: YOUR_CLIENT_SECRET
            redirectURI: https://NKP_CLUSTER_DOMAIN/dex/callback
            scopes:
            - openid
            - profile
```

```
            - email
            insecureSkipEmailVerified: true
            insecureEnableGroups: true
            userIDKey: email
            userNameKey: email
[...]
```

Access Token Lifetime

By default, the client access token lifetime is 24 hours. After this time, the token expires and cannot be used to authenticate.

For more information on access token expiration and rotation settings, see https://dexidp.io/docs/configuration/tokens/#expiration-and-rotation-settings.

Here is an example configuration for extending the token lifetime to 48 hours:

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
apps:
  dex:
    values: |
      config:
        expiry:
          idTokens: "48h"
[...]
```

Authentication

OpenID Connect is an extension of the OAuth2 authentication protocol. As required by OAuth2, the client must be registered with Dex. Do this by passing the name of the application and a callback/redirect URI. These handle the processing of the OpenID token after the user authenticates successfully. After registration, Dex returns a `client_id` and a `secret`. Authentication requests use these between the client and Dex to identify the client.

For more information on OAuth2 authentication protocol, https://oauth.net/2/.

Users access Kommander in two ways:

- To interact with Kubernetes API, usually through `kubectl`.
- To interact with the NKP UI, which has GUI dashboards for Prometheus, Grafana, etc.

In Kommander, Dex comes pre-configured with a client for these access use cases. The clients talk to Dex for authentication. Dex talks to the configured Identity Provider, or IdP, (for example LDAP, SAML, etc) to perform the actual task of authenticating the user.

If the user authenticates successfully, Dex pulls the user's information from the IdP and forms an OpenID token. The token contains this information and returns it to the respective client's callback URL. The client or end user uses this token for communicating with the NKP UI or Kubernetes API respectively.

This figure illustrates these components and their interaction at a high level:

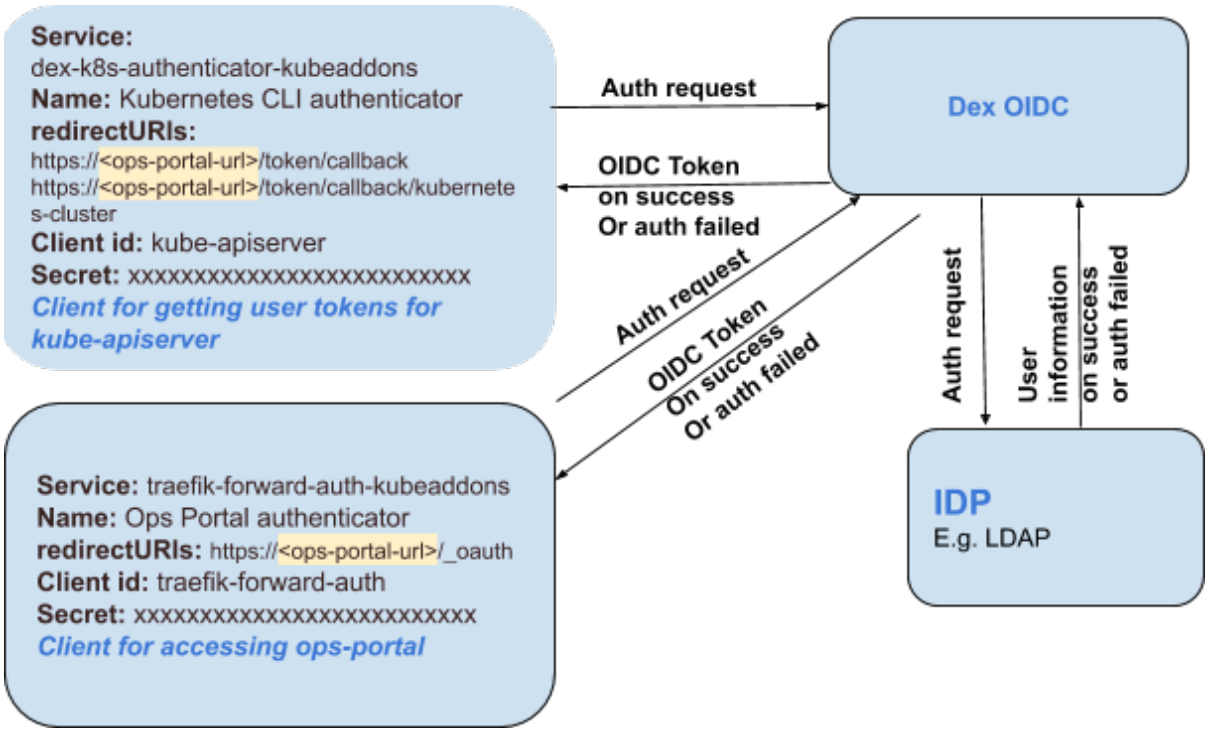


Figure 21: OIDC Authentication Flow with Dex

Connecting Kommander to an IdP Using SAML

About this task

Learn how to enforce policies using `Gatekeeper` Gatekeeper is the policy controller for Kubernetes, allowing organizations to enforce configurable policies using the Open Policy Agent, a policy engine for Cloud Native environments hosted by CNCF as a graduated-level project. This tutorial describes how to use Gatekeeper to enforce policies by rejecting non-compliant resources. Specifically, this tutorial describes two constraints as a way to use Gatekeeper as an alternative to Pod Security Policies: Prevent the running of privileged pods, Prevent mounting host path volumes

Procedure

1. Modify the `dex` configuration.
   For this step, get the following from your IdP:

   - single sign-on URL or SAML URL: `ssoURL`
   - base64 encoded, PEM encoded CA certificate: `caData`
   - username attribute name in SAML response:> `usernameAttr`
   - email attribute name in SAML response: `emailAttr`

   From above, you need the following:

   - issuer URL: `entityIssuer`
   - callback URL: `redirectURI`

   Ensure you base64 encode the contents of the PEM file. For example, the contents' prefix will result in this exact base64 prefix.

   ```
   LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tC[...]
   ```

   You can add the configuration as values field in the `dex` application.

2. Modify the `traefik-foward-auth-mgmt` configuration and add a whitelist.

   This step is required to give a user access to the NKP UI. For each user, you must give access to Kubernetes resources (see Access Control on page 354) and add an entry in the `whitelist`below.

   ```
   apiVersion: config.kommander.mesosphere.io/v1alpha1
   kind: Installation
   apps:
   ...
     traefik-forward-auth-mgmt:
       values: |
         traefikForwardAuth:
           allowedUser:
             valueFrom:
               secretKeyRef: null
           whitelist:
           - < allowed email addresses >
   ```

3. Run `kommander install --installer-config kommander.yaml` to deploy modified `dex`.
4. Visit `https://<your-cluster-host>/nkp/kommander/dashboard` to login to the NKP UI.
5. Select `Launch Console` and follow the authentication steps to complete the procedure.

Enforcing Policies Using Gatekeeper

Gatekeeper uses the OPA Constraint Framework to describe and enforce policy. Before you can define a constraint, you must first define a `ConstraintTemplate`, which describes both the `Rego` (a powerful query language) that enforces the constraint and the schema of the constraint. The schema of the constraint allows an admin to fine-tune the behavior of a constraint, much like arguments to a function.

About this task

For more information on the OPA Constraint Framework, see https://github.com/open-policy-agent/frameworks/tree/master/constraint. The Gatekeeper repository includes a library of policies to replace Pod Security Policies, which you will use. For more information, see https://github.com/open-policy-agent/gatekeeper-library/tree/master/library/pod-security-policy.

Learn how to enforce policies using Gatekeeper. Gatekeeper Gatekeeper is the policy controller for Kubernetes, allowing organizations to enforce configurable policies using the Open Policy Agent (See https://github.com/open-policy-agent/opa), a policy engine for Cloud Native environments hosted by CNCF as a graduate-level project. This tutorial describes how to use Gatekeeper to enforce policies by rejecting non-compliant resources. Specifically, this tutorial describes two constraints as a way to use Gatekeeper as an alternative to Pod Security Policies (see https://kubernetes.io/docs/concepts/policy/pod-security-policy/).

Before you begin

- You must have access to a Linux, macOS, or Windows computer with a supported operating system version.
- You must have a properly deployed and running cluster. For information about deploying Kubernetes with default settings on different types of infrastructures, see the Custom Installation and Infrastructure Tools on page 677.
- If you install Kommander with a custom configuration, make sure you enable Gatekeeper.

> **Warning:** If you intend to disable Gatekeeper, keep in mind that the app is deployed pre-configured with constraint templates that enforce multi-tenancy in projects.

Procedure

1. Preventing the Running of Privileged Pods on page 619
2. Preventing the Mounting of Host Path Volumes on page 620

Preventing the Running of Privileged Pods

Procedure

1. Define the ConstraintTemplate.

   Create the privileged pod policy constraint template `k8spspprivilegedcontainer` by running the following command.

   ```
   kubectl apply -f https://raw.githubusercontent.com/open-policy-agent/gatekeeper-library/master/library/pod-security-policy/privileged-containers/template.yaml
   ```

2. Define the Constraint. Constraints are then used to inform

   The gatekeeper that the admin wants to enforce a constraint template he privileged pod policy constraint `psp-privileged-container` by running the following command.

   ```
   kubectl apply -f https://raw.githubusercontent.com/open-policy-agent/gatekeeper-library/master/library/pod-security-policy/privileged-containers/samples/psp-privileged-container/constraint.yaml
   ```

3. Test that the constraint is enforced.

   Create a privileged pod by running the following command:

   ```
   kubectl apply -f https://raw.githubusercontent.com/open-policy-agent/gatekeeper-library/master/library/pod-security-policy/privileged-containers/samples/psp-privileged-container/example_disallowed.yaml
   ```

   You should see the following output:

   ```
   Error from server ([denied by psp-privileged-container] Privileged container is
    not allowed: nginx, securityContext: {"privileged": true}): error when creating
    "https://raw.githubusercontent.com/open-policy-agent/gatekeeper-library/master/
   library/pod-security-policy/privileged-containers/samples/psp-privileged-container/
   ```

```
example_disallowed.yaml": admission webhook "validation.gatekeeper.sh" denied the
 request: [denied by psp-privileged-container] Privileged container is not allowed:
 nginx, securityContext: {"privileged": true}
```

Preventing the Mounting of Host Path Volumes

Procedure

1. Define the ConstraintTemplate.

   Create the host path volume policy constraint template `k8spsphostfilesystem` by running the following command.

   ```
   kubectl apply -f https://raw.githubusercontent.com/open-policy-agent/gatekeeper-
   library/master/library/pod-security-policy/host-filesystem/template.yaml
   ```

2. Define the Constraint. Constraints are then used to inform Gatekeeper that the admin wants to enforce a ConstraintTemplate, and how.

   Create the host path volume policy constraint `psp-host-filesystem` by running the following command to only allow `/foo` to be mounted as a host path volume.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: constraints.gatekeeper.sh/v1beta1
   kind: K8sPSPHostFilesystem
   metadata:
     name: psp-host-filesystem
   spec:
     match:
       kinds:
         - apiGroups: [""]
           kinds: ["Pod"]
     parameters:
       allowedHostPaths:
       - readOnly: true
         pathPrefix: "/foo"
   EOF
   ```

3. Test that the constraint is enforced.

   Create a privileged pod by running the following command:

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: v1
   kind: Pod
   metadata:
     name: nginx-host-filesystem
     labels:
       app: nginx-host-filesystem
   spec:
     containers:
     - name: nginx
       image: nginx
       volumeMounts:
       - mountPath: /cache
         name: cache-volume
         readOnly: true
     volumes:
     - name: cache-volume
       hostPath:
         path: /tmp # directory location on host
   ```

```
EOF
```

You should see the following output:

```
Error from server ([denied by psp-host-filesystem] HostPath volume {"hostPath":
 {"path": "/tmp", "type": ""}, "name": "cache-volume"} is not allowed, pod: nginx-
host-filesystem. Allowed path: [{"readOnly": true, "pathPrefix": "/foo"}]): error
 when creating "STDIN": admission webhook "validation.gatekeeper.sh" denied the
 request: [denied by psp-host-filesystem] HostPath volume {"hostPath": {"path":
 "/tmp", "type": ""}, "name": "cache-volume"} is not allowed, pod: nginx-host-
filesystem. Allowed path: [{"readOnly": true, "pathPrefix": "/foo"}]
```

4. Test that the constraint to chech the allowed host paths.

   Create a pod that mounts an allowed host path by running the following command:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: nginx-host-filesystem
  labels:
    app: nginx-host-filesystem
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - mountPath: /cache
      name: cache-volume
      readOnly: true
  volumes:
  - name: cache-volume
    hostPath:
      path: /foo # directory location on host
EOF
```

   You should see the following output:

```
pod/nginx-host-filesystem created
```

Traefik-Forward-Authentication in NKP (TFA)

Traefik-Forward-Authentication (TFA) is a vital component in NKP. It handles authentication and authorization of web (HTTP) requests and how users can access the NKP UI. Regardless of the authentication method you choose (user and password, SSO, 2FA, etc.), TFA is a component that authorizes HTTP clients to enter your environment.

TFA is one of the standard applications in the Kommander component of NKP. It is deployed by a controller on all attached, managed, and management clusters.

TFA Authentication Workflow

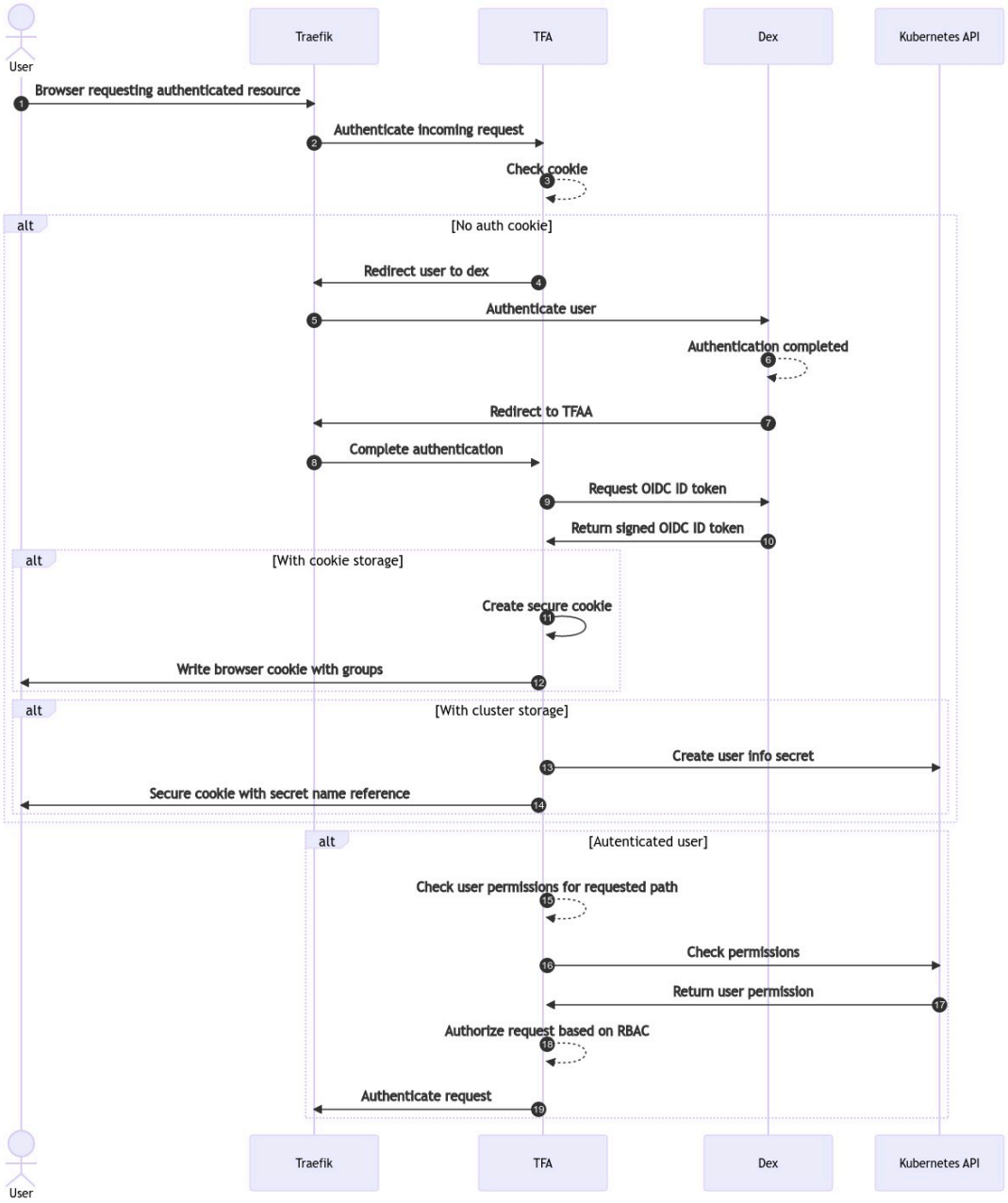


Figure 22: TFA Authentication Workflow

Default TFA Configuration in NKP

In the default configuration in NKP, TFA stores all authentication information about users through the browser cookies. When TFA authenticates users, it stores the user's metadata in encrypted browser cookies. Subsequent requests following initial authentication will use these cookies to recognize users without the need to re-authenticate them.

-
  -
    - The cookies are securely encrypted, so they cannot be modified by users.
- The cookies contain the RBAC username.

- The cookies contain a list of groups that users are associated with.

Cluster Storage Option

The browser cookie storage is limited to a maximum of 4Kb per cookie. However, if a user is assigned to a large number of groups, this limitation can be exceeded and this will return a response 500-internal server error, meaning that the user will be unable to access any web services.

To work around the cookie storage size limit, TFA can be configured to store the metadata claims in the cluster as a Kubernetes secret instead of in the browser. To do so, the `clusterStorage` option can be configured in the Traefik Forward Auth application when installing Kommander.

In order to enable the `clusterStorage` option, add the following to the cluster.yaml when installing Kommander:

```
traefik-forward-auth-mgmt:
  values: |
    clusterStorage:
      enabled: true
      namespace: kommander
```

If the `clusterStorage` feature is enabled, automatic garbage collection will delete the secrets after 12 hours. Keep in mind that enabling this feature will have performance implications for web requests, because TFA needs to load the secret to retrieve the user groups for each HTTP API request. Because of this, we recommend first trying to reduce the number of groups associated users and only enabling this option if that cannot be accomplished.

For more information on traefik-forward-authentication, see https://github.com/mesosphere/traefik-forward-auth.

Local Users

Nutanix recommends configuring an external identity provider to manage access to your cluster.

For an overview of the benefits, supported providers, and instructions on how to configure an external identity provider, see Identity Providers.

However, you can create local users as an alternative, for example, when you want to test RBAC policies quickly. NKP automatically creates a unique local user, the `admin` user, but you can create additional local users and assign them other RBAC roles.

This section shows how to create and manage local users to access your Pro or Management cluster.

There are two ways in which you can create local users:

- Creating Local Users During the Kommander Installation on page 623
- Creating Local Users After the Kommander Installation on page 624

Creating Local Users During the Kommander Installation

Create local users after you install the Kommander component of NKP. If you have not already installed Kommander yet and want to create additional users during the installation.

About this task

> **Warning:** Nutanix does not recommend creating local users for production clusters. See for instructions on how to configure an external identity provider to manage your users.

Customize the Dex AppDeployment, and add a `configOverrides` section.

Procedure

1. Open the Kommander Installer Configuration File or `kommander.yaml` file. If you do not have the `kommander.yaml` file, initialize the configuration file so you can edit it in the subsequent steps.

   Note: Initialize this file only one time, otherwise you will- overwrite previous customizations.

2. In that file, add the following customization for `dex` and create local users by establishing their credentials.

   - Replace `<example_email>` with the user's email address or username.
   - Replace `<password_bcrypt_hash>` with the bcrypt hash of the password you want to assign. You can use the htpasswd CLI to create the hash of a specific password. For example, by running `htpasswd -bnBC 10 "" password | tr -d ':\n' && echo`, you can generate the hash for the password “password”.

```
app:
  dex:
    enabled: true
    values: |
      config:
        staticPasswords:
        - email: <example_email>
          hash: <password_bcrypt_hash>
        - email: <example_email2>
          hash: <password_bcrypt_hash2>
```

3. Install Kommander using the customized installation file.`kommander.yaml` file.

   For more information, see Installing Kommander with a Configuration File on page 1036.

   After Kommander finishes installing, you have created local users, but you cannot use them until you have assigned them permissions. To complete the configuration, see Adding RBAC Roles to Local Users on page 626.

   Warning: You have created a user that does not have any permissions to see or manage your NKP cluster yet.

   You have created a user that does not have any permissions to see or manage your NKP cluster yet.

Creating Local Users After the Kommander Installation

Create local users before you install the Kommander component of NKP. If you have already installed Kommander.

About this task

Warning: Nutanix does not recommend creating local users for production clusters. See for instructions on how to configure an external identity provider to manage your users.

Customize the Dex AppDeployment, and add a `configOverrides` section:

Procedure

1. Create a `configMap` resource with the credentials of the new local user

   - Replace `<example_email>` with the user's email address or username.
   - Replace `<password_bcrypt_hash>` with the bcrypt hash of the password you want to assign. You can use the htpasswd CLI to create the hash of a specific password. For example, by running `htpasswd -bnBC 10 "" password | tr -d ':\n' && echo`, you can generate the hash for the password “password”.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: v1
   kind: ConfigMap
   metadata:
     name: dex-overrides
     namespace: kommander
   data:
     values.yaml: |
       config:
         staticPasswords:
         - email: <example_email>
           hash: <password_bcrypt_hash>
   EOF
   ```

2. Open the Dex AppDeployment to edit it.

   ```
   kubectl edit -n kommander appdeployment dex
   ```

   The editor displays the AppDeployment.

3. Copy the following values and paste them in a location in the file where they are nested in the `spec` field.

   ```
     configOverrides:
       name: dex-overrides
   ```

   Example:

   ```
   apiVersion: apps.kommander.d2iq.io/v1alpha3
   kind: AppDeployment
   metadata:
   ...
   spec:
     appRef:
       kind: ClusterApp
       name: dex-2.11.1
     clusterConfigOverrides:
     - clusterSelector:
         matchExpressions:
         - key: kommander.d2iq.io/cluster-name
           operator: In
           values:
           - host-cluster
       configMapName: dex-kommander-overrides
     configOverrides: # Copy and paste this section.
       name: dex-overrides
   status:
   ```

```
...
```

Editing the AppDeployment restarts the HelmRelease for Dex. The new users will be created after the reconciliation. However, the user creation is not completed until you assign it permissions.

> **Note:** You have created a user that does not have any permissions to see or manage your NKP cluster yet.

To complete the configuration, see Adding RBAC Roles to Local Users on page 626.

Adding RBAC Roles to Local Users

Manage access to your cluster and its resources by assigning Kubernetes RBAC roles to local users.

About this task

If you have not created local users yet, see

Creating Local Users During the Kommander Installation on page 623 or Creating Local Users After the Kommander Installation on page 624.

To assign a Role:

Procedure

Create the following `ClusterRoleBinding` resource:.

- Replace `<example_email>` with the user's email address or username.
- Replace `<cluster_admin>` with the RBAC role you want to assign to a user.
- If you have configured an Identity Provider for a specific workspace (see Multi-Tenancy in NKP on page 436), configure the `subjects.name` field to `<workspace_ID>:<user_email>`. For example, `tenant-z:jane.doe@example.com`.

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: cluster-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: <example_email>
EOF
```

After assigning the previous role to `<example_email>`, the user is able to log in to the cluster using the credentials you assigned in Creating Local Users During the Kommander Installation on page 623 or Creating Local Users After the Kommander Installation on page 624.

> **Note:** The Login page and cluster URL are the same for the default admin user and the local users you create with this method.

For more information on RBAC resources in NKP and granting access to Kubernetes and Kommander resources, see Access to Kubernetes and Kommander Resources on page 356.

For general information on RBAC as a Kubernetes resource, see https://kubernetes.io/docs/reference/access-authn-authz/rbac/.

Modifying Local Users

Procedure

1. To change the password or username of a user, edit the `dex-overrides` ConfigMap.
2. If you change the email address or username of a user, ensure you update any `RoleBindings` or RBAC resources associated with this user.

Deleting Local Users

Procedure

To delete local users, edit the `dex-overrides` ConfigMap and remove the email and hash fields for the user. Also, ensure you delete any `RoleBindings` or RBAC resources associated with this user.

Networking

Configure networking for the Konvoy cluster.

This section describes different networking components that come together to form a Konvoy networking stack. It assumes familiarity with Kubernetes networking.

Networking Service

A Service is an API resource that defines a logical set of pods and a policy by which to access them, and is an abstracted manner to expose applications as network services.

For more information on service, see https://kubernetes.io/docs/concepts/services-networking/service/#service-resource.

Kubernetes gives pods their own IP addresses and a single DNS name for a set of pods. Services are used as entry points to load-balance the traffic across the pods. A selector determines the set of Pods targeted by a Service.

For example, if you have a set of pods that each listen on TCP port `9191` and carry a label `app=MyKonvoyApp`, as configured in the following:

```
apiVersion: v1
kind: Service
metadata:
  name: my-konvoy-service
  namespace: default
spec:
  selector:
    app: MyKonvoyApp
  ports:
  - protocol: TCP
    port: 80
    targetPort: 9191
```

This specification creates a new `Service` object named `"my-konvoy-service"`, that targets TCP port `9191` on any pod with the `app=MyKonvoyApp` label.

Kubernetes assigns this Service an IP address. In particular, the `kube-proxy` implements a form of virtual IP for Services of type other than `ExternalName`.

Note:

- The name of a Service object must be a valid DNS label name.
- A Service is not a Platform Service.

Service Topology
A Service Topology is a mechanism in Kubernetes that routes traffic based on the cluster's Node topology. For example, you can configure a Service to route traffic to endpoints on specific nodes or even based on the region or availability zone of the node's location.

To enable this new feature in your Kubernetes cluster, use the feature gates `--feature-gates="ServiceTopology=true,EndpointSlice=true"` flag. After enabling, you can control Service traffic routing by defining the `topologyKeys` field in the Service API object.

In the following example, a Service defines `topologyKeys` to be routed to endpoints only in the same zone:

```
apiVersion: v1
kind: Service
metadata:
  name: my-konvoy-service
  namespace: default
spec:
  selector:
    app: MyKonvoyApp
  ports:
  - protocol: TCP
    port: 80
    targetPort: 9191
  topologyKeys:
  - "topology.kubernetes.io/zone"
```

> **Note:** If the value of the `topologyKeys` field does not match any pattern, the traffic is rejected.

EndpointSlices
EndpointSlices are an API resource that appears as a scalable and more manageable solution to network endpoints within a Kubernetes cluster. They allow for distributing network endpoints across multiple resources with a limit of 100 endpoints per EndpointSlice.

For more information, see https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/#endpointslice-resource.

An EndpointSlice contains references to a set of endpoints, and the control plane takes care of creating EndpointSlices for any service that has a selector specified. These EndpointSlices include references to all the pods that match the Service selector.

Like Services, the name of a EndpointSlice object must be a valid DNS subdomain name.

In this example, here's a sample EndpointSlice resource for the example Kubernetes Service:

```
apiVersion: discovery.k8s.io/v1beta1
kind: EndpointSlice
metadata:
  name: konvoy-endpoint-slice
  namespace: default
  labels:
    kubernetes.io/service-name: my-konvoy-service
addressType: IPv4
ports:
- name: http
  protocol: TCP
  port: 80
endpoints:
- addresses:
  - "192.168.126.168"
  conditions:
    ready: true
  hostname: ip-10-0-135-39.us-west-2.compute.internal
  topology:
```

```
    kubernetes.io/hostname: ip-10-0-135-39.us-west-2.compute.internal
    topology.kubernetes.io/zone: us-west2-b
```

DNS for Services and Pods

Every new Service object in Kubernetes gets assigned a DNS name. The Kubernetes DNS component schedules a DNS name for the pods and services created on the cluster, and then the Kubelets are configured so containers can resolve these DNS names.

Considering previous examples, assume there is a Service named `my-konvoy-service` in the Kubernetes namespace `default`. A Pod is running in namespace `default` can look up this service by performing a DNS query for `my-konvoy-service`. A Pod running in namespace `kommander` can look up this service by performing a DNS query for `my-konvoy-service.default`.

In general, a pod has the following DNS resolution:

```
pod-ip-address.namespace-name.pod-name.cluster-domain.example.
```

Similarly, a service has the following DNS resolution:

```
service-name.namespace-name.svc.cluster-domain.example.
```

For information about all the possible record types and layouts and DNS for Services and Pods, see https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/.

Ingress and Networking

Ingress is an API resource that manages external access to the services in a cluster through HTTP or HTTPS. It offers name-based virtual hosting, SSL termination, and load balancing when exposing HTTP/HTTPS routes from outside to services in the cluster.

For information on Ingress, see https://kubernetes.io/docs/concepts/services-networking/ingress/#what-is-ingress.

The traffic policies are controlled by rules as part of the Ingress definition. Each rule defines the following details:

- An optional host to which to apply the rules.
- A list of paths or routes that has an associated backend defined with a Service `name`, a port `name` and `number`.
- A backend is a combo of a Service and port names, or a custom resource backend defined as a CRD. Consequently HTTP/HTTPS requests to the Ingress that match the host and path of the rule are sent to the listed backend.

An example of an Ingress specification is:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: konvoy-ingress
  namespace: default
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - http:
      paths:
      - path: /path
        pathType: Prefix
        backend:
          service:
            name: my-konvoy-service
            port:
```

```
              number: 80
```

In Kommander, you can expose services to the outside world using Ingress objects.

Ingress Controllers

In contrast with the controllers in the Kubernetes control plane, Ingress controllers are not started with a cluster so you need to choose the desired Ingress controller.

An Ingress controller has to be deployed in a cluster for the Ingress definitions to work.

Kubernetes, as a project, currently supports and maintains GCE and nginx controllers.

These are four of the most known Ingress controllers:

- HAProxy Ingress is a highly customizable community-driven ingress controller for HAProxy. See https://haproxy-ingress.github.io/
- NGINX offers support and maintenance for the NGINX Ingress Controller for Kubernetes. See https://www.nginx.com/products/nginx-ingress-controller.
- Traefik is a fully featured Ingress controller (Let's Encrypt, secrets, http2, websocket), and has commercial support. See https://github.com/containous/traefik.
- Ambassador API Gateway EXPERIMENTAL is an Envoy based Ingress controller with the community and commercial support. See https://www.getambassador.io/.

In Kommander, `Traefik` deploys by default as a well-suited Ingress controller.

Network Policies

NetworkPolicy is an API resource that controls the traffic flow at port level 3 or 4 or at the IP address level. It enables defining constraints on how a pod communicates with various network services such as `endpoints` and `services`.

A Pod can be restricted to talk to other network services through a selection of the following identifiers:

- Namespaces that have to be accessed. There can be pods that are not allowed to talk to other namespaces.
- Other allowed IP blocks regardless of the node or IP address assigned to the targeted Pod.
- Other allowed Pods.

An example of a NetworkPolicy specification is:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: network-konvoy-policy
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - ipBlock:
        cidr: 172.17.0.0/16
        except:
        - 172.17.1.0/24
    - namespaceSelector:
```

```
      matchLabels:
        app: MyKonvoyApp
  - podSelector:
      matchLabels:
        app: MyKonvoyApp
  ports:
  - protocol: TCP
    port: 6379
egress:
- to:
  - ipBlock:
      cidr: 10.0.0.0/24
  ports:
  - protocol: TCP
    port: 5978
```

As shown in the example, when defining a pod or namespace based NetworkPolicy, you use a selector to specify what traffic is allowed to and from the Pod(s).

Adding Entries to Pod /etc/hosts with HostAliases

The Pod API resource definition has a `HostAliases` field that allows adding entries to the Pod's container `/etc/hosts` file. This field overrides the hostname resolution when DNS and other options are not applicable.

For example, to resolve `foo.node.local`, `bar.node.local` to `127.0.0.1` and `foo.node.remote`, `bar.node.remote` to `10.1.2.3`, configure the `HostAliases` values as follows:

```
apiVersion: v1
kind: Pod
metadata:
  name: hostaliases-konvoy-pod
spec:
  restartPolicy: Never
  hostAliases:
  - ip: "127.0.0.1"
    hostnames:
    - "foo.node.local"
    - "bar.node.local"
  - ip: "10.1.2.3"
    hostnames:
    - "foo.node.remote"
    - "bar.node.remote"
  containers:
  - name: cat-hosts
    image: busybox
    command:
    - cat
    args:
    - "/etc/hosts"
```

Required Domains

This section describes the required domains for NKP.

You must have access to the following domains through the customer networking rules so that Kommander can download all required images:

- http://docker.io
- http://gcr.io
- http://k8s.gcr.io

- mcr.microsoft.com
- nvcr.io
- http://quay.io
- http://us.gcr.io
- registry.k8s.io

> **Note:** In an air-gapped installation, these domains do not need to be accessible.

Load Balancing

In a Kubernetes cluster, depending on the flow of traffic direction, there are two kinds of load balancing:

- Internal load balancing for the traffic within a Kubernetes cluster
- External load balancing for the traffic coming from outside the cluster

Load Balancing for Internal Traffic

Load balancing within a Kubernetes cluster is accessed through a `ClusterIP` service type. `ClusterIP` presents a single IP address to the client, and load balances the traffic going to this IP to the backend servers. The actual load balancing happens using `iptables` or IPVS configuration. The `kube-proxy` Kubernetes component programs these. The `iptables` mode of operation uses DNAT (see https://www.linuxtopia.org/Linux_Firewall_iptables/x4013.html) rules to distribute direct traffic to real servers, whereas IPVS (see https://en.wikipedia.org/wiki/IP_Virtual_Server) leverages in-kernel transport-layer load-balancing. Read a comparison between these two methods (see https://www.projectcalico.org/comparing-kube-proxy-modes-iptables-or-ipvs/). By default, `kube-proxy` runs in `iptables` mode. The kube-proxy configuration can be altered by updating the `kube-proxy` configmap in the `kube-system` namespace.

Load Balancing for External Traffic

External traffic destined for the Kubernetes service requires a service of type `LoadBalancer`, through which external clients connect to your internal service. Under the hood, it uses a load balancer provided by the underlying infrastructure to direct the traffic.

> **Note:** In NKP environments, the external load balancer must be configured without TLS termination.

In cloud deployments, the load balancer is provided by the cloud provider. For example, in AWS, the service controller communicates with the AWS API to provision an ELB service that targets the cluster nodes.

For an on-premises Pre-provisioned deployment, NKP ships with MetalLB (see https://metallb.universe.tf/concepts/), which provides load-balancing services. The environments that use MetalLB are pre-provisioned, as well as vSphere infrastructures. For more information on how to configure MetalLB for these environments, see the following:

- Pre-provisioned Configure MetalLB on page 753
- Configure MetalLB for a vSphere infrastructure

Custom Load Balancer for External Traffic

If you want to use a non-NKP load balancer for external traffic, see External Load Balancer on page 1056.

Ingress

Kubernetes Ingress resources expose HTTP and HTTPS routes from outside the cluster to services within the cluster. In Kommander, the Traefik Ingress controller is installed by default and provides access to the NKP UI.

An Ingress performs the following:

- Gives Services externally-reachable URLs
- Load balances traffic
- Terminates SSL/TLS sessions
- Offers name-based virtual hosting

An Ingress controller fulfills the Ingress with a load balancer.

A cluster can have multiple Ingress controllers. Nutanix recommends adding your own Ingress controllers for your applications. The Traefik Ingress controller that Kommander installs for access to the NKP UI can be replaced later if a different solution is a better fit. Using your own Ingress controller in parallel for your own business requirements ensures that you are not limited by any future changes in Kommander.

Traefik v2.4

Traefik is a modern HTTP reverse proxy and load balancer that deploys microservices with ease. Kommander currently installs Traefik v2.4 by default on every cluster. Traefik creates a service of type `LoadBalancer`. In the cloud, the cloud provider creates the appropriate load balancer. In an on-premises deployment, by default, it uses MetalLB.

Traefik listens to the Kubernetes API and automatically generates and updates the routes without any further configuration or intervention so that the Services selected by the Ingress resources are connected to the outside world. Further, Traefik supports a rich set of functionality such as Name-based routing, Path-based routing, Traffic splitting, etc.

Major features highlighted in the Traefik documentation:

- Continuously updates its configuration (No restarts!)
- Supports multiple load balancing algorithms
- Provides HTTPS to your microservices
- Circuit breakers, retry
- A clean web UI
- Websocket, HTTP/2, GRPC ready
- Provides metrics (Rest, Prometheus, Datadog, StatsD, InfluxDB)
- Keeps access logs (JSON, CLF)
- Exposes a Rest API
- Packaged as a single binary file (made with go) and available as a docker image

Configuring Ingress for Load Balancing

Learn how to configure Ingress settings for load balancing (layer-7).

About this task

Ingress is the name used to describe an API object that manages external access to the services in a cluster. Typically, an Ingress exposes HTTP and HTTPS routes from outside the cluster to services running within the cluster.

The object is called an Ingress because it acts as a gateway for inbound traffic. The Ingress receives inbound requests and routes them according to the rules you defined for the Ingress resource as part of your cluster configuration.

Expose an application running on your cluster by configuring an Ingress for load balancing (layer-7).

Before you begin

You must:

- Have access to a Linux, macOS, or Windows computer with a supported operating system version.
- Have a properly deployed and running cluster.

To expose a pod using an Ingress (L7)

Procedure

1. Deploy two web application Pods on your Kubernetes cluster by running the following command.

```
kubectl run --restart=Never --image hashicorp/http-echo --labels app=http-echo-1 --
port 80 http-echo-1 -- -listen=:80 --text="Hello from http-echo-1"
kubectl run --restart=Never --image hashicorp/http-echo --labels app=http-echo-2 --
port 80 http-echo-2 -- -listen=:80 --text="Hello from http-echo-2"
```

2. Expose the Pods with a service type of ClusterIP by running the following commands.

```
kubectl expose pod http-echo-1 --port 80 --target-port 80 --name "http-echo-1"
kubectl expose pod http-echo-2 --port 80 --target-port 80 --name "http-echo-2"
```

3. Create the Ingress to expose the application to the outside world by running the following command.

```
cat <<EOF | kubectl create -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  annotations:
    kubernetes.io/ingress.class: kommander-traefik
    traefik.ingress.kubernetes.io/router.tls: "true"
  generation: 7
  name: echo
  namespace: default
spec:
  rules:
  - http:
      paths:
      - backend:
          service:
            name: http-echo-1
            port:
              number: 80
        path: /echo1
        pathType: Prefix
  - http:
      paths:
      - backend:
          service:
```

```
            name: http-echo-2
            port:
              number: 80
        path: /echo2
        pathType: Prefix
EOF
```

The configuration settings in this example illustrate:

- setting the `kind` to `Ingress`.
- setting the `service.name` to be exposed as each `backend`.

4. Run the following command to get the URL of the load balancer created on AWS for the Traefik service.

```
kubectl get svc kommander-traefik -n kommander
```

This command displays the internal and external IP addresses for the exposed service. (Note that IP addresses and host names are for illustrative purposes. Always use the information from your own cluster)

```
NAME                 TYPE           CLUSTER-IP    EXTERNAL-IP
                                     PORT(S)                                        AGE
kommander-traefik    LoadBalancer   10.0.24.215
 abf2e5bda6ca811e982140acb7ee21b7-37522315.us-west-2.elb.amazonaws.com   80:31169/
TCP,443:32297/TCP,8080:31923/TCP   4h22m
```

5. Validate that you can access the web application Pods by running the following commands: (Note that IP addresses and host names are for illustrative purposes. Always use the information from your own cluster)

```
curl -k https://abf2e5bda6ca811e982140acb7ee21b7-37522315.us-
west-2.elb.amazonaws.com/echo1
curl -k https://abf2e5bda6ca811e982140acb7ee21b7-37522315.us-
west-2.elb.amazonaws.com/echo2
```

Istio as a Microservice

Istio helps you manage cloud-based deployments by providing an open-source service mesh to connect, secure, control, and observe microservices.

This topic describes how to expose an application running on the NKP cluster using the LoadBalancer (layer-4) service type.

Deploying Istio Using NKP

About this task

Follow these steps to prepare NKP to run Istio:

Procedure

1. Review the list of available applications to obtain the current APP ID and version for Istio and its dependencies (see Platform Applications Dependencies on page 404). You need this information to executerun the following commands.

2. Install Istio's dependencies with an `AppDeployment` resource.

   Replace the `<APPID>` and `<APPID-version>` variables with the information of the application.

   The required value for the `--app` flag consists of the APP ID and version in the `APPID-version` format.

```
nkp create appdeployment <APPID> --app <APPID-version> --workspace ${WORKSPACE_NAME}
```

3. Install Istio.

   Replace `<APPID-version>` with the version of Istio you want to deploy.

   ```
   nkp create appdeployment istio --app <APPID-version> --workspace ${WORKSPACE_NAME}
   ```

   Note:

   - Create the resource in the workspace you just created, which instructs Kommander to deploy the `AppDeployment` to the `KommanderClusters` in the same workspace.
   - Observe that the `nkp create` command must be run with the `WORKSPACE_NAME` instead of the `WORKSPACE_NAMESPACE` flag.

Downloading the Istio Command Line Utility

About this task

Follow these steps to download and run Istio:

Procedure

1. Pull a copy of the corresponding Istio command line to your system.

   Replace `<your_istio_version_here>` with the Istio version you want to deploy.

   ```
   curl -L https://istio.io/downloadIstio | ISTIO_VERSION=<your_istio_version_here> sh -
   ```

2. Change to the Istio directory and set your PATH environment variable by running the following commands.

   ```
   cd istio*
   export PATH=$PWD/bin:$PATH
   ```

3. Run the following `istioctl` command and view the subsequent output.

   ```
   istioctl version
   ```

   ```
   client version: <your_istio_version_here>
   control plane version: <your_istio_version_here>
   data plane version: <your_istio_version_here> (1 proxies)
   ```

Deploying a Sample Application on Istio

About this task

The Istio `bookinfo` sample application is composed of four separate microservices that demonstrate various Istio features.

Procedure

1. Deploy the sample `bookinfo` application on the Kubernetes cluster by running the following commands.

   Important: Ensure your `nkp` configuration references the cluster where you deployed Istio by setting the `KUBECONFIG` environment variable, or using the `--kubeconfig` flag, in accordance with Kubernetes conventions (see https://kubernetes.io/docs/tasks/access-application-cluster/configure-access-multiple-clusters/).

   ```
   kubectl apply -f <(istioctl kube-inject -f samples/bookinfo/platform/kube/
   bookinfo.yaml)
   ```

```
kubectl apply -f samples/bookinfo/networking/bookinfo-gateway.yaml
```

2. Get the URL of the load balancer created on AWS for this service by running the following command.

   ```
   kubectl get svc istio-ingressgateway -n istio-system
   ```

   The command displays output similar to the following:

   ```
   NAME                   TYPE           CLUSTER-IP    EXTERNAL-IP
                                            PORT(S)

              AGE
   istio-ingressgateway   LoadBalancer   10.0.29.241
    a682d13086ccf11e982140acb7ee21b7-2083182676.us-west-2.elb.amazonaws.com
     15020:30380/TCP,80:31380/TCP,443:31390/TCP,31400:31400/TCP,15029:30756/
   TCP,15030:31420/TCP,15031:31948/TCP,15032:32061/TCP,15443:31232/TCP   110s
   ```

3. Open a browser and navigate to the external IP address for the load balancer to access the application.

   For example, the external IP address in the sample output is `a682d13086ccf11e982140acb7ee21b7-2083182676.us-west-2.elb.amazonaws.com`, enabling you to access the application using the following URL: `http://a682d13086ccf11e982140acb7ee21b7-2083182676.us-west-2.elb.amazonaws.com/productpage`

4. To understand the different Istio features, follow the steps in the Istio BookInfo Application.

   For more information on Istio, see https://istio.io/latest/docs/.

GPUs

In NKP), the nodes with NVIDIA GPUs are configured with nvidia-gpu-operator (see https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/index.html) and NVIDIA drivers to support the container runtime.

For more information, see Nutanix GPU Passthrough.

Creating a GPU Image

You must create a GPU-compatible OS image to add a GPU or a GPU node pool to a Nutanix cluster.

About this task

Create a worker node that is able to use a NVIDIA GPU by creating a machine image that has the NVIDIA drivers enabled.

To create a GPU image:

Procedure

Run the following command.

```
nkp create image nutanix  --gpu --gpu-name=${GPU_NAME} --cluster=
${NUTANIX_CLUSTER_NAME} --endpoint=${NUTANIX_PC_ENDPOINT} --subnet=${NUTANIX_SUBNET}
 ubuntu-22.04
```

> **Note:** The subnet provided must have external connectivity to configure the machine properly. If you are unable to create a machine in a subnet with external connectivity, use a bastion with the following flags.

```
nkp create image nutanix --gpu --gpu-name=${GPU_NAME} --cluster=${NUTANIX_CLUSTER_NAME}
 --endpoint=${NUTANIX_PC_ENDPOINT} --subnet=${NUTANIX_SUBNET} \
    --bastion-host=${BASTION_IP} --bastion-username=${BASTION_SSH_USERNAME} --bastion-
private-key-file=path/to/private_key.pem \
    ubuntu-22.04
```

Running the preceding commands results in the following output.

```
--> nutanix.kib_image: <Name of the GPU OS image created by NKP CLI>
```

Adding GPU to a Nutanix Cluster

You can deploy GPU-enabled Kubernetes clusters on Prism Central.

Before you begin

- Ensure that an NVIDIA GPU is available on a Nutanix cluster.
- Ensure that you have the appropriate permissions to create Kubernetes Clusters.
- Ensure to have a GPU-compatible OS image. See Creating a GPU Image on page 637.

To add a GPU to a Nutanix cluster:

Procedure

1. Create the cluster by running the following dry-run command.

```
nkp create cluster nutanix -c $CLUSTER_NAME
  --control-plane-prism-element-cluster $NUTANIX_PRISM_ELEMENT_CLUSTER_NAME
  --worker-prism-element-cluster $NUTANIX_PRISM_ELEMENT_CLUSTER_NAME
  --vm-image <Name of the GPU OS image created by NKP CLI>
  --endpoint=https://$PC_NUTANIX_ENDPOINT:$NUTANIX_PORT
  --control-plane-endpoint-ip=$CONTROL_PLANE_ENDPOINT_IP
  --control-plane-subnets=$NUTANIX_SUBNET_NAME
  --csi-storage-container=$NUTANIX_STORAGE_CONTAINER_NAME
  --worker-subnets=$NUTANIX_SUBNET_NAME
  --kubernetes-service-load-balancer-ip-range=$START_IP-$END_IP
  --dry-run -o yaml >> gpu.yaml
```

2. Modify the generated `gpu.yaml` file to include a node pool with GPUs.

```
workers:
      machineDeployments:
      <other node pools>
      - class: default-worker
        metadata:
          annotations:
            cluster.x-k8s.io/cluster-api-autoscaler-node-group-max-size: "1"
            cluster.x-k8s.io/cluster-api-autoscaler-node-group-min-size: "1"
        name: gpu-pool
        variables:
          overrides:
          - name: workerConfig
            value:
              nutanix:
                machineDetails:
                  bootType: uefi
                  cluster:
                    name: <Nutanix Prism Element Cluster Name with the GPU>
                    type: name
                  image:
                    name: <Name of the GPU OS image created by NKP CLI> # from the
 "Creating a GPU Image" topic
                    type: name
                  memorySize: 32Gi
                  subnets:
                  - name: <Nutanix Subnet>
                    type: name
```

```
                gpus:
                - type: name
                  name: <GPU_NAME>
                systemDiskSize: 80Gi
                vcpuSockets: 8
                vcpusPerSocket: 1
```

3. To apply the `gpu.yaml` file, run the following command.

```
kubectl apply -f gpu.yaml
```

Adding GPU Node Pool to a Nutanix Cluster

You can also add a GPU node pool to existing Kubernetes clusters on Prism Central.

Before you begin

- Ensure that an NVIDIA GPU is available on a Nutanix cluster.
- Ensure that you have the appropriate permissions to create Kubernetes Clusters.
- Ensure to have a GPU-compatible OS image. See Creating a GPU Image on page 637.

To add a GPU node pool to a Nutanix cluster:

Procedure

1. Create a new GPU node pool by running the following dry-run command.

```
$ nkp create nodepool nutanix -c $CLUSTER_NAME --vm-image
 $NAME_OF_GPU_OS_IMAGE_CREATED_BY_NKP_CLI --dry-run -o yaml --subnets
 $NUTANIX_SUBNET_NAME  --prism-element-cluster $NUTANIX_PRISM_ELEMENT_CLUSTER_NAME
 $NODEPOOL_NAME  >> gpu.yaml
```

2. Modify the generated `gpu.yaml` file to include a node pool with GPUs.

```
workers:
      machineDeployments:
      <other node pools>
      - class: default-worker
        metadata:
          annotations:
            cluster.x-k8s.io/cluster-api-autoscaler-node-group-max-size: "1"
            cluster.x-k8s.io/cluster-api-autoscaler-node-group-min-size: "1"
        name: gpu-pool
        variables:
          overrides:
          - name: workerConfig
            value:
              nutanix:
                machineDetails:
                  bootType: uefi
                  cluster:
                    name: <Nutanix Prism Element Cluster Name with the GPU>
                    type: name
                  image:
                    name: <Name of the GPU OS image created by NKP CLI> # from the
 "Creating a GPU Image" topic
                    type: name
                  memorySize: 32Gi
                  subnets:
                  - name: <Nutanix Subnet>
```

```
    type: name
  gpus:
  - type: name
    name: <GPU_NAME>
  systemDiskSize: 80Gi
  vcpuSockets: 8
  vcpusPerSocket: 1
```

3. To add the new GPU node pool to the NKP cluster , run the following command.

```
kubectl apply -f gpu.yaml
```

Configuring GPU for Kommander Clusters

Configure GPU for Kommander Clusters.

Before you begin

- Ensure nodes provide an NVIDIA GPU.
- If you are using a public cloud service such as AWS (see Use KIB with AWS on page 1087), create an AMI with KIB using the instructions on the KIB for GPU (see Using KIB with GPU on page 1102).
- If you are deploying in a pre-provisioned environment, ensure that you have created the appropriate secret for your GPU nodepoolan (see Creating Pre-provisioned GPU Node Pools on page 790) and have uploaded the appropriate artifacts to each node. For information on the pre-provisioned prerequisites air-gapped, see the GPU-only steps in Pre-provisioned Prerequisite Configuration on page 744.

  Warning: Specific instructions must be followed for enabling `nvidia-gpu-operator` depending on if you want to deploy the app on a Management cluster or a Attached or a Managed cluster.

Procedure

Select one of the following options:

» For instructions on enabling the NVIDIA platform application on a Management cluster, follow the instructions in the NVIDIA Platform Application Management Cluster section. See Enabling the NVIDIA Platform Application on a Management Cluster on page 640.

» For instructions on enabling the NVIDIA platform application on attached or managed clusters, follow the instructions in the NVIDIA Platform Application Attached or Managed Cluster section. See Enabling the NVIDIA Platform Application on Attached or Managed Clusters on page 642.

After `nvidia-gpu-operator` has been enabled depending on the cluster type, proceed to the **Select the correct Toolkit version for your NVIDIA GPU Operator** on each of those pages.

Enabling the NVIDIA Platform Application on a Management Cluster

About this task

If you intend to run applications that make use of GPUs on your cluster, you should install the NVIDIA GPU operator. To enable NVIDIA GPU support when installing Kommander on a management cluster, perform the following steps:

Procedure

1. Create an installation configuration file.

```
nkp install kommander --init > install.yaml
```

2. Append the following to the apps section in the `install.yaml` file to enable Nvidia platform services.

```
apps:
  nvidia-gpu-operator:
   enabled: true
```

3. Install Kommander using the configuration file you created.

```
nkp install kommander --installer-config ./install.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

   In the previous command, the `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you set the context for installing Kommander on the right cluster. For alternatives and recommendations around setting your context, refer to

   Provide Context for Commands with a kubeconfig File. See Commands within a kubeconfig File on page 32.

4. Proceed to the Selecting the Correct Toolkit Version for Your NVIDIA GPU Operator on page 641 section.

> **Tip:** Sometimes, applications require a longer period to deploy, which causes the installation to time out. Add the `--wait-timeout <time to wait>` flag and specify a period (for example, `1h`) to allocate more time to the deployment of applications.

Selecting the Correct Toolkit Version for Your NVIDIA GPU Operator

About this task

The NVIDIA Container Toolkit allows users to run GPU-accelerated containers. The toolkit includes a container runtime library and utilities to automatically configure containers to leverage NVIDIA GPU and must be configured correctly according to your base operating system.

Kommander (Management Cluster) Customization

Procedure

1. Select the correct Toolkit version based on your OS:

   The NVIDIA Container Toolkit allows users to run GPU-accelerated containers. The toolkit includes a container runtime library and utilities to configure containers to leverage NVIDIA GPU automatically, which must be configured correctly according to your base operating system.

   - **RHEL 8.6** : If you're using RHEL 8.6 as the base operating system for your GPU enabled nodes, set the `toolkit.version` parameter in your Kommander Installer Configuration file or `kommander.yaml` to the following.

```
kind: Installation
apps:
  nvidia-gpu-operator:
   enabled: true
   values: |
     toolkit:
       version: v1.14.6-ubi8
```

- **Ubuntu 18.04 and 20.04**: If you're using Ubuntu 18.04 or 20.04 as the base operating system for your GPU enabled nodes, set the `toolkit.version` parameter in your Kommander Installer Configuration file or `kommander.yaml` to the following.

```
kind: Installation
apps:
  nvidia-gpu-operator:
   enabled: true
   values: |
     toolkit:
       version: v1.14.6-ubuntu20.04
```

2. Install Kommander, using the configuration file you created.

```
nkp install kommander --installer-config ./install.yaml
```

In the previous command, the `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you set the context for installing Kommander on the right cluster. For alternatives and recommendations around setting your context and to provide Context for Commands with a `kubeconfig` file, see Commands within a kubeconfig File on page 32

> **Tip:** Sometimes, applications require a longer period to deploy, which causes the installation to time out. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, `1h`) to allocate more time to the deployment of applications.

Enabling the NVIDIA Platform Application on Attached or Managed Clusters

About this task

If you intend to run applications that utilize GPUs on Attached or Managed clusters, you must enable the `nvidia-gpu-operator` platform application in the workspace.

> **Note:**
>
> - To use the UI to enable the application, see Platform Applications on page 400.
> - To use the CLI, see Deploying Platform Applications Using CLI on page 403.
> - If only a subset of attached or managed clusters in the workspace are utilizing GPUs. For more information on how to enable only the `nvidia-gpu-operator` on specific clusters, see Enabling or Disabling an Application Per Cluster on page 416.

Procedure

After you have enabled the `nvidia-gpu-operator` app in the workspace on the necessary clusters, proceed to the next section.

Selecting the Correct Toolkit Version for Your NVIDIA GPU Operator

About this task

The NVIDIA Container Toolkit allows users to run GPU-accelerated containers. The toolkit includes a container runtime library and utilities to configure containers to leverage NVIDIA GPU automatically and must be configured correctly according to your base operating system., which

Workspace (Attached and Managed clusters) Customization

> **Note:** For how to use the CLI to customize the platform application on a workspace, see AppDeployment Resources on page 410.

If specific attached/managed clusters in the workspace require different configurations, see Customizing an Application Per Cluster on page 416.

Procedure

1. Select the correct Toolkit version based on your OS and create a `ConfigMap` with these configuration override values:

   - **RHEL 8.6** : If you're using RHEL 8.6 as the base operating system for your GPU enabled nodes, set the toolkit.version parameter in your install.yaml to the following:

     ```
     cat <<EOF | kubectl apply -f -
     apiVersion: v1
     kind: ConfigMap
     metadata:
       namespace: ${WORKSPACE_NAMESPACE}
       name: nvidia-gpu-operator-overrides-attached
     data:
       values.yaml: |
         toolkit:
           version: v1.10.0-ubi8
     EOF
     ```

   - **Ubuntu 18.04 and 20.04**: If you're using Ubuntu 18.04 or 20.04 as the base operating system for your GPU enabled nodes, set the toolkit.version parameter in your install.yaml to the following:

     ```
     cat <<EOF | kubectl apply -f -
     apiVersion: v1
     kind: ConfigMap
     metadata:
       namespace: ${WORKSPACE_NAMESPACE}
       name: nvidia-gpu-operator-overrides-attached
     data:
       values.yaml: |
         toolkit:
           version: v1.11.0-ubuntu20.04
     EOF
     ```

2. Note the name of this `ConfigMap` (`nvidia-gpu-operator-overrides-attached`) and use it to set the necessary `nvidia-gpu-operator` `AppDeployment` spec fields depending on the scope of the override. Alternatively, you can also use the UI to pass in the configuration overrides for the app per workspace or per cluster.

Validating the Application

Procedure

Run the following command to validate that your application has started correctly.

```
kubectl get pods -A | grep nvidia
```

Output example:

```
nvidia-container-toolkit-daemonset-7h2l5   1/1  Running  0              150m
nvidia-container-toolkit-daemonset-mm65g   1/1  Running  0              150m
nvidia-container-toolkit-daemonset-mv7xj   1/1  Running  0              150m
nvidia-cuda-validator-pdlz8            0/1  Completed   0              150m
nvidia-cuda-validator-r7qc4            0/1  Completed   0              150m
nvidia-cuda-validator-xvtqm            0/1  Completed   0              150m
nvidia-dcgm-exporter-9r6rl             1/1  Running  1 (149m ago)   150m
nvidia-dcgm-exporter-hn6hn             1/1  Running  1 (149m ago)   150m
```

```
nvidia-dcgm-exporter-j7g7g                1/1   Running   0              150m
nvidia-dcgm-jpr57                         1/1   Running   0              150m
nvidia-dcgm-jwldh                         1/1   Running   0              150m
nvidia-dcgm-qg2vc                         1/1   Running   0              150m
nvidia-device-plugin-daemonset-2gv8h      1/1   Running   0              150m
nvidia-device-plugin-daemonset-tcmgk      1/1   Running   0              150m
nvidia-device-plugin-daemonset-vqj88      1/1   Running   0              150m
nvidia-device-plugin-validator-9xdqr      0/1   Completed   0              149m
nvidia-device-plugin-validator-jjhdr      0/1   Completed   0              149m
nvidia-device-plugin-validator-llxjk      0/1   Completed   0              149m
nvidia-operator-validator-9kzv4           1/1   Running   0              150m
nvidia-operator-validator-fvsr7           1/1   Running   0              150m
nvidia-operator-validator-qr9cj           1/1   Running   0              150m
```

If you are seeing errors, ensure that you set the container toolkit version appropriately based on your OS, as described in the previous section.

NVIDIA GPU Monitoring

Kommander uses the NVIDIA Data Center GPU Manager to export GPU metrics towards Prometheus..

For information on NVIDIA Data Center GPU Manager, see https://developer.nvidia.com/dcgm.

By default, Kommander has a Grafana dashboard called `NVIDIA DCGM Exporter Dashboard` to monitor GPU metrics. This GPU dashboard is shown in Kommander's Grafana UI.

Configuring MIG for NVIDIA

About this task

MIG stands for Multi-Instance-GPU. It is a mode of operation for future NVIDIA GPUs that allows the user to partition a GPU into a set of MIG devices. Each set appears to the software that is consuming them as a mini-GPU with a fixed partition of memory and a fixed partition of compute resources.

> **Note:** MIG is only available for the following NVIDIA devices: H100, A100, and A30.

Procedure

1. Set the MIG strategy according to your GPU topology.

   - `mig.strategy` should be set to mixed when MIG mode is not enabled on all GPUs on a node.
   - `mig.strategy` should be set to single when MIG mode is enabled on all GPUs on a node and they have the same MIG device types across all of them.

   For the Management Cluster, this can be set at install time by modifying the Kommander configuration file to add configuration for the `nvidia-gpu-operator` application:

   ```
   apiVersion: config.kommander.mesosphere.io/v1alpha1
   kind: Installation
   apps:
     nvidia-gpu-operator:
       values: |
         mig:
           strategy: single
   ...
   ```

   Or by modifying the `clusterPolicy` object for the GPU operator once it has already been installed.

2. Set the MIG profile for the GPU you are using. In our example, we are using the A30 GPU that supports the following MIG profiles.

   ```
   4 GPU instances @ 6GB each
   ```

```
2 GPU instances @ 12GB each
1 GPU instance @ 24GB
```

Set the mig profile by labeling the node ${NODE} with the profile as in the example below:

```
kubectl label nodes ${NODE} nvidia.com/mig.config=all-1g.6gb --overwrite
```

3. Check the node labels to see if the changes were applied to your MIG enabled GPU node

```
kubectl get no -o json | jq .items[0].metadata.labels
```

```
"nvidia.com/mig.config": "all-1g.6gb",
  "nvidia.com/mig.config.state": "success",
  "nvidia.com/mig.strategy": "single"
```

4. Deploy a sample workload.

```
apiVersion: v1
kind: Pod
metadata:
  name: cuda-vector-add
spec:
  restartPolicy: OnFailure
  containers:
      - name: cuda-vectoradd
      image: "nvidia/samples:vectoradd-cuda11.2.1"
      resources:
        limits:
          nvidia.com/gpu: 1

  nodeSelector:
    "nvidia.com/gpu.product": NVIDIA-A30-MIG-1g.6gb
```

If the workload successfully finishes, then your GPU has been properly MIG partitioned.

Troubleshooting NVIDIA GPU Operator on Kommander

About this task

In case you run into any errors with NVIDIA GPU Operator, here are a couple of GPU-enabled commands you can run to troubleshoot:

Procedure

1. Connect (using SSH or similar) to your GPU enabled nodes and run the `nvidia-smi` command. Your output should be similar to the following example:

```
[ec2-user@ip-10-0-0-241 ~]$ nvidia-smi
Thu Nov  3 22:52:59 2022
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 535.183.06    Driver Version: 535.183.06    CUDA Version: 12.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla T4            On   | 00000000:00:1E.0 Off |                    0 |
| N/A   54C    P8    11W /  70W |      0MiB / 15109MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
```

```
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
```

2. Another common issue is having a misconfigured toolkit version, resulting in NVIDIA pods in a bad state. For example.

```
nvidia-container-toolkit-daemonset-jrqt2                          1/1     Running
            0             29s
nvidia-dcgm-exporter-b4mww                                        0/1     Error
            1 (9s ago)    16s
nvidia-dcgm-pqsz8                                                 0/1
 CrashLoopBackOff        1 (13s ago)   27s
nvidia-device-plugin-daemonset-7fkzr                              0/1     Init:0/1
            0             14s
nvidia-operator-validator-zxn4w                                   0/1
 Init:CrashLoopBackOff   1 (7s ago)    11s
```

   To modify the toolkit version, run the following commands to modify the `AppDeployment` for the `nvidia gpu operator` application.

   a. Provide the name of a `ConfigMap` with the custom configuration in the `AppDeployment`.

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: nvidia-gpu-operator
  namespace: kommander
spec:
  appRef:
    kind: ClusterApp
    name: nvidia-gpu-operator-1.11.1
  configOverrides:
    name: nvidia-gpu-operator-overrides
EOF
```

   b. Create the `ConfigMap` with the name provided in the previous step, which provides the custom configuration on top of the default configuration in the config map, and set the version appropriately.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: kommander
  name: nvidia-gpu-operator-overrides
data:
  values.yaml: |
    toolkit:
      version: v1.10.0-centos7
EOF
```

3. If a node has an NVIDIA GPU installed and the `nvidia-gpu-operator` application is enabled on the cluster, but the node is still not accepting GPU workloads, it's possible that the nodes do not have a label that indicates there is an NVIDIA GPU present.

   By default, the GPU operator will attempt to configure nodes with the following labels present, which are usually applied by the node feature discovery component.

```
"feature.node.kubernetes.io/pci-10de.present": "true",
 "feature.node.kubernetes.io/pci-0302_10de.present": "true",
```

```
 "feature.node.kubernetes.io/pci-0300_10de.present": "true",
```

If these labels are not present on a node that you know contains an NVIDIA GPU, you can manually label the node using the following command:

```
kubectl label node ${NODE} feature.node.kubernetes.io/pci-0302_10de.present=true
```

Disabling NVIDIA GPU Operator Platform Application on Kommander

About this task

Context for the current task

Procedure

1. Delete all GPU workloads on the GPU nodes where the NVIDIA GPU Operator platform application is present.
2. Delete the existing NVIDIA GPU Operator AppDeployment using the following command:

   ```
   kubectl delete appdeployment -n kommander nvidia-gpu-operator
   ```

3. Wait for all NVIDIA related resources in the `Terminating` state to be cleaned up. You can check pod status with the following command.

   ```
   kubectl get pods -A | grep nvidia
   ```

   For information on how to delete node pools, see Deleting Pre-provisioned Node Pools on page 790.

GPU Toolkit Versions

The NVIDIA Container Toolkit allows users to run GPU-accelerated containers. The toolkit includes a container runtime library and utilities to automatically configure containers to leverage NVIDIA GPU and must be configured correctly according to your base operating system.

- **RHEL 8.6** : If you're using RHEL 8.6 as the base operating system for your GPU enabled nodes, set the `toolkit.version` parameter in your Kommander Installer Configuration file or `kommander.yaml` to the following.

  ```
  kind: Installation
  apps:
    nvidia-gpu-operator:
     enabled: true
     values: |
       toolkit:
         version: v1.14.6-ubi8
  ```

- **Ubuntu 18.04 and 20.04**: If you're using Ubuntu 18.04 or 20.04 as the base operating system for your GPU enabled nodes, set the `toolkit.version` parameter in your Kommander Installer Configuration file or `kommander.yaml` to the following.

  ```
  kind: Installation
  apps:
    nvidia-gpu-operator:
     enabled: true
     values: |
       toolkit:
         version: v1.14.6-ubuntu20.04
  ```

Enabling GPU After Installing NKP

About this task

If you want to enable your cluster to run GPU nodes after installing NKP, enable the correct Toolkit version for your operating system in the `nvidia-gpu-operator` AppDeployment.

> **Note:** If you have not installed the Kommander component of NKP yet, set the Toolkit version in the Kommander Installer Configuration file (see GPU Toolkit Versions on page 647)and skip this section.

Procedure

1. Create a `ConfigMap` with the necessary configuration overrides to set the correct Toolkit version. For example, if you're using Ubuntu 20.04 or RHEL 7.9 as the base operating system for your GPU-enabled nodes, set the `toolkit.version` parameter.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: kommander
  name: nvidia-gpu-operator-overrides
data:
  values.yaml: |
    toolkit:
      version: v1.10.0-ubuntu20
EOF
```

2. Update the `nvidia-gpu-operator` AppDeployment in the `kommander` namespace to reference the `ConfigMap` you created.

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: nvidia-gpu-operator
  namespace: kommander
spec:
  appRef:
    name: nvidia-gpu-operator-1.11.1
    kind: ClusterApp
  configOverrides:
    name: nvidia-gpu-operator-overrides
EOF
```

Monitoring and Alerts

Using NKP you can monitor the state of the cluster and the health and availability of the processes running on the cluster. By default, Kommander provides monitoring services using a pre-configured monitoring stack based on the Prometheus open-source project and its broader ecosystem.

The default NKP monitoring stack:

- Provides in-depth monitoring of Kubernetes components and platform services.
- Includes a default set of Grafana dashboards to visualize the status of the cluster and its platform services.
- Supports predefined critical error and warning alerts. These alerts notify immediately if there is a problem with cluster operations or availability.

By incorporating Prometheus, Kommander visualizes all the metrics that are exposed from your different nodes, Kubernetes objects, and platform service applications running in your cluster. The default monitoring stack also enables you to add metrics from any of your deployed applications, making those applications part of the overall Prometheus metrics stream.

Recommendations

Recommended settings for monitoring and collecting metrics for Kubernetes, platform services, and applications deployed on the cluster

Nutanix conducts routine performance testing of Kommander. The following table provides recommended settings, based on cluster size and increasing workloads, that maintain a healthy Prometheus monitoring deployment.

> **Note:** The resource settings reflect some settings but do not represent the exact structure to be used in the platform service configuration.

Table 55: Prometheus

Cluster Size	Number of Pods	Number of Services	Resource settings

10	1k	250	`resources:`
			`  limits:`
			`  cpu: 500m`
			`  memory: 2192Mi`
			`  requests:`
			`  cpu: 100m`
			`  memory: 500Mi`
			`storage: 35Gi`

25	1k	250	`resources:`
			`  limits:`
			`  cpu: 2`
			`  memory: 6Gi`
			`  requests:`
			`  cpu: 1`
			`  memory: 3Gi`
			`storage: 60Gi`

50	1.5k	500	`resources:`
			`  limits:`
			`  cpu: 7`
			`  memory: 28Gi`
			`  requests:`
			`  cpu: 2`
			`  memory: 8Gi`
			`storage: 100Gi`

100	3k	1k	`resources:`
			`  limits:`
			`  cpu: 12`
			`  memory: 50Gi`
			`  requests:`
			`  cpu: 10`
			`  memory: 48Gi`
			`storage: 100Gi`

200	10k	3k	`resources:`
			`  limits:`
			`  cpu: 20`
			`  memory: 80Gi`
			`  requests:`
			`  cpu: 15`
			`  memory: 50Gi`
			`storage: 100Gi`

300	15k	6k	`resources:`
			`  limits:`
			`  cpu: 35`
			`  memory: 150Gi`
			`  requests:`
			`  cpu: 25`
			`  memory: 120Gi`
			`storage: 100Gi`

Grafana Dashboards

With Grafana, you can query and view collected metrics in easy-to-read graphs. Kommander ships with a set of default dashboards, including:

- Kubernetes Components: API Server, Nodes, Pods, Kubelet, Scheduler, StatefulSets and Persistent Volumes
- Kubernetes USE method: Cluster and Nodes
- Calico
- etcd
- Prometheus

Find the complete list of default-enabled dashboards on GitHub. For more information, see https://github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack/templates/grafana/dashboards-1.14.

Disabling Default Dashboards

About this task

To deactivate all of the default dashboards, follow these steps to define an overrides ConfigMap:

Procedure

1. Create a file named `kube-prometheus-stack-overrides.yaml` and paste the following YAML code into it to create the overrides ConfigMap.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: kube-prometheus-stack-overrides
  namespace: <your-workspace-namespace>
data:
 values.yaml: |
   ---
   grafana:
     defaultDashboardsEnabled: false
```

2. Use the following command to apply the YAML file.

```
kubectl apply -f kube-prometheus-stack-overrides.yaml
```

3. Edit the `kube-prometheus-stack` AppDeployment to replace the `spec.configOverrides.name` value with `kube-prometheus-stack-overrides`. To deploy an application with a custom configuration as a guide, see Customizing an Application Per Cluster on page 416. When your editing is complete, the AppDeployment will resemble this code sample.

```
apiVersion: apps.kommander.d2iq.io/v1alpha2
kind: AppDeployment
metadata:
  name: kube-prometheus-stack
  namespace: <your-workspace-namespace>
spec:
  appRef:
    name: kube-prometheus-stack-34.9.3
    kind: ClusterApp
  configOverrides:
```

```
    name: kube-prometheus-stack-overrides
```

To access the Grafana UI, browse to the landing page and then search for the Grafana dashboard, for example, `https://<CLUSTER_URL>/nkp/grafana`.

Adding Custom Dashboards

About this task

In Kommander, you can define your own custom dashboards. You can use a few methods to import dashboards to Grafana. For more information, see https://github.com/grafana/helm-charts/tree/main/charts/grafana#import-dashboards.

Procedure

1. One method is to use ConfigMaps to import dashboards. Below are steps on how to create a ConfigMap with your dashboard definition.

   For more information, see https://github.com/grafana/helm-charts/tree/main/charts/grafana#sidecar-for-dashboards.

   For simplicity, this section assumes the desired dashboard definition is in `json` format.

   ```
   {
       "annotations": {
           "list": []
       },
       "description": "etcd sample Grafana dashboard with Prometheus",
       "editable": true,
       "gnetId": null,
       "hideControls": false,
       "id": 6,
       "links": [],
       "refresh": false,
       ...
   }
   ```

2. After creating your custom dashboard json, insert it into a ConfigMap and save it as `etcd-custom-dashboard.yaml`.

   ```
   apiVersion: v1
   kind: ConfigMap
   metadata:
     name: etcd-custom-dashboard
     labels:
       grafana_dashboard: "1"
   data:
     etcd.json: |
       {
           "annotations": {
               "list": []
           },
           "description": "etcd sample Grafana dashboard with Prometheus",
           "editable": true,
           "gnetId": null,
           "hideControls": false,
           "id": 6,
           "links": [],
           "refresh": false,
           ...
   ```

```
    }
```

Apply the ConfigMap, which automatically gets imported to Grafana using the Grafana dashboard sidecar (see https://github.com/grafana/helm-charts/tree/main/charts/grafana#sidecar-for-dashboards).

```
kubectl apply -f etcd-custom-dashboard.yaml
```

Cluster Metrics

The `kube-prometheus-stack`is deployed by default on the management cluster and attached clusters. This stack deploys the following Prometheus components to expose metrics from nodes, Kubernetes units, and running apps:

- prometheus-operator: orchestrates various components in the monitoring pipeline.
- prometheus: collects metrics, saves them in a time series database, and serves queries.
- alertmanager: handles alerts sent by client applications such as the Prometheus server.
- node-exporter: deployed on each node to collect the machine hardware and OS metrics.
- kube-state-metrics: simple service that listens to the Kubernetes API server and generates metrics about the state of the objects.
- grafana: monitors and visualizes metrics.
- service monitors: collects internal Kubernetes components.

> **Note:** NKP has a listener on the `metrics.k8s.io/v1beta1/nodes` resource, which updates your backend store when that value changes. We then poll that backend store every 5 seconds, so the metrics are updated in real time every 5 seconds without the need to refresh your view.

For a detailed description of the exposed metrics, see https://github.com/kubernetes/kube-state-metrics/tree/main/docs#exposed-metrics. The `service-monitors` collect internal Kubernetes components but can also be extended to monitor customer apps as explained in the section Monitor Applications, on this page below.

Alerts Using AlertManager

To keep your clusters and applications healthy and drive productivity forward, you need to stay informed of all events occurring in your cluster. NKP helps you to stay informed of these events by using the `alertmanager` of the `kube-prometheus-stack`.

Kommander is configured with pre-defined alerts to monitor four specific events. You receive alerts related to:

- State of your nodes
- System services managing the Kubernetes cluster
- Resource events from specific system services
- Prometheus expressions exceeding some pre-defined thresholds

Some examples of the alerts currently available are:

- CPUThrottlingHigh
- TargetDown
- KubeletNotReady
- KubeAPIDown

- CoreDNSDown
- KubeVersionMismatch

For a complete list with all the pre-defined alerts on GitHub, see https://github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack/templates/prometheus/rules-1.14.

Configuring Alert Rules

About this task

Use override `ConfigMaps` to configure alert rules.

You can enable or disable the default alert rules by providing the desired configuration in an overrides ConfigMap. For example, if you want to turn off the default `node` alert rules, follow these steps to define an overrides ConfigMap:

Before you begin

1. Determine the name of the workspace where you want to perform the actions. You can use the `nkp get workspaces` command to see the list of workspace names and their corresponding namespaces.
2. Set the `WORKSPACE_NAMESPACE` environment variable to the name of the workspace's namespace where the cluster is attached:

   ```
   export WORKSPACE_NAMESPACE=<workspace_namespace>
   ```

Procedure

1. Create a file named `kube-prometheus-stack-overrides.yaml` and paste the following YAML code into it to create the overrides ConfigMap.

   ```
   apiVersion: v1
   kind: ConfigMap
   metadata:
     name: kube-prometheus-stack-overrides
     namespace: ${WORKSPACE_NAMESPACE}
   data:
    values.yaml: |
      ---
      defaultRules:
        rules:
          node: false
   ```

2. Use the following command to apply the YAML file.

   ```
   kubectl apply -f kube-prometheus-stack-overrides.yaml
   ```

3. Edit the `kube-prometheus-stack` AppDeployment to replace the `spec.configOverrides.name` value with `kube-prometheus-stack-overrides`.

   ```
   nkp edit appdeployment -n ${WORKSPACE_NAMESPACE} kube-prometheus-stack
   ```

   After your editing is complete, the AppDeployment resembles this example:

   ```
   apiVersion: apps.kommander.d2iq.io/v1alpha2
   kind: AppDeployment
   metadata:
     name: kube-prometheus-stack
     namespace: ${WORKSPACE_NAMESPACE}
   spec:
     appRef:
       name: kube-prometheus-stack-34.9.3
   ```

```
    kind: ClusterApp
  configOverrides:
    name: kube-prometheus-stack-overrides
```

4. To disable all rules, create an overrides ConfigMap with this YAML code.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: kube-prometheus-stack-overrides
  namespace: ${WORKSPACE_NAMESPACE}
data:
 values.yaml: |
   ---
   defaultRules:
     create: false
```

5. Alert rules for the Velero platform service are turned off by default. You can enable them with the following overrides ConfigMap. They should be enabled only if the `velero` platform service is enabled. If platform services are disabled disable the alert rules to avoid alert misfires.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: kube-prometheus-stack-overrides
  namespace: ${WORKSPACE_NAMESPACE}
data:
  values.yaml: |
    ---
    mesosphereResources:
      rules:
        velero: true
```

6. To create a custom alert rule named `my-rule-name`, create the overrides ConfigMap with this YAML code.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: kube-prometheus-stack-overrides
  namespace: ${WORKSPACE_NAMESPACE}
data:
  values.yaml: |
    ---
    additionalPrometheusRulesMap:
      my-rule-name:
        groups:
        - name: my_group
          rules:
          - record: my_record
            expr: 100 * my_record
```

After you set up your alerts, you can manage each alert using the Prometheus web console to mute or unmute firing alerts, and perform other operations. For more information on configuring `alertmanager`, see https://prometheus.io/docs/alerting/latest/configuration/.

To access the Prometheus Alertmanager UI, browse to the landing page and then search for the Prometheus Alertmanager dashboard, for example `https://<CLUSTER_URL>/nkp/alertmanager`

Configuring Prometheus to Send Slack Alerts

About this task

To hook up the Prometheus `alertmanager` notification system, you need to overwrite the existing configuration.

Procedure

1. The following file, named `alertmanager.yaml`, configures `alertmanager` to use the Incoming Webhooks feature of Slack (`slack_api_url: https://hooks.slack.com/services/<HOOK_ID>`) to fire all the alerts to a specific channel `#MY-SLACK-CHANNEL-NAME`.

```
global:
  resolve_timeout: 5m
  slack_api_url: https://hooks.slack.com/services/<HOOK_ID>

route:
  group_by: ['alertname']
  group_wait: 2m
  group_interval: 5m
  repeat_interval: 1h

  # If an alert isn't caught by a route, send it to slack.
  receiver: slack_general
  routes:
    - match:
        alertname: Watchdog
      receiver: "null"

receivers:
  - name: "null"
  - name: slack_general
    slack_configs:
      - channel: '#MY-SLACK-CHANNEL-NAME'
        icon_url: https://avatars3.githubusercontent.com/u/3380462
        send_resolved: true
        color: '{{ if eq .Status "firing" }}danger{{ else }}good{{ end }}'
        title: '{{ template "slack.default.title" . }}'
        title_link: '{{ template "slack.default.titlelink" . }}'
        pretext: '{{ template "slack.default.pretext" . }}'
        text: '{{ template "slack.default.text" . }}'
        fallback: '{{ template "slack.default.fallback" . }}'
        icon_emoji: '{{ template "slack.default.iconemoji" . }}'

templates:
  - '*.tmpl'
```

2. The following file, named `notification.tmpl`, is a template that defines a pretty format for the fired notifications.

```
{{ define "__titlelink" }}
{{ .ExternalURL }}/#/alerts?receiver={{ .Receiver }}
{{ end }}

{{ define "__title" }}
[{{ .Status | toUpper }}{{ if eq .Status "firing" }}:{{ .Alerts.Firing | len }}
{{ end }}] {{ .GroupLabels.SortedPairs.Values | join " " }}
{{ end }}

{{ define "__text" }}
{{ range .Alerts }}
```

```
{{ range .Labels.SortedPairs }}*{{ .Name }}*: `{{ .Value }}`
{{ end }} {{ range .Annotations.SortedPairs }}*{{ .Name }}*: {{ .Value }}
{{ end }} *source*: {{ .GeneratorURL }}
{{ end }}
{{ end }}

{{ define "slack.default.title" }}{{ template "__title" . }}{{ end }}
{{ define "slack.default.username" }}{{ template "__alertmanager" . }}{{ end }}
{{ define "slack.default.fallback" }}{{ template "slack.default.title" . }} |
 {{ template "slack.default.titlelink" . }}{{ end }}
{{ define "slack.default.pretext" }}{{ end }}
{{ define "slack.default.titlelink" }}{{ template "__titlelink" . }}{{ end }}
{{ define "slack.default.iconemoji" }}{{ end }}
{{ define "slack.default.iconurl" }}{{ end }}
{{ define "slack.default.text" }}{{ template "__text" . }}{{ end }}
```

3. Finally, apply these changes to `alertmanager` as follows. Set `${WORKSPACE_NAMESPACE}` to the workspace namespace that `kube-prometheus-stack` is deployed in.

```
kubectl create secret generic -n ${WORKSPACE_NAMESPACE} \
  alertmanager-kube-prometheus-stack-alertmanager \
  --from-file=alertmanager.yaml \
  --from-file=notification.tmpl \
  --dry-run=client --save-config -o yaml | kubectl apply -f -
```

Configuring Prometheus to Send Email Alerts

About this task

To configure the Prometheus `alertmanager` notification system to send an email for alerts, you need to overwrite the existing configuration. The steps below configure Alertmanager to send all configured alerts to a Gmail account. For example, test@gmail.com.

Procedure

1. Create a file named `alertmanager.yaml` with the following contents.

```
global:
  resolve_timeout: 5m
inhibit_rules: []
receivers:
- name: "null"
- name: test_gmail
  email_configs:
  - to: test@gmail.com
    from: test@gmail.com
    auth_username: test@gmail.com
    auth_password: password
    send_resolved: true
    require_tls: true
    smarthost: smtp.gmail.com:587
route:
  receiver: test_gmail
  group_by:
  - namespace
  group_interval: 5m
  group_wait: 30s
  repeat_interval: 12h
  routes:
  - matchers:
    - alertname =~ "InfoInhibitor|Watchdog"
```

```
    receiver: "null"
templates:
- /etc/alertmanager/config/*.tmpl
```

2. Apply these changes to `alertmanager` as follows. Set `${WORKSPACE_NAMESPACE}` to the workspace namespace that `kube-prometheus-stack` is deployed in (typically the `kommander` namespace).

```
kubectl create secret generic -n ${WORKSPACE_NAMESPACE} \


  alertmanager-kube-prometheus-stack-alertmanager \
  --from-file=alertmanager.yaml \
  --dry-run=client --save-config -o yaml | kubectl apply -f -
```

3. Allow some time for the configuration to take affect. You can then use the following command to verify that the configuration took effect.

```
kubectl exec -it alertmanager-kube-prometheus-stack-alertmanager-0 -n kommander --
 cat /etc/alertmanager/config_out/alertmanager.env.yaml
```

   For more information on configuring email alerting, see https://prometheus.io/docs/alerting/latest/configuration/.

Centralized Monitoring

Monitor clusters, created with Kommander, on any attached cluster

Kommander provides centralized monitoring, in a multi-cluster environment, using the monitoring stack running on any attached clusters. Centralized monitoring is provided by default in every managed or attached cluster.

Managed or attached clusters are distinguished by a monitoring ID. The monitoring ID corresponds to the kube-system namespace UID of the cluster. To find a cluster's monitoring ID, you can go to the Clusters tab on the NKP UI (in the relevant workspace), or go to the **Clusters** page in the **Global** workspace:

```
https://<CLUSTER_URL>/nkp/kommander/dashboard/clusters
```

Select the `View Details` link on the attached cluster card, and then select the **Configuration** tab, and find the monitoring ID under **Monitoring ID (clusterId)**.

You might also search or filter by monitoring IDs on the Clusters page, linked above.

You can also run this kubectl command, **using the correct cluster's context or kubeconfig**, to look up the cluster's kube-system namespace UID to determine which cluster the metrics and alerts correspond to:

```
kubectl get namespace kube-system -o jsonpath='{.metadata.uid}'
```

Adding Custom Dashboards

About this task

You can also define custom dashboards for centralized monitoring on Kommander. There are a few methods to import dashboards to Grafana. For more information, see https://github.com/mesosphere/charts/tree/master/stable/grafana#import-dashboards.

Procedure

1. For simplicity, assume the desired dashboard definition is in `json` format.

```
{
    "annotations":
    ...
    # Complete json file here
    ...
```

```
    "title": "Some Dashboard",
    "uid": "abcd1234",
    "version": 1
}
```

2. After creating your custom dashboard json, insert it into a ConfigMap and save it as `some-dashboard.yaml`.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: some-dashboard
  labels:
    grafana_dashboard_kommander: "1"
data:
  some_dashboard.json: |
    {
      "annotations":
      ...
      # Complete json file here
      ...
      "title": "Some Dashboard",
      "uid": "abcd1234",
      "version": 1
    }
```

3. Apply the ConfigMap, which will automatically get imported to Grafana through the Grafana dashboard sidecar.

```
kubectl apply -f some-dashboard.yaml
```

Centralized Metrics

Managed and attached clusters, collects and presents metrics from all attached clusters remotely using Thanos. You can visualize these metrics in Grafana using a set of provided dashboards.

The Thanos Query (see https://thanos.io/v0.5/components/query/#query) component is installed on attached and managed clusters. Thanos Query queries the Prometheus instances on the attached clusters, using a Thanos sidecar running alongside each Prometheus container. Grafana is configured with Thanos Query as its data source and comes with pre-installed dashboards for a global view of all attached clusters. The `Thanos Query` dashboard is also installed, by default, to monitor the Thanos Query component.

> **Note:** Cluster metrics are read remotely from Kommander; they are not backed up. If an attached cluster goes down, Kommander no longer collects or presents its metrics, including past data.

You can access the centralized Grafana UI at:

```
https://<CLUSTER_URL>/nkp/kommander/monitoring/grafana
```

> **Note:** This is a separate Grafana instance from the one installed on all attached clusters. It is dedicated specifically to components related to centralized monitoring.

Optionally, if you want to access the Thanos Query UI (essentially the Prometheus UI), the UI is accessible at:

```
https://<CLUSTER_URL>/nkp/kommander/monitoring/query/
```

You can also check that the attached cluster's Thanos sidecars are successfully added to Thanos Query by going to:

```
https://<CLUSTER_URL>/nkp/kommander/monitoring/query/stores
```

The preferred method to view the metrics for a specific cluster is to go directly to that cluster's Grafana UI.

Centralized Alerts

A centralized view of alerts from attached clusters, is provided using an alert dashboard called Karma (see https://github.com/prymitive/karma). Karma aggregates all alerts from the Alertmanagers running in the attached clusters, allowing you to visualize these alerts on one page. Using the Karma dashboard, you can get an overview of each alert and filter by alert type, cluster, and more.

> **Note:** Silencing alerts using the Karma UI is currently not supported.

You can access the Karma dashboard UI at:

```
https://<CLUSTER_URL>/nkp/kommander/monitoring/karma
```

> **Note:** When there are no attached clusters, the Karma UI displays an error message `Get https://placeholder.invalid/api/v2/status: dial tcp: lookup placeholder.invalid on 10.0.0.10:53: no such host`. This is expected, and the error disappears when clusters are connected.

Federating Prometheus Alerting Rules

About this task

You can define additional Prometheus alerting rules (see https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/) on attached and managed clusters and federate them to all of the attached clusters by following these instructions. To use these instructions you must install the kubefedctl CLI.

Procedure

1. Enable the PrometheusRule type for federation.

   ```
   kubefedctl enable PrometheusRules --kubefed-namespace kommander
   ```

2. Modify the existing alertmanager configuration.

   ```
   kubectl edit PrometheusRules/kube-prometheus-stack-alertmanager.rules -n kommander
   ```

3. Append a sample rule.

   ```
   - alert: MyFederatedAlert
     annotations:
       message: A custom alert that will always fire.
     expr: vector(1)
     labels:
       severity: warning
   ```

4. Federate the rules you just modified.

   ```
   kubefedctl federate PrometheusRules kube-prometheus-stack-alertmanager.rules --
   kubefed-namespace kommander -n kommander
   ```

5. Ensure that the cluster selection (`status.clusters`) is appropriately set for your desired federation strategy and check the propagation status.

   ```
   kubectl get federatedprometheusrules kube-prometheus-stack-alertmanager.rules -n
    kommander -oyaml
   ```

Centralized Cost Monitoring

Monitoring costs of all attached clusters with Kubecost

Kubecost (see https://www.kubecost.com/), running on Kommander, provides centralized cost monitoring for all attached clusters. This feature, installed by default in the management cluster, provides a centralized view of Kubernetes resources used on all attached clusters.

> **Note:** By default, up to 15 days of cost metrics are retained, with no backup to an external store.

Adding a Kubecost License Key to Your Clusters

About this task

By default, NKP is deployed with the free version of Kubecost, which provides cost monitoring for individual clusters and metric retention for up to 15 days. Licensed plans from Kubecost with additional features are also available. For more information on Kubecost Pricing, https://www.kubecost.com/pricing/.

> **Warning:** The license key must be applied to the Centralized Kubecost application running on the Management cluster.

> **Note: Considerations when Adding a License Key.** Until you add a license key, Kubecost caches the context of the cluster you first navigate from. This means that if you navigate to the Kubecost UI through the dashboard in the Application Dashboards tab of any cluster, you will not be able to access the centralized Kubecost UI until you clear your browser cookies/cache.

The following instructions provide information on how you can add a Kubecost license key to your clusters if you have purchased one.

Procedure

1. Obtain license key from Kubecost.
2. Access the `centralized-kubecost` dashboard with the following link:

   ```
   https://<CLUSTER_URL>/nkp/kommander/kubecost/frontend/
   ```

3. From the dashboard, select the **Settings** icon, then select “Add License Key”. Alternatively, the dashboard can be accessed through the following link: https://%3CCLUSTER_URL%3E/nkp/kommander/kubecost/frontend/settings
4. Input your license key, then select “Update.”
5. After the license key has been added, the licensed features become available in the Kubecost UI.

   > **Note:** The Kubecost Enterprise plan offers a free trial ows you to preview Ultimate features for 30 days. To access this, select “Start Free Trial” in the **Settings** pane.

Centralized Costs

Using Thanos, the management cluster collects cost metrics remotely from each attached cluster. Costs from the last day and the last seven days are displayed for each cluster, workspace, and project in the respective NKP UI pages. Further cost analysis and details can be found in the centralized Kubecost UI running on Kommander at:

```
https://<CLUSTER_URL>/nkp/kommander/kubecost/frontend/detail.html#&agg=cluster
```

For more information on cost allocation metrics and how to navigate this view in the Kubecost UI, see https://docs.kubecost.com/using-kubecost/getting-started/cost-allocation.

To identify the clusters in Kubecost, use the cluster's monitoring ID. The monitoring ID corresponds to the kube-system namespace UID of the cluster. To find the cluster's monitoring ID, select the **Clusters** tab on the NKP UI in the relevant workspace, or go to the **Clusters** page in the **Global** workspace.

```
https://<CLUSTER_URL>/nkp/kommander/dashboard/clusters
```

Select **View Details** on the attached cluster card. Select the **Configuration** tab, and find the monitoring ID under **Monitoring ID (clusterId).**

You can also search or filter by monitoring ID on the **Clusters** page.

To look up a cluster's kube-system namespace UID directly using the CLI, run the following kubectl command **using the cluster's context or kubeconfig**.

```
kubectl get namespace kube-system -o jsonpath='{.metadata.uid}'
```

Kubecost

Kubecost integrates directly with the Kubernetes API and cloud billing APIs to give you real-time visibility into your Kubernetes spending and cost allocation. By monitoring your Kubernetes spending across clusters, you can avoid overspend caused by uncaught bugs or oversights. With a cost monitoring solution in place, you can realize the full potential and cost of these resources and avoid over-provisioning resources.

To customize pricing and of clusters-of-cluster costs for AWS, you must apply these settings using the Kubecost UI running on each attached cluster. You can access the attached cluster's Kubecost Settings page at:

```
https://<MANAGED_CLUSTER_URL>/nkp/kubecost/frontend/settings.html
```

> **Warning:** Make sure you access the cluster's Kubecost UI linked above, not the centralized Kubecost UI running on the Kommander management cluster.

AWS

To configure a data feed for the AWS Spot instances and a more accurate AWS Spot pricing, follow the steps at https://docs.kubecost.com/using-kubecost/getting-started/spot-checklist#implementing-spot-nodes-in-your-cluster.

To allocate out-of-cluster costs for AWS, see https://docs.kubecost.com/install-and-configure/advanced-configuration/cloud-integration/aws-cloud-integrations/aws-out-of-cluster.

Grafana Dashboards

A set of Grafana dashboards providing visualization of cost metrics is provided in the centralized Grafana UI:

```
https://<CLUSTER_URL>/nkp/kommander/monitoring/grafana
```

These dashboards give a global view of accumulated costs from all attached clusters. From the navigation in Grafana, you can find these dashboards by selecting those tagged with `cost`, `metrics`, and `utilization`.

Application Monitoring using Prometheus

Before attempting to monitor your own applications, you should be familiar with the Prometheus conventions for exposing metrics. In general, there are two key recommendations:

- You should expose metrics using an HTTP endpoint named `/metrics`.
- The metrics you expose must be in a format that Prometheus can consume.

By following these conventions, you ensure that your application metrics can be consumed by Prometheus itself or by any Prometheus-compatible tool that can retrieve metrics, using the Prometheus client endpoint.

The `kube-prometheus-stack` for Kubernetes provides easy monitoring definitions for Kubernetes services and deployment and management of Prometheus instances. It provides a Kubernetes resource called `ServiceMonitor`.

By default, the `kube-prometheus-stack` provides the following service monitors to collect internal Kubernetes components:

- kube-apiserver
- kube-scheduler
- kube-controller-manager
- etcd
- kube-dns/coredns
- kube-proxy

The operator is in charge of iterating over all of these `ServiceMonitor` objects and collecting the metrics from these defined components.

The following example illustrates how to retrieve application metrics. In this example, there are:

- Three instances of a simple app named `my-app`
- The sample app listens and exposes metrics on port 8080
- The app is assumed to already be running

To prepare for monitoring of the sample app, create a service that selects the pods that have `my-app` as the value defined for their app label setting.

The service object also specifies the port on which the metrics are exposed. The `ServiceMonitor` has a label selector to select services and their underlying endpoint objects. For example:

```
kind: Service
apiVersion: v1
metadata:
  name: my-app
  namespace: my-namespace
  labels:
    app: my-app
spec:
  selector:
    app: my-app
  ports:
  - name: metrics
    port: 8080
```

This service object is discovered by a `ServiceMonitor`, which defines the selector to match the labels with those defined in the service. The app label must have the value `my-app`.

In this example, in order for `kube-prometheus-stack` to discover this `ServiceMonitor`, add a specific label `prometheus.kommander.d2iq.io/select: "true"` in the `yaml`:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: my-app-service-monitor
  namespace: my-namespace
  labels:
```

```
    prometheus.kommander.d2iq.io/select: "true"
spec:
  selector:
    matchLabels:
      app: my-app
  endpoints:
  - port: metrics
```

In this example, you can modify the Prometheus settings to have the operator collect metrics from the service monitor by appending the following configuration to the overrides ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: kube-prometheus-stack-overrides
  namespace: ${WORKSPACE_NAMESPACE}
data:
  values.yaml: |
    ---
    prometheus:
      additionalServiceMonitors:
        - name: my-app-service-monitor
          selector:
            matchLabels:
              app: my-app
          namespaceSelector:
            matchNames:
              - my-namespace
          endpoints:
            - port: metrics
              interval: 30s
```

Note:

Official documentation about using a `ServiceMonitor` to monitor an app with the Prometheus-operator on Kubernetes can be found on the GitHub repository.

Setting Storage Capacity for Prometheus

About this task

Follow the steps on this page to set a specific storage capacity for Prometheus.

Procedure

When defining the requirements of a cluster, you can specify the capacity and resource requirements of Prometheus by modifying the settings in the overrides ConfigMap definition, as shown below.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: kube-prometheus-stack-overrides
  namespace: ${WORKSPACE_NAMESPACE}
data:
  values.yaml: |
    ---
    prometheus:
      prometheusSpec:
        resources:
          limits:
            cpu: "4"
            memory: "8Gi"
```

```
        requests:
          cpu: "2"
          memory: "6Gi"
    storageSpec:
      volumeClaimTemplate:
        spec:
          resources:
            requests:
              storage: "100Gi"
```

Storage for Applications

NKP ships with a Rook Ceph cluster that is used as the primary blob storage for various NKP components in the logging stack and backups.

The pages in this section provide an overview of the Rook Ceph application in NKP, including information about its components, resource requirements, storage configuration information, and dashboard.

- Rook Ceph in NKP on page 665
- Bring Your Own Storage (BYOS) to NKP Clusters on page 670

Rook Ceph in NKP

NKP ships with a Rook Ceph cluster, that is used as the primary blob storage for various NKP components in the logging stack, backups, and NKP Insights.

The pages in this section provide an overview of the Rook Ceph application in NKP, including information about its components, resource requirements, storage configuration information, and dashboard.

> **Note:** The Ceph instance installed by NKP is intended only for use by Nutanix Kubernetes Platform Insights and the logging stack and `velero` platform applications. For more information, see the Nutanix Kubernetes Platform Insights Guide on page 1217.
>
> If you have an instance of Ceph that is managed outside of the NKP life cycle, see Bring Your Own Storage (BYOS) to NKP Clusters on page 670.

Rook Ceph: Prerequisites
Important information you should know prior to using Rook Ceph in NKP

> **Note:** The Ceph instance installed by NKP is intended only for use by the logging stack and `velero` platform applications.
>
> If you have an instance of Ceph that is managed outside of the NKP life cycle, see Bring Your Own Storage (BYOS) to NKP Clusters on page 670
>
> If you do not plan on using any of the logging stack components such as `grafana-loki` or `velero` for backups, then you do not need Rook Ceph for your installation and you can disable it by adding the following to your installer config file:

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
apps:
  ...
  ...
  grafana-loki:
    enabled: false
  ...
  rook-ceph:
    enabled: false
  rook-ceph-cluster:
```

```
    enabled: false
  ...
  velero:
    enabled: false
...
```

You must enable `rook-ceph` and `rook-ceph-cluster` if any of the following is true:

- If `grafana-loki` is enabled.
- If `velero` is enabled. If you applied config overrides for `velero` to use ,storage that is external to your cluster (see Usage of Velero with AWS S3 Buckets on page 572), then you do not need Ceph to be installed.

For more information on Rook Ceph, see https://rook.io/docs/rook/v1.10/Getting-Started/intro/.

Rook Ceph: Configuration

This page contains information about configuring Rook Ceph in your NKP Environment.

> **Note:** The Ceph instance installed by NKP is intended only for use by the logging stack, NKP Insights and `velero` platform applications.
>
> If you have an instance of Ceph that is managed outside of the NKP life cycle, see Bring Your Own Storage (BYOS) to NKP Clusters on page 670.
>
> If you intend to use Ceph in conjunction with NKP Insights, see NKP Insights Bring Your Own Storage (BYOS) to Insights on page 1227.
>
> Components of a Rook Ceph Cluster

Ceph supports creating clusters in different modes as listed in CephCluster CRD Rook Ceph Documentation (see https://rook.io/docs/rook/v1.10/CRDs/Cluster/ceph-cluster-crd/). NKP, specifically, is shipped with a PVC Cluster, as documented in PVC Storage Cluster Rook Ceph Documentation (see https://rook.io/docs/rook/v1.10/CRDs/Cluster/pvc-cluster/#pvc-storage-only-for-monitors). It is recommended that the PVC mode be used to keep the deployment and upgrades simple and agnostic to technicalities with node draining.

Ceph cannot be your CSI Provisioner when installing in PVC mode as Ceph relies on an existing CSI provisioner to bind the PVCs created by it. It is possible to use Ceph as your CSI provisioner, but that is outside the scope of this document. If you have an instance of Ceph that acts as the CSI Provisoner, then it is possible to reuse it for your NKP Storage needs.

When you create `AppDeployments` for `rook-ceph` and `rook-ceph-cluster` platform applications, it results in the deployment of various components as listed in the following diagram

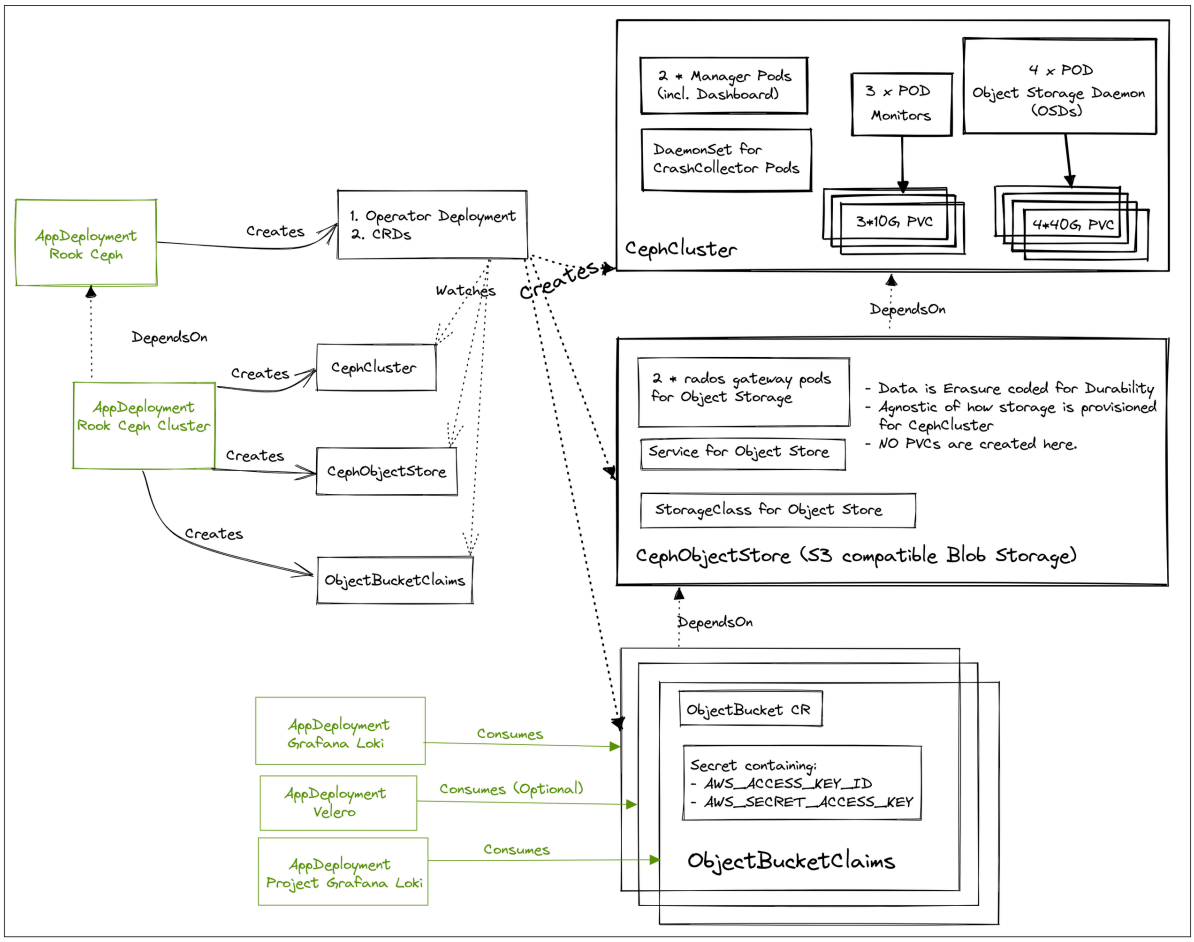


Figure 23: Rook Ceph Cluster Components

Items highlighted in green are user-facing and configurable.

For an in-depth explanation of the inner workings of the components outlined in the above diagram, see https://rook.io/docs/rook/v1.10/Getting-Started/storage-architecture/ and https://docs.ceph.com/en/quincy/architecture/.

For additional details about the data model, see https://github.com/rook/rook/blob/release-1.10/design/ceph/data-model.md.

Rook Ceph: Resource Requirements
The following is a non-exhaustive list of the resource requirements for long running components of Ceph:

Table 56: Table

Type	Resources	**Total**
CPUs	100m x # of mgr instances (default 2) 250m x # of mon instances (default 3) 250m x # of osd instances (default 4) 100m x # of crashcollector instances (Daemonset that is., # of nodes) 250m x # of rados gateway replicas (default 2)	~2000m CPU
Memory	512Mi x # of mgr instances (default 2) 512Gi x # of mon instances (default 3) 1Gi x # of osd instances (default 4) 500Mi x # of rados gateway replicas (default 2)	~8Gi Memory
Disk	4 x 40Gi PVCs with `Block` mode for ObjectStorageDaemons (see https://rook.io/docs/rook/v1.10/CRDs/Cluster/ceph-cluster-crd/#storage-selection-settings). 3 x 10Gi PVCs with `Block` or `FileSystem` mode for Mons (see https://rook.io/docs/rook/v1.10/CRDs/Cluster/ceph-cluster-crd/#mon-settings).	190Gi`PersistentVolumeClaim`s created by Ceph with `volumeMode: Block` .Your default `StorageClass` should support creation of `PersistentVolume`s that satisfy the `PersistentVolumeClaim`s

Your default `StorageClass` should support creation of `PersistentVolume`s created by Ceph with that satisfy the `volumeMode: Block`.

Rook Ceph: Storage Configuration
Ceph is highly configurable and can support Replication or Erasure Coding to ensure data durability. NKP is configured to use Erasure Coding for maximum efficiency.

For more information on data durability, see https://en.wikipedia.org/wiki/Durability_(database_systems).

The default configuration creates a `CephCluster` that creates 4 x `PersistentVolumeClaim`s of `40G` each, resulting in `160G` of raw storage. Erasure coding ensures durability with `k=3` data bits and `m=1` parity bits. This gives a storage efficiency of `75%` (refer to the primer above for calculation), which means `120G` of disk space is available for consumption by services like `grafana-loki`, `project-grafana-loki`, and `velero`.

It is possible to override the replication strategy for the logging stack (`grafana-loki`) and `velero` backups. Refer to the default configmap for the `CephObjectStore` at https://github.com/mesosphere/kommander-applications/blob/v2.4.0/services/rook-ceph-cluster/1.10.3/defaults/cm.yaml#L126-L175 and override the

replication strategy according to your needs by referring to CephObjectStore CRD documentation (see https://www.rook.io/docs/rook/v1.11/CRDs/Object-Storage/ceph-object-store-crd/).

For more information on configuring storage in Rook Ceph, refer to the following pages:

- For general information on how to configure Object Storage Daemons (OSDs), see https://www.rook.io/docs/rook/v1.11/Storage-Configuration/Advanced/ceph-osd-mgmt/.
- For information on how to set up auto-expansion of OSDs, https://www.rook.io/docs/rook/v1.11/Storage-Configuration/Advanced/ceph-configuration/?h=expa#auto-expansion-of-osds.

Replication and **Erasure Coding** are the two primary methods for storing data in a durable fashion in any distributed system.

Replication

- For a replication factor of N, data has N copies (including the original copy)
- Smallest possible replication factor is 2 (usually this means two storage nodes).
  - With replication factor of 2, data has 2 copies and this tolerates loss of one copy of data.
- Storage efficiency : `(1/N) * 100`percentage. For example,
  - If `N=2`, then efficiency is `50%`.
  - If `N=3`, then efficiency is `33%` so on.
- Fault Tolerance : `N-1` nodes can be lost without loss of data. For example,
  - If `N=2`, then atmost 1 node can be lost without data loss.
  - If `N=3`, then atmost 2 nodes can be lost without data loss and so on.

Erasure Coding

- Slices an object into `k` data fragments and computes `m` parity fragments. The erasure coding scheme guarantees that data can be recreated using any `k` fragments out of `k+m` fragments.
- The `k + m = n` fragments are spread across (`>=n`) Storage Nodes to offer durability.
- Since `k` out of `n` fragments (parity or data fragments) are needed for the recreation of data, at most `m` fragments can be lost without loss of data.
- The smallest possible count is `k = 2`, `m = 1` that is., `n = k + m = 3`. This works only if there are at least `n = 3` storage nodes.
- Storage efficiency: `k/(k+m) * 100` percentage. For example,
  - If `k=2, m=1`, then efficiency is `67%`
  - If `k=3, m=1`, then efficiency is `75%` and so on.
- Fault Tolerance: `m` nodes can be lost without loss of data. For example:
  - If `k=3, m=1` then atmost 1 out of 4 nodes can be lost without data loss.
  - If `k=4, m=2` then atmost 2 out of 6 nodes can be lost without data loss, and so on.

Bring Your Own Storage (BYOS) to NKP Clusters

You can use Ceph as the CSI Provisioner in some environments. For environments where Ceph was installed before installing NKP, you can reuse your existing Ceph installation to satisfy the storage requirements of NKP Applications.

> **Note:** This guide assumes you have a Ceph cluster that is not managed by NKP.
>
> For information on how to configure the Ceph instance installed by NKP for use by NKP platform applications, see Rook Ceph: Configuration on page 666.

Disabling NKP-managed Ceph

Process for disabling rook-ceph on non-Nutanix infrastructure.

About this task

Disable `rook-ceph` in your installer config since the default config of NKP has already installed a Ceph Cluster.

> **Note:** This task is supported through NKP Release 2.12.x.

• NKP's default config already installs installer config to prevent NKP from installing a Ceph cluster:

Procedure

Run the following command.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
apps:
  rook-ceph:
    enabled: false
  rook-ceph-cluster:
    enabled: false
  ...
...
```

The NKP instances of `velero` and `grafana-loki` rely on the storage provided by Ceph. Before installing the Kommander component of NKP, be sure to configure appropriate Ceph resources for their usage as detailed in the next section.

Creating NKP-compatible Ceph Resources

About this task

This section walks you through the creation of `CephObjectStore` and then a set of `ObjectBucketClaim`s, which can be consumed by either `velero` and `grafana-loki`.

Typically, Ceph is installed in the `rook-ceph` namespace, which is the default namespace if you have followed the Quickstart - Rook Ceph Documentation guide. For more information, see https://www.rook.io/docs/rook/v1.10/Getting-Started/quickstart/#create-a-ceph-cluster.

> **Note:** This guide assumes your Ceph instance is installed in the `rook-ceph` namespace. In subsequent steps, configure the variable #CEPH_NAMESPACE# as it applies to your environment.

Procedure

1. Create. `CephObjectStore`
   There are two ways to install Ceph:

   » Using Helm Charts (For more information on Helm Chart, see https://www.rook.io/docs/rook/v1.10/Helm-Charts/operator-chart/#release).

   This section is relevant if you have installed Ceph using `helm install` or some other managed Helm resource mechanism.

   If you have applied any configuration overrides to your Rook Ceph operator, ensure it was deployed with `currentNamespaceOnly` set to false (It is the default value, so unless you have applied any overrides, it will be `false` by default). This ensures that the Ceph Operator in the `rook-ceph` namespace is able to monitor and manage resources in other namespaces such as `kommander`.

   Note:

   - 1. Ensure the following configuration for `rook-ceph` is completed. See https://www.rook.io/docs/rook/v1.10/Helm-Charts/operator-chart/#configuration.

     ```
     # This is the default value, so need to overwrite if you are just
      using the defaults as-is
     currentNamespaceOnly: false
     ```

     2. You must enable the following configuration overrides for the `rook-ceph-cluster`. See https://www.rook.io/docs/rook/v1.10/Helm-Charts/ceph-cluster-chart/#ceph-object-stores.

     ```
     cephObjectStores:
       - name: nkp-object-store
         # see https://github.com/rook/rook/blob/master/Documentation/
     CRDs/Object-Storage/ceph-object-store-crd.md#object-store-settings
      for available configuration
         spec:
           metadataPool:
             # The failure domain: osd/host/(region or zone if available)
      - technically also any type in the crush map
             failureDomain: osd
             # Must use replicated pool ONLY. Erasure coding is not
      supported.
             replicated:
               size: 3
           dataPool:
             # The failure domain: osd/host/(region or zone if available)
      - technically also any type in the crush map
             failureDomain: osd
             # Data pool can use either replication OR erasure coding.
      Consider the following example scenarios:
             # Erasure Coding is used here with 3 data chunks and 1 parity
      chunks which assumes 4 OSDs exist.
             # Configure this according to your CephCluster specification.
             erasureCoded:
               dataChunks: 3
               codingChunks: 1
           preservePoolsOnDelete: false
           gateway:
             port: 80
             instances: 2
             priorityClassName: system-cluster-critical
             resources:
               limits:
     ```

```
                cpu: "750m"
                memory: "1Gi"
              requests:
                cpu: "250m"
                memory: "500Mi"
        healthCheck:
          bucket:
            interval: 60s
      storageClass:
        enabled: true
        name: nkp-object-store
        reclaimPolicy: Delete
```

» By directly applying Kubernetes manifests (For more information, see Bring Your Own Storage (BYOS) to NKP Clusters on page 670:

Note:

- **1.** Set a variable to refer to the namespace the `AppDeployments` are created in.

  Note: This is the `kommander` namespace on the management cluster or `Workspace` namespace on all other clusters.

  ```
  export CEPH_NAMESPACE=rook-ceph
  export NAMESPACE=kommander
  ```

  2. Create `CephObjectStore` in the same namespace as the `CephCluster`:

  ```
  cat <<EOF | kubectl apply -f -
  apiVersion: ceph.rook.io/v1
  kind: CephObjectStore
  metadata:
    name: nkp-object-store
    namespace: ${CEPH_NAMESPACE}
  spec:
    metadataPool:
      # The failure domain: osd/host/(region or zone if available) -
   technically, any type in the crush map
      failureDomain: osd
      # Must use replicated pool ONLY. Erasure coding is not supported.
      replicated:
        size: 3
    dataPool:
      # The failure domain: osd/host/(region or zone if available) -
   technically, any type in the crush map
      failureDomain: osd
      # Data pool can use either replication OR erasure coding.
   Consider the following example scenarios:
      # Erasure Coding is used here with 3 data chunks and 1 parity
   chunks which assumes 4 OSDs exist.
      # Configure this according to your CephCluster specification.
      erasureCoded:
        dataChunks: 3
        codingChunks: 1
    preservePoolsOnDelete: false
    gateway:
      port: 80
      instances: 2
      priorityClassName: system-cluster-critical
      resources:
        limits:
```

```
            cpu: "750m"
            memory: "1Gi"
          requests:
            cpu: "250m"
            memory: "500Mi"
    healthCheck:
      bucket:
        interval: 60s
  EOF
```

3. Wait for the `CephObjectStore` to be `Connected`:

   ```
   $ kubectl get cephobjectstore -A
   NAMESPACE   NAME               PHASE
   rook-ceph   nkp-object-store   Progressing
   ...
   ...
   rook-ceph   nkp-object-store   Connected
   ```

4. Create a `StorageClass` to consume the object storage:

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: storage.k8s.io/v1
   kind: StorageClass
   metadata:
     name: nkp-object-store
   parameters:
     objectStoreName: nkp-object-store
     objectStoreNamespace: ${CEPH_NAMESPACE}
   provisioner: ${CEPH_NAMESPACE}.ceph.rook.io/bucket
   reclaimPolicy: Delete
   volumeBindingMode: Immediate
   EOF
   ```

2. Create `ObjectBucketClaim`s.

   After connecting the Object Store, create the `ObjectBucketClaim` in the same namespace as `velero` and `grafana-loki`.

   This results in the creation of `ObjectBucket` , that then creates `Secret`s that are consumed by `velero` and `grafana-loki`.

   a. For `grafana-loki`.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: objectbucket.io/v1alpha1
   kind: ObjectBucketClaim
   metadata:
     name: nkp-loki
     namespace: ${NAMESPACE}
   spec:
     additionalConfig:
       maxSize: 80G
     bucketName: nkp-loki
     storageClassName: nkp-object-store
   EOF
   ```

   b. For `velero`.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: objectbucket.io/v1alpha1
   kind: ObjectBucketClaim
   metadata:
   ```

```
  name: nkp-velero
  namespace: ${NAMESPACE}
spec:
  additionalConfig:
    maxSize: 10G
  bucketName: nkp-velero
  storageClassName: nkp-object-store
EOF
```

3. Wait for the `ObjectBucket`s to be `Bound` by executing the following command:

   ```
   kubectl get objectbucketclaim -n${NAMESPACE} -ocustom-
   columns='NAME:.metadata.name,PHASE:.status.phase'
   ```

   which should display something similar to:

   ```
   NAME         PHASE
   nkp-loki     Bound
   nkp-velero   Bound
   ```

Configuring Loki to Use S3 Compatible Storage

About this task

If you want to use your own storage in NKP that is S3 compatible, create a secret that contains your AWS secret credentials.

Procedure

Run the following command.

```
apiVersion: v1
data:
  AWS_ACCESS_KEY_ID: base64EncodedValue
  AWS_SECRET_ACCESS_KEY: base64EncodedValue
kind: Secret
metadata:
  name: nkp-loki #If you want to configure a custom name here, also use it in the step
 below
  namespace: kommander
```

Overriding velero and grafana-loki Configuration

About this task

After all the buckets are in the `Bound` state, NKP applications are now ready to be installed with the following configuration overrides populated in the installer config.

Procedure

Run the following command.

```
cat <<EOF | kubectl apply -f -
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
apps:
  grafana-loki:
    enabled: true
    values: |
      loki:
        structuredConfig:
          storage_config:
            aws:
```

```
              s3: "http://rook-ceph-rgw-nkp-object-store.${CEPH_NAMESPACE}.svc:80/nkp-
loki"
      ingester:
        extraEnvFrom:
          - secretRef:
              name: nkp-loki # Optional: This is the default value
      querier:
        extraEnvFrom:
          - secretRef:
              name: nkp-loki # Optional: This is the default value
      queryFrontend:
        extraEnvFrom:
          - secretRef:
              name: nkp-loki # Optional: This is the default value
      compactor:
        extraEnvFrom:
          - secretRef:
              name: nkp-loki # Optional: This is the default value
      ruler:
        extraEnvFrom:
          - secretRef:
              name: nkp-loki # Optional: This is the default value
      distributor:
        extraEnvFrom:
          - secretRef:
              name: nkp-loki # Optional: This is the default value
  velero:
    enabled: true
    values: |
      configuration:
        backupStorageLocation:
        - bucket: nkp-velero
          provider: "aws"
          config:
            region: nkp-object-store
            s3Url: http://rook-ceph-rgw-nkp-object-store.${CEPH_NAMESPACE}.svc:80/
      credentials:
        # This secret is owned by the ObjectBucketClaim. A ConfigMap and a Secret with
 the same name as a bucket are created.
        extraSecretRef: nkp-velero
EOF
```

This installer config can be merged with your installer config with any other relevant configuration before installing NKP.

Overriding project-grafana-loki Configuration

About this task

When installing project level grafana loki, its configuration needs to be overridden similarly to workspace level grafana loki, so that the project logs can be persisted in Ceph storage.

Procedure

1. The following overrides need to be applied to `project-grafana-loki`:

   ```
   loki:
     structuredConfig:
       storage_config:
         aws:
   ```

```
        s3: "http://rook-ceph-rgw-nkp-object-store.${CEPH_NAMESPACE}.svc:80/nkp-loki"
```

These overrides can be applied from the UI directly while substituting the `${CEPH_NAMESPACE}` appropriately.

2. Set `NAMESPACE` to project namespace and `CEPH_NAMESPACE` to Ceph install namespace.

   Note: Run these commands if you are using CLI.

```
export CEPH_NAMESPACE=rook-ceph
export NAMESPACE=my-project
```

3. Create a `ConfigMap` to apply the configuration overrides.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
data:
  values.yaml: |
    loki:
      structuredConfig:
        storage_config:
          aws:
            s3: "http://rook-ceph-rgw-nkp-object-store.${CEPH_NAMESPACE}.svc:80/proj-
loki-${NAMESPACE}"
kind: ConfigMap
metadata:
  name: project-grafana-loki-ceph
  namespace: ${NAMESPACE}
EOF
```

4. Create the `AppDeployment` with a reference to the above `ConfigMap`.

   Note: The `clusterSelector` can be adjusted according to your needs.

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: project-grafana-loki
  namespace: ${NAMESPACE}
spec:
  appRef:
    kind: ClusterApp
    name: project-grafana-loki-0.48.6
  clusterSelector: {}
  configOverrides:
    name: project-grafana-loki-ceph
EOF
```

The project level Grafana Loki creates an `ObjectBucketClaim` and assumes that the Ceph operator is montitoring the project namespace, so there is no need to create `ObjectBucketClaim` manually.

6

CUSTOM INSTALLATION AND INFRASTRUCTURE TOOLS

The Konvoy component of Nutanix Kubernetes Platform (NKP) can be customized for different environments and infrastructures depending on your network technology choices. If you have already installed using Basic Installations by Infrastructure on page 52 instructions, you might find helpful tools in this section. However, if you have not already installed NKP with the Basic installation instructions, find your infrastructure and begin the process of custom NKP installation in this area.

See the following sections for details on custom installation options and infrastructure-specific details.

Universal Configurations for all Infrastructure Providers

Several areas of Nutanix Kubernetes Platform (NKP) configuration are shared amongst all infrastructure providers. Some of the universal configurations related to environment variables, flags for cluster creation, local registries, and more are described in this section.

For additional Konvoy customizations, see Additional Konvoy Configurations on page 1065.

Container Runtime Engine (CRE)

Supported CRE options for NKP.

A Container Runtime Engine (CRE) installed is required to install Nutanix Kubernetes Platform (NKP). NKP supports both Podman and Docker as container engines for cluster creation. If you choose Podman, it works with Linux on NKP.

- A Container engine or runtime installed is required to install NKP and bootstrap. You can select one of these supported CREs:
  - **Docker** container engine version 27.4.0 installed for Linux or MacOS. On MacOS, Docker runs on a virtual machine that, Docker runs in a virtual machine which needs to be configured with at least 8 GB of memory. For more information, see https://docs.docker.com/get-docker/.
  - **Podman** Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.

Configuring an HTTP or HTTPS Proxy

HTTP and HTTPS proxy settings are needed when creating an NKP Cluster.

When creating a Nutanix Kubernetes Platform (NKP) cluster in environments that use an HTTP or HTTPS proxy, you must provide proxy details. The proxy values are strings that list a set of proxy servers, URLs, or wildcard addresses specific to your environment.

When creating a NKP cluster in a proxied environment, you need to specify proxy settings for the following:

- Bootstrap cluster
- CAPI components

- NKP Kommander component

When you create a NKP cluster using the `--self-managed` flag, the bootstrap cluster and Cluster API (CAPI) components are created for you automatically, and use the HTTP and HTTPS proxy settings you specify in the `nkp create cluster <provider>...` command.

You can also create the bootstrap cluster and CAPI components manually, using the appropriate commands: `nkp create bootstrap` and `nkp create capi-components` , respectively, combined with the command line flags to include your HTTP or HTTPS proxy information.

You can also specify HTTP or HTTPS proxy information in an override file when using Konvoy Image Builder (KIB). For more information, see Use Override Files with Konvoy Image Builder on page 1118.

Without these values provided as part of the relevant `nkp create` command, NKP cannot create the requisite parts of your new cluster correctly. This is true of both management and managed clusters alike.

> **Note:** For NKP installation, create the bootstrap cluster from within the same network where the new cluster will run. Using a bootstrap cluster on a laptop with different proxy settings, for example, or residing in a different network, can cause problems.

Bootstrap Cluster HTTP Proxy Settings
When creating a bootstrap cluster, you must locate the device used to create the bootstrap in the same proxied environment in which the workload cluster will run. Nutanix does not recommend creating a bootstrap cluster from outside a proxied environment.

The Application Programming Interface (API) server doesn't exist yet in the bootstrap environment before you install Nutanix Kubernetes Platform (NKP) because the API server is created during cluster creation. To create a bootstrap server in a proxied environment, you need to include the following flags:

- `--http-proxy <<http proxy list>>`
- `--https-proxy <<https proxy list>>`
- `--no-proxy <<no proxy list>>`

The following is an example of the `nkp create bootstrap` command's syntax, with the HTTP proxy settings included.

```
nkp create bootstrap --http-proxy <<http proxy list>> --https-proxy <<https proxy
 list>> --no-proxy <<no proxy list>>
```

Creating a Bootstrap Cluster with HTTP Proxy Settings
Steps used to create a bootstrap cluster with HTTP Proxy settings.

Before you begin

If an HTTP proxy is required, locate the values to use for the `http_proxy`, `https_proxy`, and `no_proxy` flags. They will be built into the bootstrap cluster during cluster creation.

About this task

The flags can include a mix of IP addresses and domain names. Note that the delimiter between each proxy value within a flag is a comma (,) with no space character following it.

Procedure

Create a bootstrap cluster and any other flags you need using the command `nkp create bootstrap --kubeconfig $HOME/.kube/config --http-proxy <string> \ --https-proxy <string> \ --no-proxy <string>`.

Example output shows values for the proxy settings:

```
nkp create bootstrap \
```

```
   --http-proxy 10.0.0.15:3128 \
   --https-proxy  10.0.0.15:3128 \
   --no-proxy
 127.0.0.1,192.168.0.0/16,10.0.0.0/16,10.96.0.0/12,169.254.169.254,169.254.0.0/24,localhost,kubern
prometheus-server.kommander,logging-operator-logging-
fluentd.kommander.svc.cluster.local,elb.amazonaws.com
```

Creating CAPI Components with HTTP or HTTPS Proxy Settings
Creating CAPI components for an NKP cluster from the command line requires HTTP or HTTPS proxy information if your environment is proxied.

About this task

If you created a cluster without using the `--self-managed flag`, the cluster will not have any Cluster API (CAPI) controllers or the cert-manager component. This means that the cluster will be managed from the context of the cluster from which it was created, such as the bootstrap cluster. However, you can transform the cluster to a self-managed cluster by performing the commands `nkp create capi-components --kubeconfig=<newcluster>` and `nkp move --to-kubeconfig=<newcluster>`. This combination of actions is sometimes called a pivot.

When creating the CAPI components for a proxied environment using the Nutanix Kubernetes Platform (NKP) command line interface, you must include the following flags :

- `--http-proxy <<http proxy list>>`
- `--https-proxy <<https proxy list>>`
- `--no-proxy <<no proxy list>>`

The following is an example `nkp create capi-components` command's syntax with the HTTP proxy settings included:

> **Tip:**
>
> ```
> nkp create capi-components --http-proxy <<http proxy list>> --https-proxy
>  <<https proxy list>> --no-proxy <<no proxy list>>
> ```

Procedure

1. If an HTTP proxy is required, locate the values to use for the `http_proxy`, `https_proxy`, and `no_proxy` flags. They will be built into the CAPI components during their creation.
2. Create CAPI components using this command syntax and any other flags you might need.

   ```
   nkp create capi-components --kubeconfig $HOME/.kube/config \
     --http-proxy <string> \
     --https-proxy <string> \
     --no-proxy <string>
   ```

   This code sample shows the command with example values for the proxy settings:

   ```
   nkp create capi-components \
      --http-proxy 10.0.0.15:3128 \
      --https-proxy  10.0.0.15:3128 \
      --no-proxy
    127.0.0.1,192.168.0.0/16,10.0.0.0/16,10.96.0.0/12,169.254.169.254,169.254.0.0/24,localhost,kube
   prometheus-server.kommander,logging-operator-logging-
   fluentd.kommander.svc.cluster.local,elb.amazonaws.com
   ```

Clusters with HTTP or HTTPS Proxy

During cluster creation, you might need to configure the control plane and worker nodes to use an HTTP proxy. This can occur during installation of the Konvoy component of Nutanix Kubernetes Platform (NKP)or when creating a managed cluster.

If you require HTTP proxy configurations, you can apply them during the `NKP create cluster` operation by adding the appropriate flags to the command example below:

Proxy configuration	Flag
HTTP proxy for control plane machines	`--control-plane-http-proxy string`
HTTPS proxy for control plane machines	`--control-plane-https-proxy string`
No Proxy list for control plane machines	`--control-plane-no-proxy strings`
HTTP proxy for worker machines	`--worker-http-proxy string`
HTTPS proxy for worker machines	`--worker-https-proxy string`
No Proxy list for worker machines	`--worker-no-proxy strings`

Using an HTTP override file, you must apply the same configuration to any custom machine images built with the Konvoy Image Builder (KIB). For more information, see Image Overrides on page 1123.

Configure the Control Plane and Worker Nodes to Use HTTP/S Proxy

This method uses environment variables to configure the HTTP proxy values. (You are not required to use this method.)

Review this sample code to configure environment variables for the control plane and worker nodes, considering the list of considerations that follow the sample.

```
export CONTROL_PLANE_HTTP_PROXY=http://example.org:8080
export CONTROL_PLANE_HTTPS_PROXY=http://example.org:8080
export
 CONTROL_PLANE_NO_PROXY="example.org,example.com,example.net,localhost,127.0.0.1,10.96.0.0/12,192.

export WORKER_HTTP_PROXY=http://example.org:8080
export WORKER_HTTPS_PROXY=http://example.org:8080
export
 WORKER_NO_PROXY="example.org,example.com,example.net,localhost,127.0.0.1,10.96.0.0/12,192.168.0.0/
```

HTTP proxy configuration considerations to ensure the core components work correctly

- Replace `example.org,example.com,example.net` with your internal addresses
- `localhost` and `127.0.0.1` addresses that can be accessed directly, not through the proxy.
- `10.96.0.0/12` is the default Kubernetes service subnet
- `192.168.0.0/16` is the default Kubernetes pod subnet
- `kubernetes,kubernetes.default,kubernetes.default.svc,kubernetes.default.svc.cluster,kubernetes.c` is the internal Kubernetes kube-apiserver service
- The entries `.svc,.svc.cluster,.svc.cluster.local` are the internal Kubernetes services
- Auto-IP addresses `169.254.169.254` for any cloud provider
-
  - `169.254.169.254` is the AWS metadata server
  - `.elb.amazonaws.com` is for the worker nodes to allow them to communicate directly to the kube-apiserver ELB

Example of Creating a Cluster Using the Configured HTTP Proxy Variables

The following is an example of a `NKP create cluster...` command that uses the values set in the environment variables from the code sample above. Use the appropriate infrastructure provider name in line 1 from the choices listed:

```
nkp create cluster [aws, azure, gcp, preprovisoned, vsphere] \
 --cluster-name ${CLUSTER_NAME} \
 --control-plane-http-proxy="${CONTROL_PLANE_HTTP_PROXY}" \
 --control-plane-https-proxy="${CONTROL_PLANE_HTTPS_PROXY}" \
 --control-plane-no-proxy="${CONTROL_PLANE_NO_PROXY}" \
 --worker-http-proxy="${WORKER_HTTP_PROXY}" \
 --worker-https-proxy="${WORKER_HTTPS_PROXY}" \
 --worker-no-proxy="${WORKER_NO_PROXY}"
```

HTTP or HTTPS Proxy Settings for the NKP Kommander Component

After the cluster is running in the Konvoy component; you need to configure the `NO_PROXY` variable for each provider.

For example, in addition to the values above for Amazon Web Services (AWS), you need the following settings:

- The default VPC Classless Inter-Domain Routing (CIDR) range of `10.0.0.0/16`
- `kube-apiserver` internal or external ELB address

> **Note:** The `NO_PROXY` variable contains the Kubernetes Services CIDR. This example uses the default CIDR, `10.96.0.0/12`. If your cluster's CIDR differs, update the value in the `NO_PROXY` field.

Set the `httpProxy` and `httpsProxy` environment variables to the address of the HTTP and HTTPS proxy servers, respectively. (Frequently, environments use the same values for both.) Set the `noProxy` environment variable to the addresses that can be accessed directly and not through the proxy.

For the Kommander component of Nutanix Kubernetes Platform (NKP), refer to more HTTP Proxy information in Additional Kommander Configurations.

Konvoy Image Builder HTTP or HTTPS Proxy

In some networked environments, the machines used for building images can reach the Internet, but only through an HTTP or HTTPS proxy. For Nutanix Kubernetes Platform (NKP) to operate in these networks, you need a way to specify what proxies to use; see Configuring an HTTP or HTTPS Proxy on page 677. You can use an HTTP proxy override file to specify that proxy. When KIB tries installing a particular OS package, it uses that proxy to reach the Internet to download it.

The proxy setting specified here is NOT “baked into” the image - it is only used while the image is being built. The settings are removed before the image is finalized.

While it might seem logical to include the proxy information in the image, the reality is that many companies have multiple proxies - one perhaps for each geographical region or maybe even a proxy per datacenter or office datacenter. All network traffic to the Internet goes through the proxy. If you were in Germany, you probably would not want to send all your traffic to a U.S.-based proxy. Doing that slows traffic down and consumes too many network resources. If you bake the proxy settings into the image, you must create a separate image for each region. Creating an image without a proxy makes more sense, but remember that you still need a proxy to access the Internet. Thus, when creating the cluster (and installing the Kommander component of NKP), you must specify the correct proxy settings for the network environment into which you install the cluster. You will use the same base image for that cluster installed in an environment with different proxy settings.

Output Directory Flag

When creating a cluster, you can use the `--output-directory` flag to organize the cluster configuration into individual files. This is particularly useful for ease of editing and managing the cluster configuration. The flag creates multiple files in the specified directory, which must already exist.

```
Example:
--output-directory=<existing-directory>
```

Customization of Cluster CAPI Components

Familiarize yourself with Cluster API (CAPI) before editing the cluster objects because edits can prevent the cluster from deploying successfully. The result of this command will allow such edits.

Select the appropriate infrastructure provider name from the choices listed when using the command `nkp create cluster [aws, azure, gcp, preprovisoned, vsphere]`.

Example:

```
nkp create cluster aws
  --cluster-name=${CLUSTER_NAME} \
  --dry-run \
  --output=yaml \
  > ${CLUSTER_NAME}.yaml
```

The “>” shows the output from the command saved to the file named ${CLUSTER_NAME}.yaml. To edit that YAML Ain't Markup Language (YAML), you need to understand the CAPI components to avoid failure of the cluster deployment. The objects are Custom Resources defined by Cluster API components, and they belong to three different categories:

- **Cluster**

  A Cluster object references the infrastructure-specific and control plane objects. Because this is an Amazon Web Services (AWS) cluster, an AWS Cluster object describes the infrastructure-specific cluster properties. This means the AWS region, the VPC ID, subnet IDs, and security group rules required by the Pod network implementation.

- **Control Plane**

  A KubeadmControlPlane object describes the control plane, the group of machines that run the Kubernetes control plane components. Those include the `etcd` distributed database, the API server, the core controllers, and the scheduler. The object describes the configuration for these components and refers to an infrastructure-specific object that represents the properties of all control plane machines. For AWS, the object references an AWSMachineTemplate object, which means the instance type, the type of disk used, and the disk size, among other properties.

- **Node Pool**

  A Node Pool is a collection of machines with identical properties. For example, a cluster might have one Node Pool with large memory capacity and another Node Pool with graphics processing unit (GPU) support. Each Node Pool is described by three objects: The MachinePool references an object that represents the configuration of Kubernetes components (kubelet) deployed on each node pool machine, and an infrastructure-specific object that describes the properties of all node pool machines. For AWS, it references a KubeadmConfigTemplate and an AWSMachineTemplate object, which represents the instance type, the type of disk used, and the disk size, among other properties.

For more information on the objects, see the Cluster API book https://cluster-api.sigs.k8s.io/user/concepts.html or Custom Resources in https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/.

Registry and Registry Mirrors

Nutanix Kubernetes Platform (NKP) supports operation with several local registry tools.

- **Registry Mirrors** are local copies of images from a public registry that follow (or mirror) the file structure of a public registry. If you need to set up a private registry with a registry mirror or details on using the flag(s), see Using a Registry Mirror on page 1071.
- **Container registries** are collections of container repositories and can also offer API paths and access rules.
- **Container repositories** are a collection of related container images. The container image has everything the software might need to run, including code, resources, and tools. Container repositories store container images for setup and deployment, and you use the repositories to manage, pull, and push images during cluster operations.

Kubernetes does not natively provide a registry for hosting the container images you will use to run the applications you want to deploy on Kubernetes. Instead, Kubernetes requires you to use an external solution to store and share container images. A variety of Kubernetes-compatible registry options are compatible with NKP.

This is How the Registry Mirror Works

: The first time you request an image from your local registry mirror, it pulls the image from the public registry (such as Docker) and stores it locally before handing it back to you. On subsequent requests, the local registry mirror can serve the image from its storage.

Air-gapped vs. Non-air-gapped Environments

In a non-air-gapped environment, you can access the Internet. You retrieve artifacts from specialized repositories dedicated to them, such as Docker images contained in DockerHub and Helm Charts that come from a dedicated Helm Chart repository. You can also create your local repository to hold the downloaded container images needed or any custom images you've created with the Konvoy Image Builder tool.

In an air-gapped environment, you need a local repository to store Helm charts, Docker images, and other artifacts. Private registries provide security and privacy in enterprise container image storage, whether hosted remotely or on-premises locally in an air-gapped environment. NKP in an air-gapped environment requires a local container registry of trusted images to enable production-level Kubernetes cluster management. However, a local registry is also an option in a non-air-gapped environment for speed and security.

If you want to use images from this local registry to deploy applications inside your Kubernetes cluster, you'll need to set up a secret for a private registry. The secret contains your login data, which Kubernetes needs to connect to your private repository. It is not required to `export` any variables for most of the command examples. However, the export, along with an arbitrary variable name, primarily clarifies what values in the commands need to be substituted. Also, that makes it easier to copy and paste the examples. Furthermore, if multiple steps in a procedure need you to specify a variable, you export it once with the `export`and then reuse it in future commands.

```
export it once with the #export#, https/http>://<registry-address>:<registry-port>"
```

To run the `create cluster` command using that variable created above, use the example command replacing `azure` with your choice of provider [`gcp`, `vsphere`, `vcd`, `pre-provisioned`, `aws`]:

```
nkp create cluster azure --registry-mirror-url=${REGISTRY_URL}
```

A cluster administrator uses NKP CLI commands to upload the image bundle to your registry with the parameters:

```
nkp push bundle --bundle <bundle> --to-registry=${REGISTRY_URL}
```

Parameter definitions:

- `--bundle <bundle>` the group of images. The example below is for the NKP air-gapped environment bundle

- Either use exported variable`${REGISTRY_URL}` or `--to-registry=<registry-address>/<registry-name>` to provide registry location for push

Command example:

```
nkp push bundle --bundle container-images/konvoy-image-bundle-v2.12.0.tar --to-registry=333000009999.dkr.ecr.us-west-2.amazonaws.com/can-test
```

Any URL can contain an optional port specification. If no port is specified, then the default port for the protocol is assumed. For example, for HTTPS protocol, port 443 is the default, meaning these two URLs are equivalent:

```
https://docs.nutanix.com
https://docs.nutanix.com:443
```

A port specification is only required if the URL target uses a port number other than the default.

Related Information

For information on related topics or procedures, see Registry Mirror Tools on page 1069.

Managing Subnets and Pods

Describes how to change Kubernetes subnets during cluster creation.

About this task

Some subnets reserved by Kubernetes can prevent proper cluster deployment if you unknowingly configure Nutanix Kubernetes Platform (NKP) so that the Node subnet collides with either the Pod or Service subnet. Ensure your subnets do not overlap with your host subnet because the subnets cannot be changed after cluster creation.

Note: The default subnets used in NKP are /16 for pods and /20 for service.

```
spec:
  clusterNetwork:
    pods:
      cidrBlocks:
      - 192.168.0.0/16
    services:
      cidrBlocks:
      - 10.96.0.0/12
```

Changing the Kubernetes subnets must be done during cluster creation. To change the subnets, perform the following steps:

Procedure

1. Generate the YAML Ain't Markup Language (YAML) manifests for the cluster using the `--dry-run` and `-o yaml` flags, along with the desired `nkp cluster create` command.

   Example:

   ```
   nkp create cluster preprovisioned --cluster-name ${CLUSTER_NAME} --control-plane-endpoint-host <control plane endpoint host> --control-plane-endpoint-port <control plane endpoint port, if different than 6443> --dry-run -o yaml > cluster.yaml
   ```

   Note: MetalLB IP address ranges, Classless Inter-Domain Routing (CIDR), and node subnet should not conflict with the Kubernetes cluster pod and service subnets.

2. To modify the service subnet, add or edit the `spec.clusterNetwork.services.cidrBlocks` field of the `Cluster` object.

   Example:

   ```
   kind: Cluster
   spec:
     clusterNetwork:
       services:
         cidrBlocks:
         - 10.0.0.0/12
   ```

3. To modify the pod subnet, edit the `Cluster` and calico-cni `ConfigMap` resources. Cluster: Add or edit the `spec.clusterNetwork.pods.cidrBlocks` field.

   Example:

   ```
   kind: Cluster
   spec:
     clusterNetwork:
       pods:
         cidrBlocks:
         - 172.16.0.0/16
   ```

4. In the ConfigMap, edit the `data."custom-resources.yaml".spec.calicoNetwork.ipPools.cidr` field with your desired pod subnet.

   Example:

   ```
   apiVersion: v1
   data:
     custom-resources.yaml: |
       apiVersion: operator.tigera.io/v1
       kind: Installation
       metadata:
         name: default
       spec:
         # Configures Calico networking.
         calicoNetwork:
           # Note: The ipPools section cannot be modified post-install.
           ipPools:
           - blockSize: 26
             cidr: 172.16.0.0/16
   kind: ConfigMap
   metadata:
     name: calico-cni-<cluter-name>
   ```

   When you provision the cluster, the configured pod and service subnets will be applied.

Creating a Bastion Host

When creating an air-gapped cluster, the bastion Virtual Machine (VM) hosts the installation of the Nutanix Kubernetes Platform (NKP) Konvoy bundles, images, and the Docker or other local registry are needed to create and operate your cluster. In a given environment, the bastion VM must have access to the infrastructure provider's Application Programming Interface (API).

About this task

Ensure the items below are installed and the environment matches the requirements below:

- Create a bastion VM host template for the cluster nodes to use within the air-gapped network. This bastion VM host also needs access to a local registry instead of an Internet connection to pull images.
- Find and record the bastion VM's IP or hostname.

- Download the following required NKP Konvoy binaries and installation bundles are discussed in step 5 below. To access the download bundles, see Downloading NKP on page 16.
- A local registry or Docker version 27.4.0 installed. You must install Docker on the host where the NKP Konvoy CLI runs. For example, if you install Konvoy on your laptop, ensure the computer has a supported version of Docker. On macOS, Docker runs in a virtual machine that you configure with at least 8GB of memory. For information on the local registry, see Registry Mirror Tools on page 1069. For information on Docker, see https://docs.docker.com/get-docker/.
- To interact with the running cluster, install kubectl on the host where the NKP Konvoy command line interface (CLI) runs. For more information, see kubectl.

Depending on your OS, various commands exist to set up your bastion host in an air-gapped environment. The vSphere example workflow shows a generic instance for Red Hat Enterprise Linux (RHEL) Bastion nodes using Docker.

Procedure

1. Open an `ssh` terminal to the bastion host and install the tools and packages using the command `sudo yum install -y yum-utils bzip2 wget`.

2. Install `kubectl`.

   RHEL example:

   ```
   cat <<EOF | sudo tee /etc/yum.repos.d/kubernetes.repo
   [kubernetes]
   name=Kubernetes
   baseurl=https://packages.cloud.google.com/yum/repos/kubernetes-el7-\$basearch
   enabled=1
   gpgcheck=1
   gpgkey=https://packages.cloud.google.com/yum/doc/rpm-package-key.gpg
   EOF
   sudo yum install -y kubectl
   ```

3. Install Docker, for example (only on the Bastion Host), and add the repo for upstream Docker using the command `sudo yum-config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo`

   Docker Install example:

   ```
   sudo yum install -y docker-ce docker-ce-cli containerd.io
   ```

4. Download `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`, extract the tar file to a local directory using the command `tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`

5. Set the following environment variables to enable connection to an existing Docker or other registry using the command `export REGISTRY_ADDRESS=<https/http>://<registry-address>:<registry-port> export REGISTRY_CA=<path to the CA on the bastion host>`.

   > **Note:** You must create the VM template with the Konvoy Image Builder to use the registry mirror feature.

   Command variables for the export REGISTRY command.

   - `REGISTRY_ADDRESS`: The address of an existing registry accessible in the environment where the new cluster nodes will be configured to use a mirror registry when pulling images.
   - `REGISTRY_CA`: (Optional) path on the bastion host to the registry CA. Konvoy configures the cluster nodes to trust this CA. This value is only needed if the registry uses a self-signed certificate and the VMs are not already configured to trust this CA.

Provision Flatcar Linux OS

Flatcar default network interface name might require specifying. It is most likely to be `ens192` , which requires passing the parameter `--virtual-ip-interface ens192` to the `nkp create cluster aws` command. Otherwise, the cluster creation might fail because `kube-vip` can not configure the first control-plane virtual IP.

Flatcar Linux Example

These flags are also shown in context on the Create Cluster page for either air-gapped or non-air-gapped environments:

Amazon Web Services (AWS) Example is shown; replace `aws` with `vsphere` if required:

```
nkp create cluster aws \
    --cluster-name ${CLUSTER_NAME} \
    --os-hint flatcar
```

Note: For provisioning Nutanix Kubernetes Platform (NKP) on Flatcar, NKP configures cluster nodes to use Control Groups (cgroups) version 1. In versions before Flatcar 3033.3.x, a restart is required to apply the changes to the kernel.

Also note that once **Ignition** runs, it is not available on reboot.

For more information on Flatcar usage, see:

- Flatcar documentation: https://www.flatcar.org/docs/latest/container-runtimes/switching-to-unified-cgroups/#starting-new-nodes-with-legacy-cgroups
- Control Groups version 1:https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v1/cgroups.html#what-are-cgroups
- Ignition https://www.flatcar.org/docs/latest/provisioning/ignition/#ignition-only-runs-once

Load Balancers

Kubernetes has a basic load balancer solution internally but does not offer an external load balancing component directly. You must provide one, or you can integrate your Kubernetes cluster with a cloud provider. In a Kubernetes cluster, depending on the flow of traffic direction, there are two kinds of load balancing:

- Internal load balancing for the traffic within a Kubernetes cluster
- External load balancing for the traffic coming from outside the cluster

Nutanix Kubernetes Platform (NKP) includes a load balancing solution for the supported cloud infrastructure providers environments. For more information, see Load Balancing on page 632.

If you want to use a **non-NKP load balancer** (for example, as an alternative to MetalLB), NKP supports setting up an **external load balancer**.

When enabled, the external load balancer routes incoming traffic requests to a single entry point in your cluster. Users and services can access the **NKP UI** through an established IP or DNS address.

A virtual IP is a client's address to connect to the service. A load balancer is a device that distributes the client connections to the backend servers. Before you create a new NKP cluster, choose an external load balancer (LB) or virtual IP.

External load balancer

It is recommended that an external load balancer be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane

machines. Configure the load balancer to send requests only to control plane machines responding to API requests.

When enabled, the external load balancer routes incoming traffic requests to a single entry point in your cluster. Users and services can access the **NKP UI** through an established IP or DNS address. For more information, see External Load Balancer.

Built-in virtual IP

If an external load balancer is unavailable, use the built-in virtual IP. The virtual IP is **not** a load balancer. It does not distribute the request load among the control plane machines. However, if the machine receiving requests does not respond, the virtual IP automatically moves to another machine. For more information, see Universal Configurations for all Infrastructure Providers on page 677.

MetalLB

Choose one of the two protocols you want to use to announce service IPs. If your environment is not currently equipped with a load balancer, you can use MetalLB. Otherwise, your own load balancer will work, and you can continue the installation process with Pre-provisioned: Install Kommander. To use MetalLB, create a MetalLB configMap for your infrastructure. MetalLB uses one of two protocols for exposing Kubernetes services:

- Layer 2, with Address Resolution Protocol (ARP)
- Border Gateway Protocol (BGP)

  Select one of the following procedures to create your MetalLB manifest for further editing.

Layer 2 Configuration

Layer 2 mode is the simplest to configure: in many cases, you don't need any protocol-specific configuration, only IP addresses.

Layer 2 mode does not require the IPs to be bound to the network interfaces of your worker nodes. It gives the machine's MAC address to clients giving, and giving clients the machine's MAC address.

- MetalLB IP address ranges or CIDRs must be within the node's primary network subnet.
- MetalLB IP address ranges, CIDRs, and node subnets must not conflict with the Kubernetes cluster pod and service subnets.

For example, the following configuration gives MetalLB control over IPs from 192.168.1.240 to 192.168.1.250 and configures Layer 2 mode:

The following values are generic. Enter your specific values into the fields where applicable.

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    address-pools:
    - name: default
      protocol: layer2
      addresses:
      - 192.168.1.240-192.168.1.250
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

Border Gateway Protocol (BGP) Configuration

For a basic configuration featuring one BGP router and one IP address range, you need four pieces of information:

- The router IP address that MetalLB needs to connect to.
- The router's autonomous systems (AS) number.
- The AS number MetalLB to be used.
- An IP address range expressed as a Classless Inter-Domain Routing (CIDR) prefix.

As an example, if you want to give MetalLB the range 192.168.10.0/24 and AS number 64500 and connect it to a router at 10.0.0.1 with AS number 64501, your configuration will look like this:

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    peers:
    - peer-address: 10.0.0.1
      peer-asn: 64501
      my-asn: 64500
    address-pools:
    - name: default
      protocol: bgp
      addresses:
      - 192.168.10.0/24
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

Inspect Cluster for Issues

If you have issues during cluster creation, you can investigate those issues with some of the following commands..

You can investigate what is running and what has failed to try to resolve those issues independently.

Investtigate Cluster Issues

These commands can provide helpful information for troubleshooting.

- Check your pods to see if anything not running and investigate those pods. You can view your pods by checking their status with the command:

  ```
  kubectl get pods -A
  ```

- You can check the logs in your cluster API pod of your cluster infrastructure provider choice. The example below uses Nutanix infrastructure, so replace it with your infrastructure.

  ```
  kubectl logs -l cluster.x-k8s.io/provider=infrastructure-nutanix --namespace capx-
  system --kubeconfig ${CLUSTER_NAME}.conf
  ```

- If you still have your bootstrap cluster running, you can check your CAPI logs from the bootstrap with the command. The example below uses Nutanix infrastructure, so replace it with your CAPI driver and infra name.

```
kubectl logs -l cluster.x-k8s.io/provider=infrastructure-nutanix --namespace capx-system --kubeconfig ${CLUSTER_NAME}-bootstrap.conf
```

Nutanix Infrastructure

Configuration types for installing Nutanix Kubernetes Platform (NKP) on a Nutanix Infrastructure.

For an environment on the Nutanix Infrastructure, install options based on those environment variables are provided for you in this location.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Nutanix Overview

The overall process for configuring Nutanix and NKP together includes the following steps:

1. Configure Nutanix to provide the elements described in the Nutanix Prerequisites.
2. **For air-gapped environments**, create a bastion VM host. For more information see, Creating a Bastion Host on page 685.
3. Create a base OS image. For more information, see Nutanix Base OS Image Requirements on page 706.
4. Create a new cluster.
5. Verify and log on to the UI.

After creating the base OS image, the NKP image builder uses it to make a custom image if you are not using the pre-built Out-of-the-box Rocky Linux 9.4 image provided. You can use that resulting image with the `nkp create cluster nutanix` command to create the VM nodes in your cluster directly on a server. You can use NKP to provision and manage your cluster from that point.

Nutanix Infrastructure Prerequisites

Prerequisites specific to Nutanix infrastructure.

This section contains the prerequisite information specific to Nutanix infrastructure. These are in addition to the Nutanix Kubernetes Platform (NKP prerequisites for install requirements. Fulfilling the prerequisites involves completing the NKP prerequisites and the Nutanix prerequisites.

Prerequisites for Creating a Nutanix Cluster with NKP

Before using NKP to create a Nutanix cluster, verify that you have the following:

- An x86_64-based Linux or macOS machine.
- NKP binaries and NKP Image Builder (NIB) downloads. For more information, see Downloading NKP on page 16.

- A Container engine or runtime installed is required to install NKP and bootstrap:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
- A registry is needed in your environment.

  Note:

  - For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

    ```
    tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
    ```

  - Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

    For example:

    ```
    export TMPDIR=/path/to/your/directory
    ```

- kubectl 1.28.x for interacting with the running cluster, installed on the host where the NKP Konvoy command line interface (CLI) runs. For more information, see https://kubernetes.io/docs/tasks/tools/#kubectl.
- A valid Nutanix account with credentials configured.

  Note:

  - NKP uses the Nutanix CSI driver 3.0 as the default storage provider. For more information on the default storage providers, see Default storage providers.
  - For compatible storage suitable for production, you can choose from any of the storage options available for Kubernetes. For more information, see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types.
  - To turn off the default that Konvoy deploys:

    1. Set the default StorageClass as non-default.
    2. Set your newly created StorageClass to be the default.

    For more information on Changing the Default Storage Class, see https://kubernetes.io/docs/tasks/administer-cluster/change-default-storage-class/.

Nutanix Prerequisites

Before installing, verify that your environment meets the following basic requirements:

- Nutanix Prism Central version 2024.1 has role credentials configured, allowing Administrator privileges.
- AOS 6.5, 6.8, 6.10 and later

- Configure Prism Central Settings. For more information, see Prism Central Settings (Infrastructure).
  - Pre-designated subnets.
  - A subnet with unused IP addresses. The number of IP addresses required is computed as follows:
    - One IP address for each node in the Kubernetes Cluster. The default cluster size has three control plane nodes and four worker nodes, which require seven IP addresses.
    - One IP address is not part of an address pool for the Kubernetes API server (kubevip).
    - One IP address in the same CIDR as the subnet but is not part of an address pool for the load balancer service used by Traefik (metallb).
  - For air-gapped environments, a bastion VM host template with access to a configured local registry. The recommended template naming pattern is `../folder-name/NKP-e2e-bastion-template` or similar. Each infrastructure provider has its own set of bastion host instructions. For more information see, Creating a Bastion Host on page 685.
  - Access to a bastion VM or other network-connected host running NKP Image Builder.

    Note: Nutanix will provide a complete image built on Nutanix-provided if you do not want to create your own from a BaseOS image.

    - You must reach the Nutanix endpoint where the Konvoy Command Line Interface (CLI) runs.
- Verify that your OS is supported. For more information, see Supported Infrastructure Operating Systems on page 12.
- If not already complete, review the NKP installation prerequisites. For more information, see Prerequisites for Installation on page 47.

Prism Central Credential Management

Credentials are required in Nutanix Prism Central (PC) for the Nutanix Infrastructure and NKP functionality.

A Nutanix Kubernetes Platform(NKP) 2.12 Nutanix infrastructure cluster uses Prism Central credentials for three components.

Note: Long-lived credentials are preferred.

1. To manage the cluster, such as listing subnets and other infrastructure, and to create VMs in Prism Central, which the Cluster API Provider Nutanix Cloud Infrastructure (CAPX) infrastructure provider uses.
2. To manage persistent storage used by Nutanix CSI providers.
3. To discover node metadata used by the Nutanix Cloud Cost Management (CCM) provider.

PC credentials are required to authenticate to the Prism Central APIs. CAPX currently supports two mechanisms for supplying the required credentials.

- Credentials injected into the CAPX manager deployment.
- Workload cluster-specific credentials.

For examples, see the topic Credential Management.

Injected Credentials

By default, credentials will be injected into the CAPX manager deployment when CAPX is initialized. See the Getting Started Guide topic for information about getting started with Cluster API Provider Nutanix Cloud Infrastructure (CAPX).

Upon initialization, a `nutanix-creds` secret will automatically be created in the `capx-system` namespace. This secret will contain the values supplied through the `NUTANIX_USER` and `NUTANIX_PASSWORD` parameters.

The `nutanix-creds` secret will be used for workload cluster deployment if no other credential is provided.

Workload Cluster Credentials

Users can override the credentials injected in CAPX manager deployment by supplying a credential specific to a workload cluster. See the topic Credentials injected in CAPX manager deployment. The credentials can be provided by creating a secret in the same namespace as the `NutanixCluster` namespace.

The secret can be referenced by adding a `credentialRef` inside the `prismCentral` attribute contained in the `NutanixCluster`. See the topic Admin Guide to Prism Central. The secret will also be deleted when the `NutanixCluster` is deleted.

> **Caution:** Update the different credentials once a cluster has been deployed. Update the credentials on the workload (used by CCM, CSI, and other add-ons. Update credentials on the management cluster used by CAPX or keep CCM and CSI secrets in sync.

> **Note:** There is a 1:1 relation between the secret and the `NutanixCluster` object.

Updating Prism Central Credentials on Deployed NKP Clusters

Update your Prism Central credentials when you either want to change the password or the password expires.

Before you begin

Ensure that you have:

- A `kubeconfig` file for the management cluster.
- A `kubeconfig` file for each of the workload clusters that need PC credentials changed.

Update the following secrets with a new password for Prism Central:

Table 57: Table

Secret Name	Namespace	NKP Cluster
global-nutanix-credentials	capx-system	Management Cluster
${MANAGEMENT_CLUSTER_NAME}-pc-credentials	${MANAGEMENT_CLUSTER_NAMESPACE}	Management Cluster
${MANAGEMENT_CLUSTER_NAME}-pc-credentials-for-csi	${MANAGEMENT_CLUSTER_NAMESPACE}	Management Cluster
${MANAGED_CLUSTER_NAME}-pc-credentials	${MANAGED_CLUSTER_NAMESPACE}	Management Cluster
${MANAGED_CLUSTER_NAME}-pc-credentials-for-csi	${MANAGED_CLUSTER_NAMESPACE}	Management Cluster
nutanix-ccm-credentials	kube-system	• Management Cluster • Managed Cluster

Secret Name	Namespace	NKP Cluster
nutanix-csi-credentials	ntnx-system	• Management Cluster • Managed Cluster

To update the Prism Central credentials:

Procedure

1. Scale down the associated PODs.

   Scale down the following deployments or controllers.

   ```
   kubectl scale deployment capx-controller-manager -n capx-system --replicas=0
   kubectl scale deployment cluster-api-runtime-extensions-nutanix -n caren-system --
   replicas=0
   kubectl scale deployment nutanix-cloud-controller-manager -n kube-system --replicas=0
   kubectl scale deployment nutanix-csi-controller -n ntnx-system --replicas=0
   kubectl scale deployment nutanix-csi-controller -n ntnx-system --replicas=0 --
   kubeconfig=$KUBECONFIG_WORKLOAD
   kubectl scale deployment nutanix-cloud-controller-manager -n kube-system --replicas=0
    --kubeconfig=$KUBECONFIG_WORKLOAD
   ```

   Scale down the CSI pods on the management cluster.

   ```
   kubectl -n ntnx-system patch daemonset nutanix-csi-node  -p '{"spec": {"template":
    {"spec": {"nodeSelector": {"non-existing": "true"}}}}}'
   ```

   Scale down the CSI pods on each of the managed clusters.

   ```
   kubectl -n ntnx-system patch daemonset nutanix-csi-node  -p '{"spec": {"template":
    {"spec": {"nodeSelector": {"non-existing": "true"}}}}}' --kubeconfig=
   $KUBECONFIG_WORKLOAD
   ```

2. Update the secret for the following global credentials.

   ```
   ***  kubectl get secrets global-nutanix-credentials -n capx-system -o yaml
   apiVersion: v1
   data:
     additionalTrustBundle: ""
     insecure: base 64 encoded Insecure Mode
     password: base 64 encoded PC Password     < PrismCentral Password.
     prismURL: base 64 encoded PC Endpoint
     username: base 64 encoded PC Username
   kind: Secret
   metadata:
     creationTimestamp: "2024-09-30T16:16:05Z"
     labels:
       clusterctl.cluster.x-k8s.io/move: ""
       dkp-infrastructure-provider-type: nutanix-secret
     name: global-nutanix-credentials
     namespace: capx-system
     resourceVersion: "4977406"
     uid: 109a34fb-efae-40a3-9db1-bede0ebaa464
   type: Opaque



   * Password is stored in base64 format, where we will encode the new pass with
    following:

   ***  echo -n 'NEW_PASSWORD' | base64
   ```

```
TkVXX1BBU1NXT1JE



* then update global-nutanix-credentials with new password:

***  kubectl edit secrets global-nutanix-credentials -n capx-system -o yaml
apiVersion: v1
data:
  additionalTrustBundle: ""
  insecure: base 64 encoded Insecure Mode
  password: base 64 encoded PC Password     < Update new PrismCentral Password.
  prismURL: base 64 encoded PC Endpoint
  username: base 64 encoded PC Username
kind: Secret
metadata:
  creationTimestamp: "2024-09-30T16:16:05Z"
  labels:
    clusterctl.cluster.x-k8s.io/move: ""
    dkp-infrastructure-provider-type: nutanix-secret
  name: global-nutanix-credentials
  namespace: capx-system
  resourceVersion: "4977406"
  uid: 109a34fb-efae-40a3-9db1-bede0ebaa464
type: Opaque
```

3. Update individual cluster credentials (secrets).

   Secrets are in the formats `<cluster-name>-pc-credentials` and `<cluster-name>-pc-credentials-for-csi`.

4. Update the PC credentials for the Management cluster.

```
#Step 1: Get the Secret.

***  kubectl get secrets ${MANAGEMENT_CLUSTER_NAME}-pc-credentials -o yaml
apiVersion: v1
data:
  credentials: base64 encoded PC Ccredentials
kind: Secret
metadata:
  creationTimestamp: "2024-09-30T16:16:07Z"
  finalizers:
  - nutanixcluster/infrastructure.cluster.x-k8s.io
  labels:
    cluster.x-k8s.io/provider: nutanix
    konvoy.d2iq.io/cluster-name: ${MANAGEMENT_CLUSTER_NAME}
    konvoy.d2iq.io/provider: nutanix
  name: ${MANAGEMENT_CLUSTER_NAME}-pc-credentials
  namespace: default
  ownerReferences:
  - apiVersion: infrastructure.cluster.x-k8s.io/v1beta1
    kind: NutanixCluster
    name: ${MANAGEMENT_CLUSTER_NAME}-2smbz
    uid: 8a9d1bba-02d5-4287-b3d8-284199d28a18
  - apiVersion: cluster.x-k8s.io/v1beta1
    kind: Cluster
    name: ${MANAGEMENT_CLUSTER_NAME}
    uid: a74225d9-4367-4fc2-b424-53a0bd981020
  resourceVersion: "4982392"
  uid: d382fa23-5e0e-46d7-a1c0-b5b3ee0bddd3
type: Opaque
```

```
#Use the value of the `credentials` attribute in the secret
***   echo base 64 String | base64 -d
[
{
"type": "basic_auth",
"data": {
"prismCentral": {
"username": "PC Username",
"password": "PC Password"
}
}
}
]


#Step 2: Decode the secret value and save it in a file.

***  echo base 64 String | base64 -d > update_new_password.txt



#Step 3: Open the file using a text editor and modify the password.

***   cat update_new_password.txt
[
{
"type": "basic_auth",
"data": {
"prismCentral": {
"username": "PC Username",
"password": "PC Password"     << ADD new password.
}
}
}
]



#Step 4: Encode the updated secret value.

***   cat update_new_password.txt | base64 base 64 String


#Step 5: Edit the secret and update the secret value with the new value.

***  kubectl edit secrets ${MANAGEMENT_CLUSTER_NAME}-pc-credentials -o yaml


#Step 6: Validate the changes.

***  kubectl get secrets ${MANAGEMENT_CLUSTER_NAME}-pc-credentials -o yaml
apiVersion: v1
data:
credentials: base64 encoded PC Ccredentials << NEW_BASE64ENCODED_PASSWORD
kind: Secret
metadata:
creationTimestamp: "2024-09-30T16:16:07Z"
finalizers:
- nutanixcluster/infrastructure.cluster.x-k8s.io
labels:
```

```
cluster.x-k8s.io/provider: nutanix
konvoy.d2iq.io/cluster-name: ${MANAGEMENT_CLUSTER_NAME}
konvoy.d2iq.io/provider: nutanix
name: ${MANAGEMENT_CLUSTER_NAME}-pc-credentials
namespace: default
ownerReferences:
- apiVersion: infrastructure.cluster.x-k8s.io/v1beta1
kind: NutanixCluster
name: ${MANAGEMENT_CLUSTER_NAME}-2smbz
uid: 8a9d1bba-02d5-4287-b3d8-284199d28a18
- apiVersion: cluster.x-k8s.io/v1beta1
kind: Cluster
name: ${MANAGEMENT_CLUSTER_NAME}
uid: a74225d9-4367-4fc2-b424-53a0bd981020
resourceVersion: "4982392"
uid: d382fa23-5e0e-46d7-a1c0-b5b3ee0bddd3
type: Opaque
```

5. Perform the preceding step for subsequent workload clusters.
   Secrets are in the format `<cluster-name>-pc-credentials`.

```
#Step 1: Get the Secret.

**  kubectl get secrets ${WORKLOAD_CLUSTER_NAME}-pc-credentials -n kommander-default-
workspace -o yaml
apiVersion: v1
data:
  credentials: base64 encoded PC Ccredentials
kind: Secret
metadata:
  creationTimestamp: "2024-09-30T21:46:05Z"
  finalizers:
  - nutanixcluster/infrastructure.cluster.x-k8s.io
  labels:
    cluster.x-k8s.io/provider: nutanix
  name: ${MANAGED_CLUSTER_NAME}-pc-credentials
  namespace: kommander-default-workspace
  ownerReferences:
  - apiVersion: infrastructure.cluster.x-k8s.io/v1beta1
    kind: NutanixCluster
    name: ${MANAGED_CLUSTER_NAME}-7bqc2
    uid: b4832f32-970b-4990-8d26-e5b134bca625
  - apiVersion: cluster.x-k8s.io/v1beta1
    kind: Cluster
    name: ${MANAGED_CLUSTER_NAME}
    uid: 435ab407-b366-4bd7-b5e3-4b7c125244e8
  resourceVersion: "5006043"
  uid: d132a4aa-7906-4584-b178-effdfcf3c5ce
type: Opaque



**  echo base 64 String | base64 -d
[{"type":"basic_auth","data":{"prismCentral":{"username":"PC Username","password":"PC
 Password"}}}]



#Step 2: Decode the secret value and save it in a file.

**  echo -n '[{"type":"basic_auth","data":{"prismCentral":{"username":"PC
 Username","password":"PC Password"}}}]' | base64
```

```
base 64 String



#Step 3: Edit the secret and update the secret value with the new value.

**  kubectl edit  secrets ${WORKLOAD_CLUSTER_NAME}-pc-credentials -n kommander-
default-workspace -o yaml
 apiVersion: v1
data:
credentials: base 64 encoded PC Password.          <<<< UPDATE_NEW_PASSWORD
kind: Secret
metadata:
creationTimestamp: "2024-09-30T21:46:05Z"
finalizers:
- nutanixcluster/infrastructure.cluster.x-k8s.io
labels:
cluster.x-k8s.io/provider: nutanix
name: ${WORKLOAD_CLUSTER_NAME}-pc-credentials
namespace: kommander-default-workspace
ownerReferences:
- apiVersion: infrastructure.cluster.x-k8s.io/v1beta1
kind: NutanixCluster
name: ${WORKLOAD_CLUSTER_NAME}-7bqc2
uid: b4832f32-970b-4990-8d26-e5b134bca625
- apiVersion: cluster.x-k8s.io/v1beta1
kind: Cluster
name: ${WORKLOAD_CLUSTER_NAME}
uid: 435ab407-b366-4bd7-b5e3-4b7c125244e8
resourceVersion: "5006043"
uid: d132a4aa-7906-4584-b178-effdfcf3c5ce
type: Opaque
```

6. Update the CSI credentials on the management cluster for each cluster in the format `<cluster-name>-for-csi`.

```
*  kubectl get secrets ${MANAGEMENT_CLUSTER_NAME}-pc-credentials-for-csi -o yaml
apiVersion: v1
data:
  key: Key ID
kind: Secret
metadata:
  creationTimestamp: "2024-09-30T16:16:06Z"
  labels:
    cluster.x-k8s.io/provider: nutanix
    konvoy.d2iq.io/cluster-name: ${MANAGEMENT_CLUSTER_NAME}
    konvoy.d2iq.io/provider: nutanix
  name: ${MANAGEMENT_CLUSTER_NAME}-pc-credentials-for-csi
  namespace: default
  ownerReferences:
  - apiVersion: cluster.x-k8s.io/v1beta1
    kind: Cluster
    name: ${MANAGEMENT_CLUSTER_NAME}
    uid: a74225d9-4367-4fc2-b424-53a0bd981020
  resourceVersion: "5000354"
  uid: e76fe412-d904-4b5f-b029-4800571898b4
type: Opaque


*  echo base 64 String | base64 -d
PC IP:9440:PC Username:PC Password
```

```
*  echo -n 'PC IP:9440:admin:NEW_PASSWORD' | base64base 64 String     <<
 NEW_BASE64ENCODED_PASSWORD



*   kubectl edit secrets ${MANAGEMENT_CLUSTER_NAME}-pc-credentials-for-csi -o yaml


*  kubectl get secrets ${MANAGEMENT_CLUSTER_NAME}-pc-credentials-for-csi -o yaml
apiVersion: v1
data:
key: NEW_BASE64ENCODED_PASSWORD.                      << UPDATED_PASSWORD
kind: Secret
metadata:
creationTimestamp: "2024-09-30T16:16:06Z"
labels:
cluster.x-k8s.io/provider: nutanix
konvoy.d2iq.io/cluster-name: ${MANAGEMENT_CLUSTER_NAME}
konvoy.d2iq.io/provider: nutanix
name: ${MANAGEMENT_CLUSTER_NAME}-pc-credentials-for-csi
namespace: default
ownerReferences:
- apiVersion: cluster.x-k8s.io/v1beta1
kind: Cluster
name: ${MANAGEMENT_CLUSTER_NAME}
uid: a74225d9-4367-4fc2-b424-53a0bd981020
resourceVersion: "5000354"
uid: e76fe412-d904-4b5f-b029-4800571898b4
type: Opaque
*
```

Similarly, update the CSI credentials for the other clusters:

```
* kubectl get secret ${WORKLOAD_CLUSTER_NAME}-pc-credentials-for-csi  -n kommander-
default-workspace -o yaml
apiVersion: v1
data:
  key: Key ID
kind: Secret
metadata:
  creationTimestamp: "2024-09-30T21:46:05Z"
  labels:
    cluster.x-k8s.io/provider: nutanix
  name: ${WORKLOAD_CLUSTER_NAME}-pc-credentials-for-csi
  namespace: kommander-default-workspace
  ownerReferences:
  - apiVersion: cluster.x-k8s.io/v1beta1
    kind: Cluster
    name: ${WORKLOAD_CLUSTER_NAME}
    uid: 435ab407-b366-4bd7-b5e3-4b7c125244e8
  resourceVersion: "5008388"
  uid: bdf259ec-fa26-48a7-9be9-7712d3614802
type: Opaque

* echo base 64 String | base64 -dPC IP:9440:PC Username:PC Password



* echo -n 'PC IP:9440:admin:NEW_PASSWORD' | base64 base 64 String
```

```
* kubectl edit secret ${WORKLOAD_CLUSTER_NAME}-pc-credentials-for-csi  -n kommander-
default-workspace -o yaml
 kubectl get secret ${WORKLOAD_CLUSTER_NAME}-pc-credentials-for-csi -n kommander-
default-workspace -o yaml
apiVersion: v1
data:
key: base64 encoded New Password.                    << UPDATED_PASSWORD
kind: Secret
metadata:
creationTimestamp: "2024-09-30T21:46:05Z"
labels:
cluster.x-k8s.io/provider: nutanix
name: ${WORKLOAD_CLUSTER_NAME}-pc-credentials-for-csi
namespace: kommander-default-workspace
ownerReferences:
- apiVersion: cluster.x-k8s.io/v1beta1
kind: Cluster
name: ${WORKLOAD_CLUSTER_NAME}
uid: 435ab407-b366-4bd7-b5e3-4b7c125244e8
resourceVersion: "5008388"
uid: bdf259ec-fa26-48a7-9be9-7712d3614802
type: Opaque
```

7. Update the CCM and CSI credentials on individual cluster level.

   For each cluster, `--kubeconfig` changes. However, the steps remain the same.

```
* kubectl get secrets nutanix-ccm-credentials -n kube-system --kubeconfig=
$KUBECONFIG_MANAGEMENT -o yaml
apiVersion: v1
data:
  credentials: base64 encoded PC Credentials
kind: Secret
metadata:
  creationTimestamp: "2024-09-30T16:12:20Z"
  name: nutanix-ccm-credentials
  namespace: kube-system
  resourceVersion: "5017728"
  uid: 3290249e-facb-45e3-a785-62ac620d5861
type: Opaque

*  echo base 64 String | base64 -d
[
    {
        "type": "basic_auth",
        "data": {
            "prismCentral": {
                "username": "PC Username",
                "password": "PC Password"
            }
        }
    }
]

*  echo base 64 String | base64 -d > update_ccm_password.txt
  cat update_ccm_password.txt
[
    {
        "type": "basic_auth",
        "data": {
            "prismCentral": {
                "username": "PC Username",
                "password": "PC Password"
```

```
            }
        }
    }
]

* cat update_ccm_password.txt | base64 base 64 String



* kubectl edit secrets nutanix-ccm-credentials -n kube-system --kubeconfig=
$KUBECONFIG_MANAGEMENT -o yaml


* kubectl get secrets nutanix-ccm-credentials -n kube-system --kubeconfig=
$KUBECONFIG_MANAGEMENT -o yaml
apiVersion: v1
data:
  credentials: base64 encoded New Password
kind: Secret
metadata:
  creationTimestamp: "2024-09-30T16:12:20Z"
  name: nutanix-ccm-credentials
  namespace: kube-system
  resourceVersion: "5017728"
  uid: 3290249e-facb-45e3-a785-62ac620d5861
type: Opaque



CSI password update:


* kubectl get secrets nutanix-csi-credentials -n ntnx-system -o yaml
apiVersion: v1
data:
key: Key ID
kind: Secret
metadata:
creationTimestamp: "2024-09-30T16:12:20Z"
name: nutanix-csi-credentials
namespace: ntnx-system
resourceVersion: "5023111"
uid: e78d3eef-3d86-4c26-966f-4672531b30aa
type: Opaque

* echo base 64 String | base64 -d
PC IP:9440:PC Username:PC Password
echo -n 'PC IP:9440:admin:NEW_PASSWORD' | base64 base 64 String


* kubectl edit secrets nutanix-csi-credentials -n ntnx-system -o yaml
apiVersion: v1
data:
key: base64 encoded New Password                    << UPDATED_PASSWORD
kind: Secret
metadata:
creationTimestamp: "2024-09-30T16:12:20Z"
name: nutanix-csi-credentials
namespace: ntnx-system
resourceVersion: "5023111"
uid: e78d3eef-3d86-4c26-966f-4672531b30aa
type: Opaque
```

8. Update the infrastructure provider secret.

   The secret name is identified as ID from the UI. For example, `ntnx-jdjkt` is the provider ID on the UI.

```
* kubectl get secrets ntnx-jdjkt -n kommander-default-workspace -o yaml
apiVersion: v1
data:
  additionalTrustBundle: ""
  insecure: base64 encoded Insecure Mode
  password: base64 encoded PC Password
  prismURL: base64 encoded PC URL
  username: base64 encoded PC Username
kind: Secret
metadata:
  creationTimestamp: "2024-09-30T20:18:04Z"
  labels:
    dkp-infrastructure-provider-type: nutanix-secret
  name: ntnx-jdjkt
  namespace: kommander-default-workspace
  resourceVersion: "5188693"
  uid: 25afed6e-e037-4d69-b147-57066508d710
type: Opaque

* echo -n 'NEW_PASSWORD' | base64
TkVXX1BBU1NXT1JE

* kubectl edit secrets ntnx-jdjkt -n kommander-default-workspace
apiVersion: v1
data:
  additionalTrustBundle: ""
  insecure: base64 encoded Insecure Mode
  password: base64 encoded PC Password                        <<<
 NEW_BASE64ENCODED_PASSWORD
  prismURL: base64 encoded PC URL
  username: base64 encoded PC Username
kind: Secret
metadata:
  creationTimestamp: "2024-09-30T20:18:04Z"
  labels:
    dkp-infrastructure-provider-type: nutanix-secret
  name: ntnx-jdjkt
  namespace: kommander-default-workspace
  resourceVersion: "5188693"
  uid: 25afed6e-e037-4d69-b147-57066508d710
type: Opaque
```

9. Scale up the pods.

```
Scale pods on Management cluster:
* kubectl -n ntnx-system patch daemonset nutanix-csi-node --type json -p='[{"op":
 "remove", "path": "/spec/template/spec/nodeSelector/non-existing"}]'
* kubectl scale deployment cluster-api-runtime-extensions-nutanix -n caren-system --
replicas=1
* kubectl scale deployment capx-controller-manager -n capx-system --replicas=1
* kubectl scale deployment nutanix-csi-controller  -n ntnx-system --replicas=1
* kubectl scale deployment nutanix-cloud-controller-manager -n kube-system --
replicas=1

Similarly for Managed cluster:
* kubectl -n ntnx-system patch daemonset nutanix-csi-node --type json -p='[{"op":
 "remove", "path": "/spec/template/spec/nodeSelector/non-existing"}]' --kubeconfig=
$KUBECONFIG_WORKLOAD
```

```
* kubectl scale deployment nutanix-cloud-controller-manager -n kube-system --
replicas=1 --kubeconfig=$KUBECONFIG_WORKLOAD
* kubectl scale deployment nutanix-csi-controller -n ntnx-system --replicas=1 --
kubeconfig=$KUBECONFIG_WORKLOAD
```

Prism Central Pre-Defined Role

When provisioning Kubernetes clusters with Nutanix Kubernetes Platform (NKP) on Nutanix infrastructure, a role contains the minimum permissions needed for NKP to provide proper access to deploy clusters.

An NKP 2.12 Nutanix cluster uses Prism Central credentials for three components:

1. To manage the cluster for such actions as to list subnets and other infrastructure and to create Virtual Machines (VM) in Prism Central (Used by Cluster API Provider Nutanix Cloud Infrastructure (CAPX) infrastructure provider).
2. To manage persistent storage (Used by Nutanix CSI provider).
3. To discover node metadata (Used by Nutanix Cloud Cost Management (CCM) provider).

Configuring the Role Using an Authorization Policy

When provisioning Kubernetes clusters with Nutanix Kubernetes Platform (NKP) on Nutanix infrastructure, there is a pre-defined role containing the minimum permissions for deploying clusters located in Prism Central.

About this task

On the **Kubernetes Infrastructure Provision Role Details** screen, you assign the system-defined roles by creating an authorization policy. For more information see, Configuring an Authorization Policy in the Nutanix Acropolis Operating System (AOS) Security documentation for further details.

Procedure

1. Log in to **Prism Central** as an administrator.
2. Select **Admin Center** in the Application Switcher.
3. Select **IAM** and go to **Authorization Policies**.
4. To create an authorization policy, select **Create New Authorization Policy** . The Create New Authorization Policy window appears
5. In the **Choose Role** step, enter a role name by typing in the **Select the role to add to this policy** field, and select **Next** You can enter any built-in or custom roles.
6. In the **Define Scope** step, select one of the following.
   - **Full Access** - which gives all added users access to all entity types in the associated role.
   - **Configure Access** - which provides you with the option to configure the entity types and instances for the added users in the associated role.
7. Select **Next**.
8. In the **Assign Users** step, do the following.
   - From the dropdown list, select Local User to add a local user or group to the policy. Search a user or group by typing the first few letters in the text field.
   - From the dropdown list, select the available directory to add a directory user or group. Search a user or group by typing the first few letters in the text field.

9. click **Save**.

   The authorization policy configurations are saved, and the authorization policy is listed in the **Authorization Policies** window.

> **Note:** To display role permissions for any built-in role, see the Nutanix AOS Security documentation topic Displaying Role Permissions.

Prism Central Role Permissions Table
This table contains the pre-defined permissions for the **Kubernetes Infrastructure Provisions** role in Prism Central.

> **Note:** To view the minimum permissions required for CSI Volume Driver, see Operations Required to Perform CSI Actions in the *CSI Volume Driver Guide*.

Infrastructure Role	Permission Granted
AHV VM	• Create Virtual Machine • Create a Virtual Machine Disk • Delete Virtual Machine • Delete Virtual Machine Disk • Update Virtual Machine • Update Virtual Machine Project • View Virtual Machine
Category	• Create Or Update Name Category • Create Or Update Value Category • Delete Name Category • Delete Value Category • View Name Category • View Value Category
Category Mapping	• Create Category Mapping • Delete Category Mapping • Update Category Mapping • View Category Mapping
Cluster	• View Cluster • Create Image • Delete Image • View Image

Infrastructure Role	Permission Granted
Project	• View Project
Subnet	• View Subnet

Preparing Prism Central Resources for the Cluster
Actions must be taken to prepare Prism Central (PC) resources for cluster creation.

About this task

After creating your BaseOS image, you must prepare Prism Central's resources to create a cluster.

Before you begin

Locate the following information in Nutanix Prism Central Admin Center Overview.

- Prism Central Endpoint - with or without the port
  - Prism Element (PE) cluster name. For more information, see Modifying Cluster Details in the Prism Element Web Console Guide.
  - Subnet name.
  - OS image name from creating the OS Image in the previous topic.
  - Docker Hub credentials, or you will encounter Docker Hub rate limiting, and the cluster will not be created.
- Find an available control plane endpoint IP not assigned to any VMs.

In Prism Central, you will enter information.

Procedure

1. Navigate to the **Infrastructure** menu.
   a. Select **Network and Security**.
   b. Select **Subnets** and choose your subnet's IP address.
   c. Use an IP in the subnet Classless Inter-Domain Routing (CIDR) but outside the IP pool.
2. Navigate to **Compute and Storage.**
   a. Select **Storage Containers**.
   b. Select **Create New** to create a new container.
   c. Enter a **Name** for your storage container.
   d. In the **Cluster** field, ensure you use the same cluster name you plan to deploy.
   e. Select **Create**.

   Prism Central is appropriately configured to create a cluster in your air-gapped or non-air-gapped Nutanix environment.

Migrating VMs from VLAN to OVN
Describes how to create and migrate a subnet.

About this task

Migrating Virtual Machines (VMs) from VLAN basic to OVN VLAN is not done through `atlas_cli` , which is recommended by other projects in Nutanix.

Some subnets reserved by Kubernetes can prevent proper cluster deployment if you unknowingly configure Nutanix Kubernetes Platform (NKP) so that the Node subnet collides with either the Pod or Service subnet. Ensure your subnets do not overlap with your host subnet because the subnets cannot be changed after cluster creation.

> **Note:** The default subnets used in NKP are:
>
> ```
> spec:
>   clusterNetwork:
>     pods:
>       cidrBlocks:
>       - 192.168.0.0/16
>     services:
>       cidrBlocks:
>       - 10.96.0.0/12
> ```

The existing VLAN implementation is *basic* VLAN. However, *advanced* VLAN uses OVN as the control plane instead of the Acropolis. The subnet creation workflow is from Prism Central (PC) rather than Prism Element (PE). Subnet creation can be done using API or through the UI.

Procedure

1. Log in to **Prism Central**
2. Select the **Infrastructure** application from the Application Switcher Function.

   For more information, see Application Switcher Function
3. Click the **Settings** icon or navigate to **Prism Central Setting** from the Navigation Bar. The **Settings** page appears.

   For more information, see Settings and Application-specific Navigation Bar
4. Click **Network Controller**
5. On the Network Controller page, select the **Configure Network Controller-Managed VLANs as default setting** checkbox.
6. To migrate VMs, select **Migrate VMs from VLAN basic subnets**.

   For more information, see Flow Virtual Networking Network Controller 5.0 - Migrating VMs from VLAN Basic Subnets
7. Modify the subnet specification in the control plane and worker nodes to use the new subnet. `kubectl edit cluster <clustername>`.

   CAPX will roll out the new control plane and worker nodes in the new Subnet and destroy the old ones.

   > **Note:** You can choose Basic or Advanced OVN when creating the subnet(s) you used during cluster creation. If you created the cluster with basic, you can migrate to OVN.

   To modify the service subnet, add or edit the `configmap`. See the topic Managing Subnets and Pods for more details.

Nutanix Base OS Image Requirements

The base OS image is used by Nutanix Kubernetes Platform (NKP) Image Builder (NIB) to create a custom image. In creating a base OS image, you have two choices:

1. Create your custom image for Rocky Linux 9.4 or Ubuntu 22.04.
2. Use the pre-built Rocky Linux 9.4 image downloaded from the portal.

   Starter license level workload clusters are only licensed to use Nutanix pre-built images.

The Base OS Image

This Base OS Image is later used with NKP Image Builder during installation and cluster creation. It is important to consider the image requirements.

- Prism Central configuration. For more information, see Prism Central Admin Center Guide.
- Choose to use or build a pre-built Rocky Linux 9.4 image. This Base OS Image is later used with NKP Image Builder during installation and cluster creation.
  - If using a pre-built image, ensure it has been uploaded to Prism Central images folder.
  - If creating a custom image, NIB will place the new image in the Prism Central images folder upon creation.

    Note: Out-of-the-box image: Nutanix provides a complete image built on Nutanix-provided base images.
- Network configuration is required because NIB must download and install packages, and activating the network is required.
- A Container engine or runtime installed is required to install NKP and bootstrap:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.

Disk Size

For each cluster you create using this base OS image, ensure you establish the disk size of the root file system based on the following:

- The minimum NKP Resource Requirements. For more information, see Resource Requirements.
- The minimum storage requirements for your organization.
- Clusters are created with a default disk size of 80 GB.
- For clusters created with the default disk size, the base OS image root file system must be precisely 80 GB. The root file system cannot be reduced automatically when a machine first boots.

Customization

You can also specify a custom disk size when you create a cluster (see the flags available for use with the Nutanix Create Cluster command). This allows you to use one base OS image to create multiple clusters with different storage requirements.

Before specifying a disk size when you create a cluster, take into account the following:

- For some base OS images, the custom disk size option does not affect the size of the root file system. This is because some root file systems, for example, those contained in Logical volume management (LVM) Logical Volume, cannot be resized automatically when a machine first boots.

- The specified custom disk size must be equal to, or larger than, the size of the base OS image root file system. This is because a root file system cannot be reduced automatically when a machine first boots.

Create the OS Image for Prism Central

Create your image using Nutanix Image Builder (NIB).

About this task

You can create your image or use the pre-built image from the Nutanix portal.

Before you begin

- Configure Prism Central. For more information, see Prism Central Admin Center Guide.
- Ensure you have Docker or Podman installed. For more information, see Nutanix Base OS Image Requirements.
- You will need:
  - Prism Element (PE) cluster name
  - Prism Central (PC) endpoint URL
  - Subnet from Prism Central (PC)

Procedure

1. Export Nutanix Credentials using the command `export NUTANIX_USER=<user> export NUTANIX_PASSWORD=<password>`.
2. Build Rocky 9.4 image using the command `nkp create image nutanix rocky-9.4`.
   Example:

   ```
   nkp create image nutanix rocky-9.4 \
   --cluster <PE_CLUSTER_NAME> \
   --endpoint <PC_ENDPOINT_WITHOUT_PORT_EX_prismcentral.foobar.example.com> \
   --subnet <NAME_OR_UUID_OF_SUBNET>
   ```

   a. To specify the name of the base image, use the flag `--source-image <name of base image>`.
   The Output will have the name of the image created. Take note of the Image name for use in cluster creation. For example, `nutanix.kib_image: Image successfully created: nkp-rocky-9.3-1.29.6-20240612181040 (db03feec-66f5-4c4d-85b1-79797a2aecc5)`.

Create the Air-gapped OS Image for Prism Central

Create your image using Nutanix Image Builder (NIB).

About this task

You can create your image or use the pre-built image from the Nutanix portal.

Before you begin

- Configure Prism Central. For more information, see Prism Central Admin Center Guide.
- Ensure you have Docker or Podman installed. For more information, see Nutanix Base OS Image Requirements.

- You will need:
  - Prism Element (PE) cluster name
  - Prism Central (PC) endpoint URL
  - Subnet from Prism Central (PC)

Procedure

1. Download `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`, and extract the tar file to a local directory using the command `tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`
2. You must fetch the distro packages and other artifacts. You get the latest security fixes available at machine image build time by fetching the distro packages from Rocky Linux or Ubuntu repositories.
3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the Nutanix Kubernetes Platform (NKP) command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`.

   For example:

   ```
   ./nkp create package-bundle OSName --artifacts-directory </path/to/save/os-package-bundles>
   ```

   Replace `OSName` with any of the supported operating systems for air-gapped.
4. Export Nutanix Credentials.

   For example:

   ```
   export NUTANIX_USER=<user>
   export NUTANIX_PASSWORD=<password>
   ```
5. Build Rocky 9.4 image using the command `nkp create image nutanix rocky-9.4`.

   For example:

   ```
   nkp create image nutanix rocky-9.4 \
   --cluster <PE_CLUSTER_NAME> \
   --endpoint <PC_ENDPOINT_WITHOUT_PORT_EX_prismcentral.foobar.example.com> \
   --subnet <NAME_OR_UUID_OF_SUBNET>
   --artifacts-directory </path/to/saved/os-package-bundles>
   --source-image <name of base image>
   ```

Nutanix Installation in a Non-air-gapped Environment

This installation provides instructions on how to install Nutanix Kubernetes Platform (NKP) in a Nutanix non-air-gapped environment.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47
- Nutanix Prerequisites

Further Nutanix Prerequisites

In an environment with access to the Internet, you retrieve artifacts from specialized repositories dedicated to them, such as Docker images contained in DockerHub and Helm Charts that come from a dedicated Helm Chart repository. In a non-air-gapped environment, you can still use local repositories to store Helm charts, Docker images, and other artifacts.

> **Tip:** A local registry can also be used in a non-air-gapped environment for speed and security if desired. To do so, add the following steps to your non-air-gapped installation process. See the topic Registry Mirror Tools on page 1069

Bootstrapping Nutanix

To create Kubernetes clusters, NKP uses Cluster API (CAPI) controllers. These controllers run on a Kubernetes cluster.

About this task

To get started, you need a bootstrap cluster. By default, Nutanix Kubernetes Platform (NKP) creates a bootstrap cluster for you in a Docker container using the Kubernetes-in-Docker (KIND) tool.

Before you begin

Procedure

1. Complete the Nutanix Infrastructure Prerequisites. For more information, see Nutanix Infrastructure Prerequisites on page 690.
2. Ensure the `NKP` binary can be found in your `$PATH`.
3. Decide your Base OS image selection. See BaseOS Image Requirements in the Prerequisites section.

Bootstrap Cluster Life Cycle Services

Procedure

1. Review Universal Configurations for all Infrastructure Providers regarding settings, flags, and other choices and begin bootstrapping. For more information, see Universal Configurations for all Infrastructure Providers.
2. Create a bootstrap cluster using the command `nkp create bootstrap --kubeconfig $HOME/.kube/config`.

   > **Note:** Use `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Configuring an HTTP or HTTPS Proxy on page 677.

   Example output:

   ```
   # Creating a bootstrap cluster
   # Initializing new CAPI components
   ```

3. NKP creates a bootstrap cluster using KIND as a library.

   For more information, see https://github.com/kubernetes-sigs/kind.

4. NKP then deploys the following Cluster API providers on the cluster.

   - Core Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/
   - AWS Infrastructure Provider: https://github.com/kubernetes-sigs/cluster-api-provider-aws
   - Kubeadm Bootstrap Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/bootstrap/kubeadm
   - Kubeadm ControlPlane Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/controlplane/kubeadm

   For more information on Cluster APIs, see https://cluster-api.sigs.k8s.io/.

5. Ensure that the Cluster API Provider Nutanix Cloud Infrastructure (CAPX) controllers are present using the command `kubectl get pods -n capx-system`.

   Output example:

```
NAME                                        READY   STATUS    RESTARTS   AGE
capx-controller-manager-785c5978f-nnfns   1/1     Running   0          13h
```

6. NKP waits until these providers' controller-manager and webhook deployments are ready. List these deployments using the command `kubectl get --all-namespaces deployments -l=clusterctl.cluster.x-k8s.io`.

   Output example:

```
NAMESPACE                           NAME
 READY   UP-TO-DATE   AVAILABLE   AGE
capa-system                         capa-controller-manager
 1/1     1            1           1h
capg-system                         capg-controller-manager
 1/1     1            1           1h
capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-manager
 1/1     1            1           1h
capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-manager
 1/1     1            1           1h
capi-system                         capi-controller-manager
 1/1     1            1           1h
cappp-system                        cappp-controller-manager
 1/1     1            1           1h
capv-system                         capv-controller-manager
 1/1     1            1           1h
capx-system                         capx-controller-manager
 1/1     1            1           1h
capz-system                         capz-controller-manager
 1/1     1            1           1h
cert-manager                        cert-manager
 1/1     1            1           1h
cert-manager                        cert-manager-cainjector
 1/1     1            1           1h
cert-manager                        cert-manager-webhook
 1/1     1            1           1h
```

Creating a Nutanix Management Cluster

Create a Nutanix Cluster in a non-air-gapped environment.

About this task

Use these instructions to create a cluster using the Nutanix Kubernetes Platform (NKP) default settings without any edits to configuration files or additional flags and deploy your cluster on a Rocky Linux 9.4

operating system image with three control plane nodes, and four worker nodes. The system deploys all control plane notes in a single AOS cluster. The default worker node pool exists in a single AOS cluster, and you can create additional node pools in other AOS clusters by using the `nkp create nodepool` command.

> **Note:** NKP uses the Nutanix CSI driver as the default storage provider. For more information see, Default Storage Providers on page 34.

> **Note:** NKP uses a CSI storage container on your Prism Element (PE). The CSI Storage Container image names must be the same for every PE environment in which you deploy an NKP cluster.

Before you begin

Name your cluster.

> **Note:** The cluster name might only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. For more naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable for cluster name using the command `export CLUSTER_NAME=<my-nutanix-cluster>`
3. Export Nutanix PC credentials.

   ```
   export NUTANIX_USER=$user
   export NUTANIX_PASSWORD=$password
   ```

4. Ensure your subnets do not overlap with your host subnet because they cannot be changed after cluster creation. The default subnets used in NKP are.

   ```
   spec:
     clusterNetwork:
       pods:
         cidrBlocks:
         - 192.168.0.0/16
       services:
         cidrBlocks:
         - 10.96.0.0/12
   ```

   If you need to change the Kubernetes subnets, you must do this at cluster creation. See the topic Subnets for more information.

   - (Optional) Modify Control Plane Audit logs - Users can modify the `KubeadmControlplane` cluster-API object to configure different `kubelet` options. If you need to configure your control plane beyond the existing options available from flags, see Configuring your control plane.
   - (Optional) Determine what VPC Network to use. Nutanix accounts have a default preconfigured VPC Network, which will be used if you do not specify a different network. To use a different VPC network for your cluster, create one by following these instructions for Create and Manage VPC Networks. Then select the `--network $new_vpc_network_name` option on the create cluster command below.

5. Create a Kubernetes cluster. The following example shows a common configuration.

   ```
   nkp create cluster nutanix \
       --cluster-name=$CLUSTER_NAME \
   ```

```
    --control-plane-prism-element-cluster=$PE_NAME \
    --worker-prism-element-cluster=$PE_NAME \
    --control-plane-subnets=$SUBNET_ASSOCIATED_WITH_PE \
    --worker-subnets=$SUBNET_ASSOCIATED_WITH_PE \
    --control-plane-endpoint-ip=$AVAILABLE_IP_FROM_SAME_SUBNET \
    --csi-storage-container=$NAME_OF_YOUR_STORAGE_CONTAINER \
    --endpoint=$PC_ENDPOINT_URL \
    --control-plane-vm-image=$NAME_OF_OS_IMAGE_CREATED_BY_NKP_CLI\
    --worker-vm-image=$NAME_OF_OS_IMAGE_CREATED_BY_NKP_CLI \
    --kubernetes-service-load-balancer-ip-range $START_IP-$END_IP \
    --dry-run \
    --output=yaml \
> ${CLUSTER_NAME}.yaml
```

Note: Optional registry flags are `--registry-mirror-url=${REGISTRY_URL} \ --registry-mirror-username=${REGISTRY_USERNAME} \ --registry-mirror-password=${REGISTRY_PASSWORD}`

Note: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

6. Inspect or edit the cluster objects. Familiarize yourself with Cluster API before editing the cluster objects, as edits can prevent the cluster from deploying successfully.

   Note: If the cluster creation fails, check for issues with your environment, such as storage resources. If the cluster becomes self-managed before it stalls, you can independently investigate what is running and what has failed to resolve those issues. See Resource Requirements and Inspect Cluster Issues for more information.

7. Create the cluster from the objects generated from the `dry run`.

   ```
   kubectl create -f ${CLUSTER_NAME}.yaml
   ```

   Note: A warning will appear in the console if the resource already exists, requiring you to remove the resource or update your YAML.

   Note: If you used the `--output-directory` flag in your `NKP create .. --dry-run` step above, create the cluster from the objects you created by specifying the directory:

   ```
   kubectl create -f $existing-directory/.
   ```

8. Wait for the cluster control plan to be ready.

   ```
   kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
   timeout=20m
   ```

9. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. Konvoy provides a command to describe the current status of the cluster.

   ```
   nkp describe cluster -c ${CLUSTER_NAME}
   ```

   Output:

   ```
   NAME                                                                READY  SEVERITY
     REASON  SINCE  MESSAGE
   Cluster/nutanix-e2e-cluster_name-1                                     True
               13h
   ##ClusterInfrastructure - NutanixCluster/nutanix-e2e-cluster_name-1    True
               13h
   ```

```
##ControlPlane - KubeadmControlPlane/nutanix-control-plane             True
             13h
# ##Machine/nutanix--control-plane-7llgd                               True
             13h
# ##Machine/nutanix--control-plane-vncbl                               True
             13h
# ##Machine/nutanix--control-plane-wbgrm                               True
             13h
##Workers
    ##MachineDeployment/nutanix--md-0                                  True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-67rv4                           True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-n2skc                           True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-nkftv                           True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-sqklv                           True
             13h
```

Important: If you need to increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command. See Docker Hub's rate limit.

10. Check all machines has NODE_NAME assigned.

```
kubectl get machines
```

11. Verify that the kubeadm control plane is ready with the command.

```
kubectl get kubeadmcontrolplane
```

Output is similar to:

```
NAME                                CLUSTER              INITIALIZED   API SERVER
 AVAILABLE   REPLICAS   READY   UPDATED   UNAVAILABLE   AGE   VERSION
nutanix-e2e-cluster-1-control-plane   nutanix-e2e-cluster-1   true          true
               3          3       3         0             14h   v1.29.6
```

12. Describe the kubeadm control plane and check its status and events with the command.

```
kubectl describe kubeadmcontrolplane
```

Nutanix with VPC Creating a New Cluster

Create a Nutanix Cluster in an Air-gapped environment using a Virtual Private Cloud (VPC).

About this task

If you use these instructions to create a cluster using the Nutanix Kubernetes Platform (NKP) default settings without editing configuration files or adding additional flags, your cluster is deployed with three control plane nodes and four worker nodes.

Note:

- NKP uses the Nutanix CSI driver as the default storage provider. For more information see, Default Storage Providers on page 34.
- NKP uses a CSI storage container on your Prism Element (PE). The CSI Storage Container image names must be the same for every PE environment in which you deploy an NKP cluster.

Before you begin

- Ensure you have a VPC associated with the external subnet on PCVM with connectivity outside the cluster.

  > **Note:** You must configure the default route (0.0.0.0/0) to the external subnet as the next hop for connectivity outside the cluster (north-south connectivity).

- Floating IP available for Bastion VM.
- See the topic Creating a VM through Prism Central (AHV)

  > **Note:**
  >
  > - The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.
  > - Additional steps are required before and during cluster deployment if you use VPC instead of other environments.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable for cluster name using the command `export CLUSTER_NAME=<my-nutanix-cluster>`.
3. Export Nutanix PC credentials.

   ```
   export NUTANIX_USER=$user
   export NUTANIX_PASSWORD=$password
   ```

4. Create a VM (Bastion) inside a VPC subnet where you want to create the cluster. See Creating a VM through Prism Central.
5. Assign a Floating IP with the Bastion VM. See Assigning Secondary IP Addresses to Floating IPs.
6. Download the NKP Binary, as described in Downloading NKP on page 16.
7. Complete the NKP installation, as described in Creating a Nutanix Management Cluster on page 711.
8. Select the desired scenario between accessing the cluster inside or outside the VPC subnet.

   » Inside the VPC: Proceed directly to the **Create a Kubernetes cluster** step.

   > **Note:** To access the cluster in the VPC, use the Bastion VM or any other VM in the same VPC.

   » Outside the VPC: If access is needed from outside the VPC, assign a Floating IP to the private IP used as `CONTROL_PLANE_ENDPOINT_IP` while deploying the cluster. Also, assign a different Floating IP to the internal IP used as `START_IP` of the Service Load Balancer IP Range. This floating IP reaches the cluster UI and platform applications outside the VPC. For information on Floating IPs, see the topic Request Floating IPs in Flow Virtual Networking.

9. The cluster uses two subnets, one for Pods and another for Services. Ensure these subnets do not overlap with your VM subnet. These two subnets cannot be changed after cluster creation. The default subnets used in NKP are.

   ```
   spec:
   ```

```
  clusterNetwork:
    pods:
      cidrBlocks:
      - 192.168.0.0/16
    services:
      cidrBlocks:
      - 10.96.0.0/12
```

If you need to change the Kubernetes subnets, you must do this at cluster creation. See the topic Subnets for more information.

10. Create a Kubernetes cluster using the example, populating the fields starting with `$` as required.

Note:

- If you assigned a Floating IP to the control plane endpoint private IP, you must include the Floating IP in the `--extra-sans` flag when you create the cluster. This is required for the control plane to securely serve requests from outside the VPC via the Floating IP. You may include other IPs and hostnames.
- If you assigned a Floating IP to the start IP of the Service Load Balancer IP Range, you must use the Floating IP as the argument of the `--cluster-hostname` flag when you create the cluster. This is required for the UI and platform applications to securely serve requests arriving from outside the VPC via the Floating IP.
- Before running the `nkp create cluster` command, you must first configure the Docker network correctly on the Bastion VM.
  - For cluster creation: To remove and create the `kind` Docker network, which is used for the cluster creation process.

    ```
    docker network rm kind
    docker network create -o "com.docker.network.driver.mtu=1442" kind
    ```

  - (Optional) If you are using the Bastion VM for also creating a template image: The Docker network used for `nkp create image` is different from the `kind` network. Therefore, before creating the template image, ensure you configure the Docker network for the image creation.

    ```
    docker network rm <image_network_name>
    docker network create -o
     "com.docker.network.driver.mtu=1442" <image_network_name>
    ```

    Replace *<image_network_name>* with the specific network name required for the image creation process.

  By configuring both the networks (one for the cluster creation and one for the image creation, if required), you ensure that the Bastion VM can function as intended for each specific process.

```
nkp create cluster nutanix \
  --cluster-name=$CLUSTER_NAME \
  --control-plane-replicas $CONTROLPLANE_REPLICAS \
  --self-managed \
  --worker-replicas $WORKER_REPLICAS \
  --endpoint https://$PC_NUTANIX_ENDPOINT:$NUTANIX_PORT \
  --control-plane-endpoint-ip $CONTROL_PLANE_ENDPOINT_IP \
  --control-plane-vm-image $NUTANIX_MACHINE_TEMPLATE_IMAGE_NAME \
  --control-plane-prism-element-cluster $NUTANIX_PRISM_ELEMENT_CLUSTER_NAME \
  --control-plane-subnets $NUTANIX_SUBNET_NAME \
  --worker-vm-image $NUTANIX_MACHINE_TEMPLATE_IMAGE_NAME \
  --worker-prism-element-cluster $NUTANIX_PRISM_ELEMENT_CLUSTER_NAME \
```

```
  --worker-subnets $NUTANIX_SUBNET_NAME \
  --csi-storage-container $NUTANIX_STORAGE_CONTAINER_NAME \
  --registry-url $REGISTRY_URL \
  --registry-username $REGISTRY_URL_USERNAME \
  --registry-password $REGISTRY_URL_PASSWORD \
  --kubernetes-version $K8S_VERSION \
  --kubernetes-service-load-balancer-ip-range $START_IP-$END_IP \
  --extra-sans $FLOATING_IP_ASSIGNED_TO_CONTROL_PLANE_ENDPOINT_IP \
  --cluster-hostname $FLOATING_IP_ASSIGNED_TO_START_IP \
  --dry-run \
  --output=yaml \
> ${CLUSTER_NAME}.yaml
```

11. Inspect or edit the cluster objects. Familiarize yourself with Cluster API before editing the cluster objects, as edits can prevent the cluster from deploying successfully.

    Note: If the cluster creation fails, check for issues with your environment, such as storage resources. If the cluster becomes self-managed before creation fails, you can independently investigate what is running and what has failed to resolve those issues. See Resource Requirements and Inspect Cluster Issues for more information.

12. Create the cluster from the objects generated from the `dry run`.

    Note: Creating clusters from the dry-run is reserved for advanced use cases that are not supported by the flags.

    ```
    kubectl create -f ${CLUSTER_NAME}.yaml
    ```

    Note:

    - A warning will appear in the console if the resource already exists, requiring you to remove the resource or update your YAML.
    - If you used the `--output-directory` flag in your `NKP create .. --dry-run` step above, create the cluster from the objects you created by specifying the directory:

      ```
      kubectl create -f $existing-directory/.
      ```

13. Wait for the cluster control plan to be ready.

    ```
    kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
    timeout=20m
    ```

14. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. The CLI provides a command to describe the current status of the cluster.

    ```
    nkp describe cluster -c ${CLUSTER_NAME}
    ```

    Output:

    ```
    NAME                                                                READY  SEVERITY
      REASON  SINCE  MESSAGE
    Cluster/nutanix-e2e-cluster_name-1                                     True
                13h
    ##ClusterInfrastructure - NutanixCluster/nutanix-e2e-cluster_name-1    True
                13h
    ##ControlPlane - KubeadmControlPlane/nutanix-control-plane            True
                13h
    # ##Machine/nutanix--control-plane-7llgd                               True
                13h
    ```

```
# ##Machine/nutanix--control-plane-vncbl                                  True
             13h
# ##Machine/nutanix--control-plane-wbgrm                                 True
             13h
##Workers
    ##MachineDeployment/nutanix--md-0                                    True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-67rv4                             True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-n2skc                             True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-nkftv                             True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-sqklv                             True
             13h
```

Important: If you need to increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command. See Docker Hub's rate limit.

15. If the cluster needs to be accessed from **outside** the VPC, get the `kubeconfig` of the cluster.

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

Update the kubeconfig to use the Floating IP assigned to the control plane endpoint IP to access the cluster outside of the VPC.

```
kubectl \
  --kubeconfig=${CLUSTER_NAME}.conf \
  config set clusters.${CLUSTER_NAME}.server \
  $FLOATING_IP_ASSIGNED_TO_CONTROL_PLANE_ENDPOINT_IP
```

16. Check all machines has NODE_NAME assigned.

```
kubectl get machines
```

17. Verify that the kubeadm control plane is ready with the command.

```
kubectl get kubeadmcontrolplane
```

Output is similar to:

```
NAME                                CLUSTER              INITIALIZED   API SERVER
 AVAILABLE   REPLICAS   READY   UPDATED   UNAVAILABLE   AGE   VERSION
nutanix-e2e-cluster-1-control-plane   nutanix-e2e-cluster-1   true          true
               3          3       3         0             14h   v1.29.6
```

18. Describe the kubeadm control plane and check its status and events with the command.

```
kubectl describe kubeadmcontrolplane
```

19. As they progress, the controllers also create Events, which you can list using the command

    ```
    kubectl get events | grep ${CLUSTER_NAME}
    ```

    For brevity, this example uses `grep`. You can also use separate commands to get Events for specific objects, such as `kubectl get events --field-selector involvedObject.kind="NutanixCluster`."

    By default, the control-plane Nodes will be created in 3 different zones. However, the default worker Nodes will reside in a single zone. You might create additional node pools in other zones with the `nkp create nodepool` command.

    Availability zones (AZs) are isolated locations within datacenter regions where public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single Availability Zone, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple Availability Zones.

Making the Nutanix Cluster Self-Managed
How to make a Kubernetes cluster manage itself.

About this task

NKP deploys all cluster life cycle services to a bootstrap cluster, which then deploys a workload cluster. When the workload cluster is ready, move the cluster life cycle services to the workload cluster, which makes the workload cluster self-managed.

Before you begin

Ensure you can create a workload cluster as described in the topic: Nutanix Creating a New Cluster.

This page contains instructions on how to make your cluster self-managed. This is necessary if there is only one cluster in your environment or if this cluster becomes the Management cluster in a multi-cluster environment.

> **Note:** If you already have a self-managed or Management cluster in your environment, skip this page.

Follow these steps to turn your new cluster into a **Management Cluster** for an Ultimate license environment or a free-standing Pro Cluster.

Procedure

1. Deploy cluster life cycle services on the workload cluster using the command `nkp create capi-components --kubeconfig ${CLUSTER_NAME}.conf`.

   Output example:

   ```
   # Initializing new CAPI components
   ```

   > **Note:** If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP/HTTPS Proxy.

2. The cluster life cycle services on the workload cluster are ready, but the workload cluster configuration is on the bootstrap cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the bootstrap to the workload cluster. This process is

also called a Pivot. For more information, see https://cluster-api.sigs.k8s.io/reference/glossary.html?highlight=pivot#pivot.

Move the Cluster API objects from the bootstrap to the workload cluster using the command `nkp move capi-resources --to-kubeconfig ${CLUSTER_NAME}.conf`.

Output example:

```
# Moving cluster resources
You can now view resources in the moved cluster by using the --kubeconfig flag with
 kubectl.
For example: kubectl --kubeconfig=gcp-example.conf get nodes
```

> **Note:** To ensure only one set of cluster life cycle services manages the workload cluster, NKP first pauses the reconciliation of the objects on the bootstrap cluster, then creates the objects on the workload cluster. As NKP copies the objects, the cluster life cycle services on the workload cluster reconcile the objects. The workload cluster becomes self-managed after NKP creates all the objects. If it fails, the `move` command can be safely retried.

3. Wait for the cluster control plane to be ready using the command `kubectl --kubeconfig ${CLUSTER_NAME}.conf wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --timeout=20m`

   Output example:

   ```
   cluster.cluster.x-k8s.io/gcp-example condition met
   ```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster status. After moving the cluster life cycle services to the workload cluster, remember to use NKP with the workload cluster `kubeconfig`.

   ```
   nkp describe cluster --kubeconfig ${CLUSTER_NAME}.conf -c ${CLUSTER_NAME}
   ```

   Output:

   ```
   NAME                                                                READY  SEVERITY
    REASON  SINCE  MESSAGE
   Cluster/nutanix-example-1                                           True
           13h
   ##ClusterInfrastructure - nutanixCluster/nutanix-example-1          True
           13h
   ##ControlPlane - KubeadmControlPlane/nutanix-example-control-plane  True
           13h
   # ##Machine/nutanix-example-control-plane-7llgd                     True
           13h
   # ##Machine/nutanix-example-control-plane-vncbl                     True
           13h
   # ##Machine/nutanix-example-control-plane-wbgrm                     True
           13h
   ##Workers
       ##MachineDeployment/nutanix-example-md-0                        True
           13h
       ##Machine/nutanix-example-md-0-74c849dc8c-67rv4                 True
           13h
       ##Machine/nutanix-example-md-0-74c849dc8c-n2skc                 True
           13h
       ##Machine/nutanix-example-md-0-74c849dc8c-nkftv                 True
           13h
       ##Machine/nutanix-example-md-0-74c849dc8c-sqklv                 True
           13h
   ```

5. Remove the bootstrap cluster because the workload cluster is now self-managed using the command `nkp delete bootstrap --kubeconfig $HOME/.kube/config`.

```
nkp delete bootstrap --kubeconfig $HOME/.kube/config
# Deleting bootstrap cluster
```

Known Limitations

Procedure

- NKP only supports moving all namespaces in the cluster; NKP does not support migration of individual namespaces.
- Konvoy supports moving only one set of cluster objects from the bootstrap cluster to the workload cluster or vice-versa.

Verifying the Nutanix Install and Log in to the UI

Verify your Kommander Install and Log in to the Dashboard UI.

About this task

After you create the cluster, you can verify that installation of the UI and platform applications is complete using the following command. This command waits up to 15 minutes for installation to complete.

Procedure

You can check the status of the installation using the command `kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m`.

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using the command `nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf`.
2. Retrieve your credentials at any time if necessary, using the command `kubectl -n kommander get secret nkp-credentials -o go-template='Username: {{.data.username|base64decode}} {{ "\n"}}Password: {{.data.password|base64decode}}{{ "\n"}}'`
3. Retrieve the URL used for accessing the UI sing the command `kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/nkp/kommander/dashboard{{ "\n"}}'`.
   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.
   a. Rotate the password using the command `nkp experimental rotate dashboard-password`.
      Example output displaying the new password:
      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Nutanix Installation in an Air-Gapped Environment

This installation provides instructions on how to install Nutanix Kubernetes Platform (NKP) in a vSphere air-gapped environment.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47
- Nutanix Prerequisites

Note:

- For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

  ```
  tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
  ```

- Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

  For example:

  ```
  export TMPDIR=/path/to/your/directory
  ```

Further Prerequisites

In an environment with access to the Internet, you retrieve artifacts from specialized repositories dedicated to them, such as Docker images contained in DockerHub and Helm Charts that come from a dedicated Helm Chart repository. However, in an air-gapped environment, you need local repositories to store Helm charts, Docker images, and other artifacts. Tools such as JFrog, Harbor, and Nexus handle multiple types of artifacts in one local repository.

Nutanix Air-gapped Environment Loading the Registry

Before creating an air-gapped Kubernetes cluster, you must load the required images in a local registry for the Konvoy component.

About this task

The complete Nutanix Kubernetes Platform (NKP) air-gapped bundle is needed for an air-gapped environment but can also be used in a non-air-gapped environment. The bundle contains all the NKP components needed for an air-gapped environment installation and a local registry in a non-air-gapped environment.

Note: If you do not already have a local registry set up, see the Local Registry Tools page for more information.

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images, is required. This registry must be accessible from both the bastion machine and other machines that will be created for the Kubernetes cluster.

Procedure

1. If not already done in prerequisites, download the air-gapped bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
   ```

2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. EX: For the bootstrap cluster, change your directory to the `nkp-<version>` directory, similar to the example below, depending on your current location

   ```
   cd nkp-v2.12.0
   ```

3. Set an environment variable with your registry address and any other needed variables using this command.

   ```
   export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
   export REGISTRY_USERNAME=<username>
   ```

```
export REGISTRY_PASSWORD=<password>
export REGISTRY_CA=<path to the cacert file on the bastion>
```

- `REGISTRY_URL`: the address of an existing registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.
- `REGISTRY_CA`: (optional) the path on the bastion machine to the registry CA. Konvoy will configure the cluster nodes to trust this CA. This value is only needed if the registry uses a self-signed certificate and the images are not already configured to trust this CA.
- `REGISTRY_USERNAME`: optional, set to a user with pull access to this registry.
- `REGISTRY_PASSWORD`: optional if username is not set.

4. Execute the following command to load the air-gapped image bundle into your private registry using any relevant flags to apply variables from Step 3.

```
nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-
registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-
password=${REGISTRY_PASSWORD}
```

> **Note:** It may take some time to push all the images to your image registry, depending on the network performance of the machine you are running the script on and the registry.

> **Important:** To increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

5. Load the Kommander component images to your private registry using the command.

```
nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-
registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-
password=${REGISTRY_PASSWORD}
```

Optional: This step is required only if you have an Ultimate license.

For NKP Catalog Applications available with the Ultimate license, perform this image load by running the following command to load the `dkp-catalog-applications` image bundle into your private registry:

```
nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-
v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME}
 --to-registry-password=${REGISTRY_PASSWORD}
```

6. On the Bastion, load the Konvoy bootstrap image, using either the Docker or Podman command.

Docker example:

```
docker load -i konvoy-bootstrap-image-v2.12.0.tar
```

Podman example:

```
podman load -i konvoy-bootstrap-image-v2.12.0.tar
podman image tag konvoy-bootstrap:2.12.0 docker.io/konvoy-bootstrap:v2.12.0
```

Bootstrapping Air-gapped Nutanix

To create Kubernetes clusters, NKP uses Cluster API (CAPI) controllers. These controllers run on a Kubernetes cluster.

About this task

To get started, you need a bootstrap cluster. By default, Nutanix Kubernetes Platform (NKP) creates a bootstrap cluster for you in a Docker container using the Kubernetes-in-Docker (KIND) tool.

Before you begin

Procedure

1. Complete the Nutanix Infrastructure Prerequisites. For more information, see Nutanix Infrastructure Prerequisites on page 690.
2. Ensure the `NKP` binary can be found in your `$PATH`.
3. Decide your Base OS image selection. See BaseOS Image Requirements in the Prerequisites section.

Bootstrap Cluster Life Cycle Services

Procedure

1. Review Universal Configurations for all Infrastructure Providers regarding settings, flags, and other choices and then begin bootstrapping.
2. Create a bootstrap cluster using the command `nkp create bootstrap --kubeconfig $HOME/.kube/config`.

   > **Note:** Use `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Configuring an HTTP or HTTPS Proxy on page 677.

   Example output:

   ```
   # Creating a bootstrap cluster
   # Initializing new CAPI components
   ```

3. NKP creates a bootstrap cluster using KIND as a library.

   For more information, see https://github.com/kubernetes-sigs/kind.

4. NKP then deploys the following Cluster API providers on the cluster.

   - Core Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/
   - AWS Infrastructure Provider: https://github.com/kubernetes-sigs/cluster-api-provider-aws
   - Kubeadm Bootstrap Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/bootstrap/kubeadm
   - Kubeadm ControlPlane Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/controlplane/kubeadm

   For more information on Cluster APIs, see https://cluster-api.sigs.k8s.io/.

5. Ensure that the Cluster API Provider Nutanix Cloud Infrastructure (CAPX) controllers are present using the command `kubectl get pods -n capx-system`.

   Output example:

   ```
   NAME                                       READY   STATUS    RESTARTS   AGE
   capx-controller-manager-785c5978f-nnfns    1/1     Running   0          13h
   ```

6. NKP waits until these providers' controller-manager and webhook deployments are ready. List these deployments using the command `kubectl get --all-namespaces deployments -l=clusterctl.cluster.x-k8s.io.`

   Output example:

```
NAMESPACE                           NAME
 READY   UP-TO-DATE   AVAILABLE   AGE
capa-system                         capa-controller-manager
 1/1     1            1           1h
capg-system                         capg-controller-manager
 1/1     1            1           1h
capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-manager
 1/1     1            1           1h
capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-manager
 1/1     1            1           1h
capi-system                         capi-controller-manager
 1/1     1            1           1h
cappp-system                        cappp-controller-manager
 1/1     1            1           1h
capv-system                         capv-controller-manager
 1/1     1            1           1h
capx-system                         capx-controller-manager
 1/1     1            1           1h
capz-system                         capz-controller-manager
 1/1     1            1           1h
cert-manager                        cert-manager
 1/1     1            1           1h
cert-manager                        cert-manager-cainjector
 1/1     1            1           1h
cert-manager                        cert-manager-webhook
 1/1     1            1           1h
```

Nutanix Air-gapped Environment Creating a New Cluster

Create a Nutanix Cluster in an air-gapped environment.

About this task

If you use these instructions to create a cluster using the Nutanix Kubernetes Platform (NKP) default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 22.04 operating system image with three control plane nodes, and four worker nodes.

By default, the control-plane Nodes will be created in 3 different zones. However, the default worker Nodes will reside in a single zone. You maymight create additional node pools in other zones with the `nkp create nodepool` command.

Availability zones (AZs) are isolated locations within datacenter regions where public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single Availability Zone, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple Availability Zones.

> **Note:** NKP uses the Nutanix CSI driver as the default storage provider. For more information see, Default Storage Providers on page 34.

> **Note:** NKP uses a CSI storage container on your Prism Element (PE). The CSI Storage Container image names must be the same for every PE environment in which you deploy an NKP cluster.

Before you begin

- Ensure you have Nutanix Air-gapped: Loading the Registry on page 64.

- Named your cluster.

  Note: The cluster name might only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable for cluster name using the command `export CLUSTER_NAME=<my-nutanix-cluster>`.
3. Export Nutanix PC credentials.

   ```
   export NUTANIX_USER=$user
   export NUTANIX_PASSWORD=$password
   ```

4. Ensure your subnets do not overlap with your host subnet because they cannot be changed after cluster creation. The default subnets used in NKP are.

   ```
   spec:
     clusterNetwork:
       pods:
         cidrBlocks:
         - 192.168.0.0/16
       services:
         cidrBlocks:
         - 10.96.0.0/12
   ```

   If you need to change the kubernetes subnets, you must do this at cluster creation. See the topic Subnets for more information.

   » (Optional) Modify Control Plane Audit logs - Users can modify the `KubeadmControlplane` cluster-API object to configure different `kubelet` options. If you need to configure your control plane beyond the existing options available from flags, see Configuring your control plane.

   » (Optional) Determine what VPC Network to use. Nutanix accounts have a default preconfigured VPC Network, which will be used if you do not specify a different network. To use a different VPC network for your cluster, create one by following these instructions for Create and Manage VPC Networks. Then select the `--network <new_vpc_network_name>` option on the create cluster command below.

5. Create a Kubernetes cluster. The following example shows a common configuration.

   ```
   nkp create cluster nutanix \
       --cluster-name=$CLUSTER_NAME \
       --control-plane-prism-element-cluster=$PE_NAME \
       --worker-prism-element-cluster=$PE_NAME \
       --control-plane-subnets=$SUBNET_ASSOCIATED_WITH_PE \
       --worker-subnets=$SUBNET_ASSOCIATED_WITH_PE \
       --control-plane-endpoint-ip=$AVAILABLE_IP_FROM_SAME_SUBNET \
       --csi-storage-container=$NAME_OF_YOUR_STORAGE_CONTAINER \
       --endpoint=$PC_ENDPOINT_URL \
       --control-plane-vm-image=$NAME_OF_OS_IMAGE_CREATED_BY_NKP_CLI\
       --worker-vm-image=$NAME_OF_OS_IMAGE_CREATED_BY_NKP_CLI \
       --registry-mirror-url=${REGISTRY_URL} \
       --registry-mirror-username=${REGISTRY_USERNAME} \
       --registry-mirror-password=${REGISTRY_PASSWORD} \
       --kubernetes-service-load-balancer-ip-range $START_IP-$END_IP \
       --airgapped \
       --self-managed \
   ```

```
    --dry-run \
    --output=yaml \
> ${CLUSTER_NAME}.yaml
```

6. Inspect or edit the cluster objects. Familiarize yourself with Cluster API before editing the cluster objects, as edits can prevent the cluster from deploying successfully.

   Note: If the cluster creation fails, check issues with your environment such as storage resources. If the cluster becomes self-managed before it stalls, you can investigate what is running and what has failed to try to resolve those issues independently. See Resource Requirements and Inspect Cluster Issues for more information.

7. Create the cluster from the objects generated from the `dry run`.

   ```
   kubectl create -f ${CLUSTER_NAME}.yaml
   ```

   Note: A warning will appear in the console if the resource already exists, requiring you to remove the resource or update your YAML.

   Note: If you used the `--output-directory` flag in your `NKP create .. --dry-run` step above, create the cluster from the objects you created by specifying the directory:

   ```
   kubectl create -f $existing-directory/.
   ```

8. Wait for the cluster control plan to be ready.

   ```
   kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
   timeout=20m
   ```

9. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. Konvoy provides a command to describe the current status of the cluster.

   ```
   nkp describe cluster -c ${CLUSTER_NAME}
   ```

   Output:

   ```
   NAME                                                                READY  SEVERITY
     REASON  SINCE  MESSAGE
   Cluster/nutanix-e2e-cluster_name-1                                     True
                13h
   ##ClusterInfrastructure - NutanixCluster/nutanix-e2e-cluster_name-1    True
                13h
   ##ControlPlane - KubeadmControlPlane/nutanix-control-plane             True
                13h
   # ##Machine/nutanix--control-plane-7llgd                               True
                13h
   # ##Machine/nutanix--control-plane-vncbl                               True
                13h
   # ##Machine/nutanix--control-plane-wbgrm                               True
                13h
   ##Workers
       ##MachineDeployment/nutanix--md-0                                  True
                13h
       ##Machine/nutanix--md-0-74c849dc8c-67rv4                          True
                13h
       ##Machine/nutanix--md-0-74c849dc8c-n2skc                          True
                13h
       ##Machine/nutanix--md-0-74c849dc8c-nkftv                          True
                13h
   ```

```
##Machine/nutanix--md-0-74c849dc8c-sqklv                              True
         13h
```

Important: If you need to increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command. See Docker Hub's rate limit.

10. Check all machines has NODE_NAME assigned.

```
kubectl get machines
```

11. Verify that the kubeadm control plane is ready with the command.

```
kubectl get kubeadmcontrolplane
```

Output is similar to:

```
NAME                                CLUSTER              INITIALIZED   API SERVER
 AVAILABLE   REPLICAS   READY   UPDATED   UNAVAILABLE   AGE   VERSION
nutanix-e2e-cluster-1-control-plane   nutanix-e2e-cluster-1   true          true
               3          3       3         0             14h   v1.29.6
```

12. Describe the kubeadm control plane and check its status and events with the command.

```
kubectl describe kubeadmcontrolplane
```

13. As they progress, the controllers also create Events, which you can list using the command

```
kubectl get events | grep ${CLUSTER_NAME}
```

For brevity, this example uses `grep`. You can also use separate commands to get Events for specific objects, such as `kubectl get events --field-selector involvedObject.kind="NutanixCluster`."

Nutanix with VPC Creating a New Air-gapped Cluster

Create a Nutanix Cluster in an Air-gapped environment using a Virtual Private Cloud (VPC).

About this task

If you use these instructions to create a cluster using the Nutanix Kubernetes Platform (NKP) default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 22.04 operating system image with three control plane nodes, and four worker nodes.

Note: NKP uses the Nutanix CSI driver as the default storage provider. For more information see, Default Storage Providers on page 34.

Note: NKP uses a CSI storage container on your Prism Element (PE). The CSI Storage Container image names must be the same for every PE environment in which you deploy an NKP cluster.

Before you begin

- Ensure you have a VPC associated with the external subnet on PCVM with connectivity outside the cluster.

  Note: You must configure the default route (0.0.0.0/0) to the external subnet as the next hop for connectivity outside the cluster (north-south connectivity).

- Floating IP available for Bastion VM.
- See the topic Creating a VM through Prism Central (AHV)

- Ensure you have Nutanix Air-gapped: Loading the Registry on page 64.
- Named your cluster.

  Note: The cluster name might only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

  Note: Additional steps are required before and during cluster deployment if you use VPC instead of other environments.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable for cluster name using the command `export CLUSTER_NAME=$my-nutanix-cluster`.
3. Export Nutanix PC credentials.

   ```
   export NUTANIX_USER=$user
   export NUTANIX_PASSWORD=$password
   ```

4. Create a VM(Bastion) inside a VPC subnet where you want to create the cluster. See Creating a VM through Prism Central.
5. Associate a Floating IP with the Bastion VM. See Assigning Secondary IP Addresses to Floating IPs.
6. Upload the NKP package onto the Bastion VM.
7. Extract the package onto the Bastion VM: `tar -xzvf nkp-bundle_v2.12.0_linux_amd64.tar.gz`.
8. Select the desired scenario between accessing the cluster inside or outside the VPC subnet.

   » Inside the VPC: Proceed directly to the **Create a Kubernetes cluster** step.

   Note: To access the cluster in the VPC, use the Bastion VM or any other VM in the same VPC.

   » Outside the VPC: If access is needed from outside the VPC, link the floating IP to an internal IP used as `CONTROL_PLANE_ENDPOINT_IP` while deploying the cluster. For information on Floating IP, see the topic Request Floating IPs in Flow Virtual Networking.

   Note: Access the cluster in the VPC from outside using updated `kubeconfig` after creating the cluster.

   Note: To access the UI outside the VPC, you need to request three floating IPs.

   - One IP for the bastion
   - One IP for passing the `--extra-sans` flag during cluster creation
   - One IP for the UI

9. Ensure your subnets do not overlap with your host subnet because they cannot be changed after cluster creation. The default subnets used in NKP are.

   ```
   spec:
     clusterNetwork:
       pods:
         cidrBlocks:
   ```

```
      - 192.168.0.0/16
    services:
      cidrBlocks:
      - 10.96.0.0/12
```

If you need to change the kubernetes subnets, you must do this at cluster creation. For more information, see the topic Subnets .

10. Create a Kubernetes cluster using the example, populating the fields starting with `$` as required.

    Note: Use the second floating IP from step 8 for passing as `--extra-sans`.

```
nkp create cluster nutanix \
  --cluster-name=$CLUSTER_NAME \
  --control-plane-replicas $CONTROLPLANE_REPLICAS \
  --self-managed \
  --worker-replicas $WORKER_REPLICAS \
  --endpoint https://$PC_NUTANIX_ENDPOINT:$NUTANIX_PORT \
  --control-plane-endpoint-ip $CONTROL_PLANE_ENDPOINT_IP \
  --control-plane-vm-image $NUTANIX_MACHINE_TEMPLATE_IMAGE_NAME \
  --control-plane-prism-element-cluster $NUTANIX_PRISM_ELEMENT_CLUSTER_NAME \
  --control-plane-subnets $NUTANIX_SUBNET_NAME \
  --worker-vm-image $NUTANIX_MACHINE_TEMPLATE_IMAGE_NAME \
  --worker-prism-element-cluster $NUTANIX_PRISM_ELEMENT_CLUSTER_NAME \
  --worker-subnets $NUTANIX_SUBNET_NAME \
  --csi-storage-container $NUTANIX_STORAGE_CONTAINER_NAME \
  --registry-url $REGISTRY_URL \
  --registry-username $REGISTRY_URL_USERNAME \
  --registry-password $REGISTRY_URL_PASSWORD \
  --kubernetes-version $K8S_VERSION \
  --kubernetes-service-load-balancer-ip-range $START_IP-$END_IP \
  --extra-sans $FLOATING_IP
  --dry-run \
  --output=yaml \
> ${CLUSTER_NAME}.yaml
```

11. Inspect or edit the cluster objects. Familiarize yourself with Cluster API before editing the cluster objects, as edits can prevent the cluster from deploying successfully.

    Note: If the cluster creation fails, check issues with your environment such as storage resources. If the cluster becomes self-managed before it stalls, you can investigate what is running and what has failed to try to resolve those issues independently. See Resource Requirements and Inspect Cluster Issues for more information.

12. Create the cluster from the objects generated from the `dry run`.

```
kubectl create -f ${CLUSTER_NAME}.yaml
```

    Note: A warning will appear in the console if the resource already exists, requiring you to remove the resource or update your YAML.

    Note: If you used the `--output-directory` flag in your `NKP create .. --dry-run` step above, create the cluster from the objects you created by specifying the directory:

```
kubectl create -f $existing-directory/.
```

13. Wait for the cluster control plane to be ready.

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=20m
```

14. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. Konvoy provides a command to describe the current status of the cluster.

```
nkp describe cluster -c ${CLUSTER_NAME}
```

Output:

```
NAME                                                                READY  SEVERITY
  REASON  SINCE  MESSAGE
Cluster/nutanix-e2e-cluster_name-1                                    True
             13h
##ClusterInfrastructure - NutanixCluster/nutanix-e2e-cluster_name-1    True
             13h
##ControlPlane - KubeadmControlPlane/nutanix-control-plane            True
             13h
# ##Machine/nutanix--control-plane-7llgd                              True
             13h
# ##Machine/nutanix--control-plane-vncbl                              True
             13h
# ##Machine/nutanix--control-plane-wbgrm                              True
             13h
##Workers
    ##MachineDeployment/nutanix--md-0                                 True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-67rv4                          True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-n2skc                          True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-nkftv                          True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-sqklv                          True
             13h
```

> **Important:** If you need to increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command. See Docker Hub's rate limit.

15. If the cluster needs to be accessed from **outside** the VPC, get the `kubeconfig` of the cluster.

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

Update the field to the Floating IP in order to access the cluster outside of the VPC.

```
server: https://ControlPlane_EndPoint:6443
```

in `${CLUSTER_NAME}.conf` where `CONTROL_PLANE_ENDPOINT_IP` is to be replaced with FloatingIP passed as `--extra-sans` IP during k8s cluster creation.

16. Check all machines has NODE_NAME assigned.

```
kubectl get machines
```

17. Verify that the kubeadm control plane is ready with the command.

```
kubectl get kubeadmcontrolplane
```

Output is similar to:

```
NAME                                CLUSTER              INITIALIZED   API SERVER
 AVAILABLE   REPLICAS   READY   UPDATED   UNAVAILABLE   AGE   VERSION
nutanix-e2e-cluster-1-control-plane   nutanix-e2e-cluster-1   true          true
               3          3       3         0             14h   v1.29.6
```

18. Describe the kubeadm control plane and check its status and events with the command.

    ```
    kubectl describe kubeadmcontrolplane
    ```

19. As they progress, the controllers also create Events, which you can list using the command

    ```
    kubectl get events | grep ${CLUSTER_NAME}
    ```

    For brevity, this example uses `grep`. You can also use separate commands to get Events for specific objects, such as `kubectl get events --field-selector involvedObject.kind="NutanixCluster`."

    By default, the control-plane Nodes will be created in 3 different zones. However, the default worker Nodes will reside in a single zone. You might create additional node pools in other zones with the `nkp create nodepool` command.

    Availability zones (AZs) are isolated locations within datacenter regions where public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single Availability Zone, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple Availability Zones.

Making the Nutanix Air-gapped Cluster Self-Managed
How to make a Kubernetes cluster manage itself.

About this task

Nutanix Kubernetes Platform (NKP) deploys all cluster life cycle services to a bootstrap cluster, which then deploys a workload cluster. When the workload cluster is ready, move the cluster life cycle services to the workload cluster, which makes the workload cluster self-managed.

Before you begin

Ensure you can create a workload cluster as described in the topic: Nutanix Air-gapped Creating a New Cluster.

This page contains instructions on how to make your cluster self-managed. This is necessary if there is only one cluster in your environment or if this cluster becomes the Management cluster in a multi-cluster environment.

> **Note:** If you already have a self-managed or Management cluster in your environment, skip this page.

Follow these steps to turn your new cluster into a **Management Cluster** for an Ultimate license environment (or a free-standing Pro Cluster):

Procedure

1. Deploy cluster life cycle services on the workload cluster.

   ```
   nkp create capi-components --kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Initializing new CAPI components
   ```

   > **Note:** If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP/HTTPS Proxy.

2. The cluster life cycle services on the workload cluster are ready, but the workload cluster configuration is on the bootstrap cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the bootstrap to the workload cluster. This process is

called a Pivot. For more information, see https://cluster-api.sigs.k8s.io/reference/glossary.html?highlight=pivot#pivot.

Move the Cluster API objects from the bootstrap to the workload cluster:

```
nkp move capi-resources --to-kubeconfig ${CLUSTER_NAME}.conf
```

Output:

```
# Moving cluster resources
You can now view resources in the moved cluster by using the --kubeconfig flag with
 kubectl. For example: kubectl --kubeconfig=gcp-example.conf get nodes
```

> **Note:** To ensure only one set of cluster life cycle services manages the workload cluster, NKP first pauses the reconciliation of the objects on the bootstrap cluster, then creates the objects on the workload cluster. As NKP copies the objects, the cluster life cycle services on the workload cluster reconcile the objects. The workload cluster becomes self-managed after NKP creates all the objects. If it fails, the `move` command can be safely retried.

3. Wait for the cluster control plane to be ready.

```
kubectl --kubeconfig ${CLUSTER_NAME}.conf wait --for=condition=ControlPlaneReady
 "clusters/${CLUSTER_NAME}" --timeout=20m
```

Output:

```
cluster.cluster.x-k8s.io/gcp-example condition met
```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster status. After moving the cluster life cycle services to the workload cluster, remember to use NKP with the workload cluster `kubeconfig`.

```
nkp describe cluster --kubeconfig ${CLUSTER_NAME}.conf -c ${CLUSTER_NAME}
```

Output:

```
NAME                                                                READY  SEVERITY
 REASON  SINCE  MESSAGE
Cluster/nutanix-example-1                                           True
        13h
##ClusterInfrastructure - nutanixCluster/nutanix-example-1          True
        13h
##ControlPlane - KubeadmControlPlane/nutanix-example-control-plane  True
        13h
# ##Machine/nutanix-example-control-plane-7llgd                     True
        13h
# ##Machine/nutanix-example-control-plane-vncbl                     True
        13h
# ##Machine/nutanix-example-control-plane-wbgrm                     True
        13h
##Workers
    ##MachineDeployment/nutanix-example-md-0                        True
        13h
    ##Machine/nutanix-example-md-0-74c849dc8c-67rv4                 True
        13h
    ##Machine/nutanix-example-md-0-74c849dc8c-n2skc                 True
        13h
    ##Machine/nutanix-example-md-0-74c849dc8c-nkftv                 True
        13h
    ##Machine/nutanix-example-md-0-74c849dc8c-sqklv                 True
        13h
```

5. Remove the bootstrap cluster because the workload cluster is now self-managed.

```
nkp delete bootstrap --kubeconfig $HOME/.kube/config
```

```
# Deleting bootstrap cluster
```

Known Limitations

Procedure

- NKP only supports moving all namespaces in the cluster; NKP does not support migration of individual namespaces.
- Konvoy supports moving only one set of cluster objects from the bootstrap cluster to the workload cluster or vice-versa.

Installing Kommander in a Nutanix Air-gapped Environment

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped environment.

About this task

Once you have installed the Konvoy component of Nutanix Kubernetes Platform (NKP), you will continue installing the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a time period (for example, 1 hour) to allocate more time to deploy applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. If required: Customize your `kommander.yaml`.

   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.

5. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `dkp-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.nutanix.io/project-default-catalog-repository: "true"
        kommander.nutanix.io/workspace-default-catalog-repository: "true"
        kommander.nutanix.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

6. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Enable NKP Catalog Applications after Installing NKP.

Verifying the Nutanix Air-gapped Install and Log in to the UI

Verify your Kommander Install and Log in to the Dashboard UI.

About this task

After you build the Konvoy cluster and install the Kommander component for the UI, you can verify your installation. It waits for all applications to be ready by default.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/chartmuseum condition met
helmrelease.helm.toolkit.fluxcd.io/cluster-observer-2360587938 condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/gatekeeper condition met
helmrelease.helm.toolkit.fluxcd.io/gatekeeper-proxy-mutations condition met
helmrelease.helm.toolkit.fluxcd.io/karma-traefik-certs condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-operator condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-ui condition met
helmrelease.helm.toolkit.fluxcd.io/kube-oidc-proxy condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-traefik-certs condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-traefik-certs condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

By default, you can log in to the UI in Kommander with the credentials given using this command.

```
nkp get dashboard --kubeconfig=${CLUSTER_NAME}.conf
```

Nutanix Management Tools

After cluster creation and configuration, you can revisit clusters to update and change variables.

Configuring Nutanix Cluster Autoscaler

This page explains how to configure autoscaler for node pools.

About this task

Cluster Autoscaler can automatically scale up or down the number of worker nodes in a cluster based on the number of pending pods to be scheduled. Running the Cluster Autoscaler is optional. Unlike Horizontal-Pod Autoscaler, Cluster Autoscaler does not depend on any Metrics server and does not need Prometheus or any other metrics source.

The Cluster Autoscaler looks at the following annotations on a MachineDeployment to determine its scale-up and scale-down ranges:

> **Note:**
>
> ```
> cluster.x-k8s.io/cluster-api-autoscaler-node-group-min-size
> cluster.x-k8s.io/cluster-api-autoscaler-node-group-max-size
> ```

The full list of command line arguments to the Cluster Autoscaler controller is on the Kubernetes public GitHub repository.

For more information on how Cluster Autoscaler works, see these documents:

- What is Cluster Autoscaler
- How does scale-up work
- How does scale-down work
- CAPI Provider for Cluster Autoscaler

Before you begin

Ensure you have the following:

- A bootstrap cluster life cycle: Bootstrapping Nutanix on page 710
- Created a new Kubernetes Cluster.
- A Self-Managed Cluster.

Run Cluster Autoscaler on the Management Cluster

Procedure

1. Locate the machinedeployment for the worker nodes you want to adjust your auto-scaling. You can retrieve the name of your nodepool by running the command `nkp get nodepool --cluster-name ${CLUSTER_NAME}`.

   Example:

   ```
   nkp get nodepool --cluster-name ${CLUSTER_NAME}
   ```

   Locate the nodepool name under `spec.topology.workers.machineDeployments.metadata.name`, Select the nodepool you want to scale.

2. To enable autoscaling on your Nutanix cluster, edit the cluster object using the command `kubectl`.

   Example:

   ```
   kubectl edit cluster ${CLUSTER_NAME}
   ```

3. Adjust the annotation for `cluster.x-k8s.io/cluster-api-autoscaler-node-group-max-size` or `cluster.x-k8s.io/cluster-api-autoscaler-node-group-min-size` to your desired numbers, and then save the file.

4. After adjust it, it will resemble the following:

   ```
   spec:
     topology:
       workers:
         machineDeployments:
         - class: default-worker
           metadata:
             annotations:
               cluster.x-k8s.io/cluster-api-autoscaler-node-group-max-size: "7"
               cluster.x-k8s.io/cluster-api-autoscaler-node-group-min-size: "3"
           name: md-0
   ```

Configuring NTP on Nutanix NKP Clusters

You can configure Network Time Protocol (NTP) on NKP clusters to keep the system's time synchronized across all the nodes. Configure Network Time Protocol (NTP) servers on Nutanix NKP clusters if:

- You cannot use the public NTP servers set by default in the operating system.
- You prefer to use your own custom NTP server in non-air-gapped environments.

- You cannot configure NTP servers due to a cloud-init bug for Rocky Linux 9.4.

You can configure NTP on Nutanix NKP clusters in the following ways:

Creating a DaemonSet:

You can use a DaemonSet to automatically update and configure NTP servers on all the nodes, even when new nodes are added or existing ones are updated, or removed. The DaemonSet method updates the NTP Servers on the underlying nodes dynamically. It is recommended you use the DaemonSet method because it automates the NTP configuration and is reliable after upgrading NKP. For more information, see Configuring NTP using an in-cluster DaemonSet on page 739.

Modifying the KubeadmConfigTemplate:

You can use KubeadmConfigTemplate to configure NTP if you need advanced customization. However, there is a risk that your configured templates might be overwritten when upgrading NKP. If the customized NTP settings are overwritten, you will need to reconfigure the settings after the upgrade.. If you do not want to give DaemonSet access - to your node filesystem, you can use this procedure. For more information, see Configuring NKP using the KubeadmConfig Template on page 741.

Configuring NTP using an in-cluster DaemonSet

A DaemonSet ensures that all nodes in the system always have the correct NTP server settings when new nodes are added or existing ones are updated, or removed.

About this task

It is recommended that you deploy this DaemonSet to ensure you can update the NTP Servers on the underlying nodes dynamically. The DaemonSet ensures that each node is configured with the NTP servers defined in the ConfigMap.

For example, deploying this DaemonSet will update the NTP Servers on the underlying nodes dynamically when new nodes are added or nodes rotated during an upgrade.

Procedure

1. Create the ServiceAccount file named serviceaccount.yaml:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: ntp-updater-sa
  namespace: kube-system
```

   This creates an account called `ntp-updater-sa` to give the program the necessary permissions to run on the system.

2. Create the ConfigMap file named configmap.yaml:

   A ConfigMap holds the list of NTP servers such as `time.aws.com` and `time.google.com` that the system should use. These servers will be applied to all the nodes.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ntp-servers-config
  namespace: kube-system
data:
  ntp-servers: |
    time.aws.com
    time.google.com
```

3. Deploy a DaemonSet named daemonset.yaml which runs on every node in the cluster.

   - The DaemonSet runs a script that reads the list of NTP servers from the ConfigMap, checks the configuration file chrony.conf on each node, and adds the servers if they're not already there.
   - If a new NTP server is added, it restarts the NTP service on the node to apply the changes.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: ntp-updater
  namespace: kube-system
  annotations:
    configmap.reloader.stakater.com/reload: "ntp-servers-config"
spec:
  selector:
    matchLabels:
      app: ntp-updater
  template:
    metadata:
      labels:
        app: ntp-updater
    spec:
      serviceAccountName: ntp-updater-sa
      hostPID: true
      tolerations:
      - key: "node-role.kubernetes.io/master"
        operator: "Exists"
        effect: "NoSchedule"
      - key: "node-role.kubernetes.io/control-plane"
        operator: "Exists"
        effect: "NoSchedule"
      containers:
      - name: ntp-updater
        image: docker.io/library/busybox:1
        securityContext:
          privileged: true
        volumeMounts:
        - name: host-etc
          mountPath: /host/etc
        - name: ntp-config
          mountPath: /config
        command: ["/bin/sh"]
        args:
          - "-c"
          - |
            #!/bin/sh
            set -e

            NTP_CONFIG="/config/ntp-servers"
            HOST_CHRONY_CONF="/host/etc/chrony.conf"

            if [ ! -f "$HOST_CHRONY_CONF" ]; then
              echo "Chrony configuration not found on host."
              exit 1
            fi

            # Read existing server entries from chrony.conf
            EXISTING_SERVERS=$(grep "^server " "$HOST_CHRONY_CONF" | awk '{print
 $2}')

            # Process each server from the ConfigMap
```

```
            while IFS= read -r server_name; do
              if [ -n "$server_name" ]; then
                # Construct the server directive
                SERVER_DIRECTIVE="server $server_name iburst"

                # Check if the server is already configured
                if echo "$EXISTING_SERVERS" | grep -Fxq "$server_name"; then
                  echo "NTP server already exists: $server_name"
                else
                  echo "$SERVER_DIRECTIVE" >> "$HOST_CHRONY_CONF"
                  echo "Added NTP server: $server_name"
                  NEED_RESTART=true
                fi
              fi
            done < "$NTP_CONFIG"

            if [ "$NEED_RESTART" = true ]; then
              nsenter --target 1 --mount --pid --ipc -- systemctl restart chronyd ||
echo "Failed to restart chronyd"
            fi

            # Keep the container running
            tail -f /dev/null
      volumes:
      - name: host-etc
        hostPath:
          path: /etc
      - name: ntp-config
        configMap:
          name: ntp-servers-config
```

- This creates a pod that runs on every node. The pod checks the ConfigMap for the NTP server list and ensures that each node gets the correct configuration.
- The DaemonSet uses the `ServiceAccount` created in Step 1 and mounts the necessary files to update the NTP configuration on each node.
- The pod checks if the node's configuration is missing any servers and adds them if needed. If the pod adds any new servers, the pod restarts the `chronyd` service which handles time syncing to apply the changes.

4. After you apply these configurations, a DaemonSet automatically runs on every node. The DaemonSet updates the NTP settings by adding code similar to the following example to each node's configuration file /etc/chrony.conf:

   ```
   … # Other config
   server time.aws.com iburst
   server time.google.com iburst
   ```

   The `/etc/chrony.conf` file on each node will have the correct NTP servers listed, ensuring the nodes keep time synchronized. Therefore, all nodes get the same time-sync settings, even if new nodes are added or old ones are updated.

Configuring NKP using the KubeadmConfig Template

You can use the Cluster API which manages Kubernetes clusters to configure NKP settings during node creation.

About this task

Cluster API-managed Kubernetes clusters have the following templates:

- KubeadmControlPlaneTemplate to define the configuration for control plane nodes.
- KubeadmConfigTemplate to define the configuration for worker nodes.

You can configure these templates to customize NTP settings or include NTP settings before deploying new nodes. When you create new control plane nodes or worker nodes, the settings might be lost after an NKP upgrade.

Before you begin

The Cluster API method using KubeadmConfigTemplate requires modifying resources that were created by Cluster API operators using the ClusterClass of the Nutanix NKP clusters.

These resources get overwritten during an NKP upgrade and the custom configuration is not carried over.

Procedure

1. In a Cluster API-managed Kubernetes cluster, modify the control plane template.

   The KubeadmControlPlaneTemplate is used to define the control plane nodes' configuration.

   a. Edit KubeadmControlPlaneTemplate.

   b. Search for kubeadmConfigSpec.

   c. Specify the NTP settings in the `kubeadmConfigSpec.ntp` field.

   d. Add the NTP servers under `kubeadmConfigSpec.ntp`:

   ```
   apiVersion: controlplane.cluster.x-k8s.io/v1beta1
   kind: KubeadmControlPlaneTemplate
   metadata:
     name: my-cluster-control-plane
   spec:
     template
       spec:
         kubeadmConfigSpec:
           ntp:
             enabled: true
             servers:
             - time.aws.com
             - time.google.com
       # ... other configurations ...
   ```

   The new control plane nodes are rolled out with the desired NTP servers configured during the next cluster reconciliation.

2. In a Cluster API-managed Kubernetes cluster, modify the template of the worker node.

   The KubeadmConfigTemplate defines the worker nodes' configuration. Extract the current configuration, edit it to add NTP servers, then reapply.

   a. Create a copy of the existing KubeadmConfigTemplate:

   ```
   $ kubectl get kubeadmconfigtemplate my-cluster-worker -o yaml > kct.yaml
   ```

   b. Edit the new KubeadmConfigTemplate.

   c. Specify a new name.

   d. Specify the NTP settings in the `spec.ntp` field:

   ```
   apiVersion: bootstrap.cluster.x-k8s.io/v1beta1
   kind: KubeadmConfigTemplate
   metadata:
   ```

```
  name: my-cluster-worker-v2
spec:
  template:
    spec:
      ntp:
        enabled: true
        servers:
          - time.aws.com
          - time.google.com
      # ... other configurations ...
```

e. Save the changes.

f. Reapply the template. Apply the new KubeadmConfigTemplate:

```
$ kubectl apply -f kct.yaml
```

g. Edit the `spec.bootstrap.configRef` of worker machine deployment

```
apiVersion: cluster.x-k8s.io/v1beta1
kind: MachineDeployment
metadata:
  …
  name: nkp-sid-md-0-8dzhf
  namespace: default
spec:
  …
  template:
    …
    spec:
      …
      bootstrap:
        configRef:
          apiVersion: bootstrap.cluster.x-k8s.io/v1beta1
          kind: KubeadmConfigTemplate
          name: nkp-sid-m1d-0-7zvbs-v2
          namespace: default
```

The worker node deployment is configured to use the updated template so that new nodes use the new NTP configuration.

After updating the MachineDeployment, the new worker nodes are rolled out with the desired NTP servers configured.

Pre-provisioned Infrastructure

Configuration types for installing the Nutanix Kubernetes Platform (NKP) on a Pre-provisioned Infrastructure.

Create a Kubernetes cluster on pre-provisioned nodes in a bare metal infrastructure.

The following procedure describes creating an NKP cluster on a pre-provisioned infrastructure using SSH. For more information on a Pre-provisioned environment, see Pre-provisioned Infrastructure on page 23

Completing this procedure results in a Kubernetes cluster that includes a Container Networking Interface (CNI) and a Local Persistence Volume Static Provisioner that is ready for workload deployment.

Before moving to a production environment, you might add applications for logging and monitoring, storage, security, and other functions. You can use NKP to select and deploy applications or deploy your own. For more information, see Deploying Platform Applications Using CLI on page 403.

For more information, see:

- Container Networking Interface (CNI): https://docs.projectcalico.org/

- Local Persistence Volume Static Provisioner: https://github.com/kubernetes-sigs/sig-storage-local-static-provisioner

Pre-provisioned Prerequisites and Environment Variables

Pre-provisioning is the process of setting up an environment that authorized users, devices, and servers can access. Network provisioning primarily concerns connectivity and security, which means a heavy focus on device and identity management. Pre-provisioning can bring enterprises greater efficiency and more secure operations.

A cloud-based or on-premises server must first be provisioned with the correct data, software, and configuration to function on a network.

The steps in this process typically include:

- Installing an operating system, device drivers, and partitioning and setup tools
- Installing enterprise software and applications
- Setting parameters such as IP addresses
- Performing partitioning or installation of virtualization software
- Connectivity, whether an air-gapped or non-air-gapped environment, meaning it is connected to the internet

Environment prerequisites and configuration are found on these pages before installation begins.

Pre-provisioned Prerequisite Configuration

Infrastructure and machine requirements will be required to fulfill all the prerequisites for a successful implementation in a Pre-provisioned environment. Read all the sections on this page to ensure you have met all prerequisites. Before you begin using Nutanix Kubernetes Platform, you must have the following set:

- An x86_64-based Linux or macOS machine.
- The `nkp` binary for Linux or macOS.
- kubectl for interacting with the running cluster.
- Pre-provisioned hosts with SSH access enabled.
- An unencrypted SSH private key, whose public key is configured on the above hosts.
- A Container engine/runtime installed is required to install NKP and bootstrap:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
- To use a local registry, whether air-gapped or non-air-gapped environment, download and extract the bundle. Download the Complete NKP Air-gapped Bundle for this release (for example, `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`) to load the registry.
- For an air-gapped environment, create a working registry.
  - local registry on bastion or other machine
- Resource requirements

- Pre-provisioned Override Files

When in an air-gapped environment, you must also follow the steps described in the Air-gapped Define Environment and Docker Registry as a prerequisite.

NKP uses `localvolumeprovisioner` as the default storage provider. However, `localvolumeprovisioner` is not suitable for production use. Use a Kubernetes CSI compatible storage that is suitable for production.

You can choose from any of the storage options available for Kubernetes. To disable the default that Konvoy deploys, set the default StorageClass `localvolumeprovisioner` as non-default. Then, set your newly created StorageClass as the default by following the commands in the Kubernetes documentation called Changing the Default Storage Class.

Machine Specifications

You need to have at least three **Control Plane Machines**.

Each control plane machine must have the following:

- 4 cores
- 16 GB memory
- Approximately 80 GB of free space for the volume used for `/var/lib/kubelet` and `/var/lib/containerd`.
- 15% free space on the root file system.
- Multiple ports open, as described in NKP Ports.
- `firewalld` systemd service disabled. If it exists and is enabled, use the commands `systemctl stop firewalld` then `systemctl disable firewalld`, so that `firewalld` remains disabled after the machine restarts.
- For a Pre-provisioned environment using Ubuntu 20.04, ensure the machine has the `/run` directory mounted with exec permissions.

> **Note:** Swap is disabled. The `kubelet` does not have generally available support for swap. Due to variable commands, refer to your operating system documentation.

Worker Machines

You need to have at least four worker machines. The specific number of worker machines required for your environment can vary depending on the cluster workload and size of the machines.

Each worker machine must have the following:

- 8 cores
- 32 GiB memory
- Around 80 GiB of free space for the volume used for `/var/lib/kubelet` and `/var/lib/containerd`
- 15% free space on the root file system
- If you plan to use local volume provisioning to provide persistent volumes for your workloads, you must mount at least four volumes to the `/mnt/disks/` mount point on each machine. Each volume must have at least 55 GiB of capacity.
- Ensure your disk meets the resource requirements for Rook Ceph in `Block` mode for ObjectStorageDaemons as specified in the requirements table.
- Multiple ports open, as described in NKP Ports.

- `firewalld` systemd service disabled. If it exists and is enabled, use the commands `systemctl stop firewalld` then `systemctl disable firewalld`, so that `firewalld` remains disabled after the machine restarts.
- For a Pre-provisioned environment using Ubuntu 20.04, ensure the machine has the `/run` directory mounted with exec permissions.

  Note: Swap is disabled. The `kubelet` does not generally support swap. Due to variable commands, refer to your operating system documentation.

Defining Cluster Hosts and Infrastructure
Define the cluster's control plane and worker nodes.

About this task

The Konvoy component of Nutanix Kubernetes Platform (NKP) must know how to access your cluster hosts, so you must define the cluster hosts and infrastructure. This is done using inventory resources. For initial cluster creation, you must define a control-plane and at least one worker pool for air-gapped and non-air-gapped environments.

Complete the steps to set the necessary environment variables and specify the control plane and worker nodes:

Procedure

1. Export the following environment variables, ensuring that all control plane and worker nodes are included:

   ```
   export CONTROL_PLANE_1_ADDRESS="<control-plane-address-1>"
   export CONTROL_PLANE_2_ADDRESS="<control-plane-address-2>"
   export CONTROL_PLANE_3_ADDRESS="<control-plane-address-3>"
   export WORKER_1_ADDRESS="<worker-address-1>"
   export WORKER_2_ADDRESS="<worker-address-2>"
   export WORKER_3_ADDRESS="<worker-address-3>"
   export WORKER_4_ADDRESS="<worker-address-4>"
   export SSH_USER="<ssh-user>"
   export SSH_PRIVATE_KEY_SECRET_NAME="$CLUSTER_NAME-ssh-key"
   ```

   The environment variables you set in this step automatically replace the variable names when the inventory YAML file is created.

2. Use the following template to define your infrastructure.

   ```
   cat <<EOF > preprovisioned_inventory.yaml
   ---
   apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
   kind: PreprovisionedInventory
   metadata:
     name: $CLUSTER_NAME-control-plane
     namespace: default
     labels:
       cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
       clusterctl.cluster.x-k8s.io/move: ""
   spec:
     hosts:
       # Create as many of these as needed to match your infrastructure
       # Note that the command line parameter --control-plane-replicas determines how
    many control plane nodes will actually be used.
       #
       - address: $CONTROL_PLANE_1_ADDRESS
       - address: $CONTROL_PLANE_2_ADDRESS
   ```

```
    - address: $CONTROL_PLANE_3_ADDRESS
  sshConfig:
    port: 22
    # This is the username used to connect to your infrastructure. This user must be
 root or
    # have the ability to use sudo without a password
    user: $SSH_USER
    privateKeyRef:
      # This is the name of the secret you created in the previous step. It must
 exist in the same
      # namespace as this inventory object.
      name: $SSH_PRIVATE_KEY_SECRET_NAME
      namespace: default
---
apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
kind: PreprovisionedInventory
metadata:
  name: $CLUSTER_NAME-md-0
  namespace: default
  labels:
    cluster.x-k8s.io/cluster-name: $CLUSTER_NAME
    clusterctl.cluster.x-k8s.io/move: ""
spec:
  hosts:
    - address: $WORKER_1_ADDRESS
    - address: $WORKER_2_ADDRESS
    - address: $WORKER_3_ADDRESS
    - address: $WORKER_4_ADDRESS
  sshConfig:
    port: 22
    user: $SSH_USER
    privateKeyRef:
      name: $SSH_PRIVATE_KEY_SECRET_NAME
      namespace: default
EOF
```

3. To tell the bootstrap cluster which nodes you want to be control plane nodes and which nodes are worker nodes. Apply the file to the bootstrap cluster using the command `kubectl apply -f preprovisioned_inventory.yaml`.

   Example:

```
preprovisionedinventory.infrastructure.cluster.konvoy.d2iq.io/preprovisioned-example-
control-plane created
preprovisionedinventory.infrastructure.cluster.konvoy.d2iq.io/preprovisioned-example-
md-0 created
```

What to do next

Pre-provisioned Cluster Creation Customization Choices

Loading the Registry for an Air-gapped Kubernetes Cluster

Before creating an Air-gapped Kubernetes cluster, you must load the required images in a local registry for the Konvoy component.

About this task

The complete Nutanix Kubernetes Platform (NKP) air-gapped bundle is needed for an air-gapped environment but can also be used in a non-air-gapped environment. The bundle contains all the NKP components needed for an air-gapped environment installation and a local registry in a non-air-gapped environment.

This registry must be accessible from the bastion machine and the AWS EC2 instances (if deploying to AWS) or other machines that will be created for the Kubernetes cluster.

> **Note:** If you do not already have a local registry set up, see the Local Registry Tools page for more information.

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images, is required. This registry must be accessible from the bastion machine and the AWS EC2 instances (if deploying to AWS) or other machines that will be created for the Kubernetes cluster.

Procedure

1. If not already done in prerequisites, download the air-gapped bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tar file to a local directory using the command `tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`.

2. After extraction, you can access files from different directories using the command `nkp-<version>`.

   The following example shows the `nkp-<version>` command to change the directory.

   ```
   cd nkp-v2.12.0
   ```

3. Set an environment variable with your registry address and any other needed variables using this command.

   ```
   export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
   export REGISTRY_USERNAME=<username>
   export REGISTRY_PASSWORD=<password>
   export REGISTRY_CA=<path to the cacert file on the bastion>
   ```

4. To load the air-gapped image bundle into your private registry using any of the relevant flags to apply variables above use the command `nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}`

   > **Note:** It might take some time to push all the images to your image registry, depending on the network's performance between the machine you are running the script on and the registry.

   > **Important:** To increase Docker Hub's rate limit, use your credentials to create the cluster by setting the `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` flag when using the command `nkp create cluster`.

5. Load the Kommander component images to your private registry using the command `nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}`.

   Optional step required only if you have an Ultimate license: Load the image bundle into your private registry using the command `dkp-catalog-applications`.

   ```
   Example:
   nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME}
    --to-registry-password=${REGISTRY_PASSWORD}
   ```

Replacing the Pre-provisioned Driver with the Azure Disk CSI Driver

After your bootstrap is running and your cluster is created, you will need to install the Azure Disk CSI Driver on your pre-provisioned Azure Kubernetes cluster.

About this task

The Nutanix Kubernetes Platform (NKP) Pre-provisioned provider installs by default the storage-local-static-provisioner CSI driver, which is not suitable for production environments. For this reason, it needs to be replaced by the Azure Disk CSI Driver.

Before you begin

- An x86_64-based Linux or macOS machine.
- Download the `nkp` binary for Linux or macOS. To check which version of NKP you installed for compatibility reasons, run the `nkp version` command.
- A Container engine or runtime installed is required to install NKP and bootstrap:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
- CLI tool Kubectl is used to interact with the running cluster. https://kubernetes.io/docs/tasks/tools/#kubectl
- Azure CLI. For more information, see https://docs.microsoft.com/en-us/cli/azure/install-azure-cli.
- A valid Azure account with credentials configured. For more information, see https://github.com/kubernetes-sigs/cluster-api-provider-azure/blob/master/docs/book/src/topics/getting-started.md#prerequisites.
- Create a custom Azure image using KIB. For more information, see Using KIB with Azure on page 1097.
- For air-gapped environments only:
  - Ability to download artifacts from the internet and then copy those onto your Bastion machine.
  - Download the Complete NKP Air-gapped Bundle for this release - `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`.
  - An existing local registry to seed the air-gapped environment. For more information, see Registry and Registry Mirrors on page 682.

Procedure

1. Log in to Azure using the command `az login`.
   Example output:

```
[
  {
    "cloudName": "AzureCloud",
    "homeTenantId": "a1234567-b132-1234-1a11-1234a5678b90",
    "id": "b1234567-abcd-11a1-a0a0-1234a5678b90",
    "isDefault": true,
    "managedByTenants": [],
    "name": "Mesosphere Developer Subscription",
    "state": "Enabled",
    "tenantId": "a1234567-b132-1234-1a11-1234a5678b90",
    "user": {
      "name": "user@azuremesosphere.onmicrosoft.com",
```

```
      "type": "user"
    }
  }
]
```

2. Create an Azure Service Principal (SP) by using the command `az ad sp create-for-rbac --role contributor --name "$(whoami)-konvoy" --scopes=/subscriptions/$(az account show --query id -o tsv)`.

   This command will rotate the password if an SP with the name exists.

   Example output:

```
{
  "appId": "7654321a-1a23-567b-b789-0987b6543a21",
  "displayName": "azure-cli-2021-03-09-23-17-06",
  "password": "Z79yVstq_E.R0R7RUUck718vEHSuyhAB0C",
  "tenant": "a1234567-b132-1234-1a11-1234a5678b90"
}
```

   - For air-gapped environments, you need to create a resource management private link with a private endpoint to ensure the Azure CSI driver will run correctly in further steps. Private links enable you to access Azure services over a private endpoint in your virtual network. For more information, see https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/create-private-link-access-portal.

     To set up a private link resource, use the following process.

     1. Create a private resource management link using Azure CLI. For more information, see https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/create-private-link-access-commands?tabs=azure-cli#create-resource-management-private-link
     2. Create a private link association for the root management group, which also references the resource ID for the resource management private link. For more information, see https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/create-private-link-access-commands?tabs=azure-cli#create-private-link-association.
     3. Add a private endpoint referencing the resource management private link using the Azure Documentation. For more information, see https://learn.microsoft.com/en-us/azure/private-link/create-private-endpoint-cli?tabs=dynamic-ip.

3. Set the required environment variables.

   Example:

```
export AZURE_SUBSCRIPTION_ID="<id>"       # b1234567-abcd-11a1-a0a0-1234a5678b90
export AZURE_TENANT_ID="<tenant>"         # a1234567-b132-1234-1a11-1234a5678b90
export AZURE_CLIENT_ID="<appId>"          # 7654321a-1a23-567b-b789-0987b6543a21
export AZURE_CLIENT_SECRET="<password>"   # Z79yVstq_E.R0R7RUUck718vEHSuyhAB0C
export AZURE_RESOURCE_GROUP="<resource group name>" # set to the name of the
 resource group
export AZURE_LOCATION="westus"            # set to the location you are using
```

4. Set your KUBECONFIG environment variable using the command `export kubeconfig=${CLUSTER_NAME}.conf`

5. Create the Secret with the Azure credentials. The Azure CSI driver will use this.

   a. Create an `azure.json` file.

```
cat <<EOF > azure.json
{
  "cloud": "AzurePublicCloud",
  "tenantId": "$AZURE_TENANT_ID",
  "subscriptionId": "$AZURE_SUBSCRIPTION_ID",
```

```
  "aadClientId": "$AZURE_CLIENT_ID",
  "aadClientSecret": "$AZURE_CLIENT_SECRET",
  "resourceGroup": "$AZURE_RESOURCE_GROUP",
  "location": "$AZURE_LOCATION"
}
EOF
```

b. Create the Secret using the command `kubectl create secret generic azure-cloud-provider --namespace=kube-system --type=Opaque --from-file=cloud-config=azure.json`.

6. Install the Azure Disk CSI driver using the command `$ curl -skSL https://raw.githubusercontent.com/kubernetes-sigs/azuredisk-csi-driver/v1.26.2/deploy/install-driver.sh | bash -s v1.26.2 snapshot –`.

7. Check the status to see if the driver is ready for use using the command `kubectl -n kube-system get pod -o wide --watch -l app=csi-azuredisk-controller kubectl -n kube-system get pod -o wide --watch -l app=csi-azuredisk-node`.
Kubernetes knows this is Azure disk and will create clusters on Azure.

8. Create the StorageClass for the Azure Disk CSI Driver using the command `kubectl create -f https://raw.githubusercontent.com/kubernetes-sigs/azuredisk-csi-driver/master/deploy/example/storageclass-azuredisk-csi.yaml`.

9. Change the default storage class to this new StorageClass so that every new disk will be created in the Azure environment using the command `kubectl patch sc/localvolumeprovisioner -p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-default-class":"false"}}}' kubectl patch sc/managed-csi -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class":"true"}}}'`

10. Verify that the StorageClass chosen is currently the default using the command `kubectl get storageclass`.

What to do next

For more information about Azure Disk CSI for persistent storage and changing the default StorageClass, see Default Storage Providers in NKP.

Pre-provisioned Cluster Creation Customization Choices

Below are two methods to customize your cluster during creation. If none of these choices apply, proceed to the next section.

- Pre-provisioned Install in a Non-air-gapped Environment
- Pre-provisioned Install in an Air-gapped Environment

Pre-provisioned Section Topics

Many options are available when creating clusters, such as those listed in this documentation section. A brief explanation of each choice is given in the following topic summaries with a link to the more descriptive page. To use these, proceed to the cluster choice page for detailed instructions:

- **Pre-provisioned Customizing CAPI Clusters** : Familiarize yourself with the Cluster API before editing the cluster objects, as edits can prevent the cluster from deploying successfully.
- **Pre-provisioned Registry Mirrors**: In an air-gapped environment, you need a local repository to store Helm charts, Docker images, and other artifacts. In an environment with access to the Internet, you can retrieve artifacts from specialized repositories dedicated to them, such as Docker images contained in DockerHub and Helm Charts that come from a dedicated Helm Chart repository.

- **Pre-provisioned Create Secrets and Overrides**: Create necessary secrets and overrides for pre-provisioned clusters. Most applications deployed through Kubernetes https://kubernetes.io/docs/concepts/configuration/secret/ require access to databases, services, and other external resources. The easiest way to manage the login information necessary to access those resources is using secrets, which help organize and distribute sensitive information across a cluster while minimizing the risk of sensitive information exposure.
- **Pre-provisioned Define Control Plane Endpoint**: A control plane needs to have three, five, or seven nodes to remain available if one, two, or three nodes fail. A control plane with one node is not for production use.
- **Pre-provisioned Configure MetalLB**: An external load balancer (LB) is recommended to be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines responding to API requests. If you do not have one, you can use Metal LB to create a MetalLB configmap for your Pre-provisioned infrastructure.
- **Pre-provisioned Modify the Calico Installation**: Calico is a networking and security solution that enables Kubernetes and non-Kubernetes/legacy workloads to communicate seamlessly and securely. Sometimes, changes are needed, so use the information on this Pre-provisioned Modify the Calico Installation page.
- **Pre-provisioned Built-in Virtual IP**: As explained in Define the Control Plane Endpoint, we recommend using an external load balancer for the control plane endpoint but provide a built-in virtual IP when one is not available.
- **Pre-provisioned Use HTTP Proxy**: When you require HTTP proxy configurations, you can apply them during the create operation by adding the appropriate flags to the nkp create cluster command.
- **Pre-provisioned Use Alternate Pod or Service Subnets**: Some subnets are reserved by Kubernetes and can prevent proper cluster deployment if you unknowingly configure NKP so that the Node subnet collides with either the Pod or Service subnet.
- **Pre-provisioned Output Directory YAML**: You can create individual files with different smaller manifests for ease in editing using the `--output-directory` flag used with `--output=json|yaml`. You create the directory where resources are outputted to files.

Single-Node Control Plane

> **Caution:** Do not use a single-node control plane in a production cluster.

A control plane with one node can use its single node as the endpoint, so you will not require an external load balancer or a built-in virtual IP. At least one control plane node must always be running. Therefore, a spare machine must be available in the control plane inventory to upgrade a cluster with one control plane node. This machine is used to provision the new node before the old node is deleted.

When the API server endpoints are defined, you can create the cluster.

> **Note:** For more information on modifying Control Plane Audit logs settings, see Configuring the Control Plane.

Pre-provisioned Define Control Plane Endpoint

Define the control plane endpoint for your cluster and the connection mechanism. A control plane must have three, five, or seven nodes to remain available if one or more nodes fail. A control plane with one node is not for production use.

In addition, the control plane should have an endpoint that remains available if some nodes fail.

```
-------- cp1.example.com:6443
```

```
      |
      lb.example.com:6443 ---------- cp2.example.com:6443
      |
      -------- cp3.example.com:6443
```

In this example, the control plane endpoint host is `lb.example.com`, and the control plane endpoint port is `6443`. The control plane nodes are `cp1.example.com`, `cp2.example.com`, and `cp3.example.com`. The port of each API server is `6443`.

Select your Connection Mechanism

A virtual IP is the client's address to which to connect to the service. A load balancer is a device that distributes the client connections to the backend servers. Before you create a new Nutanix Kubernetes Platform (NKP) cluster, choose an external load balancer (LB) or virtual IP.

- **External load balancer**

It is recommended that an external load balancer be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines responding to API requests.

- **Built-in virtual IP**

You can use the built-in virtual IP if an external load balancer is unavailable. The virtual IP is not a load balancer; it does not distribute request load among the control plane machines. However, if the machine receiving requests does not respond, the virtual IP automatically moves to another machine.

Single-Node Control Plane

> **Caution:** Do not use a single-node control plane in a production cluster.

A control plane with one node can use its single node as the endpoint, so you will not require an external load balancer or a built-in virtual IP. At least one control plane node must always be running. Therefore, a spare machine must be available in the control plane inventory to upgrade a cluster with one control plane node. This machine is used to provision the new node before the old node is deleted. When the API server endpoints are defined, you can create the cluster using the link in the Next Step below.

> **Note:** Modify Control Plane Audit log settings using the information on the page Configure the Control Plane.

Known Limitations

The control plane endpoint port is also used as the API server port on each control plane machine. The default port is 6443. Before creating the cluster, ensure the port is available on each control plane machine.

Pre-provisioned Configure MetalLB

Create a MetalLB configmap for your Pre-provisioned Infrastructure.

Nutanix recommends that an external load balancer (LB) be the control plane endpoint. To distribute request load among the control plane machines, configure the load balancer to send requests to all the control plane machines. Configure the load balancer to send requests only to control plane machines responding to API requests. If you do not have one, you can use Metal LB to create a MetalLB configmap for your Pre-provisioned infrastructure.

Choose one of the two protocols you want to use to announce service IPs. If your environment is not currently equipped with a load balancer, you can use MetalLB, a load balancer implementation for Kubernetes. Otherwise, your load balancer will work, and you can continue the installation process with Pre-provisioned: Install Kommander. To use MetalLB, create a MetalLB configMap for your Pre-provisioned infrastructure. MetalLB uses one of two protocols to expose Kubernetes services.

Select one of the following procedures to create your MetalLB manifest for further editing:

- Layer 2, with Address Resolution Protocol (ARP)
- Border Gateway Protocol (BGP)

Layer 2 Configuration

Layer 2 mode is the simplest to configure: in many cases, you don't need any protocol-specific configuration, only IP addresses. It does not require the IPs to be bound to the network interfaces of your worker nodes. It responds to ARP requests on your local network directly and gives clients the machine's MAC address.

- MetalLB IP address ranges or CIDRs must be within the node's primary network subnet. For more information, see Managing Subnets and Pods on page 684.
- MetalLB IP address ranges, CIDRs, and node subnets must not conflict with the Kubernetes cluster pod and service subnets.

For example, the following configuration gives MetalLB control over IPs from 192.168.1.240 to 192.168.1.250 and configures Layer 2 mode:

The following values are generic; enter your specific values into the fields where applicable.

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
  config: |
    address-pools:
    - name: default
      protocol: layer2
      addresses:
      - 192.168.1.240-192.168.1.250
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

BGP Configuration

For a basic configuration featuring one BGP router and one IP address range, you need four pieces of information:

- The router IP address that MetalLB needs to connect to.
- The router's autonomous systems (AS) number.
- The AS number MetalLB is to be used.
- An IP address range is a Classless Inter-Domain Routing (CIDR) prefix.

As an example, if you want to give MetalLB the range 192.168.10.0/24 and AS number 64500 and connect it to a router at 10.0.0.1 with AS number 64501, your configuration will look like this:

```
cat << EOF > metallb-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system
  name: config
data:
```

```
  config: |
    peers:
    - peer-address: 10.0.0.1
      peer-asn: 64501
      my-asn: 64500
    address-pools:
    - name: default
      protocol: bgp
      addresses:
      - 192.168.10.0/24
EOF
```

```
kubectl apply -f metallb-conf.yaml
```

Pre-provisioned Built-in Virtual IP

As explained in Pre-provisioned Define Control Plane Endpoint on page 752, we recommend using an external load balancer for the control plane endpoint but provide a built-in virtual IP when an external load balancer is unavailable. If an external load balancer is unavailable, use the built-in virtual IP.

The virtual IP is not a load balancer; it does not distribute request load among the control plane machines. However, if the machine receiving requests does not respond to them, the virtual IP automatically moves to another machine. The built-in virtual IP uses the kube-vip project. To use the virtual IP, add these flags to the `create cluster` command:

Table 58: Create Cluster Flags

Virtual IP Configuration	Flag
Network interface to use for Virtual IP. It must exist on all control plane machines.	--virtual-ip-interface string
IPv4 address. Reserved for use by the cluster.	--control-plane-endpoint string

Virtual IP Example

```
nkp create cluster preprovisioned \
    --cluster-name ${CLUSTER_NAME} \
    --control-plane-endpoint-host 196.168.1.10 \
    --virtual-ip-interface eth1
    --pre-provisioned-inventory-file preprovisioned_inventory.yaml
    --ssh-private-key-file <path-to-ssh-private-key>
    --self-managed
```

For more information on kube-vip, see https://kube-vip.io/

Creating Secrets and Overrides

Create necessary secrets and overrides for pre-provisioned clusters.

About this task

Most applications deployed through Kubernetes require external access to databases, services, and other resources. The easiest way to manage the login information necessary to access those resources is by using secrets to help organize and distribute sensitive information across a cluster while minimizing the risk of sensitive information exposure.

Nutanix Kubernetes Platform (NKP) requires SSH access to your infrastructure with superuser privileges. You must provide an unencrypted SSH private key to NKP , so secrets are a good way to achieve this. Populate the key and create the required secret on your bootstrap cluster using the following procedure.

Before you begin

Give your cluster a unique name suitable for your environment.

> **Note:** The cluster name might only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

Procedure

Set the environment variable to be used throughout this procedure using the command `export CLUSTER_NAME=<preprovisioned-example>`

(Optional) If you want to create a unique cluster name using the command `export CLUSTER_NAME=preprovisioned-example-$(LC_CTYPE=C tr -dc 'a-z0-9' </dev/urandom | fold -w 5 | head -n1)echo $CLUSTER_NAME`.

> **Note:** This creates a unique name every time you run it, so use it carefully.

Create a Secret

About this task

Create a secret that contains the SSH key.

Procedure

1. Export the key using the command `export SSH_PRIVATE_KEY_FILE="<path-to-ssh-private-key>"`
2. Export the secret using the command `export SSH_PRIVATE_KEY_SECRET_NAME=$CLUSTER_NAME-ssh-key`.
3. Create the secret using the command `kubectl create secret generic ${SSH_PRIVATE_KEY_SECRET_NAME} --from-file=ssh-privatekey=${SSH_PRIVATE_KEY_FILE} kubectl label secret ${SSH_PRIVATE_KEY_SECRET_NAME} clusterctl.cluster.x-k8s.io/move=`.

   Example output:

   ```
   secret/preprovisioned-example-ssh-key created
   secret/preprovisioned-example-ssh-key labeled
   ```

Create Overrides

About this task

In these steps, you will point your machines at the desired Registry to obtain the container images. If your pre-provisioned machines need to have Custom Override Files, create a secret that includes all the overrides you want to provide in one file.

Procedure

1. Example CentOS7 and Docker - If you want to provide an override with Docker credentials and a different source for EPEL on a CentOS7 machine, create a file like this.

   ```
   cat > overrides.yaml << EOF
   image_registries_with_auth:
   - host: "registry-1.docker.io"
     username: "my-user"
     password: "my-password"
     auth: ""
   ```

```
  identityToken: ""

epel_centos_7_rpm: https://my-rpm-repostory.org/epel/epel-release-latest-7.noarch.rpm
EOF
```

You can then create the related secret by using the command `kubectl create secret generic $CLUSTER_NAME-user-overrides --from-file=overrides.yaml=overrides.yaml kubectl label secret $CLUSTER_NAME-user-overrides clusterctl.cluster.x-k8s.io/move=`.

2. When using Oracle 7 OS, you might wish to deploy the RHCK kernel instead of the default UEK kernel. To do so, add the following text to your `overrides.yaml`.

```
cat > overrides.yaml << EOF
---
oracle_kernel: RHCK
EOF
```

You can then create the related secret by using the command `kubectl create secret generic $CLUSTER_NAME-user-overrides --from-file=overrides.yaml=overrides.yaml kubectl label secret $CLUSTER_NAME-user-overrides clusterctl.cluster.x-k8s.io/move=`.

Creating FIPS Secrets and Overrides

Create necessary secrets and overrides for pre-provisioned clusters using FIPS.

About this task

Most applications deployed through Kubernetes require external access to databases, services, and other resources. The easiest way to manage the login information necessary to access those resources is by using secrets to help organize and distribute sensitive information across a cluster while minimizing the risk of sensitive information exposure.

Nutanix Kubernetes Platform (NKP) requires SSH access to your infrastructure with superuser privileges. You must provide an unencrypted SSH private key to NKP , so secrets are a good way to achieve this. Populate the key and create the required secret on your bootstrap cluster using the following procedure.

Before you begin

Give your cluster a unique name suitable for your environment.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. For more naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

Procedure

Set the environment variable to be used throughout this procedure using the command `export CLUSTER_NAME=<preprovisioned-example>`

(Optional) If you want to create a unique cluster name, use the command. `export CLUSTER_NAME=preprovisioned-example-$(LC_CTYPE=C tr -dc 'a-z0-9' </dev/urandom | fold -w 5 | head -n1) echo $CLUSTER_NAME`.

> **Note:** This creates a unique name every time you run it, so use it carefully.

Create a Secret

About this task

Create a secret that contains the SSH key.

Procedure

1. Export the key using the command `export SSH_PRIVATE_KEY_FILE="<path-to-ssh-private-key>"` .
2. Export the secret using the command
   `export SSH_PRIVATE_KEY_SECRET_NAME=$CLUSTER_NAME-ssh-key`
3. Create the secret using the command `kubectl create secret generic ${SSH_PRIVATE_KEY_SECRET_NAME} --from-file=ssh-privatekey=${SSH_PRIVATE_KEY_FILE} kubectl label secret ${SSH_PRIVATE_KEY_SECRET_NAME} clusterctl.cluster.x-k8s.io/move=`.
   Example output:

   ```
   secret/preprovisioned-example-ssh-key created
   secret/preprovisioned-example-ssh-key labeled
   ```

   > **Note:** Konvoy Image Builder (KIB) can produce images containing FIPS-140 compliant binaries. Use the `fips.yaml` FIPS Override Non-air-gapped files provided with the image bundles. To locate the available Override files in the Konvoy Image Builder repo, see https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides.

Create Overrides

About this task

In these steps, you will point your machines at the desired Registry to obtain the container images. If your pre-provisioned machines need Custom Override Files, create a secret that includes all the overrides you want to provide in one file.

Procedure

1. Create a secret that includes the customization Overrides for FIPS compliance.

   ```
   cat > overrides.yaml << EOF
   ---
   k8s_image_registry: docker.io/mesosphere

   fips:
     enabled: true

   build_name_extra: -fips
   kubernetes_build_metadata: fips.0
   default_image_repo: hub.docker.io/mesosphere
   kubernetes_rpm_repository_url: "https://packages.nutanix.com/konvoy/stable/linux/repos/el/kubernetes-v{{ kubernetes_version }}-fips/x86_64"
   docker_rpm_repository_url: "\
     https://containerd-fips.s3.us-east-2.amazonaws.com\
     /{{ ansible_distribution_major_version|int }}\
     /x86_64"
   EOF
   ```

2. If your pre-provisioned machines need customization with alternate package libraries, Docker image or other container registry image repos, or other Custom Override Files, add more lines to the same Overrides file.

   a. Example One - If you want to provide an override with Docker credentials and a different source for EPEL on a CentOS7 machine, create a file like this.

```
cat > overrides.yaml << EOF
---
# fips configuration
k8s_image_registry: docker.io/mesosphere

fips:
  enabled: true

build_name_extra: -fips
kubernetes_build_metadata: fips.0
default_image_repo: hub.docker.io/mesosphere
kubernetes_rpm_repository_url: "https://packages.nutanix.com/konvoy/stable/linux/
repos/el/kubernetes-v{{ kubernetes_version }}-fips/x86_64"
docker_rpm_repository_url: "\
  https://containerd-fips.s3.us-east-2.amazonaws.com\
  /{{ ansible_distribution_major_version|int }}\
  /x86_64"

# custom configuration
image_registries_with_auth:
- host: "registry-1.docker.io"
  username: "my-user"
  password: "my-password"
  auth: ""
  identityToken: ""

epel_centos_7_rpm: https://my-rpm-repostory.org/epel/epel-release-
latest-7.noarch.rpm
EOF
```

   b. Example Two - When using Oracle 7 OS, you may wish to deploy the RHCK kernel instead of the default UEK kernel. To do so, add the following text to your `overrides.yaml`.

```
cat > overrides.yaml << EOF
---
# fips configuration
k8s_image_registry: docker.io/mesosphere

fips:
  enabled: true

build_name_extra: -fips
kubernetes_build_metadata: fips.0
default_image_repo: hub.docker.io/mesosphere
kubernetes_rpm_repository_url: "https://packages.nutanix.com/konvoy/stable/linux/
repos/el/kubernetes-v{{ kubernetes_version }}-fips/x86_64"
docker_rpm_repository_url: "\
  https://containerd-fips.s3.us-east-2.amazonaws.com\
  /{{ ansible_distribution_major_version|int }}\
  /x86_64"

# custom configuration
oracle_kernel: RHCK
```

```
EOF
```

3. Create the related secret by using the command `kubectl create secret generic $CLUSTER_NAME-user-overrides --from-file=overrides.yaml=overrides.yaml kubectl label secret $CLUSTER_NAME-user-overrides clusterctl.cluster.x-k8s.io/move=`.

Modifying the Calico Installation

Calico is a networking and security solution that enables Kubernetes workloads and non-Kubernetes/legacy workloads to communicate seamlessly and securely. Sometimes changes are needed, so use the information in this Pre-provisioned Modify the Calico Installation page.

About this task

By default, Calico automatically detects the IP address to use for each node using the `first-found` method. This is not always appropriate for your particular nodes. In that case, you must modify Calico's configuration to use a different method. An alternative is to use the `interface` method by providing the interface ID.

Set the Interface

> **Note:** Azure does not set the interface. Proceed to **Change the Encapsulation Type** section below.

Procedure

1. Follow the steps outlined in this section to modify Calico's configuration. In this example, all cluster nodes use ens192 as the interface name. Get the pods running on your cluster with this command.

```
kubectl get pods -A --kubeconfig ${CLUSTER_NAME}.conf
```

Output:

```
NAMESPACE               NAME
        READY   STATUS            RESTARTS        AGE
calico-system           calico-kube-controllers-57fbd7bd59-vpn8b
        1/1     Running           0               16m
calico-system           calico-node-5tbvl
        1/1     Running           0               16m
calico-system           calico-node-nbdwd
        1/1     Running           0               4m40s
calico-system           calico-node-twl6b
        0/1     PodInitializing   0               9s
calico-system           calico-node-wktkh
        1/1     Running           0               5m35s
calico-system           calico-typha-54f46b998d-52pt2
        1/1     Running           0               16m
calico-system           calico-typha-54f46b998d-9tzb8
        1/1     Running           0               4m31s
default                 cuda-vectoradd
        0/1     Pending           0               0s
kube-system             coredns-78fcd69978-frwx4
        1/1     Running           0               16m
kube-system             coredns-78fcd69978-kkf44
        1/1     Running           0               16m
kube-system             etcd-ip-10-0-121-16.us-west-2.compute.internal
        0/1     Running           0               8s
kube-system             etcd-ip-10-0-46-17.us-west-2.compute.internal
        1/1     Running           1               16m
kube-system             etcd-ip-10-0-88-238.us-west-2.compute.internal
        1/1     Running           1               5m35s
kube-system             kube-apiserver-ip-10-0-121-16.us-west-2.compute.internal
        0/1     Running           6               7s
```

```
kube-system              kube-apiserver-ip-10-0-46-17.us-west-2.compute.internal
       1/1     Running          1               16m
kube-system              kube-apiserver-ip-10-0-88-238.us-west-2.compute.internal
       1/1     Running          1               5m34s
kube-system              kube-controller-manager-ip-10-0-121-16.us-
west-2.compute.internal   0/1     Running           0               7s
kube-system              kube-controller-manager-ip-10-0-46-17.us-
west-2.compute.internal    1/1     Running           1 (5m25s ago)   15m
kube-system              kube-controller-manager-ip-10-0-88-238.us-
west-2.compute.internal   1/1     Running           0               5m34s
kube-system              kube-proxy-gclmt
       1/1     Running          0               16m
kube-system              kube-proxy-gptd4
       1/1     Running          0               9s
kube-system              kube-proxy-mwkgl
       1/1     Running          0               4m40s
kube-system              kube-proxy-zcqxd
       1/1     Running          0               5m35s
kube-system              kube-scheduler-ip-10-0-121-16.us-west-2.compute.internal
       0/1     Running          1               7s
kube-system              kube-scheduler-ip-10-0-46-17.us-west-2.compute.internal
       1/1     Running          3 (5m25s ago)   16m
kube-system              kube-scheduler-ip-10-0-88-238.us-west-2.compute.internal
       1/1     Running          1               5m34s
kube-system              local-volume-provisioner-2mv7z
       1/1     Running          0               4m10s
kube-system              local-volume-provisioner-vdcrg
       1/1     Running          0               4m53s
kube-system              local-volume-provisioner-wsjrt
       1/1     Running          0               16m
node-feature-discovery   node-feature-discovery-master-84c67dcbb6-m78vr
       1/1     Running          0               16m
node-feature-discovery   node-feature-discovery-worker-vpvpl
       1/1     Running          0               4m10s
tigera-operator          tigera-operator-d499f5c8f-79dc4
       1/1     Running          1 (5m24s ago)   16m
```

Note: If a `calico-node` pod is not ready on your cluster, you must edit the `default` Installation resource. To edit the Installation resource, run the command: `kubectl edit installation default --kubeconfig ${CLUSTER_NAME}.conf`

2. Change the value for `spec.calicoNetwork.nodeAddressAutodetectionV4` to `interface: ens192`, and save the resource.

```
spec:
  calicoNetwork:
  ...
    nodeAddressAutodetectionV4:
      interface: ens192
```

3. Save this resource. If that pod has failed, you might need to delete the node feature discovery worker pod in the node-feature-discovery namespace. After you delete it, Kubernetes replaces the pod as part of its normal reconciliation.

Change the Encapsulation Type

About this task

Calico can leverage different network encapsulation methods to route traffic for your workloads. Encapsulation is useful when running on top of an underlying network that is unaware of workload IPs.

Procedure

- Public cloud environments where you don’t own the hardware.
- AWS across VPC subnet boundaries.
- Environments where you cannot peer Calico over BGP to the underlay or easily configure static routes.

Provider Specific Steps

About this task

The encapsulation type for networking depends on the cloud provider.IP-in-IP is Calico's default encapsulation method, which most providers use, but not Azure.

> **Note: Azure** only supports VXLAN encapsulation type. Therefore, if you install on Azure pre-provisioned VMs, you must set the encapsulation mode to VXLAN.

Nutanix recommends using the method below to change the encapsulation type after cluster creation but before production. To change the encapsulation type, follow these steps:

Procedure

1. First, remove the existing `default-ipv4-ippool` IPPool resource from `kubeconfig`. After you edit the installation resource, the resource must be deleted to be recreated. Run the command below to delete.

   ```
   kubectl delete ippool default-ipv4-ippool
   ```

2. Run the following command to edit.

   ```
   kubectl edit installation default --kubeconfig ${CLUSTER_NAME}.conf
   ```

3. Change the value for encapsulation - encapsulation: as shown below.

   ```
   spec:
   calicoNetwork:
     ipPools:
       - encapsulation: VXLAN
   ```

   > **Note: VXLAN** is a tunneling protocol that encapsulates Layer 2 Ethernet frames in UDP packets, enabling you to create virtualized Layer 2 subnets that span Layer 3 networks. It has a slightly larger header than IP-in-IP, which slightly reduces performance over IP-in-IP.

   > **Note: IPIP** IP-in-IP is an IP tunneling protocol that encapsulates one IP packet in another IP packet. An outer packet header is added with the tunnel entry and exit points. The calico implementation of this protocol uses BGP to determine the exit point, which makes this protocol unusable on networks that don’t pass BGP.

   > **Tip:** If using Windows, see this documentation on the Calico site regarding limitations: Calico for Windows VXLAN

Pre-provisioned Installation in a Non-air-gapped Environment

In pre-provisioned environments, Nutanix Kubernetes Platform (NKP) handles your cluster’s life cycle, including installation, upgrade, and node management. NKP installs Kubernetes, monitoring and logging apps, and its own UI.

In an environment with access to the Internet, you retrieve artifacts from specialized repositories dedicated to them, such as Docker images contained in DockerHub and Helm Charts that come from a dedicated

Helm Chart repository. However, in an air-gapped environment, you need local repositories to store Helm charts, Docker images, and other artifacts. Tools such as JFrog, Harbor, and Nexus handle multiple types of artifacts in one local repository.

> **Note:** If desired, a local registry can also be used in a non-air-gapped environment for speed and security. To do so, add the Pre-provisioned Air-gapped Define Environment steps to your non-air-gapped installation process.

Bootstrap Pre-provisioned
To create Kubernetes clusters, NKP uses Cluster API (CAPI) controllers. These controllers run on a Kubernetes cluster.

About this task

To get started, you need a bootstrap cluster. By default, Nutanix Kubernetes Platform (NKP) creates a bootstrap cluster for you in a Docker container using the Kubernetes-in-Docker (KIND) tool.

Prerequisites:

Before you begin, you must:

- Complete the steps in Prerequisites.
- Ensure the `nkp` binary can be found in your `$PATH`.
- IF using a Registry Mirror even though you are not in an air-gapped environment, refer to the air-gapped section for loading images: Pre-provisioned Air-gapped Define Environment

Procedure

1. Review Universal Configurations for all Infrastructure Providers regarding settings, flags, and other choices and then begin bootstrapping.
2. Create a bootstrap cluster using the command `nkp create bootstrap --kubeconfig $HOME/.kube/config`.

   > **Note:** Use `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Configuring an HTTP or HTTPS Proxy on page 677.

   Example output:

   ```
   # Creating a bootstrap cluster
   # Initializing new CAPI components
   ```

3. NKP creates a bootstrap cluster using KIND as a library.

   For more information, see https://github.com/kubernetes-sigs/kind.

4. NKP then deploys the following Cluster API providers on the cluster.

   - Core Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/
   - AWS Infrastructure Provider: https://github.com/kubernetes-sigs/cluster-api-provider-aws
   - Kubeadm Bootstrap Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/bootstrap/kubeadm
   - Kubeadm ControlPlane Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/controlplane/kubeadm

   For more information on Cluster APIs, see https://cluster-api.sigs.k8s.io/.

5. NKP waits until these providers' controller-manager and webhook deployments are ready. List these deployments using the command `kubectl get --all-namespaces deployments -l=clusterctl.cluster.x-k8s.io`.

   Example output:

```
NAMESPACE                           NAME
 READY   UP-TO-DATE   AVAILABLE   AGE
capa-system                         capa-controller-manager
 1/1     1            1           1h
capg-system                         capg-controller-manager
 1/1     1            1           1h
capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-manager
 1/1     1            1           1h
capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-manager
 1/1     1            1           1h
capi-system                         capi-controller-manager
 1/1     1            1           1h
cappp-system                        cappp-controller-manager
 1/1     1            1           1h
capv-system                         capv-controller-manager
 1/1     1            1           1h
capz-system                         capz-controller-manager
 1/1     1            1           1h
cert-manager                        cert-manager
 1/1     1            1           1h
cert-manager                        cert-manager-cainjector
 1/1     1            1           1h
cert-manager                        cert-manager-webhook
 1/1     1            1           1h
```

Creating a New Cluster

Create a new Pre-provisioned Kubernetes cluster in a non-air-gapped environment.

About this task

After defining the infrastructure and control plane endpoints, you can create the cluster by following these steps to create a new pre-provisioned cluster.

Before you begin

Procedure

1. Give your cluster a unique name suitable for your environment.

   Note: The cluster name may only contain the following characters: a-z, 0-9, and -. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

2. Set the environment variable: `export CLUSTER_NAME=<preprovisioned-example>`

What to do next

Create a Kubernetes Cluster

Before you create a new Nutanix Kubernetes Platform (NKP) cluster below, you may choose an external load balancer or virtual IP and use the corresponding `nkp create cluster` command example from that page in the docs from the links below. Other customizations are available but require different flags during the `nkp create cluster` command. Refer to Pre-provisioned Cluster Creation Customization Choices for more cluster customizations.

When you create a new NKP cluster below, choose an external load balancer (LB) or virtual IP and use the corresponding `nkp create cluster` command.

In a pre-provisioned environment, use the Kubernetes CSI and third-party drivers for local volumes and other storage devices in your datacenter.

NKP uses a local static provisioner as the default storage provider for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. Use Kubernetes CSI compatible storage that is suitable for production.

After turning off `localvolumeprovisioner`, you can choose from any of the storage options available for Kubernetes. To make that storage the default storage, use the commands in this section of the Kubernetes documentation: Changing the Default Storage Class

For Pre-provisioned environments, you define a set of existing nodes. During the cluster creation process, Konvoy Image Builder(KIB) is built into NKP and automatically runs the machine configuration process (which KIB uses to build images for other providers) against the set of nodes that you defined. This results in your pre-existing or pre-provisioned nodes being appropriately configured.

The following command relies on the pre-provisioned cluster API infrastructure provider to initialize the Kubernetes control plane and worker nodes on the hosts defined in the inventory YAML previously created.

> **Note:** (Optional) If you have overrides for your clusters, specify the secret as part of the create cluster command. If these are not specified, the overrides for your nodes will not be applied.`--override-secret-name=$CLUSTER_NAME-user-overrides`. See the topic Custom Overrides for details.

> **Note:** (Optional) Use a registry mirror. Configure your cluster to use an existing local registry as a mirror when attempting to pull images previously pushed to your registry when defining your infrastructure. Instructions in the expandable Custom Installation section. For registry mirror information, see topics Using a Registry Mirror and Registry Mirror Tools.

> **Note:** When creating the cluster, specify the `cluster-name`. Using the same #cluster-name# when defining your inventory objects would be best. See topic Defining Cluster Hosts and Infrastructure for more details.

Ensure your subnets do not overlap with your host subnet because they cannot be changed after cluster creation. If you need to change the kubernetes subnets, you must do this at cluster creation. See the topic Subnets. The default subnets used in NKP are:

```
spec:
  clusterNetwork:
    pods:
      cidrBlocks:
      - 192.168.0.0/16
    services:
      cidrBlocks:
      - 10.96.0.0/12
```

This command uses the **default external load balancer (LB)** option (see alternative Step 1 for virtual IP):

```
nkp create cluster preprovisioned \
  --cluster-name ${CLUSTER_NAME} \
  --control-plane-endpoint-host <control plane endpoint host> \
  --control-plane-endpoint-port <control plane endpoint port, if different than 6443> \
  --override-secret-name=$CLUSTER_NAME-user-overrides \
  --dry-run \ so
  --output=yaml \
  > ${CLUSTER_NAME}.yaml
```

> **Note:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

1. **ALTERNATIVE Virtual IP** - if you don't have an external LB and want to use a VIRTUAL IP provided by `kube-vip`, specify these flags example below:

```
nkp create cluster preprovisioned \
    --cluster-name ${CLUSTER_NAME} \
    --control-plane-endpoint-host 196.168.1.10 \
    --virtual-ip-interface eth1 \
    --dry-run \
    --output=yaml \
    > ${CLUSTER_NAME}.yaml
```

2. Inspect or edit the cluster objects and familiarize yourself with Cluster API before editing them, as edits can prevent the cluster from deploying successfully.
3. Create the cluster from the objects generated in the `dry run`. A warning will appear in the console if the resource already exists, requiring you to remove the resource or update your YAML.

```
kubectl create -f ${CLUSTER_NAME}.yaml
```

> **Note:** If you used the `--output-directory` flag in your `nkp create .. --dry-run` step above, create the cluster from the objects you created by specifying the directory:
>
> `kubectl create -f <existing-directory>/`

4. Use the wait command to monitor the cluster control-plane readiness:

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=30m
```

Output:

```
cluster.cluster.x-k8s.io/preprovisioned-example condition met
```

> **Note:** It will take a few minutes to create, depending on the cluster size.

When the command is complete, you will have a running Kubernetes cluster! For bootstrap and custom YAML cluster creation, refer to the Additional Infrastructure Customization section of the documentation for Pre-provisioned: Pre-provisioned Infrastructure

Use this command to get the Kubernetes `kubeconfig` for the new cluster and proceed to install the NKP Kommander UI:

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

> **Note:** If changing the Calico encapsulation, Nutanix recommends changing it after cluster creation but before production.

> **Important:** If you need to increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

Audit Logs

To modify Control Plane Audit logs settings using the information on the page Configure the Control Plane.

Making the New Pre-Provisioned Cluster Self-Managed

How to make a Kubernetes cluster manage itself.

About this task

Nutanix Kubernetes Platform (NKP) deploys all cluster life cycle services to a bootstrap cluster, which then deploys a workload cluster. When the workload cluster is ready, move the cluster life cycle services to the workload cluster, which makes the workload cluster self-managed.

Before you begin

Before starting, ensure you can create a workload cluster as described in the topic: Create a New Pre-provisioned Cluster.

This page contains instructions on how to make your cluster self-managed. This is necessary if there is only one cluster in your environment or if this cluster becomes the Management cluster in a multi-cluster environment.

> **Note:** If you already have a self-managed or Management cluster in your environment, skip this page.

Follow these steps to turn your new cluster into a **Management Cluster** for an Ultimate license environment (or a free-standing Pro Cluster):

Procedure

1. Deploy cluster life cycle services on the workload cluster.

   ```
   nkp create capi-components --kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Initializing new CAPI components
   ```

   > **Note:** If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP/HTTPS Proxy.

2. The cluster life cycle services on the workload cluster are ready, but the workload cluster configuration is on the bootstrap cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the bootstrap to the workload cluster. This process is called a Pivot. For more information, see https://cluster-api.sigs.k8s.io/reference/glossary.html?highlight=pivot#pivot.

   Move the Cluster API objects from the bootstrap to the workload cluster:

   ```
   nkp move capi-resources --to-kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Moving cluster resources
   You can now view resources in the moved cluster by using the --kubeconfig flag with
    kubectl. For example: kubectl --kubeconfig=gcp-example.conf get nodes
   ```

   > **Note:** To ensure only one set of cluster life cycle services manages the workload cluster, NKP first pauses the reconciliation of the objects on the bootstrap cluster, then creates the objects on the workload cluster. As NKP copies the objects, the cluster life cycle services on the workload cluster reconcile the objects. The workload cluster becomes self-managed after NKP creates all the objects. If it fails, the `move` command can be safely retried.

3. Wait for the cluster control plane to be ready.

   ```
   kubectl --kubeconfig ${CLUSTER_NAME}.conf wait --for=condition=ControlPlaneReady
    "clusters/${CLUSTER_NAME}" --timeout=20m
   ```

   Output:

   ```
   cluster.cluster.x-k8s.io/gcp-example condition met
   ```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster status. After moving the cluster life cycle services to the workload cluster, remember to use NKP with the workload cluster `kubeconfig`.

```
nkp describe cluster --kubeconfig ${CLUSTER_NAME}.conf -c ${CLUSTER_NAME}
```

Output:

```
NAME                                                                        READY
 SEVERITY  REASON  SINCE  MESSAGE
Cluster/preprovisioned-example                                              True
               2m31s
##ClusterInfrastructure - PreprovisionedCluster/preprovisioned-example

##ControlPlane - KubeadmControlPlane/preprovisioned-example-control-plane  True
               2m31s
# ##Machine/preprovisioned-example-control-plane-6g6nr                      True
               2m33s
# ##Machine/preprovisioned-example-control-plane-8lhcv                      True
               2m33s
# ##Machine/preprovisioned-example-control-plane-kk2kg                      True
               2m33s
##Workers

  ##MachineDeployment/preprovisioned-example-md-0                           True
               2m34s
    ##Machine/preprovisioned-example-md-0-77f667cd9-tnctd                  True
               2m33s
```

5. Remove the bootstrap cluster because the workload cluster is now self-managed.

```
nkp delete bootstrap --kubeconfig $HOME/.kube/config
```

Output:

```
# Deleting bootstrap cluster
```

Known Limitations

Procedure

- NKP only supports moving all namespaces in the cluster; NKP does not support migration of individual namespaces.
- Konvoy supports moving only one set of cluster objects from the bootstrap cluster to the workload cluster or vice-versa.

Installing Kommander

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped Pre-provisioned environment.

About this task

Once you have installed the Konvoy component of Nutanix Kubernetes Platform (NKP), you will continue installing the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.

- Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a time period (for example, 1h) to allocate more time to deploy applications.
- If the Kommander installation fails, or you wantwantreconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. Edit the installer file to include configuration overrides for the `rook-ceph-cluster`. NKP's default configuration ships Ceph with PersistentVolumeClaim (PVC) based storage, which requires your CSI provider to support PVC with type `volumeMode: Block`. As this is impossible with the default local static provisioner, you can install Ceph in host storage mode. You can choose whether Ceph's object storage daemon (osd) pods can consume all or just some of the devices on your nodes. Include one of the following Overrides.

   a. To automatically assign all raw storage devices on all nodes to the Ceph cluster.

   ```
   rook-ceph-cluster:
       enabled: true
       values: |
         cephClusterSpec:
           storage:
             storageClassDeviceSets: []
             useAllDevices: true
             useAllNodes: true
             deviceFilter: "<<value>>"
   ```

   b. To assign specific storage devices on all nodes to the Ceph cluster.

   ```
   rook-ceph-cluster:
       enabled: true
       values: |
         cephClusterSpec:
           storage:
             storageClassDeviceSets: []
             useAllNodes: true
             useAllDevices: false
   ```

```
          deviceFilter: "^sdb."
```

> **Note:** If you want to assign specific devices to specific nodes using the `deviceFilter` option, refer to Specific Nodes and Devices. For general information on the `deviceFilter` value, refer to Storage Selection Settings.

5. If required: Customize your `kommander.yaml`.

   a. See Kommanderthe Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `dkp-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.nutanix.io/project-default-catalog-repository: "true"
        kommander.nutanix.io/workspace-default-catalog-repository: "true"
        kommander.nutanix.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Enable NKP Catalog Applications after Installing NKP.

Verifying the Kommander Install and UI Log in

Verify Kommander Install and Log in to the Dashboard UI

About this task

Verify Kommander Installation

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   NKP open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

```
kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
 index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
dashboard{{ "\n"}}'
```

Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

a. Rotate the password using the following command.

```
NKP experimental rotate dashboard-password
```

The output displays the new password:

```
Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Day 2 Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of Nutanix Kubernetes Platform (NKP) . The Day 2 section allows you to manage cluster operations and their application workloads to optimize your organization's productivity.

- Continue to the NKP Dashboard.

Pre-provisioned Installation in an Air-gapped Environment

In pre-provisioned environments, Nutanix Kubernetes Platform (NKP) handles your cluster's life cycle, including installation, upgrade, and node management. NKP installs Kubernetes, monitoring and logging apps, and its UI.

In an environment with access to the Internet, you retrieve artifacts from specialized repositories dedicated to them, such as Docker images contained in DockerHub and Helm Charts that come from a dedicated Helm Chart repository. However, in an air-gapped environment, you need local repositories to store Helm charts, Docker images, and other artifacts. Tools such as JFrog, Harbor, and Nexus handle multiple types of artifacts in one local repository.

Follow these steps to deploy NKP in a Pre-provisioned, Non-air-gapped environment:

Completing the Prerequisites for a Pre-Provisioned Air-gapped Environment
Outlines how to fulfill the prerequisites for using pre-provisioned infrastructure when using an air-gapped environment.

About this task

Nutanix Kubernetes Platform (NKP) in an air-gapped environment requires a local container registry of trusted images to enable production-level Kubernetes cluster management. In an environment with access to the internet, you retrieve artifacts from specialized repositories dedicated to them, such as Docker images contained in DockerHub and Helm Charts that come from a dedicated Helm Chart repository. However, in an air-gapped environment, you need:

Before you begin

- Local repositories to store Helm charts, Docker images, and other artifacts. Tools such as ECR, jFrog, Harbor, and Nexus handle multiple types of artifacts in one local repository.

- Bastion Host - If you have not set up a Bastion Host yet, refer to that Documentation section.
- The complete NKP air-gapped bundle, which contains all the NKP components needed for an air-gapped environment installation and also to use a local registry in a non-air-gapped environment: Pre-provisioned Loading the Registry

Copy Air-gapped Artifacts onto Cluster Hosts

Procedure

1. Download `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarballtar file to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You must fetch the distro packages and other artifacts. You get the latest security fixes available at machine image build time by fetching the distro packages from distro repositories.

3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the new NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`. Example command.

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   Note:

   - For FIPS: Pass the flag: `--fips`
   - For RHEL OS: Pass your RedHat subscription/licensing manager credentials: `export RMS_ACTIVATION_KEY` or `export RHSM_USER="" export RHSM_PASS=""`

     ```
     export RHSM_ACTIVATION_KEY="-ci"
     export RHSM_ORG_ID="1232131"
     ```

4. Set up the process.

   a. The bootstrap image must be extracted and loaded onto the bastion host.

   b. Artifacts must be copied onto cluster hosts for nodes to access.

   c. If using GPU, those artifacts must be positioned locally.

   d. Registry seeded with images locally.

5. Load the bootstrap image on your bastion machine from the air-gapped bundle you downloaded (`nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`)

   ```
   docker load -i konvoy-bootstrap-image-v2.12.0.tar
   ```

6. Copy air-gapped artifacts onto cluster hosts. Using the Konvoy Image Builder, you can copy the required artifacts onto your cluster hosts. The Kubernetes image bundle will be located in `kib/artifacts/images` and you will want to verify the image and artifacts.

   a. Verify the image bundles exist in `artifacts/images`.

   ```
   $ ls artifacts/images/
   kubernetes-images-1.29.6-nutanix.1.tar kubernetes-images-1.29.6-nutanix.1-fips.tar
   ```

b. Verify the artifacts for your OS exist in the `artifacts/` directory and export the appropriate variables.

```
$ ls artifacts/
1.29.6_centos_7_x86_64.tar.gz     1.29.6_redhat_8_x86_64_fips.tar.gz
 containerd-1.6.28-d2iq.1-rhel-8.6-x86_64_fips.tar.gz pip-packages.tar.gz
1.29.6_centos_7_x86_64_fips.tar.gz  1.29.6_rocky_9_x86_64.tar.gz
 containerd-1.6.28-d2iq.1-rocky-9.0-x86_64.tar.gz
1.29.6_redhat_7_x86_64.tar.gz     1.29.6_ubuntu_20_x86_64.tar.gz
 containerd-1.6.28-d2iq.1-rocky-9.1-x86_64.tar.gz
1.29.6_redhat_7_x86_64_fips.tar.gz  containerd-1.6.28-d2iq.1-ubuntu-20.04-
x86_64.tar.gz
1.29.6_redhat_8_x86_64.tar.gz
```

c. For example, for RHEL 8.6 you set.

```
export OS_PACKAGES_BUNDLE=1.29.6_redhat_8_x86_64.tar.gz
export CONTAINERD_BUNDLE=containerd-1.6.28-nutanix.1-rhel-8.4-x86_64.tar.gz
```

7. Export the following environment variables, ensuring that all control plane and worker nodes are included.

```
export CONTROL_PLANE_1_ADDRESS="<control-plane-address-1>"
export CONTROL_PLANE_2_ADDRESS="<control-plane-address-2>"
export CONTROL_PLANE_3_ADDRESS="<control-plane-address-3>"
export WORKER_1_ADDRESS="<worker-address-1>"
export WORKER_2_ADDRESS="<worker-address-2>"
export WORKER_3_ADDRESS="<worker-address-3>"
export WORKER_4_ADDRESS="<worker-address-4>"
export SSH_USER="<ssh-user>"
export SSH_PRIVATE_KEY_FILE="<private key file>"
```

`SSH_PRIVATE_KEY_FILE` must be either the name of the SSH private key file in your working directory or an absolute path to the file in your user's home directory.

8. Generate an `inventory.yaml` , which is automatically picked up by the `konvoy-image upload` in the next step.

```
cat <<EOF > inventory.yaml
all:
  vars:
    ansible_user: $SSH_USER
    ansible_port: 22
    ansible_ssh_private_key_file: $SSH_PRIVATE_KEY_FILE
  hosts:
    $CONTROL_PLANE_1_ADDRESS:
      ansible_host: $CONTROL_PLANE_1_ADDRESS
    $CONTROL_PLANE_2_ADDRESS:
      ansible_host: $CONTROL_PLANE_2_ADDRESS
    $CONTROL_PLANE_3_ADDRESS:
      ansible_host: $CONTROL_PLANE_3_ADDRESS
    $WORKER_1_ADDRESS:
      ansible_host: $WORKER_1_ADDRESS
    $WORKER_2_ADDRESS:
      ansible_host: $WORKER_2_ADDRESS
    $WORKER_3_ADDRESS:
      ansible_host: $WORKER_3_ADDRESS
    $WORKER_4_ADDRESS:
      ansible_host: $WORKER_4_ADDRESS
EOF
```

9. Upload the artifacts onto cluster hosts with the following command.

```
konvoy-image upload artifacts \
              --container-images-dir=./kib/artifacts/images/ \
              --os-packages-bundle=./kib/artifacts/$OS_PACKAGES_BUNDLE \
              --containerd-bundle=./kib/artifacts/$CONTAINERD_BUNDLE \
              --pip-packages-bundle=./kib/artifacts/pip-packages.tar.gz
```

Flags: Use the overrides flag (for example:`--overrides overrides/fips.yaml`) and reference either `fips.yaml` or `offline-fips.yaml`

manifests located in the overrides directory . You can also see these pages in the documentation. Add GPU flags if needed: `--nvidia-runfile=./artifacts/NVIDIA-Linux-x86_64-470.82.01.run`

The `konvoy-image upload artifacts` command and copy all OS packages and other artifacts onto each machine in your inventory. When you create the cluster, provisioning connects to each node and runs commands to install those artifacts. Consequently, Kubernetes is running. KIB uses variable overrides to specify your new machine image's base and container images. The variable overrides files for NVIDIA and FIPS, which can be ignored unless an overlay feature is added. Use the `--overrides overrides/fips.yaml,overrides/offline-fips.yaml` flag with manifests located in the overrides directory

Bootstrapping Air-gapped Pre-provisioned
To create Kubernetes clusters, NKP uses Cluster API (CAPI) controllers. These controllers run on a Kubernetes cluster.

About this task

To get started, you need a bootstrap cluster. By default, Nutanix Kubernetes Platform (NKP) creates a bootstrap cluster for you in a Docker container using the Kubernetes-in-Docker (KIND) tool.

Prerequisites:

Before you begin, you must:

- Complete the Nutanix Infrastructure Prerequisites. For more information, see Nutanix Infrastructure Prerequisites on page 690.
- Ensure the `nkp` binary can be found in your `$PATH`.
- If using a Registry Mirror even though you are not in an air-gapped environment, refer to the air-gapped section for loading images: Pre-provisioned Air-gapped Define Environment

Procedure

1. Review Universal Configurations for all Infrastructure Providers regarding settings, flags, and other choices and then begin bootstrapping.
2. Create a bootstrap cluster using the command `nkp create bootstrap --kubeconfig $HOME/.kube/config`.

   > **Note:** Use `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Configuring an HTTP or HTTPS Proxy on page 677.

   Example output:

```
# Creating a bootstrap cluster
# Initializing new CAPI components
```

3. NKP creates a bootstrap cluster using KIND as a library.

   For more information, see https://github.com/kubernetes-sigs/kind.

4. NKP then deploys the following Cluster API providers on the cluster.

   - Core Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/
   - AWS Infrastructure Provider: https://github.com/kubernetes-sigs/cluster-api-provider-aws
   - Kubeadm Bootstrap Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/bootstrap/kubeadm
   - Kubeadm ControlPlane Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/controlplane/kubeadm

   For more information on Cluster APIs, see https://cluster-api.sigs.k8s.io/.

5. NKP waits until these providers' controller-manager and webhook deployments are ready. List these deployments using the command `kubectl get --all-namespaces deployments -l=clusterctl.cluster.x-k8s.io`.

   Output example:

```
NAMESPACE                           NAME
 READY   UP-TO-DATE   AVAILABLE   AGE
capa-system                         capa-controller-manager
 1/1     1            1           1h
capg-system                         capg-controller-manager
 1/1     1            1           1h
capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-manager
 1/1     1            1           1h
capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-manager
 1/1     1            1           1h
capi-system                         capi-controller-manager
 1/1     1            1           1h
cappp-system                        cappp-controller-manager
 1/1     1            1           1h
capv-system                         capv-controller-manager
 1/1     1            1           1h
capz-system                         capz-controller-manager
 1/1     1            1           1h
cert-manager                        cert-manager
 1/1     1            1           1h
cert-manager                        cert-manager-cainjector
 1/1     1            1           1h
cert-manager                        cert-manager-webhook
 1/1     1            1           1h
```

Creating a New Cluster in an Air-gapped Environment

Create a new Pre-provisioned Kubernetes cluster in an Air-gapped environment.

About this task

After defining the infrastructure and control plane endpoints, you can create the cluster by following these steps to create a new pre-provisioned cluster.

Before you begin

Procedure

1. Give your cluster a unique name suitable for your environment.

   Note: The cluster name might only contain the following characters: a-z, 0-9, and -. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information.

2. Set the environment variable: `export CLUSTER_NAME=<preprovisioned-example>`

What to do next

Create a Kubernetes Cluster

Before you create a new Nutanix Kubernetes Platform (NKP) cluster below, you may choose an external load balancer or virtual IP and use the corresponding `nkp create cluster` command example from that page in the docs from the links below. Other customizations are available but require different flags during the `nkp create cluster` command. Refer to Pre-provisioned Cluster Creation Customization Choices for more cluster customizations.

When you create a new NKP cluster below, choose an external load balancer (LB) or virtual IP and use the corresponding `nkp create cluster` command.

In a pre-provisioned environment, use the Kubernetes CSI and third-party drivers for local volumes and other storage devices in your datacenter.

Note: NKP uses a local static provisioner as the default storage provider for a pre-provisioned environment. However, `localvolumeprovisioner` is not suitable for production use. Use Kubernetes CSI compatible storage that is suitable for production.

After turning off `localvolumeprovisioner`, you can choose from any of the storage options available for Kubernetes. To make that storage the default storage, use the commands in this section of the Kubernetes documentation: Changing the Default Storage Class

For Pre-provisioned environments, you define a set of existing nodes. During the cluster creation process, Konvoy Image Builder(KIB) is built into NKP and automatically runs the machine configuration process (which KIB uses to build images for other providers) against the set of nodes you defined. This results in your pre-existing or pre-provisioned nodes being appropriately configured.

The following command relies on the pre-provisioned cluster API infrastructure provider to initialize the Kubernetes control plane and worker nodes on the hosts defined in the inventory YAML previously created.

Important: If you need to increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

Note: (Optional) If you have overrides for your clusters, specify the secret in the create cluster command. If these are not specified, the overrides for your nodes will not be applied.`--override-secret-name=$CLUSTER_NAME-user-overrides`. See the topic Custom Overrides for details.

Note: (Optional) Use a registry mirror. Configure your cluster to use an existing local registry as a mirror when attempting to pull images previously pushed to your registry when defining your infrastructure. Instructions in the expandable Custom Installation section. For registry mirror information, see topics Using a Registry Mirror and Registry Mirror Tools.

Note: When creating the cluster, specify the `cluster-name`. It would be best to use the same #cluster-name# when defining your inventory objects. See topic Defining Cluster Hosts and Infrastructure for more details.

Ensure your subnets do not overlap with your host subnet because they cannot be changed after cluster creation. If you need to change the kubernetes subnets, you must do this at cluster creation. See the topic Subnets. The default subnets used in NKP are:

```
spec:
  clusterNetwork:
    pods:
      cidrBlocks:
      - 192.168.0.0/16
    services:
      cidrBlocks:
      - 10.96.0.0/12
```

This command uses the **default external load balancer (LB)** option (see alternative Step 1 for virtual IP):

```
nkp create cluster preprovisioned --cluster-name ${CLUSTER_NAME}
  --control-plane-endpoint-host <control plane endpoint host>
  --control-plane-endpoint-port <control plane endpoint port, if different than 6443>
  --pre-provisioned-inventory-file preprovisioned_inventory.yaml
  --ssh-private-key-file <path-to-ssh-private-key>
  --registry-mirror-url=${REGISTRY_URL} \
  --dry-run \
  --output=yaml \
  > ${CLUSTER_NAME}.yaml
```

Note: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

Note: For FIPS requirements, use the flags: `--kubernetes-version=v1.29.6+fips.0 \ --etcd-version=3.5.10+fips.0 \ --kubernetes-image-repository=docker.io/mesosphere \`

1. **ALTERNATIVE Virtual IP** - if you don't have an external LB and want to use a VIRTUAL IP provided by `kube-vip`, specify these flags example below:

   ```
   nkp create cluster preprovisioned \
       --cluster-name ${CLUSTER_NAME} \
       --control-plane-endpoint-host 196.168.1.10 \
       --virtual-ip-interface eth1 \
       --dry-run \
       --output=yaml \
       > ${CLUSTER_NAME}.yaml
   ```

2. Inspect or edit the cluster objects and familiarize yourself with Cluster API before editing them, as edits can prevent the cluster from deploying successfully.
3. Create the cluster from the objects generated in the `dry run`. A warning will appear in the console if the resource already exists, requiring you to remove the resource or update your YAML.

   ```
   kubectl create -f ${CLUSTER_NAME}.yaml
   ```

   Note: If you used the `--output-directory` flag in your `nkp create .. --dry-run` step above, create the cluster from the objects you created by specifying the directory:

   ```
   kubectl create -f <existing-directory>/
   ```

4. Use the wait command to monitor the cluster control-plane readiness:

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=30m
```

Output:

```
cluster.cluster.x-k8s.io/preprovisioned-example condition met
```

> **Note:** It will take a few minutes to create, depending on the cluster size.

When the command `completes complete, you will have a running Kubernetes cluster! For bootstrap and custom YAML cluster creation, refer to the Additional Infrastructure Customization section of the documentation for Pre-provisioned Pre-provisioned Infrastructure

Use this command to get the Kubernetes `kubeconfig` for the new cluster and proceed to install the NKP Kommander UI:

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

> **Note:** If changing the Calico encapsulation, Nutanix recommends changing it after cluster creation but before production.

Audit Logs

To modify Control Plane Audit logs settings using the information on the page Configure the Control Plane.

Making the Air-gapped Pre-Provisioned Cluster Self-Managed

How to make a Kubernetes cluster manage itself.

About this task

Nutanix Kubernetes Platform (NKP) deploys all cluster life cycle services to a bootstrap cluster, which then deploys a workload cluster. When the workload cluster is ready, move the cluster life cycle services to the workload cluster, which makes the workload cluster self-managed.

Before you begin

Before starting, ensure you can create a workload cluster as described in the topic: Create a New Pre-provisioned Cluster.

This page contains instructions on how to make your cluster self-managed. This is necessary if there is only one cluster in your environment or if this cluster becomes the Management cluster in a multi-cluster environment.

> **Note:** If you already have a self-managed or Management cluster in your environment, skip this page.

Follow these steps to turn your new cluster into a **Management Cluster** for an Ultimate license environment (or a free-standing Pro Cluster):

Procedure

1. Deploy cluster life cycle services on the workload cluster.

```
nkp create capi-components --kubeconfig ${CLUSTER_NAME}.conf
```

Output:

```
# Initializing new CAPI components
```

> **Note:** If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP/HTTPS Proxy.

2. The cluster life cycle services on the workload cluster are ready, but the workload cluster configuration is on the bootstrap cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the bootstrap to the workload cluster. This process is called a Pivot. For more information, see https://cluster-api.sigs.k8s.io/reference/glossary.html?highlight=pivot#pivot.

   Move the Cluster API objects from the bootstrap to the workload cluster:

   ```
   nkp move capi-resources --to-kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Moving cluster resources
   You can now view resources in the moved cluster by using the --kubeconfig flag with
    kubectl. For example: kubectl --kubeconfig=gcp-example.conf get nodes
   ```

   Note: To ensure only one set of cluster life cycle services manages the workload cluster, NKP first pauses the reconciliation of the objects on the bootstrap cluster, then creates the objects on the workload cluster. As NKP copies the objects, the cluster life cycle services on the workload cluster reconcile the objects. The workload cluster becomes self-managed after NKP creates all the objects. If it fails, the `move` command can be safely retried.

3. Wait for the cluster control-plane to be ready.

   ```
   kubectl --kubeconfig ${CLUSTER_NAME}.conf wait --for=condition=ControlPlaneReady
    "clusters/${CLUSTER_NAME}" --timeout=20m
   ```

   Output:

   ```
   cluster.cluster.x-k8s.io/gcp-example condition met
   ```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster status. After moving the cluster life cycle services to the workload cluster, remember to use NKP with the workload cluster `kubeconfig`.

   ```
   nkp describe cluster --kubeconfig ${CLUSTER_NAME}.conf -c ${CLUSTER_NAME}
   ```

   Output:

   ```
   NAME                                                                          READY
    SEVERITY  REASON  SINCE  MESSAGE
   Cluster/preprovisioned-example                                                True
                 2m31s
   ##ClusterInfrastructure - PreprovisionedCluster/preprovisioned-example

   ##ControlPlane - KubeadmControlPlane/preprovisioned-example-control-plane  True
                 2m31s
   # ##Machine/preprovisioned-example-control-plane-6g6nr                        True
                 2m33s
   # ##Machine/preprovisioned-example-control-plane-8lhcv                        True
                 2m33s
   # ##Machine/preprovisioned-example-control-plane-kk2kg                        True
                 2m33s
   ##Workers

     ##MachineDeployment/preprovisioned-example-md-0                            True
                 2m34s
       ##Machine/preprovisioned-example-md-0-77f667cd9-tnctd                    True
                 2m33s
   ```

5. Remove the bootstrap cluster because the workload cluster is now self-managed.

```
nkp delete bootstrap --kubeconfig $HOME/.kube/config
```

Output:

```
# Deleting bootstrap cluster
```

Known Limitations

Procedure

- NKP only supports moving all namespaces in the cluster; NKP does not support migration of individual namespaces.
- Konvoy supports moving only one set of cluster objects from the bootstrap cluster to the workload cluster or vice-versa.

Installing Kommander in an Air-gapped Environment

This section provides installation instructions for the Kommander component of NKP in an air-gapped Pre-provisioned environment.

About this task

Once you have installed the Konvoy component of Nutanix Kubernetes Platform (NKP), you will continue installing the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a time period (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you want to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a Default StorageClass on page 1038.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

```
export CLUSTER_NAME=<your-management-cluster-name>
```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

```
nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
```

3. Create a configuration file for the deployment.

```
nkp install kommander --init > kommander.yaml
```

4. Edit the installer file to include configuration overrides for the `rook-ceph-cluster`. NKP's default configuration ships Ceph with PersistentVolumeClaim (PVC) based storage, which requires your CSI provider to support PVC with type `volumeMode: Block`. As this is impossible with the default local static provisioner, you can install Ceph in host storage mode. You can choose whether Ceph's object storage daemon (osd) pods can consume all or just some of the devices on your nodes. Include one of the following Overrides.

   a. To automatically assign all raw storage devices on all nodes to the Ceph cluster.

```
rook-ceph-cluster:
    enabled: true
    values: |
      cephClusterSpec:
        storage:
          storageClassDeviceSets: []
          useAllDevices: true
          useAllNodes: true
          deviceFilter: "<<value>>"
```

   b. To assign specific storage devices on all nodes to the Ceph cluster.

```
rook-ceph-cluster:
    enabled: true
    values: |
      cephClusterSpec:
        storage:
          storageClassDeviceSets: []
          useAllNodes: true
          useAllDevices: false
          deviceFilter: "^sdb."
```

> **Note:** If you want to assign specific devices to specific nodes using the `deviceFilter` option, refer to Specific Nodes and Devices. For general information on the `deviceFilter` value, refer to Storage Selection Settings.

5. If required: Customize your `kommander.yaml`.

   a. See the Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `dkp-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.nutanix.io/project-default-catalog-repository: "true"
        kommander.nutanix.io/workspace-default-catalog-repository: "true"
        kommander.nutanix.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Enable NKP Catalog Applications after Installing NKP.

Verifying the Install and UI Log in

Verify Kommander Install and log in to the Dashboard UI

About this task

After you build the Konvoy cluster and you install the Kommander component for the UI, you can verify your installation. It waits for all applications to be ready by default.

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   NKP open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      NKP experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Day 2 Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of Nutanix Kubernetes Platform (NKP) . The Day 2 section allows you to manage cluster operations and their application workloads to optimize your organization’s productivity.

- Continue to the NKP Dashboard.

Pre-Provisioned Management Tools

After cluster creation and configuration, you can revisit clusters to update and change variables.

Section Content

Deleting a Pre-provisioned Cluster

Deleting a Pre-provisioned cluster.

About this task

A self-managed workload cluster cannot delete itself. If your workload cluster is self-managed, you must first create a bootstrap cluster and move the cluster life cycle services to it before deleting the workload cluster.

Procedure

If you did not make your workload cluster self-managed, as described in Make New Cluster Self-Managed, proceed to the instructions for Delete the workload cluster.

Create a Bootstrap Cluster and Move CAPI Resources

About this task

Follow these steps to create a bootstrap cluster and move CAPI resources:

Procedure

1. The bootstrap cluster will host the Cluster API controllers that reconcile the cluster objects marked for deletion. Create a bootstrap cluster. To avoid using the wrong kubeconfig, the following steps use explicit kubeconfig paths and contexts.

   ```
   nkp create bootstrap --kubeconfig $HOME/.kube/config --with-aws-bootstrap-
   credentials=true
   ```

2. Move the Cluster API objects from the workload to the bootstrap cluster: The cluster life cycle services on the bootstrap cluster are ready, but the workload cluster configuration is on the workload cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the workload to the bootstrap cluster. This process is also called a Pivot.

   ```
   nkp move capi-resources \
    --from-kubeconfig ${CLUSTER_NAME}.conf \
    --from-context ${CLUSTER_NAME}-admin@${CLUSTER_NAME} \
    --to-kubeconfig $HOME/.kube/config \
    --to-context kind-konvoy-capi-bootstrapper
   ```

   Output:

   ```
   # Moving cluster resources
   You can now view resources in the moved cluster by using the --kubeconfig flag with
    kubectl. For example: kubectl --kubeconfig $HOME/.kube/config get nodes
   ```

3. Use the cluster life cycle services on the workload cluster to check the workload cluster's status.

   ```
   nkp describe cluster --kubeconfig $HOME/.kube/config -c ${CLUSTER_NAME}
   ```

   Output:

   ```
   NAME                                                                    READY
    SEVERITY  REASON  SINCE  MESSAGE
   Cluster/preprovisioned-example                                          True
                 2m31s
   ##ClusterInfrastructure - PreprovisionedCluster/preprovisioned-example
   ```

```
##ControlPlane - KubeadmControlPlane/preprovisioned-example-control-plane  True
               2m31s
# ##Machine/preprovisioned-example-control-plane-6g6nr                     True
               2m33s
# ##Machine/preprovisioned-example-control-plane-8lhcv                     True
               2m33s
# ##Machine/preprovisioned-example-control-plane-kk2kg                     True
               2m33s
##Workers

  ##MachineDeployment/preprovisioned-example-md-0                          True
               2m34s
    ##Machine/preprovisioned-example-md-0-77f667cd9-tnctd                 True
               2m33s
```

4. Wait for the cluster control-planecontrol plan to be ready.

```
kubectl --kubeconfig $HOME/.kube/config wait --for=condition=controlplaneready
 "clusters/${CLUSTER_NAME}" --timeout=20m
```

Output:

```
cluster.cluster.x-k8s.io/preprovisioned-example condition met
```

Delete the Workload Cluster

Procedure

1. To delete a cluster, Use `nkp delete cluster` and pass in the name of the cluster you are trying to delete with `--cluster-name` flag. Use `kubectl get clusters` to get those details (`--cluster-name` and `--namespace`) of the Kubernetes cluster to delete it.

```
kubectl get clusters
```

2. Delete the Kubernetes cluster and wait a few minutes.

> **Note:** Before deleting the cluster, Nutanix Kubernetes Platform (NKP) deletes all Services of type LoadBalancer on the cluster. To skip this step, use the flag `--delete-kubernetes-resources=false`. Do not skip this step if the VPC is managed by NKP.# NKP manages the VPC NKP deletes the cluster; it deletes the VPC. If the VPC has any AWS Classic ELBs, AWS does not allow the VPC to be deleted, and NKP cannot delete the cluster.

```
nkp delete cluster --cluster-name=${CLUSTER_NAME} --kubeconfig $HOME/.kube/config
```

Output:

```
# Deleting Services with type LoadBalancer for Cluster default/azure-example
# Deleting ClusterResourceSets for Cluster default/azure-example
# Deleting cluster resources
# Waiting for the cluster to be fully deleted
Deleted default/azure-example cluster
```

After the workload cluster is deleted, you can delete the bootstrap cluster.

Delete the Bootstrap Cluster

About this task

After you have moved the workload resources back to a bootstrap cluster and deleted the workload cluster, you no longer need the bootstrap cluster. You can safely delete the bootstrap cluster with these steps:

Procedure

Delete the bootstrap cluster.

```
nkp delete bootstrap --kubeconfig $HOME/.kube/config
```

Output:

```
# Deleting bootstrap cluster
```

Manage Pre-provisioned Node Pools

Node pools are part of a cluster and managed as a group, and you can use a node pool to manage a group of machines using the same common properties. When Konvoy creates a new default cluster, there is one node pool for the worker nodes, and all nodes in that new node pool have the same configuration. You can create additional node pools for more specialized hardware or configuration. For example, if you want to tune your memory usage on a cluster where you need maximum memory for some machines and minimal memory for others, you create a new node pool with those specific resource needs.

Nutanix Kubernetes Platform (NKP) implements node pools using Cluster API Machine Deployments. For more information, see https://cluster-api.sigs.k8s.io/developer/architecture/controllers/machine-deployment.html.

Creating Pre-provisioned Node Pools

Creating a node pool is useful when you need to run workloads that require machines with specific resources, such as a GPU, additional memory, or specialized network or storage hardware.

About this task

Availability zones (AZs) are isolated locations within datacenter regions where public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single Availability Zone, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple Availability Zones.

Procedure

1. Create an inventory object with the same name as the node pool you're creating and the details of the pre-provisioned machines you want to add to it. For example, to create a node pool named gpu-nodepool, an inventory named gpu-nodepool must be present in the same namespace.

   ```
   apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
   kind: PreprovisionedInventory
   metadata:
     name: ${MY_NODEPOOL_NAME}
   spec:
     hosts:
       - address: ${IP_OF_NODE}
     sshConfig:
       port: 22
       user: ${SSH_USERNAME}
       privateKeyRef:
         name: ${NAME_OF_SSH_SECRET}
         namespace: ${NAMESPACE_OF_SSH_SECRET}
   ```

   (Optional) If your pre-provisioned machines have overrides, you must create a secret that includes all the overrides you want to provide in one file. Create an override secret using the instructions detailed on this page.

2. Once the `PreprovisionedInventory` object and overrides are created, create a node pool.

```
nkp create nodepool preprovisioned -c ${MY_CLUSTER_NAME} ${MY_NODEPOOL_NAME} --
override-secret-name ${MY_OVERRIDE_SECRET}
```

3. Advanced users can use a combination of the `--dry-run` and `--output=yaml` or `--output-directory=<existing-directory>` flags to get a complete set of node pool objects to modify locally or store in version control.

Scaling Pre-provisioned Node Pools

While running Cluster Autoscaler, you can manually scale your node pools up or down when you need finite control over your environment.

About this task

Environment variables, such as defining the node pool name, are set in the Prepare the Environment section on the previous page. If needed, refer to that page to set those variables.

Before you begin

- You must have the bootstrap node running with the SSH key or secrets created.
- The export values in the environment variables section need to contain the addresses of the nodes that you need to add Pre-provisioned: Define Infrastructure.
- Update the `preprovisioned_inventory.yaml` with the new host addresses.
- Run the `kubectl apply` command.

Scale Up Node Pools

Procedure

1. Fetch the existing `preprovisioned_inventory`.

```
$ kubectl get preprovisionedinventory
```

2. Edit the `preprovisioned_inventory` to add additional IPs needed for additional worker nodes in the `spec.hosts:` section.

```
$ kubectl edit preprovisionedinventory <preprovisioned_inventory> -n default
```

3. Add any additional IPs that you require

```
spec:
    hosts:
    - address: <worker.ip.add.1>
    - address: <worker.ip.add.2>
```

After you edit `preprovisioned_inventory`, fetch the machine deployment. The naming convention with `md` means that it is for worker machines. For example.

```
$ kubectl --kubeconfig ${CLUSTER_NAME}.conf get machinedeployment

NAME                      CLUSTER        AGE     PHASE     REPLICAS   READY   UPDATED
  UNAVAILABLE
machinedeployment-md-0   cluster-name   9m10s   Running   4          4       4
```

4. Scale the worker node to the required number. In this example, we scale from 4 to 6 worker nodes.

```
$ kubectl --kubeconfig ${CLUSTER_NAME}.conf scale --replicas=6 machinedeployment
 machinedeployment-md-0

machinedeployment.cluster.x-k8s.io/machinedeployment-md-0 scaled
```

5. Monitor the scaling with this command by adding the -w option to watch.

```
$ kubectl --kubeconfig ${CLUSTER_NAME}.conf get machinedeployment -w

NAME                     CLUSTER        AGE   PHASE       REPLICAS   READY   UPDATED
  UNAVAILABLE
machinedeployment-md-0   cluster-name   20m   ScalingUp   6          4       6
   2
```

6. Also, you can check the machine deployment to see if it is already scaled.

   Example output

```
$ kubectl --kubeconfig ${CLUSTER_NAME}.conf get machinedeployment

NAME                     CLUSTER        AGE     PHASE     REPLICAS   READY   UPDATED
  UNAVAILABLE
machinedeployment-md-0   cluster-name   3h33m   Running   6          6       6
```

7. Alternately, you can use this command to verify the `NODENAME` column and see the additional worker nodes added and in `Running` state.

```
$ kubectl --kubeconfig ${CLUSTER_NAME}.conf get machines -o wide

NAME     CLUSTER     AGE     PROVIDERID     PHASE     VERSION     NODENAME
```

userunScaling Down Pre-provisioned Node Pools

While running Cluster Autoscaler, you can manually scale your node pools up or down when you need finite control over your environment.

About this task

Environment variables, such as defining the node pool name, are set in the Prepare the Environment section on the previous page. If needed, refer to that page to set those variables.

Procedure

1. Run this command on your worker nodes.

```
kubectl scale machinedeployment <machinedeployment-name> --replicas <new number>
```

2. For control plane nodes, execute the following command.

```
kubectl scale kubeadmcontrolplane ${CLUSTER_NAME}-control-plane --replicas <new
 number>
```

   Machines can get stuck in the provisioning stage when you scale down. You can utilize a delete operation to clear the stale machine deployment:

```
kubectl delete machine ${CLUSTER_NAME}-control-plane-<hash>
```

```
kubectl delete machine <machinedeployment-name>-<hash>
```

Deleting Pre-provisioned Node Pools

Deleting a node pool deletes the Kubernetes nodes and the underlying infrastructure.

About this task

All nodes will be drained before deletion, and the pods running on those nodes will be rescheduled.

Procedure

1. Delete a node pool from a managed cluster using the command `nkp delete nodepool ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME}`.

   In this example output,`example` is the node pool to be deleted.

   ```
   # Deleting default/example nodepool resources
   ```

2. Delete an invalid node pool using the command `nkp delete nodepool ${CLUSTER_NAME}-md-invalid --cluster-name=${CLUSTER_NAME}`.

   Example output:

   ```
   MachineDeployments or MachinePools.infrastructure.cluster.x-k8s.io "no
    MachineDeployments or MachinePools found for cluster aws-example" not found
   ```

Creating Pre-provisioned GPU Node Pools

For pre-provisioned environments, Nutanix Kubernetes Platform (NKP) has provided the `nvidia-runfile` flag for Air-gapped Pre-provisioned environments.

About this task

Add the download to the artifacts directory.

Before you begin

- If the NVIDIA runfile installer has not been downloaded, retrieve and install the download by running the command. `curl -O https://download.nvidia.com/XFree86/Linux-x86_64/470.82.01/NVIDIA-Linux-x86_64-470.82.01.run mv NVIDIA-Linux-x86_64-470.82.01.run artifacts`
- Create an `artifacts` directory if it doesn't already exist.

  Note: For using GPUs in an air-gapped on-premises environment, Nutanix recommends setting up Pod Disruption Budget before Update Cluster Nodepools. For more information, see https://kubernetes.io/docs/concepts/workloads/pods/disruptions/ and https://docs.nvidia.com/datacenter/tesla/tesla-installation-notes/index.html#runfile.

Procedure

1. In your `overrides/nvidia.yaml` file, add the following to enable GPU builds. You can also access and use the overrides repo. Create the secret that GPU nodepool uses. This secret is populated from the KIB overrides. This output example has a file called `overrides/nvidia.yaml`.

   ```
   gpu:
     types:
       - nvidia
   build_name_extra: "-nvidia"
   ```

2. Create a secret on the bootstrap cluster populated from the above file. We will name it `${CLUSTER_NAME}-user-overrides`

   ```
   kubectl create secret generic ${CLUSTER_NAME}-user-overrides --from-
   file=overrides.yaml=overrides/nvidia.yaml
   ```

3. Create an inventory and node pool with the instructions below and use the `$CLUSTER_NAME-user-overrides` secret.

   Follow these steps.

   a. Create an inventory object with the same name as the node pool you're creating and the details of the pre-provisioned machines you want to add to it. For example, to create a node pool named `gpu-nodepool` an inventory named `gpu-nodepool` must be present in the same namespace.

   ```
   apiVersion: infrastructure.cluster.konvoy.d2iq.io/v1alpha1
   kind: PreprovisionedInventory
   metadata:
     name: ${MY_NODEPOOL_NAME}
   spec:
     hosts:
       - address: ${IP_OF_NODE}
     sshConfig:
       port: 22
       user: ${SSH_USERNAME}
       privateKeyRef:
         name: ${NAME_OF_SSH_SECRET}
         namespace: ${NAMESPACE_OF_SSH_SECRET}
   ```

   b. (Optional) If your pre-provisioned machines have overrides, you must create a secret that includes all the overrides you want to provide in one file. Create an override secret using the instructions detailed on this page.

   c. Once the `PreprovisionedInventory` object and overrides are created, create a node pool.

   ```
   nkp create nodepool preprovisioned -c ${MY_CLUSTER_NAME} ${MY_NODEPOOL_NAME} --
   override-secret-name ${MY_OVERRIDE_SECRET}
   ```

   - Advanced users can use a combination of the `--dry-run` and `--output=yaml` or `--output-directory=<existing-directory>` flags to get a complete set of node pool objects to modify locally or store in version control.

AWS Infrastructure

Configuration types for installing Nutanix Kubernetes Platform (NKP) on AWS Infrastructure.

For an environment on the AWS Infrastructure, installation options based on those environment variables are provided for you in this location.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Otherwise, you can go ahead and go to the AWS Prerequisites and Permissions topic to begin your custom installation.

AWS Prerequisites and Permissions

Installation instructions for installing Nutanix Kubernetes Platform (NKP) in an AWS non-air-gapped environment.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

NKP Prerequisites

Before you begin using Konvoy, you must have:

- An x86_64-based Linux or macOS machine.
- The `NKP` binary for Linux or macOS.
- A Container engine/runtime installed is required to install NKP and bootstrap:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements
- kubectl for interacting with the running cluster.
- A valid AWS account with credentials configured.
- For a local registry, whether air-gapped or non-air-gapped environment, download and extract the bundle. Download the Complete NKP Air-gapped Bundle for this release (that is. `NKP-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`) to load registry.
- For air-gapped environment ONLY:
  - Linux machine (bastion) that has access to the existing VPC.
  - The `NKP` binary on the bastion.
  - kubectl for interacting with the running cluster on the bastion.is used to interact
  - An existing local registry.
  - Ability to download artifacts from the internet and then copy those onto your bootstrap machine.
  - An AWS Air-Gapped Machine Image

    Note: On macOS, Docker runs in a virtual machine. Configure this virtual machine with at least 8GB of memory.

Control Plane Nodes

You must have at least three control plane nodes. Each control plane node needs to have at least the following:

- 4 cores
- 16 GiB memory
- Approximately 80 GiB of free space for the volume used for `/var/lib/kubelet` and `/var/lib/containerd`.
- Disk usage must be below 85% on the root volume.

NKP on AWS defaults to deploying an `m5.xlarge` instance with an 80GiB root volume for control plane nodes, which meets the above requirements.

Worker Nodes

You must have at least four worker nodes. The specific number of worker nodes required for your environment can vary depending on the cluster workload and size of the nodes. Each worker node needs to have at least the following:

- 8 cores
- 32 GiB memory
- Around 80 GiB of free space for the volume used for `/var/lib/kubelet` and `/var/lib/containerd`.
- Disk usage must be below 85% on the root volume.

NKP on AWS defaults to deploying a `m5.2xlarge` instance with an 80GiB root volume for worker nodes, which meets the above requirements.

AWS Prerequisites

Before you begin using Konvoy with AWS, you must:

1. Follow the steps to create permissions and roles on the Minimal Permissions and Role to Create Clusters page.
2. Create Cluster IAM Policies and Roles
3. Export the AWS region where you want to deploy the cluster:

   ```
   export AWS_REGION=us-west-2
   ```

4. Export the AWS profile with the credentials you want to use to create the Kubernetes cluster:

   ```
   export AWS_PROFILE=<profile>
   ```

If using AWS ECR as your local private registry, more information can be found on the Registry Mirror Tools page.

To deploy a cluster with a custom image in a region where CAPI images are not provided, you need to use Konvoy Image Builder to create your image for the region. For more information, see https://cluster-api-aws.sigs.k8s.io/topics/images/built-amis.html.

> **Note:** Every tenant must be in a different AWS account for multi-tenancy to ensure they are truly independent of other tenants to enforce security.

Using Konvoy Image Builder

The default AWS image is not recommended for use in production. Nutanix suggests using Konvoy Image Builder to create a custom AMI and take advantage of enhanced cluster operations.

About this task

You must have at least one image before creating a new cluster. If you have an image, this step in your configuration is not required each time since that image can be used to spin up a new cluster. However, if

you need different images for different environments or providers, you must create a new custom image. Inside the sections for either non-air-gapped or air-gapped installation, you will find instructions on creating and applying custom images during cluster creation.

> **Note:** For more information and compatible KIB versions, see Konvoy Image Builder on page 1084.

AMI images contain configuration information and software to create a specific, pre-configured operating environment. For example, you can create an AMI image of your computer system settings and software. The AMI image can then be replicated and distributed, creating your computer system for other users. You can use override files to customize components installed on your machine image, such as having the FIPS versions of the Kubernetes components installed by KIB components.

Depending on which Nutanix Kubernetes Platform (NKP) version you are running, steps and flags will differ. To deploy in a region where CAPI images are not provided, you need to use KIB to create your image for the region. For a list of supported Amazon Web Services (AWS) regions, refer to the Published AMI information from AWS. To begin image creation:

Procedure

1. Run the `konvoy-image`command to build and validate the image.

   For example:

   ```
   konvoy-image build aws images/ami/rhel-86.yaml
   ```

   By default, it builds in the `us-west-2` region. To specify another region, set the `--region` flag as shown in the command below:

   ```
   konvoy-image build aws --region us-east-1 images/ami/rhel-86.yaml
   ```

   > **Tip:** Ensure you have named the correct AMI image YAML file for your OS in the `konvoy-image build` command.https://github.com/mesosphere/konvoy-image-builder/tree/main/images/ami

   After KIB provisions the image successfully, the `ami` id is printed and written to the `packer.pkr.hcl` (Packer config) file. This file has an `artifact_id` field whose value provides the name of the AMI ID, as shown in the example below. That is the `ami` you use in the `create cluster` command:

   ```
   {
     "builds": [
       {
         "name": "kib_image",
         "builder_type": "amazon-ebs",
         "build_time": 1698086886,
         "files": null,
         "artifact_id": "us-west-2:ami-04b8dfef8bd33a016",
         "packer_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05",
         "custom_data": {
           "containerd_version": "",
           "distribution": "RHEL",
           "distribution_version": "8.6",
           "kubernetes_cni_version": "",
           "kubernetes_version": "1.26.6"
         }
       }
     ],
     "last_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05"
   }
   ```

2. To use a custom Amazon Machine Images (AMI) when creating your cluster, you must first create that AMI using KIB. Then perform the export and name the custom AMI for use in the command `nkp create cluster`

   For example:

   ```
   export AWS_AMI_ID=ami-<ami-id-here>
   ```

3. **Note:** For Air-gapped AMI, there are special air-gapped bundle instructions.

   Inside the sections for either Non-air-gapped or Air-gapped cluster creation, you will find the instructions for how to apply custom images.

4. YAML for KIB AMI images found here in GitHub https://github.com/mesosphere/konvoy-image-builder/tree/7447074a6d910e71ad2e61fc4a12820d073ae5ae/images/ami

Using AWS Minimal Permissions and Role to Create Clusters
Configure IAM Prerequisites before starting a cluster.

About this task

This section guides you in creating and using a minimally-scoped policy to create Nutanix Kubernetes Platform (NKP) clusters on an AWS account. For multi-tenancy, every tenant must be in a different AWS account to ensure they are truly independent of other tenants to enforce security.

Before you begin

Before applying the IAM Policies, verify the following:

- You have a valid AWS account with credentials configured that can manage CloudFormation Stacks, IAM Policies, IAM Roles, and IAM Instance Profiles.
- The AWS CLI utility is installed.

If you use the AWS STS (Short Lived Token) to create the cluster, you must run the ./nkp update bootstrap credentials aws --kubeconfig=<kubeconfig file> before you update the node for the management cluster or managed clusters.

The following is an AWSCloudFormation stack that creates:

- A policy named `nkp-bootstrapper-policy` that enumerate the minimal permissions for a user that can create NKP aws clusters.
- A role named `nkp-bootstrapper-role` that uses the `nkp-bootstrapper-policy` with a trust policy to allow IAM users and ec2 instances from `MYAWSACCOUNTID` to use the role through STS.
- An instance profile `NKPBootstrapInstanceProfile` that wraps the `nkp-bootstrapper-role` to be used by ec2 instances.

Procedure

1. To create the resources in the CloudFormation stack, copy the following contents into a file.

   ```
   AWSTemplateFormatVersion: 2010-09-09
   Resources:
     AWSIAMInstanceProfileNKPBootstrapper:
       Properties:
         InstanceProfileName: NKPBootstrapInstanceProfile
         Roles:
         - Ref: NKPBootstrapRole
       Type: AWS::IAM::InstanceProfile
   ```

```
AWSIAMManagedPolicyNKPBootstrapper:
  Properties:
    Description: Minimal policy to create NKP clusters in AWS
    ManagedPolicyName: nkp-bootstrapper-policy
    PolicyDocument:
      Statement:
      - Action:
        - ec2:AllocateAddress
        - ec2:AssociateRouteTable
        - ec2:AttachInternetGateway
        - ec2:AuthorizeSecurityGroupIngress
        - ec2:CreateInternetGateway
        - ec2:CreateNatGateway
        - ec2:CreateRoute
        - ec2:CreateRouteTable
        - ec2:CreateSecurityGroup
        - ec2:CreateSubnet
        - ec2:CreateTags
        - ec2:CreateVpc
        - ec2:ModifyVpcAttribute
        - ec2:DeleteInternetGateway
        - ec2:DeleteNatGateway
        - ec2:DeleteRouteTable
        - ec2:DeleteSecurityGroup
        - ec2:DeleteSubnet
        - ec2:DeleteTags
        - ec2:DeleteVpc
        - ec2:DescribeAccountAttributes
        - ec2:DescribeAddresses
        - ec2:DescribeAvailabilityZones
        - ec2:DescribeInstanceTypes
        - ec2:DescribeInternetGateways
        - ec2:DescribeImages
        - ec2:DescribeNatGateways
        - ec2:DescribeNetworkInterfaces
        - ec2:DescribeNetworkInterfaceAttribute
        - ec2:DescribeRouteTables
        - ec2:DescribeSecurityGroups
        - ec2:DescribeSubnets
        - ec2:DescribeVpcs
        - ec2:DescribeVpcAttribute
        - ec2:DescribeVolumes
        - ec2:DetachInternetGateway
        - ec2:DisassociateRouteTable
        - ec2:DisassociateAddress
        - ec2:ModifyInstanceAttribute
        - ec2:ModifyInstanceMetadataOptions
        - ec2:ModifyNetworkInterfaceAttribute
        - ec2:ModifySubnetAttribute
        - ec2:ReleaseAddress
        - ec2:RevokeSecurityGroupIngress
        - ec2:RunInstances
        - ec2:TerminateInstances
        - tag:GetResources
        - elasticloadbalancing:AddTags
        - elasticloadbalancing:CreateLoadBalancer
        - elasticloadbalancing:ConfigureHealthCheck
        - elasticloadbalancing:DeleteLoadBalancer
        - elasticloadbalancing:DescribeLoadBalancers
        - elasticloadbalancing:DescribeLoadBalancerAttributes
        - elasticloadbalancing:DescribeTargetGroups
        - elasticloadbalancing:ApplySecurityGroupsToLoadBalancer
```

```
          - elasticloadbalancing:DescribeTags
          - elasticloadbalancing:ModifyLoadBalancerAttributes
          - elasticloadbalancing:RegisterInstancesWithLoadBalancer
          - elasticloadbalancing:DeregisterInstancesFromLoadBalancer
          - elasticloadbalancing:RemoveTags
          - autoscaling:DescribeAutoScalingGroups
          - autoscaling:DescribeInstanceRefreshes
          - ec2:CreateLaunchTemplate
          - ec2:CreateLaunchTemplateVersion
          - ec2:DescribeLaunchTemplates
          - ec2:DescribeLaunchTemplateVersions
          - ec2:DeleteLaunchTemplate
          - ec2:DeleteLaunchTemplateVersions
          - ec2:DescribeKeyPairs
          Effect: Allow
          Resource:
          - '*'
        - Action:
          - autoscaling:CreateAutoScalingGroup
          - autoscaling:UpdateAutoScalingGroup
          - autoscaling:CreateOrUpdateTags
          - autoscaling:StartInstanceRefresh
          - autoscaling:DeleteAutoScalingGroup
          - autoscaling:DeleteTags
          Effect: Allow
          Resource:
          - arn:*:autoscaling:*:*:autoScalingGroup:*:autoScalingGroupName/*
        - Action:
          - ecr:DescribeRepositories
          - ecr:CreateRepository
          - ecr:PutLife cyclePolicy
          - ecr:CompleteLayerUpload
          - ecr:GetAuthorizationToken
          - ecr:UploadLayerPart
          - ecr:InitiateLayerUpload
          - ecr:BatchCheckLayerAvailability
          - ecr:BatchGetImage
          - ecr:GetDownloadUrlForLayer
          - ecr:PutImage
          Effect: Allow
          Resource:
          - arn:aws:ecr:*:MYAWSACCOUNT:repository/*
        - Action:
          - iam:CreateServiceLinkedRole
          Condition:
            StringLike:
              iam:AWSServiceName: autoscaling.amazonaws.com
          Effect: Allow
          Resource:
          - arn:*:iam::*:role/aws-service-role/autoscaling.amazonaws.com/
AWSServiceRoleForAutoScaling
        - Action:
          - iam:CreateServiceLinkedRole
          Condition:
            StringLike:
              iam:AWSServiceName: elasticloadbalancing.amazonaws.com
          Effect: Allow
          Resource:
          - arn:*:iam::*:role/aws-service-role/elasticloadbalancing.amazonaws.com/
AWSServiceRoleForElasticLoadBalancing
        - Action:
          - iam:CreateServiceLinkedRole
```

```
          Condition:
            StringLike:
              iam:AWSServiceName: spot.amazonaws.com
          Effect: Allow
          Resource:
          - arn:*:iam::*:role/aws-service-role/spot.amazonaws.com/
AWSServiceRoleForEC2Spot
        - Action:
          - iam:PassRole
          Effect: Allow
          Resource:
          - arn:*:iam::*:role/*.cluster-api-provider-aws.sigs.k8s.io
        - Action:
          - secretsmanager:CreateSecret
          - secretsmanager:DeleteSecret
          - secretsmanager:TagResource
          Effect: Allow
          Resource:
          - arn:*:secretsmanager:*:*:secret:aws.cluster.x-k8s.io/*
        Version: 2012-10-17
      Roles:
      - Ref: NKPBootstrapRole
    Type: AWS::IAM::ManagedPolicy
  NKPBootstrapRole:
    Properties:
      AssumeRolePolicyDocument:
        Statement:
        - Action:
          - sts:AssumeRole
          Effect: Allow
          Principal:
            Service:
            - ec2.amazonaws.com
        - Action:
            - sts:AssumeRole
          Effect: Allow
          Principal:
                AWS: arn:aws:iam::MYAWSACCOUNT:root
        Version: 2012-10-17
      RoleName: nkp-bootstrapper-role
    Type: AWS::IAM::Role
```

If your organization uses Flatcar, add the following s3 permissions to your CloudFormation stack in the `nkp-bootstrapper-policy`.

```
- Action:
          - 's3:CreateBucket'
          - 's3:DeleteBucket'
          - 's3:PutObject'
          - 's3:DeleteObject'
          - 's3:PutBucketPolicy'
          Effect: Allow
          Resource:
          - 'arn:*:s3:::cluster-api-provider-aws-*'
```

2. Replace the following with the correct values.

   a. `MYFILENAME.yaml` - give your file a meaningful name.

   b. `MYSTACKNAME` - give your cloudformation stack a meaningful name.

   c. `MYAWSACCOUNT`- replace with an AWS Account ID number such as:`111122223333`

3. Run the following command to create the stack.

```
aws cloudformation create-stack --template-body=file://MYFILENAME.yaml --stack-
name=MYSTACKNAME --capabilities CAPABILITY_NAMED_IAM
```

Leverage the NKP Create Cluster Role

Use the `NKP-bootstrapper-role` for various access forms.

About this task

The created `nkp-bootstrapper-role` can be assumed by IAM users for temporary credentials through STS. Use temporary User Access Keys through STS below.

Procedure

1. Run the command below to use the `nkp-bootstrapper-role` .

```
aws sts assume-role --role-arn arn:aws:iam::MYAWSACCOUNT:role/nkp-bootstrapper-role
 --role-session-name EXAMPLE
```

Example output:

```
{
    "Credentials": {
        "AccessKeyId": "ASIA6RTF53ZH5B52EVM5",
        "SecretAccessKey": "BSssyvSsdfJY74jubsadfdsafdsaH7x1L+8Vk/",
        "SessionToken": "IQoJb3JpZ2z5cyChb9PtJvP0S6KAi",
        "Expiration": "2022-07-14T20:19:13+00:00"
    },
    "AssumedRoleUser": {
        "AssumedRoleId": "ASIA6RTF53ZH5B52EVM5:test",
        "Arn": "arn:aws:sts::MYAWSACCOUNTID:assumed-role/NKP-bootstrapper-role/test"
    }
}
```

2. Export the following environment variables with the results.

```
export AWS_ACCESS_KEY_ID=(.Credentials.AccessKeyId)

export AWS_SECRET_ACCESS_KEY=(.Credentials.SecretAccessKey)

export AWS_SESSION_TOKEN=(.Credentials.SessionToken)
```

> **Note:** These credentials are short-lived and need to be updated in the bootstrap cluster

Use EC2 Instance Profiles

Procedure

- The created `nkp-bootstrapper-role` can be assumed by an ec2 instance that a user then runs Nutanix Kubernetes Platform (NKP) create cluster commands from. To do this, specify the IAM Instance Profile `NKPBootstrapInstanceProfile` on creation.

Use Access Keys

About this task

AWS administrators can attach the `nkp-bootstrapper-policy` to an existing IAM user and authenticate with Access Keys on the workstation they run the `nkp create cluster` commands from by exporting the

following environment variables with the appropriate values for the IAM user. For more information, see https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_manage-attach-detach.html and https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_access-keys.html.

Procedure

- Export the environment variables.

```
export AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
export AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
export AWS_DEFAULT_REGION=us-west-2
```

> **Note:** Regarding Access Keys usage, Nutanix recommends that a system administrator always consider AWS's Best practices.

Suppose your organization uses encrypted AMIs (##Use encryption with EBS-backed AMIs - Amazon Elastic Compute Cloud#). In that case, you must add additional permissions to the control plane policy to allow access to the Amazon Key Management Services.

Return to EKS Cluster IAM Permissions and Roles or proceed to the next AWS step below.

For more information see:

- Encryption with EBS-backed AMIs - Amazon Elastic Compute Cloud: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIEncryption.html
- Amazon policies: https://docs.aws.amazon.com/kms/latest/developerguide/key-policy-default.html#key-policy-service-integration

AWS Cluster Identity and Access Management Policies and Roles

This guides a Nutanix Kubernetes Platform (NKP) user in creating Identity and Access Management (IAM) Policies and Instance Profiles used by the cluster's control plane and worker nodes using the provided Amazon Web Services (AWS) CloudFormation Stack. to ensure they are truly independent of other tenants and enforce security.

Prerequisites.

Before applying the IAM Policies, verify the following.

- You have a valid AWS account with credentials configured that can manage CloudFormation Stacks, IAM Policies, IAM Roles, and IAM Instance Profiles. For more information, see https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-files.html.
- You have the AWS CLI utility installed, see https://docs.aws.amazon.com/cli/latest/userguide/cli-chap-install.html.

Below is information regarding the setup for policies and roles. After reading the information for each area, you will find the CloudFormation Stack that creates these policies and roles in the IAM Artifacts topic.

Policies

1. `AWSIAMManagedPolicyCloudProviderControlPlane` enumerates the Actions required by the workload cluster control plane machines. It is attached to the `AWSIAMRoleControlPlane` Role.
2. `AWSIAMManagedPolicyCloudProviderNodes` enumerates the Actions required by the workload cluster worker machines. It is attached to the `AWSIAMRoleNodes` Role.
3. `AWSIAMManagedPolicyControllers` enumerates the Actions required by the workload cluster worker machines. It is attached to the `AWSIAMRoleControlPlane` Role.

Roles

1. `AWSIAMRoleControlPlane` is the Role associated with the `AWSIAMInstanceProfileControlPlane` Instance Profile.
2. `AWSIAMRoleNodes` is the Role associated with the `AWSIAMInstanceProfileNodes` Instance Profile.

For more information on how to grant cluster access to IAM users and roles, see https://docs.aws.amazon.com/eks/latest/userguide/add-user-role.html.

Creating AWS IAM Artifacts

The CloudFormation Stack from AWS Cluster IAM Policies and Roles creates these policies and roles in the IAM Artifacts stack below.

About this task

This guides a Nutanix Kubernetes Platform (NKP) user in creating Instance Profiles used by the cluster's control plane and worker nodes using the provided AWS CloudFormation Stack.

Procedure

1. `AWSIAMInstanceProfileControlPlane`, assigned to workload cluster control plane machines.

   » **Important:** If the name is changed from the default, used below, it must be passed to `nkp create cluster` with the `--control-plane-iam-instance-profile` flag.

2. `AWSIAMInstanceProfileNodes`, assigned to workload cluster worker machines.

   » If the name is changed from the default, used below, it must be passed to `nkp create cluster` with the `--worker-iam-instance-profile` flag.

```
AWSTemplateFormatVersion: 2010-09-09
Resources:
  AWSIAMInstanceProfileControlPlane:
    Properties:
      InstanceProfileName: control-plane.cluster-api-provider-aws.sigs.k8s.io
      Roles:
      - Ref: AWSIAMRoleControlPlane
    Type: AWS::IAM::InstanceProfile
  AWSIAMInstanceProfileNodes:
    Properties:
      InstanceProfileName: nodes.cluster-api-provider-aws.sigs.k8s.io
      Roles:
      - Ref: AWSIAMRoleNodes
    Type: AWS::IAM::InstanceProfile
  AWSIAMManagedPolicyCloudProviderControlPlane:
    Properties:
      Description: For the Kubernetes Cloud Provider AWS Control Plane
      ManagedPolicyName: control-plane.cluster-api-provider-aws.sigs.k8s.io
      PolicyDocument:
        Statement:
        - Action:
          - autoscaling:DescribeAutoScalingGroups
          - autoscaling:DescribeLaunchConfigurations
          - autoscaling:DescribeTags
          - ec2:DescribeInstances
          - ec2:DescribeImages
          - ec2:DescribeRegions
          - ec2:DescribeRouteTables
          - ec2:DescribeSecurityGroups
          - ec2:DescribeSubnets
          - ec2:DescribeVolumes
```

```
        - ec2:CreateSecurityGroup
        - ec2:CreateTags
        - ec2:CreateVolume
        - ec2:ModifyInstanceAttribute
        - ec2:ModifyVolume
        - ec2:AttachVolume
        - ec2:AuthorizeSecurityGroupIngress
        - ec2:CreateRoute
        - ec2:DeleteRoute
        - ec2:DeleteSecurityGroup
        - ec2:DeleteVolume
        - ec2:DetachVolume
        - ec2:RevokeSecurityGroupIngress
        - ec2:DescribeVpcs
        - elasticloadbalancing:AddTags
        - elasticloadbalancing:AttachLoadBalancerToSubnets
        - elasticloadbalancing:ApplySecurityGroupsToLoadBalancer
        - elasticloadbalancing:CreateLoadBalancer
        - elasticloadbalancing:CreateLoadBalancerPolicy
        - elasticloadbalancing:CreateLoadBalancerListeners
        - elasticloadbalancing:ConfigureHealthCheck
        - elasticloadbalancing:DeleteLoadBalancer
        - elasticloadbalancing:DeleteLoadBalancerListeners
        - elasticloadbalancing:DescribeLoadBalancers
        - elasticloadbalancing:DescribeLoadBalancerAttributes
        - elasticloadbalancing:DetachLoadBalancerFromSubnets
        - elasticloadbalancing:DeregisterInstancesFromLoadBalancer
        - elasticloadbalancing:ModifyLoadBalancerAttributes
        - elasticloadbalancing:RegisterInstancesWithLoadBalancer
        - elasticloadbalancing:SetLoadBalancerPoliciesForBackendServer
        - elasticloadbalancing:AddTags
        - elasticloadbalancing:CreateListener
        - elasticloadbalancing:CreateTargetGroup
        - elasticloadbalancing:DeleteListener
        - elasticloadbalancing:DeleteTargetGroup
        - elasticloadbalancing:DescribeListeners
        - elasticloadbalancing:DescribeLoadBalancerPolicies
        - elasticloadbalancing:DescribeTargetGroups
        - elasticloadbalancing:DescribeTargetHealth
        - elasticloadbalancing:ModifyListener
        - elasticloadbalancing:ModifyTargetGroup
        - elasticloadbalancing:RegisterTargets
        - elasticloadbalancing:SetLoadBalancerPoliciesOfListener
        - iam:CreateServiceLinkedRole
        - kms:DescribeKey
        - kms:CreateGrant
        Effect: Allow
        Resource:
        - '*'
      Version: 2012-10-17
    Roles:
    - Ref: AWSIAMRoleControlPlane
  Type: AWS::IAM::ManagedPolicy
AWSIAMManagedPolicyCloudProviderNodes:
  Properties:
    Description: For the Kubernetes Cloud Provider AWS nodes
    ManagedPolicyName: nodes.cluster-api-provider-aws.sigs.k8s.io
    PolicyDocument:
      Statement:
      - Action:
        - ec2:DescribeInstances
        - ec2:DescribeRegions
```

```
        - ecr:GetAuthorizationToken
        - ecr:BatchCheckLayerAvailability
        - ecr:GetDownloadUrlForLayer
        - ecr:GetRepositoryPolicy
        - ecr:DescribeRepositories
        - ecr:ListImages
        - ecr:BatchGetImage
        Effect: Allow
        Resource:
        - '*'
      - Action:
        - secretsmanager:DeleteSecret
        - secretsmanager:GetSecretValue
        Effect: Allow
        Resource:
        - arn:*:secretsmanager:*:*:secret:aws.cluster.x-k8s.io/*
      - Action:
        - ssm:UpdateInstanceInformation
        - ssmmessages:CreateControlChannel
        - ssmmessages:CreateDataChannel
        - ssmmessages:OpenControlChannel
        - ssmmessages:OpenDataChannel
        - s3:GetEncryptionConfiguration
        Effect: Allow
        Resource:
        - '*'
      Version: 2012-10-17
    Roles:
    - Ref: AWSIAMRoleControlPlane
    - Ref: AWSIAMRoleNodes
  Type: AWS::IAM::ManagedPolicy
AWSIAMManagedPolicyControllers:
  Properties:
    Description: For the Kubernetes Cluster API Provider AWS Controllers
    ManagedPolicyName: controllers.cluster-api-provider-aws.sigs.k8s.io
    PolicyDocument:
      Statement:
      - Action:
        - ec2:AllocateAddress
        - ec2:AssociateRouteTable
        - ec2:AttachInternetGateway
        - ec2:AuthorizeSecurityGroupIngress
        - ec2:CreateInternetGateway
        - ec2:CreateNatGateway
        - ec2:CreateRoute
        - ec2:CreateRouteTable
        - ec2:CreateSecurityGroup
        - ec2:CreateSubnet
        - ec2:CreateTags
        - ec2:CreateVpc
        - ec2:ModifyVpcAttribute
        - ec2:DeleteInternetGateway
        - ec2:DeleteNatGateway
        - ec2:DeleteRouteTable
        - ec2:DeleteSecurityGroup
        - ec2:DeleteSubnet
        - ec2:DeleteTags
        - ec2:DeleteVpc
        - ec2:DescribeAccountAttributes
        - ec2:DescribeAddresses
        - ec2:DescribeAvailabilityZones
        - ec2:DescribeInstanceTypes
```

```
        - ec2:DescribeInternetGateways
        - ec2:DescribeImages
        - ec2:DescribeNatGateways
        - ec2:DescribeNetworkInterfaces
        - ec2:DescribeNetworkInterfaceAttribute
        - ec2:DescribeRouteTables
        - ec2:DescribeSecurityGroups
        - ec2:DescribeSubnets
        - ec2:DescribeVpcs
        - ec2:DescribeVpcAttribute
        - ec2:DescribeVolumes
        - ec2:DetachInternetGateway
        - ec2:DisassociateRouteTable
        - ec2:DisassociateAddress
        - ec2:ModifyInstanceAttribute
        - ec2:ModifyInstanceMetadataOptions
        - ec2:ModifyNetworkInterfaceAttribute
        - ec2:ModifySubnetAttribute
        - ec2:ReleaseAddress
        - ec2:RevokeSecurityGroupIngress
        - ec2:RunInstances
        - ec2:TerminateInstances
        - tag:GetResources
        - elasticloadbalancing:AddTags
        - elasticloadbalancing:CreateLoadBalancer
        - elasticloadbalancing:ConfigureHealthCheck
        - elasticloadbalancing:DeleteLoadBalancer
        - elasticloadbalancing:DescribeLoadBalancers
        - elasticloadbalancing:DescribeLoadBalancerAttributes
        - elasticloadbalancing:DescribeTargetGroups
        - elasticloadbalancing:ApplySecurityGroupsToLoadBalancer
        - elasticloadbalancing:DescribeTags
        - elasticloadbalancing:ModifyLoadBalancerAttributes
        - elasticloadbalancing:RegisterInstancesWithLoadBalancer
        - elasticloadbalancing:DeregisterInstancesFromLoadBalancer
        - elasticloadbalancing:RemoveTags
        - autoscaling:DescribeAutoScalingGroups
        - autoscaling:DescribeInstanceRefreshes
        - ec2:CreateLaunchTemplate
        - ec2:CreateLaunchTemplateVersion
        - ec2:DescribeLaunchTemplates
        - ec2:DescribeLaunchTemplateVersions
        - ec2:DeleteLaunchTemplate
        - ec2:DeleteLaunchTemplateVersions
        Effect: Allow
        Resource:
        - '*'
      - Action:
        - autoscaling:CreateAutoScalingGroup
        - autoscaling:UpdateAutoScalingGroup
        - autoscaling:CreateOrUpdateTags
        - autoscaling:StartInstanceRefresh
        - autoscaling:DeleteAutoScalingGroup
        - autoscaling:DeleteTags
        Effect: Allow
        Resource:
        - arn:*:autoscaling:*:*:autoScalingGroup:*:autoScalingGroupName/*
      - Action:
        - iam:CreateServiceLinkedRole
        Condition:
          StringLike:
            iam:AWSServiceName: autoscaling.amazonaws.com
```

```
          Effect: Allow
          Resource:
          - arn:*:iam::*:role/aws-service-role/autoscaling.amazonaws.com/
AWSServiceRoleForAutoScaling
        - Action:
          - iam:CreateServiceLinkedRole
          Condition:
            StringLike:
              iam:AWSServiceName: elasticloadbalancing.amazonaws.com
          Effect: Allow
          Resource:
          - arn:*:iam::*:role/aws-service-role/elasticloadbalancing.amazonaws.com/
AWSServiceRoleForElasticLoadBalancing
        - Action:
          - iam:CreateServiceLinkedRole
          Condition:
            StringLike:
              iam:AWSServiceName: spot.amazonaws.com
          Effect: Allow
          Resource:
          - arn:*:iam::*:role/aws-service-role/spot.amazonaws.com/
AWSServiceRoleForEC2Spot
        - Action:
          - iam:PassRole
          Effect: Allow
          Resource:
          - arn:*:iam::*:role/*.cluster-api-provider-aws.sigs.k8s.io
        - Action:
          - secretsmanager:CreateSecret
          - secretsmanager:DeleteSecret
          - secretsmanager:TagResource
          Effect: Allow
          Resource:
          - arn:*:secretsmanager:*:*:secret:aws.cluster.x-k8s.io/*
        Version: 2012-10-17
      Roles:
      - Ref: AWSIAMRoleControlPlane
    Type: AWS::IAM::ManagedPolicy
  AWSIAMRoleControlPlane:
    Properties:
      AssumeRolePolicyDocument:
        Statement:
        - Action:
          - sts:AssumeRole
          Effect: Allow
          Principal:
            Service:
            - ec2.amazonaws.com
        Version: 2012-10-17
      RoleName: control-plane.cluster-api-provider-aws.sigs.k8s.io
    Type: AWS::IAM::Role
  AWSIAMRoleNodes:
    Properties:
      AssumeRolePolicyDocument:
        Statement:
        - Action:
          - sts:AssumeRole
          Effect: Allow
          Principal:
            Service:
            - ec2.amazonaws.com
        Version: 2012-10-17
```

```
      RoleName: nodes.cluster-api-provider-aws.sigs.k8s.io
    Type: AWS::IAM::Role
```

3. To create the resources in the cloudformation stack copy the contents above into a file replacing `MYFILENAME.yaml` and `MYSTACKNAME` with the intended values. Then, execute the following command.

```
 aws cloudformation create-stack --template-body=file://MYFILENAME.yaml --stack-
name=MYSTACKNAME --capabilities  CAPABILITY_NAMED_IAM
```

> **Caution:** If your organization uses encrypted AMIs, you must add additional permissions to the control plane policy `control-plane.cluster-api-provider-aws.sigs.k8s.io` to allow access to the Amazon Key Management Services. The code snippet shows how to add a particular key ARN to encrypt and decrypt AMIs.

```
---
Action:
  - kms:CreateGrant
  - kms:DescribeKey
  - kms:Encrypt
  - kms:Decrypt
  - kms:ReEncrypt*
  - kms:GenerateDataKey*
  Resource:
  - arn:aws:kms:us-west-2:111122223333:key/key-arn
  Effect: Allow
```

> **Caution:** If your organization uses Flatcar, then you will need to add additional permissions to the control plane policy `control-plane.cluster-api-provider-aws.sigs.k8s.io`

For flatcar, when using the default object storage, add the following permissions to the IAM Role `control-plane.cluster-api-provider-aws.sigs.k8s.io`:

```
     PolicyDocument:
       Statement:
       ...
       - Action:
         - 's3:CreateBucket'
         - 's3:DeleteBucket'
         - 's3:PutObject'
         - 's3:DeleteObject'
         - 's3:PutBucketPolicy'
         - 's3:PutBucketTagging'
         - 'ec2:CreateVpcEndpoint'
         - 'ec2:ModifyVpcEndpoint'
         - 'ec2:DeleteVpcEndpoints'
         - 'ec2:DescribeVpcEndpoints'
         Effect: Allow
         Resource:
         - 'arn:*:s3:::cluster-api-provider-aws-*'
```

EKS IAM Policies and Instance Profiles that govern who has access to the cluster are found here: EKS Cluster IAM Permissions and Roles. If attaching an EKS cluster, that CloudStack Formation will also need to be run to create the ARN of the bootstrapper role.

If installing EKS, return to EKS - EKS Cluster IAM Permissions and Roles or proceed to the next AWS steps below.

Using an AWS Elastic Container Registry

Use an AWS Elastic Container Registry (ECR) registry for both air-gapped and non-air-gapped, if desired, environments.

About this task

Amazon Web Services (AWS) Elastic Container Registry (ECR) is supported as your air-gapped image registry or a non-air-gapped registry mirror. Nutanix Kubernetes Platform (NKP) added support for using AWS ECR as a default registry when uploading image bundles in AWS.

Because air-gapped environments do not have direct access to the Internet, you must download, extract, and load several required images to your local container registrybefore installing NKP.

This page is to explain ECR specifics but assumes you have already downloaded and extracted the bundle from the Prerequisites. The sections below explain how you push the images to your AWS ECR registry and then use them to create a cluster.

Before you begin

- Ensure you have followed the steps to create proper permissions in AWS Minimal Permissions and Role to Create Clusters
- Ensure you have created AWS Cluster IAM Policies, Roles, and Artifacts

Upload the Air-gapped Image Bundle to the Local ECR Registry.

Procedure

A cluster administrator uses NKP CLI commands to upload the image bundle to ECR with parameters.

```
nkp push bundle --bundle <bundle> --to-registry=<ecr-registry-address>/<ecr-registry-name>
```

Parameter definitions.

» `--bundle <bundle>` the group of images. The example below is for the NKP air-gapped environment bundle. `--to-registry=<ecr-registry-address>/<ecr-registry-name>` to provide registry location for push

```
nkp push bundle --bundle container-images/konvoy-image-bundle-v2.12.0.tar --to-registry=333000009999.dkr.ecr.us-west-2.amazonaws.com/can-test
```

» You can also set an environment variable with your registry address for ECR.

```
export REGISTRY_URL=<ecr-registry-URI>
```

»

Note: `REGISTRY_URL`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.

The environment where you run the `nkp push` command must be authenticated with AWS to load your images into ECR.

Note: The cluster administrator uses existing NKP CLI commands to create the cluster and refer to their internal ECR for the image repository. The administrator does not need to provide static ECR registry credentials. See Use a Registry Mirror and Create an EKS Cluster from the CLI for more details.

Exporting Variables to Use as Flags in Cluster Creation

About this task

Below is an AWS ECR example:

Tip:

```
export REGISTRY_URL=<ecr-registry-URI>
```

Procedure

- `REGISTRY_URL`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.
- NOTE: Other local registries might use the options below.
  a. JFrog - `REGISTRY_CA`: (optional) the path on the Creating a Bastion Host on page 685 to the registry CA. This value is only needed if the registry uses a self-signed certificate and the AMIs are not already configured to trust this CA.
  b. `REGISTRY_USERNAME`: optional, set to a user with pull access to this registry.
  c. `REGISTRY_PASSWORD`: optional if username is not set.

AWS Installation in a Non-air-gapped Environment

This section provides installation instructions for installing Nutanix Kubernetes Platform (NKP) in an Amazon Web Services (AWS) non-air-gapped environment.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

AWS Prerequisites

Before you begin using Konvoy with AWS, you must:

1. Follow the steps to create permissions and roles on the Minimal Permissions and Role to Create Clusters page.
2. Create Cluster IAM Policies and Roles
3. Export the AWS region where you want to deploy the cluster:

   ```
   export AWS_REGION=us-west-2
   ```

4. Export the AWS profile with the credentials you want to use to create the Kubernetes cluster:

   ```
   export AWS_PROFILE=<profile>
   ```

If using AWS ECR as your local private registry, more information can be found on the Registry Mirror Tools page.

To deploy a cluster with a custom image in a region where CAPI images are not provided, you need to use Konvoy Image Builder to create your image for the region.

Note: For multi-tenancy, every tenant needs to be in a different AWS account to ensure they are truly independent of other tenants and enforce security.

Using Custom AMI in Cluster Creation

The default Amazon Web Services (AWS) image is not recommended for use in production. Nutanix suggests using Konvoy Image Builder to create a custom AMI and take advantage of enhanced cluster operations.

About this task

Depending on which version of Nutanix Kubernetes Platform (NKP) you are running, steps and flags will be different. To deploy in a region where CAPI images are not provided, you need to use KIB to create your image for the region. For a list of supported AWS regions, refer to the Published AMI information from AWS. To begin image creation.

Procedure

1. Run the `konvoy-image`command to build and validate the image.

   For example.

   ```
   konvoy-image build aws images/ami/rhel-86.yaml
   ```

   By default, it builds in the `us-west-2` region. To specify another region, set the `--region` flag as shown in the command below.

   ```
   konvoy-image build aws --region us-east-1 images/ami/rhel-86.yaml
   ```

   > **Tip:** Ensure you have named the correct AMI image YAML file for your OS in the `konvoy-image build` command. For more information, see https://github.com/mesosphere/konvoy-image-builder/tree/main/images/ami

   After KIB provisions the image successfully, the `ami` id is printed and written to the `packer.pkr.hcl` (Packer config) file. This file has an `artifact_id` field whose value provides the name of the AMI ID, as shown in the example below. That is the `ami` you use in the `create cluster` command.

   ```
   {
     "builds": [
       {
         "name": "kib_image",
         "builder_type": "amazon-ebs",
         "build_time": 1698086886,
         "files": null,
         "artifact_id": "us-west-2:ami-04b8dfef8bd33a016",
         "packer_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05",
         "custom_data": {
           "containerd_version": "",
           "distribution": "RHEL",
           "distribution_version": "8.6",
           "kubernetes_cni_version": "",
           "kubernetes_version": "1.26.6"
         }
       }
     ],
     "last_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05"
   }
   ```

2. To use a custom AMI when creating your cluster, you must create that AMI using KIB first. Then perform the export and name the custom AMI for use in the command `nkp create cluster`

   For example:

   ```
   export AWS_AMI_ID=ami-<ami-id-here>
   ```

3. Inside the sections for either Non-air-gapped or Air-gapped cluster creation, you will find the instructions for how to apply custom images.

4. YAML for KIB AMI images found here in GitHub https://github.com/mesosphere/konvoy-image-builder/tree/7447074a6d910e71ad2e61fc4a12820d073ae5ae/images/ami

Bootstrapping AWS

To create Kubernetes clusters, NKP uses Cluster API (CAPI) controllers. These controllers run on a Kubernetes cluster.

About this task

To get started, you need a bootstrap cluster. By default, Nutanix Kubernetes Platform (NKP) creates a bootstrap cluster for you in a Docker container using the Kubernetes-in-Docker (KIND) tool.

Prerequisites:

Before you begin, you must:

Procedure

1. Complete the Nutanix Infrastructure Prerequisites. For more information, see Nutanix Infrastructure Prerequisites on page 690.
2. Ensure the `NKP` binary can be found in your `$PATH`.

Bootstrap Cluster Life Cycle Services

Procedure

1. Review Universal Configurations for all Infrastructure Providers regarding settings, flags, and other choices and then begin bootstrapping.
2. Create a bootstrap cluster using the command `nkp create bootstrap --kubeconfig $HOME/.kube/config`.

   > **Note:** Use `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Configuring an HTTP or HTTPS Proxy on page 677.

   Example output:

   ```
   # Creating a bootstrap cluster
   # Initializing new CAPI components
   ```

3. NKP creates a bootstrap cluster using KIND as a library.

   For more information, see https://github.com/kubernetes-sigs/kind

4. NKP then deploys the following Cluster API providers on the cluster.

   - Core Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/
   - AWS Infrastructure Provider: https://github.com/kubernetes-sigs/cluster-api-provider-aws
   - Kubeadm Bootstrap Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/bootstrap/kubeadm
   - Kubeadm ControlPlane Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/controlplane/kubeadm

   For more information on Cluster APIs, see https://cluster-api.sigs.k8s.io/

5. NKP waits until these providers' controller-manager and webhook deployments are ready. List these deployments using the command `kubectl get --all-namespaces deployments -l=clusterctl.cluster.x-k8s.io`.

   Output example:

```
NAMESPACE                           NAME
 READY   UP-TO-DATE   AVAILABLE   AGE
capa-system                         capa-controller-manager
 1/1     1            1           1h
capg-system                         capg-controller-manager
 1/1     1            1           1h
capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-manager
 1/1     1            1           1h
capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-manager
 1/1     1            1           1h
capi-system                         capi-controller-manager
 1/1     1            1           1h
cappp-system                        cappp-controller-manager
 1/1     1            1           1h
capv-system                         capv-controller-manager
 1/1     1            1           1h
capz-system                         capz-controller-manager
 1/1     1            1           1h
cert-manager                        cert-manager
 1/1     1            1           1h
cert-manager                        cert-manager-cainjector
 1/1     1            1           1h
cert-manager                        cert-manager-webhook
 1/1     1            1           1h
```

Creating a New AWS Cluster

Create an AWS Cluster in a non-air-gapped environment.

About this task

Use this procedure to create a custom Amazon Web Services (AWS) cluster with Nutanix Kubernetes Platform (NKP).

Suppose you use these instructions to create a cluster on AWS using the ##NKP# default settings without any edits to configuration files or additional flags. Your cluster is deployed on an Ubuntu 20.04 operating system image with three control plane nodes and four worker nodes. For more information, see Supported Infrastructure Operating Systems on page 12

By default, the control-plane Nodes will be created in 3 different zones. However, the default worker Nodes will reside in a single zone. You may create additional node pools in other zones with the `nkp create nodepool` command.

Availability zones (AZs) are isolated locations within datacenter regions where public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single Availability Zone, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple Availability Zones.

> **Warning:** In previous NKP releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder.

> **Note:** NKP uses the AWS CSI driver as the Provider. Use a Kubernetes CSI-compatible storage that is suitable for production. For more information, see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types.

Before you begin

First, you must name your cluster.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. For Kubernetes naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/ .

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Ensure your AWS credentials are up to date.
   a. If you are using Static Credentials, refresh the credentials using the command `nkp update bootstrap credentials aws`
3. Set the environment variable of cluster name selected according to requirements and custom AMI identification using the command `export CLUSTER_NAME=<aws-example> export AWS_AMI_ID=<ami-...>`.
4. Supplying the ID of your AMI using one of the two methods.

   » Option One - Provide the ID of your AMI using the `--ami AMI_ID` command and leave the existing flag that provides the AMI ID.

   » Option Two - Provide a path for your AMI with the information required for NKP to discover the AMI using location, format, and OS information.

   Where the AMI is published using your AWS Account ID: `--ami-owner AWS_ACCOUNT_ID`

   The format or string used to search for matching AMIs and ensure it references the Kubernetes version example `--ami-base-os ubuntu-20.04` plus the base OS information example `--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*'` .

   Note:

   - The AMI must be created with Konvoy Image Builder to use the registry mirror feature.

     Example:

     ```
     export AWS_AMI_ID=<ami-...>
     ```

   - (Optional) Registry Mirror - Configure your cluster to use an existing local registry as a mirror when attempting to pull images.

     AWS ECR example output shows the `REGISTRY_URL` of an existing local registry is accessible in the VPC, where the new cluster nodes will be configured to use a mirror registry when pulling images.

     Example:

     ```
     export REGISTRY_URL=<ecr-registry-URI>
     ```

5. Ensure your subnets do not overlap with your host subnet because they cannot be changed after cluster creation.

   If you need to change the kubernetes subnets, you must do this at cluster creation. To review the default subnets used in NKP, see Managing Subnets and Pods on page 684.

   Example output:

   ```
   spec:
     clusterNetwork:
       pods:
         cidrBlocks:
         - 192.168.0.0/16
       services:
         cidrBlocks:
         - 10.96.0.0/12
   ```

   - Additional Options for your environment:

     - (Optional) Modify control plane audit logs: Modify the `KubeadmControlplane` cluster-API object to configure different kubelet options. For information beyond the existing options available from flags, see Configure the Control Plane on page 1074.
     - (Optional) Configure your cluster to use an existing local registry as a mirror when pulling images previously pushed to your registry. Set an environment variable with your registry address for ECR using the command `export REGISTRY_URL=<ecr-registry-URI>`. For more information, see Registry Mirror Tools on page 1069.

6. Create a Kubernetes cluster object with a `dry run` output for customizations.

   Example output:

   ```
   nkp create cluster aws \
   --cluster-name=${CLUSTER_NAME} \
   --ami=${AWS_AMI_ID} \
   --dry-run \
   --output=yaml \
   > ${CLUSTER_NAME}.yaml
   ```

   If providing the AMI path, use these flags in place of `AWS_AMI_ID`:

   ```
   --ami-owner AWS_ACCOUNT_ID \
   --ami-base-os ubuntu-20.04 \
   --ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*' \
   ```

   Note: More flags can be added to the `nkp create cluster` command. See the optional choices below or the complete list in the Universal Configurations for all Infrastructure Providers on page 677:

   - If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values for it to be successful. For more information, see Configuring an HTTP or HTTPS Proxy.
   - FIPS flags - To create a cluster in FIPS mode, inform the controllers of the appropriate image repository and version tags of the official Nutanix FIPS builds of Kubernetes by adding flags to the command `nkp create a cluster`. `--kubernetes-version=v1.29.6+fips.0 \ --etcd-version=3.5.10+fips.0`
   - You can create individual manifest files with different smaller manifests for ease in editing using the `--output-directory` flag. For more information, see Output Directory Flag on page 682.

7. Inspect or edit the cluster objects. Familiarize yourself with Cluster API before editing the cluster objects, as edits can prevent the cluster from deploying successfully. For more information, see Customization of Cluster CAPI Components on page 682.

8. Create the cluster from the objects generated from the dry run using the command `kubectl create -f ${CLUSTER_NAME}.yaml`

   If the resource already exists, a warning appears in the console, requiring you to remove it or update your YAML.

   Note: If you used the `--output-directory` flag in your `NKP create .. --dry-run` step above, create the cluster from the objects you created by specifying the directory:

   ```
   kubectl create -f <existing-directory>/.
   ```

9. Wait for the cluster control plan to be ready.

   ```
   kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --timeout=20m
   ```

10. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. Konvoy provides a command to describe the current status of the cluster.

    ```
    nkp describe cluster -c ${CLUSTER_NAME}
    ```

    Output:

    ```
    NAME                                                          READY  SEVERITY
     REASON  SINCE  MESSAGE
    Cluster/aws-example                                          True
          60s
    ##ClusterInfrastructure - AWSCluster/aws-example              True
          5m23s
    ##ControlPlane - KubeadmControlPlane/aws-example-control-plane  True
          60s
    # ##Machine/aws-example-control-plane-55jh4                   True
          4m59s
    # ##Machine/aws-example-control-plane-6sn97                   True
          2m49s
    # ##Machine/aws-example-control-plane-nx9v5                   True
          66s
    ##Workers
      ##MachineDeployment/aws-example-md-0                        True
          117s
     ##Machine/aws-example-md-0-cb9c9bbf7-hcl8z                 True
       3m1s
     ##Machine/aws-example-md-0-cb9c9bbf7-rtdqw                 True
       3m2s
     ##Machine/aws-example-md-0-cb9c9bbf7-t894m                 True
       3m1s
     ##Machine/aws-example-md-0-cb9c9bbf7-td29r                 True
    ```

    Note: NKP uses the AWS CSI driver as the default storage provider. Use a Kubernetes CSI-compatible storage that is suitable for production. If you're not using the default, you cannot deploy an alternate provider until after the `nkp create cluster` is finished. However, this must be determined before the Kommander installation. For more information, see:

    - Kubernetes CSI:https://kubernetes.io/docs/concepts/storage/volumes/#volume-types

- Changing the Default Storage Class: https://kubernetes.io/docs/tasks/administer-cluster/change-default-storage-class/

Note: To increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username=<username> --registry-mirror-password=<password>` on the `nkp create cluster` command. For more information, see https://docs.docker.com/docker-hub/download-rate-limit/.

Creating NKP Non-air-gapped Clusters from the UI

A guide for creating Nutanix Kubernetes Platform (NKP) clusters on the AWS console.

About this task

To create clusters on AWS from the User Interface (UI) rather than the CLI, follow these steps.

Before you begin

Configure an AWS Infrastructure, or add credentials using AWS role credentials.

Procedure

1. In the selected workspace Dashboard, select the **Add Cluster** option in the Actions dropdown menu at the top right.
2. On the **Add Cluster** page, select the **Create NKP Cluster**.
3. Provide some basic cluster details within the form.
   a. **Workspace**: The workspace where this cluster belongs (if within the Global workspace).
   b. **Kubernetes Version**: The initial Kubernetes version will be installed on the cluster.
   c. **Name**: A valid Kubernetes name for the cluster.
   d. **Add Labels**: By default, your cluster has labels that reflect the infrastructure provider provisioning. For example, your AWS cluster have a label for the datacenter region and provider: aws. Cluster labels are matched to the selectors created for Projects. Changing a cluster label may add or remove the cluster from projects.

4. Select the pre-configured AWS Infrastructureor AWS role credentials to display the remaining options specific to AWS.

   a. **Region**: Select a datacenter region or specify a custom region.

   b. **Configure Node Pools**: Specify pools of nodes, their machine types, quantity, and the IAM instance profile.

      - **Machine Type**: Machine instance type.
      - **Quantity**: Number of nodes. The control plane must be an odd number.
      - **IAM instance profile**: Name the IAM instance profile to assign to the machines.

   c. **AMI Image**: You specify the AMI as part of each pool by using either group of fields but not both groups.

      » AMI ID specifying by name.

        - **AMI ID**: AMI ID to use for all nodes.

      » AMI ID for lookup - Owner ID, Base OS, Lookup Format (all fields are required, if any are used)

        - **Base OS**: Base OS for Lookup search.

        - **Lookup Format**: Lookup Format string to generate AMI search name from.

        - **Owner ID**: Owner ID for AMI Lookup search.

   d. **Worker Availability Zone**: Worker Zones for a region (worker nodes only).

   e. **Add Infrastructure Provider Tags**: Specify tags applied on all resources created in your infrastructure for this cluster. Different infrastructure providers have varying restrictions on the usable tags. See the AWS Tags User Guide for more information on using tags in AWS.

5. Select **Create** to begin provisioning the cluster. This step may take a few minutes, taking time for the cluster to be ready and fully deploy its components. The cluster automatically tries to join and resolve after it is fully provisioned.

Making the New AWS Cluster Self-Managed

How to make a Kubernetes cluster manage itself.

About this task

Nutanix Kubernetes Platform (NKP) deploys all cluster life cycle services to a bootstrap cluster, which then deploys a workload cluster. When the workload cluster is ready, move the cluster life cycle services to the workload cluster, which makes the workload cluster self-managed.

Before you begin

Before starting, ensure you can create a workload cluster as described in the topic: Create a New AWS Cluster.

This page contains instructions on how to make your cluster self-managed. This is necessary if there is only one cluster in your environment or if this cluster becomes the Management cluster in a multi-cluster environment.

> **Note:** If you already have a self-managed or Management cluster in your environment, skip this page.

Follow these steps to turn your new cluster into a **Management Cluster** for an Ultimate license environment (or a free-standing Pro Cluster):

Procedure

1. Deploy cluster life cycle services on the workload cluster.

   ```
   nkp create capi-components --kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Initializing new CAPI components
   ```

   > **Note:** If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP/HTTPS Proxy.

2. The cluster life cycle services on the workload cluster are ready, but the workload cluster configuration is on the bootstrap cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the bootstrap to the workload cluster. This process is called a Pivot. For more information, see https://cluster-api.sigs.k8s.io/reference/glossary.html?highlight=pivot#pivot.

   Move the Cluster API objects from the bootstrap to the workload cluster:

   ```
   nkp move capi-resources --to-kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Moving cluster resources
   You can now view resources in the moved cluster by using the --kubeconfig flag with
    kubectl. For example: kubectl --kubeconfig=gcp-example.conf get nodes
   ```

   > **Note:** To ensure only one set of cluster life cycle services manages the workload cluster, NKP first pauses the reconciliation of the objects on the bootstrap cluster, then creates the objects on the workload cluster. As NKP copies the objects, the cluster life cycle services on the workload cluster reconcile the objects. The workload cluster becomes self-managed after NKP creates all the objects. If it fails, the `move` command can be safely retried.

3. Wait for the cluster control plane to be ready.

   ```
   kubectl --kubeconfig ${CLUSTER_NAME}.conf wait --for=condition=ControlPlaneReady
    "clusters/${CLUSTER_NAME}" --timeout=20m
   ```

   Output:

   ```
   cluster.cluster.x-k8s.io/gcp-example condition met
   ```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster status. After moving the cluster life cycle services to the workload cluster, remember to use NKP with the workload cluster `kubeconfig`.

   ```
   nkp describe cluster --kubeconfig ${CLUSTER_NAME}.conf -c ${CLUSTER_NAME}
   ```

   Output:

   ```
   NAME                                                                READY  SEVERITY
    REASON  SINCE  MESSAGE
   Cluster/aws-example                                                 True
       109s
   ##ClusterInfrastructure - AWSCluster/aws-example                    True
       112s
   ##ControlPlane - KubeadmControlPlane/aws-example-control-plane      True
       109s
   # ##Machine/aws-example-control-plane-55jh4                         True
       111s
   # ##Machine/aws-example-control-plane-6sn97                         True
       111s
   ```

```
# ##Machine/aws-example-control-plane-nx9v5                    True
    110s
##Workers
  ##MachineDeployment/aws-example-md-0                         True
    114s
 ##Machine/aws-example-md-0-cb9c9bbf7-hcl8z                 True
 111s
 ##Machine/aws-example-md-0-cb9c9bbf7-rtdqw                 True
 111s
 ##Machine/aws-example-md-0-cb9c9bbf7-t894m                 True
 111s
 ##Machine/aws-example-md-0-cb9c9bbf7-td29r                 True
 111s
```

5. Remove the bootstrap cluster because the workload cluster is now self-managed.

```
nkp delete bootstrap --kubeconfig $HOME/.kube/config
# Deleting bootstrap cluster
```

Known Limitations

Procedure

- NKP only supports moving all namespaces in the cluster; NKP does not support migration of individual namespaces.
- Konvoy supports moving only one set of cluster objects from the bootstrap cluster to the workload cluster or vice-versa.

Exploring the AWS Cluster

This guide explains how to use the command line to interact with your newly deployed Kubernetes cluster.

About this task

Before you start, make sure you have created a workload cluster, as described in Create a New AWS Cluster.

Procedure

1. When the workload cluster is created, the cluster life cycle services generate a `kubeconfig` file for the workload cluster and write it to a Secret. The `kubeconfig` file is scoped to the cluster administrator. Get a `kubeconfig` file for the workload cluster.

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

2. Verify the API server is up by listing the nodes.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get nodes
```

> **Note:** The Status may take a few minutes to move to `Ready` while the Pod network is deployed. The node status will change to `Ready` soon after the `calico-node` DaemonSet Pods are Ready.

Output:

```
NAME                                STATUS   ROLES                  AGE     VERSION
aws-example-control-plane-9z77w   Ready    control-plane,master   4m44s   v1.27.6
aws-example-control-plane-rtj9h   Ready    control-plane,master   104s    v1.27.6
aws-example-control-plane-zbf9w   Ready    control-plane,master   3m23s   v1.27.6
aws-example-md-0-88c46            Ready    <none>                 3m28s   v1.27.6
aws-example-md-0-fp8s7            Ready    <none>                 3m28s   v1.27.6
```

```
aws-example-md-0-qvnx7            Ready    <none>                 3m28s   v1.27.6
aws-example-md-0-wjdrg            Ready    <none>                 3m27s   v1.27.6
```

3. List the Pods with the command.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get pods -A
```

Verify the output:

```
NAMESPACE                           NAME
               READY   STATUS    RESTARTS       AGE
calico-system                       calico-kube-controllers-577c696df9-v2nzv
               1/1     Running   0              5m23s
calico-system                       calico-node-4x5rk
               1/1     Running   0              4m22s
calico-system                       calico-node-cxsgc
               1/1     Running   0              4m23s
calico-system                       calico-node-dvlnm
               1/1     Running   0              4m23s
calico-system                       calico-node-h6nlt
               1/1     Running   0              4m23s
calico-system                       calico-node-jmkwq
               1/1     Running   0              5m23s
calico-system                       calico-node-tnf54
               1/1     Running   0              4m18s
calico-system                       calico-node-v6bwq
               1/1     Running   0              2m39s
calico-system                       calico-typha-6d8c94bfdf-dkfvq
               1/1     Running   0              5m23s
calico-system                       calico-typha-6d8c94bfdf-fdfn2
               1/1     Running   0              3m43s
calico-system                       calico-typha-6d8c94bfdf-kjgzj
               1/1     Running   0              3m43s
capa-system                         capa-controller-manager-6468bc488-w7nj9
               1/1     Running   0              67s
capg-system                         capg-controller-manager-5fb47f869b-6jgms
               1/1     Running   0              53s
capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-
manager-65ffc94457-7cjdn      1/1     Running   0              74s
capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-manager-
bc7b688d4-vv8wg   1/1     Running   0              72s
capi-system                         capi-controller-manager-dbfc7b49-dzvw8
               1/1     Running   0              77s
cappp-system                        cappp-controller-manager-8444d67568-rmms2
               1/1     Running   0              59s
capv-system                         capv-controller-manager-58b8ccf868-rbscn
               1/1     Running   0              56s
capz-system                         capz-controller-manager-6467f986d8-dnvj4
               1/1     Running   0              62s
cert-manager                        cert-manager-6888d6b69b-7b7m9
               1/1     Running   0              91s
cert-manager                        cert-manager-cainjector-76f7798c9-gnp8f
               1/1     Running   0              91s
cert-manager                        cert-manager-webhook-7d4b5d8484-gn5dr
               1/1     Running   0              91s
gce-pd-csi-driver                   csi-gce-pd-controller-5bd587fbfb-lrx29
               5/5     Running   0              5m40s
gce-pd-csi-driver                   csi-gce-pd-node-4cgd8
               2/2     Running   0              4m22s
gce-pd-csi-driver                   csi-gce-pd-node-5qsfk
               2/2     Running   0              4m23s
gce-pd-csi-driver                   csi-gce-pd-node-5w4bq
               2/2     Running   0              4m18s
```

```
gce-pd-csi-driver                    csi-gce-pd-node-fbdbw
               2/2     Running   0             4m23s
gce-pd-csi-driver                    csi-gce-pd-node-h82lx
               2/2     Running   0             4m23s
gce-pd-csi-driver                    csi-gce-pd-node-jzq58
               2/2     Running   0             5m39s
gce-pd-csi-driver                    csi-gce-pd-node-k6bz9
               2/2     Running   0             2m39s
kube-system                          cluster-autoscaler-7f695dc48f-v5kvh
               1/1     Running   0             5m40s
kube-system                          coredns-64897985d-hbkqd
               1/1     Running   0             5m38s
kube-system                          coredns-64897985d-m8g5j
               1/1     Running   0             5m38s
kube-system                          etcd-aws-example-control-plane-9z77w
               1/1     Running   0             5m32s
kube-system                          etcd-aws-example-control-plane-rtj9h
               1/1     Running   0             2m37s
kube-system                          etcd-aws-example-control-plane-zbf9w
               1/1     Running   0             4m17s
kube-system                          kube-apiserver-aws-example-control-plane-9z77w
               1/1     Running   0             5m32s
kube-system                          kube-apiserver-aws-example-control-plane-rtj9h
               1/1     Running   0             2m38s
kube-system                          kube-apiserver-aws-example-control-plane-zbf9w
               1/1     Running   0             4m17s
kube-system                          kube-controller-manager-aws-example-control-
plane-9z77w         1/1     Running   0             5m33s
kube-system                          kube-controller-manager-aws-example-control-
plane-rtj9h         1/1     Running   0             2m37s
kube-system                          kube-controller-manager-aws-example-control-
plane-zbf9w         1/1     Running   0             4m17s
kube-system                          kube-proxy-bskz2
               1/1     Running   0             4m18s
kube-system                          kube-proxy-gdkn5
               1/1     Running   0             4m23s
kube-system                          kube-proxy-knvb9
               1/1     Running   0             4m22s
kube-system                          kube-proxy-tcj7r
               1/1     Running   0             4m23s
kube-system                          kube-proxy-thdpl
               1/1     Running   0             5m38s
kube-system                          kube-proxy-txxmb
               1/1     Running   0             4m23s
kube-system                          kube-proxy-vq6kv
               1/1     Running   0             2m39s
kube-system                          kube-scheduler-aws-example-control-plane-9z77w
               1/1     Running   0             5m33s
kube-system                          kube-scheduler-aws-example-control-plane-rtj9h
               1/1     Running   0             2m37s
kube-system                          kube-scheduler-aws-example-control-plane-zbf9w
               1/1     Running   0             4m17s
node-feature-discovery               node-feature-discovery-master-7d5985467-lh7dc
               1/1     Running   0             5m40s
node-feature-discovery               node-feature-discovery-worker-5qtvg
               1/1     Running   0             3m40s
node-feature-discovery               node-feature-discovery-worker-66rwx
               1/1     Running   0             3m40s
node-feature-discovery               node-feature-discovery-worker-7h92d
               1/1     Running   0             3m35s
node-feature-discovery               node-feature-discovery-worker-b4666
               1/1     Running   0             3m40s
```

```
tigera-operator                     tigera-operator-5f9bdc5c59-j9tnr
               1/1     Running   0              5m38s
```

Installing Kommander in an AWS Non-air-gapped Environment

This section provides installation instructions for the Kommander component of Nutanix Kubernetes Platform (NKP) in a non-air-gapped Amazon Web Services (AWS) environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue installing the Kommander component to bring up the UI dashboard.

> **Tip:** Tips and Recommendations
>
> - The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that the Kommander installation is on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
> - Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period (for example, 1h) to allocate more time to the deployment of applications.
> - If the Kommander installation fails, or you wishwant to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Installation Prerequisites.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want Kommander installed. If you do not know the cluster name, display it using the command `kubectl get clusters -A`.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

   > **Caution:** Nutanix support is unable to recover forgotten credentials. Prevent Kommander UI lockouts with the following best practices.
   >
   > - Backup the kubeconfig file.
   > - Save your credentials.
   > - Configure an IDP (identity provider) instead of using a username and password credentials.

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. If required: Customize your `kommander.yaml`.

   a. See the Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.

5. Only required if your cluster uses a custom AWS VPC and requires an internal load-balancer; set the traefik annotation to create an internal-facing ELB.

```
apps:
  traefik:
    enabled: true
    values: |
      service:
        annotations:
          service.beta.kubernetes.io/aws-load-balancer-internal: "true"
```

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `nkp-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.nutanix.io/project-default-catalog-repository: "true"
        kommander.nutanix.io/workspace-default-catalog-repository: "true"
        kommander.nutanix.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Enable NKP Catalog Applications after Installing NKP.

AWS Installation in an Air-gapped Environment

This installation provides instructions on how to install NKP in an Amazon Web Services (AWS) air-gapped environment.

Remember, there are always more options for custom YAML Ain't Markup Language (YAML) in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

AWS Prerequisites

Before you begin using Konvoy with AWS, you must:

1. Follow the steps to create permissions and roles on the Minimal Permissions and Role to Create Clusters page.
2. Create Cluster IAM Policies and Roles
3. Export the AWS region where you want to deploy the cluster:

   ```
   export AWS_REGION=us-west-2
   ```

4. Export the AWS profile with the credentials you want to use to create the Kubernetes cluster:

   ```
   export AWS_PROFILE=<profile>
   ```

If using AWS ECR as your local private registry, more information can be found on the Registry Mirror Tools page.

To deploy a cluster with a custom image in a region where CAPI images are not provided, you need to use Konvoy Image Builder to create your image for the region. For information on CAPI images, see https://cluster-api-aws.sigs.k8s.io/topics/images/built-amis.html.

Note:

- For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

  ```
  tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
  ```

- Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

  For example:

  ```
  export TMPDIR=/path/to/your/directory
  ```

Note: For multi-tenancy, every tenant needs to be in a different AWS account to ensure they are truly independent of other tenants to enforce security.

Using Custom AMI in Air-gapped Cluster Creation

The default Amazon Web Services (AWS) image is not recommended for use in production. Nutanix suggests using Konvoy Image Builder to create a custom AMI and take advantage of enhanced cluster operations.

About this task

If not done previously during Konvoy Image Builder download in Prerequisites, extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION` folder. Otherwise, proceed to Build the Image.

In previous Nutanix Kubernetes Platform (NKP) releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts, with the exception of the distro packages:

Warning: Previously, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP now requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder.

Explore the Customize your Image topic for more options. Using KIB, you can build an AMI without requiring access to the internet by providing an additional `--override` flag.

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Containerd tar file,

Procedure

1. Download `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`, and extract the tarball to a local directory.

   For example:

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.

3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the new NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`.

   For example:

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   Note:

   - For FIPS, pass the flag: `--fips`
   - For RHEL OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`. For example:

     ```
     export RHSM_ACTIVATION_KEY="-ci"
      export RHSM_ORG_ID="1232131"
     ```

4. Follow the instructions below to build an AMI.

   Tip: The `konvoy-image` binary and all supporting folders are also extracted. When run, `konvoy-image` bind mounts the current working directory (`${PWD}`) into the container to be used.

   - Set environment variables for AWS access. The following variables must be set using your credentials including the required required IAM:
   - export `AWS_ACCESS_KEY_ID`
   - export `AWS_SECRET_ACCESS_KEY`
   - export `AWS_DEFAULT_REGION`
   - If you have an override file to configure specific attributes of your AMI file, add it. Instructions for customizing an override file are found on this page: Image Overrides.

Building the Image

About this task

Depending on which version of NKP you are running, steps and flags will be different. To deploy in a region where CAPI images are not provided, you need to use KIB to create your own image for the region. For a list of supported AWS regions, refer to the Published AMI information from AWS. To begin image creation:

Procedure

1. Run the `konvoy-image`command to build and validate the image.

   For example:

   ```
   konvoy-image build aws images/ami/rhel-86.yaml
   ```

   By default, it builds in the `us-west-2` region. to specify another region set the `--region` flag as shown in the command below:

   ```
   konvoy-image build aws --region us-east-1 images/ami/rhel-86.yaml
   ```

   > **Tip:** Ensure you have named the correct AMI image YAML file for your OS in the `konvoy-image build` command.

   After KIB provisions the image successfully, the `ami` id is printed and written to the `packer.pkr.hcl` (Packer config) file. This file has an `artifact_id` field whose value provides the name of the AMI ID as shown in the example below. That is the `ami` you use in the `create cluster` command:

   ```
   {
     "builds": [
       {
         "name": "kib_image",
         "builder_type": "amazon-ebs",
         "build_time": 1698086886,
         "files": null,
         "artifact_id": "us-west-2:ami-04b8dfef8bd33a016",
         "packer_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05",
         "custom_data": {
           "containerd_version": "",
           "distribution": "RHEL",
           "distribution_version": "8.6",
           "kubernetes_cni_version": "",
           "kubernetes_version": "1.29.6"
         }
       }
     ],
     "last_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05"
   }
   ```

2. To use a custom AMI when creating your cluster, you must create that AMI using KIB first. Then perform the export and name the custom AMI for use in the command `NKP create cluster` link update needed creating your cluster

   For example:

   ```
   export AWS_AMI_ID=ami-<ami-id-here>
   ```

3. Inside the sections for either Non-air-gapped or Air-gapped cluster creation, you will find the instructions for how to apply custom images link updates needed Non-air-gapped

4. YAML for KIB AMI images found here in GitHub https://github.com/mesosphere/konvoy-image-builder/tree/7447074a6d910e71ad2e61fc4a12820d073ae5ae/images/ami.

Loading the Registry for an AWS Air-gapped Environment

Before creating an air-gapped Kubernetes cluster, you must load the required images in a local registry for the Konvoy component.

About this task

The complete Nutanix Kubernetes Platform (NKP) air-gapped bundle is needed for an air-gapped environment but can also be used in a non-air-gapped environment. The bundle contains all the NKP components needed for an air-gapped environment installation and for using a local registry in a non-air-gapped environment.

> **Note:** If you do not already have a local registry set up, see the Local Registry Tools page for more information.

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images, is required. This registry must be accessible from the bastion machine and the AWS EC2 instances (if deploying to AWS) or other machines that will be created for the Kubernetes cluster.

Procedure

1. If not already done in prerequisites, download the air-gapped bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tar file to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
   ```

2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. EX: For the bootstrap cluster, change your directory to the `nkp-<version>` directory, similar to the example below, depending on your current location

   ```
   cd nkp-v2.12.0
   ```

3. Set an environment variable with your registry address for ECR.

   ```
   export REGISTRY_URL=<ecr-registry-URI>
   ```

   - `REGISTRY_URL`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images
   - The environment where you are running the `nkp push` command must be authenticated with AWS to load your images into ECR.
   - Other registry variables:

     ```
     export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
     export REGISTRY_USERNAME=<username>
     export REGISTRY_PASSWORD=<password>
     export REGISTRY_CA=<path to the cacert file on the bastion>
     ```

   Before creating or upgrading a Kubernetes cluster, you must load the required images in a local registry if operating in an air-gapped environment.

4. Execute the following command to load the air-gapped image bundle into your private registry using any relevant flags to apply the above variables.

   If not ECR as shown in the example code below, use the other relevant flags: `--to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}`

   ```
   nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
   ```

   Note: It may take some time to push all the images to your image registry, depending on the network's performance between the machine you are running the script on and the registry.

   Important: To increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

5. Load the Kommander component images to your private registry using the command.

   ```
   nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
   ```

   Optional: This step is required only if you have an Ultimate license.

   For NKP Catalog Applications available with the Ultimate license, perform this image load by running the following command to load the `dkp-catalog-applications` image bundle into your private registry:

   ```
   nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
   ```

6. On the Bastion, load the Konvoy bootstrap image, using either the Docker or Podman command.

   Docker example:

   ```
   docker load -i konvoy-bootstrap-image-v2.12.0.tar
   ```

   Podman example:

   ```
   podman load -i konvoy-bootstrap-image-v2.12.0.tar
   podman image tag konvoy-bootstrap:2.12.0 docker.io/konvoy-bootstrap:v2.12.0
   ```

Bootstrapping AWS Air-gapped

To create Kubernetes clusters, NKP uses Cluster API (CAPI) controllers. These controllers run on a Kubernetes cluster.

About this task

To get started, you need a bootstrap cluster. By default, Nutanix Kubernetes Platform (NKP) creates a bootstrap cluster for you in a Docker container using the Kubernetes-in-Docker (KIND) tool.

Before you begin

Procedure

1. Complete the Nutanix Infrastructure Prerequisites. For more information, see Nutanix Infrastructure Prerequisites on page 690.
2. Ensure the `NKP` binary can be found in your `$PATH`.

Bootstrap Cluster Life Cycle Services

Procedure

1. Review Universal Configurations for all Infrastructure Providers regarding settings, flags, and other choices and then begin bootstrapping.
2. Create a bootstrap cluster using the command `nkp create bootstrap --kubeconfig $HOME/.kube/config`.

   > **Note:** Use `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Configuring an HTTP or HTTPS Proxy on page 677.

   Example output:

   ```
   # Creating a bootstrap cluster
   # Initializing new CAPI components
   ```

3. NKP creates a bootstrap cluster using KIND as a library.

   For more information, see https://github.com/kubernetes-sigs/kind.

4. NKP then deploys the following Cluster API providers on the cluster.

   - Core Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/
   - AWS Infrastructure Provider: https://github.com/kubernetes-sigs/cluster-api-provider-aws
   - Kubeadm Bootstrap Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/bootstrap/kubeadm
   - Kubeadm ControlPlane Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/controlplane/kubeadm

   For more information on Cluster APIs, see https://cluster-api.sigs.k8s.io/.

5. NKP waits until these providers' controller-manager and webhook deployments are ready. List these deployments using the command `kubectl get --all-namespaces deployments -l=clusterctl.cluster.x-k8s.io`

   Output example:

   ```
   NAMESPACE                           NAME
    READY   UP-TO-DATE   AVAILABLE   AGE
   capa-system                         capa-controller-manager
    1/1     1            1           1h
   capg-system                         capg-controller-manager
    1/1     1            1           1h
   capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-manager
    1/1     1            1           1h
   capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-manager
    1/1     1            1           1h
   capi-system                         capi-controller-manager
    1/1     1            1           1h
   cappp-system                        cappp-controller-manager
    1/1     1            1           1h
   capv-system                         capv-controller-manager
    1/1     1            1           1h
   capz-system                         capz-controller-manager
    1/1     1            1           1h
   cert-manager                        cert-manager
    1/1     1            1           1h
   ```

```
cert-manager                         cert-manager-cainjector
 1/1     1            1           1h
cert-manager                         cert-manager-webhook
 1/1     1            1           1h
```

Creating a New AWS Air-gapped Cluster

Create an AWS Cluster in an air-gapped environment.

About this task

Use this procedure to create a custom Amazon Web Services (AWS) cluster with Nutanix Kubernetes Platform (NKP).

If you use these instructions to create a cluster on AWS using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes and 4 worker nodes.

By default, the control-plane Nodes will be created in 3 different zones. However, the default worker Nodes will reside in a single zone. You may create additional node pools in other zones with the `nkp create nodepool` command.

Availability zones (AZs) are isolated locations within datacenter regions where public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single Availability Zone, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple Availability Zones.

> **Warning:** In previous NKP releases, Amazon Machine Image (AMI) images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder.

> **Note:** NKP uses the AWS CSI driver as the default storage provider. Use a Kubernetes CSI compatible storage that is suitable for production. For more information, see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types

Before you begin

First, you must name your cluster.

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes documentation for more naming information at https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Ensure your AWS credentials are up to date. If you use Static Credentials, refresh the credentials using the following command. Otherwise, proceed to the next step.

   ```
   nkp update bootstrap credentials aws
   ```

3. Set the environment variable to the name you assigned this cluster:

   ```
   export CLUSTER_NAME=<aws-example>
   ```

4. Export the variables, such as custom AMI and existing infrastructure details, for later use with the `nkp create cluster` command.

   ```
   export AWS_AMI_ID=<ami-...>
   ```

```
export AWS_VPC_ID=<vpc-...>
export AWS_SUBNET_IDS=<subnet-...,subnet-...,subnet-...>
export AWS_ADDITIONAL_SECURITY_GROUPS=<sg-...>
```

5. There are two approaches to supplying the ID of your AMI. Either provide the ID of the AMI or a way for NKP to discover the AMI using location, format, and OS information.

   a. Option One - Provide the ID of your AMI.

      Use the example command below, leaving the existing flag that provides the AMI ID: `--ami AMI_ID`

   b. Option Two - Provide a path for your AMI with the information required for image discovery.

      - Where the AMI is published using your AWS Account ID: `--ami-owner AWS_ACCOUNT_ID`
      - The format or string used to search for matching AMIs and ensure it references the Kubernetes version plus the base OS name: `--ami-base-os ubuntu-20.04`
      - The base OS information: `--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*'`

      Note:

      - The AMI must be created with Konvoy Image Builder to use the registry mirror feature.

        ```
        export AWS_AMI_ID=<ami-...>
        ```

      - (Optional) Registry Mirror - Configure your cluster to use an existing local registry as a mirror when attempting to pull images. Below is an AWS ECR example where `REGISTRY_URL`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.:

        ```
        export REGISTRY_URL=<ecr-registry-URI>
        ```

6. Ensure your subnets do not overlap with your host subnet because they cannot be changed after cluster creation. If you need to change the Kubernetes subnets, you must do this at cluster creation. The default subnets used in NKP are.

   ```
   spec:
     clusterNetwork:
       pods:
         cidrBlocks:
         - 192.168.0.0/16
       services:
         cidrBlocks:
         - 10.96.0.0/12
   ```

   » Additional Options for your environment, otherwise, proceed to the next step to create your cluster.

   (Optional) Modify Control Plane Audit logs - Users can make modifications to the `KubeadmControlplane` cluster-API object to configure different `kubelet` options. See the following guide if you wish to configure your control plane beyond the existing options available from flags.

7. Create a Kubernetes cluster object with a `dry run` output for customizations. The following example shows a common configuration.

   ```
   nkp create cluster aws
     --cluster-name=${CLUSTER_NAME} \
     --vpc-id=${AWS_VPC_ID} \
     --ami=${AWS_AMI_ID} \
     --subnet-ids=${AWS_SUBNET_IDS} \
   ```

```
  --internal-load-balancer=true \
  --additional-security-group-ids=${AWS_ADDITIONAL_SECURITY_GROUPS} \
  --registry-mirror-url=${REGISTRY_URL} \
  --dry-run \
  --output=yaml \
  > ${CLUSTER_NAME}.yaml
```

If providing the AMI path, use these flags in place of `AWS_AMI_ID`:

```
--ami-owner AWS_ACCOUNT_ID \
--ami-base-os ubuntu-20.04 \
--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*' \
```

> **Note:** More flags can be added to the `nkp create cluster` command for more options. See Choices below or refer to the topic Universal Configurations:

- If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy
- FIPS flags -

  To create a cluster in FIPS mode, inform the controllers of the appropriate image repository and version tags of the official Nutanix FIPS builds of Kubernetes by adding those flags to `nkp create cluster` command. `--kubernetes-version=v1.29.6+fips.0 \ --etcd-version=3.5.10+fips.0`
- You can create individual manifest files with different smaller manifests for ease in editing using the `--output-directory` flag.
- Flatcar OS uses this flag to instruct the bootstrap cluster to make some changes related to the installation paths: `--os-hint flatcar`

8. Inspect or edit the cluster objects. Familiarize yourself with Cluster API before editing the cluster objects, as edits can prevent the cluster from deploying successfully:

   ```
   kubectl get clusters,kubeadmcontrolplanes,machinedeployments
   ```

9. Create the cluster from the objects generated from the `dry run`. A warning will appear in the console if the resource already exists, requiring you to remove the resource or update your YAML.

   ```
   kubectl create -f ${CLUSTER_NAME}.yaml
   ```

   > **Note:** If you used the `--output-directory` flag in your `NKP create .. --dry-run` step above, create the cluster from the objects you created by specifying the directory:

   ```
   kubectl create -f <existing-directory>/.
   ```

10. Wait for the cluster control-plane to be ready.

    ```
    kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
    timeout=20m
    ```

11. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. Konvoy provides a command to describe the current status of the cluster.

    ```
    nkp describe cluster -c ${CLUSTER_NAME}
    ```

    Output:

    ```
    NAME                                                        READY  SEVERITY
     REASON  SINCE  MESSAGE
    ```

```
Cluster/aws-example                                                 True
      60s
##ClusterInfrastructure - AWSCluster/aws-example                   True
      5m23s
##ControlPlane - KubeadmControlPlane/aws-example-control-plane     True
      60s
# ##Machine/aws-example-control-plane-55jh4                        True
      4m59s
# ##Machine/aws-example-control-plane-6sn97                        True
      2m49s
# ##Machine/aws-example-control-plane-nx9v5                        True
      66s
##Workers
  ##MachineDeployment/aws-example-md-0                             True
      117s
 ##Machine/aws-example-md-0-cb9c9bbf7-hcl8z                    True
   3m1s
 ##Machine/aws-example-md-0-cb9c9bbf7-rtdqw                    True
   3m2s
 ##Machine/aws-example-md-0-cb9c9bbf7-t894m                    True
   3m1s
 ##Machine/aws-example-md-0-cb9c9bbf7-td29r                    True
```

Note: NKP uses the AWS CSI driver as the default storage provider. Use a Kubernetes CSI compatible storage that is suitable for production. For more information, see the Kubernetes documentation called ##Changing the Default Storage Class# at https://kubernetes.io/docs/concepts/storage/volumes/#volume-types. If you're not using the default, you cannot deploy an alternate provider until after the `nkp create cluster` is finished. However, this must be determined before the Kommander installation.

Note: To increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username=<username> --registry-mirror-password=<password>` on the `nkp create cluster` command. For more information see https://docs.docker.com/docker-hub/download-rate-limit/.

12. As they progress, the controllers also create Events. List the Events using the `kubectl get events | grep ${CLUSTER_NAME}` command.

Making the New AWS Air-gapped Cluster Self-Managed

How to make a Kubernetes cluster manage itself.

About this task

Nutanix Kubernetes Platform (NKP) deploys all cluster life cycle services to a bootstrap cluster, which then deploys a workload cluster. When the workload cluster is ready, move the cluster life cycle services to the workload cluster, which makes the workload cluster self-managed.

Before you begin

Before starting, ensure you can create a workload cluster as described in the topic: Create a New AWS Cluster.

This page contains instructions on how to make your cluster self-managed. This is necessary if there is only one cluster in your environment or if this cluster becomes the Management cluster in a multi-cluster environment.

Note: If you already have a self-managed or Management cluster in your environment, skip this page.

Follow these steps to turn your new cluster into a **Management Cluster** for an Ultimate license environment (or a free-standing Pro Cluster):

Procedure

1. Deploy cluster life cycle services on the workload cluster.

   ```
   nkp create capi-components --kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Initializing new CAPI components
   ```

   > **Note:** If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP/HTTPS Proxy.

2. The cluster life cycle services on the workload cluster are ready, but the workload cluster configuration is on the bootstrap cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the bootstrap to the workload cluster. This process is called a Pivot. For more information, see https://cluster-api.sigs.k8s.io/reference/glossary.html?highlight=pivot#pivot.

   Move the Cluster API objects from the bootstrap to the workload cluster:

   ```
   nkp move capi-resources --to-kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Moving cluster resources
   You can now view resources in the moved cluster by using the --kubeconfig flag with
    kubectl. For example: kubectl --kubeconfig=gcp-example.conf get nodes
   ```

   > **Note:** To ensure only one set of cluster life cycle services manages the workload cluster, NKP first pauses the reconciliation of the objects on the bootstrap cluster, then creates the objects on the workload cluster. As NKP copies the objects, the cluster life cycle services on the workload cluster reconcile the objects. The workload cluster becomes self-managed after NKP creates all the objects. If it fails, the `move` command can be safely retried.

3. Wait for the cluster control-plane to be ready.

   ```
   kubectl --kubeconfig ${CLUSTER_NAME}.conf wait --for=condition=ControlPlaneReady
    "clusters/${CLUSTER_NAME}" --timeout=20m
   ```

   Output:

   ```
   cluster.cluster.x-k8s.io/gcp-example condition met
   ```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster status. After moving the cluster life cycle services to the workload cluster, remember to use NKP with the workload cluster `kubeconfig`.

   ```
   nkp describe cluster --kubeconfig ${CLUSTER_NAME}.conf -c ${CLUSTER_NAME}
   ```

   Output:

   ```
   NAME                                                              READY  SEVERITY
    REASON  SINCE  MESSAGE
   Cluster/aws-example                                              True
       109s
   ##ClusterInfrastructure - AWSCluster/aws-example                 True
       112s
   ##ControlPlane - KubeadmControlPlane/aws-example-control-plane   True
       109s
   ```

```
# ##Machine/aws-example-control-plane-55jh4                    True
    111s
# ##Machine/aws-example-control-plane-6sn97                    True
    111s
# ##Machine/aws-example-control-plane-nx9v5                    True
    110s
##Workers
  ##MachineDeployment/aws-example-md-0                         True
    114s
 ##Machine/aws-example-md-0-cb9c9bbf7-hcl8z                 True
 111s
 ##Machine/aws-example-md-0-cb9c9bbf7-rtdqw                 True
 111s
 ##Machine/aws-example-md-0-cb9c9bbf7-t894m                 True
 111s
 ##Machine/aws-example-md-0-cb9c9bbf7-td29r                 True
 111s
```

5. Remove the bootstrap cluster because the workload cluster is now self-managed.

```
nkp delete bootstrap --kubeconfig $HOME/.kube/config
# Deleting bootstrap cluster
```

Known Limitations

Procedure

- NKP only supports moving all namespaces in the cluster; NKP does not support migration of individual namespaces.
- Konvoy supports moving only one set of cluster objects from the bootstrap cluster to the workload cluster or vice-versa.

Exploring the AWS Air-gapped Cluster

This guide explains how to use the command line to interact with your newly deployed Kubernetes cluster.

About this task

Before you start, make sure you have created a workload cluster, as described in Create a New aws Cluster.

Procedure

1. When the workload cluster is created, the cluster life cycle services generate a `kubeconfig` file for the workload cluster and write it to a Secret. The `kubeconfig` file is scoped to the cluster administrator. Get a `kubeconfig` file for the workload cluster.

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

2. Verify the API server is up by listing the nodes.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get nodes
```

> **Note:** .The Status may take a few minutes to move to `Ready` while the Pod network is deployed. The node status will change to `Ready` soon after the `calico-node` DaemonSet Pods are Ready.

Output:

```
NAME                              STATUS   ROLES                  AGE     VERSION
aws-example-control-plane-9z77w   Ready    control-plane,master   4m44s   v1.27.6
```

```
aws-example-control-plane-rtj9h   Ready    control-plane,master   104s    v1.27.6
aws-example-control-plane-zbf9w   Ready    control-plane,master   3m23s   v1.27.6
aws-example-md-0-88c46            Ready    <none>                 3m28s   v1.27.6
aws-example-md-0-fp8s7            Ready    <none>                 3m28s   v1.27.6
aws-example-md-0-qvnx7            Ready    <none>                 3m28s   v1.27.6
aws-example-md-0-wjdrg            Ready    <none>                 3m27s   v1.27.6
```

3. List the Pods with the command.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get pods -A
```

Verify the output:

```
NAMESPACE                           NAME
              READY   STATUS    RESTARTS       AGE
calico-system                       calico-kube-controllers-577c696df9-v2nzv
              1/1     Running   0              5m23s
calico-system                       calico-node-4x5rk
              1/1     Running   0              4m22s
calico-system                       calico-node-cxsgc
              1/1     Running   0              4m23s
calico-system                       calico-node-dvlnm
              1/1     Running   0              4m23s
calico-system                       calico-node-h6nlt
              1/1     Running   0              4m23s
calico-system                       calico-node-jmkwq
              1/1     Running   0              5m23s
calico-system                       calico-node-tnf54
              1/1     Running   0              4m18s
calico-system                       calico-node-v6bwq
              1/1     Running   0              2m39s
calico-system                       calico-typha-6d8c94bfdf-dkfvq
              1/1     Running   0              5m23s
calico-system                       calico-typha-6d8c94bfdf-fdfn2
              1/1     Running   0              3m43s
calico-system                       calico-typha-6d8c94bfdf-kjgzj
              1/1     Running   0              3m43s
capa-system                         capa-controller-manager-6468bc488-w7nj9
              1/1     Running   0              67s
capg-system                         capg-controller-manager-5fb47f869b-6jgms
              1/1     Running   0              53s
capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-
manager-65ffc94457-7cjdn      1/1     Running   0              74s
capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-manager-
bc7b688d4-vv8wg   1/1     Running   0              72s
capi-system                         capi-controller-manager-dbfc7b49-dzvw8
              1/1     Running   0              77s
cappp-system                        cappp-controller-manager-8444d67568-rmms2
              1/1     Running   0              59s
capv-system                         capv-controller-manager-58b8ccf868-rbscn
              1/1     Running   0              56s
capz-system                         capz-controller-manager-6467f986d8-dnvj4
              1/1     Running   0              62s
cert-manager                        cert-manager-6888d6b69b-7b7m9
              1/1     Running   0              91s
cert-manager                        cert-manager-cainjector-76f7798c9-gnp8f
              1/1     Running   0              91s
cert-manager                        cert-manager-webhook-7d4b5d8484-gn5dr
              1/1     Running   0              91s
gce-pd-csi-driver                   csi-gce-pd-controller-5bd587fbfb-lrx29
              5/5     Running   0              5m40s
gce-pd-csi-driver                   csi-gce-pd-node-4cgd8
              2/2     Running   0              4m22s
```

```
gce-pd-csi-driver                    csi-gce-pd-node-5qsfk
               2/2     Running   0             4m23s
gce-pd-csi-driver                    csi-gce-pd-node-5w4bq
               2/2     Running   0             4m18s
gce-pd-csi-driver                    csi-gce-pd-node-fbdbw
               2/2     Running   0             4m23s
gce-pd-csi-driver                    csi-gce-pd-node-h82lx
               2/2     Running   0             4m23s
gce-pd-csi-driver                    csi-gce-pd-node-jzq58
               2/2     Running   0             5m39s
gce-pd-csi-driver                    csi-gce-pd-node-k6bz9
               2/2     Running   0             2m39s
kube-system                          cluster-autoscaler-7f695dc48f-v5kvh
               1/1     Running   0             5m40s
kube-system                          coredns-64897985d-hbkqd
               1/1     Running   0             5m38s
kube-system                          coredns-64897985d-m8g5j
               1/1     Running   0             5m38s
kube-system                          etcd-aws-example-control-plane-9z77w
               1/1     Running   0             5m32s
kube-system                          etcd-aws-example-control-plane-rtj9h
               1/1     Running   0             2m37s
kube-system                          etcd-aws-example-control-plane-zbf9w
               1/1     Running   0             4m17s
kube-system                          kube-apiserver-aws-example-control-plane-9z77w
               1/1     Running   0             5m32s
kube-system                          kube-apiserver-aws-example-control-plane-rtj9h
               1/1     Running   0             2m38s
kube-system                          kube-apiserver-aws-example-control-plane-zbf9w
               1/1     Running   0             4m17s
kube-system                          kube-controller-manager-aws-example-control-
plane-9z77w         1/1     Running   0             5m33s
kube-system                          kube-controller-manager-aws-example-control-
plane-rtj9h         1/1     Running   0             2m37s
kube-system                          kube-controller-manager-aws-example-control-
plane-zbf9w         1/1     Running   0             4m17s
kube-system                          kube-proxy-bskz2
               1/1     Running   0             4m18s
kube-system                          kube-proxy-gdkn5
               1/1     Running   0             4m23s
kube-system                          kube-proxy-knvb9
               1/1     Running   0             4m22s
kube-system                          kube-proxy-tcj7r
               1/1     Running   0             4m23s
kube-system                          kube-proxy-thdpl
               1/1     Running   0             5m38s
kube-system                          kube-proxy-txxmb
               1/1     Running   0             4m23s
kube-system                          kube-proxy-vq6kv
               1/1     Running   0             2m39s
kube-system                          kube-scheduler-aws-example-control-plane-9z77w
               1/1     Running   0             5m33s
kube-system                          kube-scheduler-aws-example-control-plane-rtj9h
               1/1     Running   0             2m37s
kube-system                          kube-scheduler-aws-example-control-plane-zbf9w
               1/1     Running   0             4m17s
node-feature-discovery               node-feature-discovery-master-7d5985467-lh7dc
               1/1     Running   0             5m40s
node-feature-discovery               node-feature-discovery-worker-5qtvg
               1/1     Running   0             3m40s
node-feature-discovery               node-feature-discovery-worker-66rwx
               1/1     Running   0             3m40s
```

```
node-feature-discovery                 node-feature-discovery-worker-7h92d
                1/1     Running   0              3m35s
node-feature-discovery                 node-feature-discovery-worker-b4666
                1/1     Running   0              3m40s
tigera-operator                        tigera-operator-5f9bdc5c59-j9tnr
                1/1     Running   0              5m38s
```

Installing Kommander in an AWS Air-gapped Environment

This section provides installation instructions for the Kommander component of Nutanix Kubernetes Platform (NKP) in an air-gapped AWS environment.

About this task

Once you have installed the Konvoy component of NKP, you will continue installing the Kommander component to bring up the UI dashboard.

Tip:

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a time period (for example, 1 hour) to allocate more time to deploy applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all the prerequisites for installation.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster using the command `export CLUSTER_NAME=<your-management-cluster-name>`.
2. Copy the `kubeconfig` file of your Management cluster to your local directory using the command `nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf`.
3. Create a configuration file for the deployment using the command `nkp install kommander --init > kommander.yaml`
4. If required: Customize your `kommander.yaml`.
   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.
5. Only required if your cluster uses a custom AWS VPC and requires an internal load-balancer; set the traefik annotation to create an internal-facing ELB.

```
apps:
  traefik:
```

```
    enabled: true
    values: |
      service:
        annotations:
          service.beta.kubernetes.io/aws-load-balancer-internal: "true"
```

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `nkp-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.nutanix.io/project-default-catalog-repository: "true"
        kommander.nutanix.io/workspace-default-catalog-repository: "true"
        kommander.nutanix.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Enable NKP Catalog Applications after Installing NKP.

AWS Management Tools

After cluster creation and configuration, you can revisit clusters to update and change variables.

Deleting an AWS Cluster

Deleting an AWS cluster.

About this task

A self-managed workload cluster cannot delete itself. If your workload cluster is self-managed, you must first create a bootstrap cluster and move the cluster life cycle services to it before deleting the workload cluster.

If you did not make your workload cluster self-managed, as described in Make New Cluster Self-Managed, proceed to the instructions for Delete the workload cluster.

Procedure

Create a Bootstrap Cluster and Move CAPI Resources

About this task

Follow these steps to create a bootstrap cluster and move CAPI resources:

Procedure

1. Make sure your AWS credentials are up to date. Refresh the credentials using this command.

```
nkp update bootstrap credentials aws --kubeconfig $HOME/.kube/config
```

2. The bootstrap cluster will host the Cluster API controllers that reconcile the cluster objects marked for deletion. Create a bootstrap cluster. To avoid using the wrong kubeconfig, the following steps use explicit kubeconfig paths and contexts.

```
nkp create bootstrap --kubeconfig $HOME/.kube/config --with-aws-bootstrap-
credentials=true
```

3. Move the Cluster API objects from the workload to the bootstrap cluster: The cluster life cycle services on the bootstrap cluster are ready, but the workload cluster configuration is on the workload cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the workload to the bootstrap cluster. This process is also called a Pivot.

```
nkp move capi-resources \
 --from-kubeconfig ${CLUSTER_NAME}.conf \
 --from-context ${CLUSTER_NAME}-admin@${CLUSTER_NAME} \
 --to-kubeconfig $HOME/.kube/config \
 --to-context kind-konvoy-capi-bootstrapper
```

Output:

```
# Moving cluster resources
You can now view resources in the moved cluster by using the --kubeconfig flag with
 kubectl. For example: kubectl --kubeconfig $HOME/.kube/config get nodes
```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster's status.

```
nkp describe cluster --kubeconfig $HOME/.kube/config -c ${CLUSTER_NAME}
```

Output:

```
NAME                                                                READY  SEVERITY
 REASON  SINCE  MESSAGE
Cluster/aws-example                                                 True
    91s
##ClusterInfrastructure - AWSCluster/aws-example                    True
    103s
##ControlPlane - KubeadmControlPlane/aws-example-control-plane     True
    91s
# ##Machine/aws-example-control-plane-55jh4                         True
    102s
# ##Machine/aws-example-control-plane-6sn97                         True
    102s
# ##Machine/aws-example-control-plane-nx9v5                         True
    102s
##Workers
  ##MachineDeployment/aws-example-md-0                              True
    108s
 ##Machine/aws-example-md-0-cb9c9bbf7-hcl8z                      True
 102s
 ##Machine/aws-example-md-0-cb9c9bbf7-rtdqw                      True
 102s
 ##Machine/aws-example-md-0-cb9c9bbf7-td29r                      True
 102s
 ##Machine/aws-example-md-0-cb9c9bbf7-w64kg                      True
 102s
```

5. Wait for the cluster control-plane to be ready.

```
kubectl --kubeconfig $HOME/.kube/config wait --for=condition=controlplaneready
 "clusters/${CLUSTER_NAME}" --timeout=20m
```

Output:

```
cluster.cluster.x-k8s.io/aws-example condition met
```

Delete the Workload Cluster

Procedure

1. Make sure your AWS credentials are up to date. Refresh the credentials using this command.

```
nkp update bootstrap credentials aws --kubeconfig $HOME/.kube/config
```

> **Note:**
>
> Persistent Volumes (PVs) are not deleted automatically by design to preserve your data. However, the PVs take up storage space if not deleted. You must delete PVs manually. For more information on backing up a cluster and PVs, see Back up your Cluster's Applications and Persistent Volumes.

2. To delete a cluster, use the command `NKP delete cluster` and pass in the name of the cluster you are trying to delete with `--cluster-name` flag. Use the command `kubectl get clusters` to get those details (`--cluster-name` and `--namespace`) of the Kubernetes cluster to delete it.

```
kubectl get clusters
```

3. Delete the Kubernetes cluster and wait a few minutes.

> **Note:** Before deleting the cluster, Nutanix Kubernetes Platform (NKP) deletes all Services of type LoadBalancer on the cluster. An AWS Classic ELB backs each Service. Deleting the Service deletes the ELB that backs it. To skip this step, use the flag `--delete-kubernetes-resources=false`. Do not skip this step if # NKP manages the VPC when NKP deletes the cluster, it deletes the VPC. If the VPC has any AWS Classic ELBs, AWS does not allow the VPC to be deleted, and NKP cannot delete the cluster.

```
nkp delete cluster --cluster-name=${CLUSTER_NAME} --kubeconfig $HOME/.kube/config
```

Output:

```
# Deleting Services with type LoadBalancer for Cluster default/azure-example
# Deleting ClusterResourceSets for Cluster default/azure-example
# Deleting cluster resources
# Waiting for cluster to be fully deleted
Deleted default/azure-example cluster
```

After the workload cluster is deleted, you can delete the bootstrap cluster.

Delete the Bootstrap Cluster

About this task

After you have moved the workload resources back to a bootstrap cluster and deleted the workload cluster, you no longer need the bootstrap cluster. You can safely delete the bootstrap cluster with these steps:

Procedure

1. Make sure your AWS credentials are up to date. Refresh the credentials using this command.

```
nkp update bootstrap credentials aws --kubeconfig $HOME/.kube/config
```

2. Delete the bootstrap cluster.

```
nkp delete bootstrap --kubeconfig $HOME/.kube/config
```

Output:

```
# Deleting bootstrap cluster
```

Leveraging Multiple AWS Accounts

Leverage Multiple AWS Accounts for Kubernetes Cluster Deployments.

About this task

You can leverage multiple AWS accounts in your organization to meet specific business purposes, reflect your organizational structure, or implement a multi-tenancy strategy. Specific scenarios include:

- Implementing isolation between environment tiers such as development, testing, acceptance, and production.
- Implementing separation of concerns between management clusters, and workload clusters.
- Reducing the impact of security events and incidents.

For additional benefits of using multiple AWS accounts, See the following white paper at https://docs.aws.amazon.com/whitepapers/latest/organizing-your-aws-environment/benefits-of-using-multiple-aws-accounts.html

This document describes how to leverage the Nutanix Kubernetes Platform (NKP) to deploy a management cluster, and multiple workload clusters, leveraging multiple AWS accounts. This guide assumes you have some understanding of Cluster API concepts and basic NKP provisioning workflows on AWS.

Before you begin

Before you begin deploying NKP on AWS, you configure the prerequisites for the environment you use either non-air-gapped or air-gapped: Prerequisites for Install

Procedure

1. Deploy a management cluster in your AWS source account. NKP leverages the Cluster API provider for AWS (CAPA) to provision Kubernetes clusters in a declarative way. Customers declare the desired state of the cluster through a cluster configuration YAML file which is generated using.

```
nkp create cluster aws --cluster-name=${CLUSTER_NAME} \
--dry-run \
--output=yaml \
> ${CLUSTER_NAME}.yaml
```

2. Configure a trusted relationship between source and target accounts. Go to your target (workload) account b. Search for the role control-plane.cluster-api-provider-aws.sigs.k8s.io c. Navigate to the Trust Relationship tab and select Edit Trust Relationship d. Add the following relationship.

```
{
  "Effect": "Allow",
  "Principal": {
```

```
    "AWS": "arn:aws:iam::${mgmt-aws-account}:role/control-plane.cluster-api-
provider-aws.sigs.k8s.io"
  },
  "Action": "sts:AssumeRole"
}
```

3. Give permission to role in the source (management cluster) account to call the sts:AssumeRole API
   a. Log in to the source AWS account and attach the following inline policy to control-plane.cluster-api-provider-aws.sigs.k8s.io role.

```
{
  "Version": "2012-10-17",
  "Statement": [
     {
       "Effect": "Allow",
       "Action": "sts:AssumeRole",
       "Resource": [
"arn:aws:iam::${workload-aws-account}:role/control-plane.cluster-api-provider-
aws.sigs.k8s.io"
       ]
     }
  ]
}
```

4. Modify the management cluster configuration file and update the AWSCluster object with following details.

```
apiVersion: infrastructure.cluster.x-k8s.io/v1alpha3
kind: AWSCluster
metadata:
spec:
  identityRef:
     kind: AWSClusterRoleIdentity
     name: cross-account-role
…
…
---
apiVersion: infrastructure.cluster.x-k8s.io/v1alpha3
kind: AWSClusterRoleIdentity
metadata:
  name: cross-account-role
spec:
  allowedNamespaces: {}
  roleARN: "arn:aws:iam::${workload-aws-account}:role/control-plane.cluster-api-
provider-aws.sigs.k8s.io"
  sourceIdentityRef:
    kind: AWSClusterControllerIdentity
    name: default
```

   After performing the above steps, your Management cluster will be configured to create new managed clusters in the target AWS workload account.

Configuring AWS Cluster Autoscaler

This page explains how to configure autoscaler for node pools.

About this task

Cluster Autoscaler can automatically scale up or down the number of worker nodes in a cluster based on the number of pending pods to be scheduled. Running the Cluster Autoscaler is optional. Unlike Horizontal-Pod Autoscaler, Cluster Autoscaler does not depend on any Metrics server and does not need Prometheus or any other metrics source.

For more information see:

- Cluster Autoscaler: https://github.com/kubernetes/autoscaler/tree/master/cluster-autoscaler/cloudprovider/clusterapi
- Horizontal-Pod Autoscaler: https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/FAQ.md#how-fast-is-hpa-when-combined-with-ca

The Cluster Autoscaler looks at the following annotations on a MachineDeployment to determine its scale-up and scale-down ranges:

Note:

```
cluster.x-k8s.io/cluster-api-autoscaler-node-group-min-size
cluster.x-k8s.io/cluster-api-autoscaler-node-group-max-size
```

The full list of command line arguments to the Cluster Autoscaler controller is on the Kubernetes public GitHub repository at https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/FAQ.md#what-are-the-parameters-to-ca

For more information on how Cluster Autoscaler works, see :

- What is Cluster Autoscaler: https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/FAQ.md#what-is-cluster-autoscaler
- How does scale-up work: https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/FAQ.md#how-does-scale-up-work
- How does scale-down work: https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/FAQ.md#how-does-scale-down-work
- CAPI Provider for Cluster Autoscaler: https://github.com/kubernetes/autoscaler/tree/master/cluster-autoscaler/cloudprovider/clusterapi

Before you begin

Ensure you have the following:

- A bootstrap cluster life cycle: Bootstrapping AWS on page 810
- Created a new Kubernetes Cluster.
- A Self-Managed Cluster.

Run Cluster Autoscaler on the Management Cluster

Procedure

1. Ensure the Cluster Autoscaler controller is up and running (no restarts and no errors in the logs)

   ```
   kubectl --kubeconfig=${CLUSTER_NAME}.conf logs deployments/cluster-autoscaler
    cluster-autoscaler -n kube-system -f
   ```

2. Enable Cluster Autoscaler by setting the min & max ranges.

   ```
   kubectl --kubeconfig=${CLUSTER_NAME}.conf annotate machinedeployment ${NODEPOOL_NAME}
    cluster.x-k8s.io/cluster-api-autoscaler-node-group-min-size=2
   kubectl --kubeconfig=${CLUSTER_NAME}.conf annotate machinedeployment ${NODEPOOL_NAME}
    cluster.x-k8s.io/cluster-api-autoscaler-node-group-max-size=6
   ```

   The Cluster Autoscaler logs will show that the worker nodes are associated with node groups and that pending pods are being watched.

3. To demonstrate that it is working properly, create a large deployment that will trigger pending pods (For this example, we used Amazon Web Services (AWS) m5.2xlarge worker nodes. If you have larger worker nodes, the recommendation is to scale up the number of replicas accordingly).

```
cat <<EOF | kubectl --kubeconfig=${CLUSTER_NAME}.conf apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: busybox-deployment
  labels:
    app: busybox
spec:
  replicas: 600
  selector:
    matchLabels:
      app: busybox
  template:
    metadata:
      labels:
        app: busybox
    spec:
      containers:
      - name: busybox
        image: busybox:latest
        command:
          - sleep
          - "3600"
        imagePullPolicy: IfNotPresent
      restartPolicy: Always
EOF
```

   Cluster Autoscaler will scale up the number of Worker Nodes until there are no pending pods.

4. Scale down the number of replicas for `busybox-deployment`.

```
kubectl --kubeconfig ${CLUSTER_NAME}.conf scale --replicas=30 deployment/busybox-
deployment
```

5. Cluster Autoscaler starts to scale down the number of Worker Nodes after the default timeout of 10 minutes.

Run Cluster Autoscaler on a Managed (Workload) Cluster

About this task

Unlike the Management (self-managed) cluster instructions above, an additional instance of autoscaler is required to run autoscaler on a managed cluster. This instance is run on the management cluster but must be pointed at the managed cluster. The `nkp create cluster` command for building a managed cluster is then run against the Management cluster so that the `clusterresourcset` for that cluster's autoscaler is modified to deploy the autoscaler on the management cluster. The flags for cluster-autoscaler are also changed.

Procedure

1. Create a secret with a kubeconfig file of the primary cluster in the managed cluster with limited user permissions to only modify resources for the given cluster.

2. Mount the secret into the cluster-autoscaler deployment.

3. Add the following flag to the cluster-autoscaler command so that `/mnt//masterconfig/` value is the path where the primary cluster's kubeconfig is loaded through the secret created.

```
--cloud-config=/mnt//masterconfig/value
```

Manage AWS Node Pools

Node pools are part of a cluster and managed as a group, and you can use a node pool to manage a group of machines using the same common properties. When Konvoy creates a new default cluster, there is one node pool for the worker nodes, and all nodes in that new node pool have the same configuration. You can create additional node pools for more specialized hardware or configuration. For example, suppose you want to tune your memory usage on a cluster where you need maximum memory for some machines and minimal memory on other machines, you than others. In that case, you create a new node pool with those specific resource needs.

Nutanix Kubernetes Platform (NKP) implements node pools using Cluster API MachineDeployments For more information, see https://cluster-api.sigs.k8s.io/developer/architecture/controllers/machine-deployment.html

Creating AWS Node Pools

Creating a node pool is useful when you need to run workloads that require machines with specific resources, such as a GPU, additional memory, or specialized network or storage hardware.

About this task

Availability zones (AZs) are isolated locations within datacenter regions where public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single Availability Zone, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple Availability Zones.

The first task is to prepare the environment.

Procedure

1. Set the environment variable to the name you assigned this cluster.

```
export CLUSTER_NAME=aws-example
```

2. If your workload cluster is self-managed, as described in Make the New Cluster Self-Managed, configure kubectl to use the kubeconfig for the cluster.

```
export KUBECONFIG=${CLUSTER_NAME}.conf
```

3. Define your node pool name.

```
export NODEPOOL_NAME=example
```

Create an AWS Node Pool

Procedure

Create a new AWS node pool with 3 replicas using this command.

```
nkp create nodepool aws ${NODEPOOL_NAME} \
    --cluster-name=${CLUSTER_NAME} \
    --replicas=3
```

This example uses default values for brevity. Use flags to define custom instance types, AMIs, and other properties.

Advanced users can use a combination of the `--dry-run` and `--output=yaml` or `--output-directory=<existing-directory>` flags to get a complete set of node pool objects to modify locally or store in version control.

Listing AWS Node Pools

List the node pools of a given cluster. This returns specific properties of each node pool so that you can see the names of the Machine Deployments.

About this task

List node pools for a managed cluster.

Procedure

To list all node pools for a managed cluster, run:.

```
nkp get nodepools --cluster-name=${CLUSTER_NAME} --kubeconfig=${CLUSTER_NAME}.conf
```

The expected output is similar to the following example, indicating the desired size of the node pool, the number of replicas ready in the node pool, and the Kubernetes version those nodes are running:

```
NODEPOOL                          DESIRED              READY              KUBERNETES
 VERSION
example                           3                    3                  v1.29.6

aws-example-md-0                  4                    4                  v1.29.6
```

Attached Clusters

About this task

Before running the command to list the attached clusters, ensure that you know the following:

- KUBECONFIG for the management cluster - To find the KUBECONFIG for a cluster from the UI, refer to this section in the documentation: Access a Managed or Attached Cluster
- The CLUSTER_NAME of the attached cluster
- The NAMESPACE of the attached cluster

Procedure

To list all node pools for an attached cluster, run.

```
nkp get nodepools --cluster-name=${ATTACHED_CLUSTER_NAME} --kubeconfig=
${CLUSTER_NAME}.conf -n ${ATTACHED_CLUSTER_NAMESPACE}
```

The expected output is similar to below:

```
NODEPOOL                        DESIRED              READY              KUBERNETES
 VERSION
aws-attached-md-0               4                    4                  v1.29.6
```

Scaling AWS Node Pools

While running Cluster Autoscaler, you can manually scale your node pools up or down when you need finite control over your environment.

About this task

If you require ten machines to run a process, you can only manually set the scaling to run those ten machines. However, using the Cluster Autoscaler, you must stay within your minimum and maximum bounds. This process allows you to scale manually.

Environment variables, such as defining the node pool name, are set in the Prepare the Environment section on the previous page. If needed, refer to that page to set those variables.

Scale Up Node Pools

Procedure

1. To scale up a node pool in a cluster, run one of the following.

   » Cluster:`nkp scale nodepools ${NODEPOOL_NAME} --replicas=5 --cluster-name=${CLUSTER_NAME}`

   » Attached Cluster:`nkp scale nodepools ${ATTACHED_NODEPOOL_NAME} --replicas=5 --cluster-name=${ATTACHED_CLUSTER_NAME} --kubeconfig=${CLUSTER_NAME}.conf -n ${ATTACHED_CLUSTER_WORKSPACE}`

   Example output shows scaling is in progress.

   ```
   # Scaling node pool example to 5 replicas
   ```

2. After a few minutes, you can list the node pools.

   ```
   nkp get nodepools --cluster-name=${CLUSTER_NAME} --kubeconfig=${CLUSTER_NAME}.conf
   ```

   Example output showing that the number of DESIRED and READY replicas increased to 5.

   ```
   NODEPOOL                            DESIRED               READY
    KUBERNETES VERSION
   aws-example-md-0                     5                     5                    v1.29.6

   aws-attached-md-0                    5                     5                    v1.29.6
   ```

Scaling Down AWS Node Pools

While running Cluster Autoscaler, you can manually scale your node pools up or down when you need finite control over your environment.

About this task

If you require ten machines to run a process, you can only manually set the scaling to run those ten machines. However, using the Cluster Autoscaler, you must stay within your minimum and maximum bounds. This process allows you to scale manually.

Environment variables, such as defining the node pool name, are set in the Prepare the Environment section on the previous page. If needed, refer to that page to set those variables.

Procedure

1. To scale down a node pool, run.

   » Cluster:`nkp scale nodepools ${NODEPOOL_NAME} --replicas=4 --cluster-name=${CLUSTER_NAME}`

   » Attached Cluster:`nkp scale nodepools ${ATTACHED_NODEPOOL_NAME} --replicas=4 --cluster-name=${ATTACHED_CLUSTER_NAME} --kubeconfig=${CLUSTER_NAME}.conf -n ${ATTACHED_CLUSTER_WORKSPACE}`

   Example output indicating scaling is in progress:

   ```
   # Scaling node pool example to 4 replicas
   ```

2. After a few minutes, you can list the node pools.

   ```
   nkp get nodepools --cluster-name=${CLUSTER_NAME} --kubeconfig=${CLUSTER_NAME}.conf
   ```

   Example output showing the number of DESIRED and READY replicas decreased to 4:

   ```
   NODEPOOL                            DESIRED               READY
    KUBERNETES VERSION
   example                             4                     4                    v1.29.6

   aws-example-md-0                    4                     4                    v1.29.6
   ```

3. In a default cluster, the nodes to delete are selected at random. This behavior is controlled by CAPI's delete policy. For more information, see https://github.com/kubernetes-sigs/cluster-api/blob/v0.4.0/api/v1alpha4/machineset_types.go#L85-L105. However, when using the Nutanix Kubernetes Platform (NKP) CLI to scale down a node pool, it is also possible to specify the Kubernetes Nodes you want to delete.

   To do this, set the flag `--nodes-to-delete` with a list of nodes as below. This adds an annotation `cluster.x-k8s.io/delete-machine=yes` to the matching Machine object that contains `status.NodeRef` with the node names from `--nodes-to-delete`.

   ```
   nkp scale nodepools ${NODEPOOL_NAME} --replicas=3 --nodes-to-delete=<> --cluster-name=${CLUSTER_NAME}
   ```

   Output:

   ```
   # Scaling node pool example to 3 replicas
   ```

Scaling Node Pools Using Cluster Autoscaler

Using cluster autoscaler rather than manual scaling instructions.

About this task

If you configured the cluster autoscaler for the `demo-cluster-md-0` node pool, the value of `--replicas` must be within the minimum and maximum bounds.

Procedure

1. For example, assuming you have these annotations.

   ```
   kubectl --kubeconfig=${CLUSTER_NAME}.conf annotate machinedeployment ${NODEPOOL_NAME}
    cluster.x-k8s.io/cluster-api-autoscaler-node-group-min-size=2
   kubectl --kubeconfig=${CLUSTER_NAME}.conf annotate machinedeployment ${NODEPOOL_NAME}
    cluster.x-k8s.io/cluster-api-autoscaler-node-group-max-size=6
   ```

2. Try to scale the node pool to 7 replicas with the command.

```
nkp scale nodepools ${NODEPOOL_NAME} --replicas=7 -c demo-cluster
```

Which results in an error similar to:

```
 # Scaling node pool example to 7 replicas
failed to scale nodepool: scaling MachineDeployment is forbidden: desired replicas
 7 is greater than the configured max size annotation cluster.x-k8s.io/cluster-api-
autoscaler-node-group-max-size: 6
```

> **Note:** Similarly, scaling down to several replicas less than the configured `min-size` also returns an error.

Deleting AWS Node Pools

Deleting a node pool deletes the Kubernetes nodes and the underlying infrastructure.

About this task

All nodes will be drained before deletion, and the pods running on those nodes will be rescheduled.

Procedure

1. To delete a node pool from a managed cluster, run.

```
nkp delete nodepool ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME}
```

Here, `example` is the node pool to be deleted.

The expected output will be similar to the following example, indicating the node pool is being deleted:

```
# Deleting default/example nodepool resources
```

2. Deleting an invalid node pool results in output similar to this example.

```
nkp delete nodepool ${CLUSTER_NAME}-md-invalid --cluster-name=${CLUSTER_NAME}
```

Output:

```
MachineDeployments or MachinePools.infrastructure.cluster.x-k8s.io "no
 MachineDeployments or MachinePools found for cluster aws-example" not found
```

Installing GPUs in an AWS Environment

Install GPU Support for Supported Distributions on AWS

About this task

Using the Konvoy Image Builder, you can build an image with support using NVIDIA GPU hardware to support GPU workloads. Nutanix Kubernetes Platform (NKP) supported NVIDIA driver version is 470.x. For more information, see https://www.nvidia.com/Download/Find.aspx.

Before you begin

- Install GPU support on supported distributions on AWS
- GPU node labeling specifications described in Configure Konvoy Automatic GPU Node Labels
- Kommander also accesses resources.

Note: For using GPUs in an air-gapped on-premises environment, Nutanix recommends setting up Pod Disruption Budget before Update Cluster Nodepools. For more information, see https://kubernetes.io/docs/concepts/workloads/pods/disruptions/.

Procedure

1. In your `overrides/nvidia.yaml` file, add the following to enable GPU builds. You can also access and use the overrides repo. For more information, see https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides.

```
gpu:
    type:
    - nvidia
```

2. Build your image using the following Konvoy image builder commands.

```
konvoy-image build --region us-west-2 --source-ami=ami-12345abcdef images/ami/
ubuntu-2004.yaml --overrides overrides/nvidia.yaml
```

3. By default, your image builds in the `us-west-2` region. To specify another region, set the `--region` flag.

```
konvoy-image build --region us-east-1 --overrides override-source-ami.yaml images/
ami/<Your OS>.yaml
```

4. When the command is complete, the `ami` id is printed and written to `packer.pkr.hcl`

```
nkp create cluster aws --cluster-name=$(whoami)-aws-cluster --region us-west-2 --ami
 <ami>
```

To use the built `ami` with Konvoy Image Builder, specify it with the `--ami` flag when calling cluster create.

Note: If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP/HTTPS Proxy.

Configuring Konvoy Automatic GPU Node Labels

How to configure GPU nodes.

About this task

When using GPU nodes, they must have the proper label identifying them as Nvidia GPU nodes.

Procedure

1. Node feature discovery (NFD), by default, labels PCI hardware as.

```
"feature.node.kubernetes.io/pci-<device label>.present": "true"
```

where `<device label>` is by default as defined in this topic: https://kubernetes-sigs.github.io/node-feature-discovery/v0.7/get-started/features.html#pci

```
< class > _ < vendor >
```

However, because there is a wide variety of devices and their assigned PCI classes, you may find that the labels assigned to your GPU nodes do not always properly identify them as containing an Nvidia GPU.

2. If the default detection does not work, you can manually change the daemonset that the GPU operator creates by running the following command.

```
          nodeSelector:
            feature.node.kubernetes.io/pci-< class > _ < vendor>.present: "true"
```

   where `class` is any 4 digit number starting with `03xy` and the vendor for NVIDIA is `10de`. If this is already deployed, you can always change the `daemonset` and change the `nodeSelector` field so that it deploys to the right nodes.

Updating NVIDIA GPU Clusters

Upgrading a node pool involves draining the existing nodes in the node pool and replacing them with new nodes.

About this task

To ensure minimum downtime and maintain high availability of the critical application workloads during the upgrade process, we recommend deploying Pod Disruption Budget (Disruptions) for your critical applications.

The Pod Disruption Budget will prevent any impact on critical applications due to misconfiguration or failures during the upgrade process.

Before you begin

- Deploy Pod Disruption Budget (PDB): https://kubernetes.io/docs/concepts/workloads/pods/disruptions/
- Konvoy Image Builder (KIB)

Procedure

1. Deploy Pod Disruption Budget for your critical applications. If your application can tolerate only one replica to be unavailable at a time, then you can set the Pod disruption budget as shown in the following example. The example below is for NVIDIA GPU node pools, but the process is the same for all node pools.
2. Create the file: `pod-disruption-budget-nvidia.yaml`

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: nvidia-critical-app
spec:
  maxUnavailable: 1
  selector:
    matchLabels:
      app: nvidia-critical-app
```

3. Apply the YAML file above using the following command.

```
kubectl create -f pod-disruption-budget-nvidia.yaml
```

AWS Certificate Renewal

During cluster creation, Kubernetes establishes a Public Key Infrastructure (PKI) for generating the Transport Layer Security (TLS) certificates needed for securing cluster communication for various components such as `etcd`, `kubernetes-apiserver` and `kube-proxy`. The certificates created by these components have a default expiration of one year and are renewed when an administrator updates the cluster.

Kubernetes provides a facility to renew all certificates automatically during control plane updates. For administrators who need long-running clusters or clusters that are not upgraded often, `nkp` provides automated certificate renewal without a cluster upgrade.

- This feature requires Python 3.5 or greater to be installed on all control plane hosts.
- Complete the Bootstrap Cluster topic.

To create a cluster with automated certificate renewal, you create a Konvoy cluster using the `certificate-renew-interval` flag. The `certificate-renew-interval` is the number of days after which Kubernetes-managed PKI certificates will be renewed. For example, an `certificate-renew-interval` value of 60 means the certificates will be renewed every 60 days.

Technical Details

The following manifests are modified on the control plane hosts and are located at `/etc/kubernetes/manifests`. Modifications to these files require SUDO access.

```
kube-controller-manager.yaml
kube-apiserver.yaml
kube-scheduler.yaml
kube-proxy.yaml
```

The following annotation indicates the time each component was reset:

```
metadata:
  annotations:
    konvoy.d2iq.io/restartedAt: $(date +%s)
```

This only occurs when the PKI certificates are older than the interval given at cluster creation time. This is activated by a `systemd timer` called `renew-certs.timer` that triggers an associated `systemd` service called `renew-certs.service` that runs on all of the control plane hosts.

Debugging AWS Certificate Renewal

To debug the automatic certificate renewal feature, a cluster administrator can look at several components to see if the certificates were renewed.

About this task

An administrator might look at the control plane pod definition to check the last reset time. To determine if a scheduler pod was reset correctly.

Procedure

1. To check for reset, run the command.

   ```
   kubectl get pod -n kube-system kube-scheduler-ip-10-0-xx-xx.us-
   west-2.compute.internal -o yaml
   ```

   The output of the command will be similar to the following:

   ```
   apiVersion: v1
   kind: Pod
   metadata:
     annotations:
       konvoy.d2iq.io/restartedAt: "1626124940.735733"
   ```

2. Administrators who want more details on the execution of the `systemd` service can use `ssh` to connect to the control plane hosts and then use the `systemctl` and `journalctl` commands that follow to help diagnose potential issues.

   ```
   systemctl list-timers
   ```

3. To check the status of the `renew-certs` service, use the command.

   ```
   systemctl status renew-certs
   ```

4. To get the logs of the last run of the service, use the command.

   ```
   journalctl logs -u renew-certs
   ```

Replacing an AWS Node

In certain situations, you may want to delete a worker node and have Cluster API replace it with a newly provisioned machine.

About this task

Identify the name of the node to delete.

List the nodes

Procedure

1. List the nodes.

   ```
   kubectl --kubeconfig ${CLUSTER_NAME}.conf get nodes
   ```

   The output from this command resembles the following:

2. Export a variable with the node name for the next steps. This example uses the name `ip-10-0-100-85.us-west-2.compute.internal`.

   ```
   export NAME_NODE_TO_DELETE="<ip-10-0-100-85.us-west-2.compute.internal>"
   ```

3. Delete the Machine resource.

   ```
   export NAME_MACHINE_TO_DELETE=$(kubectl --kubeconfig ${CLUSTER_NAME}.conf get
    machine -ojsonpath="{.items[?(@.status.nodeRef.name==\"$NAME_NODE_TO_DELETE
   \")].metadata.name}")
   kubectl --kubeconfig ${CLUSTER_NAME}.conf delete machine "$NAME_MACHINE_TO_DELETE"
   ```

   Output:

   ```
   machine.cluster.x-k8s.io "aws-example-1-md-0-cb9c9bbf7-t894m" deleted
   ```

   The command will not return immediately. It will return after the Machine resource has been deleted.

   machine resource is deleted, and the corresponding node resource is also deleted.

4. Observe that the Machine resource is being replaced using this command.

   ```
   kubectl --kubeconfig ${CLUSTER_NAME}.conf get machinedeployment
   ```

   Output:

   ```
   NAME                CLUSTER       REPLICAS   READY   UPDATED   UNAVAILABLE   PHASE
      AGE     VERSION
   aws-example-md-0   aws-example   4          3       4         1             ScalingUp
      7m53s   v1.26.3
   ```

5. Identify the replacement Machine using this command.

```
export NAME_NEW_MACHINE=$(kubectl --kubeconfig ${CLUSTER_NAME}.conf get machines \
    -l=cluster.x-k8s.io/deployment-name=${CLUSTER_NAME}-md-0 \
    -ojsonpath='{.items[?(@.status.phase=="Running")].metadata.name}{"\n"}')
echo "$NAME_NEW_MACHINE"
```

Output:

```
aws-example-md-0-cb9c9bbf7-hcl8z aws-example-md-0-cb9c9bbf7-rtdqw aws-example-md-0-
cb9c9bbf7-td29r aws-example-md-0-cb9c9bbf7-w64kg
```

If the output is empty, the new Machine has probably exited the `Provisioning` phase and entered the `Running` phase.

6. Identify the replacement Node using this command.

```
kubectl --kubeconfig ${CLUSTER_NAME}.conf get nodes
```

Output:

```
NAME                                         STATUS   ROLES                  AGE
 VERSION
ip-10-0-106-183.us-west-2.compute.internal   Ready    control-plane,master   20m
 v1.29.6
ip-10-0-158-104.us-west-2.compute.internal   Ready    control-plane,master   23m
 v1.29.6
ip-10-0-203-138.us-west-2.compute.internal   Ready    control-plane,master   22m
 v1.29.6
ip-10-0-70-169.us-west-2.compute.internal    Ready    <none>                 22m
 v1.29.6
ip-10-0-77-176.us-west-2.compute.internal    Ready    <none>                 22m
 v1.29.6
ip-10-0-86-58.us-west-2.compute.internal     Ready    <none>                 57s
 v1.29.6
ip-10-0-96-61.us-west-2.compute.internal     Ready    <none>                 22m
 v1.29.6
```

If the output is empty, the Node resource is not yet available or does not yet have the expected annotation. Wait a few minutes, then repeat the command.

Configuring an Infrastructure in the UI
This page refers to working inside the AWS Console to add an Infrastructure Provider

About this task

First, you will create the Infrastructure in the UI. Then, you will fill out the provider form.

To configure an AWS Provider with a User Role in the AWS Console (UI), perform the following steps:

Procedure

1. From the top menu bar, select your target workspace.
2. Select Infrastructure Providers in the Administration section of the sidebar menu.
3. Select **Add Infrastructure Provider**.
4. Select the **Amazon Web Services (AWS)** option.
5. Ensure **Role** is selected as the **Authentication Method**.
6. Enter a name for your infrastructure provider. Select a name that matches the AWS user.
7. Enter the Role ARN.

8. You can add an External ID if you share the Role with a 3rd party. External IDs secure your environment from accidentally used roles.
9. Click **Save**.

Fill out the Add Infrastructure Provider Form

About this task

To fill out the Add Infrastructure Provider form in the AWS Console using Static Credentials, perform the following steps:

Procedure

1. In Kommander, select the Workspace associated with the credentials you are adding.
2. Navigate to Administration > Infrastructure Providers and click **Add Infrastructure Provider**.
3. Select the **Amazon Web Services (AWS)** option.
4. Ensure Static is selected as the Authentication Method.
5. Enter a name for your infrastructure provider for later reference. Consider choosing a name that matches the AWS user.
6. Fill out the access and secret keys using the keys generated above.
7. Click **Save**.

EKS Infrastructure

Configuration types for installing NKP on Amazon Elastic Kubernetes Service (EKS) Infrastructure.

When installing Nutanix Kubernetes Platform (NKP) on your EKS infrastructure, you must also set up various permissions through AWS Infrastructure.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

EKS Introduction

Nutanix Kubernetes Platform (NKP) brings value to EKS customers by providing all components needed for a production-ready Kubernetes environment. NKP provides the capability to provision EKS clusters using the NKP UI. It also provides the ability to upgrade your EKS clusters using the NKP platform, making it possible to manage the complete life cycle of EKS clusters from a centralized platform.

NKP adds value to Amazon EKS through:

- **Time to Value** in hours/days to get to production, instead of weeks/months, or even failure. Particularly in complex environments like air-gapped, customers tried various options and spending millions did not succeed or saw Day 2 later than expected. We delivered results in hours or days.

- **Less Risk**
  - **Cloud-Native Expertise** eliminates the issue of a lack of skills. Our industry-leading expertise closes skill gaps on the customer side, avoids costly mistakes, transfers skills, and improves project success rates while shortening timelines.
  - **Simplicity** mitigates operational complexity. We focus on a great user experience and automate parts of cloud-native operations to get customers to Day 2 faster and meet all Day 2 operational challenges. This frees up customer time to build what differentiates them instead of reinventing the wheel for Kubernetes operations.
  - **Military-Grade Security** alleviates security concerns. The Nutanix Kubernetes Platform can be configured to meet NSA Kubernetes security hardening guidelines. Nutanix Kubernetes Platform and supported add-on components are security scanned and secure out of the box—encryption of data-at-rest, FIPS compliance, and fully supported air-gapped deployments round out Nutanix offerings.
- **Lower TCO** with operational insights and a more straightforward platform that curates needed capabilities from Amazon EKS and the open source community that reduces the time and cost of consulting engagements and ongoing support costs.
- **Ultimate-grade Kubernetes** - comes with a curated list of Day 2 applications necessary for running Kubernetes in production.
- **One platform for all** - Single platform to manage multiple clusters on any infrastructure cloud, on-premises, and edge.
- **Nutanix GitOps and EKS** - Delivering business value through applications is the primary goal of any Kubernetes cluster. While EKS provides the hosted framework that leads the market, delivering applications to your environment requires a mature and integrated approach. Nutanix NKP provides workspace and project level constructs to a Kubernetes cluster so that application teams have a division of resources, security, and cost optimization at the project and namespace level.
  - Projects deliver applications through FluxCD's built-in GitOps—just provide a Git repository, and NKP does the rest.
  - Through integration with Kubecost, NKP monitors the utilization of project resources and provides real-time reporting for performance and cost optimization.
  - Project security is defined through ##forced the# integration of customer authentication methods by NKP and enforced through several application security layers.
- **Cluster Life cycle Management through CAPI** - Through cluster API, NKP gives customers complete life cycle management of their EKS clusters with the ability to instantiate new EKS clusters through a unified API. This allows administrators to deploy new EKS clusters through code and deliver consistent cluster configurations.
- Time to application value is significantly reduced by minimizing the steps necessary to provision a cluster segment clusters through integrated permissions.
- Secure and reliable cluster deployments.
- Automatic day 2 operations of EKS clusters (Monitoring, Logging, Central Management, Security, Cost Optimization).
- Day 2 GitOps integration with every EKS cluster.

EKS Prerequisites and Permissions

Konvoy Prerequisites

Before you begin using Konvoy, you must have:

- An x86_64-based Linux or macOS machine.
- The `nkp` binary for Linux or macOS.
- A Container engine or runtime installed is required to install Nutanix Kubernetes Platform (NKP) :
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
- kubectl for interacting with the running cluster.
- A valid AWS account with credentials configured.
- For a local registry, whether air-gapped or non-air-gapped environment, download and extract the bundle. Download the Complete NKP Air-gapped Bundle for this release (that is. `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`) to load registry.

  Note: On macOS, Docker runs in a virtual machine. Configure this virtual machine with at least 8GB of memory.

Control Plane Nodes

You need at least three control plane nodes. Each control plane node needs to have at least the following:

- 4 cores
- 16 GiB memory
- Approximately 80 GiB of free space for the volume used for /var/lib/kubelet and /var/lib/containerd.
- Disk usage must be below 85% on the root volume.

NKP on AWS defaults to deploying an `m5.xlarge` instance with an 80GiB root volume for control plane nodes, which meets the above requirements.

Worker Nodes

You need at least four worker nodes. The specific number of worker nodes required for your environment can vary depending on the cluster workload and size of the nodes. Each worker node needs to have at least the following:

- 8 cores
- 32 GiB memory
- Around 80 GiB of free space for the volume used for /var/lib/kubelet and /var/lib/containerd.
- Disk usage must be below 85% on the root volume.

NKP on AWS defaults to deploying a`m5.2xlarge` instance with an 80GiB root volume for worker nodes, which meets the above requirements.

If you use these instructions to create a cluster on AWS using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes, which match the requirements above.

AWS Prerequisites

Before you begin using Konvoy with AWS, you must:

- Create an IAM policy configuration.
- Create the Cluster IAM Policies in your AWS account.
- The user you delegate from your role must have a minimum set of permissions, see AWS Minimal Permissions and Role to Create Clusters page for AWS.
- Export the AWS region where you want to deploy the cluster.

  ```
  export AWS_REGION=us-west-2
  ```

- Export the AWS profile with the credentials you want to use to create the Kubernetes cluster.

  ```
  export AWS_PROFILE=<profile>
  ```

Minimal User Permission for EKS Cluster Creation

The following is a CloudFormation stack that adds a policy named `eks-bootstrapper` to manage the EKS cluster to the `nkp-bootstrapper-role` created by the CloudFormation stack for AWS in the Minimal Permissions and Role to Create Cluster section.

Consult the Leveraging the Role section for an example of using this role and how a system administrator wants to expose using the permissions.

EKS CloudFormation Stack

```
AWSTemplateFormatVersion: 2010-09-09
Parameters:
  existingBootstrapperRole:
    Type: CommaDelimitedList
    Description: 'Name of existing minimal role you want to add to add EKS cluster
 management permissions to'
    Default: nkp-bootstrapper-role
Resources:
  EKSMinimumPermissions:
    Properties:
      Description: Minimal user policy to manage eks clusters
      ManagedPolicyName: eks-bootstrapper
      PolicyDocument:
        Statement:
          - Action:
              - 'ssm:GetParameter'
            Effect: Allow
            Resource:
              - 'arn:*:ssm:*:*:parameter/aws/service/eks/optimized-ami/*'
          - Action:
              - 'iam:CreateServiceLinkedRole'
            Condition:
              StringLike:
                'iam:AWSServiceName': eks.amazonaws.com
            Effect: Allow
            Resource:
              - >-
                arn:*:iam::*:role/aws-service-role/eks.amazonaws.com/
AWSServiceRoleForAmazonEKS
          - Action:
              - 'iam:CreateServiceLinkedRole'
            Condition:
              StringLike:
                'iam:AWSServiceName': eks-nodegroup.amazonaws.com
            Effect: Allow
            Resource:
              - >-
```

```
              arn:*:iam::*:role/aws-service-role/eks-nodegroup.amazonaws.com/
AWSServiceRoleForAmazonEKSNodegroup
          - Action:
              - 'iam:CreateServiceLinkedRole'
            Condition:
              StringLike:
                'iam:AWSServiceName': eks-fargate.amazonaws.com
            Effect: Allow
            Resource:
              - >-
                arn:aws:iam::*:role/aws-service-role/eks-fargate-pods.amazonaws.com/
AWSServiceRoleForAmazonEKSForFargate
          - Action:
              - 'iam:GetRole'
              - 'iam:ListAttachedRolePolicies'
            Effect: Allow
            Resource:
              - 'arn:*:iam::*:role/*'
          - Action:
              - 'iam:GetPolicy'
            Effect: Allow
            Resource:
              - 'arn:aws:iam::aws:policy/AmazonEKSClusterPolicy'
          - Action:
              - 'eks:DescribeCluster'
              - 'eks:ListClusters'
              - 'eks:CreateCluster'
              - 'eks:TagResource'
              - 'eks:UpdateClusterVersion'
              - 'eks:DeleteCluster'
              - 'eks:UpdateClusterConfig'
              - 'eks:UntagResource'
              - 'eks:UpdateNodegroupVersion'
              - 'eks:DescribeNodegroup'
              - 'eks:DeleteNodegroup'
              - 'eks:UpdateNodegroupConfig'
              - 'eks:CreateNodegroup'
              - 'eks:AssociateEncryptionConfig'
              - 'eks:ListIdentityProviderConfigs'
              - 'eks:AssociateIdentityProviderConfig'
              - 'eks:DescribeIdentityProviderConfig'
              - 'eks:DisassociateIdentityProviderConfig'
            Effect: Allow
            Resource:
              - 'arn:*:eks:*:*:cluster/*'
              - 'arn:*:eks:*:*:nodegroup/*/*/*'
          - Action:
              - 'ec2:AssociateVpcCidrBlock'
              - 'ec2:DisassociateVpcCidrBlock'
              - 'eks:ListAddons'
              - 'eks:CreateAddon'
              - 'eks:DescribeAddonVersions'
              - 'eks:DescribeAddon'
              - 'eks:DeleteAddon'
              - 'eks:UpdateAddon'
              - 'eks:TagResource'
              - 'eks:DescribeFargateProfile'
              - 'eks:CreateFargateProfile'
              - 'eks:DeleteFargateProfile'
            Effect: Allow
            Resource:
              - '*'
```

```
        - Action:
            - 'iam:PassRole'
          Condition:
            StringEquals:
              'iam:PassedToService': eks.amazonaws.com
          Effect: Allow
          Resource:
            - '*'
        - Action:
            - 'kms:CreateGrant'
            - 'kms:DescribeKey'
          Condition:
            'ForAnyValue:StringLike':
              'kms:ResourceAliases': alias/cluster-api-provider-aws-*
          Effect: Allow
          Resource:
            - '*'
      Version: 2012-10-17
    Roles: !Ref existingBootstrapperRole
  Type: 'AWS::IAM::ManagedPolicy'
```

Note: If your role is not named `nkp-bootstrapper-role` , change the parameter in line 6 of the file.

To create the resources in the `cloudformation` stack, copy the contents above into a file. Before executing the following command, replace `MYFILENAME.yaml` and `MYSTACKNAME` with the intended values for your system.

```
aws cloudformation create-stack --template-body=file://MYFILENAME.yaml
--stack-name=MYSTACKNAME --capabilities CAPABILITY_NAMED_IAM
```

EKS Cluster IAM Permissions and Roles

This section guides a Nutanix Kubernetes Platform (NKP) user in creating IAM Policies and Instance Profiles that govern who has access to the cluster. The IAM Role is used by the cluster's control plane and worker nodes using the provided AWS CloudFormation Stack specific to EKS. This CloudFormation Stack has additional permissions to delegate access roles to other users.

Prerequisites from AWS

- The user you delegate from your role must have a minimum set of permissions, see User Roles and Instance Profiles page for AWS.
- Create the AWS Cluster Identity and Access Management Policies and Roles on page 800 in your AWS account.

EKS IAM Artifacts

Policies

- `controllers-eks.cluster-api-provider-aws.sigs.k8s.io` - enumerates the Actions required by the workload cluster to create and modify EKS clusters in the user's AWS Account. It is attached to the existing `control-plane.cluster-api-provider-aws.sigs.k8s.io` role
- `eks-nodes.cluster-api-provider-aws.sigs.k8s.io` - enumerates the Actions required by the EKS workload cluster's worker machines. It is attached to the existing `nodes.cluster-api-provider-aws.sigs.k8s.io`

Roles

`eks-controlplane.cluster-api-provider-aws.sigs.k8s.io` - is the Role associated with EKS cluster control planes

Note: Configure the cluster Identity and Access Management (IAM) policies and roles for Kubernetes on AWS. After you configure, the Cluster API Provider for AWS (CAPA) automatically creates and configures the `control-plane.cluster-api-provider-aws.sigs.k8s.io` and `nodes.cluster-api-provider-aws.sigs.k8s.io` roles in AWS. For information on how to configure cluster IAM policies and roles, see AWS Cluster Identity and Access Management Policies and Roles on page 800.

The following example shows a CloudFormation stack. that includes IAM policies and roles required to set up EKS Clusters. For more information, see https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html

```
AWSTemplateFormatVersion: 2010-09-09
Parameters:
  existingControlPlaneRole:
    Type: CommaDelimitedList
    Description: 'Names of existing Control Plane Role you want to add to the
 newly created EKS Managed Policy for AWS cluster API controllers'
    Default: control-plane.cluster-api-provider-aws.sigs.k8s.io
  existingNodeRole:
    Type: CommaDelimitedList
    Description: 'ARN of the Nodes Managed Policy to add to the role for nodes'
    Default: nodes.cluster-api-provider-aws.sigs.k8s.io
Resources:
  AWSIAMManagedPolicyControllersEKS:
    Properties:
      Description: For the Kubernetes Cluster API Provider AWS Controllers
      ManagedPolicyName: controllers-eks.cluster-api-provider-aws.sigs.k8s.io
      PolicyDocument:
        Statement:
          - Action:
              - 'ssm:GetParameter'
            Effect: Allow
            Resource:
              - 'arn:*:ssm:*:*:parameter/aws/service/eks/optimized-ami/*'
          - Action:
              - 'iam:CreateServiceLinkedRole'
            Condition:
              StringLike:
                'iam:AWSServiceName': eks.amazonaws.com
            Effect: Allow
            Resource:
              - >-
                arn:*:iam::*:role/aws-service-role/eks.amazonaws.com/
AWSServiceRoleForAmazonEKS
          - Action:
              - 'iam:CreateServiceLinkedRole'
            Condition:
              StringLike:
                'iam:AWSServiceName': eks-nodegroup.amazonaws.com
            Effect: Allow
            Resource:
              - >-
                arn:*:iam::*:role/aws-service-role/eks-nodegroup.amazonaws.com/
AWSServiceRoleForAmazonEKSNodegroup
          - Action:
              - 'iam:CreateServiceLinkedRole'
            Condition:
              StringLike:
                'iam:AWSServiceName': eks-fargate.amazonaws.com
            Effect: Allow
            Resource:
              - >-
```

```
                arn:aws:iam::*:role/aws-service-role/eks-fargate-
pods.amazonaws.com/AWSServiceRoleForAmazonEKSForFargate
          - Action:
              - 'iam:GetRole'
              - 'iam:ListAttachedRolePolicies'
            Effect: Allow
            Resource:
              - 'arn:*:iam::*:role/*'
          - Action:
              - 'iam:GetPolicy'
            Effect: Allow
            Resource:
              - 'arn:aws:iam::aws:policy/AmazonEKSClusterPolicy'
          - Action:
              - 'eks:DescribeCluster'
              - 'eks:ListClusters'
              - 'eks:CreateCluster'
              - 'eks:TagResource'
              - 'eks:UpdateClusterVersion'
              - 'eks:DeleteCluster'
              - 'eks:UpdateClusterConfig'
              - 'eks:UntagResource'
              - 'eks:UpdateNodegroupVersion'
              - 'eks:DescribeNodegroup'
              - 'eks:DeleteNodegroup'
              - 'eks:UpdateNodegroupConfig'
              - 'eks:CreateNodegroup'
              - 'eks:AssociateEncryptionConfig'
              - 'eks:ListIdentityProviderConfigs'
              - 'eks:AssociateIdentityProviderConfig'
              - 'eks:DescribeIdentityProviderConfig'
              - 'eks:DisassociateIdentityProviderConfig'
            Effect: Allow
            Resource:
              - 'arn:*:eks:*:*:cluster/*'
              - 'arn:*:eks:*:*:nodegroup/*/*/*'
          - Action:
              - 'ec2:AssociateVpcCidrBlock'
              - 'ec2:DisassociateVpcCidrBlock'
              - 'eks:ListAddons'
              - 'eks:CreateAddon'
              - 'eks:DescribeAddonVersions'
              - 'eks:DescribeAddon'
              - 'eks:DeleteAddon'
              - 'eks:UpdateAddon'
              - 'eks:TagResource'
              - 'eks:DescribeFargateProfile'
              - 'eks:CreateFargateProfile'
              - 'eks:DeleteFargateProfile'
            Effect: Allow
            Resource:
              - '*'
          - Action:
              - 'iam:PassRole'
            Condition:
              StringEquals:
                'iam:PassedToService': eks.amazonaws.com
            Effect: Allow
            Resource:
              - '*'
          - Action:
              - 'kms:CreateGrant'
```

```
            - 'kms:DescribeKey'
          Condition:
            'ForAnyValue:StringLike':
              'kms:ResourceAliases': alias/cluster-api-provider-aws-*
          Effect: Allow
          Resource:
            - '*'
      Version: 2012-10-17
    Roles: !Ref existingControlPlaneRole
  Type: 'AWS::IAM::ManagedPolicy'
AWSIAMManagedEKSNodesPolicy:
  Properties:
    Description: Additional Policies to nodes role to work for EKS
    ManagedPolicyName: eks-nodes.cluster-api-provider-aws.sigs.k8s.io
    PolicyDocument:
      Statement:
        - Action:
            - "ec2:AssignPrivateIpAddresses"
            - "ec2:AttachNetworkInterface"
            - "ec2:CreateNetworkInterface"
            - "ec2:DeleteNetworkInterface"
            - "ec2:DescribeInstances"
            - "ec2:DescribeTags"
            - "ec2:DescribeNetworkInterfaces"
            - "ec2:DescribeInstanceTypes"
            - "ec2:DetachNetworkInterface"
            - "ec2:ModifyNetworkInterfaceAttribute"
            - "ec2:UnassignPrivateIpAddresses"
          Effect: Allow
          Resource:
            - '*'
        - Action:
            - ec2:CreateTags
          Effect: Allow
          Resource:
            - arn:aws:ec2:*:*:network-interface/*
        - Action:
            - "ec2:DescribeInstances"
            - "ec2:DescribeInstanceTypes"
            - "ec2:DescribeRouteTables"
            - "ec2:DescribeSecurityGroups"
            - "ec2:DescribeSubnets"
            - "ec2:DescribeVolumes"
            - "ec2:DescribeVolumesModifications"
            - "ec2:DescribeVpcs"
            - "eks:DescribeCluster"
          Effect: Allow
          Resource:
            - '*'
      Version: 2012-10-17
    Roles: !Ref existingNodeRole
  Type: 'AWS::IAM::ManagedPolicy'
AWSIAMRoleEKSControlPlane:
  Properties:
    AssumeRolePolicyDocument:
      Statement:
        - Action:
            - 'sts:AssumeRole'
          Effect: Allow
          Principal:
            Service:
              - eks.amazonaws.com
```

```
        Version: 2012-10-17
      ManagedPolicyArns:
        - 'arn:aws:iam::aws:policy/AmazonEKSClusterPolicy'
      RoleName: eks-controlplane.cluster-api-provider-aws.sigs.k8s.io
    Type: 'AWS::IAM::Role'
```

To create the resources in the CloudFormation stack, copy the contents above into a file and execute the following command after replacing `MYFILENAME.yaml` and `MYSTACKNAME` with the intended values.

```
 aws cloudformation create-stack --template-body=file://MYFILENAME.yaml --stack-name=
MYSTACKNAME --capabilities  CAPABILITY_NAMED_IAM
```

Add EKS CSI Policy

AWS CloudFormation does not support attaching an existing IAM Policy to an existing IAM Role. Add the necessary IAM policy to your worker instance profile using the `aws` CLI:

```
aws iam attach-role-policy --role-name nodes.cluster-api-provider-aws.sigs.k8s.io --
policy-arn
arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy
```

Creating an EKS Cluster from the CLI

About this task

> **Note:** Ensure that the `KUBECONFIG` environment variable is set to the Management cluster by running `export KUBECONFIG=<Management_cluster_kubeconfig>.conf`.

If you prefer to work in the shell, you can continue by creating a new cluster following these steps. If you prefer to log in to the Nutanix Kubernetes Platform (NKP) UI, you can create a new cluster from there using the steps on this page: Create an EKS Cluster from the NKP UI

> **Note:** By default, the control-plane Nodes will be created in 3 different Availability Zones. However, the default worker Nodes will reside in a single zone. You may create additional node pools in other Availability Zones with the `nkp create nodepool` command.

To create an EKS cluster from the CLI, perform the following tasks.

Procedure

1. Set the environment variable to the name you assigned this cluster using the command `export CLUSTER_NAME=eks-example`.
2. Make sure your AWS credentials are up-to-date. Refresh the credentials command using the command `nkp update bootstrap credentials aws`.

   This is only necessary if using Static Credentials (Access Keys). For more information on access keys, see Using AWS Minimal Permissions and Role to Create Clusters on page 795. If you use role-based authentication on a bastion host, proceed to step 3.
3. Create the cluster using the command `nkp create cluster eks --cluster-name=${CLUSTER_NAME} --additional-tags=owner=$(whoami)`.

   Example:

   ```
   nkp create cluster eks \
     --cluster-name=${CLUSTER_NAME} \
   ```

```
  --additional-tags=owner=$(whoami)
```

> **Note:** If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Clusters with HTTP or HTTPS Proxy on page 680.

> **Note:**
>
> **Optional flag for ECR**: Configure your cluster to use an existing local registry as a mirror when attempting to pull images. Below is an AWS ECR example.
>
> `export --registry-mirror-url-string=`*`YOUR_ECR_URL`*

- `REGISTRY-MIRROR-URL-STRING`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images. Users can still pull their own images from ECR directly or use ECR as a mirror. For more information, review the `nkp create cluster eks` NKP CLI commands for `--registry-mirror` flag

```
Generating cluster resources
cluster.cluster.x-k8s.io/eks-example created
awsmanagedcontrolplane.controlplane.cluster.x-k8s.io/eks-example-control-plane
 created
machinedeployment.cluster.x-k8s.io/eks-example-md-0 created
awsmachinetemplate.infrastructure.cluster.x-k8s.io/eks-example-md-0 created
eksconfigtemplate.bootstrap.cluster.x-k8s.io/eks-example-md-0 created
clusterresourceset.addons.cluster.x-k8s.io/calico-cni-installation-eks-example
 created
configmap/calico-cni-installation-eks-example created
configmap/tigera-operator-eks-example created
clusterresourceset.addons.cluster.x-k8s.io/cluster-autoscaler-eks-example created
configmap/cluster-autoscaler-eks-example created
clusterresourceset.addons.cluster.x-k8s.io/node-feature-discovery-eks-example created
configmap/node-feature-discovery-eks-example created
clusterresourceset.addons.cluster.x-k8s.io/nvidia-feature-discovery-eks-example
 created
configmap/nvidia-feature-discovery-eks-example created
```

4. Use your favorite editor to inspect or edit the cluster objects.

> **Note:** Editing the cluster objects requires some understanding of Cluster API. Edits can prevent the cluster from deploying successfully.
>
> The objects are Custom Resources defined by Cluster API components, and they belong to three different categories:

- **Cluster**

  A *`Cluster`* object references the infrastructure-specific and control plane objects. Because this is an AWS cluster, an #AWSCluster# object describes the infrastructure-specific cluster properties. This means the AWS region, the VPC ID, subnet IDs, and security group rules required by the Pod network implementation.

- **Control Plane**

  A *`AWSManagedControlPlane`* object describes the control plane, which is the group of machines that run the Kubernetes control plane components, which include the etcd distributed database, the API server, the core controllers, and the scheduler. The object describes the configuration for these

components. The object also refers to an infrastructure-specific object that describes the properties of all control plane machines.

- **Node Pool**

  A Node Pool is a collection of machines with identical properties. For example, a cluster might have one Node Pool with large memory capacity and another Node Pool with GPU support. Each Node Pool is described by three objects: The MachinePool references an object that describes the configuration of Kubernetes components (kubelet) deployed on each node pool machine and an infrastructure-specific object that describes the properties of all node pool machines. Here, it references a `KubeadmConfigTemplate`, and an `AWSMachineTemplate` object, which describes the instance type, the type of disk used, and the disk size, among other properties.

For more information, see https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/.

For more information about the objects, see https://kubernetes.io/docs/concepts/overview/working-with-objects/.

5. To wait for the cluster control plane to be ready, use the command `kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --timeout=20m`

   Example:

   ```
   cluster.cluster.x-k8s.io/eks-example condition met
   ```

   The `READY` status will become `True` after the cluster control plane becomes ready in one of the following steps.

6. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. To describe the current status of the cluster in Konvoy, use the command `nkp describe cluster -c ${CLUSTER_NAME}`.

   Example:

   ```
   NAME                                                                READY  SEVERITY
    REASON  SINCE  MESSAGE
   Cluster/eks-example                                                 True
          10m
   ##ControlPlane - AWSManagedControlPlane/eks-example-control-plane  True
          10m
   ##Workers
     ##MachineDeployment/eks-example-md-0                              True
          26s
    ##Machine/eks-example-md-0-78fcd7c7b7-66ntt                   True
       84s
    ##Machine/eks-example-md-0-78fcd7c7b7-b9qmc                   True
       84s
    ##Machine/eks-example-md-0-78fcd7c7b7-v5vfq                   True
       84s
    ##Machine/eks-example-md-0-78fcd7c7b7-zl6m2                   True
       84s
   ```

7. As they progress, the controllers also create Events. To list the Events, use the command `kubectl get events | grep ${CLUSTER_NAME}`.

   For brevity, the example uses `grep`. Using separate commands to get Events for specific objects is also possible. For example, `kubectl get events --field-selector`

`involvedObject.kind="AWSCluster"` and `kubectl get events --field-selector involvedObject.kind="AWSMachine"`.

Example:

```
46m         Normal   SuccessfulCreateVPC
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed VPC
 "vpc-05e775702092abf09"
46m         Normal   SuccessfulSetVPCAttributes
 awsmanagedcontrolplane/eks-example-control-plane   Set managed VPC attributes for
 "vpc-05e775702092abf09"
46m         Normal   SuccessfulCreateSubnet
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Subnet
 "subnet-0419dd3f2dfd95ff8"
46m         Normal   SuccessfulModifySubnetAttributes
 awsmanagedcontrolplane/eks-example-control-plane   Modified managed Subnet
 "subnet-0419dd3f2dfd95ff8" attributes
46m         Normal   SuccessfulCreateSubnet
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Subnet
 "subnet-0e724b128e3113e47"
46m         Normal   SuccessfulCreateSubnet
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Subnet
 "subnet-06b2b31ea6a8d3962"
46m         Normal   SuccessfulModifySubnetAttributes
 awsmanagedcontrolplane/eks-example-control-plane   Modified managed Subnet
 "subnet-06b2b31ea6a8d3962" attributes
46m         Normal   SuccessfulCreateSubnet
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Subnet
 "subnet-0626ce238be32bf98"
46m         Normal   SuccessfulCreateSubnet
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Subnet
 "subnet-0f53cf59f83177800"
46m         Normal   SuccessfulModifySubnetAttributes
 awsmanagedcontrolplane/eks-example-control-plane   Modified managed Subnet
 "subnet-0f53cf59f83177800" attributes
46m         Normal   SuccessfulCreateSubnet
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Subnet
 "subnet-0878478f6bbf153b2"
46m         Normal   SuccessfulCreateInternetGateway
 awsmanagedcontrolplane/eks-example-control-plane   Created new managed Internet
 Gateway "igw-09fb52653949d4579"
46m         Normal   SuccessfulAttachInternetGateway
 awsmanagedcontrolplane/eks-example-control-plane   Internet Gateway
 "igw-09fb52653949d4579" attached to VPC "vpc-05e775702092abf09"
46m         Normal   SuccessfulCreateNATGateway
 awsmanagedcontrolplane/eks-example-control-plane   Created new NAT Gateway
 "nat-06356aac28079952d"
46m         Normal   SuccessfulCreateNATGateway
 awsmanagedcontrolplane/eks-example-control-plane   Created new NAT Gateway
 "nat-0429d1cd9d956bf35"
46m         Normal   SuccessfulCreateNATGateway
 awsmanagedcontrolplane/eks-example-control-plane   Created new NAT Gateway
 "nat-059246bcc9d4e88e7"
46m         Normal   SuccessfulCreateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Created managed RouteTable
 "rtb-01689c719c484fd3c"
46m         Normal   SuccessfulCreateRoute
 awsmanagedcontrolplane/eks-example-control-plane   Created route {...
46m         Normal   SuccessfulAssociateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Associated managed RouteTable
 "rtb-01689c719c484fd3c" with subnet "subnet-0419dd3f2dfd95ff8"
```

```
46m         Normal   SuccessfulCreateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Created managed RouteTable
 "rtb-065af81b9752eeb69"
46m         Normal   SuccessfulCreateRoute
 awsmanagedcontrolplane/eks-example-control-plane   Created route {...
46m         Normal   SuccessfulAssociateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Associated managed RouteTable
 "rtb-065af81b9752eeb69" with subnet "subnet-0e724b128e3113e47"
46m         Normal   SuccessfulCreateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Created managed RouteTable
 "rtb-03eeff810a89afc98"
46m         Normal   SuccessfulCreateRoute
 awsmanagedcontrolplane/eks-example-control-plane   Created route {...
46m         Normal   SuccessfulAssociateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Associated managed RouteTable
 "rtb-03eeff810a89afc98" with subnet "subnet-06b2b31ea6a8d3962"
46m         Normal   SuccessfulCreateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Created managed RouteTable
 "rtb-0fab36f8751fdee73"
46m         Normal   SuccessfulCreateRoute
 awsmanagedcontrolplane/eks-example-control-plane   Created route {...
46m         Normal   SuccessfulAssociateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Associated managed RouteTable
 "rtb-0fab36f8751fdee73" with subnet "subnet-0626ce238be32bf98"
46m         Normal   SuccessfulCreateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Created managed RouteTable
 "rtb-0e5c9c7bbc3740a0f"
46m         Normal   SuccessfulCreateRoute
 awsmanagedcontrolplane/eks-example-control-plane   Created route {...
46m         Normal   SuccessfulAssociateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Associated managed RouteTable
 "rtb-0e5c9c7bbc3740a0f" with subnet "subnet-0f53cf59f83177800"
46m         Normal   SuccessfulCreateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Created managed RouteTable
 "rtb-0bf58eb5f73c387af"
46m         Normal   SuccessfulCreateRoute
 awsmanagedcontrolplane/eks-example-control-plane   Created route {...
46m         Normal   SuccessfulAssociateRouteTable
 awsmanagedcontrolplane/eks-example-control-plane   Associated managed RouteTable
 "rtb-0bf58eb5f73c387af" with subnet "subnet-0878478f6bbf153b2"
46m         Normal   SuccessfulCreateSecurityGroup
 awsmanagedcontrolplane/eks-example-control-plane   Created managed SecurityGroup
 "sg-0b045c998a120a1b2" for Role "node-eks-additional"
46m         Normal   InitiatedCreateEKSControlPlane
 awsmanagedcontrolplane/eks-example-control-plane   Initiated creation of a new EKS
 control plane default_eks-example-control-plane
37m         Normal   SuccessfulCreateEKSControlPlane
 awsmanagedcontrolplane/eks-example-control-plane   Created new EKS control plane
 default_eks-example-control-plane
37m         Normal   SucessfulCreateKubeconfig
 awsmanagedcontrolplane/eks-example-control-plane   Created kubeconfig for cluster
 "eks-example"
37m         Normal   SucessfulCreateUserKubeconfig
 awsmanagedcontrolplane/eks-example-control-plane   Created user kubeconfig for
 cluster "eks-example"
27m         Normal   SuccessfulCreate                                  awsmachine/
eks-example-md-0-4t9nc                  Created new node instance with id
 "i-0aecc1897c93df740"
26m         Normal   SuccessfulDeleteEncryptedBootstrapDataSecrets   awsmachine/eks-
example-md-0-4t9nc                   AWS Secret entries containing userdata deleted
26m         Normal   SuccessfulSetNodeRef                              machine/eks-
example-md-0-78fcd7c7b7-fn7x9          ip-10-0-88-24.us-west-2.compute.internal
```

```
26m         Normal   SuccessfulSetNodeRef                              machine/eks-
example-md-0-78fcd7c7b7-g64nv          ip-10-0-110-219.us-west-2.compute.internal
26m         Normal   SuccessfulSetNodeRef                              machine/eks-
example-md-0-78fcd7c7b7-gwc5j          ip-10-0-101-161.us-west-2.compute.internal
26m         Normal   SuccessfulSetNodeRef                              machine/eks-
example-md-0-78fcd7c7b7-j58s4          ip-10-0-127-49.us-west-2.compute.internal
46m         Normal   SuccessfulCreate                                  machineset/eks-
example-md-0-78fcd7c7b7             Created machine "eks-example-md-0-78fcd7c7b7-
fn7x9"
46m         Normal   SuccessfulCreate                                  machineset/eks-
example-md-0-78fcd7c7b7             Created machine "eks-example-md-0-78fcd7c7b7-
g64nv"
46m         Normal   SuccessfulCreate                                  machineset/eks-
example-md-0-78fcd7c7b7             Created machine "eks-example-md-0-78fcd7c7b7-
j58s4"
46m         Normal   SuccessfulCreate                                  machineset/eks-
example-md-0-78fcd7c7b7             Created machine "eks-example-md-0-78fcd7c7b7-
gwc5j"
27m         Normal   SuccessfulCreate                                  awsmachine/
eks-example-md-0-7whkv                  Created new node instance with id
 "i-06dfc0466b8f26695"
26m         Normal   SuccessfulDeleteEncryptedBootstrapDataSecrets   awsmachine/eks-
example-md-0-7whkv                  AWS Secret entries containing userdata deleted
27m         Normal   SuccessfulCreate                                  awsmachine/
eks-example-md-0-ttgzv                  Created new node instance with id
 "i-0544fce0350fd41fb"
26m         Normal   SuccessfulDeleteEncryptedBootstrapDataSecrets   awsmachine/eks-
example-md-0-ttgzv                  AWS Secret entries containing userdata deleted
27m         Normal   SuccessfulCreate                                  awsmachine/
eks-example-md-0-v2hrf                  Created new node instance with id
 "i-0498906edde162e59"
26m         Normal   SuccessfulDeleteEncryptedBootstrapDataSecrets   awsmachine/eks-
example-md-0-v2hrf                  AWS Secret entries containing userdata deleted
46m         Normal   SuccessfulCreate
 machinedeployment/eks-example-md-0               Created MachineSet "eks-example-
md-0-78fcd7c7b7"
```

Known Limitations

> **Note:** Be aware of these limitations in the current release of Konvoy.

- The Konvoy version used to create a workload cluster must match the Konvoy version used to delete a workload cluster.
- EKS clusters cannot be Self-managed.
- Konvoy supports deploying one workload cluster.
- Konvoy generates a set of objects for one Node Pool.
- Konvoy does not validate edits to cluster objects.

Create an EKS Cluster from the UI

Provision a cluster from the NKP UI.

The Nutanix Kubernetes Platform (NKP) UI allows you to provision a Cluster from your browser quickly and easily.

Creating an AWS Infrastructure Provider
Describes the steps to provision a cluster from the NKP UI.

About this task

Before creating a Cluster, you must create an AWS infrastructure provider to hold your AWS or EKS Credentials.

Procedure

1. Get the AWS RoleARN.

   ```
   aws iam get-role --role-name <role-name> --query 'Role.[RoleName, Arn]' --output text
   ```

2. Select **Infrastructure Providers** from the Dashboard menu.
3. Select **Add Infrastructure Provider**.
4. Choose a workspace. If you are already in a workspace, the provider is automatically created in that workspace.
5. Ensure you select **Amazon Web Services**.
6. Add a **Name** for your Infrastructure Provider and include the **Role ARN** from Step 1 above.
7. click **Save**.

Provisioning an EKS Cluster

Workflow for provisioning an EKS cluster.

About this task

Follow these steps to provision the EKS cluster:

Procedure

1. From the top menu bar, select your target workspace.
2. To start the provisioning workflow, Select **Clusters** > **Cluster**
3. Choose **Create Cluster**.
4. Enter the **Cluster Name**.
5. Select **EKS** from the **Choose Infrastructure** choices.
6. If available, choose a **Kubernetes Version**. Otherwise, the default Kubernetes version installs.
7. Select a datacenter **region** or specify a custom **region**.
8. Edit your worker **Node Pools** as necessary. You can choose the **Number of Nodes**, the **Machine Type**, and our **IAM Instance Profile**. You can also choose a #Worker Availability Zone#for the worker pool.
9. Add any additional **Labels** or **Infrastructure Provider Tags** as necessary.
10. Validate your inputs, and then select **Create**.
    You are redirected to the **Clusters** page, where you see your **Clusters** in the **Provisioning** status. Hover over the status to view the details. Expect your cluster to change to the **Provisioned** status after 15 minutes.

What to do next

For more information on AWS IAM ARNs, see https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_identifiers.html#identifiers-arns.

Access the EKS Cluster

Retrieve a custom `kubeconfig` file from the UI with your Kommander admin credentials.

After successfully attaching the cluster (managed), you can retrieve a custom `kubeconfig` file from the UI using your Kommander administrator credentials.

Enabling

IAM-Based Cluster AccessShort Description: Configure your EKS cluster so admins can monitor all actions.

About this task

When creating an EKS cluster through the UI, the `kubeconfig` returned using the download `kubeconfig` button allows access for 15 minutes. To follow best practices for AWS security,

configure access to the EKS cluster using the IAM role or user-based authentication. This allows account administrators to monitor all actions made.

To enable IAM-based cluster access, follow the steps below:

Procedure

1. Download the `kubeconfig` by selecting the Download `kubeconfig` button on the top section of the UI.
2. Using that `kubeconfig`, edit the config map with a command similar to the example.

   ```
   kubectl --kubeconfig=MYCLUSTER.conf edit cm -n kube-system aws-auth
   ```

3. Modify the `mapRoles` and `mapUsers` objects according to the permissions as needed. The following example shows mapping the `arn:aws:iam::MYAWSACCOUNTID:role/PowerUser` role to the `systems:masters` on the Kubernetes cluster.

   ```
   apiVersion: v1
   data:
     mapRoles: |
       - groups:
         - system:bootstrappers
         - system:nodes
         rolearn: arn:aws:iam::MYAWSACCOUNTID:role/nodes.cluster-api-provider-
   aws.sigs.k8s.io
         username: system:node:{{EC2PrivateDNSName}}
       - groups:
         - system:masters
         rolearn: arn:aws:iam::MYAWSACCOUNTID:role/PowerUser
         username: admin
   kind: ConfigMap
   ```

   For more information, see:

   - Enabling IAM user and role access: https://docs.aws.amazon.com/eks/latest/userguide/add-user-role.html.
   - Kubernetes RBAC guide:https://kubernetes.io/docs/reference/access-authn-authz/rbac/.

4. From your management cluster, run the `nkp get kubeconfig` command to fetch a kubeconfig that uses IAM-based permissions.

   ```
   nkp get kubeconfig -c ${EKS_CLUSTER_NAME} -n ${KOMMANDER_WORKSPACE_NAMESPACE} >>
   ```

```
${EKS_CLUSTER_NAME}.conf
```

Granting Cluster Access

How to Grant EKS Cluster Access.

About this task

You can access your cluster using AWS IAM roles in the dashboard. When you create an EKS cluster, the IAM entity is granted `system:masters` permissions in Kubernetes Role Based Access Control (RBAC) configuration. at https://kubernetes.io/docs/reference/access-authn-authz/rbac/

> **Note:** More information about the configuration of the EKS control plane can be found on the EKS Cluster IAM Policies and Roles page.

Suppose the EKS cluster was created as a cluster using a self-managed AWS cluster that uses IAM Instance Profiles. In that case, you must modify the `IAMAuthenticatorConfig` field in the `AWSManagedControlPlane` API object to allow other IAM entities to access the EKS workload cluster. Follow the steps below:

Procedure

1. Run the following command with your `KUBECONFIG` configured to select the self-managed cluster previously used to create the workload EKS cluster. Ensure you substitute `${CLUSTER_NAME}` and `${CLUSTER_NAMESPACE}` with their corresponding values for your cluster.

   ```
   kubectl edit awsmanagedcontrolplane ${CLUSTER_NAME}-control-plane -n
    ${CLUSTER_NAMESPACE}
   ```

2. Edit the `IamAuthenticatorConfig` field with the IAM Role to the corresponding Kubernetes Role. In this example, the IAM role `arn:aws:iam::111122223333:role/PowerUser` is granted the cluster role `system:masters`. Note that this example uses an example AWS resource ##ARNs#, so remember to substitute real values in the corresponding AWS account.

   ```
   iamAuthenticatorConfig:
       mapRoles:
         - groups:
           - system:bootstrappers
           - system:nodes
           rolearn: arn:aws:iam::111122223333:role/my-node-role
           username: system:node:{{EC2PrivateDNSName}}
         - groups:
           - system:masters
           rolearn: arn:aws:iam::111122223333:role/PowerUser
           username: admin
   ```

What to do next

For more information on changing or assigning `roles` or `clusterroles` to which you can map IAM users or roles, see Amazon Enabling IAM access to your cluster at https://docs.aws.amazon.com/eks/latest/userguide/add-user-role.html.

Exploring your EKS Cluster

Interact with your Kubernetes cluster using CLI.

About this task

This section describes how to use the command line to interact with your Kubernetes cluster.

Before you begin

Create a workload cluster as described in Create a New Cluster.

Procedure

1. Get the kubeconfig file from the `Secret`, and write it to a file, using the `nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf` command.

   When the workload cluster is created, the cluster life cycle services generate a kubeconfig file for the workload cluster and write it to a `Secret`. The kubeconfig file is scoped to the cluster administrator.

   Caution: Nutanix support is unable to recover forgotten credentials. Prevent Kommander UI lockouts with the following best practices.

   - Backup the kubeconfig file.
   - Save your credentials.
   - Configure an IDP (identity provider) instead of using a username and password credentials.

2. List the nodes using the `kubectl --kubeconfig=${CLUSTER_NAME}.conf get nodes` command.

   Example output:

```
NAME                                         STATUS   ROLES    AGE   VERSION
ip-10-0-122-211.us-west-2.compute.internal   Ready    <none>   35m   v1.23.9-eks-
ba74326
ip-10-0-127-74.us-west-2.compute.internal    Ready    <none>   35m   v1.23.9-eks-
ba74326
ip-10-0-71-155.us-west-2.compute.internal    Ready    <none>   35m   v1.23.9-eks-
ba74326
ip-10-0-93-47.us-west-2.compute.internal     Ready    <none>   35m   v1.23.9-eks-
ba74326
```

   Note: The Status may take a few minutes to move to `Ready` while the Pod network is deployed. The node status will change to `Ready` soon after the `calico-node` DaemonSet Pods are Ready.

3. List the Pods using the `kubectl --kubeconfig=${CLUSTER_NAME}.conf get --all-namespaces pods` command.

```
NAMESPACE                 NAME                                                READY
 STATUS     RESTARTS      AGE
calico-system             calico-kube-controllers-7d6749878f-ccsx9            1/1
 Running    0             34m
calico-system             calico-node-2r6l8                                   1/1
 Running    0             34m
calico-system             calico-node-5pdlb                                   1/1
 Running    0             34m
calico-system             calico-node-n24hh                                   1/1
 Running    0             34m
calico-system             calico-node-qrh7p                                   1/1
 Running    0             34m
calico-system             calico-typha-7bbcb87696-7pk45                       1/1
 Running    0             34m
calico-system             calico-typha-7bbcb87696-t4c8r                       1/1
 Running    0             34m
calico-system             csi-node-driver-bz48k                               2/2
 Running    0             34m
calico-system             csi-node-driver-k5mmk                               2/2
 Running    0             34m
```

```
calico-system            csi-node-driver-nvcck                          2/2
 Running    0            34m
calico-system            csi-node-driver-x4xnh                          2/2
 Running    0            34m
kube-system              aws-node-2xp86                                 1/1
 Running    0            35m
kube-system              aws-node-5f2kx                                 1/1
 Running    0            35m
kube-system              aws-node-6lzm7                                 1/1
 Running    0            35m
kube-system              aws-node-pz8c6                                 1/1
 Running    0            35m
kube-system              cluster-autoscaler-789d86b489-sz9x2            0/1
 Init:0/1   0            36m
kube-system              coredns-57ff979f67-pk5cg                       1/1
 Running    0            75m
kube-system              coredns-57ff979f67-sf2j9                       1/1
 Running    0            75m
kube-system              ebs-csi-controller-5f6bd5d6dc-bplwm            6/6
 Running    0            36m
kube-system              ebs-csi-controller-5f6bd5d6dc-dpjt7            6/6
 Running    0            36m
kube-system              ebs-csi-node-7hmm5                             3/3
 Running    0            35m
kube-system              ebs-csi-node-l4vfh                             3/3
 Running    0            35m
kube-system              ebs-csi-node-mfr7c                             3/3
 Running    0            35m
kube-system              ebs-csi-node-v8krq                             3/3
 Running    0            35m
kube-system              kube-proxy-7fc5x                               1/1
 Running    0            35m
kube-system              kube-proxy-vvkmk                               1/1
 Running    0            35m
kube-system              kube-proxy-x6hcc                               1/1
 Running    0            35m
kube-system              kube-proxy-x8frb                               1/1
 Running    0            35m
kube-system              snapshot-controller-8ff89f489-4cfxv            1/1
 Running    0            36m
kube-system              snapshot-controller-8ff89f489-78gg8            1/1
 Running    0            36m
node-feature-discovery   node-feature-discovery-master-7d5985467-52fcn  1/1
 Running    0            36m
node-feature-discovery   node-feature-discovery-worker-88hr7            1/1
 Running    0            34m
node-feature-discovery   node-feature-discovery-worker-h95nq            1/1
 Running    0            35m
node-feature-discovery   node-feature-discovery-worker-lfghg            1/1
 Running    0            34m
node-feature-discovery   node-feature-discovery-worker-prc8p            1/1
 Running    0            35m
tigera-operator          tigera-operator-6dcd98c8ff-k97hq               1/1
 Running    0            36m
```

What to do next

Attach an Existing Cluster to the Management Cluster on page 875

Attach an Existing Cluster to the Management Cluster

You can attach existing Kubernetes clusters to the Management Cluster. After attaching the cluster, you can use the UI to examine and manage this cluster. The following chapter describes attaching an existing Amazon Elastic Kubernetes Service (EKS) cluster.

> **Note:** This procedure assumes you have an existing and spun-up Amazon EKS cluster(s) with administrative privileges. Refer to the Amazon EKS at https://aws.amazon.com/eks/
>
> for setup and configuration information.

- Install aws-iam-authenticator as described in https://docs.aws.amazon.com/eks/latest/userguide/install-aws-iam-authenticator.html. This binary is used to access your cluster using kubectl.

Attaching a Pre-existing EKS Cluster

Attach a pre-existing EKS cluster.

About this task

Attach a pre-existing EKS cluster.

Procedure

1. Ensure that the `KUBECONFIG` environment variable is set to the Management cluster.
2. Run the `export KUBECONFIG=Management_cluster_kubeconfig.conf` command.

Accessing your EKS clusters

About this task

To access your EKS clusters, do the following:

Procedure

1. Ensure you are connected to your EKS clusters, using the command `kubectl config get-contexts` `kubectl config use-context context for first eks cluster` for each of your clusters
2. Confirm `kubectl` can access the EKS cluster using the command `kubectl get nodes`.

Creating a kubeconfig File

About this task

To get started, ensure you have kubectl set up and configured with ClusterAdmin for the cluster you want to connect to Kommander. For more information, see https://kubernetes.io/docs/tasks/tools/#kubectl and https://kubernetes.io/docs/concepts/cluster-administration/.

Procedure

1. Create the necessary service account.

```
kubectl -n kube-system create serviceaccount kommander-cluster-admin
```

2. Create a token secret for the `serviceaccount`.

```
kubectl -n kube-system create  -f - <<EOF
apiVersion: v1
```

```
kind: Secret
metadata:
  name: kommander-cluster-admin-sa-token
  annotations:
    kubernetes.io/service-account.name: kommander-cluster-admin
type: kubernetes.io/service-account-token
EOF
```

For more information on Service Account Tokens, see this article in our blog at https://eng.d2iq.com/blog/service-account-tokens-in-kubernetes-v1.24/#whats-changed-in-kubernetes-v124.

3. Verify that the `serviceaccount` token is ready by running this command.

```
kubectl -n kube-system get secret kommander-cluster-admin-sa-token -oyaml
```

Verify that the `data.token` field is populated.

Example output:

```
apiVersion: v1
data:
  ca.crt: LS0tLS1CRUdJTiBDR...
  namespace: ZGVmYXVsdA==
  token: ZXlKaGJHY2lPaUpTVX...
kind: Secret
metadata:
  annotations:
    kubernetes.io/service-account.name: kommander-cluster-admin
    kubernetes.io/service-account.uid: b62bc32e-b502-4654-921d-94a742e273a8
  creationTimestamp: "2022-08-19T13:36:42Z"
  name: kommander-cluster-admin-sa-token
  namespace: default
  resourceVersion: "8554"
  uid: 72c2a4f0-636d-4a70-9f1c-55a75f15e520
type: kubernetes.io/service-account-token
```

4. Configure the new service account for `cluster-admin` permissions.

```
cat << EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: kommander-cluster-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
  name: kommander-cluster-admin
  namespace: kube-system
EOF
```

5. Set up the following environment variables with the access data that is needed for producing a new kubeconfig file.

```
export USER_TOKEN_VALUE=$(kubectl -n kube-system get secret/kommander-cluster-admin-
sa-token -o=go-template='{{.data.token}}' | base64 --decode)
export CURRENT_CONTEXT=$(kubectl config current-context)
export CURRENT_CLUSTER=$(kubectl config view --raw -o=go-
template='{{range .contexts}}{{if eq .name "'''${CURRENT_CONTEXT}'''"}}
{{ index .context "cluster" }}{{end}}{{end}}')
```

```
export CLUSTER_CA=$(kubectl config view --raw -o=go-template='{{range .clusters}}{{if
 eq .name "'''${CURRENT_CLUSTER}'''"}}"{{with index .cluster "certificate-authority-
data" }}{{.}}{{end}}"{{ end }}{{ end }}')
export CLUSTER_SERVER=$(kubectl config view --raw -o=go-template='{{range .clusters}}
{{if eq .name "'''${CURRENT_CLUSTER}'''"}}{{ .cluster.server }}{{end}}{{ end }}')
```

6. Confirm these variables have been set correctly.

```
export -p | grep -E 'USER_TOKEN_VALUE|CURRENT_CONTEXT|CURRENT_CLUSTER|CLUSTER_CA|
CLUSTER_SERVER'
```

7. Generate a kubeconfig file that uses the environment variable values from the previous step.

> **Caution:** Nutanix support is unable to recover forgotten credentials. Prevent Kommander UI lockouts with the following best practices.
>
> - Backup the kubeconfig file.
> - Save your credentials.
> - Configure an IDP (identity provider) instead of using a username and password credentials.

```
cat << EOF > kommander-cluster-admin-config
apiVersion: v1
kind: Config
current-context: ${CURRENT_CONTEXT}
contexts:
- name: ${CURRENT_CONTEXT}
  context:
    cluster: ${CURRENT_CONTEXT}
    user: kommander-cluster-admin
    namespace: kube-system
clusters:
- name: ${CURRENT_CONTEXT}
  cluster:
    certificate-authority-data: ${CLUSTER_CA}
    server: ${CLUSTER_SERVER}
users:
- name: kommander-cluster-admin
  user:
    token: ${USER_TOKEN_VALUE}
EOF
```

8. This process produces a file in your current working directory called `kommander-cluster-admin-config` . The contents of this file are used in Kommander to attach the cluster. Before importing this configuration, verify the `kubeconfig` file can access the cluster.

```
kubectl --kubeconfig $(pwd)/kommander-cluster-admin-config get all --all-namespaces
```

Attaching the EKS Cluster from the UI

About this task

Finish attaching the EKS Cluster from the UI. Starting in the Nutanix Kubernetes Platform (NKP) UI, perform the following steps.

Procedure

1. From the top menu bar, select your target workspace—task step.

2. On the Dashboard page, select the **Add Cluster** option in the **Actions** dropdown menu at the top right.
3. Select **Attach Cluster**.
4. Select the **No additional networking restrictions** card. Alternatively, if you must use network restrictions, stop following the steps below and see the instructions on the page Cluster Attachment with Networking Restrictions on page 508.
5. Upload the kubeconfig file you created in the previous section (or copy its contents) into the **Cluster Configuration** section.
6. The **Cluster Name** field automatically populates with the name of the cluster in the kubeconfig. You can edit this field using the name you want for your cluster.
7. Add labels to classify your cluster as needed.
8. Select **Create** to attach your cluster.

> **Note:** If a cluster has limited resources to deploy all the federated platform services, it will fail to stay attached to the NKP UI. If this happens, ensure your system has sufficient resources for all pods.

Manually Attaching a NKP CLI Cluster to the Management Cluster

About this task

Starting with Nutanix Kubernetes Platform (NKP) 2.6, when you create a Managed Cluster with the NKP CLI, it attaches automatically to the Management Cluster after a few moments.

However, the attached cluster will be created in the default workspace if you do not set a workspace. To ensure that the attached cluster is created in your desired workspace namespace, follow these instructions:

> **Note:** These steps only apply if you do not set a WORKSPACE_NAMESPACE when creating a cluster. If you already set a WORKSPACE_NAMESPACE, then you do not need to perform these steps since the cluster is already attached to the workspace.

Procedure

1. Confirm you have your MANAGED_CLUSTER_NAME variable set using the command `echo ${MANAGED_CLUSTER_NAME}`.
2. Retrieve your kubeconfig from the cluster you have created without setting a workspace using the command `nkp get kubeconfig --cluster-name $MANAGED_CLUSTER_NAME > $MANAGED_CLUSTER_NAME.conf`
3. You can now attach it in the UI (link to attaching it to workspace through UI that was earlier) or attach your cluster to the workspace you want in the CLI.

   > **Note:** This is only necessary if you never set the workspace of your cluster upon creation

4. Retrieve the workspace where you want to attach the cluster using the command `kubectl get workspaces -A` .
5. Set the WORKSPACE_NAMESPACE environment variable using the command `export WORKSPACE_NAMESPACE=workspace-namespace`.
6. Create a secret in the desired workspace before attaching the cluster to that workspace. Retrieve the kubeconfig secret value of your cluster using the command `kubectl -n default get secret $MANAGED_CLUSTER_NAME-kubeconfig -o go-template='{.data.value}{"\n"}'`

7. This will return a lengthy value. Copy this entire string for a secret using the template below as a reference. Create a new `attached-cluster-kubeconfig.yaml` file.

```
apiVersion: v1
kind: Secret
metadata:
  name: your-managed-cluster-name-kubeconfig
  labels:
    cluster.x-k8s.io/cluster-name: your-managed-cluster-name
type: cluster.x-k8s.io/secret
data:
  value: value-you-copied-from-secret-above
```

8. Create this secret in the desired workspace using the command `kubectl apply -f attached-cluster-kubeconfig.yaml --namespace $WORKSPACE_NAMESPACE.`

9. Create this `kommandercluster` object to attach the cluster to the workspace

   Example:

```
cat << EOF | kubectl apply -f -
apiVersion: kommander.mesosphere.io/v1beta1
kind: KommanderCluster
metadata:
  name: ${MANAGED_CLUSTER_NAME}
  namespace: $WORKSPACE_NAMESPACE
spec:
  kubeconfigRef:
    name: $MANAGED_CLUSTER_NAME-kubeconfig
  clusterRef:
    capiCluster:
      name: $MANAGED_CLUSTER_NAME
EOF
```

10. You can now view this cluster in your Workspace in the UI, and you can confirm its status by using the command `kubectl get kommanderclusters -A`.

    It may take a few minutes to reach "Joined" status.

11. If you have several Pro Clusters and want to turn one of them into a Managed Cluster to be centrally administrated by a Management Cluster, see Platform Expansion: Conversion of an NKP Pro Cluster to an NKP Ultimate Managed Cluster on page 535.

What to do next

Cluster Management on page 478.

For more information on related topics, see:

- Cluster Management on page 478
- Deleting the EKS Cluster from CLI on page 879
- *Configuring and Running Amazon EKS Clusters* in the Amazon documentation site https://aws.amazon.com/eks/.

Deleting the EKS Cluster from CLI

About this task

Note: Ensure that the `KUBECONFIG` environment variable is set to the self-managed cluster by running `export KUBECONFIG={SELF_MANAGED_AWS_CLUSTER}.conf`.

If you prefer to continue working in the terminal or shell using the CLI, the steps for deleting the cluster are listed below. If you are in the NKP UI, you can also delete the cluster from the UI using the steps on this page: Delete EKS Cluster from the NKP UI

Follow these steps for deletion from the CLI:

Procedure

1. Ensure your AWS credentials are up to date. If you use user profiles, refresh the credentials using the command below. Otherwise, proceed to step 2.

```
nkp update bootstrap credentials aws
```

2. **Important:** Do not skip this step if the VPC is managed by Nutanix Kubernetes Platform (NKP). When NKP deletes the cluster, it deletes the VPC. If the VPC has any EKS Classic ELBs, EKS does not allow the VPC to be deleted, and NKP cannot delete the cluster.

   Delete the Kubernetes cluster and wait a few minutes. Before deleting the cluster, nkp deletes all Services of type LoadBalancer on the cluster. Service is backed by an AWS Classic ELBAn, and an AWS Classic ELB backs each service. Deleting the Service deletes the ELB that backs it. To skip this step, use the flag `--delete-kubernetes-resources=false`.

```
nkp delete cluster --cluster-name=${CLUSTER_NAME}
```

3. Example output.

```
 # Deleting Services with type LoadBalancer for Cluster default/eks-example
 # Deleting ClusterResourceSets for Cluster default/eks-example
 # Deleting cluster resources
 # Waiting for cluster to be fully deleted
Deleted default/eks-example cluster
```

Known Limitations: in the current release of NKP:

- The NKP version used to create the workload cluster must match the NKP version used to delete the workload cluster.

What to do next

Day 2 - Cluster Operations Management

Deleting EKS Cluster from the NKP UI

Describes how to remove the cluster from the Nutanix Kubernetes Platform (NKP) UI.

About this task

To delete a cluster in the UI, you must first Create an EKS Cluster from the UI on page 869 and have permission to delete.

Procedure

1. Open the dashboard and select Clusters in the left menu.
2. Select the cluster you wish to delete and click the dotted icon in the bottom right corner.

3. Then select Delete in red.

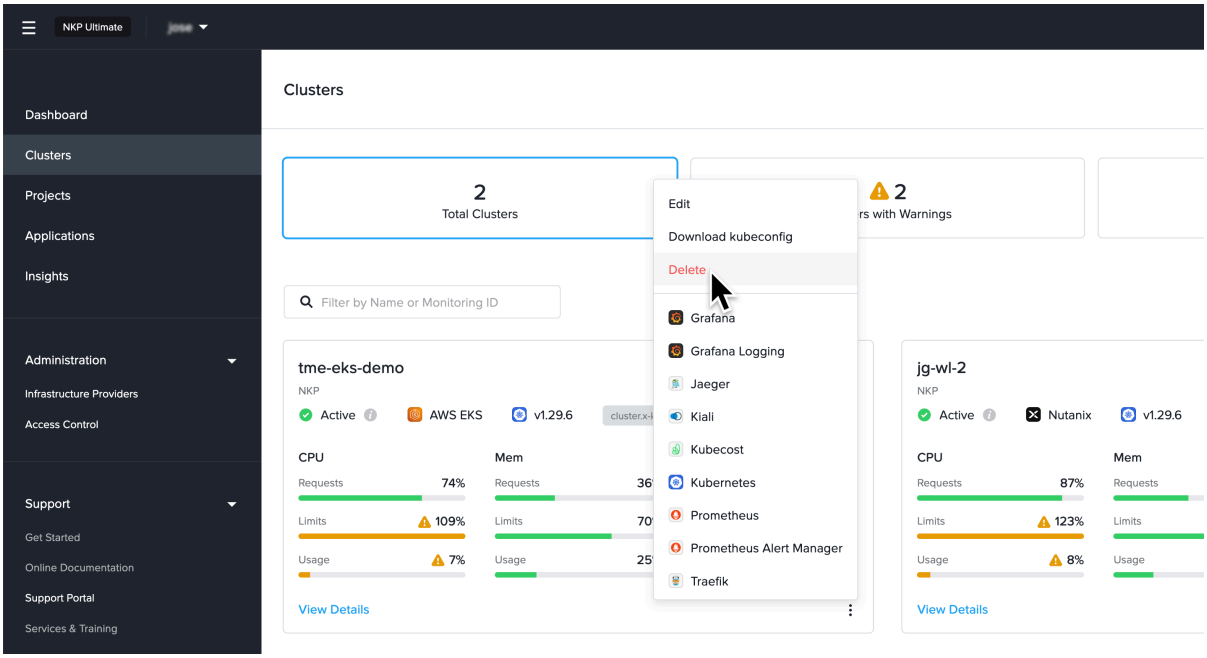


Figure 24: Delete EKS Cluster

4. When the next screen appears, copy the name of your cluster and paste it into the empty box.

5. Now execute the deletion using the Delete Cluster button.

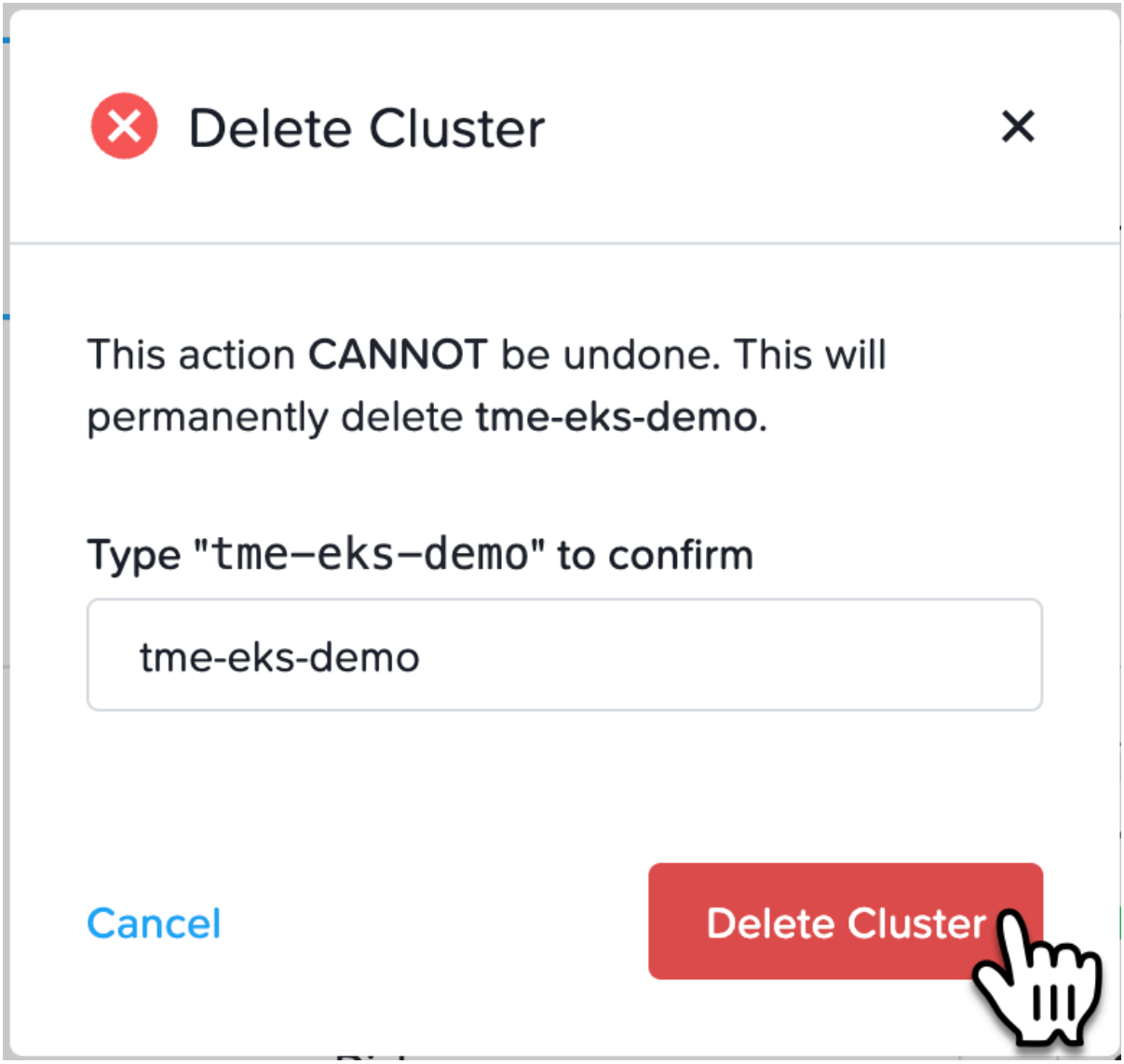


Figure 25: Delete EKS Cluster Button

6. You will see the status as “Deleting” in the top left corner of the cluster you selected for deletion.

What to do next

For a generic overview of deleting clusters within the UI and troubleshooting, see the Disconnecting or Deleting Clusters on page 558 instructions.

Manage EKS Node Pools

Note: Ensure that the `KUBECONFIG` environment variable is set to the self-managed cluster by running `export KUBECONFIG={SELF_MANAGED_AWS_CLUSTER}.conf`.

Node pools are part of a cluster and are managed as a group. They can be used to manage a group of machines using common properties. New default clusters created by Konvoy contain one node pool of worker nodes with the same configuration.

You can create additional node pools for specialized hardware or other configurations. For example, suppose you want to tune your memory usage on a cluster where you need maximum memory for some machines and minimal memory for others. In that case, you can create a new node pool with those specific resource needs.

> **Note:** Konvoy implements node pools using Cluster API MachineDeployments. For more information, see https://cluster-api.sigs.k8s.io/developer/architecture/controllers/machine-deployment.html.

Creating a Node Pool

About this task

Availability zones (AZs) are isolated locations within datacenter regions where public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single Availability Zone, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple Availability Zones.

> **Note:** By default, the first Availability Zone in the region is used for the nodes in the node pool. To create the nodes in a different Availability Zon,e set the appropriate `--availability-zone`. For more information, see https://aws.amazon.com/about-aws/global-infrastructure/regions_az/.

Procedure

To create a new EKS node pool with 3 replicas, run.

```
nkp create nodepool eks ${NODEPOOL_NAME} \
    --cluster-name=${CLUSTER_NAME} \
    --replicas=3
```

```
machinedeployment.cluster.x-k8s.io/example created
awsmachinetemplate.infrastructure.cluster.x-k8s.io/example created
eksconfigtemplate.bootstrap.cluster.x-k8s.io/example created
 # Creating default/example nodepool resources
```

Advanced users can use a combination of the `--dry-run` and `--output=yaml` flags to get a complete set of node pool objects to modify locally or store in version control.

Scaling Up Node Pools

Workflow used to scale a node pool in a cluster.

About this task

To scale up a node pool in a cluster, complete the tasks.

Procedure

1. Run: `nkp scale nodepools`.

   ```
   nkp scale nodepools ${NODEPOOL_NAME} --replicas=5 --cluster-name=${CLUSTER_NAME}
   ```

2. Example output indicating the scaling is in progress.

   ```
   # Scaling node pool example to 5 replicas
   ```

3. After a few minutes, you can list the node pools.

   ```
   nkp get nodepools --cluster-name=${CLUSTER_NAME}
   ```

4. Example output showing the number of DESIRED and READY replicas increased to 5.

```
NODEPOOL                            DESIRED              READY
 KUBERNETES VERSION
example                             5                    5                    v1.23.6

eks-example-md-0                    4                    4                    v1.23.6
```

Deleting EKS Node Pools
Task for deleting node pools in a cluster.

About this task

Deleting a node pool deletes the Kubernetes nodes and the underlying infrastructure. All nodes are drained before deletion, and the pods running on those nodes are rescheduled.

Procedure

1. To delete a node pool from a managed cluster using the command `nkp delete nodepool`.

```
nkp delete nodepool ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME}
```

   The expected output will be similar to the following example, indicating the node pool is being deleted.

2. In this example, the node pool named `example` is being deleted.

```
# Deleting default/example nodepool resources
```

3. Deleting an invalid node pool results in output similar to this example.

```
nkp delete nodepool ${CLUSTER_NAME}-md-invalid --cluster-name=${CLUSTER_NAME}
```

```
MachineDeployments or MachinePools.infrastructure.cluster.x-k8s.io "no
 MachineDeployments
 or MachinePools found for cluster eks-example" not found
```

Azure Infrastructure

Configuration types for installing NKP on Azure Infrastructure.

For an environment on the Azure Infrastructure, install options based on those environment variables are provided for you in this location.

If not already done, complete this guide's Getting Started with ##NKP# section more information, see the following topics in Getting Started with NKP on page 17.

- Resource Requirements
- Installing NKP on page 50
- Prerequisites for Install

Otherwise, proceed to the Azure Prerequisites and Permissions topic to begin your custom installation.

Azure Prerequisites

Before beginning a Nutanix Kubernetes Platform (NKP) installation, verify that you have the following:

- An x86_64-based Linux or macOS machine with a supported operating system version.
- Download the `NKP` binary for Linux or macOS. To check which version of NKP you installed for compatibility reasons, run the `NKP version -h` command.

- A Container engine/runtime installed is required to install NKP and bootstrap:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
- kubectl for interacting with the running cluster.
- Install the Azure CLI
- A valid Azure account with credentials configured.
- Create a custom Azure image using KIB.

Control Plane Nodes

You must have at least three control plane nodes. Each control plane node needs to have at least the following:

- 4 cores
- 16 GiB memory
- Approximately 80 GiB of free space for the volume used for `/var/lib/kubelet` and `/var/lib/containerd`.
- Disk usage must be below 85% on the root volume.

NKP on Azure defaults to deploying an `Standard_D4s_v3` virtual machine with a 128 GiB volume for the OS and an 80GiB volume for etcd storage, which meets the above requirements.

Worker Nodes

You must have at least four worker nodes. The specific number of worker nodes required for your environment can vary depending on the cluster workload and size of the nodes. Each worker node needs to have at least the following:

- 8 cores
- 32 GiB memory
- Around 80 GiB of free space for the volume used for `/var/lib/kubelet` and `/var/lib/containerd`.
- Disk usage must be below 85% on the root volume.

NKP on Azure defaults to deploying a `Standard_D4s_v3` instance with an 80GiB root volume for the OS, which meets the above requirements.

Azure Prerequisites

In Azure, application registration, application objects, and service principals in Azure Active Directory (Azure AD) are used for access. An application must be registered with an Azure AD tenant to delegate identity and access management functions to Azure AD. An Azure AD application is defined by its only application object, which resides in the Azure AD. To access resources secured by an Azure AD tenant, a security principal must represent the entity that requires access. This requirement is true for both users (user principal) and applications (service principal). Therefore, a service principal is a prerequisite, and the next step explains it.

Creating an Azure Service Principal

An Azure service principal is an identity created for use with applications, hosted services, and other automated tools to access Azure resources.

About this task

Service principals provide access to Azure resources with your subscription level. The access is restricted by the roles assigned to the service principal. For more information, see https://learn.microsoft.com/en-us/azure/active-directory/develop/app-objects-and-service-principals?tabs=browser and https://learn.microsoft.com/en-us/azure/databricks/security/auth-authz/access-control/service-principal-acl.

If you have already set a service principal, then the environment variables needed by KIB ([`AZURE_CLIENT_SECRET`, `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`]) are set and do not need repeated if you are still working in the same window.

They are listed below if you have not executed the Azure Service Principal steps.

Procedure

1. Sign in to Azure.

   ```
   az login
   ```

   Output

   ```
   [
     {
       "cloudName": "AzureCloud",
       "homeTenantId": "a1234567-b132-1234-1a11-1234a5678b90",
       "id": "b1234567-abcd-11a1-a0a0-1234a5678b90",
       "isDefault": true,
       "managedByTenants": [],
       "name": "Mesosphere Developer Subscription",
       "state": "Enabled",
       "tenantId": "a1234567-b132-1234-1a11-1234a5678b90",
       "user": {
         "name": "user@azuremesosphere.onmicrosoft.com",
         "type": "user"
       }
     }
   ]
   ```

2. Create an Azure Service Principal (SP).'

   ```
   az ad sp create-for-rbac --role contributor --name "$(whoami)-konvoy" --scopes=/
   subscriptions/$(az account show --query id -o tsv) --query "{ client_id: appId,
    client_secret: password, tenant_id: tenant }"
   ```

   This command will rotate the password if an SP with the name exists.

   ```
   {
     "client_id": "7654321a-1a23-567b-b789-0987b6543a21",
     "client_secret": "Z79yVstq_E.R0R7RUUck718vEHSuyhAB0C",
     "tenant_id": "a1234567-b132-1234-1a11-1234a5678b90"
   }
   ```

3. Set the `AZURE_CLIENT_SECRET` environment variable.

   ```
   export AZURE_CLIENT_SECRET="<azure_client_secret>" #
    Z79yVstq_E.R0R7RUUck718vEHSuyhAB0C
   export AZURE_CLIENT_ID="<client_id>"               # 7654321a-1a23-567b-
   b789-0987b6543a21
   export AZURE_TENANT_ID="<tenant_id>"               # a1234567-
   b132-1234-1a11-1234a5678b90
   ```

```
export AZURE_SUBSCRIPTION_ID="<subscription_id>"   # b1234567-abcd-11a1-
a0a0-1234a5678b90
```

4. Ensure you have an override file to configure specific attributes of your Azure image. Otherwise, edit the YAML file for your OS directly. For more information, see https://github.com/mesosphere/konvoy-image-builder/tree/7447074a6d910e71ad2e61fc4a12820d073ae5ae/images/azure.

Using Konvoy Image Builder with Azure

Learn how to build a custom image with NKP.

About this task

Extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION_$OS` folder. The bundled version of `konvoy-image` contains an embedded `docker` image that contains all the requirements for building.

> **Note:** The `konvoy-image` binary and all supporting folders are also extracted. When extracted, `konvoy-image` bind mounts the current working directory (`${PWD}`) into the container to be used.

This procedure describes using the Konvoy Image Builder (KIB) to create a Cluster API compliant Amazon Machine Image (AMI). KIB uses variable overrides to specify the base and container images for your new AMI.

> **Note:** The default Azure image is not recommended for use in production. We suggest using KIB for Azure to build the image to take advantage of enhanced cluster operations. For more options, see the Customize your Image topic.
>
> For more information regarding using the image in creating clusters, see Creating a New Azure Cluster on page 892.

Before you begin

- Download the Konvoy Image Builder bundle for your version of Nutanix Kubernetes Platform (NKP).
- Check the Supported Kubernetes Version for your Provider.
- Create a working `Docker` setup.

Extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION_$OS` folder. The bundled version of `konvoy-image` contains an embedded `docker` image that contains all the requirements for building.

The `konvoy-image` binary and all supporting folders are also extracted. When extracted, `konvoy-image` bind mounts the current working directory (`${PWD}`) into the container to be used.

Procedure

Run the `konvoy-image` command to build and validate the image.

```
konvoy-image build azure --client-id ${AZURE_CLIENT_ID} --tenant-id ${AZURE_TENANT_ID}
 --overrides override-source-image.yaml images/azure/ubuntu-2004.yaml
```

By default, the image builder builds in the `westus2` location. To specify another location, set the `--location` flag (shown in the example below is how to change the location to `eastus`):

```
konvoy-image build azure --client-id ${AZURE_CLIENT_ID} --tenant-id ${AZURE_TENANT_ID}
 --location eastus --overrides override-source-image.yaml images/azure/ubuntu-2004.yaml
```

When the command is complete, the image ID is printed and written to the `./packer.pkr.hcl` file. This file has an `artifact_id` field whose value provides the name of the image. Then, specify this image ID when creating the cluster.

Image Gallery

About this task

By default, Konvoy Image Builder will create a Resource Group, Gallery, and Image Name to store the resulting image.

Procedure

- To specify a specific Resource Group, Gallery, or Image Name flags may be specified:

```
--gallery-image-locations string     a list of locations to publish the image
 (default same as location)
--gallery-image-name string          the gallery image name to publish the image to
--gallery-image-offer string         the gallery image offer to set (default "nkp")
--gallery-image-publisher string     the gallery image publisher to set (default
 "nkp")
--gallery-image-sku string           the gallery image sku to set
--gallery-name string                the gallery name to publish the image in
 (default "nkp")
--resource-group string              the resource group to create the image in
 (default "nkp")
```

  When creating your cluster, you will add this flag during the creation process for your custom image: `--compute-gallery-id "<Managed Image Shared Image Gallery Id>"`." See Create a New Azure Cluster for specific consumption of image commands.

  The SKU and Image Name will default to the values in the YAML image.

  Ensure you have named the correct YAML file for your OS in the `konvoy-image build` command at https://github.com/mesosphere/konvoy-image-builder/tree/main/images/gcp.

Azure Marketplace

To allow Nutanix Kubernetes Platform (NKP) to create a cluster with Marketplace-based images, such as Rocky Linux, you must specify them with flags.

If these fields were specified in the override file during image creation, the flags must be used in cluster creation:

- `--plan-offer`, `--plan-publisher` and `--plan-sku`

```
--plan-offer rockylinux-9 --plan-publisher
 erockyenterprisesoftwarefoundationinc1653071250513
               --plan-sku rockylinux-9
```

If you see a similar error to "Creating a virtual machine from Marketplace image or a custom image sourced from a Marketplace image requires Plan information in the request." when creating a cluster, you must also set the following flags `--plan-offer`, `--plan-publisher`, `--plan-sku`. For example, when creating a cluster with Rocky Linux VMs, add the following flags to your `nkp create cluster azure` command:

- `--plan-offer`, `--plan-publisher` and `--plan-sku`

See the following for images:

- Azure Marketplace: https://azuremarketplace.microsoft.com/en-us/marketplace/apps?search=procomputers&page=1
- Rocky Linux: https://forums.rockylinux.org/t/azure-rocky-image-on-marketplace/5230

Azure Custom DNS

To use a custom Domain Name Server (DNS) on Azure, you need a DNS name in your control. Once the resource group has been created, you can create your hosted zone with the command below:

```
az network dns zone create --resource-group
              "nutanix-professional-services" --name
```

You no longer need to create a cluster issuer. Several documents explain custom DNS in the Kommander component. For more information from the Azure site, refer to their DNS Overview at https://learn.microsoft.com/en-us/azure/dns/dns-overview.

Azure Non-air-gapped Install

This installation provides instructions on how to install Nutanix Kubernetes Platform (NKP) in an Azure non-air-gapped environment.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Azure Prerequisites

Before you begin using Konvoy with Azure, you must:

1. Sign in to Azure:

```
az login
```


```
[
  {
    "cloudName": "AzureCloud",
    "homeTenantId": "a1234567-b132-1234-1a11-1234a5678b90",
    "id": "b1234567-abcd-11a1-a0a0-1234a5678b90",
    "isDefault": true,
    "managedByTenants": [],
    "name": "Nutanix Developer Subscription",
    "state": "Enabled",
    "tenantId": "a1234567-b132-1234-1a11-1234a5678b90",
    "user": {
      "name": "user@azuremesosphere.onmicrosoft.com",
      "type": "user"
    }
  }
]
```

2. Create an Azure Service Principal (SP) by running the following commands:
   1. If you have more than one Azure account, run this command to identify your account:

      ```
      echo $(az account show --query id -o tsv)
      ```

   2. Run this command to ensure you are pointing to the correct Azure subscription ID:

      ```
      az account set --subscription "Nutanix Developer Subscription"
      ```

   3. If an SP with the name exists, this command rotates the password:

      ```
      az ad sp create-for-rbac --role contributor --name "$(whoami)-konvoy" --scopes=/
      subscriptions/$(az account show --query id -o tsv) --query "{ client_id: appId,
       client_secret: password, tenant_id: tenant }"
      ```

      Output:

      ```
      {
        "client_id": "7654321a-1a23-567b-b789-0987b6543a21",
        "client_secret": "Z79yVstq_E.R0R7RUUck718vEHSuyhAB0C",
        "tenant_id": "a1234567-b132-1234-1a11-1234a5678b90"
      }
      ```

3. Set the `AZURE_CLIENT_SECRET` environment variable:

   ```
   export AZURE_CLIENT_SECRET="<azure_client_secret>" #
    Z79yVstq_E.R0R7RUUck718vEHSuyhAB0C
   export AZURE_CLIENT_ID="<client_id>"               # 7654321a-1a23-567b-
   b789-0987b6543a21
   export AZURE_TENANT_ID="<tenant_id>"               # a1234567-
   b132-1234-1a11-1234a5678b90
   export AZURE_SUBSCRIPTION_ID="<subscription_id>"   # b1234567-abcd-11a1-
   a0a0-1234a5678b90
   ```

4. Ensure you have an override file to configure specific attributes of your Azure image.

Bootstrapping Azure
To create Kubernetes clusters, NKP uses Cluster API (CAPI) controllers. These controllers run on a Kubernetes cluster.

About this task

To get started, you need a bootstrap cluster. By default, Nutanix Kubernetes Platform (NKP) creates a bootstrap cluster for you in a Docker container using the Kubernetes-in-Docker (KIND) tool.

Before you begin

Procedure

1. Complete the Nutanix Infrastructure Prerequisites. For more information, see Nutanix Infrastructure Prerequisites on page 690.
2. Ensure the `NKP` binary can be found in your `$PATH`.

Bootstrap Cluster Life Cycle Services

Procedure

1. Review Universal Configurations for all Infrastructure Providers regarding settings, flags, and other choices and then begin bootstrapping.

2. Create a bootstrap cluster using the command `nkp create bootstrap --kubeconfig $HOME/.kube/config`.

   > **Note:** Use `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Configuring an HTTP or HTTPS Proxy on page 677.

   Example output:

   ```
   # Creating a bootstrap cluster
   # Initializing new CAPI components
   ```

3. NKP creates a bootstrap cluster using KIND as a library.

   For more information, see https://github.com/kubernetes-sigs/kind.

4. NKP then deploys the following Cluster API providers on the cluster.

   - Core Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/
   - AWS Infrastructure Provider: https://github.com/kubernetes-sigs/cluster-api-provider-aws
   - Kubeadm Bootstrap Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/bootstrap/kubeadm
   - Kubeadm ControlPlane Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/controlplane/kubeadm

   For more information on Cluster APIs, see https://cluster-api.sigs.k8s.io/.

5. NKP waits until these providers' controller-manager and webhook deployments are ready. List these deployments using the command `kubectl get --all-namespaces deployments -l=clusterctl.cluster.x-k8s.io`

   Output example:

   ```
   NAMESPACE                           NAME
    READY   UP-TO-DATE   AVAILABLE   AGE
   capa-system                         capa-controller-manager
    1/1     1            1           1h
   capg-system                         capg-controller-manager
    1/1     1            1           1h
   capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-manager
    1/1     1            1           1h
   capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-manager
    1/1     1            1           1h
   capi-system                         capi-controller-manager
    1/1     1            1           1h
   cappp-system                        cappp-controller-manager
    1/1     1            1           1h
   capv-system                         capv-controller-manager
    1/1     1            1           1h
   capz-system                         capz-controller-manager
    1/1     1            1           1h
   cert-manager                        cert-manager
    1/1     1            1           1h
   cert-manager                        cert-manager-cainjector
    1/1     1            1           1h
   cert-manager                        cert-manager-webhook
    1/1     1            1           1h
   ```

Creating a New Azure Cluster

Create an Azure cluster in a non-air-gapped environment.

About this task

Use this procedure to create a Kubernetes cluster with NKP. A self-managed cluster is one in which the CAPI resources and controllers that describe and manage it run on the same cluster they are managing. First, you must name your cluster.

Before you begin

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. For more naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable using the command `export CLUSTER_NAME=<azure-example>`.

Encode your Azure Credential Variables:

Procedure

Base64 encodes the Azure environment variables set in the Azure install prerequisites step.

```
export AZURE_SUBSCRIPTION_ID_B64="$(echo -n "${AZURE_SUBSCRIPTION_ID}" | base64 | tr -d
 '\n')"
export AZURE_TENANT_ID_B64="$(echo -n "${AZURE_TENANT_ID}" | base64 | tr -d '\n')"
export AZURE_CLIENT_ID_B64="$(echo -n "${AZURE_CLIENT_ID}" | base64 | tr -d '\n')"
export AZURE_CLIENT_SECRET_B64="$(echo -n "${AZURE_CLIENT_SECRET}" | base64 | tr -d
 '\n')"
```

Create an Azure Kubernetes Cluster

About this task

If you use these instructions to create a cluster on Azure using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes.

NKP uses Azure CSI as the default storage provider. You can use a Kubernetes CSIcompatible storage solution suitable for production. For more information, see the Kubernetes documentation called ##Changing the Default Storage Class#

Availability zones (AZs), which are isolated locations within datacenter regions from which public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single Availability Zone, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple Availability Zones.

Procedure

1. To use a custom Azure Image when creating your cluster, you must create that Azure Image using KIB first and then use the flag `--compute-gallery-id` to apply the image.

   ```
   ...
   ```

```
--compute-gallery-id "<Managed Image Shared Image Gallery Id>"
```

Note:

The `--compute-gallery-id` image will be in the format

```
--compute-gallery-id /subscriptions/<subscription id>/resourceGroups/
<resource group
      name>/providers/Microsoft.Compute/galleries/<gallery name>/images/
<image definition
      name>/versions/<version id>
```

2. Ensure your subnets do not overlap with your host subnet because they cannot be changed after cluster creation. If you need to change the kubernetes subnets, you must do this at cluster creation. The default subnets used in NKP are.

```
spec:
  clusterNetwork:
    pods:
      cidrBlocks:
      - 192.168.0.0/16
    services:
      cidrBlocks:
      - 10.96.0.0/12
```

   » Additional Options for your environment; otherwise, proceed to the next step to create your cluster. (Optional) Modify Control Plane Audit logs - Users can modify the `KubeadmControlplane` cluster-API object to configure different `kubelet` options. See the following guide if you wish to configure your control plane beyond the existing options available from flags.

   » (Optional) Use a registry mirror. Configure your cluster to use an existing local registry as a mirror when attempting to pull images previously pushed to your registry.

3. Run this command to create your Kubernetes cluster using any relevant flags.

```
nkp create cluster azure \
--cluster-name=${CLUSTER_NAME} \
--dry-run \
--output=yaml \
> ${CLUSTER_NAME}.yaml
```

Note: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

4. Inspect or edit the cluster objects. Familiarize yourself with Cluster API before editing the cluster objects, as edits can prevent the cluster from deploying successfully.

5. Create the cluster from the objects generated from the `dry run`. A warning will appear in the console if the resource already exists, requiring you to remove the resource or update your YAML.

```
kubectl create -f ${CLUSTER_NAME}.yaml
```

Note: If you used the `--output-directory` flag in your `NKP create .. --dry-run` step above, create the cluster from the objects you created by specifying the directory:

```
kubectl create -f <existing-directory>/.
```

6. Wait for the cluster control plan to be ready.

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=20m
```

7. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. Konvoy provides a command to describe the current status of the cluster.

```
nkp describe cluster -c ${CLUSTER_NAME}
```

Making the New Azure Cluster Self-Managed

How to make a Kubernetes cluster manage itself.

About this task

Nutanix Kubernetes Platform (NKP) deploys all cluster life cycle services to a bootstrap cluster, which then deploys a workload cluster. When the workload cluster is ready, move the cluster life cycle services to the workload cluster, which makes the workload cluster self-managed.

Before you begin

Before starting, ensure you can create a workload cluster as described in the topic: Create a New Azure Cluster.

This page contains instructions on how to make your cluster self-managed. This is necessary if there is only one cluster in your environment or if this cluster becomes the Management cluster in a multi-cluster environment.

> **Note:** If you already have a self-managed or Management cluster in your environment, skip this page.

Follow these steps to turn your new cluster into a **Management Cluster** for an Ultimate license environment (or a free-standing Pro Cluster):

Procedure

1. Deploy cluster life cycle services on the workload cluster.

```
nkp create capi-components --kubeconfig ${CLUSTER_NAME}.conf
```

Output:

```
# Initializing new CAPI components
```

> **Note:** If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP/HTTPS Proxy.

2. The cluster life cycle services on the workload cluster are ready, but the workload cluster configuration is on the bootstrap cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the bootstrap to the workload cluster. This process is called a Pivot. For more information, see https://cluster-api.sigs.k8s.io/reference/glossary.html?highlight=pivot#pivot.

Move the Cluster API objects from the bootstrap to the workload cluster:

```
nkp move capi-resources --to-kubeconfig ${CLUSTER_NAME}.conf
```

Output:

```
# Moving cluster resources
```

```
You can now view resources in the moved cluster by using the --kubeconfig flag with
 kubectl. For example: kubectl --kubeconfig=gcp-example.conf get nodes
```

> **Note:** To ensure only one set of cluster life cycle services manages the workload cluster, NKP first pauses the reconciliation of the objects on the bootstrap cluster, then creates the objects on the workload cluster. As NKP copies the objects, the cluster life cycle services on the workload cluster reconcile the objects. The workload cluster becomes self-managed after NKP creates all the objects. If it fails, the `move` command can be safely retried.

3. Wait for the cluster control plan to be ready.

```
kubectl --kubeconfig ${CLUSTER_NAME}.conf wait --for=condition=ControlPlaneReady
 "clusters/${CLUSTER_NAME}" --timeout=20m
```

Output:

```
cluster.cluster.x-k8s.io/gcp-example condition met
```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster status. After moving the cluster life cycle services to the workload cluster, remember to use NKP with the workload cluster `kubeconfig`.

```
nkp describe cluster --kubeconfig ${CLUSTER_NAME}.conf -c ${CLUSTER_NAME}
```

Output:

```
NAME                                                              READY  SEVERITY
 REASON  SINCE  MESSAGE
Cluster/azure-example                                             True
      55s
##ClusterInfrastructure - AzureCluster/azure-example             True
      67s
##ControlPlane - KubeadmControlPlane/azure-example-control-plane  True
      55s
# ##Machine/azure-example-control-plane-67f47                     True
      58s
# ##Machine/azure-example-control-plane-7pllh                     True
      65s
# ##Machine/azure-example-control-plane-jtfgv                     True
      65s
##Workers

  ##MachineDeployment/azure-example-md-0                          True
      67s
    ##Machine/azure-example-md-0-f9cb9c79b-6nsb9                 True
      59s
    ##Machine/azure-example-md-0-f9cb9c79b-jxwl6                 True
      58s
    ##Machine/azure-example-md-0-f9cb9c79b-ktg7z                 True
      59s
    ##Machine/azure-example-md-0-f9cb9c79b-nxcm2                 True
      66s
```

5. Remove the bootstrap cluster because the workload cluster is now self-managed.

```
NKP delete bootstrap --kubeconfig $HOME/.kube/config
# Deleting bootstrap cluster
```

Known Limitations

Procedure

- NKP only supports moving all namespaces in the cluster; NKP does not support migration of individual namespaces.
- Konvoy supports moving only one set of cluster objects from the bootstrap cluster to the workload cluster or vice-versa.

Exploring the Azure Cluster

This guide explains how to use the command line to interact with your newly deployed Kubernetes cluster.

About this task

Before starting, create a workload cluster, as described in Create a New Azure Cluster.

Procedure

1. When the workload cluster is created, the cluster life cycle services generate a `kubeconfig` file for the workload cluster and write it to a Secret. The `kubeconfig` file is scoped to the cluster administrator. Get a `kubeconfig` file for the workload cluster.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
   ```

2. Verify the API server is up by listing the nodes.

   ```
   kubectl --kubeconfig=${CLUSTER_NAME}.conf get nodes
   ```

   > **Note:** The Status may take a few minutes to move to `Ready` while the Pod network is deployed. The node status will change to `Ready` soon after the `calico-node` DaemonSet Pods are Ready.

   Example output:

   ```
   NAME                                STATUS   ROLES                  AGE     VERSION
   azure-example-control-plane-7ffnl   Ready    control-plane,master   6m18s   v1.28.7
   azure-example-control-plane-l4bv8   Ready    control-plane,master   14m     v1.28.7
   azure-example-control-plane-n4g4l   Ready    control-plane,master   18m     v1.28.7
   azure-example-md-0-mpctb            Ready    <none>                 15m     v1.28.7
   azure-example-md-0-qglp9            Ready    <none>                 15m     v1.28.7
   azure-example-md-0-sgrd6            Ready    <none>                 16m     v1.28.7
   azure-example-md-0-wzbkl            Ready    <none>                 16m     v1.28.7
   ```

3. List the Pods with the command.

   ```
   kubectl --kubeconfig=${CLUSTER_NAME}.conf get pods -A
   ```

   Verify the output:

   ```
   NAMESPACE                           NAME
                  READY   STATUS    RESTARTS      AGE
   calico-system                       calico-kube-controllers-57fbd7bd59-v4tss
                  1/1     Running   0             19m
   calico-system                       calico-node-59llv
                  1/1     Running   0             17m
   calico-system                       calico-node-7t7wj
                  1/1     Running   0             16m
   calico-system                       calico-node-pf8q8
                  1/1     Running   0             17m
   calico-system                       calico-node-sh2b7
                  1/1     Running   0             8m17s
   ```

```
calico-system                       calico-node-tmxl5
                1/1     Running   0             19m
calico-system                       calico-node-vt5fh
                1/1     Running   0             18m
calico-system                       calico-node-whfs8
                1/1     Running   0             18m
calico-system                       calico-typha-797c9666d5-5w99r
                1/1     Running   0             19m
calico-system                       calico-typha-797c9666d5-hj6mj
                1/1     Running   0             18m
calico-system                       calico-typha-797c9666d5-s7rc6
                1/1     Running   0             17m
capa-system                         capa-controller-manager-74fffb5676-ch6xd
                1/1     Running   0             11m
capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-
manager-867759cc67-vg4lh       1/1     Running   0             15m
capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-
manager-5df55579c4-pc8x9   1/1     Running   1 (11m ago)   15m
capi-system                         capi-controller-manager-79cc58bf5f-xsp9t
                1/1     Running   0             15m
cappp-system                        cappp-controller-manager-85b5c77497-8ss8r
                1/1     Running   0             14m
capv-system                         capv-controller-manager-7bf4d8b66-6x2mx
                1/1     Running   0             14m
capz-system                         capz-controller-manager-5d4c6468bf-wfhcc
                1/1     Running   0             14m
capz-system                         capz-nmi-2cbrg
                1/1     Running   0             14m
capz-system                         capz-nmi-8dllm
                1/1     Running   0             14m
capz-system                         capz-nmi-95dfk
                1/1     Running   0             14m
capz-system                         capz-nmi-rtnd4
                1/1     Running   0             14m
cert-manager                        cert-manager-848f547974-gjc5p
                1/1     Running   1 (10m ago)   15m
cert-manager                        cert-manager-cainjector-54f4cc6b5-rnh4f
                1/1     Running   0             15m
cert-manager                        cert-manager-webhook-7c9588c76-rn2sd
                1/1     Running   0             15m
kube-system                         cluster-autoscaler-68c759fbf6-6vg5r
                1/1     Running   1 (11m ago)   20m
kube-system                         coredns-78fcd69978-6gx44
                1/1     Running   0             20m
kube-system                         coredns-78fcd69978-gr5q7
                1/1     Running   0             20m
kube-system                         csi-azuredisk-controller-c8fb44c8b-jhmfz
                6/6     Running   5 (11m ago)   20m
kube-system                         csi-azuredisk-controller-c8fb44c8b-lpbbs
                6/6     Running   0             20m
kube-system                         csi-azuredisk-node-2g7vw
                3/3     Running   0             8m17s
kube-system                         csi-azuredisk-node-6rdqc
                3/3     Running   0             18m
kube-system                         csi-azuredisk-node-99c6q
                3/3     Running   0             17m
kube-system                         csi-azuredisk-node-9b4ms
                3/3     Running   0             17m
kube-system                         csi-azuredisk-node-mz5pr
                3/3     Running   0             18m
kube-system                         csi-azuredisk-node-r2t99
                3/3     Running   0             16m
```

```
kube-system                         csi-azuredisk-node-t7gfs
                3/3     Running   0             20m
kube-system                         etcd-azure-example-control-plane-7ffnl
                1/1     Running   0             8m15s
kube-system                         etcd-azure-example-control-plane-l4bv8
                1/1     Running   0             16m
kube-system                         etcd-azure-example-control-plane-n4g4l
                1/1     Running   0             19m
kube-system                         kube-apiserver-azure-example-control-plane-7ffnl
                1/1     Running   0             8m16s
kube-system                         kube-apiserver-azure-example-control-plane-l4bv8
                1/1     Running   0             16m
kube-system                         kube-apiserver-azure-example-control-plane-n4g4l
                1/1     Running   0             19m
kube-system                         kube-controller-manager-azure-example-control-
plane-7ffnl        1/1     Running   0             8m17s
kube-system                         kube-controller-manager-azure-example-control-
plane-l4bv8        1/1     Running   0             16m
kube-system                         kube-controller-manager-azure-example-control-
plane-n4g4l        1/1     Running   1 (17m ago)   19m
kube-system                         kube-proxy-82zdl
                1/1     Running   0             8m17s
kube-system                         kube-proxy-fd9f9
                1/1     Running   0             18m
kube-system                         kube-proxy-l6lgc
                1/1     Running   0             17m
kube-system                         kube-proxy-lzswh
                1/1     Running   0             16m
kube-system                         kube-proxy-ndfmt
                1/1     Running   0             20m
kube-system                         kube-proxy-nxlp9
                1/1     Running   0             18m
kube-system                         kube-proxy-v9sxp
                1/1     Running   0             17m
kube-system                         kube-scheduler-azure-example-control-plane-7ffnl
                1/1     Running   0             8m16s
kube-system                         kube-scheduler-azure-example-control-plane-l4bv8
                1/1     Running   0             16m
kube-system                         kube-scheduler-azure-example-control-plane-n4g4l
                1/1     Running   1 (17m ago)   19m
node-feature-discovery              node-feature-discovery-master-84c67dcbb6-d2gm7
                1/1     Running   0             20m
node-feature-discovery              node-feature-discovery-worker-drgf6
                1/1     Running   0             17m
node-feature-discovery              node-feature-discovery-worker-hcz6k
                1/1     Running   0             17m
node-feature-discovery              node-feature-discovery-worker-pgbcd
                1/1     Running   0             16m
node-feature-discovery              node-feature-discovery-worker-vhj96
                1/1     Running   0             16m
tigera-operator                     tigera-operator-d499f5c8f-jnj8b
                1/1     Running   1 (18m ago)   19m
```

Installing Kommander in Azure

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped Azure environment.

About this task

Once you have installed the Konvoy component of Nutanix Kubernetes Platform (NKP), you will continue installing the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a time period (for example, 1 hour) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass. For more information, see Creating a Default StorageClass on page 493.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. If required: Customize your `kommander.yaml`.

   a. See the Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.

5. Only required if your cluster uses a custom AWS VPC and requires an internal load-balancer; set the traefik annotation to create an internal-facing ELB.

   ```
   apps:
     traefik:
       enabled: true
       values: |
         service:
           annotations:
             service.beta.kubernetes.io/aws-load-balancer-internal: "true"
   ```

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `nkp-catalog-appliations`.

   ```
   apiVersion: config.kommander.mesosphere.io/v1alpha1
   kind: Installation
   catalog:
   ```

```
repositories:
  - name: dkp-catalog-applications
    labels:
      kommander.nutanix.io/project-default-catalog-repository: "true"
      kommander.nutanix.io/workspace-default-catalog-repository: "true"
      kommander.nutanix.io/gitapps-gitrepository-type: "nkp"
    gitRepositorySpec:
      url: https://github.com/mesosphere/dkp-catalog-applications
      ref:
        tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Enable NKP Catalog Applications after Installing NKP.

Azure Management Tools

After cluster creation and configuration, you can revisit clusters to update and change variables.

Replacing an Azure Node

In certain situations, you may want to delete a worker node and have Cluster API replace it with a newly provisioned machine.

About this task

Identify the name of the node to delete.

List the nodes

Procedure

1. List the nodes.

```
kubectl --kubeconfig ${CLUSTER_NAME}.conf get nodes
```

The output from this command resembles the following:

```
NAME                                STATUS   ROLES                  AGE   VERSION
azure-example-control-plane-ckwm4   Ready    control-plane,master   35m   v1.28.7
azure-example-control-plane-d4fdf   Ready    control-plane,master   31m   v1.28.7
azure-example-control-plane-qrvm9   Ready    control-plane,master   33m   v1.28.7
azure-example-md-0-4w7gq            Ready    <none>                 33m   v1.28.7
azure-example-md-0-6gb9k            Ready    <none>                 33m   v1.28.7
azure-example-md-0-p2n8c            Ready    <none>                 11m   v1.28.7
azure-example-md-0-s5zbh            Ready    <none>                 33m   v1.28.7
```

2. Export a variable with the node name for the next steps. This example uses the name `ip-10-0-100-85.us-west-2.compute.internal`.

```
export NAME_NODE_TO_DELETE="<azure-example-control-plane-ckwm4>"
```

3. Delete the Machine resource.

```
export NAME_MACHINE_TO_DELETE=$(kubectl --kubeconfig ${CLUSTER_NAME}.conf get
 machine -ojsonpath="{.items[?(@.status.nodeRef.name==\"$NAME_NODE_TO_DELETE
\")].metadata.name}")
kubectl --kubeconfig ${CLUSTER_NAME}.conf delete machine "$NAME_MACHINE_TO_DELETE"
```

Output:

```
machine.cluster.x-k8s.io "aws-example-1-md-0-cb9c9bbf7-t894m" deleted
```

The command will not return immediately. It will return after the Machine resource has been deleted.

The corresponding Node resource is also deleted a few minutes after the Machine resource is deleted.

4. Observe that the Machine resource is being replaced using this command.

```
kubectl --kubeconfig ${CLUSTER_NAME}.conf get machinedeployment
```

Output:

```
NAME                 CLUSTER         REPLICAS   READY   UPDATED   UNAVAILABLE   PHASE
       AGE     VERSION
azure-example-md-0   azure-example   4          3       4         1
 ScalingUp   7m30s   v1.28.7
long-running-md-0    long-running    4          4       4         0
 Running     7m28s   v1.28.7
```

5. Identify the replacement Machine using this command.

```
export NAME_NEW_MACHINE=$(kubectl --kubeconfig ${CLUSTER_NAME}.conf get machines \
    -l=cluster.x-k8s.io/deployment-name=${CLUSTER_NAME}-md-0 \
    -ojsonpath='{.items[?(@.status.phase=="Running")].metadata.name}{"\n"}')
echo "$NAME_NEW_MACHINE"
```

Output:

```
azure-example-md-0-d67567c8b-2674r azure-example-md-0-d67567c8b-n276j azure-example-
md-0-d67567c8b-pzg8k azure-example-md-0-d67567c8b-z8km9
```

If the output is empty, the new Machine has probably exited the `Provisioning` phase and entered the `Running` phase.

6. Identify the replacement Node using this command.

```
kubectl --kubeconfig ${CLUSTER_NAME}.conf get nodes
```

Output:

```
NAME                                STATUS   ROLES                  AGE     VERSION
azure-example-control-plane-d4fdf   Ready    control-plane,master   43m     v1.28.7
azure-example-control-plane-qrvm9   Ready    control-plane,master   45m     v1.28.7
azure-example-control-plane-tz56m   Ready    control-plane,master   8m22s   v1.28.7
azure-example-md-0-4w7gq            Ready    <none>                 45m     v1.28.7
azure-example-md-0-6gb9k            Ready    <none>                 45m     v1.28.7
azure-example-md-0-p2n8c            Ready    <none>                 22m     v1.28.7
azure-example-md-0-s5zbh            Ready    <none>                 45m     v1.28.7
```

If the output is empty, the Node resource is not yet available or does not yet have the expected annotation. Wait a few minutes, then repeat the command.

Deleting an Azure Cluster
Deleting an Azure cluster.

About this task

A self-managed workload cluster cannot delete itself. If your workload cluster is self-managed, you must first create a bootstrap cluster and move the cluster life cycle services to it before deleting the workload cluster.

Procedure

If you did not make your workload cluster self-managed, as described in Make New Cluster Self-Managed, proceed to the instructions for Delete the workload cluster.

Create a Bootstrap Cluster and Move CAPI Resources

About this task

Follow these steps to create a bootstrap cluster and move CAPI resources:

Procedure

1. Make sure your AWS credentials are up to date. Refresh the credentials using this command.

   ```
   nkp update bootstrap credentials aws --kubeconfig $HOME/.kube/config
   ```

2. The bootstrap cluster will host the Cluster API controllers that reconcile the cluster objects marked for deletion. Create a bootstrap cluster. To avoid using the wrong kubeconfig, the following steps use explicit kubeconfig paths and contexts.

   ```
   nkp create bootstrap --kubeconfig $HOME/.kube/config --with-aws-bootstrap-
   credentials=true
   ```

3. Move the Cluster API objects from the workload to the bootstrap cluster: The cluster life cycle services on the bootstrap cluster are ready, but the workload cluster configuration is on the workload cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the workload to the bootstrap cluster. This process is also called a Pivot.

   ```
   nkp move capi-resources \
    --from-kubeconfig ${CLUSTER_NAME}.conf \
    --from-context ${CLUSTER_NAME}-admin@${CLUSTER_NAME} \
    --to-kubeconfig $HOME/.kube/config \
    --to-context kind-konvoy-capi-bootstrapper
   ```

   Output:

   ```
   # Moving cluster resources
   You can now view resources in the moved cluster by using the --kubeconfig flag with
    kubectl. For example: kubectl --kubeconfig $HOME/.kube/config get nodes
   ```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster's status.

   ```
   nkp describe cluster --kubeconfig $HOME/.kube/config -c ${CLUSTER_NAME}
   ```

   Output:

   ```
        102s
   ```

5. Wait for the cluster control plan to be ready.

```
kubectl --kubeconfig $HOME/.kube/config wait --for=condition=controlplaneready
 "clusters/${CLUSTER_NAME}" --timeout=20m
```

Output:

```
NAME                                                                READY  SEVERITY
 REASON  SINCE  MESSAGE
Cluster/azure-example                                               True
      15s
##ClusterInfrastructure - AzureCluster/azure-example               True
      29s
##ControlPlane - KubeadmControlPlane/azure-example-control-plane   True
      15s
# ##Machine/azure-example-control-plane-gvj5d                       True
      22s
# ##Machine/azure-example-control-plane-l8j9r                       True
      23s
# ##Machine/azure-example-control-plane-xhxxg                       True
      23s
##Workers
  ##MachineDeployment/azure-example-md-0                            True
      35s
 ##Machine/azure-example-md-0-d67567c8b-2674r                   True
   24s
 ##Machine/azure-example-md-0-d67567c8b-n276j                   True
   25s
 ##Machine/azure-example-md-0-d67567c8b-pzg8k                   True
   23s
 ##Machine/azure-example-md-0-d67567c8b-z8km9                   True
   24s
```

Delete the Workload Cluster

Procedure

1. To delete a cluster, Use `nkp delete cluster` and pass in the name of the cluster you are trying to delete with `--cluster-name` flag. Use `kubectl get clusters` to get those details (`--cluster-name` and `--namespace`) of the Kubernetes cluster to delete it.

```
kubectl get clusters
```

2. Delete the Kubernetes cluster and wait a few minutes.

> **Note:** Before deleting the cluster, Nutanix Kubernetes Platform (NKP) deletes all Services of type LoadBalancer on the cluster. To skip this step, use the flag `--delete-kubernetes-resources=false`.

```
nkp delete cluster --cluster-name=${CLUSTER_NAME} --kubeconfig $HOME/.kube/config
```

Output:

```
# Deleting Services with type LoadBalancer for Cluster default/azure-example
# Deleting ClusterResourceSets for Cluster default/azure-example
# Deleting cluster resources
# Waiting for cluster to be fully deleted
Deleted default/azure-example cluster
```

After the workload cluster is deleted, you can delete the bootstrap cluster.

Delete the Bootstrap Cluster

About this task

After you have moved the workload resources back to a bootstrap cluster and deleted the workload cluster, you no longer need the bootstrap cluster. You can safely delete the bootstrap cluster with these steps:

Procedure

Delete the bootstrap cluster.

```
nkp delete bootstrap --kubeconfig $HOME/.kube/config
```

Output:

```
# Deleting bootstrap cluster
```

Azure Certificate Renewal

During cluster creation, Kubernetes establishes a Public Key Infrastructure (PKI) for generating the TLS certificates needed for securing cluster communication for various components such as `etcd`, `kubernetes-apiserver` , and `kube-proxy`. The certificates created by these components have a default expiration of one year and are renewed when an administrator updates the cluster.

Kubernetes provides a facility to renew all certificates automatically during control plane updates. For administrators who need long-running clusters or clusters that are not upgraded often, `nkp` provides automated certificate renewal without a cluster upgrade.

- This feature requires Python 3.5 or greater to be installed on all control plane hosts.
- Complete the Bootstrap Cluster topic.

To create a cluster with automated certificate renewal, you create a Konvoy cluster using the `certificate-renew-interval` flag. The `certificate-renew-interval` is the number of days after which Kubernetes-managed PKI certificates will be renewed. For example, an `certificate-renew-interval` value of 60 means the certificates will be renewed every 60 days.

Technical Details

The following manifests are modified on the control plane hosts and are located at `/etc/kubernetes/manifests`. Modifications to these files require SUDO access.

```
kube-controller-manager.yaml
kube-apiserver.yaml
kube-scheduler.yaml
kube-proxy.yaml
```

The following annotation indicates the time each component was reset:

```
metadata:
  annotations:
    konvoy.d2iq.io/restartedAt: $(date +%s)
```

This only occurs when the PKI certificates are older than the interval given at cluster creation time. This is activated by a `systemd timer` called `renew-certs.timer` that triggers an associated `systemd` service called `renew-certs.service` that runs on all of the control plane hosts.

AKS Infrastructure

Configuration types for installing NKP on Azure Kubernetes Serves (AKS) Infrastructure.

You can choose from multiple configuration types when installing ##NKP# on Azure Kubernetes Service (AKS) infrastructure. If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50

- Prerequisites for Installation on page 47

The different types of AKS configuration types supported in NKP are covered in this section.

> **Note:** An AKS cluster cannot be a Management or Pro cluster. When installing NKP on your AKS cluster, first ensure you have a Management cluster with NKP and the Kommander component installed that handles the life cycle of your AKS cluster.

Use Nutanix Kubernetes Platform to Create a New AKS Cluster

> **Note:** Ensure that the `KUBECONFIG` environment variable is set to the Management cluster by running `export KUBECONFIG=<Management_cluster_kubeconfig>.conf`.

Name Your Cluster

Guidelines for naming your new cluster.

Give your cluster a unique name suitable for your environment.

- The cluster name may only contain the following characters: `a-z`, `0-9`, `.`, and `-`.
- Cluster creation will fail if the name includes capital letters.
- For more naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

Creating a New AKS Kubernetes Cluster

About this task

Context for the current task

Procedure

1. Set the environment variable to a name for this cluster.

   ```
   export CLUSTER_NAME=aks-example
   ```

2. Check to see what version of Kubernetes is available in your region. When deploying with Azure Kubernetes Service (AKS), you need to declare the version of Kubernetes you wish to use by running the following command, substituting `<your-location>` for the Azure region you're deploying to.

   ```
   az aks get-versions -o table --location your-location
   ```

3. Set the version of Kubernetes you've chosen.

   > **Note:** Refer to the current release Kubernetes compatibility table for the correct version to use and select an available 1.27.x version. The version listed in the command is an example.

   ```
   export KUBERNETES_VERSION=1.27.6
   ```

4. Create the cluster.

   ```
   nkp create cluster aks --cluster-name=$CLUSTER_NAME --additional-tags=owner=$(whoami)
    --kubernetes-version=$KUBERNETES_VERSION
   ```

   If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy.

   ```
   Generating cluster resources
   ```

```
cluster.cluster.x-k8s.io/aks-example created
azuremanagedcontrolplane.infrastructure.cluster.x-k8s.io/aks-example created
azuremanagedcluster.infrastructure.cluster.x-k8s.io/aks-example created
machinepool.cluster.x-k8s.io/aks-example created
azuremanagedmachinepool.infrastructure.cluster.x-k8s.io/cp6dsz8 created
machinepool.cluster.x-k8s.io/aks-example-md-0 created
azuremanagedmachinepool.infrastructure.cluster.x-k8s.io/mp6gglj created
clusterresourceset.addons.cluster.x-k8s.io/cluster-autoscaler-aks-example created
configmap/cluster-autoscaler-aks-example created
clusterresourceset.addons.cluster.x-k8s.io/node-feature-discovery-aks-example created
configmap/node-feature-discovery-aks-example created
clusterresourceset.addons.cluster.x-k8s.io/nvidia-feature-discovery-aks-example
 created
configmap/nvidia-feature-discovery-aks-example created
```

Inspecting or Editing the Cluster Objects

About this task

Perform this task using your favorite text editor.

> **Note:** Editing the cluster objects requires some understanding of Cluster API. Edits can prevent the cluster from deploying successfully.

The objects are Custom Resources defined by Cluster API components, and they belong to three different categories:

- **Cluster** A Cluster object references the infrastructure-specific and control plane objects.
- **Control Plane**
- **Node Pool** A Node Pool is a collection of machines with identical properties. For example, a cluster might have one Node Pool with large memory capacity and another Node Pool with GPU support. Each Node Pool is described by three objects: The MachinePool references an object that describes the configuration of Kubernetes components (kubelet) deployed on each node pool machine and an infrastructure-specific object that describes the properties of all node pool machines. Here, it references a `KubeadmConfigTemplate`.

  > **Note:** For more information on Custom Resources, see https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/

For more information about the objects, see *Concepts* in the Cluster API Book https://cluster-api.sigs.k8s.io/user/concepts.html.

Procedure

1. Wait for the cluster control-plane to be ready using the command `kubectl wait --for=condition=ControlPlaneReady "clusters/$CLUSTER_NAME" --timeout=20m`

   Example:

   ```
   cluster.cluster.x-k8s.io/aks-example condition met
   ```

   The `READY` status will become `True` after the cluster control-plane becomes ready.

2. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. To describe the current status of the cluster, use the command `nkp describe cluster -c $CLUSTER_NAME`

   Example:

   ```
   NAME                                                          READY  SEVERITY  REASON
    SINCE  MESSAGE
   Cluster/aks-example                                           True
    48m
   ##ClusterInfrastructure - AzureManagedCluster/aks-example
   ##ControlPlane - AzureManagedControlPlane/aks-example
   ```

3. As they progress, the controllers also create Events. To list the events, use the command `kubectl get events | grep $CLUSTER_NAME`

   Example:

   For brevity, the example uses `grep`. Using separate commands to get Events for specific objects is also possible. For example, `kubectl get events --field-selector involvedObject.kind="AKSCluster"` and `kubectl get events --field-selector involvedObject.kind="AKSMachine"`.

   ```
   48m         Normal    SuccessfulSetNodeRefs               machinepool/aks-
   example-md-0                  [{Kind: Namespace: Name:aks-mp6gglj-41174201-
   vmss000000 UID:e3c30389-660d-46f5-b9d7-219f80b5674d APIVersion: ResourceVersion:
    FieldPath:} {Kind: Namespace: Name:aks-mp6gglj-41174201-vmss000001 UID:300d71a0-
   f3a7-4c29-9ff1-1995ffb9cfd3 APIVersion: ResourceVersion: FieldPath:} {Kind:
    Namespace: Name:aks-mp6gglj-41174201-vmss000002 UID:8eae2b39-a415-425d-8417-
   d915a0b2fa52 APIVersion: ResourceVersion: FieldPath:} {Kind: Namespace: Name:aks-
   mp6gglj-41174201-vmss000003 UID:3e860b88-f1a4-44d1-b674-a54fad599a9d APIVersion:
    ResourceVersion: FieldPath:}]
   6m4s        Normal    AzureManagedControlPlane available   azuremanagedcontrolplane/
   aks-example          successfully reconciled
   48m         Normal    SuccessfulSetNodeRefs               machinepool/aks-
   example                       [{Kind: Namespace: Name:aks-mp6gglj-41174201-
   vmss000000 UID:e3c30389-660d-46f5-b9d7-219f80b5674d APIVersion: ResourceVersion:
    FieldPath:} {Kind: Namespace: Name:aks-mp6gglj-41174201-vmss000001 UID:300d71a0-
   f3a7-4c29-9ff1-1995ffb9cfd3 APIVersion: ResourceVersion: FieldPath:} {Kind:
    Namespace: Name:aks-mp6gglj-41174201-vmss000002 UID:8eae2b39-a415-425d-8417-
   d915a0b2fa52 APIVersion: ResourceVersion: FieldPath:}]
   ```

Known Limitations

Limitations for using NKP to create a new Azure Kubernetes Service (AKS) cluster.

The following are known limitations:

- The Nutanix Kubernetes Platform (NKP) version used to create a workload cluster must match the NKP version used to create a workload cluster.
- NKP supports deploying one workload cluster.
- NKP generates a single node pool deployed by default; adding additional node pools is supported.
- NKP does not validate edits to cluster objects.

Create a New AKS Cluster from the NKP UI

Provisioning an AKS cluster from your browser using the NKP UI.

The Nutanix Kubernetes Platform (NKP) UI allows you to provision an Azure Kubernetes Service (AKS) cluster from your browser quickly and easily.

Prerequisites

- Creating an AKS Infrastructure Provider on page 908

Creating an AKS Infrastructure Provider

To hold your AKS credentials, create an AKS infrastructure provider.

About this task

Before provisioning a cluster through the UI, create an Azure Kubernetes Service (AKS) infrastructure provider to hold your AKS credentials:

Procedure

1. Log in to the Azure command line.

   ```
   az login
   ```

2. Create an Azure Service Principal (SP) by running the following command.

   ```
   az ad sp create-for-rbac --role contributor --name "$(whoami)-konvoy" --scopes=/
   subscriptions/$(az account show --query id -o tsv)
   ```

3. Select **Infrastructure Providers** from the Dashboard menu.
4. Select **Add Infrastructure Provider**.
5. Choose a workspace. If you are already in a workspace, the provider is automatically created in that workspace.
6. Select **Microsoft Azure**.
7. Add a **Name** for your Infrastructure Provider.
8. Take the ID output from the log in command above and put it into the **Subscription ID** field.
9. Take the tenant used in Step 2 and put it into the **Tenant ID** field.
10. Take the appId used in Step 2 and put it into the **Client ID** field.
11. Take the password used in Step 2 and put it into the **Client Secret** field.
12. click **Save**.

Provisioning an AKS Cluster

Steps to provision an AKS cluster.

About this task

Follow these steps to provision an Azure Kubernetes Service (AKS) cluster:

Procedure

1. From the top menu bar, select your target workspace.
2. Select **Clusters** > **Add Clusters** > **.**
3. Choose **Create Cluster**.
4. Enter the **Cluster Name**.

5. From **Select Infrastructure Provider**, choose the provider created in the prerequisites section.
6. To create a **Kubernetes Version**, run the command below in the `az` CLI, and then select the version of AKS you want to use.

   ```
   azI aks get-versions -o table --location location
   ```

7. Select a datacenter **location** or specify a custom **location**.
8. Edit your worker **Node Pools**, as necessary. You can choose the **Number of Nodes**, the **Machine Type**, and for the worker nodes, you can choose a **Worker Availability Zone**.
9. Add any additional **Labels** or **Infrastructure Provider Tags**, as necessary.
10. Validate your inputs, then select **Create**.

    Note: Your cluster can take up to 15 minutes to appear in the Provisioned status.

    You are then redirected to the **Clusters** page, where you'll see your new cluster in the **Provisioning** status. Hover over the status to view the details.

Access an AKS Cluster
Use your NKP admin credentials to retrieve a custom `kubeconfig` file.

After successfully attaching the cluster (managed), you can retrieve a custom `kubeconfig` file from the UI using your NKP UI administrator credentials.

Explore New AKS Cluster

Learn to interact with your Azure Kubernetes Service (AKS) Kubernetes cluster.

This section explains how to use the command line to interact with your newly deployed Kubernetes cluster.

Before you start, make sure you have created a workload cluster, as described in Create a New Cluster.

Interacting with your AKS Kubernetes Cluster

Procedure

1. Get a kubeconfig file for the workload cluster.

   When the workload cluster is created, the cluster life cycle services generate a kubeconfig file for the workload cluster and write it to a `Secret`. The kubeconfig file is scoped to the cluster administrator.

   Caution: Nutanix support is unable to recover forgotten credentials. Prevent Kommander UI lockouts with the following best practices.

   - Backup the kubeconfig file.
   - Save your credentials.
   - Configure an IDP (identity provider) instead of using a username and password credentials.

   Get the kubeconfig from the `Secret`, and write it to a file using this command:

   ```
   nkp get kubeconfig -c $CLUSTER_NAME > $CLUSTER_NAME.conf
   ```

2. List the Nodes using this command.

```
kubectl --kubeconfig=$CLUSTER_NAME.conf get nodes
```

```
NAME                                STATUS   ROLES   AGE   VERSION
aks-cp6dsz8-41174201-vmss000000   Ready    agent   56m   v1.29.6
aks-cp6dsz8-41174201-vmss000001   Ready    agent   55m   v1.29.6
aks-cp6dsz8-41174201-vmss000002   Ready    agent   56m   v1.29.6
aks-mp6gglj-41174201-vmss000000   Ready    agent   55m   v1.29.6
aks-mp6gglj-41174201-vmss000001   Ready    agent   55m   v1.29.6
aks-mp6gglj-41174201-vmss000002   Ready    agent   55m   v1.29.6
aks-mp6gglj-41174201-vmss000003   Ready    agent   56m   v1.29.6
```

> **Note:** The Status may take a few minutes to move to `Ready` while the Pod network is deployed. The node status will change to Ready after the `calico-node` DaemonSet Pods are Ready.

3. List the Pods using the command `kubectl --kubeconfig=$CLUSTER_NAME.conf get --all-namespaces pods`

```
NAMESPACE                 NAME                                       READY
 STATUS     RESTARTS   AGE
calico-system             calico-kube-controllers-5dcd4b47b5-tgslm   1/1
 Running    0          3m58s
calico-system             calico-node-46dj9                          1/1
 Running    0          3m58s
calico-system             calico-node-crdgc                          1/1
 Running    0          3m58s
calico-system             calico-node-m7s7x                          1/1
 Running    0          3m58s
calico-system             calico-node-qfkqc                          1/1
 Running    0          3m57s
calico-system             calico-node-sfqfm                          1/1
 Running    0          3m57s
calico-system             calico-node-sn67x                          1/1
 Running    0          3m53s
calico-system             calico-node-w2pvt                          1/1
 Running    0          3m58s
calico-system             calico-typha-6f7f59969c-5z4t5              1/1
 Running    0          3m51s
calico-system             calico-typha-6f7f59969c-ddzqb              1/1
 Running    0          3m58s
calico-system             calico-typha-6f7f59969c-rr4lj              1/1
 Running    0          3m51s
kube-system               azure-ip-masq-agent-4f4v6                  1/1
 Running    0          4m11s
kube-system               azure-ip-masq-agent-5xfh2                  1/1
 Running    0          4m11s
kube-system               azure-ip-masq-agent-9hlk8                  1/1
 Running    0          4m8s
kube-system               azure-ip-masq-agent-9vsgg                  1/1
 Running    0          4m16s
kube-system               azure-ip-masq-agent-b9wjj                  1/1
 Running    0          3m57s
kube-system               azure-ip-masq-agent-kpjtl                  1/1
 Running    0          3m53s
kube-system               azure-ip-masq-agent-vr7hd                  1/1
 Running    0          3m57s
kube-system               cluster-autoscaler-b4789f4bf-qkfk2         0/1
 Init:0/1   0          3m28s
kube-system               coredns-845757d86-9jf8b                    1/1
 Running    0          5m29s
```

```
kube-system             coredns-845757d86-h4xfs                          1/1
 Running    0          4m
kube-system             coredns-autoscaler-5f85dc856b-xjb5z              1/1
 Running    0          5m23s
kube-system             csi-azuredisk-node-4n4fx                         3/3
 Running    0          3m53s
kube-system             csi-azuredisk-node-8pnjj                         3/3
 Running    0          3m57s
kube-system             csi-azuredisk-node-sbt6r                         3/3
 Running    0          3m57s
kube-system             csi-azuredisk-node-v25wc                         3/3
 Running    0          4m16s
kube-system             csi-azuredisk-node-vfbxg                         3/3
 Running    0          4m11s
kube-system             csi-azuredisk-node-w5ff5                         3/3
 Running    0          4m11s
kube-system             csi-azuredisk-node-zzgqx                         3/3
 Running    0          4m8s
kube-system             csi-azurefile-node-2rpcc                         3/3
 Running    0          3m57s
kube-system             csi-azurefile-node-4gqkf                         3/3
 Running    0          4m11s
kube-system             csi-azurefile-node-f6k8m                         3/3
 Running    0          4m16s
kube-system             csi-azurefile-node-k72xq                         3/3
 Running    0          4m8s
kube-system             csi-azurefile-node-vx7r4                         3/3
 Running    0          3m53s
kube-system             csi-azurefile-node-zc8kr                         3/3
 Running    0          4m11s
kube-system             csi-azurefile-node-zkl6b                         3/3
 Running    0          3m57s
kube-system             kube-proxy-4fpb6                                 1/1
 Running    0          3m53s
kube-system             kube-proxy-6qfbf                                 1/1
 Running    0          4m16s
kube-system             kube-proxy-6wnt2                                 1/1
 Running    0          4m8s
kube-system             kube-proxy-cspd5                                 1/1
 Running    0          3m57s
kube-system             kube-proxy-nsgq6                                 1/1
 Running    0          4m11s
kube-system             kube-proxy-qz2st                                 1/1
 Running    0          4m11s
kube-system             kube-proxy-zvh9k                                 1/1
 Running    0          3m57s
kube-system             metrics-server-6bc97b47f7-ltkkj                  1/1
 Running    0          5m28s
kube-system             tunnelfront-77d68f78bf-t78ck                     1/1
 Running    0          5m23s
node-feature-discovery  node-feature-discovery-master-65dc499cd-fxwb5    1/1
 Running    0          3m28s
node-feature-discovery  node-feature-discovery-worker-277xc              1/1
 Running    0          3m28s
node-feature-discovery  node-feature-discovery-worker-4dq5k              1/1
 Running    0          3m28s
node-feature-discovery  node-feature-discovery-worker-57nb8              1/1
 Running    0          3m28s
node-feature-discovery  node-feature-discovery-worker-b4lkl              1/1
 Running    0          3m28s
node-feature-discovery  node-feature-discovery-worker-kslst              1/1
 Running    0          3m28s
```

```
node-feature-discovery   node-feature-discovery-worker-ppjtm            1/1
 Running    0          3m28s
node-feature-discovery   node-feature-discovery-worker-x5bgf            1/1
 Running    0          3m28s
tigera-operator          tigera-operator-74c4d9cf84-k7css               1/1
 Running    0          5m25s
```

Delete an AKS Cluster

Delete the Azure Kubernetes Service (AKS) cluster and clean up your environment.

> **Note:** Ensure that the KUBECONFIG environment variable is set to the self-managed cluster by running `export KUBECONFIG=SELF_MANAGED_AZURE_CLUSTER.conf`.

Deleting the Workload Cluster

Steps to delete the AKS cluster and clean up your environment.

About this task

Context for the current task

Procedure

Delete the Kubernetes cluster and wait a few minutes.

Before deleting the cluster, NKP deletes all Services of type LoadBalancer on the cluster. Deleting the Service deletes the Azure LoadBalancer that backs it. To skip this step, use the flag `--delete-kubernetes-resources=false`.

> **Caution:** Do not skip this step; if # NKP manages the Azure Network when NKP deletes the cluster, it also deletes the Network.

```
nkp delete cluster --cluster-name=$CLUSTER_NAME
```

```
 # Deleting Services with type LoadBalancer for Cluster default/aks-example
 # Deleting ClusterResourceSets for Cluster default/aks-example
 # Deleting cluster resources
 # Waiting for cluster to be fully deleted
Deleted default/aks-example cluster
```

What to do next

To view your dashboard and continue your customization, complete the Kommander installation. For more information, see Kommander Installation Based on Your Environment on page 1016.

Known Limitations

Limitations for deleting an Azure Kubernetes Service (AKS) cluster.

The following limitations apply to the current NKP release.

- The NKP version used to create the workload cluster must match the NKP version used to delete the workload cluster.

vSphere Infrastructure

Configuration types for installing NKP on vSphere Infrastructure.

For an environment on the vSphere Infrastructure, install options based on those environment variables are provided for you in this location.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

vSphere Overview

vSphere is a more complex setup than other providers and infrastructures, so an overview of steps has been provided to help.

The overall process for configuring vSphere and Nutanix Kubernetes Platform (NKP) together includes the following steps:

1. Configure vSphere to provide the elements described in the vSphere Prerequisites.
2. For more information on air-gapped environments, see Creating a Bastion Host on page 685.
3. Create a base OS image (for use in the OVA package containing the disk images packaged with the OVF).
4. Create a CAPI VM image template that uses the base OS image and adds the needed Kubernetes cluster components.
5. Create a new self-managing cluster on vSphere.
6. Install Kommander.
7. Verify and log in to the UI.

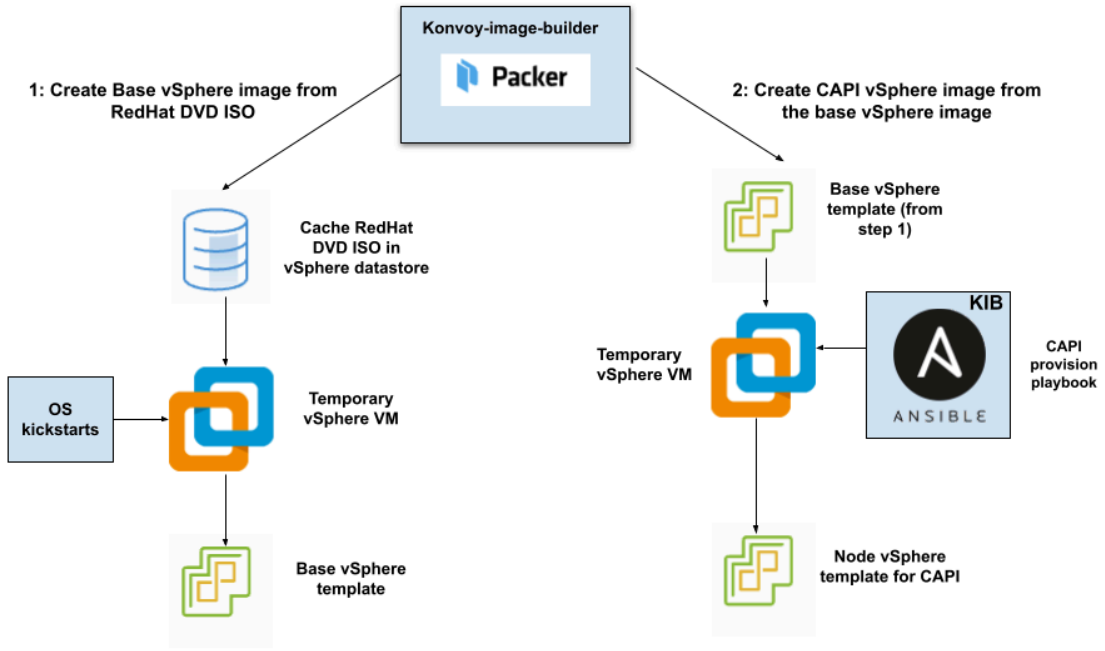


Figure 26: vSphere Image Creation Process

The workflow on the left shows the creation of a base OS image in the vCenter vSphere client using inputs from Packer. The workflow on the right shows how NKP uses that same base OS image to create CAPI-enabled VM images for your cluster.

After creating the base image, the NKP image builder uses it to create a CAPI-enabled vSphere template that includes the Kubernetes objects for the cluster. You can use that resulting template with the NKP `create cluster` command to create the VM nodes in your cluster directly on a vCenter server.

You can use ##NKP# to provision and manage your cluster from that point NKP communicates with the code in vCenter Server as the management layer for creating and managing virtual machines after ESXi v7.0.3 or later is installed and configured. For more information, see https://docs.vmware.com/en/VMware-vSphere/7.0/com.vmware.esxi.install.doc/GUID-B2F01BF5-078A-4C7E-B505-5DFFED0B8C38.html.

.

vSphere Prerequisites

This section contains all the prerequisite information specific to VMware vSphere infrastructure. These are above and beyond all of the NKP prerequisites for Install. Fulfilling the prerequisites involves completing these two areas:

1. Nutanix Kubernetes Platform (NKP) prerequisites
2. vSphere prerequisites - vCenter Server + VMware vSphere

1. NKP Prerequisites

Before using NKP to create a vSphere cluster, verify that you have:

- An x86_64-based Linux or macOS machine.
- Download NKP binaries and Konvoy Image Builder (KIB) image bundle for Linux or macOS.
- A Container engine or runtime installed is required to install NKP and bootstrap:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
- A registry needs to be installed on the host where the NKP Konvoy CLI runs. For example, if you install Konvoy on your laptop, ensure the computer has a supported version of Docker or other registry. On macOS, Docker runs in a virtual machine. Configure this virtual machine with at least 8GB of memory.
- CLI tool Kubectl 1.21.6 interacts with the running cluster, installed on the host where the NKP Konvoy command line interface (CLI) runs. For more information, see https://kubernetes.io/docs/tasks/tools/#kubectl
- A valid VMware vSphere account with credentials configured.

Note: NKP uses the vsphere CSI driver as the default storage provider. Use a Kubernetes CSI-compatible storage that is suitable for production. For more information, see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types.

Note: You can choose from any of the storage options available for Kubernetes. To disable the default that Konvoy deploys, set the default StorageClass `localvolumeprovisioner` as non-default. Then, set your newly created StorageClass as the default by following the commands in the Kubernetes documentation called Changing the Default Storage Class.

VMware vSphere Prerequisites

Before installing, verify that your VMware vSphere Client environment meets the following basic requirements:

- Access to a bastion VM or other network-connected host running vSphere Client version 7.0.3 or later.
  - You must reach the vSphere API endpoint from where the Konvoy command line interface (CLI) runs.
- vSphere account with credentials configured - this account must have Administrator privileges.
- A Red Hat subscription with a username and password for downloading DVD ISOs.
- For air-gapped environments, a bastion VM host template with access to a configured local registry. The recommended template naming pattern is `../folder-name/NKP-e2e-bastion-template` or similar. Each infrastructure provider has its own set of bastion host instructions. For more information on Creating a Bastion Host on page 685, see your provider's documentation:
  - AWS: https://aws.amazon.com/solutions/implementations/linux-bastion/
  - Azure: https://learn.microsoft.com/en-us/azure/bastion/quickstart-host-portal
  - GCP: https://blogs.vmware.com/cloud/2021/06/02/intro-google-cloud-vmware-engine-bastion-host-access-iap/
  - vSphere: https://docs.vmware.com/en/VMware-vSphere/7.0/com.vmware.vsphere.security.doc/GUID-6975426F-56D0-4FE2-8A58-580B40D2F667.html
  - VMware: https://docs.vmware.com/en/VMware-vSphere/7.0/com.vmware.vsphere.security.doc/GUID-6975426F-56D0-4FE2-8A58-580B40D2F667.html.

- Valid values for the following:
  - vCenter server URL.
  - Datacenter name.
  - Zone name that contains ESXi hosts for your cluster's nodes. For more information, see https://docs.vmware.com/en/VMware-vSphere/7.0/com.vmware.esxi.install.doc/GUID-B2F01BF5-078A-4C7E-B505-5DFFED0B8C38.html
  - Datastore name for the shared storage resource to be used for the VMs in the cluster.
    - The use of Persistent Volumes in your cluster depends on Cloud Native Storage (CNS), which is available in vSphere v7.0.3 and later versions. CNS depends on this shared datastore's configuration.
  - Datastore URL from the datastore record for the shared datastore you want your cluster to use.
    - You need this URL value to ensure the correct Datastore is used when NKP creates VMs for your cluster in vSphere.
  - Folder name.
  - Base template names, such as base-rhel-8 or base-rhel-7.
  - Name of a Virtual Network with DHCP enabled for air-gapped and non-air-gapped environments.
  - Resource Pools - at least one resource pool is needed, with every host in the pool having access to shared storage, such as VSAN.
    - Each host in the resource pool needs access to shared storage, such as NFS or VSAN, to use MachineDeployments and high-availability control planes.

Establishing vSphere Infrastructure Roles
When provisioning Kubernetes clusters with the Nutanix Kubernetes Platform (NKP) vSphere provider, four roles are needed for NKP to provide proper permissions.

About this task

Roles in vSphere are more like a policy statement for the objects in a vSphere inventory. The role is assigned to a user, and the object assignment can be inherited by any siblings through propagation if desired.

Add the permission at the highest level and set to propagate the permissions. In small vSphere environments, with just a few hosts, assigning the role or user at the top level and propagating to child resources may be appropriate. However, in most cases, this is impossible since security teams will enforce strict restrictions on who has access to specific resources.

The table below describes the level at which these permissions are assigned, followed by the steps to **Add Roles** in vCenter. These roles provide user permissions that are less than those of the admin.

Procedure

Level	Required	Propagate to Child
vCenter Server (Top Level)	No	No
Data Center	Yes	No
Resource Pool	Yes	No

Folder	Yes	Yes
Template	Yes	No

1. Open a vSphere Client connection to the vCenter Server, described in the Prerequisites.
2. Select Home > Administration > Roles > Add Role.
3. Give the new Role a name from the four choices detailed in the next section.
4. Select the Privileges from the permissions directory tree dropdown list below each of the four roles.
   - The list of permissions can be set so the provider can create, modify, or delete resources or clone templates, VMs, disks, attach network, etc.

Creating the Four vSphere Roles

The following four Roles need to be created for proper Nutanix Kubernetes Platform (NKP) access to the required Resource(s) on the correct level of vCenter and resource pools.

About this task

Set your roles to contain the permissions shown in the permissions directory tree under each role. Set the four roles - `NKP-vcenter`, `NKP-datacenter`, `NKP-k8srole`, `NKP-readonly` in the steps below.

Open vCenter:

Procedure

1. `nkp-vcenter` - This root-level permissions role applies to the Resource.
   - **1.** `vcenter root`
     1. Resource: View
     2. Cns: Searchable
     3. Profile-driven storage: Profile-driven storage view
     4. Network - Session: ValidateSession
2. `nkp-datacenter` This role allows CAPV to create resources and assign networks. It is the most extensive permission, but it is only assigned to folders, resource pools, data stores, and networks, so it can easily be separated from other environments. It applies to the Resources.
   - Do not propagate them because it gives the user view privileges on all folders and resource pools:
     1. datacenter
     2. cluster
     3. esx host 1
     4. esx host 2

Resource		
	X	View
Data Center Data Center		
	X	View
Cluster		

	X	View
ESX Host 1		
	X	View
ESX Host 2		
	X	View

3. `nkp-k8srole` - This role allows CAPV to create resources and assign networks. It is the most extensive permission, but it is only assigned to folders, resource pools, data stores, and networks, so it can easily be separated from other environments. It applies to the Resources.

   - This role can be propagated to other Resources if desired.

     1. `resource pool`
     2. `nkp folder`
     3. `nkp data store`
     4. `network`

Resource		
	X	View
Datastore		
	X	Allocate space
	X	Browse
	X	Delete File
	X	File Management
	X	Update Virtual Machine File
	X	Update Virtual Machine Data
Global		
	X	Set Custom Field
Network		
	X	Assign network
Resource		
	X	Assign vApp to Pool
	X	Assign VM to Pool
Scheduled Task		
	X	Create
	X	Delete
	X	Edit
	X	Run
	Session	

	X	ValidateSession
Storage Profile		
	X	View
Storage Views		
	X	View

4. `nkp-readonly` - This optional role allows the role to be cloned from templates in other folders and data stores but does not have write access. It applies to the Resources.

   a. `templates folder`

   b. `templates data store`

Datastore		
	X	View
Folder		
	X	View
vApp		
	X	Clone
	X	Export
Provisioning		
	X	Clone
	X	Clone template
	X	Deploy template

vSphere Minimum User Permissions

When a user needs permissions less than Admin, a role must be created with those permissions.

In small vSphere environments, with just a few hosts, assigning the role or user at the top level and propagating to child resources may be appropriate, as shown on this page in the permissions tree below.

However, this is not always possible, as security teams will enforce strict restrictions on who has access to specific resources.

The process for configuring a vSphere role with the permissions for provisioning nodes and installing includes the following steps:

1. Open a vSphere Client connection to the vCenter Server, described in the vSphere Prerequisites on page 914.
2. Select Home > Administration > Roles > Add Role.
3. Give the new role a name, then select these Privileges:

Cns	
X	Searchable
Datastore	
X	Allocate space

XLow-level file operations
Host

- Configuration

XStorage partition configuration
Profile-driven storage
XProfile-driven storage view
Network
XAssign network
Resource
Assign virtual machine to resource pool.
Virtual machine

- Change Configuration - from the list in that section, select these permissions below:

XAdd new disk
XAdd existing disk
XAdd or remove a device.
XAdvanced configuration
XChange CPU count
XChange Memory
XChange Settings
XReload from path
Edit inventory
XCreate from existing
XRemove
Interaction
XPower off
XPower on
Provisioning
XClone template
XDeploy template
Session
XValidateSession

The table below describes the level at which these permissions are assigned.

Level	Required	Propogate to Child
vCenter Server (Top Level)	No	No
Data Center	Yes	No
Resource Pool	Yes	No
Folder	Yes	Yes
Template	Yes	No

vSphere Infrastructure Storage Options

Explore storage options and considerations for using NKP with VMware vSphere

. The vSphere Container Storage plugin supports shared NFS, vNFS, and vSAN. You must provision your storage options in vCenter before creating a CAPI image in Nutanix Kubernetes Platform (NKP) for use with vSphere.

NKP has integrated the CSI 2.x driver used in vSphere. When creating your NKP cluster, NKP uses whatever configuration you provide for the Datastore name. vSAN is not required. Using NFS can reduce the tagging and permission granting needed to configure your cluster.

vSphere Base OS Image in vCenter

Creating a base OS image from DVD ISO files is a one-time process in vCenter. The base OS image file is created in the vSphere Client for use in the vSphere VM template. Therefore, Konvoy Image Builder (KIB) uses the base OS image to create a VM template to configure Kubernetes nodes by the Nutanix Kubernetes Platform (NKP) vSphere provider.

The Base OS Image

For vSphere, # SSH_USERNAME populates a username, and the user can use authorization through `SSH_PASSWORD` or `SSH_PRIVATE_KEY_FILE` environment variables and required by default by packer. This user needs administrator privileges. It is possible to configure a custom user and password when building the OS image; however, that requires the Konvoy Image Builder (KIB) configuration to be overridden.

While creating the base OS image, it is important to take into consideration the following elements:

- Storage configuration: Nutanix recommends customizing disk partitions and not configuring a SWAP partition.
- Network configuration: as KIB must download and install packages, activating the network is required.
- Connect to Red Hat: if using Red Hat Enterprise Linux (RHEL), registering with Red Hat is required to configure software repositories and install software packages.
- Software selection: Nutanix recommends choosing **Minimal Install**.
- NKP recommends installing with the packages provided by the operating system package managers. Use the version that corresponds to the major version of your operating system.

Disk Size

For each cluster you create using this base OS image, ensure you establish the disk size of the root file system based on the following:

- The minimum NKP Resource Requirements.
- The minimum storage requirements for your organization.

Disk Size

Clusters are created with a default disk size of 80 GB.

The base OS image root file system must be precisely 80 GB for clusters created with the default disk size. The root file system cannot be reduced automatically when a machine first boots.

Customization

: You can specify a custom disk size when creating a cluster (see the flags available for use with the vSphere Create Cluster command). This allows you to use one base OS image to create multiple clusters with different storage requirements.

Before specifying a disk size when you create a cluster, take into account:

- For some base OS images, the custom disk size option does not affect the size of the root file system. This is because some root file systems, for example, those contained in an LVM Logical Volume, cannot be resized automatically when a machine first boots.
- The specified custom disk size must be equal to, or larger than, the size of the base OS image root file system. This is because a root file system cannot be reduced automatically when a machine first boots.
- In VMware Cloud Director Infrastructure on page 963, the base image determines the minimum storage available for the VM.

This Base OS Image is later used toCreate during installation and cluster creation.

If using Flatcar, the documentation from Flatcar regarding disabling or enabling autologin in the Base OS Image is found here: In a vSphere or Pre-provisioned environment, anyone with access to the console of a Virtual Machine(VM) has access to the core operating system user. This is called autologin. To disable autologin, you add parameters to your base Flatcar image. For more information on using Flatcar, see

- Running Flatcar Container Linux on VMware: https://www.flatcar.org/docs/latest/installing/cloud/vmware/#disablingenabling-autologin
- Kernel modules and other settings: https://www.flatcar.org/docs/latest/setup/customization/other-settings/#adding-custom-kernel-boot-options

vSphere Installation in a Non-air-gapped Environment

This installation provides instructions on how to install Nutanix Kubernetes Platform (NKP) in a vSphere non-air-gapped environment.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Further vSphere Prerequisites

Before you begin using NKP, you must ensure you meet the other prerequisites in the vSphere Prerequisites section.

In an environment with access to the Internet, you retrieve artifacts from specialized repositories dedicated to them, such as Docker images contained in DockerHub and Helm Charts that come from a dedicated Helm Chart repository. However, in an air-gapped environment, you need local repositories to store Helm charts, Docker images, and other artifacts. Tools such as JFrog, Harbor, and Nexus handle multiple types of artifacts in one local repository.

> **Tip:** A local registry can also be used in a non-air-gapped environment for speed and security if desired. To do so, add the following steps to your non-air-gapped installation process. See the topic Registry Mirror Tools.

Creating a CAPI VM Template for vSphere

The Konvoy Image Builder (KIB) uses the values in `image.yaml` and the input base OS image to create a vSphere template directly on the vCenter server.

About this task

You must have at least one image before creating a new cluster. As long as you have an image, this step in your configuration is not required each time since that image can be used to spin up a new cluster. However, if you need different images for different environments or providers, you must create a new custom image.

Procedure

1. Users must perform the steps in the topic vSphere: Creating an Image before starting this procedure.
2. Build an image template with Konvoy Image Builder (KIB).

Create a vSphere Template for Your Cluster from a Base OS Image

Procedure

1. Set the following vSphere environment variables on the bastion VM host.

```
export VSPHERE_SERVER=your_vCenter_APIserver_URL
export VSPHERE_USERNAME=your_vCenter_user_name
export VSPHERE_PASSWORD=your_vCenter_password
```

2. Copy the base OS image file created in the vSphere Client to your desired location on the bastion VM host and make a note of the path and file name.
3. Create an `image.yaml` file and add the following variables for vSphere. Nutanix Kubernetes Platform (NKP) uses this file and these variables as inputs in the next step. To customize your `image.yaml` file, refer to this section: Customize your Image.

> **Note:** This example is Ubuntu 20.04. You will need to replace the OS name below based on your OS. Also, refer to the example YAML files located here: OVA YAML: https://github.com/mesosphere/konvoy-image-builder/tree/main/images/ova.

```
---
download_images: true
build_name: "ubuntu-2004"
packer_builder_type: "vsphere"
guestinfo_datasource_slug: "https://raw.githubusercontent.com/vmware/cloud-init-
vmware-guestinfo"
guestinfo_datasource_ref: "v1.4.0"
guestinfo_datasource_script: "{{guestinfo_datasource_slug}}/
{{guestinfo_datasource_ref}}/install.sh"
packer:
  cluster: "<VSPHERE_CLUSTER_NAME>"
  datacenter: "<VSPHERE_DATACENTER_NAME>"
  datastore: "<VSPHERE_DATASTORE_NAME>"
  folder: "<VSPHERE_FOLDER>"
  insecure_connection: "false"
  network: "<VSPHERE_NETWORK>"
```

```
  resource_pool: "<VSPHERE_RESOURCE_POOL>"
  template: "os-qualification-templates/nutanix-base-Ubuntu-20.04" # change default
 value with your base template name
  vsphere_guest_os_type: "other4xLinux64Guest"
  guest_os_type: "ubuntu2004-64"
  # goss params
  distribution: "ubuntu"
  distribution_version: "20.04"
# Use following overrides to select the authentication method that can be used with
 base template
# ssh_username: ""  # can be exported as environment variable 'SSH_USERNAME'
# ssh_password: "" # can be exported as environment variable 'SSH_PASSWORD'
# ssh_private_key_file = "" # can be exported as environment variable
 'SSH_PRIVATE_KEY_FILE'
# ssh_agent_auth: false  # is set to true, ssh_password and ssh_private_key will be
 ignored
```

4. Create a vSphere VM template with your variation of the following command.

   ```
   konvoy-image build images/ova/<image.yaml>
   ```

   - Any additional configurations can be added to this command using `--overrides` flags as shown below:

     1. Any credential overrides: `--overrides overrides.yaml`
     2. for FIPS, add this flag: `--overrides overrides/fips.yaml`
     3. for air-gapped, add this flag: `--overrides overrides/offline-fips.yaml`

5. The Konvoy Image Builder (KIB) uses the values in `image.yaml` and the input base OS image: https://github.com/mesosphere/konvoy-image-builder/tree/main/images/ova to create a vSphere template directly on the vCenter server. This template contains the required artifacts needed to create a Kubernetes cluster. When KIB successfully provisions the OS image, it creates a manifest file. The `artifact_id` field of this file contains the name of the AMI ID (AWS), template name (vSphere), or image name (GCP/Azure), for example.

   ```
   {
         "name": "vsphere-clone",
         "builder_type": "vsphere-clone",
         "build_time": 1644985039,
         "files": null,
         "artifact_id": "konvoy-ova-vsphere-rhel-86-1.21.6-1644983717",
         "packer_run_uuid": "260e8110-77f8-ca94-e29e-ac7a2ae779c8",
         "custom_data": {
           "build_date": "2022-02-16T03:55:17Z",
           "build_name": "vsphere-rhel-86",
           "build_timestamp": "1644983717",
           [...]
         }
       }
   ```

   Tip: Recommendation: Now that you can now see the template created in your vCenter, it is best to rename it to `nkp-<NKP_VERSION>-k8s-<K8S_VERSION>-<DISTRO>`, like `nkp-2.4.0-k8s-1.24.6-ubuntu` to keep templates organized.

6. The following steps are to deploy an NKP cluster using your vSphere template.

Bootstrapping vSphere

To create Kubernetes clusters, NKP uses Cluster API (CAPI) controllers. These controllers run on a Kubernetes cluster.

About this task

To get started, you need a bootstrap cluster. By default, Nutanix Kubernetes Platform (NKP) creates a bootstrap cluster for you in a Docker container using the Kubernetes-in-Docker (KIND) tool.

Procedure

1. Complete the Nutanix Infrastructure Prerequisites. For more information, see Nutanix Infrastructure Prerequisites on page 690.
2. Ensure the `NKP` binary can be found in your `$PATH`.

Bootstrap Cluster Life Cycle Services

Procedure

1. Review Universal Configurations for all Infrastructure Providers regarding settings, flags and other choices and then begin bootstrapping.
2. Create a bootstrap cluster using the command `nkp create bootstrap --kubeconfig $HOME/.kube/config`.

   > **Note:** Use `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Configuring an HTTP or HTTPS Proxy on page 677.

   Example output:

   ```
   # Creating a bootstrap cluster
   # Initializing new CAPI components
   ```

3. NKP creates a bootstrap cluster using KIND as a library.

   For more information, see https://github.com/kubernetes-sigs/kind.

4. NKP then deploys the following Cluster API providers on the cluster.

   - Core Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/
   - AWS Infrastructure Provider: https://github.com/kubernetes-sigs/cluster-api-provider-aws
   - Kubeadm Bootstrap Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/bootstrap/kubeadm
   - Kubeadm ControlPlane Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/controlplane/kubeadm

   For more information on Cluster APIs, see https://cluster-api.sigs.k8s.io/.

5. Ensure that the CAPV controllers are present using the command `kubectl get pods -n capv-system`.

   Output example:

   ```
   NAME                                       READY   STATUS    RESTARTS   AGE
   capv-controller-manager-785c5978f-nnfns    1/1     Running   0          13h
   ```

6. NKP waits until the controller-manager and webhook deployments of these providers are ready. List these deployments using the command `kubectl get --all-namespaces deployments -l=clusterctl.cluster.x-k8s.io`.

   Output example:

```
NAMESPACE                           NAME
 READY   UP-TO-DATE   AVAILABLE   AGE
capa-system                         capa-controller-manager
 1/1     1            1           1h
capg-system                         capg-controller-manager
 1/1     1            1           1h
capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-manager
 1/1     1            1           1h
capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-manager
 1/1     1            1           1h
capi-system                         capi-controller-manager
 1/1     1            1           1h
cappp-system                        cappp-controller-manager
 1/1     1            1           1h
capv-system                         capv-controller-manager
 1/1     1            1           1h
capz-system                         capz-controller-manager
 1/1     1            1           1h
cert-manager                        cert-manager
 1/1     1            1           1h
cert-manager                        cert-manager-cainjector
 1/1     1            1           1h
cert-manager                        cert-manager-webhook
 1/1     1            1           1h
```

Creating a New vSphere Cluster

Create a vSphere Management Cluster in a non-air-gapped environment.

About this task

Use this procedure to create a new Kubernetes cluster with Nutanix Kubernetes Platform (NKP).

If you use these instructions to create a cluster on vSphere using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes. First, you must name your cluster.

Before you begin

Name Your Cluster

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. For more naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable using the command `export CLUSTER_NAME=<my-vsphere-cluster>`.

3. To set the environment variables for vSphere use the command `export VSPHERE_SERVER=example.vsphere.url export VSPHERE_USERNAME=user@example.vsphere.url export VSPHERE_PASSWORD=example_password`

   Note: NKP uses the vSphere CSI driver as the default storage provider. Use a Kubernetes CSI compatible storage that is suitable for production. For more information, see the Kubernetes documentation called ##Changing the Default Storage Class# If you're not using the default, you cannot deploy an alternate provider until after the `nkp create cluster` is finished. However, this must be determined before the Kommander installation.

4. Ensure your vSphere credentials are up-to-date by refreshing the credentials with the command.

   ```
   nkp update bootstrap credentials vsphere
   ```

5. Ensure your subnets do not overlap with your host subnet because they cannot be changed after cluster creation. If you need to change the kubernetes subnets, you must do this at cluster creation. The default subnets used in NKP are.

   ```
   spec:
     clusterNetwork:
       pods:
         cidrBlocks:
         - 192.168.0.0/16
       services:
         cidrBlocks:
         - 10.96.0.0/12
   ```

6. Generate the Kubernetes cluster objects by copying and editing this command to include the correct values, including the VM template name you assigned in the previous procedure.

   ```
   nkp create cluster vsphere \
     --cluster-name ${CLUSTER_NAME} \
     --network <NETWORK_NAME> \
     --control-plane-endpoint-host <xxx.yyy.zzz.000> \
     --data-center <DATACENTER_NAME> \
     --data-store <DATASTORE_NAME> \
     --folder <FOLDER_NAME> \
     --server <VCENTER_API_SERVER_URL> \
     --ssh-public-key-file <SSH_PUBLIC_KEY_FILE> \
     --resource-pool <RESOURE_POOL_NAME> \
     --virtual-ip-interface <ip_interface_name> \
     --vm-template <TEMPLATE_NAME>
     --dry-run \
     --output=yaml \
     > ${CLUSTER_NAME}.yaml
   ```

   Note: To increase Dockerhub's rate limit, use your Dockerhub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster command`. For more information, see https://docs.docker.com/docker-hub/download-rate-limit/

   - Flatcar OS flag: Flatcar OS use `--os-hint flatcar` to instruct the bootstrap cluster to make some changes related to the installation paths:
   - HTTP: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.

» FIPS flags -

To create a cluster in FIPS mode, inform the controllers of the appropriate image repository and version tags of the official Nutanix FIPS builds of Kubernetes by adding those flags to `nkp create cluster` command:

» You can create individual manifest files with smaller manifests to ease editing using the `--output-directory` flag.

7. Inspect or edit the cluster objects. Familiarize yourself with Cluster API before editing the cluster objects, as edits can prevent the cluster from deploying successfully.

8. Create the cluster from the objects generated from the `dry run`. A warning will appear in the console if the resource already exists, requiring you to remove the resource or update your YAML.

```
kubectl create -f ${CLUSTER_NAME}.yaml
```

Note: If you used the `--output-directory` flag in your `NKP create .. --dry-run` step above, create the cluster from the objects you created by specifying the directory:

```
kubectl create -f <existing-directory>/.
```

9. Wait for the cluster control plan to be ready.

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=20m
```

10. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. Konvoy provides a command to describe the current status of the cluster.

```
nkp describe cluster -c ${CLUSTER_NAME}
```

Output:

```
NAME                                                                READY  SEVERITY
  REASON  SINCE  MESSAGE
Cluster/nutanix-e2e-cluster_name-1                                     True
             13h
##ClusterInfrastructure - VSphereCluster/nutanix-e2e-cluster_name-1    True
             13h
##ControlPlane - KubeadmControlPlane/nutanix-control-plane             True
             13h
# ##Machine/nutanix--control-plane-7llgd                               True
             13h
# ##Machine/nutanix--control-plane-vncbl                               True
             13h
# ##Machine/nutanix--control-plane-wbgrm                               True
             13h
##Workers
    ##MachineDeployment/nutanix--md-0                                 True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-67rv4                          True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-n2skc                          True
             13h
    ##Machine/nutanix--md-0-74c849dc8c-nkftv                          True
             13h
```

```
##Machine/nutanix--md-0-74c849dc8c-sqklv                                  True
         13h
```

Note: NKP uses the vSphere CSI driver as the default storage provider. Use a Kubernetes CSI compatible storage that is suitable for production. For more information, see the Kubernetes documentation called ##Changing the Default Storage Class# If you're not using the default, you cannot deploy an alternate provider until after the `nkp create cluster` is finished. However, this must be determined before the Kommander installation. For more information, see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types and https://kubernetes.io/docs/tasks/administer-cluster/change-default-storage-class/

Note: To increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username=<username> --registry-mirror-password=<password>` on the `nkp create cluster` command. For more information, see https://docs.docker.com/docker-hub/download-rate-limit/.

11. Check all machines has NODE_NAME assigned.

    ```
    kubectl get machines
    ```

12. Verify that the kubeadm control plane is ready with the command.

    ```
    kubectl get kubeadmcontrolplane
    ```

    Output is similar to:

    ```
    NAME                                CLUSTER              INITIALIZED   API SERVER
     AVAILABLE   REPLICAS   READY   UPDATED   UNAVAILABLE   AGE   VERSION
    nutanix-e2e-cluster-1-control-plane   nutanix-e2e-cluster-1   true          true
                    3          3       3         0             14h   v1.29.6
    ```

13. Describe the kubeadm control plane and check its status and events with the command.

    ```
    kubectl describe kubeadmcontrolplane
    ```

14. As they progress, the controllers also create Events, which you can list using the command

    ```
    kubectl get events | grep ${CLUSTER_NAME}
    ```

    For brevity, this example uses `grep`. You can also use separate commands to get Events for specific objects, such as `kubectl get events --field-selector involvedObject.kind="VSphereCluster"` and `kubectl get events --field-selector involvedObject.kind="VSphereMachine"`.

Making the New vSphere Cluster Self-Managed
How to make a Kubernetes cluster manage itself.

About this task

Nutanix Kubernetes Platform (NKP) deploys all cluster life cycle services to a bootstrap cluster, which then deploys a workload cluster. When the workload cluster is ready, move the cluster life cycle services to the workload cluster, which makes the workload cluster self-managed.

Before you begin

Ensure you can create a workload cluster as described in the topic: Creating a New vSphere Cluster on page 926.

This page contains instructions on how to make your cluster self-managed. This is necessary if there is only one cluster in your environment or if this cluster should become the Management cluster in a multi-cluster environment.

Note: If you already have a self-managed or Management cluster in your environment, skip this page.

Follow these steps to turn your new cluster into a **Management Cluster** for an Ultimate license environment (or a free-standing Pro Cluster):

Procedure

1. Deploy cluster life cycle services on the workload cluster.

   ```
   nkp create capi-components --kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Initializing new CAPI components
   ```

   Note: If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP/HTTPS Proxy.

2. The cluster life cycle services on the workload cluster are ready, but the workload cluster configuration is on the bootstrap cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the bootstrap to the workload cluster. This process is called a Pivot. For more information, see https://cluster-api.sigs.k8s.io/reference/glossary.html?highlight=pivot#pivot.

   Move the Cluster API objects from the bootstrap to the workload cluster:

   ```
   nkp move capi-resources --to-kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Moving cluster resources
   You can now view resources in the moved cluster by using the --kubeconfig flag with
    kubectl. For example: kubectl --kubeconfig=gcp-example.conf get nodes
   ```

   Note: To ensure only one set of cluster life cycle services manages the workload cluster, NKP first pauses reconciliation of the objects on the bootstrap cluster, then creates the objects on the workload cluster. As NKP copies the objects, the cluster life cycle services on the workload cluster reconcile the objects. The workload cluster becomes self-managed after NKP creates all the objects. If it fails, the `move` command can be safely retried.

3. Wait for the cluster control-plane to be ready.

   ```
   kubectl --kubeconfig ${CLUSTER_NAME}.conf wait --for=condition=ControlPlaneReady
    "clusters/${CLUSTER_NAME}" --timeout=20m
   ```

   Output:

   ```
   cluster.cluster.x-k8s.io/gcp-example condition met
   ```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster status. After moving the cluster life cycle services to the workload cluster, remember to use NKP with the workload cluster `kubeconfig`.

   ```
   nkp describe cluster --kubeconfig ${CLUSTER_NAME}.conf -c ${CLUSTER_NAME}
   ```

   Output:

   ```
   NAME                                                                READY  SEVERITY
    REASON  SINCE  MESSAGE
   Cluster/vsphere-example-1                                           True
           13h
   ##ClusterInfrastructure - VSphereCluster/vsphere-example-1          True
           13h
   ```

```
##ControlPlane - KubeadmControlPlane/vsphere-example-control-plane  True
        13h
# ##Machine/vsphere-example-control-plane-7llgd                     True
        13h
# ##Machine/vsphere-example-control-plane-vncbl                     True
        13h
# ##Machine/vsphere-example-control-plane-wbgrm                     True
        13h
##Workers
    ##MachineDeployment/vsphere-example-md-0                       True
        13h
    ##Machine/vsphere-example-md-0-74c849dc8c-67rv4                True
        13h
    ##Machine/vsphere-example-md-0-74c849dc8c-n2skc                True
        13h
    ##Machine/vsphere-example-md-0-74c849dc8c-nkftv                True
        13h
    ##Machine/vsphere-example-md-0-74c849dc8c-sqklv                True
        13h
```

5. Remove the bootstrap cluster because the workload cluster is now self-managed.

```
nkp delete bootstrap --kubeconfig $HOME/.kube/config
# Deleting bootstrap cluster
```

Known Limitations

Procedure

- NKP only supports moving all namespaces in the cluster; NKP does not support migration of individual namespaces.
- Konvoy supports moving only one set of cluster objects from the bootstrap cluster to the workload cluster, or vice-versa.

Exploring the vSphere Cluster

This guide explains how to use the command line to interact with your newly deployed Kubernetes cluster.

About this task

Before you start, make sure you have created a workload cluster, as described in Create a New vSphere Cluster.

Procedure

1. When the workload cluster is created, the cluster life cycle services generate a `kubeconfig` file for the workload cluster, and write it to a Secret. The `kubeconfig` file is scoped to the cluster administrator. Get a `kubeconfig` file for the workload cluster.

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

2. Create a StorageClass with a vSphere datastore.

   a. Access the Datastore tab in the vSphere client and select a datastore by name.

   b. Copy the URL of that datastore from the information dialog that displays.

   c. Return to the Nutanix Kubernetes Platform (NKP) CLI, and delete the existing `StorageClass` with the command: `kubectl delete storageclass vsphere-raw-block-sc`

   d. Run the following command to create a new StorageClass, supplying the correct values for your environment.

```
cat <<EOF > vsphere-raw-block-sc.yaml
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
  name: vsphere-raw-block-sc
provisioner: csi.vsphere.vmware.com
parameters:
  datastoreurl: "<url>"
volumeBindingMode: WaitForFirstConsumer
EOF
```

3. Verify the API server is up by listing the nodes.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get nodes
```

   Note: It may take a few minutes for the Status to move to Ready while the Pod network is deployed. The Nodes' Status will change to Ready soon after the calico-node DaemonSet Pods are Ready.

   Output:

```
NAME                                STATUS   ROLES                  AGE     VERSION
aws-example-control-plane-9z77w   Ready    control-plane,master   4m44s   v1.27.6
aws-example-control-plane-rtj9h   Ready    control-plane,master   104s    v1.27.6
aws-example-control-plane-zbf9w   Ready    control-plane,master   3m23s   v1.27.6
aws-example-md-0-88c46            Ready    <none>                 3m28s   v1.27.6
aws-example-md-0-fp8s7            Ready    <none>                 3m28s   v1.27.6
aws-example-md-0-qvnx7            Ready    <none>                 3m28s   v1.27.6
aws-example-md-0-wjdrg            Ready    <none>                 3m27s   v1.27.6
```

4. List the Pods with the command.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get pods -A
```

   Verify the output:

```
NAMESPACE                NAME
         READY   STATUS     RESTARTS      AGE
calico-system            calico-kube-controllers-57fbd7bd59-qqd96
         1/1     Running    0             20h
calico-system            calico-node-2m524
         1/1     Running    3 (19h ago)   19h
calico-system            calico-node-bbhg5
         1/1     Running    0             20h
calico-system            calico-node-cc5lf
         1/1     Running    2 (19h ago)   19h
calico-system            calico-node-cwg7x
         1/1     Running    1 (19h ago)   19h
calico-system            calico-node-d59hn
         1/1     Running    1 (19h ago)   19h
```

```
calico-system            calico-node-qmmcz
         1/1     Running    0             19h
calico-system            calico-node-wdqhx
         1/1     Running    0             19h
calico-system            calico-typha-655489d8cc-b5jnt
         1/1     Running    0             20h
calico-system            calico-typha-655489d8cc-q92x9
         1/1     Running    0             19h
calico-system            calico-typha-655489d8cc-vjlkx
         1/1     Running    0             19h
kube-system              cluster-autoscaler-68c759fbf6-7d2ck
         0/1     Init:0/1   0             20h
kube-system              coredns-78fcd69978-qn4qt
         1/1     Running    0             20h
kube-system              coredns-78fcd69978-wqpmg
         1/1     Running    0             20h
kube-system              etcd-nutanix-e2e-air-gapped-1-control-plane-7llgd
           1/1     Running    0             20h
kube-system              etcd-nutanix-e2e-air-gapped-1-control-plane-vncbl
           1/1     Running    0             19h
kube-system              etcd-nutanix-e2e-air-gapped-1-control-plane-wbgrm
           1/1     Running    0             19h
kube-system              kube-apiserver-nutanix-e2e-air-gapped-1-control-plane-7llgd
           1/1     Running    0             20h
kube-system              kube-apiserver-nutanix-e2e-air-gapped-1-control-plane-vncbl
           1/1     Running    0             19h
kube-system              kube-apiserver-nutanix-e2e-air-gapped-1-control-plane-wbgrm
           1/1     Running    0             19h
kube-system              kube-controller-manager-nutanix-e2e-air-gapped-1-control-
plane-7llgd   1/1     Running    1 (19h ago)   20h
kube-system              kube-controller-manager-nutanix-e2e-air-gapped-1-control-
plane-vncbl   1/1     Running    0             19h
kube-system              kube-controller-manager-nutanix-e2e-air-gapped-1-control-
plane-wbgrm   1/1     Running    0             19h
kube-system              kube-proxy-cpscs
         1/1     Running    0             19h
kube-system              kube-proxy-hhmxq
         1/1     Running    0             19h
kube-system              kube-proxy-hxhnk
         1/1     Running    0             19h
kube-system              kube-proxy-nsrbp
         1/1     Running    0             19h
kube-system              kube-proxy-scxfg
         1/1     Running    0             20h
kube-system              kube-proxy-tth4k
         1/1     Running    0             19h
kube-system              kube-proxy-x2xfx
         1/1     Running    0             19h
kube-system              kube-scheduler-nutanix-e2e-air-gapped-1-control-plane-7llgd
           1/1     Running    1 (19h ago)   20h
kube-system              kube-scheduler-nutanix-e2e-air-gapped-1-control-plane-vncbl
           1/1     Running    0             19h
kube-system              kube-scheduler-nutanix-e2e-air-gapped-1-control-plane-wbgrm
           1/1     Running    0             19h
kube-system              kube-vip-nutanix-e2e-air-gapped-1-control-plane-7llgd
           1/1     Running    1 (19h ago)   20h
kube-system              kube-vip-nutanix-e2e-air-gapped-1-control-plane-vncbl
           1/1     Running    0             19h
kube-system              kube-vip-nutanix-e2e-air-gapped-1-control-plane-wbgrm
           1/1     Running    0             19h
kube-system              vsphere-cloud-controller-manager-4zj7q
         1/1     Running    0             19h
```

```
kube-system              vsphere-cloud-controller-manager-87tgm
         1/1     Running    0             19h
kube-system              vsphere-cloud-controller-manager-xqmn4
         1/1     Running    1 (19h ago)   20h
node-feature-discovery   node-feature-discovery-master-84c67dcbb6-txfw9
         1/1     Running    0             20h
node-feature-discovery   node-feature-discovery-worker-8tg2l
         1/1     Running    3 (19h ago)   19h
node-feature-discovery   node-feature-discovery-worker-c5f6q
         1/1     Running    0             19h
node-feature-discovery   node-feature-discovery-worker-fjfkm
         1/1     Running    0             19h
node-feature-discovery   node-feature-discovery-worker-x6tz8
         1/1     Running    0             19h
tigera-operator          tigera-operator-d499f5c8f-r2srj
         1/1     Running    1 (19h ago)   20h
vmware-system-csi        vsphere-csi-controller-7ffd6884cc-d7rql
         7/7     Running    5 (19h ago)   20h
vmware-system-csi        vsphere-csi-controller-7ffd6884cc-k82cm
         7/7     Running    2 (19h ago)   20h
vmware-system-csi        vsphere-csi-controller-7ffd6884cc-qttkp
         7/7     Running    1 (19h ago)   20h
vmware-system-csi        vsphere-csi-node-678hw
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-6tbsh
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-9htwr
         3/3     Running    5 (20h ago)   20h
vmware-system-csi        vsphere-csi-node-g8r6l
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-ghmr6
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-jhvgm
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-rp77r
         3/3     Running    0             19h
```

Installing Kommander in a vSphere Environment

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped vSphere environment.

About this task

Once you have installed the Konvoy component of Nutanix Kubernetes Platform (NKP) , you will continue with the installation of the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, 1h) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a Default StorageClass on page 1038.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

```
export CLUSTER_NAME=<your-management-cluster-name>
```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

```
nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
```

3. Create a configuration file for the deployment.

```
nkp install kommander --init > kommander.yaml
```

4. If required: Customize your `kommander.yaml`.

   a. See Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy and External Load Balancer.

5. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `dkp-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.nutanix.io/project-default-catalog-repository: "true"
        kommander.nutanix.io/workspace-default-catalog-repository: "true"
        kommander.nutanix.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

6. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Enable NKP Catalog Applications after Installing NKP.

Verifying the vSphere Install and Log in to the UI
Verify Kommander Install and Log in to the Dashboard UI

About this task

After you build the Konvoy cluster and you install the Kommander component for the UI, you can verify your installation. It waits for all applications to be ready by default.

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      nkp experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of Nutanix Kubernetes Platform (NKP) . The Cluster Operations section allows you to manage cluster operations and their application workloads to optimize your organization’s productivity.

- Continue to the NKP Dashboard.

vSphere Installation in an Air-Gapped Environment

This installation provides instructions to install NKP in a vSphere air-gapped environment.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Note:

- For air-gapped, ensure that you download the bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` and extract the tar file to a local directory. For more information, see Downloading NKP on page 16.

  ```
  tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
  ```

- Some commands, such as `nkp push bundle`, require temporary disk space. These commands write to the temporary directory, which is usually /tmp. To override the directory, export the TMPDIR environment variable before running a command.

  For example:

  ```
  export TMPDIR=/path/to/your/directory
  ```

Further vSphere Prerequisites

Before you begin using Nutanix Kubernetes Platform (NKP) , you must ensure you already meet the other prerequisites in the vSphere Prerequisites section.

Creating an Air-gapped CAPI VM Template for vSphere

The Konvoy Image Builder (KIB) uses the values in `image.yaml` and the input base OS image to create a vSphere template directly on the vCenter server.

About this task

You must have at least one image before creating a new cluster. As long as you have an image, this step in your configuration is not required each time since that image can be used to spin up a new cluster. However, if you need different images for different environments or providers, you will need to create a new custom image.

Note: Users need to perform the steps in the topic vSphere: Creating an Image before starting this procedure.

Procedure

1. Assuming you have downloaded `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`, extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.

3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the new Nutanix Kubernetes Platform (NKP) command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`. Example command.

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   » For FIPS, pass the flag: `--fips`

   » For Red Hat Enterprise Linux (RHEL) OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`. Example command:

   ```
   export RHSM_ACTIVATION_KEY="-ci"
   export RHSM_ORG_ID="1232131"
   ```

4. Build image template with Konvoy Image Builder (KIB).

5. Follow the instructions to build a vSphere template below and if applicable, set the override `--overrides overrides/offline.yaml` flag described in Step 4 below.

Create a vSphere Template for Your Cluster from a Base OS Image

Procedure

1. Set the following vSphere environment variables on the bastion VM host.

```
export VSPHERE_SERVER=your_vCenter_APIserver_URL
export VSPHERE_USERNAME=your_vCenter_user_name
export VSPHERE_PASSWORD=your_vCenter_password
```

2. Copy the base OS image file created in the vSphere Client to your desired location on the bastion VM host and make a note of the path and file name.

3. Create an `image.yaml` file and add the following variables for vSphere. NKP uses this file and these variables as inputs in the next step. To customize your `image.yaml` file, refer to this section: Customize your Image.

> **Note:** This example is Ubuntu 20.04. You will need to replace OS name below based on your OS. Also refer to example YAML files located here: OVA YAML: https://github.com/mesosphere/konvoy-image-builder/tree/main/images/ova

```
---
download_images: true
build_name: "ubuntu-2004"
packer_builder_type: "vsphere"
guestinfo_datasource_slug: "https://raw.githubusercontent.com/vmware/cloud-init-
vmware-guestinfo"
guestinfo_datasource_ref: "v1.4.0"
guestinfo_datasource_script: "{{guestinfo_datasource_slug}}/
{{guestinfo_datasource_ref}}/install.sh"
packer:
  cluster: "<VSPHERE_CLUSTER_NAME>"
  datacenter: "<VSPHERE_DATACENTER_NAME>"
  datastore: "<VSPHERE_DATASTORE_NAME>"
  folder: "<VSPHERE_FOLDER>"
  insecure_connection: "false"
  network: "<VSPHERE_NETWORK>"
  resource_pool: "<VSPHERE_RESOURCE_POOL>"
  template: "os-qualification-templates/d2iq-base-Ubuntu-20.04" # change default
 value with your base template name
  vsphere_guest_os_type: "other4xLinux64Guest"
  guest_os_type: "ubuntu2004-64"
  # goss params
  distribution: "ubuntu"
  distribution_version: "20.04"
# Use following overrides to select the authentication method that can be used with
 base template
# ssh_username: ""  # can be exported as environment variable 'SSH_USERNAME'
# ssh_password: "" # can be exported as environment variable 'SSH_PASSWORD'
# ssh_private_key_file = "" # can be exported as environment variable
 'SSH_PRIVATE_KEY_FILE'
# ssh_agent_auth: false  # is set to true, ssh_password and ssh_private_key will be
 ignored
```

4. Create a vSphere VM template with your variation of the following command.

```
konvoy-image build images/ova/<image.yaml>
```

   - Any additional configurations can be added to this command using `--overrides` flags as shown below:

     1. Any credential overrides: `--overrides overrides.yaml`
     2. for FIPS, add this flag: `--overrides overrides/fips.yaml`
     3. for air-gapped, add this flag: `--overrides overrides/offline-fips.yaml`

5. The Konvoy Image Builder (KIB) uses the values in `image.yaml` and the input base OS image to create a vSphere template directly on the vCenter server. This template contains the required artifacts needed to create a Kubernetes cluster. When KIB provisions the OS image successfully, it creates a manifest file. The `artifact_id` field of this file contains the name of the AMI ID (AWS), template name (vSphere), or image name (GCP/Azure), for example. For more information, see https://github.com/mesosphere/konvoy-image-builder/tree/main/images/ova.

```
{
      "name": "vsphere-clone",
      "builder_type": "vsphere-clone",
      "build_time": 1644985039,
      "files": null,
      "artifact_id": "konvoy-ova-vsphere-rhel-86-1.21.6-1644983717",
      "packer_run_uuid": "260e8110-77f8-ca94-e29e-ac7a2ae779c8",
      "custom_data": {
        "build_date": "2022-02-16T03:55:17Z",
        "build_name": "vsphere-rhel-86",
        "build_timestamp": "1644983717",
        [...]
      }
    }
```

   Tip: Recommendation: Now that you can now see the template created in your vCenter, it is best to rename it to `nkp-<NKP_VERSION>-k8s-<K8S_VERSION>-<DISTRO>`, like `nkp-2.4.0-k8s-1.24.6-ubuntu` to keep templates organized.

6. Next steps are to deploy a NKP cluster using your vSphere template.

Loading the Registry in a vSphere Air-gapped Environment

Before creating an air-gapped Kubernetes cluster, you need to load the required images in a local registry for the Konvoy component.

About this task

The complete Nutanix Kubernetes Platform (NKP) air-gapped bundle is needed for an air-gapped environment but can also be used in a non-air-gapped environment. The bundle contains all the NKP components needed for an air-gapped environment installation and also to use a local registry in a non-air-gapped environment.

Note: If you do not already have a local registry set up, see the Local Registry Tools page for more information.

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images is required. This registry must be accessible from both the bastion machine or other machines that will be created for the Kubernetes cluster.

Procedure

1. If not already done in prerequisites, download the air-gapped bundle `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
   ```

2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. EX: For the bootstrap cluster, change your directory to the `nkp-<version>` directory similar to example below depending on your current location

   ```
   cd nkp-v2.12.0
   ```

3. Set an environment variable with your registry address for ECR.

   ```
   export REGISTRY_URL=<ecr-registry-URI>
   ```

   - `REGISTRY_URL`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images
   - The environment where you are running the `nkp push` command must be authenticated with AWS in order to load your images into ECR.
   - Other registry variables:

     ```
     export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
     export REGISTRY_USERNAME=<username>
     export REGISTRY_PASSWORD=<password>
     export REGISTRY_CA=<path to the cacert file on the bastion>
     ```

   Before creating or upgrading a Kubernetes cluster, you need to load the required images in a local registry if operating in an air-gapped environment.

4. Execute the following command to load the air-gapped image bundle into your private registry using any of the relevant flags to apply variables above.

   If not ECR as shown in example code below, use the other relevant flags: `--to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}`

   ```
   nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
   ```

   Note: It may take some time to push all the images to your image registry, depending on the performance of the network between the machine you are running the script on and the registry.

   Important: To increase Docker Hub's rate limit use your Docker Hub credentials when creating the cluster, by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command.

5. Load the Kommander component images to your private registry using the command.

```
nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

Optional: This step is required only if you have an Ultimate license.

For NKP Catalog Applications available with the Ultimate license, perform this image load by running the following command to load the `dkp-catalog-applications` image bundle into your private registry:

```
nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}
```

Bootstrapping Air-gapped vSphere

To create Kubernetes clusters, NKP uses Cluster API (CAPI) controllers. These controllers run on a Kubernetes cluster.

About this task

To get started, you need a bootstrap cluster. By default, Nutanix Kubernetes Platform (NKP) creates a bootstrap cluster for you in a Docker container using the Kubernetes-in-Docker (KIND) tool.

Before you begin

Procedure

1. Complete the Nutanix Infrastructure Prerequisites. For more information, see Nutanix Infrastructure Prerequisites on page 690.
2. Ensure the `NKP` binary can be found in your `$PATH`.

Bootstrap Cluster Life Cycle Services

Procedure

1. Review Universal Configurations for all Infrastructure Providers regarding settings, flags, and other choices and then begin bootstrapping.
2. Create a bootstrap cluster using the command `nkp create bootstrap --kubeconfig $HOME/.kube/config`.

> **Note:**
>
> - Use `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Configuring an HTTP or HTTPS Proxy on page 677.
> - Flatcar OS use `--os-hint` to instruct the bootstrap cluster to make some changes related to the installation paths:

Example output:

```
# Creating a bootstrap cluster
# Initializing new CAPI components
```

3. NKP creates a bootstrap cluster using KIND as a library.

   For more information, see https://github.com/kubernetes-sigs/kind.

4. NKP then deploys the following Cluster API providers on the cluster.

   - Core Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/
   - AWS Infrastructure Provider: https://github.com/kubernetes-sigs/cluster-api-provider-aws
   - Kubeadm Bootstrap Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/bootstrap/kubeadm
   - Kubeadm ControlPlane Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/controlplane/kubeadm

   For more information on Cluster APIs, see https://cluster-api.sigs.k8s.io/.

5. Ensure that the CAPV controllers are present using the command `kubectl get pods -n capv-system`

   Output example:

```
NAME                                        READY   STATUS    RESTARTS   AGE
capv-controller-manager-785c5978f-nnfns   1/1     Running   0          13h
```

6. NKP waits until these providers' controller-manager and webhook deployments are ready. List these deployments using the command `kubectl get --all-namespaces deployments -l=clusterctl.cluster.x-k8s.io`.

   Output example:

```
NAMESPACE                           NAME
 READY   UP-TO-DATE   AVAILABLE   AGE
capa-system                         capa-controller-manager
 1/1     1            1           1h
capg-system                         capg-controller-manager
 1/1     1            1           1h
capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-manager
 1/1     1            1           1h
capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-manager
 1/1     1            1           1h
capi-system                         capi-controller-manager
 1/1     1            1           1h
cappp-system                        cappp-controller-manager
 1/1     1            1           1h
capv-system                         capv-controller-manager
 1/1     1            1           1h
capz-system                         capz-controller-manager
 1/1     1            1           1h
cert-manager                        cert-manager
 1/1     1            1           1h
cert-manager                        cert-manager-cainjector
 1/1     1            1           1h
cert-manager                        cert-manager-webhook
 1/1     1            1           1h
```

vSphere Creating an Air-gapped Cluster
Create a vSphere Management Cluster in an air-gapped environment.

About this task

If you use these instructions to create a cluster on vSphere using the Nutanix Kubernetes Platform (NKP) default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes. First, you must name your cluster.

Before you begin

Name Your Cluster

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.`, and `-`. Cluster creation will fail if the name has capital letters. See Kubernetes for more naming information at https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable:

```
export CLUSTER_NAME=<my-vsphere-cluster>
```

3. Use the following command to set the environment variables for vSphere.

```
export VSPHERE_SERVER=example.vsphere.url
export VSPHERE_USERNAME=user@example.vsphere.url
export VSPHERE_PASSWORD=example_password
```

4. Load the image, using either the `docker` or `podman` command

   » `docker load -i konvoy-bootstrap-image-v2.12.0.tar`

   » `podman load -i konvoy-bootstrap-image-v2.12.0.tar`

5. Generate the Kubernetes cluster objects by copying and editing this command to include the correct values, including the VM template name you assigned in the previous procedure.

   > **Note:** NKP uses the vSphere CSI driver as the default storage provider. Use a Kubernetes CSI compatible storage that is suitable for production. For more information, see the Kubernetes documentation called ##Changing the Default Storage Class# If you're not using the default, you cannot deploy an alternate provider until after the `nkp create cluster` is finished. However, this must be determined before the Kommander installation. For more information, see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types and Changing the Default Storage Class

```
nkp create cluster vsphere \
  --cluster-name ${CLUSTER_NAME} \
  --network <NETWORK_NAME> \
  --control-plane-endpoint-host <CONTROL_PLANE_IP> \
  --data-center <DATACENTER_NAME> \
  --data-store <DATASTORE_NAME> \
  --folder <FOLDER_NAME> \
  --server <VCENTER_API_SERVER_URL> \
  --ssh-public-key-file </path/to/key.pub> \
  --resource-pool <RESOURCE_POOL_NAME> \
  --vm-template konvoy-ova-vsphere-os-release-k8s_release-vsphere-timestamp \
  --virtual-ip-interface eth0 \
  --extra-sans "127.0.0.1" \
  --registry-mirror-url=${REGISTRY_URL} \
  --registry-mirror-cacert=${REGISTRY_CA} \
```

```
  --registry-mirror-username=${REGISTRY_USERNAME} \
  --registry-mirror-password=${REGISTRY_PASSWORD} \
  --dry-run \
  --output=yaml \
> ${CLUSTER_NAME}.yaml
```

> **Note:** To increase Dockerhub's rate limit use your Dockerhub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username= --registry-mirror-password=` on the `nkp create cluster` command. For more information, see https://docs.docker.com/docker-hub/download-rate-limit/.

- Flatcar OS flag: Flatcar OS use `--os-hint flatcar` to instruct the bootstrap cluster to make some changes related to the installation paths:
- HTTP: If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677.
- FIPS flags -

  To create a cluster in FIPS mode, inform the controllers of the appropriate image repository and version tags of the official Nutanix FIPS builds of Kubernetes by adding those flags to `nkp create cluster` command:
- You can create individual manifest files with different smaller manifests for ease in editing using the `--output-directory` flag.

6. Inspect or edit the cluster objects. Familiarize yourself with Cluster API before editing the cluster objects, as edits can prevent the cluster from deploying successfully.
7. Create the cluster from the objects generated from the `dry run`. A warning will appear in the console if the resource already exists, requiring you to remove the resource or update your YAML.

   ```
   kubectl create -f ${CLUSTER_NAME}.yaml
   ```

   > **Note:** If you used the `--output-directory` flag in your `NKP create .. --dry-run` step above, create the cluster from the objects you created by specifying the directory:

   ```
   kubectl create -f <existing-directory>/.
   ```

8. Wait for the cluster control-plane to be ready.

   ```
   kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
   timeout=20m
   ```

9. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. Konvoy provides a command to describe the current status of the cluster.

   ```
   nkp describe cluster -c ${CLUSTER_NAME}
   ```

   Output:

   ```
   NAME                                                                READY  SEVERITY
    REASON  SINCE  MESSAGE
   Cluster/nutanix-e2e-cluster_name-1                                     True
             13h
   ##ClusterInfrastructure - VSphereCluster/nutanix-e2e-cluster_name-1    True
             13h
   ##ControlPlane - KubeadmControlPlane/nutanix-control-plane             True
             13h
   # ##Machine/nutanix--control-plane-7llgd                               True
             13h
   ```

```
# ##Machine/nutanix--control-plane-vncbl                                  True
           13h
# ##Machine/nutanix--control-plane-wbgrm                                 True
           13h
##Workers
    ##MachineDeployment/nutanix--md-0                                    True
           13h
    ##Machine/nutanix--md-0-74c849dc8c-67rv4                             True
           13h
    ##Machine/nutanix--md-0-74c849dc8c-n2skc                             True
           13h
    ##Machine/nutanix--md-0-74c849dc8c-nkftv                             True
           13h
    ##Machine/nutanix--md-0-74c849dc8c-sqklv                             True
           13h
```

Making the vSphere Air-gapped Cluster Self-Managed

How to make a Kubernetes cluster manage itself.

About this task

Nutanix Kubernetes Platform (NKP) deploys all cluster life cycle services to a bootstrap cluster, which then deploys a workload cluster. When the workload cluster is ready, move the cluster life cycle services to the workload cluster, which makes the workload cluster self-managed.

Before you begin

Ensure you can create a workload cluster as described in the topic: Create a New Air-gapped vSphere Cluster.

This page contains instructions on how to make your cluster self-managed. This is necessary if there is only one cluster in your environment or if this cluster becomes the Management cluster in a multi-cluster environment.

> **Note:** If you already have a self-managed or Management cluster in your environment, skip this page.

Follow these steps to turn your new cluster into a **Management Cluster** for an Ultimate license environment (or a free-standing Pro Cluster):

Procedure

1. Deploy cluster life cycle services on the workload cluster.

   ```
   nkp create capi-components --kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Initializing new CAPI components
   ```

   > **Note:** If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP/HTTPS Proxy.

2. The cluster life cycle services on the workload cluster are ready, but the workload cluster configuration is on the bootstrap cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the bootstrap to the workload cluster. This process is

called a Pivot. For more information, see https://cluster-api.sigs.k8s.io/reference/glossary.html?highlight=pivot#pivot.

Move the Cluster API objects from the bootstrap to the workload cluster:

```
nkp move capi-resources --to-kubeconfig ${CLUSTER_NAME}.conf
```

Output:

```
# Moving cluster resources
You can now view resources in the moved cluster by using the --kubeconfig flag with
 kubectl. For example: kubectl --kubeconfig=gcp-example.conf get nodes
```

> **Note:** To ensure only one set of cluster life cycle services manages the workload cluster, NKP first pauses the reconciliation of the objects on the bootstrap cluster, then creates the objects on the workload cluster. As NKP copies the objects, the cluster life cycle services on the workload cluster reconcile the objects. The workload cluster becomes self-managed after NKP creates all the objects. If it fails, the `move` command can be safely retried.

3. Wait for the cluster control-plane to be ready.

```
kubectl --kubeconfig ${CLUSTER_NAME}.conf wait --for=condition=ControlPlaneReady
 "clusters/${CLUSTER_NAME}" --timeout=20m
```

Output:

```
cluster.cluster.x-k8s.io/gcp-example condition met
```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster status. After moving the cluster life cycle services to the workload cluster, remember to use NKP with the workload cluster `kubeconfig`.

```
nkp describe cluster --kubeconfig ${CLUSTER_NAME}.conf -c ${CLUSTER_NAME}
```

Output:

```
NAME                                                                READY  SEVERITY
 REASON  SINCE  MESSAGE
Cluster/vsphere-example-1                                           True
        13h
##ClusterInfrastructure - VSphereCluster/vsphere-example-1          True
        13h
##ControlPlane - KubeadmControlPlane/vsphere-example-control-plane  True
        13h
# ##Machine/vsphere-example-control-plane-7llgd                     True
        13h
# ##Machine/vsphere-example-control-plane-vncbl                     True
        13h
# ##Machine/vsphere-example-control-plane-wbgrm                     True
        13h
##Workers
    ##MachineDeployment/vsphere-example-md-0                        True
        13h
    ##Machine/vsphere-example-md-0-74c849dc8c-67rv4                 True
        13h
    ##Machine/vsphere-example-md-0-74c849dc8c-n2skc                 True
        13h
    ##Machine/vsphere-example-md-0-74c849dc8c-nkftv                 True
        13h
    ##Machine/vsphere-example-md-0-74c849dc8c-sqklv                 True
        13h
```

5. Remove the bootstrap cluster because the workload cluster is now self-managed.

```
nkp delete bootstrap --kubeconfig $HOME/.kube/config
```

```
# Deleting bootstrap cluster
```

Known Limitations

Procedure

- NKP only supports moving all namespaces in the cluster; NKP does not support migration of individual namespaces.
- Konvoy supports moving only one set of cluster objects from the bootstrap cluster to the workload cluster or vice-versa.

Exploring the vSphere Air-gapped Cluster

This guide explains how to use the command line to interact with your newly deployed Kubernetes cluster.

About this task

Before you start, make sure you have created a workload cluster, as described in Create a New vSphere Air-gapped Cluster.

Procedure

1. When the workload cluster is created, the cluster life cycle services generate a `kubeconfig` file for the workload cluster and write it to a Secret. The `kubeconfig` file is scoped to the cluster administrator. Get a `kubeconfig` file for the workload cluster.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
   ```

2. Create a StorageClass with a vSphere datastore.

   a. Access the Datastore tab in the vSphere client and select a datastore by name.

   b. Copy the URL of that datastore from the information dialog displayed.

   c. Return to the Nutanix Kubernetes Platform (NKP) CLI, and delete the existing `StorageClass` with the command: `kubectl delete storageclass vsphere-raw-block-sc`

   d. Run the following command to create a new StorageClass, supplying the correct values for your environment.

   ```
   cat <<EOF > vsphere-raw-block-sc.yaml
   kind: StorageClass
   apiVersion: storage.k8s.io/v1
   metadata:
     annotations:
       storageclass.kubernetes.io/is-default-class: "true"
     name: vsphere-raw-block-sc
   provisioner: csi.vsphere.vmware.com
   parameters:
     datastoreurl: "<url>"
   volumeBindingMode: WaitForFirstConsumer
   EOF
   ```

3. Verify the API server is up by listing the nodes.

   ```
   kubectl --kubeconfig=${CLUSTER_NAME}.conf get nodes
   ```

   Note: The Status may take a few minutes to move to Ready while the Pod network is deployed. The Nodes' Status will change to Ready soon after the calico-node DaemonSet Pods are Ready.

4. List the Pods with the command.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get pods -A
```

Verify the output:

```
NAMESPACE                NAME
         READY   STATUS     RESTARTS      AGE
calico-system            calico-kube-controllers-57fbd7bd59-qqd96
         1/1     Running    0             20h
calico-system            calico-node-2m524
         1/1     Running    3 (19h ago)   19h
calico-system            calico-node-bbhg5
         1/1     Running    0             20h
calico-system            calico-node-cc5lf
         1/1     Running    2 (19h ago)   19h
calico-system            calico-node-cwg7x
         1/1     Running    1 (19h ago)   19h
calico-system            calico-node-d59hn
         1/1     Running    1 (19h ago)   19h
calico-system            calico-node-qmmcz
         1/1     Running    0             19h
calico-system            calico-node-wdqhx
         1/1     Running    0             19h
calico-system            calico-typha-655489d8cc-b5jnt
         1/1     Running    0             20h
calico-system            calico-typha-655489d8cc-q92x9
         1/1     Running    0             19h
calico-system            calico-typha-655489d8cc-vjlkx
         1/1     Running    0             19h
kube-system              cluster-autoscaler-68c759fbf6-7d2ck
         0/1     Init:0/1   0             20h
kube-system              coredns-78fcd69978-qn4qt
         1/1     Running    0             20h
kube-system              coredns-78fcd69978-wqpmg
         1/1     Running    0             20h
kube-system              etcd-nutanix-e2e-cluster-1-control-plane-7llgd
        1/1     Running    0             20h
kube-system              etcd-nutanix-e2e-cluster-1-control-plane-vncbl
        1/1     Running    0             19h
kube-system              etcd-nutanix-e2e-cluster-1-control-plane-wbgrm
        1/1     Running    0             19h
kube-system              kube-apiserver-nutanix-e2e-cluster-1-control-plane-7llgd
        1/1     Running    0             20h
kube-system              kube-apiserver-nutanix-e2e-cluster-1-control-plane-vncbl
        1/1     Running    0             19h
kube-system              kube-apiserver-nutanix-e2e-cluster-1-control-plane-wbgrm
        1/1     Running    0             19h
kube-system              kube-controller-manager-nutanix-e2e-cluster-1-control-
plane-7llgd   1/1     Running    1 (19h ago)   20h
kube-system              kube-controller-manager-nutanix-e2e-cluster-1-control-plane-
vncbl   1/1     Running    0             19h
kube-system              kube-controller-manager-nutanix-e2e-cluster-1-control-plane-
wbgrm   1/1     Running    0             19h
kube-system              kube-proxy-cpscs
         1/1     Running    0             19h
kube-system              kube-proxy-hhmxq
         1/1     Running    0             19h
kube-system              kube-proxy-hxhnk
         1/1     Running    0             19h
kube-system              kube-proxy-nsrbp
         1/1     Running    0             19h
```

```
kube-system              kube-proxy-scxfg
         1/1     Running    0             20h
kube-system              kube-proxy-tth4k
         1/1     Running    0             19h
kube-system              kube-proxy-x2xfx
         1/1     Running    0             19h
kube-system              kube-scheduler-nutanix-e2e-cluster-1-control-plane-7llgd
         1/1     Running    1 (19h ago)   20h
kube-system              kube-scheduler-nutanix-e2e-cluster-1-control-plane-vncbl
         1/1     Running    0             19h
kube-system              kube-scheduler-nutanix-e2e-cluster-1-control-plane-wbgrm
         1/1     Running    0             19h
kube-system              kube-vip-nutanix-e2e-cluster-1-control-plane-7llgd
         1/1     Running    1 (19h ago)   20h
kube-system              kube-vip-nutanix-e2e-cluster-1-control-plane-vncbl
         1/1     Running    0             19h
kube-system              kube-vip-nutanix-e2e-cluster-1-control-plane-wbgrm
         1/1     Running    0             19h
kube-system              vsphere-cloud-controller-manager-4zj7q
         1/1     Running    0             19h
kube-system              vsphere-cloud-controller-manager-87tgm
         1/1     Running    0             19h
kube-system              vsphere-cloud-controller-manager-xqmn4
         1/1     Running    1 (19h ago)   20h
node-feature-discovery   node-feature-discovery-master-84c67dcbb6-txfw9
         1/1     Running    0             20h
node-feature-discovery   node-feature-discovery-worker-8tg2l
         1/1     Running    3 (19h ago)   19h
node-feature-discovery   node-feature-discovery-worker-c5f6q
         1/1     Running    0             19h
node-feature-discovery   node-feature-discovery-worker-fjfkm
         1/1     Running    0             19h
node-feature-discovery   node-feature-discovery-worker-x6tz8
         1/1     Running    0             19h
tigera-operator          tigera-operator-d499f5c8f-r2srj
         1/1     Running    1 (19h ago)   20h
vmware-system-csi        vsphere-csi-controller-7ffd6884cc-d7rql
         7/7     Running    5 (19h ago)   20h
vmware-system-csi        vsphere-csi-controller-7ffd6884cc-k82cm
         7/7     Running    2 (19h ago)   20h
vmware-system-csi        vsphere-csi-controller-7ffd6884cc-qttkp
         7/7     Running    1 (19h ago)   20h
vmware-system-csi        vsphere-csi-node-678hw
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-6tbsh
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-9htwr
         3/3     Running    5 (20h ago)   20h
vmware-system-csi        vsphere-csi-node-g8r6l
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-ghmr6
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-jhvgm
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-rp77r
         3/
```

Installing Kommander in a vSphere Air-gapped Environment

This section provides installation instructions for the Kommander component of Nutanix Kubernetes Platform (NKP) in an air-gapped vSphere environment.

About this task

After you have installed the Konvoy component of NKP, you will continue installing the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a time period (for example, 1 hour) to allocate more time to deploy applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. If required: Customize your `kommander.yaml`.

   a. See the Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.

5. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `dkp-catalog-appliations`.

   ```
   apiVersion: config.kommander.mesosphere.io/v1alpha1
   kind: Installation
   catalog:
     repositories:
       - name: dkp-catalog-applications
         labels:
           kommander.nutanix.io/project-default-catalog-repository: "true"
           kommander.nutanix.io/workspace-default-catalog-repository: "true"
           kommander.nutanix.io/gitapps-gitrepository-type: "nkp"
         gitRepositorySpec:
   ```

```
      url: https://github.com/mesosphere/dkp-catalog-applications
      ref:
        tag: v2.12.0
```

6. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Enable NKP Catalog Applications after Installing NKP.

Verifying the vSphere Air-gapped Install and Log in to the UI

Verify the Kommander Install and Log in to the Dashboard UI

About this task

You can verify your installation after you build the Konvoy cluster and install the Kommander component for the UI. bu default, verification waits for all applications to be ready.

> **Note:** If the Kommander installation fails or you wish to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

You can check the status of the installation using the following command.

```
kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
```

> **Note:** If you prefer the CLI to not wait for all applications to become ready, you can set the `--wait=false` flag.

The first wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

```
helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
helmrelease.helm.toolkit.fluxcd.io/dex condition met
helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
helmrelease.helm.toolkit.fluxcd.io/grafana-loki condition met
helmrelease.helm.toolkit.fluxcd.io/karma condition met
helmrelease.helm.toolkit.fluxcd.io/kommander condition met
helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
helmrelease.helm.toolkit.fluxcd.io/reloader condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
```

```
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

Failed HelmReleases

Procedure

If an application fails to deploy, check the status of a `HelmRelease` using the command `kubectl -n kommander get helmrelease <HELMRELEASE_NAME>`

If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress”, trigger a reconciliation of the `HelmRelease` using the commands `kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`

Log in to the UI

Procedure

1. By default, you can log in to the UI in Kommander with the credentials given using this command.

   ```
   nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
   ```

2. Retrieve your credentials at any time if necessary.

   ```
   kubectl -n kommander get secret NKP-credentials -o go-template='Username:
    {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
   {{ "\n"}}'
   ```

3. Retrieve the URL used for accessing the UI with the following.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/NKP/kommander/
   dashboard{{ "\n"}}'
   ```

   Only use these static credentials to access the UI for configuring an external identity provider. Treat them as back up credentials rather than use them for normal access.

   a. Rotate the password using the following command.

      ```
      nkp experimental rotate dashboard-password
      ```

      The output displays the new password:

      ```
      Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
      ```

Dashboard UI Functions

Procedure

After installing Konvoy component and building a cluster as well as successfully installing Kommander and logging into the UI, you are now ready to customize configurations using the Cluster Operations Management section of the documentation. The majority of this customization such as attaching clusters and deploying applications will take place in the dashboard or UI of Nutanix Kubernetes Platform (NKP) . The Cluster Operations section allows you to manage cluster operations and their application workloads to optimize your organization’s productivity.

- Continue to the NKP Dashboard.

vSphere Management Tools

After cluster creation and configuration, you can revisit clusters to update and change variables.

Manage vSphere Node Pools

Node pools are part of a cluster and managed as a group, and you can use a node pool to manage a group of machines using the same common properties. When Konvoy creates a new default cluster, there is one node pool for the worker nodes, and all nodes in that new node pool have the same configuration. You can create additional node pools for more specialized hardware or configuration. For example, suppose you want to tune your memory usage on a cluster where you need maximum memory for some machines and minimal memory for others. In that case, you create a new node pool with those specific resource needs.

Nutanix Kubernetes Platform (NKP) implements node pools using Cluster API MachineDeployments. For more information on node pools, see these sections:

Creating vsphere Node Pools

Creating a node pool is useful when you need to run workloads that require machines with specific resources, such as a GPU, additional memory, or specialized network or storage hardware.

About this task

Availability zones (AZs) are isolated locations within datacenter regions where public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single Availability Zone, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple Availability Zones.

The first task is to prepare the environment.

Procedure

1. Set the environment variable to the name you assigned this cluster.

   ```
   export CLUSTER_NAME=<my-vsphere-cluster>
   ```

2. If your workload cluster is self-managed, as described in Make the New Cluster Self-Managed, configure kubectl to use the kubeconfig for the cluster.

   ```
   export KUBECONFIG=${CLUSTER_NAME}.conf
   ```

3. Define your node pool name.

   ```
   export NODEPOOL_NAME=example
   ```

Create a vSphere Node Pool

Procedure

Create a new node pool with three replicas using this command.

```
nkp create nodepool vsphere ${NODEPOOL_NAME} \
  --cluster-name=${CLUSTER_NAME} \
  --network=example_network \
  --data-center=example_datacenter \
  --data-store=example_datastore \
  --folder=example_folder \
  --server=example_vsphere_api_server_url\
  --resource-pool=example_resource_pool \
```

```
  --vm-template=example_vm_template \
  --replicas=3
```

Advanced users can use a combination of the `--dry-run` and `--output=yaml` or `--output-directory=<existing-directory>` flags to get a complete set of node pool objects to modify locally or store in version control.

Listing vSphere Node Pools

List the node pools of a given cluster. This returns specific properties of each node pool so that you can see the name of the MachineDeployments.

About this task

List node pools for a managed cluster.

Procedure

To list all node pools for a managed cluster, run:.

```
nkp get nodepools --cluster-name=${CLUSTER_NAME} --kubeconfig=${CLUSTER_NAME}.conf
```

The expected output is similar to the following example, indicating the desired size of the node pool, the number of replicas ready in the node pool, and the Kubernetes version those nodes are running:

```
NODEPOOL                        DESIRED              READY                KUBERNETES
 VERSION
demo-cluster-md-0               4                    4                    v1.27.6
example                         3                    0                    v1.27.6
```

Scaling vsphere Node Pools

While running Cluster Autoscaler, you can manually scale your node pools up or down when you need finite control over your environment.

About this task

If you require ten machines to run a process, you can only manually set the scaling to run those ten machines. However, using the Cluster Autoscaler, you must stay within your minimum and maximum bounds. This process allows you to scale manually.

Environment variables, such as defining the node pool name, are set in the Prepare the Environment section on the previous page. If needed, refer to that page to set those variables.

Scale Up Node Pools

Procedure

1. To scale up a node pool in a cluster, run one of the following.

   ```
   nkp scale nodepools ${NODEPOOL_NAME} --replicas=5 --cluster-name=${CLUSTER_NAME}
   ```

   Output example indicating scaling is in progress:

   ```
   INFO[2021-07-26T08:54:35-07:00] Running scale nodepool command
       clusterName=demo-cluster managementClusterKubeconfig= namespace=default
    src="nodepool/scale.go:82"
   INFO[2021-07-26T08:54:35-07:00] Nodepool example scaled to 5 replicas
    clusterName=demo-cluster managementClusterKubeconfig= namespace=default
    src="nodepool/scale.go:94"
   ```

2. After a few minutes, you can list the node pools.

```
nkp get nodepools --cluster-name=${CLUSTER_NAME} --kubeconfig=${CLUSTER_NAME}.conf
```

Example output showing the number of DESIRED and READY replicas increased to 5:

```
NODEPOOL                        DESIRED                READY                KUBERNETES
 VERSION
example                         5                      5                    v1.29.6
demo-cluster-md-0               4                      4                    v1.29.6
```

Scaling Down vSphere Node Pools

While running Cluster Autoscaler, you can manually scale your node pools up or down when you need finite control over your environment.

About this task

If you require ten machines to run a process, you can only manually set the scaling to run those ten machines. However, using the Cluster Autoscaler, you must stay within your minimum and maximum bounds. This process allows you to scale manually.

Environment variables, such as defining the node pool name, are set in the Prepare the Environment section on the previous page. If needed, refer to that page to set those variables.

Procedure

1. To scale down a node pool, run.

```
nkp scale nodepools ${NODEPOOL_NAME} --replicas=4 --cluster-name=${CLUSTER_NAME}
```

Output example indicating that scaling is in progress:

```
INFO[2021-07-26T08:54:35-07:00] Running scale nodepool command
    clusterName=demo-cluster managementClusterKubeconfig= namespace=default
 src="nodepool/scale.go:82"
INFO[2021-07-26T08:54:35-07:00] Nodepool example scaled to 4 replicas
 clusterName=demo-cluster managementClusterKubeconfig= namespace=default
 src="nodepool/scale.go:94"
```

In a default cluster, the nodes to delete are selected at random. This behavior is controlled by CAPI's delete policy For more information, see https://github.com/kubernetes-sigs/cluster-api/blob/v0.4.0/api/v1alpha4/machineset_types.go#L85-L105. However, when using the Konvoy CLI to scale down a node pool, you can specify the Kubernetes Nodes you want to delete.

To do this, set the flag `--nodes-to-delete` with a list of nodes, as shown in the next command. This adds an annotation `cluster.x-k8s.io/delete-machine=yes` to the matching Machine object that contains `status.NodeRef` with the node names from `--nodes-to-delete`.

2. After a few minutes, you can list the node pools.

```
nkp get nodepools --cluster-name=${CLUSTER_NAME} --kubeconfig=${CLUSTER_NAME}.conf
```

Example output showing that the number of DESIRED and READY replicas decreased to 4:

```
INFO[2021-07-26T08:54:35-07:00] Running scale nodepool command
    clusterName=demo-cluster managementClusterKubeconfig= namespace=default
 src="nodepool/scale.go:82"
INFO[2021-07-26T08:54:35-07:00] Nodepool example scaled to 3 replicas
 clusterName=demo-cluster managementClusterKubeconfig= namespace=default
 src="nodepool/scale.go:94"
```

3. In a default cluster, the nodes to delete are selected at random. CAPI's delete policy controls this behavior. However, when using the Nutanix Kubernetes Platform (NKP) CLI to scale down a node pool, it is also possible to specify the Kubernetes Nodes you want to delete.

   To do this, set the flag `--nodes-to-delete` with a list of nodes as below. This adds an annotation `cluster.x-k8s.io/delete-machine=yes` to the matching Machine object that contains `status.NodeRef` with the node names from `--nodes-to-delete`.

   ```
   nkp scale nodepools ${NODEPOOL_NAME} --replicas=3 --nodes-to-delete=<> --cluster-
   name=${CLUSTER_NAME}
   ```

   Output:

   ```
   # Scaling node pool example to 3 replicas
   ```

Deleting vSphere Node Pools

Deleting a node pool deletes the Kubernetes nodes and the underlying infrastructure.

About this task

All nodes will be drained before deletion, and the pods running on those nodes will be rescheduled.

Procedure

1. To delete a node pool from a managed cluster, run.

   ```
   nkp delete nodepool ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME}
   ```

   Here, `example` is the node pool to be deleted.

   The expected output will be similar to the following example, indicating the node pool is being deleted:

   ```
   # Deleting default/example nodepool resources
   ```

2. Deleting an invalid node pool results in output similar to this example.

   ```
   nkp delete nodepool ${CLUSTER_NAME}-md-invalid --cluster-name=${CLUSTER_NAME}
   ```

   Output:

   ```
   nkp delete nodepool ${CLUSTER_NAME}-md-invalid --cluster-name=${CLUSTER_NAME}

   INFO[2021-07-28T17:11:44-07:00] Running nodepool delete command
         Nodepool=demo-cluster-md-invalid clusterName=nutanix-e2e-cluster-1
    managementClusterKubeconfig= namespace=default src="nodepool/delete.go:80"
   Error: failed to get nodepool with name demo-cluster-md-invalid in namespace
    default : failed to get nodepool with name demo-cluster-md-invalid in namespace
    default : machinedeployments.cluster.x-k8s.io "demo-cluster-md-invalid" not found
   ```

vSphere Certificate Renewal

During cluster creation, Kubernetes establishes a Public Key Infrastructure (PKI) for generating the TLS certificates needed for securing cluster communication for various components such as `etcd`, `kubernetes-apiserver` and `kube-proxy`. The certificates created by these components have a default expiration of one year and are renewed when an administrator updates the cluster.

Kubernetes provides a facility to renew all certificates automatically during control plane updates. For administrators who need long-running clusters or clusters that are not upgraded often, `nkp` provides automated certificate renewal without a cluster upgrade.

- This feature requires Python 3.5 or greater to be installed on all control plane hosts.

- Complete the Bootstrap Cluster topic.

To create a cluster with automated certificate renewal, you create a Konvoy cluster using the `certificate-renew-interval` flag. The `certificate-renew-interval` is the number of days after which Kubernetes-managed PKI certificates will be renewed. For example, an `certificate-renew-interval` value of 60 means the certificates will be renewed every 60 days.

- Complete the bootstrap.
- To enable the automated certificate renewal, create a Konvoy cluster using the `certificate-renew-interval` flag:

  ```
  nkp create cluster vsphere --certificate-renew-interval=60 --cluster-name=long-running
  ```

  The `certificate-renew-interval` is the number of days after which Kubernetes-managed PKI certificates are renewed. For example, a `certificate-renew-interval` value of 30 means the certificates are renewed every 30 days.

Technical Details

The following manifests are modified on the control plane hosts and are located at `/etc/kubernetes/manifests`. Modifications to these files require SUDO access.

```
kube-controller-manager.yaml
kube-apiserver.yaml
kube-scheduler.yaml
kube-proxy.yaml
```

The following annotation indicates the time each component was reset:

```
metadata:
  annotations:
    konvoy.d2iq.io/restartedAt: $(date +%s)
```

This only occurs when the PKI certificates are older than the interval given at cluster creation time. This is activated by a `systemd timer` called `renew-certs.timer` that triggers an associated `systemd` service called `renew-certs.service` that runs on all of the control plane hosts.

Configuring vSphere Cluster Autoscaler

This page explains how to configure autoscaler for node pools.

About this task

Cluster Autoscaler provides the ability to automatically scale up or scale down the number of worker nodes in a cluster based on the number of pending pods to be scheduled. Running the Cluster Autoscaler is optional. Unlike Horizontal-Pod Autoscaler, Cluster Autoscaler does not depend on any Metrics server and does not need Prometheus or any other metrics source.

The Cluster Autoscaler looks at the following annotations on a MachineDeployment to determine its scale-up and scale-down ranges:

> **Note:**
>
> ```
> cluster.x-k8s.io/cluster-api-autoscaler-node-group-min-size
> cluster.x-k8s.io/cluster-api-autoscaler-node-group-max-size
> ```

The full list of command line arguments to the Cluster Autoscaler controller is on the Kubernetes public GitHub repository.

For more information about how Cluster Autoscaler works, see these documents:

- What is Cluster Autoscaler

- How does scale-up work
- How does scale-down work
- CAPI Provider for Cluster Autoscaler

Before you begin

Ensure you have the following:

- Bootstrap cluster Life cycle: Bootstrapping vSphere on page 924
- Creating a New vSphere Cluster on page 926.
- Self-Managed Cluster.

Run Cluster Autoscaler on the Management Cluster

Procedure

1. Ensure the Cluster Autoscaler controller is up and running (no restarts and no errors in the logs)

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf logs deployments/cluster-autoscaler
 cluster-autoscaler -n kube-system -f
```

2. Enable Cluster Autoscaler by setting the min & max ranges.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf annotate machinedeployment ${NODEPOOL_NAME}
 cluster.x-k8s.io/cluster-api-autoscaler-node-group-min-size=2
kubectl --kubeconfig=${CLUSTER_NAME}.conf annotate machinedeployment ${NODEPOOL_NAME}
 cluster.x-k8s.io/cluster-api-autoscaler-node-group-max-size=6
```

   The Cluster Autoscaler logs will show that the worker nodes are associated with node-groups and that pending pods are being watched.

3. To demonstrate that it is working properly, create a large deployment that will trigger pending pods (For this example, we used AWS m5.2xlarge worker nodes. If you have larger worker-nodes, you need to scale up the number of replicas accordingly).

```
cat <<EOF | kubectl --kubeconfig=${CLUSTER_NAME}.conf apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: busybox-deployment
  labels:
    app: busybox
spec:
  replicas: 600
  selector:
    matchLabels:
      app: busybox
  template:
    metadata:
      labels:
        app: busybox
    spec:
      containers:
      - name: busybox
        image: busybox:latest
        command:
          - sleep
          - "3600"
        imagePullPolicy: IfNotPresent
```

```
      restartPolicy: Always
EOF
```

Cluster Autoscaler will scale up the number of Worker Nodes until there are no pending pods.

4. Scale down the number of replicas for `busybox-deployment`.

```
kubectl --kubeconfig ${CLUSTER_NAME}.conf scale --replicas=30 deployment/busybox-deployment
```

5. Cluster Autoscaler starts to scale down the number of Worker Nodes after the default timeout of 10 minutes.

Run Cluster Autoscaler on a Managed (Workload) Cluster

About this task

Unlike the Management(self-managed) cluster instructions above, an additional instance of autoscaler is required to run autoscaler on a managed cluster. This instance is run on the management cluster but must be pointed at the managed cluster. The `nkp create cluster` command for building a managed cluster then runs against the Management cluster so that the `clusterresourcset` for that cluster's autoscaler is modified to deploy the autoscaler on the management cluster itself. The flags for cluster-autoscaler are changed as well.

Procedure

1. Create a secret with a kubeconfig file of the master cluster in the managed cluster with limited user permissions to only modify resources for the given cluster.
2. Mount the secret into the cluster-autoscaler deployment.
3. Add the following flag to the cluster-autoscaler command so that `/mnt//masterconfig/` value is the path where the master cluster's kubeconfig is loaded through the secret created.

```
--cloud-config=/mnt//masterconfig/value
```

Deleting a vSphere Cluster

Deleting a vsphere cluster.

About this task

A self-managed workload cluster cannot delete itself. If your workload cluster is self-managed, you must first create a bootstrap cluster and move the cluster life cycle services to it before deleting the workload cluster.

Procedure

If you did not make your workload cluster self-managed, as described in Make New Cluster Self-Managed, proceed to the instructions for Delete the workload cluster.

Create a Bootstrap Cluster and Move CAPI Resources

About this task

Follow these steps to create a bootstrap cluster and move CAPI resources:

Procedure

1. The bootstrap cluster will host the Cluster API controllers that reconcile the cluster objects marked for deletion. Create a bootstrap cluster. To avoid using the wrong kubeconfig, the following steps use explicit kubeconfig paths and contexts.

```
nkp create bootstrap --kubeconfig $HOME/.kube/config --with-vSphere-bootstrap-
credentials=true
```

2. Move the Cluster API objects from the workload to the bootstrap cluster: The cluster life cycle services on the bootstrap cluster are ready, but the workload cluster configuration is on the workload cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the workload to the bootstrap cluster. This process is also called a Pivot.

```
nkp move capi-resources \
 --from-kubeconfig ${CLUSTER_NAME}.conf \
 --from-context ${CLUSTER_NAME}-admin@${CLUSTER_NAME} \
 --to-kubeconfig $HOME/.kube/config \
 --to-context kind-konvoy-capi-bootstrapper
```

Output:

```
# Moving cluster resources
You can now view resources in the moved cluster by using the --kubeconfig flag with
 kubectl. For example: kubectl --kubeconfig $HOME/.kube/config get nodes
```

3. Use the cluster life cycle services on the workload cluster to check the workload cluster's status.

```
nkp describe cluster --kubeconfig $HOME/.kube/config -c ${CLUSTER_NAME}
```

Output:

```
NAME                                                                READY  SEVERITY
 REASON  SINCE  MESSAGE
Cluster/nutanix-e2e-cluster_name-1                                    True
           13h
##ClusterInfrastructure - VSphereCluster/nutanix-e2e-cluster_name-1    True
           13h
##ControlPlane - KubeadmControlPlane/nutanix-control-plane             True
           13h
# ##Machine/nutanix--control-plane-7llgd                              True
           13h
# ##Machine/nutanix--control-plane-vncbl                              True
           13h
# ##Machine/nutanix--control-plane-wbgrm                              True
           13h
##Workers
    ##MachineDeployment/nutanix--md-0                                 True
           13h
    ##Machine/nutanix--md-0-74c849dc8c-67rv4                          True
           13h
    ##Machine/nutanix--md-0-74c849dc8c-n2skc                          True
           13h
    ##Machine/nutanix--md-0-74c849dc8c-nkftv                          True
           13h
    ##Machine/nutanix--md-0-74c849dc8c-sqklv                          True
           13h
```

4. Wait for the cluster control-plane to be ready.

```
kubectl --kubeconfig $HOME/.kube/config wait --for=condition=controlplaneready
 "clusters/${CLUSTER_NAME}" --timeout=20m
```

Output:

```
cluster.cluster.x-k8s.io/vSphere-example condition met
```

Delete the Workload Cluster

Procedure

1. Make sure your vSphere credentials are up to date. Refresh the credentials using this command.

```
nkp update bootstrap credentials vSphere --kubeconfig $HOME/.kube/config
```

> **Note:**
>
> Persistent Volumes (PVs) are not deleted automatically by design to preserve your data. However, the PVs take up storage space if not deleted. You must delete PVs manually. Information for backup of a cluster and PVs is on the Back up your Cluster's Applications and Persistent Volumes page.

2. To delete a cluster, Use `NKP delete cluster` and pass in the name of the cluster you are trying to delete with `--cluster-name` flag. Use `kubectl get clusters` to get those details (`--cluster-name` and `--namespace`) of the Kubernetes cluster to delete it.

```
kubectl get clusters
```

3. Delete the Kubernetes cluster and wait a few minutes.

> **Note:** Before deleting the cluster, Nutanix Kubernetes Platform (NKP) deletes all Services of type LoadBalancer on the cluster. A vSphere Classic ELB backs each Service. Deleting the Service deletes the ELB that backs it. To skip this step, use the flag `--delete-kubernetes-resources=false`. Do not skip this step if # NKP manages the VPC. When NKP deletes the cluster, it deletes the VPC. If the VPC has any vSphere Classic ELBs, vSphere does not allow the VPC to be deleted, and NKP cannot delete the cluster.

```
nkp delete cluster --cluster-name=${CLUSTER_NAME} --kubeconfig $HOME/.kube/config
```

Output:

```
INFO[2022-03-30T11:53:42-07:00] Running cluster delete command
 clusterName=nutanix-e2e-cluster-1 managementClusterKubeconfig= namespace=default
 src="cluster/delete.go:95"
INFO[2022-03-30T11:53:42-07:00] Waiting for cluster to be fully deleted
 src="cluster/delete.go:123"
INFO[2022-03-30T12:14:03-07:00] Deleted default/nutanix-e2e-cluster-1 cluster
 src="cluster/delete.go:129"
```

After the workload cluster is deleted, you can delete the bootstrap cluster.

Delete the Bootstrap Cluster

About this task

After you have moved the workload resources back to a bootstrap cluster and deleted the workload cluster, you no longer need the bootstrap cluster. You can safely delete the bootstrap cluster with these steps:

Procedure

1. Make sure your vSphere credentials are up to date. Refresh the credentials using this command.

   ```
   nkp update bootstrap credentials vSphere --kubeconfig $HOME/.kube/config
   ```

2. Delete the bootstrap cluster.

   ```
   nkp delete bootstrap --kubeconfig $HOME/.kube/config
   ```

   Output:

   ```
   # Deleting bootstrap cluster
   ```

VMware Cloud Director Infrastructure

For an environment that is on the AWS Infrastructure, install options based on those environment variables are provided for you in this location.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Special Resource Requirements for VMware Cloud Director

You can control the virtual machine (VM) resource allocation and placement on a specific cluster or host by using VM sizing policies, VM placement policies, and vGPU policies.

VM Sizing Policy - CPU and Memory

For CPU and Memory, the VMware Cloud Director (VCD) Provider creates the appropriate VM Sizing Policies. The VM Sizing Policy defines CPU and memory resources available to the VM.

1. For CPU and Memory, the VCD Provider must create the appropriate VM Sizing Policies.
2. When creating the cluster, the Provider must reference these VM Sizing Policies by using the `--control-plane-sizing-policy` and `--worker-sizing-policy` flags.
3. See Attributes of VM Sizing Policies regarding parameters like vCPU Speed or CPU Reservation Guarantee that require consideration in VM Sizing Policy. For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.4/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-73C48A9C-79AF-402C-9746-4C81CAC2B35A.html

VM Placement Policy

The VCD Provider (SP) determines the VM Placement Policy For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.4/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-236A070E-83E6-4648-8F2F-557248C9735D.html. This policy defines the infrastructure where the VM runs, meaning its placement of a VM on a host. When you assign a VM Placement Policy to a virtual machine, the placement engine adds this virtual machine to the corresponding VM group of the cluster on which it resides.

Storage

For storage, follow Create a Base OS image in vSphere vCenter when creating the KIB base image. In VCD, the base image determines the minimum storage available for the VM.

VMware Cloud Director Prerequisites

Before continuing to install Nutanix Kubernetes Platform (NKP) on Cloud Director, verify that your VMware vSphere Client environment is running vSphere Client version v7.0.3 or later version with ESXi. You must reach the vSphere API endpoint from where the Konvoy command line interface (CLI) runs and have a vSphere account containing Administrator privileges. A RedHat subscription is required with a username and password for downloading DVD ISOs and valid vSphere values for the following: vCenter API server URL, Datacenter name, and Zone name that contains ESXi hosts for your cluster's nodes. For more infirmation, see https://docs.vmware.com/en/VMware-vSphere/7.0/com.vmware.vsphere.vm_admin.doc/GUID-55238059-912E-411F-A0E9-A7A536972A91.html and https://docs.vmware.com/en/VMware-vSphere/index.html.

- Ensure you have met VMware Cloud Director (VCD) requirements: VMware Cloud Director 10.4 Release Notes
- Download and install VMware Cloud Director: Download VMware Cloud Director
- Considerations regarding CPU and Memory during configuration:
  - For CPU and Memory, the VCD Provider creates the appropriate VM Sizing Policies.
  - The Provider (or tenant user with proper permissions) references these *VM Sizing Policies* when creating the cluster, using the `--control-plane-sizing-policy` and `--worker-sizing-policy` flags.
  - See Attributes of VM Sizing Policies regarding parameters like vCPU Speed or CPU Reservation Guarantee that require consideration in VM Sizing Policy. For example, the recommended vCPU minimum speed is at least 3 GHz.
- Considerations for Storage during configuration:
  - For storage, follow Base OS image in vSphere when creating the KIB base image. In VCD, the base image determines the minimum storage available for the VM.

VCD Overview

VMware Cloud Director (VCD) provides a private cloud-like experience in data centers. VCD allows the creation of virtual data centers for more self-service administration tasks.

VMware Cloud Director is a platform that turns physical datacenter into virtual data centers and runs on one or more vCenters. After you have vSphere vCenter, a brief overview of the general workflow from this section of the documentation is below:

- Use vCenter to create and configure virtual infrastructure, including:
  - Creating the Organization (tenant) and its users. For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-2F217F99-48C1-42F3-BF06-5ABBACADE2BA.html
  - Creating Roles, Rights, and other related permissions necessary for the tenant users and other software components
  - Creating Base Images and VM Templates that will be uploaded to the vApp catalog For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Tenant-Portal-Guide/GUID-D5737821-C3A4-4C73-8959-CA293C12A7DE.htmlof the VCD tenant Organization
- Use Nutanix Kubernetes Platform (NKP) to create and manage clusters and node pools

An overview of the steps has been provided to help. The overall process for configuring VCD and NKP together includes the following steps:

1. Configure vSphere to provide the elements described in the vSphere Prerequisites.
2. **For air-gapped environments**: Create a Creating a Bastion Host on page 685.
3. Create a base OS image (for use in the OVA package containing the disk images packaged with the OVF).
4. Create a CAPI VM image template that uses the base OS image and adds the needed Kubernetes cluster components.
5. Create a Tenant Organization
6. Configure Virtual Data Centers (VDCs)
7. MSP will upload the appropriate OVA to the tenant organization's catalog
    1. MSP uses `konvoy-image build vsphere` to create an OVA
    2. MSP creates a vApp from the OVA
    3. Before the MSP creates a cluster in a VCD tenant, it makes this vApp available to that tenant
8. The user must be able to reach the VCD API endpoint from where the NKP CLI runs
9. Create a new self-managing NKP cluster on VCD.
10. Install Kommander
11. Verify and log in to the UI

> **Note:** To see a visual architecture structure related to Tenants, Organization(or Tenant) VDCs, and Provider VDCs for VMWare Cloud Director, refer to the page linked here for their documentation under the title: New to VMware Cloud Director. For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/index.html

Cloud Director Concepts and Terms

Before attempting to create a Nutanix Kubernetes Platform (NKP) cluster in VMware Cloud Director (VCD), we recommend familiarizing yourself with the following concepts and related terminology.

VMware Cloud Director Concepts

The VMware Cloud Director is based on the following concepts:

- **Provider**: Service Provider(SP) that administrates the data centers and provisions virtual infrastructure for Organizations(tenants).
- **Organizations (Tenants)**: An administration unit for users, groups, and computing resources. Tenant users are managed at the **Organization** level.
- **System Administrators**: This role exists only in the provider organization (SP) and can create and provision tenant # organizations# and the portal. The #System Administrator# role has all VMware Cloud Director rights by default.
- **Organization Administrators**: This role creates user groups and service catalogs. Tenant is a predefined role that can manage users in their **Organization** and assign them roles.
- **Rights**: Each right provides view or manage access to a particular object type in VCD. Also see:
- **Rights Bundle**: A collection of rights for the **Organization** .
- **Roles**: A role is a set of rights assigned to one or more users and groups. When you create or import a user or group, you must assign it a role.
- **Users and Groups**: Administrators can create users manually, programmatically, or integrate with a directory service like LDAP to import user accounts and groups at scale.
- **Virtual Data Centers (VDC)**: An isolated environment provided to a cloud user in which they can provision resources, deploy, store, and operate applications.

- **Organization VMware Cloud Director Networks**: Similar to the Amazon concept of Virtual Private Cloud, a VMware Cloud Director network is available only to a specific VMware Cloud Director and available to all vApps in the **Organization**. It can be connected to external networks as needed.
- **vApp Networks**: Similar to the concept of a subnet, a vApp network is an isolated network within a VMware Cloud Director network that allows specific vApps to communicate with each other.
- **vApp**: One or more virtual machines(VMs) that come preconfigured to provide a specific cloud service. vApp is a virtual app that defines computing, storage, and networking metadata.
- **Media Files and Catalogs**: VMware Cloud Director organizes deployable resources through media files. Virtual machines and vApp templates (machine images) can be used as an initial boot program for a VM. The **Organization Administrator** organizes these files into catalogs, allowing users within the **Organization** to provision the resources they need.
- **Storage Profiles**: VCD concept to organize storage For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.5/VMware-Cloud-Director-Tenant-Guide/GUID-17DDD3AC-0ABB-49EC-B3AE-28DFB2D2B80B.html?hWord=N4IghgNiBcIMoBcD2AnMBzApgAgMITAGdCQBfIA (ex. Gold, Platinum)
- **NSXT gateway**: A Logical Router configured in a traditional hardware switch. Gateways provide connectivity to external networks and between different logical networks.
- **Tier-0 Gateway**: The Tier-0 gateway connects external networks using static routes or Border Gateway Protocol (BGP). The Tier-0 gateway primarily handles the North-South traffic between the virtualized environment and the external physical network.
- **Tier-1 Gateway**: The Tier-1 Gateway acts as a tenant router. The Tier-1 gateway is optimized for East-West traffic.
- **Edge Gateway**: A gateway that provides a VDC with connectivity and other features. It can provide NAT, firewall, and other network features.
- **Provider Gateway**: A logical gateway representing Tier-0 gateway managed by the SP.
- **Organization Edge Gateway**: An organization-specific gateway in the Provider Gateway. They are created using network segments in the provider Gateway.

Port Permissions

There are various VCD abstractions such as Edge Gateway Firewall Rules, IP sets, and Security Groups that work together. For many Kubernetes components in your Nutanix Kubernetes Platform (NKP) cluster, access is required. Allow access to ports 6443 and 443 from outside the cluster. There are other ports listed in the NKP ports page that must be allowed among the machines in the cluster.

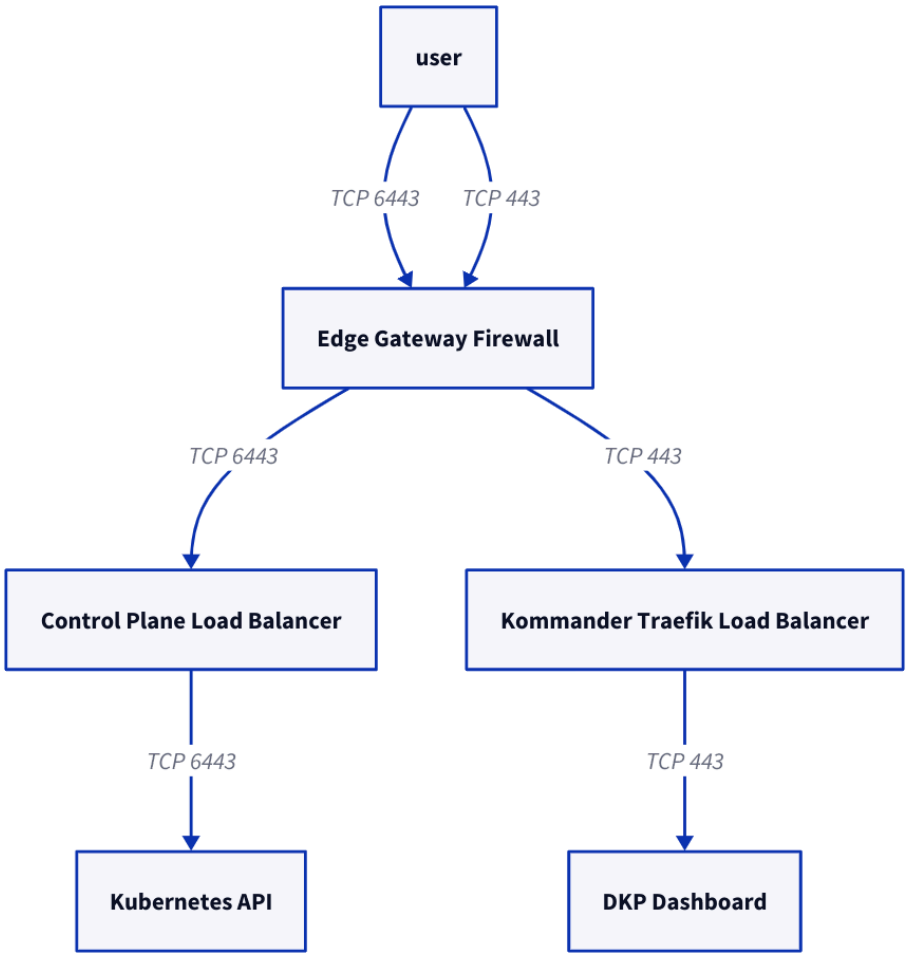


Figure 27: VMware Port Permissions

Cloud Director Configure the Organization

The following steps document the minimum configuration needed when creating and configuring an Organization(tenant) for use with Nutanix Kubernetes Platform (NKP). For additional tenant organization configuration information, see VMware's VCD documentation at https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-1AE706A9-79F2-4A0E-8B0D-FFE9E87109CD.html f.

The two main sections of the VMware Cloud Director documentation are for system administration and the tenant configuration and access:

- **Service Provider Admin Portal**: https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-F8F4B534-49B2-43B2-AEEE-7BAEE8CE1844.html

- **Organization (tenant) Portal**: https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Tenant-Portal-Guide/GUID-120992A9-4FCB-4900-B19C-9AACFCB3F40B.html

Prerequisites to Create and Configure an Organization

1. Add a vCenter Server to VCD - Log in to the Cloud Director portal, select the **Resources** tab across the top menu - **Infrastructure Resources - vCenter Server Instances and select** and select **ADD** to begin the process of connecting your vCenter server by following the on-screen instructions.
2. Create Organization (Tenant) - create a new organization from the VMware Cloud Director Admin Portal.

Configuring the Organization
How to configure the organization or tenant.

About this task

Ensure you have completed the prerequisites, then continue with organization creation.

Procedure

1. Create the Organization's VDCs: For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-09851831-142E-46B9-A278-6488784D5B6A.html
2. Configure Edge Gateway: For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-45C0FEDF-84F2-4487-8DB8-3BC281EB25CD.html
3. Create the Tenant Network: For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Tenant-Portal-Guide/GUID-8F806B38-2489-4D36-82FF-B23BAFC3B294.html
4. Configure Policies: For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-DBA11D01-E102-47A4-926C-BFDB681B75F2.html

Use System Administrator Menu

About this task

After a tenant Organization is created, use the menus in the System Administrator portal to configure the following settings:

Procedure

In the System Administration menu.

- Under the Data Center tab, select Virtual Data Center. This is the location where you can define CPU size, memory, and storage.
- Under the Data Center tab - Networking - Edges: select Configuration-External Networks and supply the publicly accessible IP address range under the Subnet column.

- Under the Libraries tab - Content Libraries in the left menu: specify the vApp Templates to import the VM Templates from vCenter that you want to make available to the tenant. EX: KIB templates from vCenter

  Note: The Service Provider (SP) can create a shared Catalog where items are placed to be automatically imported.

- Under the Networking tab - Networks: the tenant Organization Administrator will configure a network. Select the network name to be taken to its General properties, such as Gateway CIDR address, where all VMs will receive an IP address from the private IP space.

  Note: The LoadBalancer (LB) will use the routable external network, which is automatically created using the CAPVCD controller.

- Under the Resources tab - Edge Gateway - Services - NAT - External IP: Specify the IP address that allows VMs to access external networks. Either create an SNAT rule or provide Egresses to the VMs.
- Edge Gateway Firewall

  Important: —Allow port access from outside the cluster for TCP 6443 to the control plane endpoint load balancers and TCP 443 to Kubernetes load balancers (e.g., to reach the ##Nutanix Kubernetes Platform # (##NKP#) dashboard). Other ports must be allowed access among the cluster machines. The tenant is required to get one public IP to create an Edge Gateway. The Service Provider(SP) will allocate the pool of IPs from which the tenant pulls. After you have associated an external network through a gateway, the tenant can ask for IPs. Otherwise, if you have chosen an IP address, you can specify it.

  Note: After the tenant Organization is in production, various policies will need to be defined for storage and resources. The Configure Organization Policy Section of VMware documentation will provide more detail. For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-DBA11D01-E102-47A4-926C-BFDB681B75F2.html

Cloud Director Roles, Rights, and Rights Bundles

VMware Cloud Director (VCD) has a variety of ways to assign permissions. As a Service Provider (Cloud Provider), you have numerous options for giving tenants access to VMware Cloud Director features. There are:

- **Rights**: Provide view or manage access to a particular object type in VMware Cloud Director and belong to different categories depending on the objects to which they relate, such as vApp, Catalog, or an Organization. For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.4/VMware-Cloud-Director-Tenant-Portal-Guide/GUID-9B462637-E350-43B6-989A-621F226A56D4.html.
- **Roles**: A collection of rights for a User and defines what an individual user has access to. For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.4/VMware-Cloud-Director-Tenant-Portal-Guide/GUID-9B462637-E350-43B6-989A-621F226A56D4.html.
- **Rights Bundles**: A collection of rights for the tenant Organization as a whole and defines what a tenant Organization can access. For more information, see https://blogs.vmware.com/cloudprovider/2019/12/effective-rights-bundles.html.

Various Rights are common to multiple predefined Global Roles. These Rights are granted by default to all new organizations and are available for use in other Roles created by the tenant Organization Administrator. The VMware documentation explains some predefined Roles for both Provider and Tenant.

Service Provider(SP) System Administrator

The System Administrator role exists only in the provider organization. As a Service Provider(SP), you will have vSphere vCenter and VMware Cloud Director(VCD) roles.

- vCenter/NXT/AVI Infrastructure System Administrator: Manages physical infra for vCenter, NXT network fabric, AVI load balancers (Nutanix SRE / Service provider(OVH cloud))
- System Administrator - Provider(SP): Manages Virtual infra in VCD that uses vCenter(s), NXT(s), AVI(s) etc. (Nutanix SRE)
- Organization(Tenant) Administrator: Manages Virtual infra (org, orgvdc, network, catalogs, templates, users, etc.) for a tenant. Users that can create k8s cluster

Through the VMware Cloud Director(VCD) Service Provider Admin Portal, the SP can add System Administrators for Cloud Director and see the predefined list of rights in any role. The System Administrator manages the virtual infrastructure in VCD that uses vCenter(s), NXT(s), AVI(s)

, and other components of the VCD environment.For more information on Managing System Administrators and Roles from the VMware documentation site, see.

Organization (Tenant) Administrator

As an Organization Administrator, you can create, edit, import, and delete users from the tenant portal. The tenant Organization Administrator manages the virtual infrastructure that includes the organization itself, which consists of the related network, catalogs, templates, and such for the Tenant Organization.

The Tenant Organization Administrator is predefined and can use the VCD tenant portal to create and manage users in their organization and assign them roles.

A Tenant Organization Administrator can access roles if allowed. They can only view the global tenant roles that a System Administrator has published to the organization but cannot modify them. The Organization Administrator can create custom tenant roles with similar rights and assign them to the users within their own tenant Organization.

Tenant Roles and Rights

Several predefined global tenant roles are described in the VMware documentation, including which components they can access and change. User rights may be available to a role, but a rights bundle needs to be published to a tenant organization for those user role permissions to work. For more information, see:

1. Create a Rights Bundle (a collection of rights) that includes all required Rights. See the page for the List of Rights on page 974.
2. Create a Tenant Role that uses all these Rights.

The SP needs to publish these to tenant Organizations that need to create clusters. Assign the Tenant Role to a VCD user and create an API Token. The API token will be used in the Nutanix Kubernetes Platform (NKP) CLI commands to authenticate the CLI to vCenter. They will assign the Role to a VCD User, create an API Token, and pass the Token to the NKP CLI.

For more information, see the VMware documentation links:

- **Create Users**:https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Tenant-Portal-Guide/GUID-1CACBB2E-FE35-4662-A08D-D2BCB174A43C.html
- **Managing System Administrators and Roles**:https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-9DFB2238-23FB-4D07-B563-144AC4E9EDAF.html
- **Manage Users**:https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Tenant-Portal-Guide/GUID-A358E190-BFC0-4187-9406-66E82C92564A.html
- **Organization Administrator**:https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Tenant-Portal-Guide/GUID-BC504F6B-3D38-4F25-AACF-ED584063754F.html#GUID-BC504F6B-3D38-4F25-AACF-ED584063754F

- **Predefined Roles and Their Rights**:https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-BC504F6B-3D38-4F25-AACF-ED584063754F.html
- **Rights Bundle**:https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-CFB0EFEE-0D4C-498D-A937-390811F11B8E.html
- **System Administrator Rights**:https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-438B2F8C-65B0-4895-AF40-6506E379A89D.html
- **Tenant Organization Administrator**:https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Tenant-Portal-Guide/GUID-74C9E10D-9197-43B0-B469-126FFBCB5121.html
- **Tenant Role**:https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-0D991FCF-3800-461D-B123-FAE7CFF34216.html
- **Tenant Roles and Rights**:https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-AE42A8F6-868C-4FC0-B224-87CA0F3D6350.html
- **vApp Author**: role can use catalogs and create vApps https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Tenant-Portal-Guide/GUID-BC504F6B-3D38-4F25-AACF-ED584063754F.html.
- **VMware Cloud Director™ Service Provider Admin Portal Guide**: https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-F8F4B534-49B2-43B2-AEEE-7BAEE8CE1844.html

Related Information

The CAPVCD provider uses a related component called CSE. Some of the permissions necessary to create a VCD cluster are defined using this component. Note that the term Role Based Access Control (RBAC) used in the CSE documentation refers ONLY to the VCD rights and permissions necessary to perform life cycle management of Kubernetes clusters using VCD. It has no impact on the RBAC configuration of any clusters created using VCD

Role Based Access Control (RBAC) from GitHub - The RBAC on that page refers to the roles and rights required for the tenants to perform the life cycle management of Kubernetes clusters. If anything has nothing to do with the RBAC inside the Kubernetes cluster itself.

For more information, see the related RBAC links:

- CSE: https://vmware.github.io/container-service-extension/cse3_1/INTRO.html
- Role Based Access Control (RBAC): https://vmware.github.io/container-service-extension/cse3_1/RBAC.html#additional-required-rights

Service Provider (SP) System Administrator

The System Administrator role exists only in the provider organization. For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-438B2F8C-65B0-4895-AF40-6506E379A89D.html

As a Service Provider (SP), you will have vSphere vCenter and VMware Cloud Director (VCD) roles.

- **vCenter, NXT or AVI Infrastructure System Administrator:** Manages physical infra for vCenter, NXT network fabric, AVI load balancers (Nutanix SRE or Service provider (OVH cloud))
- **System Administrator - Provider (SP):** Manages Virtual infra in VCD that uses vCenter(s), NXT(s), AVI(s), etc. (Nutanix SRE)

- **Organization(Tenant) Administrator:** Manages Virtual infra (org, orgvdc, network, catalogs, templates, users, etc.) for a tenant. Users that can create k8s cluster

Through the VMware Cloud Director (VCD) Service Provider Admin Portal, the SP can add **System Administrators** for Cloud Director and see the predefined list of rights included in any role. The **System Administrators** manage the virtual infrastructure in VCD that uses vCenter(s), NXT(s), AVI(s), and other components of the VCD environment. For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-F8F4B534-49B2-43B2-AEEE-7BAEE8CE1844.html.

Also, refer to Managing System Administrator and Roles at https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-9DFB2238-23FB-4D07-B563-144AC4E9EDAF.html documentation from the VMware documentation site.

Organization (Tenant) Administrator

As an **Organization Administrator**, from the tenant portal you can create, edit, import, and delete users. The tenant organization administrator manages the virtual infrastructure that includes the organization itself, including the related network, catalogs, templates, and such for the tenant organization. Important predefined role information is below:

- Tenant Organization Administrator is predefined and can use the VCD tenant portal to create and manage users in their organization and assign them roles.
- vApp Author role can use catalogs and create vApps

A tenant **Organization Administrator** can access roles if allowed. They can only view the global tenant roles that a **System Administrator** has published to the organization but cannot modify them. The **Organization Administrator** can create **custom** tenant roles with similar rights and assign them to the users within their tenant **Organization**.

There are some predefined Global Tenant Roles as well, which are explained in the VMware documentation: https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-BC504F6B-3D38-4F25-AACF-ED584063754F.html as well as the https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-AE42A8F6-868C-4FC0-B224-87CA0F3D6350.html.

Creating Tenant Roles and Rights

About this task

Several predefined global tenant roles are described in the VMware documentation regarding which components they can access and change. User rights may be available to a role, but a rights bundle needs to be published to a tenant organization for those user role permissions to work.

The SP needs to publish these to tenant **Organizations** that need to create clusters. Assign the Tenant Role to a VCD user and create an API Token. The API token will be used in the Nutanix Kubernetes Platform (NKP) CLI commands to authenticate the CLI to vCenter. They will assign the Role to a VCD User, create an API Token, and pass the Token to the NKP CLI.

Procedure

1. Create a Rights Bundle (a collection of rights) that includes all required Rights. See the page for the Creating Tenant Roles and Rights on page 972. For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-CFB0EFEE-0D4C-498D-A937-390811F11B8E.html.

2. Create a Tenant Role that uses all these Rights. For more information, see https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-0D991FCF-3800-461D-B123-FAE7CFF34216.html.

Related Information

For information on related topics or procedures, see:

- The CAPVCD provider uses a related component called CSE; see https://vmware.github.io/container-service-extension/cse3_1/INTRO.html.
- Permissions necessary to create a VCD cluster are defined using this component. Note that the term Role Based Access Control (RBAC) used in the CSE documentation refers ONLY to the VCD rights and permissions necessary to perform life cycle management of Kubernetes clusters using VCD. It does not impact the RBAC configuration of any clusters created using VCD. For more information, see https://vmware.github.io/container-service-extension/cse3_1/RBAC.html#additional-required-rights.
- Role Based Access Control (RBAC) from GitHub - The RBAC in that page refers to the roles and rights required for the tenants to manage Kubernetes clusters' life cycle. It does not have anything to do with the RBAC inside the Kubernetes cluster itself; see https://vmware.github.io/container-service-extension/cse3_1/RBAC.html#additional-required-rights.

Cloud Director CAPVCD User Rights

CAPVCD requires specific user rights set for the tenant organization to have a role that can successfully execute ##Nutanix Kubernetes Platform# (##NKP#) clusters, and those rights must be specified. When creating an ##NKP# workload cluster machine, it must register with VCD Cloud Provider Interface (CPI) and get node references.

The remainder of this page provides more information about what the CAPVCD User requires regarding Rights and Rights Bundles.

- Terminology related to Rights and Roles: Managing Rights and Roles
- VCD Cloud Provider Interface(CPI): https://github.com/vmware/cloud-provider-for-cloud-director/tree/main#terminology

CAPVCD User

CAPVCD uses the credentials (username, password, or API token) of a VCD User to manage the cluster. The VCD Cloud Provider Interface(CPI) and CSI controllers also use the same credentials. This same user needs specific API permissions, known as Rights, for the CAPVCD, CPI, and CSI controllers to work correctly. For a User to be granted a Right, the User must be associated with a Role that consumes this Right, meaning the Role grants Rights to the User.

If the User belongs to an Organization, the Provider must publish the Right to the Tenant Organization in a Rights Bundle. The Rights Bundle grants Rights to the tenant Organization.

For more information, see https://github.com/vmware/cloud-provider-for-cloud-director/tree/main#terminology.

Creating the VCD User Required by CAPVCD

This page describes what the CAPVCD User requires about the Rights and Rights Bundles.

About this task

Cluster API for VMware Cloud Director (CAPVCD) requires all the Rights in the default vApp Author Global Role, plus the Rights needed for the VMware Cloud Director (VCD) Cloud Provider Interface (CPI), the VCD CSI, and some Rights required by CAPVCD itself. The procedure for Creating a Rights Bundle is found in VMware Documentation: Create a Rights Bundle at https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-CFB0EFEE-0D4C-498D-A937-390811F11B8E.html and the steps for creating the Role: Create a Global Tenant Role at https://docs.vmware.com/en/VMware-Cloud-Director/10.3/VMware-Cloud-Director-Service-Provider-Admin-Portal-Guide/GUID-0D991FCF-3800-461D-B123-FAE7CFF34216.html

Procedure

1. A provider administrator creates a rights bundle that enumerates all the rights listed below. We recommend the name Nutanix Kubernetes Platform (NKP) Cluster Admin for the Rights Bundle.
2. A Provider administrator creates a Global Role that enumerates all the below rights. We recommend the name NKP Cluster Admin for the Global Role.
3. A Provider administrator publishes both the Rights Bundle and Global Role to every Organization that will deploy NKP clusters
4. An Organization administrator creates a User and associates it with the Global Role.

List of Rights

Cluster API for VMware Cloud Director (CAPVCD) requires the following Rights:

- The Rights cataloged by the predefined `vApp Author`Global Role: https://docs.vmware.com/en/VMware-Cloud-Director/10.4/VMware-Cloud-Director-Tenant-Portal-Guide/GUID-AE42A8F6-868C-4FC0-B224-87CA0F3D6350.html#GUID-AE42A8F6-868C-4FC0-B224-87CA0F3D6350
- Additional Rights listed in the CAPVCD documentation: https://github.com/vmware/cloud-director-named-disk-csi-driver/blob/1.3.2/README.md#additional-rights-for-csi.
- Additional Rights listed in the VCD Cloud Provider Interface (CPI) documentation: https://github.com/vmware/cloud-provider-for-cloud-director/blob/1.3.0/README.md#additional-rights-for-cpi.
- Additional Rights listed in the VCD CSI documentation: https://github.com/vmware/cluster-api-provider-cloud-director/blob/b315c9c5e4b1b05600ccec20eb5116cc6173f845/docs/VCD_SETUP.md#publish-the-rights-to-the-tenant-organizations.

The majority of the Rights are from the `vApp Author` Global Role. When independent components, like CAPVCD and CPI, need the same Right, that Right appears in multiple sources.

Below are the lists of the rights from the above sources and the rights required but not documented by CAPVCD. The last list includes the Rights from all sources, with duplicates removed:

Required by the `vApp Author` Role

Some Rights appear in multiple sources. This list includes the Rights from all sources, with duplicates removed in the final list below:

```
Catalog: Add vApp from My Cloud
Catalog: View Private and Shared Catalogs
Organization vDC Compute Policy: View
Organization vDC Disk: View IOPS
Organization vDC Named Disk: Create
Organization vDC Named Disk: Delete
Organization vDC Named Disk: Edit Properties
```

```
Organization vDC Named Disk: View Encryption Status
Organization vDC Named Disk: View Properties
Organization vDC Network: View Properties
Organization vDC: VM-VM Affinity Edit
Organization: View
UI Plugins: View
vApp Template / Media: Copy
vApp Template / Media: Edit
vApp Template / Media: View
vApp Template: Checkout
VAPP_VM_METADATA_TO_VCENTER
vApp: Copy
vApp: Create / Reconfigure
vApp: Delete
vApp: Download
vApp: Edit Properties
vApp: Edit VM Compute Policy
vApp: Edit VM CPU
vApp: Edit VM Hard Disk
vApp: Edit VM Memory
vApp: Edit VM Network
vApp: Edit VM Properties
vApp: Manage VM Password Settings
vApp: Power Operations
vApp: Sharing
vApp: Snapshot Operations
vApp: Upload
vApp: Use Console
vApp: View ACL
vApp: View VM and VM's Disks Encryption Status
vApp: View VM metrics
vApp: VM Boot Options
```

Additional Rights Required by CAPVCD

```
API Tokens: Manage
Organization vDC Gateway: Configure Load Balancer
Organization vDC Gateway: Configure NAT
Organization vDC Gateway: View
vApp: Allow All Extra Config
```

Rights Required to List Catalogs (Not Documented by CAPVCD)

These Rights are not documented by CAPVCD but are required. They allow CAPVCD

to list the Catalogs in the Organization.

Implied Rights (Not Documented by CAPVCD)

These Rights are not documented by CAPVCD but are implied by the documented Rights.

```
General: Administrator View
Access All Organization VDCs
```

Additional Rights Required by CPI

```
API Tokens: Manage
Organization vDC Gateway: Configure Load Balancer
Organization vDC Gateway: Configure NAT
Organization vDC Gateway: View
```

All Sources Merged with Duplicates Removed

```
Access All Organization VDCs
API Tokens: Manage
Catalog: Add vApp from My Cloud
Catalog: View Private and Shared Catalogs
Certificate Library: View
General: Administrator View
Organization vDC Compute Policy: View
Organization vDC Disk: View IOPS
Organization vDC Gateway: Configure Load Balancer
Organization vDC Gateway: Configure NAT
Organization vDC Gateway: View
Organization vDC Gateway: View Load Balancer
Organization vDC Gateway: View NAT
Organization vDC Named Disk: Create
Organization vDC Named Disk: Delete
Organization vDC Named Disk: Edit Properties
Organization vDC Named Disk: View Encryption Status
Organization vDC Named Disk: View Properties
Organization vDC Network: View Properties
Organization vDC Shared Named Disk: Create
Organization vDC: VM-VM Affinity Edit
Organization: View
UI Plugins: View
vApp Template / Media: Copy
vApp Template / Media: Edit
vApp Template / Media: View
vApp Template: Checkout
VAPP_VM_METADATA_TO_VCENTER
vApp: Allow All Extra Config
vApp: Copy
vApp: Create / Reconfigure
vApp: Delete
vApp: Download
vApp: Edit Properties
vApp: Edit VM Compute Policy
vApp: Edit VM CPU
vApp: Edit VM Hard Disk
vApp: Edit VM Memory
vApp: Edit VM Network
vApp: Edit VM Properties
vApp: Manage VM Password Settings
vApp: Power Operations
vApp: Sharing
vApp: Snapshot Operations
vApp: Upload
vApp: Use Console
vApp: View ACL
vApp: View VM and VM's Disks Encryption Status
vApp: View VM metrics
vApp: VM Boot Options
```

Cloud Director Install NKP

Before continuing to install Nutanix Kubernetes Platform (NKP) on Cloud Director, verify that your VMware vSphere Client environment is running vSphere v7.0.3 or later versions with ESXi. For more information, see https://docs.vmware.com/en/VMware-vSphere/7.0/com.vmware.vsphere.vm_admin.doc/GUID-55238059-912E-411F-A0E9-A7A536972A91.html. You must reach the vSphere API endpoint from where the Konvoy command line interface (CLI) runs and have a vSphere account containing Administrator privileges. A RedHat subscription is required with a username and password for downloading DVD ISOs

and valid vSphere values for the following: vCenter API server URL, Datacenter name, and Zone name that contains ESXi hosts for your cluster's nodes.

vSphere Prerequisites

- Resource Requirements
- Installing NKP on page 50
- Prerequisites for Install

Cloud Director Create Image and Template

As a vSphere vCenter administrator, you will create the images for your VMWare Cloud Director (VCD) tenants.

1. The vSphere Base OS Image and VM Template are made in vCenter for Ubuntu 20.04. The base image determines the minimum storage available for the VM.
2. An OVA is created from the image for each VCD tenant accordingly.
3. That image is exported from vSphere and imported into the VCD tenant Organization catalog of vApp Templates.

Refer to the VMWare documentation for specifics on creating and deploying OVA for your tenants: Deploying OVF and OVA TemplatesDeploying OVF and OVA Templates and Working with vApp Templates.

If you have not already done so, create your base image and template in vSphere and import them into the VCD tenant catalog to make them available for tenant use.

Cloud Director Bootstrap the Cluster

To create Kubernetes clusters, Nutanix Kubernetes Platform (NKP) uses Cluster API (CAPI) controllers. These controllers run on a Kubernetes cluster. To get started, you need a bootstrap cluster. Bootstrapping creates a new Kubernetes cluster from scratch and involves setting up the control plane and worker nodes. It also determines which node has the correct information for synchronization with all other nodes.

Bootstrapping Cluster Life Cycle Services

Create a bootstrap cluster.

About this task

To get started, you need a bootstrap cluster. By default, Nutanix Kubernetes Platform (NKP) creates a bootstrap cluster for you in a Docker container using the Kubernetes-in-Docker (KIND) tool.

Before you begin

Procedure

1. Complete the Nutanix Infrastructure Prerequisites. For more information, see Nutanix Infrastructure Prerequisites on page 690.
2. Ensure the `NKP` binary can be found in your `$PATH`.

Bootstrap Cluster Life cycle Services

Procedure

1. Review Universal Configurations for all Infrastructure Providers regarding settings, flags, and other choices and then begin bootstrapping.
2. Create a bootstrap cluster using the command `nkp create bootstrap --kubeconfig $HOME/.kube/config`.

   > **Note:** Use `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Configuring an HTTP or HTTPS Proxy on page 677.

   Example output:

   ```
   # Creating a bootstrap cluster
   # Initializing new CAPI components
   ```

3. NKP creates a bootstrap cluster using KIND as a library.

   For more information, see https://github.com/kubernetes-sigs/kind.

4. NKP then deploys the following Cluster API providers on the cluster.

   - Core Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/
   - AWS Infrastructure Provider: https://github.com/kubernetes-sigs/cluster-api-provider-aws
   - Kubeadm Bootstrap Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/bootstrap/kubeadm
   - Kubeadm ControlPlane Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/controlplane/kubeadm

   For more information on Cluster APIs, see https://cluster-api.sigs.k8s.io/.

5. Ensure that the CAPV controllers are present using the command `kubectl get pods -n capv-system`.

   Output example:

   ```
   NAME                                        READY   STATUS    RESTARTS   AGE
   capv-controller-manager-785c5978f-nnfns   1/1     Running   0          13h
   ```

6. NKP waits until these providers' controller-manager and webhook deployments are ready. List these deployments using the command `kubectl get --all-namespaces deployments -l=clusterctl.cluster.x-k8s.io`.

   Output example:

   ```
   NAMESPACE                           NAME
    READY   UP-TO-DATE   AVAILABLE   AGE
   capa-system                         capa-controller-manager
    1/1     1            1           1h
   capg-system                         capg-controller-manager
    1/1     1            1           1h
   capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-manager
    1/1     1            1           1h
   capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-manager
    1/1     1            1           1h
   capi-system                         capi-controller-manager
    1/1     1            1           1h
   ```

```
cappp-system                        cappp-controller-manager
 1/1     1            1           1h
capv-system                         capv-controller-manager
 1/1     1            1           1h
capz-system                         capz-controller-manager
 1/1     1            1           1h
cert-manager                        cert-manager
 1/1     1            1           1h
cert-manager                        cert-manager-cainjector
 1/1     1            1           1h
cert-manager                        cert-manager-webhook
 1/1     1            1           1h
```

Cloud Director Create a New Cluster

Before creating your new cluster, confirm you can reach any IPs in the range allocated to the load balancer pool so the CAPVCD drivers can reach it during cluster creation. The bootstrap cluster will connect to the CAPVCD controller and connect to the load balancer (LB) in VCD using `sshuttle`. The bootstrap IP address will connect to the load balancer IP to deploy the Container Network Interface (CNI).

Flags Specific to VMware Cloud Director Cluster Creation

When creating a VCD cluster, CPU and Memory flags are needed:

- For CPU and Memory, the VCD Provider creates the appropriate VM Sizing Policies. Then, the Provider references these VM Sizing Policies when creating the cluster, using the flags:
  - `--control-plane-sizing-policy`
  - `--worker-sizing-policy`

If the Service Provider(SP) has given a tenant user the permissions to create clusters inside their own Organization, then that tenant user will need to reference those flags are well.

Making the New VCD Cluster Self-Managed

How to make a Kubernetes cluster manage itself.

About this task

Nutanix Kubernetes Platform (NKP) deploys all cluster life cycle services to a bootstrap cluster, which then deploys a workload cluster. When the workload cluster is ready, move the cluster life cycle services to the workload cluster, which makes the workload cluster self-managed.

Before you begin

Before starting, ensure you can create a workload cluster as described in the topic: .

This page contains instructions on how to make your cluster self-managed. This is necessary if there is only one cluster in your environment or if this cluster becomes the Management cluster in a multi-cluster environment.

> **Note:** If you already have a self-managed or Management cluster in your environment, skip this page.

Follow these steps to turn your new cluster into a **Management Cluster** for an Ultimate license environment (or a free-standing Pro Cluster):

Procedure

1. Deploy cluster life cycle services on the workload cluster.

   ```
   nkp create capi-components --kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Initializing new CAPI components
   ```

   > **Note:** If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP/HTTPS Proxy.

2. The cluster life cycle services on the workload cluster are ready, but the workload cluster configuration is on the bootstrap cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the bootstrap to the workload cluster. This process is called a Pivot. For more information, see https://cluster-api.sigs.k8s.io/reference/glossary.html?highlight=pivot#pivot.

   Move the Cluster API objects from the bootstrap to the workload cluster:

   ```
   nkp move capi-resources --to-kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Moving cluster resources
   You can now view resources in the moved cluster by using the --kubeconfig flag with
    kubectl. For example: kubectl --kubeconfig=gcp-example.conf get nodes
   ```

   > **Note:** To ensure only one set of cluster life cycle services manages the workload cluster, NKP first pauses the reconciliation of the objects on the bootstrap cluster, then creates the objects on the workload cluster. As NKP copies the objects, the cluster life cycle services on the workload cluster reconcile the objects. The workload cluster becomes self-managed after NKP creates all the objects. If it fails, the `move` command can be safely retried.

3. Wait for the cluster control plane to be ready.

   ```
   kubectl --kubeconfig ${CLUSTER_NAME}.conf wait --for=condition=ControlPlaneReady
    "clusters/${CLUSTER_NAME}" --timeout=20m
   ```

   Output:

   ```
   cluster.cluster.x-k8s.io/gcp-example condition met
   ```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster status. After moving the cluster life cycle services to the workload cluster, remember to use NKP with the workload cluster `kubeconfig`.

   ```
   nkp describe cluster --kubeconfig ${CLUSTER_NAME}.conf -c ${CLUSTER_NAME}
   ```

   Output:

   ```
   NAME                                                                        READY
    SEVERITY  REASON  SINCE  MESSAGE
   Cluster/vcd-example                                                         True
                 14s
   ##ClusterInfrastructure - vcdCluster/vcd-example
   ##ControlPlane - KubeadmControlPlane/vcd-example-control-plane              True
                 14s
   # ##Machine/vcd-example-control-plane-6fbzn                                 True
                 17s
   # # ##MachineInfrastructure - vcdMachine/vcd-example-control-plane-62g6s
   # ##Machine/vcd-example-control-plane-jf6s2                                 True
                 17s
   ```

```
# # ##MachineInfrastructure - vcdMachine/vcd-example-control-plane-bsr2z
# ##Machine/vcd-example-control-plane-mnbfs                              True
              17s
#   ##MachineInfrastructure - vcdMachine/vcd-example-control-plane-s8xsx
##Workers
  ##MachineDeployment/vcd-example-md-0                                   True
              17s
    ##Machine/vcd-example-md-0-68b86fddb8-8glsw                          True
              17s
    # ##MachineInfrastructure - vcdMachine/vcd-example-md-0-zls8d
    ##Machine/vcd-example-md-0-68b86fddb8-bvbm7                          True
              17s
    # ##MachineInfrastructure - vcdMachine/vcd-example-md-0-5zcvc
    ##Machine/vcd-example-md-0-68b86fddb8-k9499                          True
              17s
    # ##MachineInfrastructure - vcdMachine/vcd-example-md-0-k8h5p
    ##Machine/vcd-example-md-0-68b86fddb8-l6vfb                          True
              17s
      ##MachineInfrastructure - vcdMachine/vcd-example-md-0-9h5vn
```

5. Remove the bootstrap cluster because the workload cluster is now self-managed.

```
nkp delete bootstrap --kubeconfig $HOME/.kube/config
# Deleting bootstrap cluster
```

Known Limitations

Procedure

- NKP only supports moving all namespaces in the cluster; NKP does not support migration of individual namespaces.
- Konvoy supports moving only one set of cluster objects from the bootstrap cluster to the workload cluster or vice-versa.

Exploring the VCD Cluster

This guide explains how to use the command line to interact with your newly deployed Kubernetes cluster.

About this task

Before you start, ensure you have created a workload cluster, as described in .

Procedure

1. When the workload cluster is created, the cluster life cycle services generate a `kubeconfig` file for the workload cluster and write it to a Secret. The `kubeconfig` file is scoped to the cluster administrator. Get a `kubeconfig` file for the workload cluster.

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

2. Create a StorageClass with a vSphere datastore.

   a. Access the Datastore tab in the vSphere client and select a datastore by name.

   b. Copy the URL of that datastore from the information dialog displayed.

   c. Return to the Nutanix Kubernetes Platform (NKP) CLI, and delete the existing `StorageClass` with the command: `kubectl delete storageclass vsphere-raw-block-sc`

   d. Run the following command to create a new StorageClass, supplying the correct values for your environment.

```
cat <<EOF > vsphere-raw-block-sc.yaml
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
  name: vsphere-raw-block-sc
provisioner: csi.vsphere.vmware.com
parameters:
  datastoreurl: "<url>"
volumeBindingMode: WaitForFirstConsumer
EOF
```

3. Verify the API server is up by listing the nodes.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get nodes
```

   Note: The Status may take a few minutes to move to Ready while the Pod network is deployed. The Nodes' Status will change to Ready soon after the calico-node DaemonSet Pods are Ready.

   Output:

```
NAME                                STATUS   ROLES                  AGE     VERSION
aws-example-control-plane-9z77w     Ready    control-plane,master   4m44s   v1.27.6
aws-example-control-plane-rtj9h     Ready    control-plane,master   104s    v1.27.6
aws-example-control-plane-zbf9w     Ready    control-plane,master   3m23s   v1.27.6
aws-example-md-0-88c46              Ready    <none>                 3m28s   v1.27.6
aws-example-md-0-fp8s7              Ready    <none>                 3m28s   v1.27.6
aws-example-md-0-qvnx7              Ready    <none>                 3m28s   v1.27.6
aws-example-md-0-wjdrg              Ready    <none>                 3m27s   v1.27.6
```

4. List the Pods with the command.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get pods -A
```

   Verify the output:

```
NAMESPACE                NAME
         READY   STATUS     RESTARTS      AGE
calico-system            calico-kube-controllers-57fbd7bd59-qqd96
         1/1     Running    0             20h
calico-system            calico-node-2m524
         1/1     Running    3 (19h ago)   19h
calico-system            calico-node-bbhg5
         1/1     Running    0             20h
calico-system            calico-node-cc5lf
         1/1     Running    2 (19h ago)   19h
calico-system            calico-node-cwg7x
         1/1     Running    1 (19h ago)   19h
calico-system            calico-node-d59hn
         1/1     Running    1 (19h ago)   19h
```

```
calico-system            calico-node-qmmcz
         1/1     Running    0             19h
calico-system            calico-node-wdqhx
         1/1     Running    0             19h
calico-system            calico-typha-655489d8cc-b5jnt
         1/1     Running    0             20h
calico-system            calico-typha-655489d8cc-q92x9
         1/1     Running    0             19h
calico-system            calico-typha-655489d8cc-vjlkx
         1/1     Running    0             19h
kube-system              cluster-autoscaler-68c759fbf6-7d2ck
         0/1     Init:0/1   0             20h
kube-system              coredns-78fcd69978-qn4qt
         1/1     Running    0             20h
kube-system              coredns-78fcd69978-wqpmg
         1/1     Running    0             20h
kube-system              etcd-nutanix-e2e-air-gapped-1-control-plane-7llgd
           1/1     Running    0             20h
kube-system              etcd-nutanix-e2e-air-gapped-1-control-plane-vncbl
           1/1     Running    0             19h
kube-system              etcd-nutanix-e2e-air-gapped-1-control-plane-wbgrm
           1/1     Running    0             19h
kube-system              kube-apiserver-nutanix-e2e-air-gapped-1-control-plane-7llgd
           1/1     Running    0             20h
kube-system              kube-apiserver-nutanix-e2e-air-gapped-1-control-plane-vncbl
           1/1     Running    0             19h
kube-system              kube-apiserver-nutanix-e2e-air-gapped-1-control-plane-wbgrm
           1/1     Running    0             19h
kube-system              kube-controller-manager-nutanix-e2e-air-gapped-1-control-
plane-7llgd   1/1     Running    1 (19h ago)   20h
kube-system              kube-controller-manager-nutanix-e2e-air-gapped-1-control-
plane-vncbl   1/1     Running    0             19h
kube-system              kube-controller-manager-nutanix-e2e-air-gapped-1-control-
plane-wbgrm   1/1     Running    0             19h
kube-system              kube-proxy-cpscs
         1/1     Running    0             19h
kube-system              kube-proxy-hhmxq
         1/1     Running    0             19h
kube-system              kube-proxy-hxhnk
         1/1     Running    0             19h
kube-system              kube-proxy-nsrbp
         1/1     Running    0             19h
kube-system              kube-proxy-scxfg
         1/1     Running    0             20h
kube-system              kube-proxy-tth4k
         1/1     Running    0             19h
kube-system              kube-proxy-x2xfx
         1/1     Running    0             19h
kube-system              kube-scheduler-nutanix-e2e-air-gapped-1-control-plane-7llgd
           1/1     Running    1 (19h ago)   20h
kube-system              kube-scheduler-nutanix-e2e-air-gapped-1-control-plane-vncbl
           1/1     Running    0             19h
kube-system              kube-scheduler-nutanix-e2e-air-gapped-1-control-plane-wbgrm
           1/1     Running    0             19h
kube-system              kube-vip-nutanix-e2e-air-gapped-1-control-plane-7llgd
           1/1     Running    1 (19h ago)   20h
kube-system              kube-vip-nutanix-e2e-air-gapped-1-control-plane-vncbl
           1/1     Running    0             19h
kube-system              kube-vip-nutanix-e2e-air-gapped-1-control-plane-wbgrm
           1/1     Running    0             19h
kube-system              vsphere-cloud-controller-manager-4zj7q
         1/1     Running    0             19h
```

```
kube-system              vsphere-cloud-controller-manager-87tgm
         1/1     Running    0             19h
kube-system              vsphere-cloud-controller-manager-xqmn4
         1/1     Running    1 (19h ago)   20h
node-feature-discovery   node-feature-discovery-master-84c67dcbb6-txfw9
         1/1     Running    0             20h
node-feature-discovery   node-feature-discovery-worker-8tg2l
         1/1     Running    3 (19h ago)   19h
node-feature-discovery   node-feature-discovery-worker-c5f6q
         1/1     Running    0             19h
node-feature-discovery   node-feature-discovery-worker-fjfkm
         1/1     Running    0             19h
node-feature-discovery   node-feature-discovery-worker-x6tz8
         1/1     Running    0             19h
tigera-operator          tigera-operator-d499f5c8f-r2srj
         1/1     Running    1 (19h ago)   20h
vmware-system-csi        vsphere-csi-controller-7ffd6884cc-d7rql
         7/7     Running    5 (19h ago)   20h
vmware-system-csi        vsphere-csi-controller-7ffd6884cc-k82cm
         7/7     Running    2 (19h ago)   20h
vmware-system-csi        vsphere-csi-controller-7ffd6884cc-qttkp
         7/7     Running    1 (19h ago)   20h
vmware-system-csi        vsphere-csi-node-678hw
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-6tbsh
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-9htwr
         3/3     Running    5 (20h ago)   20h
vmware-system-csi        vsphere-csi-node-g8r6l
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-ghmr6
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-jhvgm
         3/3     Running    0             19h
vmware-system-csi        vsphere-csi-node-rp77r
         3/3     Running    0             19h
```

Installing Kommander in a VMware Cloud Director Environment
This section provides installation instructions for the Kommander component of NKP.

About this task

Once you have installed the Konvoy component of Nutanix Kubernetes Platform (NKP), you will continue installing the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a time period (for example, 1 hour) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Before you begin:

- Ensure you have reviewed all Prerequisites for Install.

- Ensure you have a default StorageClass.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

This section contains instructions for installing the Kommander component on air-gapped, non-air-gapped, or pre-provisioned environments. It also includes instructions on how to enable NKP catalog apps or NKP Insights, if desired.

Procedure

1. To begin configuring Kommander, run the following command to initialize a default configuration file.

   » For a non-air-gapped environment, run the following command:

   ```
   nkp install kommander --init > kommander.yaml
   ```

   » For an air-gapped environment, run the following command:

   ```
   nkp install kommander --init --airgapped > kommander.yaml
   ```

2. After the initial deployment of Kommander, you can find the application Helm Charts by checking the `spec.chart.spec.sourceRef` field of the associated `HelmRelease`.

   ```
   kubectl get helmreleases <application> -o yaml -n kommander
   ```

   Inline configuration (using values) :

   In this example, you configure the `centralized-grafana` application with resource limits by defining the Helm Chart values in the Kommander configuration file.

   ```
   apiVersion: config.kommander.mesosphere.io/v1alpha1
   kind: Installation
   apps:
     centralized-grafana:
       values: |
         grafana:
           resources:
             limits:
               cpu: 150m
               memory: 100Mi
             requests:
               cpu: 100m
               memory: 50Mi
   ...
   ```

   Reference another YAML file (using valuesFrom):

   Alternatively, you can create another YAML file containing the configuration for `centralized-grafana` and reference using `valuesFrom`. Point to this file using either a relative path (from the configuration file location) or an absolute path.

   ```
   cat > centralized-grafana.yaml <<EOF
   grafana:
     resources:
       limits:
         cpu: 150m
         memory: 100Mi
       requests:
         cpu: 100m
         memory: 50Mi
   ```

```
EOF
```

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
apps:
  centralized-grafana:
    valuesFrom: centralized-grafana.yaml
...
```

3. If you have an Ultimate License, you can enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section. Add these values (if you are enabling NKP Catalog Apps) for `dkp-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.nutanix.io/project-default-catalog-repository: "true"
        kommander.nutanix.io/workspace-default-catalog-repository: "true"
        kommander.nutanix.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

4. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Enable NKP Catalog Applications after Installing NKP.

5. After the Kommander component is installed according to your environment, log in to the UI using the instructions in the topic Logging into the UI with Kommander.

Cloud Director Management Tools

After cluster creation and configuration, you can revisit clusters to update and change variables.

Cloud Director Cluster Autoscaler

Cluster Autoscaler provides the ability to automaticallyscale upp or scale down the number of worker nodes in a cluster based on the number of pending pods to be scheduled. Running the Cluster Autoscaler is optional.

Unlike Horizontal-Pod Autoscaler, Cluster Autoscaler does not depend on any Metrics server and does not need Prometheus or other metrics source.

The Cluster Autoscaler looks at the following annotations on a MachineDeployment to determine its scale-up and scale-down ranges:

```
cluster.x-k8s.io/cluster-api-autoscaler-node-group-min-size
cluster.x-k8s.io/cluster-api-autoscaler-node-group-max-size
```

The full list of command line arguments to the Cluster Autoscaler controller is on the Kubernetes public GitHub repository.

For more information about how Cluster Autoscaler works, see:

- What is Cluster Autoscaler
- How does scale-up work
- How does scale-down work
- CAPI Provider for Cluster Autoscaler

Running the Cluster Autoscaler

About this task

The Cluster Autoscaler controller runs on the workload cluster. Upon creating the workload cluster, this controller does not have all the objects required to function correctly until after a `nkp move` is issued from the bootstrap cluster.

Run the following steps to enable Cluster Autoscaler:

Procedure

1. Ensure the Cluster Autoscaler controller is up and running (no restarts and no errors in the logs)

   ```
   kubectl --kubeconfig=${CLUSTER_NAME}.conf logs deployments/cluster-autoscaler
    cluster-autoscaler -n kube-system -f
   ```

2. Enable Cluster Autoscaler by setting the min & max ranges

   ```
   kubectl --kubeconfig=${CLUSTER_NAME}.conf annotate machinedeployment ${NODEPOOL_NAME}
    cluster.x-k8s.io/cluster-api-autoscaler-node-group-min-size=2
   kubectl --kubeconfig=${CLUSTER_NAME}.conf annotate machinedeployment ${NODEPOOL_NAME}
    cluster.x-k8s.io/cluster-api-autoscaler-node-group-max-size=6
   ```

3. The Cluster Autoscaler logs will show that the worker nodes are associated with node groups and that pending pods are being watched.

4. To demonstrate that it is working properly, create a large deployment that will trigger pending pods (For this example, we used Cloud Director n2-standard-8 worker nodes. If you have larger worker nodes, you need to scale up the number of replicas accordingly).

   ```
   cat <<EOF | kubectl --kubeconfig=${CLUSTER_NAME}.conf apply -f -
   apiVersion: apps/v1
   kind: Deployment
   metadata:
     name: busybox-deployment
     labels:
       app: busybox
   spec:
     replicas: 600
     selector:
       matchLabels:
         app: busybox
     template:
       metadata:
         labels:
           app: busybox
       spec:
         containers:
         - name: busybox
           image: busybox:latest
           command:
   ```

```
          - sleep
          - "3600"
        imagePullPolicy: IfNotPresent
      restartPolicy: Always
EOF
```

5. Cluster Autoscaler will scale up the number of Worker Nodes until there are no pending pods.
6. Scale down the number of replicas for `busybox-deployment`.

```
kubectl --kubeconfig ${CLUSTER_NAME}.conf scale --replicas=30 deployment/busybox-deployment
```

7. Cluster Autoscaler starts to scale down the number of Worker Nodes after the default timeout of 10 minutes.

Cloud Director Manage Node Pools

Node pools are part of a cluster and managed as a group, and you can use a node pool to manage a group of machines using the same common properties. When Konvoy creates a new default cluster, there is one node pool for the worker nodes, and all nodes in that new node pool have the same configuration. You can create additional node pools for more specialized hardware or configuration. For example, suppose you want to tune your memory usage on a cluster where you need maximum memory for some machines and minimal memory on other machines. In that case, you create a new node pool with those specific resource needs.

Creating a node pool is useful when running workloads requiring machines with specific resources, such as additional memory or specialized network or storage hardware.

Konvoy implements node pools using Cluster API MachineDeployments.

Preparing the Environment

Steps needed to prepare the environment for node pool creation.

Before you begin

Make sure you have created a New VCD Cluster.

About this task

Complete this task to prepare the environment.

Procedure

1. Set the environment variable to the name you assigned this cluster using the command `export CLUSTER_NAME=vcd-example`.
2. If your workload cluster is self-managed, as described in Make the New Cluster Self-Managed, configure `kubectl` to use the kubeconfig for the cluster.

```
export KUBECONFIG=${CLUSTER_NAME}.conf
```

3. Define your node pool name.

```
export NODEPOOL_NAME=example
```

Creating a VCD Node Pool

About this task

Availability zones (AZs) are isolated locations within datacenter regions from which public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single Availability Zone, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple Availability Zones.

Create a new AWS node pool with three replicas using this command:

Procedure

Set the `--zone` flag to a zone in the same region as your cluster.

```
nkp create nodepool vcd ${NODEPOOL_NAME} \
    --cluster-name=${CLUSTER_NAME} \
    --image $IMAGE_NAME \
    --zone us-west1-b \
    --replicas=3
```

This example uses default values for brevity. Use flags to define custom instance types and other properties.

```
machinedeployment.cluster.x-k8s.io/example created
## Creating default/example nodepool resources
vcdmachinetemplate.infrastructure.cluster.x-k8s.io/example created
kubeadmconfigtemplate.bootstrap.cluster.x-k8s.io/example created
 # Creating default/example nodepool resources
```

Advanced users can use a combination of the `--dry-run` and `--output=yaml` or `--output-directory=/flags` to get a complete set of node pool objects to modify locally or store in version control.

Cloud Director List Node Pools

Node pools are part of a cluster and managed as a group, and you can use a node pool to manage a group of machines using the same common properties. When Konvoy creates a new default cluster, there is one node pool for the worker nodes, and all nodes in that new node pool have the same configuration. You can create additional node pools for more specialized hardware or configuration. For example, suppose you want to tune your memory usage on a cluster where you need maximum memory for some machines and minimal memory on other machines. In that case, you create a new node pool with those specific resource needs.

Konvoy implements node pools using Cluster API MachineDeployments.

Cloud Director Scale Node Pools

While you can run Cluster Autoscaler, you can manually scale your node pools up or down when you need more finite control over your environment. For example, if you require 10 machines to run a process, you can only manually set the scaling to run those 10 machines. However, using the Cluster Autoscaler, you must stay within your minimum and maximum bounds.

Scaling Up Node Pools

About this task

To scale up a node pool in a cluster, complete the following task.

Procedure

1. To scale up a node pool in a cluster, use the command `nkp scale nodepool ${NODEPOOL_NAME} --replicas=5 --cluster-name=${CLUSTER_NAME}`.

   Example output indicating the scaling is in progress:

   ```
   # Scaling node pool example to 5 replicas
   ```

2. After a few minutes, you can list the node pools using the command `nkp get nodepool --cluster-name=${CLUSTER_NAME}`.

   Example output showing that the number of DESIRED and READY replicas increased to 5:

   ```
   NODEPOOL                            DESIRED               READY
    KUBERNETES VERSION
   example                             5                     5                    v1.28.7

   vcd-example-md-0                    4                     4                    v1.28.7
   ```

Scaling Down Node Pools

About this task

To scale down a node pool in a cluster, complete the following task.

Procedure

1. To scale down a node pool in a cluster, use the command `nkp scale nodepool ${NODEPOOL_NAME} --replicas=4 --cluster-name=${CLUSTER_NAME}`.

   Example output:

   ```
   # Scaling node pool example to 4 replicas
   ```

2. After a few minutes, you can list the node pools using the command `nkp get nodepool --cluster-name=${CLUSTER_NAME}`.

   Example output showing the number of DESIRED and READY replicas decreased to 4:

   ```
   NODEPOOL                            DESIRED               READY
    KUBERNETES VERSION
   example                             4                     4                    v1.28.7

   vcd-example-md-0                    4                     4                    v1.28.7
   ```

   In a default cluster, the nodes to delete are selected at random. • CAPI's delete policy controls this behavior. However, when using the Nutanix Kubernetes Platform (NKP) CLI to scale down a node pool, it is also possible to specify the Kubernetes Nodes you want to delete.

   a. To do this, set the flag `--nodes-to-delete` with a list of nodes as below. This adds an annotation `cluster.x-k8s.io/delete-machine=yes` to the matching Machine object that contains `status.NodeRef` with the node names from `--nodes-to-delete`.

      ```
      nkp scale nodepool ${NODEPOOL_NAME} --replicas=3 --cluster-name=${CLUSTER_NAME}
      ```

      ```
      # Scaling node pool example to 3 replicas
      ```

Scaling Node Pools When Using Cluster Autoscaler

About this task

If you configured the cluster autoscaler for the `demo-cluster-md-0` node pool, the value of `--replicas` must be within the minimum and maximum bounds.

Procedure

1. For example, assuming you have these annotations.

```
kubectl annotate machinedeployment ${NODEPOOL_NAME} cluster.x-k8s.io/cluster-api-
autoscaler-node-group-min-size=2
kubectl annotate machinedeployment ${NODEPOOL_NAME} cluster.x-k8s.io/cluster-api-
autoscaler-node-group-max-size=6
```

2. Try to scale the node pool to 7 replicas with the command.

```
nkp scale nodepool ${NODEPOOL_NAME} --replicas=7 --cluster-name=${CLUSTER_NAME}
```

3. This results in an error similar to that of

```
 # Scaling node pool example to 7 replicas
failed to scale nodepool: scaling MachineDeployment is forbidden: desired replicas
 7 is greater than the configured max size annotation cluster.x-k8s.io/cluster-api-
autoscaler-node-group-max-size: 6
```

   Similarly, scaling down to several replicas less than the configured `min-size` also returns an error.

Deleting Cloud Director Node Pools

About this task

Deleting a node pool deletes the Kubernetes nodes and the underlying infrastructure. All nodes will be drained before deletion, and the pods running on those nodes will be rescheduled.

Procedure

1. To delete a node pool from a managed cluster, run the command `nkp delete nodepool ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME}`.

   In the example output, `example` is the named node pool to be deleted.

```
# Deleting default/example nodepool resources
```

2. Deleting an invalid node pool results in output similar to this example.

```
nkp delete nodepool ${CLUSTER_NAME}-md-invalid --cluster-name=${CLUSTER_NAME}
```

```
MachineDeployments or MachinePools.infrastructure.cluster.x-k8s.io "no
 MachineDeployments or MachinePools found for cluster vcd-example" not found
```

Cloud Director Delete a Cluster

The steps to delete a cluster are determined by whether your cluster is self-managed.

Section Content

Preparing to Delete a Self-managed Workload Cluster

About this task

A self-managed cluster cannot delete itself. If your workload cluster is self-managed, you must create a bootstrap cluster and move the cluster life cycle services to the bootstrap cluster before deleting the workload cluster.

For clusters that are not self-managed, as described in Make the New Cloud Director Cluster Self-Managed, proceed to Delete the workload cluster section.

Procedure

1. Create a bootstrap cluster using the command `nkp create bootstrap --vcd-bootstrap-credentials=true --kubeconfig $HOME/.kube/config`.

   The bootstrap cluster will host the Cluster API controllers that reconcile the cluster objects marked for deletion.

   > **Note:** To avoid using the wrong `kubeconfig`, the following steps use explicit `kubeconfig` paths and contexts.

   Example output:

   ```
   # Creating a bootstrap cluster
   # Initializing new CAPI components
   ```

2. Move the Cluster API objects from the workload to the bootstrap cluster: The cluster life cycle services on the bootstrap cluster are ready, but the workload cluster configuration is on the workload cluster. The `move` command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the workload to the bootstrap cluster. This process is called a Pivot. For more information, see https://cluster-api.sigs.k8s.io/reference/glossary.html?highlight=pivot#pivot.

   ```
   nkp get nodepool --cluster-name=${CLUSTER_NAME}
   ```

   Example output showing the number of DESIRED and READY replicas decreased to 4:

   ```
   nkp move capi-resources \
       --from-kubeconfig ${CLUSTER_NAME}.conf \
       --to-kubeconfig $HOME/.kube/config
   ```

   In a default cluster, the nodes to delete are selected at random. CAPI's delete policy controls this behavior. However, when using the Nutanix Kubernetes Platform (NKP) CLI to scale down a node pool, it is also possible to specify the Kubernetes Nodes you want to delete.

3. Use the cluster life cycle services on the workload cluster to check the workload cluster status.

   ```
   nkp describe cluster --kubeconfig $HOME/.kube/config -c ${CLUSTER_NAME}
   ```

   ```
   NAME                                                                    READY
    SEVERITY  REASON  SINCE  MESSAGE
   Cluster/vcd-example                                                     True
                 34s
   ##ClusterInfrastructure - vcdCluster/vcd-example
   ##ControlPlane - KubeadmControlPlane/vcd-example-control-plane          True
                 34s
   # ##Machine/vcd-example-control-plane-6fbzn                             True
                 37s
   # # ##MachineInfrastructure - vcdMachine/vcd-example-control-plane-62g6s
   # ##Machine/vcd-example-control-plane-jf6s2                             True
                 37s
   # # ##MachineInfrastructure - vcdMachine/vcd-example-control-plane-bsr2z
   # ##Machine/vcd-example-control-plane-mnbfs                             True
                 37s
   ```

```
#   ##MachineInfrastructure - vcdMachine/vcd-example-control-plane-s8xsx
##Workers
  ##MachineDeployment/vcd-example-md-0                                  True
              37s
    ##Machine/vcd-example-md-0-68b86fddb8-8glsw                         True
              37s
    # ##MachineInfrastructure - vcdMachine/vcd-example-md-0-zls8d
    ##Machine/vcd-example-md-0-68b86fddb8-bvbm7                         True
              37s
    # ##MachineInfrastructure - vcdMachine/vcd-example-md-0-5zcvc
    ##Machine/vcd-example-md-0-68b86fddb8-k9499                         True
              37s
    # ##MachineInfrastructure - vcdMachine/vcd-example-md-0-k8h5p
    ##Machine/vcd-example-md-0-68b86fddb8-l6vfb                         True
              37s
      ##MachineInfrastructure - vcdMachine/vcd-example-md-0-9h5vn
```

Note: After moving the cluster life cycle services to the workload cluster, remember to use `nkp` with the workload cluster kubeconfig.

Deleting the Workload Cluster

About this task

Context for the current task

Procedure

1. Evict all workloads that use Persistent Volumes.

   We recommend using `nkp` with the bootstrap cluster to delete all worker node pools. This evicts all workloads that use Persistent Volumes. See information on Deleting Cloud Director Node Pools on page 991.

   Note: Persistent Volumes (PVs) are not deleted automatically by design to preserve your data. However, they take up storage space if not deleted. You must delete PVs manually. Information for the backup of a cluster and PVs is in the documentation called Cluster Applications and Persistent Volumes Backup on page 537.

2. Use `nkp` with the bootstrap cluster to delete the workload cluster. Delete the Kubernetes cluster and wait a few minutes.

   Before deleting the cluster, Nutanix Kubernetes Platform (NKP) deletes all Services of type LoadBalancer on the cluster. To skip this step, use the flag `--delete-kubernetes-resources=false`.

   ```
   nkp delete cluster --kubeconfig $HOME/.kube/config --cluster-name ${CLUSTER_NAME}
   ```

   ```
   # Deleting Services with type LoadBalancer for Cluster default/vcd-example
   # Deleting ClusterResourceSets for Cluster default/vcd-example
   # Deleting cluster resources
   # Waiting for cluster to be fully deleted
   Deleted default/vcd-example cluster
   ```

Deleting the Bootstrap Cluster

About this task

Context for the current task

Procedure

Delete the bootstrap cluster using the command `nkp delete bootstrap --kubeconfig $HOME/.kube/config`.

Example output:

```
# Deleting bootstrap
    cluster
```

Deleting a Managed Cluster

About this task

In the CLI, use the kubeconfig for the #Managed# cluster you want to delete. In order to properly delete a **Managed** cluster, you must first delete the PVCs using these steps.

Procedure

1. Export your workspace namespace using the command `export WORKSPACE_NAMESPACE=<your-workspace-namespace>`.
2. Get all of the PVCs that you need to delete from the Applications deployed by Kommander.

   ```
   kubectl get pvc --namespace ${WORKSPACE_NAMESPACE}
   ```

3. Delete those PVCs and the Managed cluster deletion process will continue.

   ```
   kubectl delete pvc --namespace ${WORKSPACE_NAMESPACE} <pvc-name-1> <pvc-name-2>
   ```

Known Limitations

The following are known limitations:

- The Nutanix Kubernetes Platform (NKP) version used to create the workload cluster must match the NKP version used to delete the workload cluster.

Google Cloud Platform (GCP) Infrastructure

Configuration types for installing NKP on GCP Infrastructure.

For an environment on the Google Cloud Platform (GCP) Infrastructure, install options based on those environment variables are provided for you in this location.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

Otherwise, proceed to the GCP Prerequisites and Permissions topic to begin your custom installation.

GCP Prerequisites

Before beginning a Nutanix Kubernetes Platform (NKP) installation, verify that you have the following:

- An x86_64-based Linux or macOS machine with a supported operating system version.

- Download the NKP binary for Linux or macOS from Downloading NKP on page 16. To check which version of NKP you installed for compatibility reasons, use the command `NKP version -h`.
- A Container engine or runtime installed is required to install NKP and bootstrap:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
- kubectl for interacting with the running cluster.
- Install the GCP `gcloud` CLI by following the Install the gcloud CLI | Google Cloud CLI Documentation

Control Plane Nodes

You must have at least three control plane nodes. Each control plane node needs to have at least the following:

- Four (4) cores
- 16 GiB memory
- Approximately 80 GiB of free space for the volume used for `/var/lib/kubelet` and `/var/lib/containerd`.
- Disk usage must be below 85% on the root volume.

NKP on GCP defaults to deploying an `n2-standard-4` instance with an 80GiB root volume for control plane nodes, which meets the above requirements.

Worker Nodes

You must have at least four worker nodes. The specific number of worker nodes required for your environment can vary depending on the cluster workload and size of the nodes. Each worker node needs to have at least the following:

- Eight (8) cores
- 32 GiB memory
- Around 80 GiB of free space for the volume used for `/var/lib/kubelet` and `/var/lib/containerd`.
- Disk usage must be below 85% on the root volume.

NKP on GCP defaults to deploying a `n2-standard-8` instance with an 80GiB root volume for worker nodes, which meets the above requirements.

GCP Prerequisites

- Verify that your Google Cloud project does not enable the OS Login feature.

GCP projects may have the Enable OS Login feature enabled by default. If this feature is enabled, KIB cannot `ssh` to the VM instances it creates, and the image creation fails.

To check if it is enabled, use the Google commands to inspect the metadata configured in your project. If you find the `enable-oslogin` flag set to TRUE, you must remove or set it to FALSE to use KIB. For more information on Set and Remove Custom Metadata, see https://cloud.google.com/compute/docs/metadata/setting-custom-metadata#console_2

- The user creating the Service Accounts needs additional privileges besides the Editor role. For more information, see GCP Roles

GCP Roles

Service accounts are a special type of Google account that grants permissions to virtual machines instead of end users. The primary purpose of Service accounts is to ensure safe, managed connections to APIs and Google Cloud services.

These roles are needed when creating an image using Konvoy Image Builder.

GCP Prerequisite Roles

If you are creating your image on either a non-GCP instance or one that does not have the required roles (`Editor` role), you must either:

- Create a GCP service account.
- If you have already created a service account, retrieve the credentials for an existing service account.
- Export the static credentials that you will use to create the cluster using the command `export GCP_B64ENCODED_CREDENTIALS=$(base64 < "${GOOGLE_APPLICATION_CREDENTIALS}" | tr -d '\n')`.

  Note: To enhance security, rotate static credentials regularly.

Role Options

- Either create a new GCP service account or retrieve credentials from an existing one.
- (Option 1) Create a GCP Service Account using the following `gcloud` commands:

```
export GCP_PROJECT=<your GCP project ID> export GCP_SERVICE_ACCOUNT_USER=<some
 new service account user> export GOOGLE_APPLICATION_CREDENTIALS="$HOME/.gcloud/
credentials.json"
               gcloud iam service-accounts create "$GCP_SERVICE_ACCOUNT_USER"
               --project=$GCP_PROJECT gcloud projects add-iam-policy-binding
 $GCP_PROJECT
               --member="serviceAccount:$GCP_SERVICE_ACCOUNT_USER@
$GCP_PROJECT.iam.gserviceaccount.com"
               --role=roles/editor gcloud iam service-accounts keys create
 $GOOGLE_APPLICATION_CREDENTIALS
               --iam-account="$GCP_SERVICE_ACCOUNT_USER@
$GCP_PROJECT.iam.gserviceaccount.com"
```

- (Option 2) Retrieve the credentials for an existing service account using the following `gcloud` commands:

```
export GCP_PROJECT=<your GCP project ID> export GCP_SERVICE_ACCOUNT_USER=<existing
 service account user> export GOOGLE_APPLICATION_CREDENTIALS="$HOME/.gcloud/
credentials.json"
               gcloud iam service-accounts keys create
 $GOOGLE_APPLICATION_CREDENTIALS
               --iam-account="$GCP_SERVICE_ACCOUNT_USER@
$GCP_PROJECT.iam.gserviceaccount.com"
```

- Export the static credentials you will use to create the cluster using the command `export GCP_B64ENCODED_CREDENTIALS=$(base64 < "${GOOGLE_APPLICATION_CREDENTIALS}" | tr -d '\n')`

To create a **GCP Service Account** with the `Editor` role, the user creating the GCP Service Account needs the `Editor`, `RoleAdministrator`, and `SecurityAdmin` roles. However, those pre-defined roles grant more permissions than the minimum set needed to create a Nutanix Kubernetes Platform (NKP) cluster. Granting unnecessary permissions can lead to potential security risks and should be avoided.

Note: For NKP cluster creation, a minimal set of roles and permissions needed for the user creating the GCP Service Account is the `Editor` role plus the following additional permissions:

- `compute.disks.setIamPolicy`
- `compute.instances.setIamPolicy`
- `iam.roles.create`
- `iam.roles.delete`
- `iam.roles.update`
- `iam.serviceAccounts.setIamPolicy`
- `resourcemanager.projects.setIamPolicy`

For more information on GCP service accounts, see GCP's documentation:

- GCP service account: https://cloud.google.com/iam/docs/service-account-overview
- Create service accounts: https://cloud.google.com/iam/docs/creating-managing-service-accounts
- Best practices for using service accounts: https://cloud.google.com/iam/docs/best-practices-service-accounts

GCP Using Konvoy Image Builder

This procedure describes using the Konvoy Image Builder (KIB) to create a Cluster API-compliant GCP image. GCP images contain configuration information and software to create a specific, pre-configured operating environment. For example, you can create a GCP image of your computer system settings and software. Then, replicate the GCP image and distribute it to others to use a replica of your computer system settings and software. KIB uses variable overrides to specify your new GCP image base and container images.

Note: Google Cloud Platform does not publish images. You must first build the image using Konvoy Image Builder. Explore the Customize your Image topic for more options. For more information regarding using the image in creating clusters, refer to the GCP Infrastructure section of the documentation.

NKP Prerequisites

Before you begin, you must:

- Download the KIB bundle for your Nutanix Kubernetes Platform (NKP) version.
- Check the Supported Infrastructure Operating Systems.
- Check the Supported Kubernetes version for your provider in Upgrade NKP on page 1144.
- Create a working Docker or other registry setup.
- On Debian-based Linux distributions, install a version of the cri-tools package compatible with the Kubernetes and container runtime versions. For more information, see https://github.com/kubernetes-sigs/cri-tools.
- Verify that your Google Cloud project does not enable the Enable OS Login feature. See below for more information:

Note: GCP projects may have the Enable OS Login feature enabled by default. If this feature is enabled, KIB cannot `ssh` to the VM instances it creates, and the image creation fails.

To check if it is enabled, use the Google commands to inspect the metadata configured in your project. If you find the `enable-oslogin` flag set to TRUE, you must remove or set it to FALSE to use KIB. For more information on Set and Remove Custom Metadata, see https://cloud.google.com/compute/docs/metadata/setting-custom-metadata#console_2

GCP Prerequisite Roles

If you are creating your image on either a non-GCP instance or one that does not have the required GCP Roles on page 996, you must either:

- Create a GCP service account. For more information, see https://cloud.google.com/iam/docs/service-account-overview.
- If you have already created a service account, retrieve the credentials for an existing service account.
- Export the static credentials that you will use to create the cluster using the command `export GCP_B64ENCODED_CREDENTIALS=$(base64 < "${GOOGLE_APPLICATION_CREDENTIALS}" | tr -d '\n')`.

Tip: Make sure to rotate static credentials for increased security.

GCP Installation in a Non-air-gapped Environment

This installation provides instructions on how to install the Nutanix Kubernetes Platform (NKP) in a GCP non-air-gapped environment.

Remember, there are always more options for custom YAML in the Custom Installation and Additional Infrastructure Tools section, but this will get you operating with basic features.

If not already done, see the documentation for:

- Resource Requirements on page 39
- Installing NKP on page 50
- Prerequisites for Installation on page 47

GCP Prerequisites

Verify that your Google Cloud project does not enable the OS Login feature.

Note:

GCP projects may have the Enable OS Login feature enabled by default. If this feature is enabled, KIB cannot `ssh` to the VM instances it creates, and the image creation fails.

To check if it is enabled, use the Google commands to inspect the metadata configured in your project. If you find the `enable-oslogin` flag set to TRUE, you must remove or set it to FALSE to use KIB. For more information on Set and Remove Custom Metadata, see https://cloud.google.com/compute/docs/metadata/setting-custom-metadata#console_2

- The user creating the Service Accounts needs additional privileges in addition to the Editor role. For more information, see GCP Roles

Bootstrapping GCP

NKP creates Kubernetes clusters using Cluster API (CAPI) controllers, which run on a Kubernetes cluster.

About this task

To get started, you need a bootstrap cluster. By default, Nutanix Kubernetes Platform (NKP) creates a bootstrap cluster for you in a Docker container using the Kubernetes-in-Docker (KIND) tool.

Before you begin

Procedure

1. Complete the Nutanix Infrastructure Prerequisites. For more information, see Nutanix Infrastructure Prerequisites on page 690.
2. Ensure the `NKP` binary can be found in your `$PATH`.

Bootstrap Cluster Life Cycle Services

Procedure

1. Review Universal Configurations for all Infrastructure Providers regarding settings, flags, and other choices and then begin bootstrapping.
2. Create a bootstrap cluster using the command `nkp create bootstrap --kubeconfig $HOME/.kube/config`.

   > **Note:** Use `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. For more information, see Configuring an HTTP or HTTPS Proxy on page 677.

   Example output:

   ```
   # Creating a bootstrap cluster
   # Initializing new CAPI components
   ```

3. NKP creates a bootstrap cluster using KIND as a library.

   For more information, see https://github.com/kubernetes-sigs/kind.

4. NKP then deploys the following Cluster API providers on the cluster.

   - Core Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/
   - AWS Infrastructure Provider: https://github.com/kubernetes-sigs/cluster-api-provider-aws
   - Kubeadm Bootstrap Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/bootstrap/kubeadm
   - Kubeadm ControlPlane Provider: https://github.com/kubernetes-sigs/cluster-api/tree/v0.3.20/controlplane/kubeadm

   For more information on Cluster APIs, see https://cluster-api.sigs.k8s.io/.

5. NKP waits until these providers' controller-manager and webhook deployments are ready. List these deployments using the command `kubectl get --all-namespaces deployments -l=clusterctl.cluster.x-k8s.io`.

   Output example:

   ```
   NAMESPACE                           NAME
    READY   UP-TO-DATE   AVAILABLE   AGE
   capa-system                         capa-controller-manager
    1/1     1            1           1h
   ```

```
capg-system                          capg-controller-manager
 1/1     1            1           1h
capi-kubeadm-bootstrap-system        capi-kubeadm-bootstrap-controller-manager
 1/1     1            1           1h
capi-kubeadm-control-plane-system    capi-kubeadm-control-plane-controller-manager
 1/1     1            1           1h
capi-system                          capi-controller-manager
 1/1     1            1           1h
cappp-system                         cappp-controller-manager
 1/1     1            1           1h
capv-system                          capv-controller-manager
 1/1     1            1           1h
capz-system                          capz-controller-manager
 1/1     1            1           1h
cert-manager                         cert-manager
 1/1     1            1           1h
cert-manager                         cert-manager-cainjector
 1/1     1            1           1h
cert-manager                         cert-manager-webhook
 1/1     1            1           1h
```

Creating a New GCP Cluster

Create a Google Cloud Platform (GCP) Cluster in a non-air-gapped environment.

About this task

Use this procedure to create a custom GCP cluster with Nutanix Kubernetes Platform (NKP). First, you must name your cluster.

Name Your Cluster

> **Note:** The cluster name may only contain the following characters: `a-z`, `0-9`,`.` , and `-`. Cluster creation will fail if the name has capital letters. For more naming information, see https://kubernetes.io/docs/concepts/overview/working-with-objects/names/.

> **Note:** NKP uses the GCP CSI driver as the default storage provider. Use a Kubernetes CSI compatible storage that is suitable for production.

Procedure

1. Give your cluster a unique name suitable for your environment.
2. Set the environment variable using the command `export CLUSTER_NAME=<gcp-example>`.

   > **Note:** To increase Docker Hub's rate limit, use your Docker Hub credentials when creating the cluster by setting the following flag `--registry-mirror-url=https://registry-1.docker.io --registry-mirror-username=<username> --registry-mirror-password=<password>` on the `nkp create cluster` command. For more information, see https://docs.docker.com/docker-hub/download-rate-limit/

Create a New GCP Cluster

About this task

If you use these instructions to create a cluster on GCP using the NKP default settings without any edits to configuration files or additional flags, your cluster is deployed on an Ubuntu 20.04 operating system image with 3 control plane nodes, and 4 worker nodes.

By default, the control-plane Nodes will be created in 3 different zones. However, the default worker Nodes will reside in a single zone. You may create additional node pools in other zones with the `nkp create nodepool` command.

Availability zones (AZs) are isolated locations within datacenter regions where public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single Availability Zone, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple Availability Zones.

> **Note:** Google Cloud Platform does not publish images. You must first build the image using Konvoy Image Builder.

Procedure

1. Create an image using Konvoy Image Builder (KIB) and then export the image name.

   ```
   export IMAGE_NAME=projects/${GCP_PROJECT}/global/images/<image_name_from_kib>
   ```

2. Ensure your subnets do not overlap with your host subnet because they cannot be changed after cluster creation. If you need to change the kubernetes subnets, you must do this at cluster creation. The default subnets used in NKP are.

   ```
   spec:
     clusterNetwork:
       pods:
         cidrBlocks:
         - 192.168.0.0/16
       services:
         cidrBlocks:
         - 10.96.0.0/12
   ```

   » Additional Options for your environment; otherwise, proceed to the next step to create your cluster.

   (Optional) Modify Control Plane Audit logs - Users can modify the `KubeadmControlplane` cluster-API object to configure different `kubelet` options. See the following guide if you wish to configure your control plane beyond the existing options available from flags.

   » (Optional) Determine what VPC Network to use. All GCP accounts come with a default preconfigured VPC Network, which will be used if you do not specify a different network. To use a different VPC network for your cluster, create one by following these instructions for Create and Manage VPC Networks. Then select the `--network <new_vpc_network_name>` option on the create cluster command below. More information is available on GCP Cloud Nat and network flag.

   » (Optional) Use a registry mirror. Configure your cluster to use an existing local registry as a mirror when pulling images previously pushed to your registry.

3. Create a Kubernetes cluster object with a `dry run` output for customizations. The following example shows a common configuration.

   ```
   nkp create cluster gcp \
   --cluster-name=${CLUSTER_NAME} \
   --additional-tags=owner=$(whoami) \
   --with-gcp-bootstrap-credentials=true \
   --project=${GCP_PROJECT} \
   --image=${IMAGE_NAME} \
   --dry-run \
   --output=yaml \
   ```

```
> ${CLUSTER_NAME}.yaml
```

Note: More flags can be added to the `nkp create cluster` command for more options. See Choices below or refer to the topic Universal Configurations:

- If your environment uses HTTP or HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP or HTTPS Proxy on page 677
- You can create individual manifest files with different smaller manifests for ease in editing using the `--output-directory` flag.

4. Inspect or edit the cluster objects. Familiarize yourself with Cluster API before editing the cluster objects, as edits can prevent the cluster from deploying successfully.
5. Create the cluster from the objects generated from the `dry run`. A warning will appear in the console if the resource already exists, requiring you to remove the resource or update your YAML.

```
kubectl create -f ${CLUSTER_NAME}.yaml
```

Note: If you used the `--output-directory` flag in your `NKP create .. --dry-run` step above, create the cluster from the objects you created by specifying the directory:

```
kubectl create -f <existing-directory>/.
```

6. Wait for the cluster control plan to be ready.

```
kubectl wait --for=condition=ControlPlaneReady "clusters/${CLUSTER_NAME}" --
timeout=20m
```

7. After the objects are created on the API server, the Cluster API controllers reconcile them. They create infrastructure and machines. As they progress, they update the Status of each object. Konvoy provides a command to describe the current status of the cluster.

```
nkp describe cluster -c ${CLUSTER_NAME}
```

Output:

```
NAME                                                                    READY
 SEVERITY  REASON  SINCE  MESSAGE
Cluster/gcp-example                                                     True
              52s
##ClusterInfrastructure - GCPCluster/gcp-example
##ControlPlane - KubeadmControlPlane/gcp-example-control-plane          True
              52s
# ##Machine/gcp-example-control-plane-6fbzn                             True
              2m32s
# # ##MachineInfrastructure - GCPMachine/gcp-example-control-plane-62g6s
# ##Machine/gcp-example-control-plane-jf6s2                             True
              7m36s
# # ##MachineInfrastructure - GCPMachine/gcp-example-control-plane-bsr2z
# ##Machine/gcp-example-control-plane-mnbfs                             True
              54s
#   ##MachineInfrastructure - GCPMachine/gcp-example-control-plane-s8xsx
##Workers
  ##MachineDeployment/gcp-example-md-0                                  True
              78s
    ##Machine/gcp-example-md-0-68b86fddb8-8glsw                         True
              2m49s
    # ##MachineInfrastructure - GCPMachine/gcp-example-md-0-zls8d
    ##Machine/gcp-example-md-0-68b86fddb8-bvbm7                         True
              2m48s
```

```
# ##MachineInfrastructure - GCPMachine/gcp-example-md-0-5zcvc
##Machine/gcp-example-md-0-68b86fddb8-k9499                          True
          2m49s
# ##MachineInfrastructure - GCPMachine/gcp-example-md-0-k8h5p
##Machine/gcp-example-md-0-68b86fddb8-l6vfb                          True
          2m49s
  ##MachineInfrastructure - GCPMachine/gcp-example-md-0-9h5vn
```

Note: NKP uses the GCP CSI driver as the default storage provider. Use a Kubernetes CSI compatible storage that is suitable for production. For more information, see the Kubernetes documentation Changing the Default Storage Class. If you're not using the default, you cannot deploy an alternate provider until after the `nkp create cluster` is finished. However, this must be determined before the Kommander installation.

Making the New GCP Cluster Self-Managed

How to make a Kubernetes cluster manage itself.

About this task

Nutanix Kubernetes Platform (NKP) deploys all cluster life cycle services to a bootstrap cluster, which then deploys a workload cluster. When the workload cluster is ready, move the cluster life cycle services to the workload cluster, which makes the workload cluster self-managed.

Before you begin

Before starting, ensure you can create a workload cluster as described in the topic: Create a New GCP Cluster.

This page contains instructions on how to make your cluster self-managed. This is necessary if there is only one cluster in your environment or if this cluster becomes the Management cluster in a multi-cluster environment.

Note: If you already have a self-managed or Management cluster in your environment, skip this page.

Follow these steps to turn your new cluster into a **Management Cluster** for an Ultimate license environment (or a free-standing Pro Cluster):

Procedure

1. Deploy cluster life cycle services on the workload cluster.

   ```
   nkp create capi-components --kubeconfig ${CLUSTER_NAME}.conf
   ```

   Output:

   ```
   # Initializing new CAPI components
   ```

   Note: If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in Configuring an HTTP/HTTPS Proxy.

2. The cluster life cycle services on the workload cluster are ready, but the workload cluster configuration is on the bootstrap cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the bootstrap to the workload cluster. This process is

called a Pivot. For more information, see https://cluster-api.sigs.k8s.io/reference/glossary.html?highlight=pivot#pivot.

Move the Cluster API objects from the bootstrap to the workload cluster:

```
nkp move capi-resources --to-kubeconfig ${CLUSTER_NAME}.conf
```

Output:

```
# Moving cluster resources
You can now view resources in the moved cluster by using the --kubeconfig flag with
 kubectl. For example: kubectl --kubeconfig=gcp-example.conf get nodes
```

> **Note:** To ensure only one set of cluster life cycle services manages the workload cluster, NKP first pauses the reconciliation of the objects on the bootstrap cluster, then creates the objects on the workload cluster. As NKP copies the objects, the cluster life cycle services on the workload cluster reconcile the objects. The workload cluster becomes self-managed after NKP creates all the objects. If it fails, the `move` command can be safely retried.

3. Wait for the cluster control plan to be ready.

```
kubectl --kubeconfig ${CLUSTER_NAME}.conf wait --for=condition=ControlPlaneReady
 "clusters/${CLUSTER_NAME}" --timeout=20m
```

Output:

```
cluster.cluster.x-k8s.io/gcp-example condition met
```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster status. After moving the cluster life cycle services to the workload cluster, remember to use NKP with the workload cluster `kubeconfig`.

```
nkp describe cluster --kubeconfig ${CLUSTER_NAME}.conf -c ${CLUSTER_NAME}
```

Output:

```
NAME                                                                        READY
 SEVERITY  REASON  SINCE  MESSAGE
Cluster/gcp-example                                                         True
             14s
##ClusterInfrastructure - GCPCluster/gcp-example
##ControlPlane - KubeadmControlPlane/gcp-example-control-plane             True
             14s
# ##Machine/gcp-example-control-plane-6fbzn                                 True
             17s
# # ##MachineInfrastructure - GCPMachine/gcp-example-control-plane-62g6s
# ##Machine/gcp-example-control-plane-jf6s2                                 True
             17s
# # ##MachineInfrastructure - GCPMachine/gcp-example-control-plane-bsr2z
# ##Machine/gcp-example-control-plane-mnbfs                                 True
             17s
#   ##MachineInfrastructure - GCPMachine/gcp-example-control-plane-s8xsx
##Workers
  ##MachineDeployment/gcp-example-md-0                                      True
             17s
    ##Machine/gcp-example-md-0-68b86fddb8-8glsw                             True
             17s
    # ##MachineInfrastructure - GCPMachine/gcp-example-md-0-zls8d
    ##Machine/gcp-example-md-0-68b86fddb8-bvbm7                            True
             17s
    # ##MachineInfrastructure - GCPMachine/gcp-example-md-0-5zcvc
    ##Machine/gcp-example-md-0-68b86fddb8-k9499                            True
             17s
    # ##MachineInfrastructure - GCPMachine/gcp-example-md-0-k8h5p
```

```
   ##Machine/gcp-example-md-0-68b86fddb8-l6vfb                              True
            17s
     ##MachineInfrastructure - GCPMachine/gcp-example-md-0-9h5vn
```

5. Remove the bootstrap cluster because the workload cluster is now self-managed.

```
NKP delete bootstrap --kubeconfig $HOME/.kube/config
# Deleting bootstrap cluster
```

Known Limitations

Procedure

- NKP only supports moving all namespaces in the cluster; NKP does not support migration of individual namespaces.
- Konvoy supports moving only one set of cluster objects from the bootstrap cluster to the workload cluster or vice-versa.

Exploring the GCP Cluster

This guide explains how to use the command line to interact with your newly deployed Kubernetes cluster.

About this task

Before you start, make sure you have created a workload cluster, as described in Create a New GCP Cluster.

Procedure

1. When the workload cluster is created, the cluster life cycle services generate a `kubeconfig` file for the workload cluster and write it to a Secret. The `kubeconfig` file is scoped to the cluster administrator. Get a `kubeconfig` file for the workload cluster.

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

2. Verify the API server is up by listing the nodes.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get nodes
```

> **Note:** The Status may take a few minutes to move to `Ready` while the Pod network is deployed. The node status will change to `Ready` soon after the `calico-node` DaemonSet Pods are Ready.

Output:

```
NAME                                STATUS   ROLES                  AGE     VERSION
gcp-example-control-plane-9z77w     Ready    control-plane,master   4m44s   v1.27.6
gcp-example-control-plane-rtj9h     Ready    control-plane,master   104s    v1.27.6
gcp-example-control-plane-zbf9w     Ready    control-plane,master   3m23s   v1.27.6
gcp-example-md-0-88c46              Ready    <none>                 3m28s   v1.27.6
gcp-example-md-0-fp8s7              Ready    <none>                 3m28s   v1.27.6
gcp-example-md-0-qvnx7              Ready    <none>                 3m28s   v1.27.6
gcp-example-md-0-wjdrg              Ready    <none>                 3m27s   v1.27.6
```

3. List the Pods with the command.

```
kubectl --kubeconfig=${CLUSTER_NAME}.conf get pods -A
```

Verify the output:

```
NAMESPACE                           NAME
               READY   STATUS    RESTARTS       AGE
```

```
calico-system                       calico-kube-controllers-577c696df9-v2nzv
               1/1     Running   0             5m23s
calico-system                       calico-node-4x5rk
               1/1     Running   0             4m22s
calico-system                       calico-node-cxsgc
               1/1     Running   0             4m23s
calico-system                       calico-node-dvlnm
               1/1     Running   0             4m23s
calico-system                       calico-node-h6nlt
               1/1     Running   0             4m23s
calico-system                       calico-node-jmkwq
               1/1     Running   0             5m23s
calico-system                       calico-node-tnf54
               1/1     Running   0             4m18s
calico-system                       calico-node-v6bwq
               1/1     Running   0             2m39s
calico-system                       calico-typha-6d8c94bfdf-dkfvq
               1/1     Running   0             5m23s
calico-system                       calico-typha-6d8c94bfdf-fdfn2
               1/1     Running   0             3m43s
calico-system                       calico-typha-6d8c94bfdf-kjgzj
               1/1     Running   0             3m43s
capa-system                         capa-controller-manager-6468bc488-w7nj9
               1/1     Running   0             67s
capg-system                         capg-controller-manager-5fb47f869b-6jgms
               1/1     Running   0             53s
capi-kubeadm-bootstrap-system       capi-kubeadm-bootstrap-controller-
manager-65ffc94457-7cjdn      1/1     Running   0              74s
capi-kubeadm-control-plane-system   capi-kubeadm-control-plane-controller-manager-
bc7b688d4-vv8wg   1/1     Running   0              72s
capi-system                         capi-controller-manager-dbfc7b49-dzvw8
               1/1     Running   0             77s
cappp-system                        cappp-controller-manager-8444d67568-rmms2
               1/1     Running   0             59s
capv-system                         capv-controller-manager-58b8ccf868-rbscn
               1/1     Running   0             56s
capz-system                         capz-controller-manager-6467f986d8-dnvj4
               1/1     Running   0             62s
cert-manager                        cert-manager-6888d6b69b-7b7m9
               1/1     Running   0             91s
cert-manager                        cert-manager-cainjector-76f7798c9-gnp8f
               1/1     Running   0             91s
cert-manager                        cert-manager-webhook-7d4b5d8484-gn5dr
               1/1     Running   0             91s
gce-pd-csi-driver                   csi-gce-pd-controller-5bd587fbfb-lrx29
               5/5     Running   0             5m40s
gce-pd-csi-driver                   csi-gce-pd-node-4cgd8
               2/2     Running   0             4m22s
gce-pd-csi-driver                   csi-gce-pd-node-5qsfk
               2/2     Running   0             4m23s
gce-pd-csi-driver                   csi-gce-pd-node-5w4bq
               2/2     Running   0             4m18s
gce-pd-csi-driver                   csi-gce-pd-node-fbdbw
               2/2     Running   0             4m23s
gce-pd-csi-driver                   csi-gce-pd-node-h82lx
               2/2     Running   0             4m23s
gce-pd-csi-driver                   csi-gce-pd-node-jzq58
               2/2     Running   0             5m39s
gce-pd-csi-driver                   csi-gce-pd-node-k6bz9
               2/2     Running   0             2m39s
kube-system                         cluster-autoscaler-7f695dc48f-v5kvh
               1/1     Running   0             5m40s
```

```
kube-system                         coredns-64897985d-hbkqd
               1/1     Running   0             5m38s
kube-system                         coredns-64897985d-m8g5j
               1/1     Running   0             5m38s
kube-system                         etcd-gcp-example-control-plane-9z77w
               1/1     Running   0             5m32s
kube-system                         etcd-gcp-example-control-plane-rtj9h
               1/1     Running   0             2m37s
kube-system                         etcd-gcp-example-control-plane-zbf9w
               1/1     Running   0             4m17s
kube-system                         kube-apiserver-gcp-example-control-plane-9z77w
               1/1     Running   0             5m32s
kube-system                         kube-apiserver-gcp-example-control-plane-rtj9h
               1/1     Running   0             2m38s
kube-system                         kube-apiserver-gcp-example-control-plane-zbf9w
               1/1     Running   0             4m17s
kube-system                         kube-controller-manager-gcp-example-control-
plane-9z77w         1/1     Running   0             5m33s
kube-system                         kube-controller-manager-gcp-example-control-
plane-rtj9h         1/1     Running   0             2m37s
kube-system                         kube-controller-manager-gcp-example-control-
plane-zbf9w         1/1     Running   0             4m17s
kube-system                         kube-proxy-bskz2
               1/1     Running   0             4m18s
kube-system                         kube-proxy-gdkn5
               1/1     Running   0             4m23s
kube-system                         kube-proxy-knvb9
               1/1     Running   0             4m22s
kube-system                         kube-proxy-tcj7r
               1/1     Running   0             4m23s
kube-system                         kube-proxy-thdpl
               1/1     Running   0             5m38s
kube-system                         kube-proxy-txxmb
               1/1     Running   0             4m23s
kube-system                         kube-proxy-vq6kv
               1/1     Running   0             2m39s
kube-system                         kube-scheduler-gcp-example-control-plane-9z77w
               1/1     Running   0             5m33s
kube-system                         kube-scheduler-gcp-example-control-plane-rtj9h
               1/1     Running   0             2m37s
kube-system                         kube-scheduler-gcp-example-control-plane-zbf9w
               1/1     Running   0             4m17s
node-feature-discovery              node-feature-discovery-master-7d5985467-lh7dc
               1/1     Running   0             5m40s
node-feature-discovery              node-feature-discovery-worker-5qtvg
               1/1     Running   0             3m40s
node-feature-discovery              node-feature-discovery-worker-66rwx
               1/1     Running   0             3m40s
node-feature-discovery              node-feature-discovery-worker-7h92d
               1/1     Running   0             3m35s
node-feature-discovery              node-feature-discovery-worker-b4666
               1/1     Running   0             3m40s
tigera-operator                     tigera-operator-5f9bdc5c59-j9tnr
               1/1     Running   0             5m38s
```

GCP Kommander Installation

This section provides installation instructions for the Kommander component of NKP in a non-air-gapped GCP environment.

About this task

Once you have installed the Konvoy component of Nutanix Kubernetes Platform (NKP), you will continue installing the Kommander component that will bring up the UI dashboard.

Tip: Tips and Recommendations

- The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures you install Kommander on the correct cluster. For alternatives, see Provide Context for Commands with a `kubeconfig` File.
- Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a time period (for example, 1 hour) to allocate more time to the deployment of applications.
- If the Kommander installation fails, or you wish to reconfigure applications, rerun the `install` command to retry.

Prerequisites:

- Ensure you have reviewed all Prerequisites for Install.
- Ensure you have a Default StorageClass on page 1038.
- Note the name of the cluster where you want to install Kommander. If you do not know the cluster name, use `kubectl get clusters -A` to display and find it.

Create your Kommander Installation Configuration File

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} >> ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. If required: Customize your `kommander.yaml`.

   a. See the Kommander Customizations page for customization options. Some options include Custom Domains and Certificates, HTTP proxy, and External Load Balancer.

5. Only required if your cluster uses a custom AWS VPC and requires an internal load-balancer; set the traefik annotation to create an internal-facing ELB.

   ```
   apps:
     traefik:
       enabled: true
       values: |
         service:
           annotations:
             service.beta.kubernetes.io/aws-load-balancer-internal: "true"
   ```

6. Enable NKP Catalog Applications and Install Kommander in the same `kommander.yaml` from the previous section, add these values (if you are enabling NKP Catalog Apps) for `nkp-catalog-appliations`.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.nutanix.io/project-default-catalog-repository: "true"
        kommander.nutanix.io/workspace-default-catalog-repository: "true"
        kommander.nutanix.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
```

7. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

> **Note:** If you only want to enable catalog applications to an existing configuration, add these values to an existing installer configuration file to maintain your Management cluster's settings.
>
> If you want to enable NKP Catalog applications after installing NKP, see Enable NKP Catalog Applications after Installing NKP.

GCP Management Tools

After cluster creation and configuration, you can revisit clusters to update and change variables.

Deleting a GCP Cluster

Deleting a GCP cluster.

About this task

A self-managed workload cluster cannot delete itself. If your workload cluster is self-managed, you must first create a bootstrap cluster and move the cluster life cycle services to it before deleting the workload cluster.

Procedure

If you did not make your workload cluster self-managed, as described in Make New Cluster Self-Managed, proceed to the instructions for Delete the workload cluster.

Create a Bootstrap Cluster and Move CAPI Resources

About this task

Follow these steps to create a bootstrap cluster and move CAPI resources:

Procedure

1. Make sure your AWS credentials are up to date. Refresh the credentials using this command.

```
nkp update bootstrap credentials aws --kubeconfig $HOME/.kube/config
```

2. The bootstrap cluster will host the Cluster API controllers that reconcile the cluster objects marked for deletion. Create a bootstrap cluster. To avoid using the wrong kubeconfig, the following steps use explicit kubeconfig paths and contexts.

```
nkp create bootstrap --kubeconfig $HOME/.kube/config --with-aws-bootstrap-
credentials=true
```

3. Move the Cluster API objects from the workload to the bootstrap cluster: The cluster life cycle services on the bootstrap cluster are ready, but the workload cluster configuration is on the workload cluster. The move command moves the configuration, which takes the form of Cluster API Custom Resource objects, from the workload to the bootstrap cluster. This process is also called a Pivot.

```
nkp move capi-resources \
 --from-kubeconfig ${CLUSTER_NAME}.conf \
 --from-context ${CLUSTER_NAME}-admin@${CLUSTER_NAME} \
 --to-kubeconfig $HOME/.kube/config \
 --to-context kind-konvoy-capi-bootstrapper
```

Output:

```
# Moving cluster resources
You can now view resources in the moved cluster by using the --kubeconfig flag with
 kubectl. For example: kubectl --kubeconfig $HOME/.kube/config get nodes
```

4. Use the cluster life cycle services on the workload cluster to check the workload cluster's status.

```
nkp describe cluster --kubeconfig $HOME/.kube/config -c ${CLUSTER_NAME}
```

Output:

```
     102s
```

5. Wait for the cluster control plan to be ready.

```
kubectl --kubeconfig $HOME/.kube/config wait --for=condition=controlplaneready
 "clusters/${CLUSTER_NAME}" --timeout=20m
```

Output:

```
NAME                                                                    READY
 SEVERITY  REASON  SINCE  MESSAGE
Cluster/gcp-example                                                     True
              34s
##ClusterInfrastructure - GCPCluster/gcp-example
##ControlPlane - KubeadmControlPlane/gcp-example-control-plane          True
              34s
# ##Machine/gcp-example-control-plane-6fbzn                             True
              37s
# # ##MachineInfrastructure - GCPMachine/gcp-example-control-plane-62g6s
# ##Machine/gcp-example-control-plane-jf6s2                            True
              37s
# # ##MachineInfrastructure - GCPMachine/gcp-example-control-plane-bsr2z
# ##Machine/gcp-example-control-plane-mnbfs                            True
              37s
#   ##MachineInfrastructure - GCPMachine/gcp-example-control-plane-s8xsx
##Workers
  ##MachineDeployment/gcp-example-md-0                                 True
              37s
    ##Machine/gcp-example-md-0-68b86fddb8-8glsw                        True
              37s
    # ##MachineInfrastructure - GCPMachine/gcp-example-md-0-zls8d
    ##Machine/gcp-example-md-0-68b86fddb8-bvbm7                        True
              37s
    # ##MachineInfrastructure - GCPMachine/gcp-example-md-0-5zcvc
```

```
##Machine/gcp-example-md-0-68b86fddb8-k9499                              True
          37s
# ##MachineInfrastructure - GCPMachine/gcp-example-md-0-k8h5p
##Machine/gcp-example-md-0-68b86fddb8-l6vfb                              True
          37s
  ##MachineInfrastructure - GCPMachine/gcp-example-md-0-9h5vn
```

Delete the Workload Cluster

Procedure

1. To delete a cluster, Use `nkp delete cluster` and pass in the name of the cluster you are trying to delete with `--cluster-name` flag. Use `kubectl get clusters` to get those details (`--cluster-name` and `--namespace`) of the Kubernetes cluster to delete it.

   ```
   kubectl get clusters
   ```

2. Delete the Kubernetes cluster and wait a few minutes.

   Note: Before deleting the cluster, Nutanix Kubernetes Platform (NKP) deletes all Services of type LoadBalancer on the cluster. An AWS Classic ELB backs each Service. Deleting the Service deletes the ELB that backs it. To skip this step, use the `--delete-kubernetes-resources=false` flag. Do not skip this step if NKP manages the VPC when NKP deletes the cluster, it deletes the VPC. If the VPC has any AWS Classic ELBs, AWS does not allow the VPC to be deleted, and NKP cannot delete the cluster.

   ```
   nkp delete cluster --cluster-name=${CLUSTER_NAME} --kubeconfig $HOME/.kube/config
   ```

   Output:

   ```
   # Deleting Services with type LoadBalancer for Cluster default/azure-example
   # Deleting ClusterResourceSets for Cluster default/azure-example
   # Deleting cluster resources
   # Waiting for cluster to be fully deleted
   Deleted default/azure-example cluster
   ```

   After the workload cluster is deleted, you can delete the bootstrap cluster.

Delete the Bootstrap Cluster

About this task

After you have moved the workload resources back to a bootstrap cluster and deleted the workload cluster, you no longer need the bootstrap cluster. You can safely delete the bootstrap cluster with these steps:

Procedure

Delete the bootstrap cluster.

```
nkp delete bootstrap --kubeconfig $HOME/.kube/config
```

Output:

```
# Deleting bootstrap cluster
```

Manage GCP Node Pools

Node pools are part of a cluster and managed as a group, and you can use a node pool to manage a group of machines using the same common properties. When Konvoy creates a new default cluster, there is one node pool for the worker nodes, and all nodes in that new node pool have the same configuration. You can create additional node pools for more specialized hardware or configuration. For example, if you want to tune your memory usage on a cluster where you need maximum memory for some machines and minimal memory on others, you create a new node pool with those specific resource needs.

Nutanix Kubernetes Platform (NKP) implements node pools using Cluster API MachineDeployments For more information on node pools, see these sections: .

Creating GCP Node Pools

Creating a node pool is useful when you need to run workloads that require machines with specific resources, such as a GPU, additional memory, or specialized network or storage hardware.

About this task

Availability zones (AZs) are isolated locations within datacenter regions where public cloud services originate and operate. Because all the nodes in a node pool are deployed in a single Availability Zone, you may wish to create additional node pools to ensure your cluster has nodes deployed in multiple Availability Zones.

The first task is to prepare the environment.

Procedure

1. Set the environment variable to the name you assigned this cluster.

   ```
   export CLUSTER_NAME=gcp-example
   ```

2. If your workload cluster is self-managed, as described in Make the New Cluster Self-Managed, configure kubectl to use the kubeconfig for the cluster.

   ```
   export KUBECONFIG=${CLUSTER_NAME}.conf
   ```

3. Define your node pool name.

   ```
   export NODEPOOL_NAME=example
   ```

Create a GCP Node Pool

Procedure

Set the `--zone` flag to a zone in the same region as your cluster. Create a new node pool with three replicas using this command.

```
nkp create nodepool gcp ${NODEPOOL_NAME} \
    --cluster-name=${CLUSTER_NAME} \
    --image $IMAGE_NAME \
    --zone us-west1-b \
    --replicas=3
```

This example uses default values for brevity. Use flags to define custom instance types, images, and other properties.

Advanced users can use a combination of the `--dry-run` and `--output=yaml` or `--output-directory=<existing-directory>` flags to get a complete set of node pool objects to modify locally or store in version control.

Listing GCP Node Pools

List the node pools of a given cluster. This returns specific properties of each node pool so that you can see the name of the MachineDeployments.

About this task

List node pools for a managed cluster.

Procedure

To list all node pools for a managed cluster, run:.

```
nkp get nodepools --cluster-name=${CLUSTER_NAME} --kubeconfig=${CLUSTER_NAME}.conf
```

The expected output is similar to the following example, indicating the desired size of the node pool, the number of replicas ready in the node pool, and the Kubernetes version those nodes are running:

```
NODEPOOL                          DESIRED              READY                KUBERNETES
 VERSION
example                           3                    3                    v1.29.6

gcp-example-md-0                  4                    4                    v1.29.6
```

Scaling GCP Node Pools

While you can run Cluster Autoscaler, you can also manually scale your node pools up or down when you need finite control over your environment.

About this task

If you require 10 machines to run a process, you can only manually set the scaling to run those 10 machines. However, if you also use Cluster Autoscaler, you must stay within your minimum and maximum bounds. This process allows you to scale manually.

Environment variables, such as defining the node pool name, are set in the Prepare the Environment section on the previous page. If needed, refer to that page to set those variables.

Scale Up Node Pools

Procedure

1. To scale up a node pool in a cluster, run one of the following.

   - Cluster:`nkp scale nodepools ${NODEPOOL_NAME} --replicas=5 --cluster-name=${CLUSTER_NAME}`
   - Attached Cluster:`nkp scale nodepools ${ATTACHED_NODEPOOL_NAME} --replicas=5 --cluster-name=${ATTACHED_CLUSTER_NAME} --kubeconfig=${CLUSTER_NAME}.conf -n ${ATTACHED_CLUSTER_WORKSPACE}`

   Example output indicating that scaling is in progress:

   ```
   # Scaling node pool example to 5 replicas
   ```

2. After a few minutes, you can list the node pools.

   ```
   nkp get nodepools --cluster-name=${CLUSTER_NAME} --kubeconfig=${CLUSTER_NAME}.conf
   ```

   Example output showing the number of DESIRED and READY replicas increased to 5:

   ```
   NODEPOOL                          DESIRED              READY
    KUBERNETES VERSION
   gcp-example-md-0                   5                    5                   v1.29.6

   gcp-attached-md-0                  5                    5                   v1.29.6
   ```

Scaling Down GCP Node Pools

While you can run Cluster Autoscaler, you can also manually scale your node pools down when you need more finite control over your environment.

About this task

If you require 10 machines to run a process, you can only manually set the scaling to run those 10 machines. However, if also using the Cluster Autoscaler, you must stay within your minimum and maximum bounds. This process allows you to scale manually.

Environment variables, such as defining the node pool name, are set in the Prepare the Environment section on the previous page. If needed, refer to that page to set those variables.

Procedure

1. To scale down a node pool, run.

   ```
   nkp scale nodepool ${NODEPOOL_NAME} --replicas=4 --cluster-name=${CLUSTER_NAME}
   ```

   Example output shows scaling is in progress.

   ```
   # Scaling node pool example to 4 replicas
   ```

2. After a few minutes, you can list the node pools.

   ```
   nkp get nodepools --cluster-name=${CLUSTER_NAME} --kubeconfig=${CLUSTER_NAME}.conf
   ```

   Example output showing that the number of DESIRED and READY replicas decreased to 4.

   ```
   NODEPOOL                            DESIRED               READY
    KUBERNETES VERSION
   example                             4                     4                    v1.29.6

   gcp-example-md-0                    4                     4                    v1.29.6
   ```

3. In a default cluster, the nodes to delete are selected at random. • CAPI's delete policy controls this behavior. However, when using the Nutanix Kubernetes Platform (NKP) CLI to scale down a node pool, it is also possible to specify the Kubernetes Nodes you want to delete.

   To do this, set the flag `--nodes-to-delete` with a list of nodes as below. This adds an annotation `cluster.x-k8s.io/delete-machine=yes` to the matching Machine object that contains `status.NodeRef` with the node names from `--nodes-to-delete`.

   ```
   nkp scale nodepools ${NODEPOOL_NAME} --replicas=3 --nodes-to-delete=<> --cluster-
   name=${CLUSTER_NAME}
   ```

   Output:

   ```
   # Scaling node pool example to 3 replicas
   ```

Deleting GCP Node Pools

Deleting a node pool deletes the Kubernetes nodes and the underlying infrastructure.

About this task

All nodes will be drained before deletion, and the pods running on those nodes will be rescheduled.

Procedure

1. To delete a node pool from a managed cluster, run.

   ```
   nkp delete nodepool ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME}
   ```

   Here, `example` is the node pool to be deleted.

   The expected output will be similar to the following example, indicating the node pool is being deleted:

   ```
   # Deleting default/example nodepool resources
   ```

2. Deleting an invalid node pool results in output similar to this example.

   ```
   nkp delete nodepool ${CLUSTER_NAME}-md-invalid --cluster-name=${CLUSTER_NAME}
   ```

   Output:

   ```
   MachineDeployments or MachinePools.infrastructure.cluster.x-k8s.io "no
    MachineDeployments or MachinePools found for cluster aws-example" not found
   ```

7

ADDITIONAL KOMMANDER CONFIGURATION

The Kommander component of NKP can be configured differently depending on your environment type and other desired customizations. This section contains instructions for installing the Kommander component in air-gapped, non-air-gapped, or pre-provisioned environments. It also contains instructions on how to enable NKP catalog apps or NKP Insights, if desired.

For more information on installing Kommander by environment, see Kommander Installation Based on Your Environment on page 1016.

This section contains instructions for enabling a custom configuration of a Kommander component,see such as custom domains or certificates, an HTTP Proxy, an external load balancer, etc.

Kommander Installation Based on Your Environment

This section provides installation instructions for the Kommander component of NKP according to your environment type.

Pre-checks Before Installation

Before you install the Kommander component, perform the following checks:

- Whether your environment Air-gapped or Non-Air-gapped?

  In an **air-gapped environment**, your environment is isolated from unsecured networks, like the Internet. In a **non-air-gapped environment**, your environment has two-way access to and from the Internet.

  For more information, see Installing Kommander in an Air-gapped Environment on page 1017 and Installing Kommander in a Non-Air-gapped Environment on page 1021.

- Your License Type

  NKP Pro and **NKP Government** Pro are self-managed single-cluster Kubernetes solutions that give you a feature-rich, easy-to-deploy, and easy-to-manage entry-level cloud container platform. The NKP Pro and NKP Gov licenses give the user access to the entire Konvoy cluster environment, as well as manage the Kommander platform application manager.

  NKP Ultimate and **NKP Government Advanced** are multi-cluster solutions centered around a management cluster that manage multiple attached or managed Kubernetes clusters through a centralized management dashboard. For this license type, you will determine whether or not to use the NKP Catalog applications.

  For more information, see Licenses on page 24.

- Whether you want to enable NKP Insights, for more information, see Nutanix Kubernetes Platform Insights Guide on page 1217.

  NKP Insights is a predictive analytics capability that detects anomalies that occur either in the present or future and generates an alert in the NKP UI.

- The AI Navigator is an AI chatbot that offers real-time, interactive communication to answer a wide range of user queries, spanning basic instructions to complex functionalities. NKP installs this application by default in non-air-gapped environments. However, you can disable it, if desired, as part of the installation of the NKP Kommander component.

  Typically, we recommend that you generate a Kommander configuration file so that you can customize the configuration prior to installing Kommander. You can use this method to disable AI Navigator. If you prefer to use a CLI approach to the installation, there is also a flag for disabling the application.

  Note: For security purposes, AI Navigator is not installed for air-gapped environments.

Installation Type Based on Your Environment

Select the installation type according to your environment:

- Installing Kommander in an Air-gapped Environment on page 1017
- Installing Kommander in a Non-Air-gapped Environment on page 1021
- Installing Kommander in a Pre-provisioned Air-gapped Environment on page 1024
- Installing Kommander in a Pre-provisioned, Non-Air-gapped Environment on page 1029
- Installing Kommander in a Small Environment on page 1032

Installing Kommander in an Air-gapped Environment

Follow these steps to install the Kommander component of NKP in an air-gapped environment.

Before you begin

- Ensure you have reviewed all the Prerequisites for Installation on page 47.
- Ensure you have a default StorageClass.
- Ensure you have loaded all necessary images for your configuration. For more information, see Images Download into Your Registry: Air-gapped Environments on page 1019.
- Note down the name of the cluster where you want to install Kommander. If you do not know it, use `kubectl get clusters -A` to display it.

About this task

Create your Kommander Installer Configuration File as follows:

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

```
nkp install kommander --init --airgapped > kommander.yaml
```

4. If required, customize your `kommander.yaml` file.

   For customization options, see Installing Kommander with a Configuration File on page 1036.

   Some of them include:

   - Custom Domains and Certificates
   - HTTP proxy
   - External Load Balancer
   - GPU utilization, etc.
   - Rook Ceph customization for Pre-provisioned environments

5. If required: If your cluster uses a custom **AWS VPC** and requires an internal load-balancer, set the `traefik` annotation to create an internal-facing ELB.

```
...
apps:
  traefik:
    enabled: true
    values: |
      service:
        annotations:
          service.beta.kubernetes.io/aws-load-balancer-internal: "true
...
```

6. Expand one of the following sets of instructions, depending on your license and application environments.

   If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful.

   Tips and recommendations:

   - The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you **install Kommander on the correct cluster**. For alternatives, see Commands within a kubeconfig File on page 32.
   - Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period (for example, `1h`) to allocate more time to the deployment of applications.
   - If the Kommander installation fails, or you want to reconfigure applications, rerun the `install` command to retry.

What to do next

See Verifying Kommander Installation on page 1034.

If you want to enable a solution that detects current and future anomalies in workload configurations or Kubernetes clusters, see Nutanix Kubernetes Platform Insights Guide on page 1217.

Pro License: Installing Kommander in an Air-gapped Environment
Use the customized `kommander.yaml` to install NKP in an air-gapped environment.

Procedure

In the `kommander.yaml` file, run the following command.

```
nkp install kommander \
--installer-config kommander.yaml --kubeconfig=${CLUSTER_NAME}.conf \
--kommander-applications-repository ./application-repositories/kommander-applications-
v2.12.0.tar.gz \
--charts-bundle ./application-charts/nkp-kommander-charts-bundle-v2.12.0.tar.gz
```

Ultimate License: Installing Kommander in an Air-gapped Environment
You can install Kommander in an Air-gapped Environment with NKP Catalog Applications.

About this task

> **Note:** If you want to enable NKP Catalog applications after installing NKP, see NKP Catalog Applications Enablement after Installing NKP on page 1063.

Procedure

1. In the same `kommander.yaml` of the previous section, add the following values to enable NKP Catalog Applications.

   ```
   ...
   catalog:
     repositories:
       - name: dkp-catalog-applications
         labels:
           kommander.d2iq.io/project-default-catalog-repository: "true"
           kommander.d2iq.io/workspace-default-catalog-repository: "true"
           kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
         path: ./dkp-catalog-applications-v2.12.0.tar.gz
   ...
   ```

   > **Warning:** If you only want to enable catalog applications to an **existing** configuration, add these values to an **existing** installer configuration file to maintain your Management cluster's settings.

2. Use the customized `kommander.yaml` to install NKP.

   ```
   nkp install kommander \
   --installer-config kommander.yaml --kubeconfig=${CLUSTER_NAME}.conf \
   --kommander-applications-repository ./application-repositories/kommander-
   applications-v2.12.0.tar.gz \
   --charts-bundle ./application-charts/nkp-kommander-charts-bundle-v2.12.0.tar.gz \
   --charts-bundle ./application-charts/dkp-catalog-applications-charts-bundle-
   v2.12.0.tar.gz
   ```

Images Download into Your Registry: Air-gapped Environments
Because air-gapped environments do not have direct access to the Internet, you must download, extract, and load the required images before installing NKP.

For more information on downloading images, see Registry Mirror Tools on page 1069.

Downloading all Images for Air-gapped Deployments

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images, is required. See below for prerequisites to download and then how to push the necessary images to this registry.

Procedure

1. Download the NKP air-gapped bundle for this release to load registry images as explained below. For more information, see Downloading NKP on page 16.

   ```
   nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
   ```

2. Connectivity with clusters attaching to the management cluster is required.

   - Both management and attached clusters must be able to connect to the local registry.
   - The management cluster must be able to connect to all the attached cluster API servers.
   - The management cluster must be able to connect to any load balancers created for platform services on the management cluster.

Extracting Air-gapped Images and Set Variables

Follow these steps to extract the air-gapped image bundles into your private registry:

Procedure

1. Assuming you have downloaded `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`, extract the tar file to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
   ```

2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. EX: For the bootstrap, change your directory to the `nkp-<version>`directory, similar to the example below, depending on your current location.

   ```
   cd nkp-v2.12.0
   ```

3. Set an environment variable with your registry address.

   ```
   export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
   export REGISTRY_USERNAME=<username>
   export REGISTRY_PASSWORD=<password>
   ```

Loading Images to Your Private Registry - Konvoy

Before creating or upgrading a Kubernetes cluster, you need to load the required images in a local registry if operating in an air-gapped environment. This registry must be accessible from both the bastion machine and either the AWS EC2 instances or other machines that will be created for the Kubernetes cluster.

About this task

For more information on creating a bastion machine, see Creating a Bastion Host on page 685.

> **Warning:** If you do not already have a local registry, set up one. For more information, see Registry Mirror Tools on page 1069.

Procedure

Run the following command to load the air-gapped image bundle into your private registry.

```
nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-
registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-
password=${REGISTRY_PASSWORD}
```

> **Note:** It might take some time to push all the images to your image registry, depending on the performance of the network between the machine you are running the script on and the registry.

Loading Images to Your Private Registry - Kommander

Load Kommander images to your Private Registry

Procedure

For the air-gapped `kommander` image bundle, run the command below. Run the following command to load the image bundle.

```
nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-
registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-
password=${REGISTRY_PASSWORD}
```

Loading Images to Your Private Registry - NKP Catalog Applications

Optional: This step is required only if you have an Ultimate license.

About this task

For **NKP Catalog Applications**, also perform an image load:

Procedure

Run the following command to load the `dkp-catalog-applications` image bundle into your private registry.

```
nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-
v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME}
 --to-registry-password=${REGISTRY_PASSWORD}
```

It can take a while to push all the images to your image registry, depending on the performance of the network between the machine you are running the script on and the registry.

Installing Kommander in a Non-Air-gapped Environment

Install the Kommander components of NKP non-air-gapped environments.

Before you begin

- Ensure you have reviewed all Identifying and Modifying Your StorageClass on page 1038.
- Ensure you have a default StorageClass. See Identifying and Modifying Your StorageClass on page 1038.
- Note down the name of the cluster where you want to install Kommander. If you do not know it, use `kubectl get clusters -A` to display it.

About this task

Create your Kommander Installer Configuration File as follows:

Procedure

1. Set the environment variable for your cluster.

   ```
   export CLUSTER_NAME=<your-management-cluster-name>
   ```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

   ```
   nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
   ```

3. Create a configuration file for the deployment.

   ```
   nkp install kommander --init > kommander.yaml
   ```

4. If required, customize your `kommander.yaml` file.

   For customization options, see Installing Kommander with a Configuration File on page 1036.

   Some of them include:

   - Custom Domains and Certificates
   - HTTP proxy
   - Disabling the AI Navigator application
   - External Load Balancer
   - GPU utilization, etc.

   For more information on installing Kommanderthe dataset, Installing Kommander in a Pre-provisioned Air-gapped Environment on page 1024.

5. If required: If your cluster uses a custom **AWS VPC** and requires an internal load-balancer, set the `traefik` annotation to create an internal-facing ELB.

   ```
   ...
   apps:
     traefik:
       enabled: true
       values: |
         service:
           annotations:
             service.beta.kubernetes.io/aws-load-balancer-internal: "true
   ...
   ```

6. Expand one of the following sets of instructions, depending on your license and application environments.

   If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in.

   Tips and recommendations:

   - The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you **install Kommander on the correct cluster**. For alternatives and to Provide Context for Commands with a kubeconfig File, see Commands within a kubeconfig File on page 32.
   - Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period (for example, `1h`) to allocate more time to the deployment of applications.
   - If the Kommander installation fails, or you want to reconfigure applications, rerun the `install` command to retry.

What to do next

See Verifying Kommander Installation on page 1034.

If you want to enable a solution that detects current and future anomalies in workload configurations or Kubernetes clusters, see Nutanix Kubernetes Platform Insights Guide on page 1217.

Pro License: Installing Kommander in a Non-Air-gapped Environment

Use the customized `kommander.yaml` to install NKP in a non-air-gapped environment.

Procedure

In the `kommander.yaml` file, run the following command.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

Ultimate License: Installing Kommander in Non-Air-gapped with NKP Catalog Applications

You can install Kommander in a non-air-gapped Environment with NKP Catalog Applications.

About this task

> **Note:** If you want to enable NKP Catalog applications after installing NKP, see NKP Catalog Applications Enablement after Installing NKP on page 1063.

Procedure

1. In the same `kommander.yaml` of the previous section, add the following values to enable NKP Catalog Applications.

```
...
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
```

```
          tag: v2.12.0
...
```

> **Warning:** If you only want to enable catalog applications to an **existing** configuration, add these values to an **existing** installer configuration file to maintain your Management cluster's settings.

2. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

Installing Kommander in a Pre-provisioned Air-gapped Environment

Follow these steps to install the Kommander component of NKP in a Pre-provisioned environment.

Before you begin

- Ensure you have reviewed all Identifying and Modifying Your StorageClass on page 1038.
- Ensure you have a default StorageClass. See Identifying and Modifying Your StorageClass on page 1038.
- Note down the name of the cluster where you want to install Kommander. If you do not know it, use `kubectl get clusters -A` to display it.
- Ensure you have completed all the Prerequisites for Installation on page 47.
- Ensure you have a default StorageClass. See Identifying and Modifying Your StorageClass on page 1038.
- Ensure you have loaded all necessary images for your configuration. See Images Download into Your Registry: Air-gapped Environments on page 1019.
- Note down the name of the cluster where you want to install Kommander. If you do not know it, use `kubectl get clusters -A` to display it.

About this task

Create and customize your Kommander Installer Configuration File as follows:

Procedure

1. Set the environment variable for your cluster.

```
export CLUSTER_NAME=<your-management-cluster-name>
```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

3. Create a configuration file for the deployment.

   For more information, see Installing Kommander with a Configuration File on page 1036.

```
nkp install kommander --init > kommander.yaml
```

4. Edit the installer file to include configuration overrides for the `rook-ceph-cluster`.

   NKP's default configuration ships Ceph with PVC based storage (see https://rook.io/docs/rook/v1.10/CRDs/Cluster/pvc-cluster/) which requires your CSI provider to support PVC with type `volumeMode: Block`. As this is not possible with the default local static provisioner (see Default Storage Providers on

page 34), you can install Ceph in host storagemode (see https://rook.io/docs/rook/v1.10/CRDs/Cluster/host-cluster/).

You can choose whether Ceph's object storage daemon (osd) pods should consume all or just some of the devices on your nodes. Include one of the following Overrides.

a. To automatically assign all raw storage devices on all nodes to the Ceph cluster.

```
...
rook-ceph-cluster:
    enabled: true
    values: |
      cephClusterSpec:
        storage:
          storageClassDeviceSets: []
          useAllDevices: true
          useAllNodes: true
          deviceFilter: "<<value>>"
...
```

b. To assign specific storage devices on all nodes to the Ceph cluster.

```
...
rook-ceph-cluster:
    enabled: true
    values: |
      cephClusterSpec:
        storage:
          storageClassDeviceSets: []
          useAllNodes: true
          useAllDevices: false
          deviceFilter: "^sdb."
...
```

Note: If you want to assign specific devices to specific nodes using the `deviceFilter` option, see https://rook.io/docs/rook/v1.10/CRDs/Cluster/host-cluster/#specific-nodes-and-devices.

Note: For general information on the `deviceFilter` value, see https://rook.io/docs/rook/v1.10/CRDs/Cluster/ceph-cluster-crd/#storage-selection-settings.

5. If required, customize your `kommander.yaml` file.

   For customization options, see Installing Kommander with a Configuration File on page 1036

   Some of them include:

   - Custom Domains and Certificates
   - HTTP proxy
   - External Load Balancer
   - GPU utilization, etc.
   - Rook Ceph customization for Pre-provisioned environments. See Installing Kommander with a Configuration File on page 1036.

6. If required: If your cluster uses a custom **AWS VPC** and requires an internal load-balancer, set the `traefik` annotation to create an internal-facing ELB.

```
...
apps:
...
  traefik:
```

```
    enabled: true
    values: |
      service:
        annotations:
          service.beta.kubernetes.io/aws-load-balancer-internal: "true
...
```

7. Expand one of the following sets of instructions, depending on your license and application environments.

   If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful.

   Tips and recommendations:

   - The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Commands within a kubeconfig File on page 32.
   - Applications can take longer to deploy and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period (for example, `1h`) to allocate more time to the deployment of applications.
   - If the Kommander installation fails, or you want to reconfigure applications, rerun the `install` command to retry.

What to do next

See Verifying Kommander Installation on page 1034.

If you want to enable a solution that detects current and future anomalies in workload configurations or Kubernetes clusters, see Nutanix Kubernetes Platform Insights Guide on page 1217.

Pro License: Installing Kommander in a Pre-provisioned Air-gapped Environment
Use the customized `kommander.yaml` to install NKP in a non-air-gapped environment.

Procedure

In the `kommander.yaml` file, run the following command.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf \
--kommander-applications-repository ./application-repositories/kommander-applications-
v2.12.0.tar.gz \
--charts-bundle ./application-charts/nkp-kommander-charts-bundle-v2.12.0.tar.gz
```

Ultimate License: Installing Kommander in a Pre-provisioned, Air-gapped Environment
You can install Kommander in a non-air-gapped Environment with NKP Catalog Applications.

About this task

> **Note:** If you want to enable NKP Catalog applications after installing NKP, see NKP Catalog Applications Enablement after Installing NKP on page 1063.

Procedure

1. In the same `kommander.yaml` of the previous section, add the following values to enable NKP Catalog Applications.

```
...
catalog:
  repositories:
    - name: dkp-catalog-applications
```

```
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      path: ./dkp-catalog-applications-v2.12.0.tar.gz
...
```

Warning: If you only want to enable catalog applications to an **existing** configuration, add these values to an **existing** installer configuration file to maintain your Management cluster's settings.

2. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf \
--kommander-applications-repository ./application-repositories/kommander-
applications-v2.12.0.tar.gz \
--charts-bundle ./application-charts/nkp-kommander-charts-bundle-v2.12.0.tar.gz \
--charts-bundle ./application-charts/dkp-catalog-applications-charts-bundle-
v2.12.0.tar.gz
```

Images Download into Your Registry: Air-gapped, Pre-provisioned Environments
Because air-gapped environments do not have direct access to the Internet, you must download, extract, and load the required images before installing NKP.

For more information on downloading images, see Registry Mirror Tools on page 1069.

Downloading all Images for Air-gapped Pre-provisioned Deployments

If you are operating in an air-gapped environment, the local container registry containing all the necessary installation images, including the Kommander images, is required. See below for prerequisites to download and then how to push the necessary images to this registry.

Procedure

1. Download the NKP air-gapped Bundle for this release (that is. `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`) to load registry images as explained below. See Downloading NKP on page 16.
2. Connectivity with clusters attaching to the management cluster is required.
   - Both management and attached clusters must be able to connect to the local registry.
   - The management cluster must be able to connect to all the attached cluster's API servers.
   - The management cluster must be able to connect to any load balancers created for platform services on the management cluster.

Extracting Air-gapped Pre-provisioned Images and Set Variables

Follow these steps to extract the air-gapped image bundles into your private registry:

Procedure

1. Assuming you have downloaded `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`, extract the tar file to a local directory.

```
tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
```

2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. EX: For the bootstrap, change your directory to the `nkp-<version>`directory similar, to the example below, depending on your current location.

```
cd nkp-v2.12.0
```

3. Set an environment variable with your registry address.

```
export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
export REGISTRY_USERNAME=<username>
export REGISTRY_PASSWORD=<password>
```

(Only Pre-provisioned) Loading Images for Deployments - Konvoy

For Pre-provisioned air-gapped environments only, you must run `konvoy-image upload artifacts` and copy the artifacts onto the cluster hosts before you begin upgrading the CAPI components process.

Procedure

1. The Kubernetes image bundle will be located in `kib/artifacts/images` , and you will want to verify the image and artifacts.

   a. Verify the image bundles exist in `kib/artifacts/images`.

   ```
   $ ls kib/artifacts/images/
   kubernetes-images-1.29.6-d2iq.1.tar kubernetes-images-1.29.6-d2iq.1-fips.tar
   ```

   b. Verify the artifacts for your OS exist in the `kib/artifacts/`directory and export the appropriate variables.

   ```
   $ ls artifacts/
   1.29.6_centos_7_x86_64.tar.gz     1.29.6_redhat_8_x86_64_fips.tar.gz
    containerd-1.6.28-d2iq.1-rhel-8.6-x86_64_fips.tar.gz pip-packages.tar.gz
   1.29.6_centos_7_x86_64_fips.tar.gz  1.29.6_rocky_9_x86_64.tar.gz
    containerd-1.6.28-d2iq.1-rocky-9.0-x86_64.tar.gz
   1.29.6_redhat_7_x86_64.tar.gz     1.29.6_ubuntu_20_x86_64.tar.gz
    containerd-1.6.28-d2iq.1-rocky-9.1-x86_64.tar.gz
   1.29.6_redhat_7_x86_64_fips.tar.gz  containerd-1.6.28-d2iq.1-ubuntu-20.04-
   x86_64.tar.gz
   1.29.6_redhat_8_x86_64.tar.gz
   ```

   c. Set the bundle values with the name from the private registry location.

   ```
   export OS_PACKAGES_BUNDLE=<name_of_the_OS_package>
   export CONTAINERD_BUNDLE=<name_of_the_containerd_bundle>
   ```

   For example, for RHEL 8.6, set.

   ```
   export OS_PACKAGES_BUNDLE=1.29.6_redhat_8_x86_64.tar.gz
   export CONTAINERD_BUNDLE=containerd-1.6.28-d2iq.1-rhel-8.6-x86_64.tar.gz
   ```

2. Upload the artifacts onto cluster hosts.

```
konvoy-image upload artifacts \
    --container-images-dir=./kib/artifacts/images/ \
    --os-packages-bundle=./kib/artifacts/${OS_PACKAGES_BUNDLE} \
    --containerd-bundle=./kib/artifacts/${CONTAINERD_BUNDLE} \
    --pip-packages-bundle=./kib/artifacts/pip-packages.tar.gz
```

Loading Images to Your Private Registry - Konvoy

Before creating or upgrading a Kubernetes cluster, you need to load the required images in a local registry if operating in an air-gapped environment. This registry must be accessible from both the bastion machine and either the AWS EC2 instances or other machines that will be created for the Kubernetes cluster.

About this task

For more information on creating a bastion machine, see Creating a Bastion Host on page 685.

> **Warning:** If you do not already have a local registry, set up one. For more information, see Registry Mirror Tools on page 1069.

Procedure

Run the following command to load the air-gapped image bundle into your private registry.

```
nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-
registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-
password=${REGISTRY_PASSWORD}
```

> **Note:** It might take some time to push all the images to your image registry, depending on the performance of the network between the machine you are running the script on and the registry.

Loading Images to Your Private Registry - Kommander

Load Kommander images to your Private Registry

Procedure

For the air-gapped `kommander` image bundle, run the command below. Run the following command to load the image bundle.

```
nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-
registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-
password=${REGISTRY_PASSWORD}
```

Installing Kommander in a Pre-provisioned, Non-Air-gapped Environment

Install the Kommander components of NKP.

Before you begin

For more information on pre-provisioned environments, see Pre-provisioned Installation Options on page 69. For more information on non-air-gapped environments, see Air-Gapped or Non-Air-Gapped Environment on page 22.

- Ensure you have reviewed all the prerequisites for installation (see Prerequisites for Installation on page 47).
- Ensure you have a default StorageClass. For more information, see
- Note down the name of the cluster where you want to install Kommander. If you do not know it, use `kubectl get clusters -A` to display it.

> **Warning:** You must modify the Kommander installer configuration file (`kommander.yaml`) before installing the Kommander component of NKP in a pre-provisioned environment.

About this task

Create and customize your Kommander Installer Configuration File as follows:

Procedure

1. Set the environment variable for your cluster.

```
export CLUSTER_NAME=<your-management-cluster-name>
```

2. Copy the `kubeconfig` file of your Management cluster to your local directory.

```
nkp get kubeconfig -c ${CLUSTER_NAME} > ${CLUSTER_NAME}.conf
```

3. Create a configuration file.
   For more information, see Installing Kommander with a Configuration File on page 1036.

```
nkp install kommander --init --airgapped > kommander.yaml
```

4. If required, customize your `kommander.yaml` file.

   The customization options include:

   - Custom Domains and Certificates
   - HTTP proxy
   - Disabling the AI Navigator application
   - External Load Balancer
   - GPU utilization, etc.
   - Rook Ceph customization for Pre-provisioned environments. See Installing Kommander in a Pre-provisioned Air-gapped Environment on page 1024.

5. If required: If your cluster uses a custom **AWS VPC** and requires an internal load-balancer, set the `traefik` annotation to create an internal-facing ELB.

```
...
apps:
  traefik:
    enabled: true
    values: |
      service:
        annotations:
          service.beta.kubernetes.io/aws-load-balancer-internal: "true
...
```

6. Expand one of the following sets of instructions, depending on your license and application environments.

   If your environment uses HTTP/HTTPS proxies, you must include the flags `--http-proxy`, `--https-proxy`, and `--no-proxy` and their related values in this command for it to be successful. More information is available in

   Configuring an HTTP/HTTPS Proxy.

   Tips and recommendations:

   - The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you install Kommander on the correct cluster. For alternatives, see Commands within a kubeconfig File on page 32.
   - Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, `1h`) to allocate more time to the deployment of applications.
   - If the Kommander installation fails, or you want to reconfigure applications, rerun the `install` command to retry.

What to do next

See Verifying Kommander Installation on page 1034.

If you want to enable a solution that detects current and future anomalies in workload configurations or Kubernetes clusters, see Nutanix Kubernetes Platform Insights Guide on page 1217.

Pro License: Installing Kommander in a Pre-provisioned, Non-Air-gapped Environment

Use the customized `kommander.yaml` to install NKP in a non-air-gapped environment.

Procedure

In the `kommander.yaml` file, run the following command.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

Ultimate License: Installing Kommander in Pre-provisioned, Non-Air-gapped with NKP Catalog Applications

You can install Kommander in a non-air-gapped Environment with NKP Catalog Applications.

About this task

> **Note:** If you want to enable NKP Catalog applications after installing NKP, see NKP Catalog Applications Enablement after Installing NKP on page 1063.

Procedure

1. In the same `kommander.yaml` of the previous section, add the following values to enable NKP Catalog Applications.

```
...
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
```

```
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
...
```

> **Warning:** If you only want to enable catalog applications to an **existing** configuration, add these values to an **existing** installer configuration file to maintain your Management cluster's settings.

2. Use the customized `kommander.yaml` to install NKP.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

Installing Kommander in a Small Environment

You can install the Kommander component of NKP on a small, single-cluster environment with smaller memory, storage, and CPU requirements for testing and demo purposes. This topic describes methods for installing Kommander in these environments.

About this task

Minimal Kommander installation:

The YAML file that is used to install a minimal configuration of Kommander contains the bare minimum setup that allows you to deploy applications and access the NKP UI. It does not include applications for cost monitoring, logging, alerting, object storage, etc.

In this YAML file, you can find the lines that correspond to all platform applications included in a normal Kommander setup. Applications that have `enabled` set to `false` are not taken into account during installation. If you want to test an additional application, you can enable it individually to be installed by setting `enabled` to `true` on the corresponding line in the YAML file.

For example, if you want to enable the logging stack, set `enabled` to `true` for `grafana-logging`, `grafana-lokilogging-operator`, `rook-ceph` and `rook-ceph-cluster`. Note that depending on the size of your cluster, enabling several platform applications can exhaust your cluster's resources.

> **Warning:** Some applications depend on other applications to work properly. To find out which other applications you need to enable to test the target application, see See Platform Applications on page 400

.

Before you begin

> **Warning:** Ultimate considerations: Nutanix recommends performing testing and demo tasks in a single-cluster environment. The Ultimate license is designed for multi-cluster environments and fleet management, which require a minimum number of resources. Applying an Ultimate license key to the previous installation adds modifications to your environment that can exhaust a small environment's resources.

Ensure you have done the following:

- You have acquired a NKP license.
- You have installed Basic Installations by Infrastructure on page 52.
- You have reviewed the prerequisite section pertaining to your air-gapped, or networked environment.

Procedure

1. Initialize your Kommander installation and name it `kommander_minimal.yaml`.

```
nkp install kommander --init --kubeconfig=${CLUSTER_NAME}.conf -oyaml >
 kommander_minimal.yaml
```

2. Edit your `kommander_minimal.yaml` file to match the following example.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
apps:
  dex:
    enabled: true
  dex-k8s-authenticator:
    enabled: true
  nkp-insights-management:
    enabled: false
  gatekeeper:
    enabled: true
  git-operator:
    enabled: true
  grafana-logging:
    enabled: false
  grafana-loki:
    enabled: false
  kommander:
    enabled: true
  kommander-ui:
    enabled: true
  kube-prometheus-stack:
    enabled: false
  kubernetes-dashboard:
    enabled: false
  kubefed:
    enabled: true
  kubetunnel:
    enabled: false
  logging-operator:
    enabled: false
  prometheus-adapter:
    enabled: false
  reloader:
    enabled: true
  rook-ceph:
    enabled: false
  rook-ceph-cluster:
    enabled: false
  traefik:
    enabled: true
  traefik-forward-auth-mgmt:
    enabled: true
  velero:
    enabled: false
ageEncryptionSecretName: sops-age
clusterHostname: ""
```

3. Install Kommander on your cluster with the following command.

```
nkp install kommander --installer-config ./kommander_minimal.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

In the previous command, the `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you set the context to install Kommander on the right cluster.for installing For alternatives and recommendations around setting your context, see Commands within a kubeconfig File on page 32 .

> **Tip:** Sometimes, applications require a longer period of time to deploy, which causes the installation to time out. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, `1h`) to allocate more time to the deployment of applications.

If the Kommander installation fails, or you wish to reconfigure applications, you can rerun the install command, and you can view the progress by increasing the log verbosity by adding the flag `-v 2`.

Verifying Kommander Installation

After you build the Konvoy cluster and you install Kommander, verify your installation. After the CLI successfully installs the components, it waits for all applications to be ready by default.

About this task

> **Note:** If the Kommander installation fails or you wishwant to reconfigure applications, you can rerun the install command to retry the installation.

Procedure

1. Do one of the following.

   » If you prefer the CLI not to wait for all applications to become ready, you can set the `--wait=false` flag.

   » If you choose not to wait through the NKP CLI, you can check the status of the installation using the following command:

   ```
   kubectl -n kommander wait --for condition=Ready helmreleases --all --timeout 15m
   ```

   This will wait for each of the helm charts to reach their `Ready` condition, eventually resulting in an output resembling below:

   ```
   helmrelease.helm.toolkit.fluxcd.io/centralized-grafana condition met
   helmrelease.helm.toolkit.fluxcd.io/dex condition met
   helmrelease.helm.toolkit.fluxcd.io/dex-k8s-authenticator condition met
   helmrelease.helm.toolkit.fluxcd.io/fluent-bit condition met
   helmrelease.helm.toolkit.fluxcd.io/grafana-logging condition met
   helmrelease.helm.toolkit.fluxcd.io/grafana-lokiLoki condition met
   helmrelease.helm.toolkit.fluxcd.io/karma condition met
   helmrelease.helm.toolkit.fluxcd.io/kommander condition met
   helmrelease.helm.toolkit.fluxcd.io/kommander-appmanagement condition met
   helmrelease.helm.toolkit.fluxcd.io/kube-prometheus-stack condition met
   helmrelease.helm.toolkit.fluxcd.io/kubecost condition met
   helmrelease.helm.toolkit.fluxcd.io/kubecost-thanos-traefik condition met
   helmrelease.helm.toolkit.fluxcd.io/kubefed condition met
   helmrelease.helm.toolkit.fluxcd.io/kubernetes-dashboard condition met
   helmrelease.helm.toolkit.fluxcd.io/kubetunnel condition met
   helmrelease.helm.toolkit.fluxcd.io/logging-operator condition met
   helmrelease.helm.toolkit.fluxcd.io/logging-operator-logging condition met
   helmrelease.helm.toolkit.fluxcd.io/prometheus-adapter condition met
   helmrelease.helm.toolkit.fluxcd.io/prometheus-thanos-traefik condition met
   helmrelease.helm.toolkit.fluxcd.io/reloader condition met
   helmrelease.helm.toolkit.fluxcd.io/rook-ceph condition met
   ```

```
helmrelease.helm.toolkit.fluxcd.io/rook-ceph-cluster condition met
helmrelease.helm.toolkit.fluxcd.io/thanos condition met
helmrelease.helm.toolkit.fluxcd.io/traefik condition met
helmrelease.helm.toolkit.fluxcd.io/traefik-forward-auth-mgmt condition met
helmrelease.helm.toolkit.fluxcd.io/velero condition met
```

2. In case of failed HelmReleases, do one of the following.

   » If an application fails to deploy, check the status of a `HelmRelease` with:

   ```
   kubectl -n kommander get helmrelease <HELMRELEASE_NAME
   ```

   » If you find any `HelmReleases` in a “broken” release state, such as “exhausted” or “another rollback/release in progress” trigger a reconciliation of the `HelmRelease` using the following commands:

   ```
   kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -
   p='[{"op": "replace", "path": "/spec/suspend", "value": true}]'
   kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -
   p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'
   ```

What to do next

After building the Konvoy cluster and installing Kommander, you can verify your Kommander installation Logging into the UI with Kommander on page 1037.

Configuring Applications After Installing Kommander

After you have a default configuration file, you can then configure each `app` either inline or by referencing another YAML file. The configuration values for each `app` correspond to the Helm Chart values for the application.

Procedure

After the initial deployment of Kommander, you can find the application Helm Charts by checking the `spec.chart.spec.sourceRef` field of the associated `HelmRelease`.

```
kubectl get helmreleases <application> -o yaml -n kommander
```

Inline configuration (using values) :

In this example, you configure the `centralized-grafana` application with resource limits by defining the Helm Chart values in the Kommander configuration file.

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
apps:
  centralized-grafana:
    values: |
      grafana:
        resources:
          limits:
            cpu: 150m
            memory: 100Mi
          requests:
            cpu: 100m
            memory: 50Mi
...
```

Reference another YAML file (using valuesFrom):

Alternatively, you can create another YAML file containing the configuration for `centralized-grafana` and reference that using `valuesFrom`. You can point to this file by using either a relative path (from the configuration file location) or by using an absolute path.

```
cat > centralized-grafana.yaml <<EOF
grafana:
  resources:
    limits:
      cpu: 150m
      memory: 100Mi
    requests:
      cpu: 100m
      memory: 50Mi
EOF
```

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
apps:
  centralized-grafana:
    valuesFrom: centralized-grafana.yaml
...
```

Installing Kommander with a Configuration File

About this task

In the following command, the `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you set the context for installing Kommander on the right cluster. For alternatives and recommendations around setting your context, see Commands within a kubeconfig File on page 32.

Procedure

1. Add the `--installer-config` flag to the `kommander install` command to use a custom configuration file.
2. To reconfigure applications, you can also run this command after the initial installation.

   ```
   nkp install kommander --installer-config kommander.yaml --kubeconfig=
   ${CLUSTER_NAME}.conf
   ```

   Tip: Sometimes, applications require a longer period of time to deploy, which causes the installation to time out. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, `1h`) to allocate more time to the deployment of applications.

Installling Kommander

You can configure the Kommander component of NKP during the initial installation and post-installation using the NKP CLI.

About this task

You can install Kommander with a bare minimum of applications on a small environment with smaller memory, storage, and CPU requirements for testing and demo purposes, see Installing Kommander in a Small Environment on page 1032.

Before you begin

- To ensure your cluster has enough resources, review the Management Cluster Application Requirements on page 43.

- Ensure you have a default `StorageClass`, as shown in Identifying and Modifying Your StorageClass on page 1038.

Initialize a Kommander Installer Configuration File as follows:

Procedure

To begin configuring Kommander, run the following command to initialize a default configuration file.

» For an air-gapped environment, run the following command:

```
nkp install kommander --init --airgapped > kommander.yaml
```

» For a non-air-gapped environment, run the following command:

```
nkp install kommander --init > kommander.yaml
```

Dashboard UI Functions

After installing the Konvoy component and building a cluster, as basic, successfully installing Kommander and logging into the UI, you are now ready to customize configurations.

For more information, see Basic Installations by Infrastructure on page 52. The majority of the customization, such as attaching clusters and deploying applications, will take place in the dashboard or UI of NKP. The Basic Installation section allows you to manage cluster operations and their application workloads to optimize your organization's productivity.

If you want to enable a solution that detects current and future anomalies in workload configurations or Kubernetes clusters, see Nutanix Kubernetes Platform Insights Guide on page 1217.

Logging into the UI with Kommander

Logging into the UI with Kommander

Procedure

1. By default, you can log on to the UI in Kommander with the credentials given using this command.

```
nkp open dashboard --kubeconfig=${CLUSTER_NAME}.conf
```

2. You can also retrieve your credentials at any time using the following command.

```
kubectl -n kommander get secret nkp-credentials -o go-template='Username:
 {{.data.username|base64decode}}{{ "\n"}}Password: {{.data.password|base64decode}}
{{ "\n"}}'
```

3. You can also retrieve the URL used for accessing the UI using the following command.

```
kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with
 index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/nkp/kommander/
dashboard{{ "\n"}}'
```

You should only use these static credentials to access the UI and configure an external identity provider. For more information, see Identity Providers on page 364. Treat the backup as backup credentials rather than use them for normal access.

4. You rotate the password using the following command.

```
nkp experimental rotate dashboard-password
```

The output displays the new password.

```
Password: kqZ31lMBSCLcBjUKVwLJMQL2PxalipIzZw5Pjyw09wDqjWV3dz2wPSSBYi09JGJp
```

You can perform the following operations on Identity Providers:

- Create an Identity Provider
- Temporarily Disable an Identity Provider
- Create Groups

Default StorageClass

Kommander requires a default StorageClass.

For the supported cloud providers, the Konvoy component handles the creation of a default `StorageClass`.

For pre-provisioned environments, the Konvoy component handles the creation of a StorageClass in the form of a local volume provisioner, which is not suitable for production use. Before installing the Kommander component, you should identify and install a Kubernetes CSI (see https://kubernetes.io/docs/concepts/storage/volumes/#volume-types) compatible storage provider that is suitable for production, and then ensure it is set as the default, as shown below. For more information, see Provisioning a Static Local Volume on page 38..

For infrastructure driver specifics, see Default Storage Providers on page 34.

Identifying and Modifying Your StorageClass

This `StorageClass` is required to install Kommander.

Procedure

1. Execute the following command to verify one is configured.

```
kubectl get sc --kubeconfig ${CLUSTER_NAME}.conf
```

For example, output, note the (default) after the name:

```
NAME               PROVISIONER       RECLAIMPOLICY   VOLUMEBINDINGMODE
 ALLOWVOLUMEEXPANSION   AGE
ebs-sc (default)   ebs.csi.aws.com   Delete          WaitForFirstConsumer   false
              41s
```

2. If the desired `StorageClass` is not set as default, add the following annotation to the `StorageClass` manifest.

```
annotations:
  storageclass.kubernetes.io/is-default-class: "true"
```

For more information on setting a StorageClass as default, see https://kubernetes.io/docs/tasks/administer-cluster/change-default-storage-class/.

Kommander Configuration Reference

This page contains the configuration parameters for the Kommander component of NKP.

Configuration Parameters

For additional information about configuring the Kommander component of NKP during initial installation, see Installing Kommander with a Configuration File on page 1036.

Table 59: Configuration Parameters

Parameter	Description	Default Value

aaps	List of platform applications that will be installed on the management cluster.	`apps:`
		`  ai-navigator-app:`
		`  enabled: true`
		`  dex:`
		`  enabled: true`
		`  dex-k8s-authenticator:`
		`  enabled: true`
		`  nkp-insights-management:`
		`  enabled: true`
		`  gatekeeper:`
		`  enabled: true`
		`  git-operator:`
		`  enabled: true`
		`  grafana-logging:`
		`  enabled: true`
		`  grafana-loki:`
		`  enabled: true`
		`  kommander:`
		`  enabled: true`
		`  kube-prometheus-stack:`
		`  enabled: true`
		`  values: <shortened for brevity>`
		`  kubefed:`
		`  enabled: true`
		`  kubernetes-dashboard:`
		`  enabled: true`
		`  kubetunnel:`
		`  enabled: true`
		`  logging-operator:`
		`  enabled: true`
		`  prometheus-adapter:`
		`  enabled: true`
		`  reloader:`
		`  enabled: true`
		`  rook-ceph:`
		`  enabled: true`
		`  rook-ceph-cluster:`
		`  enabled: true`
		`  traefik:`
		`  enabled: true`
		`  traefik-forward-auth-mgmt:`
		`  enabled: true`
		`  velero:`
		`  enabled: true`

ageEncryptionSecretName	Defines the name of the secret in which to store the Age encryption.	`sops-age`

Parameter	Description	Default Value
clusterHostName	Allows users to provide a hostname that is used for accessing the cluster's ingresses.	
ingressCertificate	Allows users to provide a custom certificate that's used for TLS in the cluster's ingresses.	
acme	Enable automatic ingress certificate management through ACME.	
appManagementImageTag	Specifies image tag of the AppManagement container.	
appManagementImageRepository	Specifies the image repository of AppManagement container	
appManagementKubetoolsImageRepository	Specifies the image repository of the Kubetools container	
kommanderChartsVersion	Specifies NKP Kommander Helm chart version.	
air-gapped	Specifies parameters for an air-gapped environment.	
catalog	Specifies parameters for installing default catalog repositories. For additional information, see NKP Catalog Applications Enablement after Installing NKP on page 1063.	

AppConfig Parameters

Table 60: AppConfig Parameters

Parameter	Description	Default Value
enabled	Denotes whether the specific app should be deployed or not. **Note:** The `ai-navigator-app` entry defaults to `true` unless you are installing in an air-gapped environment. Set the value to `false` if you do not want to install the AI Navigator on page 1205 application.	false

Parameter	Description	Default Value
valuesFrom	File path containing the values that are passed onto the application's `HelmRelease`. This is a Helm values file for all applications at the moment. The path in this field must either be a relative file location, which is then interpreted to be relative to the location of the configuration file, or an absolute path. **Note:** Only one of `valuesFrom` or `values` might be set; both cannot be set. neither can	
values	Contains the values that are passed to the application's `HelmRelease`. **Note:** Only one of `valuesFrom` or `values` may be set; both cannot be set.	

IngressCertificate

Table 61: IngressCertificate Parameters

Parameter	Description	Default Value
certificate	The path to a certificate PEM file.	
private_key	The path to the certificate's private key (PEM).	
ca	The path to the certificate's CA bundle; a PEM file containing root and intermediate certificates.	

Airgapped Parameters

Table 62: Airgapped Parameters

Parameter	Description	Default Value
enabled	Specifies if installation happens in an air-gapped environment.	
helmMirrorImageTag	Specifies an image tag of the Helm-mirror container.	

Parameter	Description	Default Value
helmMirrorImageRepository	Specifies image repository of Helm-mirror container.	

Next Step:

Configuring HTTP proxy for the Kommander Clusters on page 1058

Configuring the Kommander Installation with a Custom Domain and Certificate

Configure a custom domain and certificate during the installation of the Kommander component, for Management or Pro clusters only.

There are two configuration methods:

Table 63: Configuration Methods

Configuration Methods	Supported cluster types
While installing the Kommander component	Only Pro or Management clusters
After installing the Kommander component	All cluster types For more information, see Custom Domains and Certificates Configuration for All Cluster Types on page 553.

NKP supports configuring a custom domain name for accessing the UI and other platform services, as well as setting up manual or automatic certificate renewal or rotation. This section provides instructions and examples on how to configure the NKP installation to add a customized domain and certificate to your Pro cluster or Management cluster.

Reasons for Setting Up a Custom Domain or Certificate

Reasons for Setting Up a Custom Domain or Certificate

Reasons for Using a Custom DNS Domain

NKP supports the customization of domains to allow you to use your domain or hostname for your services. For example, you can set up your NKP UI or any of your clusters to be accessible with your custom domain name instead of the domain provided by default.

To set up a custom domain (without a custom certificate), see Configuring a Custom Domain Without a Custom Certificate on page 1048.

Reasons for Using a Custom Certificate

NKP's default CA identity supports the encryption of data exchange and traffic (between your client and your environment's server). To configure an additional security layer that validates your environment's server authenticity, NKP supports configuring a custom certificate issued by a trusted Certificate Authority either directly in a Secret or managed automatically using the ACME protocol (for example, Let's Encrypt).

Changing the default certificate for any of your clusters can be helpful. For example, you can adapt it to classify your NKP UI or any other type of service as trusted (when accessing a service through a browser).

To set up a custom domain and certificate, refer to the following pages respectively:

- Configure a custom domain and certificate as part of the cluster's installation process. This is only possible for your Management or Pro cluster. For more information, see Configuring the Kommander Installation with a Custom Domain and Certificate on page 1042.
- Update your cluster's current domain and certificate configuration as part of your cluster operations. You can do this for any cluster type in your environment. For more information, see Cluster Operations Management on page 353.

> **Note:** Using Let's Encrypt or other public ACME certificate authorities does not work in air-gapped scenarios, as these services require a connection to the Internet for their setup. For air-gapped environments, you can either use self-signed certificates issued by the cluster (the default configuration) or a certificate created manually using a trusted Certificate Authority.

Certificate Issuer and KommanderCluster Concepts

If you set up an advanced configuration of your custom domain, ensure you understand the following concepts.

`KommanderCluster` Object

The `KommanderCluster` resource is an object that contains key information for all types of clusters that are part of your environment, such as:

- Cluster access and endpoint information
- Cluster attachment information
- Cluster status and configuration information

Issuer Objects

`Issuer`, `ClusterIssuer` or `certificateSecret`?

If you use a certificate issued and managed automatically by `cert-manager`, you need an `Issuer` or `ClusterIssuer`that you reference in your `KommanderCluster` resource. The referenced object must contain the information of your certificate provider.

If you want to use a manually-created certificate, you need a `certificateSecret` that you reference in your `KommanderCluster` resource.

Location of the `KommanderCluster` and Issuer Objects

In the Management or Pro cluster, both the `KommanderCluster` and issuer objects are stored on the same cluster. The issuer can be referenced as an `Issuer`, `ClusterIssuer` or `certificateSecret`.

In Managed and Attached clusters, the `KommanderCluster` object is stored on the Management cluster. The `Issuer`, `ClusterIssuer` or `certificateSecret` is stored on the Managed or Attached cluster.

HTTP and DNS solver

When configuring a certificate for your NKP cluster, you can set up an HTTP solver or a DNS solver. The HTTP protocol exposes your cluster to the public Internet, whereas DNS keeps your traffic hidden. If you use HTTP, your cluster must be publicly accessible (through the ingress or load balancer). If you use DNS, this is not a requirement. For HTTP and DNS configuration options, see Advanced Configuration: ClusterIssuer on page 1047.

> **Note:**
>
> If you are enabling a proxied access (see Proxied Access to Network-Restricted Clusters on page 525) for a network-restricted cluster, this configuration is restricted to DNS.

Certificate Authority

The following table defines some values you require to set up a custom certificate according to your Certificate Authority (CA).

Use these values for Configuring the Kommander Installation with a Custom Domain and Certificate on page 1042.

Table 64: Table

Certificate Authority	Prerequisites	Kommander Installervalues
Let's Encrypt	None	Generated automatically by Kommander when acme is enabled
ZeroSSL	An access and a secret key provided by ZeroSSL	`acme.server: https://acme.zerossl.com/v2/DV90`
SSL.com	An access and a secret key provided by SSL	`acme.server: https://acme.ssl.com/sslcom-dv-rsa`

Certificate Configuration Options

There are three certificate configuration options.

For more information on values that are specific to your Certificate Authority or CA, see Certificate Authority on page 1044. Choose an ACME-supported Certificate Authority if you want the `cert-manager` to handle certificate renewal and rotation automatically.

> **Warning: Certificates issued by another Issuer**: You can also configure a certificate issued by another Certificate Authority. In this case, the CA will determine which information to include in the configuration.
>
> - For configuration examples, https://cert-manager.io/docs/configuration/.
> - The `ClusterIssuer's` name must be `kommander-acme-issuer`.

Next Step:

Verifying and Troubleshooting the Domain and Certificate Customization on page 1049

Using an Automatically-generated Certificate with ACME and Required Basic Configuration
When you enable ACME, by default NKP generates an ACME-supported certificate with an HTTP01 solver. The `cert-manager` automatically issues a trusted certificate for the configured custom domain, and takes care of renewing the certificate before expiration.

Procedure

1. Open the Kommander Installer Configuration File or `kommander.yaml` file.
   a. If you do not have the `kommander.yaml` file to initialize the configuration file so that you can edit it in the following steps.

      > **Warning:** Initialize this file only ONE time, otherwise, you will overwrite previous customizations.

   b. If you have initialized the configuration file already, open the `<kommander.yaml>` with the editor of your choice.

2. In that file, configure the custom domain for your cluster.

```
[...]
clusterHostname: <mycluster.example.com>
[...]
```

3. Enable ACME by adding `acme` value, the issuer's server, and your e-mail. If you don't provide a server, NKP sets up Let's Encrypt as your certificate provider.

```
acme:
  email: <your_email>
  server: <your_server>
[...]
```

4. Use the configuration file to install Kommander.

> **Note:** Basic configuration in this topic refers to the ACME server without EAB (External Account Bindings) and HTTP solver.

Using an Automatically-generated Certificate with ACME and Required Advanced Configuration
If you require additional configuration options like DNS solver, EAB, among others, create a `ClusterIssuer` with the required configurations before you run the installation of Kommander. The `cert-manager` automatically issues a trusted certificate for the configured custom domain, and takes care of renewing the certificate before expiration.

About this task

For more information on the `ClusterIssuer`, other objects, and where to store them, see Advanced Configuration: ClusterIssuer on page 1047 and Certificate Issuer and KommanderCluster Concepts on page 1043.

Procedure

1. Create a `ClusterIssuer` and store it in the target cluster. It must be called `kommander-acme-issuer:`

   a. If you require an HTTP solver, adapt the following example with the properties required for your certificate and run the command:

```
cat <<EOF | kubectl apply -f -
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: kommander-acme-issuer # This part is important
spec:
  acme:
    email: <your_email>
    server: <https://acme.server.example>
    skipTLSVerify: true
    privateKeySecretRef:
      name: kommander-acme-issuer-account # Set this to <name>-account
    solvers:
    - http01:
        ingress:
          ingressTemplate:
            metadata:
              annotations:
                kubernetes.io/ingress.class: kommander-traefik
                "traefik.ingress.kubernetes.io/router.priority": "2147483647"
```

```
EOF
```

> **Warning:** The values `kommander-acme-issuer`, `kommander-acme-issuer-account` and, `"traefik.ingress.kubernetes.io/router.priority": "2147483647"` are not placeholders and must be filled out exactly as in the example.

In on-premises environments, replace the annotation in the previous example with `traefik.ingress.kubernetes.io/router.tls: "true"`.

b. If you require a DNS solver, adapt the following example with the properties required for your certificate and run the command:

```
cat <<EOF | kubectl apply -f -
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: kommander-acme-issuer # This part is important
spec:
  acme:
    email: <your_email>
    server: <https://acme.server.example>
    privateKeySecretRef:
      name: kommander-acme-issuer-account # Set this to <name>-account
    solvers:
      - dns01:
          route53:
            region: us-east-1
            role: arn:aws:iam::YYYYYYYYYYYY:role/dns-manager
EOF
```

> **Warning:** The values `kommander-acme-issuer`, and `kommander-acme-issuer-account` are not placeholders and MUST be filled out exactly as in the example.

2. (Optional) If you require External Account Bindings to link your ACME account to an external database, see https://cert-manager.io/docs/configuration/acme/#external-account-bindings.

3. (Optional): Create a DNS record by setting up the external-dns service. For more information, DNS Record Creation with External DNS on page 1050. This way, the `external-dns` will take care of pointing the DNS record to the ingress of the cluster automatically.

   You can also manually create a DNS record , that maps your domain name or IP address to the cluster ingress. If you choose to create a DNS record manually, finish installing the Kommander component, and then manually create a DNS record that points to the load balancer address.

4. Open the Kommander Installer Configuration File or `kommander.yaml` file.

   a. If you do not have the `kommander.yaml` file, initialize the configuration file, so you can edit it in the following steps.

   > **Warning:** Initialize this file only ONCE, otherwise you will overwrite previous customizations.

   b. If you have initialized the configuration file already, open the `kommander.yaml` with the editor of your choice.

5. In that file, configure the cluster to use your custom domain.

```
[...]
clusterHostname: <mycluster.example.com>
[...]
```

6. Enable ACME by configuring the issuer's server and your e-mail.

```
[...]
acme:
  email: <your_email>
  server: <your_server>
[...]
```

7. Use the configuration file to install Kommander.

Using a Manually-generated Certificate
Nutanix supports the use of a manually-created certificate. In this case, there is no certificate controller that handles the renewal and update of your certificate automatically, so you will have to take care of these tasks manually.

Before you begin

Obtain the PEM files of your certificate and store them in the target cluster's namespace:

- Certificate
- certificate's private key
- CA bundle (containing the root and intermediate certificates)

Procedure

1. Open the Kommander Installer Configuration File or `<kommander.yaml>` file.
   a. If you do not have the `kommander.yaml` file, initialize the configuration file so that you can edit it in the following steps.

      Warning: Initialize this file only ONCE; otherwise, you will overwrite previous customizations.

   b. If you have initialized the configuration file already, open the `kommander. yaml` with the editor of your choice.
2. In the Kommander Installer Configuration file, provide your custom domain and the paths to the PEM files of your certificate.

```
[...]
clusterHostname: <mycluster.example.com>
ingressCertificate:
  certificate: <certs/cert.pem>
  private_key: <certs/key.pem>
  ca: <certs/ca.pem>
[...]
```

3. Use the configuration file to install Kommander.

Advanced Configuration: ClusterIssuer

When you enable ACME to understand the basic concepts for certificate configuration. by default NKP generates an ACME-supported certificate with an HTTP01 solver that is provided by Let's Encrypt.

For this, see Configuring the Kommander Installation with a Custom Domain and Certificate on page 1042.

Ensure you review Certificate Issuer and KommanderCluster Concepts on page 1043

You can also set up an advanced configuration for a Custom Domain and Certificate. In these cases, the custom configuration cannot be done completely through the `installer config` file, but must be specified

further in a `ClusterIssuer`. For more information, see Certificate Issuer and KommanderCluster Concepts on page 1043.

Whether it is sufficient to establish the configuration of your custom certificate in the `installer config` file only or you require a `ClusterIssuer` to define further configuration options depends on the degree of customization.

> **Warning:** If you require a `ClusterIssuer`, you must create it before you run the Kommander installation.

When do You Need a ClusterIssuer?

The configuration of the `ClusterIssuer` resource depends on your NKP landscape:

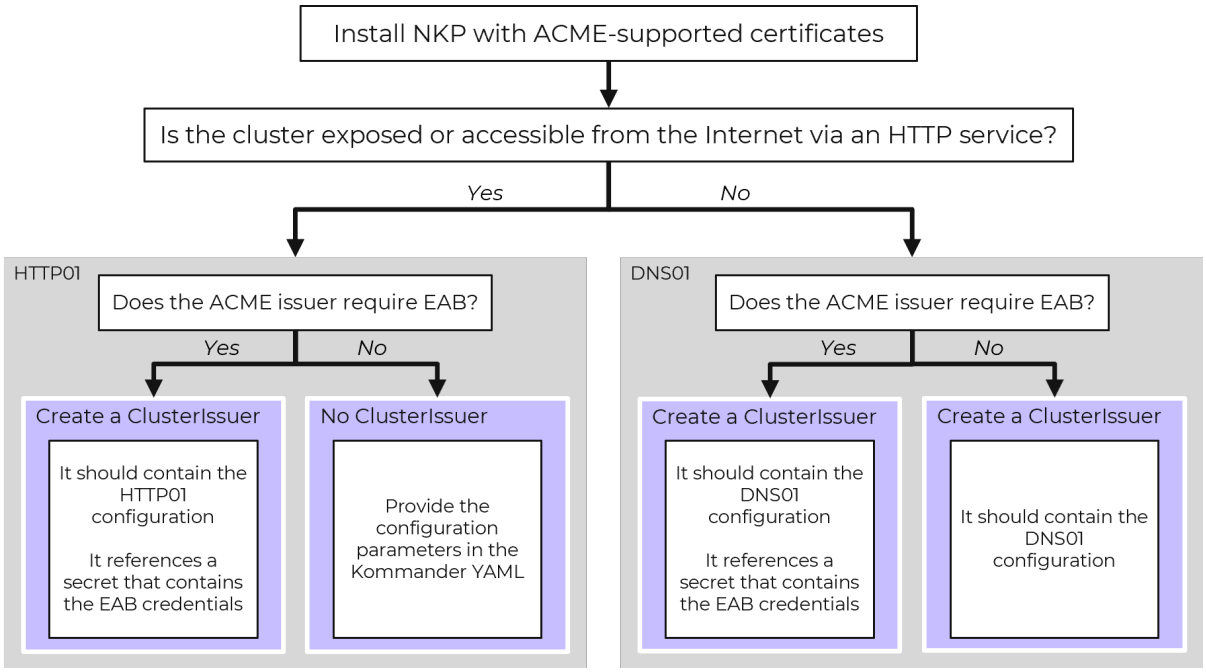


Figure 28: Cluster Domain in the NKP Landscape

How do You Configure a ClusterIssuer?

The following image describes the configurable fields of a `ClusterIssuer`:

For more information on the available options, see https://cert-manager.io/docs/configuration/acme/.

Examples

For configuration steps and examples, see Configuring the Kommander Installation with a Custom Domain and Certificate on page 1042.

> **Warning:** If you need to make changes in the configuration of your domain or certificate after you have installed NKP, or if you want to set up a custom domain and certificate for Attached or Managed clusters , modify the `ingress` in the `KommanderCluster` object as shown in the Custom domains and certificates configuration section.

Configuring a Custom Domain Without a Custom Certificate

To configure Kommander to use a custom domain, the domain name must be provided in an installation `config` file.

Procedure

1. Open the Kommander Installer Configuration File or `kommander.yaml` file.

   a. If you do not have the `kommander.yaml` file, see Installing Kommander with a Configuration File on page 1036, so you can edit it in the following steps.

      > **Warning:** Initialize this file only ONCE, otherwise, you will overwrite previous customizations.

   b. If you have initialized the configuration file already, open the `kommander.yaml` with the editor of your choice.

2. In that file, configure the custom domain for your cluster by adding this line.

   ```
   [...]
   clusterHostname: <mycluster.example.com>
   [...]
   ```

3. This configuration can be used when installing or reconfiguring Kommander by passing it to the `nkp install kommander` command.

   ```
   nkp install kommander --installer-config <kommander.yaml> --kubeconfig=
   ${CLUSTER_NAME}.conf
   ```

   > **Note:** To ensure Kommander is installed on the right cluster, use the `--kubeconfig=cluster_name.conf` flag as an alternative to KUBECONFIG.

4. After the command completes, obtain the cluster ingress IP address or hostname using the following command.

   ```
   kubectl -n kommander get svc kommander-traefik -o go-template='{{with
    index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}{{ "\n"}}'
   ```

   If required, create a DNS record (for example, by using external-dns) for your custom hostname that resolves to the cluster ingress load balancer hostname or IP address. If the previous command returns a hostname, you should create a CNAME DNS entry that resolves to that hostname. If the cluster ingress is an IP address, create a DNS A record.

   > **Warning:** The domain must be resolvable from the client (your browser) and fro the cluster.by the client (your browser) and from If you set up an `external-dns` service, it will take care of pointing the DNS record to the ingress of the cluster automatically. If you are manually creating a DNS record, you have to install Kommander first to obtain the load balancer address required for the DNS record.

   For more details and examples on how and when to set up the DNS record, see Configuring the Kommander Installation with a Custom Domain and Certificate on page 1042.

Verifying and Troubleshooting the Domain and Certificate Customization

If you want to ensure the customization for a domain and certificate is completed, or if you want to obtain more information on the status of the customization, display the status information for the `KommanderCluster`.

About this task

On the Management cluster, perform the following:

Procedure

1. Inspect the modified `KommanderCluster` object.

   ```
   kubectl describe kommandercluster -n <workspace_name> <cluster_name>
   ```

2. If the ingress is still being provisioned, the output looks similar to this.

```
[...]
Conditions:
  Last Transition Time:  2022-06-24T07:48:31Z
  Message:               Ingress service object was not found in the cluster
  Reason:                IngressServiceNotFound
  Status:                False
  Type:                  IngressAddressReady
[...]
```

If the provisioning has been completed, the output looks similar to this.

```
[...]
Conditions:
  Last Transition Time:  2022-06-28T13:43:33Z
  Message:               Ingress service address has been provisioned
  Reason:                IngressServiceAddressFound
  Status:                True
  Type:                  IngressAddressReady
  Last Transition Time:  2022-06-28T13:42:24Z
  Message:               Certificate is up to date and has not expired
  Reason:                Ready
  Status:                True
  Type:                  IngressCertificateReady
[...]
```

The same command also prints the actual customized values for the `KommanderCluster.Status.Ingress`. Here is an example.

```
[...]
   ingress:
    address: 172.20.255.180
    caBundle: LS0tLS1CRUdJTiBD...<output has been shortened>...DQVRFLS0tLS0K
[...]
```

DNS Record Creation with External DNS

This section describes how to use `external-dns` to maintain your DNS records

When you set up a custom domain, that is, Custom Domains and Certificates Configuration for All Cluster Types for your cluster, you require a DNS record that maps the configured domain or IP address to the cluster's ingress. You can either create one manual or set up the `external-dns` service to manage your DNS record automatically.

If you choose to use `external-dns` to maintain your DNS records, the `external-dns` will take care of pointing the DNS record to the ingress of the cluster automatically.

Select one of the following options to configure the `external-dns` service for Management or Pro clusters or for Managed or Attached clusters.

- Configuring External DNS with the CLI: Management or Pro Cluster on page 1050
- Configuring the External DNS Using the UI on page 1052
- Verifying Your External DNS Configuration on page 1054

If you choose to create a DNS record manually, finish installing the Kommander component and then manually create a DNS record that points to the load balancer address.

Configuring External DNS with the CLI: Management or Pro Cluster

This page contains information on how to configure an `external-dns` service to manage DNS records automatically in your Management or Pro cluster.

Before you begin

Ensure you have configured a DNS zone with your cloud provider.

About this task

Configure External DNS and Customize Traefik. The configuration varies depending on your cloud provider.

Procedure

1. Open the Kommander Installer Configuration File or `kommander.yaml` file.

   a. If you do not have the `kommander.yaml` file, see Installing Kommander with a Configuration File on page 1036so that you can edit it in the following steps.

      Warning: Initialize this file only ONCE; otherwise, you will overwrite previous customizations.

   b. If you have installed the Kommander component already, open the existing `kommander.yaml` with the editor of your choice.

2. Adjust the `app` section of your `kommander.yaml` file to include these values.

   AWS Example: Replace the placeholders `<...>`with your environment's information.

   The following example shows how to configure `external-dns` to manage DNS records in AWS Route 53 automatically:

```
apps:
  external-dns:
    enabled: true
    values: |
      aws:
        credentials:
          secretKey: <secret-key>
          accessKey: <access-key>
        region: <provider-region>
        preferCNAME: true
      policy: upsert-only
      txtPrefix: local-
      domainFilters:
        - <example.com>
```

   Azure Example: Replace the placeholders `<...>`with your environment's information.

```
apps:
  external-dns:
    enabled: true
    values: |
      azure:
        cloud: AzurePublicCloud
        resourceGroup: <resource-group>
        tenantId: <tenant-id>
        subscriptionId: <your-subscription-id>
        aadClientId: <client-id>
        aadClientSecret: <client-secret>
      domainFilters:
        - <example.com>
      txtPrefix: txt-
      policy: sync
      provider: azure
```

3. In the same `app` section, adjust the `traefik`section to include the following.

```
traefik:
    enabled: true
    values: |
      service:
        annotations:
          external-dns.alpha.kubernetes.io/hostname: <mycluster.example.com>
```

4. Use the configuration file to install or update the Kommander component.

```
nkp install kommander --installer-config kommander.yaml --kubeconfig=
${CLUSTER_NAME}.conf
```

For more information and configuring `external-dns` to use other DNS providers like Google Cloud DNS, CloudFlare, or on-site providers, see https://artifacthub.io/packages/helm/bitnami/external-dns.

What to do next

Verifying Your External DNS Configuration on page 1054

Configuring the External DNS Using the UI

Configure External DNS with the UI: All Clusters.

About this task

This page contains information on how to configure an `external-dns` service to manage DNS records automatically and applies to all cluster types.

The configuration varies depending on your cloud provider.

Before you begin

Ensure you have configured a DNS zone with your cloud provider.

Procedure

1. Select the target workspace from the top navigation bar.

   It must be the workspace that contains the cluster, for which you want to configure External DNS. In the case of the Management cluster, it is the Management cluster workspace.

2. Select **Applications** from the sidebar menu.

3. Search for the **External DNS** application.

4. On the application card, select the **three-dot menu** > **Enable**.

5. On the **Enable Workspace Platform Application** page, select **Configuration** from the sidebar menu.

6. Copy and paste the following contents into the code editor and replace the placeholders `<...>` with your environment's information.

   Here is an example configuration.

   AWS Example: Replace the placeholders `<...>`with your environment's information.

   The following example shows how to configure `external-dns` to manage DNS records in AWS Route 53 automatically.

```
aws:
  credentials:
    secretKey: <secret-key>
    accessKey: <access-key>
```

```
  region: <provider-region>
  preferCNAME: true
policy: upsert-only
txtPrefix: local-
domainFilters:
  - <example.com>
```

Azure Example: Replace the placeholders `<...>`with your environment's information.

```
azure:
  cloud: AzurePublicCloud
  resourceGroup: <resource-group>
  tenantId: <tenant-id>
  subscriptionId: <your-subscription-id>
  aadClientId: <client-id>
  aadClientSecret: <client-secret>
domainFilters:
  - <example.com>
txtPrefix: txt-
policy: sync
provider: azure
```

For more configuration options, see https://artifacthub.io/packages/helm/bitnami/external-dns.

Customizing the Traefik Deployment Using the UI

Customize the Traefik Deployment Using the UI

About this task

> **Note:** NKP deploys Traefik to all clusters by default.

Procedure

1. Select the target workspace from the top navigation bar.
   It must be the workspace that contains the cluster, for which you want to configure External DNS. In the case of the Management cluster, it is the Management cluster workspace.
2. Select **Applications** from the sidebar menu.
3. Search for the **Traefik** application.
4. On the application card, select the **three-dot menu** > **Edit**.'
5. Select the **Configuration** lateral tab to add a customization.
6. Copy and paste the following configuration into the code editor:
   Use the Cluster Application Configuration Override code editor to apply a configuration per cluster.

```
service:
  annotations:
    external-dns.alpha.kubernetes.io/hostname: <mycluster.example.com>
```

> **Warning:** Ensure you set up a domain per cluster, for example: `<mycluster1.example.com>`, `<mycluster2.example.com>` and `<mycluster3.example.com>`.

What to do next

Verifying Your External DNS Configuration on page 1054

Verifying Your External DNS Configuration

This page contains commands to verify that the `external-dns` service is functioning correctly.

About this task

If the `external-dns` service is not working properly, these commands also provide aids to find the cause or identify the issue.

To verify that the deployment was triggered:

Procedure

1. Set the environment variable to the Management/Pro cluster by exporting the `kubeconfig` file in your terminal window or using the `--kubeconfig=${CLUSTER_NAME}.conf` as explained in Commands within a kubeconfig File on page 32.
2. Verify that the `external-dns` deployment is present.

   Replace `<target_WORKSPACE_NAMESPACE>` in the namespace `-n` flag with the target cluster's workspace namespace.

   ```
   kubectl get appdeployments.apps.kommander.d2iq.io -n <target_WORKSPACE_NAMESPACE>
    external-dns
   ```

   The output should look like this:

   ```
   NAME           APP                           AGE
   external-dns   external-dns-<app_version>   36s
   ```

   The CLI has triggered the application's deployment. However, this does not mean that the application has been installed completely and successfully.

Verifying Whether the DNS Deployment Is Successful

Verify whether the DNS deployment is successful or not.

Procedure

1. Set the environment variable to the target cluster (where you enabled `external-dns`) by exporting the `kubeconfig`file in your terminal window or using the `--kubeconfig=${CLUSTER_NAME}.conf` as explained in Commands within a kubeconfig File on page 32
2. Verify that the `external-dns` deployment is ready.

   Replace `<target_WORKSPACE_NAMESPACE>` in the namespace `-n` flag with the target cluster's workspace namespace.

   ```
   kubectl get deployments.apps -n <target_WORKSPACE_NAMESPACE> external-dns
   ```

   The deployment should display a ready state.

   ```
   NAME           READY   UP-TO-DATE   AVAILABLE   AGE
   external-dns   1/1     1            1           42s
   ```

   The CLI has deployed the application completely and successfully.

Examining the Cluster's Ingress

Examine the cluster's ingress.

Procedure

1. Set the environment variable to the target cluster (where you enabled `external-dns`) by exporting the `kubeconfig`file in your terminal window or using the `--kubeconfig=${CLUSTER_NAME}.conf` as explained in Commands within a kubeconfig File on page 32.

2. Verify that the cluster's ingress contains the correct hostname annotation.

   Replace `<target_WORKSPACE_NAMESPACE>` in the namespace `-n` flag with the target cluster's workspace namespace.

   ```
   kubectl get services -n <target_WORKSPACE_NAMESPACE> kommander-traefik -o yaml
   ```

   The output looks like this.

   Ensure that the service object contains the `external-dns.alpha.kubernetes.io/hostname: <mycluster.example.com>` annotation.

   ```
   apiVersion: v1
   kind: Service
   metadata:
     annotations:
       meta.helm.sh/release-name: kommander-traefik
       meta.helm.sh/release-namespace: kommander
       external-dns.alpha.kubernetes.io/hostname: <mycluster.example.com>
     creationTimestamp: "2023-06-21T04:52:49Z"
     finalizers:
   [...]
   ```

   The `external-dns` service has been linked to the cluster correctly.

Verifying the DNS Record

Verify that the `external-dns` service has created a DNS record.

About this task

It can take a few minutes for the `external-dns` service to create a DNS record. The delay depends on your cloud provider.

Procedure

1. Set the environment variable to the target cluster (where you enabled `external-dns`) by exporting the `kubeconfig`file in your terminal window or using the `--kubeconfig=${CLUSTER_NAME}.conf` as explained in Commands within a kubeconfig File on page 32.

2. Access and run the required image.

   ```
   kubectl run -it --image=nicolaka/netshoot --rm test-dns -- /bin/bash
   ```

3. Use the image to check your domain and see the record.

   Replace `<mycluster.example.com>` with the domain you assigned to your target cluster.

   ```
   nslookup <mycluster.example.com>
   ```

   The output should look like this.

   ```
   Server:  192.168.178.1
   Address: 192.168.178.1#53

   Non-authoritative answer:
   Name: <mycluster.example.com>
   ```

```
Address: 134.568.789.12
```

The `external-dns` service is working, and the DNS provider recognizes the record created by the service. If the command displays an error, the configuration is failing on the end of the DNS provider.

Note: If your deployment has not succeeded and the previous steps have not helped you identify the issue, you can also check the logs for the `external-dns` deployment:

- 1. Set the environment variable to the target cluster (where you enabled `external-dns`) by exporting the `kubeconfig`file in your terminal window or using the `--kubeconfig=${CLUSTER_NAME}.conf` as explained in Commands within a kubeconfig File on page 32.
  2. Verify the `external-dns` logs:

     Replace `<target_WORKSPACE_NAMESPACE>` in the namespace `-n` flag with the target cluster's workspace namespace.

     ```
     kubectl logs -n kommander deployment/external-dns
     ```

     The output displays the pod's logs for the `external-dns` deployment. Here is an example:

     ```
     ...
     time="2023-07-04T06:56:35Z" level=info msg="Instantiating new Kubernetes
      client"
     time="2023-07-04T06:56:35Z" level=info msg="Using inCluster-config based
      on serviceaccount-token"
     time="2023-07-04T06:56:35Z" level=info msg="Created Kubernetes client
      https://10.96.0.1:443"
     time="2023-07-04T06:56:35Z" level=error msg="records retrieval failed:
      failed to list hosted zones:
     ...
     ```

External Load Balancer

Load Balancing for External Traffic in NKP.

NKP includes a load-balancing solution for the Supported Infrastructure Operating Systems on page 12 and for pre-provisioned environments. For more information, see Load Balancing on page 632.

If you want to use a non-NKP load balancer (for example, as an alternative to MetalLB in pre-provisioned environments), NKP supports setting up an external load balancer.

When enabled, the external load balancer routes incoming traffic requests to a single point of entry in your cluster. Users and services can then access the NKP UI through an established IP or DNS address.

Note: In NKP environments, the external load balancer must be configured without TLS termination.

Configuring Kommander to Use an External Load Balancer

To configure an external load balancer, configure a custom hostname (static IP or dynamic DNS address) and specify the target `nodePorts` for your cluster.

About this task

Procedure

1. Open the Kommander Installer Configuration File or `kommander.yaml` file.

   a. If you do not have the `kommander.yaml` file, see Installing Kommander with a Configuration File on page 1036so that you can edit it in the following steps.

   > **Warning:** Initialize this file only ONCE. Otherwise you will overwrite previous customizations.

   b. If you have installed the Kommander component already, open the existing `kommander.yaml` with the editor of your choice.

2. In that file, add the following line for the IP address or DNS name:

   > **Warning:** ACME does not support the automatic creation of a certificate if you select an IP address for your `clusterHostname`.

```
[...]
clusterHostname: <mycluster.example.com OR IP_address>
[...]
```

3. (Optional): If you require a custom certificate for your `clusterHostname`, see Configuring the Kommander Installation with a Custom Domain and Certificate on page 1042.

4. In the same **Kommander Installer Configuration File**, configure Kommander to use the `NodePort` service by adding a custom configuration under `traefik`.

   > **Warning:** You can specify the `nodePort` entry points for the load balancer. Ensure the port is within the Kubernetes default (30 000 - 32 768). If not specified, Kommander assigns a port dynamically.

```
traefik:
  enabled: true
  values: |-
    ports:
      web:
        nodePort: 32080 #if not specified, will be assigned dynamically
      websecure:
        nodePort: 32443 #if not specified, will be assigned dynamically
    service:
      type: NodePort
```

5. Use the configuration file to install Kommander.

Configuring the External Load Balancer to Target the Specified Ports

Configure the External Load Balancer to Target the Specified Ports. The `traefik` service of the Kommander component now actively listens to the pod IPs and is accessible through the specified ports on every node.

Procedure

Configure the load balancer targets to include every worker node address (DNS name or IP address) and node port combination by following this format.

```
<node1>:<nodePort_web> # for example, my.node1.internal:32080
<node2>:<nodePort_web>
<node3>:<nodePort_web>
[...]
<node1>:<nodePort_websecure> # for example, my.node1.internal:32443
<node2>:<nodePort_websecure>
<node3>:<nodePort_websecure>
```

```
[...]
```

The exact configuration depends on your load balancer provider.

HTTP Proxy Configuration Considerations

To ensure that core components work correctly, always add these addresses to the `noProxy`:

- Loopback addresses (`127.0.0.1` and `localhost`)
- Kubernetes API Server addresses
- Kubernetes Pod IPs (for example, `192.168.0.0/16`). This comes from two places:
  - Calico pod CIDR - Defaults to `192.168.0.0/16`
  - The `podSubnet` is configured in CAPI objects and needs to match above Calico's - Defaults to `192.168.0.0/16` (same as above)
- Kubernetes Service addresses (for example, `10.96.0.0/12`, `kubernetes`, `kubernetes.default`, `kubernetes.default.svc`, `kubernetes.default.svc.cluster`, `kubernetes.default.svc.cluster.local`, `.svc`, `.svc.cluster`, `.svc.cluster.local`, `.svc.cluster.local.`)
- Auto-IP addresses `169.254.169.254,169.254.0.0/24`

In addition to the values above, the following settings are needed when installing on AWS:

- The default VPC CIDR range of `10.0.0.0/16`
- `kube-apiserver` internal/external ELB address

Warning:

- The `NO_PROXY` variable contains the Kubernetes Services CIDR. This example uses the default CIDR, `10.96.0.0/12`. If your cluster's CIDR is different, update the value in the `NO_PROXY` field.
- Based on the order in which the Gatekeeper Deployment is Ready (in relation to other Deployments), not all the core services are guaranteed to be mutated with the proxy environment variables. Only the user user-deployed workloads are guaranteed to be mutated with the proxy environment variables. If you need a core service to be mutated with your proxy environment variables, you can restart the AppDeployment for that core service.

Configuring HTTP proxy for the Kommander Clusters

Configure HTTP proxy for the Kommander clusters.

About this task

Kommander supports environments that connect through an HTTP/HTTPS proxy when access to the Internet is restricted. Use the information in this section to configure the Kommander component of NKP correctly.

In these environments, you must configure Kommander to use the HTTP/HTTPS proxy. In turn, Kommander configures all platform services to use the HTTP/HTTPS proxy.

Note: Kommander follows a common convention for using an HTTP proxy server. The convention is based on three environment variables, and is supported by many, though not all, applications.

- `HTTP_PROXY`: the HTTP proxy server address

- `HTTPS_PROXY`: the HTTPS proxy server address
- `NO_PROXY`: a list of IPs and domain names that are not subject to proxy settings

Before you begin

- The `curl` command-line tool is available on the host.
- The proxy server address is `http://proxy.company.com:3128`.
- The HTTP and HTTPS proxy server addresses use the `http` scheme.
- The proxy server can reach `www.google.com` using HTTP or HTTPS.

On each cluster node, run:

Procedure

1. Verify the cluster nodes can access the Internet through the proxy server.
2. On each cluster node, run.

   ```
   curl --proxy http://proxy.company.com:3128 --head http://www.google.com
   curl --proxy http://proxy.company.com:3128 --head https://www.google.com
   ```

   If the proxy is working for HTTP and HTTPS, respectively, the `curl` command returns a `200 OK` HTTP response.

Enabling Gatekeeper

Gatekeeper acts as a Kubernetes mutating webhook.

About this task

For more information, see https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/#mutatingadmissionwebhook.

You can use this to mutate the Pod resources with `HTTP_PROXY`, `HTTPS_PROXY` and `NO_PROXY` environment variables.

Procedure

1. Create (if necessary) or update the Kommander installation configuration file. If one does not already exist, then create it using the following commands.

   ```
   nkp install kommander --init > kommander.yaml
   ```

2. Append this `apps` section to the `kommander.yaml` file with the following values to enable Gatekeeper and configure it to add HTTP proxy settings to the pods.

   Note:

   Only pods created after applying this setting will be mutated. Also, this will only affect pods in the namespace with the `"gatekeeper.d2iq.com/mutate=pod-proxy"` label.

   ```
   apps:
     gatekeeper:
       values: |
         disableMutation: false
         mutations:
           enablePodProxy: true
           podProxySettings:
   ```

```
          noProxy:
 "127.0.0.1,192.168.0.0/16,10.0.0.0/16,10.96.0.0/12,169.254.169.254,169.254.0.0/24,localhost,kub
prometheus-server.kommander,logging-operator-logging-
fluentd.kommander.svc.cluster.local,elb.amazonaws.com"
          httpProxy: "http://proxy.company.com:3128"
          httpsProxy: "http://proxy.company.com:3128"
        excludeNamespacesFromProxy: []
        namespaceSelectorForProxy:
          "gatekeeper.d2iq.com/mutate": "pod-proxy"
```

3. Create the `kommander` and `kommander-flux` namespaces, or the namespace where Kommander will be installed. Label the namespaces to activate the Gatekeeper mutation on them.

```
kubectl create namespace kommander
kubectl label namespace kommander gatekeeper.d2iq.com/mutate=pod-proxy

kubectl create namespace kommander-flux
kubectl label namespace kommander-flux gatekeeper.d2iq.com/mutate=pod-proxy
```

Creating Gatekeeper ConfigMap in the Kommander Namespace

To configure Gatekeeper so that these environment variables are mutated in the pods, create the following `gatekeeper-overrides` ConfigMap in the `kommander` Workspace you created in a previous step:

Procedure

Run the following command.

```
export NAMESPACE=kommander
```

```
cat << EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: gatekeeper-overrides
  namespace: ${NAMESPACE}
data:
  values.yaml: |
    ---
    # enable mutations
    disableMutation: false
    mutations:
      enablePodProxy: true
      podProxySettings:
        noProxy:
 "127.0.0.1,192.168.0.0/16,10.0.0.0/16,10.96.0.0/12,169.254.169.254,169.254.0.0/24,localhost,kuber
prometheus-server.kommander,logging-operator-logging-
fluentd.kommander.svc.cluster.local,elb.amazonaws.com"
        httpProxy: "http://proxy.company.com:3128"
        httpsProxy: "http://proxy.company.com:3128"
      excludeNamespacesFromProxy: []
      namespaceSelectorForProxy:
        "gatekeeper.d2iq.com/mutate": "pod-proxy"
EOF
```

Set the `httpProxy` and `httpsProxy` environment variables to the address of the HTTP and HTTPS proxy servers, respectively. Set the `noProxy` environment variable to the addresses that should be accessed directly, not through the proxy.

Performing this step before installing Kommander allows the Flux components to respect the proxy configuration in this ConfigMap.

Installing Kommander Using the Configuration Files and ConfigMap

Kommander installs with the NKP CLI.

About this task

Install Kommander using the configuration files and ConfigMap from the previous steps:

> **Note:** To ensure Kommander is installed on the workload cluster, use the `--kubeconfig=cluster_name.conf` flag:

Procedure

Run the following command.

```
nkp install kommander --installer-config kommander.yaml
```

Configuring the Workspace or Project

Configure the Workspace or Project in which you want to use the proxy.

Procedure

1. To have Gatekeeper mutate the manifests, create the `Workspace` (or `Project`) with the following label.

   ```
   labels:
     gatekeeper.d2iq.com/mutate: "pod-proxy"
   ```

2. This can be done when creating the Workspace (or Project) from the UI OR by running the following command from the CLI after creating the namespace.

   ```
   kubectl label namespace <NAMESPACE> "gatekeeper.d2iq.com/mutate=pod-proxy"
   ```

Configuring HTTP Proxy in Attached Clusters

To ensure that Gatekeeper is deployed before everything else in the attached clusters that you want to configure with proxy configuration, you must manually create the exact Namespace of the Workspace in which the cluster is going to be attached before attaching the cluster

Procedure

1. Execute the following command in the attached cluster before attaching it to the host cluster.

   ```
   kubectl create namespace <NAMESPACE>
   ```

2. Then, to configure the pods in this namespace to use proxy configuration, you must label the Workspace with `gatekeeper.d2iq.com/mutate=pod-proxy` when creating it so that Gatekeeper deploys a `validatingwebhook` to mutate the pods with proxy configuration.

   ```
   kubectl label namespace <NAMESPACE> "gatekeeper.d2iq.com/mutate=pod-proxy"
   ```

Creating Gatekeeper ConfigMap in the Workspace Namespace

Procedure

1. To configure Gatekeeper so that these environment variables are mutated in the pods, create the following `gatekeeper-overrides` ConfigMap in the Workspace Namespace.

   ```
   export NAMESPACE=<NAMESPACE>
   ```

   ```
   cat << EOF | kubectl apply -f -
   apiVersion: v1
   ```

```
kind: ConfigMap
metadata:
  name: gatekeeper-overrides
  namespace: ${NAMESPACE}
data:
  values.yaml: |
    ---
    # enable mutations
    disableMutation: false
    mutations:
      enablePodProxy: true
      podProxySettings:
        noProxy:
 "127.0.0.1,192.168.0.0/16,10.0.0.0/16,10.96.0.0/12,169.254.169.254,169.254.0.0/24,localhost,kub
prometheus-server.kommander,logging-operator-logging-
fluentd.kommander.svc.cluster.local,elb.amazonaws.com"
        httpProxy: "http://proxy.company.com:3128"
        httpsProxy: "http://proxy.company.com:3128"
      excludeNamespacesFromProxy: []
      namespaceSelectorForProxy:
        "gatekeeper.d2iq.com/mutate": "pod-proxy"
EOF
```

Set the `httpProxy` and `httpsProxy` environment variables to the address of the HTTP and HTTPS proxy servers, respectively. Set the `noProxy` environment variable to the addresses that should be accessed directly, not through the proxy. To view the list of the recommended settings, see HTTP Proxy Configuration Considerations on page 1058.

2.

Configuring Your Applications

In a default installation with `gatekeeper` enabled, you can have proxy environment variables applied to all your pods automatically by adding the following label to your namespace.

Procedure

Run the following command.

```
"gatekeeper.d2iq.com/mutate": "pod-proxy"
```

> **Note:** No further manual changes are required.

Configuring Your Application Manually

About this task

> **Note:** If Gatekeeper is not installed and you need to use an HTTP proxy, you must manually configure your applications.

In this example, the environment variables are set for a container in a Pod:

Procedure

1. Some applications follow the convention of `HTTP_PROXY`, `HTTPS_PROXY`, and `NO_PROXY` environment variables.
2. In this example, the environment variables are set for a container in a Pod.

   For more information, see https://kubernetes.io/docs/tasks/inject-data-application/define-environment-variable-container/#define-an-environment-variable-for-a-container.

What to do next

Next Steps: Now select your environment, and finish your Kommander Installation using one of the following:

- Installing Kommander in an Air-gapped Environment on page 1017
- Installing Kommander in a Non-Air-gapped Environment on page 1021
- Installing Kommander in a Small Environment on page 1032

NKP Catalog Applications Enablement after Installing NKP

NKP supports configuring NKP Catalog applications for environments with an Ultimate license.

There are two possible ways of enabling the Ultimate catalog:

- Enable an Ultimate catalog during the installation of NKP.

  Depending on your environment, see Installing Kommander in an Air-gapped Environment on page 1017.

- Enable an Ultimate catalog after installing NKP.

Configuring a Default Ultimate Catalog after Installing NKP

Procedure

1. Locate the Kommander Installer Configuration file you are using for your current deployment. This file is stored locally on your computer.

   The default file name is `kommander.yaml`, but you can provide a different name for your file.

2. Open that file and go into edit mode.

3. Enable the default catalog repository by adding the following values to your existing and already configured Kommander Installer Configuration file.

```
...
catalog:
  repositories:
    - name: dkp-catalog-applications
      labels:
        kommander.d2iq.io/project-default-catalog-repository: "true"
        kommander.d2iq.io/workspace-default-catalog-repository: "true"
        kommander.d2iq.io/gitapps-gitrepository-type: "nkp"
      gitRepositorySpec:
        url: https://github.com/mesosphere/dkp-catalog-applications
        ref:
          tag: v2.12.0
...
```

4. Reconfigure NKP by reinstalling the Kommander component. In the following example, replace `kommander.yaml` with the name of your Kommander Installer Configuration file.

   > **Warning:** Ensure you are using the correct name for the Kommander Installer Configuration file to maintain your cluster settings. Installing a different `kommander.yaml` than the one your environment is using overwrites all of your previous configurations and customizations.

   ```
   nkp install kommander --installer-config kommander.yaml --kubeconfig=
   ${CLUSTER_NAME}.conf
   ```

   Tips and recommendations:

   - The `--kubeconfig=${CLUSTER_NAME}.conf` flag ensures that you **install Kommander on the correct cluster**. For alternatives, see Commands within a kubeconfig File on page 32.
   - Applications can take longer to deploy, and time out the installation. Add the `--wait-timeout <time to wait>` flag and specify a period of time (for example, `1h`) to allocate more time to the deployment of applications.
   - If the Kommander installation fails, or you want to reconfigure applications, rerun the `install` command to retry.

NKP Catalog Application Labels

The following section describes each label:

Table 65: NKP Catalog Application Labels

Label	Description
`kommander.d2iq.io/project-default-catalog-repository`	Indicates this acts as a Catalog Repository in all projects
`kommander.d2iq.io/workspace-default-catalog-repository`	Indicates this acts as a Catalog Repository in all workspaces
`kommander.d2iq.io/gitapps-gitrepository-type`	Indicates this Catalog Repository (and all its Applications) are certified to run on NKP

8

ADDITIONAL KONVOY CONFIGURATIONS

When installing Nutanix Kubernetes Platform (NKP for a project, line-of-business, or enterprise, the first step is to determine the infrastructure on which you want to deploy. The infrastructure you select then determines the specific requirements for a successful installation.

For basic recommended installations by infrastructure, see Basic Installations by Infrastructure on page 52.

For custom or advanced installations by infrastructure, see Custom Installation and Infrastructure Tools on page 677.

If you have decided to uninstall NKP, see the same infrastructure documentation you have selected in the Basic Installations by Infrastructure on page 52.

The different Infrastructures and components for further configuration are listed below:

FIPS 140-2 Compliance

Understand FIPS-140 Operating Mode and Requirements

Developed by a working group of government, industry operators, and vendors, the Federal Information Processing Standard (FIPS), FIPS-140 defines security requirements for cryptographic modules. FIPS defines what cryptographic cyphers can be used. Kubernetes uses encryption by default between various components, and FIPS support ensures that the ciphers used for those communications meet those standards. The standard provides a broad spectrum of data sensitivity, transaction values, and various application environment security situations. The standard specifies four security levels for each of the eleven requirement areas. Each successive level offers increased security.

NIST introduced FIPS 140-2 validation by accredited third-party laboratories as a formal, rigorous process to protect sensitive digitally stored information not under Federal security classifications.

FIPS Support in NKP

Describes FIPS infrastructure support in NKP.

Nutanix Kubernetes Platform (NKP) supports provisioning a FIPS-enabled Kubernetes control plane. Core Kubernetes components are compiled using a version of Go called goboring, which uses a FIPS-certified cryptographic module for all cryptographic functions. For more information, see https://csrc.nist.gov/CSRC/media/projects/cryptographic-module-validation-program/documents/security-policies/140sp3702.pdf

Before provisioning NKP, follow your OS vendor's instructions to ensure that your OS or OS images are prepared for operating in FIPS mode. To view an example for Red Hat Enterprise Linux (RHEL), see https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/7/html/security_guide/chap-federal_standards_and_regulations#sec-Enabling-FIPS-Mode.

Additional helpful reading:

- Search | CSRC, See https://csrc.nist.gov/publications/fips
- FEDERAL INFORMATION PROCESSING STANDARDS PUBLICATION https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.140-2.pdf

Note: You cannot apply FIPS mode to an existing cluster. You must create a new cluster with FIPS enabled. Similarly, a FIPS-mode cluster must remain a FIPS-mode cluster; you cannot change its FIPS status after creating it.

Infrastructure Requirements for FIPS 140-2 Mode

Describes FIPS 140-2 infrastructure requirements.

Be sure that your environment meets the FIPS 140 Mode Performance Impact requirements. For more information see, FIPS 140 Mode Performance Impact.

Supported Operating Systems

Supported Operating Systems for FIPS mode are Red Hat Enterprise Linux and CentOS. For details on the tested and supported versions, see Supported Infrastructure Operating Systems.

Deploying Clusters in FIPS Mode

To create a cluster in FIPS mode, we must inform the bootstrap controllers about the appropriate image repository and version tags of Kubernetes's official Nutanix FIPS builds.

Component	Repository	Version
Kubernetes	docker.io/mesosphere	v1.29.6+fips.0
etcd	docker.io/mesosphere	3.5.10+fips.0

AWS Example

When creating a cluster, use the following command line options:

- `--ami <fips enabled AMI> (AWS only)`
- `--kubernetes-version <version>+fips.<build>`
- `--etcd-version <version>+fips.<build>`
- `--kubernetes-image-repository docker.io/mesosphere`
- `--etcd-image-repository docker.io/mesosphere`

```
nkp create cluster aws --cluster-name myFipsCluster \
--ami=ami-03dcaa75d45aca36f \
--kubernetes-version=v1.29.6+fips.0 \
--kubernetes-image-repository=docker.io/mesosphere \
--etcd-image-repository=docker.io/mesosphere \
--etcd-version=3.5.10+fips.0
```

vSphere Example

```
nkp create cluster vsphere \
  --cluster-name ${CLUSTER_NAME} \
  --network <NETWORK_NAME> \
  --control-plane-endpoint-host <xxx.yyy.zzz.000> \
  --data-center <DATACENTER_NAME> \
  --data-store <DATASTORE_NAME> \
  --folder <FOLDER_NAME> \
  --server <VCENTER_API_SERVER_URL> \
  --ssh-public-key-file <SSH_PUBLIC_KEY_FILE> \
  --resource-pool <RESOURE_POOL_NAME> \
  --vm-template <TEMPLATE_NAME> \
  --self-managed \
  --kubernetes-version=v1.29.6+fips.0 \
  --kubernetes-image-repository=docker.io/mesosphere \
```

```
  --etcd-image-repository=docker.io/mesosphere --etcd-version=3.5.10+fips.0
```

FIPS 140 Images: Non-Air-Gapped Environments

Describes using FIPS-140 to create images in a non-air-gapped environment.

Non-air-gapped Environment Create FIPS-140 images

Use the `fips.yaml` override file provided with the image bundles to produce images containing FIPS-140 compliant binaries.

> **Note:** To locate the available Override files in the Konvoy Image Builder repo, see https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides.

Examples

The following snippets show the creation of FIPS-compliant Kubernetes components. If you need the underlying OS to be FIPS-compliant, provide the specific FIPS-compliant OS image using the `--source-ami` flag for AWS.

- A non-air-gapped environment example of override file use is the command below, which produces a FIPS-compliant image on RHEL 8.6 for AWS: Replace `ami` with your infrastructure provisioner

```
konvoy-image build --overrides overrides/fips.yaml images/ami/rhel-86.yaml
```

- vSphere FIPS-complaint using `image.yaml` created during VM Template configuration:

```
konvoy-image build --overrides overrides/fips.yaml images/ova/<image.yaml>
```

> **Note:** In the Using Override Files with Konvoy Image Builder, there is a list of Override Files and how to apply them.

FIPS 140 Images: Air-gapped Environment

Describes using FIPS-140 to create images in an air-gapped environment.

Use the `fips.yaml` override file provided with the image bundles to produce images containing FIPS-140 compliant binaries.

> **Note:** To locate the available Override files in the Konvoy Image Builder repo, see https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides.

Examples:

The following snippets show the creation of FIPS-compliant Kubernetes components. If you need the underlying OS to be FIPS-compliant, provide the specific FIPS-compliant OS image using the `--source-ami` flag for AWS.

- An air-gapped environment example of override file use is the command below, which produces an AWS FIPS-compliant image on RHEL 8.6:

```
konvoy-image build --overrides offline-fips.yaml --overrides overrides/fips.
yaml images/ami/rhel-86.yaml
```

- vSphere FIPS-compliant air-gapped environment example:

```
konvoy-image build --overrides offline-fips.yaml --overrides overrides/fips.
yaml images/ova/<image.yaml>
```

Creating FIPS Clusters in Pre-provisioned FIPS Infrastructure

Steps to create a FIPS-compliant cluster in a Pre-provisioned FIPS Infrastructure

About this task

If you are targeting Pre-provisioned Installation Options on page 69, you can create a FIPS-compliant cluster by doing the following:

Procedure

1. Create a Pre-provisioned: Bootstrap Cluster
2. Create a secret on the bootstrap cluster with the contents from `fips.yaml`override file and any other user overrides you wish to provide.

   ```
   kubectl create secret generic $CLUSTER_NAME-fips-overrides --from-
   file=overrides.yaml=overrides.yaml
   kubectl label secret $CLUSTER_NAME-fips-overrides clusterctl.cluster.x-k8s.io/move=
   ```

   To view the complete list of FIPS Override Files, see FIPS Override Non-air-gapped Files on page 1118.

Validate FIPS 140 in Cluster

Describes using the FIPS tool for validating FIPS compliance in a cluster.

You can use the FIPS validation tool to verify that specific components and services are FIPS-compliant. The tool checks the components by comparing their file signatures against those stored in a signed signature file and by checking that services use the certified algorithms.

Run FIPS Validation

To verify the FIPS compliant cluster, run `NKP check cluster fips`. This command reads from the signature files embedded in the `NKP` executable to validate that specific components and services are FIPS-compliant. Run the command:

The full command usage and flags include:

```
NKP check cluster fips
```

Flags:

```
 -h, --help                          Help for fips
     --kubeconfig string             Path to the kubeconfig file for the fips cluster.
If unspecified, default discovery rules apply.
 -n, --namespace string              If present, the namespace scope for this CLI
request. (default "default")
     --output-configmap string       ConfigMap to store result of the fips check.
(default "check-cluster-fips-output") (DEPRECATED: This flag will be removed in a
future release.)
     --signature-configmap string    ConfigMap with fips signature data to verify.
     --signature-file string         File containing fips signature data.
     --timeout duration              The length of time to wait before giving up. Zero
means wait forever (e.g. 1s, 2m, 3h). (default 10m0s)
```

Run FIPS Validation with Custom Signature File

To validate FIPS-mode operation with the custom signature file, you can use the `signature-file` flag, as in the following command. It would be best to use the `signature-configmap` flag to set the name of the ConfigMap used to store your custom signature file.

```
NKP check cluster fips \
 --signature-file custom.json.asc \
```

```
--signature-configmap custom-signature-file
```

Signature Files

The following signature files are embedded in the `nkp` executable. This information is for reference only. You do not need to download them to run the FIPS check.

Table 66: NKP Version 2.12

NKP Version	Operating System Version	Kubernetes Version	Containerd Version	Signature File URL
2.12.1	RHEL 8.6	v1.29.9	1.6.28	RHEL 8.6
2.12.1	RHEL 8.8	v1.29.9	1.6.28	RHEL 8.8
2.12	RHEL 8.6	v1.29.6	1.6.28	RHEL 8.6
2.12	RHEL 8.8	v1.29.6	1.6.28	RHEL 8.8

FIPS 140 Mode Performance Impact

Describes the performance impact of operating your cluster in FIPS 140 mode.

The Go language cryptographic module, Goboring, relies on CGO's foreign function interface to call C-language functions exposed by the cryptographic module. Each call into the C library starts with a base overhead of 200ns.

One benchmark finds that the time to encrypt a single AES-128 block increased from 13ns to 209ns over the internal Golang implementation. The preferred mode of Nutanix's FIPS module is `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`.

The aggregate impact on a stable control plane seems to be an increase of around 10% CPU utilization over default operation. Workloads that do not directly interact with the control plane are not affected.

For more information, see https://github.com/golang/go/issues/21525

Registry Mirror Tools

Using an external solution for storing and sharing container images.

Kubernetes does not natively provide a registry for hosting the container images you will use to run the applications you want to deploy on Kubernetes. Instead, Kubernetes requires you to use an external solution to store and share container images. There are a variety of Kubernetes-compatible registry options that are compatible with Nutanix Kubernetes Platform (NKP).

How Does it Work?

The first time you request an image from your local registry mirror, it pulls the image from the public registry (such as Docker) and stores it locally before handing it back to you. On subsequent requests, the local registry mirror can serve the image from its own storage.

Air-gapped vs. Non-air-gapped Environments

In a non-air-gapped environment, you can access the Internet. You retrieve artifacts from specialized repositories dedicated to them, such as Docker images contained in DockerHub and Helm Charts that come from a dedicated Helm Chart repository. You can also create your local repository to hold the downloaded container images needed or any custom images you've created with the Konvoy Image Builder tool.

In an air-gapped environment, you need a local repository to store Helm charts, Docker images, and other artifacts. Private registries provide security and privacy in enterprise container image storage, whether hosted remotely or on-premises locally in an air-gapped environment. Nutanix Kubernetes Platform (NKP) in an air-gapped environment requires a local container registry of trusted images to enable production-level Kubernetes cluster management. However, a local registry is also an option in a non-air-gapped environment for speed and security.

If you want to use images from this local registry to deploy applications inside your Kubernetes cluster, you'll need to set up a secret for a private registry. The secret contains your login data, which Kubernetes needs to connect to your private repository.

Local Registry Tools Compatible with NKP

Tools such as JFrog Artifactory, Amazon AWS ECR, Harbor, and Nexus handle multiple artifacts in one local repository.

AWS ECR

AWS ECR (Elastic Container Registry) is supported as your air-gapped image registry or a non-air-gapped registry mirror. Nutanix Kubernetes Platform (NKP) added support for using AWS ECR as a default registry when uploading image bundles in AWS.

Prerequisites

- Ensure you have followed the steps to create proper permissions in AWS Minimal Permissions and Role to Create Clusters
- Ensure you have created AWS Cluster IAM Policies, Roles, and Artifacts

Upload the Air-gapped Image Bundle to the Local ECR Registry:

A cluster administrator uses NKP CLI commands to upload the image bundle to ECR with parameters:

```
NKP push bundle --bundle <bundle> --to-registry=<ecr-registry-address>/<ecr-registry-name>
```

Parameter Definitions:

- `--bundle <bundle>` the group of images. The example below is for the NKP air-gapped environment bundle
- `--to-registry=<ecr-registry-address>/<ecr-registry-name>` to provide registry location for push

An example command:

```
NKP push bundle --bundle container-images/konvoy-image-bundle-v2.12.0.tar --to-registry=333000009999.
dkr.ecr.us-west-2.amazonaws.com/can-test
```

> **Note:** You can also set an environment variable with your registry address for **ECR**:

```
export REGISTRY_URL=<ecr-registry-URI>
```

- `REGISTRY_URI`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.
- The environment where you are running the `NKP push` command must be authenticated with AWS to load your images into ECR.

Air-gapped Environment Information regarding your AWS ECR Account

The cluster administrator uses existing NKP CLI commands to create the cluster and refer to their internal ECR for image repository. The administrator does not need to provide static ECR registry credentials. See Use a Registry Mirror and Create an EKS Cluster from the CLI for more details.

JFrog Artifactory

JFrog Artifactory can function as a container registry and an automated management tool for binaries and artifacts of all types. If you use JFrog Artifactory or JFrog Container Registry, you must update to a new software version. Use a build newer than version 7.11; older versions are not compatible.

For more information, see https://jfrog.com/artifactory/.

Nexus Registry

Nexus Repository is a package registry for your Docker images and Helm Chart repositories and supports Proxy, Hosted, and Group repositories. It can be used as a single registry for all your Kubernetes deployments.

For more information, see https://www.nexusregistry.com/info/.

Harbor Registry

Install Harbor and configure any HTTP access require ands the system level parameters in the `harbor.yml` file. Then, run the installer script. If you upgrade from a previous version of Harbor, you update the configuration file and migrate your data to fit the database schema of the later version. For information about upgrading, see https://goharbor.io/docs/2.0.0/administration/upgrade/ and https://goharbor.io/docs/2.0.0/install-config/download-installer/. A version than Harbor Registry v2.1.1-5f52168e will support OCI images.

While seeding, you may see error messages such as the following:

```
2023/09/12 20:01:18 retrying without mount: POST https://harbor-registry.daclusta/v2/
harbor-registry
/mesosphere/kube-proxy/blobs/uploads/?from=mesosphere%2Fkube-
proxy&mount=sha256%3A9fd5070b83085808ed850ff84acc
98a116e839cd5dcfefa12f2906b7d9c6e50d&origin=REDACTED: UNAUTHORIZED: project not found,
 name: mesosphere
: project not found, name: mesosphere
```

This indicates that the image was not successfully pushed to your Harbor docker registry but is a false positive error message. This will only affect the version of the Nutanix Kubernetes Platform (NKP) binary newer than NKP 2.4.0. This does not affect any other Local Registry solution, such as Nexus or Artifactory. You can safely ignore these error messages.

Bastion Host

If you have not set up a Bastion Host yet, refer to that section of the documentation.

Related Information

If you need to configure a private registry with a registry mirror, see Use a Registry Mirror.

Using a Registry Mirror

Registry mirrors are local copies of images from a public registry that follows (or mirrors) the file structure of a public registry. You can push container images to a local registry from downloaded images or images you create with the Konvoy Image Builder. If your environment allows Internet access, the mirror registry consults its upstream registries when an image is not found locally. This kind of registry contains no images other than the ones requested.

Export Variables
Adding your registry information to the environment variable.

Set the environment variable with your registry information.

```
export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
export REGISTRY_USERNAME=<username>
export REGISTRY_PASSWORD=<password>
export REGISTRY_CA=<path to the cacert file on the bastion>
```

Definitions:

`REGISTRY_URL`: the address of an existing local registry accessible in the VPC that the new cluster nodes will be configured to use a mirror registry when pulling images.

- For example, `https://registry.example.com`

Other local registries may use the options below:

- `REGISTRY_USERNAME`: optional-set to a user with pull access to this registry.
- `REGISTRY_PASSWORD`: optional if username is not set.
- JFrog - `REGISTRY_CA`: (optional) the path on the bastion machine to the registry CA. This value is only needed if the registry uses a self-signed certificate and the AMIs are not already configured to trust this CA.
- To increase Docker Hub's rate limit use your Docker Hub credentials when creating the cluster by setting flags `----registry-mirror-url=https://registry-1.docker.io --registry-mirror-username=<your-username> --registry-mirror-password=<your-password>` when running `NKP create cluster`.

Use Flags in Cluster Creation
Usage of Flags in Cluster Creation

If you set the `--registry-mirror` flag during cluster creation, the Kubelet will now send requests to the dynamic-credential-provider with a different config. Only use one image registry per cluster.

To apply private registry configurations during the `NKP cluster create` operation, add the appropriate flags to the command:

Table 67: Table

Registry Configuration	Flag
CA certificate chain to use while communicating with the registry mirror using Transport Layer Security(TLS)	`--registry-mirror-cacert file`
Password to authenticate the registry mirror	`--registry-mirror-password string` OR apply variable `${REGISTRY_PASSWORD}`
URL of a container registry to use as a mirror in the cluster	`--registry-mirror-url string` OR apply variable `${REGISTRY_URL}`
Set to a user that has pull access to this registry	`--registry-mirror-username string` OR apply variable `${REGISTRY_USERNAME}`

This is useful when using an internal registry and when Internet access is unavailable, such as in an air-gapped environment. However, registry mirrors can also be used in non-air-gapped environments for security and speed.

Note: AWS ECR - Adding the mirror flags to EKS enables new clusters also to use ECR as image mirror. If you set the `--registry-mirror` flag, the Kubelet will now send requests to the `dynamic-credential-provider` with a different config. You can still pull your images from ECR directly or use ECR as a mirror.

You can deploy and test workloads when the cluster is up and running.

Registry Mirror Cluster Example

Selecting your provider, run:

```
NKP create cluster [aws, azure, gcp, preprovisoned, vsphere] \
    --cluster-name=${CLUSTER_NAME} \
    --registry-mirror-cacert /tmp/registry.pem \
    --registry-mirror-url=${REGISTRY_URL}
```

More information is found in the Custom Installation and Infrastructure Tools on page 677 sections under the Create a New Cluster section of each Infrastructure Provider. Mirrors can be used in both air-gapped and non-air-gapped environments by adding the flag to the `NKP create cluster` command.

Seeding the Registry for an Air-gapped Cluster

About this task

Before creating an air-gapped Kubernetes cluster, you must load the required images in a local registry for the Konvoy component. This registry must be accessible from both the Creating a Bastion Host on page 685 and either the Amazon Elastic Compute Cloud (Amazon EC2) instances (if deploying to Amazon Web Services (AWS)) or other machines that will be created for the Kubernetes cluster.

Note: If you do not already have a local registry set up, see the Local Registry Tools page for more information.

Procedure

1. If not already done in prerequisites, download the air-gapped bundle `NKP-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz` , and extract the tarball to a local directory.

   ```
   tar -xzvf NKP-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz
   ```

2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. Change your directory to the ###NKP#-<version># directory for the bootstrap cluster example,

   ```
   cd NKP-v2.12.0
   ```

3. Set an environment variable with your registry address and any other needed variables using this command.

   ```
   export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>"
   export REGISTRY_USERNAME=<username>
   export REGISTRY_PASSWORD=<password>
   export REGISTRY_CA=<path to the cacert file on the bastion>
   ```

4. Execute the following command to load the air-gapped image bundle into your private registry using any relevant flags to apply the above variables.

   ```
   NKP push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL}
   ```

```
 --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=
${REGISTRY_PASSWORD}
```

Note: It may take some time to push all the images to your image registry, depending on the network's performance between the machine you are running the scripttoscriptn and the registry.

Note: To use Elastic Container Registry (ECR), set an environment variable with your registry address for **ECR**:

```
export REGISTRY_URL=<ecr-registry-URI>
```

- `REGISTRY_URL`: the address of an existing local registry accessible in the Virtual Private Cloud (VPC) that the new cluster nodes will be configured to use a mirror registry when pulling images.
- The environment where you are running the `NKP push` command must be authenticated with AWS in order to load your images into ECR.

You are now ready to create an air-gapped bootstrap cluster for a custom cluster for your infrastructure provideror create an air-gapped cluster from the Day 1 - Basic Installs section for your provider.

Configure the Control Plane

Users can modify the `KubeadmControlplane` cluster-API object to configure different kubelet options. See the following guide if you wish to configure your control plane beyond the existing options available from flags.

Prerequisites

Before you begin, make sure you have created your cluster using a bootstrap cluster from the respective Infrastructure Providers section.

Modifying Audit Logs

Modify control plane audit logs.

About this task

To modify the control plane option, get the appropriate `cluster-api` objects that describe the cluster by running the following command:

Note: The following example uses AWS, but can be used for `gcp`, `azure`, `preprovisioned`, and `vsphere` clusters.

```
NKP create cluster aws -c {MY_CLUSTER_NAME} -o yaml --dry-run >>
 {MY_CLUSTER_NAME}.yaml
```

Procedure

1. When you open `{MY_CLUSTER_NAME}.yaml` with your favorite text editor, look for the `KubeadmControlPlane` object for your cluster. For example.

```
apiVersion: controlplane.cluster.x-k8s.io/v1beta1
kind: KubeadmControlPlane
metadata:
  name: my-cluster-control-plane
  namespace: default
spec:
  kubeadmConfigSpec:
    clusterConfiguration:
      apiServer:
```

```
        extraArgs:
          audit-log-maxage: "30"
          audit-log-maxbackup: "10"
          audit-log-maxsize: "100"
          audit-log-path: /var/log/audit/kube-apiserver-audit.log
          audit-policy-file: /etc/kubernetes/audit-policy/apiserver-audit-policy.yaml
          cloud-provider: aws
          encryption-provider-config: /etc/kubernetes/pki/encryption-config.yaml
        extraVolumes:
        - hostPath: /etc/kubernetes/audit-policy/
          mountPath: /etc/kubernetes/audit-policy/
          name: audit-policy
        - hostPath: /var/log/kubernetes/audit
          mountPath: /var/log/audit/
          name: audit-logs
      controllerManager:
        extraArgs:
          cloud-provider: aws
          configure-cloud-routes: "false"
      dns: {}
      etcd:
        local:
          imageTag: 3.5.7
      networking: {}
      scheduler: {}
    files:
    - content: |
        # Taken from https://github.com/kubernetes/kubernetes/blob/master/cluster/
gce/gci/configure-helper.sh
        # Recommended in Kubernetes docs
        apiVersion: audit.k8s.io/v1
        kind: Policy
        rules:
          # The following requests were manually identified as high-volume and low-
risk,
          # so drop them.
          - level: None
            users: ["system:kube-proxy"]
            verbs: ["watch"]
            resources:
              - group: "" # core
                resources: ["endpoints", "services", "services/status"]
          - level: None
            # Ingress controller reads 'configmaps/ingress-uid' through the unsecured
 port.
            # TODO(#46983): Change this to the ingress controller service account.
            users: ["system:unsecured"]
            namespaces: ["kube-system"]
            verbs: ["get"]
            resources:
              - group: "" # core
                resources: ["configmaps"]
          - level: None
            users: ["kubelet"] # legacy kubelet identity
            verbs: ["get"]
            resources:
              - group: "" # core
                resources: ["nodes", "nodes/status"]
          - level: None
            userGroups: ["system:nodes"]
            verbs: ["get"]
            resources:
```

```
            - group: "" # core
              resources: ["nodes", "nodes/status"]
        - level: None
          users:
            - system:kube-controller-manager
            - system:kube-scheduler
            - system:serviceaccount:kube-system:endpoint-controller
          verbs: ["get", "update"]
          namespaces: ["kube-system"]
          resources:
            - group: "" # core
              resources: ["endpoints"]
        - level: None
          users: ["system:apiserver"]
          verbs: ["get"]
          resources:
            - group: "" # core
              resources: ["namespaces", "namespaces/status", "namespaces/finalize"]
        - level: None
          users: ["cluster-autoscaler"]
          verbs: ["get", "update"]
          namespaces: ["kube-system"]
          resources:
            - group: "" # core
              resources: ["configmaps", "endpoints"]
        # Don't log HPA fetching metrics.
        - level: None
          users:
            - system:kube-controller-manager
          verbs: ["get", "list"]
          resources:
            - group: "metrics.k8s.io"
        # Don't log these read-only URLs.
        - level: None
          nonResourceURLs:
            - /healthz*
            - /version
            - /swagger*
        # Don't log events requests.
        - level: None
          resources:
            - group: "" # core
              resources: ["events"]
        # node and pod status calls from nodes are high-volume and can be large,
don't log responses for expected updates from nodes
        - level: Request
          users: ["kubelet", "system:node-problem-detector",
"system:serviceaccount:kube-system:node-problem-detector"]
          verbs: ["update","patch"]
          resources:
            - group: "" # core
              resources: ["nodes/status", "pods/status"]
          omitStages:
            - "RequestReceived"
        - level: Request
          userGroups: ["system:nodes"]
          verbs: ["update","patch"]
          resources:
            - group: "" # core
              resources: ["nodes/status", "pods/status"]
          omitStages:
            - "RequestReceived"
```

```
        # deletecollection calls can be large, don't log responses for expected
namespace deletions
        - level: Request
          users: ["system:serviceaccount:kube-system:namespace-controller"]
          verbs: ["deletecollection"]
          omitStages:
            - "RequestReceived"
        # Secrets, ConfigMaps, and TokenReviews can contain sensitive & binary
data,
        # so only log at the Metadata level.
        - level: Metadata
          resources:
            - group: "" # core
              resources: ["secrets", "configmaps"]
            - group: authentication.k8s.io
              resources: ["tokenreviews"]
          omitStages:
            - "RequestReceived"
        # Get responses can be large; skip them.
        - level: Request
          verbs: ["get", "list", "watch"]
          resources:
            - group: "" # core
            - group: "admissionregistration.k8s.io"
            - group: "apiextensions.k8s.io"
            - group: "apiregistration.k8s.io"
            - group: "apps"
            - group: "authentication.k8s.io"
            - group: "authorization.k8s.io"
            - group: "autoscaling"
            - group: "batch"
            - group: "certificates.k8s.io"
            - group: "extensions"
            - group: "metrics.k8s.io"
            - group: "networking.k8s.io"
            - group: "node.k8s.io"
            - group: "policy"
            - group: "rbac.authorization.k8s.io"
            - group: "scheduling.k8s.io"
            - group: "settings.k8s.io"
            - group: "storage.k8s.io"
          omitStages:
            - "RequestReceived"
        # Default level for known APIs
        - level: RequestResponse
          resources:
            - group: "" # core
            - group: "admissionregistration.k8s.io"
            - group: "apiextensions.k8s.io"
            - group: "apiregistration.k8s.io"
            - group: "apps"
            - group: "authentication.k8s.io"
            - group: "authorization.k8s.io"
            - group: "autoscaling"
            - group: "batch"
            - group: "certificates.k8s.io"
            - group: "extensions"
            - group: "metrics.k8s.io"
            - group: "networking.k8s.io"
            - group: "node.k8s.io"
            - group: "policy"
            - group: "rbac.authorization.k8s.io"
```

```
              - group: "scheduling.k8s.io"
              - group: "settings.k8s.io"
              - group: "storage.k8s.io"
            omitStages:
              - "RequestReceived"
          # Default level for all other requests.
          - level: Metadata
            omitStages:
              - "RequestReceived"
      path: /etc/kubernetes/audit-policy/apiserver-audit-policy.yaml
      permissions: "0600"
    - content: |
        #!/bin/bash
        # CAPI does not expose an API to modify KubeProxyConfiguration
        # this is a workaround to use a script with preKubeadmCommand to modify the
kubeadm config files
        # https://github.com/kubernetes-sigs/cluster-api/issues/4512
        for i in $(ls /run/kubeadm/ | grep 'kubeadm.yaml\|kubeadm-join-config.yaml');
do
          cat <<EOF>> "/run/kubeadm//$i"
        ---
        kind: KubeProxyConfiguration
        apiVersion: kubeproxy.config.k8s.io/v1alpha1
        metricsBindAddress: "0.0.0.0:10249"
        EOF
        done
      path: /run/kubeadm/konvoy-set-kube-proxy-configuration.sh
      permissions: "0700"
    - content: |
        [metrics]
          address = "0.0.0.0:1338"
          grpc_histogram = false
      path: /etc/containerd/conf.d/konvoy-metrics.toml
      permissions: "0644"
    - content: |
        #!/bin/bash
        systemctl restart containerd

        SECONDS=0
        until crictl info
        do
          if (( SECONDS > 60 ))
          then
             echo "Containerd is not running. Giving up..."
             exit 1
          fi
          echo "Containerd is not running yet. Waiting..."
          sleep 5
        done
      path: /run/konvoy/restart-containerd-and-wait.sh
      permissions: "0700"
    - contentFrom:
        secret:
          key: value
          name: my-cluster-etcd-encryption-config
      owner: root:root
      path: /etc/kubernetes/pki/encryption-config.yaml
      permissions: "0640"
    format: cloud-config
    initConfiguration:
      localAPIEndpoint: {}
      nodeRegistration:
```

```
      kubeletExtraArgs:
        cloud-provider: aws
      name: '{{ ds.meta_data.local_hostname }}'
  joinConfiguration:
    discovery: {}
    nodeRegistration:
      kubeletExtraArgs:
        cloud-provider: aws
      name: '{{ ds.meta_data.local_hostname }}'
  preKubeadmCommands:
  - systemctl daemon-reload
  - /run/konvoy/restart-containerd-and-wait.sh
  - /run/kubeadm/konvoy-set-kube-proxy-configuration.sh
machineTemplate:
  infrastructureRef:
    apiVersion: infrastructure.cluster.x-k8s.io/v1beta1
    kind: AWSMachineTemplate
    name: my-cluster-control-plane
    namespace: default
  metadata: {}
replicas: 3
rolloutStrategy:
  rollingUpdate:
    maxSurge: 1
  type: RollingUpdate
version: v1.29.6
```

Note: If you use the previous example as-is, update the Kubernetes version number on the final line by replacing the x with your version.

2. Now, you can configure the fields below for the log backend. The log backend will write audit events to a file in JSON format. You can configure the log audit backend using the `kube-apiserver` flags shown in the example.

```
audit-log-maxage
audit-log-maxbackup
audit-log-maxsize
audit-log-path
```

Note: For more information, see upstream documentation at https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/#log-backend.

3. After modifying the values appropriately, you can create the cluster by running the `kubectl create -f {MY_CLUSTER_NAME}.yaml` command.

```
kubectl create -f {MY_CLUSTER_NAME}.yaml
```

4. Once the cluster is created, users can get the corresponding kubeconfig for the cluster by running the `NKP get kubeconfig -c {MY_CLUSTER_NAME} >> {MY_CLUSTER_NAME}.conf` command.

```
NKP get kubeconfig -c {MY_CLUSTER_NAME} >> {MY_CLUSTER_NAME}.conf
```

Viewing the Audit Logs

View the NKP audit logs.

About this task

Fluent Bit is disabled by default on the management cluster. To view the audit logs, perform the following task.

Procedure

1. Run the `NKP diagnose --kubeconfig={MY_CLUSTER_NAME}.conf` command.

   ```
   NKP diagnose --kubeconfig={MY_CLUSTER_NAME}.conf
   ```

   A file similar to `support-bundle-2022-08-15T02_28_48.tar.gz` is created.

2. Untar the file.
   For example:

   ```
   tar -xzf support-bundle-2022-08-15T02_28_48.tar.gz
   ```

3. Navigate to the node-diagnostics sub-directory from the extracted file.
   For example:

   ```
   cd support-bundle-2022-08-15T02_28_48/node-diagnostics
   ```

4. To find the audit logs, run the following command.

   ```
   $ find . -type f | grep audit.log
   ./ip-10-0-142-117.us-west-2.compute.internal/data/kube_apiserver_audit.log
   ./ip-10-0-148-139.us-west-2.compute.internal/data/kube_apiserver_audit.log
   ./ip-10-0-128-181.us-west-2.compute.internal/data/kube_apiserver_audit.log
   ```

What to do next

For information on related topics or procedures, see Fluent bit.

Updating Cluster Node Pools

About this task

Upgrading a node pool involves draining the existing nodes in the node pool and replacing them with new nodes. To ensure minimum downtime and maintain high availability of the critical application workloads during the upgrade process, we recommend deploying Pod Disruption Budget (Disruptions) for your critical applications. For more information, see https://kubernetes.io/docs/concepts/workloads/pods/disruptions/

The Pod Disruption Budget will prevent any impact on critical applications as a result of misconfiguration or failures during the upgrade process.

Before you begin

- Deploy Pod Disruption Budget (PDB). For more information, see https://kubernetes.io/docs/concepts/workloads/pods/disruptions/
- Konvoy Image Builder (KIB)

Procedure

1. Deploy Pod Disruption Budget for your critical applications. If your application can tolerate only one replica to be unavailable at a time, then you can set the Pod disruption budget as shown in the following example. The example below is for NVIDIA GPU node pools, but the process is the same for all.

   Note: Repeat this step for each additional node pool

2. Create the `pod-disruption-budget-nvidia.yaml` file.

   ```
   apiVersion: policy/v1
   kind: PodDisruptionBudget
   ```

```
metadata:
  name: nvidia-critical-app
spec:
  maxUnavailable: 1
  selector:
    matchLabels:
      app: nvidia-critical-app
```

3. Apply the YAML file above using the command `kubectl create -f pod-disruption-budget-nvidia.yaml`.
4. Prepare OS image for your node pool using the Konvoy Image Builder.

What to do next

For information on related topics or procedures, see Upgrade NKP Pro.

Cluster and NKP Installation Verification

Check Nutanix Kubernetes Platform (NKP) components to verify the status of your cluster

This section contains information on how to verify a NKP installation.

Checking the Cluster Infrastructure and Nodes

Diagnosis tools check your cluster infrastructure and nodes.

About this task

Nutanix Kubernetes Platform (NKP) ships with default diagnosis tools to check your cluster, such as the `describe` command. You can use those tools to validate your installation.

Procedure

1. If you have not done so already, set the environment variable for your cluster name, substituting `NKP-example` with the name of your cluster.

```
export CLUSTER_NAME=NKP-example
```

2. Then, run this command to check the health of the cluster infrastructure.

```
NKP describe cluster --cluster-name=${CLUSTER_NAME}
```

> **Note:** For more details on the `NKP describe cluster` command, see https://docs.d2iq.com/dkp/2.8/dkp-describe-cluster.

A healthy cluster returns an output similar to this example.

```
NAME                                                          READY  SEVERITY
 REASON  SINCE  MESSAGE
Cluster/NKP-example                                           True
    121m
##ClusterInfrastructure - AWSCluster/NKP-example              True
    121m
##ControlPlane - KubeadmControlPlane/NKP-example-control-plane  True
    121m
# ##Machine/NKP-example-control-plane-h52t6                   True
    121m
# ##Machine/NKP-example-control-plane-knrrh                   True
    121m
# ##Machine/NKP-example-control-plane-zmjjx                   True
    121m
```

```
##Workers

  ##MachineDeployment/NKP-example-md-0                          True
    121m
    ##Machine/NKP-example-md-0-88488cb74-2vxjq                  True
    121m
    ##Machine/NKP-example-md-0-88488cb74-84xsd                  True
    121m
    ##Machine/NKP-example-md-0-88488cb74-9xmc6                  True
    121m
    ##Machine/NKP-example-md-0-88488cb74-mjf6s                  True
    121m
```

3. Use this `kubectl` command to see if all cluster nodes are ready.

```
kubectl get nodes
```

Example output showing all statuses set to `Ready`.

```
NAME                                          STATUS   ROLES                  AGE
 VERSION
ip-10-0-112-116.us-west-2.compute.internal    Ready    <none>                 135m
 v1.21.6
ip-10-0-122-142.us-west-2.compute.internal    Ready    <none>                 135m
 v1.21.6
ip-10-0-186-214.us-west-2.compute.internal    Ready    control-plane,master   133m
 v1.21.6
ip-10-0-231-82.us-west-2.compute.internal     Ready    control-plane,master   135m
 v1.21.6
ip-10-0-71-114.us-west-2.compute.internal     Ready    <none>                 135m
 v1.21.6
ip-10-0-71-207.us-west-2.compute.internal     Ready    <none>                 135m
 v1.21.6
ip-10-0-85-253.us-west-2.compute.internal     Ready    control-plane,master   137m
 v1.21.6
```

Monitor the CAPI Resources

Obtain a list of all resources that comprise a cluster and their statuses.

```
kubectl get cluster-api
```

Verify all Pods

All pods installed by Nutanix Kubernetes Platform (NKP) need to be in `Running` or `Completed` status for a successful install.

```
kubectl get pods --all-namespaces
```

Troubleshooting

If any pod is not in `Running` or `Completed` status, you need to investigate further. If something has not been deployed properly or thoroughly, run the `NKP diagnose` command, this collects information from pods and infrastructure.

GPU for Konvoy

In this version of Nutanix Kubernetes Platform (NKP), the nodes with NVIDIA GPUs are configured with nvidia-gpu-operator (Overview — NVIDIA Cloud Native Technologies documentation) and NVIDIA drivers to support the container runtime.

This page will link you to all the necessary GPU pages.

- Nutanix GPU Passthrough
- Updating Cluster Nodepools
- KIB for GPU

The remainder of GPU information is found in Cluster Management Operations section of the documentation.

Delete a NKP Cluster with One Command

Use a single CLI to delete a Kubernetes cluster on any of the platforms supported by NKP.

About this task

You can use a single command line entry to delete a Kubernetes cluster on any platforms supported by Nutanix Kubernetes Platform (NKP). Deleting a cluster means removing the cluster, all of its nodes, and all the platform applications deployed on it as part of its creation. Use this command with extreme care, as it is not reversible!

> **Note:** You need to delete the attachment for any clusters attached in Kommander before running the `delete` command.

> **Important:** Persistent Volumes (PVs) are not automatically deleted by design to preserve your data. However, they take up storage space if not deleted. You must delete PVs manually. Information for backup of a cluster and PVs is in the documentation called Cluster Applications and Persistent Volumes Backup on page 537. With Vsphere clusters, `NKP delete` doesn't delete the virtual disks backing the PVs for NKP add-ons. Therefore, the internal VMware cluster eventually runs out of storage. These PVs are only visible if VSAN is installed, giving users a Container Native Storage tab.

Procedure

1. Set the environment variable to be used throughout this documentation.

   ```
   export CLUSTER_NAME=cluster-example
   ```

2. The basic NKP `delete` command structure is.

   ```
   NKP delete cluster --cluster-name=${CLUSTER_NAME} --self-managed --kubeconfig=
   ${CLUSTER_NAME}.conf
   ```

   When you use the `--self-managed` flag, the prerequisite components and resources are moved from the self-managed cluster before deleting. When you omit this flag (the default value is false), the resources are assumed to be installed in a management cluster. The default value is false or no flag.

   This command performs the following actions:

   - Creates a local bootstrap cluster
   - Moves controllers to it
   - Deletes the management cluster
   - Deletes the local bootstrap cluster

9

KONVOY IMAGE BUILDER

The Konvoy Image Builder (KIB) is a complete solution for building Cluster API compliant images. Konvoy Image Builder aims to produce a common operating surface to run Konvoy across heterogeneous infrastructure. KIB relies on:

- **Ansible** to install software, configure, and sanitize systems for running Konvoy.
- **Packer** is used to build images for cloud environments.
- **Goss** is used to validate systems are capable of running Konvoy.

This section describes using KIB to create Cluster API compliant machine images. Machine images contain configuration information and software to create a specific, pre-configured operating environment. For example, you can create an image of your computer system settings and software. The machine image can then be replicated and distributed to your computer system for other users. KIB uses variable overrides to specify the base image and container images to use in your new machine image. The variable overrides files for Nvidia and FIPS, which can be ignored unless an overlay feature is added.

How KIB Works

You can use KIB to build machine images, but first, you must know its default behaviors. As stated, KIB installs kubeadm and the other basic components you need so that when the machine boots for the first time, it becomes a Kubernetes control plane or worker node and then can form or join a cluster.

KIB does this by booting a computer using a stripped-down base image, like an AMI, and then runs a series of steps to install all of the components that Nutanix Kubernetes Platform (NKP) needs. When the installation is complete, KIB takes a snapshot or backup of that machine image and saves it. This becomes the image or AMI, and so on, that you use when building the cluster.

Prerequisites

Before you begin, you must ensure your versions of KIB and NKP are compatible:

- Download the Konvoy Image Builder bundle from the KIB Version column of the chart below for your version of NKP prefixed with `konvoy-image-bundle` for your Operating System.
- Check the Supported Infrastructure Operating Systems and the Supported Kubernetes Version for your Provider.
- An x86_64-based Linux or MacOS machine.
- A Container engine/runtime installed is required to install NKP and bootstrap:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. On macOS, Docker runs in a virtual machine, which needs to be configured with at least 8 GB of memory. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.

- For air-gapped only - a local registry.

Additional Configuration to Know Using KIB

A variety of flags can be used to pass variables.

- **Override files**

  You can use override files to customize some of the components installed on this machine image. The KIB base override files are located in this GitHub repository https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides. For more information on using override flags, see Use Override Files with Konvoy Image Builder on page 1118

- **Customize image YAML**

  Begin creating an image, interrupt the process so that the manifest.jason gets built, and you can open and edit keys in the YAML. For more information, see Customize your Image YAML or Manifest File.

- **Using HTTP or HTTPS Proxies**

  In some networked environments, the machines used for building images can reach the Internet, but only through an HTTP or HTTPS proxy. For NKP to operate in these networks, you need a way to specify what proxies to use. Further explanation is found in Use HTTP or HTTPS Proxy with KIB Images on page 1126

Compatible NKP to KIB Versions

Along with the KIB Bundle, we publish a file containing checksums for the bundle files. The recommendation for using these checksums is to verify the integrity of the downloads.

On the corresponding link page, download the package prefixed with `konvoy-image-builder` for your OS.

Table 68: KIB Version Table

NKP/ Version	KIB Version (Click to download bundle)
v2.12.1	v2.14.1
v2.7.1	v2.8.7
v2.12.0	v2.13.2
v2.6.2	v2.5.5
v2.6.1	v2.5.4
v2.6.0	v2.5.0
v2.5.2	v2.2.13

Extract KIB Bundle

Extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION` folder. The bundled version of `konvoy-image` contains an embedded `docker` image that contains all the requirements for building the image.

> **Note:** The `konvoy-image` binary and all supporting folders are also extracted, and bind mount places the current working directory (`${PWD}`) into the container to be used. For more information, see https://docs.docker.com/storage/bind-mounts/.

Set the environment variables for AWS access. The following variables must be set using your credentials, including required IAM. For more information, see https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-envvars.html.

```
export AWS_ACCESS_KEY_ID
export AWS_SECRET_ACCESS_KEY
export AWS_DEFAULT_REGION
```

Next Steps

Either return to Basic Install or Custom Install instructions, or for more KIB specific provider information, you can continue to the provider link below for additional information:

- Basic Install
- Custom Install instructions

Creating an Air-gapped Package Bundle

The air-gapped bundle can be customized and modified depending on your environment these are operating system dependencies required for NKP that are fetched from the internet..

About this task

Creating the Operating System package bundles can be customizable to the environment in which it is built as well as customizable in the packages that it attempts to get.

Before you begin

In previous NKP releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts with the exception of the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- `containerd` tarball

> **Note:**
>
> To modify repository templates for RHEL, these files are required for operating systems that will use NVIDIA GPU drivers.
>
> - RHEL 8.6 - Repo templates
> - RHEL 8.6 - Repo templates

Procedure

1. Download `nkp-air-gapped-bundle_v2.8.1_linux_amd64.tar.gz`, and extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.8.1_linux_amd64.tar.gz && cd nkp-v2.8.1/kib
   ```

   > **Note:** RPMs are packaged into the KIB create-package-bundle workflow for RHEL and Rocky. For other distros the current behavior remains the same.

2. You will need to fetch the distro packages as well as other artifacts. By fetching the distro packages from distro repositories, you get the latest security fixes available at machine image build time.

3. In your download location, with internet access, you need to create an OS package bundle for the Target OS you use for the nodes in your NKP cluster. To create it, run the new NKP command `create-package-bundle`.This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`.

   Example command using Rockly Linux:

   ```
   ./konvoy-image create-package-bundle --os rocky-9.1 --output-directory=artifacts
   ```

   Other supported air-gapped Operating Systems (OSs) can be specified in place of `--os rocky-9.1` using the flag and corresponding OS name:

   - `redhat-8.6`
   - `redhat-8.8`
   - `ubuntu-20.04`
   - `ubuntu-22.04`

   Note: For FIPS, pass the flag: `--fips`

   Note: For RHEL OS, pass your RedHat subscription/licensing manager credentials.

   ```
   export RHSM_ACTIVATION_KEY="-ci"
   export RHSM_ORG_ID="1232131"
   ```

   OR

   ```
   export RHSM_USER=""
   export RHSM_PASS=""
   ```

4. Run the `konvoy-image` command to build and validate the image. Ensure you have named the correct AMI image YAML file for your OS in the `konvoy-image build` command.

   ```
   konvoy-image build aws images/ami/rhel-86.yaml
   ```

   Note:

   - 1. **To enable EUS**, add the `--enable-eus-repos` flag which fetches packages from EUS repositories during RHEL package bundles creation. It is disabled by default.
     2. **If kernel headers** are required for GPU, add the `--fetch-kernel-headers` flag which fetches kernel headers for the target operating system. To modify the version, edit the file at bundles/{OS_NAME} {VERSION}

5. Use custom image with NKP by specifying it with the image flag during the create cluster command.

Use KIB with AWS

How to use Konvoy Image Builder (KIB) with Amazon Web Services (AWS).

The following section describes using Konvoy Image Builder (KIB) with Amazon Web Services (AWS). Ensure that the minimum required set of permissions is met before going to the next step. To view an example in the AWS Image Builder Book showing the required set of permissions, see https://image-builder.sigs.k8s.io/capi/providers/aws.html#required-permissions-to-build-the-aws-amis

Creating Minimal IAM Permissions for KIB

Guides you in creating and using a minimally-scoped policy to create an Image for an AWS.

Before you begin

Before applying the IAM Policies, verify the following:

- You have a valid AWS account with credentials configured that can manage CloudFormation Stacks, IAM Policies, IAM Roles, and IAM Instance Profiles. For more information, see https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-profiles.html
- The AWS CLI utility is installed. For more information, see https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-profiles.html.

About this task

This section guides you in creating and using a minimally-scoped policy to create an Image for an AWS account using Konvoy Image Builder.

Minimal Permissions

The following example is an AWSCloudformation stack that creates the minimal policy to run KIB in AWS.

Procedure

1. Copy the following into a file.

```
AWSTemplateFormatVersion: 2010-09-09
Resources:
  AWSIAMInstanceKIBUser:
    Properties:
      InstanceProfileName: KIBUserInstnaceProfile
      Roles:
        - Ref: KIBUserRole
    Type: AWS::IAM::InstanceProfile
  AWSIAMManagedPolicyKIBPolicy:
    Properties:
      Description: Minimal policy to run KIB in AWS
      ManagedPolicyName: kib-policy
      PolicyDocument:
        Statement:
          - Action:
              - ec2:AssociateRouteTable
              - ec2:AssociateRouteTable
              - ec2:AttachInternetGateway
              - ec2:AttachVolume
              - ec2:AuthorizeSecurityGroupIngress
              - ec2:CreateImage
              - ec2:CreateInternetGateway
              - ec2:CreateKeyPair
              - ec2:CreateRoute
              - ec2:CreateRouteTable
              - ec2:CreateSecurityGroup
              - ec2:CreateSubnet
              - ec2:CreateTags
              - ec2:CreateVolume
              - ec2:CreateVpc
              - ec2:DeleteInternetGateway
              - ec2:DeleteKeyPair
              - ec2:DeleteRouteTable
              - ec2:DeleteSecurityGroup
              - ec2:DeleteSnapshot
              - ec2:DeleteSubnet
              - ec2:DeleteVolume
              - ec2:DeleteVpc
              - ec2:DeregisterImage
```

```
            - ec2:DescribeAccountAttributes
            - ec2:DescribeImages
            - ec2:DescribeInstances
            - ec2:DescribeInternetGateways
            - ec2:DescribeKeyPairs
            - ec2:DescribeNetworkAcls
            - ec2:DescribeNetworkInterfaces
            - ec2:DescribeRegions
            - ec2:DescribeRouteTables
            - ec2:DescribeSecurityGroups
            - ec2:DescribeSubnets
            - ec2:DescribeVolume
            - ec2:DescribeVpcAttribute
            - ec2:DescribeVpcClassicLink
            - ec2:DescribeVpcClassicLinkDnsSupport
            - ec2:DescribeVpcs
            - ec2:DetachInternetGateway
            - ec2:DetachVolume
            - ec2:DisassociateRouteTable
            - ec2:ModifyImageAttribute
            - ec2:ModifySnapshotAttribute
            - ec2:ModifySubnetAttribute
            - ec2:ModifyVpcAttribute
            - ec2:RegisterImage
            - ec2:RevokeSecurityGroupEgress
            - ec2:RunInstances
            - ec2:StopInstances
            - ec2:TerminateInstances
          Effect: Allow
          Resource:
            - '*'
      Version: 2012-10-17
    Roles:
      - Ref: KIBUserRole
  Type: AWS::IAM::ManagedPolicy
  Version: 2012-10-17
KIBUserRole:
  Properties:
    AssumeRolePolicyDocument:
      Statement:
        - Action:
            - sts:AssumeRole
          Effect: Allow
          Principal:
            Service:
              - ec2.amazonaws.com
        - Action:
            - sts:AssumeRole
          Effect: Allow
          Principal:
            AWS: arn:aws:iam::MYAWSACCOUNTID:root
      Version: 2012-10-17
    RoleName: kib-user-role
  Type: AWS::IAM::Role
```

2. Replace the following with the correct values.

   - `MYFILENAME.yaml` - give your file a meaningful name.
   - `MYSTACKNAME` - give your cloudformation stack a meaningful name.

3. Create the stack using the command `aws cloudformation create-stack --template-body=file://MYFILENAME.yaml -- stack-name=MYSTACKNAME --capabilities CAPABILITY_NAMED_IAM`..

Integrating your AWS Image with NKP CLI

This procedure describes creating a Cluster API compliant Amazon Machine Image (AMI).

About this task

A customized image requires the Konvoy Image Builder tool to be downloaded, and Use Override Files with Konvoy Image Builder on page 1118 to specify the base image and container images to use in your new AMI. To create a custom AMI and take advantage of enhanced cluster operations, see the Use KIB with AWS on page 1087 topics for more options.

> **Note:** In previous Nutanix Kubernetes Platform (NKP) releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder. A customized image requires the Konvoy Image Builder tool to be downloaded. You can use variable overrides to specify the base image and container images for your new custom AMI.

Before you begin

- Check the Supported Infrastructure Operating Systems.
- Check the supported Kubernetes Version for your provider in the Upgrade Compatibility Tables on page 1145.
- Create a working `Docker` or other Registry setup.
- Ensure you have met the minimal set of permissions from the AWS Image Builder Book. For more information, see AWS Image Builder Book https://image-builder.sigs.k8s.io/capi/providers/aws.html#required-permissions-to-build-the-aws-amis.
- For information on custom AMIs, see Create a Custom AMI on page 1091.

Building the Image: The steps and flags may vary based on the version of NKP you are running.

Procedure

1. Build and validate the image using the command `dkp-image-builder create image aws/centos-79.yaml`.

   By default, it builds in the `us-west-2` region. To specify another region, set the `--region` flag using the command `Run the nkp-image-builder create command to build and validate the image. nkp-image-builder create image aws/centos-79.yaml`

2. Once NKP provisions the image successfully, the `ami` id is printed and written to the `packer.pkr.hcl` file.

   Example output showing an `artifact_id` field whose value provides the name of the AMI ID.

   ```
   {
         "name": "rhel-8.6-fips",
         "builder_type": "amazon-ebs",
         "build_time": 1659486130,
         "files": null,
         "artifact_id": "us-west-2:ami-0f2ef742482e1b829",
         "packer_run_uuid": "0ca500d9-a5f0-815c-6f12-aceb4d46645b",
         "custom_data": {
           "containerd_version": "",
   ```

```
      "distribution": "RHEL",
      "distribution_version": "8.6",
      "kubernetes_cni_version": "",
      "kubernetes_version": "1.24.5+fips.0"
    }
  }
```

Create a Custom AMI

Build a custom AMI for use with NKP.

AMI images contain configuration information and software to create a specific, pre-configured operating environment. For example, you can create an AMI image of your computer system settings and software. The AMI image can then be replicated and distributed, creating your computer system for other users. You can use override files to customize components installed on your machine image. For example, you can tell KIB to install the FIPS versions of the Kubernetes components. For example, having the FIPS versions of the Kubernetes components installed by KIB.

This procedure describes using the Konvoy Image Builder (KIB) to create a Cluster API compliant Amazon Machine Image (AMI). KIB uses variable overrides to specify the base and container images for your new AMI.

> **Important:** In previous Nutanix Kubernetes Platform (NKP) releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder. Explore the Customize your Image topic for more options about overrides.

Prerequisites

Before you begin, you must:

- Download the KIB bundle for your version of NKP prefixed with `konvoy-image-bundle` for your OS. To locate KIB download links, see the Compatible NKP to KIB Versions table in KIB.
- Check the Supported Infrastructure Operating Systems.
- Check the Supported Kubernetes Version for your Provider.
- Create a working registry:
  - Docker container engine version 27.4.0 installed for Linux or MacOS. For more information, see https://docs.docker.com/get-docker/.
  - Podman Version 4.0 or later for Linux. For more information, see https://podman.io/getting-started/installation. For host requirements, see https://kind.sigs.k8s.io/docs/user/rootless/#host-requirements.
- For information on Cluster APIs, see https://cluster-api.sigs.k8s.io/.
- Ensure you have met the minimal set of permissions from the AWS Image Builder Book. For more information, see https://image-builder.sigs.k8s.io/capi/providers/aws.html#required-permissions-to-build-the-aws-amis.
- A Minimal IAM Permissions for KIB to create an Image for an AWS account using Konvoy Image Builder. For more information, see Creating Minimal IAM Permissions for KIB on page 1087.

Extracting the KIB Bundle for an Air-gapped AMI
Extract the KIB bundle from the local directory.

Before you begin

If not done previously during Konvoy Image Builder download in Prerequisites, extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION` folder.

About this task

In previous Nutanix Kubernetes Platform (NKP) releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts, except for the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Containerd tarball

Procedure

1. Download `nkp-air-gapped-bundle_v2.8.0_linux_amd64.tar.gz`, and extract the tarball to a local directory using the command `tar -xzvf nkp-air-gapped-bundle_v2.8.0_linux_amd64.tar.gz && cd nkp-v2.8.0/kib`
2. Fetch the distro packages as well as other artifacts.

   Distro packages from distro repositories provide the latest security fixes available at machine image build time.
3. Build an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml` using the command `./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts`.

   - The bundles directory is located in your downloads location and contains all the steps to create an OS package bundle for a particular OS.
   - For FIPS, pass the flag: `--fips`.
   - For RHEL OS, pass your Red Hat subscription manager credentials: `export RMS_ACTIVATION_KEY`.

     Example:

     ```
     export RHSM_ACTIVATION_KEY="-ci" export RHSM_ORG_ID="1232131"
     ```
4. Build an AMI.

   The `konvoy-image` binary and all supporting folders are also extracted. When run, `konvoy-image` bind mounts the current working directory (`${PWD}`) into the container to be used.

   - Set environment variables for AWS access. The following variables must be set using your credentials, including Creating Minimal IAM Permissions for KIB on page 1087.

     Example Output:

     ```
     export AWS_ACCESS_KEY_ID
     export AWS_SECRET_ACCESS_KEY
     export AWS_DEFAULT_REGION
     ```
   - If you have an override file to configure specific attributes of your AMI file, add it. For more information on customizing an override file, see Image Overrides on page 1123.

AWS Air-gapped AMI

To create an image using Konvoy Image Builder (KIB) in an air-gapped cluster, follow these instructions. Amazon Machine Image (AMI) images contain configuration information and software to create a specific, pre-configured operating environment. For example, you can create an AMI image of your computer system settings and software. The AMI image can then be replicated and distributed, creating your computer system for other users. You can use override files to customize components installed on your machine image. For example, having the Federal Information Processing Standards (FIPS) versions of the Kubernetes components installed by KIB.

Prerequisites

- Minimal IAM Permissions for KIB to create an image for an Amazon Web Services (AWS) account using Konvoy Image Builder.

  Important: In previous Nutanix Kubernetes Platform (NKP) releases, AMI images provided by the upstream CAPA project were used if you did not specify an AMI. However, the upstream images are not recommended for production and may not always be available. Therefore, NKP requires you to specify an AMI when creating a cluster. To create an AMI, use Konvoy Image Builder (KIB).

Building the Image

Describes building the image using Konvoy Image Builder (KIB).

About this task

Depending on which version of NKP you are running, steps and flags may differ. To deploy in a region where CAPI images are not provided, you need to use KIB to create your image for the region. For a list of supported AWS regions, see the Published Amazon Machine Images (AMI)https://cluster-api-aws.sigs.k8s.io/topics/images/built-amis.html information from AWS.

Procedure

1. Build and validate the image using the command `konvoy-image build aws images/ami/rhel-86.yaml`.
2. Set the `--region` flag to specify a region other than the `us-west-2` default region.

   Example:

   ```
   konvoy-image build aws --region us-east-1 images/ami/rhel-86.yaml
   ```

   Note: Ensure you have named the correct AMI image YAML file for your OS in the `konvoy-image build` command. For more information, see https://github.com/mesosphere/konvoy-image-builder/tree/main/images/ami.
3. After KIB provisions the image successfully, locate the `artifact_id` field in the `packer.pkr.hcl` (Packer config) file.

   The `artifact_id` field value provides the name of the AMI ID. Use this `ami` value in the next step.

   Example output:

   ```
   {
     "builds": [
       {
         "name": "kib_image",
         "builder_type": "amazon-ebs",
         "build_time": 1698086886,
         "files": null,
         "artifact_id": "us-west-2:ami-04b8dfef8bd33a016",
         "packer_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05",
   ```

```
      "custom_data": {
        "containerd_version": "",
        "distribution": "RHEL",
        "distribution_version": "8.6",
        "kubernetes_cni_version": "",
        "kubernetes_version": "1.26.6"
      }
    }
  ],
  "last_run_uuid": "80f8296c-e975-d394-45f9-49ef2ccc6e05"
}
```


4. To use a custom AMI when creating your cluster, you must create that AMI using KIB first.
5. Perform the export using the command `export AWS_AMI_ID=ami-<ami-id-here>`.
6. Name the custom AMI using the command `NKP create cluster`.
   For more information, see Creating a New AWS Cluster on page 811.
7. Apply custom images for your environment:
   - » Non-air-gapped
   - » Air-gapped

Creating an AWS Air-gapped AMI
Create an image for an Amazon Web Services (AWS) account using KIB.

About this task

Using Konvoy Image Builder (KIB), you can build an AMI without requiring access to the internet by providing an additional `--override` flag.

Before you begin

In previous Nutanix Kubernetes Platform (NKP) releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts, except for the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Containerd tarball

Procedure

1. Download `NKP-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`.
2. Extract the tarball to a local directory using the command `tar -xzvf NKP-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd NKP-v2.12.0/kib`.
3. Fetch the distro packages from distro repositories; these include the latest security fixes available at machine image build time.

4. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the NKP `create-package-bundle` command. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`.

   Example:

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   Other supported air-gapped Operating Systems (OSs) can be specified in place of `--os redhat-8.4` using the flag and corresponding OS name:

   - `redhat-8.6`
   - `redhat-8.8`
   - `rocky-9.1`
   - `ubuntu-20.04`

   Note:

   - For Federal Information Processing Standards (FIPS), pass the flag: `--fips`.
   - For Red Hat Enterprise Linux (RHEL) OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`. Example command:

     ```
     export RHSM_ACTIVATION_KEY="-ci"
     export RHSM_ORG_ID="1232131"
     ```

     or

     ```
     export RHSM_USER=""
     export RHSM_PASS=""
     ```

   Continue to the next step, which is to build an AMI. Depending on which version of NKP you are running, steps and flags will be different. To deploy in a region where CAPI images are not provided, you need to use KIB to create your image for the region. For a list of supported AWS regions, See the Published AMI information from AWS at https://cluster-api-aws.sigs.k8s.io/topics/images/built-amis.html.

5. Run the `konvoy-image` command to build and validate the image. Ensure you have named the correct AMI image YAML file for your OS in the `konvoy-image build` command. For more information, see https://github.com/mesosphere/konvoy-image-builder/tree/main/images/ami.

   ```
   ./konvoy-image build aws images/ami/centos-79.yaml --overrides overrides/offline.yaml
   ```

   By default, it builds in the `us-west-2` region. To specify another region, set the `--region` flag as shown in the example.

   ```
   ./konvoy-image build aws --region us-east-1 images/ami/centos-79.yaml --
   overrides overrides/offline.yaml
   ```

   To customize an override file see, Image Overrides.

   After KIB provisions the image successfully, the `ami` id is printed and written to the `packer.pkr.hcl` (Packer config) file. This file has an `amazon-ebs.kib_image` field whose value provides the name of the AMI ID as shown in the example below. That is the `ami` you use in the `NKP create cluster` command.

   ```
   ...
       amazon-ebs.kib_image: Adding tag: "distribution_version": "8.6"
       amazon-ebs.kib_image: Adding tag: "gpu_nvidia_version": ""
       amazon-ebs.kib_image: Adding tag: "kubernetes_cni_version": ""
       amazon-ebs.kib_image: Adding tag: "build_timestamp": "20231023182049"
       amazon-ebs.kib_image: Adding tag: "gpu_types": ""
       amazon-ebs.kib_image: Adding tag: "kubernetes_version": "1.29.6"
   ```

```
==> amazon-ebs.kib_image: Creating snapshot tags
    amazon-ebs.kib_image: Adding tag: "ami_name": "konvoy-ami-
rhel-8.6-1.26.6-20231023182049"
==> amazon-ebs.kib_image: Terminating the source AWS instance...
==> amazon-ebs.kib_image: Cleaning up any extra volumes...
==> amazon-ebs.kib_image: No volumes to clean up, skipping
==> amazon-ebs.kib_image: Deleting temporary security group...
==> amazon-ebs.kib_image: Deleting temporary keypair...
==> amazon-ebs.kib_image: Running post-processor:  (type manifest)
Build 'amazon-ebs.kib_image' finished after 26 minutes 52 seconds.

==> Wait completed after 26 minutes 52 seconds

==> Builds finished. The artifacts of successful builds are:
--> amazon-ebs.kib_image: AMIs were created:
us-west-2: ami-04b8dfef8bd33a016

--> amazon-ebs.kib_image: AMIs were created:
us-west-2: ami-04b8dfef8bd33a016
```

After you complete these steps, you can Air-gapped Seed the Registry.

What to do next

Explore the Customize your Image topic for more options.

> **Note:** If using AWS ECR as your local private registry, more information can be found on the Registry Mirror Tools. page.

Use Custom Amazon Machine Images to Build Cluster

Describes ways to use your custom Amazon Machine Images (AMI).

Launch a NKP Cluster with a Custom AMI

To use the built `ami` with Nutanix Kubernetes Platform (NKP), specify it with the `--ami` flag when calling cluster create. By default, `konvoy-image` will name the AMI so that `NKP` can discover the latest AMI for a base OS and Kubernetes version.

```
NKP create cluster aws --cluster-name=$(whoami)-aws-cluster --ami ami-0123456789
```

Or

, To launch a NKP Cluster with Custom AMI Lookup that will use the latest AMI, specify the `--ami-format`, `--ami-base-os` and `--ami-owner` flags, use this command instead.

```
NKP create cluster aws --cluster-name=$(whoami)-aws-cluster --ami-format "konvoy-ami-
{{.BaseOS}}-?{{.K8sVersion}}-*" --ami-base-os ubuntu-2004 --ami-owner 123456789012
```

Using Custom Source AMIs

When using Konvoy Image Builder (KIB) for building machine images for Amazon, the default source AMIs we provide are modeled by looking up an AMI based on the owner. Then, we apply a filter for that operating system and version. Sometimes, a particular upstream AMI may not be available in your region or renamed, so you create a custom source AMI.

You can view an example of that with the provided `ubuntu-2004.yaml` snippet.

```
download_images: true

packer:
  ami_filter_name: "ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server*"
  ami_filter_owners: "099720109477"
  source_ami: ""
  distribution: "Ubuntu"
```

```
  distribution_version: "20.04"
  ssh_username: "ubuntu"
  root_device_name: "/dev/sda1"
  volume_size: "15"
...
```

Other times, you want to provide a custom AMI. If this is the case, you will want to edit or create your own YAML file that looks up based on the `source_ami` field. For example, you can select images that are otherwise deprecated.

Once you select the source AMI you want, you can declare it when running your build command.

```
konvoy-image build aws path/to/ami/ubuntu-2004.yaml --source-ami ami-0123456789
```

Alternatively, you can add it to your YAML file or make your own file. You add that AMI ID into the `source_ami` in the YAML file.

```
download_images: true

packer:
  ami_filter_name: "ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server*"
  ami_filter_owners: "099720109477"
  source_ami: ""
  distribution: "Ubuntu"
  distribution_version: "20.04"
  ssh_username: "ubuntu"
  root_device_name: "/dev/sda1"
  volume_size: "15"
...
```

When you're done selecting your `source_ami`, you can build your KIB image as usual.

```
konvoy-image build aws path/to/ami/ubuntu-2004.yaml
```

KIB for EKS

AWS EKS best practices discourage building custom images. The Amazon EKS Optimized AMI is the preferred way to deploy containers for EKS. If the image is customized, it breaks some of the autoscaling and security capabilities of EKS.

For more information, see https://docs.aws.amazon.com/eks/latest/userguide/eks-optimized-amis.html.

Using KIB with Azure

Learn how to build a custom Azure Image with NKP.

This procedure describes using the Konvoy Image Builder (KIB) to create a Cluster API compliant Azure Virtual Machine (VM) Image. The VM Image contains the base operating system you specify and all the necessary Kubernetes components. The Konvoy Image Builder uses variable overrides to specify the base and container images for your new Azure VM image. For more information, see https://cluster-api.sigs.k8s.io/.

> **Note:** The default Azure image is not recommended for use in production. We suggest using KIB for Azure to build the image to take advantage of enhanced cluster operations. For more options, see the Customize your Image topic.
>
> For more information regarding using the image in creating clusters, see Creating a New Azure Cluster on page 892.

Prerequisites

Before you begin, you must:

- Download the Konvoy Image Builder bundle for your version of Nutanix Kubernetes Platform (NKP).
- Check the Supported Infrastructure Operating Systems.
- Check the update link Supported Kubernetes Version for your provider.
- Create a working `Docker` setup.

Extract the KIB Bundle

Extract the bundle and `cd` into the extracted `konvoy-image-bundle-$VERSION_$OS` folder. The bundled version of `konvoy-image` contains an embedded `docker` image that contains all the requirements for building the image.

The `konvoy-image` binary and all supporting folders are also extracted. When extracted, `konvoy-image` bind mounts the current working directory (`${PWD}`) into the container to be used.

Configure Azure Prerequisites

If you have already followed the Azure Prerequisites on page 884, then the environment variables needed by KIB (`[AZURE_CLIENT_SECRET`, `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID]`) are set and do not need repeated if you are still working in the same window.

Complete the following task if you have not executed the Azure Prerequisite steps.

1. Sign in to Azure using the command `az login`.

```
Example:
 [
  {
    "cloudName": "AzureCloud",
    "homeTenantId": "a1234567-b132-1234-1a11-1234a5678b90",
    "id": "b1234567-abcd-11a1-a0a0-1234a5678b90",
    "isDefault": true,
    "managedByTenants": [],
    "name": "Mesosphere Developer Subscription",
    "state": "Enabled",
    "tenantId": "a1234567-b132-1234-1a11-1234a5678b90",
    "user": {
      "name": "user@azuremesosphere.onmicrosoft.com",
      "type": "user"
    }
  }
]
```

2. Create an Azure Service Principal (SP) by using the command `az ad sp create-for-rbac --role contributor --name "$(whoami)-konvoy" --scopes =/subscriptions/$(az account show --query id -o tsv) --query "{ client_id: appId, client_secret: password, tenant_id: tenant }"`.

   Note: The command will rotate the password if an SP with the name exists.

```
Example:
{
  "client_id": "7654321a-1a23-567b-b789-0987b6543a21",
  "client_secret": "Z79yVstq_E.R0R7RUUck718vEHSuyhAB0C",
  "tenant_id": "a1234567-b132-1234-1a11-1234a5678b90"
}
```

3. Set the `AZURE_CLIENT_SECRET` environment variable.

```
Example:
export AZURE_CLIENT_SECRET="<azure_client_secret>" #
 Z79yVstq_E.R0R7RUUck718vEHSuyhAB0C
```

```
export AZURE_CLIENT_ID="<client_id>"                # 7654321a-1a23-567b-
b789-0987b6543a21
export AZURE_TENANT_ID="<tenant_id>"                # a1234567-
b132-1234-1a11-1234a5678b90
export AZURE_SUBSCRIPTION_ID="<subscription_id>"    # b1234567-abcd-11a1-
a0a0-1234a5678b90
```

4. Ensure you have an override file to configure specific attributes of your Azure image.

   Note: Information on related topics or procedures:

   - Using Override Files on page 1123
   - Use Override Files with Konvoy Image Builder on page 1118
   - To ensure you have named the correct file for your OS in the `konvoy-image build` command, see https://github.com/mesosphere/konvoy-image-builder/tree/main/images/azure.

Building the Image

To build and validate the image, use the `konvoy-image build azure --client-id $AZURE_CLIENT_ID --tenant-id ${AZURE_TENANT_ID} --overrides override-source-image.yaml images/azure/ubuntu-2004.yaml` command.

By default, the image builder builds in the `westus2` location. To specify another location, set the `--location` flag (shown in the example below, which shows how to change the location to `eastus`).

```
Example:
konvoy-image build azure --client-id $AZURE_CLIENT_ID --tenant-id ${AZURE_TENANT_ID}
--location eastus --overrides override-source-image.yaml images/azure/ubuntu-2004.yaml
```

When the command is complete, the image ID is printed and written to the `./packer.pkr.hcl` file. This file has an `artifact_id` field whose value provides the name of the image. Specify this image ID when creating the cluster.

Image Gallery

By default, Konvoy Image Builder will create a Resource Group, Gallery, and Image Name to store the resulting image. To specify a specific Resource Group, Gallery, or Image Name flags may be specified:

```
Example:
--gallery-image-locations string     a list of locations to publish the image (default
 same as location)
--gallery-image-name string          the gallery image name to publish the image to
--gallery-image-offer string         the gallery image offer to set (default "nkp")
--gallery-image-publisher string     the gallery image publisher to set (default "nkp")
--gallery-image-sku string           the gallery image sku to set
--gallery-name string                the gallery name to publish the image in (default
 "nkp")
--resource-group string              the resource group to create the image in (default
 "nkp")
```

When creating your cluster, add the `--compute-gallery-id "<Managed Image Shared Image Gallery Id>"` flag while creating your custom image. See the update link once the topic is created Creating a New Azure Cluster on page 892 for specific consumption of image commands.

The SKU and Image Name will default to the values in the YAML image.

Ensure you have named the correct YAML file for your OS in the `konvoy-image build` command. For more information, see https://github.com/mesosphere/konvoy-image-builder/tree/main/images/gcp.

Marketplace Images for Rocky Linux

Similar to Image Gallery, additional flags allow NKP to create a cluster with Marketplace-based images for Rocky Linux 9.0: `--plan-offer`, `--plan-publisher` and `--plan-sku`.

- If you use these fields in the override file when you create a machine image with KIB, you must also set the corresponding flags when you create your cluster with NKP.
- Conversely, if you do not use these fields when you create a machine image with KIB, you do not need to set these flags when you create your cluster with NKP.

Example output:

```
---
download_images: true

packer:
  distribution: "rockylinux-9"                # Offer
  distribution_version: "rockylinux-9"  # SKU
  # Azure Rocky linux official image: https://portal.azure.com/#view/
Microsoft_Azure_Marketplace/
GalleryItemDetailsBladeNopdl/id/
erockyenterprisesoftwarefoundationinc1653071250513.rockylinux-9
  image_publisher: "erockyenterprisesoftwarefoundationinc1653071250513"
  image_version: "latest"
  ssh_username: "azureuser"
  plan_image_sku: "rockylinux-9" # SKU
  plan_image_offer: "rockylinux-9" # offer
  plan_image_publisher: "erockyenterprisesoftwarefoundationinc1653071250513" #
 publisher

build_name: "rocky-90-az"
packer_builder_type: "azure"
python_path: ""
```

KIB for AKS

Because custom virtual machine images are discouraged in Azure Kubernetes Service (AKS), Konvoy Image Builder (KIB) does not include any support for building custom machine images for AKS. If the image is customized, it breaks some of AKS' autoscaling and security capabilities. The AKS managed image ensures everything is patched and tuned for the container workload.

Using KIB with GCP

Describes how to use the (KIB) to create a compliant GCP image

This procedure describes how to use the Konvoy Image Builder (KIB) to create a Cluster API compliant GCP image. GCP images contain configuration information and software to create a specific, pre-configured operating environment. For example, you can create a GCP image of your computer system settings and software. The GCP image can then be replicated and distributed, creating your computer system for other users. KIB uses variable overrides to specify the base and container images of your new GCP image.

> **Note:** Google Cloud Platform does not publish images. You must first build the image using Konvoy Image Builder. Explore the Customize your Image topic for more options. For more information regarding using the image in creating clusters, refer to the GCP Infrastructure section of the documentation.

Nutanix Kubernetes Platform (NKP) Prerequisites

Before you begin, you must:

- Download the KIB bundle for your version of NKP.
- Check the Supported Infrastructure Operating Systems
- Check the Supported Kubernetes Version for your Provider.
- Create a working Docker or other registry setup.
- On Debian-based Linux distributions, install a version of the cri-tools package compatible with Kubernetes and container runtime versions.
- Verify that your Google Cloud project does not have the Enable OS Login feature enabled. For more information, see https://github.com/kubernetes-sigs/cri-tools.

  Note: The Enable OS Login feature is sometimes enabled by default in GCP projects. If the OS login feature is enabled, KIB will be unable to `ssh` to the VM instances it creates and cannot successfully create an image.

  Inspect the metadata configured in your project to check if it is enabled. If you find the `enable-oslogin` flag set to TRUE, you must remove it (or set it to FALSE) to use KIB successfully. For more information, see https://cloud.google.com/compute/docs/metadata/setting-custom-metadata#console_2

GCP Prerequisite Roles

If you are creating your image on either a non-GCP instance or one that does not have the required roles, you must either:

- Create a GCP service account. For more information, see GCP service account.
- If you have already created a service account, retrieve the credentials for an existing service account.
- Export the static credentials that will be used to create the cluster using the command `export GCP_B64ENCODED_CREDENTIALS=$(base64 < "$GOOGLE_APPLICATION_CREDENTIALS" | tr -d '\n')`

  Tip: Make sure to rotate static credentials for increased security.

Building the GCP Image

Use the Konvoy Image Builder (KIB) to create a Cluster API compliant GCP image.

About this task

Use the Konvoy Image Builder (KIB) to create a Cluster API compliant GCP image.

Procedure

1. Run the `konvoy-image` command to build and validate the image.

   ```
   ./konvoy-image build gcp --project-id $GCP_PROJECT --network $NETWORK_NAME images
   /gcp/ubuntu-2004.yaml
   ```

2. KIB will run and print out the name of the created image, use this name when creating a Kubernetes cluster. See the sample output below.

   ```
   ...
   ==> ubuntu-2004-focal-v20220419: Deleting instance...
       ubuntu-2004-focal-v20220419: Instance has been deleted!
   ==> ubuntu-2004-focal-v20220419: Creating image...
   ==> ubuntu-2004-focal-v20220419: Deleting disk...
       ubuntu-2004-focal-v20220419: Disk has been deleted!
   ```

```
==> ubuntu-2004-focal-v20220419: Running post-processor: manifest
Build 'ubuntu-2004-focal-v20220419' finished after 7 minutes 46 seconds.
==> Wait completed after 7 minutes 46 seconds
==> Builds finished. The artifacts of successful builds are:
--> ubuntu-2004-focal-v20220419: A disk image was created: konvoy-
ubuntu-2004-1-23-7-1658523168
--> ubuntu-2004-focal-v20220419: A disk image was created: konvoy-
ubuntu-2004-1-23-7-1658523168
```

> **Note:** Ensure you have named the correct YAML file for your OS in the `konvoy-image build` command. For more information, see https://github.com/mesosphere/konvoy-image-builder/tree/main/images/gcp

3. To find a list of images you have created in your account, use the command `gcloud compute images list --no-standard-images`

What to do next

Refer to Nutanix Kubernetes Platform (NKP) Documentation regarding roles and minimum permission for use with Konvoy Image Builder: GCP Roles.

Creating a Network (Optional)

Creating a network is an optional task.

About this task

Building an image requires a Network with firewall rules that allow SSH access to the VM instance.

Procedure

1. Set your GCP Project ID for your `gcp` account using the command `export GCP_PROJECT=`*`your GCP project ID`*`.`
2. Create a new network using the command `export NETWORK_NAME=kib-ssh-network gcloud compute networks create "$`*`NETWORK_NAME`*`" --project="$`*`GCP_PROJECT`*`" --subnet-mode=auto --mtu=1460 --bgp-routing-mode=regional`
3. Create the firewall rule to allow Ingress access on port 22 using the command `gcloud compute firewall-rules create "$`*`NETWORK_NAME`*`-allow-ssh" --project="$`*`GCP_PROJECT`*`" --network="projects/$`*`GCP_PROJECT`*`/global/networks/$`*`NETWORK_NAME`*`" --description="Allows TCP connections from any source to any instance on the network using port 22." --direction=INGRESS --priority=65534 --source-ranges=0.0.0.0/0 --action=ALLOW --rules=tcp:22`
   With your KIB image now created, you can now move on to create your cluster and set up your Cluster API (CAPI) controllers.

What to do next

For more information on related topics, see

- Network: https://cloud.google.com/vpc/docs/vpc
- Cluster API:https://cluster-api.sigs.k8s.io/

Using KIB with GPU

Create a GPU-supported OS image using Konvoy Image Builder

About this task

Using the Konvoy Image Builder, you can build an image that supports the three of NVIDIA GPU hardware to support GPU workloads.

> **Note:** The NVIDIA driver requires a specific Linux kernel version. Ensure the base image for the OS version has the required kernel version.
>
> See Supported Infrastructure Operating Systems for a list of OS versions and the corresponding kernel versions known to work with the NVIDIA driver.

If the NVIDIA runfile installer has not been downloaded, retrieve and install the download first by running the following command. The first line in the command below downloads and installs the runfile, and the second line places it in the artifacts directory (you must create an `artifacts` directory if you don't already have one). For more information, see https://docs.nvidia.com/datacenter/tesla/tesla-installation-notes/index.html#runfile.

```
curl -O https://download.nvidia.com/XFree86/Linux-x86_64/535.183.06/NVIDIA-Linux-x86_64535.183.06.run
```

```
mv NVIDIA-Linux-x86_64-535.183.06.run artifacts
```

> **Note:** The Nutanix Kubernetes Platform (NKP) supported NVIDIA driver version is 470.x.

About this task

To build an image for GPU-enabled hardware, perform the following steps.

Procedure

1. add the following to enable GPU builds in your override file. Otherwise, you can access and use the overrides in our repo, the documentation under Nvidia GPU Override File or Offline Nvidia Override file. For more information, see https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides and https://www.nvidia.com/Download/Find.aspx

   » Non-air-gapped GPU override `overrides/nvidia.yaml`:

   ```
   gpu:
     types:
       - nvidia
   build_name_extra: "-nvidia"
   ```

   » Air-gapped GPU override `overrides/offline-nvidia.yaml`:

   ```
   # Use this file when building a machine image, not as an override secret for
   preprovisioned environments
   nvidia_runfile_local_file: "{{ playbook_dir}}/../artifacts/
   {{ nvidia_runfile_installer }}"
   gpu:
     types:
       - nvidia
   build_name_extra: "-nvidia"
   ```

   > **Note:** For RHEL Pre-provisioned Override Files used with KIB, see specific note for GPU.

2. Build your image using the Konvoy Image Builder command, including the flag --instance-type that specifies an AWS instance with an available GPU.

   AWS Example:

   ```
   konvoy-image build --region us-east-1 --instance-type=p2.xlarge --source-
   ami=ami-12345
   abcdef images/ami/ubuntu-2004.yaml --overrides overrides/nvidia.yaml
   ```

   In this example, we chose an instance type with an NVIDIA GPU using the `--instance-type` flag, and we provided the NVIDIA overrides using the `--overrides` flag. See Using KIB with AWS for more information on creating an AMI.

3. When the Konvoy Image Builder image is complete, the custom `ami` id is printed and written to `./.`. To use the built `ami` with Konvoy, specify it with the `--ami` flag when calling cluster create.

What to do next

Additional helpful information can be found in the for Kubernetes instructions at: .

- NVIDIA documentation: https://nvidia.custhelp.com/app/answers/detail/a_id/131/kw/driver%20installation%20docs/related/1
- NVIDIA Device Plug-in: https://github.com/NVIDIA/k8s-device-plugin/blob/master/README.md
- Installation Guide of Supported Platforms: https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/install-guide.html

Verification

About this task

Perform the following task to verify that the NVIDIA driver is working,

Procedure

Connect to the node and execute the `nvidia-smi` command.

When drivers are successfully installed, the display looks like the following sample output:

```
Fri Jun 11 09:05:31 2021
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 535.183.06    Driver Version: 535.183.06    CUDA Version: 12.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla K80           Off  | 00000000:00:1E.0 Off |                    0 |
| N/A   35C    P0    73W / 149W |      0MiB / 11441MiB |     99%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
```

Using KIB with vSphere

vSphere for Nutanix Kubernetes Platform (NKP) using Konvoy Image Builder

This section assumes you have access to a vSphere environment and have credentials with proper privileges and access. For more information, see https://www.vmware.com/products/vcenter-server.html and https://docs.vmware.com/en/VMware-vSphere/index.html.

Create a vSphere Base OS Image

Creating a base OS image from DVD ISO files is a one-time process. Building an OS image creates a base vSphere template in your vSphere environment. The base OS image is used by Konvoy Image Builder (KIB) to create a VM template to configure Kubernetes nodes by the Nutanix Kubernetes Platform (NKP) vSphere provider. For more information about images in vSphere vCenter, see https://docs.vmware.com/en/VMware-Cloud-Foundation/4.5/vcf-deploy/GUID-2674DA5A-8DF7-4212-A4A9-88CD798DC303.html.

Create the Base OS Image

For vSphere, a username and password are populated by `SSH_USERNAME` , and the user can use authorization through `SSH_PASSWORD` or `SSH_PRIVATE_KEY_FILE` environment variables and required by default by packer. This user needs administrator privileges. Configuring a custom user and password when building the OS image is possible. However, that requires the Konvoy Image Builder (KIB) configuration to be overridden. Git location for packer folder, see https://github.com/mesosphere/konvoy-image-builder/blob/0523fd2c5e6e1ad1d4962f60a47039aa145a6e42/pkg/packer/manifests/vsphere/packer.pkr.hcl

While creating the base OS image, it is essential to take into consideration the following elements:

- **Storage configuration**: Nutanix recommends customizing disk partitions and not configuring a SWAP partition.
- **Network configuration**: as KIB must download and install packages, activating the network is required.
- **Connect to Red Hat**: If using Red Hat Enterprise Linux (RHEL), registering with Red Hat is required to configure software repositories and install software packages.
- **Software selection**: Nutanix recommends choosing **Minimal Install**.
- NKP recommends installing with the packages provided by the operating system package managers. Use the version that corresponds to the primary version of your operating system.

Disk Size

For each cluster you create using this base OS image, ensure you establish the disk size of the **root file system** based on the following:

- The minimum NKP Resource Requirements.
- The minimum storage requirements for your organization.

Defaults

- Clusters are created with a default disk size of 80 GB.

  Important: The base OS image root file system must be 80 GB for clusters created with the default disk size. The root file system cannot be reduced automatically when a machine first boots.

Customization

You can also specify a custom disk size when you create a cluster add when the topic is made (see the flags available for use with the Creating a New vSphere Cluster on page 926 command). This allows you to use one base OS image to create multiple clusters with different storage requirements.

Before specifying a disk size when you create a cluster, take into account:

1. **For some base OS images, the custom disk size option does not affect the size of the root file system.** This is because some root file systems, for example, those contained in an LVM Logical Volume, cannot be resized automatically when a machine first boots.
2. **The specified custom disk size must be equal to, or larger than, the size of the base OS image root file system.** This is because a root file system cannot be reduced automatically when a machine first boots.
3. In VMware Cloud Director Infrastructure on page 963, the base image determines the minimum storage available for the VM.

For additional information and examples of KIB vSphere overrides, see this page: Image Overrides

If using Flatcar, the documentation from Flatcar regarding disabling or enabling autologin in the Base OS Image is found here: In a vSphere or Pre-provisioned environment, anyone with access to the console of a Virtual Machine (VM) has access to the core operating system user. This is called autologin. To disable autologin, you add parameters to your base Flatcar image. For more information on how to use Flatcar, see:

- Running Flatcar Container Linux on VMware: https://www.flatcar.org/docs/latest/installing/cloud/vmware/#disablingenabling-autologin.
- Kernel modules and other settings:https://www.flatcar.org/docs/latest/setup/customization/other-settings/#adding-custom-kernel-boot-options.
- vCenter:https://www.vmware.com/products/vcenter-server.html.
- vSphere Client:https://docs.vmware.com/en/VMware-vSphere/index.html

Create a vSphere Virtual Machine Template

Create a vSphere template for your cluster from a base OS image

You must have at least one image before creating a new cluster. As long as you have an image, this step in your configuration is not required each time since that image can be used to spin up a new cluster. However, if you need different images for different environments or providers, you must create a new custom image.

The Konvoy Image Builder (KIB) uses the values in `image.yaml` and the input base OS image to create a vSphere template directly on the vCenter server. Using KIB, you can build an image without requiring access to the internet by providing an additional offline `--override` flag. You can use these override files to customize some components installed on your machine image. For example, you can tell KIB to install the FIPS versions of the Kubernetes components.

Prerequisites

- Users must create a base OS image in their vSphere client before starting this procedure.
- Konvoy Image Builder (KIB) downloaded and extracted.

Creating an Air-gapped vSphere Virtual Machine Template

About this task

In previous Nutanix Kubernetes Platform (NKP) releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts, except for the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Containerd tarball

Procedure

1. Download `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`.
2. Extract the tarball to a local directory:

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

3. Fetch the distro packages as well as other artifacts. You get the latest security fixes available at machine image build time by fetching the distro packages from distro repositories.
4. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To create it, run the NKP command `create-package-bundle`. This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`.

   For example:

   ```
   ./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
   ```

   Other supported air-gapped Operating Systems (OSs) can be specified in place of `--os redhat-8.4` using the flag and corresponding OS name:

   - `redhat-8.6`
   - `redhat-8.8`
   - `rocky-9.1`
   - `ubuntu-20.04`

   Note:

   - For FIPS, pass the flag: `--fips`.
   - For Red Hat Enterprise Linux (RHEL) OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`. As shown in the example.

     ```
     export RHSM_ACTIVATION_KEY="-ci"
     export RHSM_ORG_ID="1232131"
     ```

5. Follow the instructions to build a vSphere template below and set the override `--overrides overrides/offline.yaml` flag.

Creating a vSphere Template for Your Cluster from a Base OS Image
This task is to create a vSphere template for your cluster from a base OS image.

About this task

Using the base OS image created in a previous procedure, Nutanix Kubernetes Platform (NKP) creates the new vSphere template directly on the vCenter server.

Procedure

1. Set the following vSphere environment variables on the bastion VM host.

   ```
   export VSPHERE_SERVER=your_vCenter_APIserver_URL
   export VSPHERE_USERNAME=your_vCenter_user_name
   export VSPHERE_PASSWORD=your_vCenter_password
   ```

2. Copy the base OS image file created in the vSphere Client to your desired location on the bastion VM host and make a note of the path and file name.

3. Create an `image.yaml` file and add the following variables for vSphere. NKP uses this file and these variables as inputs in the next step. To customize your `image.yaml` file, see Customize your Image on page 1112.

   Important: Use the YAML file that matches your OS name. The following example is for Ubuntu 20.04. See the Additional YAML File Examples on page 1109 for alternate copy and paste options for the last step.

```
---
download_images: true
build_name: "ubuntu-2004"
packer_builder_type: "vsphere"
guestinfo_datasource_slug: "https://raw.githubusercontent.com/vmware/cloud-init-
vmware-guestinfo"
guestinfo_datasource_ref: "v1.4.0"
guestinfo_datasource_script: "{{guestinfo_datasource_slug}}/
{{guestinfo_datasource_ref}}/install.sh"
packer:
  cluster: "<VSPHERE_CLUSTER_NAME>"
  datacenter: "<VSPHERE_DATACENTER_NAME>"
  datastore: "<VSPHERE_DATASTORE_NAME>"
  folder: "<VSPHERE_FOLDER>"
  insecure_connection: "false"
  network: "<VSPHERE_NETWORK>"
  resource_pool: "<VSPHERE_RESOURCE_POOL>"
  template: "os-qualification-templates/nutanix-base-Ubuntu-20.04" # change default
 value with your base template name
  vsphere_guest_os_type: "other4xLinux64Guest"
  guest_os_type: "ubuntu2004-64"
  # goss params
  distribution: "ubuntu"
  distribution_version: "the 20.04"
# Use following overrides to select the authentication method that can be used with
 the base template
# ssh_username: ""  # can be exported as environment variable 'SSH_USERNAME'
# ssh_password: "" # can be exported as environment variable 'SSH_PASSWORD'
# ssh_private_key_file = "" # can be exported as environment variable
 'SSH_PRIVATE_KEY_FILE'
# ssh_agent_auth: false  # is set to true, ssh_password and ssh_private_key will be
 ignored
```

4. Create a vSphere VM template with your variation of the following command.

```
konvoy-image build images/ova/<image.yaml>
```

   Any additional configurations can be added to this command using `--overrides` flags as shown below:

   - Any credential overrides: `--overrides overrides.yaml`
   - for FIPS, add this flag: `--overrides overrides/fips.yaml`
   - for air-gapped, add this flag: `--overrides overrides/offline-fips.yaml`

5. The Konvoy Image Builder (KIB) uses the values in image.yaml and the input base OS image to create a vSphere template directly on the vCenter server. This template contains the required artifacts needed to create a Kubernetes cluster.

   When KIB successfully provisions the OS image, it creates a manifest file. The `artifact_id` field of this file contains the name of the AMI ID (AWS), template name (vSphere), or image name (GCP/Azure).

Github image folder location, see https://github.com/mesosphere/konvoy-image-builder/tree/main/images/ova

Example:

```
{
      "name": "vsphere-clone",
      "builder_type": "vsphere-clone",
      "build_time": 1644985039,
      "files": null,
      "artifact_id": "konvoy-ova-vsphere-rhel-86-1.21.6-1644983717",
      "packer_run_uuid": "260e8110-77f8-ca94-e29e-ac7a2ae779c8",
      "custom_data": {
        "build_date": "2022-02-16T03:55:17Z",
        "build_name": "vsphere-rhel-86",
        "build_timestamp": "1644983717",
        [...]
      }
    }
```

Tip: Recommendation: Now we can now see the template created in our vCenter, it is best to rename it to `nkp-<NKP_VERSION>-k8s-<K8S_VERSION>-<DISTRO>`, like `nkp-2.4.0-k8s-1.24.6-ubuntu` to keep templates organized.

6. Deploy a NKP cluster using your vSphere template.

Additional YAML File Examples

Copy and paste the YAML files that match your OS or select one from GitHub.

RHEL 8.6

```
---
download_images: true
build_name: "rhel-86"
packer_builder_type: "vsphere"
guestinfo_datasource_slug: "https://raw.githubusercontent.com/vmware/cloud-init-vmware-
guestinfo"
guestinfo_datasource_ref: "v1.4.0"
guestinfo_datasource_script: "{{guestinfo_datasource_slug}}/
{{guestinfo_datasource_ref}}/install.sh"
packer:
  cluster: ""
  datacenter: ""
  datastore: ""
  folder: ""
  insecure_connection: "false"
  network: ""
  resource_pool: ""
  template: "os-qualification-templates/nutanix-base-RHEL-86" # change default value
 with your base template name
  vsphere_guest_os_type: "rhel8_64Guest"
  guest_os_type: "rhel8-64"
  # goss params
  distribution: "RHEL"
  distribution_version: "8.6"
# Use following overrides to select the authentication method that can be used with
 base template
# ssh_username: ""  # can be exported as environment variable 'SSH_USERNAME'
# ssh_password: "" # can be exported as environment variable 'SSH_PASSWORD'
# ssh_private_key_file = "" # can be exported as environment variable
 'SSH_PRIVATE_KEY_FILE'
```

```
# ssh_agent_auth: false  # is set to true, ssh_password and ssh_private_key will be
 ignored
```

Rocky Linux 9.1

```
---
download_images: true
build_name: "rocky-91"
packer_builder_type: "vsphere"
guestinfo_datasource_slug: "https://raw.githubusercontent.com/vmware/cloud-init-vmware-
guestinfo"
guestinfo_datasource_ref: "v1.4.0"
guestinfo_datasource_script: "{{guestinfo_datasource_slug}}/
{{guestinfo_datasource_ref}}/install.sh"
packer:
  cluster: ""
  datacenter: ""
  datastore: ""
  folder: ""
  insecure_connection: "false"
  network: ""
  resource_pool: ""
  template: "os-qualification-templates/nutanix-base-RockyLinux-9.1" # change default
 value with your base template name
  vsphere_guest_os_type: "other4xLinux64Guest"
  guest_os_type: "rocky9-64"
  # goss params
  distribution: "rocky"
  distribution_version: "9.1"
# Use following overrides to select the authentication method that can be used with
 base template
# ssh_username: ""  # can be exported as environment variable 'SSH_USERNAME'
# ssh_password: "" # can be exported as environment variable 'SSH_PASSWORD'
# ssh_private_key_file = "" # can be exported as environment variable
 'SSH_PRIVATE_KEY_FILE'
# ssh_agent_auth: false  # is set to true, ssh_password and ssh_private_key will be
 ignored
```

Flatcar

```
---
download_images: true
build_name: "flatcar-3033.3.16"
packer_builder_type: "vsphere"
guestinfo_datasource_slug: "https://raw.githubusercontent.com/vmware/cloud-init-vmware-
guestinfo"
guestinfo_datasource_ref: "v1.4.0"
guestinfo_datasource_script: "{{guestinfo_datasource_slug}}/
{{guestinfo_datasource_ref}}/install.sh"
packer:
  cluster: ""
  datacenter: ""
  datastore: ""
  folder: ""
  insecure_connection: "false"
  network: ""
  resource_pool: ""
  template: "nutanix-base-templates/nutanix-base-Flatcar-3033.3.16"
  vsphere_guest_os_type: "flatcar_64Guest"
  guest_os_type: "flatcar-64"
  # goss params
  distribution: "flatcar"
  distribution_version: "3033.3.16"
```

```
# Use following overrides to select the authentication method that can be used with
 base template
# ssh_username: ""  # can be exported as environment variable 'SSH_USERNAME'
# ssh_password: "" # can be exported as environment variable 'SSH_PASSWORD'
# ssh_private_key_file = "" # can be exported as environment variable
 'SSH_PRIVATE_KEY_FILE'
# ssh_agent_auth: true  # is set to true, ssh_password and ssh_private_key will be
 ignored
```

Using KIB with Pre-provisioned Environments

Using KIB running inside the bootstrap cluster against a set of nodes you define.

About this task

In Pre-provisioned environments, KIB runs automatically inside the `bootstrap cluster`, against the list of nodes you define. Meanwhile, with other providers, you manually run KIB outside the cluster to build your images.

For Pre-provisioned, you define a set of nodes that already exist. During the cluster creation process, KIB is built into Nutanix Kubernetes Platform (NKP) and automatically runs the machine configuration process (which KIB uses to build images for other providers) against the set of nodes you defined. This results in your pre-existing or pre-provisioned nodes being appropriately configured. The remainder of the cluster provisioning happens automatically after that.

- In a Pre-provisioned environment, you have existing machines, and NKP consumes them to form a cluster.
- When you have another provisioner (for example, cloud providers such AWS, vSphere , and others), you build images with KIB, and NKP consumes the images to provision machines and form a cluster.

Before NKP 2.6, you had to specify the HTTP proxy in the KIB override setup and then again in the `nkp create cluster` command. After NKP 2.6, an HTTP proxy gets created from the Konvoy flags for the control plane proxy and workers proxy values. The flags in the NKP command for Pre-provisioned clusters populate a Secret automatically in the bootstrap cluster. That Secret has a known name that the Pre-provisioned controller finds and applies when it runs the KIB provisioning job.

> **Note:** For a Pre-provisioned air-gapped environment, you must build the OS packages after fetching the packages from the distro repositories.

In previous NKP releases, the distro package bundles were included in the downloaded air-gapped bundle. Currently, that air-gapped bundle contains the following artifacts, except for the distro packages:

- NKP Kubernetes packages
- Python packages (provided by upstream)
- Containerd tarball

Procedure

1. Download`nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`, and extract the tarball to a local directory.

   ```
   tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz && cd nkp-v2.12.0/kib
   ```

2. You must fetch the distro packages and other artifacts. You get the latest security fixes available at machine image build time by fetching the distro packages from distro repositories.

3. In your download location, there is a bundles directory with all the steps to create an OS package bundle for a particular OS. To make it, run the new NKP command `create-package-bundle`.

This builds an OS bundle using the Kubernetes version defined in `ansible/group_vars/all/defaults.yaml`.

For Example:

```
./konvoy-image create-package-bundle --os redhat-8.4 --output-directory=artifacts
```

Note:

- For FIPS, pass the flag: `--fips`.
- For Red Hat Enterprise Linux (RHEL) OS, pass your RedHat subscription manager credentials: `export RMS_ACTIVATION_KEY`. As shown in the example.

```
export RHSM_ACTIVATION_KEY="-ci"
export RHSM_ORG_ID="1232131"
```

Customize your Image

There are several ways to customize an image. Some methods are simple, and some are more complex customizations. Each section in this portion of the documentation will explain these options and give directions.

Customize your Image YAML or Manifest File

Nutanix Kubernetes Platform (NKP) provides default YAML files for each Infrastructure Provider and their supported Operating Systems. Those default YAML files can easily be modified by replacing values. The simplest way is to open your OS in an editor and change a line. For example, in the following YAML, you can name a specific image rather than accepting the default `source_ami: ""`.

AWS Example: `Example images/ami/rhel-86.yaml`

```
---
# Example images/ami/rhel-86.yaml
download_images: true

packer:
  ami_filter_name: "RHEL-8.6.0_HVM-*"
  ami_filter_owners: "309956199498"
  distribution: "RHEL"
  distribution_version: "8.6"
  source_ami: ""
  ssh_username: "ec2-user"
  root_device_name: "/dev/sda1"
  volume_size: "15"

build_name: "rhel-8.6"
packer_builder_type: "amazon"
python_path: ""
```

Once the YAML for the image is edited, you create your image using the customized YAML by applying it in the command.`konvoy-image build aws images/ami/rhel-86.yaml`.

For more information about the default YAML files, see https://github.com/mesosphere/konvoy-image-builder/tree/7447074a6d910e71ad2e61fc4a12820d073ae5ae/images

Generating a Packer File to Customize

Editing your Packer template is an alternate method for customizing the YAML or Manifest file.

About this task

Another customization technique is editing your Packer template. When first running the Konvoy Image Builder (KIB) command to generate an image, it automatically creates the files you need to edit in a work folder path.

First, you will build an image and then edit the generated output to customize another image. During the attempt to build the image, a `packer.pkr.hcl` is generated under the path `work/centos-7-xxxxxxx-yyyyy/packer.pkr.hcl`.

Procedure

1. Run the image creation command.

   ```
   konvoy-image build aws images/ami/ubuntu-2004.yaml
   ```

   > **Note:** To avoid fully creating the image and reduce image charges, interrupt the command right after running it. This will generate the needed files but avoid a full image creation charge.

   a. Overrides for FIPS: You can also use FIPS overrides during initial image creation.

   ```
   konvoy-image build aws images/ami/ubuntu-2004.yaml --overrides overrides/fips.yaml
   ```

2. A work file is generated (for example: `work/ubuntu-20-3486702485-iulvY`), which contains the following files to be opened in an editor, and the values are added or changed.

   - `ansible_vars.yaml`
   - `packer_vars.json`
   - `packer.pkr.hcl`

3. Open the `packer.pkr.hcl` file with an editor and change the desired values.

```
packer {
 required_plugins {
  amazon = {
     version = ">= 1.1.3"
     source  = "github.com/hashicorp/amazon"
  }
  ansible = {
     version = ">= 1.1.0"
     source  = "github.com/hashicorp/ansible"
  }
  goss = {
    version = ">=3.1.5"
    source  = "github.com/supershal/goss"
  }
 }
}
variable "ami_groups" {
  type    = string
  default = ""
}
variable "ami_regions" {
  type    = string
  default = "us-west-2"
}
variable "ami_users" {
  type    = string
  default = ""
}
```

```
variable "ansible_extra_vars" {
  type    = string
  default = ""
}
variable "aws_access_key" {
  type    = string
  default = ""
}
variable "aws_instance_type" {
  type    = string
  default = "t3.small"
}
variable "aws_profile" {
  type    = string
  default = ""
}
variable "aws_region" {
  type    = string
  default = "us-west-2"
}
```

4. Save and rename these changes as `manifest.pkr.kcl` so they can be applied during image creation using the --packer-manifest flag.

   a. For a complete list of KIB flags, run the command: `konvoy-image build aws --help`.

5. Once the YAML for the image is edited, you create another image using your customized YAML by applying it with the flag `--packer-manifest`. Provision the new image applying the recently edited and renamed `manifest.pkr.kcl`.

   AWS example:

```
konvoy-image build aws --packer-manifest manifest.pkr.hcl --overrides overrides/
fips.yaml
```

6. Review the final image to verify the correct variables.

Customize your Packer Configuration

Note:

Although several parameters are specified by default in the Packer templates for each provider, it is possible to **override** the default values.

For a complete list of the variables that can be modified for each Packer builder, see:

- AWS Packer template: https://github.com/mesosphere/konvoy-image-builder/blob/main/pkg/packer/manifests/aws/packer.pkr.hcl
- Azure Packer template: https://github.com/mesosphere/konvoy-image-builder/blob/main/pkg/packer/manifests/azure/packer.pkr.hcl
- GCP Packer template: https://github.com/mesosphere/konvoy-image-builder/blob/main/pkg/packer/manifests/gcp/packer.pkr.hcl
- vSphere Packer template: https://github.com/mesosphere/konvoy-image-builder/blob/main/pkg/packer/manifests/vsphere/packer.pkr.hcl

Using Flags

One option is to execute KIB with specific flags to override the following values:

- The source AMI (`--source-ami`)
- AMI region (`--ami-regions`)
- AWS EC2 instance type (`--aws-instance-type`)

For a comprehensive list of flags, run `./konvoy-image build --help`.

Using Override Files

Another option is to create a file with the parameters to be overridden and specify the `--overrides` flag as shown below:

```
./konvoy-image build images/<builder-type>/<os>-<version>.yaml --overrides
 overrides.yaml
```

> **Note:** While CLI flags can be combined with override files, CLI flags take priority over any override files.

Nutanix Kubernetes Platform (NKP) provides several default override files and explains how to create custom override files. Refer to whichever information you need:

- Default Override Files
- Custom Override Files

Override Credentials in Packer

To abide by security practices; you can set their username and password while creating the base OS image and override the default credentials in KIB.

AWS Example:

For example, when using the AWS Packer builder to override the credentials, set them under the `packer` key. This overrides the image search and forces the use of the specified credentials.

Create your override file `overrides-packer-credentials.yaml`:

```
---
packer:
  ssh_username: "USERNAME"
  ssh_password: "PASSWORD"
```

After creating the override file for your credentials, pass your override file by using the `--overrides` flag when building our image:

```
./konvoy-image build aws images/ami/ubuntu-2004.yaml --overrides override-packer-
credentials.yaml
```

vSphere Example:

A vSphere example shows the current base image description at `images/vsphere/rhel-86.yaml` , similar to the example.

```
---
download_images: true
build_name: "rhel-86"
packer_builder_type: "vsphere"
guestinfo_datasource_slug: "https://raw.githubusercontent.com/vmware/cloud-init-vmware-
guestinfo"
guestinfo_datasource_ref: "v1.4.0"
guestinfo_datasource_script: "{{guestinfo_datasource_slug}}/
{{guestinfo_datasource_ref}}/install.sh"
```

```
packer:
  cluster: "zone1"
  datacenter: "dc1"
  datastore: "esxi-06-disk1"
  folder: "cluster-api"
  insecure_connection: "false"
  network: "Airgapped"
  resource_pool: "Users"
  template: "base-rhel-8"
  vsphere_guest_os_type: "rhel8_64Guest"
  guest_os_type: "rhel8-64"
  #goss params
  distribution: "RHEL"
  distribution_version: "8.6"
```

For more information, see Supported Infrastructure Operating Systems on page 12.

Adding Custom Tags to your Image

Add a custom tag to the image by modifying the `packer.pkr.hcl` file.

About this task

Some cloud environments enforce the tagging of resources. Adding a custom tag as an *attribute* to the image is achieved by modifying the `packer.pkr.hcl` file.

To add a tag, complete the following task.

Procedure

1. Generate manifest files for KIB by executing the following command. The following Amazon Web Services (AWS) example shows the Ubuntu image being built:

   ```
   ./konvoy-image build images/ami/ubuntu-2004.yaml
   ```

   Note: Other provider commands are located in the Konvoy Image Builder.

2. Send SIGINT to the process to halt it after seeing the output `...writing new packer configuration to work/ubuntu-2004`. During the attempt to build the image, a `packer.pkr.hcl` is generated under the path `work/ubuntu-2004/packer.pkr.hcl`.

3. Edit the `packer.pkr.hcl` file by adding the parameter `"run_tags"` to the `packer.pkr.hcl` file as seen in the AWS example below:

   ```
    "builders": [
   {
   "name": "{{(user `distribution`) | lower}}-{{user `distribution_version`}}{{user
    `build_name
   _extra`}}",
   "type": "amazon-ebs",
   "run_tags": {"my_custom_tag": "tag"},
   "instance_type": "{{user `aws_instance_type`}}",
   "source_ami_filter": {
   "filters": {
   "virtualization-type": "hvm",
   "name": "{{user `ami_filter_name`}}",
   "root-device-type": "ebs",
   "architecture": "x86_64"
   },
   ```

4. Use the `--packer-manifest` flag to apply it when you build the image.

```
./konvoy-image build aws images/ami/centos-79.yaml --packer-manifest=work/
ubuntu-2004-
1658174984-TycMM/packer.pkr.hcl
2022/09/07 18:23:33 writing new packer configuration to work/ubuntu-2004-1662575013-
zJUhP
2022/09/07 18:23:33 starting packer build
ubuntu-20.04: output will be in this color.
==> ubuntu-20.04: Prevalidating any provided VPC information
==> ubuntu-20.04: Prevalidating AMI Name: konvoy-ami-ubuntu-20.04-1.29.6-1662575013
ubuntu-20.04: Found Image ID: ami-0686851c4e7b11234
==> ubuntu-20.04: Creating temporary keypair: packer_6318e1a6-9f45-c01b-c7ca-
f54047351234
==> ubuntu-20.04: Creating temporary security group for this instance:
 packer_6318e1aa-7448-
d74b-9b90-166f26d21234
==> ubuntu-20.04: Authorizing access to port 22 from [0.0.0.0/0] in the temporary
 security groups...
==> ubuntu-20.04: Launching a source AWS instance...
ubuntu-20.04: Adding tag: "my_custom_tag": "tag"
ubuntu-20.04: Instance ID: i-04d457b207123456
==> ubuntu-20.04: Waiting for instance (i-04d457b207123456) to become ready...
==> ubuntu-20.04: Using SSH communicator to connect: 34.222.123.123
==> ubuntu-20.04: Waiting for SSH to become available...
```

Now that your tags have been added to the image, you can continue to Custom Installation and Additional Infrastructure Tools. You will build the image with your edited manifest and finish building your clusters under your infrastructure.

What to do next

Using Override Files with Konvoy Image Builder

Ansible Variables

Ansible variables are generally preset when using KIB. To change the variables, you change the Ansible runbook or preprogrammed playbook.

Example `ansible_vars.yaml`

```
build_name: ubuntu-20
download_images: true
kubernetes_full_version: 1.29.6
kubernetes_version: 1.29.6
packer:
  ami_filter_name: ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server*
  ami_filter_owners: "099720109477"
  distribution: Ubuntu
  distribution_version: "20.04"
  dry_run: false
  goss_arch: amd64
  goss_entry_file: goss/goss.yaml
  goss_format: json
  goss_format_options: pretty
  goss_inspect_mode: false
  goss_tests_dir: goss
  goss_url: null
  goss_vars_file: ansible/group_vars/all/system.yaml
  goss_version: 0.3.16
  root_device_name: /dev/sda1
  source_ami: ""
  ssh_username: ubuntu
```

```
  volume_size: "15"
packer_builder_type: amazon
python_path: ""
```

Use Override Files with Konvoy Image Builder

Learn how to use override files with Konvoy Image Builder

You can use override files to customize some of the components installed on your machine image. For example, having the FIPS versions of the Kubernetes components installed by KIB components. You can also add `offline` overrides used for air-gapped environments. That will upload the necessary OS packages and other files to the image so that KIB doesn't try to look for those using the Internet. Or use an override file to provide similar configurations for a GPU-enabled cluster. To locate the KIB base override files in the Github repository, see https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides.

You can only change specific, predefined parameters using this mechanism. The parameters you can change depend on which of the supported infrastructure providers you use. For example, for vSphere, you use many settings to tell the KIB how to talk to your vCenter implementation, where to store the images, which credentials to use, and so on.

The Konvoy Image Builder uses `override` files to configure specific attributes. These files provide information to override default values for certain parts of your image file. These `override` files can modify the version and parameters of the image description and the Ansible playbook. For more information on KIB, see Konvoy Image Builder.

Default Override Files

Learn to use the default override files provided by Konvoy Image Builder.

This section discusses the different default override files the Konvoy Image Builder provides.

FIPS Override Non-air-gapped Files

FIPS override non-air-gapped file options.

There are two types of Non-air-gapped override files: Cloud Provisioners Override File Non-air-gapped and the FIPS Override Files to Non-air-gapped Pre-provisioned Environments.

Cloud Provisioners Override File Non-air-gapped

Add the following FIPS override file to your environment using the command `--overrides overrides/fips.yaml`.

```
Example:
---
k8s_image_registry: docker.io/mesosphere
fips:
  enabled: true
build_name_extra: -fips
kubernetes_build_metadata: fips.0
default_image_repo: hub.docker.io/mesosphere
kubernetes_rpm_repository_url: "https://packages.nutanix.com/konvoy/stable/linux/repos/
el/kubernetes-v{{ kubernetes_version }}-fips/x86_64"
docker_rpm_repository_url: "\
  https://containerd-fips.s3.us-east-2.amazonaws.com\
  /{{ ansible_distribution_major_version|int }}\
  /x86_64"
```

To locate the available Override files in the Konvoy Image Builder repo, see https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides.

Adding the FIPS Override File to Non-air-gapped Pre-provisioned Environments

About this task

Perform this task to add the FIPS override file to your pre-provisioned environment.

> **Note:** To locate the available Override files in the Konvoy Image Builder repo, see https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides.

Procedure

1. If your pre-provisioned machines need a default Override file like FIPS, create a secret that includes the overrides in a file.

   Example:

   ```
   cat > fips.yaml << EOF
   ---
   k8s_image_registry: docker.io/mesosphere
   fips:
     enabled: true
   build_name_extra: -fips
   kubernetes_build_metadata: fips.0
   default_image_repo: hub.docker.io/mesosphere
   kubernetes_rpm_repository_url: "https://packages.nutanix.com/konvoy/stable/linux/
   repos/el/kubernetes-v{{ kubernetes_version }}-fips/x86_64"
   docker_rpm_repository_url: "\
     https://containerd-fips.s3.us-east-2.amazonaws.com\
     /{{ ansible_distribution_major_version|int }}\
     /x86_64"
   EOF
   ```

2. Create the related secret by using the command `kubectl create secret generic $CLUSTER_NAME-user-overrides --from-file=fips.yaml=fips.yaml kubectl label secret $CLUSTER_NAME-user-overrides clusterctl.cluster.x-k8s.io/move=`.

FIPS Override Air-gapped Environment Files

Offline FIPS override file options:

Cloud Provisioners Override File (Air-gapped)

Add the following FIPS offline override file to your environment:

```
--overrides overrides/offline-fips.yaml
```

```
# fips os-packages
os_packages_local_bundle_file: "{{ playbook_dir }}/../
artifacts/{{ kubernetes_version }}_{{ ansible_distribution|
lower }}_{{ ansible_distribution_major_version }}_x86_64_fips.tar.gz"
containerd_local_bundle_file: "{{ playbook_dir }}/../artifacts/
{{ containerd_tar_file }}"
pip_packages_local_bundle_file: "{{ playbook_dir }}/../artifacts/pip-packages.tar.gz"
images_local_bundle_dir: "{{ playbook_dir}}/../artifacts/images"
```

> **Note:** All available Override files are in the Konvoy Image Builder repo, at https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides.

Adding the FIPS Override File to Pre-provisioned Air-gapped Environments

Perform this task to add the FIPS override file to your air-gapped pre-provisioned environment.

About this task

Add the following FIPS override file to your environment.

> **Note:** All available Override files are in the Konvoy Image Builder repo. For more information, see https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides.

Procedure

1. If your pre-provisioned machines need a default Override file like FIPS, create a secret that includes the overrides in a file.

```
cat > fips.yaml << EOF
# fips os-packages
os_packages_local_bundle_file: "{{ playbook_dir }}/../artifacts/
{{ kubernetes_version }}_{{ ansible
_distribution|lower }}_{{ ansible_distribution_major_version }}_x86_64_fips.tar.gz"
containerd_local_bundle_file: "{{ playbook_dir }}/../artifacts/
{{ containerd_tar_file }}"
pip_packages_local_bundle_file: "{{ playbook_dir }}/../artifacts/pip-packages.tar.gz"
images_local_bundle_dir: "{{ playbook_dir}}/../artifacts/images"
EOF
```

2. Create the related secret by running the following command.

```
kubectl create secret generic $CLUSTER_NAME-user-overrides --from-
file=fips.yaml=fips.yaml
kubectl label secret $CLUSTER_NAME-user-overrides clusterctl.cluster.x-k8s.io/move=
```

Related Information

For information on related topics or procedures, see Private Registry in Air-gapped Override.

Offline Override File

You can find these offline (air-gapped) override files in the Konvoy Image Builder repo at https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides.

```
--overrides overrides/offline.yaml
```

```
os_packages_local_bundle_file: "{{ playbook_dir }}/../artifacts/
{{ kubernetes_version }}_
{{ ansible_distribution|
lower }}_{{ ansible_distribution_major_version }}_x86_64.tar.gz"
containerd_local_bundle_file: "{{ playbook_dir }}/../artifacts/
{{ containerd_tar_file }}"
pip_packages_local_bundle_file: "{{ playbook_dir }}/../artifacts/pip-packages.tar.gz"
images_local_bundle_dir: "{{ playbook_dir}}/../artifacts/images"
```

> **Note:** For Ubuntu 20.04, when Konvoy Image Builder runs, it will temporarily disable all defined Debian repositories by appending a `.disabled` suffix. Each repository will revert to its original name at the end of installation. In case of failures, the files will not be renamed back.

For GPU Offline Override File, you can use `nvidia-runfile` flag for GPU support if you have downloaded the runfile installer at https://docs.nvidia.com/datacenter/tesla/tesla-installation-notes/index.html#runfile.

Related Information

For information on related topics or procedures, see:

- Offline Nvidia GPU Override file
- Private Registry in Air-gapped Override

Nvidia GPU Override Files

Nvidia GPU override file options:

Nvidia GPU override file options:

GPU Override File

This override file is used to create images with ##Nutanix Kubernetes Platform# (##NKP#), which uses GPU hardware. These override files can also be located in the Konvoy Image Builder repo at https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides.

```
--overrides overrides/offline-nvidia.yaml
```

```
# Use this file when building a machine image, not as a override secret for
 preprovisioned environments
nvidia_runfile_local_file: "{{ playbook_dir}}/../artifacts/
{{ nvidia_runfile_installer }}"
gpu:
  types:
    - nvidia
build_name_extra: "-nvidia"
```

Adding the GPU Override File to Air-gapped Pre-provisioned Environments

Perform this task to add the Nvidia GPU override file to your Air-gapped Pre-provisioned environment.

About this task

If you require your environment to consume GPU resources, add the following FIPS Overrides file to your environment:

Procedure

1. If your pre-provisioned machines need a default Override file like FIPS, create a secret that includes the overrides in a file.

   ```
   cat > nvidia.yaml << EOF
   ---
   gpu:
     types:
       - nvidia
   build_name_extra: "-nvidia"
   EOF
   ```

2. Create the related secret by running the following command.

   ```
   kubectl create secret generic $CLUSTER_NAME-user-overrides --from-
   file=nvidia.yaml=nvidia.yaml
   kubectl label secret $CLUSTER_NAME-user-overrides clusterctl.cluster.x-k8s.io/move=
   ```

Offline Nvidia GPU Override Files

Offline Nvidia GPU override file and task to add the Nvidia GPU override file to your air-gapped pre-provisioned environment.

Offline Nvidia GPU override file options:

GPU Override File

This override file is used to create images with ##Nutanix Kubernetes Platform# (##NKP#), which uses GPU hardware. These override files can also be located in the Konvoy Image Builder repo at https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides.

```
--overrides overrides/offline-nvidia.yaml
```

```
# Use this file when building a machine image, not as a override secret for
 preprovisioned environments
nvidia_runfile_local_file: "{{ playbook_dir}}/../artifacts/
{{ nvidia_runfile_installer }}"
gpu:
  types:
    - nvidia
build_name_extra: "-nvidia"
```

Adding the GPU Override File to Air-gapped Pre-provisioned Environments

Perform this task to add the Nvidia GPU override file to your Air-gapped Pre-provisioned environment.

About this task

If you require your environment to consume GPU resources, add the following GPU Overrides file to your environment:

Procedure

1. If your pre-provisioned machines need a default Override file like GPU, create a secret that includes the overrides in a file.

   ```
   cat > nvidia.yaml << EOF
   ---
   nvidia_runfile_local_file: "{{ playbook_dir}}/../artifacts/
   {{ nvidia_runfile_installer }}"
   gpu:
     types:
       - nvidia
   build_name_extra: "-nvidia"
   EOFcat > nvidia.yaml << EOF
   ---
   nvidia_runfile_local_file: "{{ playbook_dir}}/../artifacts/
   {{ nvidia_runfile_installer }}"
   gpu:
     types:
       - nvidia
   build_name_extra: "-nvidia"
   EOF
   ```

2. Create the related secret by running the following command.

   ```
   kubectl create secret generic $CLUSTER_NAME-user-overrides --from-
   file=nvidia.yaml=nvidia.yaml
   kubectl label secret $CLUSTER_NAME-user-overrides clusterctl.cluster.x-k8s.io/move=
   ```

Oracle Red Hat Linux Kernel Override File

You can find these override files in the Konvoy Image Builder repo at https://github.com/mesosphere/konvoy-image-builder/tree/main/overrides.

```
--overrides overrides/rhck.yaml
```

```
---
oracle_kernel: RHCK
```

Custom Override Files

Custom files you create to use with Konvoy Image Builder.

The following sections describe how to use custom override files:

Image Overrides

When using KIB to create an OS image that is compliant with Nutanix Kubernetes Platform (NKP), the parameters for building and configuring the image are included in the file located in `images/<builder-type>/<os-version.yaml` where <builder-type> is infrastructure provider specific such as `ami` or `ova`.

Although several parameters are specified by default in the Packer templates for each provider, it is possible to override the default values with flags and override files.

Run the `./konvoy-image build images/<builder-type>/{os}-{version}.yaml` command for those files:

Azure Usage

Azure also requires the `--client-id --tenant-id` flag. For more information, see Using KIB with Azure.

> **Note:** To locate the YAML Ain't Markup Language (YAML) for each provider, see https://github.com/mesosphere/konvoy-image-builder/tree/7447074a6d910e71ad2e61fc4a12820d073ae5ae/images.

Using Flags

To execute KIB with specific flags to override the following values:

- The source AMI (`--source-ami`)
- AMI region (`--ami-regions`)
- AWS EC2 instance type (`--aws-instance-type`)

For a comprehensive list of these flags, run: `./konvoy-image build --help`.

Using Override Files

Another option is creating a file with the parameters to be overridden and specifying the `--overrides` flag, as shown in the example.

```
./konvoy-image build images/<builder-type>/<os>-<version>.yaml --overrides
 overrides.yaml
```

> **Note:** While CLI flags can be combined with override files, CLI flags take priority over any override files.

Custom Override Examples

AWS Example on page 1124

vSphere Example on page 1124

AWS Example

About this task

When using the AWS Packer builder to override an image with another image, create an override file and set the `source_ami` under the packer key. This overrides the image search and forces using the specified `source_ami`.

Procedure

1. Create your AWS override file `overrides-source-ami.yaml`

```
---
packer:
  source_ami: "ami-0123456789"
```

2. After creating the override file for our `source_ami`, we can pass our override file by using the`--overrides` flag when building our image:

```
./konvoy-image build aws images/ami/ubuntu-2004.yaml --overrides override-source-ami.yaml
```

vSphere Example

About this task

Context for the current task

Procedure

1. Create your vSphere overrides file `overrides/vcenter.yaml` and fill in the relevant details for your vSphere environment.

```
---
packer:
  vcenter_server: <FQDN of your vcenter>
  vsphere_username: <username for vcenter e.g. administrator@vsphere.local>
  vsphere_password: <password for vcenter>
  ssh_username: builder
  ssh_password: <password for your VMs builder user>
  linked_clone: false
  cluster: <vsphere cluster to use>
  datacenter: <vsphere datacenter to use>
  datastore: <vsphere datastore to use for templates>
  folder: <vsphere folder to store the template in>
  insecure_connection: "true"
  network: <a DHCP-enabled network in vcenter>
  resource_pool: <vsphere resource pool to use>
```

2. After creating the override file for your `source_ova`, pass your override file by using the `--overrides` flag when building your image:

```
konvoy-image build images/ova/ubuntu-2004.yaml \
  --overrides overrides/kube-version.yaml \
  --overrides overrides/vcenter.yaml
```

Pre-Provisioned Override Files

Pre-provisioned environments require specific override files to work correctly. The override files can contain **HTTP Proxy** info and other factors, including whether you want the image to be **FIPS-enabled**, **NVIDIA** optimized, and so on.

Those override files must also have a **Secret** that includes all of the overrides you wish to provide in one file.

Before Nutanix Kubernetes Platform 2.6, you had to specify the proxy in the KIB override setup. Then again, in the `NKP create cluster` command, even though, in this case, they both always use the same proxy setting. As of NKP 2.6, an HTTP proxy gets created from the Konvoy flags for the control plane proxy and workers' proxy values. The flags in the NKP command for Pre-provisioned clusters populate a Secret automatically in the bootstrap cluster. That Secret has a known name that the Pre-provisioned controller finds and applies when it runs the KIB provisioning job. More information about these flags is on the Clusters with HTTP or HTTPS Proxy on page 680 page.

Proxy Configuration	Flag
HTTP proxy for control plane machines	`--control-plane-http-proxy string`
HTTPS proxy for control plane machines	`--control-plane-https-proxy string`
No proxy list for control plane machines	`--control-plane-no-proxy strings`
HTTP proxy for worker machines	`--worker-http-proxy string`
HTTPS proxy for worker machines	`--worker-https-proxy string`
No proxy list for worker machines	`--worker-no-proxy strings`

The nodes that get Kubernetes on them through CAPP automatically get the HTTP proxy secrets that you set using the flags. You no longer have to put the proxy info in both the overrides AND the `NKP create cluster` command as an argument.

For example, if you wish to provide an override with Docker credentials and a different source for EPEL on a CentOS7 machine, you can create a file like this:

```
image_registries_with_auth:
- host: "registry-1.docker.io"
  username: "my-user"
  password: "my-password"
  auth: ""
  identityToken: ""
```

Note: For Red Hat Enterprise Linux (RHEL) Pre-provisioned using GPU, provide the following additional lines of information in your override file:

```
rhsm_user: ""

rhsm_password: ""
```

Example:

```
image_registries_with_auth:
- host: "registry-1.docker.io"
  username: "my-user"
  password: "my-password"
  auth: ""
  identityToken: ""
rhsm_user: ""
rhsm_password: ""
```

Use HTTP or HTTPS Proxy with KIB Images

In some networked environments, the machines used for building images can reach the Internet, but only through an HTTP or HTTPS proxy. For Nutanix Kubernetes Platform (NKP) to operate in these networks, you need a way to specify what proxies to use. You can use an HTTP proxy override file to specify that proxy. When KIB tries installing a particular OS package, it uses that proxy to reach the Internet to download it.

> **Important:** The proxy setting specified here is NOT “baked into” the image - it is only used while the image is being built. The settings are removed before the image is finalized.

While it might seem logical to include the proxy information in the image, the reality is that many companies have multiple proxies - one perhaps for each geographical region or maybe even a proxy per datacenter or office. All network traffic to the Internet goes through the proxy. If you were in Germany, you probably would not want to send all your traffic to a U.S.-based proxy. Doing that slows traffic down and consumes too many network resources. If you bake the proxy settings into the image, you must create a separate image for each region. Creating an image without a proxy makes more sense, but remember that you still need a proxy to access the Internet. Thus, when creating the cluster (and installing the Kommander component of NKP), you must specify the correct proxy settings for the network environment into which you install the cluster. You will use the same base image for that cluster as one installed in an environment with different proxy settings.

See, HTTP Proxy Override Files

Next Step

Either navigate to the main Konvoy Image Builder section of the documentation or back to your installation section:

- If using the Day 1- Basic Install instructions, proceed (or return) to that section Basic Installs by Infrastructure combinations to install and set up NKP based on your infrastructure environment provider.
- If using the Custom Install instructions, proceed (or return) to that section and select the infrastructure provider you are using: Custom Installation and Additional Infrastructure Tools

Related Information

For information on related topics or procedures, see Pre-Provisioned Override Files.

HTTP Proxy Override Files

You can use an HTTP proxy configuration when creating your image. The Ansible playbooks create `systemd`drop-in files for `containerd` and `kubelet` to configure the `http_proxy`, `http_proxy`, and `no_proxy` environment variables for the service from the file `/etc/konvoy_http_proxy.conf`.

To configure a proxy for use during image creation, create a new override file and specify the following:

```
# Example override-proxy.yaml
---
http_proxy: http://example.org:8080
https_proxy: http://example.org:8081
no_proxy: example.org,example.com,example.net
```

These values are only used for the image creation. After creating the image, the Ansible playbooks remove the `/etc/konvoy_http_proxy.conf` file.

You can use the `NKP` command to configure the `KubeadmConfigTemplate` object to create this file on the start-up of the image with values supplied during the `NKP` invocation. This enables using different proxy settings for image creation and runtime.

Next Steps

Pre-Provisioned Override Files or return to cluster creation in the Custom Installation and Additional Infrastructure Tools under your Infrastructure Provider.

Private Registry in Air-gapped Override

To create an Air-gapped registry override for the Docker hub, run the following command:

```
[plugins."io.containerd.grpc.v1.cri".registry.mirrors."docker.io"]
 endpoint = ["https://my-local-registry.local/v2/harbor-registry","https://
registry-1.docker.io"]
```

Tell the override how to create a wildcard mirror with this command:

```
[plugins."io.containerd.grpc.v1.cri".registry.mirrors."*"]
 endpoint = ["https://my-local-registry.local/v2/harbor-registry"]
```

Konvoy Image Builder CLI

Detailed Konvoy command line interface (CLI) content.

Additional Resources:

- Github KIB CLI: https://github.com/mesosphere/konvoy-image-builder/tree/main/docs/cli
- NKP CLI: https://docs.d2iq.com/dkp/2.8/cli-commands

konvoy-image build

Konvoy-image builds command pages.

Build and provision images:

Synopsis

Build and Provision images. Specifying AWS arguments is deprecated and will be removed in a future version. Use the `aws` subcommand instead.

```
konvoy-image build <image.yaml> [flags]
```

Options

```
      --ami-groups stringArray           a list of AWS groups which are allowed use the
image, using 'all' result in a public image
      --ami-regions stringArray          a list of regions to publish amis
      --ami-users stringArray            a list AWS user accounts which are allowed use
the image
      --containerd-version string        the version of containerd to install
      --dry-run                          do not create artifacts, or delete them after
creating. Recommended for tests.
      --extra-vars strings               flag passed Ansible's extra-vars
  -h, --help                             help for build
      --instance-type string             instance type used to build the AMI; the type
must be present in the region in which the AMI is built
      --kubernetes-version string        The version of kubernetes to install. Example:
1.21.6
      --overrides strings                a comma separated list of override YAML files
      --packer-manifest string           provide the path to a custom packer manifest
      --packer-on-error string           [advanced] set error strategy for packer.
strategies [cleanup, abort, run-cleanup-provisioner]
      --packer-path string               the location of the packer binary (default
"packer")
      --region string                    the region in which to build the AMI
```

```
      --source-ami string                the ID of the AMI to use as the source; must
be present in the region in which the AMI is built
      --source-ami-filter-name string    restricts the set of source AMIs to ones whose
Name matches filter
      --source-ami-filter-owner string   restricts the source AMI to ones with this
owner ID
      --work-dir string                  path to custom work directory generated by the
generate command
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

konvoy-image build aws

Build and provision AWS images.

Build and provision AWS images.

```
konvoy-image build aws <image.yaml> [flags]
```

Examples

```
aws --region us-west-2 --source-ami=ami-12345abcdef images/ami/centos-79.yaml
```

Options

```
      --ami-groups stringArray           a list of AWS groups which are allowed use the
image, using 'all' result in a public image
      --ami-regions stringArray          a list of regions to publish amis
      --ami-users stringArray            a list AWS user accounts which are allowed use
the image
      --containerd-version string        the version of containerd to install
      --dry-run                          do not create artifacts, or delete them after
creating. Recommended for tests.
      --extra-vars strings               flag passed Ansible's extra-vars
  -h, --help                             help for aws
      --instance-type string             instance type used to build the AMI; the type
must be present in the region in which the AMI is built
      --kubernetes-version string        The version of kubernetes to install. Example:
1.21.6
      --overrides strings                a comma separated list of override YAML files
      --packer-manifest string           provide the path to a custom packer manifest
      --packer-on-error string           [advanced] set error strategy for packer.
strategies [cleanup, abort, run-cleanup-provisioner]
      --packer-path string               the location of the packer binary (default
"packer")
      --region string                    the region in which to build the AMI
      --source-ami string                the ID of the AMI to use as the source; must
be present in the region in which the AMI is built
      --source-ami-filter-name string    restricts the set of source AMIs to ones whose
Name matches filter
      --source-ami-filter-owner string   restricts the source AMI to ones with this
owner ID
      --work-dir string                  path to custom work directory generated by the
generate command
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

See Also

- konvoy-image build - build and provision images

konvoy-image build azure
Build and provision Azure images.

Build and provision azure images

```
konvoy-image build azure <image.yaml> [flags]
```

Examples

```
azure --location westus2 --subscription-id <sub_id> images/azure/centos-79.yaml
```

Options

```
      --client-id string                      the client id to use for the build
      --cloud-endpoint string                 Azure cloud endpoint. Which can be one of
[Public USGovernment China] (default "Public")
      --containerd-version string             the version of containerd to install
      --dry-run                               do not create artifacts, or delete them
after creating. Recommended for tests.
      --extra-vars strings                    flag passed Ansible's extra-vars
      --gallery-image-locations stringArray   a list of locations to publish the image
(default [westus])
      --gallery-image-name string             the gallery image name to publish the
image to
      --gallery-image-offer string            the gallery image offer to set (default
"nkp")
      --gallery-image-publisher string        the gallery image publisher to set
(default "nkp")
      --gallery-image-sku string              the gallery image sku to set
      --gallery-name string                   the gallery name to publish the image in
(default "nkp")
 -h, --help                                   help for azure
      --instance-type string                  the Instance Type to use for the build
(default "Standard_D2s_v3")
      --kubernetes-version string             The version of kubernetes to install.
Example: 1.21.6
      --location string                       the location in which to build the image
(default "westus2")
      --overrides strings                     a comma separated list of override YAML
files
      --packer-manifest string                provide the path to a custom packer
manifest
      --packer-on-error string                [advanced] set error strategy for packer.
strategies [cleanup, abort, run-cleanup-provisioner]
      --packer-path string                    the location of the packer binary
(default "packer")
      --resource-group string                 the resource group to create the image in
(default "nkp")
      --subscription-id string                the subscription id to use for the build
      --tenant-id string                      the tenant id to use for the build
      --work-dir string                       path to custom work directory generated
by the generate command
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
 -v, --v int      select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

See Also

- konvoy-image build - build and provision images

konvoy-image build gpt
Build and provision GCP images.

Build and provision azure images

```
konvoy-image build gcp <image.yaml> [flags]
```

Examples

```
gcp ... images/gcp/centos-79.yaml
```

Options

```
      --containerd-version string   the version of containerd to install
      --dry-run                     do not create artifacts, or delete them after
creating. Recommended for tests.
      --extra-vars strings          flag passed Ansible's extra-vars
  -h, --help                        help for gcp
      --kubernetes-version string   The version of kubernetes to install. Example:
1.21.6
      --network string              the network to use when creating an image
      --overrides strings           a comma separated list of override YAML files
      --packer-manifest string      provide the path to a custom packer manifest
      --packer-on-error string      [advanced] set error strategy for packer.
strategies [cleanup, abort, run-cleanup-provisioner]
      --packer-path string          the location of the packer binary (default
"packer")
      --project-id string           the project id to use when storing created image
      --region string               the region in which to launch the instance (default
"us-west1")
      --work-dir string             path to custom work directory generated by the
generate command
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

See Also

- konvoy-image build - build and provision images

konvoy-image build vsphere
Build and provision vSphere images.

Build and provision Azure images using the command `konvoy-image build vsphere <image.yaml> [flags]`.

Example output:

```
vsphere --datacenter dc1 --cluster zone1 --datastore nfs-store1 --network public --
template=nutanix-base-
templates/nutanix-base-ubuntu20 images/ami/ubuntu-2004.yaml
```

Options

```
      --cluster string               vSphere cluster to be used. Alternatively set
host. Required value: you can pass the cluster name through an override file or image
definition file.
      --containerd-version string    the version of containerd to install
      --datacenter string            The vSphere datacenter. Required value: you can
pass the datacenter name through an override file or image definition file.
      --datastore string             vSphere datastore used to build and store the
image template. Required value: you can pass the datastore name through an override
file or image definition file.
      --dry-run                      do not create artifacts, or delete them after
creating. Recommended for tests.
      --extra-vars strings           flag passed Ansible's extra-vars
      --folder string                vSphere folder to store the image template
  -h, --help                         help for vsphere
      --host string                  vSphere host to be used. Alternatively set
cluster. Required value: you can pass the host name through an override file or image
definition file.
      --kubernetes-version string    The version of kubernetes to install. Example:
1.21.6
      --network string               vSphere network used to build image template.
Ensure the host running the command has access to this network. Required value: you
can pass the network name through an override file or image definition file.
      --overrides strings            a comma separated list of override YAML files
      --packer-manifest string       provide the path to a custom packer manifest
      --packer-on-error string       [advanced] set error strategy for packer.
strategies [cleanup, abort, run-cleanup-provisioner]
      --packer-path string           the location of the packer binary (default
"packer")
      --resource-pool string         vSphere resource pool to be used to build image
template
      --ssh-privatekey-file string   Path to ssh private key which will be used to log
into the base image template
      --ssh-publickey string         Path to SSH public key which will be copied to the
image template. Ensure to set ssh-privatekey-file or load the private key into ssh-
agent
      --ssh-username string          username to be used with the vSphere image
template
      --template string              Base template to be used. Can include folder.
<templatename> or <folder>/<templatename>. Required value: you can pass the template
name through an override file or image definition file.
      --work-dir string              path to custom work directory generated by the
generate command
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

See Also

- Konvoy Image Builder on page 1084 - build and provision images

konvoy-image completion

Konvoy-image completion command pages.

Synopsis

Generate the autocompletion script for the konvoy-image for the specified shell. See each sub-command's help for details on how to use the generated script.

Options

```
  -h, --help   help for completion
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

konvoy-image completion bash

Generate the autocompletion script for bash.

Synopsis

Generate the autocompletion script for the bash shell.

This script depends on the 'bash-completion' package. You can install it through your OS's package manager if it has not been installed yet.

To load completions in your current shell session:

```
source <(konvoy-image completion bash)
```

To load completions for every new session, execute once:

Linux:

```
konvoy-image completion bash > /etc/bash_completion.d/konvoy-image
```

macOS

```
konvoy-image completion bash > $(brew --prefix)/etc/bash_completion.d/konvoy-image
```

You will need to start a new shell for this setup to take effect.

```
konvoy-image completion bash
```

Options

```
  -h, --help              help for bash
      --no-descriptions   disable completion descriptions
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

See Also

- konvoy-image completion - Generate the autocompletion script for the specified shell

konvoy-image completion fish

Generate the autocompletion script for fish.

Synopsis

Generate the autocompletion script for the fish shell.

To load completions in your current shell session:

```
konvoy-image completion fish | source
```

To load completions for every new session, execute once:

```
konvoy-image completion fish > ~/.config/fish/completions/konvoy-image.fish
```

You must start a new shell for this setup to take effect.

```
konvoy-image completion fish [flags]
```

Options

```
  -h, --help              help for fish
      --no-descriptions   disable completion descriptions
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

konvoy-image completion powershell

Generate the autocompletion script for powershell.

Synopsis

Generate the autocompletion script for the powershell.

To load completions in your current shell session:

```
konvoy-image completion powershell | Out-String | Invoke-Expression
```

To load completions for every new session, add the output of the above command to your powershell profile.

```
konvoy-image completion powershell [flags]
```

Options

```
  -h, --help              help for powershell
      --no-descriptions   disable completion descriptions
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

konvoy-image completion zsh

Generate the autocompletion script for Zookeeper Shell (ZSH).

Synopsis

Generate the autocompletion script for the zsh shell.

If shell completion is not already enabled in your environment, you will need to enable it. You can execute the following once:

```
echo "autoload -U compinit; compinit" >> ~/.zshrc
```

To load completions in your current shell session:

```
source <(konvoy-image completion zsh)
```

To load completions for every new session, execute once:

Linux

```
konvoy-image completion zsh > "${fpath[1]}/_konvoy-image"
```

macOS

```
konvoy-image completion zsh > $(brew --prefix)/share/zsh/site-functions/_konvoy-image
```

Options

```
  -h, --help              help for zsh
      --no-descriptions   disable completion descriptions
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

konvoy-image generate-docs

Konvoy-image generate-docs command page.

Generate docs in the path

Examples

```
generate-docs /tmp/docs
```

Options

```
  -h, --help   help for generate-docs
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

konvoy-image generate

Konvoy-image generates command pages.

Generate files relating to building images.

Synopsis

Generate files relating to building images. Specifying AWS arguments is deprecated and will be removed in a future version. Use the `aws` subcommand instead.

```
konvoy-image generate <image.yaml> [flags]
```

Options

```
      --ami-groups stringArray           a list of AWS groups which are allowed use the
image, using 'all' result in a public image
      --ami-regions stringArray          a list of regions to publish amis
      --ami-users stringArray            a list AWS user accounts which are allowed use
the image
      --containerd-version string        the version of containerd to install
      --extra-vars strings               flag passed Ansible's extra-vars
  -h, --help                             help for generate
      --instance-type string             instance type used to build the AMI; the type
must be present in the region in which the AMI is built
      --kubernetes-version string        The version of kubernetes to install. Example:
1.21.6
      --overrides strings                a comma separated list of override YAML files
      --region string                    the region in which to build the AMI
      --source-ami string                the ID of the AMI to use as the source; must
be present in the region in which the AMI is built
```

```
      --source-ami-filter-name string    restricts the set of source AMIs to ones whose
Name matches filter
      --source-ami-filter-owner string   restricts the source AMI to ones with this
owner ID
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

konvoy-image generate aws
Generate files relating to building aws images.

Generate files relating to building AWS images

```
konvoy-image generate aws <image.yaml> [flags]
```

Examples

```
aws --region us-west-2 --source-ami=ami-12345abcdef images/ami/centos-79.yaml
```

Options

```
aws --region us-west-2 --source-ami=ami-12345abcdef images/ami/centos-79.yaml
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

konvoy-image generate azure
Generate files relating to building Azure images

Generate files relating to building Azure images.

```
konvoy-image generate azure <image.yaml> [flags]
```

Examples

```
azure --location westus2 --subscription-id <sub_id> images/azure/centos-79.yaml
```

Options

```
      --client-id string                      the client id to use for the build
      --cloud-endpoint string                 Azure cloud endpoint. Which can be one of
[Public USGovernment China] (default "Public")
      --containerd-version string             the version of containerd to install
      --extra-vars strings                    flag passed Ansible's extra-vars
      --gallery-image-locations stringArray   a list of locations to publish the image
(default [westus])
      --gallery-image-name string             the gallery image name to publish the
image to
      --gallery-image-offer string            the gallery image offer to set (default
"nkp")
      --gallery-image-publisher string        the gallery image publisher to set
(default "nkp")
      --gallery-image-sku string              the gallery image sku to set
      --gallery-name string                   the gallery name to publish the image in
(default "nkp")
  -h, --help                                  help for azure
      --instance-type string                  the Instance Type to use for the build
(default "Standard_D2s_v3")
```

```
      --kubernetes-version string             The version of kubernetes to install.
Example: 1.21.6
      --location string                       the location in which to build the image
(default "westus2")
      --overrides strings                     a comma separated list of override YAML
files
      --resource-group string                 the resource group to create the image in
(default "nkp")
      --subscription-id string                the subscription id to use for the build
      --tenant-id string                      the tenant id to use for the build
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

See Also

- konvoy-image generate - generate files relating to building images

konvoy-image provision

Konvoy-image provision command page.

Provision to an inventory.yaml or hostname. Note the comma at the end of the hostname.

```
konvoy-image provision <inventory.yaml|hostname,> [flags]
```

Examples

```
provision --inventory-file inventory.yaml
```

Options

```
      --containerd-version string   the version of containerd to install
      --extra-vars strings          flag passed Ansible's extra-vars
  -h, --help                        help for provision
      --inventory-file string       an ansible inventory defining your infrastructure
      --kubernetes-version string   The version of kubernetes to install. Example:
1.21.6
      --overrides strings           a comma separated list of override YAML files
      --provider string             specify a provider if you wish to install provider
specific utilities
      --work-dir string             path to custom work directory generated by the
generate command
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

konvoy-image upload

Konvoy-image uploads command pages.

Upload one of the [artifacts].

Options

```
  -h, --help   help for upload
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

konvoy-image upload artifacts

Upload offline artifacts to hosts defined in the inventory-file.

Upload offline artifacts to hosts defined in the inventory-file.

```
konvoy-image upload artifacts [flags]
```

Options

```
      --container-images-dir string   path to container images for install on remote
hosts.
      --containerd-bundle string      path to Containerd tar file for install on remote
hosts.
      --extra-vars strings            flag passed Ansible's extra-vars
  -h, --help                          help for artifacts
      --inventory-file string         an ansible inventory defining your infrastructure
(default "inventory.yaml")
      --nvidia-runfile string         path to nvidia runfile to place on remote hosts.
      --os-packages-bundle string     path to os-packages tar file for install on
remote hosts.
      --overrides strings             a comma separated list of override YAML files
      --pip-packages-bundle string    path to pip-packages tar file for install on
remote hosts.
      --work-dir string               path to custom work directory generated by the
command
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

konvoy-image validate

Konvoy-image validate command pages.

Validate existing infrastructure

```
konvoy-image validate [flags]
```

Options

```
      --apiserver-port int      apiserver port (default 6443)
  -h, --help                    help for validate
      --inventory-file string   an ansible inventory defining your infrastructure
(default "inventory.yaml")
      --pod-subnet string       ip addresses used for the pod subnet (default
"192.168.0.0/16")
      --service-subnet string   ip addresses used for the service subnet (default
"10.96.0.0/12")
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

konvoy-image version

Konvoy-image version command pages.

Version for konvoy-image

```
konvoy-image version [flags]
```

Options

```
  -h, --help   help for version
```

Options Inherited from Parent Commands

```
      --color     enable color output (default true)
  -v, --v int     select verbosity level, should be between 0 and 6
      --verbose   enable debug level logging (same as --v 5)
```

10

NUTANIX IMAGE BUILDER

The Nutanix image builder enables you to create and manage pre-configured operating system (OS) images specifically for running Kubernetes on Nutanix infrastructure. You can first create an image with all the necessary configurations, drivers, and software needed for Kubernetes. After creating an image, you can use this image to add a new node to your Kubernetes cluster quickly and consistently This reduces Kubernetes cluster deployment and upgrade errors caused by misconfiguration of OS images and packages. Images helps you easily manage large Kubernetes deployments on Nutanix systems.

You can add images in the following ways:

- Use an image provided by Nutanix. For more information, see Using an Image provided by Nutanix on page 1139
- Build your own customized image. For more information, see Building a Custom Image on page 1140.
- Upload a Base ISO or Cloud OS image to Prism Central. For more information, see https://portal.nutanix.com/page/documents/details?targetId=Prism-Central-Guide-vpc_2024_2:mul-image-import-pc-t.html.

You can use the Nutanix image builder only with Nutanix infrastructure. You can build images using AWS, Azure, vSphere, or GCP in the Konvoy image builder. For information on how to build images using Konvoy image builder, see Konvoy Image Builder on page 1084.

Using an Image provided by Nutanix

Nutanix provides an OS image that contains all the tools and configurations to build NKP Kubernetes clusters.

Before you begin

Ensure that you meet the following prerequisites:

- AMD64 architecture.
- Subscribe to one of the following NKP licenses:
  - NKP Starter
  - NKP Pro
  - NKP Ultimate

About this task

To use the image provided by Nutanix, follow these steps:

Procedure

1. Download the image from the Nutanix Support Portal.

2. Upload the image to Prism Central.
   For information on how to upload images to Prism Central, see Importing Images to Prism Central.

   Note: Ensure that the name of the image is in the following format: *<NKP version>:<OS name>-<Kubernetes version> timestamp*

Building a Custom Image

You can build your own customized image according to your requirements.

Before you begin

Ensure that you meet the following prerequisites:

- Container Runtime
  - Docker version 27.4.0
  - Podman 4.0 or later
- Nutanix Kubernetes Platform Command-Line Interface (NKP CLI)
  - Operating System: Linux or macOS
  - Architecture: AMD64 or ARM64
- Export Nutanix Prism Central credentials as environment variables:
  - NUTANIX_USERNAME
  - NUTANIX_PASSWORD
- Subscribe to one of the following NKP licenses:
  - NKP Pro
  - NKP Ultimate
- Upload the OS base image to Prism Central. The name of this base image is referenced in the `nkp create image` parameter `<base_image_name_or_UUID_or_URL>`.

  For more information, see Importing Images to Prism Central in *Prism Central Infrastructure Guide*. For more information about supported operating system, see Supported Infrastructure Operating Systems on page 12.

  Note:

  - If you require Ubuntu as an operating system, ensure that you select the cloud Ubuntu image as a base image and avoid using the default Ubuntu image.
  - Your base image must use UEFI Boot type.

About this task

To create an OS image, follow these steps:

Procedure

1. In the NKP, run the following command.

```
nkp create image nutanix <OS_Name> \
     --cluster <Prism_Element_cluster_name> \
     --endpoint <Prism_Central_endpoint_URL> \
     --subnet <subnet_name_or_UUID_associated_with_Prism_Element> \
     --source-image <base_image_name_or_UUID_or_URL>
```

   A new NKP compatible OS image for a non air-gapped environment is created in Prism Central. You can use this image to create new NKP clusters.

2. To create images according to your requirement, add more flags to the preceding command.

   For example, you can do the following:

   - Create an image for GPU workloads. For more information, see Creating a GPU Image on page 637.
   - Create a customized image using YAML files if customizations cannot be done using the available flags.
   - Create an image for air-gapped environments.

   For information on these use cases, see Building a Custom Image Use Cases on page 1141.

   To view all the flags and their purpose, run the following command.

```
nkp create image nutanix --help
```

3. (Optional) For additional customizations that cannot be performed using flags or YAML files, contact Nutanix support.

Building a Custom Image Use Cases

The following examples illustrate how you can build your own images using the Nutanix image builder for different scenarios.

Table 69: Use Cases

Use Cases	Procedure
To create an image for a non air-gapped environment	In the NKP CLI, run the following command. `nkp create image nutanix <OS_Name> \ --cluster <Prism_Element_cluster_name> \ --endpoint <Prism_Central_endpoint_URL> \ --subnet <subnet_name_or_UUID_associated_with_Prism_Element> \ --source-image <base_image_name_or_UUID_or_URL>`

Use Cases	Procedure
To create an image for an air-gapped environment without a bastion VM	To create an image for an air-gapped environment, follow these steps: 1. Download the NKP air-gapped bundle from the Nutanix portal. For more information, see Downloading NKP on page 16 2. (Optional) Create your own air-gapped infrastructure. 3. In the NKP CLI, run the following command. `nkp create image nutanix <OS_Name> \ --cluster <Prism_Element_cluster_name> \ --endpoint <Prism_Central_endpoint_URL> \ --subnet <subnet_name_or_UUID_associated_with_Prism_Element> \ --source-image <base_image_name_or_UUID_or_URL> \ --artifacts-directory <path_to_artifact_bundle_DIR>`
To create an an image for an air-gapped environment with a bastion VM	To create an image for an air-gapped environment, follow these steps: 1. Download the NKP air-gapped bundle from the Nutanix Support Portal. For more information, see Downloading NKP on page 16 2. (Optional) Create your own air-gapped infrastructure. 3. In the NKP CLI, run the following command. `nkp create image nutanix <OS_Name> \ --cluster <Prism_Element_cluster_name> \ --endpoint <Prism_Central_endpoint_URL> \ --subnet <subnet_name_or_UUID_associated_with_Prism_Element> \ --source-image <base_image_name_or_UUID_or_URL> \ --artifacts-directory <path_to_artifact_bundle_DIR> \ --bastion-host <IP_or_hostname> \ --bastion-username <SSH_user_on_bastion> \ --bastion-private-key-file <path_to_SSH_private_key_file>`

Use Cases	Procedure
To create an image for GPU workloads	In the NKP CLI, run the following command. `nkp create image nutanix <OS_Name> \ --cluster <Prism_Element_cluster_name> \ --endpoint <Prism_Central_endpoint_URL> \ --subnet <subnet_name_or_UUID_associated_with_Prism_Element> \ --source-image <base_image_name_or_UUID_or_URL> \ --gpu \ --gpu-name <name_of_GPU_attached _to_the_cluster>` For more information, see Creating a GPU Image on page 637.
To add additional customizations that cannot be done using flags, you can use override files.	In the NKP CLI, run the command. `nkp create image nutanix <OS_Name> \ --cluster <Prism_Element_cluster_name> \ --endpoint <Prism_Central_Endpoint_URL> \ --subnet <subnet_name_or_UUID_associated_with_Prism_Element> \ --source-image <base_image_name-or_UUID_or_URL> \ --overrides <override_file1.yaml, override_file2.yaml>`

11

UPGRADE NKP

Describe upgrade prerequisites and paths for upgrading your air-gapped and non-air-gapped environments to the current NKP version that is compatible with the latest Kubernetes version.

The Nutanix Kubernetes Platform (NKP) upgrade is essential to your environment's life cycle. It ensures that you are up-to-date with the latest features and can benefit from the most recent improvements, enhanced cluster management, and better performance. This section describes how to upgrade your air-gapped and non-air-gapped environment to the latest NKPversion compatible with the latest Kubernetes version.

Prerequisites

Review the following before starting your upgrade:

- NKP Release Notes
- Significant Kubernetes changes that may affect your system. For more information, see https://portal.nutanix.com/page/documents/details?targetId=Release-Notes-Nutanix-Kubernetes-Platform-v2_12:rel-release-notes-nkp-v2_12-r.html.
- Verify the NKP version you have downloaded using the command `nkp version`.
- To review the supported Kubernetes versions, see Upgrade Compatibility Tables.

Supported Upgrade Paths

Use the table to determine the upgrade path for your environment.

	Upgrading from Release…		
…to Release		2.8.0	2.8.1
	2.8.0	NA	NA
	2.8.1	Yes	NA
	2.12	Yes	Yes

Upgrade Order

Perform the upgrade sequentially, beginning with the Kommander component and moving to upgrade clusters and CAPI components in the Konvoy component. The process for upgrading your entire NKP product is different depending on your license and environment.

To get started, select the license that matches your environment and license:

- Upgrade NKP Pro on page 1173: A stand-alone Management Cluster.
- Upgrade NKP Ultimate on page 1152: A multi-cluster environment that includes a combination of a Management cluster(s) and managed or attached workspace clusters.

Upgrade Compatibility Tables

Links to NKP versions for Kubernetes, applications, and components.

This page contains the latest Nutanix Kubernetes Platform (NKP) versions for Kubernetes, applications, and components. Individual sections will have links to pages with further information if required.

Supported Operating Systems

The supported Operating System (OS) is essential for upgrades. During an upgrade, you need to create new AMI's or images with the Kubernetes version to which you want to upgrade. That requires selecting an OS and ensuring you have the correct version. To review the Supported Infrastructure Operating Systems, see Supported Infrastructure Operating Systems

Konvoy Image Builder

To view the components versions and compatibility table, see Compatible NKP to KIB Versions on page 1085.

Deploying Clusters Versions

The latest supported Kubernetes versions for NKP are listed below. In NKP 2.12, the Kubernetes version installed by default is 1.29.6. The newest and oldest Kubernetes versions used with NKP must be within one minor version.

- The latest supported Kubernetes version for deploying clusters is **1.29.6**.
- The supported Kubernetes version for deploying clusters in EKS is **1.28.7** .

Kubernetes 1.29.6 benefits you from upstream Kubernetes's latest features and security fixes. This release comes with 60 enhancements, such as Node log access through Kubernetes API, Seccomp profile defaulting, and much more. For more information, see https://github.com/kubernetes/enhancements/issues/2258.

To read more about significant features in this release, see https://github.com/kubernetes/sig-release/blob/master/releases/release-1.27/README.md and https://kubernetes.io/blog/2023/04/11/kubernetes-v1-27-release/.

Supported Kubernetes Cluster Versions

Table mapping the supported Kubernetes® version by produce to NKP 2.12.0. NKP for Kubernetes v.1.29.6. In NKP 2.12, the Kubernetes version installed by default is 1.29.6. The Kubernetes versions used with NKP must be within one minor version.For deploying clusters, the supported Kubernetes version is 1.29.6.For deploying clusters in EKS, the supported Kubernetes versions is 1.28.7. Kubernetes ® is a registered trademark of The Linux Foundation in the United States and other countries, and is used pursuant to a license from The Linux Foundation.

Table 70: Supported Kubernetes Versions

Product	Compatible Kubernetes Versions
AKS	1.27.x
EKS	1.26.x
GKE	1.27.x

Important:

- Attaching Kubernetes clusters with versions greater than n-1 is not recommended.

- Attaching a cluster with an earlier Kubernetes version than the NKP default, requires you to build and use an image with that earlier version of Kubernetes. Failure to do this can result in an error. For more information on building images, see Konvoy Image Builder.

Upgrading Cluster Node Pools

Before you begin

Review the following:

- Deploy Pod Disruption Budget (PDB). For more information, see https://kubernetes.io/docs/concepts/workloads/pods/disruptions/.
- Konvoy Image Builder (KIB)

About this task

Upgrading a node pool involves draining the existing nodes in the node pool and replacing them with new nodes. To ensure minimum downtime and maintain high availability of the critical application workloads during the upgrade process, we recommend deploying Pod Disruption Budget (Disruptions) for your critical applications. For more information, see https://kubernetes.io/docs/concepts/workloads/pods/disruptions/

The Pod Disruption Budget prevents critical applications from being affected by misconfiguration or failures during the upgrade process.

Procedure

1. Deploy a Pod Disruption Budget for your critical applications. If your application can tolerate only one replica being unavailable at a time, you can set a Pod disruption budget, as shown in the following example. The example below is for NVIDIA graphics processing unit (GPU) node pools, but the process is the same for all node pools.

   Note: Repeat this step for each additional node pool.

2. Create the file using the command `pod-disruption-budget-nvidia.yaml`.

   Example output:

   ```
   apiVersion: policy/v1
   kind: PodDisruptionBudget
   metadata:
     name: nvidia-critical-app
   spec:
     maxUnavailable: 1
     selector:
       matchLabels:
         app: nvidia-critical-app
   ```

3. Apply the YAML Ain't Markup Language (YAML) file using the command `kubectl create -f pod-disruption-budget-nvidia.yaml`.

4. Prepare an OS image for your node pool using Konvoy Image Builder.

What to do next

For information on related topics or procedures, see Ultimate: Upgrade the Management Cluster Kubernetes Version.

Upgrade Prerequisites

Prerequisites for Kommander and Konvoy components.

The prerequisites vary depending on your environment. Ensure you follow the correct lists for your specific configurations.

Prerequisites for the Kommander Component

Prerequisites specific to the Kommander component.

The prerequisites for Kommander consist of two parts: the first section covers all environments, and the subsections depend on air-gapped or non-air-gapped and optional applications. Ensure you follow the correct section for your specific environment type.

All Kommander Environments

For all Kommander environments, regardless of network type and applications, complete these prerequisites:

- Before upgrading, we **strongly recommend** reading the release notes and verifying your current setup against any possible breaking changes.
- Review the Components and Application versions that are part of this upgrade.
- Ensure the applications you are running will be compatible with Kubernetes v1.27.x.
- If you have attached clusters, ensure your applications will be compatible with Kubernetes v1.27.x.
- Review the list of major Kubernetes changes that may affect your system.
- Ensure you are attempting to run a supported NKP upgrade. For more information, see Upgrade NKP on page 1144.
- **REQUIRED** Create a Backup and Restore on page 567 of your current configuration with Velero before upgrading.
- Download and install this release's supported Nutanix Kubernetes Platform (NKP) CLI binary on your computer.

The remaining prerequisites are necessary if you have one of the following environments or additions to basic Kommander. Otherwise, proceed to the prerequisites for the Konvoy component.

Air-gapped Kommander Environments

If you operate Kommander in an air-gapped environment, complete these prerequisites:

- There are several sets of images you will need to push to your local registry:
  - The Kommander component -`kommander-image-bundle-v2.12.0.tar`
  - The Konvoy component for Kubernetes cluster - `konvoy-image-bundle-v2.12.0.tar`
  - Optional: Catalog Applications (Ultimate only) -`dkp-catalog-applications-image-bundle-v2.12.0.tar`

  For more information, see Upgrade: For Air-gapped Environments Only on page 1149.
- For air-gapped environments with catalog applications: Ensure you have updated your catalog repository before upgrading. The catalog repository contains the Docker registry with the Workspace Catalog Application Upgrade on page 427 images and a charts bundle file containing Downloading NKP on page 16 charts.

NKP Workspace Catalog Applications Only

If you use any of the NKP Workspace Catalog Applications, complete these prerequisites:

- Review the release notes to ensure your clusters run on a supported Kubernetes version for this release of NKP, and that said the Kubernetes version is also compatible with your Downloading NKP on page 16.
- We recommend keeping all clusters on the same Kubernetes version for customers with an Ultimate license and a multi-cluster environment. This ensures that your NKP catalog application can run on all clusters in a given workspace.
- Konvoy installs a new Kubernetes version with each upgrade. Therefore, your Workspace Catalog Application Upgrade on page 427 must be compatible with the Kubernetes version that comes with this release of NKP.

  > **Note:** If your current Catalog application version is incompatible with this release's Kubernetes version, upgrade the application to a compatible version BEFORE upgrading the Konvoy component on Managed clusters or BEFORE upgrading the Kubernetes version on Attached clusters.

- **Air-gapped environments with catalog applications:** Ensure you have updated your catalog repository before upgrading. The catalog repository contains the Docker registry with the NKP catalog application images and a charts bundle file containing NKP catalog application charts.

Related Topics

Additional information resources for upgrading NKP.

For more information on supported components and applications, see the NKP Release Notes at https://portal.nutanix.com/page/documents/details?targetId=Release-Notes-Nutanix-Kubernetes-Platform-v2_12:rel-release-notes-nkp-v2_12-r.html.

Prerequisites for the Konvoy Component

Konvoy specific prerequisites.

The prerequisites for Konvoy also consists of two parts, the first section for all environments, and the subsections are specific to your network type, air-gapped, non-air-gapped, and optional applications, follow the directions for your environment type.

All Konvoy Environments

For all Konvoy environments, regardless of network type and applications, complete these prerequisites:

- Verify your current Nutanix Kubernetes Platform (NKP) version using the CLI command `nkp version`.
- Set the environment variables:
  - For Air-gapped environments, download the required bundles from Nutanix, at https://support.d2iq.com/hc/en-us.
  - For Azure, set the required environment variables. For more information, see Azure Prerequisites on page 884.
  - For AWS and EKS, set the required AWS Infrastructure and AWS Installation on page 163.
  - For vSphere, see vSphere Prerequisites: All Installation Types on page 259.
  - For GCP, set the required Google Cloud Platform (GCP) Infrastructure.
- **vSphere only:** If you want to resize your disk, ensure you have reviewed Create a vSphere Base OS Image.

Air-gapped Konvoy Environments Only

If you operate Konvoy in an air-gapped environment, complete these prerequisites:

- Verify your current NKP version using the CLI command `nkp version`.
- Ensure that all platform applications in the management cluster have been upgraded to avoid compatibility issues with the Kubernetes version included in this release. This is done automatically when upgrading Kommander, so ensure that you upgrade Kommander prior to upgrading Konvoy.
- For air-gapped environments, seed the registry. For more information, see Upgrade: For Air-gapped Environments Only

Third-Party and Open Source Attributions

Third-party and open source attribution information.

Topic	Link
List of third-party and open source attributions.	https://d2iq.com/legal/3rd

Next Steps

Links to your next step after completing the Nutanix Kubernetes Platform (NKP) upgrade prerequisites.

Depending on your license type, you will follow the relevant link:

- Upgrade NKP Ultimate
- Upgrade NKP Pro

Upgrade: For Air-gapped Environments Only

Because air-gapped environments do not have direct access to the Internet, you must download, extract and load several required images to your local container registry, before installing or upgrading Nutanix Kubernetes Platform (NKP) . The information below will be covered as a step during either **Ultimate** or **Pro** Upgrade steps, but feel free to familiarize yourself with the concept below if desired. Otherwise, depending on your license type, follow the relevant link to begin upgrading:

- Upgrade NKP Ultimate
- Upgrade NKP Pro

Overview of Seeding the Registry for Air-gapped Environment

Downloading all Images for Air-gapped Deployments

Download the installation images for and air-gapped environment.

About this task

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images is required. See below for prerequisites to download and then how to push the necessary images to this registry.

Procedure

1. Download the Complete Nutanix Kubernetes Platform (NKP) Air-gapped Bundle for this release (that is. `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`) to load registry images as explained below. For more the download link, see Downloading NKP on page 16.

2. Connectivity with clusters attaching to the management cluster is required.

   - Both management and attached clusters must be able to connect to the local registry.
   - The management cluster must be able to connect to all attached cluster's Application Programming Interface (API) servers.
   - The management cluster must be able to connect to any load balancers created for platform services on the management cluster.

Extracting Air-gapped Images and Set Variables

Work flow for extracting the image bundled into your private registry.

About this task

After you have downloaded the image to a local directory, perform these steps to extract the air-gapped image bundles into your private registry:

Procedure

1. Extract the tarball to a local directory using the command `tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`.
2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories.

   For example, for the bootstrap, change your directory to the `nkp-version` directory similar to example below depending on your current location.

   ```
   cd nkp-v2.12.0
   ```

3. Set an environment variable with your registry address using the command `export REGISTRY_URL="https/http://registry-address:registry-port" export REGISTRY_USERNAME=username export REGISTRY_PASSWORD=password`

Loading Images for Deployments - Konvoy Pre-provisioned

Copying the artifacts onto the cluster hosts before you begin the upgrade.

About this task

For Pre-provisioned air-gapped environments only, you must run konvoy-image upload artifacts to copy the artifacts onto the cluster hosts before you begin the upgrade the Cluster API (CAPI) components process later in the upgrade steps.

Procedure

1. The Kubernetes image bundle will be located in `kib/artifacts/images`. Verify the image and artifacts exist.

   a. To verify the image bundles exist, use the following command.

   ```
   $ ls kib/artifacts/images/
   kubernetes-images-1.29.6-nutanix.1.tar kubernetes-images-1.29.6-nutanix.1-fips.tar
   ```

   b. To verify the artifacts for your OS exist in the directory and export the appropriate variables, use the following command.

   ```
   $ ls artifacts/
   ```

```
1.29.6_centos_7_x86_64.tar.gz     1.29.6_redhat_8_x86_64_fips.tar.gz
 containerd-1.6.28-d2iq.1-rhel-8.6-x86_64_fips.tar.gz pip-packages.tar.gz
1.29.6_centos_7_x86_64_fips.tar.gz  1.29.6_rocky_9_x86_64.tar.gz
 containerd-1.6.28-d2iq.1-rocky-9.0-x86_64.tar.gz
1.29.6_redhat_7_x86_64.tar.gz     1.29.6_ubuntu_20_x86_64.tar.gz
 containerd-1.6.28-d2iq.1-rocky-9.1-x86_64.tar.gz
1.29.6_redhat_7_x86_64_fips.tar.gz  containerd-1.6.28-d2iq.1-ubuntu-20.04-
x86_64.tar.gz
1.29.6_redhat_8_x86_64.tar.gz           images
```

c. Set the bundle values with the name from the private registry location, use the `export OS_PACKAGES_BUNDLE` command as shown in the example.

```
export OS_PACKAGES_BUNDLE=name_of_the_OS_package
export CONTAINERD_BUNDLE=name_of_the_containerd_bundle
```

For example, for RHEL 8.6 set the following:

```
export OS_PACKAGES_BUNDLE=1.29.6_redhat_8_x86_64.tar.gz
export CONTAINERD_BUNDLE=containerd-1.6.28-nutanix.1-rhel-8.6-x86_64.tar.gz
```

2. Sample output after uploading the artifacts onto cluster hosts.

Example:

```
konvoy-image upload artifacts \
    --container-images-dir=./kib/artifacts/images/ \
    --os-packages-bundle=./kib/artifacts/${OS_PACKAGES_BUNDLE} \
    --containerd-bundle=./kib/artifacts/${CONTAINERD_BUNDLE} \
    --pip-packages-bundle=./kib/artifacts/pip-packages.tar.gz
```

Load Images to your Private Registry - Konvoy

Before creating or upgrading a Kubernetes cluster, you need to load the required images in a local registry if operating in an air-gapped environment. This registry must be accessible from both the Creating a Bastion Host on page 685 and either the Amazon Elastic Compute Cloud (Amazon EC2) instances or other machines that will be created for the Kubernetes cluster.

> **Important:** If you do not already have a local registry set up, refer to Local Registry Tools page for more information.

Execute the following command to load the air-gapped image bundle into your private registry:

```
nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-
registry=
$REGISTRY_URL --to-registry-username=$REGISTRY_USERNAME --to-registry-password=
$REGISTRY_PASSWORD
```

It may take some time to push all the images to your image registry, depending on the performance of the network between the machine you are running the script on and the registry.

Load Images to your Private Registry - Kommander

Load Kommander images to your Private Registry.

For the air-gapped `kommander` image bundle, run the `nkp push bundle` as shown in the example.

```
nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-
registry=
$REGISTRY_URL --to-registry-username=$REGISTRY_USERNAME --to-registry-password=
$REGISTRY_PASSWORD
```

Load Images to your Private Registry - NKP Catalog Applications

NKP Ultimate license requirement: Load the image bundle to your private registry.

Optional: This step is required only if you have an Nutanix Kubernetes Platform (NKP) Ultimate license.

For NKP Catalog Applications run the `nkp push bundle` command to load the `dkp-catalog-applications` image bundle into your private registry as shown in the example.

```
nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-
v2.12.0.tar --
to-registry=$REGISTRY_URL --to-registry-username=$REGISTRY_USERNAME --to-registry-
password=$REGISTRY_
PASSWORD
```

Next Step

Depending on your license type, you will follow the relevant link:

- Upgrade NKP Ultimate on page 1152
- Upgrade NKP Pro on page 1173

Upgrade NKP Ultimate

NKP Ultimate upgrade overview.

> **Note:** Before you begin the upgrade, ensure you review the Upgrade Prerequisites.

This section describes how to upgrade your Nutanix Kubernetes Platform (NKP) clusters to the latest NKP version in an Ultimate, multi-cluster, environment. The NKP upgrade represents an important step of your environment's life cycle, as it ensures that you are up-to-date with the latest features and can benefit from the most recent improvements, enhanced cluster management, and better performance. If you are on this scenario of the upgrade section of documentation, you have an Ultimate License. For an NKP Pro license, refer to the NKP Upgrade Pro section instead.

> **Important:** Ensure that all platform applications in the management cluster have been upgraded to avoid compatibility issues with the Kubernetes version included in this release. This is done automatically when upgrading Kommander, so ensure that you follow these sections in order to upgrade the Kommander component prior to upgrading the Konvoy component.

Overview of Steps

Preform the following steps in the order presented.

1. Ultimate: For Air-gapped Environments Only
2. Ultimate: Upgrade the Management Cluster and Platform Applications
3. Ultimate: Upgrade Platform Applications on Managed and Attached Clusters
4. Ultimate: Upgrade Workspace NKP Catalog Applications
5. Ultimate: Upgrade Project Catalog Applications
6. Ultimate: Upgrade Custom Applications
7. Ultimate: Upgrade the Management Cluster CAPI Components
8. Ultimate: Upgrade the Management Cluster Core Addons
9. Ultimate: Upgrade the Management Cluster Kubernetes Version
10. Ultimate: Upgrade Managed Clusters
11. Ultimate: Upgrade images used by Catalog Applications

Next Step:

- If you are in an air-gapped environment, first load your local registry. For more information see, Ultimate: For Air-gapped Environments Only.
- For all other environments, your next step is to Upgrade the Management Cluster and Platform Applications.

Ultimate: For Air-gapped Environments Only

If you are operating in an air-gapped environment, a local container registry containing all the necessary installation images, including the Kommander images is required. This section will guide you through how to push the necessary images to this registry. For more information on local contained registries, see Registry Mirror Tools on page 1069

- Download the complete NKP air-gapped bundle for this release (`nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`)
- Connectivity with clusters attaching to the management cluster:
  - Both management and attached clusters must be able to connect to the local registry.
  - The management cluster must be able to connect to all attached cluster's API servers.
  - The management cluster must be able to connect to any load balancers created for platform services on the attached cluster.

  Note: You can also attach clusters when there are networking restrictions between the management cluster and attached cluster. See Attach a Cluster with Networking Restrictions for more information.

Extracting Air-gapped Images and Set Variables

This procedure extracts the air-gapped image bundles into your private registry.

About this task

After you have downloaded the image to a local directory, perform these steps to extract the air-gapped image bundles into your private registry.

Procedure

1. Extract the tarball to a local directory using the command `tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`.
2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. EX: For the bootstrap, change your directory to the `nkp-version` directory similar to example below depending on your current location.

   ```
   cd nkp-v2.12.0
   ```

3. Set an environment variable with your registry address using the command `export REGISTRY_URL="https/http://registry-address:registry-port" export REGISTRY_USERNAME=username export REGISTRY_PASSWORD=password`.

Load Images to Your Private Registry

Examples of loading air-gapped images to your private registry.

The following sections show examples of loading images to your private registry for Kommander, Nutanix Kubernetes Platform (NKP) Catalog Applications, and Konvoy images.

Note: It may take some time to push all the images to your image registry, depending on the performance of the network between the machine you are running the script on and the registry.

Kommander

To load the air-gapped kommander image bundle to your private registry use the command `nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-registry=$REGISTRY_URL --to-registry-username=$REGISTRY_USERNAME --to-registry-password=$REGISTRY_PASSWORD`.

(Optional) NKP Catalog Applications

Important: This step is required only if you have an Ultimate license.

To load the NKP Catalog Applications image bundle to your private registry, use the command `nkp push bundle --bundle ./container-images/dkp-catalog-applications-image-bundle-v2.12.0.tar --to-registry=$REGISTRY_URL --to-registry-username=$REGISTRY_USERNAME --to-registry-password=$REGISTRY_PASSWORD`.

Konvoy

Before creating or upgrading a Kubernetes cluster, you need to load the required images in a local registry if operating in an air-gapped environment. This registry must be accessible from both the Bastion Host on page 1071 and either the AWS EC2 instances or other machines that will be created for the Kubernetes cluster.

Note: If you do not already have a local registry set up, refer to Registry Mirror Tools on page 1069 page for more information.

To load the air-gapped Konvoy image bundle to your private registry use the command `nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-registry=$REGISTRY_URL --to-registry-username=$REGISTRY_USERNAME --to-registry-password=$REGISTRY_PASSWORD`.

Ultimate: Upgrade the Management Cluster and Platform Applications

Describes the options to upgrade your Kommander Management cluster in air-gapped, non-air-gapped and on-premises environments.

This section describes how to upgrade your Kommander Management cluster and all Platform Applications to their supported versions in air-gapped, non-air-gapped and on-premises environments. To prevent compatibility issues, you must first upgrade Kommander on your Management Cluster before upgrading to Nutanix Kubernetes Platform (NKP) .

Note: It is important you upgrade Kommander BEFORE upgrading the Kubernetes version (or Konvoy version for Managed Konvoy clusters) in attached clusters. This ensures that any changes required for new or changed Kubernetes API's are already present.

Important: Authentication token changes were made. In previous releases, you used the same token against clusters attached to the management cluster. In this release, users will be logged out of attached clusters until the upgrade process is complete. The `kubeconfig` must then be retrieved from the endpoint and shared with all users of the attached clusters. The URL to download a new `kubeconfig` can be generated using the command:`kubectl -n kommander get svc kommander-traefik -o go-template='https://{{with index .status.loadBalancer.ingress 0}}{{or .hostname .ip}}{{end}}/token/plugin/kubeconfig{{ "\n"}}'`

Prerequisites

- Use the `--kubeconfig=${CLUSTER_NAME}.conf` flag or set the `KUBECONFIG` environment variable to ensure that you upgrade Kommander on the right cluster. For alternatives and recommendations around setting your context, refer to Provide Context for Commands with a kubeconfig File.

  Important: Limited availability of NKP resources

  - If you have configured a custom domain, running the `upgrade` command may result in an inaccessibility of your services through your custom domain for a few minutes.
  - The NKP UI and other Application Programming Interfaces (API) may be inconsistent or unavailable until the upgrade is complete.

Upgrade Kommander Examples

This section provides command-line interface (CLI) examples of Kommander environment upgrades.

Non-air-gapped Environments

To upgrade Kommander and all the Platform Applications in the Management Cluster use the command:

```
nkp upgrade kommander
```

Note: If you want to disable the Artificial Intelligence (AI) Navigator, add the flag `--disable-appdeployments ai-navigator-app` to this command.

Example output:

```
# Ensuring upgrading conditions are met
# Ensuring application definitions are updated
# Ensuring helm-mirror implementation is migrated to ChartMuseum
...
```

After the upgrade, if you have Nutanix Kubernetes Platform (NKP) Catalog Applications deployed, proceed to Update the NKP Catalog Applications GitRepository on page 1158.

Air-gapped Environments

To upgrade Kommander and all the Platform Applications in the Management Cluster use the command `nkp upgrade kommander \ --charts-bundle ./application-charts/nkp-kommander-charts-bundle-v2.12.1.tar.gz \ --kommander-applications-repository ./application-repositories/kommander-applications-v2.12.1.tar.gz`.

Example output:

```
# Ensuring upgrading conditions are met
# Ensuring application definitions are updated
# Ensuring helm-mirror implementation is migrated to ChartMuseum
...
```

After the upgrade, if you have Nutanix Kubernetes Platform (NKP) Catalog Applications deployed, proceed to Update the NKP Catalog Applications GitRepository on page 1158.

Air-gapped Environments with NKP Catalog Applications

To upgrade Kommander and all the Platform Applications in the Management Cluster use the command `nkp upgrade kommander \ --charts-bundle ./application-charts/nkp-kommander-charts-bundle-v2.12.1.tar.gz \ --charts-bundle ./application-charts/nkp-catalog-applications-charts-bundle-v2.12.1.tar.gz \ --kommander-applications-repository ./application-repositories/kommander-applications-v2.12.1.tar.gz`.

Example output:

```
# Ensuring upgrading conditions are met
# Ensuring application definitions are updated
# Ensuring helm-mirror implementation is migrated to ChartMuseum
...
```

After the upgrade, if you have NKP Catalog Applications deployed, proceed to Ultimate: Upgrade Workspace NKP Catalog Applications on page 1157.

Troubleshooting
Troubleshooting tips for upgrading Ultimate management cluster and platform applications.

If the upgrade fails, perform the troubleshooting commands as needed.

- If the upgrade fails, run the `nkp upgrade kommander -v 6` command to get more information on the upgrade process.
- If you find any `HelmReleases` in a “broken” release state such as “exhausted” or “another rollback/release in progress”, you can trigger a reconciliation of the `HelmRelease` using the command`kubectl -n kommander patch helmrelease HELMRELEASE_NAME --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease HELMRELEASE_NAME --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`.

Ultimate: Upgrade Platform Applications on Managed and Attached Clusters

For the Management Cluster (and the `kommander` workspace namespace), `nkp upgrade` handles all Platform applications; no other steps are necessary. However, for managed or attached clusters, you MUST manually upgrade the Platform applications by upgrading the workspace.

> **Important:** If you are upgrading your Platform applications as part of the NKP upgrade, upgrade your Platform applications on any additional Workspaces before proceeding with the Konvoy upgrade. Some applications in the previous release are not compatible with the Kubernetes version of this release, and upgrading Kubernetes is part of the Nutanix Kubernetes Platform (NKP) Konvoy upgrade process.

Prerequisites

Before you begin, you must:

- Set the `WORKSPACE_NAMESPACE` environment variable to the name of the workspace’s namespace where the cluster is attached using the command `export WORKSPACE_NAMESPACE=workspace_namespace`.
- Set the `WORKSPACE_NAME` environment variable to the name of the workspace where the cluster is attached using the command `export WORKSPACE_NAME=workspace_name`

Use CLI to Upgrade Platform Applications
CLI to upgrade all platform applications in a workspace and its projects to the same version as the platform applications running on the management cluster.

To upgrade all platform applications in a workspace and its projects to the same version as the platform applications running on the management cluster use the command `nkp upgrade workspace $WORKSPACE_NAME`

Example output:

```
# Ensuring HelmReleases are upgraded on clusters in namespace "$WORKSPACE_NAME"
...
```

Troubleshooting

If the upgrade fails or times out, retry the command with later verbosity to get more information on the upgrade process:

- If the upgrade fails, run the `nkp upgrade kommander -v 6` command to get more information on the upgrade process.
- If you find any `HelmReleases` in a “broken” release state such as “exhausted” or “another rollback/release in progress”, you can trigger a reconciliation of the `HelmRelease` using the command`kubectl -n kommander patch helmrelease HELMRELEASE_NAME --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true}]' kubectl -n kommander patch helmrelease HELMRELEASE_NAME --type='json' -p='[{"op": "replace", "path": "/spec/suspend", "value": false}]'`.

Ultimate: Upgrade Workspace NKP Catalog Applications

Upgrade catalog applications using the command-line interface (CLI) and UI

Considerations for Upgrading NKP Catalog Applications with Spark Operator Post NKP 2.7

> **Important:** Starting in Nutanix Kubernetes Platform (NKP) 2.7, the Spark operator app is removed, resulting in the complete uninstallation of the app after upgrading the NKP Catalog Applications GitRepository.
>
> This section provides instructions on how you can continue using Spark in NKP 2.7 and after.
>
> If you do not plan on using Spark or if you’re content with having Spark be uninstalled automatically, you can skip this section and proceed to the Update the NKP Catalog Applications GitRepository on page 1158

Orphaning Existing Instances of Spark so that it is Unmanaged by NKP or Flux

About this task

Context for the current task

Procedure

1. Set the `WORKSPACE_NAMESPACE` variable to the namespace where the spark-operator is deployed using the comman `export WORKSPACE_NAMESPACE=<WORKSPACE_NAMESPACE>`
2. Set `suspend: true` on the cluster Kustomization.

   Example output:

   ```
   kubectl -n ${WORKSPACE_NAMESPACE} patch kustomization cluster --type='json' -
   p='[{"op":
    "replace", "path": "/spec/suspend", "value": true}]'
   ```

   > **Note:** This needs to be reverted in the last step.

3. Set `suspend: true` and `prune: false` on the `spark-operator` Kustomizations

   Example output:

   ```
   export SPARK_OPERATOR_APPDEPLOYMENT_NAME=<spark operator AppDeployment name>
   export SPARK_OPERATOR_APPDEPLOYMENT_VERSION=<spark operator AppDeployment version>
   kubectl -n ${WORKSPACE_NAMESPACE} patch kustomization
    ${SPARK_OPERATOR_APPDEPLOYMENT_NAME} --type=
   'json' -p='[{"op": "replace", "path": "/spec/suspend", "value": true},{"op":
    "replace", "path":
   "/spec/prune", "value": false}]'
   ```

```
kubectl -n ${WORKSPACE_NAMESPACE} patch kustomization spark-operator-
${SPARK_OPERATOR_APPDEPLOYMENT_VERSION}-defaults --type='json' -p='[{"op": "replace",
 "path": "/spec/suspend", "value": true},{"op": "replace", "path": "/spec/prune",
 "value": false}]'
```

4. Update the `dkp-catalog-applications` GitRepository with the new version,
5. Unsuspend the cluster Kustomization using the command `kubectl -n ${WORKSPACE_NAMESPACE} patch kustomization cluster --type='json' -p='[{"op" : "replace", "path": "/spec/suspend", "value": false}]'`

> **Note:** If the Kustomization is left suspended, Kommander will be unable to function properly.

Removing Spark from the UI

About this task

Follow these steps to remove the `spark-operator` AppDeployment from the Nutanix Kubernetes Platform (NKP) UI:

Procedure

1. From the top menu bar, select your target workspace.
2. Select **Applications** from the sidebar menu.
3. Select the three dot menu from the bottom-right corner of the Spark application tile, and then select **Uninstall**.
4. click **Save**.

Removing Spark with CLI

About this task

Context for the current task

Procedure

1. Execute the following command to get the `WORKSPACE_NAMESPACE` of your workspace using the command `nkp get workspaces`.

   Copy the values under the `NAMESPACE` column for your workspace.
2. Export the `WORKSPACE_NAMESPACE` variable using the command `export WORKSPACE_NAMESPACE=<WORKSPACE_NAME>`
3. To delete the `spark-operator` AppDeployment use the command `kubectl delete AppDeployment <spark operator appdeployment name> -n ${WORKSPACE_NAMESPACE}`
   This results in the `spark-operator` Kustomizations being deleted, but the `HelmRelease` and default `ConfigMap` remains in the cluster. From here, you can continue to manage the `spark-operator` through the `HelmRelease`.

Update the NKP Catalog Applications GitRepository

Follow the instructions in this section to update the GitRepository `dkp-catalog-applications`.

Note: When upgrading from DKP 2.8, use the catalog name `dkp-catalog-applications` while running your catalog repository upgrade commands instead of the catalog name or GitRepository name `nkp-catalog-applications` because you are updating an existing `dkp-catalog-applications` named resource. For example:

```
kubectl patch gitrepository -n kommander dkp-catalog-applications --type merge
 --patch
        '{"spec": {"ref":{"tag":"v2.12.0"}}}'
```

Updating the GitRepository in an Air-gapped Environments

Instructions to update the GitRepository in an air-gapped environment.

About this task

Follow the instructions to update the NKP Catalog Applications GitRepository.

Note: Ensure you modify the most recent `kommander.yaml` configuration file. It must be the file that reflects the current state of your environment. Reinstalling Kommander with an outdated `kommander.yaml` overwrites the list of platform applications that are currently running in your cluster.

Procedure

1. In the `kommander.yaml` you are currently using for your environment, update the NKP Catalog Applications by setting the correct NKP version.

   Example output:

   ```
   ...
   # The list of enabled/disabled apps here should reflect the current state of the
    environment, including
   configuration overrides!
   ...
   catalog:
     repositories:
       - name: dkp-catalog-applications
         labels:
           kommander.nutanix.io/project-default-catalog-repository: "true"
           kommander.nutanix.io/workspace-default-catalog-repository: "true"
           kommander.nutanix.io/gitapps-gitrepository-type: "nkp"
         path: ./nkp-catalog-applications-v2.12.1.tar.gz # modify this version to match
    the nkp upgrade version
   ...
   ```

2. Refresh the `kommander.yaml` to apply the updated tarball using the command `nkp install kommander --installer-config kommander.yaml`.

   Note: Ensure the `kommander.yaml` is the Kommander Installer Configuration file you are currently using for your environment. Otherwise, your configuration will be overwritten and previous configuration lost.

Updating the GitRepository in a Non-air-gapped Environments

Instructions to update the GitRepository in an non-air-gapped environment.

About this task

Upgrade the workspace Catalog Applications in `kommander` on each additional workspace on which catalog apps are deployed. In air-gapped scenarios, the images are included in the air-gapped bundle. In non-air-gapped scenarios, refresh the Git Repository as shown below.

Procedure

1. Update the GitRepository with the tag of your updated NKP version on the `kommander` workspace using the command `kubectl patch gitrepository`.

   Example:

   ```
   kubectl patch gitrepository -n kommander nkp-catalog-applications --type merge --
   patch '{"spec": {"ref":{"tag":"v2.12.0"}}}'
   ```

   > **Note:** This command updates the catalog application repositories for all workspaces.

2. For any additional Catalog `GitRepository` resources created outside the `kommander.yaml` configuration (e.g. such as in a project or workspace namespace), set the `WORKSPACE_NAMESPACE` environment variable to the namespace of the workspace.

   Example:

   ```
   export WORKSPACE_NAMESPACE=<workspace namespace>
   ```

3. Update the GitRepository for the additional workspace using the command `kubectl patch gitrepository`.

   Example:

   ```
   kubectl patch gitrepository -n ${WORKSPACE_NAMESPACE} nkp-catalog-applications --type
    merge --patch '{"spec":{"ref":{"tag":"v2.12.1"}}}'
   ```

Upgrade Catalog Applications

Before upgrading also, keep in mind the distinction between Platform applications and Catalog applications. Platform applications are deployed and upgraded as a set for each cluster or workspace. Catalog applications are deployed separately, so that you can deploy and upgrade them individually for each project.

Upgrading with UI

About this task

Follow these steps to upgrade an application from the Nutanix Kubernetes Platform (NKP) UI:

Procedure

1. From the top menu bar, select your target workspace.
2. Select **Applications** from the sidebar menu.
3. Select the three dot menu from the bottom-right corner of the desired application tile, and then select **Edit**.
4. Select the **Version** dropdown list, and select a new version. This dropdown list will only be available if there is a newer version to upgrade to.
5. click **Save**.

Upgrading with CLI

About this task

Follow these steps to upgrade an application from the Nutanix Kubernetes Platform (NKP) UI.

> **Note:** The following commands are using the workspace **name** and not namespace.
>
> - You can retrieve the workspace name by running the command `nkp get workspaces`.
> - To view a list of the deployed apps to your workspace, run the command `nkp get appdeployments --workspace=`.

Procedure

1. To see what app(s) and app versions are available to upgrade, use the command `kubectl get apps -n ${WORKSPACE_NAMESPACE}`

   > **Note:** You can reference the app version by going into the app name (e.g. `<APP ID>-<APP VERSION>`)

   You can also use this command to display the apps and app versions, for example:

   ```
   kubectl get apps -n ${WORKSPACE_NAMESPACE} -o jsonpath='{range .items[*]}
   {@.spec.appId}
   {"----"}{@.spec.version}{"\n"}{end}'
   ```

   Example output:

   ```
   kafka-operator----0.20.0
   kafka-operator----0.20.2
   kafka-operator----0.23.0-dev.0
   zookeeper-operator----0.2.13
   zookeeper-operator----0.2.14
   ```

2. To upgrade an application from the NKP CLI use the command `nkp upgrade catalogapp <appdeployment-name> --workspace=${WORKSPACE_NAME} --to-version=<version.number>`.

   The output example upgrades the Kafka Operator application, named `kafka-operator-abc`, in a workspace to version `0.25.1`:

   ```
   nkp upgrade catalogapp kafka-operator-abc --workspace=${WORKSPACE_NAME} --to-
   version=0.25.1
   ```

3. Repeat this process for on each workspace where you have deployed the application.

> **Note:** Platform applications cannot be upgraded on a one-off basis, and must be upgraded in a single process for each workspace. If you attempt to upgrade a platform application with these commands, you receive an error and the application is not upgraded.

> **Important:** As of NKP 2.7, these are the supported versions of Catalog application, all other have been deprecated:
>
> - `kafka-operator-0.25.1`
> - `zookeeper-operator-0.2.16-nkp.1`
>
> If you plan on upgrading to NKP 2.7 or later, ensure that you upgrade these applications to the latest compatible version.
>
> - To find what versions of applications are available for upgrade, use the command `kubectl get apps -n ${WORKSPACE_NAMESPACE}` :
>
> For more information, see Downloading NKP on page 16.

> **Important:** To ensure you do not install images with known CVEs, specify a custom image for `kafka` and `zookeeper` by following these instructions:
>
> - For kafka: Kafka Operator in a Workspace on page 422
> - For zookeeper: Zookeeper Operator in Workspace on page 423

Ultimate: Upgrade Project Catalog Applications

Before upgrading your catalog applications, verify the current and supported versions of the application. Also, keep in mind the distinction between Platform applications and Catalog applications. Platform applications are deployed and upgraded as a set for each cluster or workspace. Catalog applications are deployed separately, so that you can deploy and upgrade them individually for each project.

> **Note:** Catalog applications must be upgraded to the latest version BEFORE upgrading the Konvoy components for Managed clusters or Kubernetes version for attached clusters.

Upgrading with UI

About this task

Context for the current task

Procedure

1. From the top menu bar, select your target workspace.
2. From the side menu bar, select **Projects**.
3. Select your target project.
4. Select Applications from the project menu bar.
5. Select the three dot menu from the bottom-right corner of the desired application tile, and then select **Edit**.

6. Select the **Version** dropdown list, and select a new version. This dropdown list will only be available if there is a newer version to upgrade to.
7. click **Save**.

Upgrading with CLI

About this task

Context for the current task

Procedure

1. To see what app(s) and app versions are available to upgrade use the command `kubectl get apps -n ${PROJECT_NAMESPACE}`.

   > **Note:** You can reference the app version by going into the app name (e.g. <APP ID>-<APP VERSION>)

   You can also use this command to display the apps and app versions, for example:

   ```
   kubectl get apps -n ${PROJECT_NAMESPACE} -o jsonpath='{range .items[*]}{@.spec.appId}{"----"}{@.spec.version}{"\n"}{end}'
   ```

   Example output:

   ```
   zookeeper-operator----0.2.13
   zookeeper-operator----0.2.14
   zookeeper-operator----0.2.15
   ```

2. Run the following command to upgrade an application from the Nutanix Kubernetes Platform (NKP) CLI using the command `nkp upgrade catalogapp <appdeployment-name> --workspace=my-workspace --project=my-project --to-version=<version.number>`

   In the following example shows the upgrade of the Zookeeper Operator application, named `zookeeper-operator-abc`, in a workspace to version `0.2.15`:

   ```
   nkp upgrade catalogapp zookeeper-operator-abc --workspace=my-workspace --to-version=0.2.15
   ```

   > **Note:** Platform applications cannot be upgraded on a one-off basis, and must be upgraded in a single process for each workspace. If you attempt to upgrade a platform application with these commands, you receive an error and the application is not upgraded.

Ultimate: Upgrade Custom Applications

Verify the compatibility of Custom Applications with the current Kubernetes version

We recommend upgrading your Custom Applications to the latest compatible version as soon as possible. Since Custom Applications are not created, maintained or supported by Nutanix, you must upgrade them manually.

> **Note:** Ensure you validate any Custom Applications you run for compatibility issues against the Kubernetes version in the new release. If the Custom Application's version is not compatible with the Kubernetes version, do not continue with the Konvoy upgrade. Otherwise, your custom Applications may stop running.

After you have ensured your Custom Applications are compatible with the current Kubernetes version, return to the Upgrade NKP on page 1144 documentation, to review the next steps depending on your environment and license type.

Ultimate: Upgrade the Management Cluster CAPI Components

Upgrade the CAPI Components on the Management cluster.

> **Note:** For a Pre-provisioned air-gapped environment only, ensure you have reviewed: Ultimate: For Air-gapped Environments Only on page 1153.

Upgrade the CAPI Components

New versions of Nutanix Kubernetes Platform (NKP) come pre-bundled with newer versions of CAPI, newer versions of infrastructure providers, or new infrastructure providers. When using a new version of the NKP CLI, upgrade all of these components first.

If you are running on more than one management cluster, you must upgrade the CAPI components on each of these clusters.

> **Note:** Ensure your NKP configuration references the management cluster where you want to run the upgrade by setting the `KUBECONFIG` environment variable, or using the `--kubeconfig` flag, in accordance with Kubernetes conventions

Execute the upgrade command for the CAPI components.

> **Note:** If you created CAPI components using flags to specify values, use those same flags during Upgrade to preserve existing values while setting additional values.
>
> - Refer to nkp create cluster aws CLI for flag descriptions for `--with-aws-bootstrap-credentials` and `--aws-service-endpoints`
> - Refer to the HTTP section for details: Clusters with HTTP or HTTPS Proxy on page 680
>
> ```
> nkp upgrade capi-components
> ```
>
> The output resembles the following:
>
> ```
> # Upgrading CAPI components
> # Waiting for CAPI components to be upgraded
> # Initializing new CAPI components
> ```

If the upgrade fails, review the prerequisites section and ensure that you've followed the steps in the Upgrade NKP on page 1144 overview. Furthermore, ensure you have adhered to the Prerequisites at the top of this page.

Ultimate: Upgrade the Management Cluster Core Addons

Upgrade the Core Addons on the Management Cluster.

For Pre-provisioned air-gapped environments only, you must run konvoy-image upload artifacts to copy the artifacts onto the cluster hosts before you begin the Upgrade the CAPI Components section below.

```
konvoy-image upload artifacts \
    --container-images-dir=./artifacts/images/ \
    --os-packages-bundle=./artifacts/$OS_PACKAGES_BUNDLE \
    --containerd-bundle=artifacts/$CONTAINERD_BUNDLE \
    --pip-packages-bundle=./artifacts/pip-packages.tar.gz
```

To install the core addons, Nutanix Kubernetes Platform (NKP) relies on the `ClusterResourceSet` Cluster API feature. In the CAPI component upgrade, we deleted the previous set of outdated global `ClusterResourceSet` because in past releases, some addons were installed using a global configuration. In order to support individual cluster upgrades, NKP now installs all addons with a unique set of `ClusterResourceSet` and corresponding referenced resources, all named using the cluster's name as a suffix. For example: `calico-cni-installation-my-aws-cluster`.

Note: If you have modified any of the `ClusterResourceSet` definitions, these changes will not be preserved when running the command `nkp upgrade addons <provider>`. You must define the cloud provider before you use the `--dry-run -o yaml` options to save the new configuration to a file and remake the same changes upon each upgrade.

Your cluster comes preconfigured with a few different core addons that provide functionality to your cluster upon creation. These include: CSI, Container Network Interface (CNI), Cluster Autoscaler, and Node Feature Discovery. New versions of NKP may come pre-bundled with newer versions of these addons.

Perform the following steps to update these addons:

1. If you have any additional managed clusters, you will need to upgrade the core addons and Kubernetes version for each one.
2. Ensure your NKP configuration references the management cluster where you want to run the upgrade by setting the `KUBECONFIG` environment variable, or using the `--kubeconfig` flag, in accordance with Kubernetes conventions.
3. Upgrade the core addons in a cluster using the `nkp upgrade addons` command specifying the cluster infrastructure (choose `aws`, `azure`, `vsphere`,`vcd`, `eks`, `gcp`, `preprovisioned`) and the name of the cluster.

   Note: If you need to verify or discover your cluster name to use with this example, first run the `kubectl get clusters` command.

Examples for upgrade core addons commands:

```
export CLUSTER_NAME=my-azure-cluster
nkp upgrade addons azure --cluster-name=${CLUSTER_NAME}
```

or

```
export CLUSTER_NAME=my-aws-cluster
nkp upgrade addons aws --cluster-name=${CLUSTER_NAME}
```

AWS example output:

```
Generating addon resources
clusterresourceset.addons.cluster.x-k8s.io/calico-cni-installation-my-aws-cluster
 upgraded
configmap/calico-cni-installation-my-aws-cluster upgraded
clusterresourceset.addons.cluster.x-k8s.io/tigera-operator-my-aws-cluster upgraded
configmap/tigera-operator-my-aws-cluster upgraded
clusterresourceset.addons.cluster.x-k8s.io/aws-ebs-csi-my-aws-cluster upgraded
configmap/aws-ebs-csi-my-aws-cluster upgraded
clusterresourceset.addons.cluster.x-k8s.io/cluster-autoscaler-my-aws-cluster upgraded
configmap/cluster-autoscaler-my-aws-cluster upgraded
clusterresourceset.addons.cluster.x-k8s.io/node-feature-discovery-my-aws-cluster
 upgraded
configmap/node-feature-discovery-my-aws-cluster upgraded
clusterresourceset.addons.cluster.x-k8s.io/nvidia-feature-discovery-my-aws-cluster
 upgraded
configmap/nvidia-feature-discovery-my-aws-cluster upgraded
```

Additional References:

For more information, see:

- NKP upgrade addons: https://d2iq.atlassian.net/wiki/spaces/DENT/pages/29923443/dkp+upgrade+addons for more CLI command help.
- Cluster API feature:https://cluster-api.sigs.k8s.io/
- Kubernetes conventions:https://kubernetes.io/docs/tasks/access-application-cluster/configure-access-multiple-clusters/

Ultimate: Upgrading the Management Cluster Kubernetes Version

Upgrade the Kubernetes Version

About this task

Complete the follow steps to upgrade the Kubernetes version of a cluster:

> **Note:** If you have FIPS clusters you should note the additional considerations for FIPS if using FIPS configuration:

Procedure

1. Upgrade the control plane using the infrastructure specific command.
2. Upgrade the node pools using the infrastructure specific command.
3. If you have any additional managed clusters, you need to upgrade the core addons and Kubernetes version for each one. Attached clusters need the Kubernetes version upgraded to a supported Nutanix Kubernetes Platform (NKP) version using the tool that created that cluster.
4. During an upgrade, you need to create new AMI's or images with the Kubernetes version to which you want to upgrade which requires selecting your OS and ensuring you have the current supported version. Build a new image if applicable.

   - If an AMI was specified when initially creating a cluster for AWS, you must build a new one with Create a Custom AMI on page 1091 and set the flag(s) in the update commands. Either AMI ID`--ami AMI_ID`, or the lookup image flags: `--ami-owner AWS_ACCOUNT_ID`, `--ami-base-os ubuntu-20.04`, and `--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*'`.

     > **Important:**
     >
     > The AMI lookup method will return an error if the lookup uses the upstream CAPA account ID.

   - If an Azure Machine Image was specified for Azure, you must build a new one Using KIB with Azure on page 1097 .
   - If a vSphere template Image was specified for vSphere, you must build a new one Using KIB with vSphere on page 1104.
   - You must build a new GCP image Using KIB with GCP on page 1100.

5. Upgrade the Kubernetes version of the control plane. Each cloud provider has distinctive commands. Select the drop down list next to your provider for compliant CLI.

   AWS Example:

   ```
   nkp update controlplane aws --cluster-name=${CLUSTER_NAME} --kubernetes-
   version=v1.28.7
   ```

   OR for AMI ID lookup Example:

   > **Note:** If you created your initial cluster with a custom AMI using the `--ami` flag, it is required to set the `--ami` flag during the Kubernetes upgrade.

   ```
   nkp update controlplane aws \
   --cluster-name=${CLUSTER_NAME} \
   --ami-owner AWS_ACCOUNT_ID \
   --ami-base-os ubuntu-20.04 \
   --ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*' \
   ```

```
--kubernetes-version=v1.29.6
```

Azure Example:

```
nkp update controlplane azure --cluster-name=${CLUSTER_NAME} --kubernetes-
version=v1.29.6 --
compute-gallery-id <Azure Compute Gallery built by KIB for Kubernetes v1.29.6>
```

If the `--plan-offer`, `--plan-publisher` and `--plan-sku` fields were specified in the override file during image creation, the flags must be used in upgrade:

```
Example:
--plan-offer rockylinux-9
--plan-publisher erockyenterprisesoftwarefoundationinc1653071250513
--plan-sku rockylinux-9
```

vSphere Example:

```
nkp update controlplane vsphere --cluster-name=${CLUSTER_NAME} --kubernetes-
version=v1.29.6
--vm-template <vSphere template built by KIB for Kubernetes v1.29.6>
```

VCD Example:

```
nkp update controlplane vcd --cluster-name=${CLUSTER_NAME} --kubernetes-
version=v1.29.6
--catalog <tenant catalog image> --vapp-template <vApp template built in vSphere KIB
 for Kubernetes
v1.29.6>
```

GCP Example:

```
nkp update controlplane gcp --cluster-name=${CLUSTER_NAME} --kubernetes-
version=v1.29.6 --
image=projects/${GCP_PROJECT}/global/images/<GCP image built by KIB for Kubernetes
 v1.29.6>
```

Pre-Provisioned Example:

```
nkp update controlplane preprovisioned --cluster-name=${CLUSTER_NAME} --kubernetes-
version=v1.29.6
```

EKS Example:

```
nkp update controlplane eks --cluster-name=${CLUSTER_NAME} --kubernetes-
version=v1.26.12
```

Additional Considerations for upgrading a FIPS cluster:

If upgrading a FIPS cluster, to correctly upgrade the Kubernetes version using the command `nkp update controlplane aws --cluster-name=${CLUSTER_NAME} --kubernetes-version=v1.29.6+fips. 0 --ami=<ami-with-fips-id>`

Example output which includes the etcd FIPS information:

```
Updating control plane resource controlplane.cluster.x-k8s.io/v1beta1,
 Kind=KubeadmControlPlane
default/my-aws-cluster-control-plane
Waiting for control plane update to finish.
```

```
# Updating the control plane
```

> **Note:** Some advanced options are available for various providers. To see all the options for your particular provider use the command `nkp update controlplane aws|vsphere|preprovisioned|azure|gcp|eks --help`
>
> For more advance options like this example for AWS AMI instance type: `aws: --ami, --instance-type` includes some of the options mentioned.
>
> > **Tip:** The command `nkp update controlplane {provider}` has a 30 minute default timeout for the update process to finish. If you see the error " `timed out waiting for the condition`", you can check the control plane nodes version using the command `kubectl get machines -o wide $KUBECONFIG` before trying again.

6. Upgrade the Kubernetes version of your node pools. Upgrading a nodepool involves draining the existing nodes in the nodepool and replacing them with new nodes. In order to ensure minimum downtime and maintain high availability of the critical application workloads during the upgrade process, we recommend deploying Pod Disruption Budget (Disruptions) for your critical applications. For more information, see Updating Cluster Node Pools on page 1080.

   a. List all node pools available in your cluster by using the command `nkp get nodepool --cluster-name ${CLUSTER_NAME}`.

   b. Select the nodepool you want to upgrade using the command `export NODEPOOL_NAME=my-nodepool`.

   c. Update the selected nodepool using the command for your cloud provider as shown in the examples. The first example command shows AWS language, so select the dropdown list for your provider for the correct command. Execute the `update` command for each of the node pools listed in the previous command.

   AWS Example:

   If you created your initial cluster with a custom AMI using the `--ami` flag, it is required to set the `--ami` flag during the Kubernetes upgrade.

   ```
   nkp update nodepool aws ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
   kubernetes-version=v1.29.6
   ```

   Azure Example:

   ```
   nkp update nodepool azure ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
   kubernetes-version=v1.29.6 --compute-gallery-id <Azure Compute Gallery built by
    KIB for
   Kubernetes v1.29.6>
   ```

   If the `--plan-offer`, `--plan-publisher` and `--plan-sku` fields were specified in the override file during image creation, the flags must be used in upgrade:

   ```
   Example:
   --plan-offer rockylinux-9
   --plan-publisher erockyUltimatesoftwarefoundationinc1653071250513
   --plan-sku rockylinux-9
   ```

   vSphere Example:

   ```
   nkp update nodepool vsphere ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
   kubernetes-version=v1.29.6 --vm-template <vSphere template built by KIB for
    Kubernetes v1.29.6>
   ```

   VCD Example:

   ```
   nkp update nodepool vcd ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
   kubernetes-version=v1.29.6
   ```

```
--catalog <tenant catalog image> --vapp-template <vApp template built in vSphere
 KIB for Kubernetes v1.29.6>
```

GCP Example:

```
nkp update nodepool gcp ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
kubernetes-version=v1.29.6
--image=projects/${GCP_PROJECT}/global/images/<GCP image built by KIB for
 Kubernetes v1.29.6>
```

Pre-Provisioned Example:

```
nkp update nodepool preprovisioned ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME}
 --kubernetes-version=v1.29.6
```

EKS Example:

```
nkp update nodepool eks ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
kubernetes-version=v1.26.12
```

Output example showing the name of the infrastructure provider shown accordingly:

```
Updating node pool resource cluster.x-k8s.io/v1beta1, Kind=MachineDeployment
 default/my-aws-cluster-my-nodepool
Waiting for node pool update to finish.
 # Updating the my-aws-cluster-my-nodepool node pool
```

d. Repeat this step for each additional node pool.

> **Note:** Additional Considerations for upgrading a FIPS cluster:
>
> If upgrading a FIPS cluster, to correctly upgrade the Kubernetes version use the command `nkp update nodepool aws ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --kubernetes-version=v1.29.6+fips.0 --ami= <ami-with-fips-id>`
>
> When all nodepools have been updated, your upgrade is complete. For the overall process for upgrading to the latest version of NKP, refer back to Upgrade NKP on page 1144 for more details.

Ultimate: Upgrading Managed Clusters

Upgrade Managed Clusters

About this task

If you have managed clusters, follow these steps to upgrade each cluster similar to the management cluster process order:

Procedure

1. Using the kubeconfig of your management cluster, find your cluster name and be sure to copy the information for all of your clusters.

   Example:

   ```
   kubectl get clusters -A
   ```

2. Set your cluster variable using the command `export CLUSTER_NAME=<your-managed-cluster-name>`.

3. Set your cluster's workspace variable using the command `export CLUSTER_WORKSPACE=<your-workspace-namespace>`.

4. Then, upgrade the core addons (replacing `aws` with whatever infrastructure provider you are using) the command `nkp upgrade addons aws --cluster-name=${CLUSTER_NAME} -n ${CLUSTER_WORKSPACE}`

Upgrading Kubernetes Version on a Managed Cluster

About this task

After you complete the previous steps for all managed clusters and you update your core addons, begin upgrading the Kubernetes version.

> **Note:** First complete the upgrade of your Kommander Management Cluster before upgrading any managed clusters.

Procedure

1. To begin upgrading the Kubernetes version use the command `nkp update controlplane`. Review the example that applies to your cloud provider.

   AWS Example:

   ```
   nkp update controlplane aws --cluster-name=${CLUSTER_NAME} --kubernetes-
   version=v1.29.6 -n
    ${CLUSTER_WORKSPACE}
   ```

   EKS Example:

   ```
   nkp update nodepool eks ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --kubernetes-
   version=v1.28.8 -n ${CLUSTER_WORKSPACE}
   ```

   Azure Example:

   ```
   nkp update controlplane azure --cluster-name=${CLUSTER_NAME} --kubernetes-
   version=v1.29.6
   --compute-gallery-id <Azure Compute Gallery built by KIB for Kubernetes v1.29.6> -n
    ${CLUSTER_WORKSPACE}
   ```

   vSphere Example:

   ```
   nkp update controlplane vsphere --cluster-name=${CLUSTER_NAME} --kubernetes-
   version=v1.29.6
   --vm-template <vSphere template built by KIB for Kubernetes v1.29.6> -n
    ${CLUSTER_WORKSPACE}
   ```

   VCD Example:

   ```
   nkp update controlplane vcd --cluster-name=${CLUSTER_NAME} --kubernetes-
   version=v1.29.6 --catalog <tenant
    catalog image> --vapp-template <vApp template built in vSphere KIB for Kubernetes
    v1.29.6> -n ${CLUSTER_WORKSPACE}
   ```

   GCP Example:

   ```
   nkp update controlplane gcp --cluster-name=${CLUSTER_NAME} --kubernetes-
   version=v1.29.6 --image=projects/${
   GCP_PROJECT}/global/images/<GCP image built by KIB for Kubernetes v1.29.6> -n
    ${CLUSTER_WORKSPACE}
   ```

   Pre-provisioned Example:

   ```
   nkp update controlplane preprovisioned --cluster-name=${CLUSTER_NAME} --kubernetes-
   version=v1.29.6 -n
   ${CLUSTER_WORKSPACE}
   ```

2. Get a list of all node pools available in your cluster by running the command `nkp get nodepools -c ${CLUSTER_NAME} -n ${CLUSTER_WORKSPACE} export NODEPOOL_NAME=`.

3. To upgrade the node pools use the command `nkp update nodepool` for your cloud provider.

   AWS Example:

   ```
   nkp update nodepool aws ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --kubernetes-
   version=v1.29.6 -n ${CLUSTER_WORKSPACE}
   ```

   EKS Example:

   ```
   nkp update nodepool eks ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --kubernetes-
   version=v1.25.11 -n ${CLUSTER_WORKSPACE}
   ```

   Azure Example:

   ```
   nkp update nodepool azure ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
   kubernetes-
   version=v1.29.6 --compute-gallery-id <Azure Compute Gallery built by KIB for
    Kubernetes v1.29.6
   > -n ${CLUSTER_WORKSPACE}
   ```

   vSphere Example:

   ```
   nkp update nodepool vsphere ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
   kubernetes-
   version=v1.29.6 --vm-template <vSphere template built by KIB for Kubernetes v1.29.6>
    -n ${
   CLUSTER_WORKSPACE}
   ```

   VCD Example:

   ```
   nkp update nodepool vcd ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --kubernetes-
   version=v1.29.6 --catalog <tenant catalog image> --vapp-template <vApp template built
    in vSphere KIB
    for Kubernetes v1.29.6>  -n ${CLUSTER_WORKSPACE}
   ```

   GCP Example:

   ```
   nkp update nodepool gcp ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --kubernetes-
   version
   =v1.29.6 --image=projects/${GCP_PROJECT}/global/images/<GCP image built by KIB for
    Kubernetes v1.29.6
   > -n ${CLUSTER_WORKSPACE}
   ```

   Pre-provisioned Example:

   ```
   nkp update nodepool preprovisioned ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
   kubernetes
   -version=v1.29.6 -n ${CLUSTER_WORKSPACE}
   ```

What to do next

Upgrade Attached Cluster

Since any attached clusters you have are managed by their corresponding cloud provider, none of the components are upgraded using the Nutanix Kubernetes Platform (NKP) process. The tool used to create the cluster is used to upgrade it. Ensure that it has the Kubernetes version that is compatible with the versions we support.

Ultimate: Upgrade images used by Catalog Applications

To reduce the risk caused by vulnerabilities present in older Kafka and Zookeeper images, you need to upgrade image references in the `ZookeeperCluster` and `KafkaCluster` custom resources on attached clusters.

Note that you must perform this step after completing your upgrade of Nutanix Kubernetes Platform (NKP) . Perform this step after upgrading the `zookeeper-operator` and `kafka-operator` applications themselves.

Note that these images can be upgraded individually, on a cluster-by-cluster basis.

Upgrading a Zookeeper Image Reference in a Zookeeper Cluster Resource

About this task

The Nutanix Kubernetes Platform (NKP) catalog in 2.12 includes an updated Zookeeper 3.7.2 image, which does not contain CVEs present in earlier versions. This image is hosted at `ghcr.io/mesosphere/zookeeper:0.2.15-dkp`. To update an existing Zookeeper cluster to use this image, perform the following steps:

Procedure

1. Ensure that your workloads that use this Zookeeper cluster are compatible with Zookeeper 3.7.2.
2. Set the spec.image.repository and spec.image.tag fields of each ZookeeperCluster custom resource to reference the new image. For example, you can usekubectl patch to upgrade the image in a ZookeeperCluster named zk-3 that has been manually created in a namespace ws-1 as follows.

   ```
   kubectl patch zookeepercluster -n ws-1 zk-3 --type='json' -p='[{"op": "replace",
    "path":
    "/spec/image/repository", "value": "ghcr.io/mesosphere/zookeeper"}, {"op":
    "replace", "path":
    "/spec/image/tag", "value": "0.2.15-dkp"}]'
   ```

What to do next

For more information, refer to Zookeeper Operator documentation: https://github.com/pravega/zookeeper-operator/blob/master/README.md#trigger-the-upgrade-manually

Upgrading the Kafka image reference in a KafkaCluster resource

About this task

The Nutanix Kubernetes Platform (NKP) catalog in 2.12 includes an updated Kafka 3.4.1 image, which does not contain CVEs present in earlier versions. This image is hosted at `ghcr.io/banzaicloud/kafka:2.13-3.4.1`. To update an existing `KafkaCluster`to use this image, perform the following steps:

Procedure

1. Ensure that all workloads that use this Kafka cluster are compatible with this version of Kafka.
2. Identify the current version of Kafka, which is specified in the `.spec.clusterImage` field of the `KafkaCluster` resource.
3. Ensure that the `spec.readOnlyConfig` field of the `KafkaCluster` contains a `inter.broker.protocol.version=X.Y.Z` line, where `X.Y.Z` stands for the current version of Kafka, and no other lines setting this value are present in the `KafkaCluster` resource. This ensures all the brokers in the cluster can communicate with each other as the cluster is upgraded using the specified protocol version.

   For example, if you use the `kubectl edit` command to perform this step on a `KafkaCluster` currently using the `ghcr.io/banzaicloud/kafka:2.13-2.8.1` image.

   KafkaCluster manifest example output:

   ```
   apiVersion: kafka.banzaicloud.io/v1beta1
   ```

```
...
spec:
  clusterImage: "ghcr.io/banzaicloud/kafka:2.13-2.8.1"
  ...
  readOnlyConfig: |
    auto.create.topics.enable=false
    cruise.control.metrics.topic.auto.create=true
    cruise.control.metrics.topic.num.partitions=1
    cruise.control.metrics.topic.replication.factor=2
    inter.broker.protocol.version=2.8.1
    ...
```

For more information, refer to Kafka upgrade documentation: https://kafka.apache.org/documentation/#upgrade

4. Wait until the `kafka-operator` reconciles the broker protocol change. The expected result is for the `kubectl get -A kafkaclusters` to report a `ClusterRunning` status for the KafkaCluster.
5. Set the `spec.clusterImage` to the chosen image.

   Example output showing the `KafkaCluster` manifest:

```
...
spec:
  clusterImage: "ghcr.io/banzaicloud/kafka:2.13-3.4.1"
  ...
  readOnlyConfig: |
    ...
    inter.broker.protocol.version=2.8.1
    ...
```

6. Wait until `kafka-operator` fully applies the image change. After that, the cluster will run a new version of Kafka, but will still use the old protocol version.
7. Verify the behavior and performance of this Kafka cluster and workloads that use it.

   Important: Do not postpone this verification. The next step makes a rollback to the previous Kafka version impossible.

8. Bump the protocol version to the current one by modifying the `spec.readOnlyConfig` field so that `inter.broker.protocol.version` equals the new Kafka version.

   Example output showing `KafkaCluster` manifest:

```
...
spec:
  clusterImage: "http://ghcr.io/banzaicloud/kafka:2.13-3.4.1"
  ...
  readOnlyConfig: |
    ...
    inter.broker.protocol.version=3.4.1
    ...
```

9. Wait until `kafka-operator` fully applies the protocol version change. The upgrade of the Kafka image for this cluster is now complete.

Upgrade NKP Pro

Overview of the steps to upgrade NKP Pro using CLI.

The Nutanix Kubernetes Platform (NKP) upgrade represents an important step of your environment's life cycle, as it ensures that you are up-to-date with the latest features and can benefit from the most recent improvements, enhanced cluster management, and better performance. If you have are on this scenario

of the upgrade section of documentation, you have an Pro License. If you have an NKP Ultimate license, jump to that section of the Upgrade NKP Ultimate documentation.

Ensure that all platform applications in the management cluster have been upgraded to avoid compatibility issues with the Kubernetes version included in this release. This is done automatically when upgrading Kommander, so ensure that you follow these sections in order to upgrade the Kommander component prior to upgrading the Konvoy component.

If you have more than one Pro cluster, repeat all of these steps for each Pro cluster. For a full list of NKP Pro features, see Licenses on page 24.

Prerequisites

Review the following before starting your upgrade:

- Upgrade Prerequisites.
- If you are in an air-gapped environment, also see, NKP Pro: For Air-gapped Environments Only on page 1174.

Proceed to the NKP Pro: Upgrade the Cluster and Platform Applications on page 1175 section and begin your upgrade.

NKP Pro: For Air-gapped Environments Only

Push the necessary images to this registry.

If you are operating in an air-gapped environment, a Local Registry Tools Compatible with NKP on page 1070 containing all the necessary installation images, including the Kommander images is required. Continue to for steps to push the necessary images to this registry.

- Download the Complete Nutanix Kubernetes Platform (NKP) Air-gapped Bundle for this release (that is. `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`. For more information, see Downloading NKP on page 16).
- Connectivity: Your Pro cluster must be able to connect to the local registry.

Extracting Air-gapped Images and Set Variables
Steps to extract the air-gapped image bundles into your private registry.

About this task

Follow these steps to extract the air-gapped image bundles into your private registry.

Procedure

1. Assuming you have downloaded `nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`, extract the tarball to a local directory using the command `tar -xzvf nkp-air-gapped-bundle_v2.12.0_linux_amd64.tar.gz`.
2. The directory structure after extraction can be accessed in subsequent steps using commands to access files from different directories. For example: For the bootstrap, change your directory to the `nkp-<version>` directory similar to example below depending on your current location.

   Example:

   ```
   cd nkp-v2.12.0
   ```

3. Set an environment variable with your registry address using the command `export REGISTRY_URL="<https/http>://<registry-address>:<registry-port>" export REGISTRY_USERNAME=<username> export REGISTRY_PASSWORD=<password>`.

Load Images to your Private Registry - Kommander

For the air-gapped kommander image bundle, use the command `nkp push bundle --bundle ./container-images/kommander-image-bundle-v2.12.0.tar --to-registry=$ {REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}`.

Load Images to your Private Registry - Konvoy

Before creating or upgrading a Kubernetes cluster, you need to load the required images in a local registry if operating in an air-gapped environment. This registry must be accessible from both the Creating a Bastion Host on page 685 and either the AWS EC2 instances or other machines that will be created for the Kubernetes cluster.

> **Note:** If you do not already have a local registry set up, continue to the Local Registry Tools Compatible with NKP on page 1070 page for more information.

To load the air-gapped image bundle into your private registry use the command `nkp push bundle --bundle ./container-images/konvoy-image-bundle-v2.12.0.tar --to-registry=${REGISTRY_URL} --to-registry-username=${REGISTRY_USERNAME} --to-registry-password=${REGISTRY_PASSWORD}`.

> **Note:** It may take some time to push all the images to your image registry, depending on the performance of the network between the machine you are running the script on and the registry.

NKP Pro: Upgrade the Cluster and Platform Applications

This section describes how to upgrade your Kommander Management cluster and all Platform Applications to their supported versions in air-gapped, non-air-gapped and on-premises environments. To prevent compatibility issues, you must first upgrade the Kommander component on your Pro Cluster before upgrading to Nutanix Kubernetes Platform (NKP) .

> **Note:** It is important you upgrade Kommander BEFORE upgrading the Kubernetes version. This ensures that any changes required for new or changed Kubernetes API's are already present.

Upgrade Kommander

Prerequisites

- Use the `--kubeconfig=${CLUSTER_NAME}.conf` flag or set the `KUBECONFIG` environment variable to ensure that you upgrade Kommander on the right cluster. For alternatives and recommendations around setting your context, see Commands within a kubeconfig File on page 32 .
- If you have Nutanix Kubernetes Platform (NKP) Insights installed, ensure you uninstall it before upgrading NKP. For more information, see Upgrade NKP Insights to 1.0.0 on page 1221.

> **Important:** Limited availability of NKP resources
>
> - If you have configured a **custom domain**, running the `upgrade` command can result in an inaccessibility of your services through your custom domain for a few minutes.
> - The **NKP UI** and other APIs may be inconsistent or unavailable until the upgrade is complete.

Upgrade Air-gapped Environments

To upgrade Kommander and all the platform applications in the management cluster, use the command `nkp upgrade kommander \ --charts-bundle ./application-charts/nkp-kommander-charts-bundle-v2.12.0.tar.gz \ --kommander-applications-repository ./application-repositories/kommander-applications-v2.12.0.tar.gz`.

Example output:

```
# Ensuring upgrading conditions are met
# Ensuring application definitions are updated
# Ensuring helm-mirror implementation is migrated to ChartMuseum
...
```

Upgrade Non-air-gapped Environments

To upgrade Kommander and all the Platform Applications in the Management Cluster on a non-air-gapped environment, use the command `nkp upgrade kommander`.

> **Note:** If you want to disable the Artificial Intelligence (AI) Navigator, add the flag `--disable-appdeployments ai-navigator-app` to this command.

Example output:

```
# Ensuring upgrading conditions are met
# Ensuring application definitions are updated
# Ensuring helm-mirror implementation is migrated to ChartMuseum
...
```

Troubleshooting

If the upgrade fails, get more information on the upgrade process using the command `nkp upgrade kommander -v 6`

If you find any `HelmReleases` in a “broken” release state such as “exhausted” or “another rollback/release in progress”, you can trigger a reconciliation of the `HelmRelease` using the following commands:

```
kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op":
 "replace"
, "path": "/spec/suspend", "value": true}]'
kubectl -n kommander patch helmrelease <HELMRELEASE_NAME> --type='json' -p='[{"op":
 "replace", "path":
 "/spec/suspend", "value": false}]'
```

NKP Pro: Upgrade the Cluster CAPI Components

Upgrade the CAPI Components on the Management cluster.

- For a Pre-provisioned air-gapped environment only, ensure you have uploaded the artifacts.
- For air-gapped environments, ensure you have created the air-gapped bundle correctly.

Upgrade the CAPI Components

New versions of Nutanix Kubernetes Platform (NKP) come pre-bundled with newer versions of CAPI, newer versions of infrastructure providers, or new infrastructure providers. When using a new version of the NKP CLI, upgrade all of these components first.

If you are running on more than one management cluster, you must upgrade the CAPI components on each of these clusters.

> **Note:** Ensure your NKP configuration references the management cluster where you want to run the upgrade by setting the `KUBECONFIG` environment variable, or using the `--kubeconfig` flag. For more information, see https://kubernetes.io/docs/tasks/access-application-cluster/configure-access-multiple-clusters/.

Execute the upgrade command for the CAPI components using the command `nkp upgrade capi-components`

Example output:

```
# Upgrading CAPI components
# Waiting for CAPI components to be upgraded
# Initializing new CAPI components
```

> **Note:**
>
> If you created CAPI components using flags to specify values, use those same flags during Upgrade to preserve existing values while setting additional values.
>
> - Refer to `nkp create cluster aws` CLI for flag descriptions for `--with-aws-bootstrap-credentials` and `--aws-service-endpoints`.
> - Refer to the HTTP or HTTPS section for details: Configuring an HTTP or HTTPS Proxy on page 677.

If the upgrade fails, review the prerequisites section and ensure that you've followed the steps in the Upgrade NKP on page 1144 overview. Furthermore, ensure you have adhered to the Prerequisites at the top of this page.

NKP Pro: Upgrade the Cluster Core Addons

For Pre-provisioned air-gapped environments only, you must upload artifacts to copy the artifacts onto the cluster hosts before you upgrade the CAPI components using the command `konvoy-image upload artifacts \ --container-images-dir=./artifacts/images/ \ --os-packages-bundle=./artifacts/$OS_PACKAGES_BUNDLE \ --containerd-bundle=artifacts/$CONTAINERD_BUNDLE \ --pip-packages-bundle=./artifacts/pip-packages.tar.gz`

Upgrading the Cluster Core Addons

About this task

To install the core addons, Nutanix Kubernetes Platform (NKP) relies on the `ClusterResourceSet` Cluster API feature. In the CAPI component upgrade, we deleted the previous set of outdated global `ClusterResourceSets` because in past releases, some addons were installed using a global configuration. In order to support individual cluster upgrades, NKP now installs all addons with a unique set of `ClusterResourceSets` and corresponding referenced resources, all named using the cluster's name as a suffix. For example: `calico-cni-installation-my-aws-cluster`. For more Cluster API feature information, see https://cluster-api.sigs.k8s.io/

If you modify any of the `ClusterResourceSet` definitions, these changes are not be preserved when running the command `nkp upgrade addons`. You must use the `--dry-run -o yaml` options to save the new configuration to a file and continue the same changes upon each upgrade.

Your cluster comes preconfigured with a few different core addons that provide functionality to your cluster upon creation. These include: CSI, Container Network Interface (CNI), Cluster Autoscaler, and Node Feature Discovery. New versions of NKP may come pre-bundled with newer versions of these addons.

> **Important:** If you have more than one Pro cluster, ensure your `nkp` configuration references the cluster where you want to run the upgrade by setting the `KUBECONFIG` environment variable, or using the `--kubeconfig` flag, in accordance with Kubernetes conventions. For information on Kubernetes conventions, see https://kubernetes.io/docs/tasks/access-application-cluster/configure-access-multiple-clusters/

Perform the following steps to update your addons.

Procedure

1. Ensure your `nkp` configuration references the cluster where you want to run the upgrade by setting the `KUBECONFIG` environment variable, or use the `--kubeconfig` flag.

2. Upgrade the core addons in a cluster using the `nkp upgrade addons` command.

   Specify the cluster infrastructure (choose `aws`, `azure`, `vsphere`, `vcd`, `eks`,`gcp`, `preprovisioned`) and the name of the cluster.

   Examples for upgrade core addons commands:

   ```
   export CLUSTER_NAME=my-azure-cluster
   nkp upgrade addons azure --cluster-name=${CLUSTER_NAME}
   ```

   or

   ```
   export CLUSTER_NAME=my-aws-cluster
   nkp upgrade addons aws --cluster-name=${CLUSTER_NAME}
   ```

   AWS example output:

   ```
   Generating addon resources
   clusterresourceset.addons.cluster.x-k8s.io/calico-cni-installation-my-aws-cluster
    upgraded
   configmap/calico-cni-installation-my-aws-cluster upgraded
   clusterresourceset.addons.cluster.x-k8s.io/tigera-operator-my-aws-cluster upgraded
   configmap/tigera-operator-my-aws-cluster upgraded
   clusterresourceset.addons.cluster.x-k8s.io/aws-ebs-csi-my-aws-cluster upgraded
   configmap/aws-ebs-csi-my-aws-cluster upgraded
   clusterresourceset.addons.cluster.x-k8s.io/cluster-autoscaler-my-aws-cluster upgraded
   configmap/cluster-autoscaler-my-aws-cluster upgraded
   clusterresourceset.addons.cluster.x-k8s.io/node-feature-discovery-my-aws-cluster
    upgraded
   configmap/node-feature-discovery-my-aws-cluster upgraded
   clusterresourceset.addons.cluster.x-k8s.io/nvidia-feature-discovery-my-aws-cluster
    upgraded
   configmap/nvidia-feature-discovery-my-aws-cluster upgraded
   ```

What to do next

For more NKP CLI command help, see NKP upgrade addons CLI.

NKP Pro: Upgrading the Kubernetes Version

About this task

When upgrading the Kubernetes version of a cluster:

> **Important: Additional Considerations for upgrading a FIPS cluster** :
>
> To correctly upgrade the Kubernetes version use the command `nkp update controlplane aws --cluster-name=${CLUSTER_NAME} --kubernetes-version=v1.29.6 +fips.0 --ami=<ami-with-fips-id>`

Procedure

1. Upgrade the control plane first using the infrastructure specific command.
2. Upgrade the node pools second using the infrastructure specific command.
3. Build a new image if applicable.

   - If an AMI was specified when initially creating a cluster for AWS, you must build a new one Using Konvoy Image Builder on page 793 and set the flag(s) in the update commands. Either AMI ID

`--ami AMI_ID`, or the lookup image flags: `--ami-owner AWS_ACCOUNT_ID`, `--ami-base-os ubuntu-20.04`, and `--ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*'`.

> **Important:** The AMI lookup method will return an error if the lookup uses the upstream CAPA account ID.

- If an Azure Machine Image was specified for Azure, you must build a new one with Using KIB with Azure on page 1097.
- If a vSphere template Image was specified for vSphere, you must build a new one with Using KIB with vSphere on page 1104.
- You must build a new GCP image with Using KIB with GCP on page 1100.

4. Upgrade the Kubernetes version of the control plane. Each cloud provider has distinctive commands. Below is the AWS command example. Select the dropdown list menu next to your provider for compliant CLI.

   Examples:

   - **AWS**

     > **Note:** The first example below is for AWS. If you created your initial cluster with a custom AMI using the `--ami` flag, it is required to set the `--ami` flag during the Kubernetes upgrade.

     ```
     nkp update controlplane aws \
     --cluster-name=${CLUSTER_NAME} \
     --ami AMI_ID \
     --kubernetes-version=v1.29.6
     ```

   - **AWS AMI ID lookup**

     ```
     nkp update controlplane aws \
     --cluster-name=${CLUSTER_NAME} \
     --ami-owner AWS_ACCOUNT_ID \
     --ami-base-os ubuntu-20.04 \
     --ami-format 'example-{{.BaseOS}}-?{{.K8sVersion}}-*' \
     --kubernetes-version=v1.29.6
     ```

   - **Azure**

     ```
     nkp update controlplane azure --cluster-name=${CLUSTER_NAME} --kubernetes-
     version=v1.29.6 --
     compute-gallery-id <Azure Compute Gallery built by KIB for Kubernetes v1.29.6>
     ```

     If these fields were specified in the Default Override Files on page 1118 during Azure: Creating an Image on page 324, the flags must be used in upgrade:

     `--plan-offer`, `--plan-publisher` and `--plan-sku`.

     ```
     --plan-offer rockylinux-9
     --plan-publisher erockyenterprisesoftwarefoundationinc1653071250513
     --plan-sku rockylinux-9
     ```

   - **vSphere**

     ```
     nkp update controlplane vsphere --cluster-name=${CLUSTER_NAME} --kubernetes-
     version=v1.29.6 --
     vm-template <vSphere template built by KIB for Kubernetes v1.29.6>
     ```

   - **VCD**

     ```
     nkp update controlplane vcd --cluster-name=${CLUSTER_NAME} --kubernetes-
     version=v1.29.6 --catalog
     ```

```
<tenant catalog vApp template> --vapp-template <vApp template built in vSphere
KIB for Kubernetes v1.29.6>
```

- **GCP**

```
nkp update controlplane gcp --cluster-name=${CLUSTER_NAME} --kubernetes-
version=v1.29.6 --image=
projects/${GCP_PROJECT}/global/images/<GCP image built by KIB for Kubernetes
 v1.29.6>
```

- **Pre-provisioned**

```
nkp update controlplane preprovisioned --cluster-name=${CLUSTER_NAME} --
kubernetes-version=v1.29.6
```

- **EKS**

```
nkp update controlplane eks --cluster-name=${CLUSTER_NAME} --kubernetes-
version=v1.26.6
```

- Example output, showing the provider name corresponding to the CLI you executed from the choices above.

```
Updating control plane resource controlplane.cluster.x-k8s.io/v1beta1,
 Kind=KubeadmControlPlane default/my-aws-cluster-control-plane
Waiting for control plane update to finish.
 # Updating the control plane
```

Tip: Some advanced options are available for various providers. To see all the options for your particular provider, run the command `nkp update controlplane aws|vsphere|preprovisioned|azure|gcp|eks --help`

- AWS AMI instance example:

  `aws: --ami, --instance-type`

includes some of the options mentioned in the previous note.

> **Note:** The command `nkp update controlplane {provider}` has a 30 minute default timeout for the update process to finish. If you see the error "`timed out waiting for the condition`", you can check the control plane nodes version using the command `kubectl get machines -o wide --kubeconfig $KUBECONFIG` before trying again.

5. Upgrade the Kubernetes version of your node pools. Upgrading a nodepool involves draining the existing nodes in the nodepool and replacing them with new nodes. In order to ensure minimum downtime and maintain high availability of the critical application workloads during the upgrade process, we recommend deploying Pod Disruption Budget (Disruptions) for your critical applications. For more information, see Updating Cluster Node Pools on page 1080.

   a. First, get a list of all node pools available in your cluster by using the command `nkp get nodepool --cluster-name ${CLUSTER_NAME}`

   b. Select the nodepool you want to upgrade using the command `export NODEPOOL_NAME=my-nodepool`.

   c. Then update the selected nodepool using the command below. Upgrading a node pool involves draining the existing nodes in the node pool and replacing them with new nodes. we recommend deploying Pod Disruption Budget (Disruptions) for your critical applications. Refer to Updating Cluster Node Pools on page 1080 for more information. The first example command shows AWS

language, so select the dropdown list for your provider for the correct command. Execute the `update` command for each of the node pools listed in the previous command.

> **Note:** The first example below is for AWS. If you created your initial cluster with a custom AMI using the `--ami` flag, it is required to set the `--ami` flag during the Kubernetes upgrade.

- **AWS Example:**

```
nkp update nodepool aws ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
kubernetes-version=v1.29.6
```

- **Azure Example:**

```
nkp update nodepool azure ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
kubernetes-version=v1.29.6 --compute-gallery-id <Azure Compute Gallery built by
 KIB for
 Kubernetes v1.29.6>
```

If these fields were specified in the Use Override Files with Konvoy Image Builder on page 1118 during Azure: Creating an Image on page 324, the flags must be used in upgrade:

```
--plan-offer
```

```
--plan-publisher
```

```
--plan-sku
```

```
--plan-offer rockylinux-9
--plan-publisher erockyenterprisesoftwarefoundationinc1653071250513
--plan-sku rockylinux-9
```

- **vSphere Example:**

```
nkp update nodepool vsphere ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME}
 --kubernetes-version=v1.29.6 --vm-template <vSphere template built by KIB for
 Kubernetes v1.29.6>
```

- **VCD Example:**

```
nkp update nodepool vcd ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
kubernetes-version=v1.29.6 --catalog <tenant catalog vApp template> --vapp-
template <vApp template built in vSphere KIB for Kubernetes v1.29.6>
```

- **GCP Example:**

```
nkp update nodepool gcp ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
kubernetes-version=v1.29.6 --image=projects/${GCP_PROJECT}/global/images/<GCP
 image built by KIB for Kubernetes v1.29.6>
```

- **Pre-provisioned Example:**

```
nkp update nodepool preprovisioned ${NODEPOOL_NAME} --cluster-name=
${CLUSTER_NAME} --kubernetes-version=v1.29.6
```

- **EKS Example:**

```
nkp update nodepool eks ${NODEPOOL_NAME} --cluster-name=${CLUSTER_NAME} --
kubernetes-version=v1.26.6
```

- Example output, showing the infrastructure provider name.

```
Updating node pool resource cluster.x-k8s.io/v1beta1, Kind=MachineDeployment
 default/my-aws-cluster-my-nodepool
Waiting for node pool update to finish.
```

```
# Updating the my-aws-cluster-my-nodepool node pool
```

d. Repeat this step for each additional node pool.

When all nodepools have been updated, your upgrade is complete. For the overall process for upgrading to the latest version of Nutanix Kubernetes Platform (NKP) , refer back to Upgrade NKP on page 1144 for more details.

12

TROUBLESHOOTING GUIDE

Collection of tips for self-remedying common NKP user issues.

The sections in this chapter represent the most common areas of questions directed at our Support Department. Starting here, you can answer those frequently asked questions.

Gather Environment Data for Nutanix NKP Support

Steps for gathering the data needed to troubleshoot a Kubernetes cluster.

The following pages provide the steps to troubleshoot a Kubernetes cluster customization. These subtopics offer a way to supply a support bundle file back to Nutanix for further analysis.

Generate a Support Bundle

To generate a support bundle with data collected for the last 48 hours of the cluster's life, complete the following tasks.

Prerequisites

Before generating a support bundle, verify that you have:

- An AMD 64-based Linux or macOS machine with a supported version of the operating system.
- A running Kubernetes cluster.
- Access to the NKP binary.

Diagnostic Bundle

`nkp diagnose` was developed by NKP and builds on the open source `troubleshoot.sh` project.

> **Note:** The command `nkp diagnose` is based on version `0.13.16` of `troubleshoot.sh` with custom modifications. The NKP fork is open source and available from this public GitHub repository: https://github.com/mesosphere/troubleshoot.

`nkp diagnose` supports multiple support bundle collectors and can be configured as a `SupportBundle` Kubernetes resource in a YAML file. For more information, see https://troubleshoot.sh/docs/collect/all/.

The following list is the minimum set of resources required to debug a cluster, but it can be further customized.

The bundle uses the following collectors:

- `clusterInfo` collects basic information about the cluster. For more information, see https://troubleshoot.sh/docs/collect/cluster-info/.
- `clusterResources` collects a subset of available resources in the cluster. For more information, see https://troubleshoot.sh/docs/collect/cluster-resources/.
- `configMap` collects the values of Kubernetes ConfigMaps. For more information, see https://troubleshoot.sh/docs/collect/configmap/.

- `secrets` collects the names and metadata of secrets, but NOT the secret values or keys. For more information, see https://troubleshoot.sh/docs/collect/secret/.
- `execCopyFromHost` runs a container on each node on the cluster and copies the created data. For more information, see Gathering information using the ExecCopyFromHost Collector on page 1186.
- `allLogs` can collect logs from all containers on the cluster. For more information, see Using the AllLogs Collector on page 1188.
- `cpuMemoryUtilisation` collects the average CPU and memory utilization of the last 30 days.

Generating a Support Bundle

About this task

The command `nkp diagnose` uses the same Kubernetes configuration as the command `kubectl`. The `nkp diagnose` command can also be pointed at a specific configuration using the `--kubeconfig` parameter.

To generate the support bundle, perform the following steps:

Procedure

1. Run the default collectors configuration using the `nkp diagnose` command.

   Output example:

   ```
   Collecting support bundle ...
   support-bundle-2021-08-13T14_44_23.tar.gz
   ```

2. To view the bundle contents, extract the bundle and replace `support-bundle-2021-08-13T14_44_23.tar.gz` with the location from the previous step.

   Example:

   ```
   tar -xzvf support-bundle-2021-08-13T14_44_23.tar.gz
   ```

   Example output:

   ```
   cluster-info  cluster-resources  configmaps  node-diagnostics  pod-logs  secrets
    version.yaml
   ```

Collecting Information from a Bootstrap Cluster

About this task

If your bootstrap cluster has not yet pivoted towards your Konvoy cluster, you can collect log information from that bootstrap cluster, and there is a preconfigured set of relevant collectors. Specify an additional bootstrap cluster `kubeconfig` using the `--bootstrap-kubeconfig` parameter to activate bootstrap cluster diagnostics. You will receive an additional support bundle named `bootstrap-support-bundle-<date created>`.

> **Note:** The bootstrap cluster diagnostics are independent of the *main* or Konvoy cluster diagnostics configuration. We run a static collector set that collects the following bootstrap cluster information:

- ClusterInfo
- ClusterResources
- AllLogs
- ConfigMaps
- Secrets

Procedure

Run the `nkp diagnose` command with bootstrap bundle configuration.

Example:

```
nkp diagnose bundle.yaml
```

Using Customizations

About this task

To print the default collectors configuration, complete the following task:

Procedure

1. Run the `nkp diagnose default-config > bundle.yaml` command.
2. Edit the file to make appropriate modifications.

> **Note:** By default, `nkp diagnose` does not require that you supply a configuration. You can print the default bundle by running `nkp diagnose default-config`.

Using SSH Fallback and Ansible

About this task

In some cases, the Kubernetes API is unavailable for the cluster. In those cases, you can collect node-level information using SSH access to the diagnosed nodes. Be aware that not all clusters have SSH access configured. If they do not, then access using SSH fallback is not possible.

To get node-level information from your cluster using SSH access, perform the following steps:

Procedure

To specify the nodes to access for data collection, run the `nkp diagnose ssh <path/to/ansible-inventory.yaml>` command.

> **Note:** This collector does not use the full Ansible `inventory.yaml` format, only a limited subset to describe the infrastructure.

Only the following attributes of the `ansible-inventory.yaml` are supported. All other group definitions are ignored.

- Support for `all` shared variables.
- Support for `hosts` key in `all` groups.
- Supported behavioral inventory is limited to:
  - `ansible_host`
  - `ansible_port`
  - `ansible_user`
  - `ansible_ssh_private_key_file`

The following is an example `inventory.yaml` file:

```
all:
  vars:
    ansible_user: centos
```

```
  hosts:
    host-1:
      ansible_host: 192.168.10.1
    host-2:
      ansible_host: 192.168.10.22
      ansible_port: 2222
```

More information on these Ansible parameters can be found in the Ansible User Guide at https://docs.ansible.com/ansible/latest/user_guide/intro_inventory.html#connecting-to-hosts-behavioral-inventory-parameters.

> **Note:** All other group definitions are in the `inventory.yaml` files are ignored.

Example:

```
all:
  vars:
    ansible_user: centos
  hosts:
    host-1:
      ansible_host: 192.168.10.1
    host-2:
      ansible_host: 192.168.10.22
      ansible_port: 2222
```

The fallback collector runs a bash script over SSH and copies the collected data. The format of the created bundle matches that of `nkp diagnose` collector-generated bundles.

Example:

```
node-diagnostics/<HOSTNAME_PORT>/data/
        - dmesg
        - ....
```

Redactors are supported and are in the same format as the main `nkp diagnose` command. Per node collection, timeouts are supported using the `--timeout` parameter.

See also: NKP CLI at https://docs.d2iq.com/dkp/2.7/cli-commands.

Create Diagnostic Bundles using Custom Collectors

For creating diagnostic bundles, NKP uses a customized version of `troubleshoot.sh` integrated into `nkp-diagnose`.

Customizations

To meet the specific needs of diagnosing NKP clusters, we have developed custom collectors and modified the behavior of upstream collectors. For more information on the changes, see our repository at https://github.com/mesosphere/troubleshoot/blob/v0.13-d2iq/README.d2iq.md.

Gathering information using the `ExecCopyFromHost` Collector

About this task

This collector is explicitly created to gather host-level information from cluster nodes. The collector allows you to run a provided container image in a privileged mode as a root user, with additional Linux capabilities and the host filesystem mounted in the container.

You can collect host-level information other than copying host-level files. (This is already possible with the `CopyFromHost` collector.) Like the `CopyFromHost` collector, this collector runs as a Kubernetes `DaemonSet` executed on all nodes in the system. The data produced by the container are copied from a pre-defined

directory into the diagnostics bundle under each node name. The name of the parent directory in the diagnostics bundle is determined by the name of the collector specified in its configuration.

Procedure

The data written into the diagnostics bundle follows this format `<collector-name> / <node-name> / data / (file1|file2|...)`

The following is a sample configuration file:

```
spec:
  collectors:
    - execCopyFromHost:
        name: node-diagnostics
        image: mesosphere/nkp-diagnostics-node-collector:latest
        timeout: 30s
        command:
          - "/bin/bash"
          - "-c"
          - "/diagnostics/container.sh --hostroot /host --hostpath ${PATH} --
outputroot /output"
        workingDir: "/diagnostics"
        includeControlPlane: true
        privileged: true
        capabilities:
          - AUDIT_CONTROL
          - AUDIT_READ
          - BLOCK_SUSPEND
          - BPF
          - CHECKPOINT_RESTORE
          - DAC_READ_SEARCH
          - IPC_LOCK
          - IPC_OWNER
          - LEASE
          - LINUX_IMMUTABLE
          - MAC_ADMIN
          - MAC_OVERRIDE
          - NET_ADMIN
          - NET_BROADCAST
          - PERFMON
          - SYS_ADMIN
          - SYS_BOOT
          - SYS_MODULE
          - SYS_NICE
          - SYS_PACCT
          - SYS_PTRACE
          - SYS_RAWIO
          - SYS_RESOURCE
          - SYS_TIME
          - SYS_TTY_CONFIG
          - SYSLOG
          - WAKE_ALARM
        extractArchive: true
```

The following is an example of the data produced by running this collector:

```
### node-diagnostics
#   ### troubleshoot-control-plane
#   #   ### data
#   #       ### certs_expiration_kubeadm
#   #       ### containerd_config.toml
                ...
#   #       ### whoami_validate
#   ### troubleshoot-worker
```

```
#       ### data
#           ### containerd_config.toml
#           ### containers_crictl
                ...
#           ### whoami_validate
```

If an error occurs while collecting node diagnostics, the `node-diagnostics/<node>/pod-collector.json` file contains the serialized JSON representations of the running pod. This helps debug the reasons for the collection failure. The `node-diagnostics/<node>/pod-collector.log` file contains standard output from the collector container that runs the diagnostics script. In addition, the command may also produce certain `-error.txt` files. `file-copy-error.txt` and `pod-collector-files-copy-error.txt` are two file examples. These files contain error messages generated while fetching log files from the collector.

When using this collector for node-level information, you must run additional docker containers and must have the following docker images:

- `mesosphere/pause-alpine:3.2`
- `mesosphere/nkp-diagnostics-node-collector:$(nkp-diagnose version)`

For more information on the configuration options, see the `ExecCopyFromHost` in the `pkg/apis/troubleshoot/v1beta2/exec_copy_from_host.go` file.

Using the `AllLogs` Collector

About this task

This collector gathers pod logs from specified namespaces or all namespaces if none are selected. You can collect logs of all the pods from all the namespaces. The pod logs are collected under the `allPodLogs` directory.

Procedure

The data written into the diagnostics bundle follows this format: `<collector-name> / <namespace-name> / <pod-name> - (container1|container2|...)`

The following is a sample configuration file to collect logs from all the pods from all the namespaces:

```
spec:
  collectors:
    - allLogs:
        namespaces:
          - "*"
```

The following is a sample configuration file to collect logs from all the pods from specific namespaces:

```
spec:
  collectors:
    - allLogs:
        namespaces:
          - default
          - dev
          - prod
```

The following is an example of the data produced by running this collector:

```
### node-diagnostics
#   ### troubleshoot-control-plane
#   #   ### data
#   #       ### certs_expiration_kubeadm
#   #       ### containerd_config.toml
                ...
#   #       ### whoami_validate
#   ### troubleshoot-worker
```

```
#       ### data
#           ### containerd_config.toml
#           ### containers_crictl
                ...
#           ### whoami_validate
```

If an error occurs while collecting node diagnostics, the `node-diagnostics/<node>/pod-collector.json` file contains the serialized JSON representations of the running pod. This helps debug the reasons for the collection failure. The `node-diagnostics/<node>/pod-collector.log` file contains stdout from the collector container that runs the diagnostics script.

When using this collector for node-level information, you must run additional docker containers and must have the following docker images:

- `mesosphere/pause-alpine:3.2`
- `mesosphere/nkp-diagnostics-node-collector:$(nkp-diagnose version)`

For more information on the configuration options, see the `ExecCopyFromHost` in the `pkg/apis/troubleshoot/v1beta2/exec_copy_from_host.go` file.

Collect from all Namespaces for `ConfigMap` and `Secret` Collector

Support for collecting from all namespaces for`ConfigMap`and`Secret`collector.

In the original collectors `namespace` there is a required parameter. This supports collecting from all namespaces by not setting the `namespace` (or setting it to `""`). Note: To collect all config maps or secrets, an empty selector must be used (`selector: [""]`).

Support for Optional support-bundle Name Prefix

The `nkp diagnose` command includes an optional `--bundle-name-prefix` option that is used to add a prefix to the bundle names automatically. The original bundle naming convention (i.e. `support-bundle-<date/time>`) is used if this option is not specified.

`ClusterResources` Collector

Another customization is added to collect custom resource definitions and all custom resources in the cluster.

Common Information Discovery Commands

Discovery commands provide information about the environment and system status used to identify problems.

When troubleshooting Cluster Operations Management, sometimes just knowing what, where, and status can help you determine where the problem is and help reveal a solution to an issue.

Several `kubectl` commands can give you information from your environment that can be useful in determining where a problem is occurring.

Application Discovery Commands

Application definitions originate from the source Git repo. These are configured in the cluster for Flux to consume. You can use common `kubectl` verbs to investigate status and gather log data to help determine any problem areas.

Verb and Purpose	Examples

Investigate the Git repo using `kubectl`.	`kubectl get gitrepository -A`
	or
	`kubectl describe -n <namespace> gitrepository <name>`

Check the status logs of the `source-controller` pod	`kubectl describe -n kommander-flux deploy/source-controller`
	or
	`kubectl logs -n kommander-flux deploy/source-controller --tail -1`

Verify the status of the associated Apps, ClusterApps, and AppDeployments	`kubectl get apps,clusterapps,appdeployments -A`

Check the status and logs of the `kommander-appmanagement` pod	`kubectl describe -n kommander deploy/kommander-appmanagement`
	or
	`kubectl logs -n kommander deploy/kommander-appmanagement --tail -1`

Kustomizations	

Check the status of all Kustomizations created by AppDeployment:	`kubectl get kustomization -A`
	or
	`kubectl describe -n <namespace> kustomization <name>`

Check the status and logs of the `kustomize-controller` pod since it manages Kustomizations in the `kommander-flux` Namespace.	`kubectl describe -n kommander-flux deploy/kustomize-controller`
	or
	`kubectl logs -n kommander-flux deploy/kustomize-controller --tail -1`

Verb and Purpose	Examples
HelmRelease	

HelmRelease(s) are deployed by each Kustomization, and for the Application to function, these resources must be in the “Ready” state. To check:	`kubectl get helmrelease -A`
	or
	`kubectl describe -n <namespace> helmrelrease <name>`

Each HelmRelease also consumes a HelmChart. We can ensure the status of HelmCharts with the following:	`kubectl get helmchart -A`
	or
	`kubectl describe -n <namespace> helmchart <name>`

The controller that manages HelmReleases and HelmCharts is the `helm-controller` in the `kommander-flux` Namespace. It can be helpful to check the status and logs of the `helm-controller` pod:	`kubectl describe -n kommander-flux deploy/helm-controller`
	or
	`kubectl logs -n kommander-flux deploy/helm-controller --tail -1`

Workspace Discovery Commands

Workspaces are managed by a Custom Resource named Workspaces. You can use common `kubectl` verbs to investigate status and gather log data to help determine any problem areas:

Examples	Examples

Leverage `kubectl` to investigate workspace information:	`kubectl get workspaces`
	or
	`kubectl describe workspace <name>`

The controller that manages Workspaces is the `kommander-cm` deployment in the `kommander` Namespace. Check status logs :	`kubectl describe -n kommander deployment/kommander-cm`
	or
	`kubectl logs -n kommander deployment/kommander-cm --all-containers --tail -1`

Federated* Resources	

Workspace resources federated to Attached clusters are managed by `Federated* resources`. To view the status of all Federated resources for the Workspace Namespace, we can leverage the following command(s):	`for resource in $(kubectl api-resources | awk '$1 ~ /^federated.*$/ {print $1}'); do kubectl get -n <workspace-namespace> $resource --show-kind --ignore-not-found; done`
	or
	`kubectl describe -n <workspace-namespace> <resource>`

The controller that manages **Federated* resources** is the `kubefed-controller-manager` in the `kube-federation-system` Namespace. It can be helpful to check into the status and logs of the `kubefed-controller-manager` pod:	`kubectl describe -n kube-federation-system deployment/kubefed-controller-manager`
	or
	`for pod in $(kubectl get po -n kube-federation-system -lkubefed-control-plane=controller-manager -oname); do kubectl logs -n kube-federation-system $pod --all-containers --tail -1; done`

Examples	Examples
Attached Clusters	
To check	`$body` or `$body`
Ensure the status with:	`$body` or `$body`
It can be helpful to check the status and logs	`$body` or `$body`

Application Troubleshooting

Describes common application behaviors when the root cause is with an application deployed on the Kommander component.

Not sure if you are having problems with the Applications deployed on the Kommander component of NKP? This section will explain some behaviors you may observe if you have issues with Applications. Also, you see context around the machinery that manages the resources and how they interact.

Applications Context

Application definitions originate from the source Git Repository(ies). They are configured inside the cluster for Flux to consume. The controller that manages GitRepositories is the `source-controller` deployment in the `kommander-flux` **Namespace**.

Kommander Applications are deployed as AppDeployments and reference `Apps` and `ClusterApps`. The controller that manages `Apps`, `ClusterApps`, and `AppDeployments` is the `kommander-appmanagement` deployment in the `kommander` **Namespace**. For more information on AppDeployments, see Applications on page 390.

Each AppDeployment creates a `Kustomization`. The controller that manages `Kustomizations` is the `kustomize-controller` in the `kommander-flux` **Namespace**.

Each `Kustomization` typically deploys a few resources, with HelmRelease(s) being the most notable. For the application functionality, these resources should always be ready. Each HelmRelease also consumes a HelmChart. The controller that manages HelmReleases and HelmCharts is the `helm-controller` in the `kommander-flux` **Namespace**.

> **Note:** Application version information is in the Nutanix Kubernetes® Platform Release Notes.

Troubleshooting Approach

To properly troubleshoot an issue, you need to be familiar with how all the pieces connected to Applications work. This knowledge will assist in identifying the problem and where to start troubleshooting.

> **Note:** For specific symptoms, you can always search the NKP Knowledge Base for resolutions to behaviors. For more information, see https://portal.nutanix.com/page/documents/kbs/list?filterVal=Nutanix%20Kubernetes%20Platform%20(NKP).

Example issues might be:

- My Kommander Apps are not getting federated down to my attached clusters.
- The process has halted. How can I check my status?

Use the subtopics in this section for more specific help:

- Applications Deployment CLI vs UI on page 1193
- Application YAML Customizations on page 1197

Related Topics

Additional troubleshooting information is located in the following sections of the Cluster Operations Management on page 353 chapter of this guide:

- Platform Applications on page 400
- Deploying Platform Applications Using CLI on page 403
- Workspaces on page 411

Applications Deployment CLI vs UI

When installing NKP, an `AppDeployment` resource is created for each enabled **Platform Application**. This `AppDeployment` resource references a `ClusterApp`, which then references the repository that contains a concrete declarative and preconfigured setup of an application, usually in the form of a `HelmRelease`. `ClusterApps` are cluster-scoped, so these Platform Applications are deployable to all Workspaces or Projects. Also, refer to AppDeployment Resources on page 410.

Applications can be enabled and configured using the UI or CLI at both levels:

- **Workspace level** for all clusters in a Workspace
- **Cluster level** for specific clusters within a Workspace.

See the following table for an overview:

Desired level of Configuration	CLI	UI
Cluster-scoped customization	Create an AppDeployment, and specify target clusters in `spec.clusterSelector`. Reference `ConfigMap` with customization overrides in `clusterConfigOverrides`. To see an example deployment, see Enabling an Application Per Cluster on page 415.	Go to the target workspace and enable through the application card, selecting the target clusters. For an example deployment, see Pro: Enabling an Application Using the UI on page 398.
Workspace-scoped deployment	Create an AppDeployment without a `spec.clusterSelector` section. Automatically deploys to all clusters in the workspace. To see an example deployment, see Deploying Platform Applications Using CLI on page 403.	Go to the target workspace and enable it through the application card, selecting all target clusters. For an example deployment, see Pro: Customizing an Application Using the UI on page 399.

Desired level of Configuration	CLI	UI
Cluster-scoped customization	Create an AppDeployment, and specify target clusters in `spec.clusterSelector`. Reference `ConfigMap` with customization overrides in `clusterConfigOverrides`. To see an example deployment, see Customizing an Application Per Cluster on page 416.	Go to target workspace and enable through the application card. Select the target clusters and establish customizations in the configuration service per cluster. For an example deployment, see Pro: Customizing an Application Using the UI on page 399.
Workspace-scoped customization	Create an AppDeployment without a `spec.clusterSelector` section. Reference `ConfigMap` with customization overrides in `clusterConfigOverrides`. Automatically deploys customization to all clusters in the workspace. To see an example deployment, see Customizing Your Application on page 390.	Go to the target workspace and enable it through the application card. Manually select all clusters and copy-paste the customization in the configuration service for all clusters. For an example deployment, see Pro: Customizing an Application Using the UI on page 399.

Applications Deployment in a Workspace
Print and Review the Current State of an `AppDeployment` Resource.

If you want to know how the `AppDeployment` resource is currently configured, use the commands below to print a table of the declared information. If the `AppDeployment` is configured for several clusters in a workspace, a column will display a list of the clusters.

Reviewing all AppDeployments in a Workspace

About this task

To review the state of the `AppDeployment` resource for a specific workspace, complete the following task.

Procedure

Using the name of your workspace, run the `nkp get appdeployments -w kommander-workspace` command.

The output should contain a list of all your applications, as shown in the example.

```
NAME APP CLUSTERS
[...]
kube-oidc-proxy                kube-oidc-proxy-0.3.2           host-cluster
kube-prometheus-stack          kube-prometheus-stack-46.8.0    host-cluster
kubecost                       kubecost-0.35.1                 host-cluster
[...]
```

Reviewing a Specific AppDeployment of an Application in a Workspace

About this task

To review the state of a specific `AppDeployment` of an application, complete the following task.

Procedure

Using the application's name and your workspace, run the `nkp get appdeployment kube-prometheus-stack -w kommander-workspace` command.

The output should contain a list of all your applications, as shown in the example.

```
NAME APP CLUSTERS
kube-prometheus-stack kube-prometheus-stack-46.8.0 host-cluster
```

Deployment Scope

In a single-cluster environment with an **Pro license**, `AppDeployments` enable customizing any platform application.

In a multi-cluster environment with an **Ultimate license**, `AppDeployments` enable Resource Requirements on page 39, Project Applications on page 441, and Cluster-scoped Application for Existing AppDeployments on page 414.

Use Case Example

About this task

As a user, I have multiple clusters in a Workspace. I want to allow an application for some but not all clusters in that workspace. I also want to enable a custom configuration.

To define which cluster should have an application deployed onto them, use the AppDeployment's `spec.clusterSelector` field.

> **Note:** The user can edit the AppDeployment at anytime to add or remove clusters from the spec.

Enable Workspace Applications for a subset of clusters and custom configurations for each cluster:

Procedure

1. Create an AppDeployment using the command `nkp create appdeployment` to enable and disable clusters and set cluster config overrides per cluster:
2. Use the code editor to edit the AppDeployment spec to directly:
   a. Enable or disable clusters
   b. Add or remove cluster config overrides for clusters. using the `nkp edit appdeployment APPDEPLOYMENT_NAME -n NAMESPACE` command.

      Example:

      ```
      nkp edit appdeployment kube-oidc-proxy -n kommander
      ```

      > **Note:** `NAMESPACE` is the namespace in which the AppDeployment resides - this can be a Workspace or a Project Namespace. To list all available workspace namespaces, run `kubectl get workspaces`.

Troubleshooting

About this task

For up-to-date status on which clusters an application has been enabled, or what configuration overrides have been applied to each cluster,

Procedure

1. Check the AppDeployment status using the command `nkp get appdeployment`.
2. Use the `kubectl get appdeployment` command to get the entire YAML object, including cluster config overrides that might be set on a cluster.

   Example:

   ```
   nkp get appdeployment --output yaml
   ```

   Example output:

   ```
   apiversion: apps.kommander.d2iq.io/v1alpha3
   kind: AppDeployment
   metadata:
     generation: 1  # this means there is only one instance of the application, must
    match with the number of instances you expect to see
     name: kube-oidc-proxy  # APP ID of the application as displayed in the Release
    Notes
     namespace: kommander # equal to the workspace namespace of the workspace where the
    app is deployed
   spec:
     appRef:
       kind: ClusterApp
       name: kube-oidc-proxy-0.3.2 # APP ID with the app version as displayed in the
    Release Notes
     clusterConfigOverrides:  # present only for apps with custom configurations
     - clusterSelector:   # present only for workspaces with several clusters
         matchLabels:
           kommander.d2iq.io/cluster-name: host-cluster  # cluster name where app is
    deployed
       configMapName: kube-oidc-proxy-host-cluster  # name of the ConfigMap file that
    contains the Overrides with the custom configuration for this specific cluster
     clusterSelector:
       matchExpressions:
       - key: kommander.d2iq.io/cluster-name
         operator: In
         values:
         - host-cluster # cluster name where app is deployed
   status:
     clusters:
     - clusterConfigOverridesRef:
       name: kube-oidc-proxy-host-cluster # name of the ConfigMap file that contains the
    Overrides with the custom configuration for this specific cluster
     conditions:
     - status: "True" # must be True for app to be deployed successfully
       type: AppDeploymentEnabled
     name: host-cluster
   observedGeneration: 1
   ```

Verifying Applications

About this task

Once the applications are enabled, verify the deployment using this task.

Procedure

Connect to the attached cluster and check the `HelmReleases` using the command `kubectl get helmreleases -n ${WORKSPACE_NAMESPACE}`.

Example output:

```
NAMESPACE                NAME                READY   STATUS
 AGE
workspace-test-vjsfq    kafka-operator     True    Release reconciliation succeeded
 7m3s
```

Application YAML Customizations

Customized application deployment objects establish how applications are deployed.

NKP allows configuring an application for all clusters in a workspace and a subset of clusters in a workspace. You can also apply customizations per cluster or workspace.

An `AppDeployment` object creates a local HelmRelease YAML file that establishes how an Application is deployed. To further customize the application, you can create a `ConfigMap` object with overriding configuration.

You can apply the same `ConfigMap` to several clusters in a workspace or create different customizations in individual `ConfigMaps` for each cluster.

The AppDeployment specifies the name of the app, the version of the app, and the workspace where it applies. Additionally, you can specify a subset of clusters where it should be deployed and, if required, any customizations.

For example, this is the default `AppDeployment` for the Kube Prometheus Stack Platform Application:

```
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: kube-prometheus-stack
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    name: kube-prometheus-stack-46.8.0
    kind: ClusterApp
```

Overriding ALL Clusters within a Workspace

About this task

If you want to change the default configuration (customize an application), complete the following task.

Procedure

1. Add a `ConfigMap` with your override values.
2. Add that reference to the `AppDeployment` by setting the `configOverrides` field to the name of that `ConfigMap`.

   ```
   kind: HelmRelease
   metadata:
     ...
     name: dex
     namespace: kommander
   ...
   spec:
   ...
     valuesFrom:
     - kind: ConfigMap
       name: dex-2.13.5-d2iq-defaults
     - kind: ConfigMap
   ```

```
    name: dex-cluster-overrides
```

When the HelmRelease refreshes, it will add references to those ConfigMaps. This overrides the app's configuration for all clusters within the Workspace unless specified differently in the AppDeployment through the `spec.clusterSelector`.

Overriding Example: Customize the Kube Prometheus Stack

About this task

This is an example of how to customize the `AppDeployment` of Kube Prometheus Stack:

Procedure

1. Provide the name of a ConfigMap with the custom configuration in the AppDeployment:

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: kube-prometheus-stack
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    name: kube-prometheus-stack-46.8.0
    kind: ClusterApp
  configOverrides:
    name: kube-prometheus-stack-overrides-attached
EOF
```

> **Note:** Line 12 references the name of the ConfigMap with the customization.

2. Create the `ConfigMap` with the name from the previous step, which provides the custom configuration on top of the default configuration:

```
cat <<EOF > kube-prometheus-stack-overrides-attached | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: ${WORKSPACE_NAMESPACE}
  name: kube-prometheus-stack-overrides-attached
data:
  values.yaml: |
    prometheus:
      prometheusSpec:
        storageSpec:
          volumeClaimTemplate:
            spec:
              resources:
                requests:
                  storage: 150Gi
EOF
```

Available Customizations

Application YAML can be customized, but which customizations can you do, and why would you customize? Take a look below for the answers.

- **Why Customize?** Your environment and provider, combined with various image locations or security requirements, might dictate the why behind customizing your Applications.

  - Enable certain images
  - Provide more storage
  - Bump memory resources

  Depending on which level you need your customizations determines the how. Refer to the pages that follow for specifics about how to customize your Applications.

How do I Customize?

About this task

To get started, create a `ConfigMap`. A `ConfigMap` lets you decouple environment-specific configuration from your container images, making your applications easily portable. Use a `ConfigMap` to set configuration data separate from the Application code. Provide the name of a `ConfigMap` in the **AppDeployment**, which provides custom configuration on top of the default configuration.

This is an example of how to customize the **AppDeployment** of the Kube Prometheus Stack:

Procedure

1. Create the `ConfigMap`, which provides the custom configuration on top of the default configuration:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: ${WORKSPACE_NAMESPACE}
  name: kube-prometheus-stack-overrides-attached
data:
  values.yaml: |
    prometheus:
      prometheusSpec:
        storageSpec:
          volumeClaimTemplate:
            spec:
              resources:
                requests:
                  storage: 150Gi
EOF
```

2. Provide the name of a `ConfigMap` with the custom configuration in the **AppDeployment**. Override the default configuration of an Application by setting the **configOverrides** field on the **AppDeployment** to that `ConfigMap`.

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: kube-prometheus-stack
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    name: kube-prometheus-stack-46.8.0
    kind: ClusterApp
  configOverrides:
    name: kube-prometheus-stack-overrides-attached
```

```
EOF
```

Note:

- The changes are applied only if the YAML file has a valid syntax.
- Set up only one cluster override ConfigMap per cluster. Only one will be applied if several ConfigMaps are configured for a cluster.
- Cluster override ConfigMaps must be created on the Management cluster.

Enable or Disable an App per Cluster

For newly attached clusters into the workspace, all Applications enabled for the Workspace are automatically enabled on and deployed to the new clusters. If you want to see what clusters your application is currently deployed in, follow the steps below.

Print and Review the Current State of an `AppDeployment` Resource

If you want to know how the `AppDeployment` resource is currently configured, use the commands below to print a table of the declared information. If the `AppDeployment` is configured for several clusters in a workspace, a column will display a list of the clusters.

Review all `AppDeployments` in a workspace

About this task

To review the state of the `AppDeployment` resource for a specific workspace, complete this task.

Procedure

Run the `nkp get appdeployments -w kommander-workspace` command with the name of your workspace.

The output should contain a list of all your applications, as shown in this example:

```
NAME APP CLUSTERS
[...]
kube-oidc-proxy                kube-oidc-proxy-0.3.2          host-cluster
kube-prometheus-stack          kube-prometheus-stack-46.8.0   host-cluster
kubecost                       kubecost-0.35.1                host-cluster
[...]
```

Review a specific `AppDeployment` of an application in a workspace

About this task

To review the state of a specific `AppDeployment` of an application, complete this task.

Procedure

Run the `nkp get appdeployment kube-prometheus-stack -w kommander-workspace` command with the name of the application and your workspace, as in this example:

Example output

```
NAME APP CLUSTERS
kube-prometheus-stack kube-prometheus-stack-46.8.0 host-cluster
```

Deployment Scope

In a single-cluster environment with an **Pro license**, `AppDeployments` enable customizing any platform application.

In a multi-cluster environment with an **Ultimate license**, `AppDeployments` enable Resource Requirements on page 39, Project Applications on page 441, and Cluster-scoped Application for Existing AppDeployments on page 414.

Define a Custom Configuration

Enable or Disable an Application per Cluster after enabling it at the workspace level. You can enable or disable applications at any time. After you have enabled the application at the workspace level, the `spec.clusterSelector` field populates.

Edit the `AppDeployment` YAML by adding or removing the names of the clusters where you want to enable your application in the `clusterSelector` section.

The following snippet is an example. To customize, replace the following according to your requirements:

- Application name
- Version
- Workspace name
- Cluster names

See Components and Applications in the NKP Release Notes for the compatible versions.

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.kommander.d2iq.io/v1alpha3
kind: AppDeployment
metadata:
  name: kube-prometheus-stack
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    name: kube-prometheus-stack-46.8.0
    kind: ClusterApp
  clusterSelector:
    matchExpressions:
    - key: kommander.d2iq.io/cluster-name
      operator: In
      values:
      - attached-cluster1
      - attached-cluster3-new
EOF
```

To verify the Current Configuration of your Application, see Verifying Applications on page 1196.

Object Locations and Explanations

Troubleshoot using Kommander component resource locations.

Objects are created and interact in many steps with coordination between various components and applications. What deploys the Platform Applications, where are the charts stored, what is the mechanism that deploys them, and how are settings customized.

After installing the Konvoy components of NKP using the NKP CLI, install the Kommander component. For more information, see NKP Concepts and Terms on page 18.

Kommander

Kommander uses many individual applications to manage resources. The following explains the process(es) that deploy them.

Table 71: Kommander Objects and the Resources they Manage

Kommander Object	Managed Resource
`kommander-applications` repository	
Flux	Deploys Flux from a set of Kubernetes manifest files.
Helm Repositories	Applies the Helm Repository resources from this repository.
Applications Definitions	Populate the management Git repository hosted by Gitea by copying the local `kommander-applications` repo provided via the `--kommander-applications-repository` flag.
ChartMuseum	(Air-gapped Only) ChartMuseum stores the Helm Charts for Air-gapped installations. In non-air-gapped installations, the charts are fetched from upstream repositories, and ChartMuseum is not installed.
Gatekeeper	Deploys Gatekeeper by applying the HelmRelease directly to the cluster. AppDeployment adopts the HelmRelease.
`kommander-appmanagement`	
AppManagement	Deploys `kommander-appmanagement` via applying the HelmRelease directly to the cluster.
Core AppDeployments	Creates AppDeployments for the core components: Flux, `kommander-appmanagement` , and ChartMuseum. These components were previously deployed, but this creates the AppDeployments, which will eventually manage the Applications.
`apps-kommander`	
Bootstrap Repository	Creates the `apps-kommander` Kubstomization, which deploys apps into the `kommander` Namespace on the Management Cluster. Commits manifests and directories to the Management git repository to set up the Management Cluster and the repository structure.
`kommander-ca`	
Root CA	Creates Cert-manager custom resources: `kommander-ca` clusterIssuer, `kommander-ca` Certificate, and a self-signed ClusterIssuer. Then, waits for the `kommander-ca` certificate to be created and accessible.
`kommander`	
Ingress Certificate	Chart values are used to create the self-attached KommanderCluster with custom domain/custom certificate settings. It also creates ACME resources(ClusterIssuer) as defined in the installer configuration.

Related information

https://github.com/mesosphere/kommander-applications/tree/main/common/helm-repositories

Kommander Resource Locations

Installation happens in many steps, with coordination between various components and applications. This section describes the process and locations for multiple repositories. After installing the Konvoy components of NKP using the NKP CLI, install the Kommander component, by completing the following tasks.

Kommander Installation Part I:

1. **Base** - Deploys base resources required, including Kommander and `kommander flux` namespaces.
2. **Flux** - Deploys Flux from a set of Kubernetes manifest files in the `kommander-applications` repository (deployments, etc).
3. **Root CA** - Creates Cert-manager custom resources: `kommander-ca` clusterIssuer, `kommander-ca` Certificate, and a self-signed ClusterIssuer. Then, it waits for the `kommander-ca` Certificate to be created and accessible.
4. **ChartMuseum** - Installs via the Helm library. The ChartMuseum Helm Release is created but will reconcile when the Git repository is created and populated. This is because ChartMuseum is being stored in the GitRepository. The installer will also set the helmRepositoryURL in the internal installer configuration to be used during the Helm repository installation in the next step.
5. **Helm Repositories** - Applies the Helm Repository resources from the `kommander-applications` repo https://github.com/mesosphere/kommander-applications/tree/main/common/helm-repositories. If the helmRepositoryURL were set in the previous step, it would be applied via a substitution variable when the Helm Repository objects are applied. EX: "${helmMirrorURL:=https://kubernetes.github.io/dashboard}"
6. **Ingress Certificate** - Sets `kommander` chart values to create the self-attached KommanderCluster with custom domain/custom certificate settings. It also creates ACME resources (ClusterIssuer) as defined in the installer configuration.
7. **Applications Definitions** - Populates the management Git repository by copying the local kommander-applications repo provided via the `--kommander-applications-repository` flag.
8. **Bootstrap Repository** - Commits manifests and directories to the Management git repository to set up the Management Cluster and the repository structure. It also creates the `apps-kommander` kustomization, which deploys apps into the `kommander` namespace on the Management Cluster.
9. **Age** - Create an Age Encryption section for SOPs encryption and add the Age recipient(public) key to git.
10. **Flux Configuration** - Populates the `kommander-vars` ConfigMap with substitution variables as well as creates the management GitRepository object and commits it to Git
11. **Gatekeeper** Deploys Gatekeeper by applying the HelmRelease directly to the cluster. AppDeployment adopts the HelmRelease.
12. **AppManagement** - Deploys `kommander-appmanagement` via applying the HelmRelease directly to the cluster.
13. **Core AppDeployments** - Creates AppDeployments for the core components: Flux, `kommander-appmanagement` and ChartMuseum. These components were previously deployed, but this creates the AppDeployments, which will eventually manage the Applications.
14. **Optional AppDeployments** - Creates all the AppDeployments for the applications defined in the installer configuration.
15. **Catalog Repository Loader** - Creates and commits the GitRepository objects to the management Git repo defined in the installer config (catalog). These default catalog Git repositories are then managed by a controller (DefaultCatalogGitRepository), which propagates the GitRepository objects to workspaces and projects based on labels.

> **Note:** Application version information is in the Nutanix Kubernetes® Platform Release Notes.

Kommander Installation Part II:

This part of the installation deals with Kommander the Controller and Helm Charts.

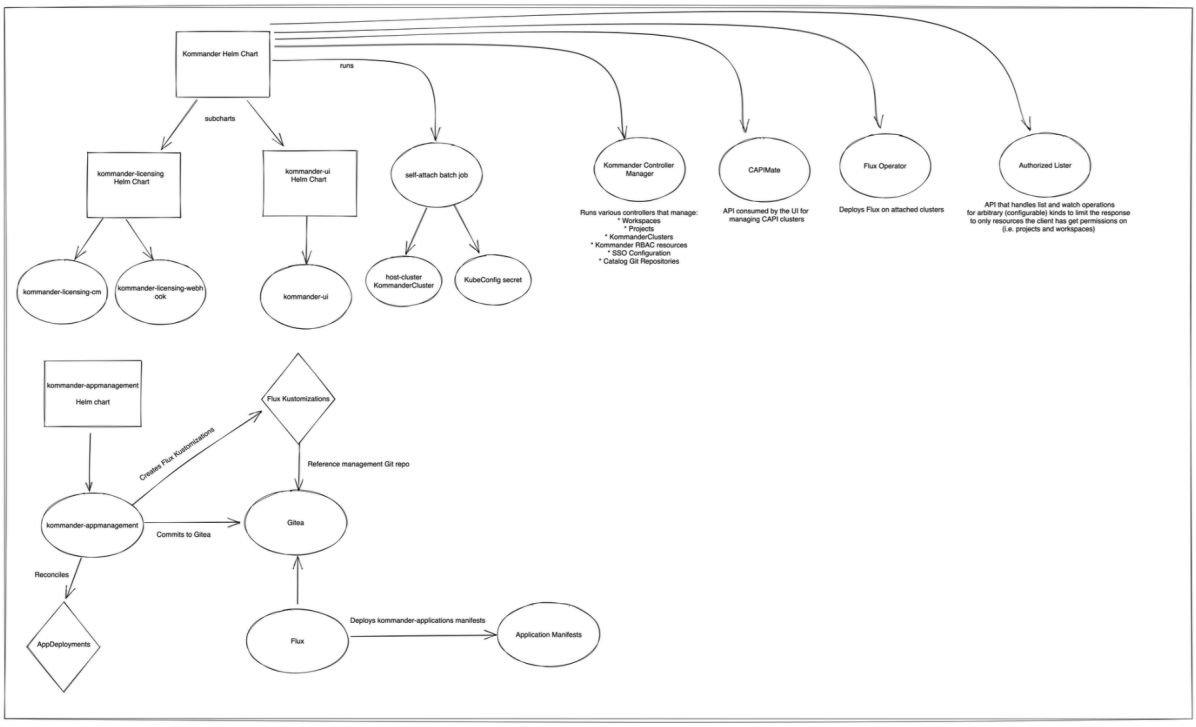


Figure 29: Kommander Installation Diagram

13

AI NAVIGATOR

Introduction to the Nutanix Kubernetes Platform (NKP) chatbot, AI Navigator.

The Artificial Intelligence (AI) Navigator chatbot ensures real-time, interactive communication to answer various user queries, spanning basic instructions to complex functionalities. AI Navigator was introduced in DKP 2.6.0, and is trained in product documentation and support knowledge base. Use of AI Navigator requires a licensed copy of DKP version 2.6.0 or later as NKP 2.12 and you must accept the User Agreement. The AI chatbot is offered for Internet-connected environments regardless of your license type.

> **Note:** AI Navigator is not supported in air-gapped implementations to help support heightened security in public sector, defense, and network-restricted customer environments.

Section Content

AI Navigator User Agreement and Guidelines

Information on the user agreement and general usage guidelines.

Introducing the Artificial Intelligence (AI) Navigator, engineered with the power of AI and models that include our product documentation and Support knowledge base. This chatbot ensures real-time, interactive communication to answer a wide range of user queries. The chatbot is offered for Internet-connected environments regardless of NKP license type.

Use of the AI Navigator requires that you are logged in to a licensed copy of NKP 2.6.0 or later.

If you want to the AI Navigator to include cluster live data, additionally enable the NKP AI Navigator Cluster Info Agent: Obtain Live Cluster Information.

Accept the User Agreement

AI Navigator User Agreement to use the tool.

When you first access the NKP AI Navigator, it displays the user agreement for acceptance. Select **Yes, I agree** to continue. If you select **No, not now** to decline, you cannot use AI Navigator.

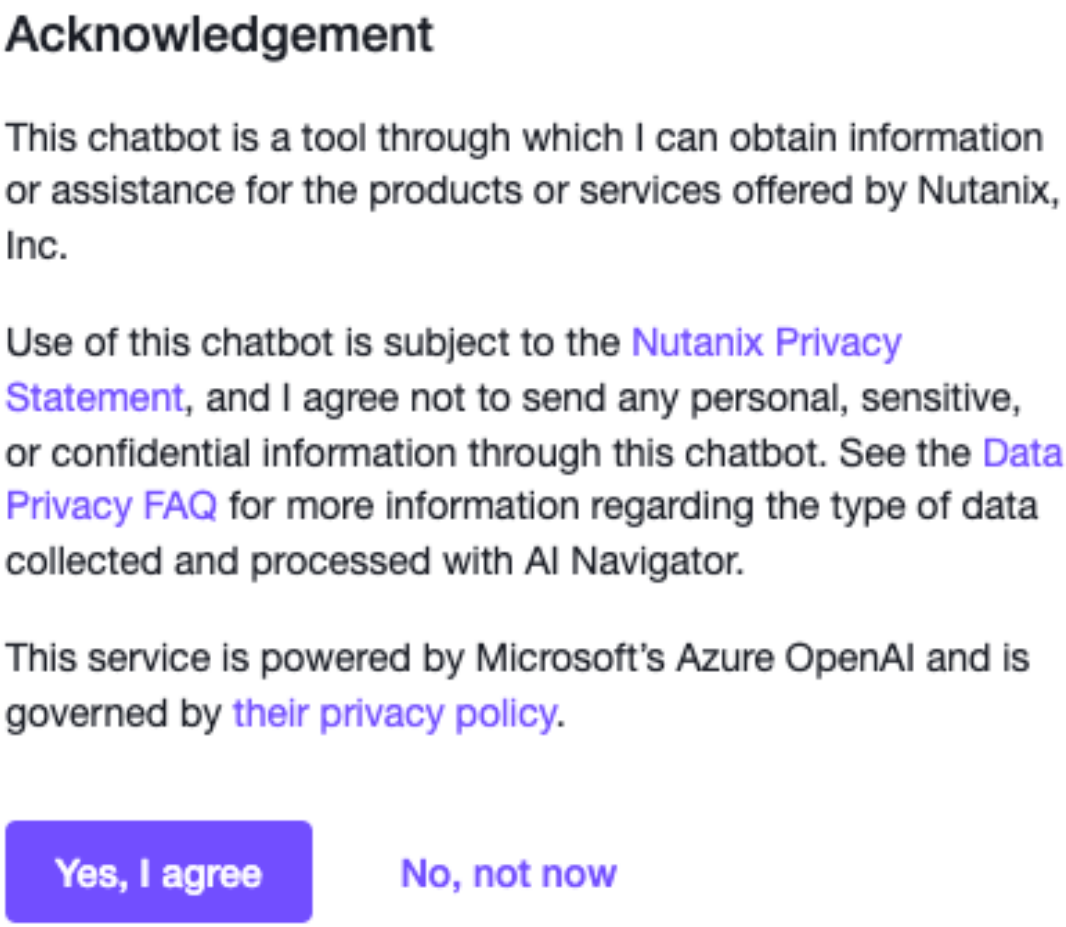


Figure 30: AI Navigator User Acknowledgement Message

AI Navigator Guidelines

General usage guidelines for AI Navigator.

In addition to applying any of your own organization's specific rules about the kinds of prompts you should enter, Nutanix recommends that you **NOT** enter sensitive information, including:

- Full names, addresses, or other personally identifying information (PII)
- Technical secrets:
  - Actual server names
  - Real IP addresses
  - Private keys
  - Access tokens
  - Any other technical details that might be useful to bad actors
- Government ID numbers

- Corporate, financial, or investment information
- Company Confidential information
- Company credentials, including actual usernames or passwords
- Secured or classified information

AI Navigator Installation and Upgrades

Information about installing, disabling, and upgrading the AI Navigator.

Use of Artificial Intelligence (AI) Navigator requires that you are logged in to a licensed copy of Nutanix Kubernetes Platform (NKP). The AI chatbot is offered for Internet-connected environments regardless of NKP license type.

Installing AI Navigator

When installing this version of NKP for the first time, you have a choice as to whether you installation AI Navigator. To installation the chatbot application, perform the installation according to the procedures for your chosen infrastructure provider, and NKP installs AI Navigator by default.

Disabling AI Navigator

Disable AI Navigator app using CLI.

About this task

To disable AI Navigator perform the following task:

Procedure

In the `apps:` section of the file, modify the Kommander configuration file after generating it using the command `ai-navigator-app`.

For more information on modifying the Kommander configuration file, see Installing Kommander in an Air-gapped Environment on page 1017

Upgrades to 2.7.0 or Later

By default, NKP installs the AI Navigator application as part of installing the Kommander component. NKP 2.6.0 installed AI Navigator by default as an experimental technical preview. However, beginning in NKP 2.6.1 and later, you have a choice to deactivate it. To disable AI Navigator, modify the Kommander configuration file after generating it, to disable the `ai-navigator-app` entry in the `apps:` section of the file. For more information on modifying the Kommander configuration file, see Installing Kommander in an Air-gapped Environment on page 1017

Upgrading AI Navigator in NKP Ultimate Environments

About this task

Under the NKP Ultimate: Upgrade the Management Cluster and Platform Applications on page 1154 section complete the following task:

Procedure

1. Expand the section for **For non-air-gapped environments**.
2. Follow the instructions to add a CLI flag to the listed `nkp upgrade kommander` command to disable the chatbot.

Upgrading AI Navigator in NKP Pro Environments

About this task

Under the NKP NKP Pro: Upgrade the Cluster and Platform Applications on page 1175 section complete the following task:

Procedure

1. Expand the section for **For non-air-gapped environments**.
2. Follow the instructions to add a CLI flag to the listed `nkp upgrade kommander` command to disable the chatbot.

Related Information

Links to related AI Navigator topics.

The following provides additional information on the installation and upgrade of AI Navigator.

- For information about the Kommander configuration file in the topic, see Kommander Configuration Reference on page 1038
- If you want to the AI Navigator to include cluster live data, see NKP AI Navigator Cluster Info Agent: Obtain Live Cluster Information.

Accessing the AI Navigator

Describes how to access and close the AI Navigator interface.

About this task

In a browser window, open the Nutanix Kubernetes Platform (NKP) UI.

Procedure

1. Click the Artificial Intelligence (AI) Navigator icon in the lower right corner.

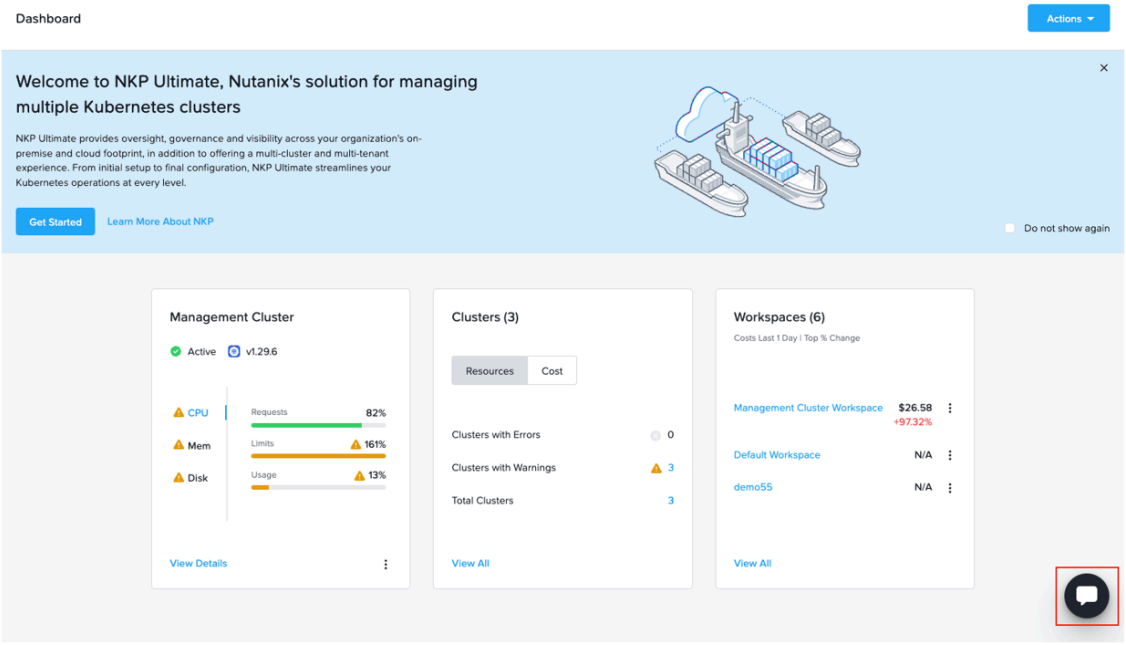


Figure 31: NKP User Interface with AI Navigator Icon

2. Agree to the user agreement terms to use AI Navigator.

   When you open the chatbot for the first time, it prompts you with a user agreement. If you decline, the chatbot window closes and you cannot use it. For more information on the user agreement, see AI Navigator User Agreement and Guidelines on page 1205.

   > **Note:** AI Navigator maintains your query history for the duration of your browser session, whether or not you close the AI Navigator application.

3. To close the AI Navigator, click **Close** below the prompt entry field.

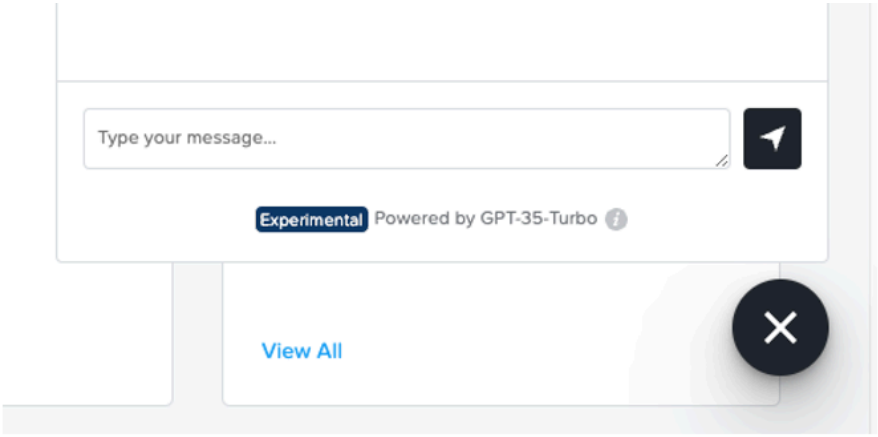


Figure 32: AI Navigator Close Button

AI Navigator Queries

Prompt engineering is the process of creating a chatbot that returns the data you really want. For AI chatbots trained on very large language models, there are some tips you can use to get better information with fewer prompt attempts. Some of those techniques are useful with the Artificial Intelligence (AI) Navigator as well.

When you create a prompt, an AI chatbot breaks down your entry into discrete parts to help it search its model. The more precise you can make your prompt, the better the chatbot is at returning the information you wanted.

Another technique for getting to the desired information is called fine-tuning. This involves adjusting the parameters and database model that the chatbot has to search. Nutanix Kubernetes Platform (NKP) fine-tunes the AI Navigator model by adding both the NKP and Insights documentation, and the NKP Support Knowledge Base. This helps to keep answers focused and fast.

What Goes in a Prompt

You can include straight text, inline code snippets such as kubectl commands, and code block snippets, within a limit of 4096 characters.

AI Navigator maintains your query history for the duration of your browser session, whether or not you close the AI Navigator application.

Inline Commands or Code Snippets

Use single back-tick marks to enclose a word, phrase, or command, for example:

```
What are the parameters I can use with the `nkp create cluster` command?
```

…creates this output in the chatbot window:

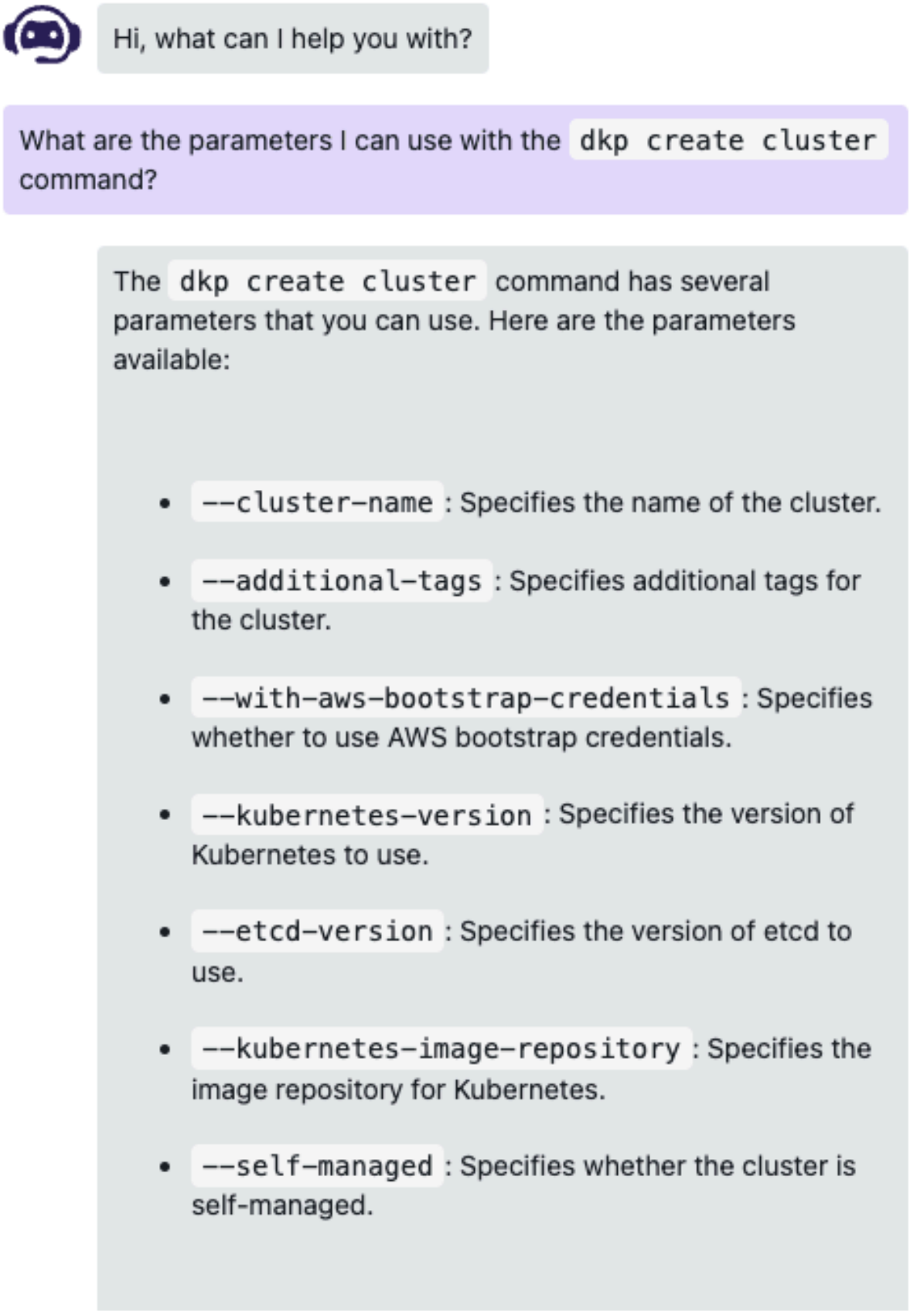


Figure 33: AI Navigator Chat Example

Code Blocks

Use a triple back-tick to open a code block for entry, in a manner similar to many popular chat and collaboration applications, like this:

````
Where do I need to use this code:
```
nkp create cluster aws \
    --http-proxy=127.0.0.1
    --https-proxy=127.0.0.2
````

…creates this entry and answer pair:

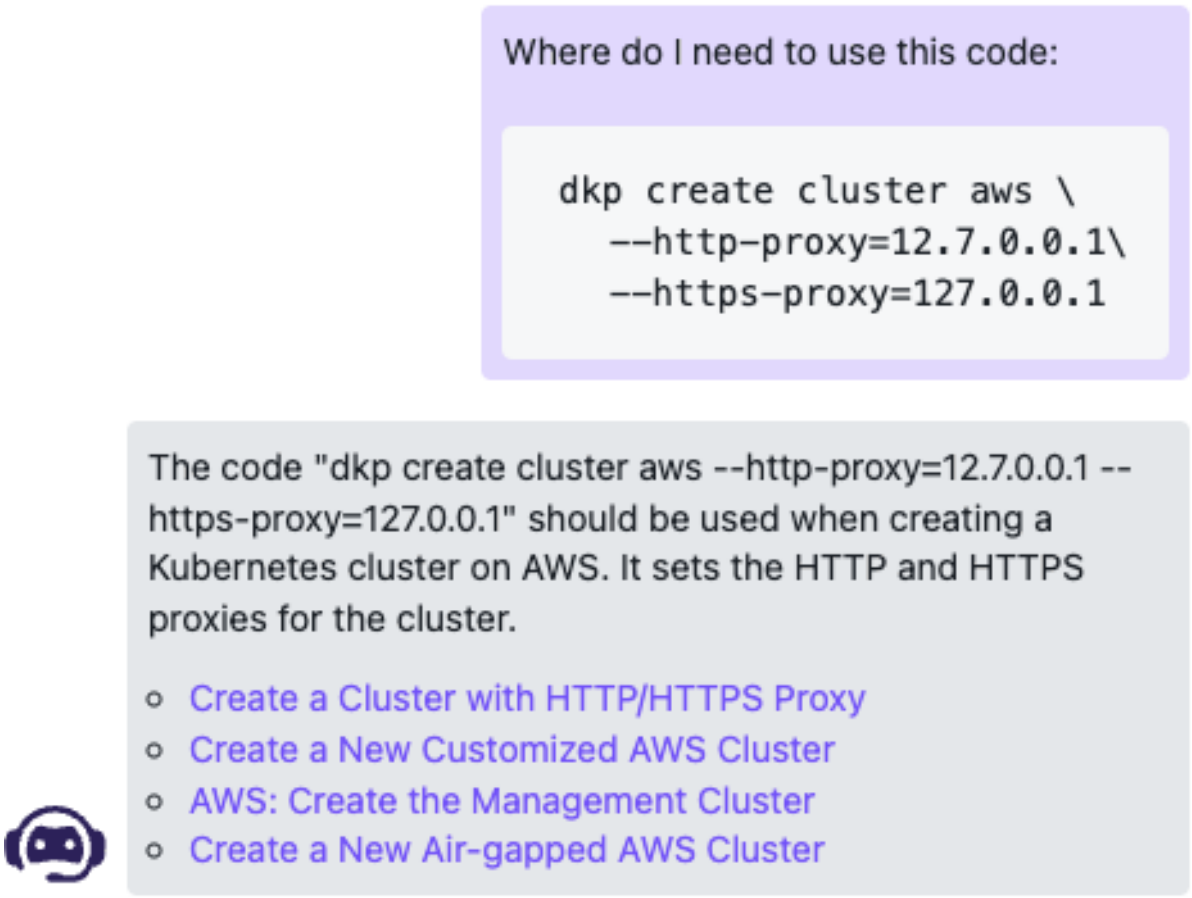


Figure 34: AI Navigator Prompt with Code Block

Selected Prompt Examples

You can use the AI Navigator to help you with a variety of tasks related to creating and managing clusters, and prompts can be fairly complex, and not necessarily limited to NKP. Consider these examples:

- **Evaluate a YAML Ain't Markup Language (YAML) Manifest**:

  For this query:

  ```
  Explain this yaml:
  apiVersion: policy/v1
  kind: PodDisruptionBudget
  metadata:
    name: nvidia-critical-app
  ```
  ```

```
spec:
  maxUnavailable: 1
  selector:
    matchLabels:
      app: nvidia-critical-app
```

The AI Navigator returns output like this example:

```
This YAML file is defining a Pod Disruption Budget (PDB) for an application named
"nvidia-critical-app".

A Pod Disruption Budget is a Kubernetes resource that specifies the minimum
number of pods that must be available during a disruption, such as a node
upgrade or maintenance. It helps ensure high availability of critical
applications by preventing too many pods from being unavailable at the same time.

In this specific YAML, the PDB is configured with the following properties:

maxUnavailable: 1: This specifies that only one pod of the "nvidia-critical-app"
can be unavailable at a time during a disruption. This ensures that at least
one pod is always available to handle requests.

selector: This specifies the labels used to select the pods that are part of
the "nvidia-critical-app". In this case, it selects the pods with the label
app: nvidia-critical-app.


Overall, this YAML is creating a Pod Disruption Budget that allows only one
pod of the "nvidia-critical-app" to be unavailable at a time during disruptions.

Links
-------
[Update Cluster Nodepools](https://beta-docs.nutanix.com/nkp/2.6/update-nvidia-gpu-
cluster-nodepools)
[Setting Priority Classes in NKP Applications](https://beta-docs.nutanix.com/nkp/2.6/
setting-priority-classes-in-nkp-applications)
[Project Quotas & Limit Ranges](https://beta-docs.nutanix.com/nkp/2.6/project-quotas-
limit-ranges)
[Configure the Control Plane](https://beta-docs.nutanix.com/nkp/2.6/configuring-the-
control-plane)
```

- **Updating a Transport Layer Security (TLS) Thumbprint**

  For this query:

  ```
  How do I update a TLS Thumbprint?
  ```

  The AI Navigator returns output like this example:

  ```
  To update a TLS thumbprint in NKP, you need to update the following objects:

  1. Update the vspherecluster object:

     - Use the command: kubectl patch vspherecluster <CLUSTER_NAME> --type=merge -p
   '{"spec": {"thumbprint": "<TLS_THUMBPRINT>"}}'

  2. Update the vsphere-config-secret secret in the vmware-system-csi namespace:

     - Use the command: kubectl patch secret vsphere-config-secret -n vmware-system-csi
   --type='json'-p='[{"op" : "replace" ,"path" : "/data/csi-vsphere.conf" ,"value" :
   "<BASE64 Encoded>"}]'

     - Make sure to encode the value of csi-vsphere.conf in base64.
  ```

```
3. Update the vsphere-cloud-config configmap in the kube-system namespace:

   - Use the command: kubectl --kubeconfig <CLUSTER_NAME>-workload.conf patch cm
 vsphere-cloud-config -n kube-system --type=merge -p'{"data": {"vsphere.conf":
 "global:\n  secretName: cloud-provider-vsphere-credentials\n  secretNamespace:
 kube-system\n  thumbprint: <TLS_THUMBPRINT>\nvcenter:\n  <vCenter_Address>:\n
 datacenters:\n    - 'dc1'\n    secretName: cloud-provider-vsphere-credentials\n
   secretNamespace: kube-system\n    server: '<vCenter_Address>'\n    thumbprint:
 <TLS_THUMBPRINT>\n" }}'


Replace <CLUSTER_NAME>, <TLS_THUMBPRINT>, and <BASE64 Encoded> with the actual values
 specific to your environment.

Links
------
Updating vCenter Server TLS Certificate Thumbprint in NKP – Nutanix
Updating vCenter Server TLS Certificate Thumbprint in NKP – Nutanix
```

- **Describe Clock Skew in Ceph Cluster Status**

  For this query:

  ```
  What is clock skew in Ceph cluster status?
  ```

  The AI Navigator returns output like this example:

  ```
  Clock skew refers to the difference in time between the clocks of different nodes
   in a Ceph cluster. In the context of a Ceph cluster, clock skew can affect the
   monitors, which are responsible for maintaining the cluster state. The monitors
   rely on a tight time alignment to coordinate with other Ceph daemons. If there
   is a significant clock skew between the monitor nodes, it can lead to issues in
   the cluster, such as inconsistent cluster maps and coordination problems. It is
   important to keep the time in-sync across nodes to ensure the proper functioning of
   the Ceph cluster.
  Links
  -------
  How to detect if a clock skew is affecting Rook-Ceph monitors in NKP? – Nutanix
  Create new cluster fails to complete due to time drift – Nutanix
  ```

AI Navigator Cluster Info Agent: Obtain Live Cluster Information

Do you want to know which **etcd version** and **image** your cluster is using? Inquire about the frequency of a specific **CronJob**? Or just know which version of Insights you are using?

You no longer need to rely on your NKP and `kubectl` knowledge to obtain answers. By enabling the Cluster Info Agent, the Artificial Intelligence (AI) Navigator can get the answers for you.

When enabled, the **Cluster Info Agent** can fetch live cluster data from your Pro or Management Cluster, and help you find out more about your cluster and its deployments. Just start a new query with the AI Navigator. The answers will then include data related to your cluster's current status, allowing for enhanced specificity and context, and leading to quicker problem resolution and enhanced productivity.

Like the AI Navigator chatbot, the Cluster Info Agent is offered for Internet-connected environments regardless of Nutanix Kubernetes Platform (NKP) license type.

For more information around data collection and processing, see Data Privacy FAQs on page 1215.

> **Note:** To help support heightened security in public sector, defense, and network-restricted customer environments, AI Navigator is not offered for air-gapped implementations.

Enable the NKP AI Navigator Cluster Info Agent

Describes how to enable the Nutanix Kubernetes Platform (NKP) AI Navigator cluster info agent

Prerequisites

- Ensure Artificial Intelligence (AI) Navigator is enabled.
- Assign the Workspace AI Navigator View Role (`workspace-ai-navigator-cluster-info-view`) role to the user or user group that should have access to cluster-specific information when interacting with the AI Navigator. For more information, see Configuring Workspace Role Bindings on page 435.

Enable the NKP AI Navigator Cluster Info Agent: Enable the Cluster Info Agent application on your Pro or Management cluster as explained in Enable an Application using the UI.

- Customers with an Ultimate license must ensure they have selected the Management Cluster Workspace before deploying the Agent.
- The Cluster Info Agent requires a few hours to index the complete cluster dataset. Until then, you can still use the NKP AI Navigator, but it will not include the entire cluster-specific dataset when providing answers.

Customizing the AI Navigator Cluster Info Agent

Describes how to customize your Artificial Intelligence (AI) Navigator Cluster info.

About this task

The Cluster Info Agent automatically observes the `kommander` workspace namespace (that corresponds to the Pro or Management cluster). However, you can configure the agent to scan a broader context, for example, other namespaces, as long as they are in the same workspace.

To customize a AI Navigator Cluster Info Agent installation.

Procedure

1. Log in to your Nutanix Kubernetes Platform (NKP) UI.
   For more information, see Logging into the UI with Kommander on page 1037.
2. **Ultimate only**: Select the Management Cluster Workspace from the top navigation bar.
3. Select **Applications** from the sidebar and search for **AI Navigator Cluster Info Agent**.
4. Using the three-dot menu in the application card, select **Edit** > **Configuration**.
5. Add other namespaces for scanning, as shown in this example.

   ```
   watchNamespaces: namespace1,namespace2,namespace3
   ```

6. **Uninstall the Cluster Info Agent**: Uninstall the Cluster Info Agent application like any other application as explained in Ultimate: Disabling an Application Using the UI on page 398

Data Privacy FAQs

Questions and answers to general date privacy questions.

This section answers common questions about general date privacy questions.

What Happens to My Data During an AI Navigator Query if the Cluster Info Agent is Enabled?

The Nutanix Kubernetes Platform does not store any type of data related to your queries, nor Pro or Management cluster data. Further, no data is stored by the Azure OpenAI Service. Your data is not available to OpenAI, Nutanix, or any other customers, and your data is not used to improve the OpenAI

model. To learn more about the data protection measures in place with the Azure OpenAI Service, see https://learn.microsoft.com/en-us/legal/cognitive-services/openai/data-privacy.

Query-related data collected by the Artificial Intelligence (AI) Navigator (with Cluster Info Agent enabled) is stored directly in your environment. Here is an overview of what happens during a query:

1. You make a query requesting information related to your live cluster.
2. AI Navigator leverages LangChain to retrieve query-relevant data from the Vector store.
3. AI Navigator sends relevant data from your cluster to our production service to be transmitted and processed by the Azure OpenAI Service.
4. The Azure OpenAI Service returns an answer to your query while including contextual cluster information.

What Type of Data is Collected and Processed?

AI Navigator collects the following types of data:

- `nodes`
- `pods`
- `services`
- `events`
- `endpoints`
- `deployments`
- `statefulsets`
- `daemonsets`
- `replicasets`
- `ingresses`
- `jobs`
- `cronjobs`
- `helm.toolkit.fluxcd.io/helmreleases`

For Which Purpose is My Cluster Data Being Processed?

We prioritize your data's privacy and security. When you enable this feature, any potentially sensitive information from your Pro or Management cluster will be securely transmitted and processed by our trusted cloud service provider, Azure OpenAI. This data is used exclusively to add additional context to your queries to provide to the Cluster Info Agent service. We adhere to strict data protection and privacy standards, ensuring responsible handling of your information.

Use of the AI Navigator chatbot with the Cluster Info Agent is governed by the terms of the Nutanix License and Service Agreement, the Nutanix privacy policy and the Azure OpenAI privacy policy. For more information, see https://learn.microsoft.com/en-us/legal/cognitive-services/openai/data-privacy.

Do I Have Control Over Where or How Data is Stored?

You cannot modify the way the chatbot processes or stores data. However, the AI Navigator Cluster Info Agent application is opt-in, which means that you can choose to enable or disable it at your discretion. For more information, see Customizing the AI Navigator Cluster Info Agent on page 1215.

Your informed consent is paramount, and we ensure clarity by offering a straightforward option to enable or disable this feature according to your preference.

14

NUTANIX KUBERNETES PLATFORM INSIGHTS GUIDE

Nutanix Kubernetes Platform Insights (NKP Insights) tool is a predictive analytics solution for the NKP .

NKP for Kubernetes v1.29.6. Kubernetes® is a registered trademark of The Linux Foundation in the United States and other countries and is used pursuant to a license from The Linux Foundation.

Nutanix Kubernetes Platform Insights Overview

Overview of the NKP Insights product.

Nutanix Kubernetes Platform Insights (NKP Insights) is a predictive analytics solution that detects current and future anomalies in workload configurations or Kubernetes clusters that can or could occur.

It consists of two components:

- **NKP Insights Management** runs on the NKP Management cluster.
- **NKP Insights Engine** runs on each Attached or Managed Kubernetes cluster or on the NKP Management cluster for NKP Pro (single-cluster environments) customers.

When the NKP Insights engine detects an anomaly, it sends an insight alert to NKP Management and displays a summary in the Insights Alert table.

An Insight alert details corresponding to an insight alert provide the anomaly description, root cause analysis, and recommended steps to resolve the anomaly at a much lower MTTR in a Management Cluster and Managed or Attached Clusters. This maximizes your production environment uptime and saves you time and money.

NKP Insights assists Kubernetes Administrators or Application Owners with routine tasks such as:

- Resolving common anomalies
- Checking security issues
- Verifying whether workloads follow best practices

Access the NKP Insights Alert table by selecting **Insights** from the left-side navigation menu.

You must explicitly enable the NKP Insights Engine to explicitly allow the NKP to Insights Engine on each Attached cluster or Management cluster. The NKP Insights Setup and Configuration on page 1219 section contains instructions for configuring these components.

NKP Insights Alert Table

The Nutanix Kubernetes Platform Insights(NKP Insights) Alert table provides an overview of all insight alerts, such as the type of the alert and some basic summary information. From the Alert table, select an insight alert to view the details.

NKP Insights Engine

The Nutanix Kubernetes Platform Insights (NKP Insights) Engine collects events and metrics on your Attached cluster and uses rule-based heuristics to detect potential anomalies of varying criticality, which helps you quickly identify and resolve any anomalies. The NKP Insights alert summaries display in the NKP Insights Alert table, where you can filter and sort insight alerts for a selected cluster or project by:

- Project name
- Cluster name
- Description
- Type

From the NKP Insights Alert table, you can also toggle by each Severity level:

- Critical
- Warning
- Notice

NKP Insights Architecture

This diagram details the architecture for Nutanix Kubernetes Platform Insights (NKP Insights).

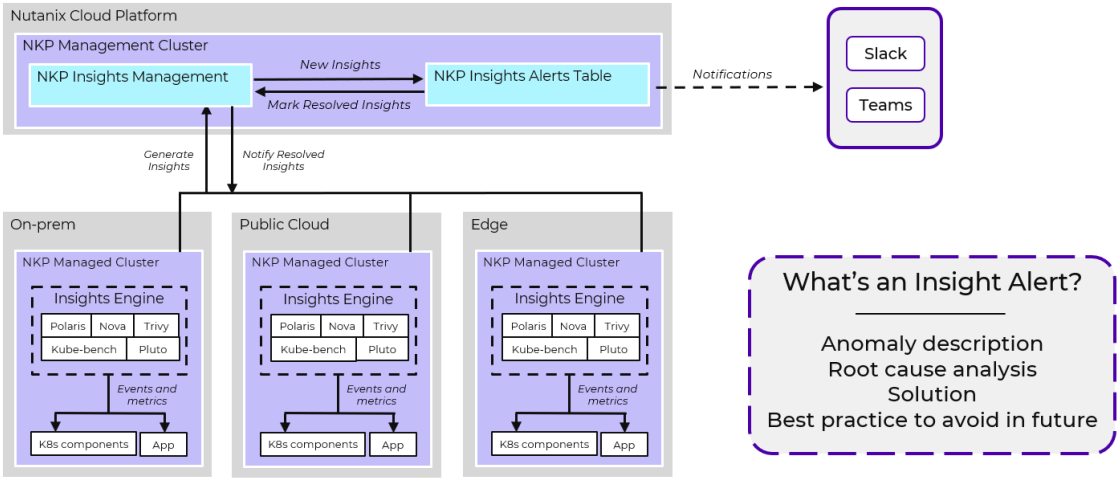


Figure 35: NKP Insights Architecture

Nutanix Kubernetes Platform Insights Setup

Nutanix Kubernetes Platform Insights (NKP Insights) Management installs installed by default on your Management Cluster as a Platform Application. You must install NKP Insights on each managed or attached cluster you want to monitor.

NKP Insights Resource Requirements

Table of the resource requirements needed to use Nutanix Kubernetes Platform Insights.

Table 72: NKP Insights Resource Requirements

Common Name	Application Name	Minimum Resources Suggested	Minimum Persistent Storage Required	Default Priority Class
NKP Insights Management	nkp-insights-management	CPU: 100m Memory: 128Mi		NKP Critical (100002000)
NKP Insights Engine	nkp-insights-backend-deployment	CPU: 250m Memory: 128Mi		NKP Critical (100002000)
	nkp-insights-backend-reforwarder	CPU: 100m Memory: 64Mi		
	nkp-insights-postgresql	CPU: 256Mi Memory: 250m	# of PVs: 1 PV Sizes: 8Gi	
	nkp-insights-resolution	CPU: 100m Memory: 64Mi		

NKP Insights Setup and Configuration

This chapter contains the following setup and configuration topics:

Install NKP Insights

This section describes installing Nutanix Kubernetes Platform Insights (NKP Insights) based on your license type.

Ultimate License Installation

Setup and Configuration for the NKP Ultimate License Installation.

Installation of Nutanix Kubernetes Platform Insights (NKP Insights) in multi-cluster environments with an Ultimate License.

Prerequisites

Prerequisites for the Ultimate License Installation.

Before starting your installation, complete the following:

- You have installed NKP.
- You have an active license key for NKP Insights.
- Decide whether you want to receive alerts of a single cluster (NKP Pro license) or several clusters (NKP Ultimate license).
- Ensure you have **Rook Ceph**, **Rook Ceph Cluster**, and **Kube Prometheus Stack** deployed on the clusters where you perform the NKP Insights installation.

Note: The Management/Pro cluster comes with Rook Ceph, Rook Ceph Cluster, and Kube Prometheus Stack by default. Deploy Rook Ceph and Rook Ceph Cluster on any Managed or Attached clusters using the UI or CLI.

Installing NKP Insights Ultimate License

NKP Insights Ultimate License Installation task.

Before you begin

Enable Nutanix Kubernetes Platform Insights (NKP Insights) using the UI or the CLI. The application will not start functioning until you apply for a license key.

About this task

NKP Insights consists of two applications:

- The **Insights Management** application or `nkp-insights-management`.
- The **Insights Engine** application or `nkp-insights`.

The *Insights Management* application is enabled by default in the Management cluster. You must enable the *Insights Engine* application on the clusters you want to monitor.

Procedure

1. Enable the Insights Engine or `nkp-insights` on the clusters you want to monitor. There are several options:
   - » You can enable NKP Insights per cluster or workspace using the UI.
   - » You can enable NKP Insights per workspace using the CLI. This enables `nkp-insights` in all clusters in the target workspace.
   - » You can enable NKP Insights per cluster using the CLI. This enables `nkp-insights` in selected clusters within a workspace.
2. (Optional) Nutanix recommends enabling the Insights Engine Application on the `kommander` workspace to monitor your Management cluster.

   Note: If you only want to monitor workload clusters, skip this step.

   a. Enable NKP Insights on the Management Cluster Workspace using the UI.

   b. Create an AppDeployment for NKP Insights in the `kommander` namespace (the workspace name is `kommander-workspace`). Specify the correct application version in the latest Release Notes from the applications table.

   For example:

   ```
   nkp create appdeployment nkp-insights --app nkp-insights-1.2.1 --workspace
    kommander-workspace
   ```

3. Apply a valid NKP Insights license key to allow NKP Insights to start analyzing your environment. NKP Insights now displays alerts for clusters where you have installed the Insights Engine (`nkp-insights`).

Upgrade NKP Insights to 1.0.0

NKP supports upgrading Nutanix Kubernetes Platform Insights (NKP Insights). Upgrades are supported from version 2.6.0 to versions 2.6.x and 2.7.x. Here is an overview:

Table 73: NKP Insights Upgrade Support Matrix

Current Version	Desired Upgrade Version	Support
0.4.x (DKP 2.5.0)	0.5.x (DKP 2.6.x)	No
0.5.0 (DKP 2.6.0)	0.5.1 (DKP 2.6.1)	Yes
0.5.0 (DKP 2.6.0)	1.0.x (DKP 2.7.x)	Yes
0.5.1 (DKP 2.6.1)	1.0.x (DKP 2.7.x)	Yes

For the **supported upgrade**, both Insights applications (*Insights Management* and *Insights Engine*) are upgraded automatically as part of the DKP upgrade.

Unsupported upgrade paths require uninstalling DKP Insights, upgrading DKP, and installing the new version of DKP Insights. See https://docs.nutanix.com/dins/2.6/dkp-insights-upgrade for a comprehensive example.

Grant View Rights to Users Using the UI

Overview of the Admin task for granting view rights to users using the NKP Insights UI.

Only admin users can access the Insights section of the NKP UI and the detailed alert view by default.

To allow additional users and user groups to view these Insights resources, create roles with rights to view them. Then, assign these roles to users or user groups.

> **Note:** Access control to Insights summary cards and Insights Alert Details is performed via Kubernetes RBAC based on the namespace to which the Insight Alert is tied.

Workspace-Based Access Control

Workspace-based tasks for adding view rights to Nutanix Kubernetes Platform Insights.

This chapter contains the following Workspace-Based Access Control topics:

Creating a Role with View Rights to Summary Cards

Admin task for creating a workspace-based role with view rights to Nutanix Kubernetes Platform Insights summary cards.

About this task

When assigned, this role allows users and user groups to view the summary table of all Nutanix Kubernetes Platform Insights alerts for all workspaces and projects.

Procedure

1. Select the **Management Cluster Workspace**. The workspace selector is located at the top navigation bar. (Option available for Ultimate customers only)
2. Select **Administration** > **Access Control** in the sidebar menu.
3. Select **Create Role**, and add a **Role Name**

4. Select **NKP Role**, as you provide access to NKP UI resources.
5. Select **+ Add Rule** in the **Rules** section.
6. Enter the following information:

Field	Value
Resources	`insights`
Resource Names	[Leave this field empty]
API Groups	`nkp-insights.nutanix.io`
Verbs	`get`, `list` and `watch`

7. Select **Save** to exit the rule configuration window.
8. Select **Save** again to create the new role.
9. Assign the roles you created to a user group as explained in Workspace Role Bindings in the Nutanix Kubernetes Platform Guide.

> **Note:** It will take a few minutes to create the resource.

Creating a Role with View Rights to NKP Insights Alert Details

Admin task for creating a project-based role with view rights to Nutanix Kubernetes Platform Insights details.

About this task

When assigned to a user or user group, this role allows them to view alert details for an alert generated in a specific project.

Procedure

1. Select the workspace you want to grant view rights to. The workspace selector is located at the top navigation bar. (Option available for Ultimate customers only)
2. Select **Projects** in the sidebar menu. Select or create a Project for which you want to create a role.
3. Select the **Roles** tab, and **Create** Role.
4. Assign a name to the role.
5. Select **Role**, as you provide access to Insights resources across clusters.
6. Select **+ Add Rule** in the **Rules** section.
7. Enter the following information:

Field	Value
Select Rule Type	Resources
Resources	`insights`, `rca`, `solutions`
Resource Names	[Leave this field empty]
API Groups	`virtual.backend.nkp-insights.nutanix.io`
Verbs	`get`

8. Select Save to exit the rule configuration window and Save again to create the new role.
9. Assign the role you created to a user group as explained in Workspace Role Bindings.
10. If you want to grant view rights to the alert details for clusters in another Workspace, repeat the same procedure on a per-workspace basis.

> **Note:**
>
> - It will take a few minutes for the resource to be created.
> - `insights`, `rca`, `solutions` are virtual resources and are not listed as a Kubernetes API resource.

Project-Based Access Control
Project-based tasks for adding view rights to Nutanix Kubernetes Platform Insights.

This chapter contains the following Project-Based Access Control topics:

Creating a Role with View Rights to Summary Cards

Admin task for creating a project-based role with view rights to Nutanix Kubernetes Platform Insights summary cards.

About this task

When assigned, this role allows users and user groups to view the summary table of all Nutanix Kubernetes Platform Insights alerts for all workspaces and projects.

Procedure

1. Select the **Management Cluster Workspace**. The workspace selector is located at the top navigation bar. (Option available for Ultimate customers only)
2. Select **Projects** in the sidebar menu.
3. Select or create a Project for which you want to create a role.
4. Select the **Roles** tab and **Create** Role.
5. Select **NKP Role**, as you are providing access to NKP UI resources, and add a **Role Name**
6. Select **+ Add Rule** in the **Rules** section.
7. Enter the following information:

Field	Value
Resources	`insights`
Resource Names	[Leave this field empty]
API Groups	`nkp-insights.nutanix.io`
Verbs	`get`, `list` and `watch`

8. Select **Save** to exit the rule configuration window.
9. Select **Save** again to create the new role.

10. Assign the roles you created to a user group as explained in Workspace Role Bindings in the Nutanix Kubernetes Platform Guide.

    Note: It will take a few minutes to create the resource.

Creating a Role with View Rights to Insights Alert Details

Admin task for creating a workspace-based role with view rights to Nutanix Kubernetes Platform Insights details.

About this task

When assigned to a user or user group, this role allows them to view alert details for an alert generated in a specific workspace.

Procedure

1. Select the workspace you want to grant view rights to. The workspace selector is located at the top navigation bar. (Option available for Ultimate customers only)
2. Select **Administration** > **Access Control** in the sidebar menu.
3. Select the **Cluster Roles** tab, and **Create** Role.
4. Provide a name for the role.
5. Select **Cluster Roles**, as you provide access to Insights resources across clusters.
6. Select **+ Add Rule** in the **Rules** section.
7. Enter the following information:

Field	Value
Select Rule Type	Resources
Resources	`insights`, `rca`, `solutions`
Resource Names	[Leave this field empty]
API Groups	`virtual.backend.nkp-insights.nutanix.io`
Verbs	`get`

8. Select Save to exit the rule configuration window and Save again to create the new role.
9. Assign the role you created to a user group as explained in Workspace Role Bindings.
10. If you want to grant view rights to the alert details for clusters in another Workspace, repeat the same procedure on a per-workspace basis.

    Note:

    - It will take a few minutes for the resource to be created.
    - `insights`, `rca`, `solutions` are virtual resources and are not listed as a Kubernetes API resource.

Uninstall NKP Insights

Overview of the applications you need to disable for uninstalling NKP Insights.

Nutanix Kubernetes Platform Insights (NKP Insights) consists of two applications:

- The **Insights Management** application or `nkp-insights-management`.
- The **Insights Engine** application or `nkp-insights`.

To uninstall NKP Insights, disable both applications.

Disable NKP Insights Engine on the Management or Pro Cluster
Overview of disabling the NKP Insights Engine.

Disable the NKP Insights Engine application on your Management or Pro cluster. You can use the UI or the CLI to do this.

Disabling NKP Insights Engine Using the UI

About this task

Disable the Nutanix Kubernetes Platform Insights (NKP Insights) Engine application on the Management or Pro cluster using the UI:

Procedure

1. Access the NKP UI.
2. Select the **Management Cluster Workspace** from the top menu bar.
3. Select **Applications** and search for NKP Insights.
4. Select **Uninstall** from the three-dot menu in the application card.
5. Confirm that you want to uninstall by following the pop-up window.
6. Wait until the application is removed completely before you continue deleting persistent volume claims.
7. Verify that the application has been removed completely:
   a. Verify that the application has been removed completely:
   b. Ensure NKP Insights is no longer deployed.

   > **Note:** The NKP Insights Engine can take several minutes to delete completely.

Disabling NKP Insights Engine Using the CLI

About this task

Disable the Nutanix Kubernetes Platform Insights (NKP Insights) Engine application on the Management or Pro cluster using the CLI:

Procedure

1. Disable the NKP Insights Engine application on the Management/Pro cluster by deleting the `nkp-insights` AppDeployment:

   ```
   kubectl delete appdeployment -n kommander nkp-insights
   ```

2. Wait until the `HelmRelease` is removed:

```
kubectl -n kommander wait --for=delete helmrelease/nkp-insights --timeout=5m
```

> **Note:** The Insights Engine can take several minutes to delete completely.

Disable NKP Insights Engine on Additional Clusters

This is only required in Ultimate environments where you have installed Nutanix Kubernetes Platform Insights (NKP Insights) on additional Managed or Attached clusters.

Disabling NKP Insights Engine on Additional Clusters via the UI

About this task

Disable the Nutanix Kubernetes Platform Insights (NKP Insights) Engine application on additional clusters using the UI

Procedure

1. Access the NKP UI.
2. Select the target **Workspace** from the top menu bar.
3. Select **Applications** and search for NKP Insights.
4. Select **Uninstall** from the three-dot menu in the application card.
5. Confirm that you want to uninstall by following the pop-up window.
6. Wait until the application is entirely removed before you continue deleting persistent volume claims.
7. Verify that the application has been removed entirely from a Managed or Attached cluster.
   a. Select the target Workspace > **Clusters** > **View Details** > **Applications** tab.
   b. Ensure Insights is no longer deployed.

> **Note:** The Insights Engine can take several minutes to delete completely.

Disabling NKP Insights Engine on Additional Clusters via CLI

About this task

Export the environment variable for the target workspace:

Procedure

1. To list all workspaces and their namespaces using the command `kubectl get workspaces`

```
export WORKSPACE_NAMESPACE=<target_workspace_namespace>
```

2. Disable the Nutanix Kubernetes Platform Insights (NKP Insights) Engine application on all attached/managed clusters by deleting the `nkp-insights` AppDeployment:

```
kubectl delete appdeployment -n ${WORKSPACE_NAMESPACE} nkp-insights
```

3. Wait until the `HelmRelease` is removed:

```
kubectl -n ${WORKSPACE_NAMESPACE} wait --for=delete helmrelease/nkp-insights --
timeout=5m
```

> **Note:** The Insights Engine can take several minutes to delete completely.

Deleting Persistent Volumes on Additional Clusters

About this task

Ensure you delete all remaining data by deleting Insights-related PVs. This is only required in Ultimate environments where you have installed Nutanix Kubernetes Platform Insights (NKP Insights) on additional Managed or Attached clusters.

Procedure

1. Set the environment variable for the additional cluster using the command `export KUBECONFIG=<attached/managed_cluster_kubeconfig>`
2. Delete all remaining data from the Engine clusters on any managed or attached clusters. To execute the following command:

```
kubectl delete pvc \
data-nkp-insights-postgresql-0 \
-n ${WORKSPACE_NAMESPACE}
```

> **Note:**
>
> - Ensure your configuration references the cluster where the Insights Engine is installed. For more information, see the *Provide Context for Commands with a kubeconfig File* in the Nutanix Kubernetes Platform Guide.
> - Ensure your configuration references the correct `${WORKSPACE_NAMESPACE}`.

3. Delete Insights-related data usng the command `kubectl delete insights --all -A`.

Disabling NKP Insights Management

Procedure

Disable the Nutanix Kubernetes Platform Insights (NKP Insights) Management application on the Management/Pro cluster by deleting the `nkp-insights-management` AppDeployment:

```
kubectl delete appdeployment -n kommander nkp-insights-management
```

NKP Insights Bring Your Own Storage (BYOS) to Insights

Ceph can be used as the CSI Provisioner in some environments. For environments where Ceph was installed before installing NKP, you can reuse your existing Ceph installation to satisfy the storage requirements of NKP Applications.

> **Note:** This guide assumes you have a Ceph cluster that NKP does not manage.
>
> For information on configuring the Ceph instance installed by NKP for use by NKP platform applications, see the *Rook Ceph Configuration* chapter in the Nutanix Kubernetes Platform Guide.

This guide also assumes that you have already disabled NKP Managed Ceph. For more information, see the *BYOS (Bring Your Own Storage) to NKP Clusters* | *Disable-NKP-Managed-Ceph* chapter in the Nutanix Kubernetes Platform Guide.

Requirements

The following are required for using Nutanix Kubernetes Platform Insights with your storage:

- You only need to disable the Object Bucket claim if you use an S3 Provider that does not use an object bucket claim.
- If you disable the Object Bucket Claim, then a S3 bucket needs to be created.
- If you create the bucket, it must support the following:
  - At least 1GB of storage
  - TTL is set to a value of “n days” per S3 spec example: 7 Days. Insights will set TTL to this value on initialization, which fails if it can not be set.
  - we assume the storage is hosted in the same cluster with fast networking access.
  - The bandwidth usage is ~100Mb over the course of a day
  - Latency and speed should be <10 ms and >1Gbs

Create a Secret to support BYOS for Nutanix Kubernetes Platform Insights

About this task

You must create a secret with the AWS S3 credentials for the S3 bucket created for Insights.

Procedure

Create a secret in the same namespace where you installed NKP Insights

```
# Set to the workspace namespace insights is installed in
export WORKSPACE_NAMESPACE=kommander

# Replace your AWS S3 credentials
kubectl create secret generic nkp-insights -n ${WORKSPACE_NAMESPACE} --from-
literal='AWS_ACCESS_KEY_ID=<<AWS Key>>' --from-literal='AWS_SECRET_ACCESS_KEY=<<AWS
 secret>>'
```

Helm Values for Insights Storage

The Helm Values for Insights storage and each component's description are as follows:

```
backend:
  s3:
    port: 80
    region: "us-east-1"
    endpoint: "rook-ceph-rgw-nkp-object-store"
    bucketSize: "1G"
    storageClassName: nkp-object-store
    enableObjectBucketClaim: true
cleanup:
  insightsTTL: "168h"
```

Note: Object bucket claims (OBC) are a custom resource that declares object storage.

Ceph is one provider that uses custom resource definitions (CRD).

If you are using Ceph or another provider that supports object bucket claims, you want to leave it on. This creates an OBC as part of the installation. If you're going to use S3 directly or create the storage container manually, you should turn it off.

Table 74: Helm Values for Insights Storage

Name	Default Value	Description
`port`	`80`	Port of S3 storage provider.
`region`	`us-east-1`	AWS Region for S3 storage provider. It may only be needed for some providers. (Set to a dummy value.)
`endpoint`	`rook-ceph-rgw-nkp-object-store`	Endpoint URL for S3 storage provider. Exclude HTTP://
`bucketSize`	`1G`	Bucket size of bucket created in with Object Bucket Claim.
`storageClassName`	`nkp-object-store`	Storage class to use for the Object Bucket Claim.
`enableObjectBucketClaim`	`true`	To bring your own storage other than Ceph, set it to false. This will require you to create your bucket manually outside of the insights installation.
`insightsTTL`	`168h`	The time in hours spent maintaining insights data in the database and S3. For S3, this is rounded up to the nearest day.

Installing NKP Insights Special Storage in the UI

About this task

For Insights installed with special storage, complete the following task:

Procedure

Add the following in the UI:

Note: For more information on editing the values, see the Enable Nutanix Kubernetes Platform Insights (NKP Insights) Engine in an Air-gapped Environment chapter.

```
backend:
  s3:
    port: 80
    region: "us-east-1"
    endpoint: "rook-ceph-rgw-nkp-object-store"
    bucketSize: "1G"
    storageClassName: nkp-object-store
    enableObjectBucketClaim: true
cleanup:
  insightsTTL: "168h"
```

Installing Insights Storage using CLI

About this task

To configure storage for Insights via the CLI, complete the following:

Procedure

1. Create the `ConfigMap` with the name provided in the step above, with the custom configuration:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: ${WORKSPACE_NAMESPACE}
  name: nkp-insights-overrides
data:
  values.yaml: |
  # helm values here
    backend:
      s3:
        port: 80
        region: "us-east-1"
        endpoint: "rook-ceph-rgw-nkp-object-store"
        bucketSize: "1G"
        storageClassName: nkp-object-store
        enableObjectBucketClaim: true
    cleanup:
      insightsTTL: "168h"
EOF
```

> **Note:** Kommander waits for the `ConfigMap` to be present before deploying the `AppDeployment` to the managed or attached clusters.

2. Provide the name of a `ConfigMap` in the `AppDeployment`, which provides a custom configuration on top of the default configuration:

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.kommander.nutanix.io/v1alpha2
kind: AppDeployment
metadata:
  name: nkp-insights
  namespace: ${WORKSPACE_NAMESPACE}
spec:
  appRef:
    kind: App
    name: nkp-insights-0.4.1
  configOverrides:
    name: nkp-insights-overrides
EOF
```

Installing Nutanix Kubernetes Platform Insights in an Air-gapped Environment

About this task

Follow the steps below to configure storage for Nutanix Kubernetes Platform Insights (NKP Insights) in air-gapped environments:

Procedure

In `kommander.yaml`, enable NKP Insights and NKP Catalog Applications by setting the following:

```
apiVersion: config.kommander.mesosphere.io/v1alpha1
kind: Installation
apps:
  ...
  nkp-insights-management:
    enabled: true
    # helm values here
    backend:
      s3:
        port: 80
        region: "us-east-1"
        endpoint: "rook-ceph-rgw-nkp-object-store"
        bucketSize: "1G"
        storageClassName: nkp-object-store
        enableObjectBucketClaim: true
    cleanup:
      insightsTTL: "168h"
...
catalog:
  repositories:
    - name: insights-catalog-applications
      labels:
        kommander.nutanix.io/workspace-default-catalog-repository: "true"
        kommander.nutanix.io/gitapps-gitrepository-type: "nkp"
      path: ./application-repositories/nkp-insights-v2.5.0.tar.gz
    - name: nkp-catalog-applications
      labels:
        kommander.nutanix.io/project-default-catalog-repository: "true"
        kommander.nutanix.io/workspace-default-catalog-repository: "true"
        kommander.nutanix.io/gitapps-gitrepository-type: "nkp"
      path: ./application-repositories/nkp-catalog-applications-v2.5.0.tar.gz
```

What to do next

For more information, see the *Install Air-gapped Kommander with NKP Insights and NKP Catalog Applications* in the Nutanix Kubernetes Platform Guide.

Manually Creating the Object Bucket Claim

About this task

If you need to manually create an OBC, such as when you do not want the Helm Chart to generate one automatically, follow these steps:

Procedure

If needed, an `ObjectBucketClaim` can be created manually in the same namespace as `nkp-insights`. This results in the creation of `ObjectBucket` , which creates a `Secret` consumed by `nkp-insights`.

```
For nkp-insights:
cat <<EOF | kubectl apply -f -
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  name: nkp-insights
  namespace: ${NAMESPACE}
spec:
  additionalConfig:
    maxSize: 1G
```

```
  bucketName: nkp-insights
  storageClassName: nkp-object-store
EOF
```

Note:

- Bucket name cannot be changed.
- The storage class and the maxSize can be configured as needed.

Upgrade NKP Insights to 1.0.0

NKP supports upgrading Nutanix Kubernetes Platform Insights (NKP Insights). Upgrades are supported from version 2.6.0 to versions 2.6.x and 2.7.x. Here is an overview:

Table 75: NKP Insights Upgrade Support Matrix

Current Version	Desired Upgrade Version	Support
0.4.x (DKP 2.5.0)	0.5.x (DKP 2.6.x)	No
0.5.0 (DKP 2.6.0)	0.5.1 (DKP 2.6.1)	Yes
0.5.0 (DKP 2.6.0)	1.0.x (DKP 2.7.x)	Yes
0.5.1 (DKP 2.6.1)	1.0.x (DKP 2.7.x)	Yes

For the **supported upgrade**, both Insights applications (*Insights Management* and *Insights Engine*) are upgraded automatically as part of the DKP upgrade.

Unsupported upgrade paths require uninstalling DKP Insights, upgrading DKP, and installing the new version of DKP Insights. See https://docs.nutanix.com/dins/2.6/dkp-insights-upgrade for a comprehensive example.

Nutanix Kubernetes Platform Insights Alerts

Note: A detected Nutanix Kubernetes Platform Insights (NKP Insights) Alert persists for 72 hours. If an existing Insight Alert is not updated in that period, it expires and is removed from the table. If the anomaly recurs, the Insight Alert reappears in the table.

After enabling the NKP Insights Engine in a Managed or Attached cluster, select a workspace in the NKP UI that includes the cluster to view the NKP Insights Alerts. An Insights summary card in the **Dashboard** tab displays the most recent Insight Alerts, as well as the number of insights within each severity level of *Critical*, *Warning*, and *Notices*.

Select **View All** from the NKP Insights summary card or **Insights** from the sidebar to access the NKP Insights Alert table. This table provides an overview of all the NKP Insights Alerts. You can filter these NKP Insights Alerts in several different ways:

- Use the search dialog to search by description keyword.

- View by status:

  Note: *Muted* and *Resolved* status are manually set, as described below.

  - Open
  - Muted
  - Resolved
- Toggle your view by the following NKP Insights types:
  - All types
  - Availability
  - Best Practices
  - Configuration
  - Security
- Select **All Clusters** or an individual cluster.
- Select **All Projects** or an individual project.
- Toggle between **All**, **Critical**, **Warning**, and **Notices**.

To clear filters and reset your view to all NKP Insights items, select **Clear All**.

Resolving or Muting Alerts

About this task

Note: When you *resolve* an alert, it is impossible to move it back to *Open* or *Muted*. Ensure you only resolve alerts once you fix the issue.

Procedure

1. From the Insight Alert table, filter and check the boxes for the alerts.

2. From the top of the Insight Alert table, select:

   - **Resolved** if you have resolved the issues or
   - **Mute** if you want to silence the alert

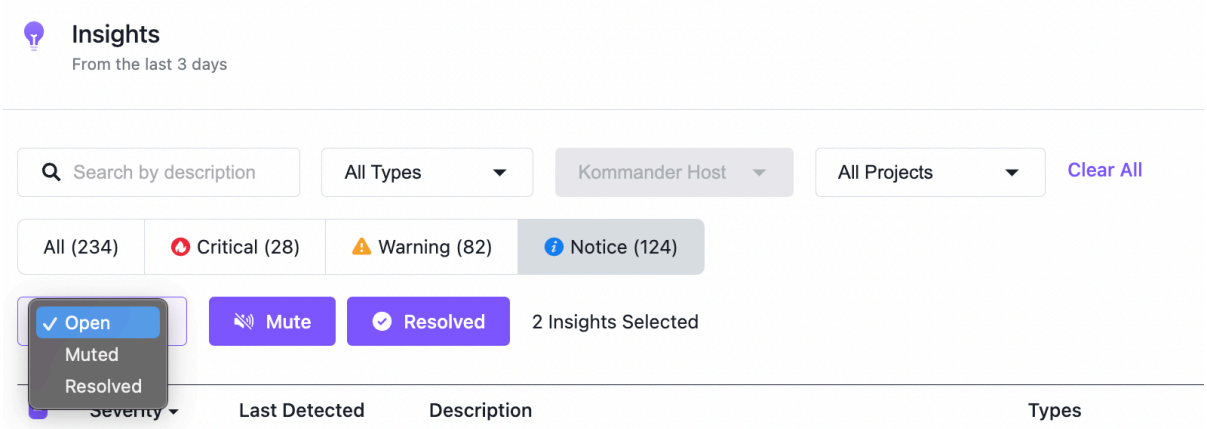


Figure 36: Insights Alerts UI

3. A confirmation prompt for the status change appears once you resolve or mute an Insight Alert.

Viewing Resolved or Muted Alerts

About this task

> **Note:** Once you set an alert to *Resolved* or *Muted*, it does not appear in the *Open* Insight Alert table view.

To view Resolved or Muted Alerts, perform the following steps:

Procedure

From the Insights Alert field, select the desired filter from the drop-down list.

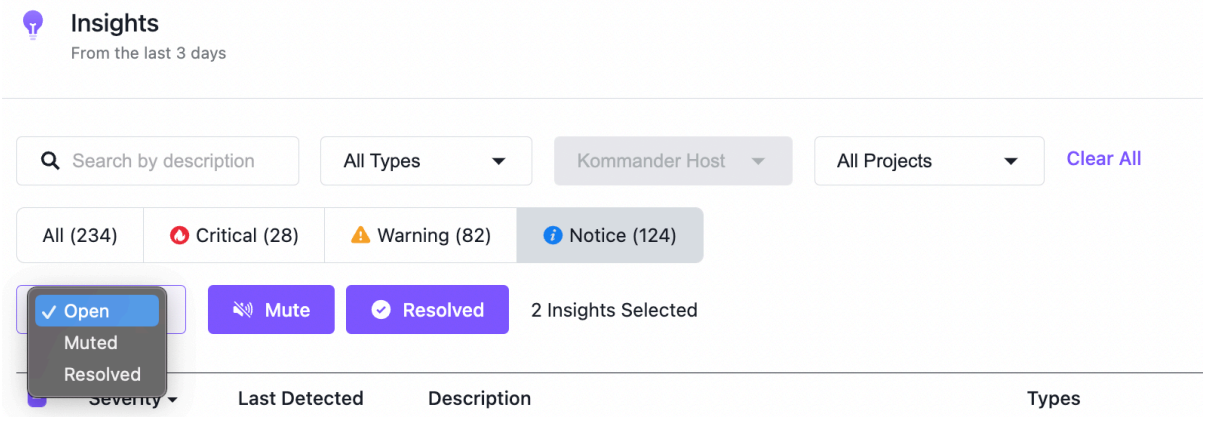


Figure 37: Insights Alerts Filter List

Insight Alert Usage Tips

For an Insight Alert you do not want to use, check the box corresponding to that alert and select **Mute**. This only silences the individual Insight Alert. The Insight remains muted if the anomaly recurs, but the **Last Detected** timestamp is updated.

NKP Insight Alert Details

From the NKP Insight Alert table, select the **Details** link to view additional information.

The standard sections that alerts contain are:

- Severity
- Last Detected
- Description
- Types
- Cluster
- Project, if applicable

Root Cause Analysis

This section provides all the information you need to understand the cause of the anomaly.

Solutions

This section contains recommended steps to resolve the anomaly.

NKP Insights Alert Notifications With Alertmanager

NKP Insights supports the configuration of Kube Prometheus Stack's Alertmanager to send alert notifications to environment administrators and users through Slack and Microsoft Teams.

> **Note:** You can configure NKP Insights to send notifications to other communication platforms like PagerDuty or e-mail. However, we have only included examples for Slack and Microsoft Teams.

Table 76: Common Questions about Kube Prometheus Stack's Alertmanager

Why Should I Send Notifications for NKP Insights Alerts?	Activating this feature eliminates the need to check your cluster's health manually. NKP Insights, combined with Alertmanager, can automatically warn users about critical issues. They can then take measures to keep your environment healthy and avoid possible downtime.

How Do NKP Insights and Alertmanager Work Together?	Alertmanager acts as a central component for managing and routing alerts. It is available by default in your NKP installation and automatically monitors several NKP-defined alerts. By enabling NKP Insights to route alerts to Alertmanager, you add another source of alerting. In the examples provided in this section, you use an `AlertmanagerConfig` YAML file to enable Alertmanager to group and filter NKP Insights alerts according to rules and send notifications to a communication platform. **Note:** You add new configurations by applying the #AlertmanagerConfig# example files referenced in this section. Existing default or custom Alertmanager configurations remain unaffected.
What Type of Configuration Options Are Possible?	In the `AlertmanagerConfig` object, you can define the following parameters: **Routes:** Routes define which alert types generate notifications and which do not. In the provided examples, we configure Alertmanager to send notifications for all **Critical** and **Warning** NKP Insights based on Severity. **Receivers:** Receivers define the communication platform where you want to receive the notifications. The provided examples show how to configure notifications for Slack and Microsoft Teams. **Message content and format:** The receiver configuration also defines the display format for the alert message. The examples provide message formatting designed for Slack and Microsoft Teams. The provided notification examples display all the informational fields you can find when looking at an alert in the NKP UI. **Note:** You can customize the `AlertmanagerConfig` YAML file to include other routes, receivers, or a different message formatting. However, this requires advanced knowledge of `AlertmanagerConfig` specifications and Helm and Golang templating rules.
How Do I Enable and Configure Alertmanager?	For more information, see: • *Slack: Send NKP Insights Alert Notifications to a Channel* topics at Slack: Send Nutanix Kubernetes Platform Insights Alert Notification to a Channel. • For configuration templates, see the *Microsoft Teams: Send NKP Insights Alert Notifications to a Channel* topic at Slack: Send Nutanix Kubernetes Platform Insights Alert Notification to a Channel

Slack: Send Nutanix Kubernetes Platform Insights Alert Notifications to a Channel

This page contains information on setting up a configuration for Alertmanager to send alert notifications through Slack. See NKP Insights Alert Notifications With Alertmanager on page 1235 for more information about this function.

Prerequisites

- Kube Prometheus Stack installed on the Management cluster (included in the default configuration)
- A Slack Incoming Webhook created by a Slack workspace admin. For more information, see https://api.slack.com/messaging/webhooks#create_a_webhook.
- Nutanix Kubernetes Platform Insights installed. For more information, see Nutanix Kubernetes Platform Insights Setup on page 1218.

Preparing your Environment

About this task

Complete the following steps to prepare your environment:

Procedure

1. Set your environment variable to the `kommander` workspace namespace:

   ```
   export WORKSPACE_NAMESPACE=kommander
   ```

2. Set the Slack Webhook variable to the URL you obtained from Slack for this purpose: The webhook format is similar to `https://hooks.slack.com/services/T00000000/B00000000/XXXXXXXXXXXXXXXXXXXXXXXX`.

   ```
   export SLACK_WEBHOOK=<endpoint_URL>
   ```

Enable Nutanix Kubernetes Platform Insights to Send Notifications with Alertmanager

About this task

Create an `AlertmanagerConfig` object and apply it on the `kommander` workspace namespace.

Procedure

1. Create a secret for the Alertmanager-Slack integration:

   ```
   kubectl create secret generic slack-webhook -n ${WORKSPACE_NAMESPACE} \
     --from-literal=slack-webhook-url=${SLACK_WEBHOOK} \
     --dry-run=client --save-config -o yaml | kubectl apply -f -
   ```

2. Create the AlertmanagerConfig YAML file and name it `alertmanager-slack-config.yaml`. This example allows Alertmanager to send notifications for **all Critical Insights** of all alert types that occur in any workspace to Slack.

   > **Note:** Replace <#target_slack_channel> with the name of the Slack channel where you want to receive the notifications.

   ```
   apiVersion: monitoring.coreos.com/v1alpha1
   kind: AlertmanagerConfig
   metadata:
     name: slack-config
     namespace: kommander
   spec:
     route:
       groupBy: ['source', 'insightClass', 'severity', 'cluster']
       groupWait: 3m
       groupInterval: 15m
   ```

```
   repeatInterval: 1h
   receiver: 'slack'
   routes:
     - receiver: 'slack'
       matchers:
         - name: source
           value: Insights
           matchType: =
         - name: severity
           value: Critical
           matchType: =
       continue: true
 receivers:
   - name: 'slack'
     slackConfigs:
       - apiURL:
           name: slack-webhook
           key: slack-webhook-url
         channel: '#<target_slack_channel>'
         username: Insights Slack Notifier
         iconURL: https://avatars3.githubusercontent.com/u/3380462
         title: |-
           {{ .Status | toUpper -}}{{ if eq .Status "firing" }}: {{ .Alerts.Firing
| len }} {{- end}} Insights Alert{{ if gt (len .Alerts.Firing) 1 }}s{{ end }}
({{ .CommonLabels.insightClass }})
         titleLink: 'https://{{ (index .Alerts 0).Annotations.detailsURL }}'
         text: |-
           {{- if (index .Alerts 0).Labels.namespace }}
             {{- "\n" -}}
             *Namespace:* `{{ (index .Alerts 0).Labels.namespace }}`
           {{- end }}
           {{- if (index .Alerts 0).Labels.severity }}
             {{- "\n" -}}
             *Severity:* `{{ (index .Alerts 0).Labels.severity }}`
           {{- end }}
           {{- if (index .Alerts 0).Labels.cluster }}
             {{- "\n" -}}
             *Cluster:* `{{ (index .Alerts 0).Labels.cluster }}`
           {{- end }}
           {{- if (index .Alerts 0).Annotations.description }}
             {{- "\n" -}}
             *Description:* {{ (index .Alerts 0).Annotations.description }}
           {{- end }}
           {{- if (index .Alerts 0).Annotations.categories }}
             {{- "\n" -}}
             *Categories:* {{ (index .Alerts 0).Annotations.categories }}
           {{- end }}
         actions:
           - text: 'Go to Insight :mag:'
             type: button
             url: 'https://{{ (index .Alerts 0).Annotations.detailsURL }}'
```

3. Apply the `AlertmanagerConfig` file:

```
kubectl -n ${WORKSPACE_NAMESPACE} apply -f alertmanager-slack-config.yaml
```

Microsoft Teams: Send NKP Insights Alert Notifications to a Channel

This page contains information on setting up a configuration for Alertmanager to send alert notifications through Microsoft Teams. See NKP Insights Alert Notifications With Alertmanager on page 1235 for more information about this function.

Prerequisites

- Kube Prometheus Stack installed on the Management cluster (included in the default configuration)
- A Microsoft Teams Incoming Webhook. For more information, see https://learn.microsoft.com/en-us/microsoftteams/platform/webhooks-and-connectors/how-to/add-incoming-webhook?tabs=newteams%2Cdotnet.
- Nutanix Kubernetes Platform Insights installed. For more information, see Nutanix Kubernetes Platform Insights Setup on page 1218.

Preparing your Environment

About this task

Complete the following steps to prepare your environment:

Procedure

1. Set your environment variable to the `kommander` workspace namespace.

   ```
   export WORKSPACE_NAMESPACE=kommander
   ```

2. Set the Microsoft Teams Webhook variable to the URL you obtained from Microsoft Teams for this purpose: The webhook format is similar to: `https://xxxx.webhook.office.com/xxxxxxxxx`.

   ```
   export TEAMS_WEBHOOK=<endpoint_URL>
   ```

Enabling Nutanix Kubernetes Platform Insights to Send Notifications with Alertmanager

About this task

Install an extension for Kube Prometheus Stack that adds compatibility with Microsoft Teams. Then, create an

`AlertmanagerConfig` object and apply it to the `kommander` workspace namespace.

Procedure

1. Add the following repository to enable Microsoft Teams configuration:

   ```
   helm repo add prometheus-msteams https://prometheus-msteams.github.io/prometheus-msteams/
   ```

2. Create a custom configuration of Kube Prometheus Stack, and name it `teams-proxy-config.yaml`Replace `<teams_webhook_URL>` with the webhook you obtained from Microsoft Teams. The format is similar to`https://xxxxx.webhook.office.com/xxxxxxxxx`.

   ```
   replicaCount: 1
   image:
     repository: quay.io/prometheusmsteams/prometheus-msteams
     tag: v1.5.1
   connectors:
     - alertmanager: <teams_webhook_URL>
   container:
     additionalArgs:
       - -debug
   metrics:
     serviceMonitor:
       enabled: true
   ```

```
    additionalLabels:
      release: kube-prometheus-stack-prometheus
    scrapeInterval: 30s
```

3. Create a custom display format for your message in Microsoft Teams message, and name the file `custom-card.tmpl`:

```
{{ define "teams.card" }}
{
  "@type": "MessageCard",
  "@context": "http://schema.org/extensions",
  "themeColor": "{{- if eq .Status "resolved" -}}2DC72D
                {{- else if eq .Status "Firing" -}}
                   {{- if eq .CommonLabels.severity "Critical" -}}8C1A1A
                   {{- else if eq .CommonLabels.severity "Warning" -}}FFA500
                   {{- else -}}808080{{- end -}}
                {{- else -}}808080{{- end -}}",
  "summary": "{{- if eq .CommonAnnotations.description "" -}}
                  {{- if eq .CommonLabels.insightClass "" -}}
                    {{- if eq .CommonLabels.alertname "" -}}
                      Prometheus Alert
                    {{- else -}}
                      {{- .CommonLabels.alertname -}}
                    {{- end -}}
                  {{- else -}}
                    {{- .CommonLabels.insightClass -}}
                  {{- end -}}
              {{- else -}}
                  {{- .CommonAnnotations.description -}}
              {{- end -}}",
  "title": "{{ .Status | toUpper -}}{{ if eq .Status "firing" }}: {{ .Alerts.Firing
 | len }} {{- end}} Insights Alert{{ if gt (len .Alerts.Firing) 1 }}s{{ end }}
 ({{ .CommonLabels.insightClass }})",
  "sections": [ {{$externalUrl := (index .Alerts 0).Annotations.detailsURL }}
    {
      "activityTitle": "[{{ (index .Alerts 0).Annotations.description }}]
({{ $externalUrl }})",
      "facts": [
        {{- if (index .Alerts 0).Labels.namespace }}
        {
          "name": "Namespace:",
          "value": "{{ (index .Alerts 0).Labels.namespace }}"
        },
        {{- end }}
        {{- if (index .Alerts 0).Labels.severity }}
        {
          "name": "Severity:",
          "value": "{{ (index .Alerts 0).Labels.severity }}"
        },
        {{- end }}
        {{- if (index .Alerts 0).Labels.cluster }}
        {
          "name": "Cluster:",
          "value": "{{ (index .Alerts 0).Labels.cluster }}"
        },
        {{- end }}
        {{- if (index .Alerts 0).Annotations.categories }}
        {
          "name": "Categories:",
          "value": "{{ (index .Alerts 0).Annotations.categories }}"
        }
        {{- end }}
```

```
      ],
      "markdown": true
    }
  ]
}
{{ end }}
```

4. Upgrade the Helm values to apply the configuration in Step 4:

```
helm upgrade --install prometheus-msteams \
  --namespace ${WORKSPACE_NAMESPACE} -f teams-proxy-config.yaml \
  --set-file customCardTemplate=custom-card.tmpl \
  prometheus-msteams/prometheus-msteams
```

5. Create the `AlertmanagerConfig` YAML file and name it `alertmanager-teams-config.yaml`. This example allows Alertmanager to send notifications for **all Critical alerts** of all alert types that occur in any workspace to Microsoft Teams.

```
apiVersion: monitoring.coreos.com/v1alpha1
kind: AlertmanagerConfig
metadata:
  name: alertmanager-teams-config.yaml
  namespace: kommander
spec:
  route:
    groupBy: ['source', 'insightClass', 'severity', 'cluster']
    groupWait: 3m
    groupInterval: 15m
    repeatInterval: 1h
    receiver: 'default'
    routes:
      - receiver: 'teams'
        matchers:
          - name: source
            value: Insights
            matchType: =
          - name: severity
            value: Critical
            matchType: =
        continue: true
  receivers:
    - name: 'default'
    - name: 'teams'
      webhookConfigs:
        - url: 'http://prometheus-msteams:2000/alertmanager'
```

6. Apply the `AlertmanagerConfig` file:

```
kubectl -n ${WORKSPACE_NAMESPACE} apply -f alertmanager-teams-config.yaml
```

Verifying that Alertmanager Sends Notifications

Verify the configuration to ensure that alerts are correctly routed to your communication platform and the notifications reach the intended recipients.

Prerequisite

You have enabled an Alertmanager configuration for Slack or Microsoft Teams:

- Slack: Send Nutanix Kubernetes Platform Insights Alert Notifications to a Channel on page 1236

- Microsoft Teams: Send NKP Insights Alert Notifications to a Channel on page 1238

Sending a Test Alert

About this task

Trigger a mock Nutanix Kubernetes Platform Insights alert to confirm the successful configuration.

Procedure

1. Open a local port for the Alertmanager mock alert:

```
kubectl -n kommander port-forward svc/kube-prometheus-stack-alertmanager 8083:9093
```

2. In another terminal session, send a mock alert to the open port:

```
curl -L 'http://localhost:8083/api/v2/alerts' \
-H 'Content-Type: application/json' \
-H 'Accept: application/json' \
-d \
'[{
    "labels":
    {
        "alertname": "Test Insight Alert",
        "namespace": "kommander",
        "status": "Open",
        "source": "Insights",
        "severity": "Critical",
        "cluster": "Kommander Host (Test)"
    },
    "annotations":
    {
        "description": "This is a mock Insight for testing",
        "generatorURL": "https://test-endpoint.com",
        "categories": "Best-Practices, Configuration"
    }
}]'
```

   This sends a **Critical** mock Insights alert to Alertmanager, which triggers sending a notification to the configured communication platform.

Troubleshooting

This chapter contains the following troubleshooting topics:

Verifying the Alertmanager Dashboard

About this task

Verify if Nutanix Kubernetes Platform Insights (NKP Insights) alerts are displayed in the Alertmanager Dashboard. If you can see NKP Insights alerts present, the NKP Insights-Alertmanager route is configured successfully.

> **Note:** You can recognize NKP Insights alerts from other default NKP alerts because the alert severity tags are capitalized. For example, an NKP Insights alert is `Critical`, whereas other non-Insights alerts are `critical`.

Procedure

1. Access the NKP UI.
2. For Ultimate only: Select **Management Cluster Workspace**.
3. Select **Application Dashboards**, and look for the **Prometheus Alert Manager** application card.
4. Select **Dashboard** to open the Alertmanager console.

Verifying the Alertmanager Log Files

About this task

If Slack does not display any Alert messages, but you can see Nutanix Kubernetes Platform Insights (NKP Insights) alerts in the Alertmanager console, perform the following task:

Procedure

Verify the deployment logs using the command `kubectl -n kommander logs alertmanager-kube-prometheus-stack-alertmanager-0`.
If the output is blank, the configuration has been successful. The output displays errors if the deployment has failed.

Enable NKP-Related Insights Alerts

By default, Nutanix Kubernetes Platform Insights (NKP Insights) analyzes your environment and focuses on troubleshooting issues related to your organization's workloads. This default configuration provides a baseline or reference point that you can use to start deploying workloads and observe the alerts they may cause.

However, to ensure compliance with CVE databases or monitor NKP resources and your workload resources, you can allow NKP Insights to analyze and create alerts for underlying NKP resources and Kubernetes components.

You can do so by enabling customization of NKP Insights per workspace.

> **Note:** NKP Insights displays alerts related to `DiskFull` and `PVCFull`, regardless of whether they are rooted in your environment's underlying NKP resources, Kubernetes resources, or one of your production workloads. Ensure you have allocated sufficient disk capacity and have assigned adequate storage in your PVC objects to allow your environment to run uninterruptedly and ensure no data is lost.

Navigating to your NKP Insights Configuration Service

About this task

To customize an Nutanix Kubernetes Platform Insights (NKP Insights) Installation on a per-workspace basis:

Procedure

1. Log in to your NKP UI.
2. **Ultimate only:** Select the target workspace from the top navigation bar.
3. Select **Applications** from the sidebar and search for **NKP Insights**.
4. Select the three-dot menu in the application card, and **Edit** > **Configuration**.

Adding a Custom Configuration to Receive NKP Alerts

About this task

To enable NKP-related Insight alerts, complete the following task:

Procedure

1. Copy the following customization and paste it into the code editor:

```
backend:
  engineConfig:
    dkpIdentification:
       enabled: false
```

2. Select **Save** and exit.
3. Repeat the configuration steps included in this page for each workspace.

Configuration Anomalies

In Kubernetes, a class of problems arises from incorrect or insufficient configuration in workload and Kubernetes cluster deployments. We refer to them as configuration anomalies.

We integrated third-party open-source components into the Nutanix Kubernetes Platform Insights(NKP Insights) Engine that handles specific classes of configuration anomalies:

Polaris

Polaris by Fairwinds is an open-source project that identifies Kubernetes deployment configuration errors. Polaris runs over a dozen checks to help users discover Kubernetes misconfigurations that frequently cause security vulnerabilities, outages, scaling limitations, and more. Using Polaris, you can avoid problems and ensure you’re using Kubernetes best practices.

Polaris checks configurations against a set of best practices for workloads and Kubernetes cluster deployments, such as:

- Health Checks
- Images
- Networking
- Resources
- Security

It informs you about potential problems in configurations through insight alerts.

To see which Polaris version is included in this release, see the Nutanix Kubernetes Platform Insights(NKP Insights) Release Notes.

Enabling or Disabling Polaris Insights

About this task

To enable or disable Polaris insights, complete the following task:

Procedure

1. Edit the Service configuration with the following values:

```
polaris:
  enabled: true
```

2. To modify an installation, select **Workspace** > **Applications** > **NKP-Insights** > **Edit**.

Changing the Frequency of Polaris Audit Scans

About this task

To change the default scan frequency, perform the following task:

Procedure

1. Polaris Audits run by default every 37 minutes and use Cron syntax. You can change the default by editing the Service configuration with the following values:

```
polaris:
  schedule: "@every 37m"
```

> **Note:** For more information on Cron syntax, see https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/#cron-schedule-syntax.

2. To modify an installation, select **Workspace** > **Applications** > **NKP-Insights** > **Edit**.

Modifying Severities of Polaris Insights

About this task

Polaris Audit specifies a default severity for each of these types:

- Security:https://polaris.docs.fairwinds.com/checks/security/
- Efficiency: https://polaris.docs.fairwinds.com/checks/efficiency/
- Reliability: https://polaris.docs.fairwinds.com/checks/reliability/

Procedure

1. You can change these defaults by modifying the Service configuration with the following values:

```
polaris:
  config:
    # See https://github.com/FairwindsOps/polaris/blob/master/examples/config.yaml
    checks:
      # reliability
      deploymentMissingReplicas: warning
      priorityClassNotSet: ignore
      tagNotSpecified: danger
      pullPolicyNotAlways: warning
      readinessProbeMissing: warning
      livenessProbeMissing: warning
      metadataAndNameMismatched: ignore
      pdbDisruptionsIsZero: warning
      missingPodDisruptionBudget: ignore

      # efficiency
```

```
      cpuRequestsMissing: warning
      cpuLimitsMissing: warning
      memoryRequestsMissing: warning
      memoryLimitsMissing: warning
      # security
      hostIPCSet: danger
      hostPIDSet: danger
      notReadOnlyRootFilesystem: warning
      privilegeEscalationAllowed: danger
      runAsRootAllowed: danger
      runAsPrivileged: danger
      dangerousCapabilities: danger
      insecureCapabilities: warning
      hostNetworkSet: danger
      hostPortSet: warning
      tlsSettingsMissing: warning
```

2. To modify an installation, select **Workspace** > **Applications** > **NKP-Insights** > **Edit**.

> **Note:** When you mark a Polaris Audit Insight alert as **Not-Useful**, newly generated alerts are set to the lowest Notice severity.

Adding Exemptions to Polaris Insights

About this task

Context for the current task

Procedure

1. You can exclude a particular workload from a Polaris Audit via its Exemptions. This example shows how to exempt the workload **dummy-deployment**, which currently has an issue where **CPU Limits are Missing**. Change the exceptions list by modifying the Service configuration with the following values:

```
polaris:
  config:
    exemptions:
      - controllerNames:
          - dummy-deployment
        rules:
          - cpuLimitsMissing
```

2. To modify an installation, select **Workspace** > **Applications** > **NKP-Insights** > **Edit**.

Pluto

Pluto by Fairwinds is a tool that scans Live Helm releases running in your cluster for deprecated Kubernetes API versions. It sends an alert about deprecated `apiVersions` deployed in your Helm releases.

In Nutanix Kubernetes Platform Insights(NKP Insights), Pluto scans Live Helm releases running in your cluster for deprecated API versions and sends an alert about any deprecated `apiVersions` deployed in your Helm releases.

To know which Pluto version is included in this release, see the Nutanix Kubernetes Platform Insights(NKP Insights) Release Notes.

For more information on Pluto, see https://pluto.docs.fairwinds.com/

Enabling or Disabling Pluto Insights

About this task

Context for the current task

Procedure

1. Enable or disable Helm release scanning with Pluto Insights by editing the Service configuration with the following values:

```
pluto:
  enabled: true
```

2. To modify an installation, select **Workspace** > **Applications** > **NKP-Insights** > **Edit**.

Changing the Frequency of Pluto Scans

About this task

To change the default scan frequency, perform the following task:

Procedure

1. Pluto scans run by default every 41 minutes and uses Cron syntax. You can change the default by editing the values of the Service configuration:

```
pluto:
  schedule: "@every 41m"
```

2. To modify an installation, select **Workspace** > **Applications** > **NKP-Insights** > **Edit**.

Severities of Pluto Insights

Table 77: Pluto Alert Severities

Pluto Result	Insights Alert Level	Meaning
Deprecated	Warning	Kubernetes API is scheduled to be removed in a future version of Kubernetes.
Removed	Critical	Kubernetes API has been removed in the current running version of Kubernetes.

Nova

Nova by Fairwinds adds the ability for the Insights engine to check the helm chart version of the current workload deployment. It scans the latest helm chart version available from the configured Helm repositories and then sends a structural Insight alert if there is an issue. The alert details show an RCA and a solution to resolve the problem.

Nova adds the ability for the Insights engine to check the helm chart version of the current workload deployment. It scans the latest helm chart version available from the helm repository and then sends a structural insight alert if there is an issue. The alert details show an RCA and a solution to resolve the problem.

To know which Nova version is included in this release, see the Nutanix Kubernetes Platform Insights(NKP Insights) Release Notes.

For more information on Nova, see https://nova.docs.fairwinds.com/.

Enabling or Disabling Nova Insight

About this task

Edit the Service configuration:

Procedure

1. Set the `nova.enabled` value to `true`
2. Set the `helmRepositoryURLs` to the URLs for the Helm repositories used by your workloads where you want Helm chart versions to be scanned.

```
nova:
  enabled: true
  helmRepositoryURLs:
  - https://charts.bitnami.com/bitnami/
  - https://charts.jetstack.io
```

3. To modify an installation, select **Workspace** > **Applications** > **NKP-Insights** > **Edit**.

Changing the Frequency of Nova Scans

About this task

To change the default scan frequency, perform the following task:

Procedure

1. Nova runs every 37 minutes by default and uses the Cron syntax. You can change the default by editing the Service configuration with the following values:

```
nova:
  schedule: "@every 34m"
```

2. To modify an installation, select **Workspace** > **Applications** > **NKP-Insights** > **Edit**.

Trivy

> **Note:** This function is disabled in the default configuration of Nutanix Kubernetes Platform Insights(NKP Insights) .

This and later versions of Insights come with CVE scanning functionality for customer-deployed workload clusters and deployments.

CVE/CIS databases are updated every couple of hours. When enabled, the CVE scanning feature scans these databases and runs an analysis against your workloads to flag any potential security issues.

Trivy is an open-source vulnerability and misconfiguration scanner that scans to detect vulnerabilities in:

- Container Images
- Rootfs
- Filesystems

To know which Trivy version is included in this release, see the NKP Insights Release Notes.

For more information on Trivy, see https://aquasecurity.github.io/trivy/v0.44/docs/scanner/vulnerability/.

Enabling or Disabling Trivy Insights

About this task

Enable or disable CVE scanning with Trivy Insights by completing this task.

Procedure

1. Edit the Service configuration with the following values:

```
trivy:
  enabled: true
```

2. To modify an installation, select **Workspace** > **Applications** > **NKP-Insights** > **Edit**.

Changing the Frequency of Trivy CVE Scans

About this task

To change the default scan frequency, perform the following task:

Procedure

1. Trivy scans run by default every 2 hours and uses Cron syntax. You can change the default by editing the values of the Service configuration:

```
trivy:
  schedule: "@every 2h"
```

2. To modify an installation, select **Workspace** > **Applications** > **NKP-Insights** > **Edit**.

Severities of Trivy Insights

Table 78: Trivy Alert Severities

Trivy Severity Leve	Insights Alert Level	Example (depends on the categorization of the source database)
CRITICAL	Critical	Denial of crucial service
HIGH	Warning	Exposure of information to an unauthorized user
MEDIUM		
LOW	Notice	Insufficient validation
UNKNOWN		

Update Trivy Database in Air-Gapped Environments

All Trivy versions include databases that are updated regularly.

In **non-air-gapped environments**, Nutanix Kubernetes Platform Insights (NKP Insights) automatically updates the Trivy database before each scheduled run (every two hours, by default) to support the latest security updates.

In **air-gapped environments**, NKP Insights uses the Trivy database bundled with the NKP release, but you can manually update this database as required.

This section shows you how to update the Trivy databases manually in your air-gapped environments.

Prerequisites

- Install Git: https://git-scm.com/book/en/v2/Getting-Started-Installing-Git
- Install Docker: https://docs.docker.com/engine/install/
- You have enabled Trivy: https://docs.d2iq.com/dins/2.7/trivy

Verifying the Trivy Version

About this task

Procedure

Obtain the currently used Trivy version:

```
kubectl get cronjob -n <workspace_namespace> nkp-insights-trivy -o
 jsonpath='{.spec.jobTemplate.spec.template.spec.initContainers[?
(@.name=="trivy")].image}' | cut -d ":" -f 2
```

The output displays the Trivy version, followed by the database timestamp.

In the example output, the Trivy version is `0.42.1` , and the database timestamp is `20230816T060333Z`.

```
0.42.1-20230816T060333Z
```

Creating a Bundle with the New Trivy Database

About this task

Create an air-gapped Trivy bundle from the trivy-bundles public repository. For more information about trivy-bundles, see https://github.com/mesosphere/trivy-bundles.

Starting on an internet-connected machine:

Procedure

1. Clone the Nutanix Kubernetes Platform Insights(NKP Insights) - Trivy Bundles repository to your local machine using the command `git clone https://github.com/mesosphere/trivy-bundles.git`
2. Specify the Trivy Version included in this version of NKP Insights using the command `export TRIVY_VERSION=`
3. Build the air-gapped bundle using the command `make create-airgapped-image-bundle`
   In this example output, the bundle is called `trivy-bundles-0.42.1-20230908T185308Z.tar.gz`.

   ```
   Executing target: install-mindthegap
   Executing target: latest_image_tag
   [+] Building 7.3s (10/10) FINISHED
                                                        docker:default
    => [internal] load build definition from Dockerfile
                                                                  0.0s
   ```

```
 => => transferring dockerfile: 534B

                                                                    0.0s
 => [internal] load .dockerignore

                                                                    0.0s
 => => transferring context: 2B

                                                                    0.0s
 => [internal] load metadata for docker.io/aquasec/trivy:0.42.1

                                                                    0.3s
 => [1/7] FROM docker.io/aquasec/
trivy:0.42.1@sha256:49a0b08589b7577f3e21a7d479284c69dc4d27cbb86bd07ad36773f075581313

                             0.0s
 => CACHED [2/7] RUN mkdir /trivy_cache

                                                                    0.0s
 => CACHED [3/7] RUN chown 65532:65532 /trivy_cache

                                                                    0.0s
 => [4/7] RUN echo 20230908T185308Z

                                                                    0.3s
 => [5/7] RUN trivy image --download-db-only --cache-dir /trivy_cache

                                                                    4.5s
 => [6/7] RUN ls -Rl /trivy_cache

                                                                    0.3s
 => exporting to image

                                                                    1.8s
 => => exporting layers

                                                                    1.8s
 => => writing image
 sha256:62f71725212e5b680a3cef771bcb312e931e05445c50632fa4495e216793c9cf

                                          0.0s
 => => naming to docker.io/mesosphere/trivy-bundles:0.42.1-20230908T185308Z

                                                                    0.0s
Executing target: create-airgapped-image-bundle


 # Checking if output file already exists
 # Parsing image bundle config
 # Creating temporary directory
 # Starting temporary Docker registry
 # Pulling requested images [==================================>1/1] (time elapsed
 23s)
 # Archiving images to trivy-bundles-0.42.1-20230908T185308Z.tar.gz
```

4. Transfer the created bundle to the air-gapped bastion host or node you used to install NKP.

Uploading the Bundle to your Air-Gapped Environment

About this task

The air-gapped bundle can now be uploaded to the private registry.

Procedure

1. Go to the air-gapped bastion host or node you used for installing NKP.
2. Export the environment variables for your registry. For more information, see the Local Registry.

```
export REGISTRY_ADDRESS=<registry-address>:<registry-port>
export REGISTRY_USERNAME=<username>
export REGISTRY_PASSWORD=<password>
```

3. Run the following command to load the air-gapped Trivy bundle into your private registry: Replace `<trivy-bundle-name.tar.gz>` with the name of the bundle you created in the previous section.

```
nkp push bundle --bundle <trivy-bundle-name.tar.gz> --to-registry $REGISTRY_ADDRESS
 --to-registry-username $REGISTRY_USERNAME --to-registry-password $REGISTRY_PASSWORD
```

4. Update Nutanix Kubernetes Platform Insights (NKP Insights) in the air-gapped environment to use the refreshed database. Edit the service configuration on each workspace by providing the path to the Docker image. To modify an existing installation, select **Workspace**, **Applications**, **NKP-Insights**, and then **Edit**. Replace `<docker-image-name>` with the path to the Docker image. It looks similar to `docker.io/mesosphere/trivy-bundles:0.42.1-20230908T185308Z`

```
trivy:
  enabled: true
  image:
    imageFull: <docker-image-path>
```

Verify the Database

After Insights has completed deploying, check the currently used Trivy database shown in Verifying the Trivy Version on page 1250 to ensure the configuration has been deployed correctly.

Kube-bench

Kube-bench by Aqua Security is a tool that verifies that Kubernetes clusters run securely. This tool checks against the best practices and guidelines specified in the CIS Kubernetes Benchmark developed by the Center for Internet Security to ensure that your clusters comply with the latest security configuration standards.

Whenever a standard is not met during a scan, an Insights alert is created with comprehensive information. For more information about this application, refer to the official documentation from Kube-bench.

Kube-bench adds the ability to ensure that Kubernetes clusters run securely. This tool checks against the best practices and guidelines specified in the CIS Kubernetes Benchmark.

Whenever a security standard is not met during a scan, an Insights alert is created with comprehensive information.

To know which Kube-bench version is included in this release, see the Nutanix Kubernetes Platform Insights(NKP Insights) Release Notes.

For more information on Kube-bench, see https://www.aquasec.com/products/kubernetes-security/ and https://aquasecurity.github.io/kube-bench/v0.6.12/. For more information on the Center for Internet Security, see https://www.cisecurity.org/.

Enabling or Disabling Kube-bench

About this task

Kube-bench is enabled by default, but you can disable it anytime.

Procedure

1. Edit the Service configuration with the following values:

```
kubeBench:
  enabled: true
```

2. To modify an installation, select **Workspace** > **Applications** > **NKP-Insights** > **Edit**.

Changing the frequency of Kube-bench Scans

About this task

To change the default scan frequency, perform the following task:

Procedure

1. Kube-bench scans run by default every 35 minutes and uses Cron syntax. You can change the default by editing the Service configuration with the following values:

```
kubeBench:
  schedule: "@every 35m"
```

> **Note:** For more information on Cron syntax, see https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/#cron-schedule-syntax.

2. To modify an installation, select **Workspace** > **Applications** > **NKP-Insights** > **Edit**.

Changing CIS Benchmark Version

About this task

By default, Kube-bench attempts to auto-detect the running version of Kubernetes and map this to the corresponding CIS Benchmark version. For example, Kubernetes version 1.15 is mapped to CIS Benchmark version `cis-1.15` , the benchmark version valid for Kubernetes 1.15. For an existing or a new configuration instance,

Procedure

1. You can change this default behavior and define a CIS benchmark version to check against, editing the service configuration with the following values:

   The example configuration configures Kube-bench to check against the `cis-1.15` regardless of the Kubernetes version.

```
kubeBench:
  config:
    instances:
      defaultSetup:
        additionalArgs: ["--version", "cis-1.15"]
```

2. To modify an installation, select **Workspace** > **Applications** > **NKP-Insights** > **Edit**.

Severity Levels

Kube-bench validation runs only have three possible outcomes:

- If the validation runs correctly and does not detect any anomalies, no Insight is created.
- If the validation runs and fails due to a detected anomaly, an Insight is created with the alert level **Warning**.
- If the validation check cannot run or is incomplete, an Insight is created with the alert level **Warning**.

Known Issues and Mitigations

`kube-bench` analyses security-related aspects of your cluster and creates alerts when your Kubernetes cluster is not compliant with the best practices established in the CIS benchmark.

Some issue alerts relate to cluster elements created with Konvoy, NKP's provisioning tool.

For customers who require CIS Benchmark compliance, this page provides an overview of mitigating these known alerts or why addressing the issue is not feasible.

- For issues that can be mitigated, create patch files with the mitigations, then create a cluster kustomization that references these patch files, and, lastly, create a new cluster based on the kustomization file as shown in Mitigate Issues by Creating Custom Clusters on page 1254.
- For issues that cannot be mitigated, see the List of CIS Benchmark Explanations at https://docs.d2iq.com/dins/2.7/list-of-cis-benchmark-explanations.

Mitigate Issues by Creating Custom Clusters

For issues that can be mitigated, create patch files with the mitigations, then create a cluster kustomization that references these patch files, and, lastly, create a new cluster based on the kustomization file

Creating Patch Files with CIS Benchmark Mitigations

About this task

Context for the current task

> **Note:** All files you create in this and the following sections must be present in the same directory.

Procedure

1. Establish a name for the cluster you will create by setting the `CLUSTER_NAME` environment variable: Replace the placeholder `<name_of_the_cluster>` with the actual name you want to use.

   ```
   export CLUSTER_NAME=<name_of_the_cluster>
   ```

2. Create CIS patch files for the issues you want to mitigate. These are the issues that you can mitigate:

CIS 1.2.5

ID	Text	Remediation
1.2.5	Ensure that the `--kubelet-certificate-authority` argument is set as appropriate (Automated).	Follow the Kubernetes documentation and setup the TLS connection between the apiserver and kubelets. Then, edit the API server pod specification file `$apiserverconf` on the control plane node and set the `--kubelet-certificate-authority` parameter to the path to the cert file for the certificate `authority.--kubelet-certificate-authority=<ca-string>`.

NKP Explanation

The `--kubelet-certificate-authority` flag needs to be set on each API Server after the cluster has been fully provisioned, adding it earlier causes issues with the creation and adding of worker nodes via CAPI and kubeadm.

CIS 1.2.12

ID	Text	Remediation
1.2.12	Ensure that the admission control plugin AlwaysPullImages is set (Manual).	Edit the API server pod specification file $apiserverconf on the control plane node and set the --enable-admission-plugins parameter to include AlwaysPullImages:--enable-admission-plugins=...,AlwaysPullImages,...

NKP Mitigation

Create a file called `cis-1.2.12-patches.yaml` with the following in the same folder as `kustomization.yaml`:

```
cat <<EOF > cis-1.2.12-patches.yaml
apiVersion: controlplane.cluster.x-k8s.io/v1beta1
kind: KubeadmControlPlane
metadata:
  name: ${CLUSTER_NAME}-control-plane
spec:
  kubeadmConfigSpec:
    clusterConfiguration:
      apiServer:
        extraArgs:
          enable-admission-plugins: "AlwaysPullImages"
EOF
```

CIS 1.2.15

ID	Text	Remediation
1.2.15	Ensure that the `--profiling` argument is set to false (Automated).	Edit the API server pod specification file `$apiserverconf` on the control plane node and set the parameter:`--profiling=false`

NKP Mitigation

Create a file called `cis-1.2.16-patches.yaml` with the following in the same folder as `kustomization.yaml`:

```
cat <<EOF > cis-1.2.16-patches.yaml
apiVersion: controlplane.cluster.x-k8s.io/v1beta1
kind: KubeadmControlPlane
metadata:
```

```
  name: ${CLUSTER_NAME}-control-plane
spec:
  kubeadmConfigSpec:
    clusterConfiguration:
      apiServer:
        extraArgs:
          profiling: "false"
EOF
```

CIS 1.2.18

ID	Text	Remediation
1.2.18	Ensure that the --profiling argument is set to false (Automated).	Edit the API server pod specification file $apiserverconf on the control plane node and set the below parameter:--profiling=false

NKP Mitigation

Create a file called `cis-1.2.18-patches.yaml` with the following in the same folder as `kustomization.yaml`:

```
cat <<EOF > cis-1.2.18-patches.yaml
apiVersion: controlplane.cluster.x-k8s.io/v1beta1
kind: KubeadmControlPlane
metadata:
  name: ${CLUSTER_NAME}-control-plane
spec:
  kubeadmConfigSpec:
    clusterConfiguration:
      apiServer:
        extraArgs:
          profiling: "false"
EOF
```

CIS 1.2.32

ID	Text	Remediation
1.2.32	Ensure that the API Server only makes use of Strong Cryptographic Ciphers (Manual)	Edit the API server pod specification file /etc/kubernetes/manifests/kube-apiserver.yaml on the control plane node and set the below parameter. --tls-cipher-suites=TLS_AES_128_GCM_SHA256,TLS_AES_256_GCM_SHA TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256,TLS

NKP Mitigation

Create a file called `cis-1.2.32-patches.yaml` with the following in the same folder as `kustomization.yaml`:

```
cat <<EOF > cis-1.2.32-patches.yaml
apiVersion: controlplane.cluster.x-k8s.io/v1beta1
kind: KubeadmControlPlane
metadata:
  name: ${CLUSTER_NAME}-control-plane
spec:
  kubeadmConfigSpec:
    clusterConfiguration:
      apiServer:
```

```
        extraArgs:
          tls-cipher-suites:
 "TLS_AES_128_GCM_SHA256,TLS_AES_256_GCM_SHA384,TLS_CHACHA20_POLY1305_SHA256,TLS_ECDHE_ECDSA_WITH_
EOF
```

CIS 1.3.1

ID	Text	Remediation
1.3.1	Ensure that the --terminated-pod-gc-threshold argument is set as appropriate (Manual).	Edit the Controller Manager pod specification file $controllermanagerconf on the control plane node and set the --terminated-pod-gc-threshold to an appropriate threshold, for example:--terminated-pod-gc-threshold=10

NKP Mitigation

Create a file called `cis-1.3.1-patches.yaml` with the following in the same folder as `kustomization.yaml`:

```
cat <<EOF > cis-1.3.1-patches.yaml
apiVersion: controlplane.cluster.x-k8s.io/v1beta1
kind: KubeadmControlPlane
metadata:
  name: ${CLUSTER_NAME}-control-plane
spec:
  kubeadmConfigSpec:
    clusterConfiguration:
      controllerManager:
        extraArgs:
          terminated-pod-gc-threshold: "12500"
EOF
```

CIS 1.3.2

ID	Text	Remediation
1.3.2	Ensure that the --profiling argument is set to false (Automated).	Edit the Controller Manager pod specification file $controllermanagerconf on the control plane node and set the below parameter:--profiling=false

NKP Mitigation

Create a file called `cis-1.3.2-patches.yaml` with the following in the same folder as `kustomization.yaml`:

```
cat <<EOF > cis-1.3.2-patches.yaml
apiVersion: controlplane.cluster.x-k8s.io/v1beta1
kind: KubeadmControlPlane
metadata:
  name: ${CLUSTER_NAME}-control-plane
spec:
  kubeadmConfigSpec:
    clusterConfiguration:
      controllerManager:
        extraArgs:
          profiling: "false"
EOF
```

CIS 1.4.1

ID	Text	Remediation
1.4.1	Ensure that the --profiling argument is set to false (Automated).	Edit the Controller Manager pod specification file $schedulerconf on the control plane node and set the below parameter:--profiling=false

NKP Mitigation

Create a file called `cis-1.4.1-patches.yaml` with the following in the same folder as `kustomization.yaml`:

```
cat <<EOF > cis-1.4.1-patches.yaml
apiVersion: controlplane.cluster.x-k8s.io/v1beta1
kind: KubeadmControlPlane
metadata:
  name: ${CLUSTER_NAME}-control-plane
spec:
  kubeadmConfigSpec:
    clusterConfiguration:
      scheduler:
        extraArgs:
          profiling: "false"
EOF
```

CIS 4.1.1

ID	Text	Remediation
4.1.1	Ensure that the kubeletservice file permissions are set to 600 or more restrictive(Automated). All known nodes are affected.	Run the below command (based on the file location on your system) on the each node. For example, `chmod 600 /lib/systemd/system/kubelet.service`

NKP Mitigation

Create a file called `cis-4.1.1-patches.yaml` with the following in the same folder as `kustomization.yaml`:

```
cat <<EOF > cis-4.1.1-patches.yaml
apiVersion: controlplane.cluster.x-k8s.io/v1beta1
kind: KubeadmControlPlane
metadata:
  name: ${CLUSTER_NAME}-control-plane
spec:
  kubeadmConfigSpec:
    postKubeadmCommands:
    - chmod 600 /lib/systemd/system/kubelet.service
---
apiVersion: bootstrap.cluster.x-k8s.io/v1beta1
kind: KubeadmConfigTemplate
metadata:
  name: ${CLUSTER_NAME}-md-0
spec:
  template:
    spec:
      postKubeadmCommands:
      - chmod 600 /lib/systemd/system/kubelet.service
EOF
```

CIS 4.1.9

ID	Text	Remediation
4.1.9	If the kubelet `config.yaml`configuration file is being used validate permissions set to 600 or more restrictive.	Run the following command (using the config file location identified in the Audit step):`chmod 600 /var/lib/kubelet/config.yaml`

NKP Mitigation

Create a file called `cis-4.1.9-patches.yaml` with the following in the same folder as `kustomization.yaml`:

```
cat <<EOF > cis-4.2.6-patches.yaml
apiVersion: controlplane.cluster.x-k8s.io/v1beta1
kind: KubeadmControlPlane
metadata:
  name: ${CLUSTER_NAME}-control-plane
spec:
  kubeadmConfigSpec:
    postKubeadmCommands:
    - chmod 600 /var/lib/kubelet/config.yaml
---
apiVersion: bootstrap.cluster.x-k8s.io/v1beta1
kind: KubeadmConfigTemplate
metadata:
  name: ${CLUSTER_NAME}-md-0
spec:
  template:
    spec:
      postKubeadmCommands:
      - chmod 600 /var/lib/kubelet/config.yaml
EOF
```

CIS 4.2.6

ID	Text	Remediation
4.2.6	Ensure that the --protect-kernel-defaults argument is set to true (Automated).	If using a Kubelet config file, edit the file to set `protectKernelDefaults` to `true`. If using command line arguments, edit the kubelet service file $kubeletsvc on each worker node and set the below parameter in KUBELET_SYSTEM_PODS_ARGS variable:--protect-kernel-defaults=trueBased on your system, restart the kubelet service. For example systemctl daemon-reloadsystemctl restart kubelet.service

NKP Mitigation

Create a file called `cis-4.2.6-patches.yaml` with the following in the same folder as `kustomization.yaml`:

```
cat <<EOF > cis-4.2.6-patches.yaml
apiVersion: controlplane.cluster.x-k8s.io/v1beta1
kind: KubeadmControlPlane
metadata:
  name: ${CLUSTER_NAME}-control-plane
spec:
  kubeadmConfigSpec:
    initConfiguration:
      nodeRegistration:
        kubeletExtraArgs:
```

```
          protect-kernel-defaults: "true"
    joinConfiguration:
      nodeRegistration:
        kubeletExtraArgs:
          protect-kernel-defaults: "true"
---
apiVersion: bootstrap.cluster.x-k8s.io/v1beta1
kind: KubeadmConfigTemplate
metadata:
  name: ${CLUSTER_NAME}-md-0
spec:
  template:
    spec:
      joinConfiguration:
        nodeRegistration:
          kubeletExtraArgs:
            protect-kernel-defaults: "true"
EOF
```

CIS 4.2.9

ID	Text	Remediation
4.2.9	Ensure that the eventRecordQPS argument is set to a level that ensures appropriate event capture (Manual).	If using a Kubelet config file, edit the file to set `eventRecordQPS` to an appropriate level. If using command line arguments, edit the kubelet service file$kubeletsvc on each worker node and set the parameter below in the KUBELET_SYSTEM_PODS_ARGS variable. Based on your system, restart the kubelet service. For example, systemctl daemon-reloadsystemctl restart kubelet.service

NKP Mitigation

`eventRecordQPS` can also be configured with the `--event-qps` argument on the kubelet's arguments.

Create a file called `cis-4.2.9-patches.yaml` with the following in the same folder as `kustomization.yaml`:

```
cat <<EOF > cis-4.2.9-patches.yaml
apiVersion: controlplane.cluster.x-k8s.io/v1beta1
kind: KubeadmControlPlane
metadata:
  name: ${CLUSTER_NAME}-control-plane
spec:
  kubeadmConfigSpec:
    initConfiguration:
      nodeRegistration:
        kubeletExtraArgs:
          event-qps: "0"
    joinConfiguration:
      nodeRegistration:
        kubeletExtraArgs:
          event-qps: "0"
---
apiVersion: bootstrap.cluster.x-k8s.io/v1beta1
kind: KubeadmConfigTemplate
metadata:
  name: ${CLUSTER_NAME}-md-0
spec:
  template:
    spec:
```

```
      joinConfiguration:
        nodeRegistration:
          kubeletExtraArgs:
            event-qps: "0"
EOF
```

CIS 4.2.13

ID	Text	Remediation
4.2.13	Ensure that the Kubelet only makes use of Strong Cryptographic Ciphers (Manual)	If using a Kubelet config file, edit the file to set `TLSCipherSuites` to TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,TLS_EC or to a subset of these values. If using executable arguments, edit the kubelet service file $kubeletsvc on each worker node and set the --tls-cipher-suites parameter as follows or to a subset of these values. --tls-cipher-suites=TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,T Based on your system, restart the kubelet service. For example, systemctl daemon-reload systemctl restart kubelet.service

NKP Mitigation

Create a file called `cis-4.2.13-patches.yaml` with the following in the same folder as `kustomization.yaml`:

```
cat <<EOF > cis-4.2.13-patches.yaml
apiVersion: controlplane.cluster.x-k8s.io/v1beta1
kind: KubeadmControlPlane
metadata:
  name: ${CLUSTER_NAME}-control-plane
spec:
  kubeadmConfigSpec:
    initConfiguration:
      nodeRegistration:
        kubeletExtraArgs:
          tls-cipher-suites:
 "TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_ECDSA_WI
    joinConfiguration:
      nodeRegistration:
        kubeletExtraArgs:
          tls-cipher-suites:
 "TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_ECDSA_WI
---
apiVersion: bootstrap.cluster.x-k8s.io/v1beta1
kind: KubeadmConfigTemplate
metadata:
  name: ${CLUSTER_NAME}-md-0
spec:
  template:
    spec:
```

```
      joinConfiguration:
        nodeRegistration:
          kubeletExtraArgs:
            tls-cipher-suites:
 "TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_ECDSA_WI
EOF
```

Create a Cluster Kustomization

Create a cluster kustomization that references the CIS patch files you created in the previous section.

> **Note:** The `kustomization.yaml` file you create in this section must be in the same directory as the CIS patch files.

Prerequisites

Refer to Customizing CAPI Components for a Cluster to familiarize yourself with the customization procedure and options. We will use similar terms on this page.

For more information, see Customizing CAPI Components for a Cluster at https://docs.d2iq.com/dkp/2.8/customizing-capi-components-for-a-cluster

Creating a Kustomization YAML File

About this task

Create a cluster YAML using the NKP CLI,

Procedure

1. Create a cluster YAML and modify any arguments as necessary:

```
nkp create cluster aws
  --cluster-name=${CLUSTER_NAME} \
  --dry-run \
  --output=yaml \
  > ${CLUSTER_NAME}.yaml
```

2. Create a `kustomization.`The yaml# file will include patches for each of the CIS mitigations.

   We use the ##CIS-1.2.18# patch in this example, but you can include all mitigation files you created in the first section.

```
cat <<EOF > kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
bases:
  - ${CLUSTER_NAME}.yaml
patches:
  - cis-1.2.18-patch.yaml
 #- Add more CIS patch files here.
 EOF
```

Create a Cluster with the Kustomization

About this task

Context for the current task

> **Note:** The CIS patch, `kustomization.yaml`, and `${CLUSTER_NAME}.yaml` files must be in the same directory.

Procedure

1. Create a Bootstrap Cluster. Ensure that the bootstrap cluster has been created for the desired provider.

   Note: Supported providers include AWS, Azure, GCP, Pre-Provisioned, and vSphere.

2. To apply the customizations and create a new cluster, use the command `kubectl create -k.`

3. Monitor and watch the cluster creation.

List of CIS Benchmark Explanations

CIS 1.1.10

ID	Text	Remediation
1.1.10	Ensure that the Container Network Interface file ownership is set to root:root (Manual)	Run the below command (based on the file location on your system) on the control plane node. For example,chown root:root <path/to/cni/files>

NKP Explanation

The kubelet config `--cni-config-dir` has been deprecated and removed since Kubernetes v1.24. Calico, used for CNI stores, is configured at `/etc/cni/net.d` and has ownership set to `root:root`.

CIS 1.1.9

ID	Text	Remediation
1.1.9	Ensure that the Container Network Interface file permissions are set to 644 or more restrictive (Manual)	Run the below command (based on the file location on your system) on the control plane node. For example, chmod 644 <path/to/cni/files>

NKP Explanation

The kubelet config `--cni-config-dir` has been deprecated and removed since Kubernetes v1.24. Calico, which is used for CNI, stores its configuration at `/etc/cni/net.d` and has permissions set to `644`.

CIS 1.1.12

ID	Text	Remediation
1.1.12	Ensure that the etcd data directory ownership is set to etcd:etcd (Automated)	On the etcd server node, get the etcd data directory, passed as an argument --data-dir, from the command 'ps -ef | grep etcd.'Run the below command (based on the etcd data directory found above).For example, chown etcd:etcd /var/lib/etcd

NKP Explanation

etcd files are owned by root. Creating another user adds additional attack vectors. On previous STIGs, this has been acceptable to leave as root:root.

CIS 1.2.1

ID	Text	Remediation
1.2.1	Ensure that the --anonymous-auth argument is set to false (Manual)	Edit the API server pod specification file $apiserverconfon the control plane node and set the below parameter.--anonymous-auth=false

NKP Explanation

Although the `--anonymous-auth` flag defaults to `true`, we also set the `--authorization-mode=Node,RBAC` flag. Having anonymous authorization enabled is generally used for discovery and health checking. This is also important for `kubeadm join` to function properly. For more information, see https://github.com/aws/eks-anywhere/pull/3122#issuecomment-1226581563.

CIS 1.2.5

ID	Text	Remediation
1.2.5	Ensure that the `--kubelet-certificate-authority` argument is set as appropriate (Automated).	Follow the Kubernetes documentation and setup the TLS connection between the apiserver and kubelets. Then, edit the API server pod specification file `$apiserverconf` on the control plane node and set the `--kubelet-certificate-authority` parameter to the path to the cert file for the certificate `authority.--kubelet-certificate-authority=<ca-string>`.

NKP Explanation

The `--kubelet-certificate-authority` flag needs to be set on each API Server after the cluster has been fully provisioned, adding it earlier causes issues with the creation and adding of worker nodes via CAPI and kubeadm.

CIS 1.2.6

ID	Text	Remediation
1.2.6	Ensure that the --kubelet-certificate-authority argument is set as appropriate (Automated)	Follow the Kubernetes documentation and set up the TLS connection between the apiserver and kubelets. Then, edit the API server pod specification file$apiserverconf on the control plane node and set the--kubelet-certificate-authority parameter to the certificate authority's path to the cert file.--kubelet-certificate-authority=<ca-string>

NKP Explanation

The `--kubelet-certificate-authority` flag needs to be set on each API Server after the cluster has been fully provisioned; adding it earlier causes issues with the creation and adding of worker nodes via CAPI and kubeadm.

CIS 1.2.10

ID	Text	Remediation
4.2.10	Ensure that the --tls-cert-file and --tls-private-key-file arguments are set as appropriate (Manual)	If using a Kubelet config file, edit the file to set `tlsCertFile` to the location of the certificate file to identify this Kubelet and `tlsPrivateKeyFile`to the location of the corresponding private key file. If using command line arguments, edit the kubelet service file$kubeletsvc on each worker node and the below parameters in KUBELET_CERTIFICATE_ARGS variable.--tls-cert-file=<path/to/tls-certificate-file>--tls-private-key-file=<path/to/tls-key-file>Based on your system, restart the kubelet service. For example,systemctl daemon-reloadsystemctl restart kubelet.service

NKP Explanation

This remediation refers to a serving certificate on the kubelet, where the https endpoint on the kubelet is used. By default, a self-signed certificate is used here. Connecting to a kubelet's https endpoint should only be used for diagnostic or debugging purposes where applying a provided key and certificate isn't expected.

For more information, see *Client and serving certificates* at https://kubernetes.io/docs/reference/access-authn-authz/kubelet-tls-bootstrapping/#client-and-serving-certificates.

CIS 1.2.13

ID	Text	Remediation
1.2.13	Ensure that the admission control plugin SecurityContextDeny is set if PodSecurityPolicy is not used (Manual)	Edit the API server pod specification file $apiserverconfon the control plane node and set the --enable-admission-plugins parameter to includeSecurityContextDeny, unless PodSecurityPolicy is already in place.--enable-admission-plugins=...,SecurityContextDeny,...

NKP Explanation

The Kubernetes Project recommends not using this admission controller, as it is deprecated and will be removed in a future release. For more information, see Admission Controllers Reference https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/#securitycontextdeny.

CIS 4.2.8

ID	Text	Remediation
4.2.8	Ensure that the --hostname-override argument is not set (Manual)	Edit the kubelet service file $kubeletsvcon each worker node and remove the --hostname-override argument from theKUBELET_SYSTEM_PODS_ARGS variable. Based on your system, restart the kubelet service. For example,systemctl daemon-reloadsystemctl restart kubelet.service

NKP Explanation

The `hostname-override` argument is used by various infrastructure providers to provision nodes; removing this argument will impact how CAPI works with the infrastructure provider.

CIS 4.2.10

ID	Text	Remediation
1.2.10	Ensure that the admission control plugin EventRateLimit is set (Manual)	Follow the Kubernetes documentation and set the desired limits in a configuration file.Then, edit the API server pod specification file $apiserverconfand set the below parameters.--enable-admission-plugins=...,EventRateLimit,...--admission-control-config-file=<path/to/configuration/file>

NKP Explanation

Kubernetes recommends the use of API Priority and Fairness using the `--max-requests-inflight` and `--max-mutating-requests-inflight` flags to control how the Kubernetes API Server behaves in overload situations. The `APIPriorityAndFairness` Feature Gate has been enabled by default since Kubernetes v1.20.

For more information, see:

- API Priority and Fairness: https://kubernetes.io/docs/concepts/cluster-administration/flow-control/
- Feature Gate: https://kubernetes.io/docs/reference/command-line-tools-reference/feature-gates/#feature-gates-for-alpha-or-beta-features